STB-SUITE

The Siesta Toolbox Suite

User and Reference Manual

Command-line tools for every stage of a SIESTA density-functional-theory project: input generation, structure building, post-processing, complete property workflows, and machine-learning-potential simulations.

Carlos M. O. Bastos

University of Brasília (UnB), Brazil

Version 2.0.0, the first stable release.
Source, examples and issue tracker: https://github.com/stb-suite/stb
Documentation site: https://stb-suite.github.io/stb/

Distributed under the MIT License.

About this manual

Stable version 2.0.0, under testing

This is the first stable release, and it is still being tested in real projects. **Verify every result** before relying on it: check it independently, for example against a reference calculation or the literature. Every tool prints the checks it ran and flags what it could not verify, but those diagnostics are an aid, not a guarantee.

This manual documents every command of STB-SUITE, the Siesta Toolbox Suite [1]: a collection of independent Python command-line tools that assist users of the SIESTA density-functional-theory (DFT) code [2, 3] through every step of a project. It is meant to be read in two ways.

As a user guide.
Part I explains how the suite is installed, how its tools are run (directly or through the interactive **`stb-suite`** menu), and the conventions every tool shares: how a SIESTA input is split into a structure file and a calculation template, how directives are forced onto a template without editing it, how reports, plots and bibliography files are written, and how the multi-stage workflows hand data from one stage to the next.

As a reference.
Part II (Chapters 7–9) follows the six categories of the interactive menu, one section per menu item. Each section opens with an *At a glance* box (menu code, command, example folder), continues with the hands-on guide of the documentation site when one exists, adds implementation details, theory and caveats taken from the source code itself, and ends with the complete command-line reference: every option of every command, with its type, default value, allowed choices and whether it is required, extracted directly from the tool's argument parser.

Part III describes the internal architecture (the shared `stb.core` library, the menu dispatcher, versioning, extensions and the documentation tooling) for those who want to extend the suite. The appendices hold a map of every menu code, an alphabetical table of all commands, units and conversion factors, and a glossary. The bibliography collects every method, code and database the tools rely on; the index lists every command and every long option.

Typographic conventions

- **`stb-kgrid`** – a console command installed by the package.
- `-density` – a command-line option; `calc.fdf` – a file or folder.
- `MD.TypeOfRun` – a SIESTA fdf directive.
- **5.8.2** – a code of the interactive menu (category 4, item 8, stage 2).
- Shell sessions are set in framed monospace blocks; lines starting with `#` are comments.
- Report markers are quoted as printed by the tools: `[OK]`, `[NOTE]`, `[WARNING]`, `[ERROR]`, `[SKIPPED]`, `[UNVERIFIED]`, `[EXTRAPOLATED]`.

Relation to the documentation site

The documentation site is generated from the repository itself: its *Guides* are the `README.md` files of the `examples/` folder and its *Reference* pages are the `--help` texts of the commands. Both are included here (the guides verbatim, converted to LaTeX; the help texts restructured into option lists). Everything else – the overview chapters, the implementation notes, the theory boxes, the sections of tools that have no guide yet, the developer chapter and the appendices – exists only in this manual.

Contents

About this manual i

I Getting started 1

1 Introduction 2

1.1 What STB-SUITE is . . . 2
1.2 Design principles . . . 2
1.3 How this manual is organized . . . 3
1.4 Citing STB-SUITE and the methods it uses . . . 3

2 Installation 4

2.1 Requirements . . . 4
2.2 Installing from source . . . 4
2.3 Checking the installation . . . 5
2.4 Version numbers . . . 5
2.5 External programs you may want . . . 5

3 Running the tools 6

3.1 Two ways to run any tool . . . 6
3.2 The interactive menu . . . 6
3.3 Options shared by every tool . . . 7
3.4 Reports . . . 8
3.5 Bibliography files . . . 8
3.6 Plots . . . 8
3.7 Trajectory formats . . . 8

4 SIESTA files and conventions 10

4.1 The structure file and the calculation template . . . 10
4.2 `config_extra.fdf`: forcing directives without editing a template . . . 11
4.3 Pseudopotentials . . . 11
4.4 Vacuum and dimensionality . . . 12
4.5 Output files of SIESTA read by the suite . . . 12

4.6 Atoms near the cell boundary . . . 13

5 Workflows in practice 14

5.1 The prepare–run–analyse cycle . . . 14
5.2 Running many folders . . . 14
5.3 What the analysis stages check . . . 15
5.4 Converge first . . . 15

6 Machine-learning potentials 16

6.1 MACE and the MACE-MP-0 foundation model . . . 16
6.2 Where MACE appears in the suite . . . 16
6.3 Choosing the model . . . 16
6.4 Device and precision . . . 17
6.5 Dispersion . . . 17
6.6 Caveats that apply to every ML result . . . 17

II Tool reference 18

7 Inputs 19

7.1 Input File Generator . . . 20
7.2 Pseudopotential Resolver . . . 26
7.3 K-Grid Generator . . . 31
7.4 K-Path Generator . . . 35
7.5 DFT+U / Hubbard Block Generator . . . 39
7.6 Structure Fetcher . . . 45
7.7 ML Pre-Relaxation . . . 50

8 Analysis 59

8.1 Bands Analyzer . . . 61
8.2 PDOS XML Parser . . . 67
8.3 DOS Processor (Convolution) . . . 73
8.4 Structure Analyzer . . . 79
8.5 Symmetry Analyzer . . . 86
8.6 Bader Charge Analysis . . . 93
8.7 Work Function Calculator . . . 99
8.8 Density Plotter (RHO) . . . 106
8.9 Powder XRD Simulator . . . 112

8.10 Fatbands Analyzer . . . 119
8.11 STM Simulator . . . 126
8.12 Wavefunction Density . . . 133
8.13 STS Spectroscopy Simulator . . . 140
8.14 COOP/COHP Bonding Analysis . . . 146
8.15 IPR Analyzer . . . 152
8.16 Effective Mass / Velocity . . . 157
8.17 Spin Texture Analyzer . . . 163
8.18 AIMD Trajectory Analysis . . . 168
8.19 Native Charges . . . 175

9 Utils **186**
9.1 File Translator . . . 187
9.2 Clean File Tools . . . 203
9.3 Grid to Cube Converter . . . 216
9.4 Wantibexos Interface . . . 231
9.5 AIMD Trajectory Converter . . . 244
9.6 Status Checker . . . 257
9.7 Archive Packager . . . 268

10 Structures **280**
10.1 2D Monolayer Stacker . . . 281
10.2 Supercell Builder . . . 288
10.3 Slab Builder . . . 294
10.4 Nanotube/Nanoribbon Builder . . . 301
10.5 Point Defect Generator . . . 308
10.6 SQS Generator . . . 314
10.7 Unit Cell Finder . . . 322
10.8 Crystal Builder . . . 328
10.9 Surface Passivator . . . 335
10.10 Reference Molecule Builder . . . 341
10.11 Amorphous Structure Generator . . . 346
10.12 Random Crystal Generator . . . 354

11 Workflows **363**

11.1 Stress-Strain . . . 365
11.2 Elastic Constants . . . 379
11.3 Cohesive Energy . . . 394
11.4 Phonons . . . 406
11.5 Convergence Tests . . . 422
11.6 Structure Solution (XRD) . . . 434
11.7 Hubbard U (Linear Response) . . . 449
11.8 Adsorption . . . 464
11.9 NEB / Reaction Path . . . 490
11.10 2D Stacking Fault (Gamma-Surface) . . . 505
11.11 Raman Spectrum . . . 516
11.12 IR Spectrum . . . 534
11.13 Generalized Quasi-Chemical Approximation (GQCA) . . . 549
11.14 Optical Properties . . . 571
11.15 Equation of State . . . 595
11.16 Charge Density Difference . . . 617
11.17 Hirshfeld-I Charge Partitioning . . . 641

12 Catalysis **665**

13 ML Simulations **666**

13.1 Custom ML Force Field (MACE fine-tuning) . . . 667
13.2 ML Molecular Dynamics . . . 708
13.3 ML Phonons . . . 741
13.4 ML Elastic Constants . . . 773
13.5 ML NEB . . . 800
13.6 ML Adsorption-Site Screening . . . 827

III Developer reference **853**

14 Architecture **854**

14.1 Repository layout . . . 854
14.2 Thin tools over a shared core . . . 854
14.3 The interactive menu . . . 855
14.4 Extensions . . . 855
14.5 Versioning . . . 856

14.6 Documentation 856
14.7 Adding a tool 856
14.8 Testing 856

A Menu code map **857**

B All commands **860**

C Units and conversion factors **865**

D Glossary **866**

Bibliography **867**

Index of commands, options and topics **874**

Part I

Getting started

Chapter 1

Introduction

In this chapter

1.1 What STB-SUITE is 2
1.2 Design principles 2
1.3 How this manual is organized 3
1.4 Citing STB-SUITE and the methods it uses 3

1.1 What STB-SUITE is

SIESTA [2, 3] solves the Kohn–Sham equations [4, 5] with norm-conserving pseudopotentials and a basis of strictly localized numerical atomic orbitals [6]. That design makes it fast and able to treat large systems, but it also leaves a great deal of work to the user: the input file must be written by hand, pseudopotentials must be found and copied, k-point grids and band paths must be derived from the cell, and every *property* beyond the total energy – elastic constants, phonons, adsorption energies, barriers, optical spectra – is a small project of its own, made of many calculations prepared consistently and analysed together.

STB-SUITE automates that work. It is a set of *independent* command-line programs, each installed as an `stb-*` console command, grouped in six categories:

#	Category	What it is for
1	Inputs	Set up a SIESTA run: input file, pseudopotentials, k-grid, k-path, DFT+U
2	Structures	Build, generate or transform structure files
3	Analysis	Post-process finished SIESTA runs: bands, DOS, charges, STM, …
4	Workflow	Prepare-and-analyse pipelines for one physical property
5	ML Simulations	Simulations driven by a MACE machine-learning potential
6	Utils	Format conversion, run bookkeeping and file management

The interactive program **`stb-suite`** wraps all of them: it asks the questions the command-line flags would answer and then runs the very same command, so the two ways of working always give identical results.

1.2 Design principles

A handful of rules shape every tool, and knowing them makes the whole suite predictable.

STB never runs SIESTA for you.
Every tool either *prepares* folders that are ready to run, or *reads* folders that SIESTA has already finished. How SIESTA is launched (locally, with MPI, through a queue system) stays entirely in

your hands. This is why the property workflows of Category 4 are split into stages: a preparation stage writes folders, you run SIESTA in them, and an analysis stage aggregates the results. The one deliberate exception to "never run" is the machine-learning category, where a MACE potential replaces SIESTA and the whole calculation happens in memory.

Your templates are never edited.
When a workflow must impose a setting (a single-point evaluation, a fixed cell, spin polarization, a dipole correction, …) it writes it into a small sidecar file, `config_extra.fdf`, and adds one `%include` line at the very top of the copied `calc.fdf`. SIESTA keeps the first value it reads for a repeated label, so the forced value wins while your own file stays readable and intact (Section 4.2).

Every result is reported with its diagnostics.
Tools print a numbered report (`[0] RUN METADATA`, `[1] . . .`), where each check is listed as `[OK]`, `[NOTE]`, `[WARNING]` or `[ERROR]` rather than summarized as a single pass/fail. Quantities that rest on an approximation are labelled as such (`[UNVERIFIED]`, `[EXTRAPOLATED]`); unphysical inputs are refused with a clear `[ERROR]` instead of being "fixed" silently.

Every result can be cited.
Whenever an output depends on a citable method, code or database, a `references.bib` is written next to it; an existing file in the same folder is merged, not overwritten.

Level of theory is propagated, not assumed.
A stage that builds new calculations from a previous stage's relaxed geometry reads the settings that stage *actually* used (from its `config_extra.fdf` and logs) instead of assuming defaults, so a BSSE correction, a vibrational Hessian or a ghost-fragment density is always computed at the level of theory of the calculation it corrects.

One shared library.
Every command is a thin script around the `stb.core` library (Chapter 14). A file format, a parser or a physical convention is implemented once and reused, so that, for example, a strain labelled `xy` means exactly the same deformation in the DFT and the ML elastic tools.

1.3 How this manual is organized

Part I (this part) covers installation, the two ways of running a tool, the options every tool shares, the SIESTA file conventions, the workflow pattern and the machine-learning potentials. Part II is the tool reference, one chapter per menu category and one section per menu item. Part III is for developers. Appendix A maps every menu code to its command.

1.4 Citing STB-SUITE and the methods it uses

Please cite SIESTA [2, 3] for any calculation, and the methods, libraries and databases listed in the `references.bib` written by each tool. The main libraries the suite is built on are pymatgen [7], ASE [8], spglib [9], sisl [10], phonopy [11, 12], icet [13], PyXtal [14] and, for the ML tools, MACE [15] with the MACE-MP-0 foundation model [16].

Chapter 2

Installation

In this chapter

2.1 Requirements 4
2.2 Installing from source 4
2.3 Checking the installation 5
2.4 Version numbers 5
2.5 External programs you may want 5

2.1 Requirements

STB-SUITE is a pure-Python package (`stb_suite`, import name `stb`) for Python 3.9 to 3.12. Installing it pulls in the scientific stack listed in Table 2.1. SIESTA itself is *not* a Python dependency: install it separately on the machine where you will run the calculations (the workflows and their examples were exercised with SIESTA 5.x).

Table 2.1: Dependencies of the package and what they are used for.

Package	Role in the suite
`numpy`, `scipy`, `pandas`	numerics, fitting, Kramers–Kronig, tables
`matplotlib`	interactive previews (`-view`) and PNG figures
`ase` [8]	structures, optimizers, NEB, MD, EOS fits, 3D viewer, trajectory I/O
`pymatgen` [7]	structure objects, slabs, defects, adsorption sites, ZSL matching, Materials Project access
`spglib` $\geq$ 2.2 [9]	space-group and layer-group analysis
`sisl` [10]	SIESTA binary files: grids (`.RHO`, `.VT`, `.LDOS`), `.WFSX`, `.HSX`, `.DM`
`pybader` [17]	grid-based Bader partitioning
`phonopy` [11, 12]	finite-displacement phonons, irreducible representations
`icet` [13]	SQS generation and pair-cluster enumeration (GQCA)
`pyxtal` [14]	random symmetric structures, powder XRD engine
Optional extra `ml`	
`torch`, `mace-torch` [15]	MACE potentials for Category 5 and every `-ml-*` option
`torch-dftd`	Grimme D3 dispersion added to MACE when the matching DFT forces DFT-D3

2.2 Installing from source

There is no PyPI package; install from a clone of the repository:

```
git clone https://github.com/stb-suite/stb.git
cd stb/stb-suite
pip install .              # regular install
pip install -e .           # or: editable install, for development
```

The machine-learning tools additionally need PyTorch and `mace-torch`, installed by the optional `ml` extra:

```
pip install ".[ml]"                # from stb/stb-suite, instead of the plain install
pip install "stb_suite[ml]"        # or, to add it to an existing install
```

For a GPU, install a CUDA-enabled PyTorch build first (see the PyTorch installation selector) and then the extra; Section 6.4 explains how the tools choose and validate the device.

A conda recipe (`stb-suite/meta.yaml`) mirrors the pip dependencies and builds the release version by default (`STB_VERSION` overrides it):

```
conda build stb-suite
```

2.3 Checking the installation

Every tool is now on your `PATH`:

```
stb-kgrid --version          # prints 2.0.0
stb-kgrid --help             # every option of one tool
stb-suite                    # the interactive menu
```

A tool that needs an optional dependency checks for it when it starts and exits with an installation hint if it is missing (for example `pip install stb_suite[ml]` for the MACE tools), instead of failing half-way.

2.4 Version numbers

The suite has a single version for every tool. The `main` branch holds the stable releases (`2.0.0`, then `2.0.1`, … for bug fixes); the `dev` branch holds the next version and reports it as `2.1.0.dev<N>`, N being the number of commits since the last release tag, so it grows with every commit. An installed copy reports the number baked into its metadata when it was built; the environment variable `STB_VERSION` overrides both. Each tool shows it in its banner and with `-version`. The per-tool numbers used before (1.0.0–3.0.0) are retired; their notes are kept in `stb-suite/CHANGELOG.md`.

2.5 External programs you may want

- **SIESTA** [2, 3] – to run the folders the tools prepare.
- **gnuplot** – the analysis stages of the workflows write `.dat+.gplot` pairs (Section 3.6).
- **A 3D viewer** – VESTA [18], XCrySDen [19], VMD [20] or OVITO [21] for `.cube`, `.xsf` and trajectory files.
- **A display** (local, or `ssh -X`/`-Y`) for the interactive `-view` windows; every other output works headless.

Chapter 3

Running the tools

In this chapter

3.1 Two ways to run any tool . . . 6
3.2 The interactive menu . . . 6
3.3 Options shared by every tool . . . 7
3.4 Reports . . . 8
3.5 Bibliography files . . . 8
3.6 Plots . . . 8
3.7 Trajectory formats . . . 8

3.1 Two ways to run any tool

Directly, with flags. – scriptable, and what you use once you know a tool's options:

```
stb-kgrid -f structure.fdf -d 0.2
```

Through the interactive menu. – guided questions instead of flags; a good way to discover what a tool can do and what its defaults are:

```
stb-suite
```

Both paths run the same underlying command and produce the same output. Every guided example in the repository proves it by driving both with the same inputs and comparing the results.

3.2 The interactive menu

`stb-suite` starts with a banner and the main menu, which lists the seven categories: Inputs, Analysis, Utils, Structures, Workflows, Catalysis and ML Simulations. At the main prompt you can:

- type a category number (`1`–`7`) to open its sub-menu, then the item number; workflow items open one more level with their stages;
- type a tool's *dotted code* to jump straight to it – `1.3` for the k-grid generator, `5.1.2` for the second stage of the stress–strain workflow, `7.3` for the ML phonons;
- type `0` to quit (in a sub-menu, `0` goes back one level); `Ctrl+C` also leaves a sub-menu or the program.

After the questions, the menu prints nothing of its own: it runs the installed command (for example `stb-kgrid -f structure.fdf -d 0.2`) as a subprocess, shows its output, and waits for `Enter` before returning to the menu. If a command is missing from your `PATH`, the menu says so instead of failing. Appendix A lists every code.

The menu also checks things the command line cannot ask about interactively: for example, before offering `cuda` for a MACE tool it tells you whether a usable GPU was detected, and some analysis items offer to list a structure's atoms before asking for atom indices.

Tools meant to be called from scripts rather than interactively are not in the menu. The only one today is **`stb-nebCycle`**, one step of a real-DFT NEB optimization, called from a job-submission loop (Section 11.9).

3.3 Options shared by every tool

Every command accepts three options that are omitted from the option lists of Part II:

`-h, --help`
: Print the tool's usage, every option with its default, and usage examples, then exit.

`-v, --version`
: Print the suite version (for example `2.0.0`) and exit.

`--no-intro`
: Skip the animated banner printed when the tool starts – useful in scripts and logs. The menu passes it for you.

Most tools also share the following options; their exact wording is in each tool's reference.

`--save-report`
: (95 of the 101 commands) Also write the console report to a text file, normally `stb_<tool>_report.txt` in the output folder. The report is identical; the flag only persists it.

`-o, --output-dir / --output`
: Where the files go (default: the current folder, or a tool-specific name such as `mlff_runs/`).

`--view`
: Open an interactive view at the end of the run: ASE's 3D viewer for structures, a matplotlib window for curves and maps. Off by default and never needed for any file output.

`--save-gnuplot`
: (Analysis tools) Write each computed curve or map as a `.dat` file plus a ready-to-run `.gplot` script. Off by default.

`--save-data / --show / --png`
: (Machine-learning and newer workflow analyses) Write per-item data tables, show the figure, or save it as PNG.

`--vacuum-gap`
: Minimum empty span, in Å, for an axis to count as non-periodic (default 10 Å; Section 4.4).

`--symprec / --angle-tolerance`
: Tolerances handed to spglib for every symmetry analysis.

`-p, --pseudo-dir`
: A bundled pseudopotential bank (`dojo`, `virtual_vault`) or a folder (Section 4.3).

`--model / --custom-model / --custom-model-card / --device`
: The MACE potential and the device it runs on; a fine-tuned model is always given with its model card (Chapter 6).

3.4 Reports

Every tool prints a report in numbered sections. A typical one opens with `[0] RUN METADATA` (tool, version, date, command line, input files), then one section per step of the method – input data, checks, results, interpretation – and, in the newer tools, closes with a `WARNINGS` section that collects every warning raised during the run, so none is lost in a long output.

The markers have fixed meanings:

`[OK]`	a check passed (the threshold is printed next to it)
`[INFO]` / `[NOTE]`	context: what was assumed, detected or chosen, and why
`[WARNING]`	the result was produced but deserves attention
`[ERROR]`	the input was refused; nothing questionable was written
`[SKIPPED]`	a check did not apply (for example atomic density of a slab)
`[UNVERIFIED]`	a formula or value that could not be pinned to a primary source
`[EXTRAPOLATED]`	a prediction outside the range the data constrain

Structure-reading tools run the same malformation checklist (`core/structure_checks.py`): *atom proximity* (minimum-image distance below 0.5 Å is flagged), *lattice handedness* (a negative cell determinant), and *atomic density* (outside 0.01–0.15 atoms/Å^3; skipped for vacuum-padded cells, whose volume is mostly vacuum by design).

3.5 Bibliography files

Whenever an output depends on a citable method, code or database (SIESTA itself, Monkhorst–Pack, a k-path convention, a pseudopotential bank, MACE, a dispersion correction, a database a structure came from, …), the tool writes a `references.bib` next to its output, with no flag needed. A `references.bib` already present in the folder is merged entry by entry, so running several tools in one working folder builds one combined bibliography.

3.6 Plots

Two conventions coexist, by category:

- The **Analysis** tools and the **Workflow** analysis stages write gnuplot pairs: a `.dat` with the numbers and a `.gplot` script that reads it. The scripts use bare file names, so run them from the folder they are in: `cd out/ && gnuplot plot.gplot`. With `-view` the same data are shown in a matplotlib window, never saved.
- The **ML Simulations** tools and the MLFF/optical/GQCA analyses write matplotlib PNG files directly (often multi-panel dashboards), shown with `-show`.

3.7 Trajectory formats

Tools that write several frames (**`stb-ani2traj`**, **`stb-mlmd`**, **`stb-mlphonons`**) share three formats, all written by ASE's multi-frame writers:

`xsf` (default)
: animated XSF (AXSF), read by XCrySDen, VESTA and OVITO, carries the cell of every frame;

pdb
: multi-model PDB with a `CRYST1` record, for VMD;

xyz
: extended XYZ with a `Lattice=` tag and optional per-frame metadata (energy, temperature, …).

Chapter 4

SIESTA files and conventions

In this chapter

4.1 The structure file and the calculation template . . . 10
4.2 `config_extra.fdf`: forcing directives without editing a template . . . 11
4.3 Pseudopotentials . . . 11
4.4 Vacuum and dimensionality . . . 12
4.5 Output files of SIESTA read by the suite . . . 12
4.6 Atoms near the cell boundary . . . 13

4.1 The structure file and the calculation template

Most tools work with the usual SIESTA split of an input into two files:

`structure.fdf`
: the geometry: `NumberOfAtoms`, `NumberOfSpecies`, `%block ChemicalSpeciesLabel`, `LatticeConstant`, `%block LatticeVectors`, `AtomicCoordinatesFormat` and `%block AtomicCoordinatesAndAtomicSpecies`;

`calc.fdf`
: everything else – `SystemLabel`, basis, XC functional, mesh cutoff, k-grid, SCF and MD settings – with a line `%include structure.fdf`.

A preparation tool rewrites only the structure file in each folder it writes and copies the template verbatim, so the level of theory is defined once, by you. **`stb-inputfile`** (1.1) writes a sensible pair from any structure. Several tools also accept one self-contained `.fdf` that holds both.

How the suite reads a structure file.. The single `.fdf` reader (`core/structure_io.py`) honours the unit of `LatticeConstant` (Å, Bohr/au, nm) and stores the cell in Å; a cell written as `LatticeConstant 1.0 Bohr` (as phonopy's SIESTA writer does) is therefore read correctly. All coordinate formats SIESTA accepts are understood (Å, Bohr, fractional/`ScaledByLatticeVectors`, `ScaledCartesian`). When a tool rewrites an existing file's `%block LatticeVectors` it writes the block in the units of the file's own `LatticeConstant`, so the constant is never applied twice. Every file the suite writes numbers `%block ChemicalSpeciesLabel` sequentially from 1, in the order of the original indices: SIESTA requires the block to have no gaps.

Fractional coordinates for strained cells.. Tools that deform a cell (strain, elastic constants, EOS scans) replace only the lattice vectors. That is correct only when positions are fractional, which follow the cell for free; these tools check the coordinate format and refuse Cartesian input instead of producing a wrongly strained structure.

Relaxed geometries.. Wherever a tool reads "the result of a previous run", it accepts either a `.fdf` file or a folder. Given a folder, it prefers the finished `<SystemLabel>.XV` over the input guess and cross-checks the species of every atom in the `.XV` against the expected order, so an atom swap cannot go unnoticed.

4.2 config_extra.fdf: forcing directives without editing a template

SIESTA's fdf reader has two properties the suite relies on:

1. labels are compared ignoring case and the characters `.`, `_` and `-` (`MD.TypeOfRun`, `md-typeofrun` and `MDTypeOfRun` are the same label);
2. when a label appears more than once, the *first* occurrence wins.

A workflow that must impose a setting therefore writes it into `config_extra.fdf` and *prepends* `%include config_extra.fdf` at the very top of the copied `calc.fdf`. Whatever your template says later about the same label is ignored by SIESTA, and the template text stays exactly as you wrote it. Typical forced blocks are:

Purpose	Directives
single point	`MD.TypeOfRun CG`, `MD.Steps 0` / `MD.NumCGsteps 0`
fixed cell	`MD.VariableCell false`
relaxation	`MD.TypeOfRun CG`, `MD.NumCGsteps N`, `MD.VariableCell T`
spin	`Spin polarized`
slab dipole	`Slab.DipoleCorrection` [22, 23]
dispersion	`DFTD3 .true.` [24, 25]
grids for differences	one explicit `Mesh.Cutoff`, `SaveRho true`
k-sampling	a `%block kgrid.MonkhorstPack` matched to each cell

Because the forced settings live in a separate file, later stages can *read them back*. This is how the level of theory propagates: a stage that derives new calculations from an earlier one inspects that stage's `config_extra.fdf` to learn whether spin, dipole correction or dispersion were really applied there.

Why prepend, not append

An `%include` added at the *end* of a template – or after its structure include – is silently ignored for every label the template already sets, because the template's own value is read first. A relaxation template that sets `MD.TypeOfRun` and `MD.Steps` is the common case; only the prepended include reliably turns it into a single point.

4.3 Pseudopotentials

Tools that write ready-to-run folders copy the pseudopotential of every species into each folder. The source, given with `-p`/`--pseudo-dir` (or `--pp-path`), is either a folder of your own or one of two banks shipped with the package:

Name	Format	Content
`dojo`	PSML [26]	PseudoDojo v0.5, PBE, standard accuracy [27]; 72 elements
`virtual_vault`	PSF	SIESTA Pseudopotentials Virtual Vault (Troullier–Martins [28]); 69 elements

Files are *copied*, never symlinked: a link into the package or into another folder breaks as soon as the run folder is copied to a cluster. **`stb-pseudo`** (**1.2**) resolves and copies pseudopotentials on its own and can fall back to a second bank for missing elements.

A bank is not a guarantee of quality

Check a new pseudopotential against a known property (a lattice constant, a bond length) before production. In one test the PseudoDojo sulfur gave a wrong Mo–S bond length in MoS_2; the optical example uses the Virtual Vault carbon for graphene for this kind of reason.

4.4 Vacuum and dimensionality

Molecules, wires and slabs are modelled in SIESTA as periodic cells padded with vacuum. Many tools need to know which axes are truly periodic: k-grids must use a single point along a vacuum axis, elastic and stress tensors are reported per area or per length, EOS and melting refuse non-bulk cells, strain must not be applied across vacuum. The suite detects this the same way everywhere (`core/kspace.py`): along each lattice axis, the fractional coordinates of all atoms are placed on a ring (so the gap that wraps around the cell boundary is counted) and the largest empty span is measured; if it exceeds `--vacuum-gap` (default 10 Å), the axis is non-periodic. The number of periodic axes gives the dimensionality: 3D bulk, 2D slab, 1D wire, 0D molecule. The same wrap-aware gap is used to find *where* a surface is (the atom just below the largest gap), which matters when a fetched structure has its atoms straddling the cell boundary with the vacuum in the middle of the cell.

4.5 Output files of SIESTA read by the suite

SIESTA names its output files after `SystemLabel`. The table lists those the analysis tools read and the directive that makes SIESTA write each one [29].

File	Written when	Read by
`.out` (stdout)	always (redirect it, e.g. `calc.out`)	almost every analysis: energies, Fermi level, stress, forces, cell, SCF convergence, optical block, native charges
`.XV`	always (final cell and positions)	every tool that reads a relaxed geometry
`.FA`	always (forces)	phonons, Raman, IR, MLFF, NEB cycles
`.bands`	`BandLinesScale` + `%block BandLines`	**stb-bands**, **stb-fatbands**, **stb-ipr**
`.PDOS` (XML)	`%block ProjectedDensityOfStates`	**stb-dos**
`.EIG`	always (eigenvalues)	**stb-opticalAnalysis** (Kohn–Sham gap)
`.RHO`	`SaveRho true`	**stb-bader**, **stb-density**, **stb-cube**, charge difference, Hirshfeld-I
`.VT`, `.VH`	`SaveTotalPotential` / `SaveElectrostaticPotential`	**stb-workfunction**, **stb-cube**
`.LDOS`	`%block LocalDensityOfStates`	**stb-stm**
`.bands.WFSX`	`WFS.Write.For.Bands T`	**stb-fatbands**, **stb-ipr**, **stb-spintexture**
`.selected.WFSX`	`WriteWaveFunctions T` + `%block WaveFuncKPoints`	**stb-wfdensity**, **stb-sts**, **stb-coop**, **stb-effmass**
`.HSX`	`SaveHS true`	overlap-aware projections, COOP/COHP, effective masses, the Hamiltonian export of **stb-siesta2wtb**
`.DM`	always	bond orders in **stb-coop**
`.ANI`, `.MDE`	molecular dynamics	**stb-ani2traj**, **stb-aimdAnalysis**
`.EPSIMG`	`Optical.Calculate T`	**stb-opticalAnalysis**, **stb-ramanAnalysis**
`.BC`	`BornCharge T`	**stb-irAnalysis**
`.ion`, `.ion.xml`	always (basis)	real-space orbitals for **stb-wfdensity**, **stb-sts**

Logs with any name.. Real jobs often redirect SIESTA's output to a generic name (`calc.out`, `siesta.out`, `slurm-1234.out`). Tools that need the log locate it (`core/siesta_log.find_out_file`) rather than assuming `<SystemLabel>.out`, and those that combine several files accept an explicit path for each (`–geometry-file`, `–fermi-file`, `–hsx-file`, ...) in addition to `–label`, because the input `.fdf` is very often not named after its own `SystemLabel`.

Fermi level.. Tools that shift energies to the Fermi level resolve it in a fixed order: an explicit `–fermi`, then a `.bands` file, then `–fermi-file`, then a log found automatically.

Parsers never crash a scan.. Every log parser returns "not found" rather than raising, because analysis stages scan many folders and must tolerate a run that is unfinished or failed. The report then lists which folders were usable and why the others were not.

4.6 Atoms near the cell boundary

After any step that can move atoms (a relaxation, MD, an ML pre-relaxation), fractional coordinates are wrapped into $[0, 1)$ before anything compares or writes them. An unwrapped coordinate near a cell face makes "the same atom" in two snapshots look far apart, which corrupts NEB endpoint matching and displacement diagnostics. Distances between atoms are always computed with the minimum-image convention.

Chapter 5

Workflows in practice

In this chapter

5.1 The prepare–run–analyse cycle 14
5.2 Running many folders 14
5.3 What the analysis stages check 15
5.4 Converge first 15

5.1 The prepare–run–analyse cycle

The property workflows of Category 4 all follow the same pattern:

1. **Prepare** (stage 1, sometimes more): from your relaxed structure and your calculation template, write one ready-to-run folder per calculation the method needs – strained cells, displaced supercells, isolated atoms, image folders, polarization directions, … Each folder holds the structure, the copied template with its `config_extra.fdf`, and the pseudopotentials.
2. **Run SIESTA** in every folder, in whatever way suits your machine.
3. **Analyse** (the last stage): point the analysis tool at the top folder; it finds every finished run, checks each one (SCF convergence, residual forces, level of theory), and computes the property.

Some workflows need an intermediate preparation stage because the folders it writes depend on results that do not exist before SIESTA runs: the Hubbard-U perturbations (5.7.2), the BSSE ghost fragments at relaxed adsorption geometries (5.8.2), the Raman/IR optical displacements along computed phonon modes (5.11.2, 5.12.2), the Hirshfeld-I ions (5.17.2). As a rule, an analysis stage never writes new geometry; the single documented exception is `–compute-gibbs` of 5.8.3, which prepares the vibrational folders of stage 4.

5.2 Running many folders

A local loop is enough for small studies:

```
for d in strain_out/xx/strain_xx_*/; do
  (cd "$d" && mpirun -np 8 siesta < calc.fdf > calc.out)
done
```

On a cluster, submit one job per folder, or a job array over the folder list. **`stb-status`** (3.6) summarizes any number of folders – run type, SCF convergence, maximum force, final energy, which outputs exist – so you can see at a glance which runs finished; **`stb-clean`** (3.2) resets a folder for a fresh run and **`stb-archive`** (3.7) packages a finished one.

5.3 What the analysis stages check

Before computing a property, the analysis stages verify that the runs they combine are comparable and finished:

- every run reached SCF convergence and, for relaxations, a residual force below the requested tolerance (or did not stop because the step budget ran out);
- all folders share the same settings that matter – mesh cutoff, basis, XC functional, k-sampling (compared as $k \times a$, so a supercell with a divided grid still matches), pseudopotential checksums, spin, dispersion;
- the geometry is the one expected (cell, number and species of atoms).

Inconsistent folders are reported, and either left out or flagged in the result.

5.4 Converge first

The precision of every derived property is bounded by the precision of the underlying calculations. Before a property study, use the convergence workflow (**5.5**) to choose `Mesh.Cutoff`, `PAO.EnergyShift` and the k-grid density. Because SIESTA evaluates part of the energy on a real-space grid, forces and stresses carry a small grid-position-dependent noise (the "eggbox effect"), so that workflow judges convergence on the *relaxed geometry* as well as on the energy.

Chapter 6

Machine-learning potentials

In this chapter

6.1 MACE and the MACE-MP-0 foundation model 16
6.2 Where MACE appears in the suite 16
6.3 Choosing the model 16
6.4 Device and precision 17
6.5 Dispersion 17
6.6 Caveats that apply to every ML result 17

6.1 MACE and the MACE-MP-0 foundation model

The ML tools use MACE [15], an equivariant message-passing neural network that predicts energies, forces and stresses from atomic positions. The default model is MACE-MP-0 [16], a *foundation model* trained on Materials Project relaxation data across the periodic table, available in three sizes, `small`, `medium` (the usual default) and `large`; it is downloaded and cached on first use. A foundation model gives qualitatively sensible structures, phonons and elastic constants for a broad range of materials [30, 31], at a tiny fraction of the cost of DFT, but it is *not* SIESTA: it was trained on a different code, functional set and energy reference, and its errors are material dependent.

6.2 Where MACE appears in the suite

- **ML Simulations** (Category 5) run whole simulations with MACE instead of SIESTA: MD, phonons, elastic constants, EOS, NEB, vacancy diffusion, adsorption screening.
- **Pre-screening and pre-relaxation** inside structure and workflow tools: **`stb-mlrelax`**, **`stb-amorphize`**, `-ml-relax` of the structure builders, `-ml-rank` of **`stb-defect`** and **`stb-adsorb`**, the MACE path of **`stb-neb`**. Here MACE only prepares or ranks candidates; SIESTA still produces the result.
- **A custom force field**: the MLFF workflow (**7.1**) fine-tunes MACE-MP on your own SIESTA data, validates it, and extends the training set by active learning. The resulting `.model` file can be used by every tool that takes `-custom-model`, always together with its model card (`-custom-model-card`, see below).

6.3 Choosing the model

`-model small|medium|large`
a MACE-MP-0 size. A larger model is not necessarily closer to DFT: for silicon the large one was the softest (Sections 13.3 and 13.4).

-custom-model PATH
: a fine-tuned `.model`. Tools that accept it also run the foundation model on the same structure and overlay the two results (plots and tables), unless `-skip-foundation-comparison` is given – the difference shows what the fine-tuning changed.

-custom-model-card JSON
: required with `-custom-model`: the `mlff_model_card.json` that **stb-mlffAnalysis** writes next to every model it trains. It records the elements, the cell sizes and bond lengths of the training set, the SIESTA level of theory (including whether the labels already contain DFT-D3), the energy scale, the training recipe and the accuracy reached. The tool refuses a card that does not belong to the model file (SHA-256), never adds a second dispersion correction to a model whose labels already have it, stops with a clear message for an element the model was not trained on, and warns once when the structure leaves the trained range (cell size, shortest bond) or the predicted forces exceed the largest training force by 50%. The options are spelled `-ml-custom-model-card` in **stb-adsorb** and **stb-stackingfault**, and `-mace-custom-model-card` in **stb-xrdsearch**.

6.4 Device and precision

`-device` (or `-ml-device`, `-mace-device`) is `cpu` by default. `cuda` is checked before anything is loaded: if PyTorch was built without CUDA or no GPU is visible, the tool stops with an explanation instead of falling back to the CPU without telling you. The one deliberate exception is the MACE pre-screening of **stb-adsorb**, which degrades to CPU with a `[WARNING]`, because MACE is only a preliminary step there. Geometry optimizations run in double precision (float64, as recommended for MACE); molecular dynamics may use float32. MLFF training (**7.1.2**) selects `cuda` automatically when available.

6.5 Dispersion

When the DFT calculation a MACE step approximates forces Grimme's DFT-D3 correction (adsorption sites, NEB with `-force-vdw`), the MACE step adds D3(BJ) [24, 25] on top of the model (package `torch-dftd`), so both see the same level of theory. A model fine-tuned on DFT-D3 labels already contains the dispersion; never add it a second time.

6.6 Caveats that apply to every ML result

- Validate against DFT for your system before trusting a number; the MLFF workflow exists for exactly this.
- Methods have their own limits independent of the potential, documented in each tool's section (for example, single-phase heating superheats past the melting point).

Part II

Tool reference

Chapter 7

Inputs

In this chapter

7.1 Input File Generator 20
7.2 Pseudopotential Resolver 26
7.3 K-Grid Generator 31
7.4 K-Path Generator 35
7.5 DFT+U / Hubbard Block Generator 39
7.6 Structure Fetcher 45
7.7 ML Pre-Relaxation 50

The tools of this category set up a SIESTA run: they turn a structure into a complete input (**stb-inputfile**), resolve and copy its pseudopotentials (**stb-pseudo**), choose the Brillouin-zone sampling (**stb-kgrid**) and the band-structure path (**stb-kpath**), write a DFT+U block (**stb-dftu**), obtain a starting structure from a database (**stb-fetch**) and pre-relax it cheaply with a machine-learning potential (**stb-mlrelax**).

A typical start of a project chains them:

```
stb-fetch --source cod --cod-id 1010369 --unitcell primitive -o structure.fdf   # 1.6
stb-mlrelax -f structure.fdf --relax-cell -o relaxed.fdf                        # 1.7
stb-inputfile relaxed.fdf -t relax -p dojo                                      # 1.1
stb-kpath -f relaxed.fdf -o kpath_bs.fdf                                        # 1.4
```

stb-inputfile already computes the k-grid with the logic of **stb-kgrid** and copies the pseudopotentials with that of **stb-pseudo**; the two standalone tools are for inspecting or redoing those choices on their own.

Code	Tool	Purpose
1.1	Input File Generator	Create a 'calc.fdf' input file from a structure file.
1.2	Pseudopotential Resolver	Resolve the pseudopotentials a structure needs and copy them into a run folder.
1.3	K-Grid Generator	Suggest a Monkhorst-Pack grid (k-points) based on desired density.
1.4	K-Path Generator	Generate a high-symmetry k-path for band structure calculations.
1.5	DFT+U / Hubbard Block Generator	Generate a ready-to-use %block LDAU.proj snippet from a user-supplied U (and J).
1.6	Structure Fetcher	Fetch a structure from Materials Project or COD and write it as .fdf.
1.7	ML Pre-Relaxation	Fast pre-relaxation with the MACE-MP-0 potential (needs the optional 'ml' extra).

7.1 Input File Generator

At a glance

Menu code: 1.1 **Command:** `stb-inputfile`
Guided example: `examples/1.1-stb-inputfile/`
Summary: Create a 'calc.fdf' input file from a structure file.

7.1.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/1.1-stb-inputfile/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

7.1.1.1 What this tool does

You almost always start a SIESTA project with a *structure* — a set of atoms in a box, with no calculation settings attached (that's what `structure.fdf` in this folder is). `stb-inputfile` takes that bare structure and turns it into a complete, ready-to-run `calc.fdf`: it works out a basis set, computes a Monkhorst-Pack k-point grid from the cell's own density, and drops in the right SIESTA blocks for whatever kind of run you asked for. Instead of hand-assembling a `.fdf` file from scratch (and re-deriving the k-grid by hand every time your cell changes), you answer one question — "what am I calculating?" — and get a working input file back.

Before writing `calc.fdf`, it also always runs a quick structure validation pass — symmetry (space group / crystal system) plus a handful of malformation checks (atoms too close together, a left-handed cell, an implausible density, a stale atom-count header) — and reports whatever it finds as `[WARNING]` lines. This never blocks generation; it's a heads-up, not a gate.

7.1.1.2 When you'd reach for it

Any time you have a structure file (freshly built with `stb-slab`, `stb-supercell`, `stb-crystalbuilder`, fetched with `stb-fetch`, converted with `stb-translate`, ...) and need to turn it into an actual SIESTA calculation. It's normally the very last step of structure preparation and the very first step of running SIESTA.

7.1.1.3 Two ways to run it

A — direct CLI, the fastest once you know the flags:

```
stb-inputfile structure.fdf -t relax -p dojo
```

B — interactive `stb-suite` menu, which asks the same questions as guided prompts instead of flags:

```
stb-suite
# at the main prompt, type: 1.1
```

`1.1` jumps straight past the category menus (`1` = Inputs, item `1` = Input File Generator) to this tool. It then asks you, one at a time: which structure file, which calculation mode (numbered list), whether

to enable DFT-D3, whether to enable spin polarization, an optional pseudopotential source, whether to save a text report, and whether to open the interactive 3D viewer — building the exact same `stb-inputfile ... -t ... -d3 -s -p ... --view` command underneath and running it for you. Good for exploring what a tool can do before you've memorized its flags; the CLI is faster once you have.

Both paths run the identical underlying tool and produce byte-identical output — `example_1.1.sh` demonstrates this directly, running the same `relax` calculation both ways and diffing the results.

7.1.1.4 Files in this folder

- `structure.fdf` — bulk-silicon example structure (conventional cubic diamond cell, 8 atoms, 3D/fully periodic).
- `structure_molecule.fdf` / `structure_chain.fdf` / `structure_graphene.fdf` — an isolated CH4 molecule (0D), an infinite carbon chain (1D), and a graphene monolayer (2D).
- `example_1.1.sh` — the guided walkthrough (**not** an automated test — see `test/1-inputs/1-input_file/test.sh` for that). Pauses between sections so you can read before moving on; safe to re-run.
- `output/` — created by `example_1.1.sh` when you run it (git-ignored, not checked in). See below.

7.1.1.5 Running the walkthrough

```
./example_1.1.sh
```

`stb-inputfile` always needs pseudopotentials to be handed off to SIESTA for real, so every case below is generated with `-p dojo` and written into its **own folder** under `output/` — each one already self-contained (structure file + `calc.fdf` + pseudopotentials, and `kpath_bs.fdf` for `bands/`) and ready to run in SIESTA as-is, no extra assembly needed:

Folder	Command (conceptually)
`output/relax/`	`stb-inputfile structure.fdf -t relax -p dojo`
`output/relax_d3/`	`stb-inputfile structure.fdf -t relax -d3 -p dojo` — same mode, DFT-D3 switched on
`output/relax_spin/`	`stb-inputfile structure.fdf -t relax -s -p dojo` — same mode, spin-polarized
`output/bands/`	`stb-inputfile structure.fdf -t bands -p dojo`, then `stb-kpath -f structure.fdf` to generate the `kpath_bs.fdf` that `bands` mode references
`output/molecule/`	`stb-inputfile structure_molecule.fdf -t total_energy -p dojo`
`output/chain/`	`stb-inputfile structure_chain.fdf -t total_energy -p dojo`
`output/graphene/`	`stb-inputfile structure_graphene.fdf -t total_energy -p dojo`
`output/silicon/`	`stb-inputfile structure.fdf -t total_energy -p dojo`

The script also drives the exact same `relax` case through the interactive `stb-suite` menu (non-interactively, via a piped `printf`, no folder kept for it) and diffs the result against `output/relax/calc.fdf` — proving the CLI and the menu produce identical output.

Structure validation (always on, every case above). Every run prints a `[2] STRUCTURE VALIDATION` section, right after parsing the structure and before the k-grid is even computed. It always runs -- there's no flag to turn it on, and it never blocks generation. Every check is listed **explicitly**, one row per check, each marked `[OK]`/`[WARNING]`/`[SKIPPED]` (via the shared `core/structure_checks.py` module, also used by `stb-fetch`/`stb-mlrelax`) -- not just a summary line -- so it's always clear exactly what was verified:

```
Check                | Result                                                                    | Status
---------------------------------------------------------------------------------------------
```

```
Atom proximity         | min. distance = 2.211 Ang (>= 0.5 Ang threshold) | [OK]
Lattice handedness     | right-handed (positive determinant)              | [OK]
Atomic density         | 0.0500 atoms/Ang^3 (within [0.01, 0.15])         | [OK]
NumberOfAtoms header   | declared 8, matches actual atom count            | [OK]
NumberOfSpecies header | declared 1, matches actual species count         | [OK]
```

Five specific things get checked:

1. **Atoms too close together** -- any two atoms closer than 0.5 Ang (minimum-image distance). Usually means two overlapping/duplicate atoms from a bad edit or a bad supercell/merge operation. `[SKIPPED]` if the structure has fewer than 2 atoms.
2. **Left-handed cell** -- the lattice vectors' determinant is negative (negative cell volume). A silent geometry bug: SIESTA will still run, but stress/chirality-sensitive downstream analysis can pick up sign errors from it.
3. **Implausible atomic density** -- atoms-per-Angˆ3 outside roughly `[0.01, 0.15]`, a deliberately generous band covering everything from light/loosely-packed solids to dense covalent ones. The classic trigger is a `LatticeConstant` given in the wrong unit (e.g. Bohr instead of Angstrom) -- the cell ends up ~6.7x too large or too small in volume. **Only checked for a genuine 3D bulk structure** (see the vacuum note below) -- a vacuum-padded cell's volume is dominated by the artificial vacuum by design, so this metric would be meaningless (and would false-positive) there; `[SKIPPED]` instead, explicitly, rather than silently absent.
4. **Stale atom-count header** -- the `.fdf` file's own declared `NumberOfAtoms`/`NumberOfSpecies` line doesn't match what's actually in `%block AtomicCoordinatesAndAtomicSpecies`/`%block ChemicalSpeciesLabel`. Catches a header left un-updated after hand-editing the structure. `[SKIPPED]` if a given header line isn't present at all (they're not required) -- two independent rows, since either header can be present/absent on its own.
5. **Symmetry** -- space group and crystal system, always printed (informational, not a warning by itself). **This is where dimensionality matters**: an ordinary space group only means what it says for a fully 3D-periodic structure. `molecule/`, `chain/`, and `graphene/` below are each vacuum-padded along one or more axes, so `stb-inputfile` still reports *a* space group (spglib will happily analyze the padded cell as if the vacuum were just another periodic direction), but appends this extra `[WARNING]`:

   ```
   [WARNING] Structure has a vacuum-padded axis -- the space group above
   treats it as an ordinary periodic direction and may not reflect the
   true symmetry. Use stb-symmetry (code 2.5) for a dimension-aware
   layer-group/point-group analysis.
   ```

   In other words: for `silicon/` (fully 3D) the printed space group is the real, physically meaningful answer. For `molecule//chain//graphene/` (0D/1D/2D) it's only a rough approximation of a real 3D crystal that doesn't actually exist -- the vacuum thickness itself can even change which "space group" comes out, since spglib has nothing else to base a 3D classification on. If you need the *correct* symmetry for a non-3D structure (a proper 2D layer group, or a 0D point group), run `stb-symmetry` (code 2.5) directly instead of trusting this quick check.

Run `stb-inputfile structure.fdf -t total_energy -p dojo` yourself and look for `[2] STRUCTURE VALIDATION` in the output to see all of this live.

Visualizing the structure (`--view`). `--view` opens the structure in ASE's interactive 3D viewer right before the tool exits (needs a local display, or `ssh -X`/`-Y`). Off by default -- not used in this script (an unattended walkthrough shouldn't pop a GUI window), but try it yourself:

```
stb-inputfile structure.fdf -t relax -p dojo --view
```

Why `bands/` needs an extra command. `stb-inputfile`'s `bands` mode writes a `calc.fdf` that references `%include kpath_bs.fdf`, but doesn't generate that file itself — that's `stb-kpath`'s (code 1.4) job. The script calls it too, so `output/bands/` ends up genuinely complete instead of missing a file SIESTA would need.

Why dimensionality matters. A k-point grid only makes physical sense along a genuinely periodic direction. `stb-inputfile` measures the largest empty gap along each lattice vector and, above a threshold (10 Ang), treats that axis as vacuum instead of periodic when computing the k-grid — automatically, from the geometry alone, no flag needed:

Folder	Structure	Dimensionality	Periodic axes	Typical k-grid shape
`molecule/`	`structure_molecule.fdf` (CH4)	0D	0	`1 1 1` (isolated)
`chain/`	`structure_chain.fdf` (carbon chain)	1D	1	`N 1 1` (a wire)
`graphene/`	`structure_graphene.fdf` (graphene)	2D	2	`N N 1` (a slab)
`silicon/`	`structure.fdf` (bulk silicon)	3D	3	`N N N` (a crystal)

This is why the same `stb-inputfile <structure> -t total_energy` command works unchanged whether the structure is a molecule, a wire, a slab, or a bulk crystal — the vacuum gap in the structure file itself is the signal, so you never need a separate "molecule mode" or "slab mode" flag.

7.1.1.6 Try it yourself

Once you've read through the walkthrough, experiment directly:

```
stb-inputfile structure.fdf -t total_energy -p dojo
stb-inputfile structure.fdf -t aimd -p dojo
stb-inputfile structure.fdf -t bands -d3 -s -p dojo   # -d3 and -s combine with any mode
```

Open each generated `calc.fdf` and compare it against `structure.fdf` and against the other modes — that's the fastest way to see exactly what each mode adds.

7.1.1.7 Flag reference

```
stb-inputfile <structure_file> -t <mode> [-d3] [-s] [-p <pseudo-source>] [--view]
```

- `-t/--type` (required): `total_energy`, `relax`, `aimd`, or `bands`.
- `-d3/--d3` (optional): enable the DFT-D3 (Grimme) dispersion correction, independent of and combinable with any mode above.
- `-s/--spin-polarized` (optional): enable spin polarization, also independent of and combinable with any mode above. Defaults to non-polarized when omitted.
- `-p/--pp-path` (optional): a bundled bank name (`dojo`, ...) or a path to a folder containing `.psml`/`.psf` pseudopotential files.
- `--view` (optional): open the structure in ASE's interactive 3D viewer after generation. Off by default; needs a display.

Structure validation (symmetry + malformation warnings, see below) always runs before generation -- there's no flag for it, it's always on.

Run `stb-inputfile --help` for the full list of options.

7.1.1.8 What's next

Any `output/<case>/` folder is ready to hand to SIESTA as-is. After the run finishes, `stb-status` gives you a quick summary of what happened (converged? final energy? which output files were produced), and the `Analysis` category tools (`stb-bands`, `stb-dos`, ...) pick up from there.

7.1.2 Implementation details

What is written.. **`stb-inputfile`** always writes a file named `calc.fdf` in the *current folder* (the name is fixed), plus `references.bib`, and copies one pseudopotential per species when `-p` is given. It does not rewrite the structure file: `calc.fdf` contains `%include <structure file name>`, so keep the two files together. The steps, which are also the sections of the report, are: `[0]` run metadata, `[1]` template selection, `[2]` structure validation (malformation checklist, a check that the declared `NumberOfAtoms` and `NumberOfSpecies` match the blocks, space group and crystal system), `[3]` k-grid, `[4]` writing, `[5]` references, `[6]` pseudopotentials and `[7]` summary.

The four templates.. Each `-type` selects a complete, commented template; only four things are substituted in it: the `%include` line, the k-grid, `DFTD3` (`-d3`) and `Spin` (`-s`). Everything else is a starting point to review, summarized in Table 7.4. Commented lines list the most useful optional directives of each section (basis tuning, mixer variants, relaxation tolerances, output flags), ready to be uncommented.

Table 7.4: Active directives of the **`stb-inputfile`** templates.

Directive	`total_energy`	`relax`	`aimd`	`bands`
`PAO.BasisSize`	DZP	DZP	SZ	DZP
`PAO.EnergyShift`	0.02 Ry	0.01 Ry	0.02 Ry	0.02 Ry
`Mesh.CutOff` / `FilterCutoff`	320 Ry / 150 Ry in all templates			
`XC.Functional/XC.Authors`	GGA / PBE [32]			
`kgrid.MonkhorstPack`	from density	from density	$1 \times 1 \times 1$ (kept)	from density
SCF	`MaxSCFIterations` 300, `SCF.Mixer.Weight` 0.1, `SCF.Mixer.History` 6, 300 K smearing			
`SCF.DM.Tolerance`	10^{-5}	10^{-5}	10^{-4}	10^{-5}
MD	–	CG, variable cell, 150 steps, 0.01 eV/Å	Nosé [33], 1 fs, 5000 steps, 300 K, $2\times2\times2$ `Supercell`	–
extra	–	commented `Geometry.Constraints`	`MD.UseSaveXV`	`%include kpath_bs.fdf`, PDOS block

The k-grid.. For `total_energy`, `relax` and `bands` the grid is computed with a fixed density of 0.2 $Å^{-1}$ by the same routine as **`stb-kgrid`** (1.3), with one k-point along every vacuum-padded axis (10 Å threshold). For `aimd` the template's Γ-only grid is kept on purpose: an MD supercell is large and the template already replicates the cell $2 \times 2 \times 2$. Run **`stb-kgrid`** if you want a different density, and paste its line.

The bands template.. It expects a `kpath_bs.fdf` with the `%block BandLines` written by **`stb-kpath`** (1.4) next to `calc.fdf`; the guide's section on `bands/` shows the extra command. It also asks SIESTA for a projected DOS (`ProjectedDensityOfStates` from -20 to 20 eV with 8000 points and a 0.01 eV broadening, on a $20 \times 20 \times 20$ `PDOS.kgrid_Monkhorst_Pack`; reduce it along vacuum axes), which **`stb-dos`** (2.2) reads.

Pseudopotentials.. With `-p` the tool looks in the bank or folder for a file named after each species label of `ChemicalSpeciesLabel` (`Si.psml`, `Si.psf`); when both a `.psml` and a `.psf` exist, the `.psml` is preferred.

Missing elements are listed as warnings in section [6b]; the input is still written. **stb-pseudo** (1.2) offers more control (fallback bank, dry run).

Citations.. The references.bib contains exactly what the generated calculation uses: SIESTA, PBE, Monkhorst–Pack (except AIMD), DFT-D3 if -d3, the Nosé thermostat for AIMD, and the pseudopotential bank when a bundled one was used.

Before running

The templates are generic. Converge Mesh.CutOff, PAO.EnergyShift and the k-grid for your system (5.5), choose spin and dispersion consciously, and change SystemLabel if several runs share a folder.

7.1.3 Command-line reference

7.1.3.1 stb-inputfile

Script to prepare a Siesta input file (.fdf).

Synopsis.

```
usage: stb-inputfile [-h] -t {total_energy,relax,aimd,bands} [-d3] [-s] [-p PP_PATH]
                     [--save-report] [--view] [-v] [--no-intro]
                     structure_file
```

Positional arguments

structure_file
: Path to the structure file (e.g., struct.fdf) — required

Options

-t, --type {total_energy,relax,aimd,bands}
: Calculation mode. Valid choices: total_energy, relax, aimd, bands — required; choices: total_energy, relax, aimd, bands

-d3, --d3
: Enable the DFT-D3 (Grimme) van der Waals dispersion correction, on top of any calculation mode. — switch, off by default

-s, --spin-polarized
: Enable spin polarization. Default: non-polarized. — switch, off by default

-p, --pp-path PP_PATH
: Pseudopotentials source (optional): a bundled bank (dojo, virtual_vault) or a folder path.

--save-report
: Also persist the report to stb_inputfile_report.txt. Off by default. — switch, off by default

--view
: Open an interactive 3D view of the structure (via ASE) after generating calc.fdf. Needs a display. Off by default. — switch, off by default

Like every STB command it also accepts -h/-help, -v/-version and -no-intro (Section 3.3).

From the built-in help.

```
Usage examples:
  stb-inputfile struct.fdf -t relax -d3 -p /path/to/pps
  stb-inputfile system.fdf -t bands
  stb-inputfile system.fdf --type aimd -s
```

7.2 Pseudopotential Resolver

At a glance

Menu code: 1.2 **Command:** `stb-pseudo`
Guided example: `examples/1.2-stb-pseudo/`
Summary: Resolve the pseudopotentials a structure needs and copy them into a run folder.

7.2.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/1.2-stb-pseudo/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

7.2.1.1 What this tool does

Before SIESTA can run a calculation, every chemical species in your structure needs a matching pseudopotential file (`.psf` or `.psml`) sitting in the run folder, named after the species *label* declared in `%block ChemicalSpeciesLabel`. `stb-pseudo` automates that step: give it a structure (or a plain list of elements) and a pseudopotential source, and it tells you exactly which elements are covered, which aren't, and copies the ones that are into a folder ready for `calc.fdf`. `--fallback-dir` lets you patch gaps from a second source automatically, instead of hunting down a missing file by hand.

7.2.1.2 Why this matters (a bit of theory)

A real atom's electrons split into two very different roles: a handful of **core** electrons, tightly bound near the nucleus, essentially inert to chemistry, and a few **valence** electrons, which is where all the actual bonding physics happens. Solving the all-electron problem exactly means resolving both — including the core electrons' deep, rapidly-oscillating wavefunctions near the nucleus, which is numerically expensive and, for chemistry, mostly wasted effort.

A **pseudopotential** replaces the true nucleus-plus-core-electrons Coulomb potential with a smoother *effective* potential, tuned so that outside a chosen core radius `r_c` the resulting pseudo-wavefunctions reproduce the real (all-electron) valence wavefunctions almost exactly. Inside `r_c` the pseudo-wavefunction is deliberately smooth (no more sharp oscillations to resolve), which is what actually makes the calculation cheap.

Norm-conserving pseudopotentials — the kind both banks `stb-pseudo` ships (see below) provide — add one more constraint: the pseudo- and all-electron wavefunctions must enclose the *same charge* inside `r_c`. This single constraint is what makes a pseudopotential **transferable**: it keeps the scattering properties (logarithmic derivatives) correct not just at one reference energy, but over the whole range of energies the atom actually experiences in different chemical environments. Without it, a pseudopotential

fitted for one bonding situation could silently give wrong results in another — transferability is the reason a pseudopotential library can be trusted as a general-purpose building block, rather than needing a fresh all-electron calculation for every new compound.

Two format notes that matter in practice:

- **`.psf`** is SIESTA's older native pseudopotential format (the `virtual_vault` bank ships these). **`.psml`** is a newer, code-agnostic exchange format that SIESTA also reads directly (the `dojo` bank ships these). Numerically they describe the same kind of object — `stb-pseudo` copies whichever is present, preferring `.psml` when both exist for a species.
- A pseudopotential is generated for one specific **exchange-correlation functional** (both bundled banks here are PBE). Using it in a calculation set up for a *different* functional is a real, silent correctness trap — `stb-pseudo` does not check this (it has no way to know what functional your `calc.fdf` will use), so it's on you to keep the two consistent.

7.2.1.3 When you'd reach for it

Right before (or right after) `stb-inputfile` (example `1.1`), to make sure every element your structure needs actually has a pseudopotential available — and to get them all copied into the run folder in one step, instead of resolving each one by hand. It's also the tool to reach for when a bundled bank is missing one particular element: `--fallback-dir` lets a second source fill just that gap, without abandoning the first source for everything else.

7.2.1.4 Two ways to run it

A — direct CLI:

```
stb-pseudo --file sic.fdf --pp-path dojo --output out/
```

B — interactive `stb-suite` menu:

```
stb-suite
# at the main prompt, type: 1.2
```

`1.2` first asks what you want to do — resolve from a structure file **[1]**, resolve from an explicit element list **[2]**, or just browse a bundled bank's contents **[3]** — then walks through the same primary source / fallback source / output directory / dry-run / save-report questions the CLI flags cover. `example_1.2.sh` proves the CLI and the menu agree.

7.2.1.5 Files in this folder

- `sic.fdf` — a 2-atom SiC primitive cell (Si + C, both covered by the bundled `dojo` bank).
- `example_1.2.sh` — the guided walkthrough (**not** an automated test — see `test/1-inputs/2-pseudo/test.sh` for that). Pauses between sections so you can read before moving on; safe to re-run.
- `output/` — created by `example_1.2.sh` when you run it (git-ignored, not checked in). See below.

7.2.1.6 Running the walkthrough

```
./example_1.2.sh
```

Folder	Command	What it shows
(none — just prints)	`stb-pseudo --list-elements dojo`	browsing a bank without any structure
`output/sic/`	`stb-pseudo -f sic.fdf -p dojo -o output/sic`	resolve + copy from a real structure
`output/fallback/`	`stb-pseudo --species Si At -p dojo --fallback-dir virtual_vault -o output/fallback`	filling a gap `dojo` can't cover
`output/dry_run/`	`stb-pseudo -f sic.fdf -p dojo -o output/dry_run --dry-run`	report only, nothing copied

Bundled banks: what's actually inside `dojo` vs `virtual_vault`. `stb-pseudo --list-elements` (used above with no structure at all) is the quickest way to check whether a bank covers what you need *before* committing to it:

- **`dojo`** — PseudoDojo v0.5, PBE, `.psml` format, 72 elements.
- **`virtual_vault`** — the SIESTA Pseudopotentials Virtual Vault, PBE, `.psf` format, a different (overlapping but not identical) element coverage.

Neither bank covers every element you might need — which is exactly why `--fallback-dir` exists, rather than forcing you to pick one bank and live with its gaps.

Filling the gaps: `--fallback-dir` made concrete. Astatine (`At`) is a real, if obscure, case: it's absent from `dojo` but present in `virtual_vault`. Resolving `Si At` against `dojo` alone reports `At` as `MISSING`; adding `--fallback-dir virtual_vault` resolves it from the second source instead, and the report says exactly which source supplied each element (`primary` vs `fallback`) — nothing is silently mixed up.

Proof: CLI and the interactive menu agree. The script also drives the same `sic.fdf` / `dojo` case through the interactive `stb-suite` menu (non-interactively, via a piped `printf`) and diffs the resulting "Resolved : N/N" line against the direct CLI run — proving the two paths produce identical output.

7.2.1.7 Try it yourself

```
stb-pseudo --species Fe O -p dojo -o out/          # a common oxide pair
stb-pseudo --list-elements virtual_vault           # what's in the other bank?
```

7.2.1.8 Flag reference

```
stb-pseudo (-f/--file <structure.fdf> | --species EL [EL ...]) -p/--pp-path <BANK_OR_PATH>
           [--fallback-dir <BANK_OR_PATH>] [-o/--output <dir>] [--dry-run] [--save-report]
stb-pseudo --list-elements <BANK>
```

- `-f/--file`: a SIESTA structure file (`.fdf`) to read the required species from. Mutually exclusive with `--species`.
- `--species`: element symbol(s) to resolve directly, instead of reading a structure file.

- `-p/--pp-path` (required unless `--list-elements` is given): a bundled bank name (`dojo`, `virtual_vault`) or a directory of your own `.psf`/`.psml` files.
- `--fallback-dir` (optional): a second source, consulted only for elements missing from `-p/--pp-path`.
- `-o/--output` (optional, default `.`): directory the resolved pseudopotentials are copied into.
- `--dry-run` (optional): only run the availability check — copy nothing.
- `--list-elements <BANK>`: print the elements available in a bundled bank and exit — no structure or `-p/--pp-path` needed.
- `--save-report` (optional): also persist the report to `stb_pseudo_report.txt`. Off by default.

`references.bib` (SIESTA plus a citation for every bank actually used) is always written — there's no flag for it.

Run `stb-pseudo --help` for the full list of options.

7.2.1.9 What's next

Point `stb-inputfile`'s (example `1.1`) `-p/--pp-path`, or your `calc.fdf`'s pseudopotential setup directly, at the output directory `stb-pseudo` just filled — every species it resolved is already sitting there with the right filename.

7.2.2 Implementation details

Resolution order.. The required elements are the distinct species labels of the structure's `ChemicalSpeciesLabel` block (or the `–species` list), sorted. For each one the primary source (`-p`) is searched for `<El>.psf` or `<El>.psml`; elements not found there are searched in `–fallback-dir`, if given. Both sources accept a bank name (`dojo`, `virtual_vault`) or a folder path; a bank prints its origin and citation when it is resolved. The report has six sections: metadata, required species, sources, an availability table (`FOUND (primary, .psml)`, `FOUND (fallback, .psf)` or `MISSING` per element), the action taken, and references.

Copying.. Files are copied (never linked) into `-o` (created if needed); an existing file with the same name is left untouched, so re-running is harmless. When a source holds both formats for an element, the copy takes the `.psml`. `–dry-run` stops after the availability table.

Exit status.. Missing elements produce a `[WARNING]` listing them; the resolved ones are still copied. Invalid argument combinations (no source, both `-f` and `–species`) stop with an `[ERROR]` and exit status 1.

Mixing banks.. `–fallback-dir` makes it easy to fill gaps, but a calculation that mixes pseudopotentials generated with different recipes (for example PSML from PseudoDojo and PSF from the Virtual Vault) is consistent only if all of them use the same XC functional as the calculation; check the header of each file.

7.2.3 Command-line reference

7.2.3.1 `stb-pseudo`

Resolves the pseudopotentials a structure needs and copies them into a run folder. Give a structure file (`-f/--file`, .fdf) or an explicit element list (`--species`), plus a pseudopotential source (`-p/--pp-path` – a bundled bank name or a directory of your own .psf/.psml files). Reports which elements were found and which are missing, and copies the resolved files into `-o/--output` (default: the current directory) unless

`--dry-run` is given. `--fallback-dir` supplies a second source consulted only for elements missing from the primary one. `--list-elements` browses a bundled bank without needing a structure at all.

Synopsis.

```
usage: stb-pseudo [-h] [-f PATH] [--species SPECIES [SPECIES ...]] [-p BANK_OR_PATH]
                  [--fallback-dir BANK_OR_PATH] [--list-elements BANK] [-o OUTPUT]
                  [--dry-run] [--save-report] [-v] [--no-intro]
```

Options

`-f, --file PATH`
: Path to a SIESTA structure file (.fdf) to read the required species from. Required unless `--species` or `--list-elements` is given.

`--species SPECIES [SPECIES ...]`
: Element symbol(s) to resolve directly, instead of reading a structure file. Mutually exclusive with `-f/--file`.

`-p, --pp-path BANK_OR_PATH`
: Pseudopotential source: a bundled bank name (dojo, virtual_vault) or a directory of your own .psf/.psml files. Required unless `--list-elements` is given.

`--fallback-dir BANK_OR_PATH`
: A second pseudopotential source, consulted only for elements missing from `-p/--pp-path` – fills gaps automatically instead of just reporting them as missing.

`--list-elements BANK`
: Print the elements available in a bundled bank and exit – no structure or `-p/--pp-path` needed. choices: dojo, virtual_vault

`-o, --output OUTPUT`
: Directory to copy the resolved pseudopotentials into. Default: the current directory.

`--dry-run`
: Only run the availability check and report – do not copy any files. switch, off by default

`--save-report`
: Also persist the report to `stb_pseudo_report.txt`. Off by default. switch, off by default

Like every STB command it also accepts `-h/-help`, `-v/-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Usage examples:
  stb-pseudo -f structure.fdf -p dojo -o .
  stb-pseudo --species Fe O -p dojo --fallback-dir virtual_vault
  stb-pseudo -f structure.fdf -p dojo --dry-run
  stb-pseudo --list-elements dojo
```

7.3 K-Grid Generator

At a glance

Menu code: 1.3 **Command:** `stb-kgrid`
Guided example: `examples/1.3-stb-kgrid/`
Summary: Suggest a Monkhorst-Pack grid (k-points) based on desired density.

7.3.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/1.3-stb-kgrid/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

7.3.1.1 What this tool does

SIESTA needs to know how finely to sample reciprocal space for a periodic calculation — the `kgrid.MonkhorstPack` block in `calc.fdf`. `stb-kgrid` computes a sensible one for you: given a structure and a target k-point *density*, it works out how many k-points to use along each lattice direction, correctly forcing a single k-point along any vacuum-padded (non-periodic) direction. Instead of guessing a grid or hand-deriving one from the reciprocal lattice, you answer one question — "how dense?" — and get back a ready-to-use grid.

7.3.1.2 Why this matters (a bit of theory)

A periodic crystal's electronic states are labeled by a wavevector **k** (Bloch's theorem) ranging continuously over the **Brillouin zone** (BZ) — reciprocal space's own unit cell. Total energy, forces, and everything else a DFT code computes are, in principle, an *integral* over that zone. In practice, that integral is approximated by a weighted sum over a finite set of sampled k-points: too few, and the result is a poor approximation that keeps changing as you add more; enough, and it converges to a stable value.

Monkhorst-Pack (Monkhorst & Pack, 1976 — the exact reference `stb-kgrid` writes to `references.bib`, see below) is the standard convention for *how* to lay out that finite set: a uniform mesh of `N1 x N2 x N3` points along the three reciprocal lattice directions, instead of picking points ad hoc.

`stb-kgrid` itself does **not** re-derive that paper's own numerical-optimality argument for where exactly to place points — what it computes is simpler and more practical: given a target k-point **density** (in 1/Ang), it picks a mesh size per axis,

```
N_i = ceil( |b_i| / density )
```

where `b_i` is the length of reciprocal lattice vector `i`. A finer density (smaller number) means more k-points — more accurate, slower. A coarser one means fewer k-points — faster, less accurate. Metals (sharp features at the Fermi surface) typically need a denser mesh than semiconductors/insulators for the same accuracy — this is exactly what the density-recommendation table (shown below, before you even pick a value) summarizes.

Dimensionality: Bloch's theorem, and the whole idea of a k-point, only applies along a genuinely *periodic* direction. `stb-kgrid` detects a vacuum-padded axis (the same `core/kspace.py::detect_vacuum_axes` heuristic `stb-inputfile` also uses) and forces a single k-point there regardless of density — there's no dispersion to sample along a direction with no real periodicity, so "finer" sampling there wouldn't mean anything physically.

7.3.1.3 When you'd reach for it

Whenever you need a k-grid for a periodic calculation and don't want to guess one by hand. In practice you rarely run this standalone — `stb-inputfile` (example `1.1`) already calls the same underlying `core/kspace.py` logic to fill in `calc.fdf`'s own `kgrid.MonkhorstPack` line automatically. `stb-kgrid` is for when you want to inspect, justify, or experiment with that choice on its own (e.g. deciding how much accuracy a given density buys you before committing to a full calculation).

7.3.1.4 Two ways to run it

A — direct CLI:

```
stb-kgrid --file structure.fdf --density 0.2
```

B — interactive `stb-suite` menu:

```
stb-suite
# at the main prompt, type: 1.3
```

`1.3` asks for the structure file, then shows the density recommendation guide **before** asking for a density, so the choice is informed instead of guessed, then runs the exact same `stb-kgrid` command underneath. `example_1.3.sh` proves both paths agree.

7.3.1.5 Files in this folder

- `structure.fdf` — bulk-silicon example structure (same 8-atom conventional cubic diamond cell used throughout `examples/1.1-stb-inputfile/`).
- `structure_molecule.fdf` / `structure_chain.fdf` / `structure_graphene.fdf` — an isolated CH4 molecule (0D), an infinite carbon chain (1D), and a graphene monolayer (2D).
- `example_1.3.sh` — the guided walkthrough (**not** an automated test — see `test/1-inputs/3-k-grid/test.sh` for that). Pauses between sections so you can read before moving on; safe to re-run.
- `output/` — created by `example_1.3.sh` when you run it (git-ignored, not checked in). See below.

7.3.1.6 Running the walkthrough

```
./example_1.3.sh
```

Every case below is generated with `--save-report`, each into its own folder under `output/` — `stb_kgrid_report.txt` (the exact console report) plus `references.bib` (SIESTA + Monkhorst-Pack citations):

Folder	Structure	Dimensionality	Grid (density 0.2)
`output/silicon/`	`structure.fdf` (bulk silicon)	3D	`6 6 6` (fully periodic)
`output/molecule/`	`structure_molecule.fdf` (CH4)	0D	`1 1 1` (isolated)
`output/chain/`	`structure_chain.fdf` (carbon chain)	1D	`25 1 1` (a wire)
`output/graphene/`	`structure_graphene.fdf` (graphene)	2D	`15 15 1` (a slab)

The accuracy/cost tradeoff, made concrete. The script also scans the *same* silicon structure at three densities and prints the resulting k-point *count* (the product of the grid divisions — the actual driver of SCF cost) side by side:

Density	Grid	k-points
0.1	12 12 12	1728
0.2	6 6 6	216
0.3	4 4 4	64

Going from 0.3 to 0.1 (3x finer) costs **27x** more k-points — for a 3D solid, mesh size scales with the *cube* of the linear density change. This is exactly why picking a sensible density (not just the smallest possible one) matters in practice.

The `--vacuum-gap` threshold is a real, tunable choice. Detecting "vacuum" isn't magic — it's a threshold (default 10 Ang): the largest empty gap along an axis, wrapped periodically, must exceed it to be treated as non-periodic. The script builds a structure with a 12 Ang gap along `c` and shows it classified as 2D (grid `6 6 1`) at the default threshold, but 3D (grid `6 6 2`) once `--vacuum-gap 15` is given — same structure, different threshold, different physical interpretation *and* different k-grid.

Proof: CLI and the interactive menu agree. The script also drives the same silicon/density-0.2 case through the interactive `stb-suite` menu (non-interactively, via a piped `printf`, no folder kept for it), confirms the density guide prints before the density prompt, and diffs the resulting grid line against `output/silicon/` — proving the CLI and the menu produce identical output.

7.3.1.7 Try it yourself

```
stb-kgrid --file structure.fdf --density 0.05   # high precision, expensive
stb-kgrid --file structure.fdf --density 0.5    # low precision, cheap
```

Compare the resulting grids and k-point counts against the density-scan table above.

7.3.1.8 Flag reference

```
stb-kgrid --file <structure.fdf> --density <value> [--vacuum-gap <Ang>] [--save-report]
```

- `-f/--file` (required): path to a SIESTA structure file (`.fdf` only).
- `-d/--density` (required): target k-point density, in 1/Ang.
- `--vacuum-gap` (optional, default `10.0`): minimum empty span (Ang) along an axis, wrapped periodically, to treat it as vacuum-padded.
- `--save-report` (optional): also persist the report to `stb_kgrid_report.txt`. Off by default.

`references.bib` (SIESTA + Monkhorst-Pack citations) is always written — there's no flag for it.

Run `stb-kgrid --help` for the full list of options.

7.3.1.9 What's next

The suggested grid is exactly what belongs in `calc.fdf`'s `kgrid.MonkhorstPack` line — but you rarely need to copy it there by hand: `stb-inputfile` (example `1.1`) already computes and writes it for you, using this same underlying logic.

7.3.2 Implementation details

The grid formula

With real-space lattice vectors $\mathbf{a}_1, \mathbf{a}_2, \mathbf{a}_3$ and cell volume $V = \mathbf{a}_1 \cdot (\mathbf{a}_2 \times \mathbf{a}_3)$, the reciprocal vectors are

$$\mathbf{b}_1 = \frac{2\pi}{V}\,\mathbf{a}_2 \times \mathbf{a}_3, \qquad \mathbf{b}_2 = \frac{2\pi}{V}\,\mathbf{a}_3 \times \mathbf{a}_1, \qquad \mathbf{b}_3 = \frac{2\pi}{V}\,\mathbf{a}_1 \times \mathbf{a}_2,$$

and the number of divisions along each axis is

$$N_i = \begin{cases} 1 & \text{if axis } i \text{ is vacuum-padded,} \\ \max(1,\ \lceil |\mathbf{b}_i|/d \rceil) & \text{otherwise,} \end{cases}$$

where d is `-density` in Å^{-1}. Because $|\mathbf{b}_i|$ includes the factor 2π, d is the largest allowed spacing between neighbouring k-points: the actual spacing $|\mathbf{b}_i|/N_i$ is always $\leq d$. The result is reported as a Monkhorst–Pack mesh [34] in SIESTA's compact form `kgrid.MonkhorstPack [N1 N2 N3]`.

Checks.. A zero (or numerically zero) cell volume and a non-positive density are refused with an `[ERROR]`. Vacuum detection uses the wrap-aware largest gap of Section 4.4, and the report states the dimensionality it implies (0D molecule, 1D wire, 2D slab, 3D bulk) before the grid.

Density guide.. The report (and the menu, *before* asking for a value) prints the same recommendation table:

Density (Å^{-1})	Accuracy
0.05–0.10	high precision
0.10–0.30	medium precision (0.2–0.3 is usually enough)
0.30–0.50	low precision

Metals need the denser end of the range for the same accuracy, because their occupations change abruptly at the Fermi surface. Always confirm with a convergence scan (**5.5**), which uses this very formula for its k-grid sweep.

Supercells.. The formula is applied to the cell you give. A supercell $n_1 \times n_2 \times n_3$ of a converged primitive cell needs about N_i/n_i divisions; workflows that build supercells (phonons, MLFF active learning) divide the reference grid accordingly.

7.3.3 Command-line reference

7.3.3.1 `stb-kgrid`

Compute a Monkhorst-Pack k-point grid from a SIESTA structure file (.fdf) and a target k-point density.

Synopsis.

```
usage: stb-kgrid [-h] -f STRUCTURE_FILE -d DENSITY [--vacuum-gap VACUUM_GAP]
                 [--save-report] [-v] [--no-intro]
```

Options

`-f, --file STRUCTURE_FILE`
: Path to the SIESTA structure file (.fdf). required

`-d, --density DENSITY`
: Target k-point density (in 1/Ang). Example: 0.2 required; type: float

`--vacuum-gap VACUUM_GAP`
: Minimum empty span (in Angstrom) between atoms along an axis, wrapped periodically, to treat that axis as vacuum-padded (non-periodic) and force a single k-point there regardless of density. Default: 10.0type: float

`--save-report`
: Also persist the report to `stb_kgrid_report.txt`. Off by default. switch, off by default

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Usage examples:
  stb-kgrid -f structure.fdf -d 0.2
  stb-kgrid -f slab.fdf -d 0.2 --vacuum-gap 15
```

7.4 K-Path Generator

At a glance

Menu code: 1.4 **Command:** `stb-kpath`
Guided example: `examples/1.4-stb-kpath/`
Summary: Generate a high-symmetry k-path for band structure calculations.

7.4.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/1.4-stb-kpath/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

7.4.1.1 What this tool does

For a band-structure calculation, SIESTA needs a path of k-points through the Brillouin zone (BZ) — the `%block BandLines` in `kpath_bs.fdf`. `stb-kpath` builds one automatically: given a structure, it detects its dimensionality and Bravais lattice, then generates the correct high-symmetry path for that lattice type, dimension-aware (a 1D wire, a 2D slab, and a 3D bulk crystal each have a genuinely different Brillouin zone shape, hence a different path).

7.4.1.2 Why this matters (a bit of theory)

`stb-kgrid` (example `1.3`) builds a Monkhorst-Pack **grid** — k-points spread uniformly over the *whole* BZ, used to approximate an *integral* (total energy, charge density). A band structure plot is a different job: it shows how the energy `E(k)` varies *along* a route through the BZ, so it's traced along a **path** connecting the BZ's high-symmetry points instead — `E(k)` is smooth, and the physically interesting features (band

extrema, gaps, crossings) are captured well by a handful of well-chosen points and the segments between them.

Which points count as "high-symmetry", and in what order to connect them, isn't arbitrary — it depends on the shape of the BZ, which in turn depends on the crystal's **Bravais lattice**. **Setyawan & Curtarolo (2010)** — the exact reference `stb-kpath` writes to `references.bib` — standardized this into one path convention per Bravais lattice type, now used across essentially every DFT code for reproducible band-structure plots. ASE implements that exact convention (`Cell.bandpath`) and extends it to 1D/2D systems via a periodic-axes mask; `stb-kpath` is a thin, dimension-aware wrapper around it — it does not invent or choose paths itself.

Bravais lattice vs. space group — two different questions. `stb-kpath` reports both, and they answer different things:

- **Bravais lattice** — from the raw *lattice vectors* alone (ignoring the atoms). Determines the BZ shape and which path template applies. This is what the k-path is actually built from.
- **Space group** — the *full* crystallographic symmetry, lattice + atomic basis together (via the same `core/symmetry.py` accessors `stb-inputfile` also uses).

These can legitimately disagree: a structure file can list its atoms in a non-primitive (conventional) cell, so the raw lattice vectors alone look lower-symmetry than the real crystal actually is once the atomic arrangement is taken into account. `example_1.4.sh` shows exactly this on bulk silicon: a plain cubic Bravais lattice, but an `Fd-3m` (diamond) space group.

Why a 0D structure is a hard error. Bloch's theorem, and the whole idea of a k-point, only applies along a genuinely periodic direction (see example `1.3`'s own theory section). An isolated molecule (0D — vacuum-padded on every axis) has no periodic direction at all, so a k-path isn't just low quality there, it's not physically meaningful. `stb-kpath` treats this as a real error (exit code 1, nothing written) rather than a soft warning.

7.4.1.3 When you'd reach for it

Whenever you need `kpath_bs.fdf` for a band-structure calculation. In practice you rarely run this standalone — `stb-inputfile`'s `bands` mode (example `1.1`) already calls `stb-kpath` itself to produce it. `stb-kpath` is for inspecting or double-checking the path it chose, or for generating one on its own.

7.4.1.4 Two ways to run it

A — direct CLI:

```
stb-kpath --file structure.fdf
```

B — interactive `stb-suite` menu:

```
stb-suite
# at the main prompt, type: 1.4
```

`1.4` asks for the structure file, then — the same "advanced settings" pattern used elsewhere in the suite — offers to configure the vacuum-gap, the Bravais-lattice tolerance, and the symmetry tolerances, all gated behind a single `[y/N]` question so the essential flow stays short. `example_1.4.sh` proves both paths agree.

7.4.1.5 Files in this folder

- `structure.fdf` — bulk-silicon example structure (same 8-atom conventional cubic cell used throughout `examples/1.1-stb-inputfile/` and `examples/1.3-stb-kgrid/`).
- `structure_chain.fdf` / `structure_graphene.fdf` — a carbon chain (1D) and a graphene monolayer (2D).
- `structure_molecule.fdf` — an isolated CH4 molecule (0D), used to demonstrate the hard error above.
- `example_1.4.sh` — the guided walkthrough (**not** an automated test — see `test/1-inputs/4-k-path/test.sh` for that). Pauses between sections so you can read before moving on; safe to re-run.
- `output/` — created by `example_1.4.sh` when you run it (git-ignored, not checked in). See below.

7.4.1.6 Running the walkthrough

```
./example_1.4.sh
```

Every successful case is generated with `--save-report`, each into its own folder under `output/` — `stb_kpath_report.txt`, `references.bib` (SIESTA + Setyawan-Curtarolo citations), and `kpath_bs.fdf`:

Folder	Structure	Dimensionality	Bravais lattice	Path
`output/silicon/`	`structure.fdf` (bulk silicon)	3D	primitive cubic (CUB)	Γ-X-M-Γ-R-X | M-R
`output/chain/`	`structure_chain.fdf` (carbon chain)	1D	primitive line (LINE)	Γ-X
`output/graphene/`	`structure_graphene.fdf` (graphene)	2D	primitive hexagonal (HEX2D)	Γ-M-K-Γ
`output/molecule/`	`structure_molecule.fdf` (CH4)	0D	*(hard error — no path, no file)*	—

For the 1D/2D cases, the script also prints the same vacuum-padded-axis caveat `stb-inputfile` uses: the space group shown treats the vacuum as an ordinary periodic direction and may not be exact — use `stb-symmetry` (code `2.5`) for a proper layer-group/point-group analysis of those.

The `--vacuum-gap` threshold changes the physics, not just a number. Same idea as example `1.3`: a structure with a 12 Ang gap along `c` sits just above the default 10 Ang vacuum threshold (2D, square BZ, path `M-Γ-X-M`) but below a stricter 15 Ang one (3D, tetragonal BZ, a completely different, longer path) — same atoms, different threshold, different Brillouin zone entirely.

Proof: CLI and the interactive menu agree. The script also drives the same silicon case through the interactive `stb-suite` menu's advanced-settings gate (vacuum-gap/eps/symprec/angle all entered explicitly at their defaults) and diffs the resulting path against `output/silicon/` — proving the CLI and the menu produce identical output.

7.4.1.7 Try it yourself

```
stb-kpath --file structure.fdf --vacuum-gap 5    # a tighter vacuum threshold
stb-kpath --file structure_chain.fdf -o my_path.fdf
```

7.4.1.8 Flag reference

```
stb-kpath --file <structure.fdf> [-p/--prec <eps>] [--vacuum-gap <Ang>]
          [--symprec <tol>] [--angle-tolerance <deg>] [-o/--output <file>]
          [--save-report]
```

- `-f/--file` (required): path to a SIESTA structure file (`.fdf` only).
- `-p/--prec` (optional, default `0.0002`): tolerance for ASE's Bravais-lattice detection.
- `--vacuum-gap` (optional, default `10.0`): minimum empty span (Ang) along an axis, wrapped periodically, to treat it as vacuum-padded.
- `--symprec/--angle-tolerance` (optional, defaults `0.001/5.0`): tolerances for the crystallographic space-group analysis.
- `-o/--output` (optional, default `kpath_bs.fdf`): output filename.
- `--save-report` (optional): also persist the report to `stb_kpath_report.txt`. Off by default.

`references.bib` (SIESTA + Setyawan-Curtarolo citations) is always written whenever a path is actually generated — there's no flag for it.

Run `stb-kpath --help` for the full list of options.

7.4.1.9 What's next

`kpath_bs.fdf` is exactly what `stb-inputfile`'s `bands` mode (example `1.1`) references via `%include kpath_bs.fdf` — but you rarely need to generate it separately: that example's own `bands/` step already calls `stb-kpath` for you, right after `stb-inputfile` itself.

7.4.2 Implementation details

Method.. The path is computed with ASE's `Cell.bandpath` [8], which identifies the Bravais lattice of the cell (tolerance `–prec`, the `eps` of ASE) and returns its special points and standard path following Setyawan and Curtarolo [35]. The vacuum-padded axes found with `–vacuum-gap` are passed to ASE as non-periodic, so a slab gets a 2D lattice (oblique, rectangular, centred rectangular, square or hexagonal) and a wire a 1D one; a structure with no periodic axis is refused, since a band path needs at least one. The space group and crystal system printed alongside come from spglib (`–symprec`, `–angle-tolerance`); for 1D/2D structures they treat the vacuum axis as periodic and are only indicative – use **`stb-symmetry`** for the layer or rod group.

Output file.. The output (default `kpath_bs.fdf`) contains `BandLinesScale ReciprocalLatticeVectors` and a `%block BandLines` with the special points in fractional reciprocal coordinates. The first point has count 1 and every following point 50, i.e. 50 k-points per segment. Point Γ is written as `\Gamma`, which SIESTA prints as a label in the `.bands` file and **`stb-bands`** turns into Γ on the axis.

Disjoint paths.. Standard paths of low-symmetry lattices have breaks (`Y-H|M-D`). The tool writes all segments in one sequence and prints a note at each break; SIESTA then computes the straight stretch between the two ends of a break as if it were part of the path. Remember this when reading the band plot, or edit the block to split the calculation.

Using it.. `calc.fdf` written by **`stb-inputfile -t bands`** already contains `%include kpath_bs.fdf`. A band structure must be computed on the *same* cell orientation used to generate the path; if you reduce or rotate the cell afterwards, generate the path again. **`stb-kpath`** references SIESTA and Setyawan–Curtarolo in

`references.bib`; see also the more recent crystallography-based convention of Hinuma *et al.* [36] if you need paths that are invariant to the choice of cell.

7.4.3 Command-line reference

7.4.3.1 `stb-kpath`

Finds the high-symmetry k-path for a SIESTA structure file (.fdf). Dimension-aware (1D/2D/3D) via ASE's Bravais-lattice path finder.

Synopsis.

```
usage: stb-kpath [-h] -f FILENAME [-p EPS] [--vacuum-gap VACUUM_GAP]
                 [--symprec SYMPREC] [--angle-tolerance ANGLE_TOLERANCE] [-o OUTPUT]
                 [--save-report] [-v] [--no-intro]
```

Options

`-f, --file FILENAME`
The structure file name (e.g., `struct.fdf`). required

`-p, --prec EPS`
Tolerance (ase.cell.Cell's 'eps') for Bravais-lattice/symmetry detection.
Default: 0.0002 type: float

`--vacuum-gap VACUUM_GAP`
Minimum empty span (in Angstrom) between atoms along an axis, wrapped periodically, to treat that axis as vacuum-padded (non-periodic) when detecting the system's dimensionality. Default: 10.0 type: float

`--symprec SYMPREC`
Symmetry-detection tolerance for the crystallographic space-group analysis. Default: 0.01 (pymatgen's own default). Loosen further (e.g. 0.02-0.05) for a structure relaxed with a looser force tolerance – a tighter value than the relaxation's own residual numerical noise can misdetect the true space group. type: float

`--angle-tolerance ANGLE_TOLERANCE`
Symmetry angle tolerance (degrees) for the crystallographic space-group analysis. Default: 5.0 type: float

`-o, --output OUTPUT`
Output .fdf file name (default: `kpath_bs.fdf`).

`--save-report`
Also persist the report to `stb_kpath_report.txt`. Off by default. switch, off by default

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

7.5 DFT+U / Hubbard Block Generator

At a glance

Menu code: 1.5 **Command:** `stb-dftu`
Guided example: `examples/1.5-stb-dftu/`
Summary: Generate a ready-to-use %block LDAU.proj snippet from a user-supplied U (and J).

7.5.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/1.5-stb-dftu/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

7.5.1.1 What this tool does

For materials with localized d or f electrons (transition-metal oxides, lanthanides, actinides), plain DFT often gets the electronic structure wrong. `stb-dftu` generates a ready-to-use `%block LDAU.proj` snippet for SIESTA's DFT+U (Hubbard) correction — one stanza per species, with the correlated shell auto-detected and the Hubbard `U` (and exchange `J`) supplied by you. It never estimates `U` on its own; it only offers a literature table as an optional, clearly-labeled starting point.

7.5.1.2 Why this matters (a bit of theory)

Ordinary DFT (LDA/GGA) has a **self-interaction error**: an electron partly interacts with itself. For most materials this barely matters, but for *localized* d or f electrons it artificially favors delocalization — the textbook failure is **NiO**, a real antiferromagnetic insulator that plain GGA predicts as a metal.

DFT+U (the Hubbard correction) patches this with a mean-field term that penalizes *fractional* occupation of the correlated orbital, pushing it toward an integer occupation (0 or 1) — closer to how a real localized electron behaves. It's a cheap, physically-motivated patch, not a first-principles fix. Two parameters describe it:

- **`U`** — on-site Coulomb repulsion: the energy cost of putting two electrons in the same correlated orbital.
- **`J`** — Hund's-rule exchange energy (usually smaller than `U`).

The SIESTA block also needs `n/l` — the correlated shell's principal and angular quantum numbers (`l=2` for a d shell, `l=3` for f) — auto-detected by periodic-table block: transition metals → `nd`, lanthanides → `4f`, actinides → `5f` (see `core/dftu_data.py::SHELL_NAMES/DEFAULT_SHELL`).

Why `stb-dftu` never guesses `U` for you. Unlike `n/l` (uncontroversial chemistry), `U` is *not* a universal constant — it depends on the material, the functional, the pseudopotential, and the basis set (screening, hybridization with neighboring ligands all shift it). `stb-dftu` bundles one literature table — **Wang, Maxisch & Ceder, Phys. Rev. B 73, 195107 (2006)**, the Materials Project's own GGA+U oxide calibration, and the exact reference this tool writes to `references.bib` — purely as a starting-point *sanity check*, never a validated value. You always have to pass `--u` explicitly, or opt in with `--use-reference`.

For a real, first-principles `U` computed from *your own* system (not a generic oxide table), use `stb-hubbardu/stb-hubbarduAnalysis` (Workflow menu) instead — the Cococcioni & de Gironcoli linear-response method. `stb-dftu` itself already points there in its own `--help` text.

7.5.1.3 When you'd reach for it

Any time you're setting up a SIESTA calculation for a transition-metal, lanthanide, or actinide compound and need a `DFT+U` correction — pasting the generated block into `calc.fdf`'s DFT+U section (the same file `stb-inputfile`, example `1.1`, produces).

7.5.1.4 Two ways to run it

A — direct CLI:

```
stb-dftu --species Mn --u 3.9
```

B — interactive `stb-suite` menu:

```
stb-suite
# at the main prompt, type: 1.5
```

`1.5` itself offers two sub-modes: **[1]** read species off a structure file and auto-fill `U` from the reference table, or **[2]** enter species one at a time by hand. Both end with the same save-block/save-report prompts. `example_1.5.sh` proves the CLI and the menu agree.

7.5.1.5 Files in this folder

- `sc_fe_o.fdf` — a synthetic 3-species fixture (Sc, Fe, O) used to demonstrate `--fdf --use-reference`'s 3-way split (see below).
- `example_1.5.sh` — the guided walkthrough (**not** an automated test — see `test/1-inputs/5-dftu/test.sh` for that). Pauses between sections so you can read before moving on; safe to re-run.
- `output/` — created by `example_1.5.sh` when you run it (git-ignored, not checked in). See below.

7.5.1.6 Running the walkthrough

```
./example_1.5.sh
```

Every case is generated with `--save-report`, each into its own folder under `output/` — `stb_dftu_report.txt` + `references.bib`:

Folder	Command (conceptually)
`output/single/`	`stb-dftu --species Mn --u 3.9` (auto-detected 3d shell)
`output/multi/`	`stb-dftu --species Fe Co --u 5.3 3.32 --j 0.5 0.5 --shell 3d 3d`
`output/from-structure/`	`stb-dftu --fdf sc_fe_o.fdf --use-reference`
`output/list-reference/`	`stb-dftu --list-reference` (no block, table only)
`output/suggest/`	`stb-dftu --suggest Ni` (no block, one value only)

`from-structure/` — the 3-way split, live. `sc_fe_o.fdf` has 3 species: **Sc** (a metal with *no* tabulated reference), **Fe** (a metal *with* one), and **O** (a non-metal). `--use-reference` treats all three differently:

- **Fe** → tabulated, included in the block, with a loud `[WARNING]` that a literature (not validated) value is being used.
- **Sc** → a metal, but untabulated → skipped, with its own `[WARNING]` so it's never silently dropped.
- **O** → not a metal at all → silently skipped (DFT+U doesn't apply to a non-correlated species, no warning needed).

`list-reference`/`suggest` — lookups only. Neither generates a block, so neither's `references.bib` gets a SIESTA citation (nothing SIESTA-bound was produced) — just the Wang/Maxisch/Ceder reference-table citation.

"Never guesses" in practice. The script also runs `stb-dftu --species Si --u 2.0` — Si has no default correlated shell and no tabulated reference, so this is a real error (exit code 1, no block, no silent fallback), not a guess.

7.5.1.7 Try it yourself

```
stb-dftu --species Ni --use-reference     # single species, auto-filled U
stb-dftu --species Cr --u 3.7 --shell 3d
```

7.5.1.8 Flag reference

```
stb-dftu (--species <label>... --u <eV>... | --fdf <path> [--use-reference])
         [--j <eV>...] [--shell <3d|4d|5d|4f|5f>...] [-o/--output <file>]
         [--list-reference] [--suggest <element>] [--save-report]
```

- `--species`/`--fdf` — one is required (unless `--list-reference`/ `--suggest`); `--fdf` reads species off a structure file, `--species` filters/overrides.
- `--u` (eV, one per species) — required unless `--use-reference`.
- `--use-reference` — auto-fill `U` from the bundled literature table for any species without an explicit `--u`.
- `--j` (eV, default `0.0`) / `--shell` (default: auto-detected).
- `-o/--output` — also save the block to a file.
- `--list-reference`/`--suggest <element>` — print the reference table (or one value) and exit; no block generated.
- `--save-report` (optional): also persist the report to `stb_dftu_report.txt`. Off by default.

`references.bib` is always written — there's no flag for it.

Run `stb-dftu --help` for the full list of options.

7.5.1.9 What's next

Paste the generated block into `calc.fdf`'s DFT+U section (the file `stb-inputfile`, example `1.1`, produces). If you need a real, system-specific `U` instead of the bundled literature table, see `stb-hubbardu`/`stb-hubbarduAnalysis` in the Workflow menu.

7.5.2 Implementation details

DFT+U in SIESTA

DFT+U adds to the energy functional an on-site penalty for fractional occupation of a localized shell. In the rotationally invariant simplified form of Dudarev *et al.* [37],

$$E_{\mathrm{DFT}+U} = E_{\mathrm{DFT}} + \frac{U_{\mathrm{eff}}}{2} \sum_{\sigma} \mathrm{Tr}[\mathbf{n}^{\sigma} - \mathbf{n}^{\sigma}\mathbf{n}^{\sigma}], \qquad U_{\mathrm{eff}} = U - J,$$

where $\mathbf{n}^{\sigma}$ is the occupation matrix of the shell projected on atom-centred projectors. The original formulations are those of Anisimov *et al.* [38] and Liechtenstein *et al.* [39]. In SIESTA each correlated shell is declared in `%block LDAU.proj`: a line with the species label and the number of projectors, then for each projector the principal quantum number and angular momentum (`n=3 2` for 3d), U

and J in eV, and the projector cut-off radius r_c and the width ω of its Fermi-function cut (0.0 lets SIESTA choose them).

The block written.. For each species one stanza is written:

```
%block LDAU.proj
Mn   1
n=3    2
3.900    0.000
0.000    0.000
%endblock LDAU.proj
```

It is printed on screen and, with `-o`, saved to a file you can `%include`. The other DFT+U directives (`LDAU.ProjectorGenerationMethod`, `LDAU.EnergyShift`, ...) keep SIESTA's defaults.

Shells.. The correlated shell is chosen from the periodic-table block: 3d for Sc–Zn, 4d for Y–Cd, 5d for La and Hf–Hg, 4f for Ce–Lu, 5f for Th–Lr. Any other species needs an explicit `–shell` (3d, 4d, 5d, 4f or 5f); `–u`, `–j` and `–shell` are lists matched one-to-one with the species.

The reference table.. `–list-reference`, `–suggest` and `–use-reference` use the Materials Project's GGA+U values for oxides, derived from Wang, Maxisch and Ceder [40] (V 3.25, Cr 3.7, Mn 3.9, Fe 5.3, Co 3.32, Ni 6.2, Mo 4.38, W 6.2 eV). They were calibrated on oxide formation energies with VASP, a plane-wave code with PAW potentials; the projectors of SIESTA are different, so the same number does not mean the same correction. They are a sanity check, and the tool says so whenever they appear. With `–fdf` and `–use-reference`, non-metal species are skipped silently and metals without a tabulated value are skipped with a warning. For a value computed for your own system, use the linear-response workflow **5.7** [41].

7.5.3 Command-line reference

7.5.3.1 `stb-dftu`

Generates a ready-to-use %block LDAU.proj snippet for one or more species. You supply U (and optionally J) explicitly – this tool never guesses or estimates U for you. `--list-reference` prints a small table of literature GGA+U values (citation included) as a convenience starting point; `--suggest` prints that value for one element with a clear disclaimer. Neither is ever used automatically – `--u` must always be given to produce a block, unless you pass `--use-reference` to explicitly opt in to auto-filling U from that same table. `--fdf` reads the species list off an existing structure file instead of typing `--species` by hand – combine it with `--use-reference` to build a block straight from a structure: only metal species with a tabulated value go into the block, with a loud warning; non-metals are skipped silently and untabulated metals are skipped with a warning. For a first-principles U computed from your own system via the Cococcioni & de Gironcoli linear-response method, see the Workflow menu's "Hubbard U (Linear Response)" entry (`stb-hubbardu` / `stb-hubbarduAnalysis`) instead.

Synopsis.

```
usage: stb-dftu [-h] [--fdf PATH] [--species SPECIES [SPECIES ...]] [--u U [U ...]]
                [--use-reference] [--j J [J ...]]
                [--shell {3d,4d,4f,5d,5f} [{3d,4d,4f,5d,5f} ...]] [-o OUTPUT]
                [--list-reference] [--suggest ELEMENT] [--save-report] [-v]
                [--no-intro]
```

Options

`--fdf PATH`
: Read the species list off this .fdf structure file instead of typing `--species` by hand. If `--species` is also given, it filters down to that subset (each one must be present in the file).

`--species SPECIES [SPECIES ...]`
: Species label(s) to add a DFT+U correction to. Required unless `--fdf` is given.

`--u U [U ...]`
: Hubbard U (eV), one per species, in the same order. Required unless `--use-reference` is given (and every species has a tabulated value). type: float

`--use-reference`
: For any species without an explicit `--u`, auto-fill U from the tabulated Materials Project reference table (see `--list-reference`). A non-metal species (no default correlated shell, e.g. O, Si) is silently skipped – DFT+U doesn't apply to it. A metal species with no tabulated value is skipped too, but with a warning. The block is built from whatever is left; errors out only if nothing qualifies. switch, off by default

`--j J [J ...]`
: Exchange J (eV), one per `--species` (default: 0.0 for each). type: float

`--shell {3d,4d,4f,5d,5f} [{3d,4d,4f,5d,5f} ...]`
: Correlated shell per species (3d/4d/5d/4f/5f). Default: the standard shell for each species (transition metals -> (n)d, lanthanides -> 4f, actinides -> 5f). choices: 3d, 4d, 4f, 5d, 5f

`-o, --output OUTPUT`
: Also save the block to this file (in addition to printing it).

`--list-reference`
: Print the literature GGA+U reference table (with citation) and exit. switch, off by default

`--suggest ELEMENT`
: Print the literature reference U for one element (with a disclaimer) and exit – does not generate a block.

`--save-report`
: Also persist the report to `stb_dftu_report.txt`. Off by default. switch, off by default

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Usage examples:
  stb-dftu --species Mn --u 3.9
  stb-dftu --species Fe Co --u 5.3 3.32 --shell 3d 3d
  stb-dftu --fdf calc.fdf --use-reference
  stb-dftu --fdf calc.fdf --species Mn --u 3.9
  stb-dftu --list-reference
  stb-dftu --suggest Ni
```

7.6 Structure Fetcher

At a glance

Menu code: 1.6 **Command:** `stb-fetch`
Guided example: `examples/1.6-stb-fetch/`
Summary: Fetch a structure from Materials Project or COD and write it as .fdf.

7.6.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/1.6-stb-fetch/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

7.6.1.1 What this tool does

`stb-fetch` looks up a structure in an online database — the Crystallography Open Database (COD, no account needed), Materials Project (needs a free `PMG_MAPI_KEY`), or any OPTIMADE-compliant database (AFLOW, JARVIS, 2D-materials databases like `twodmatpedia`, ...) — and writes it straight out as a SIESTA `.fdf`. It's the suite's first network-dependent tool: everything else processes files you already have locally.

Real database entries are frequently disordered even for common materials (e.g. magnetite's octahedral Fe site is often given as a Fe2+/Fe3+ mixed -occupancy split rather than plain Fe). Same-element-different-oxidation -state disorder is collapsed automatically to a single element at full occupancy; genuine multi-element disorder (two different elements sharing a site) is refused with a clear error instead of silently guessing.

7.6.1.2 When you'd reach for it

Any time the material you want already exists in the literature and you'd rather fetch it than hand-build a `POSCAR`/CIF/`.fdf` yourself — as a starting point for `stb-inputfile` (example `1.1`), or for `stb-symmetry`/`stb-kpath` once you have the structure in hand.

7.6.1.3 Two ways to run it

A — direct CLI:

```
stb-fetch --source cod --cod-id 1010369
```

B — interactive `stb-suite` menu:

```
stb-suite
# at the main prompt, type: 1.6
```

`1.6` walks you through picking a source/provider, an exact id or a formula search, an optional unit-cell reduction, and the save-report/`--view` prompts — the same choices the CLI flags below control directly. `example_1.6.sh` proves the CLI and the menu agree.

7.6.1.4 Files in this folder

- `example_1.6.sh` — the guided walkthrough. Pauses between sections so you can read before moving on; safe to re-run. **Needs internet access** — unlike every other example so far, this tool queries COD/OPTIMADE live; if a request fails (offline, or a remote endpoint is down), re-run once connectivity is back.
- `output/` — created by `example_1.6.sh` when you run it (git-ignored, not checked in). See below.

7.6.1.5 Running the walkthrough

```
./example_1.6.sh
```

Folder	Command (conceptually)
`output/cod-by-id/`	`stb-fetch --source cod --cod-id 1010369` (magnetite, disordered, bulk 3D)
`output/cod-primitive/`	same, + `--unitcell primitive`
`output/optimade-2d/`	`stb-fetch --source optimade --provider twodmatpedia --optimade-id 2dm-3150` (monolayer MoS2, 2D)
`output/formula-search/`	`--formula MoS2` with no exact id — multiple candidates, informative error
`output/merged-citations/`	two fetches (COD + OPTIMADE) into the same folder — `references.bib` merges

Every case that generates a structure is run with `--save-report`, each into its own `stb_fetch_report.txt` + `references.bib`.

Structure validation (always on, every fetch). Every run prints a `[3] STRUCTURE VALIDATION` section, right after the structure is fetched and disorder-resolved. It always runs — there's no flag to turn it on, and it never blocks writing the `.fdf`. It checks the same three malformation cases `stb-inputfile` (example `1.1`) checks on a hand-built structure:

1. **Atoms too close together** — any two atoms closer than 0.5 Å (minimum-image distance).
2. **Left-handed cell** — the lattice vectors' determinant is negative.
3. **Implausible atomic density** — atoms-per-Å^3 outside roughly `[0.01, 0.15]`. **Only checked for a genuine 3D bulk structure** — a vacuum-padded cell's volume is dominated by the artificial vacuum by design, so this metric would be meaningless there.

Real database entries are usually clean, so these three rarely fire in practice — but a fetched structure gets exactly the same scrutiny as one you built by hand.

Dimension-aware group label. Right after the malformation checks, the same section reports the structure's symmetry group — **the label only**, not the full list of operations or a Wyckoff-site table (that's `stb-symmetry`, code `2.5`'s job):

- **Bulk (3D, no vacuum axis)**: the ordinary space group, e.g. `Fd-3m (No. 227)`.
- **Slab (exactly one vacuum-padded axis, e.g. a 2D material)**: the **layer group** instead — the physically correct classification for a genuinely 2D-periodic structure (an ordinary 3D space group would treat the vacuum gap as just an unusually tall periodic cell). Same detection `stb-symmetry` uses (`spglib.get_layergroup`), needs `spglib >= 2.1.0`.
- **Wire (two vacuum-padded axes)**: neither spglib nor pymatgen has a rod-group equivalent, so this case only gets a note — same documented limitation as `stb-symmetry`.
- **Isolated molecule (all three axes vacuum-padded)**: the **point group** instead (pymatgen's `PointGroupAnalyzer`). In practice, none of COD/ Materials Project/OPTIMADE hand back an

isolated 0D structure — they're crystal-structure databases — so this case is implemented for completeness/robustness but isn't reachable through the walkthrough.

`references.bib` — always written, and it merges. `references.bib` is always written — there's no flag for it. Citations always include SIESTA, spglib (symmetry detection now always runs), and one entry for whichever database the structure came from (COD/Materials Project/OPTIMADE). Fetching from two different sources into the same output folder merges citations by BibTeX key instead of the second run erasing the first's — see `output/merged-citations/`.

`--view` (optional, off by default). Opens the final structure in ASE's interactive 3D viewer after writing the `.fdf` — needs a display. Not run by the walkthrough script (no display in a CI/headless environment); try it yourself:

```
stb-fetch --source cod --cod-id 1010369 --view
```

7.6.1.6 Try it yourself

```
stb-fetch --list-providers                    # every curated OPTIMADE alias
stb-fetch --source cod --formula Si --limit 5 # a formula search
stb-fetch --source materials-project --material-id mp-19306 --api-key <your PMG_MAPI_KEY>
```

7.6.1.7 Flag reference

```
stb-fetch --source <cod|materials-project|optimade>
          (--material-id <id> | --cod-id <id> | --optimade-id <id> | --formula <formula>)
          [--most-stable] [--api-key <key>] [--provider <alias|url>]
          [--unitcell <primitive|conventional|refined>] [--symprec <val>] [--angle-tolerance <deg>]
          [-o/--output <file>] [--save-report] [--view]
```

- `--source` (required unless `--list-providers`): which database to query.
- One of `--material-id`/`--cod-id`/`--optimade-id`/`--formula` is required — exact id, or a formula search that may need a follow-up pick.
- `--most-stable` (materials-project + `--formula` only): auto-pick the lowest energy-above-hull candidate.
- `--unitcell` (optional): also reduce to the primitive/conventional/ refined cell (same logic as `stb-unitcell`) before writing.
- `-o/--output` (default `fetched.fdf`).
- `--save-report` (optional): also persist the report to `stb_fetch_report.txt`. Off by default.
- `--view` (optional): open the structure in ASE's interactive 3D viewer after writing. Off by default; needs a display.

Structure validation and the dimension-aware group label (see above) always run before writing — there's no flag for either, they're always on. `references.bib` is always written too.

Run `stb-fetch --help` for the full list of options.

7.6.1.8 What's next

The `.fdf` this tool writes is a plain structure file — feed it to `stb-inputfile` (example `1.1`) to build a full SIESTA calculation, or to `stb-symmetry` (code `2.5`) for the full operations/Wyckoff-site symmetry analysis this tool only ever summarizes as a single label.

7.6.2 Implementation details

Sources..

`cod`
: the Crystallography Open Database [42], through its public web interface (`https://www.crystallography.net/cod`): a formula search lists the entries (COD id, formula, space group, a); one entry's CIF is downloaded and parsed with pymatgen. No account needed.

`materials-project`
: the Materials Project [43] through its API (`mp-api`); an API key is needed, either with `-api-key` or as configured for the `MPRester` client (environment variable `MP_API_KEY`). The candidate list shows the material id, formula, space group and energy above the convex hull; `-most-stable` picks the lowest E_{hull} among *all* matches.

`optimade`
: any OPTIMADE provider [44], chosen with `-provider`; `-list-providers` prints the curated list (`twodmatpedia`, `jarvis`, `aflow`, `oqmd`, `alexandria.pbe`, `odbx`, `mcloud.mc2d`, `mcloud.2dtopo`, `cmr`), but any provider alias known to the OPTIMADE client works. Formula searches use the element set and element count, which every provider implements, rather than the Hill-formula filter that some providers reject.

Formulas can be typed in any form pymatgen understands (`Fe3O4`, `Fe3 O4`); they are converted to the Hill notation the databases expect. Network requests are retried three times with back-off on transient server errors.

Choosing among several matches.. A formula search with more than one hit prints the first `-limit` candidates (default 20). In a terminal you are asked to pick one; in a script (no terminal on standard input) the tool stops and tells you which id option to use, so a batch job never picks a structure for you silently – except with `-most-stable`, which is an explicit choice.

Disorder.. Database entries are often crystallographically disordered. When every mixed site holds a single element in different oxidation states (magnetite's Fe^{2+}/Fe^{3+} site, for example), the site is collapsed to that element at full occupancy and the report says so. A site shared by *different* elements cannot be written as a definite `.fdf` and the entry is refused; pick an ordered entry, or build an ordered model yourself (**`stb-sqs`**, 4.6, for alloys). Partially occupied sites are also written fully occupied, so check the composition of the result.

After fetching.. The structure goes through the checklist of Section 3.4, its space group is printed (and its layer group when one axis is vacuum-padded), and `-unitcell primitive|conventional|refined` reduces it with the same code as **`stb-unitcell`** before it is written (default output `fetched.fdf`). The database's citation and spglib's are added to `references.bib`.

Experimental versus computed structures

COD holds experimental structures (lattice constants at the measurement temperature, sometimes with disorder); the Materials Project and most OPTIMADE databases hold DFT-relaxed ones (PBE or PBE+U lattice constants). Either is a starting point: relax it with your own SIESTA settings before computing properties.

7.6.3 Command-line reference

7.6.3.1 `stb-fetch`

Fetches a structure from an online database and writes it as .fdf. Look it up by exact id, or search by formula and pick a candidate.

Synopsis.

```
usage: stb-fetch [-h] [--source {materials-project,cod,optimade}]
                 [--material-id MATERIAL_ID | --cod-id COD_ID |
                 --optimade-id OPTIMADE_ID | --formula FORMULA] [--most-stable]
                 [--api-key API_KEY] [--provider PROVIDER] [--list-providers]
                 [--limit LIMIT] [--unitcell {primitive,conventional,refined}]
                 [--symprec SYMPREC] [--angle-tolerance ANGLE_TOLERANCE] [-o OUTPUT]
                 [--save-report] [--view] [-v] [--no-intro]
```

Options

`--source {materials-project,cod,optimade}`
Database to query. Required unless `--list-providers` is given. choices: materials-project, cod, optimade

`--material-id MATERIAL_ID`
Exact Materials Project id (e.g. 'mp-19306'). materials-project only. exclusive group 1

`--cod-id COD_ID`
Exact Crystallography Open Database numeric id. cod only. type: int; exclusive group 1

`--optimade-id OPTIMADE_ID`
Exact id within the chosen `--provider`. optimade only. exclusive group 1

`--formula FORMULA`
Search by chemical formula (e.g. 'Fe3O4'). Works with any source; prints a candidate table if more than one match is found. exclusive group 1

`--most-stable`
With `--formula` on materials-project, auto-pick the candidate with the lowest energy above hull instead of requiring an explicit `--material-id`. Not available for `--source` cod/optimade (no DFT energy to rank by). switch, off by default

`--api-key API_KEY`
Materials Project API key. Default: the PMG_MAPI_KEY environment variable / pymatgen's own settings file.

`--provider PROVIDER`
OPTIMADE provider: an alias (see `--list-providers` for a curated subset; any alias from pymatgen's own OptimadeRester.aliases also works) or a raw OPTIMADE base URL. Required with `--source` optimade.

`--list-providers`
Print a curated list of OPTIMADE providers and exit. Does not require `--source` or a selection flag.switch, off by default

`--limit LIMIT`
: Cap on how many `--formula` candidates to display (default: 20). type: int

`--unitcell {primitive,conventional,refined}`
: Also reduce the fetched structure to its primitive/conventional/refined unit cell (same logic as `stb-unitcell`) before writing. Default: keep the structure exactly as fetched. choices: primitive, conventional, refined

`--symprec SYMPREC`
: Symmetry precision, used for the fetched structure's space group summary and for `--unitcell` (default: 0.01, pymatgen's own default, matches `stb-unitcell/stb-symmetry`). type: float

`--angle-tolerance ANGLE_TOLERANCE`
: Symmetry angle tolerance in degrees, used the same way as `--symprec` (default: 5.0, pymatgen's own default). type: float

`-o, --output OUTPUT`
: Output .fdf file name (default: `fetched.fdf`).

`--save-report`
: Also persist the report to `stb_fetch_report.txt`. Off by default. switch, off by default

`--view`
: Open an interactive 3D view of the structure (via ASE) after writing the .fdf. Needs a display. Off by default. switch, off by default

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Usage examples:
  stb-fetch --source cod --cod-id 1010369
  stb-fetch --source cod --formula 'Fe3O4'
  stb-fetch --source materials-project --material-id mp-19306
  stb-fetch --source materials-project --formula Fe3O4 --most-stable
  stb-fetch --source optimade --provider twodmatpedia --formula MoS2
  stb-fetch --source optimade --provider twodmatpedia --optimade-id 2dm-2127
  stb-fetch --source cod --cod-id 1010369 --unitcell primitive
  stb-fetch --list-providers
```

7.7 ML Pre-Relaxation

At a glance

Menu code: 1.7 **Command:** `stb-mlrelax`
Guided example: `examples/1.7-stb-mlrelax/`
Summary: Fast pre-relaxation with the MACE-MP-0 potential (needs the optional 'ml' extra).

7.7.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/1.7-stb-mlrelax/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

7.7.1.1 What this tool does

`stb-mlrelax` runs a fast, heuristic geometry pre-relaxation using **MACE-MP-0** — a "foundation" machine-learned interatomic potential (MLIP) trained on ~1.5 million DFT relaxation snapshots from the Materials Project — or a custom model you fine-tuned yourself with `stb-mlffAnalysis`, before you commit to an expensive real SIESTA relaxation. It cleans up obviously -wrong geometry (a bad lattice guess, a freshly-added defect/passivant atom at the wrong bond length) in seconds, so the follow-up DFT relaxation starts much closer to equilibrium and needs far fewer, cheaper ionic steps.

This is a heuristic, not a substitute for DFT. Universal MLIPs are trained mostly on Materials-Project-like inorganic crystals and are less reliable far from that distribution. Always relax the result with SIESTA afterward.

7.7.1.2 Why this matters (a bit of theory)

A real SIESTA relaxation is expensive because every ionic step needs a full DFT self-consistent-field (SCF) cycle — solving the many-electron problem from (nearly) scratch just to take one small step downhill in energy. An MLIP sidesteps that entirely: it's a function, trained in advance to *approximate* the DFT potential energy surface (PES), so evaluating an energy and its forces on a new structure is just a forward pass through a neural network — microseconds to milliseconds, not minutes.

MACE (the architecture) is an *equivariant* message-passing graph neural network: atoms are nodes, atoms within a cutoff radius are connected by edges, and each message-passing "round" lets an atom build up a description of its local environment from its neighbors — two rounds in practice, since each one implicitly reaches one hop further (MACE's own efficiency trick over earlier, more rounds-hungry equivariant GNNs). *Equivariant* is the physically important part: if you rotate the whole structure, the network's predicted forces rotate the exact same way — a hard architectural constraint, not something the model has to learn approximately, so a MACE potential can't accidentally break rotational symmetry the way an ordinary (non-equivariant) network could.

MACE-MP-0 is one specific *trained* MACE model — the "foundation" part means it was fit once, broadly, across the Materials Project's public inorganic-crystal database, rather than for one particular material. That's exactly what makes it directly usable out of the box on almost any structure you hand it, at the cost of being a generalist, not a specialist (a model you fine-tune yourself on your own SIESTA data, via `stb-mlffAnalysis`, trades that generality for accuracy on your specific system — usable here via `--custom-model`).

Convergence methods: FIRE, BFGS, LBFGS, and the `fmax` criterion. Once MACE can predict a force on every atom (`F = -dE/dR`, the same physics as DFT's Hellmann-Feynman forces), an ordinary geometry optimizer walks "downhill" in energy, one step at a time, until the structure stops moving appreciably. `--optimizer` picks *which* algorithm does that walk — all three converge to the same kind of answer (a local energy minimum), they just take different-shaped paths to get there, with different costs per step:

- **`FIRE`** (Fast Inertial Relaxation Engine, the default) — treats the atoms like a damped mechanical system: move along the force direction as inertia would carry it, accelerating while the motion keeps pointing downhill, and adaptively resetting/damping the "velocity" whenever it doesn't. Each step only needs the current forces (no extra bookkeeping, no matrix), so it's cheap and very robust even from a badly-guessed starting geometry — but because it only ever looks at the local force direction, it can need more steps to fully settle once it's already close to the minimum.
- **`BFGS`** (Broyden-Fletcher-Goldfarb-Shanno) — a *quasi-Newton* method: instead of only following the current force, it builds up an approximate curvature (Hessian) of the energy surface from the

history of positions and forces seen so far, and uses that curvature to jump more directly toward the minimum. This typically needs noticeably fewer steps once the structure is already reasonably close to equilibrium (superlinear convergence), at the cost of maintaining a full curvature matrix -- memory that grows with the *square* of the number of degrees of freedom (3 x atoms, +6 more if `--relax-cell`), and a per-step cost that grows with it too.

- **`LBFGS`** (Limited-memory BFGS) — the same curvature-based idea as BFGS, but keeps only a short rolling history of recent position/force changes instead of the full curvature matrix. Nearly all of BFGS's fast, few-steps-to-converge behavior, at a memory/step cost that stays small regardless of system size -- the practical choice once a system is too large for full BFGS to be cheap.

Convergence itself is judged by **`fmax`**: the largest force component on any single atom, in eV/Ang (default target `0.05`, tunable via `--fmax`) — the same style of criterion SIESTA's own relaxation uses, so "relaxed enough for `stb-mlrelax`" and "relaxed enough for SIESTA" mean the same physical thing, just possibly at different thresholds. All three optimizers above are judged by the exact same `fmax` target -- `--optimizer` only changes *how fast* that target is reached, never what "converged" means.

Measured live on the same 8-atom `si_defect.fdf` (see `output/optimizer-comparison/` below):

`--optimizer`	Steps to converge	Wall time	Final energy
`FIRE` (default)	24	1.2 s	-42.952733 eV
`BFGS`	11	0.7 s	-42.953736 eV
`LBFGS`	11	0.7 s	-42.953736 eV

BFGS/LBFGS needed under half the steps here -- the expected quasi-Newton advantage once already reasonably close to a minimum. BFGS and LBFGS land on identical numbers for this tiny 8-atom case: with so few degrees of freedom, "limited memory" isn't actually limiting anything yet -- the gap between them only opens up on much larger structures, where full BFGS's curvature matrix becomes expensive to build and store.

7.7.1.3 When you'd reach for it

- After `stb-defect`/`stb-passivate`/`stb-slab` adds a new atom at an approximate bond length.
- After hand-editing or converting a structure from another source (unit slips, wrong lattice constants).
- Before a real SIESTA relaxation, to cut down the number of ionic steps SIESTA itself needs.

7.7.1.4 Two ways to run it

A — direct CLI:

```
stb-mlrelax -f defect.fdf
```

B — interactive `stb-suite` menu:

```
stb-suite
# at the main prompt, type: 1.7
```

`example_1.7.sh` proves the CLI and the menu agree.

7.7.1.5 What every run does (always on)

- **`[2]`/`[6]` structure validation**, before *and* after relaxing — the same shared checklist `stb-inputfile`/`stb-fetch` use (see `core/structure_checks.py`): atom proximity, lattice handedness, and atomic density, each printed as its own row with an explicit `[OK]`/`[WARNING]`/`[SKIPPED]` status, e.g.:

```
Check             | Result                                                | Status
------------------------------------------------------------------------------
Atom proximity    | min. distance = 2.211 Ang (>= 0.5 Ang threshold)  | [OK]
Lattice handedness | right-handed (positive determinant)              | [OK]
Atomic density    | 0.0500 atoms/Ang^3 (within [0.01, 0.15])         | [OK]
```

  Every check that ran is listed explicitly -- not just a summary line -- so it's always clear exactly what was verified, in both the console output and the report file.
- **`references.bib`** — always written, no flag: SIESTA (the output is a `.fdf`), the MACE architecture paper, and (unless `--custom-model`) the MACE-MP-0 foundation-model paper. Merges with an existing `references.bib` in the same folder instead of overwriting it.
- **A detailed before/after comparison table** (`[5]`, `Quantity | Before | After | Change`) — energy (total and per-atom), max force, full cell parameters + volume + density (if `--relax-cell`), mean/RMS/max atomic displacement (and which atom moved most), the nearest-neighbor distance before vs. after, the **space group** before vs. after, and the max **stress**-tensor component in GPa. See "Two more physical diagnostics" below for what these last two actually mean.
- **A convergence plot** (`<stem>_relax_convergence.png`, always) — energy and max force vs. optimizer step, so you can see *how* the relaxation converged, not just the final numbers.
- **MACE simulation detail** (`[3]`) — model type/size, device, precision, and, best-effort, the loaded model's approximate parameter count and cutoff radius (see the model-size table below).

7.7.1.6 Optional (off by default)

- `--save-report` — also persist the full numbered report to `stb_mlrelax_report.txt`.
- `--save-data` — also dump the raw per-step convergence trace to `<stem>_relax_convergence.dat`.
- `--view` — open an interactive ASE 3D viewer with **both** the structure before and after relaxation as two pageable frames (needs a display).

7.7.1.7 Two more physical diagnostics: space group and stress

Space group before vs. after is a genuine diagnostic, not decoration: if a structure is supposed to be a specific symmetric phase but relaxes into a *lower* symmetry, that's often a sign the starting guess (or the potential itself, this far from its training distribution) is misbehaving — whereas recovering or keeping *high* symmetry through relaxation is a good physical sanity check (see `si_wrong_a.fdf` below: a badly-guessed cubic Si cell relaxes back into a properly cubic one, not something lower-symmetry).

Stress is force's generalization to the cell itself — how hard the structure is "pushing" outward (or pulling inward) on its own periodic boundary, per unit area, reported here in GPa (best-effort: a custom model trained without stress targets just reports "not available" instead of failing the run). A converged *positions-only* relaxation at a fixed cell will generally still show nonzero stress — expected, since only `--relax-cell` explicitly relaxes the cell toward near-zero stress; it's extra information here, not a second convergence criterion.

7.7.1.8 Model size: small vs. medium vs. large

"Size" here means the trained network's capacity: how many features each atom carries internally (channels), how many message-passing interaction blocks it has, and how finely it represents angular information (the spherical-harmonic order used for the equivariant features -- see "Why this matters" above). None of that is something `stb-mlrelax` controls; `--model` just picks which of the 3 pre-trained MACE-MP-0 checkpoints to load. More capacity costs more compute but (usually, across many structures — not guaranteed for any one structure) predicts closer to the underlying DFT reference.

Cutoff radius (also shown below) is a *separate* architectural choice: it's the distance, in Angstrom, within which two atoms are connected by an edge in the graph at all -- a larger cutoff means every atom "sees" a bigger local neighborhood per message-passing round, at the cost of more edges (more compute) per step. It's worth noting this doesn't scale neatly with the other two sizes either: `large` actually uses a *smaller* cutoff (4.50 Ang) than `small`/`medium` (6.00 Ang) below -- a reminder that "bigger" model checkpoints aren't a strict superset of the smaller ones in every dimension, just an independently-tuned, overall more expressive network.

Numbers below were measured live on the same 8-atom `si_defect.fdf` (see `example_1.7.sh`'s `output/model-comparison/`, which reproduces this exact table from the tool's own `[3]`/`[4]` report sections, not a separate claim):

`--model`	Parameters (approx.)	Cutoff radius	Wall time (8 atoms, CPU)	Final energy
`small` (default)	~3.85 M	6.00 Ang	1.2 s	-42.9527 eV
`medium`	~4.69 M	6.00 Ang	2.3 s	-42.7278 eV
`large`	~15.85 M	4.50 Ang	5.0 s	-42.8940 eV

Use `small` (the default) for routine pre-relaxation; reach for `medium`/`large` when a small-model result looks suspicious (e.g. the

> 15% lattice-change warning fired) or as an extra sanity check before committing to expensive real DFT. `--custom-model` bypasses this table entirely with your own fine-tuned model.

7.7.1.9 Files in this folder

- `si_wrong_a.fdf` — bulk Si, correct chemistry, but a deliberately wrong lattice constant (5.00 Ang instead of ~5.43-5.46 Ang) — for `--relax-cell`.
- `si_defect.fdf` — bulk Si, correct cell, one atom nudged ~0.16 Ang off its ideal site — for the default positions-only mode.
- `si_too_close.fdf` — bulk Si with a mis-typed atom sitting ~0.09 Ang from its neighbor — deliberately malformed, to demonstrate structure validation actually catching a real problem.
- `example_1.7.sh` — the guided walkthrough (**not** an automated test — see `test/1-inputs/7-mlrelax/test.sh` for that). Pauses between sections so you can read before moving on; safe to re-run.
- `output/` — created by `example_1.7.sh` when you run it (git-ignored, not checked in). See below.

7.7.1.10 Running the walkthrough

```
./example_1.7.sh
```

Folder	Command (conceptually)	What it shows
`output/positions-only/`	`stb-mlrelax -f si_defect.fdf`	default mode; cell is bit-identical before/after
`output/relax-cell/`	`stb-mlrelax -f si_wrong_a.fdf --relax-cell`	a=5.00 Ang recovers to ~5.46 Ang
`output/bad-structure/`	`stb-mlrelax -f si_too_close.fdf`	pre-relax `[WARNING]`, clean post-relax check
`output/model-comparison/`	`stb-mlrelax -f si_defect.fdf --model {small,medium,large}`	the speed/accuracy table above, measured live
`output/optimizer-comparison/`	`stb-mlrelax -f si_defect.fdf --optimizer {FIRE,BFGS,LBFGS}`	the FIRE/BFGS/LBFGS table above, measured live
`output/full-report/`	`stb-mlrelax -f si_defect.fdf --save-report --save-data`	full report, `references.bib`, convergence files

`bad-structure/` — the structure check catching a real problem. `si_too_close.fdf` has one atom ~0.09 Ang from its neighbor (well under the 0.5 Ang threshold). `[2] STRUCTURE VALIDATION (pre-relaxation)` flags it with a `[WARNING]` before anything is relaxed.

Why the starting energy is enormous: at ~0.09 Ang apart, the two atoms' electron clouds are forced to overlap almost completely. Physically this is **Pauli repulsion** — electrons can't occupy the same state, so pushing two atoms' electron clouds together costs a steeply rising energy penalty (this is also, fundamentally, why atoms have a "size" at all). MACE learned this repulsive wall from its DFT training data just like everything else, so the predicted force here is huge and points straight apart — resolving the overlap within a handful of optimizer steps.

After relaxation, `[6] STRUCTURE VALIDATION (post-relaxation)` reports "No malformation issues detected." — so you can see the problem *and* confirm it's actually resolved, not just relaxed into a different problem.

7.7.1.11 Try it yourself

```
stb-mlrelax -f your_structure.fdf --view
stb-mlrelax -f your_slab.fdf --relax-cell   # vacuum axis stays exactly fixed
stb-mlrelax -f your_structure.fdf --custom-model my_finetuned.model \
    --custom-model-card mlff_model_card.json
stb-mlrelax -f your_structure.fdf --model large   # slower, usually more accurate
stb-mlrelax -f your_structure.fdf --optimizer BFGS   # fewer steps once already close
```

7.7.1.12 Flag reference

```
stb-mlrelax -f <structure.fdf> [--relax-cell] [--vacuum-gap <Ang>]
            [--model small|medium|large | --custom-model <path> --custom-model-card <json>]
            [--fmax <eV/Ang>] [--max-steps <N>] [--optimizer FIRE|BFGS|LBFGS]
            [--device cpu|cuda] [-o/--output <file>]
            [--save-report] [--save-data] [--view]
```

- `-f/--file` — required, the input structure (`.fdf`).
- `--relax-cell` — also relax the lattice (auto-adapts to periodicity: a vacuum-padded axis, e.g. a slab, is held exactly fixed; a fully isolated structure, e.g. a molecule, has nothing to relax and the flag is a no-op).
- `--model`/`--custom-model` — MACE-MP-0 foundation size, or your own fine-tuned model file. A fine-tuned model is always given together with its model card, `--custom-model-card` (the `mlff_model_card.json` that `stb-mlffAnalysis` writes next to it).

- `--fmax/--max-steps/--optimizer/--device` — relaxation controls.
- `-o/--output` — output `.fdf` name (default `relaxed.fdf`).
- `--save-report/--save-data/--view` — all optional, off by default.

`references.bib` is always written — there's no flag for it. Structure validation (before and after relaxation) always runs too — there's no flag for it either.

Run `stb-mlrelax --help` for the full list of options.

7.7.1.13 What's next

Take the relaxed `.fdf` and run a real SIESTA relaxation on it (e.g. via `stb-inputfile`, example `1.1`) — it should converge in far fewer ionic steps than starting from the original, un-relaxed guess. If you have your own SIESTA training data, `stb-mlffAnalysis` (Workflow menu) can fine-tune a custom MACE model for `--custom-model` that's more accurate for your specific material than the generic foundation checkpoint.

7.7.2 Implementation details

The relaxation.. The structure is converted to an ASE `Atoms` object, given a MACE calculator (Chapter 6; double precision) and relaxed with the chosen optimizer – FIRE [45] (default), BFGS or LBFGS from ASE – until the largest force component falls below `–fmax` (default 0.05 eV/Å) or `–max-steps` (default 200) is reached. With `–relax-cell` the cell is relaxed too, through ASE's `FrechetCellFilter`, with a six-component Voigt mask that freezes the normal strain of every vacuum-padded axis and every shear that involves one: a slab relaxes its in-plane lattice, a wire only its periodic axis, and the vacuum thickness never changes. A molecule (all axes vacuum) is relaxed at fixed cell with a note. The optimizer's trace (energy and max force per step) is recorded.

Outputs..

- the relaxed structure, `-o` (default `relaxed.fdf`);
- `<stem>_relax_convergence.png`: energy and maximum force (log scale) per optimizer step, with the `–fmax` target; with `–save-data` also the numbers in `<stem>_relax_convergence.dat`;
- `references.bib` (MACE, and MACE-MP-0 when the foundation model was used);
- the report, with sections for the input structure, validation before and after, the model, the relaxation, and a before/after table: energy (total and per atom), maximum force, lattice parameters, angles and volume (cell relaxation), displacement statistics with the atom that moved most, minimum interatomic distance, space group, and maximum stress.

Diagnostics worth reading.. A lattice parameter that changes by more than 15 % is flagged: either the input was far from equilibrium or the model is outside its domain. A change of space group means the relaxation broke (or found) a symmetry. "Hit step cap, NOT converged" means the structure should not be trusted as relaxed. The stress row is skipped for a custom model trained without stress.

A pre-relaxation, not a relaxation

MACE-MP-0 was trained on PBE(+U) data from a different code; its equilibrium lattice constants and bond lengths differ from those of your SIESTA basis, pseudopotentials and functional by an amount that depends on the material (often of the order of a per cent for lattice constants). The output is an excellent *starting point* – it removes bad contacts and gross strain, so the SIESTA relaxation needs

fewer and safer steps – but the final geometry must come from SIESTA.

7.7.3 Command-line reference

7.7.3.1 `stb-mlrelax`

Fast ML pre-relaxation with the MACE-MP-0 foundation potential. Cleans up obviously-wrong geometry (bad lattice guesses, freshly-added defect/passivant atoms at the wrong bond length) cheaply, so a follow-up SIESTA relaxation starts much closer to its own equilibrium and needs far fewer ionic steps. This is a pre-relaxation heuristic, NOT a substitute for a real DFT relaxation – universal MLIPs are trained mostly on Materials-Project-like inorganic crystals and are less reliable far from that distribution.

Synopsis.

```
usage: stb-mlrelax [-h] -f FILENAME [--relax-cell] [--vacuum-gap VACUUM_GAP]
                   [--model {small,medium,large}] [--custom-model PATH]
                   [--custom-model-card JSON] [--fmax FMAX] [--max-steps MAX_STEPS]
                   [--optimizer {FIRE,BFGS,LBFGS}] [--device {cpu,cuda}] [-o OUTPUT]
                   [--save-report] [--save-data] [--view] [-v] [--no-intro]
```

Options

`-f, --file FILENAME`
: Path to the input structure file (.fdf). required

`--relax-cell`
: Also relax the lattice, not just atomic positions. Automatically adapts to the structure's periodicity: for a slab/wire (vacuum-padded axes, e.g. from `stb-slab`), only the genuinely periodic direction(s) relax and the vacuum thickness is held exactly fixed – relaxing stress along a vacuum direction would be physically meaningless. A fully isolated structure (vacuum in all 3 directions, e.g. a molecule) has nothing to relax; `--relax-cell` is ignored. switch, off by default

`--vacuum-gap VACUUM_GAP`
: Gap (Ang) used to detect vacuum-padded axes for `--relax-cell` (default: 10.0, matches `stb-kgrid`).type: float

`--model {small,medium,large}`
: MACE-MP-0 model size: speed/accuracy tradeoff (default: medium). Ignored if `--custom-model` is given. choices: small, medium, large

`--custom-model PATH`
: Use a custom MACE model file instead of the MACE-MP-0 foundation potential – e.g. one fine-tuned on your own SIESTA data via `stb-mlffAnalysis`. Overrides `--model`.

`--custom-model-card JSON`
: Model card of the `--custom-model` file: the `mlff_model_card.json` that `stb-mlffAnalysis` wrote next to it (training data, level of theory, accuracy, energy scale). Required with `--custom-model`: the card is checked against the model file and used to warn when a structure is outside what the model was trained on.

`--fmax FMAX`
: Force convergence threshold, eV/Ang (default: 0.05). type: float

`--max-steps MAX_STEPS`
: Maximum optimizer steps (default: 200). type: int

`--optimizer {FIRE,BFGS,LBFGS}`
: ASE optimizer to use (default: FIRE). choices: FIRE, BFGS, LBFGS

`--device {cpu,cuda}`
: Device to run the model on (default: cpu). choices: cpu, cuda

`-o, --output OUTPUT`
: Output .fdf file name (default: `relaxed.fdf`).

`--save-report`
: Also persist the report to `stb_mlrelax_report.txt`. Off by default. switch, off by default

`--save-data`
: Also write the raw per-step relaxation convergence trace (`<stem>_relax_convergence.dat`). Off by default. switch, off by default

`--view`
: Open an interactive 3D view (via ASE) comparing the structure before and after relaxation (page through frames in ase-gui). Needs a display. Off by default. switch, off by default

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Usage examples:
  stb-mlrelax -f defect.fdf
  stb-mlrelax -f bulk.fdf --relax-cell -o bulk_relaxed.fdf
  stb-mlrelax -f slab2d.fdf --relax-cell -o slab2d_relaxed.fdf
  stb-mlrelax -f defect.fdf --save-report --save-data --view

Note: an initial guess that's very far from equilibrium (e.g. a lattice constant off by 15-20%% or more) can
occasionally lead a universal MLIP to a spurious low-energy state instead of the real minimum -- sanity-check a
large --relax-cell change before trusting it.
```

Chapter 8

Analysis

In this chapter

8.1 Bands Analyzer . . . 61
8.2 PDOS XML Parser . . . 67
8.3 DOS Processor (Convolution) . . . 73
8.4 Structure Analyzer . . . 79
8.5 Symmetry Analyzer . . . 86
8.6 Bader Charge Analysis . . . 93
8.7 Work Function Calculator . . . 99
8.8 Density Plotter (RHO) . . . 106
8.9 Powder XRD Simulator . . . 112
8.10 Fatbands Analyzer . . . 119
8.11 STM Simulator . . . 126
8.12 Wavefunction Density . . . 133
8.13 STS Spectroscopy Simulator . . . 140
8.14 COOP/COHP Bonding Analysis . . . 146
8.15 IPR Analyzer . . . 152
8.16 Effective Mass / Velocity . . . 157
8.17 Spin Texture Analyzer . . . 163
8.18 AIMD Trajectory Analysis . . . 168
8.19 Native Charges . . . 175

The analysis tools read the files of a *finished* SIESTA run and turn them into numbers, tables and plots. They do not write new calculations. They fall into four groups:

Electronic structure
band structures and gaps (`stb-bands`), projected DOS (`stb-dos`, `stb-convdos`), orbital-projected bands (`stb-fatbands`), localization (`stb-ipr`), effective masses and band velocities (`stb-effmass`), spin textures (`stb-spintexture`).

Real-space quantities
charge density maps (`stb-density`), Bader charges (`stb-bader`), work function (`stb-workfunction`), single wavefunctions (`stb-wfdensity`), simulated STM images and STS spectra (`stb-stm`, `stb-sts`), bonding analysis by COOP/COHP (`stb-coop`), and the atomic charges SIESTA itself printed, side by side (`stb-nativecharges`).

Structure
coordination and bond statistics (**stb-structural**), space/layer/point-group analysis (**stb-symmetry**), powder diffraction (**stb-xrd**).

Dynamics
radial distribution, diffusion and vibrational DOS of an MD trajectory (**stb-aimdAnalysis**).

They share one report style – numbered sections, `-save-report`, `-save-gnuplot` for `.dat+.gplot` files, `-view` for an interactive preview – and the same way of locating the files of a run: either `-label` (the `SystemLabel`, from which `<label>.bands`, `<label>.WFSX`, … are derived) or an explicit path for each file.

Wavefunction-based tools.. **stb-fatbands**, **stb-ipr**, **stb-spintexture**, **stb-wfdensity**, **stb-sts**, **stb-coop** and **stb-effmass** read SIESTA's `.WFSX` through sisl [10], and need different companions:

Tool	.WFSX kind	Also needs
stb-fatbands, **stb-ipr**	band path (`.bands.WFSX`)	`.bands`; `.HSX` preferred (overlap-aware weights), `.fdf` fallback
stb-spintexture	any (non-collinear/SOC)	`.HSX` preferred
stb-wfdensity, **stb-sts**	any / full-BZ mesh	`.fdf` with its `.ion` files (real-space orbitals)
stb-coop, **stb-effmass**	full-BZ mesh / any	`.HSX` required (overlap and Hamiltonian)

No file records which `.WFSX` belongs to which `.bands` or `.fdf`; the tools check orbital and k-point counts and cross-check a few eigenvalues, but always pair files from the same run.

Code	Tool	Purpose
2.1	Bands Analyzer	Analyze .bands files and calculate band gaps.
2.2	PDOS XML Parser	Extract data from `PDOS.xml` by total, atom, and species.
2.3	DOS Processor (Convolution)	Apply Gaussian convolution to Density of States (DOS) files.
2.4	Structure Analyzer	Calculate ECN and analyze structural properties.
2.5	Symmetry Analyzer	Analyze the symmetry of crystal structures.
2.6	Bader Charge Analysis	Calculate atomic charges using the Bader AIM method from .RHO and .XV files.
2.7	Work Function Calculator	Calculate Work Function from electrostatic potential (.VT).
2.8	Density Plotter (RHO)	Export 2D Charge Density Maps or 3D Clouds.
2.9	Powder XRD Simulator	Simulate a powder XRD pattern (peak table + optional plot) from a structure, optionally compared against an experimental pattern.
2.10	Fatbands Analyzer	Orbital-projected / character-colored band structure from .bands + .WFSX.
2.11	STM Simulator	Simulate an STM image (Tersoff-Hamann approximation) from a SIESTA .LDOS grid.
2.12	Wavefunction Density	Real-space |psi|^2 of a single (k, band) eigenstate from .WFSX -> Gaussian cube.
2.13	STS Spectroscopy Simulator	Simulated dI/dV(E) spectroscopy curve at a fixed tip point from a full-BZ .WFSX.
2.14	COOP/COHP Bonding Analysis	Energy-resolved Crystal Orbital Overlap/Hamilton Population curves for selected atom pairs.
2.15	IPR Analyzer	Inverse Participation Ratio (localization measure) scattered onto a band structure.
2.16	Effective Mass / Velocity	Effective mass tensor and band velocity for one chosen (k, band) eigenstate.
2.17	Spin Texture Analyzer	Spin moments (<Sx>,<Sy>,<Sz>) scattered onto a band structure from a non-collinear/SOC .WFSX.

Code	Tool	Purpose
2.18	AIMD Trajectory Analysis	RDF, MSD/diffusion coefficient, and VACF-derived VDOS from an AIMD trajectory.
2.19	Native Charges	Read and compare SIESTA's own Mulliken/Hirshfeld/Voronoi charges: per atom, per species, method spread, sum checks, point-charge dipole.

8.1 Bands Analyzer

At a glance

Menu code: 2.1 **Command:** `stb-bands`
Guided example: `examples/2.1-stb-bands/`
Summary: Analyze .bands files and calculate band gaps.

8.1.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/2.1-stb-bands/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

8.1.1.1 What this tool does

`stb-bands` parses a SIESTA `.bands` file (band-structure eigenvalues along a high-symmetry k-path) and reports the Valence Band Maximum (VBM), Conduction Band Minimum (CBM), whether the gap is direct or indirect, and whether the material is metallic — no SIESTA re-run needed. Optionally cross-checks that gap against the full SCF k-mesh (`--eig-file`, a `.EIG` file) in case the path missed the true extrema, and reports each spin channel (+ half-metallic character) separately for spin-polarized calculations.

This is the first example for the Analysis category (3.x) — the same guided-walkthrough format 2.x already uses, applied to a tool that *analyzes* an existing SIESTA output rather than building/transforming a structure.

8.1.1.2 Why this matters (a bit of theory)

The `.bands` file format. Line 1 is the Fermi energy; line 4 is `nbands nspin nk`; then one block per k-point (the k value, followed by `nbands*nspin` energies — wrapped onto extra lines if there are many bands); a footer lists which of those k-points are the high-symmetry ones, with labels (e.g. `'GAMMA'`, `'X'`). Spin-up and spin-down values are **not interleaved per band** — the first `nbands` values at each k are the full spin-up channel, the next `nbands` (if `nspin=2`) are spin-down (confirmed against `sisl`'s own reader).

VBM, CBM, and direct vs. indirect gaps. At **each** k-point, the highest energy at or below the Fermi level is that k-point's local VBM; the lowest energy above it is the local CBM.

- The **indirect (fundamental) gap** is `CBM - VBM` using the *global* extrema — the true VBM might be at a completely different k-point than the true CBM.
- The **direct gap** is the smallest `CBM(k) - VBM(k)` restricted to the *same* k-point.

Since the direct gap is the same search with an extra constraint (same k), **indirect gap ≤ direct gap always** — `example_2.1.sh`'s first case shows this on a real 21-band structure (indirect 1.2457 eV, direct 1.4484 eV, so `Gap type: Indirect`).

Why a "gap" is never negative, and what `--gap-tol` actually decides. If bands cross the Fermi level, the naive `CBM - VBM` can come out negative (global CBM below global VBM) — `stb-bands` clamps this to exactly `0.0` rather than ever reporting a negative gap. Very small *positive* gaps and true metals are otherwise indistinguishable, so `--gap-tol` (default `0.01` eV) is the threshold: an indirect gap below it is classified `Metallic`. `example_2.1.sh`'s `metallic-threshold/` case proves this is a genuine threshold effect — the *same* real ~1.25 eV gap is classified `Indirect` at a tight `--gap-tol` and `Metallic` at a loose one; the physical gap itself never moved.

Half-metals: one spin channel with a gap, the other without. For a spin-polarized calculation, `stb-bands` runs the exact same VBM/CBM search independently on each spin channel. A **half-metallic** material has one channel metallic and the other with a real gap — it conducts current of only one spin orientation, which is exactly why half-metals matter for spintronics. `example_2.1.sh`'s `half_metal.bands` fixture is a purpose-built synthetic example: spin-up channel gap ~0.0068 eV (Metallic, below the default `--gap-tol`), spin-down channel gap exactly 2.0 eV (Direct) — a clean, unambiguous half-metal.

Why a k-mesh comparison can only go one direction. A `.bands` file only samples a 1-D **path** through the Brillouin zone — the true VBM/CBM could sit anywhere else in the zone, entirely off that path. A `.EIG` file (the full SCF k-**mesh**, `--eig-file`) samples much more of the zone, so it can catch this. The key asymmetry: a denser mesh can only find a fundamental gap **smaller than or equal to** any 1-D subset of it (the path *is* a subset of the mesh in the ideal case), never larger. So:

- mesh gap **smaller** than the line gap → expected and informative: the path missed the true extrema, trust the mesh.
- mesh gap **larger** than the line gap → a red flag: the mesh itself is too coarse to have captured what the path already found.

`example_2.1.sh`'s `mesh-vs-line/` case demonstrates the first (smaller) case directly, with a synthetic 3-point mesh whose extrema are known by hand (`mesh.EIG`/`mesh.KP`).

`--shift`: which energy reference the plotted bands are drawn against. A `.bands` file's raw eigenvalues are absolute (whatever zero the SCF calculation happened to use) — not very readable on a plot. `--shift` subtracts a reference energy from every eigenvalue before writing `bands_gnuplot.dat`/plotting, so the reference itself sits at 0 eV:

- `fermi` — the Fermi energy itself. The most common choice: occupied states end up at negative energy, empty states at positive energy, regardless of whether the material is a metal or an insulator.
- `vbm` — the Valence Band Maximum. Common for semiconductors/ insulators specifically, since it puts the top of the occupied manifold exactly at 0 eV — convenient when comparing gaps or band-edge positions across different materials.
- `cbm` — the Conduction Band Minimum, the mirror choice of `vbm`.
- `manual` — any value you choose yourself (`--manual-value`), e.g. to match a reference used in a paper or a previous plot.

This only affects the *plotted*/written bands (`bands_gnuplot.dat`, `bands.gplot`) — the VBM/CBM/gap analysis in the report itself is always computed from the raw, unshifted eigenvalues against the file's own Fermi energy, so `--shift` never changes any reported physics, only where the plot's zero line ends

up.

8.1.1.3 When you'd reach for it

- Getting the band gap and its direct/indirect character from a finished SIESTA band-structure calculation, without opening a plotting tool.
- Sanity-checking whether your k-path actually captured the true gap (`--eig-file`), before quoting a gap value from a path alone.
- Checking for half-metallic character in a spin-polarized calculation.

8.1.1.4 Two ways to run it

A — direct CLI:

```
stb-bands --file semiconductor.bands --shift fermi
```

B — interactive `stb-suite` menu:

```
$ stb-suite
Select an option (0-7, or a tool code like 5.1.2): 2.1
```

Both paths call the exact same underlying tool and produce the exact same output — `example_2.1.sh` proves this directly at the end.

8.1.1.5 What every run does (always on)

- **A numbered report** (`[0] RUN METADATA ... [7] SUMMARY & FILES`) printed to the console.
- **`bands_gnuplot.dat` + `bands.gplot`** — the actual plot data (run `gnuplot bands.gplot` for a PDF).
- **`references.bib`** — SIESTA (every `.bands` file is SIESTA output).

8.1.1.6 Optional (off by default)

- **`--save-report`** — also persists the full numbered report to `stb_bands_report.txt`. Without it, only the plot data and `references.bib` are written — no text report file at all.
- **`--eig-file`** (+ `--kp-file`) — k-mesh vs. k-path gap comparison.
- **`--gap-tol`** — the Metallic/Indirect-or-Direct threshold (default `0.01` eV).
- **`--label`** — shorthand for `--file <label>.bands`, auto-detecting a matching `<label>.EIG`/`<label>.KP` if present.

8.1.1.7 Files in this folder

- `semiconductor.bands` — a real 21-band structure, 4 k-points (Γ→X→M→Γ), with a genuine indirect gap. Copied from `test/2-analysis/1-bands/siesta_edge_nbands21.bands` (kept small deliberately — a real `.bands` file for a decent-sized cell is easily 1+ MB, far too heavy for a lightweight example).
- `half_metal.bands` — synthetic spin-polarized fixture, purpose-built for the half-metallic demo (from `siesta_spin2.bands`).
- `mesh.EIG` / `mesh.KP` — a tiny synthetic 3-point k-mesh with known-by-hand extrema (from `mesh_kxkyz.{EIG,KP}`).

- `example_2.1.sh` — the guided walkthrough (**not** an automated test — see `test/2-analysis/1-bands/test.sh` for that, which additionally covers the real, ~1.2 MB reference fixture this example deliberately skips).
- `.gitignore` — excludes `output/` and every file this tool itself generates on a run (`bands_gnuplot.dat`, `bands.gplot`, `references.bib`, `stb_bands_report.txt`).

8.1.1.8 Running the walkthrough

```
cd examples/2.1-stb-bands
./example_2.1.sh
```

It pauses between sections (`[Press Enter to continue]`) and always starts by wiping its own `output/`. Five self-contained cases are generated:

Folder	What it shows
`gap-analysis/`	VBM/CBM and direct-vs-indirect gap on a real (tiny) indirect-gap structure
`metallic-threshold/`	The same real gap classified Indirect vs. Metallic, only `--gap-tol` changes
`half-metallic/`	One spin channel metallic, the other with a 2 eV gap
`mesh-vs-line/`	`--eig-file`/`--kp-file`: the k-mesh catching what the k-path missed
`full-report/`	Default (no report file) vs. `--save-report` (`stb_bands_report.txt`), `references.bib`

8.1.1.9 Try it yourself

```
# Your own finished SIESTA calculation
stb-bands --label my_calc --shift fermi --save-report

# Cross-check the path-based gap against the full SCF mesh
stb-bands --file my_calc.bands --eig-file my_calc.EIG --shift fermi
```

8.1.1.10 Flag reference

Flag	Meaning
`--label`	SIESTA label; shorthand for `--file <label>.bands` + auto-detected `.EIG`/`.KP`
`--file`	Explicit path to the `.bands` file
`--shift`	Energy reference for the plot: `vbm`/`cbm`/`fermi`/`manual`
`--manual-value`	Custom shift value (required with `--shift manual`)
`--gap-tol`	Metallic-classification threshold, eV (default `0.01`)
`--eig-file`	Full SCF k-mesh eigenvalues, for the mesh-vs-line gap comparison
`--kp-file`	Cartesian k-points matching `--eig-file` (requires `--eig-file`)
`-o`/`--output-dir`	Where to write the plot data (and report, with `--save-report`)
`--save-report`	Persist the full numbered report to `stb_bands_report.txt`

8.1.1.11 What's next

`core/siesta_bands.py`'s VBM/CBM machinery is shared with `stb-fatbands` (orbital-projected bands) and, via `select_band_vbm_cbm`, with `stb-effmass`/`stb-wfdensity` (finding the VBM/CBM band directly from a `.WFSX` with no `.bands` file at all). `stb-dos` covers the same occupied/empty-states question from the density-of-states side instead of individual bands.

8.1.2 Implementation details

Reading the file.. SIESTA's `.bands` file holds the Fermi energy, the range of the path coordinate, the number of bands, spin channels and k-points, then for every k-point its path coordinate and all eigenvalues (eV), and finally the labels of the special points and their positions. **`stb-bands`** reads it with `-file` or `-label` (`<label>.bands`), keeps the eigenvalues per k-point and spin, and uses the labels as tick marks (Γ written `\Gamma` or `Gamma` is shown as Γ).

Band edges and gaps as computed

For each k-point the eigenvalues are split by the Fermi energy E_F read from the file: $\epsilon \le E_F$ counts as occupied, $\epsilon > E_F$ as empty. Then

$$\mathrm{VBM} = \max_k \max_{\epsilon_{nk} \le E_F} \epsilon_{nk}, \qquad \mathrm{CBM} = \min_k \min_{\epsilon_{nk} > E_F} \epsilon_{nk},$$

the fundamental gap is $E_g = \max(0, \mathrm{CBM} - \mathrm{VBM})$ and the direct gap is the smallest $\min_{\epsilon > E_F} \epsilon_{nk} - \max_{\epsilon \le E_F} \epsilon_{nk}$ at a single k. The system is classified *Metallic* if $E_g <$ `-gap-tol` (default 0.01 eV), *Direct* if the direct gap exceeds E_g by less than `-gap-tol`, and *Indirect* otherwise. For spin-polarized runs the analysis is repeated per channel, and the system is flagged *half-metallic* when one channel is metallic and the other is not.

The Fermi level matters.. The occupied/empty split uses SIESTA's Fermi energy, which for an insulator lies somewhere in the gap (its exact position depends on the smearing). If no state lies on one side of E_F the tool stops and says so: the file and the Fermi energy do not belong together.

Line versus mesh.. A band path only samples lines, so it can miss the true band edges. With `-eig-file` the same analysis runs on the eigenvalues of the SCF k-mesh (`<label>.EIG`, read with sisl; `-kp-file` adds the k-vectors from `<label>.KP`) and the report compares the edges and gaps found on the path and on the mesh; the guide explains how to read a difference between the two.

Shift and output.. `-shift` sets the zero of energy of the written data (VBM, CBM, Fermi level, or a `manual` value with `-manual-value`). The tool writes the shifted bands for gnuplot (`.dat` plus `.gplot`, with the special points as tics) in `-output-dir`, plus the report. Kohn–Sham gaps with semilocal functionals (LDA/GGA) usually underestimate experimental gaps substantially; treat them as qualitative.

8.1.3 Command-line reference

8.1.3.1 `stb-bands`

Process the band structure data.

Synopsis.

```
usage: stb-bands [-h] [--label LABEL] [--file INPUT_FILE]
                 --shift {vbm,cbm,fermi,manual} [--manual-value MANUAL_VALUE]
                 [--gap-tol GAP_TOL] [--eig-file EIG_FILE] [--kp-file KP_FILE]
                 [-o OUTPUT_DIR] [--save-report] [-v] [--no-intro]
```

Options

`--label LABEL`
: SIESTA output label (e.g. 'siesta'). Shorthand for `--file <label>.bands`, and auto-detects `<label>.EIG` next to it for the mesh (k-grid) gap comparison – if that .EIG isn't there, the mesh comparison is skipped rather than treated as an error. Also auto-detects <label>.KP (mesh k-point coordinates) alongside the .EIG, if present. Mutually exclusive with `--file/--eig-file/--kp-file`.

`--file INPUT_FILE`
: Path to the input file containing band structure data (e.g., `siesta.bands`). Alternative to `--label` when you need an explicit path.

`--shift {vbm,cbm,fermi,manual}`
: Reference energy shift:
- 'vbm' : Valence Band Maximum
- 'cbm' : Conduction Band Minimum
- 'fermi' : Fermi level
- 'manual' : Custom shift value (requires `--manual-value`). required; choices: vbm, cbm, fermi, manual

`--manual-value MANUAL_VALUE`
: Custom energy shift value (required if `--shift` manual is used). type: float

`--gap-tol GAP_TOL`
: Energy tolerance in eV below which a gap is classified as Metallic (default: 0.01). type: float

`--eig-file EIG_FILE`
: Optional SIESTA .EIG file (full k-mesh eigenvalues from the SCF run). When given, compares the k-mesh gap against the k-path gap from `--file` – the high-symmetry path may not pass through the true VBM/CBM. Requires sisl.

`--kp-file KP_FILE`
: Optional SIESTA .KP file matching `--eig-file`'s k-mesh (Cartesian kx,ky,kz in 1/Ang). When given, mesh VBM/CBM/direct-gap k-points are reported as (kx, ky, kz) instead of a bare mesh index. Requires `--eig-file`.

`-o, --output-dir OUTPUT_DIR`
: Directory to write `bands_gnuplot.dat` and `bands.gplot` into (and stb_bands_report.txt/references.bib, with `--save-report`) (default: current directory). Created if it doesn't exist.

`--save-report`
: Also persist the full run report to `stb_bands_report.txt`. Off by default. switch, off by default

Like every STB command it also accepts `-h/-help`, `-v/-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Example usage:
  stb_bands --label siesta --shift fermi
  stb_bands --file siesta.bands --shift fermi
  stb_bands --file siesta.bands --shift manual --manual-value 0.5
  stb_bands --file siesta.bands --shift fermi --gap-tol 0.05
  stb_bands --file siesta.bands --shift fermi --eig-file siesta.EIG
  stb_bands --file siesta.bands --shift fermi --save-report
```

8.2 PDOS XML Parser

At a glance

Menu code: 2.2 **Command:** `stb-dos`
Guided example: `examples/2.2-stb-dos/`
Summary: Extract data from `PDOS.xml` by total, atom, and species.

8.2.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/2.2-stb-dos/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

8.2.1.1 What this tool does

`stb-dos` reads a SIESTA `<label>.PDOS.xml` file (the projected density of states) and writes it out as plain-text, per-orbital data ready to plot:

- `dos_total.dat` — the DOS summed over every atom, still split into one column per orbital (never summed across orbitals — see below).
- `dos_per_atom/<species>_<index>.dat` — one file per atom.
- `dos_per_species/dos_<species>.dat` — one file per chemical species.

Each file has one column per orbital angular momentum (`s`, `p`, `d`, `f` with `--projection l`, the default; or the individual `m` sub-orbitals — `px`, `py`, `pz`, `dxy`, ... — with `--projection ml`), split into spin-up/spin-down columns for a spin-polarized (`nspin=2`) calculation. `--type` picks which of the three output kinds to write (default: all three).

8.2.1.2 Why this matters (a bit of theory)

What a PDOS.xml actually holds. Unlike a `.bands/.EIG` file (discrete eigenvalues at specific k-points), a `PDOS.xml` is already a **broadened, continuous curve**: SIESTA applies a Gaussian/Lorentzian smearing to the discrete eigenvalues and evaluates the result on a shared energy grid, once per orbital of every atom. This is what makes it directly plottable as a DOS curve, but it also means there is no exact, discrete VBM/CBM sitting in the file the way there is in a `.bands/.EIG` — see `--shift vbm/cbm` below.

`--shift`: which energy sits at 0 eV in the output.

- `fermi` (default) — the calculation's own Fermi energy, from `<fermi_energy>` in the XML.
- `vbm` / `cbm` — the Valence Band Maximum / Conduction Band Minimum (see next section).
- a manual number — any value you choose.

This only changes where the written energy axis's zero sits — it never changes the shape of the DOS curve itself.

`--shift vbm/cbm`: reusing `stb-bands`' own VBM/CBM machinery. Since a PDOS.xml alone has no exact band-edge value, `--shift vbm/cbm` looks for a **companion file** next to it and computes the VBM/CBM from that, using the exact same `core/siesta_bands.py` code `stb-bands` itself uses — so the number reported here always matches what `stb-bands` would report for the same file, never a second,

independently-computed value. One `--label` covers all three files at once (`<label>.PDOS.xml` + `<label>.bands/<label>.EIG`); if you pass a `PDOS.xml` path directly instead, the label is still derived automatically from its own filename.

The lookup follows a strict hierarchy, most-precise source first:

1. **`<label>.bands`** — the k-**path** file `stb-bands` defaults to. Tried first.
2. **`<label>.EIG`** — the full SCF k-**mesh**, read via `sisl`. Used only if no `.bands` is present.
3. **An estimate from the DOS itself** (`--estimate-from-dos`) — the last resort, only used if *neither* of the above exists, and only when explicitly opted into.

A `.bands` k-path and a `.EIG` k-mesh are, in general, different samplings of the Brillouin zone (the same distinction `stb-bands`' own `--eig-file` mesh-vs-line comparison is built around) — they can legitimately report different VBM/CBM values for the same calculation, which is exactly why the hierarchy always prefers a real eigenvalue file over the DOS-based guess, and why a mismatch between the two files' own Fermi energies (or `nspin`) is reported as a warning, not silently ignored.

Without either file, and without `--estimate-from-dos`, `stb-dos` refuses outright rather than silently substituting a different-physics approximation for an exact value.

The DOS-estimate fallback (`--estimate-from-dos`). When neither a `.bands` nor a `.EIG` is available, `--estimate-from-dos` walks outward from the Fermi energy on each side of the (summed) total DOS until it first rises above `--dos-threshold-frac * peak_DOS` (default `0.01`, i.e. 1% of the tallest point in the curve) — the energy where the curve leaves the near-zero gap region. This is explicitly an **approximation**: the true band edge is blurred by the PDOS's own broadening, so the estimated value can differ meaningfully from the exact one a `.bands`/`.EIG` would give. `stb-dos` always prints an explicit warning when this path is used, rather than presenting it as exact.

8.2.1.3 The report: console output, `--save-report`, `--save-gnuplot`, `--view`

Every run prints a numbered report — the same `[0]...[5]` style every newer tool in this suite uses:

- `[0] RUN METADATA` — input file, resolved label, requested `--type`(s), projection mode, shift mode, output directory.
- `[1] INPUT DATA` — Fermi energy, spin channels, energy grid, orbitals/ atoms/species found, plus any non-fatal parsing warnings.
- `[2] ENERGY SHIFT` — which reference was used and why (the `--shift vbm/cbm` hierarchy messages above live here).
- `[3] WRITING OUTPUT FILES` — one `[OK]` line per `.dat`/directory actually written.
- `[4] REFERENCES` — SIESTA citations, written to `references.bib`.
- `[5] SUMMARY & FILES` — every file this run produced.

`--save-report` additionally persists that exact report to `stb_dos_report.txt` — off by default, so a plain run only ever writes the `.dat` files and `references.bib`, no text report file.

`--save-gnuplot` writes one `.gplot` script **per output category** — off by default. Every column is plotted as its **own curve, never summed** together (a 3-orbital atom contributes 3 separate lines, not 1):

- `dos_total.gplot` (next to `dos_total.dat`) — plots every orbital/spin column of the total DOS, using gnuplot's own `columnheader(i)` to read column names straight from the `.dat` file's own `#Energy(eV) s p ...` header line. Since there's only ever one `total` file, this is the most useful single-file view.

- `dos_per_atom/dos_per_atom.gplot` — ONE script overlaying **every atom's every orbital/spin column** as its own curve (e.g. `Sn_1: s`, `Sn_1: p`, `Sn_2: s`, ...), so every atom's full orbital breakdown can be compared in one plot instead of opening one file per atom.
- `dos_per_species/dos_per_species.gplot` — the same idea, one curve per (species, orbital/spin column) pair (e.g. `Si: s`, `Si: p`, `O: s`).

Run `gnuplot <name>.gplot` (from inside the same folder as the script) for a PDF.

`--view` opens an interactive matplotlib preview right before the tool exits — off by default, one figure per output category actually written (`total`/`atom`/`species`, mirroring `--save-gnuplot`'s own grouping and its same never-summed, one-curve-per-column convention), shown together only after every report line and file has already been written, so a blocking preview window never delays or hides them.

8.2.1.4 When you'd reach for it

- Plotting a projected density of states from a finished SIESTA calculation, split by atom/species/orbital.
- Referencing the DOS plot to the VBM or CBM instead of the Fermi energy — e.g. to line it up visually with a `stb-bands` plot shifted the same way.
- A quick, no-`.bands`/`.EIG`-available estimate of where the band edges sit, when only a `PDOS.xml` exists.

8.2.1.5 Two ways to run it

A — direct CLI:

```
stb-dos --label example --shift vbm
```

B — interactive `stb-suite` menu:

```
$ stb-suite
Select an option (0-7, or a tool code like 5.1.2): 2.2
```

Both paths call the exact same underlying tool and produce the exact same output — `example_2.2.sh` proves this directly at the end. The menu additionally offers `--shift` as a **numbered menu** (`1` Fermi level, `2` VBM, `3` CBM, `4` custom value — re-prompting on an invalid choice) instead of typing the keyword, plus separate `y/N` prompts for `--save-report`/`--save-gnuplot`/`--view`, each mapped straight onto the matching CLI flag.

8.2.1.6 Files in this folder

- `example.PDOS.xml` — a small, hand-built synthetic fixture: 1 atom (Si), 1 orbital (s), 13 energy points from -3 to +3 eV, Fermi energy 0.0 eV. Its DOS drops to near-zero between -1 and +1 eV — a deliberate "gap" shape, used to demonstrate `--estimate-from-dos`.
- `example.bands` / `example.EIG` — a matching "same calculation" trio (same Fermi energy), with known-by-hand VBM/CBM at each level of the `--shift vbm/cbm` hierarchy.
- `multi.PDOS.xml` — a second fixture, 3 atoms/2 species (`Si_1`/`Si_2`, each with an `s` AND a `p` orbital; `O_3` with only `s`), same energy grid/"gap" shape as `example.PDOS.xml` but each orbital scaled differently so every curve is visually distinct and hand-checkable — used to demonstrate `--save-gnuplot`/`--view` overlaying multiple atoms/species AND multiple orbitals, all as separate curves, never summed together (`example.PDOS.xml`'s own single atom/orbital can't show that).

- `example_2.2.sh` — the guided walkthrough (**not** an automated test — see `test/2-analysis/2-dos/test.sh` for that, which additionally covers a real ~22 MB reference PDOS.xml this example deliberately skips).
- `.gitignore` — excludes `output/`.

8.2.1.7 Running the walkthrough

```
cd examples/2.2-stb-dos
./example_2.2.sh
```

It pauses between sections (`[Press Enter to continue]`) and always starts by wiping its own `output/`. Five self-contained cases are generated:

Folder	What it shows
`basic-run/`	The three output types (`dos_total.dat`, `dos_per_atom/`, `dos_per_species/`)
`vbm-hierarchy/`	`--shift vbm`'s `.bands` → `.EIG` → `--estimate-from-dos` hierarchy, live
`label-shorthand/`	`--label` resolving both the `PDOS.xml` and its `.bands` in one go
`multi-atom-plots/`	`--save-gnuplot`/`--view` overlaying every atom's/species' own orbitals as separate curves (never summed), rendered with a real `gnuplot`
`full-report/`	Default (no report/`.gplot` files) vs. `--save-report --save-gnuplot`

8.2.1.8 Try it yourself

```
# Your own finished SIESTA calculation
stb-dos --label my_calc --shift vbm

# No .bands/.EIG available for my_calc -- opt into the DOS-based estimate
stb-dos --label my_calc --shift cbm --estimate-from-dos
```

8.2.1.9 Flag reference

Flag	Meaning
`filename`	Path to the `.PDOS.xml` file (positional). Optional if `--label` is given.
`--label`	SIESTA label; shorthand for `filename='<label>.PDOS.xml'`, and (with `--shift vbm/cbm`) used to locate `<label>.bands`/`<label>.EIG`.
`--type`	Which output(s) to write: `total`, `atom`, `species` (default: all three).
`--shift`	Energy reference: `fermi` (default), `vbm`, `cbm`, or a manual number.
`--estimate-from-dos`	With `--shift vbm/cbm`: fall back to a DOS-based estimate if no `.bands`/`.EIG` is found. Off by default.
`--dos-threshold-frac`	Threshold (fraction of peak DOS) used only by `--estimate-from-dos` (default `0.01`).
`--projection`	Orbital detail: `l` (s/p/d/f, default) or `ml` (individual m sub-orbitals).
`-o/--output-dir`	Where to write the output files (default: current directory).
`--save-report`	Persist the full numbered report to `stb_dos_report.txt`. Off by default.
`--save-gnuplot`	Write one `.gplot` script per output category (total/atom/species). Off by default.
`--view`	Show an interactive matplotlib preview, one figure per output category. Off by default.

8.2.1.10 What's next

The VBM/CBM machinery this tool reuses (`core/siesta_bands.py`) is shared with `stb-bands`, `stb-fatbands`, and — via `select_band_vbm_cbm` — with `stb-effmass`/`stb-wfdensity`. `stb-bands` covers the same

occupied/empty -states question from the discrete-eigenvalue side instead of the DOS.

8.2.2 Implementation details

Input.. SIESTA writes the projected DOS in XML (`<label>.PDOS.xml` or `<label>.PDOS`) when `%block ProjectedDensityOfStates` is set: the energy grid, the Fermi energy, the number of spin channels, and one `orbital` element per basis orbital with its atom index, species, quantum numbers n, l, m, zeta and polarization flags, and its PDOS on the grid. The PDOS of all orbitals sums to the total DOS. Every orbital's data are read once, without printing, and then grouped.

Grouping.. `-type` selects any combination of

- `total` – the sum over all atoms, in `dos_total.dat`;
- `atom` – one file per atom in `dos_per_atom/`;
- `species` – one file per species in `dos_per_species/`;

and `-projection` the orbital resolution of the columns: `l` (s, p, d, f; default), `ml` (p_x, p_y, p_z, d_{xy}, …, with SIESTA's real spherical harmonics) or `none` (one total column). For spin-polarized runs every column is split into `_up` and `_down`. Orbitals with $l > 3$ are reported and counted only in the `none` projection. Every `.dat` has a header naming its columns, which **`stb-convdos`** and the generated gnuplot scripts read.

Energy reference.. `-shift fermi` (default) subtracts the Fermi energy of the file; a number shifts by that amount (`0.0` keeps absolute energies). `vbm`/`cbm` find the band edges with the algorithm of **`stb-bands`**, from a companion `<label>.bands` (preferred) or `<label>.EIG`, located with `-label`. If neither exists, `-estimate-from-dos` walks outward from E_F on the total DOS to the first points exceeding `-dos-threshold-frac` (default 1 %) of its maximum – an estimate that depends on the broadening and is labelled as such.

Broadening and integration.. SIESTA broadens each eigenvalue with a Gaussian whose width is the third number of the `ProjectedDensityOfStates` line; **`stb-convdos`** adds further broadening. The k-sampling of the PDOS is `PDOS.kgrid.MonkhorstPack` if present, otherwise the SCF grid – a DOS needs a denser grid than the SCF.

8.2.3 Command-line reference

8.2.3.1 `stb-dos`

Parse a `PDOS.xml` file and generate Gnuplot-ready .dat files.

Synopsis.

```
usage: stb-dos [-h] [--label LABEL]
               [--type {total,atom,species} [{total,atom,species} ...]]
               [--shift SHIFT] [--estimate-from-dos]
               [--dos-threshold-frac DOS_THRESHOLD_FRAC] [--projection {l,ml,none}]
               [-o OUTPUT_DIR] [--save-report] [--save-gnuplot] [--view] [-v]
               [--no-intro]
               [filename]
```

Positional arguments

`[filename]`

The input .PDOS.xml file to process. Give this before `--type` (see epilog). Optional if `--label` is given (resolves to '<label>.PDOS.xml').

Options

`--label LABEL`

SIESTA label. Shorthand for filename='<label>.PDOS.xml' if filename isn't given directly, and (with `--shift` vbm/cbm) used to locate a companion '<label>.bands'/'<label>.EIG' next to it. One of filename or `--label` is required. If filename is given explicitly without `--label`, the label is still derived automatically from its own name for `--shift` vbm/cbm.

`--type {total,atom,species} [{total,atom,species} ...]`

Type(s) of DOS to output.
total: Sum of all atoms.
atom: One file for each atom.
species: One file for each chemical species (e.g., C, N, B).
You can select multiple, e.g., `--type` total species (default: all three) choices: total, atom, species

`--shift SHIFT`

Energy shift to apply.
'fermi': Automatically shift by the Fermi energy (default).
'vbm'/'cbm': Shift by the Valence Band Maximum / Conduction Band Minimum, computed from a companion '`<label>.bands`' (preferred) or '`<label>.EIG`' file – see `--label`. Falls back to an approximate estimate from the DOS itself only with `--estimate-from-dos`.
'0.0': Use an absolute energy scale (no shift).
'-1.23': Apply a manual shift of -1.23 eV.

`--estimate-from-dos`

With `--shift` vbm/cbm: if no companion '<label>.bands'/'<label>.EIG' is found, fall back to an approximate VBM/CBM estimated directly from the DOS itself (blurred by its own energy broadening – less precise than a real .bands/.EIG). Off by default: without it, a missing .bands/.EIG is a clear error, not a silent approximation. switch, off by default

`--dos-threshold-frac DOS_THRESHOLD_FRAC`

Only used by `--shift` vbm/cbm's `--estimate-from-dos` fallback: fraction of the peak total DOS above which the curve is considered 'occupied'/'empty' again past the gap region (default: 0.01). type: float

`--projection {l,ml,none}`

Orbital projection detail level.
l: Aggregate by angular momentum (s, p, d, f). (default)
ml: Project by magnetic quantum number (s, px, py, pz, dxy, etc.).
none: No orbital breakdown – every orbital (including l>3, e.g. g) collapses into a single 'total' column per atom/species/grand-total. choices: l, ml, none

`-o, --output-dir OUTPUT_DIR`

Directory to write `dos_total.dat`, dos_per_atom/, and dos_per_species/ into (default: current directory). Created if it doesn't exist.

`--save-report`

Also persist the full run report to `stb_dos_report.txt`. Off by default. switch, off by default

`--save-gnuplot`

Also write a .gplot gnuplot script next to every .dat file this run generates (`dos_total.dat`, each dos_per_atom/*.dat, each dos_per_species/*.dat). Off by default. switch, off by default

`--view`

Show an interactive matplotlib preview of the total DOS before exiting. Off by default. switch, off by default

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Example usage:
  stb-dos siesta.PDOS.xml --type total species --projection ml
Put the filename first: --type takes one or more values, so anything
written after it (including the filename) is consumed as another type.
```

8.3 DOS Processor (Convolution)

At a glance

Menu code: 2.3 **Command:** `stb-convdos`
Guided example: `examples/2.3-stb-convdos/`
Summary: Apply Gaussian convolution to Density of States (DOS) files.

8.3.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/2.3-stb-convdos/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

8.3.1.1 What this tool does

`stb-convdos` applies Gaussian broadening (convolution) to every DOS column of a whitespace-separated data file (energy first, one or more DOS columns after — typically `stb-dos`'s own output). No structure, no SIESTA re-run — it's pure post-processing of an already-computed density of states.

Two input modes:

- **`--file <file> --out <file>`** — broaden one file.
- **`--dir <folder>`** — broaden **every** `.dat` file found recursively under a folder (e.g. a whole `stb-dos` output tree: `dos_total.dat` + `dos_per_atom/*.dat` + `dos_per_species/*.dat`) with the exact same `--sigma`/`--fwhm`/`--size`, writing a broadened mirror of the entire tree — same subfolders, same filenames — under `--output-dir` (default: `<folder>_filtered`, a sibling directory). A file that fails validation (e.g. accidentally not a DOS file) is skipped with a warning instead of aborting the whole batch.

8.3.1.2 Why this matters (a bit of theory)

Why broaden a DOS at all?. A DFT calculation gives you the density of states at a finite, discrete set of k-points. Especially with a coarse k-mesh, the "raw" DOS looks like a comb of sharp, almost delta-function-like spikes rather than a smooth curve — an artifact of finite sampling, not the real physical density of states (which, for a bulk crystal, genuinely is smooth). Convolving with a Gaussian kernel simulates the effect that any real measurement or a denser calculation would show: finite instrumental/thermal broadening, or just enough smoothing to see the underlying trend instead of sampling noise.

`example_2.3.sh`'s first case makes this completely explicit with a synthetic "stick spectrum" (`dos_total.dat` below), watching sharp spikes turn into smooth peaks.

`sigma` vs. `FWHM`. Both describe the *width* of the Gaussian broadening kernel, just in two conventions:

- **`sigma`** (standard deviation) — the natural parameter of a Gaussian, `exp(-(E-E0)^2 / (2*sigma^2))`.
- **`FWHM`** (Full Width at Half Maximum) — the width of the peak measured at half its height, the value more commonly quoted in spectroscopy and experimental resolution specs.

They're related by an exact constant:

```
FWHM = 2*sqrt(2*ln(2)) * sigma  ≈  2.3548 * sigma
```

`--fwhm` is simply converted to the equivalent `sigma` internally (`sigma = fwhm / 2.3548`) before anything else happens — `--sigma 50` and `--fwhm 117.741` are the exact same broadening, just expressed differently (`example_2.3.sh` proves this directly).

Why `--sigma`/`--fwhm` are in meV, not "samples". The convolution kernel itself operates on **grid samples** — a "width of 5" only means something once you know how far apart the file's own energy points actually are. `stb-convdos` instead asks for `--sigma`/ `--fwhm` in **meV**, a physical, file-independent unit, and converts it internally:

```
sigma_samples = (sigma_meV / 1000) / d_energy
```

where `d_energy` is the **median** spacing between consecutive energy points in that specific file (median, not mean — robust to the rare irregular point). This is why the same `--sigma 50` gives the same physical broadening whether the input file has 100 or 10,000 energy points — and why, in `--dir` mode, files with different native energy grids (e.g. a `stb-dos` run with a different grid resolution) can each get a different `sigma_samples`/kernel size internally while still representing the identical 50 meV physical width (see `[2] BROADENING PARAMETERS` in the report, which shows this per-file when it happens).

The kernel size, and why it must be odd. By default, the kernel width (in samples) is sized automatically to cover about **3 standard deviations** on each side of center — over 99% of a Gaussian's mass:

```
half_width = ceil(3 * sigma_samples)
size = 2 * half_width + 1
```

It's forced to be **odd** so there's a single, unambiguous center sample — an even-sized kernel would shift the filtered curve by half a bin. `--size` overrides this if you want a specific width; `stb-convdos` rejects an even or non-positive value outright rather than silently producing a shifted or garbage result.

The conservation check. A properly normalized Gaussian kernel (this one sums to exactly 1) should barely change the **total integrated DOS** of each column — broadening redistributes weight in energy, it doesn't create or destroy states. `[3] CONSERVATION CHECK` integrates (trapezoidal rule) each column before and after broadening and reports both numbers side by side. A **large** drift almost always means the energy window is too narrow for the requested broadening — weight simply falls off the zero-padded edges of the window and is lost. This is a genuine runtime physics sanity check, not just a formatting nicety: if you ever see the "before" and "after" numbers diverge by more than a percent or so, trust the warning, not the filtered file.

`--dir`: broadening a whole `stb-dos` output tree at once. `stb-dos` (2.2) can write dozens of files for a real structure — `dos_total.dat`, one `dos_per_atom/<species>_<index>.dat` per atom, one `dos_per_species/dos_<species>.dat` per species. Running `stb-convdos` by hand on every one of them individually doesn't scale. `--dir` walks the whole tree recursively, applies the identical broadening to every `.dat` file found, and writes a **mirrored** tree (same relative paths, same filenames) under `--output-dir` — a drop-in "broadened copy" of the original `stb-dos` output, ready to plot the same way.

Only the **plot** behaves differently in `--dir` mode: with dozens of files, showing a before/after figure for every single one would be impractical (and likely to overwhelm the display). Only the **first** file processed gets its plot shown; the rest are still broadened and written normally, just without a plot window. `--no-plot` turns off that one remaining plot too.

8.3.1.3 When you'd reach for it

- Making a jagged, under-sampled DOS from a quick/coarse SIESTA calculation presentable, without re-running with a denser k-mesh.
- Matching a calculated DOS's resolution to an experimental spectrum's known instrumental broadening (if you know its FWHM).
- Broadening an entire `stb-dos` output folder (total + per-atom + per -species) in one command instead of one `stb-convdos` call per file.

8.3.1.4 Two ways to run it

A — direct CLI:

```
stb-convdos --file dos_total.dat --sigma 50 --out dos_filtered.dat
```

B — interactive `stb-suite` menu:

```
$ stb-suite
Select an option (0-7, or a tool code like 5.1.2): 2.3
```

Both paths call the exact same underlying tool and produce the exact same output — `example_2.3.sh` proves this directly at the end. The menu adds one extra first choice (single file vs. a whole folder) before the usual `--sigma`/`--fwhm`/`--size`/plot prompts.

8.3.1.5 What every run does (always on)

- **A numbered report** (`[0] RUN METADATA ... [6] SUMMARY & FILES`) printed to the console.
- **The filtered `.dat` file(s)** — one for `--file`, a mirrored tree for `--dir`.
- **`references.bib`** — SIESTA (every DOS file processed here, directly or via `stb-dos`, is ultimately derived from a SIESTA calculation).

8.3.1.6 Optional (off by default)

- **`--save-report`** — also persists the full numbered report to `stb_convdos_report.txt`.
- **`--size`** — explicit kernel width in samples (must be odd), instead of the automatic 3-sigma sizing.
- **`--output-dir`** (`--dir` mode only) — where the broadened tree goes; default `<dir>_filtered`.
- **`--no-plot`** — skip the before/after plot entirely (`--dir`: skips the one plot it would otherwise show, for the first file).

8.3.1.7 Files in this folder

- `dos_total.dat` — a small, hand-built synthetic "stick spectrum": 41 energy points (-2 to +2 eV, 0.1 eV spacing), two columns (s, p) that are exactly zero except for 2 sharp spikes each — deliberately built to make the broadening effect (spike -> smooth peak) and the conservation check (`before`/`after` areas match exactly) easy to see and verify by hand.
- `multi.PDOS.xml` — a small synthetic PDOS (3 atoms/2 species), the same fixture `examples/2.2-stb-dos/` uses — processed live by `example_2.3.sh` via a real `stb-dos` run to generate a genuine `dos_total.dat` + `dos_per_atom/*.dat` + `dos_per_species/*.dat` tree, used to demonstrate `--dir` on the exact use case it's built for.
- `example_2.3.sh` — the guided walkthrough (**not** an automated test — see `test/2-analysis/3-dos-convolution/test.sh` for that).
- `.gitignore` — excludes `output/`.

8.3.1.8 Running the walkthrough

```
cd examples/2.3-stb-convdos
./example_2.3.sh
```

It pauses between sections (`[Press Enter to continue]`) and always starts by wiping its own `output/`. Self-contained cases are generated:

Folder	What it shows
`basic-broadening/`	Sharp spikes -> smooth peaks, the numbered report, the conservation check
`sigma-vs-fwhm/`	`--sigma 50` and the equivalent `--fwhm 117.741` give the identical broadening
`dir-mode/`	A real `stb-dos` output tree broadened in one `--dir` command, mirrored structure, one skipped-file example
`full-report/`	Default (no report file) vs. `--save-report` (`stb_convdos_report.txt`), `references.bib`

8.3.1.9 Try it yourself

```
# Broaden one file from a real stb-dos run
stb-convdos --file dos_total.dat --sigma 50 --out dos_total_filtered.dat

# Broaden an entire stb-dos output folder in one command
stb-convdos --dir my_calc_dos/ --sigma 50 --save-report
```

8.3.1.10 Flag reference

Flag	Meaning
`--file`	Input file to broaden. Requires `--out`. Alternative to `--dir`.
`--dir`	Input folder to broaden recursively (every `.dat` found). Alternative to `--file`.
`--sigma`	Gaussian standard deviation, meV. Alternative to `--fwhm`.
`--fwhm`	Gaussian full width at half maximum, meV (`= 2.3548 * sigma`). Alternative to `--sigma`.
`--size`	Explicit kernel width in samples (must be odd). Default: auto (3-sigma rule).
`--out`	Output file. Required with `--file`, invalid with `--dir`.
`-o/--output-dir`	Output tree root for `--dir` (default `<dir>_filtered`). Invalid with `--file`.
`--no-plot`	Skip the before/after plot.
`--save-report`	Persist the full numbered report to `stb_convdos_report.txt`.

8.3.1.11 What's next

`stb-convdos` is a pure signal-processing post-processor — it has no opinion about *where* its input DOS files came from, only that they're whitespace-separated columns with energy first. `stb-dos` (2.2) is its natural upstream partner (and the `--dir` mode above is built specifically around that pairing), but any DOS-like data file in the same column convention works the same way.

8.3.2 Implementation details

Gaussian convolution on a grid

The broadened DOS is the convolution

$$\tilde{g}(E) = \int g(E') \, \frac{1}{\sigma\sqrt{2\pi}} \exp\left[-\frac{(E-E')^2}{2\sigma^2}\right] dE', \qquad \mathrm{FWHM} = 2\sqrt{2\ln 2}\,\sigma \approx 2.3548\,\sigma.$$

On the discrete grid it is computed with a normalized kernel of odd width K samples, $w_j \propto \exp[-j^2\Delta E^2/2\sigma^2]$, $j = -(K-1)/2, \ldots, (K-1)/2$, where ΔE is the median spacing of the grid. By default K is the smallest odd number covering about 3σ on each side.

Units and checks.. `-sigma` or `-fwhm` are given in meV (exactly one of them) and converted to samples with the grid spacing; the report warns if the grid is not uniform, since then a single spacing does not represent the whole grid. An explicit `-size` must be a positive odd number (an even kernel has no central sample and shifts the curve by half a bin). Near the ends of the energy window the convolution assumes zeros outside the data, so the first and last $\sim 3\sigma$ are less reliable.

Conservation.. A normalized kernel preserves the integral of each column. The report lists the trapezoidal integral of every column before and after; a difference larger than the edge effect means the window was too narrow.

Batch mode.. `-file` broadens one file (energy in the first column, any number of DOS columns, an optional `#` header whose labels are kept). `-dir` broadens every `.dat` below a folder – typically the whole output tree of **`stb-dos`** – and writes a mirror of it under `-output-dir`. A preview figure (original and broadened side by side, one row per column) is shown unless `-no-plot` is given.

8.3.3 Command-line reference

8.3.3.1 `stb-convdos`

Apply Gaussian convolution to broaden a DOS file's columns.

Synopsis.

```
usage: stb-convdos [-h] (--file INPUT_FILE | --dir INPUT_DIR) (--sigma SIGMA |
                   --fwhm FWHM) [--size SIZE] [--out OUTFILE] [-o OUTPUT_DIR]
                   [--no-plot] [--save-report] [-v] [--no-intro]
```

Options

`--file INPUT_FILE`
: Input DOS file: whitespace-separated columns, energy first. Requires `--out`. Alternative to `--dir`. exclusive group 1

`--dir INPUT_DIR`
: Recursively broaden every `*.dat` file found under this directory (e.g. a `stb-dos` output folder: `dos_total.dat` + dos_per_atom/*.dat + dos_per_species/*.dat) with the same `--sigma/--fwhm/--size`. Each file is written under `--output-dir` preserving its relative path and filename – a broadened mirror of the whole input tree. A file that fails validation is skipped with a warning, not fatal to the rest of the batch. Alternative to `--file`.exclusive group 1

`--sigma SIGMA`
: Gaussian broadening standard deviation, in meV. Converted internally to grid samples using each input file's own energy spacing, so the same `--sigma` gives the same physical broadening regardless of how densely a file is sampled. Must be positive. Alternative to `--fwhm`. type: float; exclusive group 2

`--fwhm FWHM`
: Gaussian broadening full width at half maximum, in meV (FWHM = 2.3548 * sigma) – the width more commonly quoted in spectroscopy/DOS literature. Must be positive. Alternative to `--sigma`. type: float; exclusive group 2

`--size SIZE`
: Gaussian kernel width, in samples. Optional – by default sized automatically from `--sigma/--fwhm` to cover about 3 standard deviations on each side. Must be a positive odd number if given explicitly (an even size shifts the filtered curve by half a bin, since it has no single center sample). type: int

`--out OUTFILE`
: Output file for the filtered data. Required with `--file`, invalid with `--dir`.

`-o, --output-dir OUTPUT_DIR`
: Root of the broadened output tree for `--dir` (default: '<dir>_filtered', a sibling directory). Invalid with `--file` (use `--out` there).

`--no-plot`
: Skip the before/after plot (all columns are still filtered and written). With `--dir`, only the first processed file is ever plotted regardless of this flag's default; `--no-plot` turns that single plot off too. switch

`--save-report`
: Also persist the full run report to `stb_convdos_report.txt`. Off by default. switch, off by default

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Example usage:
  stb-convdos --file dos_total.dat --sigma 50 --out dos_filtered.dat
  stb-convdos --file dos_total.dat --fwhm 100 --size 41 --out dos_filtered.dat --no-plot
  stb-convdos --dir dos_output/ --sigma 50 --output-dir dos_output_filtered/
```

8.4 Structure Analyzer

At a glance

Menu code: 2.4 **Command:** `stb-structural`
Guided example: `examples/2.4-stb-structural/`
Summary: Calculate ECN and analyze structural properties.

8.4.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/2.4-stb-structural/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

8.4.1.1 What this tool does

`stb-structural` reads a SIESTA structure file (`.fdf` or a post-relaxation `.STRUCT_OUT`) and reports a broad set of local and extended structural properties, with no SIESTA re-run needed:

- Lattice parameters, cell volume, density.
- **Effective Coordination Number (ECN)**, computed by 4 independent methods side by side.
- Bond distances and bond angles, pooled per species-pair / ligand-center -ligand triplet.
- Coordination-polyhedron **distortion**: bond-length distortion (BLD), bond-angle variance (BAV), and polyhedron volume.
- The shortest same-species contact distance in the structure.
- Coordination-polyhedron **connectivity** (corner-/edge-/face-sharing) between same-species centers.
- Optionally, the **radial distribution function** g(r).

`--mode mean` analyzes every atom in the structure, aggregated per species. `--mode list --list 1,4,5` restricts the atom-level analyses (ECN, distortion) to only the atoms you name — see "`--mode mean` vs. `--mode list`" below for exactly what does and doesn't get scoped down.

8.4.1.2 Why this matters (a bit of theory)

There is no single, universally-agreed "coordination number" for a real (often distorted, sometimes disordered) structure — every method below is a different **operational definition** of "how many neighbors does this atom have," built on a different criterion for where a neighbor shell ends. That's exactly why this tool reports 4 of them side by side instead of picking one: where they agree, you can trust the number; where they disagree noticeably, that itself is informative (a strong sign the local environment is genuinely ambiguous or distorted, not a bug in one of the methods).

Effective Coordination Number (ECN) — the 4 methods. All four are computed with **weighted** coordination numbers (a continuous, "effective" count — e.g. `5.98` rather than a flat integer `6`) rather than a hard neighbor cutoff, via pymatgen's `local_env` module.

- **MinDistNN** (`MinimumDistanceNN`) — the simplest of the four: find the closest neighbor at distance `d_min`, then include every neighbor within `(1 + tol) * d_min` (default `tol = 0.1`, i.e. within 10% of the closest bond). Each included neighbor's weight is `d_min / d_i`, so a neighbor right at the closest distance contributes weight 1.0 and one further away contributes proportionally less. **Limitation**:

a single global relative tolerance either lumps a legitimately slightly-longer bond into the count or excludes it entirely — there's no adaptation to how spread out a given site's own bond lengths naturally are.

- **BrunnerNN** (`BrunnerNNRelative`) — sorts *every* neighbor distance up to a cutoff, then finds the largest **relative gap** (the biggest ratio `d_{i+1} / d_i` between two consecutive sorted distances). That gap is taken as the boundary between the first coordination shell and everything beyond it; every neighbor closer than the gap is included, weighted the same way as MinDistNN (`d_min / d_i`). **Limitation**: it assumes a clearly separated shell exists at all. In densely/continuously packed environments (common in metals or disordered structures) there may be no single obviously-largest gap, in which case the chosen cutoff is not physically meaningful, just whichever ratio happened to be biggest.
- **EconNN** (Hoppe, 1979 — the original ECoN, "Effective Coordination Number") — a smooth, self-consistent formula instead of any hard cutoff: each neighbor's weight is `w_i = exp(1 - (d_i / d_avg)^6)`, where `d_avg` is itself solved **iteratively** as the weighted mean bond length (start from the simple mean, recompute the weights, recompute the mean, repeat to convergence). The final `ECoN = sum(w_i)`. Because the weight decays smoothly rather than switching on/off at a cutoff, a neighbor doesn't have to be "in" or "out" — it can contribute a small fraction. **Limitation**: designed originally for ionic/ceramic bonding (Hoppe's full formula can use "fictive ionic radii" instead of raw bond distances); this tool uses raw bond distances only (`use_fictive_radius=False`), so the ionic-radius refinement isn't applied here.
- **CrystalNN** (Zimmermann & Jain) — the most elaborate of the four: a Voronoi tessellation gives each candidate neighbor a solid-angle-based weight, which is then refined by a smooth distance cutoff (based on the sum of covalent radii) and a preference for larger Pauling electronegativity differences (chemically motivated: a cation is more likely bonded to an anion than to another cation at a similar distance). Its own docstring notes the default parameters were benchmarked specifically for inorganic crystal structures. **Limitation**: the most "opinionated" of the four — its result depends on more internal parameters (electronegativity weighting, distance -cutoff smoothing) than a purely geometric method, so it can diverge most from the others for chemistries or bonding types (e.g. molecular crystals, MOFs) its defaults weren't tuned for.

JmolNN was deliberately left out. It was tested and consistently gave the worst agreement with the actual local geometry of the five candidate methods — its reference distance comes from a fixed Jmol bonding-radius lookup table rather than the atom's own closest-neighbor distance, so its weight isn't on a comparable scale to the other four.

Bond distances and bond angles. For every atom in scope (all atoms in `mean` mode, just the requested ones in `list` mode), every neighbor found by `CrystalNN` (unweighted here — a plain neighbor list, not a coordination-number calculation) at that site contributes one bond distance and one angle per neighbor pair at that same center. Results are pooled **by species pair** (bonds) or **ligand-center-ligand species triplet** (angles) — e.g. every `O-Sn` bond in the structure is averaged together, every `O-Sn-O` angle separately from every `Sn-O-Sn` angle — rather than one meaningless structure-wide average mixing chemically distinct bond types.

In `mean` mode, the same physical bond can be counted from both of its endpoints' neighbor lists if the neighbor relationship is symmetric (`n=` in the report makes this visible, so it's never hidden). This doesn't bias the pooled *average* value — it's the same number counted twice — but does mean `n` is not simply "number of bonds in the structure."

Coordination-polyhedron distortion: BLD, BAV, Volume.

- **BLD (Bond-Length Distortion, %)** — Baur's index: the mean absolute deviation of a site's own bond lengths from *that site's own* mean bond length, as a percentage:

  ```
  BLD = 100 * mean(|d_i - d_avg| / d_avg)
  ```

  Limitation: needs at least 2 bonds. It only measures how *unequal* the bond lengths are — a site with 6 identical-length bonds arranged in a wildly non-octahedral shape (e.g. all pointing into one hemisphere) would score BLD = 0%, since BLD says nothing about angles.

- **BAV (Bond-Angle Variance, deg^2)** — based on Robinson et al. (1971), but **deliberately modified** here in two ways, both because a fractional/"effective" coordination number has no single well-defined ideal polyhedron to compare against:

  1. Uses the site's own **observed mean angle** as the reference, instead of a theoretical ideal angle (90°/109.5°/etc. for an idealized octahedron/tetrahedron).
  2. Includes **every** pairwise ligand-center-ligand angle at that site, not just the 12 "cis" (~90°) angles Robinson's original octahedral formula uses — there's no general way to separate "cis" from "trans" pairs outside an assumed 6-coordinate octahedral topology.

  For a real 6-coordinate site, including the ~180° "trans" angles too inflates BAV well above textbook octahedral values. **Limitation (important)**: because of this, **BAV values from this tool are not directly comparable to BAV values quoted in the literature** — they're still a valid, self-consistent distortion metric for comparing sites/species *within one report*, just not against external benchmarks. Needs at least 3 neighbors (2 angles).

- **Volume (Å^3)** — the volume of the convex hull (`scipy.spatial. ConvexHull`) of the neighbor atoms' Cartesian positions. **Limitation**: needs at least 4 non-coplanar neighbors to bound a 3D volume (`N/A` otherwise — you'll see this for e.g. a 3-coordinate site in the example below). It also says nothing about *where* the central atom sits relative to that hull — an atom pushed far off-center inside an otherwise-normal-sized polyhedron still reports a normal volume.

Same-species minimum distance. The single shortest distance between any two atoms of the same species anywhere in the structure (e.g. the closest `O···O` contact). A quick packing/steric sanity check — an implausibly short same-species distance usually flags a bad structure (overlapping atoms) before you waste time on a real SIESTA run — or, for metals, a genuine metal-metal bonding distance. **Limitation**: it's a single number, the global minimum — not a distribution. One unusually close pair dominates the report; it says nothing about the *typical* same-species spacing.

Coordination-polyhedron connectivity. How two same-species coordination polyhedra (e.g. two `SnO_x` polyhedra) are linked, classified by how many ligand atoms they have in common:

Shared ligands	Classification
1	corner-sharing
2	edge-sharing
3+	face-sharing

This is a standard way to describe extended structural topology in crystallography (e.g. rutile's edge-sharing `TiO6` octahedral chains vs. perovskite's corner-sharing `BO6` network). **Limitation**: the neighbor list used here comes from a separate, **unweighted** `CrystalNN` pass — a plain yes/no neighbor list, not the same *weighted* coordination number reported in the ECN section above. The same atom can therefore show e.g. `CrystalNN: 4.004` in the ECN table while its connectivity classification was built from a discrete neighbor set that isn't identical to "the nearest 4 atoms."

Radial distribution function g(r). The pair correlation function: `g(r)` is proportional to the probability of finding an atom at distance `r` from a reference atom, relative to what a uniform (ideal-gas-like) density would predict. `g(r) = 1` everywhere would mean "no structure at all"; the real, sharp peaks you see are the coordination shells.

Standard number-density normalization, per species pair (A as reference, B as target):

```
g_AB(r) = <n_AB(r, r+dr)> / (4*pi*r^2*dr*rho_B)
```

averaged over every A-type atom as the reference. The **first peak**'s position is the average nearest-neighbor A-B distance; **integrating g(r) under that first peak recovers the coordination number**:

```
N_AB = 4*pi*rho_B * integral( r^2 * g_AB(r) dr )   [from 0 to the shell boundary]
```

This is a genuine independent cross-check against the ECN section above (same physics, computed a completely different way) — the walkthrough below does exactly this integral by hand and recovers the known O-Sn coordination number. **Limitations**: needs a large enough `--rdf-rmax` (and, implicitly, a large enough cell/supercell) that periodic-image finite-size effects don't distort the tail of the curve; the bin width `dr` (fixed at 0.05 Å here) trades peak resolution against statistical noise; and `g(r)` is purely radial — it carries no information about **angular** order (that's what the bond-angle section above is for).

`--mode mean` vs. `--mode list`: what stays whole-structure. `--mode list --list 1,7` restricts the **ECN** and **distortion** (BLD/BAV/Volume) sections to only the atoms you name — computing (and printing) results for the whole structure would be wasted work if you only care about one or two sites.

The **same-species minimum distance** and **coordination-polyhedron connectivity** sections, however, are always computed over the *entire* structure regardless of `--mode`/`--list` — both describe properties of the extended network (the closest contact anywhere; how polyhedra link up across the whole cell), not a property of any single atom, so scoping them down to a couple of named atoms wouldn't make physical sense.

8.4.1.3 When you'd reach for it

- A quick coordination-environment sanity check on a structure before committing to a real SIESTA run (spotting an accidentally-collapsed or overlapping structure, an unphysical bond length, etc.).
- Comparing coordination number/bond length/distortion across several candidate structures (e.g. different space-group guesses from `stb-crystalcast`, or before/after a relaxation).
- Characterizing one or two specific sites of interest (`--mode list`) — e.g. a defect site, a substitution site, a surface atom — without wading through the whole structure's statistics.

8.4.1.4 Two ways to run it

A — direct CLI:

```
stb-structural --file structure.fdf --format fdf --mode mean
```

B — interactive `stb-suite` menu:

```
$ stb-suite
Select an option (0-7, or a tool code like 5.1.2): 2.4
```

Both paths call the exact same underlying tool and produce the exact same output — `example_2.4.sh` proves this directly at the end.

8.4.1.5 What every run does (always on)

- **A numbered report** (`[0] RUN METADATA ... [11] SUMMARY & FILES`) printed to the console.
- **`rdf.dat`** — the full g(r) curve, unless `--no-rdf` is given.
- **`references.bib`** — SIESTA (every structure file analyzed here is SIESTA input/output).

8.4.1.6 Optional (off by default)

- **`--save-report`** — also persists the full numbered report to `stb_structural_report.txt`. Without it, only `rdf.dat` and `references.bib` are written — no text report file at all (the old, always-on `structural_information.dat` this tool used to write unconditionally is gone; a stale one left by an older run is cleaned up automatically instead of being mistaken for current output).
- **`--no-rdf`** — skip the radial distribution function entirely.
- **`--rdf-rmax`** — g(r) cutoff radius in Å (default `10.0`).

8.4.1.7 Files in this folder

- `structure.fdf` — a real, small (14-atom, Sn_3O_4-composition) SIESTA structure, copied from `test/2-analysis/4-structure/structure.fdf`. Small enough to be a lightweight example, but rich enough to exercise every section of the report: two chemical species with clearly different coordination environments (6-coordinate Sn, 3-coordinate O — including the `Volume: N/A` case for a site with fewer than 4 neighbors), a genuine O-O and Sn-Sn same-species contact, and both corner- and edge-sharing polyhedra.
- `example_2.4.sh` — the guided walkthrough (**not** an automated test — see `test/2-analysis/4-structure/test.sh` for that, which additionally covers the post-relaxation `.STRUCT_OUT` format on the same structure).
- `.gitignore` — excludes `output/`.

8.4.1.8 Running the walkthrough

```
cd examples/2.4-stb-structural
./example_2.4.sh
```

It pauses between sections (`[Press Enter to continue]`) and always starts by wiping its own `output/`. Self-contained cases are generated:

Folder	What it shows
`mode-mean/`	Full-structure analysis: ECN (4 methods) per species, bond distances/angles, distortion, connectivity
`mode-list/`	`--mode list --list 1,7`: ECN/distortion scoped to 2 atoms, same-species distance/connectivity still whole-structure
`rdf/`	g(r), and hand-integrating its first O-Sn peak to independently recover the ECN section's coordination number
`full-report/`	Default (no report file) vs. `--save-report` (`stb_structural_report.txt`), `references.bib`

8.4.1.9 Try it yourself

```
# Your own finished (or pre-relaxation) SIESTA structure
stb-structural --file my_calc.fdf --format fdf --mode mean --save-report
```

```
# Focus on specific atoms (e.g. a defect site you just built)
stb-structural --file my_calc.fdf --format fdf --mode list --list 12,45
```

8.4.1.10 Flag reference

Flag	Meaning
`--file`	Path to the structure file.
`--format`	`fdf` (SIESTA input) or `struct_out` (post-relaxation `.STRUCT_OUT`).
`--mode`	`mean` (whole structure, per species) or `list` (specific atoms).
`--list`	Comma-separated 1-based atom indices. Required with `--mode list`.
`-o/--output-dir`	Where to write `rdf.dat`/`references.bib` (and the report, with `--save-report`).
`--no-rdf`	Skip the radial distribution function (`rdf.dat` not written).
`--rdf-rmax`	g(r) cutoff radius, Å (default `10.0`).
`--save-report`	Persist the full numbered report to `stb_structural_report.txt`.

8.4.1.11 What's next

The ECN methods here (`pymatgen.analysis.local_env`) are used only by this tool in the suite — no shared `core/` module for coordination geometry exists yet (extract-on-second-use, per this project's own convention, once a second tool needs the same machinery). `stb-symmetry` (2.5) covers a complementary question — the structure's space-group/ point-group symmetry — rather than its local coordination environment.

8.4.2 Implementation details

Input and modes.. The structure is read from a SIESTA `.fdf` (`-format fdf`) or from the `<label>.STRUCT_OUT` written at the end of a relaxation (`-format struct_out`). `-mode mean` reports per-species averages over the whole structure; `-mode list -list 1,4,5` reports the listed atoms (1-based) one by one. The RDF and the same-species minimum distances always describe the whole structure.

The quantities, as defined in the code

Coordination numbers
four estimators from pymatgen's local-environment module, all in their weighted form: `MinDistNN` (neighbours within a tolerance of the shortest distance), `CrystalNN` (Voronoi solid angles with chemical and distance penalties, weighted), `BrunnerNNRelative` (largest relative gap in the sorted distances) and `EconNN`, Hoppe's effective coordination number

$$\mathrm{ECoN} = \sum_i \exp\left[1 - \left(\frac{l_i}{l_{\mathrm{av}}}\right)^6\right],$$

with l_{av} a self-consistently weighted mean bond length. Disagreement between estimators is itself information: it signals an ambiguous first shell.

Bond-length distortion
(Baur) $\Delta = \frac{100\%}{n}\sum_i |l_i - \bar{l}|/\bar{l}$ over a site's $n \geq 2$ bonds.

Bond-angle variance
$\sigma_\theta^2 = \frac{1}{m-1}\sum_i(\theta_i - \bar{\theta})^2$ over the ligand–centre–ligand angles, measured around their own mean rather than an ideal polyhedron angle, so it applies to any coordination.

Polyhedron volume
the convex hull of the neighbours (needs ≥ 4 non-coplanar neighbours).

Connectivity
two same-species polyhedra sharing 1, 2 or ≥ 3 ligands are corner-, edge- or face-sharing.

RDF
partial $g_{ab}(r)$ for every species pair and the total $g(r)$, from a periodic neighbour search out to `-rdf-rmax` (default 10 Å), normalized by the ideal-gas shell density so that $g \to 1$ at large r in a dense solid; the report quotes each first peak and `rdf.dat` has the curves.

Notes.. Coordination analysis of slabs and molecules is valid (the vacuum simply contributes no neighbours), but their $g(r)$ does not tend to 1 because the cell volume includes the vacuum. CrystalNN's warnings about missing oxidation states are expected for plain `.fdf` input and are filtered.

8.4.3 Command-line reference

8.4.3.1 `stb-structural`

Compute ECN and structural properties from a SIESTA structure file.

Synopsis.

```
usage: stb-structural [-h] --file FILE --format {fdf,struct_out} --mode {list,mean}
                      [--list LIST] [-o OUTPUT_DIR] [--no-rdf] [--rdf-rmax RDF_RMAX]
                      [--save-report] [-v] [--no-intro]
```

Options

`--file FILE`
Path to structure file. required

`--format {fdf,struct_out}`
Input file format:
fdf: SIESTA structure input (%block LatticeVectors etc.)
struct_out: SIESTA post-relaxation output (.STRUCT_OUT) required; choices: fdf, struct_out

`--mode {list,mean}`
Calculation mode: list or mean required; choices: list, mean

`--list LIST`
List of atom indices (comma-separated, 1-based). Example: 1,4,5,7 - Required for 'list' mode

`-o, --output-dir OUTPUT_DIR`
Directory to write `rdf.dat` and `references.bib` into (and `stb_structural_report.txt`, with `--save-report`) (default: current directory). Created if it doesn't exist.

`--no-rdf`
Skip the radial distribution function g(r) (`rdf.dat` is not written). switch

`--rdf-rmax RDF_RMAX`
Cutoff radius for g(r), in Å (default: 10.0). Larger values capture more coordination shells but take longer for large structures. type: float

`--save-report`
Also persist the full run report to `stb_structural_report.txt`. Off by default. switch, off by default

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Example usage:
  stb-structural --file structure.fdf --format fdf --mode mean
  stb-structural --file siesta.STRUCT_OUT --format struct_out --mode list --list 1,4,5
  stb-structural --file structure.fdf --format fdf --mode mean --save-report
```

8.5 Symmetry Analyzer

At a glance

Menu code: 2.5 **Command:** `stb-symmetry`
Guided example: `examples/2.5-stb-symmetry/`
Summary: Analyze the symmetry of crystal structures.

8.5.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/2.5-stb-symmetry/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

8.5.1.1 What this tool does

`stb-symmetry` reads a SIESTA structure file (`.fdf` or a post-relaxation `.STRUCT_OUT`) and reports its symmetry, with no SIESTA re-run needed:

- The **3D space group**, Wyckoff positions/orbits, site symmetry, and the full list of symmetry operations.
- For a **vacuum-padded 2D slab** (exactly one vacuum axis): the **layer group** instead — the physically correct 2D classification.
- For a **vacuum-padded isolated molecule** (all 3 axes vacuum-padded): the **point group** instead.
- A per-atom **distortion** (Å): how far each atom sits from where the detected symmetry operations would place it exactly.
- Optionally: a **tolerance-sensitivity scan** (`--scan-symprec`), a **comparison** between two structures (`--compare-to`, e.g. before/after a SIESTA relaxation), and a **symmetry-refined structure** (`--write-refined`).

8.5.1.2 Why this matters (a bit of theory)

Space groups, Wyckoff positions, and symmetry operations. A crystal's **space group** is the complete set of symmetry operations (rotations, reflections, screw axes, glide planes, and the translations that repeat the unit cell) that map the structure onto itself. `stb-symmetry` detects this via spglib, wrapped by pymatgen's `SpacegroupAnalyzer` — the same detector `stb-unitcell` and `stb-fetch --unitcell` already use in this suite (`core/symmetry.py`).

Atoms related to each other by a symmetry operation belong to the same **Wyckoff orbit** (e.g. NaCl's 4 Na atoms all share orbit `4a`, its 4 Cl atoms all share `4b` — 8 atoms, but only 2 physically distinct environments).

Each orbit also has a **site symmetry** — the subgroup of operations that fix that particular position (e.g. `m-3m` for a rock-salt ion sitting exactly on a high-symmetry point).

Symmetry **operations** are printed in compact `x,y,z` (Seitz) notation — e.g. `-x, -y, -z` is an inversion through the origin, `x, y+1/2, z+1/2` is a translation by half the cell along `b` and `c`.

The distortion metric — and why it's *not* an absolute measure. Real structures are rarely perfectly symmetric (numerical noise from a DFT relaxation, or a deliberately displaced defect). `stb-symmetry` reports a per-atom **distortion** (Å): apply every *detected* symmetry operation to an atom's position and measure the distance to the nearest same-species atom that operation should map it onto exactly if the structure had perfect symmetry; the atom's distortion is the largest such distance over all non-identity operations.

Limitation: this is bounded by construction to roughly `<= --symprec` (that's the definition of `symprec`: the tolerance within which spglib accepted this operation set as valid in the first place) — it's a finer-grained, per-atom breakdown *within* that tolerance budget, not an absolute "how far from ideal" number independent of what tolerance you asked for. An earlier attempt used `SpacegroupAnalyzer.get_refined_structure()` directly, but that method empirically returns the input positions essentially unchanged (verified: every atom's distance to its own "refined" counterpart came out exactly 0.0, even for a deliberately noisy structure) — recomputing the residuals directly from the operations themselves, as done here, is what actually produces meaningful non-zero values correlated with the injected noise.

Why a vacuum-padded structure needs LAYER GROUP or POINT GROUP. spglib's 3D space-group detection has no concept of "vacuum" — it treats an empty gap along one axis as just an unusually tall periodic cell. For a slab (one vacuum axis), this means the reported 3D space group actually depends on the arbitrary vacuum thickness you chose, **not** the real 2D symmetry of the sheet itself. `stb-symmetry` calls this out explicitly (see the `[1] DIMENSIONALITY & VACUUM AXES` section) and, when possible, reports the physically meaningful classification instead:

- **1 vacuum axis** (a slab) → the **layer group**, via spglib's own dedicated `get_layergroup()` (needs spglib $\geq$ 2.1.0; $\geq$ 2.2.0 for a Hall-number bugfix) — a real, separate 2D-symmetry detector, not a heuristic filter of the 3D result.
- **3 vacuum axes** (an isolated molecule) → the **point group**, via pymatgen's separate, non-periodic `PointGroupAnalyzer` (it treats the structure as a plain molecule — Cartesian coordinates only, no lattice — the same way a quantum-chemistry package would). Its detection tolerance is a molecular, Å-scale parameter, **independent of `--symprec`** (a much tighter, crystallographic default) — pymatgen's own default (0.3 Å) is used unless overridden by `--scan-symprec`'s own point-group sweep.
- **2 vacuum axes** (a wire) → **neither section** — see Limitations below.

`--symprec` / `--angle-tolerance` and `--scan-symprec`. `--symprec` (Å) and `--angle-tolerance` (degrees) are the tolerances spglib uses to decide whether two atoms/angles are "the same" for symmetry purposes. A structure fresh out of a DFT relaxation often has small positional noise that hides its true symmetry at a tight tolerance — `--scan-symprec` sweeps a range of tolerances (tight to loose) and reports the first one at which the detected group changes, revealing symmetry that's really there but momentarily obscured by noise.

`--write-refined`. Writes a symmetry-refined structure: positions snapped to the detected symmetry, same reduction `stb-unitcell --mode refined` performs. Refines against whichever classification is physically meaningful for the structure's dimensionality (3D space group / layer group / point group). The file is written **inside `--output-dir`** (a plain filename, not a path of its own) alongside `references.bib`/the report, and carries a header documenting exactly how it was generated — source file/format,

`--symprec`/`--angle-tolerance`, and which group (space/layer/point) it was refined against — so the file is self-describing even opened on its own, without the console report next to it. **Known caveat** (documented in `core/symmetry.py`): the refined structure's atom order and coordinate origin are **never guaranteed** to match the input, in any mode — spglib/pymatgen rebuild the cell from the detected symmetry operations from scratch and are free to pick any symmetry-equivalent origin, which even tiny input noise can flip for highly symmetric structures. Not a bug; it's the same crystal, just possibly relabeled.

8.5.1.3 Limitations

- **No rod-group detection.** A wire (1D-periodic, 2 vacuum axes) has no fallback at all — spglib doesn't implement rod groups (the periodic analogue of a layer group for a 1D system). `stb-symmetry` reports this explicitly rather than silently falling back to the physically meaningless 3D space group.
- **Layer-group detection can fail even with exactly 1 vacuum axis** — the structure may not be genuinely 2D-periodic within `--symprec`, or the installed spglib may predate `get_layergroup()` ($\geq$ 2.1.0).
- **The point-group tolerance is a separate parameter**, not tied to `--symprec` — don't expect `--symprec` to change point-group detection sensitivity; use `--scan-symprec` to see the point group's *own* tolerance sweep instead.
- **The distortion metric is relative to `--symprec`**, not an absolute measure of "how far from an ideal position" (see above).
- **`--write-refined`'s atom order/origin is not preserved** (see above).
- **A single isolated atom** (all 3 axes vacuum-padded, e.g. `stb-cohesive`'s own isolated-atom reference structures) is a geometrically degenerate case for pymatgen's `PointGroupAnalyzer` — it's caught and reported as "detection... failed" rather than crashing the whole run, but no point group is available for it either.

8.5.1.4 The report: console output, `--save-report`, `--view`

Every run prints a numbered report to the console:

Section	Content
`[0] RUN METADATA`	date/time, input file/format, tolerances, output dir, active options
`[1] DIMENSIONALITY & VACUUM AXES`	which axes (if any) are vacuum-padded, and which section below is physically meaningful
`[2] SPACE GROUP`	the 3D space group, Hall symbol, point group, crystal system, Pearson symbol
`[3] LAYER GROUP` / `[3] POINT GROUP`	(conditional, exactly one or neither) the 2D or 0D classification
`[4] LATTICE`	cell parameters, volume, reduced formula, lattice vectors
`[5] TOLERANCE SENSITIVITY SCAN`	(conditional, `--scan-symprec`) how the detected group changes with tolerance
`[6] SYMMETRICALLY DISTINCT SITES`	the Wyckoff orbits
`[7] ATOMIC SITES`	per-atom fractional coordinates + distortion
`[8] SYMMETRY OPERATIONS`	(conditional, on unless `--no-operations`) the full x,y,z-notation list
`[9] SYMMETRY COMPARISON`	(conditional, `--compare-to`) space/layer/point group comparison between two structures
`[10] REFERENCES`	writes `references.bib` (SIESTA)
`[11] SUMMARY & FILES`	status and a recap of every file written

- **`--save-report`** — also persists the full numbered report to `stb_symmetry_report.txt`. Off by default, so a plain run only ever writes `references.bib` — no text report file at all (the old, always-on

`symmetry.dat` this tool used to write unconditionally is gone; a stale one left by an older run is cleaned up automatically instead of being mistaken for current output).
- **`--view`** — opens an interactive 3D view (via ASE) of the analyzed structure — or, if `--write-refined`/`--compare-to` is also given, both structures side by side (page through frames in ase-gui). Needs a display. Off by default.

8.5.1.5 When you'd reach for it

- A quick sanity check on a structure's symmetry before committing to a real SIESTA run (e.g. confirming a hand-built structure has the space group you intended).
- Checking whether symmetry **survived a relaxation** (`--compare-to` a pre-relaxation `.fdf` against its post-relaxation `.STRUCT_OUT`).
- Recovering symmetry hidden by small numerical noise (`--scan-symprec`).
- Classifying a 2D slab or an isolated molecule correctly, instead of trusting a 3D space group that only describes the padded supercell.
- Generating a clean, symmetry-refined structure (`--write-refined`) for a subsequent calculation.

8.5.1.6 Two ways to run it

A — direct CLI:

```
stb-symmetry --file nacl.fdf --format fdf
```

B — interactive `stb-suite` menu:

```
$ stb-suite
Select an option (0-7, or a tool code like 5.1.2): 2.5
```

Both paths call the exact same underlying tool and produce the exact same output — `example_2.5.sh` proves this directly at the end.

8.5.1.7 Files in this folder

- `nacl.fdf` — textbook rock-salt NaCl (space group Fm-3m, No. 225).
- `nacl_noisy.fdf` — the same NaCl with a small random atomic displacement, for the `--scan-symprec` demonstration.
- `graphene_slab.fdf` — a 2-atom graphene layer, 20 Å vacuum along `c`.
- `molecule.fdf` — an isolated water molecule, 10 Å vacuum on `a`/`b`/`c`.
- `wire.fdf` — a 1D-periodic chain, 10 Å vacuum on `a`/`b` — the documented "no rod group" limitation case.
- `example_2.5.sh` — the guided walkthrough (**not** an automated test — see `test/2-analysis/5-symmetry/test.sh` for that).
- `.gitignore` — excludes `output/` and other generated files.

8.5.1.8 Running the walkthrough

```
cd examples/2.5-stb-symmetry
./example_2.5.sh
```

It pauses between sections (`[Press Enter to continue]`) and always starts by wiping its own `output/`. Self-contained cases are generated:

Folder	What it shows
`bulk-nacl/`	Space group, Wyckoff orbits, symmetry operations, zero distortion on textbook NaCl
`scan-symprec/`	`--scan-symprec` recovering symmetry hidden by numerical noise
`layer-group/`	Why a 2D slab needs LAYER GROUP instead of (or in addition to) SPACE GROUP
`point-group/`	An isolated water molecule's POINT GROUP (C2v)
`limitation-wire/`	The documented "no rod group" limitation for a 1D wire
`full-report/`	Default (no report file) vs. `--save-report` (`stb_symmetry_report.txt`), `references.bib`

8.5.1.9 Try it yourself

```
# A structure you just built or relaxed
stb-symmetry --file my_calc.fdf --format fdf --save-report

# Check symmetry survived a relaxation
stb-symmetry --file my_calc.fdf --format fdf --compare-to my_relaxed.STRUCT_OUT --compare-format
struct_out

# Recover symmetry hidden by small numerical noise
stb-symmetry --file my_relaxed.STRUCT_OUT --format struct_out --scan-symprec
```

8.5.1.10 Flag reference

Flag	Meaning
`--file`	Path to the structure file.
`--format`	`fdf` (SIESTA input) or `struct_out` (post-relaxation `.STRUCT_OUT`).
`--symprec`	Symmetry-detection tolerance in Å (default `1e-3`).
`--angle-tolerance`	Symmetry-detection angle tolerance in degrees (default `5.0`).
`--no-operations`	Skip the full symmetry-operations list (keeps just the count).
`--scan-symprec`	Sweep a range of tolerances and report where the group changes.
`--compare-to`	Also analyze a second structure and compare symmetry.
`--compare-format`	File format of `--compare-to` (required if given).
`--write-refined`	Write the symmetry-refined structure as this filename, inside `--output-dir`, with a provenance header.
`-o/--output-dir`	Where to write `references.bib` (and the report, with `--save-report`).
`--save-report`	Persist the full numbered report to `stb_symmetry_report.txt`.
`--view`	Open an interactive 3D view (via ASE) after the report is written.

8.5.1.11 What's next

`stb-structural` (2.4) covers a complementary question — a structure's local coordination environment (bond lengths/angles, effective coordination number) — rather than its global space/layer/point-group symmetry. `stb-unitcell` reduces a structure to its primitive/conventional cell using the same underlying `core/symmetry.py` machinery this tool's `--write-refined` calls for a genuinely 3D bulk structure.

8.5.2 Implementation details

Which group is computed.. The tool first detects vacuum axes (Section 4.4) and reports the group that fits the dimensionality:

- 3D: the space group from spglib [9] via pymatgen – symbol, number, Hall symbol, point group, crystal system, Wyckoff letters and site symmetries, the orbits of equivalent atoms, and (unless `–no-operations`) every symmetry operation as rotation matrix and translation;
- 2D: in addition, the *layer group* (spglib's `get_layergroup`, spglib $\geq$ 2.1), which treats the vacuum axis as non-periodic; the 3D space group of the padded cell is printed but flagged as not physical;
- 0D: the molecular point group (pymatgen's `PointGroupAnalyzer`, which works in Cartesian space without periodicity).

Distortion metric.. For every atom, each detected operation is applied to its position and the distance to the nearest atom of the same species is measured (with periodic images); the largest of these distances is the atom's deviation from exact symmetry, in Å. It tells how close the structure is to the group found at the current tolerance, not an absolute measure of distortion – the guide explains why.

Tolerance scans and comparisons.. `–scan-symprec` repeats the detection at 10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}, 5×10^{-2} and 10^{-1} Å (layer group for slabs; point-group tolerances 0.01–1.0 for molecules) and reports the first tolerance at which the group changes: a group that only appears at loose tolerance is approximate. `–compare-to` analyses a second structure (for instance after relaxation) and reports whether the space group and the number of operations were preserved. `–write-refined` writes the symmetrized structure. `–view` opens the structure in ASE's viewer.

8.5.3 Command-line reference

8.5.3.1 `stb-symmetry`

Analyze crystal symmetry (space group, Wyckoff positions, symmetry operations) from a SIESTA structure file.

Synopsis.

```
usage: stb-symmetry [-h] --file FILE --format {fdf,struct_out} [--symprec SYMPREC]
                    [--angle-tolerance ANGLE_TOLERANCE] [--no-operations]
                    [--scan-symprec] [--compare-to PATH]
                    [--compare-format {fdf,struct_out}] [--write-refined FILENAME]
                    [-o OUTPUT_DIR] [--save-report] [--view] [-v] [--no-intro]
```

Options

`--file FILE`

Path to structure file. required

`--format {fdf,struct_out}`

Input file format:
fdf: SIESTA structure input (%block LatticeVectors etc.)
struct_out: SIESTA post-relaxation output (.STRUCT_OUT) required; choices: fdf, struct_out

`--symprec SYMPREC`

Symmetry-detection tolerance in Å (default: 0.01, pymatgen's own default). Loosen further (e.g. 0.02-0.05) for a structure relaxed with a looser force tolerance – a tighter value than the relaxation's own residual numerical noise can misdetect the true space group; use `--scan-symprec` to check where a given structure's symmetry call is sensitive. type: float

`--angle-tolerance ANGLE_TOLERANCE`

Symmetry-detection angle tolerance in degrees (default: 5.0). type: float

`--no-operations`

Skip the full symmetry-operations list in the report – keeps just the count in the SPACE GROUP section. Useful for high-symmetry structures where the full list is long. switch

`--scan-symprec`

Also detect symmetry at a range of tolerances (1e-05, 0.0001, 0.001, 0.01, 0.05, 0.1) and report where the space group changes – reveals symmetry that's present but hidden by small numerical noise (e.g. from a DFT relaxation) at the `--symprec` you asked for. For a structure with exactly one vacuum axis, also scans the layer group; for one with all 3 axes vacuum-padded (an isolated molecule), scans the point group's own tolerance instead (both physically meaningful there, in their own report sections). switch, off by default

`--compare-to PATH`

Also analyze a second structure file and report whether it shares the same space group – e.g. compare a pre-relaxation .fdf against its post-relaxation .STRUCT_OUT to check whether symmetry survived. Analyzed with the same `--symprec`/`--angle-tolerance` as `--file`. If both structures have a detected layer group (e.g. comparing a 2D slab) or point group (e.g. comparing an isolated molecule), that's compared too.

`--compare-format {fdf,struct_out}`

File format of `--compare-to` (required if `--compare-to` is given). choices: fdf, struct_out

`--write-refined FILENAME`

Also write the symmetry-refined structure (conventional cell, positions snapped to the detected symmetry) as this .fdf file inside `--output-dir`, with a header documenting the source file, tolerances, and which group it was refined against – the same reduction `stb-unitcell --mode` refined uses. For a structure with exactly one vacuum axis, refines against the layer group instead; for one with all 3 axes vacuum-padded (an isolated molecule), symmetrizes it as a molecule instead (both physically meaningful there, unlike the 3D space group) – skipped with an error for a wire (2 vacuum axes, no rod-group equivalent exists) or if the applicable detection failed.

`-o, --output-dir OUTPUT_DIR`

Directory to write `references.bib` into (and `stb_symmetry_report.txt`, with `--save-report`) (default: current directory). Created if it doesn't exist.

`--save-report`

Also persist the full run report to `stb_symmetry_report.txt`. Off by default. switch, off by default

`--view`

Open an interactive 3D view (via ASE) of the analyzed structure – or, if `--write-refined`/`--compare-to` is also given, both structures side by side (page through frames in ase-gui) – after the report is written. Needs a display. Off by default. switch, off by default

Like every STB command it also accepts `-h`/`--help`, `-v`/`--version` and `--no-intro` (Section 3.3).

From the built-in help.

```
Example usage:
  stb-symmetry --file structure.fdf --format fdf
  stb-symmetry --file siesta.STRUCT_OUT --format struct_out --symprec 1e-4
```

8.6 Bader Charge Analysis

At a glance

Menu code: 2.6 **Command:** `stb-bader`
Guided example: `examples/2.6-stb-bader/`
Summary: Calculate atomic charges using the Bader AIM method from .RHO and .XV files.

8.6.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/2.6-stb-bader/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

8.6.1.1 What this tool does

`stb-bader` computes Bader (Atoms-in-Molecules) atomic charges from a finished SIESTA calculation, with no extra SIESTA run needed:

- Reads `<label>.RHO` (the real-space charge-density grid) plus `<label>.XV` or `<label>.fdf` (the geometry), converts to a `.cube` file via sisl, and partitions the density into atomic basins with PyBader.
- Reports each atom's **population** (integrated electrons in its own basin), **net charge** (valence − population), and whether it's acting as a donor or acceptor.
- Auto-detects each species' **valence electron count** (`Z_val`) from `<label>.out` (or `--ref`), falling back to a hardcoded periodic-table guess per species when it can't.
- If the `.RHO` file is spin-polarized, also reports each atom's net spin (magnetic moment).
- Also reports each atom's **Bader volume** and minimum **surface distance**, a **per-species mean/std summary**, and cross-checks **symmetry-equivalent atoms** against each other.

8.6.1.2 Why this matters (a bit of theory)

Bader's Atoms-in-Molecules (AIM) theory. There is no operator in quantum mechanics for "how much charge belongs to this atom" — a molecule or crystal's electron density is one continuous field, not divided into per-atom pieces by anything in the underlying physics. Bader's AIM theory (1990) gives a rigorous, *purely density-based* way to divide it anyway: partition all space into regions separated by **zero-flux surfaces** — surfaces where the gradient of the electron density, $\nabla\rho(\mathbf{r})$, has zero component perpendicular to the surface at every point. Each such region (a "basin") is assigned to whichever nucleus it encloses, and integrating the density over a basin gives that atom's Bader population.

This is attractive specifically *because* it needs nothing extra: no fitted atomic radii, no basis-set-dependent partitioning (unlike e.g. Mulliken charges) — just the density itself, which is in principle an observable quantity. **Limitation**: it's still one specific, if well-motivated, choice of how to divide up a field that isn't naturally divided at all — "the charge on this atom" is not a direct experimental observable, and different partitioning schemes (Bader, Hirshfeld, Voronoi, ...) can disagree, sometimes substantially, especially for strongly covalent or highly delocalized bonding.

The grid-based algorithm — and its own approximation. Finding the *exact* zero-flux surfaces analytically is only tractable for toy densities. PyBader instead uses the **grid-based** approach of Tang, Sanville & Henkelman (*J. Phys.: Condens. Matter* **21**, 084204, 2009): starting from each voxel of the charge-density grid, follow the steepest ascent of the density (via a weighted average of neighboring

voxels) until you land on a density maximum (a nucleus) — every voxel that flows to the same maximum belongs to that atom's basin. The "near-grid" refinement step (this tool's default) corrects a known bias of the naive on-grid method, where basin edges can align suspiciously with the grid axes themselves instead of the true (grid-independent) zero-flux surface.

Limitation: this is fundamentally a **finite-resolution approximation** — basin boundaries are only as accurate as the density grid itself (SIESTA's real-space mesh, set by `MeshCutoff`/`PAO.MeshCutoff` originally). `--speed fast` switches to the plain on-grid method (faster, no edge-refinement pass) — useful for a quick look, but explicitly **not** recommended for a number you intend to quote.

Z_val detection and the unit-correction factor. A Bader *population* (raw integrated electron count in a basin) only becomes a *net charge* once you know how many valence electrons that species' pseudopotential started with (`Z_val` — net charge = `Z_val − population`). `stb-bader` parses this straight out of the actual SIESTA `.out` log for the pseudopotential really used (falling back to a generic periodic-table guess, with a loud warning, for any species it can't find there — see Limitations).

Because the whole calculation is on a finite real-space grid, the total integrated population across every atom can drift slightly from the theoretical total (`sum(Z_val)`) — usually a small, uniform effect of mesh resolution. If that drift exceeds 10%, `stb-bader` applies a single global multiplicative **correction factor** so the reported total matches the target. **Limitation**: this is one scalar applied uniformly to every atom — it's the right fix if the deviation really is a uniform mesh-resolution effect, but if the real cause is localized (one bad atom, a coarse region of the mesh, or a mismatched `.RHO`/geometry pairing), the correction just smears that same error evenly across every atom's reported charge instead of fixing it. A large correction factor is a cue to investigate further, not a guarantee the correction is physically right.

Symmetry cross-check. Atoms the structure's space group treats as equivalent (e.g. every O atom on the same Wyckoff site) are *physically required* to carry the same Bader charge. `stb-bader` runs pymatgen's `SpacegroupAnalyzer` on the geometry (best-effort — silently skipped if symmetry detection fails, e.g. for an already-relaxed structure with no exact symmetry left) and flags any symmetry-equivalent group whose charges disagree by more than `0.1 e-` — a red flag for numerical noise, a resolution problem localized to part of the cell, or a genuinely mis-set-up structure, invisible from looking at any single atom's row on its own.

8.6.1.3 Limitations

- **Non-nuclear attractors.** A real Bader analysis can occasionally find a density maximum that isn't centered on any atom at all (a "non-nuclear attractor" — most common in ionic or metallic bonding). PyBader always folds such a basin into the nearest atom; `stb-bader` cannot detect or correct for this happening.
- **`Z_val` accuracy.** Detected values come from parsing the actual pseudopotential generation log in `.out` — accurate for the exact pseudopotential used. The hardcoded `FALLBACK_VALENCE` table (used only for a species the `.out` parse can't find) is a generic "standard valence" guess and can be wrong for a semicore-inclusive pseudopotential (e.g. one that treats normally-core `d`/`f` electrons as valence).
- **`--speed fast`** trades basin-boundary accuracy for speed (see above) — a quick preview, not a final number.
- **Near-zero population is a red flag, not a clean result.** An atom reporting essentially `0` electrons (below `0.01 e-`) usually means PyBader found no density feature of its own to anchor a basin on there (e.g. a pseudopotential with deep semicore states frozen into the core, leaving too little resolvable valence density near that nucleus) and its whole region got folded into a neighbor's basin instead — this fixture's own Sn atoms #1/#2 demonstrate exactly this (see the

walkthrough below).

- **The unit-correction factor is a single global scalar** (see above) — it can mask a localized problem instead of fixing it.
- **A wrong `.RHO`/geometry pairing isn't always detectable.** The lattice and atom-count/order cross-checks catch a genuinely different calculation, but neither can catch a same-cell, same-atom-order mismatch — a `.RHO` grid carries no independent atomic-position record to compare against.
- **A spin-polarized `.RHO` has 2 raw components — up-spin and down-spin — not "total" and "spin".** An earlier version of this tool read them directly (`index=0`/`index=1`) as if they already were the total charge and the net spin; both were wrong (verified against a real spin-polarized O_2 calculation, a textbook triplet with SIESTA's own reported `|S| = 2.0`: the old reading integrated to ~7 e "charge" — the up channel alone — instead of the correct 12 e). Fixed by using sisl's own combination convention (`index='total'` = up+down, `index='z'` = up−down), the same one `stb-density` (2.8) independently needed the identical fix for.
- **PyBader's own cube reader has a separate, pre-existing bug with mixed-sign (negative-valued) cube files** — its fixed-line-width buffered read desyncs whenever a value's sign character shifts a line's length (confirmed with a minimal, stb-suite-independent reproduction: an all-positive synthetic cube reads fine, the same grid with negative values doesn't). A net spin/magnetization density is exactly this kind of mixed-sign data, so `--spin`-resolved analysis can hit this and gracefully fall back to charge-only (a `[WARN]`, not a crash) — unrelated to the up/down fix above, and outside this suite's own code to fix (it's inside the third-party `pybader` package).

8.6.1.4 The report: console output, `--save-report`

Every run prints a numbered report to the console:

Section	Content
`[0] RUN METADATA`	label, output dir, speed mode, `--ref`, vacuum tolerance, threads, cube/export flags
`[1] VALENCE (Z_val) SETUP`	Z_val source (detected vs. hardcoded fallback) per species
`[2] PYBADER CONFIGURATION`	method, refine method, threads, vacuum tolerance, spin-polarization
`[3] PER-ATOM BADER POPULATIONS`	the main per-atom table (population, Z_val, net charge, state, volume/surface distance, spin)
`[4] PER-SPECIES SUMMARY`	mean/std net charge per species
`[5] DIAGNOSTICS & WARNINGS`	unit-correction factor, near-zero-population atoms, symmetry-inconsistent groups, general limitations
`[6] REFERENCES`	writes `references.bib` (SIESTA); names the Bader/PyBader theory papers
`[7] SUMMARY & FILES`	status and a recap of every file written

- **`--save-report`** — also persists the full numbered report to `stb_bader_report.txt`. Off by default, so a plain run only ever writes the `.cube` file(s) and `references.bib` — no text report file at all (the old, always-on `<label>_BADER.txt` this tool used to write unconditionally is gone; a stale one left by an older run is cleaned up automatically instead of being mistaken for current output).
- **`--output-dir`** — where the `.cube` file(s), `references.bib`, the report, and any `--export-volumes` files all land (default: current directory).

8.6.1.5 When you'd reach for it

- A quick charge-transfer sanity check after a SIESTA SCF run (e.g. confirming an oxide's cations/anions show the expected donor/acceptor sign and roughly the expected magnitude).

- Comparing charge distribution across a series of related structures (a defect, a substitution, before/after adsorption).
- Visualizing individual atomic volumes in VESTA/VMD (`--export-volumes`).
- A magnetic-moment estimate per atom, for free, when the `.RHO` is spin-polarized.

8.6.1.6 Two ways to run it

A — direct CLI:

```
stb-bader --label Sn3O4
```

B — interactive `stb-suite` menu:

```
$ stb-suite
Select an option (0-7, or a tool code like 5.1.2): 2.6
```

Both paths call the exact same underlying tool and produce the exact same output — `example_2.6.sh` proves this directly at the end.

8.6.1.7 Files in this folder

- `Sn3O4.out`, `Sn3O4.RHO`, `Sn3O4.XV` — a real, finished SIESTA calculation on a 14-atom Sn_3O_4 structure (copied from `test/2-analysis/6-bader/`). Chosen because it genuinely exercises several of the tool's diagnostics at once: two chemical species with different bonding character (donor Sn, acceptor O), a real near-zero-population case on 2 of the 6 Sn atoms (see the walkthrough), and enough symmetry for the equivalent-atom cross-check to run.
- `example_2.6.sh` — the guided walkthrough (**not** an automated test — see `test/2-analysis/6-bader/test.sh` for that).
- `.gitignore` — excludes generated `.cube` files, `references.bib`, the report, and `output/`.

8.6.1.8 Running the walkthrough

```
cd examples/2.6-stb-bader
./example_2.6.sh
```

It pauses between sections (`[Press Enter to continue]`) and always starts by wiping its own `output/`. Self-contained cases are generated (each run takes a little while — PyBader is partitioning a real, fairly dense charge-density grid):

Folder	What it shows
`basic/`	Z_val detection, the per-atom population/net-charge table, per-species summary
`speed-fast/`	`--speed fast`'s on-grid method vs. the default near-grid method
`export-volumes/`	`--export-volumes`: one `.cube` per atom, for VESTA/VMD
`full-report/`	Default (no report file) vs. `--save-report` (`stb_bader_report.txt`), `references.bib`

8.6.1.9 Try it yourself

```
# A finished SIESTA calculation of your own
stb-bader --label my_calc --save-report

# Point at a specific .out if <label>.out isn't the right one
stb-bader --label my_calc --ref relax/my_calc.out --save-report

# Visualize individual atomic volumes in VESTA/VMD
stb-bader --label my_calc --export-volumes -o bader_out
```

8.6.1.10 Flag reference

Flag	Meaning
`-l/--label`	SystemLabel used in SIESTA.
`-o/--output-dir`	Where the `.cube` file(s)/`references.bib` (and the report, with `--save-report`) land.
`--save-report`	Persist the full numbered report to `stb_bader_report.txt`.
`--ref`	Path to a specific `.out` file to read `Z_val` from.
`--speed`	`normal` (near-grid, default) or `fast` (on-grid, less precise edges).
`--threads`	Worker threads for PyBader.
`--vacuum-tol`	Charge-density threshold below which a voxel is bucketed as vacuum (slabs/wires/molecules).
`--no-cube`	Delete the intermediate `.cube` file(s) after the run.
`--export-volumes`	Also write each atom's individual Bader volume as its own `.cube` file.

8.6.1.11 What's next

`stb-structural` (2.4) and `stb-symmetry` (2.5) both analyze a structure file directly (coordination environment, space/layer/point-group symmetry); `stb-bader` is this suite's only tool that instead analyzes a finished **SCF charge density**, giving physically grounded atomic charges rather than a purely geometric picture.

8.6.2 Implementation details

Bader partitioning

Bader's quantum theory of atoms in molecules [46] divides space into atomic basins bounded by zero-flux surfaces of the electron density, $\nabla\rho(\mathbf{r}) \cdot \hat{\mathbf{n}}(\mathbf{r}) = 0$; each basin contains one density maximum, normally at a nucleus. The Bader charge of atom A is $q_A = Z_A^{\mathrm{val}} - \int_{\Omega_A} \rho(\mathbf{r})\, d\mathbf{r}$. On a grid the basins are found by following steepest-ascent paths from every voxel [47]; the near-grid refinement [48] removes the bias of the grid directions.

Pipeline..

1. `<label>.RHO` and the geometry (`<label>.XV`, else the `.fdf`) are read with sisl and written as a standard Gaussian cube (Bohr units, density in e/Bohr^3).
2. The grid is handed to PyBader [17] with its default near-grid method; `-speed fast` switches to on-grid assignment (quicker, coarser basin boundaries). `-threads` sets the worker count (default: `SLURM_CPUS_PER_TASK` inside a SLURM job, otherwise all cores).

3. The valence charge Z^{val} of each species is read from the SIESTA log (`<label>.out`, or `-ref`); the guide explains the unit check applied to the integrated charge.
4. With `-vacuum-tol`, voxels below that density are collected in a separate "vacuum" basin instead of being forced into the nearest atom – recommended for slabs and molecules.
5. For a spin-polarized `.RHO`, the magnetization density is integrated over the same basins to give a moment per atom.

Consistency checks.. The report tabulates charge, population and volume per atom and per species, and checks them: an atom whose basin holds almost no electrons ($< 0.01\ e$) is flagged; atoms that the space group makes equivalent must have the same charge (spread $\leq 0.1\ e$); and a species whose atoms all carry a large ($|q| > 1\ e$ on average) and nearly identical ($\sigma < 0.15\ e$) charge is flagged, because that is the typical signature of a pseudopotential whose valence density has no resolvable maximum of its own (its charge is absorbed by the neighbours' basins). `-export-volumes` writes each atom's basin as its own cube file for inspection; `-no-cube` deletes the intermediate cube afterwards.

Grids and pseudopotentials.. SIESTA's `.RHO` contains only the *valence* pseudo-density, without core electrons, so basin boundaries near heavy atoms are less sharp than with all-electron densities, and the result converges with the real-space grid: increase `Mesh.Cutoff` until the charges stop changing. Bader charges are not oxidation states; compare them between related systems rather than reading them in isolation.

8.6.3 Command-line reference

8.6.3.1 `stb-bader`

Computes Bader (AIM) atomic charges from a SIESTA charge-density grid. Reads `<label>.RHO` (charge density) plus `<label>.XV` or `<label>.fdf` (geometry), converts to a .cube file via sisl, and partitions it into atomic basins with PyBader. Z_val (valence electron count per species) is parsed automatically from `<label>.out` when available (or `--ref` points at a specific SIESTA output); species it doesn't mention fall back to a hardcoded periodic-table guess, per species, with a warning. If the .RHO file is spin-polarized, a per-atom net spin (magnetic moment) is reported alongside the charge automatically. Dummy/no-element sites (Z<=0) are excluded to match PyBader's own handling; SIESTA's own ghost/BSSE atoms are unaffected. Also reports each atom's Bader volume and minimum surface distance, a per-species mean/std summary, and cross-checks symmetry-equivalent atoms against each other – on top of the per-atom warnings for a suspiciously low (near-zero) population.

Synopsis.

```
usage: stb-bader [-h] -l LABEL [-o OUTPUT_DIR] [--save-report] [--ref REF]
                 [--speed {normal,fast}] [--threads N] [--vacuum-tol TOL] [--no-cube]
                 [--export-volumes] [-v] [--no-intro]
```

Options

`-l, --label LABEL`
SystemLabel used in Siesta — required

`-o, --output-dir OUTPUT_DIR`
Directory to write the .cube file(s)/references.bib into (and `stb_bader_report.txt`, with `--save-report`; also any `--export-volumes` files) (default: current directory). Created if it doesn't exist.

`--save-report`
: Also persist the full run report to `stb_bader_report.txt`. Off by default. switch, off by default

`--ref REF`
: Path to a specific .out file to read Z_val from (overrides `<label>.out`)

`--speed {normal,fast}`
: 'fast' uses PyBader's own bundled speed profile (on-grid charge assignment instead of near-grid) – quicker, less precise basin boundaries (default: normal) choices: normal, fast

`--threads N`
: Worker threads for PyBader, >= 1 (default: $SLURM_CPUS_PER_TASK if set, else every CPU core – multiprocessing.cpu_count() can overcount under a job scheduler's cgroup limits) type: int

`--vacuum-tol TOL`
: Charge-density threshold (e/Ang^3) below which a voxel is assigned to a separate 'vacuum' bucket instead of the nearest atom – useful for slabs/wires/isolated molecules with empty space in the cell (default: disabled, matches PyBader's own default) type: float

`--no-cube`
: Delete the intermediate .cube file(s) after the calculation (kept by default – they can be large for dense grids) switch

`--export-volumes`
: Also write each atom's individual Bader volume as its own 'Bader-atoms-<N>.cube' file (plus a vacuum one if `--vacuum-tol` is set) inside `--output-dir`, for visual inspection in VESTA/VMD – one file per atom, each as large as the main grid, so this is opt-in (default: off) switch, off by default

Like every STB command it also accepts `-h`/`--help`, `-v`/`--version` and `--no-intro` (Section 3.3).

From the built-in help.

```
Usage examples:
  stb-bader --label siesta
  stb-bader --label siesta --ref relax/siesta.out --save-report
  stb-bader --label siesta --speed fast --threads 8
  stb-bader --label slab --vacuum-tol 1e-3
  stb-bader --label siesta --export-volumes
```

8.7 Work Function Calculator

At a glance

Menu code: 2.7 **Command:** `stb-workfunction`
Guided example: `examples/2.7-stb-workfunction/`
Summary: Calculate Work Function from electrostatic potential (.VT).

8.7.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/2.7-stb-workfunction/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

8.7.1.1 What this tool does

`stb-workfunction` computes a material's **work function** — the minimum energy needed to remove an electron from the material and place it at rest just outside its surface, in vacuum — straight from a finished SIESTA calculation, with no extra SIESTA run needed:

- Reads `<label>.VT` (the total electrostatic potential, on SIESTA's real-space grid) and planar-averages it along the direction normal to the surface (`--axis`).
- Reads the Fermi energy from `<label>.out` (or `--fermi` to override).
- Automatically finds the flat **vacuum plateau(s)** in that averaged profile and computes `Φ = E_vacuum - E_F` for each one.
- Handles an **asymmetric slab** (different terminations on each side) correctly: two genuinely different vacuum levels are reported separately, not averaged into one physically meaningless number.
- Auto-detects the vacuum-padded axis from the structure's own geometry (`<label>.XV`/`.fdf`) when available.

8.7.1.2 Why this matters (a bit of theory)

The work function itself. The work function Φ is one of the most basic, experimentally measurable properties of a surface — it sets the energy barrier for electron emission (thermionic/photoemission), governs Schottky-barrier heights at metal-semiconductor junctions, and determines band alignment at heterointerfaces. In DFT, it's computed from two ingredients you already have after any slab calculation:

```
Φ = E_vacuum - E_F
```

`E_F` is the Fermi energy (the "top of the electron sea" inside the material); `E_vacuum` is the electrostatic potential energy an electron would have sitting at rest far outside the surface, in the vacuum region of the simulation cell.

Why this only makes sense for a slab, not a 3D bulk crystal. A genuinely 3D-periodic crystal has **no vacuum** anywhere in its cell — `E_vacuum` simply isn't defined. `stb-workfunction` needs a structure with a real vacuum gap along one axis (a slab, a 2D monolayer, a wire) so that axis's planar-averaged potential has a region *flat enough* to call "vacuum." This is exactly why `--axis` auto-detection reuses `core.kspace.detect_vacuum_axes` — the same vacuum-gap detector used by `stb-kgrid`/`stb-mlrelax` elsewhere in this suite — and why the tool warns explicitly if you point it at an axis that doesn't actually look vacuum-padded.

The planar average, and why it "flattens" in vacuum. `V(x,y,z)` on the raw 3D grid is full of atomic-scale wiggles (it peaks sharply near every nucleus). Averaging it over the two in-plane directions at each point along the surface normal — `V(z) = <V(x,y,z)>_xy` — washes out those in-plane wiggles while keeping the physically meaningful variation *along* the surface normal. Deep in the vacuum, far from any atom, there's nothing left to average over that varies at all — the profile goes flat, and that flat value **is** `E_vacuum`. `stb-workfunction` finds this flat region automatically (the lowest -gradient contiguous run of the averaged profile — see `find_vacuum_plateaus`), rather than asking you to eyeball a plot and pick a point.

A concrete distinction worth knowing: the reported "vacuum size" is the width of this genuinely-flat region only — not the whole geometric vacuum gap in your cell. Near the slab-vacuum interface the potential is still transitioning (not yet flat), so the flat plateau is always *narrower* than `cell length - slab thickness`. In the CrS example below, the real vacuum gap is about 20 Å (a 23.35 Å cell minus a ~3.1 Å-thick layer), but the genuinely flat plateau `stb-workfunction` finds and averages over is only ~3.4

Å wide — both numbers are correct, they just answer different questions ("how much empty space is there" vs. "how much of it is flat enough to trust as *the* vacuum level").

Asymmetric slabs and the surface dipole. If a slab's two surfaces are different (different terminations, an adsorbate on only one side, a deliberately asymmetric cut), the two vacuum regions genuinely sit at **different** potential levels — there are two distinct work functions, and averaging them would throw away real physics. The difference between the two vacuum levels, `dV`, is directly proportional to the slab's own surface dipole moment (`dV ~ 4π·μ/A`, in Gaussian-like units). `stb-workfunction` reports `dV` itself rather than converting it to an absolute dipole moment in `e·Å`, since that conversion needs a convention-specific prefactor (see e.g. Bengtsson, *Phys. Rev. B* **59**, 12301 (1999) for the full derivation).

A systematic slope = a missing dipole correction. A genuine vacuum plateau should be **flat**, not sloped. A slow, systematic ramp across an otherwise-flat vacuum region (as opposed to just noisy scatter) is the classic numerical symptom of an uncancelled periodic-image electric field on an asymmetric slab — SIESTA's own fix for this is `SlabDipoleCorrection`. `stb-workfunction` fits a line to each plateau and flags this automatically (`detect_plateau_slope`), without needing to parse whether the flag was actually set in your `.fdf` — it's detected from the physical symptom itself, so it works regardless of SIESTA version or how exactly that setting is logged.

8.7.1.3 Limitations

- **The chosen axis must genuinely have vacuum.** If none of the 3 axes has a real vacuum gap (a 3D bulk crystal), there's nothing to compute — the tool fails cleanly rather than returning a meaningless number (see the "bulk, no vacuum" case in `test/2-analysis/7-workfunction/test.sh`).
- **A "noisy" plateau is a warning, not an automatic correction.** If the detected vacuum region's standard deviation is above a small tolerance, the reported vacuum level/work function should be treated as approximate.
- **`dV` is not converted to an absolute dipole moment** (see above) — it needs a convention-specific prefactor this tool doesn't assume for you.
- **An atom projecting into the "vacuum" plateau may mean it isn't real vacuum** — e.g. an isolated adsorbate on one side of an otherwise symmetric slab. `stb-workfunction` flags this rather than silently trusting the plateau.
- **The geometry cross-check can't catch every mismatch.** Comparing the `.XV`/`.fdf` cell against the `.VT` grid's own cell catches a genuinely different calculation being paired by accident, but a same-cell, different-atomic-position mismatch isn't detectable from these files alone.

8.7.1.4 The report: console output, `--save-report`, `--save-gnuplot`, `--view`

Every run prints a numbered report to the console:

Section	Content
`[0] RUN METADATA`	label, Fermi/grid file, axis (requested or auto-detected), tolerances, output dir, active options
`[1] FERMI ENERGY`	value and its source (detected `.out` vs. `--fermi` override)
`[2] VACUUM AXIS DETECTION`	axis used, and the geometry/grid cross-check result
`[3] VACUUM PLATEAU DETECTION`	table of detected plateaus (position, mean potential, noise), with warnings (atoms projecting in, systematic slope)
`[4] WORK FUNCTION RESULTS`	vacuum size/slab thickness, Φ per plateau, average + `dV` if asymmetric
`[5] OUTPUT DATA & PLOTS`	whether `workfunction_data.dat`/`.gplot` were written (only with `--save-gnuplot`)

Section	Content
[6] REFERENCES	writes references.bib (SIESTA)
[7] SUMMARY & FILES	status and a recap of every file written

- **--save-report** — also persists the full numbered report to stb_workfunction_report.txt. Off by default.
- **--save-gnuplot** — also writes workfunction_data.dat (the planar-averaged potential profile, with the detected vacuum level(s)/ work function(s) noted in its header) and workfunction.gplot (an annotated gnuplot script — Fermi level and each vacuum level drawn as dashed reference lines, with the work function itself marked between them). Off by default — this tool used to write both **unconditionally** on every run; that's gone, they're opt-in now, same convention as --save-gnuplot on stb-dos/stb-bader/etc.
- **--view** — shows an interactive matplotlib preview of the same profile before exiting. Off by default (this **replaces** the old --no-plot flag, which showed a plot by default and only let you turn it *off* — inverted here to match every other tool in this suite: opt-in, not opt-out).

8.7.1.5 When you'd reach for it

- Right after any slab/surface SIESTA calculation, to get the work function without hand-plotting the .VT file yourself.
- Comparing work functions across a series of surfaces/terminations/ adsorbate coverages.
- Checking for a missing SlabDipoleCorrection on an asymmetric slab (the systematic-slope warning) before trusting any surface-related result from that calculation.

8.7.1.6 Two ways to run it

A — direct CLI:

```
stb-workfunction --label siesta --file calc.out
```

B — interactive stb-suite menu:

```
$ stb-suite
Select an option (0-7, or a tool code like 5.1.2): 2.7
```

Both paths call the exact same underlying tool and produce the exact same output — example_2.7.sh proves this directly at the end.

8.7.1.7 Files in this folder

- calc.out, siesta.VT, siesta.XV — a real, finished, converged SIESTA calculation (copied from test/2-analysis/7-workfunction/) on a CrS monolayer (SystemLabel siesta) — a genuinely 2D material fetched from the twodmatpedia OPTIMADE database via stb-fetch, with a large vacuum gap along c, making it a real, physically meaningful work-function case (not a synthetic slab). --file calc.out is needed because this run's own log file isn't named siesta.out.
- example_2.7.sh — the guided walkthrough (**not** an automated test — see test/2-analysis/7-workfunction/test.sh for that, which uses small synthetic grids instead, to exercise every code path — asymmetric slabs, sloped vacuum, mismatched geometry, etc. — without committing large binary fixtures).
- .gitignore — excludes output/ and other generated files.

8.7.1.8 Running the walkthrough

```
cd examples/2.7-stb-workfunction
./example_2.7.sh
```

It pauses between sections ([Press Enter to continue]) and always starts by wiping its own output/. Self-contained cases are generated:

Folder	What it shows
basic/	The full report on a real, converged DFT calculation: real Fermi energy, real work function
save-gnuplot/	--save-gnuplot: workfunction_data.dat + the annotated workfunction.gplot
full-report/	Default (no report file) vs. --save-report (stb_workfunction_report.txt), references.bib

8.7.1.9 Try it yourself

```
# Right after a slab calculation
stb-workfunction --label my_slab --save-report

# Also get the annotated gnuplot script for a publication-ready plot
stb-workfunction --label my_slab --save-gnuplot

# Force the Fermi energy if your .out file isn't the default <label>.out
stb-workfunction --label my_slab --file relax/my_slab.out
```

8.7.1.10 Flag reference

Flag	Meaning
-l/--label	SystemLabel used in SIESTA.
-f/--file	.out file to read the Fermi energy from (default <label>.out).
-g/--grid	Potential grid file (default <label>.VT).
-z/--axis	Axis normal to the surface, 0/1/2 (default: auto-detected).
--fermi	Manually force the Fermi energy (eV), instead of reading .out.
--asymmetric-tol	Vacuum levels within this many eV are treated as one plateau (default 0.05).
-o/--output-dir	Where references.bib (and the report/data+gnuplot, with their flags) land.
--save-report	Persist the full numbered report to stb_workfunction_report.txt.
--save-gnuplot	Also write workfunction_data.dat/workfunction.gplot.
--view	Show an interactive matplotlib preview of the potential profile.

8.7.1.11 What's next

stb-density (2.8) reads the same class of SIESTA real-space grid files (.RHO instead of .VT) for charge-density maps/clouds rather than a single planar-averaged profile; stb-bader (2.6) partitions the charge density into atomic basins instead of averaging the electrostatic potential. All three read a finished SCF calculation's own real-space output, no re-run needed.

8.7.2 Implementation details

Work function from the electrostatic potential

The work function of a surface is $\Phi = V_{\text{vac}} - E_F$, where V_{vac} is the electrostatic potential energy of an electron in the vacuum far from the slab. With the planar average

$$\bar{V}(z) = \frac{1}{A}\iint_A V(x, y, z)\, dx\, dy$$

of SIESTA's total (`.VT`) or Hartree (`.VH`) potential along the surface normal z, V_{vac} is the value of $\bar{V}$ on its flat plateau in the vacuum region. Both faces of an asymmetric slab, or of a slab with a net dipole, have different plateaus and therefore different work functions; without a dipole correction [22, 23] the periodic images impose an artificial field that tilts the plateau.

Inputs.. `-label` locates `<label>.VT` (`-g/-grid` selects another potential grid, e.g. a `.VH`), the geometry (`<label>.XV` or `.fdf`) and the log `<label>.out` (`-f/-file`), from which the Fermi energy is read unless `-fermi` gives it. The averaging axis is `-z/-axis`; by default it is the vacuum axis detected from the geometry, when that geometry matches the grid's cell, and z otherwise. A cell with no vacuum along the axis has no plateau, and the report says that no vacuum level could be found.

Finding the plateaus.. Instead of taking "the flattest points" of $\bar{V}$, which could mix the two faces or a flat spot inside the slab, the tool:

1. computes $|d\bar{V}/dz|$ with periodic central differences and marks as flat the 15 % of the grid points with the smallest gradient (plus points equal within the float32 noise of the grid);
2. bridges interruptions narrower than 1 Å whose two sides agree within 0.05 eV;
3. keeps only contiguous runs at least 2 Å wide, treating the axis as periodic so a vacuum region crossing the cell boundary is one plateau;
4. merges plateaus whose levels agree within the merging tolerance and checks that no atom lies inside a plateau.

Each plateau gets its mean, standard deviation (flagged as noisy above 0.05 eV) and a linear fit across its width (flagged as sloped above 0.05 eV of total variation, the signature of a missing dipole correction), and its own work function. Vacuum levels within `-asymmetric-tol` (default 0.05 eV) of each other are treated as one level; otherwise the slab is reported as asymmetric and both work functions are given. Slab thickness and vacuum size are reported from the plateau extents.

Practical advice.. Use `SaveTotalPotential true` (or `SaveElectrostaticPotential` for `.VH`), `Slab.DipoleCorrection` for any asymmetric slab, at least 10–15 Å of vacuum, and check convergence with slab thickness. `-save-gnuplot` writes $\bar{V}(z)$ with the plateaus marked; `-view` shows it.

8.7.3 Command-line reference

8.7.3.1 `stb-workfunction`

Calculates the work function from a SIESTA electrostatic potential grid. Reads `<label>.VT` (total potential, planar-averaged along `--axis`) plus the Fermi energy from `<label>.out` (or `--fermi` to override), and locates the vacuum plateau(s) in the averaged potential to compute Phi = E_vac - E_F. If `<label>.XV` or

<label>.fdf is present (and its cell matches <label>.VT's), --axis is auto-detected from the structure's actual vacuum-padded direction (core.kspace.detect_vacuum_axes, same mechanism as stb-kgrid/stb-mlrelax); otherwise it defaults to z (2). An asymmetric slab (different terminations on each side) genuinely has two different vacuum levels – this is detected automatically and reported as two separate work functions rather than one physically meaningless average, along with the vacuum-level difference (proportional to the surface dipole moment). Also reports the slab thickness/vacuum size along the axis (when geometry is available), and warns if a "vacuum" plateau has a systematic slope (a common symptom of a missing dipole correction) rather than being genuinely flat.

Synopsis.

```
usage: stb-workfunction [-h] -l LABEL [-f FILE] [-g GRID] [-z {0,1,2}] [--fermi FERMI]
                        [--asymmetric-tol EV] [-o OUTPUT_DIR] [--save-report]
                        [--save-gnuplot] [--view] [--no-intro] [-v]
```

Options

`-l, --label LABEL`
System Label (required). required

`-f, --file FILE`
Output file (.out). Default: label.out

`-g, --grid GRID`
Potential grid file (.VT recommended). Default: label.VT

`-z, --axis {0,1,2}`
Axis normal to surface (0=x, 1=y, 2=z). Default: auto-detected from label.XV/label.fdf if available and matching label.VT's cell, else z (2) choices: 0, 1, 2; type: int

`--fermi FERMI`
Manually force Fermi Energy (eV). type: float

`--asymmetric-tol EV`
Vacuum levels within this many eV of each other are treated as the same physical plateau rather than a genuinely asymmetric slab (default: 0.05) type: float

`-o, --output-dir OUTPUT_DIR`
Directory to write references.bib into (and the report/data+gnuplot files, with --save-report/--save-gnuplot) (default: current directory). Created if it doesn't exist.

`--save-report`
Also persist the full run report to stb_workfunction_report.txt. Off by default. switch, off by default

`--save-gnuplot`
Also write workfunction_data.dat and workfunction.gplot (the planar-averaged potential profile, annotated with the detected vacuum level(s)/work function(s)). Off by default. switch, off by default

`--view`
Show an interactive matplotlib preview of the potential profile before exiting. Off by default. switch, off by default

Like every STB command it also accepts -h/-help, -v/-version and -no-intro (Section 3.3).

From the built-in help.

```
Example usage:
```

```
stb-workfunction -l graphene
stb-workfunction -l slab --axis 2
stb-workfunction -l slab --fermi -4.5 --save-report --save-gnuplot
```

8.8 Density Plotter (RHO)

At a glance

Menu code: 2.8 **Command:** `stb-density`
Guided example: `examples/2.8-stb-density/`
Summary: Export 2D Charge Density Maps or 3D Clouds.

8.8.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/2.8-stb-density/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

8.8.1.1 What this tool does

`stb-density` converts a SIESTA charge-density grid (`.RHO`) into 2D slices, 3D point clouds, or a planar-averaged 1D profile — with an interactive matplotlib preview and/or gnuplot data+script export:

- **2D slice** (default) — a planar cut through the density, at a given position along a chosen axis.
- **3D point cloud** (`--3d`) — the full volumetric grid as an (X, Y, Z, value) point cloud, optionally filtered by `--iso-min`.
- **1D profile** (`--profile`) — the planar average along one axis, useful for slabs/interfaces/superlattices.
- If the `.RHO` is **spin-polarized**, the net spin (magnetization) density is now **detected and processed automatically**, alongside the total charge — no flag needed to notice it.
- `--rho2` subtracts a second `.RHO` (charge transfer / bonding analysis).
- `--cube` additionally writes a Gaussian `.cube` file for real 3D isosurface rendering in VESTA/VMD/Avogadro.

8.8.1.2 Why this matters (a bit of theory)

The charge density itself. $\rho(r)$ is one of DFT's most fundamental outputs — literally the electron probability density that determines every ground-state property, per the Hohenberg-Kohn theorems. Slicing/averaging/clouding it is the most direct possible way to *see* a calculation's result: bonding character (covalent bridges of density between atoms vs. ionic, spherical-atom-like density), where charge accumulates or depletes upon adsorption/defect formation (`--rho2`), or how density decays into a slab's vacuum region (`--profile`).

Spin-polarized calculations: a real, verified pitfall. For a **collinear spin-polarized** SIESTA calculation, `.RHO` stores **two raw components** — but critically, they are the **up-spin** and **down-spin** densities directly, *not* "total" and "spin" as separately meaningful quantities. Reading component 0 alone is only half the electrons; reading component 1 alone is only the *other* half — neither is the total charge, and neither is the net magnetization on its own. The physically meaningful quantities are:

```
Total charge density   = up + down
Net spin (moment) density = up - down
```

This was verified against a real calculation, not assumed: a collinear spin-polarized SIESTA run on an O_2 molecule (`o2.RHO` in this folder — a textbook case, since O_2's real ground state is a **triplet**, 2 unpaired electrons, SIESTA's own log reports `|S| = 2.0`). Naively reading raw component 0 as "charge" integrates to **7.0 e** (the up channel alone) instead of the correct **12.0 e** (2 oxygen atoms × 6 valence electrons each); reading raw component 1 as "spin" integrates to **5.0 e** (the down channel alone) instead of the correct **2.0 e** net moment — which *does* match SIESTA's own reported value exactly. An earlier version of this tool made exactly this mistake. It's fixed now by using sisl's own combination convention (`index='total'` for `up+down`, `index='z'` for `up-down`) — the walkthrough below reproduces both the wrong and the corrected numbers directly, so you can see the difference.

Planar averaging (`--profile`). Averaging ρ`(x,y,z)` over the two in-plane directions at each point along one axis washes out atomic-scale in-plane wiggles while keeping the physically meaningful variation *along* that axis — e.g. how charge density decays from a slab's surface out into vacuum. Same technique `stb-workfunction` (2.7) uses for the electrostatic potential.

Colorbar convention. A **non-negative** quantity (total charge) is anchored at `cbrange [0:max]` — white always means "no charge" on a consistent baseline. A **signed** quantity (net spin density, or a `--rho2` difference) instead gets a range *symmetric around zero*, so the diverging blue-white-red palette's white midpoint actually lands on zero — otherwise autorange would center white at `(min+max)/2`, which is only 0 by coincidence.

8.8.1.3 Limitations

- **Non-collinear / spin-orbit calculations aren't handled** — those have a genuinely different 4-component spin texture (charge, m_x, m_γ, m_z), not the simple 2-component up/down split this tool (and the `total`/`z` combination) assumes. `stb-cube`'s own `detect_spin_configuration` (reading `.HSX`) is the authoritative way to tell collinear-polarized apart from non-collinear/spin-orbit, if you need to check first.
- **A skewed cut plane** (common for hexagonal cells) can render visibly distorted in both the gnuplot `pm3d map` and the matplotlib `imshow` preview, even though the underlying data file's coordinates are correct — flagged with a warning, not auto-corrected.
- **`--vmin`/`--vmax` apply only to the primary (charge, or spin if `--spin`) quantity** — an auto-detected spin section always uses its own zero-symmetric range, since a manually-fixed range for one quantity rarely makes sense for the other.
- **`--cube` applies to one quantity at a time** (charge, or spin if `--spin` was given) — not both automatically in the same run.
- **A large `--3d` matplotlib preview can be slow/cluttered** — `--iso-min` is the same filter used for the exported point cloud; a tighter value gives a clearer preview too.

8.8.1.4 The report: console output, `--save-report`, `--save-gnuplot`, `--view`

Every run prints a numbered report to the console:

Section	Content
`[0] RUN METADATA`	label, mode, axis, spin-polarization status, output dir, active options
`[1] CHARGE DENSITY`	integrated total charge (sanity check), mode-specific details, files written

Section	Content
`[2] SPIN DENSITY`	(conditional — auto-detected, or `--spin`) integrated net moment, mode-specific details, files written
`[3] CUBE FILE`	(conditional, `--cube`)
`[4] REFERENCES`	writes `references.bib` (SIESTA)
`[5] SUMMARY & FILES`	status and a recap of every file written

- **`--save-report`** — persist the full report to `stb_density_report.txt`.
- **`--save-gnuplot`** — also write a `.gplot` script next to each `.dat` file (off by default — this tool used to write the `.gplot` script **unconditionally** on every run; that's no longer the case, matching `--save-gnuplot` elsewhere in this suite).
- **`--view`** — show an interactive matplotlib preview of each quantity (heatmap for a slice, line plot for a profile, 3D scatter for a point cloud) — the same blue-white-red / white-yellow-red palette convention as the gnuplot script, so the two views read as the same plot.

8.8.1.5 When you'd reach for it

- A quick visual sanity check on a finished SCF calculation's charge distribution.
- Visualizing charge transfer upon adsorption/defect formation (`--rho2`).
- Checking a slab's charge-density decay into vacuum (`--profile`).
- Seeing where a magnetic moment actually sits, spatially, for a spin-polarized system (the auto-detected spin section).
- A `.cube` file for a proper 3D isosurface in VESTA/VMD/Avogadro.

8.8.1.6 Two ways to run it

A — direct CLI:

```
stb-density --label Sn3O4
```

B — interactive `stb-suite` menu:

```
$ stb-suite
Select an option (0-7, or a tool code like 5.1.2): 2.8
```

Both paths call the exact same underlying tool and produce the exact same output — `example_2.8.sh` proves this directly at the end.

8.8.1.7 Files in this folder

- `Sn3O4.RHO` — a real, non-spin-polarized SIESTA calculation (copied from `test/2-analysis/8-density/`), a 14-atom Sn_3O_4 structure.
- `o2.RHO/o2.XV/o2.fdf` — a real, **spin-polarized** SIESTA calculation on an isolated O_2 molecule (copied from `test/3-utils/3-cube/`) — O_2's real triplet ground state, the exact fixture used above to verify the up/down-vs-total/spin fix.
- `example_2.8.sh` — the guided walkthrough (**not** an automated test — see `test/2-analysis/8-density/test.sh` for that).
- `.gitignore` — excludes generated `.dat`/`.gplot`/`.cube` files, `references.bib`, the report, and `output/`.

8.8.1.8 Running the walkthrough

```
cd examples/2.8-stb-density
./example_2.8.sh
```

It pauses between sections (`[Press Enter to continue]`) and always starts by wiping its own `output/`. Self-contained cases are generated:

Folder	What it shows
`basic-slice/`	A 2D slice of Sn_3O_4's total charge density
`spin-auto/`	The O_2 spin-polarization auto-detection: correct total (12 e) and net moment (2 e)
`rho2-diff/`	`--rho2`: an (admittedly trivial, self-subtracted) charge-transfer difference, symmetric colorbar
`full-report/`	Default (no report/gnuplot files) vs. `--save-report --save-gnuplot`, `references.bib`

8.8.1.9 Try it yourself

```
# A quick look at a finished calculation
stb-density --label my_calc --save-gnuplot --view

# Charge transfer upon adsorption
stb-density --label adsorbed --rho2 isolated.RHO --view

# A slab's charge decay into vacuum
stb-density --label slab --profile --axis 2 --view

# Real 3D isosurfaces in VESTA/VMD
stb-density --label my_calc --cube
```

8.8.1.10 Flag reference

Flag	Meaning
`-l/--label`	SystemLabel (looks for `<label>.RHO`).
`-o/--output-dir`	Where all generated files (and `references.bib`) land.
`--3d/--profile`	3D point cloud / 1D planar-averaged profile (default: 2D slice).
`-a/--axis`	Axis normal to the cut plane, or the profile axis (0/1/2).
`-p/--pos`	Slice position in Å (default: cell center).
`--spin`	Process ONLY the net spin density, skipping charge.
`--rho2`	A second `.RHO` to subtract (charge/spin transfer).
`--iso-min`	For `--3d`: filter out points below this `|density|`.
`--cube`	Also write a Gaussian `.cube` file.
`--vmin/--vmax`	Fix the colorbar range manually (primary quantity only).
`--contour`	Overlay contour lines (2D slice only).
`--save-report`	Persist the full report to `stb_density_report.txt`.
`--save-gnuplot`	Also write `.gplot` script(s).
`--view`	Show an interactive matplotlib preview.

8.8.1.11 What's next

`stb-workfunction` (2.7) reads the same class of SIESTA real-space grid file (`.VT` instead of `.RHO`) for a planar-averaged electrostatic potential; `stb-bader` (2.6) partitions the charge density into atomic basins instead of slicing/averaging it directly.

8.8.2 Implementation details

Reading.. `<label>.RHO` is read with sisl [10]; for a spin-polarized run the file holds one grid per spin, and the tool always forms the total density $\rho_\uparrow + \rho_\downarrow$ (never just the first channel – the guide shows the pitfall) and, when present, the spin density $\rho_\uparrow - \rho_\downarrow$, reported automatically. `-spin` restricts the output to the spin density. `-rho2 FILE` subtracts a second density on the same grid, $\Delta\rho = \rho_1 - \rho_2$; for a proper charge-density difference between a combined system and its fragments, use the dedicated workflow **5.16**, which also guarantees identical grids and basis sets.

Outputs..

2D slice (default)
: the plane perpendicular to `-axis` (0, 1, 2 for a, b, c; default 2) at `-pos` (Å; default the middle of the cell), with colour limits `-vmin`/`-vmax` (default symmetric around zero for difference and spin densities) and optional `-contour` lines;

`-profile`
: the planar average $\bar{\rho}(z)$ along `-axis`;

`-3d`
: a point cloud of the voxels above `-iso-min`, for gnuplot;

`-cube`
: a Gaussian cube file for VESTA [18] or VMD.

With `-save-gnuplot` each output is written as `.dat+.gplot`; `-view` shows a matplotlib preview. The report gives the grid shape and spacing, the integrated charge (number of valence electrons, a sanity check of the units) and, for spin densities, the integrated moment.

Units.. sisl returns the density in electrons per Å^3; integrals use the voxel volume of the cell. SIESTA's `.RHO` is the valence pseudo-density, without core electrons.

8.8.3 Command-line reference

8.8.3.1 `stb-density`

Converts a SIESTA charge-density grid (.RHO) to 2D maps, 3D point clouds, or planar-averaged 1D profiles. Plots the total charge density by default. If the .RHO file is spin-polarized, the spin density (up-down) is now detected and reported automatically alongside it – `--spin` no longer needs to be remembered just to notice a magnetic calculation; pass `--spin` to look at ONLY the spin density (skip the charge section) instead. `--rho2` subtracts a second .RHO file's density from the first (Delta rho = rho1 - rho2), the standard way to visualize charge transfer/bonding (e.g. adsorbate+substrate vs. isolated fragments) – both files must be on the same grid. `--iso-min` filters low -|density| points out of a –3d export, since an unfiltered production-size grid can produce a multi-GB file dominated by near-zero vacuum/interstitial points. `--profile` writes the planar average along the chosen axis instead of a map (useful for slabs/interfaces). `--cube` additionally writes a standard Gaussian cube file (needs a `<label>.XV` or `<label>.fdf` for the geometry) for real isosurface rendering in VESTA/VMD/Avogadro. `--vmin`/`--vmax` fix the colorbar range manually for the primary quantity, e.g. to compare several slices on the same scale.

Synopsis.

```
usage: stb-density [-h] -l LABEL [-o OUTPUT_DIR] [--3d | --profile] [-a {0,1,2}]
                   [-p POS] [--spin] [--rho2 FILE] [--iso-min RHO] [--cube]
                   [--vmin VMIN] [--vmax VMAX] [--contour] [--save-report]
                   [--save-gnuplot] [--view] [--no-intro] [-v]
```

Options

`-l, --label LABEL`

System Label (looks for .RHO file) required

`-o, --output-dir OUTPUT_DIR`

Directory to write the data/gnuplot/cube/report files and `references.bib` into (default: current directory). Created if it doesn't exist.

`--3d`

Export full 3D volume switch, off by default; exclusive group 1

`--profile`

Write the planar average of the density along `--axis` as a 1D profile (average taken over the other two axes) instead of a 2D/3D map – useful for slabs, interfaces, and superlattices switch, off by default; exclusive group 1

`-a, --axis {0,1,2}`

For a slice/profile: the axis normal to the cut plane (slice) or the axis the profile varies along (profile) (0=X, 1=Y, 2=Z) default: 2; choices: 0, 1, 2; type: int

`-p, --pos POS`

Position (Angstrom) of the 2D cut (ignored in `--profile` mode) type: float

`--spin`

Process ONLY the spin density (up-down), skipping the charge density section entirely – only available for spin-polarized .RHO files. Without this flag, spin density is now detected and reported automatically alongside the charge density whenever the .RHO file has it, no flag needed. switch, off by default

`--rho2 FILE`

A second .RHO file to subtract from the first (Delta rho = rho1 - rho2), e.g. to visualize charge transfer upon adsorption or defect formation. Must be on the same grid (same shape) as `<label>.RHO`. If both files are spin-polarized, the spin difference is computed too.

`--iso-min RHO`

For –3d: only export/preview points with |density| >= this threshold (e/Ang^3) – keeps the point cloud a manageable size and visually meaningful instead of dominated by near-zero points type: float

`--cube`

Also write the primary quantity's full 3D grid as a Gaussian .cube file (needs `<label>.XV` or `<label>.fdf` for the geometry) – for real 3D isosurface rendering in VESTA/VMD/Avogadro, which handle volumetric data better than a point cloud. Applies to the charge density, or the spin density if `--spin` was given – not both at once. switch, off by default

`--vmin VMIN`

Fix the colorbar/palette lower bound manually (e/Ang^3) for the primary quantity, e.g. to compare several slices on the same scale. Auto-detected spin density always uses its own zero-symmetric range regardless of this. type: float

`--vmax VMAX`

Fix the colorbar/palette upper bound manually (e/Ang^3), for the primary quantity (see `--vmin`). type: float

`--contour`

Overlay contour lines on a 2D slice map/preview (ignored for –3d/–profile) switch, off by default

`--save-report`
: Also persist the full run report to `stb_density_report.txt`. Off by default. switch, off by default

`--save-gnuplot`
: Also write a .gplot script next to each .dat file this run generates. Off by default. switch, off by default

`--view`
: Show an interactive matplotlib preview of each quantity plotted this run (slice heatmap / profile line / 3D scatter, matching the mode). Off by default. switch, off by default

Like every STB command it also accepts `-h`/`--help`, `-v`/`--version` and `--no-intro` (Section 3.3).

From the built-in help.

```
Example usage:
  stb-density --label siesta
  stb-density --label siesta --3d --iso-min 0.01
  stb-density --label siesta --spin
  stb-density --label adsorbed --rho2 isolated.RHO
  stb-density --label siesta --profile --axis 2
  stb-density --label siesta --cube --view
```

8.9 Powder XRD Simulator

At a glance

Menu code: 2.9 **Command:** `stb-xrd`
Guided example: `examples/2.9-stb-xrd/`
Summary: Simulate a powder XRD pattern (peak table + optional plot) from a structure, optionally compared against an experimental pattern.

8.9.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/2.9-stb-xrd/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

8.9.1.1 What this tool does

`stb-xrd` simulates a powder X-ray diffraction pattern (peak list + optional plot) from a structure, using pyxtal's diffraction engine. It reports:

- **Peak positions and intensities** — every reflection (`hkl`) that falls inside a scanned 2-theta range, sorted by intensity — saved to `xrd_pattern.dat` (`--save-gnuplot`), not printed to the console/report.
- **Structure/symmetry info** — reduced formula, space group, crystal system, point group, lattice parameters, cell volume, density.
- Optionally, a **similarity score** against an experimental pattern (`--compare-to`), and an interactive matplotlib preview (`--view`).

8.9.1.2 Why this matters (a bit of theory)

Bragg's law. A crystal is a periodic stack of atomic planes. X-rays reflecting off successive parallel planes interfere constructively only at specific angles — Bragg's law:

```
n * lambda = 2 * d_hkl * sin(theta)
```

where `lambda` is the X-ray wavelength, `d_hkl` is the spacing between `(hkl)` planes, and `theta` is half the scattering angle (`2*theta` is what's actually measured). A powder sample contains crystallites in every possible orientation, so every set of planes that satisfies Bragg's law for *some* orientation contributes a peak at its own `2*theta` — the "powder pattern" is the full peak table as a function of angle.

The structure factor and systematic absences. Not every geometrically allowed reflection actually shows up: the **structure factor** `F_hkl` (a sum over every atom's scattering contribution, weighted by a phase factor depending on its position) sets each peak's intensity, and a crystal's symmetry can force `F_hkl = 0` *exactly* for certain `(hkl)` combinations — "systematic absences". This is why a peak table is a fingerprint of a structure's symmetry, not just its cell size: two structures with the same lattice parameters but different space groups can have visibly different missing/present peaks.

Wavelength choice. Different X-ray sources (Cu-Kalpha, Mo-Kalpha, ...) have different wavelengths, which rescales the whole pattern via Bragg's law — a shorter wavelength (e.g. Mo-Kalpha, ~0.71 Ang) reaches higher resolution (smaller `d_hkl`) within the same 2-theta range than a longer one (e.g. Cu-Kalpha, ~1.54 Ang). `--wavelength` accepts a named source or a plain number in Ang.

The similarity score (`--compare-to`). Given an experimental pattern (2-theta, intensity pairs), `--compare-to` interpolates both the simulated and experimental patterns onto a common grid and reports pyxtal's cosine-weighted similarity (0-1, higher is more similar) — a quick, single-number check of "does this candidate structure plausibly match the measured pattern", useful e.g. for narrowing down candidate phases before a full Rietveld refinement.

Required column format for the `--compare-to` file: 2-theta (degrees) in the **first** column, intensity in the **last** column — whitespace- or comma-separated, blank lines and `#` comments skipped. Any columns in between are ignored, so this accepts both a plain 2-column file AND stb-xrd's own `--save-gnuplot` output (`xrd_pattern.dat`, 6 columns: `2theta d h k l intensity`) directly, without needing to strip the `d`/`h`/`k`/`l` columns out yourself first.

Peak lists are auto-broadened before comparing. A real, continuously -scanned diffractogram (many closely and roughly evenly spaced points) is compared as-is. But a *discrete peak list* — e.g. a literature-reported indexed peak table, or stb-xrd's own stick-pattern `xrd_pattern.dat` — has no peak width at all, while the simulated pattern `--compare-to` compares it against is always a Gaussian-broadened continuous profile (FWHM=0.1 deg). Comparing the two representations directly is apples-to-oranges: pyxtal's cubic interpolation draws a spurious curve through the empty gaps between sparse peaks, which does NOT match the broadened profile's true near-zero baseline there — even for the exact same structure. stb-xrd detects this (a peak list has a highly uneven spacing between consecutive 2-theta points — measured via the coefficient of variation of that spacing — vs. a real scan's near-constant step) and automatically Gaussian-broadens a detected peak list the same way before computing the similarity. `--raw-experimental` disables this and compares the file exactly as read, for the rare case you want that instead.

Real bug found and fixed (verified live): feeding stb-xrd's own `xrd_pattern.dat` for a structure back into `--compare-to` **for that same structure** used to score as low as 0.32-0.46 similarity instead of ~1.0, from two compounding issues: (1) `--compare-to` used to always read intensity from column index 1,

which in a 6-column `xrd_pattern.dat` is `d` (the d-spacing in Ang), not intensity; and (2) even after fixing that, comparing the raw, un-broadened stick pattern against the broadened simulated profile still only scored ~0.32, for the reason explained above. Fixed by reading intensity from the last column always, and by auto-broadening peak-list-shaped input before comparing — verified live that the exact same self-comparison now scores 1.0000, that a genuinely different structure's pattern still scores well below 1.0 (so the fix doesn't just make everything score 1.0), and that a genuine dense/uniform continuous scan is correctly left un-broadened.

8.9.1.3 Limitations

- **This is an ideal, simulated powder pattern** — no peak broadening, no instrumental profile, no preferred orientation, no temperature (Debye- Waller) factors. Good for phase identification/comparing candidate structures, not for Rietveld-quality quantitative refinement.
- **The similarity score needs >= 4 experimental points** (pyxtal's cubic interpolation requirement) — fewer raises a clear error instead of a cryptic one from deep inside pyxtal.
- **A vacuum-padded structure (a slab/2D monolayer) is still simulated as fully 3D-periodic** — this tool has no concept of "this axis is just vacuum". A huge artificial lattice parameter along the vacuum direction produces spurious LOW-ANGLE `(001)` peaks with very short real-space periodicity that don't correspond to anything a real bulk powder sample would show (there's no actual repeating unit at that spacing — it's the simulation cell's vacuum gap, not a physical lattice plane). The walkthrough below hits this directly: CrS's own real geometry has a huge vacuum gap along `c`, and the strongest simulated peak is exactly this kind of `(001)` low-angle artifact. Worth knowing before reading too much into a low-angle peak from a slab/monolayer structure.

8.9.1.4 The report: console output, `--save-report`, `--save-gnuplot`, `--view`

Every run prints a numbered report to the console:

Section	Content
`[0] RUN METADATA`	input file/format, wavelength, 2-theta range, `--top` setting, output dir, active options
`[1] STRUCTURE`	formula, site count, space group/crystal system/point group/Hall symbol/layer group, lattice parameters, cell volume, density
`[2] DIFFRACTION PATTERN`	peak count, strongest peak, resolution (min d-spacing) — a compact summary only, no peak-by-peak table
`[3] EXPERIMENTAL COMPARISON`	(conditional — `--compare-to`) file used, point count, whether a peak list was detected/auto-broadened, similarity score
`[4] OUTPUT DATA & PLOTS`	whether `xrd_pattern.dat/.gplot` were written
`[5] REFERENCES`	writes `references.bib` (SIESTA + pyxtal)
`[6] SUMMARY & FILES`	status and a recap of every file written

The full peak-by-peak table is not printed anywhere (console or `--save-report`'s file) — it used to be, in full, which could run to hundreds of lines for a low-symmetry structure over a wide 2-theta range. It only ever lives in `xrd_pattern.dat`.

- **`--save-report`** — persist the (table-free) report to `stb_xrd_report.txt`.
- **`--save-gnuplot`** — write `xrd_pattern.dat` (the actual peak list — 2theta, d, h, k, l, intensity — with a complete, human-readable header: structure/formula/space group/wavelength/range, not just pyxtal's own terse one-liner) + `xrd_pattern.gplot` (a stick-pattern plot script) together — off by default (this tool used to write the data file **unconditionally** on every run; that's no longer the

case). `--top` controls how many peaks land in this file (it used to just trim the console table, which no longer exists).
- **`--view`** — show an interactive matplotlib preview of the simulated stick pattern (or, with `--compare-to`, an overlay of both patterns). Renamed from the old `--plot`, same off-by-default behavior — but fixed: the old code delegated to a non-blocking call (`fig.show()`, or pyxtal's own `plot_pxrd()`, which was checked directly and also only ever calls the non-blocking `fig.show()` internally), so the window would disappear as soon as the script exited. `--view` now builds its own plot and calls the blocking `plt.show()`, the same convention already used correctly by `stb-workfunction`/`stb-density`'s own `--view`.

8.9.1.5 When you'd reach for it

- A quick, cheap check of what a candidate structure's powder pattern should look like, before running a real diffraction experiment.
- Narrowing down candidate phases against a measured pattern (`--compare-to`).
- Sanity-checking a structure's symmetry: does the saved peak list show the systematic absences you'd expect from its space group?

8.9.1.6 Two ways to run it

A — direct CLI:

```
stb-xrd --file structure.fdf --format fdf
```

B — interactive `stb-suite` menu:

```
$ stb-suite
Select an option (0-7, or a tool code like 5.1.2): 2.9
```

Both paths call the exact same underlying tool and produce the exact same output — `example_2.9.sh` proves this directly at the end.

8.9.1.7 Files in this folder

- `siesta.XV` — a real, converged SIESTA calculation's geometry (copied from `examples/2.7-stb-workfunction/`) on a CrS 2D monolayer (fetched from the twodmatpedia OPTIMADE database via `stb-fetch`).
- `mock_experimental.dat` — a **synthetic** "experimental" pattern (small angle jitter + intensity noise added to this structure's own simulated peaks), used to exercise `--compare-to` — not independent lab data.
- `example_2.9.sh` — the guided walkthrough (**not** an automated test — see `test/2-analysis/9-xrd/test.sh` for that). Converts `siesta.XV` to `crs_structure.fdf` itself, up front (see the note below).
- `.gitignore` — excludes `crs_structure.fdf` (regenerated by the script), generated `.dat`/`.gplot`/`.pdf` files, `references.bib`, the report, and `output/`.

Why the script converts `siesta.XV` via `sisl` directly, not `stb-translate`: `stb-xrd` only accepts `.fdf`/`.STRUCT_OUT` structure files. While preparing this example, `stb-translate`'s own `siesta` input reader (`getatomsandvectors_siesta` in `translate.py`) was found to mis-handle this real file — it treats the `.XV`'s Cartesian (Bohr) positions as fractional coordinates without converting either the positions or the cell

from Bohr to Angstrom, which then fails downstream with a cryptic ASE cell-shape error. This is a separate, pre-existing bug in `translate.py`, out of scope for `stb-xrd`'s own migration — reported rather than silently worked around. `sisl` itself reads this exact file correctly (confirmed directly, and it's the same mechanism `stb-workfunction`'s own vacuum-axis detection already uses successfully on it), so the example script uses `sisl` + `core.structure_io.write_fdf` directly instead.

8.9.1.8 Running the walkthrough

```
cd examples/2.9-stb-xrd
./example_2.9.sh
```

It pauses between sections (`[Press Enter to continue]`) and always starts by wiping its own `output/`. Self-contained cases are generated:

Folder	What it shows
`basic/`	A real structure's (CrS) space group/lattice info and pattern summary
`save-gnuplot/`	`--save-gnuplot`: the stick-pattern `.dat`/`.gplot` pair, only with the flag
`compare/`	`--compare-to` a synthetic mock-experimental pattern, similarity score
`self-check/`	`--compare-to` the structure's OWN `xrd_pattern.dat` — the fixed column/peak-list-broadening bug, and `--raw-experimental` to see the old (wrong) score
`full-report/`	Default (no report/gnuplot files) vs. `--save-report`, `references.bib`

8.9.1.9 Try it yourself

```
# A quick look at a candidate structure's pattern
stb-xrd --file structure.fdf --format fdf --save-gnuplot --view

# A different X-ray source, narrower range, top peaks only
stb-xrd --file structure.fdf --format fdf --wavelength MoKa --two-theta-range 10 80 --top 10

# Compare against a measured pattern
stb-xrd --file structure.fdf --format fdf --compare-to experimental.dat --view
```

8.9.1.10 Flag reference

Flag	Meaning
`--file`/`--format`	Structure file and its format (`fdf` or `struct_out`).
`--wavelength`	X-ray source name or a wavelength in Ang (default: CuKa).
`--two-theta-range`	2-theta range to scan, in degrees (default: 0-90).
`--top`	Only include the N strongest peaks in the saved data file (`--save-gnuplot`).
`--compare-to`	Experimental pattern file (2-theta first column, intensity last column); prints a similarity score.
`--raw-experimental`	With `--compare-to`, never auto-broaden a peak-list-looking experimental file — compare it exactly as read.
`-o`/`--output-dir`	Where all generated files (and `references.bib`) land.
`--save-report`	Persist the full report to `stb_xrd_report.txt`.
`--save-gnuplot`	Also write `xrd_pattern.dat`/`xrd_pattern.gplot`.
`--view`	Show an interactive matplotlib preview.

8.9.1.11 What's next

`stb-xrdsearch`/`stb-xrdrank` (Workflow) use this same simulated-pattern machinery to guide a structure search against a real experimental pattern, rather than just reporting one structure's pattern in isolation.

8.9.2 Implementation details

The calculation.. The pattern is computed by PyXtal's diffraction engine [14] for every reflection hkl in the `--two-theta-range` (default 0–90°): Bragg angle from $\lambda = 2d_{hkl}\sin\theta$, structure factor $F_{hkl} = \sum_j f_j(\sin\theta/\lambda)\, e^{2\pi i(hx_j+ky_j+lz_j)}$ with tabulated atomic scattering factors, multiplicity, and the Lorentz–polarization factor of an unpolarized powder experiment. Intensities are normalized to the strongest peak; `--top` limits the table to the strongest peaks. The wavelength is a source name known to pymatgen (`CuKa`, default, 1.54184 Å; `MoKa`, …) or a number in Å. Thermal (Debye–Waller) factors and preferred orientation are not applied to the simulated pattern.

Comparing with an experiment.. `--compare-to FILE` reads a measured pattern (two columns, 2θ and intensity; a header and extra columns are handled) and computes PyXtal's similarity score, a cross-correlation of the two broadened profiles that tolerates small peak shifts; 1 means identical. Both sides must be continuous profiles: the simulated pattern is broadened with Gaussians of 0.1° FWHM on a 0.01° grid, and a file that looks like a list of peak positions (irregular 2θ spacing, detected from the spread of the spacings) is broadened the same way before scoring – comparing sticks with a profile would deflate the score. `--raw-experimental` disables that step. At least 4 experimental points are needed.

Using the score.. The similarity is a ranking tool: use it to compare candidate structures against the same measurement (as 5.6 does), not as an absolute measure of agreement. DFT lattice constants differ from room-temperature experiment by about a per cent, which shifts high-angle peaks; refine or scale the cell before concluding from small differences.

8.9.3 Command-line reference

8.9.3.1 `stb-xrd`

Simulates a powder XRD pattern from a structure. Uses pyxtal's own diffraction engine (not pymatgen's) so the same dependency already needed by `stb-crystalcast` covers this too.

Synopsis.

```
usage: stb-xrd [-h] --file FILE --format {fdf,struct_out} [--wavelength WAVELENGTH]
               [--two-theta-range MIN MAX] [--top TOP] [--compare-to PATH]
               [--raw-experimental] [-o OUTPUT_DIR] [--save-report] [--save-gnuplot]
               [--view] [-v] [--no-intro]
```

Options

`--file FILE`
Path to structure file. required

`--format {fdf,struct_out}`
Input file format:
fdf: SIESTA structure input (%block LatticeVectors etc.)
struct_out: SIESTA post-relaxation output (.STRUCT_OUT) required; choices: fdf, struct_out

`--wavelength WAVELENGTH`

X-ray source: a known name (CuKa, CuKa1, MoKa, CrKa, FeKa, CoKa, AgKa, ...) or a wavelength in Ang as a plain number (default: CuKa, 1.54184 Ang).

`--two-theta-range MIN MAX`

2-theta range in degrees to compute (default: 0 90). type: float

`--top TOP`

Only include the N strongest peaks in the saved data file (`--save-gnuplot`); default: include all of them. No effect without `--save-gnuplot` – there's no console/report peak table anymore, see [2] DIFFRACTION PATTERN for a summary instead. type: int

`--compare-to PATH`

Compare the simulated pattern against an experimental one: a text file with 2-theta in the first column and intensity in the LAST column (any columns in between, e.g. d/h/k/l, are ignored – so `stb-xrd`'s own `--save-gnuplot xrd_pattern.dat` is a valid input too), whitespace- or comma-separated, '#' comments and blank lines allowed. If the file looks like a sparse list of discrete peaks rather than a continuous scan (e.g. a literature peak table, or this tool's own stick-pattern output), it is automatically Gaussian-broadened before comparing – pass `--raw-experimental` to disable this and compare it unbroadened instead. Prints a similarity score (pyxtal's cosine-weighted metric, 0-1, higher is more similar); combine with `--view` for a visual overlay.

`--raw-experimental`

With `--compare-to`, never auto-broaden a peak-list-looking experimental file – compare it exactly as read. Off by default (auto-detection normally handles this correctly). switch, off by default

`-o, --output-dir OUTPUT_DIR`

Directory to write `references.bib` into (and the report/data+gnuplot files, with `--save-report`/`--save-gnuplot`) (default: current directory). Created if it doesn't exist.

`--save-report`

Also persist the full run report to `stb_xrd_report.txt`. Off by default. switch, off by default

`--save-gnuplot`

Also write `xrd_pattern.dat` (all peaks, pyxtal's own format) and `xrd_pattern.gplot` (a stick-pattern plot script) together. Off by default – this tool used to write the data file unconditionally on every run; that's no longer the case. switch, off by default

`--view`

Show an interactive matplotlib preview of the pattern (or, with `--compare-to`, an overlay of both patterns) before finishing. Off by default. switch, off by default

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Usage examples:
  stb-xrd --file structure.fdf --format fdf
  stb-xrd --file structure.fdf --format fdf --wavelength MoKa --top 10
  stb-xrd --file relaxed.STRUCT_OUT --format struct_out \
      --two-theta-range 10 80 --view
  stb-xrd --file structure.fdf --format fdf \
      --compare-to experimental.dat --view --save-report --save-gnuplot
```

8.10 Fatbands Analyzer

At a glance

Menu code: 2.10 **Command:** `stb-fatbands`
Guided example: `examples/2.10-stb-fatbands/`
Summary: Orbital-projected / character-colored band structure from .bands + .WFSX.

8.10.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/2.10-stb-fatbands/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

8.10.1.1 What this tool does

`stb-fatbands` combines a SIESTA `.bands` file (k-path eigenvalues) with a `.WFSX` file (the wavefunction coefficients for that same k-path) to color and size every `(k, band)` point on a band-structure plot by its **orbital character** — which atomic orbital(s), atom(s), or chemical species build up a given state. Optionally cross-checked/weighted with a `.HSX` file (Hamiltonian + overlap) for physically exact weights.

Five projection modes (`--projection`):

- `l` — angular momentum only (`s`, `p`, `d`, `f`)
- `ml` — full orbital detail (`s`, `px`, `py`, `pz`, `dxy`, ...)
- `atom` — per atom index
- `species` — per chemical species
- `species_l` (**default**) — species AND angular momentum combined (`Sn-s`, `Sn-p`, `O-s`, `O-p`, ...)

For a spin-polarized calculation, every category is automatically split into `<category>_up`/`<category>_down` — see the theory section below for why that matters.

8.10.1.2 Why this matters (a bit of theory)

LCAO wavefunctions: what a `.WFSX` file actually holds. SIESTA is a **linear-combination-of-atomic-orbitals (LCAO)** code: every Bloch state is expanded as

```
psi_n(k, r) = sum_mu  c_mu,n(k) * phi_mu(r)
```

where `phi_mu` are the localized, numerical atomic orbitals of the basis set (one or more per atom, depending on `PAO.BasisSize`: s/p/d/..., possibly several per angular momentum for a multiple-zeta basis like DZP), and `c_mu,n(k)` are complex expansion coefficients — one per `(orbital, band, k-point)` triple. **These coefficients are exactly what a `.WFSX` file stores.** A plain `.bands` file only has the eigenvalues (WHERE a state sits in energy); the `.WFSX` additionally has WHAT that state is made of.

The projection weight: a Mulliken-like population analysis. Because the numerical atomic orbitals overlap each other in space, the basis is **non-orthogonal**: `S_mu,nu = <phi_mu|phi_nu>` is not the identity matrix (unlike, e.g., a plane-wave basis). The physically correct weight of orbital `mu` in state `n` at k is not simply `|c_mu,n(k)|^2` but

```
w_mu,n(k) = Re[ c*_mu,n(k) * sum_nu S_mu,nu * c_nu,n(k) ]
```

— the same orbital-resolved decomposition of `<psi|S|psi>` a **Mulliken population analysis** uses (`stb-fatbands` gets this via sisl's own `norm2(projection="hadamard")`, applied to `psi` and the real SIESTA overlap matrix `S`). Summed over every orbital of a category (a species, an `l` shell, ...), it tells you what *fraction of that state's own norm* lives in that category — and, up to the `l > 3` exclusion noted below, summing across **every** category at a fixed `(n, k)` recovers the full norm (≈1): the categories are a complete partition of the state, not an arbitrary rescaling.

Computing `S` needs a real Hamiltonian object, read from `<label>.HSX` (written when `SaveHS T` is set) — **without it**, sisl falls back to an *implicit orthogonal basis* (`S = identity`), giving the cruder `|c_mu|^2` instead. Same order of magnitude, not exact — `stb-fatbands` always prints (and reports) which of the two it actually used, so you know exactly how much to trust the numbers.

Why `WFS.Write.For.Bands T` is required in the SIESTA calculation. This is the one setting this tool cannot work without. SIESTA computes the same diagonalization for a `.bands` k-path regardless, but it only *writes out* the wavefunction coefficients if explicitly told to — the file can be large (one complex number per orbital, per band, per k-point), so it is never saved by default. Concretely, in the `.fdf` (see `spin/Ospin.fdf` in this folder for the complete, minimal, working example):

```
WFS.Write.For.Bands T
BandLinesScale ReciprocalLatticeVectors
%block BandLines
  1   0.000  0.000  0.000  GAMMA
  5   0.500  0.000  0.000  X
%endblock BandLines
```

Add `SaveHS T` too, so `stb-fatbands` can use the accurate, overlap-aware weights instead of falling back to the approximation above. Without `WFS.Write.For.Bands T`, SIESTA still writes `<label>.bands` (the plain eigenvalues) but **no** `<label>.bands.WFSX` at all — `stb-fatbands` refuses to run outright in that case (a clear `No .WFSX file found` error) rather than silently producing nothing or a wrong plot. If you already have a finished calculation without this flag, you normally don't need a fresh SCF cycle: add the flag, set `DM.UseSaveDM T`, and re-run — SIESTA restarts from the converged density matrix and only needs to redo the band-path diagonalization.

The five projection modes, and why `species_l` is the default.

- `l` and `ml` tell you *which orbital character* dominates a band, but not *which atom/species* it belongs to in a multi-species structure.
- `species` (and `atom`) tell you *which chemical element* (or specific site) dominates, but not *which orbital character* within it.
- `species_l` gives you both at once (`Sn-s`, `O-p`, ...) — the single most informative view for understanding chemical bonding character (e.g. "the valence band top is mostly O-p, the conduction band bottom is mostly Sn-s") without needing two separate runs. This is why it is the default as of v2.1.0 (it used to be plain `l`).
- `atom` is still useful on its own for comparing symmetry-inequivalent sites of the *same* species (e.g. surface vs. bulk atoms in a slab) — `species`/`species_l` cannot distinguish those, since both group by species/orbital, never by individual atom index.

Spin-polarized calculations: every category is split automatically. For a spin-polarized (`nspin=2`) calculation, the spin-up (majority) and spin-down (minority) channels can have genuinely different dispersion and orbital character — not just a rigid energy shift, since that difference *is* what magnetism

means at the electronic-structure level. Merging both channels into one category would silently combine two physically distinct band sets into one indistinguishable series. `stb-fatbands` splits every category into `<category>_up/<category>_down` for `nspin=2` (same `_up/_down` naming `stb-dos` already uses for spin-resolved PDOS) — demonstrated live in this walkthrough on a real, converged spin-polarized calculation (see `spin/` below), where verifying this exact behavior caught a genuine bug: the original implementation merged both spin channels into one series, and two band sets whose conduction-band minima differ by ~29 eV were being plotted (and averaged into the report's per-category weight statistics) as if they were one.

The band gap analysis "for free". Resolving `--shift vbm`/`--shift cbm` already needs the exact same VBM/CBM search `stb-bands` performs on the same `.bands` file — so `stb-fatbands` reports it too (`[2] BAND GAP ANALYSIS`, reusing `core/siesta_bands.py`'s `cbm_vbm`, spin-resolved for `nspin=2`), at no extra computational cost and guaranteed to agree with what `stb-bands` itself would report for the identical file.

The `.bands`/`.WFSX` correspondence check. There is no file metadata tying a specific `.WFSX` to a specific `.bands` file — nothing stops you from accidentally pointing `--wfsx` at the wrong run. `stb-fatbands` guards this two ways: a hard k-point-count and orbital-count match, and an eigenvalue cross-check (`--k-tol`, default 0.001 eV) at three sampled k-points (first/middle/last) between what the `.WFSX` itself stored and what the `.bands` file reports. Always prefer `--label` (auto-pairs same-run files) over hand-picking `--file`/`--wfsx` from different calculations.

8.10.1.3 The report: console output, `--save-report`, `--save-gnuplot`, `--view`

Every run prints a numbered report, the same `[0]...[6]` style every newer tool in this suite uses:

Section	Content
`[0] RUN METADATA`	input files, shift mode, projection, category filter, output dir, active options
`[1] INPUT DATA`	Fermi energy, spin channels, bands x k-points, orbital count, weight accuracy (accurate/approximate), `.bands`/`.WFSX` correspondence check
`[2] BAND GAP ANALYSIS (k-path)`	VBM/CBM, direct/indirect gap, gap type — spin-resolved (+ half-metallic flag) for `nspin=2`
`[3] ORBITAL PROJECTION`	categories found/used, `1 > 3` exclusions (if any), spin-split note, and a per-category table (orbital count, mean/max weight)
`[4] WRITING OUTPUT FILES`	whether `fatbands_<category>.dat`/`.gplot` were written
`[5] REFERENCES`	writes `references.bib` (SIESTA)
`[6] SUMMARY & FILES`	status and a recap of every file written

- **`--save-report`** — persist the (text) report to `stb_fatbands_report.txt`. Off by default.
- **`--save-gnuplot`** — write `fatbands_<category>.dat` (one file per category — `k_position/energy(eV)/value` columns) + `fatbands.gplot` (all categories overlaid) together. Off by default — this tool used to write both **unconditionally** on every run; that's no longer the case.
- **`--view`** — show an interactive matplotlib preview (single-category runs get a colorbar-encoded scatter; multi-category runs get one discrete-colored series per category, shared legend). Off by default — this tool used to always show it, with no way to skip a blocking window. Marker sizes are deliberately smaller than `stb-ipr`/`stb-stm`'s own shared defaults (`core/band_scatter.py`): a fatbands plot scatters every `(k, band[, category])` point at once, several times denser than those tools' single-series plots.

8.10.1.4 When you'd reach for it

- Identifying which orbital/species/atom character dominates a valence or conduction band, e.g. before discussing chemical bonding or assigning a band's character in a paper.
- Distinguishing majority/minority spin channels visually in a magnetic material's band structure.
- Comparing symmetry-inequivalent atoms of the same species (`--projection atom`), e.g. surface vs. bulk sites in a slab.
- A quick sanity check that a `.WFSX` and `.bands` pair truly belong to the same calculation, before trusting a plot built from hand-picked files.

8.10.1.5 Two ways to run it

A — direct CLI:

```
stb-fatbands --file Sn3O4.bands --wfsx Sn3O4.bands.WFSX --hsx-file Sn3O4.HSX --shift fermi
```

B — interactive `stb-suite` menu:

```
$ stb-suite
Select an option (0-7, or a tool code like 5.1.2): 2.10
```

Both paths call the exact same underlying tool and produce the exact same output — `example_2.10.sh` proves this directly at the end. The menu additionally defaults `--shift` to Fermi level and `--projection` to `species_l` (just press Enter through both), plus separate `y/N` prompts for `--save-report`/`--save-gnuplot`/`--view`.

8.10.1.6 Files in this folder

- `Sn3O4.bands` / `Sn3O4.bands.WFSX` / `Sn3O4.HSX` / `calc.fdf` / `structure.fdf` / `Sn.ion(.xml)` / `O.ion(.xml)` — a real, converged bulk Sn3O4 calculation (copied from `test/2-analysis/10-fatbands/`, itself a deliberately SHORT band path — Gamma-X-Y-Gamma, 16 k-points — re-run from the same converged density matrix used elsewhere in this suite's tests, purely so `.bands.WFSX` stays a few MB instead of the 100+ MB a full, finely-sampled path would produce). Real dispersion, a genuine indirect gap, and two chemical species — used for most of this walkthrough. `calc.fdf`'s own `### BANDS` section shows `WFS.Write.For.Bands T` in a real, full production input (alongside DOS/relaxation settings you don't need for this tool specifically).
- `spin/Ospin.fdf` / `spin/Ospin.bands` / `spin/Ospin.bands.WFSX` / `spin/Ospin.HSX` / `spin/O.ion(.xml)` — a real SIESTA calculation run specifically for this walkthrough: a single, spin-polarized oxygen atom in a large vacuum box, seeded (`%block DM.InitSpin`) to converge to its physical 2 Bohr-magneton triplet ground state (confirmed in the calculation's own log: `spin moment ... |S| = 2.00000`). Deliberately the smallest possible real system that is genuinely spin-polarized — `Ospin.fdf` is short enough to read top-to-bottom as a complete, self-contained example of every fdf setting this tool needs (`WFS.Write.For.Bands T`, `SaveHS T`, `Spin polarized`).
- `example_2.10.sh` — the guided walkthrough (**not** an automated test — see `test/2-analysis/10-fatbands/test.sh` for that).
- `.gitignore` — excludes `output/` and every file this tool itself generates on a run.

8.10.1.7 Running the walkthrough

```
cd examples/2.10-stb-fatbands
./example_2.10.sh
```

It pauses between sections ([Press Enter to continue]) and always starts by wiping its own output/. Self-contained cases are generated:

Folder	What it shows
species-l-default/	The default species_l projection on a real, dispersive, multi-species structure; accurate (.HSX-based) overlap weights
projection-modes/	The same structure/bands, contrasting l vs. species vs. species_l categories
category-filter/	--category restricting the output to specific species/orbital combinations
accuracy-fallback/	No .HSX available: the approximate (|c|^2) fallback, via --geometry-file
spin-polarized/	The real spin-polarized fixture: spin-resolved gap analysis AND <category>_up/<category>_down splitting
full-report/	Default (no report/data files) vs. --save-report --save-gnuplot, references.bib

8.10.1.8 Try it yourself

```
# Your own finished SIESTA calculation (needs WFS.Write.For.Bands T -- see above)
stb-fatbands --label my_calc --shift fermi --save-report --save-gnuplot

# Which orbital character dominates each band, split by species AND l
stb-fatbands --label my_calc --shift fermi --projection species_l --category Fe-d O-p

# A spin-polarized calculation -- categories split into _up/_down automatically
stb-fatbands --label my_magnetic_calc --shift fermi --view
```

8.10.1.9 Flag reference

Flag	Meaning
--label	SIESTA label; shorthand for --file <label>.bands + auto-detected .bands.WFSX/.HSX/.fdf.
--file/--wfsx	Explicit .bands/.WFSX paths (alternative to --label).
--hsx-file	Explicit .HSX for accurate, overlap-weighted norms.
--geometry-file	Explicit .fdf (+ .ion/.ion.xml) fallback parent when no .HSX is available.
--shift	Energy reference: vbm/cbm/fermi/manual (same vocabulary as stb-bands).
--manual-value	Custom shift value (required with --shift manual).
--gap-tol	Metallic-classification threshold, eV (default 0.01).
--projection	l / ml / atom / species / species_l (default).
--category	Restrict output to specific category values (default: all found).
--k-tol	.bands/.WFSX eigenvalue cross-check tolerance, eV (default 0.001).
-o/--output-dir	Where all generated files (and references.bib) land.
--save-report	Persist the full report to stb_fatbands_report.txt.
--save-gnuplot	Also write fatbands_<category>.dat/fatbands.gplot.
--view	Show an interactive matplotlib preview.

8.10.1.10 What's next

core/band_scatter.py's scatter-plot machinery is shared with stb-ipr (a single continuous inverse-participation-ratio value per (k, band), no category) and stb-spintexture (signed spin-expectation

components). `core/siesta_wfsx.py`'s `.WFSX`/`.HSX` loading is shared with `stb-wfdensity`, `stb-sts`, and `stb-coop` — tools that need the same wavefunction coefficients for a full k-mesh instead of a single k-path. `stb-bands` covers the plain (non-orbital-resolved) band gap question this tool's own `[2] BAND GAP ANALYSIS` section reuses.

8.10.2 Implementation details

Projection weights in a non-orthogonal basis

SIESTA expands each Bloch state in atomic orbitals, $|\psi_{n\mathbf{k}}\rangle = \sum_\mu c_{\mu,n\mathbf{k}}|\phi_{\mu\mathbf{k}}\rangle$, with overlap $S_{\mu\nu}(\mathbf{k})$. The weight of orbital μ in the state is the Mulliken-like quantity

$$w_{\mu,n\mathbf{k}} = \mathrm{Re}\Big[c^*_{\mu,n\mathbf{k}} \sum_\nu S_{\mu\nu}(\mathbf{k})\, c_{\nu,n\mathbf{k}}\Big], \qquad \sum_\mu w_{\mu,n\mathbf{k}} = 1,$$

(sisl's `norm2(projection="hadamard")` with the Hamiltonian as parent). A category (an atom, a species, an angular momentum, …) receives the sum of the weights of its orbitals. Without the overlap matrix the tool falls back to $|c_\mu|^2$, which is exact only for an orthogonal basis; it says so.

Files.. The `.bands` file (path, eigenvalues, Fermi level) is combined with the wavefunction file SIESTA writes for the same path, `<label>.bands.WFSX` (`WFS.Write.For.Bands T`), and, preferably, `<label>.HSX` (`SaveHS T`) for the overlap; `-geometry-file` gives the `.fdf` when no `.HSX` is available. Nothing in the files ties a `.WFSX` to a `.bands`: the tool checks that k-point and orbital counts agree and compares eigenvalues at a few sampled k-points within `-k-tol` (default 0.001 eV) before trusting the pairing.

Categories.. `-projection` chooses the grouping: `species_l` (default; e.g. `Sn-s`, `O-p`), `species`, `atom`, `l` or `ml`; `-category` restricts the output to selected categories. Spin-polarized runs split every category into `_up`/`_down`; orbitals with $l > 3$ count only in the atom and species modes. The VBM/CBM analysis of **`stb-bands`** (with `-gap-tol`) is repeated and `-shift` sets the energy zero. With `-save-gnuplot` each category becomes a column of `fatbands_<category>.dat` with a script that draws marker sizes proportional to the weight.

8.10.3 Command-line reference

8.10.3.1 `stb-fatbands`

Orbital-projected (fatbands) band structure from .bands + .WFSX.

Synopsis.

```
usage: stb-fatbands [-h] [--label LABEL] [--file INPUT_FILE] [--wfsx WFSX]
                    [--hsx-file HSX_FILE] [--geometry-file GEOMETRY_FILE]
                    --shift {vbm,cbm,fermi,manual} [--manual-value MANUAL_VALUE]
                    [--gap-tol GAP_TOL] [--projection {l,ml,atom,species,species_l}]
                    [--category CATEGORY [CATEGORY ...]] [--k-tol K_TOL]
                    [-o OUTPUT_DIR] [--save-report] [--save-gnuplot] [--view] [-v]
                    [--no-intro]
```

Options

`--label LABEL`
SIESTA output label. Shorthand for `--file <label>.bands`, auto-detects <label>.bands.WFSX (falling back to `<label>.WFSX` with a note), and auto-detects `<label>.HSX` (optional, upgrades weight accuracy) then `<label>.fdf` (geometry+basis fallback – NOT `<label>.XV`, which has no per-orbital basis information). Mutually exclusive with `--file/--wfsx/--hsx-file/--geometry-file`.

`--file INPUT_FILE`
Explicit path to the .bands file (alternative to `--label`).

`--wfsx WFSX`
Explicit path to the .WFSX file. Required if `--file` is used instead of `--label`.

`--hsx-file HSX_FILE`
Optional explicit .HSX path (Hamiltonian+overlap) for physically correct overlap-weighted norms. Falls back to `--geometry-file` / auto-detected .fdf with an accuracy warning if omitted.

`--geometry-file GEOMETRY_FILE`
Optional explicit .fdf geometry path (must have the basis's .ion/.ion.xml files alongside it), used as the fallback parent when no .HSX is available. A bare .XV has no per-orbital basis information and will fail the orbital-count check against the .WFSX – use `--hsx-file` or a .fdf instead. Requires sisl.

`--shift {vbm,cbm,fermi,manual}`
Reference energy shift, same vocabulary as `stb-bands`:
- 'vbm' : Valence Band Maximum
- 'cbm' : Conduction Band Minimum
- 'fermi' : Fermi level
- 'manual' : Custom shift value (requires `--manual-value`). required; choices: vbm, cbm, fermi, manual

`--manual-value MANUAL_VALUE`
Custom energy shift value (required if `--shift` manual is used). type: float

`--gap-tol GAP_TOL`
Energy tolerance in eV for VBM/CBM classification when `--shift` is vbm/cbm (default: 0.01). Same meaning as `stb-bands`. type: float

`--projection {l,ml,atom,species,species_l}`
Category to color/size markers by (default: species_l):
l: aggregate by angular momentum (s, p, d, f)
ml: detailed orbital (s, px, py, pz, dxy, ...)
atom: per atom index
species: per chemical species
species_l: chemical species AND angular momentum combined
(e.g. 'Sn-s', 'Sn-p', 'O-s', 'O-p') – shows which
orbital character within a species dominates a band,
which plain 'species' or plain 'l' cannot on their own
Single choice only – unlike `stb-dos's --type`, the plot needs one weight per marker; run again for another projection. choices: l, ml, atom, species, species_l

`--category CATEGORY [CATEGORY ...]`
Restrict output to specific category values (e.g. `--projection` species `--category` O Sn, or `--projection` species_l `--category` Sn-s O-p). Default: all categories found.

`--k-tol K_TOL`
Cross-check tolerance (eV) between WFSX-stored eigenvalues and .bands eigenvalues at sampled k-points (default: 0.001). This is a file-order sanity guard, not a physics tolerance like `--gap-tol` – raising it only weakens the check that .bands/.WFSX truly correspond to the same calculation. type: float

`-o, --output-dir OUTPUT_DIR`
Directory to write fatbands_<category>.dat/fatbands.gplot (with `--save-gnuplot`) and

stb_fatbands_report.txt/references.bib into (default: current directory). Created if it doesn't exist.

`--save-report`
: Also persist the full run report to `stb_fatbands_report.txt`. Off by default. switch, off by default

`--save-gnuplot`
: Also write `fatbands_<category>.dat` (one file per category, k_position/energy(eV)/value columns) and `fatbands.gplot` together. Off by default – this tool used to write both unconditionally on every run; that's no longer the case. switch, off by default

`--view`
: Show an interactive matplotlib preview of the fatbands plot before finishing. Off by default – this tool used to always show it, with no way to skip. switch, off by default

Like every STB command it also accepts `-h`/`–help`, `-v`/`–version` and `–no-intro` (Section 3.3).

From the built-in help.

```
Example usage:
  stb-fatbands --label siesta --shift fermi --projection l
  stb-fatbands --label siesta --shift fermi --projection species --category O Sn
  stb-fatbands --label siesta --shift fermi --projection species_l
  stb-fatbands --file siesta.bands --wfsx siesta.bands.WFSX --hsx-file siesta.HSX \
      --shift fermi --save-report --save-gnuplot --view
```

8.11 STM Simulator

At a glance

Menu code: 2.11 **Command:** `stb-stm`
Guided example: `examples/2.11-stm/`
Summary: Simulate an STM image (Tersoff-Hamann approximation) from a SIESTA .LDOS grid.

8.11.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/2.11-stm/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

8.11.1.1 What this tool does

`stb-stm` turns a SIESTA energy-integrated local density of states grid (`<label>.LDOS`) into a simulated STM image, under the **Tersoff-Hamann approximation**: tunneling current is proportional to the LDOS at the tip position. Two modes:

- `--mode current` (**default**) — the classic *constant-current* STM image: for every `(x, y)`, finds the height where the LDOS first drops to `--iso` coming in from vacuum — the topography a real tip follows to hold tunneling current constant.
- `--mode height` — the simpler *constant-height* image: a flat 2D LDOS map at a fixed height `--z` above the surface.

8.11.1.2 Why this matters (a bit of theory)

The Tersoff-Hamann approximation. A real STM tip is a complicated, atomically-sharp, chemically-specific object — modeling it exactly is a hard many-body tunneling problem. Tersoff and Hamann (1985) showed that if you approximate the tip as a structureless **s-wave point probe**, Bardeen's tunneling theory collapses to a strikingly simple result:

```
I(x, y, z) is proportional to LDOS(x, y, z; E_F)
```

the tunneling current at tip position `(x, y, z)` is proportional to the **sample's own local density of states, evaluated at the tip position**, integrated over the electrons that can actually tunnel at the applied bias (see below). This is *why* an STM image can be simulated directly from a DFT calculation's electron density alone, with no separate tip calculation at all — and why it is only a **proportionality**, not a calibrated, absolute tunneling current in Amperes: `--iso` (or the LDOS value under the tip in `--mode height`) is in the grid's own units (e/Bohr^3), tuned for sensible image contrast, not read off a real experiment's setpoint current.

What a `.LDOS` file actually holds, and why the bias WINDOW matters. `%block LocalDensityOfStates Emin Emax` tells SIESTA to integrate the local density of states over an energy window `[Emin, Emax]` and write the result as a real-space grid — the same binary format as a charge-density `.RHO` file, but holding

```
LDOS(r) = integral from Emin to Emax of  rho(r, E) dE
```

instead of the density integrated over *all* occupied states. **This bias window is a property of the SIESTA run itself, baked into the grid file — `stb-stm` cannot see or change it after the fact.** The energies can be given as absolute values, or relative to the Fermi level using the `EF` keyword as the first token — this is the setting used to generate this example's own `siesta.LDOS`:

```
%block LocalDensityOfStates
EF -3.50 0.00  eV
%endblock LocalDensityOfStates
```

`EF -3.50 0.00 eV` integrates from 3.50 eV *below* the Fermi level up to the Fermi level itself — i.e., **occupied states only**. This is the STM analog of a negative sample bias in a real experiment (electrons tunnel *from the sample to the tip*): the image shows where the *filled* states live. A positive window (e.g. `EF 0.00 3.50 eV`) would instead image **empty** states, and in general a different bias window can show visibly different orbital character for the same material (a real, well-known STM effect, not a simulation artifact) — always know which window you (or whoever ran the calculation) actually used before interpreting an image.

Note on the exact fdf syntax: verified live while building this example that SIESTA's parser requires `EF Emin Emax` and the unit (`eV`) on the **same line** — splitting them across two lines (`EF -3.50 0.00` then `eV` on the next line) raises a hard `ERROR in LocalDensityOfStates block!` and aborts the run. Always write it as one line, exactly as shown above.

`--mode height` vs. `--mode current`: two different physical pictures.

- `--mode height` is the cheaper, simpler case: just read off the LDOS grid at one fixed height everywhere. It is NOT what a real constant -current STM does, and can be misleading for a corrugated surface — the tip would physically crash into a protruding atom at a height that's perfectly safe

elsewhere. It also has no built-in safeguard against a poorly chosen `--z`: verified live in this walkthrough that the default `3.0` Ang gives an entirely flat, near-zero image for this example's own occupied-states LDOS (already decayed by that height), while `--mode current`'s feedback loop always finds a height with real signal (or clearly reports when it can't) on the exact same grid.

- `--mode current` reproduces the real feedback loop: at every `(x, y)`, the tip retreats from far above the surface until the LDOS first reaches `--iso` — exactly mimicking a real STM's constant-current topography scan, and the mode that produces the textbook "STM image" with visible atomic corrugation. `stb-stm` searches **outside-in** (farthest height first) so it finds the same point a real retreating tip would stop at, not an arbitrary crossing.

Why `--iso`/`--z` now have defaults, and what they mean physically. Both used to be required with no default. `--iso` defaults to `0.001` e/Bohr^3, `--z` to `3.0` Ang — reasonable starting points for a typical slab, not a universally "correct" value for every material/bias window; tune them if the default image looks saturated (too many points crash into the surface or never reach `--iso`) or empty (nothing does — see `iso-sensitivity/` below for exactly this on a real structure).

Locating the surface: a real bug found using this exact fixture. Every height reported by this tool is **relative to the topmost atom** along the detected (or `--axis`-forced) surface-normal direction — never the raw simulation cell origin. This example's own `siesta.XV` (a real CrS monolayer, fetched from the twodmatpedia database — the same structure already used in `examples/2.7-stb-workfunction/` and `examples/2.9-stb-xrd/`) is exactly what caught a real, verified bug while building this walkthrough: its 4 atoms sit at fractional z = `0, 0, 0.066, 0.934` (2 Cr at the cell's stored origin, 2 S buckled slightly to either side) — a naive "topmost atom = `max(z)`" picks the `z=0.934` S atom and searches **upward** from there, straight into the tiny ~7% sliver before wrapping back to the Cr atoms — collapsing the whole search window to ~1.5 Ang instead of the genuine ~20 Ang vacuum region that actually separates this thin monolayer's two faces. Fixed with a new `core/kspace.py::find_surface_reference` helper: it reuses the same gap-finding logic that already decides *whether* an axis is vacuum-padded (`detect_vacuum_axes`), now also used to find *where* the real vacuum starts, correctly identifying the `z=0.066` S atom as the reference instead.

A second, related subtlety this same fixture exposed: a periodic cell's stacking axis is topologically a ring, so **any** single compact atomic region surrounded by vacuum has TWO faces exposed to that same gap (a "top" and a "bottom"), not one. Searching the *entire* ~20 Ang gap gave wildly unphysical "corrugation" values (18-20 Ang — real STM corrugation is sub-Angstrom to a few Ang) at several `--iso` values tried live, because the outside-in scan eventually crossed into the *far* face's own LDOS tail. `find_surface_reference` caps the search at **half** the identified gap — matching this tool's own pre-existing documented limitation ("only images the surface exposed in the +axis direction... a slab with two exposed faces only gets its 'top' one imaged this way"), just now enforced numerically instead of relying on the atom arrangement to accidentally already imply it (as a perfectly centered structure, like this suite's own graphene test fixture, happens to). Verified live: identical numbers as before on every existing (conventional) fixture — this fix only changes anything for a structure like this one — and a physically sensible ~1.0-1.7 Ang corrugation on this real CrS monolayer once fixed.

`stb-stm` refuses to run rather than guess if it can't find exactly one vacuum-padded axis at all (unlike `stb-workfunction`, which can degrade to an assumed axis — an STM image without a genuine, confirmed surface is meaningless, not just less precise).

8.11.1.3 The report: console output, `--save-report`, `--save-gnuplot`, `--view`

Every run prints a numbered report, the same `[0]...[5]` style every newer tool in this suite uses:

Section	Content
`[0] RUN METADATA`	input files, mode, `--iso`/`--z` (whichever applies), output dir, active options
`[1] INPUT DATA`	grid shape, LDOS range, surface-normal axis, vacuum-padded axes, topmost atom height, axis length
`[2] STM IMAGE`	mode-specific results: actual height used + LDOS stats (`height`), or iso/coverage/height-map stats + corrugation (`current`)
`[3] OUTPUT DATA & PLOTS`	whether `stm_<mode>.dat`/`.gplot` were written
`[4] REFERENCES`	writes `references.bib` (SIESTA + Tersoff-Hamann)
`[5] SUMMARY & FILES`	status and a recap of every file written

- **`--save-report`** — persist the report to `stb_stm_report.txt`. Off by default.
- **`--save-gnuplot`** — write `stm_<mode>.dat` (a pm3d-blocked X/Y/value file) + `stm_<mode>.gplot` together. Off by default — this tool used to write both **unconditionally** on every run. The `.gplot` script's own `set output`/`splot` filenames are always bare basenames (`stm_current.pdf`, never `some_dir/stm_current.pdf`) — a real, verified bug found while rewriting this tool: the old `set output` line kept the `--output-dir` prefix (inconsistently with `splot`, which already stripped it), so running `gnuplot stm_current.gplot` from *inside* the output directory itself (the intended, documented usage) tried to write to a nested, usually-nonexistent `output_dir/output_dir/...` path.
- **`--view`** — show an interactive matplotlib preview (a filled 2D color map, mirroring the `.gplot`'s own `pm3d map`). Off by default — this tool previously had **no** matplotlib option at all.

8.11.1.4 When you'd reach for it

- Simulating what a real STM experiment would see on a finished SIESTA slab calculation, before (or instead of) a costly real measurement.
- Comparing occupied- vs. empty-states images (different `%block LocalDensityOfStates` bias windows) to understand bias-dependent contrast on a real material.
- Getting a quick, physically grounded corrugation estimate (`--mode current`) to compare against a real STM measurement's own topography.

8.11.1.5 Two ways to run it

A — direct CLI:

```
stb-stm --file siesta.LDOS --geometry-file siesta.XV --mode current
```

B — interactive `stb-suite` menu:

```
$ stb-suite
Select an option (0-7, or a tool code like 5.1.2): 2.11
```

Both paths call the exact same underlying tool and produce the exact same output — `example_2.11.sh` proves this directly at the end. The menu defaults `--iso`/`--z` the same way the CLI does (just press Enter), plus separate `y/N` prompts for `--save-report`/`--save-gnuplot`/`--view`.

8.11.1.6 Files in this folder

- `siesta.LDOS` / `siesta.XV` / `calc.fdf` / `structure.fdf` — a real SIESTA calculation of a CrS monolayer (fetched via `stb-fetch` from the twodmatpedia OPTIMADE database, id `2dm-2617` — the same

structure already used in `examples/2.7-stb-workfunction//examples/2.9-stb-xrd/`), with `%block LocalDensityOfStates EF -3.50 0.00 eV` (occupied states only, see the theory section above) producing `siesta.LDOS`. This exact fixture is what caught the real vacuum-side bug described above.

- `example_2.11.sh` — the guided walkthrough (**not** an automated test — see `test/2-analysis/11-stm/test.sh` for that, which uses its own separate graphene fixture with a conventional, centered vacuum axis).
- `.gitignore` — excludes `output/` and every file this tool itself generates on a run.

8.11.1.7 Running the walkthrough

```
cd examples/2.11-stm
./example_2.11.sh
```

It pauses between sections (`[Press Enter to continue]`) and always starts by wiping its own `output/`. Self-contained cases are generated:

Folder	What it shows
`constant-current/`	The default mode with default `--iso`: the real STM feedback-loop image, corrugation reported
`constant-height/`	`--mode height` at the default `--z` going flat (LDOS already decayed to ~0 there) vs. a closer `--z` recovering real contrast
`iso-sensitivity/`	The same structure at three `--iso` values -- too high (mostly NaN), sensible (default), and very low
`full-report/`	Default (no report/data files) vs. `--save-report --save-gnuplot`, `references.bib`

8.11.1.8 Try it yourself

```
# Your own finished SIESTA slab calculation (needs %block LocalDensityOfStates -- see above)
stb-stm --label my_slab --mode current --save-report --save-gnuplot

# A shallower/deeper occupied-states window, or an empty-states one, needs a NEW SIESTA run
# (the bias window is baked into the .LDOS file itself, not a stb-stm option)

# Constant-height at a custom tip distance, with a live preview
stb-stm --label my_slab --mode height --z 4.0 --view
```

8.11.1.9 Flag reference

Flag	Meaning
`--label`	SIESTA label; shorthand for auto-detected `.LDOS`/`.STM.LDOS` + `.XV`/`.fdf`.
`--file`/`--geometry-file`	Explicit `.LDOS`/geometry paths (alternative to `--label`).
`--axis`	Surface-normal axis (0/1/2). Auto-detected from vacuum padding if omitted.
`--mode`	`current` (default) or `height`.
`--z`	Height above the topmost atom for `--mode height` (default `3.0` Ang).
`--iso`	LDOS threshold for `--mode current` (default `0.001` e/Bohr^3).
`--z-max`	Upper bound of the search/plot window (default: half the detected vacuum gap).
`-o`/`--output-dir`	Where all generated files (and `references.bib`) land.
`--save-report`	Persist the full report to `stb_stm_report.txt`.
`--save-gnuplot`	Also write `stm_<mode>.dat`/`stm_<mode>.gplot`.
`--view`	Show an interactive matplotlib preview.

Flag	Meaning

8.11.1.10 What's next

`core/kspace.py`'s vacuum-axis detection (and now, the surface-reference gap-finding this tool's own fix added) is shared with `stb-workfunction`/`stb-mlrelax`/`stb-kgrid` and every other tool that needs to tell a genuinely periodic axis apart from a vacuum-padded one. `stb-density` covers the same real-space-grid family (`.RHO`, total charge/spin density) with a richer set of cut modes (slice/3D/profile) for a quantity that isn't tied to a surface the way an STM image is.

8.11.2 Implementation details

Tersoff–Hamann

For a tip modelled as an s-wave, the tunnelling current at small bias V is proportional to the local density of states of the sample at the tip position integrated over the bias window [49, 50],

$$I(\mathbf{r}_0, V) \propto \int_{E_F}^{E_F+eV} \rho(\mathbf{r}_0, E)\, dE, \qquad \rho(\mathbf{r}, E) = \sum_{n\mathbf{k}} |\psi_{n\mathbf{k}}(\mathbf{r})|^2 \delta(E - \epsilon_{n\mathbf{k}}).$$

SIESTA computes the integrated quantity on the real-space grid when `%block LocalDensityOfStates` gives the window `Emin Emax` (absolute energies), and writes it to `<label>.LDOS`.

Modes..

`-mode current` (default)
: constant-current topography: for every (x, y) column the grid is scanned from the vacuum towards the surface and the height at which the LDOS first reaches `-iso` (default 0.001, in the e/Bohr^3 units of the `.LDOS` file) is recorded; `-z-max` (Å above the topmost atom; default the cell boundary) sets the top of the search window. The report says how many columns reached the threshold – tune `-iso` if too few or too many did.

`-mode height`
: constant-height map: the LDOS on the plane `-z` Å (default 3.0) above the topmost atom.

Heights are measured from the surface atom found with the wrap-aware vacuum gap of Section 4.4 (`-axis` sets the surface normal; by default the single detected vacuum axis), using the geometry from `<label>.XV` or `-geometry-file`.

Limitations.. Only the face on the +axis side of the largest vacuum gap is imaged (mirror the slab to image the other one); the axis must be Cartesian-aligned; `-iso` is a relative threshold, not a calibrated current; and the `.LDOS` file does not record its bias window – keep it with your input. Positive windows above E_F image empty states (positive sample bias), windows below E_F image filled states.

8.11.3 Command-line reference

8.11.3.1 `stb-stm`

Simulate an STM image (Tersoff-Hamann approximation) from a SIESTA energy-integrated LDOS grid (`<label>.LDOS`).

Synopsis.

```
usage: stb-stm [-h] [--label LABEL] [--file LDOS_FILE] [--geometry-file GEOMETRY_FILE]
               [--axis {0,1,2}] [--mode {height,current}] [--z Z] [--iso ISO]
               [--z-max Z_MAX] [-o OUTPUT_DIR] [--save-report] [--save-gnuplot]
               [--view] [-v] [--no-intro]
```

Options

`--label LABEL`
: SIESTA output label. Auto-detects `<label>.LDOS` (falling back to <label>.STM.LDOS with a note) and <label>.XV/<label>.fdf for the topmost-atom reference. Mutually exclusive with `--file`/`--geometry-file`.

`--file LDOS_FILE`
: Explicit path to the .LDOS/.STM.LDOS file (alternative to `--label`).

`--geometry-file GEOMETRY_FILE`
: Explicit .XV/.fdf geometry path (alternative/override to `--label`'s auto-detect). Required if `--file` is used instead of `--label`.

`--axis {0,1,2}`
: Surface-normal axis (0=X, 1=Y, 2=Z). Auto-detected from the geometry's vacuum padding if omitted and exactly one vacuum axis is found. choices: 0, 1, 2; type: int

`--mode {height,current}`
: 'current' (default): classic constant-current STM image – the topography a tip follows to hold the LDOS at `--iso`. 'height': constant-height LDOS map at `--z` above the surface. choices: height, current

`--z Z`
: Height (Ang) above the topmost surface atom for `--mode` height (default: 3.0, a commonly used representative constant-height tip-sample separation). type: float

`--iso ISO`
: LDOS threshold (same units as the .LDOS file, e/Bohr^3) defining the constant-current contour for `--mode` current (default: 0.001 – a relative threshold, not a calibrated absolute current; tune it for your own system's LDOS magnitude if the default leaves too many/few points reaching it, see [3] in the report). type: float

`--z-max Z_MAX`
: Upper bound (Ang above the topmost atom) of the search/plot window along `--axis`. Default: distance to the cell boundary. type: float

`-o, --output-dir OUTPUT_DIR`
: Directory to write stm_<mode>.dat/.gplot into (with `--save-gnuplot`) and stb_stm_report.txt/references.bib (default: current directory). Created if it doesn't exist.

`--save-report`
: Also persist the full run report to `stb_stm_report.txt`. Off by default. switch, off by default

`--save-gnuplot`
: Also write `stm_<mode>.dat` + `stm_<mode>.gplot` together. Off by default – this tool used to write both unconditionally on every run. switch, off by default

`--view`
: Show an interactive matplotlib preview of the STM image before finishing. Off by default.switch, off by default

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Example usage:
  stb-stm --label siesta --mode current --iso 0.001
  stb-stm --label siesta --mode height --z 3.0
  stb-stm --label siesta --save-report --save-gnuplot --view
```

8.12 Wavefunction Density

At a glance

Menu code: 2.12 **Command:** `stb-wfdensity`
Guided example: `examples/2.12-wfdensity/`
Summary: Real-space |psi|^2 of a single (k, band) eigenstate from .WFSX -> Gaussian cube.

8.12.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/2.12-wfdensity/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

8.12.1.1 What this tool does

`stb-wfdensity` computes the real-space probability density `|psi(r)|^2` of **one chosen electronic eigenstate** — band N at k-point K — from a SIESTA `.WFSX` file, and writes it as a Gaussian `.cube` file (the primary output, for VESTA/VMD/Avogadro) plus an optional 2D-slice or 1D-profile gnuplot/matplotlib preview. Useful as a diagnostic: "what does the CBM look like at this k-point", "is this defect state localized on one atom or spread across the lattice" — **not** a physical ground-state observable by itself (see the theory section below for why).

8.12.1.2 Why this matters (a bit of theory)

From LCAO coefficients to a real-space value. SIESTA is an LCAO code: every state is a linear combination of localized atomic orbitals,

```
psi_n(k, r) = sum_mu  c_mu,n(k) * phi_mu(r)
```

`stb-fatbands` uses this same expansion to compute *how much weight* each orbital contributes (a projection). `stb-wfdensity` instead evaluates the *actual value of psi at every point of a real-space grid* — summing every orbital's own numerical radial function, phase and all, at each grid point — then takes `|psi(r)|^2` to get a real, plottable 3D density for that one state.

Why only a real `.fdf` (+ `.ion`/`.ion.xml`) works — never `.XV`/`.HSX`. Evaluating `phi_mu(r)` at a real-space point needs the orbital's actual *radial shape* (a numerical function with a finite cutoff radius) — not just bookkeeping about how many orbitals exist. Verified live on this example's own structure:

```
Geometry from siesta.XV  : 4 orbitals total (1 per atom), cutoff radius = -1 (no shape at all)
Geometry from calc.fdf   : 64 real orbitals, cutoff radius = 3.61 Ang
```

A bare `.XV` (or `.HSX`) carries no radial-function information at all — sisl fills in one placeholder "orbital" per atom just so the bookkeeping doesn't crash, with a cutoff radius of `-1` (not "small", literally *undefined*). Only a `.fdf`, read together with the `.ion`/`.ion.xml` files SIESTA itself wrote for that basis, carries the real numerical radial functions `phi_mu(r)` needs. This is a hard physical/data limitation, not an arbitrary restriction — `--geometry-file` (or `--label`'s auto-detected `<label>.fdf`) is not optional.

The normalization check: a built-in quality control. A single-particle wavefunction is normalized to exactly one electron:

```
integral |psi(r)|^2 dV = 1
```

`stb-wfdensity` computes this directly from the evaluated grid and reports it in `[3] WAVEFUNCTION DENSITY`, warning if it deviates from 1 by more than 5%. This is a genuinely useful diagnostic, not decoration — verified live on this example's own VBM state:

```
--spacing 0.3 (default) : integral = 0.939287  -- [WARNING] more than 5% off
--spacing 0.2           : integral = 1.009170  -- [OK]
```

The coarser grid was clipping the tails of the real orbitals (each with a genuine several-Angstrom cutoff radius) badly enough to lose ~6% of the normalization — the exact failure mode the warning exists to catch, demonstrated live in `grid-resolution/` below.

Not a physical observable by itself. `|psi_n(k, r)|^2` for one single band at one single k-point is **not**, by itself, a measurable ground-state quantity — the real physical charge density sums `|psi_n(k, r)|^2` over **every occupied** band and k-point, weighted by the Brillouin-zone integration weights (that's exactly what `stb-density`'s `.RHO`-based total charge density already is). This tool is a diagnostic/visualization aid for **one** state at a time — e.g. "is this defect level localized", "what does the band edge look like at Gamma" — never a substitute for the real total density.

Selecting the state: `--band N`, or `vbm`/`cbm` with a Fermi energy. `--band vbm`/`cbm` searches the *entire* k-mesh in the `.WFSX` for the global valence-band-maximum/conduction-band-minimum state, exactly the same search `stb-fatbands`/`stb-bands` use. This needs a Fermi energy, resolved in priority order: `--fermi` (explicit value) > `--bands-file` (a real `.bands` file) > `--fermi-file` (an explicit `.out` log) > an auto-detected `.out` in the current directory — **not** assumed to be named after the label (see the next section for exactly why that decoupling matters in practice).

A real bug this example's own fixture caught: `--label` + `--geometry-file`. This folder's `calc.fdf`/`structure.fdf`/`siesta.bands.WFSX`/`calc.out` are a real SIESTA run with `SystemLabel siesta` -- but the actual input file is named `calc.fdf`, and there is **no** `siesta.fdf` anywhere (a completely ordinary, common mismatch for anything generated by this suite's own workflow tools). `--label siesta` alone used to fail outright ("No usable geometry found") because `--label` and `--geometry-file` together were rejected as mutually exclusive — forcing `--wfsx` to be given explicitly too, just to change the geometry source. Fixed: `--label` now auto-detects the `.WFSX` as before, and `--geometry-file` (given alongside it) simply overrides the geometry source instead of being rejected. There are actually **three** `.fdf` files in this exact folder (`calc.fdf`, `structure.fdf`, and `kpath_bs.fdf`, a k-path-only fragment kept here specifically to show this) -- none named `siesta.fdf` -- so `--geometry-file` still has to be given explicitly; only the "can't combine with `--label`" restriction was the actual bug.

Another real bug this example's own fixture caught: where the slice cuts. `--mode`-equivalent slice output (`--save-gnuplot`/`--view` without `--profile`) used to always cut at the **geometric center** of

the cell along `--axis` — with no way to change it. This structure's 4 atoms sit at fractional z = `0, 0, 0.066, 0.934` (clustered near the cell boundary), with the real vacuum in the **middle** of the cell — so the old geometric-center default landed squarely in that vacuum, showing essentially nothing. Fixed: the default is now the plane where the planar-averaged `|psi|^2` is largest (verified live: index 0, right at the atoms, versus the old geometric center at roughly the halfway point which sits deep in the vacuum) — `--pos` (Angstrom along `--axis`, same name/semantics as `stb-density`'s own flag) overrides it manually. See `slice-position/` below for both, side by side, on the same real grid.

Grid resolution: `--spacing` vs. `--shape`. `--spacing` (Angstrom per voxel, default `0.3`) is the usual control; `--shape NX NY NZ` sets exact voxel counts instead, mutually exclusive with `--spacing`. A coarser grid is faster but risks the normalization warning above; there's no universal "correct" value, only "checked, via the normalization warning, and tightened if it fires."

Spin channels and what's *not* modeled. `--spin` selects which channel to evaluate (0/1, must be 0 for a non-spin-polarized calculation) — this example's own calculation is genuinely spin-polarized (`Spin polarized` in `calc.fdf`, `nspin=2` in the `.WFSX`). Only the first spinor component is evaluated at all times; non-collinear/SOC wavefunctions are not fully represented.

8.12.1.3 The report: console output, `--save-report`, `--save-gnuplot`, `--view`

Every run prints a numbered report, the same `[0]...[6]` style every newer tool in this suite uses:

Section	Content
`[0] RUN METADATA`	input files, band/Fermi selection, grid spec, slice/profile mode + position, output dir, active options
`[1] INPUT DATA`	orbital count, k-points in the `.WFSX`, spin channels
`[2] STATE SELECTION`	k-index/vector, spin, band, eigenvalue -- and the VBM/CBM search details if used
`[3] WAVEFUNCTION DENSITY`	grid shape, the normalization check (and its `[OK]`/`[WARNING]`)
`[4] OUTPUT DATA & PLOTS`	the `.cube` file (always), and whether the slice/profile `.dat`/`.gplot` were written
`[5] REFERENCES`	writes `references.bib` (SIESTA)
`[6] SUMMARY & FILES`	status and a recap of every file written

- **`--save-report`** — persist the report to `stb_wfdensity_report.txt`. Off by default.
- **`--save-gnuplot`** — write the slice/profile `.dat` + a real `.gplot` script together. Off by default — this tool used to write the `.dat` file unconditionally (opted out via a now-removed `--cube-only`) but never actually wrote a `.gplot` script at all, despite its own docstring calling this "gnuplot output". The `.cube` file itself is always written regardless (the tool's primary output).
- **`--view`** — an interactive matplotlib preview (2D slice or 1D profile, matching `--profile`). Off by default — this tool previously had no matplotlib option at all.

8.12.1.4 When you'd reach for it

- Checking whether a defect/impurity state is localized on one site or delocalized across the lattice.
- Visualizing what the VBM or CBM actually looks like at a specific k-point (Gamma, or the true band-edge k-point from a VBM/CBM search).
- A quick, real-space sanity check on one state before trusting a more abstract projection (`stb-fatbands`) or a derived quantity.

8.12.1.5 Two ways to run it

A — direct CLI:

```
stb-wfdensity --label siesta --geometry-file calc.fdf --band vbm --fermi-file calc.out --spacing 0.2
```

B — interactive `stb-suite` menu:

```
$ stb-suite
Select an option (0-7, or a tool code like 5.1.2): 2.12
```

Both paths call the exact same underlying tool and produce the exact same output — `example_2.12.sh` proves this directly at the end. The menu asks for the label and the `.fdf` structure path **separately** (a real fix — it used to only ask for one label and assume `<label>.fdf` named everything), and for VBM/CBM offers a small Fermi-source submenu (a direct value, a `.bands` file, an explicit `.out` log, or auto-detect — matching the CLI's own priority order).

8.12.1.6 Files in this folder

- `siesta.bands.WFSX` / `siesta.XV` / `calc.fdf` / `structure.fdf` / `calc.out` — a real, spin-polarized SIESTA band-structure calculation of a CrS monolayer (fetched via `stb-fetch` from the twod-matpedia OPTIMADE database, id `2dm-2617` — the same structure already used in `examples/2.7-stb-workfunction/`, `examples/2.9-stb-xrd/` and `examples/2.11-stm/`). `SystemLabel siesta`, but no `siesta.fdf` — this exact mismatch is what caught the `--label+--geometry-file` bug above.
- `kpath_bs.fdf` — a genuine k-path-only fdf fragment from the same real run, kept here specifically to show that this directory really does have multiple `.fdf` files, none matching the label — it doesn't even parse as a geometry at all (no `LatticeConstant` block), which is fine: it's never touched unless you explicitly pointed `--geometry-file` at it by mistake.
- `Cr.ion`/`Cr.ion.xml`, `S.ion`/`S.ion.xml` — the real numerical basis orbitals for this calculation, needed alongside `calc.fdf`/`structure.fdf` for real-space wavefunction evaluation (see the theory section above).
- `example_2.12.sh` — the guided walkthrough (**not** an automated test — see `test/2-analysis/12-wfdensity/test.sh` for that, which uses a separate, non-spin-polarized bulk Sn3O4 fixture).
- `.gitignore` — excludes `output/` and every file this tool itself generates on a run.

8.12.1.7 Running the walkthrough

```
cd examples/2.12-wfdensity
./example_2.12.sh
```

It pauses between sections (`[Press Enter to continue]`) and always starts by wiping its own `output/`. Self-contained cases are generated:

Folder	What it shows
`basic-run/`	A plain `--k-index`/`--band` run, the `.cube` file, and the normalization check
`vbm-cbm/`	`--band vbm`/`cbm` searching the whole k-mesh, Fermi energy from `--fermi-file`
`slice-position/`	The real bug: default (auto-detected `|psi|^2` peak, right at the atoms) vs. `--pos` at the old geometric center (near-empty vacuum)
`grid-resolution/`	`--spacing 0.3` (normalization warning fires) vs. `0.2` (passes) -- the same real state

Folder	What it shows
`full-report/`	Default (no report/data files) vs. `--save-report --save-gnuplot`, `references.bib`, a real gnuplot render

8.12.1.8 Try it yourself

```
# Your own finished SIESTA calculation (needs a real .fdf + .ion/.ion.xml files)
stb-wfdensity --label my_calc --k-index 0 --band 1 --save-report --save-gnuplot

# The VBM at whatever k-point it actually sits at, reading Fermi from the .out log
stb-wfdensity --label my_calc --band vbm --fermi-file my_calc.out --view

# A specific k-point by vector instead of index, at a finer grid
stb-wfdensity --label my_calc --k-point 0.0 0.0 0.0 --band 5 --spacing 0.2
```

8.12.1.9 Flag reference

Flag	Meaning
`--label`	SIESTA label; auto-detects `.WFSX` + (unless `--geometry-file` overrides it) `<label>.fdf`.
`--wfsx/--geometry-file`	Explicit `.WFSX`/`.fdf` paths.
`--k-index/--k-point`	Select k by index (default 0) or by vector.
`--spin`	Spin channel (0/1, default 0).
`--band`	Band number (1-based), or `vbm`/`cbm`.
`--fermi/--bands-file/--fermi-file`	Fermi energy sources for `--band vbm/cbm`, in priority order (plus auto-detected `.out`).
`--gap-tol`	VBM/CBM classification tolerance, eV (default `0.01`).
`--spacing/--shape`	Grid resolution (Ang/voxel) or exact voxel counts.
`--axis`	Axis for the optional slice/profile (default 2).
`--profile`	1D planar-averaged profile instead of a 2D slice.
`--pos`	Slice position, Ang along `--axis` (default: auto-detected `|psi|^2` peak).
`-o/--output-dir`	Where all generated files (and `references.bib`) land.
`--save-report`	Persist the full report to `stb_wfdensity_report.txt`.
`--save-gnuplot`	Also write the slice/profile `.dat`/`.gplot`.
`--view`	Show an interactive matplotlib preview.

8.12.1.10 What's next

`core/siesta_wfsx.py`'s `.WFSX`/geometry loading (`load_parent`, mode `"orbitals_required"`) is the same machinery `stb-sts`/`stb-coop` use for a full-BZ-mesh `.WFSX`, and shares its overall shape with `stb-fatbands`'s own `"hamiltonian_preferred"` mode (which additionally needs a `.HSX` for overlap-weighted orbital projections — this tool never touches `.HSX` at all). `stb-density` covers the real, physical ground -state charge density (`.RHO`, summed over every occupied state/k) that this tool's own single-eigenstate density is explicitly not a substitute for.

8.12.2 Implementation details

From coefficients to a grid.. The state $\psi_{n\mathbf{k}}(\mathbf{r}) = \sum_\mu c_{\mu,n\mathbf{k}}\,\phi_\mu(\mathbf{r}-\mathbf{R}_\mu)\,e^{i\mathbf{k}\cdot\mathbf{R}_\mu}$ is evaluated by sisl on a real-space grid of the cell, with spacing –`spacing` (Å, default 0.3) or an explicit –`shape`. The radial

functions ϕ_μ come from the `.ion/ .ion.xml` files next to the `.fdf` (`–geometry-file`); geometries read from `.HSX` or `.XV` carry no orbital shapes and are refused. The grid of $|\psi|^2$ is integrated over the cell as a normalization check: it should be close to 1; a value far from 1 means the grid is too coarse or the basis does not match.

Choosing the state.. `–k-index` (or `–k-point`, a k vector in the units stored in the `.WFSX`) and `–band` (a 1-based index, or `vbm`/`cbm`, found with `–gap-tol` from the Fermi level given by `–fermi`, `–bands-file`, `–fermi-file` or a log found automatically) select the state; `–spin` the channel. `vbm`/`cbm` search the whole `.WFSX`, which can be slow for a dense mesh.

Outputs.. The primary output is a Gaussian cube of $|\psi|^2$ for VESTA or VMD. With `–save-gnuplot`, a 2D slice perpendicular to `–axis` – by default at the plane where the planar average of $|\psi|^2$ is largest, or at `–pos` – or, with `–profile`, the planar average along the axis.

Interpretation.. A single $|\psi_{n\mathbf{k}}|^2$ is a diagnostic (is the CBM localized on the defect? does the VBM sit on the surface?), not an observable; physical densities sum over occupied states with weights (**`stb-density`**). Only the first spinor component of non-collinear states is evaluated, and the confined orbitals of SIESTA make the density exactly zero a few Å away from every atom.

8.12.3 Command-line reference

8.12.3.1 `stb-wfdensity`

Real-space |psi|^2 of one chosen (k, band) eigenstate from a SIESTA .WFSX file -> Gaussian .cube.

Synopsis.

```
usage: stb-wfdensity [-h] [--label LABEL] [--wfsx WFSX]
                     [--geometry-file GEOMETRY_FILE] [--k-index K_INDEX |
                     --k-point KX KY KZ] [--spin {0,1}] --band BAND [--fermi FERMI]
                     [--bands-file BANDS_FILE] [--fermi-file FERMI_FILE]
                     [--gap-tol GAP_TOL] [--spacing SPACING | --shape NX NY NZ]
                     [--axis {0,1,2}] [--profile] [-p POS] [-o OUTPUT_DIR]
                     [--save-report] [--save-gnuplot] [--view] [-v] [--no-intro]
```

Options

`--label LABEL`

SIESTA output label. Auto-detects `<label>.WFSX`, falling back to <label>.selected.WFSX (written by WriteWaveFunctions T + %block WaveFuncKPoints) then <label>.bands.WFSX, with a note – and, unless `--geometry-file` overrides it, `<label>.fdf` (with its .ion/.ion.xml files alongside it – NOT <label>.XV/.HSX, neither of which carries real orbital shapes). Mutually exclusive with `--wfsx` (not `--geometry-file` – see its own help: the real fdf is very often NOT named `<label>.fdf`).

`--wfsx WFSX`

Explicit path to the .WFSX file (alternative to `--label`).

`--geometry-file GEOMETRY_FILE`

Explicit .fdf geometry path (must have the basis's .ion/.ion.xml files alongside it). Required if `--wfsx` is used instead of `--label`; optional (and common) together WITH `--label` too, since the real fdf is often not literally named `<label>.fdf` (e.g. SystemLabel 'siesta' with the actual input file called `calc.fdf`) – `--label` still auto-detects the .WFSX in that case, only the geometry source changes.

`--k-index K_INDEX`
: 0-based index into the .WFSX's own k-point order (default: 0 if neither `--k-index` nor `--k-point` is given).type: int; exclusive group 1

`--k-point KX KY KZ`
: Match by k-point vector instead of index (same units as the .WFSX's own k, matched via sisl's read_eigenstate). type: float; exclusive group 1

`--spin {0,1}`
: Spin channel (default: 0; must be 0 for a non-polarized calculation). choices: 0, 1; type: int

`--band BAND`
: Which band at the chosen k: an integer N (1-based), or 'vbm'/'cbm' (requires `--fermi`, `--bands-file`, `--fermi-file`, or an auto-detected .out log – see below). required

`--fermi FERMI`
: Fermi energy (eV) for `--band` vbm/cbm. Highest-priority source if given. type: float

`--bands-file BANDS_FILE`
: Companion .bands file to read the Fermi energy from, for `--band` vbm/cbm. No correspondence guard against `--wfsx` like `stb-fatbands` has.

`--fermi-file FERMI_FILE`
: Explicit SIESTA .out log to read the Fermi energy from, for `--band` vbm/cbm – an alternative to `--bands-file` for a run with no saved .bands file. Not assumed to be named after `--label` (see below).

`--gap-tol GAP_TOL`
: Energy tolerance in eV for VBM/CBM classification (default: 0.01). Same meaning as `stb-bands/stb-fatbands`. type: float

`--spacing SPACING`
: Grid spacing in Angstrom (default: 0.3). Mutually exclusive with `--shape`. type: float; exclusive group 2

`--shape NX NY NZ`
: Explicit grid shape (voxel counts), alternative to `--spacing`. type: int; exclusive group 2

`--axis {0,1,2}`
: Axis for the optional `--save-gnuplot` slice/profile (default: 2). choices: 0, 1, 2; type: int

`--profile`
: Write a planar-averaged 1D profile instead of a 2D slice, for the optional `--save-gnuplot` output.switch, off by default

`-p, --pos POS`
: Position (Angstrom) along `--axis` of the 2D slice (ignored in `--profile` mode). Default: auto-detected at the plane where the planar-averaged |psi|^2 is largest – NOT the geometric center of the cell, which can land in empty vacuum for a structure whose atoms aren't centered along `--axis` (verified live: a real structure with atoms clustered near the cell boundary and vacuum in the middle had its default slice land on a point with ~1000x less density than the real peak, right at the atoms). type: float

`-o, --output-dir OUTPUT_DIR`
: Directory to write the .cube (and, with `--save-gnuplot`, the slice/profile data + report/references.bib) into (default: current directory). Created if it doesn't exist.

`--save-report`
: Also persist the full run report to `stb_wfdensity_report.txt`. Off by default. switch, off by default

`--save-gnuplot`
: Also write a 2D-slice or (with `--profile`) 1D-profile .dat + .gplot pair of the |psi|^2 density. Off by default – this tool used to write the .dat file (but never a .gplot script) unconditionally, opted out of via the old `--cube-only` flag (now removed). switch, off by default

`--view`
: Show an interactive matplotlib preview (2D slice or 1D profile, matching `--profile`) before finishing. Off by default – this tool previously had no matplotlib option at all. switch, off by default

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Example usage:
  stb-wfdensity --label siesta --k-index 0 --band 1
  stb-wfdensity --label siesta --band vbm --fermi -4.2
  stb-wfdensity --label siesta --band cbm --save-report --save-gnuplot --view
```

8.13 STS Spectroscopy Simulator

At a glance

Menu code: 2.13 **Command:** `stb-sts`
Guided example: `examples/2.13-sts/`
Summary: Simulated dI/dV(E) spectroscopy curve at a fixed tip point from a full-BZ .WFSX.

8.13.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/2.13-sts/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

`stb-sts` simulates a scanning tunneling spectroscopy (STS) curve — dI/dV as a function of energy — at **one fixed point in real space**, from a SIESTA full-Brillouin-zone `.WFSX` file. It answers the question "if I parked a tip at this exact (x, y, z) and swept the bias voltage, what would the tunneling spectrum look like?" — the energy-resolved twin of `stb-stm`'s spatial STM images.

This example walks through the theory, the real-world file conventions you need to get right (a full-BZ `.WFSX`, not a band-path one), and two real bugs found and fixed while reviewing this tool — using a real SIESTA calculation of monolayer graphene.

8.13.1.1 Theory: the Tersoff-Hamann proxy, extended to an energy axis

What a real STS measures. In a scanning tunneling microscope, the tunneling current at bias voltage `V` and tip position **r** is, in the simplest (Tersoff-Hamann, s-wave tip) approximation, proportional to the **integrated local density of states**:

```
I(V, r) ~  integral_0^{eV}  LDOS(r, E) dE
```

so the differential conductance dI/dV is directly proportional to the LDOS itself at that one bias energy:

```
dI/dV(r, E) ~ LDOS(r, E) = sum_{n,k} |psi_{n,k}(r)|^2 * delta(E - eps_{n,k})
```

`stb-stm` already uses this exact same LDOS proxy for a **spatial** STM image: fix the energy window, scan `r` over a grid. `stb-sts` does the opposite reduction: fix `r` at one point, scan `E`. Conceptually it is "PDOS projected onto a single real-space point instead of onto atomic orbitals" — the delta function is Gaussian-broadened (same `--sigma`/`--fwhm` convention as `stb-dos`/`stb-coop`) and every `(k, band, spin)` eigenstate is weighted by its own Brillouin-zone weight, exactly the k-mesh sum a real DOS/LDOS needs.

Where this tool sits among the suite's other real-space/energy tools. This suite has four tools that each keep a different pair of "real space" and "energy" fixed vs. integrated. It's worth seeing them side by side — each is genuinely useful for a different question:

Tool	Real space (r)	Energy (E)	Answers
`stb-wfdensity`	full 3D grid	ONE state (k, band)	"What does this one Bloch state look like?"
`stb-dos`/`stb-coop`	integrated over all r	full curve, BZ-summed	"How many states (or bonds) exist at each E?"
`stb-stm`	full 2D/3D grid	ONE window (near E_F)	"What does the surface look like to a tip at fixed bias?"
`stb-sts` (here)	ONE fixed point	full curve, BZ-summed	"What spectrum would a tip see, parked right here?"

`stb-sts` and `stb-stm` are the two members of this family that actually use the Tersoff-Hamann tunneling proxy (they simulate a tip); `stb-dos`/ `stb-coop` are pure electronic-structure quantities with no tip model at all. `stb-wfdensity` is the only one of the four that looks at a single eigenstate rather than a BZ-summed quantity.

1.3 Why a full-BZ `.WFSX`, not a band-path one. `stb-fatbands` reads a `.WFSX` written along a **band path** (`WFS.Write.For.Bands T` + `%block BandLines`) — perfect for a band structure, useless for a DOS-like quantity, since it isn't a real, properly-weighted Brillouin-zone sample. `stb-sts` needs the OTHER mechanism: `WriteWaveFunctions T` + an explicit `%block WaveFuncKPoints` listing a genuine k-mesh, which SIESTA names `<label>.selected.WFSX` by its own convention. This example's fixture (`calc.fdf`) uses exactly that — a modest 9-point mesh (spanning one quadrant of the graphene Brillouin zone, `kx, ky in {0, 0.25, 0.5}` in reciprocal-lattice units) is enough for a didactic, fast-to-run demo; a real study would want a much denser mesh for a converged spectrum, the same way a real DOS calculation needs a converged k-grid.

Confirmed live while building this fixture: `WriteWaveFunctions T` alone, with no `WaveFuncKPoints` block, silently writes nothing at all — SIESTA needs the explicit k-point list to know which states to save.

The real per-orbital basis requirement. Just like `stb-wfdensity`, evaluating `psi(r)` at an arbitrary point needs the orbitals' actual numerical radial shape — only available from a real `.fdf` (with its `.ion`/`.ion.xml` files alongside it), never from a bare `.XV`/`.HSX`. This example's fixture confirms the same asymmetry already verified for `stb-wfdensity`: `calc.fdf` gives 26 real orbitals (13 per carbon atom in this DZP basis: 2 radial functions each for 2s/2p, plus a polarization d-shell), while `Graphene.XV` alone carries none.

The confined-basis cutoff radius, verified live. SIESTA's numerical atomic orbitals (PAOs) are strictly zero beyond a finite cutoff radius — unlike a plane-wave or Gaussian basis, this is not a numerically-small tail, it is *exactly* zero. For this fixture's carbon DZP basis:

```
>>> max(a.maxR() for a in geometry.atoms)
2.576278646253102   # Ang
```

Sweeping `--height` above the graphene sheet (topmost/only atomic plane at Z = 10.000 Ang, half the 20 Ang cell) confirms this exactly:

`--height` (Ang)	Absolute Z (Ang)	Peak dI/dV
1.5	11.5	3.25e-02 (real signal)
3.0	13.0	0.0 (beyond the 2.576 Ang cutoff)
5.0 - 15.0	15.0 - 25.0	0.0

`--height 3.0` already lands 0.42 Ang beyond the cutoff radius and gives an exactly-flat, zero curve with no error — `stb-sts` warns about this (`[WARNING] |psi(r_tip)|^2 is exactly zero for every state ...`), but it's easy to miss if you're not watching for it. Always keep `--height` a couple of Angstrom at most for a typical DZP basis.

A real, verified physics finding: the vacuum axis is still periodic. This one is worth knowing before trusting a large `--height`. SIESTA's "vacuum" axis in a slab calculation is a real periodic lattice vector — just padded with empty space, not actually non-periodic. `stb-sts` evaluates `psi(r_tip)` by translating the geometry so `r_tip` lands at the origin of a periodic grid; if `r_tip` sits far enough along the vacuum axis, it silently wraps around to the **next periodic image of the surface**, not to "farther into empty space."

Verified live on this exact fixture (cell length along Z = 20 Ang, atoms at Z = 10 Ang, so the periodic image of the atomic plane recurs at Z = 10 + 20 = 30 Ang, i.e. `--height` = 20):

`--height` (Ang)	Absolute Z (Ang)	Peak dI/dV	What's really being sampled
8.0 - 15.0	18.0 - 25.0	0.0	genuine vacuum (beyond the PAO cutoff either way)
19.0	29.0	2.46e-01	wrapping back toward the periodic image at Z=30
20.0	30.0	3.90e-01	**exactly** the periodic image of the atomic plane

Both the near-tip region (Z = 11.5-13) and the near-periodic-image region (Z = 29-30) give strong, non-zero signal for the same underlying reason (both are within ~2.6 Ang of a real carbon atom — one is the original, the other its periodic copy). A large `--height` does NOT probe some Platonic "far from any surface" limit; it eventually samples the *other* side of the same periodic slab. In practice this only matters if your vacuum padding is small relative to `--height` — this suite's own `stb-slab`/`stb-2Dstacking` typically pad with 15-20+ Ang of vacuum specifically so this doesn't bite a normal STM/STS height (a few Ang), but it's worth keeping in mind if you ever go looking for the true vacuum plateau by cranking `--height` up.

8.13.1.2 Output and options

- **Numbered `[0]...[6]` report** (`RUN METADATA`, `INPUT DATA`, `TIP POSITION`, `STS CURVE`, `OUTPUT DATA & PLOTS`, `REFERENCES`, `SUMMARY & FILES`), matching `stb-bands`/`stb-dos`/`stb-wfdensity`.
- **`--save-report`** persists the full report to `stb_sts_report.txt` (off by default).
- **`--save-gnuplot`** writes `sts.dat` and a working `.gplot` script together, both behind this one off-by-default flag.
- **`--view`** shows the matplotlib preview (off by default).
- **`--shift` `fermi`'s Fermi-energy source** follows a priority-ordered hierarchy, the same one as `stb-wfdensity`'s `--band vbm/cbm`: `--fermi` (explicit) > `--bands-file` > `--fermi-file` > an auto-detected `.out` log in the current directory (not assumed to be named `<label>.out` -- many real SIESTA jobs redirect stdout to a generic name instead).
- **`--label` can be combined with `--geometry-file`.** This example's own fixture shows why that

matters: `SystemLabel` is `Graphene`, but the real input file is `calc.fdf` — there is no `Graphene.fdf` anywhere.
- **The surface-normal axis-alignment check** (shared with `stb-stm`, for a sheared cell where "height above the surface" along one axis isn't simply Cartesian) prints its result, exactly like `stb-stm` does.
- **The `--xy`/`--height` tip height is measured from the true topmost atom.** A plain `xyz[:, axis].max()` picks the wrong bounding atom for a structure whose atoms straddle the periodic cell boundary (verified on a real CrS monolayer with atoms at fractional Z = 0, 0, 0.066, 0.934 — see the `2.11-stm` example for that live verification), so `stb-sts` uses the same gap-aware surface reference `stb-stm` does. This example's own graphene fixture has its atoms exactly centered (fractional Z = 0.5 for both), so the naive and gap-aware methods agree here — the gap-aware method is defense against structures that aren't this well-behaved, not something this particular fixture can show diverging.
- The interactive `stb-suite` menu (item `2.13`) asks for the label and the `.fdf` path separately, and has an energy-shift submenu offering the same Fermi-source options — see the "Two ways to run it" section below.

8.13.1.3 Known limitations

- **Not a full STS simulation** — the tip is a structureless point probe (same philosophy as `stb-stm`'s constant-height/current modes), not a real tip-orbital/decay model.
- **No `--shift vbm/cbm`** (unlike `stb-fatbands`/`stb-wfdensity`) — would need a second, expensive full pass over the whole `.WFSX` just to find the extremum; use `--shift fermi` directly.
- **Only the first spinor component is evaluated** — non-collinear/SOC wavefunctions aren't fully represented (this fixture is a plain non-polarized calculation, so this doesn't matter here).
- **Cost scales as O(n_k * n_bands)** point-wavefunction evaluations — 234 contributions for this tiny 9 k-point / 26-orbital fixture already takes a couple of seconds; a real, converged k-mesh will take considerably longer.
- **No WFSX-mesh-quality metadata** — `stb-sts` can only print an advisory if the k-point count/weights look suspiciously like a band-path file (few points, all equal weight), never block outright.

8.13.1.4 Files in this folder

File	What it is
`calc.fdf`	The real SIESTA input (`SystemLabel Graphene` -- note the mismatched filename, see section 2 above)
`structure.fdf`	`%included` geometry: 2-atom primitive hexagonal graphene cell, 20 Ang vacuum along Z
`C.ion/C.ion.xml`	The DZP carbon basis set (needed alongside `calc.fdf` for real orbital shapes)
`Graphene.selected.WFSX`	The full-BZ wavefunction file (9 explicit k-points via `WaveFuncKPoints`)
`Graphene.XV`	Geometry-only file (kept to demonstrate why it's NOT sufficient on its own -- no orbitals)
`example_2.13.sh`	This walkthrough's runnable script

8.13.1.5 Running it

```
bash example_2.13.sh
```

Or try the tool directly:

```
stb-sts --label Graphene --geometry-file calc.fdf \
    --xy 0 0 --height 1.5 --erange -5 5 --sigma 50 \
    --shift fermi --fermi-file calc.out \
    --save-report --save-gnuplot --view
```

8.13.1.6 What's next

- `stb-stm` (`2.11-stm/`) — the spatial-map twin of this tool: fix the energy window, scan real space instead of fixing real space and scanning energy.
- `stb-wfdensity` (`2.12-wfdensity/`) — a single eigenstate's full 3D `|psi|^2`, the tool this one shares its Fermi-source hierarchy and `--label`/`--geometry-file` fix with.
- `stb-dos` (`2.2-stb-dos/`) — the same BZ-summed, energy-resolved idea, but integrated over all of real space instead of evaluated at one point.

8.13.2 Implementation details

Formula.. For a tip at $\mathbf{r}_0$,

$$\frac{dI}{dV}(E) \;\propto\; \rho(\mathbf{r}_0, E) \approx \sum_{\mathbf{k}} w_{\mathbf{k}} \sum_{n} |\psi_{n\mathbf{k}}(\mathbf{r}_0)|^2 \, \frac{1}{\sigma\sqrt{2\pi}} e^{-(E-\epsilon_{n\mathbf{k}})^2/2\sigma^2},$$

evaluated on `-npoints` (default 400) energies in `-erange` (relative to the chosen energy zero), with the Gaussian width given by `-sigma` or `-fwhm` in meV (the convention of **`stb-convdos`**). The k-weights $w_{\mathbf{k}}$ are those stored in the `.WFSX`, so the file must sample the whole Brillouin zone (`WriteWaveFunctions T` plus a `%block WaveFuncKPoints` listing the mesh, which SIESTA names `<label>.selected.WFSX`); the tool warns when the k-points look like a band path.

The tip position.. Either `-point X Y Z` (Cartesian, Å) or `-xy X Y` with `-height`, a height above the topmost atom along `-axis`. Because SIESTA's orbitals vanish beyond their cutoff radii (about 2.5–3.5 Å for a typical DZP basis), a tip further than that from every atom sees exactly zero; keep the height small, or extend the basis (longer orbitals, or ghost atoms in the vacuum).

Energy zero.. `-shift fermi` uses the Fermi level from `-fermi`, `-bands-file`, `-fermi-file` or a log found automatically; `manual` uses `-manual-value`; `none` keeps absolute energies. Cost grows as (number of k-points)×(number of bands); start with a coarse mesh.

8.13.3 Command-line reference

8.13.3.1 `stb-sts`

Simulated STS dI/dV(E) spectroscopy curve at a fixed tip point, from a full-BZ SIESTA .WFSX.

Synopsis.

```
usage: stb-sts [-h] [--label LABEL] [--wfsx WFSX] [--geometry-file GEOMETRY_FILE]
               (--point X Y Z | --xy X Y) [--height HEIGHT] [--axis {0,1,2}]
               --erange EMIN EMAX [--npoints NPOINTS] [--shift {none,fermi,manual}]
```

```
[--fermi FERMI] [--bands-file BANDS_FILE] [--fermi-file FERMI_FILE]
[--manual-value MANUAL_VALUE] (--sigma SIGMA | --fwhm FWHM)
[-o OUTPUT_DIR] [--save-report] [--save-gnuplot] [--view] [-v]
[--no-intro]
```

Options

`--label LABEL`

SIESTA output label. Auto-detects `<label>.WFSX`, falling back to <label>.selected.WFSX then <label>.bands.WFSX, and, unless `--geometry-file` overrides it, `<label>.fdf` (with its .ion/.ion.xml files alongside it). Mutually exclusive with `--wfsx` (not `--geometry-file` – see its own help: the real fdf is very often NOT named `<label>.fdf`).

`--wfsx WFSX`

Explicit path to the .WFSX file (alternative to `--label`).

`--geometry-file GEOMETRY_FILE`

Explicit .fdf geometry path (must have the basis's .ion/.ion.xml files alongside it). Required if `--wfsx` is used instead of `--label`; optional (and common) together WITH `--label` too, since the real fdf is often not literally named `<label>.fdf` (e.g. SystemLabel 'siesta' with the actual input file called `calc.fdf`) – `--label` still auto-detects the .WFSX in that case, only the geometry source changes.

`--point X Y Z`

Absolute Cartesian tip position (Ang). type: float; exclusive group 1

`--xy X Y`

In-plane Cartesian position (Ang); needs `--height` too. `stb-stm-style`: the tip sits `--height` Ang above the topmost surface atom along the vacuum axis. type: float; exclusive group 1

`--height HEIGHT`

Height (Ang) above the topmost surface atom, for `--xy` mode. type: float

`--axis {0,1,2}`

Surface-normal axis for `--xy` mode. Auto-detected from the geometry's vacuum padding if omitted and exactly one vacuum axis is found. choices: 0, 1, 2; type: int

`--erange EMIN EMAX`

Energy window (eV), relative to `--shift`'s reference. required; type: float

`--npoints NPOINTS`

Number of energy samples (default: 400). type: int

`--shift {none,fermi,manual}`

Energy reference (default: none, i.e. raw eigenvalues as stored in the .WFSX). No vbm/cbm option here – see the module docstring. choices: none, fermi, manual

`--fermi FERMI`

Fermi energy (eV) for `--shift` fermi. Highest-priority source if given. type: float

`--bands-file BANDS_FILE`

Companion .bands file to read the Fermi energy from, for `--shift` fermi.

`--fermi-file FERMI_FILE`

Explicit SIESTA .out log to read the Fermi energy from, for `--shift` fermi – an alternative to `--bands-file` for a run with no saved .bands file. Not assumed to be named after `--label`.

`--manual-value MANUAL_VALUE`

Custom shift value (eV), required if `--shift` manual. type: float

`--sigma SIGMA`

Gaussian broadening standard deviation, in meV. type: float; exclusive group 2

`--fwhm FWHM`
: Gaussian broadening full width at half maximum, in meV. type: float; exclusive group 2

`-o, --output-dir OUTPUT_DIR`
: Directory to write `sts.dat` (and, with `--save-gnuplot`/`--save-report`, the .gplot script/report/references.bib) into (default: current directory). Created if it doesn't exist.

`--save-report`
: Also persist the full run report to `stb_sts_report.txt`. Off by default. switch, off by default

`--save-gnuplot`
: Also write a real .gplot script alongside `sts.dat`. Off by default – this tool used to write the .dat file unconditionally but never wrote a .gplot script at all. switch, off by default

`--view`
: Show an interactive matplotlib preview before finishing. Off by default (replaces the old `--no-plot`, which was on by default). switch, off by default

Like every STB command it also accepts `-h`/`–help`, `-v`/`–version` and `–no-intro` (Section 3.3).

From the built-in help.

```
Example usage:
  stb-sts --label siesta --xy 0 0 --height 3.0 --erange -3 3 --sigma 50
  stb-sts --label siesta --point 1.2 0.7 12.0 --erange -5 5 --fwhm 100
  stb-sts --label siesta --xy 0 0 --height 3.0 --erange -5 5 --sigma 50 --shift fermi --fermi-file calc.out
  --save-report --save-gnuplot --view
```

8.14 COOP/COHP Bonding Analysis

At a glance

Menu code: 2.14 **Command:** `stb-coop`
Guided example: `examples/2.14-coop/`
Summary: Energy-resolved Crystal Orbital Overlap/Hamilton Population curves for selected atom pairs.

8.14.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/2.14-coop/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

`stb-coop` computes energy-resolved **Crystal Orbital Overlap Population (COOP)** and **Crystal Orbital Hamilton Population (COHP)** curves between user-chosen atom pairs, from a SIESTA full-Brillouin-zone `.WFSX` + `.HSX`. Where `stb-dos` tells you *how many* electronic states exist at each energy, COOP/COHP tell you *whether those states are bonding or antibonding* between two specific atoms — the quantum-chemical bridge between a band structure and a picture of chemical bonds.

This example walks through the theory, and a real, serious bug found and fixed while reviewing this tool — using a real SIESTA calculation of the tin oxide Sn3O4.

8.14.1.1 Theory

What COOP/COHP actually measure. For a crystal orbital (Bloch state) `psi_{n,k} = sum_mu c_mu,n(k) phi_mu`, the overlap population between two atomic orbitals `mu` (on atom I) and `nu` (on atom J) is `c_mu,n(k) * c_nu,n(k) * S_{mu,nu}(k)` — a term that is **positive when the orbital combination is bonding** (in-phase, builds up density between the atoms) and **negative when it's antibonding** (out-of-phase, depletes density between the atoms). Summing this over every orbital pair between atoms I and J, over every band `n`, and Gaussian-broadening around each band's eigenvalue, gives a continuous curve:

```
COOP_IJ(E) = sum_(n,k) w_k * [sum_(mu in I, nu in J) c_mu,n(k) c_nu,n(k) S_mu,nu(k)] * delta(E - eps_n(k))
```

COHP replaces the overlap matrix `S` with the Hamiltonian matrix `H` in the same formula — a Hamilton-weighted version of the same idea (some literature plots `-COHP` so that positive also means bonding; this suite keeps sisl's own raw sign, i.e. **COOP/COHP > 0 = bonding**, verified below rather than just asserted).

This suite reuses sisl's own `EigenstateElectron.COOP()/.COHP()` (explicitly marked "experimental" in sisl's own docs), which computes one k-point at a time; `stb-coop` itself loops every k-point, weights by the `.WFSX`'s own Brillouin-zone weights, and reduces the resulting orbital-pair matrix down to the atom pairs you actually asked for.

1.2 Why a full-BZ `.WFSX`, and why a real `.HSX`. Same requirement as `stb-sts`: COOP/COHP is a Brillouin-zone-integrated, DOS-like quantity, so it needs a genuine k-mesh `.WFSX` (`WriteWaveFunctions T` + an explicit `%block WaveFuncKPoints`, named `<label>.selected.WFSX` by SIESTA's own convention) -- NOT the band-path `.WFSX` `stb-fatbands` uses. Unlike `stb-wfdensity`/`stb-sts`, there is also no approximate fallback here: COOP needs the real overlap matrix `Sk()` and COHP needs the real Hamiltonian `Hk()`, neither of which exists on a bare `.fdf`-derived `Geometry` — only a real `.HSX` carries them.

This example's fixture (`Sn3O4.selected.WFSX` + `Sn3O4.HSX`) is a real, non-polarized GGA-PBE (+DFT-D3) calculation of the tin oxide Sn3O4 (14 atoms, 182 orbitals, DZP basis), sampled at 8 explicit k-points (a 2x2x2-equivalent mesh).

A worked example: a real bond vs. a non-bond. This fixture's atom 0 (Sn) has a real Sn-O bond to atom 6 (O) at 2.077 Ang, and sits 3.800 Ang from atom 1 (another Sn) -- too far to be a real bond. Watch how differently these two pairs behave.

With a window that includes UNoccupied states (`--erange -10 5`, well above the real Fermi energy of -3.204 eV), the picture is misleading:

Pair	Largest |value|	Integrated over `--erange`
0-6 (Sn-O, bonded)	-1.101e-01 at E=3.75 eV	-1.490e-01 (**antibonding**)
0-1 (Sn-Sn, not bonded)	-1.508e-02 at E=5.00 eV	+7.507e-03 (bonding)

That looks backwards -- the real bond reads "antibonding" and the non-bond reads "bonding"! The reason: this window captures a lot of UNoccupied antibonding character for the real Sn-O bond (states above E_F that would be antibonding if they were ever filled), which dominates the integral. **Restricting the window to the actual occupied states** (`--erange -25 -3.204`, i.e. up to the real Fermi energy) gives the physically meaningful picture instead:

Pair	Largest |value|	Integrated over `--erange`
0-6 (Sn-O, bonded)	+2.861e-02 at E=-21.65 eV	**+1.330e-01 (bonding)**
0-1 (Sn-Sn, not bonded)	-3.501e-03 at E=-5.998 eV	**-7.660e-04 (essentially zero)**

Now it makes physical sense: the real bond shows net bonding character over the occupied states, and the non-bonded pair is essentially flat at zero. **The energy window you integrate over matters** -- `stb-coop`'s own `[3] COOP/COHP CURVE` report section prints this integrated value with an explicit reminder that it's over `--erange`, not automatically "the occupied states".

1.4 A real, serious bug found and fixed: `--bond-order`. `--bond-order` is meant to be a cheap, energy-INTEGRATED complementary sanity check on the COOP curve above: the **Mulliken bond order**, `B_IJ = 2 * sum M_ij * S_ij` (summed over orbitals), where `M` is the **density matrix**. This tool used to call `H.bond_order(...)` directly on the Hamiltonian object `H` -- but sisl's `bond_order()` just uses whatever sparse matrix `self` holds as `M` in that formula. Passing the Hamiltonian silently substituted `H_ij` (Hamiltonian matrix elements, in eV, tens of eV for on-site terms) for a density-matrix population (dimensionless, typically O(0.1-2)).

Verified live on this exact fixture, before the fix:

Pair	Old (broken) "bond order"
0-0 (Sn on-site)	-92.38
0-6 (Sn-O, REAL bond, 2.08 Ang)	-38.32
0-1 (Sn-Sn, NOT bonded, 3.80 Ang)	-29.04

A genuinely bonded pair and a genuinely non-bonded pair gave numbers of the same order of magnitude -- the smoking gun that this measured nothing about bonding at all, just Hamiltonian-matrix-element scale.

The fix: read a REAL density matrix (`--dm-file`, or an auto-detected `<label>.DM`) and call `bond_order()` on THAT instead. One more wrinkle needed fixing too: SIESTA's `.DM` file format doesn't store the overlap matrix at all (confirmed live: `DM.Sk()` gave an exactly-zero overlap trace right after a bare read) -- only `.HSX`/`.TSHS` do. So the already-loaded Hamiltonian's own overlap column is spliced into the freshly-read density matrix before calling `bond_order()`.

After the fix, on the exact same pairs:

Pair	Fixed bond order	Physical read
0-6 (Sn-O, bonded)	**+0.527**	a real, single-bond-scale value
0-1 (Sn-Sn, not bonded)	**-0.010**	essentially zero, as expected

...and, as section 1.3 already showed, the energy-integrated COOP curve (a COMPLETELY different computation -- from eigenstate wavefunction coefficients, not the density matrix) **agrees in sign** with the fixed bond order for both pairs (+0.133 vs +0.527 for the bond, -0.0008 vs -0.010 for the non-bond). Two independent methods agreeing is real evidence, not just an assertion, that this suite's COOP sign convention (positive = bonding) is correct.

8.14.1.2 Output and options

- **Numbered `[0]...[7]` report** (`RUN METADATA`, `INPUT DATA`, `PAIR SELECTION`, `COOP/COHP CURVE`, `BOND ORDER`, `OUTPUT DATA & PLOTS`, `REFERENCES`, `SUMMARY & FILES`), matching `stb-wfdensity`/`stb-sts`.
- **`--save-report`** persists the report to `stb_coop_report.txt` (off by default).

- **`--save-gnuplot`** writes `coop.dat`/`cohp.dat` and a real multi-pair gnuplot script (one curve per selected pair, via `columnheader()`), together behind one off-by-default flag.
- **`--view`** shows the matplotlib preview (off by default).
- **`--shift fermi`'s Fermi-energy source** follows the same priority-ordered hierarchy as `stb-wfdensity`/`stb-sts` (`--fermi` > `--bands-file` > `--fermi-file` > an auto-detected `.out` log).
- **`--label` can be combined with `--hsx-file`**, which takes precedence over `<label>.HSX`.
- **`--bond-order`** uses a real density matrix (`--dm-file`/auto-detected `<label>.DM`), not the Hamiltonian -- see section 1.4 above. It errors clearly if no usable `.DM` is found, instead of silently returning a wrong number.
- The interactive `stb-suite` menu (item `2.14`) asks for the label and `.HSX` path separately, has a Fermi-source submenu, and a bond-order/`.DM` prompt.

8.14.1.3 Known limitations

- sisl's COOP/COHP API is explicitly "experimental" and memory-heavy for many energy points/large systems -- start modest with `--npoints`.
- No metadata distinguishes a full-BZ-mesh `.WFSX` from a band-path one; `stb-coop` only prints an advisory (few k-points, all equal weight), never blocks.
- No `--shift vbm/cbm` (would need a second, expensive full pass over the whole `.WFSX`) -- use `--shift fermi` instead.
- Not tested for non-collinear states (sisl's own docstring caveat, passed through unchanged).

8.14.1.4 Files in this folder

File	What it is
`Sn3O4.HSX`	The real Hamiltonian + overlap matrix (mandatory -- no approximate fallback)
`Sn3O4.selected.WFSX`	The full-BZ wavefunction file (8 explicit k-points via `WaveFuncKPoints`)
`Sn3O4.DM`	The real density matrix, needed for `--bond-order` (see section 1.4)
`Sn3O4.XV`	Geometry-only file (not actually needed by `stb-coop`, kept for reference)
`example_2.14.sh`	This walkthrough's runnable script

8.14.1.5 Running it

```
bash example_2.14.sh
```

Or try the tool directly:

```
stb-coop --label Sn3O4 --quantity coop --pair 0 6 --pair 0 1 \
    --erange -25 -3.204 --sigma 300 --bond-order \
    --save-report --save-gnuplot --view
```

8.14.1.6 What's next

- `stb-sts` (`2.13-sts/`) -- the other Brillouin-zone-integrated, energy-resolved tool, with the same Fermi-source hierarchy.
- `stb-dos`/`stb-ipr` -- related energy-resolved electronic-structure quantities, without the atom-pair bonding character COOP/COHP add.
- `stb-fatbands` (`2.10-stb-fatbands/`) -- orbital-projected band structure, the band-path analog (not BZ-integrated) of the same per-orbital coefficients COOP/COHP consume here.

8.14.2 Implementation details

COOP and COHP

For a pair of atoms A, B the crystal orbital overlap population [51] and the crystal orbital Hamilton population [52] are

$$\mathrm{COOP}_{AB}(E) = \sum_{\mathbf{k}} w_{\mathbf{k}} \sum_{n} \sum_{\mu\in A,\nu\in B} \mathrm{Re}\left[c^{*}_{\mu n\mathbf{k}} S_{\mu\nu}(\mathbf{k}) c_{\nu n\mathbf{k}}\right] \delta(E-\epsilon_{n\mathbf{k}}),$$

and the same with $H_{\mu\nu}(\mathbf{k})$ in place of $S_{\mu\nu}(\mathbf{k})$ for COHP. Positive COOP (negative COHP) means bonding, negative COOP (positive COHP) antibonding; energy integrals up to E_F measure the net bond strength. Plots often show $-$COHP so that bonding points the same way as COOP.

How it is computed.. sisl [10] returns, for one k-point at a time, the orbital-pair- and energy-resolved matrices over the whole auxiliary supercell, so bonds to periodic images are included. The tool loops over the k-points of a full-Brillouin-zone `.WFSX`, weights them with the file's k-weights, and sums the rows and columns of the requested atoms: `-pair I J` (0-based, repeatable; $I = J$ gives the on-site term) or `-pair-species A B` (all pairs between two species, including on-site terms when $A = B$). The energy grid is `-erange` with `-npoints` points (default 300), each state broadened with `-sigma`/`-fwhm` (meV). It needs `<label>.HSX`: COOP uses the overlap and COHP the Hamiltonian.

Bond order.. `-bond-order` adds the Mulliken bond order $2\sum_{\mu\in A,\nu\in B} D_{\mu\nu}S_{\mu\nu}$ computed from the density matrix (`<label>.DM`, or `-dm-file`), with the overlap taken from the `.HSX` because the `.DM` file does not store it. Its sign agrees with the integrated COOP and gives an independent check.

Cautions.. sisl marks this API as experimental and it can use a lot of memory: start with a small window and few points. Non-collinear states are not tested. COOP/COHP depend on the basis (they are not basis-set invariant): compare pairs within one calculation, or calculations with the same basis.

8.14.3 Command-line reference

8.14.3.1 `stb-coop`

Energy-resolved COOP/COHP bonding/antibonding curves for selected atom pairs, from a full-BZ SIESTA .WFSX + .HSX.

Synopsis.

```
usage: stb-coop [-h] [--label LABEL] [--wfsx WFSX] [--hsx-file HSX_FILE]
                --quantity {coop,cohp} [--pair I J] [--pair-species A B]
                --erange EMIN EMAX [--npoints NPOINTS] [--shift {none,fermi,manual}]
                [--fermi FERMI] [--bands-file BANDS_FILE] [--fermi-file FERMI_FILE]
                [--manual-value MANUAL_VALUE] (--sigma SIGMA | --fwhm FWHM)
                [--atol ATOL] [--bond-order] [--dm-file DM_FILE] [-o OUTPUT_DIR]
                [--save-report] [--save-gnuplot] [--view] [-v] [--no-intro]
```

Options

`--label LABEL`
: SIESTA output label. Auto-detects `<label>.WFSX`, falling back to <label>.selected.WFSX then <label>.bands.WFSX, and, unless `--hsx-file` overrides it, `<label>.HSX` (mandatory – no approximate fallback). Mutually exclusive with `--wfsx` (not `--hsx-file` – see its own help).

`--wfsx WFSX`
: Explicit path to the .WFSX file (alternative to `--label`).

`--hsx-file HSX_FILE`
: Explicit .HSX path. Required if `--wfsx` is used instead of `--label`; optional (and common) together WITH `--label` too, since the real file is often not literally named `<label>.HSX` – `--label` still auto-detects the .WFSX in that case, only the Hamiltonian source changes.

`--quantity {coop,cohp}`
: 'coop': Crystal Orbital Overlap Population. 'cohp': Crystal Orbital Hamilton Population.required; choices: coop, cohp

`--pair I J`
: 0-based atom-index pair (repeatable). I == J is allowed (on-site term). type: int

`--pair-species A B`
: Species-pair (repeatable), e.g. `--pair-species` Sn O – aggregates ALL atom-index pairs between the two species into one combined curve. If A == B (e.g. Sn Sn), the aggregate also includes each atom's own on-site term, mixed in with the inter-atomic Sn-Sn terms – use `--pair` I J for a purely inter-atomic curve instead.

`--erange EMIN EMAX`
: Energy window (eV), relative to `--shift`'s reference. required; type: float

`--npoints NPOINTS`
: Number of energy samples (default: 300). sisl's COOP/COHP is memory-heavy for many points – start modest. type: int

`--shift {none,fermi,manual}`
: Energy reference (default: none). No vbm/cbm option – see the module docstring.choices: none, fermi, manual

`--fermi FERMI`
: Fermi energy (eV) for `--shift` fermi. Highest-priority source if given. type: float

`--bands-file BANDS_FILE`
: Companion .bands file to read the Fermi energy from, for `--shift` fermi.

`--fermi-file FERMI_FILE`
: Explicit SIESTA .out log to read the Fermi energy from, for `--shift` fermi – an alternative to `--bands-file` for a run with no saved .bands file. Not assumed to be named after `--label`.

`--manual-value MANUAL_VALUE`
: Custom shift value (eV), required if `--shift` manual. type: float

`--sigma SIGMA`
: Gaussian broadening standard deviation, in meV. type: float; exclusive group 1

`--fwhm FWHM`
: Gaussian broadening full width at half maximum, in meV. type: float; exclusive group 1

`--atol ATOL`
: Per-state energy-window cutoff passed straight to sisl's COOP/COHP (default: 1e-10) – a performance knob, not a physics tolerance. type: float

`--bond-order`
: Also compute Hamiltonian.bond_order(method='mulliken', projection='atom') per pair from a REAL density matrix – a cheap, energy-INTEGRATED complementary sanity check, not a substitute for the curve. Needs `--dm-file` or an auto-detected `<label>.DM` (see below). switch, off by default

`--dm-file DM_FILE`
: Explicit .DM density-matrix path, for `--bond-order`. Falls back to an auto-detected `<label>.DM` if omitted. SIESTA's .DM format doesn't store the overlap matrix – it's spliced in from the already-loaded Hamiltonian, so the .DM must be from the same calculation as the .HSX/–hsx-file.

`-o, --output-dir OUTPUT_DIR`
: Directory to write coop.dat/cohp.dat (and, with `--save-gnuplot`/`--save-report`, the .gplot script/report/references.bib) into (default: current directory). Created if it doesn't exist.

`--save-report`
: Also persist the full run report to `stb_coop_report.txt`. Off by default. switch, off by default

`--save-gnuplot`
: Also write a real .gplot script alongside coop.dat/cohp.dat. Off by default – this tool used to write the .dat file unconditionally but never wrote a .gplot script at all. switch, off by default

`--view`
: Show an interactive matplotlib preview before finishing. Off by default (replaces the old `--no-plot`, which was on by default). switch, off by default

Like every STB command it also accepts `-h`/`–help`, `-v`/`–version` and `–no-intro` (Section 3.3).

From the built-in help.

```
Example usage:
  stb-coop --label siesta --quantity coop --pair 0 1 --erange -10 5 --sigma 200
  stb-coop --label siesta --quantity cohp --pair-species Sn O --erange -10 5 --fwhm 300
  stb-coop --label siesta --quantity coop --pair 0 1 --erange -10 5 --sigma 200 \
      --bond-order --save-report --save-gnuplot --view
```

8.15 IPR Analyzer

At a glance

Menu code: 2.15 **Command:** `stb-ipr`
Guided example: `examples/2.15-ipr/`
Summary: Inverse Participation Ratio (localization measure) scattered onto a band structure.

8.15.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/2.15-ipr/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

`stb-ipr` scatters one number per (k, band) onto a band structure: the **Inverse Participation Ratio (IPR)**, a standard diagnostic for whether an electronic state is *localized* (weight concentrated on a few atoms) or *extended* (weight spread evenly across the whole cell). It reuses exactly the same input files and orbital-projection machinery as `stb-fatbands` — this example builds directly on that one.

8.15.1.1 Theory

What IPR measures. For a Bloch state `psi_n(k) = sum_i c_i` (sum over orbitals `i`, weighted by the real, non-orthogonal overlap matrix), the generalized IPR is:

```
I_q(k, band) = sum_i |psi_i|^(2q) / [sum_i |psi_i|^2]^q
```

which, for a properly normalized state (`sum_i |psi_i|^2 = 1`), reduces to simply `sum_i |psi_i|^(2q)` — this suite's own module docstring formula. Verified directly against sisl's source (`EigenstateElectron.ipr()`):

```
state_abs2 = self.norm2(projection="hadamard").real
return (state_abs2**q).sum(-1) / state_abs2.sum(-1)**q
```

`norm2(projection="hadamard")` is the **exact same per-orbital quantity** `stb-fatbands` already uses for its own orbital weights — IPR is really just a different reduction of the same underlying numbers `stb-fatbands` scatters by orbital/atom/species category. That's why `stb-ipr` shares `stb-fatbands`' entire `.bands/.WFSX` loading, cross-check, and `.HSX`-accurate/`.fdf`-approximate machinery (`core/band_scatter.py` was literally extracted from `fatbands.py` once `ipr.py` became its second consumer).

The classic Anderson-localization interpretation, for `q=2` (the default, and the only value sisl's `ipr()` will accept below 2 — it hard-asserts `q >= 2` internally):

```
lim (L -> infinity) I_2 =  1/L^d    for an EXTENDED state (d = dimensionality)
                         = const.   for a LOCALIZED state
```

A state with weight on just 1 of N orbitals has `I_2 = 1`; a state spread perfectly evenly over all N has `I_2 = 1/N`. **There is no universal absolute scale** — the value depends on orbital count/cell size, so only compare IPR values within the same system/basis, never across different structures.

Verified live: deep states are more localized than near-Fermi ones. Running on this example's real Sn3O4 fixture (the same one `2.10-stb-fatbands/` uses):

```
$ stb-ipr --file Sn3O4.bands --wfsx Sn3O4.bands.WFSX --hsx-file Sn3O4.HSX --shift fermi
```

Series	States	Mean IPR	Min IPR (most extended)	Max IPR (most localized)
ipr	2912	0.0493	0.0172 (E=51.99 eV)	0.3056 (E=75.23 eV)

Deep, high-energy core-like states (E > 50 eV above the shifted Fermi reference, from this DZP basis's own semi-core-like orbitals) average **~0.107** mean IPR; states within 2 eV of the Fermi level average only **~0.052** — deep states really are more localized than the itinerant, bonding-derived states near the Fermi level, exactly the physically expected picture for a real oxide's electronic structure.

1.3 The `--q` order parameter: higher q sharpens the localized/extended contrast.

```
$ stb-ipr ... --q 2   ->  mean 0.0493, max 0.3056, min 0.0172
$ stb-ipr ... --q 4   ->  mean 0.0008, max 0.0832, min 0.0000
$ stb-ipr ... --q 6   ->  mean 0.0001, max 0.0240, min 0.0000
```

As `q` increases, every value shrinks (raising fractions below 1 to a higher power), but the **contrast** sharpens: the most-extended state's IPR collapses toward exactly 0 much faster than the most-localized state's does. Higher `q` is a cheap way to emphasize which states are *really* localized versus merely "somewhat concentrated" at `q=2`.

Accuracy: real overlap matrix vs. the orthogonal-basis approximation. Just like `stb-fatbands`, `stb-ipr` needs a real `.HSX` (Hamiltonian + overlap) for a physically correct, overlap-weighted `norm2`; without one it falls back to an implicit-orthogonal approximation (`|c|^2`, no true `S` matrix) with a printed warning. Verified live on the same fixture, same states:

Source	Mean IPR
`Sn3O4.HSX` (accurate, overlap-aware)	0.04930
`calc.fdf` only (approximate, no `.HSX`)	0.04356

A genuine, ~12% difference — not a rounding-level discrepancy — confirming the accuracy warning is worth heeding whenever a real `.HSX` is available.

8.15.1.2 Output and options

- **Numbered `[0]...[6]` report** (`RUN METADATA`, `INPUT DATA`, `BAND GAP ANALYSIS`, `IPR ANALYSIS`, `WRITING OUTPUT FILES`, `REFERENCES`, `SUMMARY & FILES`) — the `IPR ANALYSIS` section reports per-series mean/min/max IPR AND which exact (k, energy) state is the most localized/most extended.
- **`--save-report`**, **`--save-gnuplot`** (writes `ipr.dat`/`ipr.gplot`) and **`--view`** (matplotlib preview) are all off by default.
- **`--label` can be combined with `--hsx-file`/`--geometry-file`**: an explicit `--hsx-file` is preferred over `<label>.HSX`. Only `--label` + `--file`/`--wfsx` is rejected (ambiguous).
- **`--q` must be at least 2**: sisl's own `ipr()` asserts `q >= 2`, so `--q 1`/0/negative is rejected up front with a clear error instead of a raw `AssertionError`.
- **nspin=2 keeps the two spin channels separate** — see section 3 below for the live verification.

`stb-ipr` has no Fermi-energy source hierarchy (unlike `stb-wfdensity/` `stb-sts`/`stb-coop`): like `stb-fatbands`, it already requires a `.bands` file as its primary input, which carries its own embedded Fermi energy — there is nothing to decouple.

8.15.1.3 A real bug found and fixed: spin channels silently merged

This tool's row-building loop used to dump BOTH spin channels into one flat, spin-blind list — for a genuinely magnetic system this silently combines two physically different IPR series into one indistinguishable scatter/data file (the exact same bug `stb-fatbands` already found and fixed for its own orbital weights). Fixed the same way: `nspin=2` now splits into `ipr_up`/`ipr_down`, written as separate `.dat` files and reported as separate table rows; `nspin=1` is completely unaffected (still plain `ipr`).

Verified live against a real spin-polarized calculation (a single oxygen atom in a large vacuum box, seeded to converge to its physical 2 Bohr-magneton triplet ground state — the exact same fixture `2.10-stb-fatbands/spin/` already uses and verified):

```
$ stb-ipr --label Ospin --shift fermi
```

Series	Mean IPR
ipr_up	0.9515
ipr_down	1.0870

A real, physically meaningful difference (the two spin channels see a different effective potential in a magnetic atom) that used to be silently averaged away into one merged series. (Both values sitting close to/slightly above 1 makes sense here too — with only 13 orbitals total for a single, near-isolated

atom, several states really are close to fully localized on one or two orbitals; values fractionally above the theoretical q=2 maximum of exactly 1.0 reflect small numerical noise in the overlap-weighted normalization, not a code defect.)

8.15.1.4 Known limitations

- No file metadata ties a given `.WFSX` to a given `.bands` file — same k-count/orbital-count guard and `--k-tol` eigenvalue cross-check as `stb-fatbands`.
- IPR values are basis-size-dependent, never directly comparable across different structures/basis sets (section 1.1 above).
- Non-collinear/SOC (nspin 4/8) inputs get a raw `_s{n}` per-channel suffix, same fallback `dos.py`/`fatbands.py` use — not specially handled or verified beyond that.

8.15.1.5 Files in this folder

File	What it is
`Sn3O4.bands/Sn3O4.bands.WFSX/Sn3O4.HSX`	The real band-path calculation (same fixture as `2.10-stb-fatbands/`)
`calc.fdf/structure.fdf/Sn.ion(.xml)/O.ion(.xml)`	The structure/basis this calculation used
`spin/Ospin.*`	A real spin-polarized isolated-O-atom calculation, for the nspin=2 section
`example_2.15.sh`	This walkthrough's runnable script

8.15.1.6 Running it

```
bash example_2.15.sh
```

Or try the tool directly:

```
stb-ipr --file Sn3O4.bands --wfsx Sn3O4.bands.WFSX --hsx-file Sn3O4.HSX \
    --shift fermi --q 4 --save-report --save-gnuplot --view
```

8.15.1.7 What's next

- `stb-fatbands` (`2.10-stb-fatbands/`) — the direct sibling this tool shares its entire input/loading/accuracy machinery with; orbital projection instead of a single localization measure.
- `stb-dos` (`2.2-stb-dos/`) — a complementary, energy-only view of the same electronic structure, with no per-state localization information.

8.15.2 Implementation details

Inverse participation ratio

With the overlap-aware orbital weights $w_{\mu,n\mathbf{k}}$ of **stb-fatbands** (Section 8.10),

$$\mathrm{IPR}_q(n\mathbf{k}) = \frac{\sum_\mu w_{\mu,n\mathbf{k}}^q}{\left(\sum_\mu w_{\mu,n\mathbf{k}}\right)^q}, \qquad q \geq 2.$$

A state spread evenly over N orbitals gives $\mathrm{IPR}_2 = 1/N$; a state on a single orbital gives 1. It is the standard diagnostic of localization [53]; larger -q emphasizes the most localized states.

Use.. The IPR of every (k, band) of a band path is scattered onto the band structure, as marker size and colour, using the same `.bands`/`.bands.WFSX`/`.HSX` inputs and pairing checks as **stb-fatbands**; spin-polarized runs give separate `ipr_up`/`ipr_down` series. Typical uses: spotting defect or surface states in a band structure, distinguishing localized from band-like states near the gap.

Scale.. Values depend on the number of orbitals in the cell and on the basis, so compare IPRs within one calculation, or between calculations of the same cell and basis, never across different systems.

8.15.3 Command-line reference

8.15.3.1 stb-ipr

Inverse Participation Ratio (IPR) scattered onto a band structure, from .bands + .WFSX.

Synopsis.

```
usage: stb-ipr [-h] [--label LABEL] [--file INPUT_FILE] [--wfsx WFSX]
               [--hsx-file HSX_FILE] [--geometry-file GEOMETRY_FILE]
               --shift {vbm,cbm,fermi,manual} [--manual-value MANUAL_VALUE]
               [--gap-tol GAP_TOL] [--q Q] [--k-tol K_TOL] [-o OUTPUT_DIR]
               [--save-report] [--save-gnuplot] [--view] [-v] [--no-intro]
```

Options

`--label LABEL`
: SIESTA output label. Shorthand for `--file <label>.bands`, auto-detects <label>.bands.WFSX (falling back to `<label>.WFSX` then <label>.selected.WFSX, with a note), and, unless `--hsx-file`/`--geometry-file` override it, auto-detects `<label>.HSX` (optional, upgrades weight accuracy) then `<label>.fdf` (geometry+basis fallback). Mutually exclusive with `--file`/`--wfsx` (not `--hsx-file`/`--geometry-file` – see their own help).

`--file INPUT_FILE`
: Explicit path to the .bands file (alternative to `--label`).

`--wfsx WFSX`
: Explicit path to the .WFSX file. Required if `--file` is used instead of `--label`.

`--hsx-file HSX_FILE`
: Optional explicit .HSX path for physically correct overlap-weighted IPR. Falls back to `--geometry-file` / auto-detected .fdf with an accuracy warning if omitted. Optional (and common) together WITH `--label` too, since the real file is often not literally named `<label>.HSX`.

`--geometry-file GEOMETRY_FILE`
: Optional explicit .fdf geometry path, used as the fallback parent when no .HSX is available. Requires sisl.

`--shift {vbm,cbm,fermi,manual}`
Reference energy shift, same vocabulary as `stb-bands/stb-fatbands`. required; choices: vbm, cbm, fermi, manual

`--manual-value MANUAL_VALUE`
Custom energy shift value (required if `--shift` manual is used). type: float

`--gap-tol GAP_TOL`
Energy tolerance in eV for VBM/CBM classification when `--shift` is vbm/cbm (default: 0.01). type: float

`--q Q`
IPR order parameter (default: 2, the standard choice). Passed straight to sisl's EigenstateElectron.ipr(q=...), which requires q >= 2. Higher q emphasizes localization more strongly. type: int

`--k-tol K_TOL`
Cross-check tolerance (eV) between WFSX-stored eigenvalues and .bands eigenvalues at sampled k-points (default: 0.001). Same file-order sanity guard as `stb-fatbands`. type: float

`-o, --output-dir OUTPUT_DIR`
Directory to write ipr.dat/ipr.gplot (with `--save-gnuplot`) and stb_ipr_report.txt/references.bib into (default: current directory). Created if it doesn't exist.

`--save-report`
Also persist the full run report to `stb_ipr_report.txt`. Off by default. switch, off by default

`--save-gnuplot`
Also write `ipr.dat` (or ipr_up.dat/ipr_down.dat for a spin-polarized run) + `ipr.gplot`. Off by default – this tool used to write both unconditionally on every run. switch, off by default

`--view`
Show an interactive matplotlib preview before finishing. Off by default – this tool previously always showed it, with no way to skip. switch, off by default

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Example usage:
  stb-ipr --label siesta --shift fermi
  stb-ipr --label siesta --shift fermi --q 4
  stb-ipr --label siesta --shift fermi --save-report --save-gnuplot --view
```

8.16 Effective Mass / Velocity

At a glance

Menu code: 2.16 **Command:** `stb-effmass`
Guided example: `examples/2.16-effmass/`
Summary: Effective mass tensor and band velocity for one chosen (k, band) eigenstate.

8.16.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/2.16-effmass/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

Welcome! `stb-effmass` computes two of the most fundamental single-state electronic-structure quantities: the **band velocity** (the group velocity of a wavepacket at that state) and the **effective mass tensor** (how the band curves in energy around that k-point) for one chosen (k, band) eigenstate — most usefully the valence band maximum (VBM) or conduction band minimum (CBM), where these quantities govern carrier transport.

This example walks through the theory, a real crash a user hit on their own spin-polarized calculation (now fixed), and a second, deeper physics subtlety uncovered along the way.

8.16.1.1 Theory

Band velocity and effective mass, from first principles. Near a band extremum, the energy dispersion can be Taylor-expanded:

```
E_n(k) ~ E_n(k0) + hbar * v . (k - k0) + (hbar^2/2) (k-k0)^T [M^-1] (k-k0) + ...
```

- **v** (band velocity) is the *first* derivative, `v = (1/hbar) dE/dk` — how fast a wavepacket built from this state moves through the crystal.
- **M** (the effective mass tensor) comes from the *second* derivative, `[M^-1]_ij = (1/hbar^2) d^2E/dk_i dk_j` — how strongly the band curves, which determines how heavy or light a carrier behaves under an applied field (`F = M^-1 . p`, Newton's second law with the vacuum mass replaced by this tensor).

`stb-effmass` gets both from sisl's `EigenstateElectron.velocity()`/ `.effective_mass()`, which compute these derivatives **analytically** from the Hamiltonian's own k-derivatives (`dH/dk`, d^2H/dk^2) — not by finite-differencing a band structure, so there's no step-size to tune and no numerical-noise tradeoff.

A real, verified subtlety: the 2nd-order curvature correction needs energy differences between ALL bands at that k-point (a Berry-curvature- like term), so it must always be computed on the FULL multi-band eigenstate — verified live that calling it after `.sub(band_index)` gives wildly wrong values (differing by orders of magnitude on a real test system). This tool always computes on the full state and only indexes the band of interest afterward.

A real bug found and fixed: spin-polarized calculations used to crash. A user ran this tool on their own real, spin-polarized SIESTA calculation and hit:

```
TypeError: SparseOrbitalBZ._ddPk() got an unexpected keyword argument 'spin'
```

Verified directly against sisl's own source: `Hamiltonian.dPk()` (the 1st-order derivative `velocity()` uses) accepts a `spin` argument, but `Hamiltonian.ddPk()` (the 2nd-order derivative `effective_mass()` needs) **does not have a `spin` parameter at all** in this sisl version. This tool used to assume only non-collinear/spin-orbit (nspin=4/8) calculations were affected — verified live that it's actually **any** spin-polarized Hamiltonian (nspin=2 too). Fixed: `stb-effmass` now detects any spin-resolved Hamiltonian up front and reports velocity only, with a clear warning instead of a raw traceback (plus a `try/except` as a second line of defense). See section 3 below for the live demonstration.

A second, deeper finding: per-axis values can be misleading. sisl's own `effective_mass()` returns each Voigt component (`m*_xx`, `m*_yy`, `m*_zz`, `m*_yz`, `m*_xz`, `m*_xy`) as an **independent, element-wise reciprocal** of the corresponding curvature term — NOT a proper matrix inversion of the full 3x3 curvature tensor. This means `m*_xx/yy/zz` are only rigorously "the effective mass along x/y/z" when the off-diagonal terms are small.

Verified live on this example's own Sn3O4 VBM — assembling the FULL curvature tensor from all 6 Voigt values, inverting THAT matrix, and diagonalizing it gives the properly basis-independent **principal effective masses**:

	m*_xx	m*_yy	m*_zz	Principal masses
Sn3O4 VBM	-0.629	-0.757	-0.515	**-1.320, -0.844, -0.343**

Substantially different (up to 2-4x) from the naive per-axis reading of the exact same state! This is common, not a rare corner case: away from a crystal's principal axes (Sn3O4 has low symmetry), the curvature tensor is generically NOT diagonal in the Cartesian basis. `stb-effmass` now reports BOTH values, with an automatic `[OK]`/`[WARNING]` flag comparing how much the off-diagonal curvature contributes.

8.16.1.2 Worked examples: what a real oxide's states actually look like

A deep, core-like state at Gamma (band 1).

```
$ stb-effmass --wfsx Sn3O4.selected.WFSX --hsx-file Sn3O4.HSX --k-index 0 --band 1
```

Component	Value (m0)
m*_xx	0.444
m*_yy	0.121
m*_zz	0.085
m*_yz	0.118
m*_xz	0.427
m*_xy	0.493

Off-diagonal/diagonal curvature ratio: **1.144** — off-diagonal terms actually EXCEED the diagonal ones here, flagging `[WARNING]`. The properly diagonalized principal masses (0.052, 0.585, 0.762 m0) are all positive and physically ordinary — but along directions that are NOT simply x, y, or z.

The valence band maximum: negative (hole-like) effective mass.

```
$ stb-effmass --wfsx Sn3O4.selected.WFSX --hsx-file Sn3O4.HSX --band vbm --fermi -3.200055
```

Every principal effective mass here is **negative** (-1.320, -0.844, -0.343 m0) — exactly the expected sign for a valence band maximum: the band curves DOWNWARD in energy away from the VBM, so a hole (missing electron) there behaves as if it has positive mass while the electron itself has negative curvature-derived mass. A physically satisfying, textbook-correct sign, now backed by a properly diagonalized value instead of 6 independent per-axis numbers.

The conduction band minimum: a caution about coarse k-meshes.

```
$ stb-effmass --wfsx Sn3O4.selected.WFSX --hsx-file Sn3O4.HSX --band cbm --fermi -3.200055
```

Principal masses here come out **mixed-sign** (-0.820, -0.522, +0.734 m0) — not the all-positive signature a genuine local energy minimum should have in every direction. This fixture's `.WFSX` only samples a coarse 2x2x2-equivalent k-mesh (8 explicit k-points); the point this mesh identifies as the "global CBM" may not be an exact true minimum along all three directions of the real, continuous band structure. A real, honest limitation of ANY effective-mass calculation done at a mesh-identified extremum, not a bug in this tool — worth knowing before trusting a single coarse-mesh CBM/VBM effective mass at face value.

8.16.1.3 The real bug, demonstrated live

```
$ stb-effmass --wfsx Ospin.bands.WFSX --hsx-file Ospin.HSX --k-index 0 --band 3
```

This fixture (`spin/Ospin.*`) is a real, spin-polarized isolated oxygen atom (converged to its physical 2 Bohr-magneton triplet ground state). Before the fix, this exact command would crash with the raw `TypeError` shown in section 1.2. Now:

```
[WARNING] Effective mass (2nd-order k-derivative) is not supported by
sisl for spin-resolved Hamiltonians (nspin=2) in this environment --
reporting velocity only.
...
[3] EFFECTIVE MASS (per-axis Voigt, ...): N/A (unsupported for nspin=2, ...)
[4] EFFECTIVE MASS (principal, full tensor): N/A (unsupported for nspin=2, ...)
[5] BAND VELOCITY: (still computed normally)
```

A non-collinear/spin-orbit fixture (`nc/IsolatedO.*`, nspin=4) shows the exact same graceful degradation — this was already handled before the fix; the new fix simply extends the SAME treatment to nspin=2, which used to be (incorrectly) assumed safe.

8.16.1.4 Output and options

- **Numbered `[0]...[8]` report** (`RUN METADATA`, `INPUT DATA`, `STATE SELECTION`, `EFFECTIVE MASS (per-axis Voigt)`, `EFFECTIVE MASS (principal, full tensor)`, `BAND VELOCITY`, `OUTPUT DATA & PLOTS`, `REFERENCES`, `SUMMARY & FILES`).
- **`--save-report`**, **`--save-gnuplot`** (writes `effmass.dat`/ `velocity.dat` + a real 2-panel bar-chart `.gplot` script) and **`--view`** (matplotlib bar charts) are all off by default.
- **nspin=2 is handled gracefully** (section 1.2/3 above).
- **The principal effective mass** (section 1.3 above) is reported alongside the per-axis values.
- **`--label` can be combined with `--hsx-file`.**
- **`--band` `vbm/cbm`'s Fermi-energy source** is independent of `--label`: the same priority-ordered hierarchy (`--fermi` > `--bands-file` > `--fermi-file` > an auto-detected `.out` log) as `stb-wfdensity`/ `stb-sts`/`stb-coop`.

8.16.1.5 Known limitations

- Only a single (k, band) point, not swept along a path — most meaningful at a true band extremum (VBM/CBM); a coarse k-mesh's "extremum" may not be an exact local minimum in every direction (section 2.3 above).
- A component reading exactly 0 along a vacuum-padded (non-periodic) axis is expected, not an error — and the principal-mass diagonalization is skipped there (the curvature tensor is singular).
- Berry curvature correction to the velocity itself is NOT included (sisl's own documented limitation).
- Effective mass (not velocity) is unavailable for ANY spin-resolved Hamiltonian (section 1.2/3).

8.16.1.6 Files in this folder

File	What it is
`Sn3O4.selected.WFSX/Sn3O4.HSX`	The real full-BZ Sn3O4 calculation (same fixture `2.14-coop/` uses)

File	What it is
`spin/Ospin.bands.WFSX/spin/Ospin.HSX`	A real spin-polarized isolated-O-atom calculation (the exact bug-fix fixture)
`nc/IsolatedO.selected.WFSX/nc/IsolatedO.HSX`	A real non-collinear/SOC calculation (pre-existing graceful-degradation case)
`example_2.16.sh`	This walkthrough's runnable script

8.16.1.7 Running it

```
bash example_2.16.sh
```

Or try the tool directly:

```
stb-effmass --wfsx Sn3O4.selected.WFSX --hsx-file Sn3O4.HSX --band vbm \
    --fermi -3.200055 --save-report --save-gnuplot --view
```

8.16.1.8 What's next

- `stb-coop` (`2.14-coop/`) — shares this exact Sn3O4 fixture, a different (bonding-character) question about the same electronic structure.
- `stb-fatbands` (`2.10-stb-fatbands/`) — the same spin-polarized `Ospin` fixture, used there to find/fix the analogous nspin=2 category-merging bug in orbital-projected weights.
- `stb-bands`/`stb-dos` — the band structure / density of states this single-state quantity sits within.

8.16.2 Implementation details

Effective mass and band velocity

For band n at $\mathbf{k}$,

$$v_i = \frac{1}{\hbar}\frac{\partial \epsilon_n}{\partial k_i}, \qquad \left(\frac{1}{m^*}\right)_{ij} = \frac{1}{\hbar^2}\frac{\partial^2 \epsilon_n}{\partial k_i \partial k_j}.$$

sisl computes these derivatives analytically from $\partial H/\partial \mathbf{k}$ and $\partial^2 H/\partial \mathbf{k}^2$ (second-order perturbation theory over all bands at that $\mathbf{k}$), which requires the real Hamiltonian (`.HSX`).

What is reported..

- the velocity in km/s (sisl's Å/ps $\times 0.1$);
- sisl's per-component masses – each Voigt component of the curvature inverted on its own, $m^*_{ij} = \hbar^2/(\partial^2\epsilon/\partial k_i \partial k_j)$ – converted to electron masses with the factor 3.301296×10^{-6};
- the *principal* effective masses: the full 3×3 curvature tensor is built from the six components, inverted as a matrix and diagonalized, giving the masses along its principal axes and the axis directions. Away from high-symmetry points the curvature tensor is generally not diagonal in Cartesian axes, and the two descriptions can differ by tens of per cent; the report flags a significant difference.

The derivatives are always computed on the full multi-band state (taking a single band first would discard the inter-band terms and give wrong curvatures).

Selecting the state.. As in **stb-wfdensity**: -k-index or -k-point, -band N or vbm/cbm, -spin. A mass is physically meaningful at a band extremum; at any other point it describes the local curvature only.

Limitations.. Along vacuum axes the derivatives vanish and the curvature tensor is singular; the principal analysis is then skipped. For spin-polarized and non-collinear Hamiltonians the installed sisl cannot compute second derivatives, so the tool reports the velocity only, with a warning. The Berry-curvature (anomalous) velocity is not included.

8.16.3 Command-line reference

8.16.3.1 stb-effmass

Effective mass tensor (Voigt) and band velocity for one chosen (k, band) eigenstate, from a SIESTA .WFSX + .HSX.

Synopsis.

```
usage: stb-effmass [-h] [--label LABEL] [--wfsx WFSX] [--hsx-file HSX_FILE]
                   [--k-index K_INDEX | --k-point KX KY KZ] [--spin {0,1}] --band BAND
                   [--fermi FERMI] [--bands-file BANDS_FILE] [--fermi-file FERMI_FILE]
                   [--gap-tol GAP_TOL] [-o OUTPUT_DIR] [--save-report]
                   [--save-gnuplot] [--view] [-v] [--no-intro]
```

Options

`--label LABEL`
: SIESTA output label. Auto-detects <label>.WFSX, falling back to <label>.selected.WFSX then <label>.bands.WFSX, and, unless `--hsx-file` overrides it, <label>.HSX (mandatory – no approximate fallback). Mutually exclusive with `--wfsx` (not `--hsx-file` – see its own help).

`--wfsx WFSX`
: Explicit path to the .WFSX file (alternative to `--label`).

`--hsx-file HSX_FILE`
: Explicit .HSX path. Required if `--wfsx` is used instead of `--label`; optional (and common) together WITH `--label` too, since the real file is often not literally named <label>.HSX.

`--k-index K_INDEX`
: 0-based index into the .WFSX's own k-point order (default: 0 if neither `--k-index` nor `--k-point` is given).type: int; exclusive group 1

`--k-point KX KY KZ`
: Match by k-point vector instead of index (matched via sisl's read_eigenstate). type: float; exclusive group 1

`--spin {0,1}`
: Spin channel (default: 0; must be 0 for a non-polarized calculation). choices: 0, 1; type: int

`--band BAND`
: Which band at the chosen k: an integer N (1-based), or 'vbm'/'cbm' (requires `--fermi`, `--bands-file`, `--fermi-file`, or an auto-detected .out log – see below). required

`--fermi FERMI`
: Fermi energy (eV) for `--band` vbm/cbm. Highest-priority source if given. type: float

`--bands-file BANDS_FILE`
: Companion .bands file to read the Fermi energy from, for `--band` vbm/cbm.

`--fermi-file FERMI_FILE`
: Explicit SIESTA .out log to read the Fermi energy from, for `--band` vbm/cbm – an alternative to `--bands-file` for a run with no saved .bands file. Not assumed to be named after `--label`.

`--gap-tol GAP_TOL`
: Energy tolerance in eV for VBM/CBM classification (default: 0.01). Same meaning as `stb-bands/stb-fatbands/stb-wfdensity`. type: float

`-o, --output-dir OUTPUT_DIR`
: Directory to write effmass.dat/velocity.dat (with `--save-gnuplot`) and stb_effmass_report.txt/references.bib into (default: current directory). Created if it doesn't exist.

`--save-report`
: Also persist the full run report to `stb_effmass_report.txt`. Off by default. switch, off by default

`--save-gnuplot`
: Also write effmass.dat/velocity.dat + a real .gplot bar-chart script. Off by default. switch, off by default

`--view`
: Show a matplotlib bar-chart preview (effective mass + velocity components) before finishing. Off by default. switch, off by default

Like every STB command it also accepts `-h`/`–help`, `-v`/`–version` and `–no-intro` (Section 3.3).

From the built-in help.

```
Example usage:
  stb-effmass --label siesta --band vbm --fermi -4.2
  stb-effmass --label siesta --k-index 0 --band 1
  stb-effmass --label siesta --band vbm --fermi-file calc.out \
      --save-report --save-gnuplot --view
```

8.17 Spin Texture Analyzer

At a glance

Menu code: 2.17 **Command:** `stb-spintexture`
Guided example: `examples/2.17-spintexture/`
Summary: Spin moments (<Sx>,<Sy>,<Sz>) scattered onto a band structure from a non-collinear/SOC .WFSX.

8.17.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/2.17-spintexture/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

`stb-spintexture` scatters <Sx>, <Sy>, <Sz> — the quantum-mechanical expectation values of the Pauli spin operators — onto a band structure, one point per (k, band). It's the tool you reach for after a non-collinear or spin-orbit-coupled (SOC) SIESTA calculation, to see how each electronic state's spin is oriented: Rashba splitting, topological surface states, and canted magnetic textures all show up here.

8.17.1.1 Theory

What a spin texture actually is. A collinear (non-polarized or simple spin-up/down) calculation only ever needs a single number per state: "how much spin-up vs. spin-down". A non-collinear/SOC calculation instead represents each electron as a full 2-component spinor, and its spin direction can point ANYWHERE in 3D — described by the expectation values of the three Pauli matrices:

```
<Sx> = <psi| sigma_x |psi>,  <Sy> = <psi| sigma_y |psi>,  <Sz> = <psi| sigma_z |psi>
```

For a properly normalized spinor, these three numbers always satisfy:

```
|S| = sqrt(<Sx>^2 + <Sy>^2 + <Sz>^2)  <=  1
```

`stb-spintexture` gets these from sisl's `EigenstateElectron. spin_moment()`, which needs the real overlap matrix `Sk()` — the same "hamiltonian_preferred" accuracy tradeoff as `stb-fatbands`/`stb-ipr`: a real `.HSX` gives the true overlap-weighted value; a bare `.fdf` falls back to an implicit-orthogonal approximation with a printed warning.

A real, live-verified reason the accuracy warning matters. This isn't a theoretical concern. Running this example's own fixture (a real non-collinear isolated oxygen atom) both ways:

```
$ stb-spintexture --wfsx IsolatedO.selected.WFSX --hsx-file IsolatedO.HSX      # accurate
$ stb-spintexture --wfsx IsolatedO.selected.WFSX --geometry-file calc.fdf      # approximate
```

Source	Sz values	max |S|
`IsolatedO.HSX` (accurate)	all exactly ±1 (to numerical noise)	**1.000001**
`calc.fdf` only (approximate)	ranges from -18.16 to +18.45	**18.451336**

The approximate fallback doesn't just give a slightly-off number here — it gives values that are **physically impossible** (a Pauli expectation value can never exceed 1 in magnitude for a normalized spinor). This tool's new `[3] SPIN TEXTURE ANALYSIS` report section catches this automatically:

```
Sz        | -0.000000 | -18.163734 | 18.451336
|S| (Pauli expectation magnitude): mean=6.216040, max=18.451336
[WARNING] |S| exceeds 1 by more than 2% for at least one state --
this should never happen for a properly normalized spinor; check the
.WFSX/.HSX correspondence.
```

Always pass a real `--hsx-file` for spin texture work — the approximate fallback exists for tools like `stb-fatbands`/`stb-ipr` where it's merely less accurate; for spin texture, without the true overlap matrix the result can be outright unphysical.

1.3 No `.bands` file needed — and why the x-axis is different. Unlike `stb-fatbands`/`stb-ipr`, this tool does NOT need a companion `.bands` file: `spin_moment()` works directly from whatever k-points the `.WFSX` itself contains (a band path, a full-BZ mesh, or even — as in this example — a single Gamma-only point). Without a `.bands` file's own k-path geometry, the x-axis is simply the 0-based k-INDEX, not a physical k-path arc length. For a real SOC band-structure spin-texture plot (Rashba splitting, topological surface states), pass a genuine band-path `.WFSX` and expect evenly-spaced k-index ticks, not a `stb-bands`-style scaled k-path.

8.17.1.2 This example's fixture: what it does and doesn't show

`IsolatedO.HSX/IsolatedO.selected.WFSX` is a real, non-collinear SIESTA calculation (a single oxygen atom in a large vacuum box, Gamma-point only). Without an explicit initial spin canting (`%block DM.InitSpin`), it converges with:

```
$ stb-spintexture --label IsolatedO
Sz        | 0.000000  | -1.000000 | 1.000001
Most spin-polarized state: |S|=1.000001 at k-index=0, E=23.908838 eV
  (Sx=0.0000, Sy=-0.0000, Sz=1.0000)
```

`Sz` is strongly non-zero (±1.0, the physically expected value for this open-shell atom's spin), while `Sx`/`Sy` are ~0 (numerical noise, ~1e-15). This **validates the numerical pipeline is correct** (a real, non-trivial |S| = 1 result), but does NOT demonstrate a genuinely "textured" spin (canted, k-dependent `Sx`/`Sy`) — that needs real spin-orbit coupling on a heavier element with a genuine k-path, which this suite doesn't bundle a fixture for. The `[3]` report section's "Most spin-polarized state" line is exactly the piece of information you'd use to find where in a real k-path the most interesting spin texture occurs.

8.17.1.3 Output and options

- **Numbered `[0]`...`[6]` report** (`RUN METADATA`, `INPUT DATA`, `ENERGY REFERENCE`, `SPIN TEXTURE ANALYSIS`, `OUTPUT DATA & PLOTS`, `REFERENCES`, `SUMMARY & FILES`) — `SPIN TEXTURE ANALYSIS` reports per-component mean/min/max, the `|S| <= 1` normalization check (section 1.2), and the single most spin-polarized state found.
- **`--save-report`**, **`--save-gnuplot`** (writes `spintexture_S{x,y,z}.dat/.gplot`) and **`--view`** (matplotlib preview) are all off by default.
- **`--label` can be combined with `--hsx-file`/`--geometry-file`.**
- **`--shift` `fermi/vbm/cbm`'s Fermi-energy source** follows the same priority-ordered hierarchy as `stb-effmass` (`--fermi` > `--bands-file`

  `--fermi-file` > an auto-detected `.out` log).

8.17.1.4 Known limitations

- Needs `nspin=4` (non-collinear) or `nspin=8` (spin-orbit); a clean error for any other `nspin`.
- This example's own fixture doesn't demonstrate a genuinely canted (k-dependent) spin texture (section 2 above) — a real SOC calculation on a heavier element with a genuine k-path would be needed for that.
- Only "diagonal" projection (`spin_moment` per band), not the orbital-resolved "hadamard" projection sisl also supports.

8.17.1.5 Files in this folder

File	What it is
`IsolatedO.HSX/IsolatedO.selected.WFSX`	The real non-collinear calculation (mandatory `.WFSX`, optional `.HSX` for accuracy)
`calc.fdf`	Geometry-only fallback input (used to demonstrate the real accuracy gap in section 1.2)
`example_2.17.sh`	This walkthrough's runnable script

8.17.1.6 Running it

```
bash example_2.17.sh
```

Or try the tool directly:

```
stb-spintexture --label Isolated0 --shift fermi --fermi -12.0 \
    --save-report --save-gnuplot --view
```

8.17.1.7 What's next

- `stb-fatbands` (`2.10-stb-fatbands/`)/`stb-ipr` (`2.15-ipr/`) — share this tool's exact `.HSX`-accurate/`.fdf`-approximate accuracy tradeoff, and its `core/band_scatter.py` scatter-plot machinery.
- `stb-effmass` (`2.16-effmass/`) — shares this tool's exact Fermi-energy source hierarchy for `--shift fermi/vbm/cbm`.

8.17.2 Implementation details

Physics.. For a two-component spinor state, the spin texture is the expectation value of the Pauli matrices,

$$\langle S_\alpha \rangle_{n\mathbf{k}} = \langle \psi_{n\mathbf{k}} | \sigma_\alpha | \psi_{n\mathbf{k}} \rangle, \qquad \alpha = x, y, z,$$

evaluated by sisl's `spin_moment` with the overlap of the non-orthogonal basis (`.HSX` preferred, `.fdf` fallback with a warning). It exists only for non-collinear (`Spin non-collinear`) or spin–orbit (`Spin spin-orbit`) calculations; collinear runs are refused. For a normalized spinor $|\langle \mathbf{S} \rangle| \leq 1$, and the report checks this bound.

Output.. Any `.WFSX` works (band path, mesh, or a single point); no `.bands` file is needed, so the horizontal axis is the k-point index, not the path length. Each of $\langle S_x \rangle$, $\langle S_y \rangle$, $\langle S_z \rangle$ is scattered on the bands as colour, with `-shift` choosing the energy zero (Fermi level from the usual sources). Rashba or Dresselhaus splittings and topological surface states appear as in-plane spin components that reverse between the two branches and with $\mathbf{k} \to -\mathbf{k}$.

Note.. The test fixture of the suite is an isolated O atom with no initial spin canting, which validates the pipeline (large S_z, vanishing S_x, S_y) but not a textured case; check your own results against symmetry expectations.

8.17.3 Command-line reference

8.17.3.1 `stb-spintexture`

Spin texture (<Sx>,<Sy>,<Sz>) scattered onto a band structure, from a non-collinear/SOC SIESTA .WFSX.

Synopsis.

```
usage: stb-spintexture [-h] [--label LABEL] [--wfsx WFSX] [--hsx-file HSX_FILE]
                       [--geometry-file GEOMETRY_FILE]
                       [--shift {vbm,cbm,fermi,manual,none}]
                       [--manual-value MANUAL_VALUE] [--fermi FERMI]
```

```
[--bands-file BANDS_FILE] [--fermi-file FERMI_FILE]
[--gap-tol GAP_TOL] [-o OUTPUT_DIR] [--save-report]
[--save-gnuplot] [--view] [-v] [--no-intro]
```

Options

`--label LABEL`

SIESTA output label. Auto-detects `<label>.WFSX`, falling back to <label>.selected.WFSX then <label>.bands.WFSX, and, unless `--hsx-file/--geometry-file` override it, `<label>.HSX` (optional, upgrades accuracy) then `<label>.fdf`. Mutually exclusive with `--wfsx` (not `--hsx-file/--geometry-file` – see their own help).

`--wfsx WFSX`

Explicit path to the .WFSX file (alternative to `--label`).

`--hsx-file HSX_FILE`

Optional explicit .HSX path for physically correct overlap-weighted spin moments. Falls back to `--geometry-file` / auto-detected .fdf with an accuracy warning if omitted. Optional (and common) together WITH `--label` too, since the real file is often not literally named `<label>.HSX`.

`--geometry-file GEOMETRY_FILE`

Optional explicit .fdf geometry path, used as the fallback parent when no .HSX is available.

`--shift {vbm,cbm,fermi,manual,none}`

Energy reference (default: none, i.e. raw eigenvalues as stored in the .WFSX). choices: vbm, cbm, fermi, manual, none

`--manual-value MANUAL_VALUE`

Custom shift value (eV), required if `--shift` manual. type: float

`--fermi FERMI`

Fermi energy (eV) for `--shift` fermi/vbm/cbm. Highest-priority source if given. type: float

`--bands-file BANDS_FILE`

Companion .bands file to read the Fermi energy from, for `--shift` fermi/vbm/cbm.

`--fermi-file FERMI_FILE`

Explicit SIESTA .out log to read the Fermi energy from, for `--shift` fermi/vbm/cbm – an alternative to `--bands-file` for a run with no saved .bands file. Not assumed to be named after `--label`.

`--gap-tol GAP_TOL`

Energy tolerance in eV for VBM/CBM classification when `--shift` is vbm/cbm (default: 0.01). type: float

`-o, --output-dir OUTPUT_DIR`

Directory to write spintexture_S{x,y,z}.dat/.gplot (with `--save-gnuplot`) and stb_spintexture_report.txt/references.bib into (default: current directory). Created if it doesn't exist.

`--save-report`

Also persist the full run report to `stb_spintexture_report.txt`. Off by default. switch, off by default

`--save-gnuplot`

Also write spintexture_S{x,y,z}.dat + `spintexture.gplot`. Off by default – this tool used to write both unconditionally on every run. switch, off by default

`--view`

Show an interactive matplotlib preview before finishing. Off by default – this tool previously always showed it, with no way to skip. switch, off by default

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Example usage:
  stb-spintexture --label siesta --shift fermi --fermi -4.2
  stb-spintexture --label siesta --shift fermi --fermi-file calc.out \
      --save-report --save-gnuplot --view
```

8.18 AIMD Trajectory Analysis

At a glance

Menu code: 2.18 **Command:** `stb-aimdAnalysis`
Guided example: `examples/2.18-aimdanalysis/`
Summary: RDF, MSD/diffusion coefficient, and VACF-derived VDOS from an AIMD trajectory.

8.18.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/2.18-aimdanalysis/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

`stb-aimdAnalysis` turns a raw AIMD/MD trajectory -- a SIESTA `<label>.ANI` (+ `.out` for the true per-step cell) or a generic ASE-readable file (e.g. one written by `stb-mlmd`) -- into physics: structure (RDF), transport (MSD/diffusion), vibrations (VACF-derived VDOS), thermodynamics (energy/temperature/volume/pressure, from SIESTA's own `.MDE` file), and per-atom displacement/atom-pair relative-distance tracking. It complements `stb-ani2traj`, which only converts a trajectory's file *format* for external viewers and computes nothing.

8.18.1.1 Theory

Radial distribution function g(r). For every frame, the minimum-image distance between every requested pair of atoms is histogrammed and normalized against the ideal-gas shell density -- the standard structural fingerprint of a liquid/amorphous/gas system (bond lengths show up as sharp peaks). `--pair A-B` restricts this to one species pair (e.g. `O-H`); the default is the total RDF over every pair. Correctly re-derived per frame from that frame's OWN cell, so a variable-cell (NPT/Parrinello-Rahman) run stays correct.

Mean-squared displacement (MSD) and diffusion coefficient. `MSD(t) = <|r(t) - r(0)|^2>`, averaged over multiple sliding time origins (not just `t=0`, the standard way to reduce statistical noise) and over every atom of a given species. A linear fit of the diffusive -regime part of the curve (default: the second half of the trajectory) gives the diffusion coefficient via the 3D Einstein relation `D = slope/6`, converted from Ang^2/fs to the conventional cm^2/s.

Vibrational density of states (VDOS). `.ANI`/most generic trajectory formats carry only POSITIONS, not velocities -- velocities are estimated by central finite difference, `v(t) = (x(t+dt) - x(t-dt))/(2 dt)`. The velocity autocorrelation function (VACF) is computed via FFT (fast, `O(N log N)`) and its Fourier transform (Wiener-Khinchin theorem) gives the VDOS. This amplifies positional noise into the derivative, so the result is explicitly qualitative, not benchmark-grade -- useful for "where's the dominant vibrational frequency", not a publication-grade phonon DOS.

Single-atom displacement tracking. `--track-atom N` (0-based atom index) reports ONE specific atom's own Cartesian displacement from its initial position, frame by frame -- built on the same PBC-unwrapped trajectory the MSD already uses, so a real diffusing/hopping atom's displacement stays physically continuous instead of resetting every time it crosses a periodic boundary. Useful for watching one diffusing adatom/dopant/defect directly, instead of only the species-averaged MSD.

Atom-pair relative-distance tracking. `--track-pair I-J` (two 0-based atom indices, e.g. `0-5`) reports the relative separation between two SPECIFIC atoms every frame, using the SAME minimum-image convention as the RDF -- deliberately NOT the unwrapped trajectory `--track-atom` uses. What matters physically for a bond length, a hydrogen-bond distance, or a reaction coordinate is the atoms' TRUE instantaneous separation; unwrapping the two atoms independently would not give this in general, since each atom can accumulate its own separate drift/unwrap path that doesn't cancel back down to the true short separation.

Thermodynamic time series (energy/temperature/volume/pressure). `[7] THERMODYNAMIC TIME SERIES` reports and plots four quantities in one 4-panel figure. Volume is always available (computed directly from each frame's own cell, any input source). For a `--label` run, Energy/ Temperature/Pressure are read straight from SIESTA's own dedicated `<label>.MDE` file (`core.siesta_log.get_mde_trajectory`) -- a small, clean per-step table (`Step T(K) E_KS(eV) E_tot(eV) Vol(Ang^3) P(kBar)`) SIESTA already writes for exactly this purpose, rather than re-scraping it back out of scattered `.out` log lines. Unlike the INPUT `.fdf`, `.MDE` is always named after `SystemLabel` (like `.XV`/`.ANI`), so no `--geometry-file`-style override is needed for it.

Both `E_tot` (total energy, kinetic+potential) and `E_KS` (the electronic/potential-like energy alone) are plotted together on the Energy panel when both are available -- `E_tot` trades between kinetic and potential energy by design, so only the total is meaningful to check for stability/drift (the same distinction `stb-mlmd`'s own energy tracking relies on). For a generic `--trajectory` input, Energy/ Temperature fall back to each frame's own embedded `Epot`/`Temp` info if present (e.g. from `stb-mlmd --out-format xyz`); Pressure is never available there (no known source).

Why energy is plotted per atom, not as an absolute total. Energy is an **extensive** quantity -- it scales with how many atoms are in the simulation cell. `-1029.53 eV` for an 8-atom cell and `-1029.53 eV` for a 64-atom cell of the SAME material would be wildly different physical states (roughly 8x less bound per atom in the second case), yet the absolute number alone doesn't tell you that. Dividing by the atom count gives an **intensive** quantity (`eV/atom`) -- independent of system size, and the natural axis for judging stability/drift or comparing two different-sized calculations of the same material. This is why the Energy panel (both the matplotlib `--view` figure and the gnuplot `<stem>_energy.gplot`/`<stem>_thermo .gplot` scripts) plots `E_tot`/`E_KS` in `eV/atom` -- the raw absolute totals are still written as extra columns in `<stem>_energy.dat` and still printed in the `[7]` report table, just not what's on the y-axis.

8.18.1.2 2. Cross-check: `--track-pair` against the RDF's own first peak

`aimd.ANI/.XV/.fdf/.out` is a real SIESTA AIMD run: a 5-step Verlet trajectory of an isolated O2 dimer in vacuum. With only 2 atoms, `--track-pair 0-1` IS the O-O bond -- and it independently agrees with the RDF's own first peak, two completely separate code paths (a statistical histogram over all frames vs. one specific pair's per-frame minimum-image distance) landing on the same answer:

```
$ stb-aimdAnalysis --label aimd --track-atom 0 --track-pair 0-1 --no-intro
...
First peak        : r = 1.087 Ang (g(r) = 269.148)
...
```

```
Tracked pair      : atom 0 (O) -- atom 1 (O)
Initial distance (Ang)          | 1.0864
Final distance (Ang)            | 1.2284
```

The bond visibly stretches over the 5-step trajectory (1.086 -> 1.228 Ang) -- too short a run to mean anything thermodynamically, but exactly the kind of event `--track-pair` exists to let you watch directly instead of only seeing it blurred into a population-averaged RDF.

8.18.1.3 Features at a glance

- **`--geometry-file <path>`**: an explicit, `--label`-decoupled path to the real SIESTA input `.fdf` (needed for the true MD timestep) -- the same `--sts`/`--coop`/`--ipr`/`--effmass`/`--spintexture` pattern this suite already uses for `.fdf`/`.HSX` inputs. Details and a live reproduction of the real-world filename mismatch it fixes: section 4.
- **`--list-atoms`**: a standalone early-exit mode -- prints every atom's 0-based index, species, and Cartesian coordinates (from the first frame only, so it's fast regardless of trajectory length), then exits immediately without running RDF/MSD/VDOS/anything else. Deliberately a separate opt-in mode rather than a table always printed in the report -- a real structure can have hundreds of atoms. The interactive `stb-suite` menu asks "List every atom's index/ species/coordinates?" (y/N) right before prompting for `--track-atom`/`--track-pair`.
- **`--track-atom N`** (single-atom displacement) and **`--track-pair I-J`** (atom-pair relative distance) -- report sections `[5]`/`[6]`, always printed (say "Not requested" if the flag is omitted, same convention as `[8]`'s gnuplot section). Theory: sections 1.4/1.5.
- **`[7] THERMODYNAMIC TIME SERIES`** -- energy/temperature/volume/ pressure in one 4-panel figure (matplotlib via `--view`, gnuplot via `--save-gnuplot`), reading Energy/Temperature/Pressure from SIESTA's own dedicated `<label>.MDE` file for a `--label` run (section 1.6). The Energy panel's y-axis plots `eV/atom`, not the raw absolute total (an intensive, system-size-independent quantity -- section 1.7); `[7]`'s report table and `<stem>_energy.dat`'s extra columns still carry the absolute eV values too. Verified on a real 500-step SIESTA NVT run: section 5.
- **Numbered `[0]`...`[10]` report** (`RUN METADATA`, `INPUT DATA`, `RDF`, `MSD & DIFFUSION`, `VACF/VDOS`, the two tracking sections, the thermodynamic section, `WRITING OUTPUT FILES`, `REFERENCES`, `SUMMARY & FILES`).
- **`--save-gnuplot`** writes a real `.dat` + `.gplot` pair for every computed quantity (RDF, MSD, VACF, VDOS, volume, energy, temperature, pressure, and the atom-displacement/pair-distance series if used) plus one combined 4-panel `<stem>_thermo.gplot`.
- **`--view`** shows the same figures as an interactive matplotlib preview, off by default.
- **`-o`/`--output-dir`** selects where every output file is written (default: the current directory).
- **`[9] REFERENCES`** writes `references.bib` citing SIESTA for a `--label` run; correctly prints a note instead for a generic `--trajectory` input, since that path isn't guaranteed to be a SIESTA run at all (section 6).

8.18.1.4 4. `--geometry-file`: the real-world `.fdf` filename mismatch, in detail

`--label` mode reads two things from a `.fdf`: the true MD timestep (`MD.InitialTimeStep`/`MD.LengthTimeStep`) and, only as a LAST-resort lattice fallback (after `<label>.out`'s per-step cell and `<label>.XV`), the geometry itself. SIESTA always names `.XV`/`.ANI`/`.HSX`/`.WFSX` after `SystemLabel`, but the INPUT `.fdf`'s own filename is chosen by the user and is very often different (e.g. `SystemLabel siesta` with the real input called `calc.fdf`) -- without an explicit override, this silently degrades to "assume 1 fs per frame" with only an easy-to-miss warning. Renaming this example's own `aimd.fdf` to `calc.fdf` reproduces the scenario and its fix directly:

```
$ stb-aimdAnalysis --label aimd --no-intro   # no aimd.fdf present -- only calc.fdf
[WARNING] MD.LengthTimeStep not found in 'aimd.fdf (auto-detected)' --
assuming 1 fs per (strided) frame for MSD/VDOS time axes. If the real
input file isn't named 'aimd.fdf', pass --geometry-file <path>.

$ stb-aimdAnalysis --label aimd --geometry-file calc.fdf --no-intro
Geometry file     : calc.fdf
Timestep (post-stride) | 1.0000 fs
```

`--geometry-file` only applies to `--label` mode (a `--trajectory` input already carries its own per-frame cell, no `.fdf` involved at all) -- and is validated to exist up front, same as every other explicit-path flag in this suite.

8.18.1.5 5. A real 500-step SIESTA NVT run: `.MDE` + `--geometry-file` together

`siesta.ANI/.XV/.MDE` + `calc.fdf/structure.fdf` are a SECOND, real SIESTA run bundled with this example (distinct from the small 5-step `aimd.*` fixture above) -- 500 MD steps, an 8-atom SiC supercell, Nose thermostat at a 500 K target. `SystemLabel siesta` but the real input is `calc.fdf` -- reproducing the exact `--geometry-file` scenario above on a richer run, AND giving `[7] THERMODYNAMIC TIME SERIES` real `.MDE` data to read:

```
$ stb-aimdAnalysis --label siesta --geometry-file calc.fdf --save-gnuplot --no-intro
[7] THERMODYNAMIC TIME SERIES (ENERGY / TEMPERATURE / VOLUME / PRESSURE)
Quantity                   | Mean        | Std dev  | Min         | Max
Volume (Ang^3)             | 691.4382    | 0.0000   | 691.4382    | 691.4382
Energy, total (eV)         | -1029.5308  | 0.0029   | -1029.5339  | -1029.5154
Energy, potential (eV)     | -1030.4177  | 0.4158   | -1031.0612  | -1029.5250
Temperature (K)            | 503.0835    | 251.5614 | 137.9600    | 1250.3500
Pressure (kBar)            | -4.7330     | 13.6222  | -23.0030    | 29.1470
Energy, total (eV/atom)    | -128.691355 | 0.000363 | -128.691739 | -128.689429
Energy, potential (eV/atom) | -128.802211 | 0.051971 | -128.882655 | -128.690628
```

`-1029.5308 eV / 8 atoms = -128.691355 eV/atom` -- the per-atom rows are exactly what `siesta_energy.dat`'s plotted columns (2/3) and `siesta_energy.gplot/siesta_thermo.gplot`'s y-axis carry (section 1.7 above), while these absolute-eV rows and their own extra `.dat` columns stay available for reference.

Three independent physical sanity checks all pass on this real data:

- **Volume is exactly constant** (std = 0.0000 Angˆ3) -- correct for a fixed-cell NVT run (Nose thermostat controls temperature, not cell shape/volume).
- **Temperature fluctuates around the 500 K Nose target** (mean 503.1 K) -- expected for a small 8-atom system's canonical-ensemble fluctuations, not clamped the way a barostat/thermostat's instantaneous value never is.
- **`E_tot` is ~140x more stable than `E_KS`** (std 0.0029 eV vs. 0.4158 eV) -- confirming `E_tot`, not `E_KS`, is the physically appropriate "conserved-ish" quantity to judge stability by (section 1.6 above).

8.18.1.6 6. `--trajectory`: independent of SIESTA

Not every trajectory this tool can read comes from SIESTA -- `stb-mlmd` (a MACE-driven MD run) writes the same xsf/pdb/xyz formats `stb-aimdAnalysis --trajectory` accepts. Because that input isn't guaranteed to be a SIESTA calculation, `[9] REFERENCES` correctly skips the SIESTA citation for it instead of assuming one:

```
$ stb-aimdAnalysis --trajectory synthetic_traj.xyz --track-pair 0-1 --no-intro
[9] REFERENCES
No SIESTA-specific references for a generic --trajectory input -- cite
whichever tool produced it instead (e.g. stb-mlmd's own MACE/foundation
-model references).
```

An extended-xyz trajectory written with per-frame `Time` info (e.g. `stb-mlmd --out-format xyz`) auto-detects `dt`; `xsf`/`pdb` carry no such data, so `--dt` is required for those.

8.18.1.7 Known, deliberate limitations

- The VDOS is qualitative, not benchmark-grade (section 1.3).
- This example's small `aimd.*` fixture (5 frames) is far too short for the MSD/diffusion or VDOS numbers to carry real physical meaning -- it only demonstrates that the pipeline runs correctly end to end.
- `--track-pair`'s minimum-image convention assumes no atom moves more than half a cell length between frames (the same assumption the RDF and PBC-unwrapping already make elsewhere in this suite).

8.18.1.8 Files in this folder

File	What it is
`aimd.ANI/.XV/.fdf/.out`	The real 5-step SIESTA AIMD run (O2 dimer in vacuum)
`siesta.ANI/.XV/.MDE`, `calc.fdf`, `structure.fdf`	The real 500-step SIESTA NVT run (8-atom SiC supercell, Nose thermostat) -- `SystemLabel siesta`, real input `calc.fdf`
`example_2.18.sh`	This walkthrough's runnable script

8.18.1.9 Running it

```
bash example_2.18.sh
```

Or try the tool directly:

```
stb-aimdAnalysis --label aimd --track-atom 0 --track-pair 0-1 \
    --save-report --save-gnuplot --view
```

8.18.1.10 What's next

- `stb-ani2traj` (Utils) -- converts the same `.ANI` trajectory's *format* for external viewers (OVITO/VMD), computing nothing; run it alongside this tool to actually watch the motion this tool quantifies.
- `stb-mlmd` (ML Simulations) -- a MACE-driven MD run whose own xsf/pdb/xyz output is exactly what `--trajectory` reads in section 6 above.

8.18.2 Implementation details

Input.. Either a SIESTA run, `-label` (`<label>.ANI` for positions, `<label>.MDE` for energy, temperature, volume and pressure, the cell of every step from the log, the `.XV` or the `.fdf`, and the time step from the input `.fdf`, which `-geometry-file` points to when it is not named after the label), or any multi-frame file ASE reads, `-trajectory` (for example from **stb-mlmd**), with `-dt` the time between frames unless an extended-XYZ file carries it. `-stride` keeps every N-th frame and `-skip` discards the first frames as equilibration. Variable-cell (NPT) runs are handled: each frame is analysed with its own cell.

The quantities

RDF
$g(r) = \frac{V}{N_a N_b} \left\langle \sum_{i\in a} \sum_{j\in b} \frac{\delta(r - r_{ij})}{4\pi r^2} \right\rangle$, with minimum-image distances, averaged over frames (`-pair A-B` for one species pair, `-r-max` up to half the smallest cell width, `-n-bins`).

MSD and diffusion
$\mathrm{MSD}(t) = \langle |\mathbf{r}_i(t_0+t) - \mathbf{r}_i(t_0)|^2 \rangle_{i,t_0}$ on unwrapped coordinates with all time origins, per species; the Einstein relation $D = \lim_{t\to\infty} \mathrm{MSD}(t)/6t$ is applied to a linear fit between `-fit-start` and `-fit-end` (default: second half), reported in cm^2/s.

VDOS
the Fourier transform of the velocity autocorrelation function (Wiener–Khinchin), in cm^{-1}. Velocities are central differences of positions, since the `.ANI` has none, so the spectrum is qualitative and limited by the frame interval.

Thermodynamics
energy per atom (E_{tot} and E_{KS}), temperature, volume and pressure against time, in one four-panel figure; only E_{tot} should be conserved (NVE) or fluctuate about a constant (NVT).

Tracking single atoms.. `-list-atoms` prints indices, species and positions of the first frame and exits. `-track-atom N` follows one atom's displacement from its start (unwrapped, continuous across boundaries); `-track-pair I-J` follows the true minimum-image distance between two atoms – for a bond length or a reaction coordinate.

Statistics.. A diffusion coefficient needs a clearly linear MSD over a time much longer than the ballistic regime; with short runs or few atoms the error bar is large. Compare fits over two windows, and prefer the NVT or NVE production part of the run (`-skip`).

8.18.3 Command-line reference

8.18.3.1 `stb-aimdAnalysis`

Physical post-processing of an AIMD/MD trajectory – a SIESTA AIMD run (`<label>.ANI`) or a generic ASE-readable trajectory (e.g. one written by `stb-mlmd`): radial distribution function g(r), mean-squared displacement (MSD) and diffusion coefficient, a VACF-derived vibrational density of state (VDOS), single-atom displacement tracking, and atom-pair relative distance tracking. Complements `stb-ani2traj`, which only converts a SIESTA trajectory's format for external viewers.

Synopsis.

```
usage: stb-aimdAnalysis [-h] [-l LABEL] [--geometry-file PATH] [--trajectory PATH]
                        [--dt FS] [--stride STRIDE] [--skip SKIP] [--pair A-B]
```

```
[--r-max R_MAX] [--n-bins N_BINS] [--fit-start FS]
[--fit-end FS] [--track-atom N] [--track-pair I-J]
[--list-atoms] [-o OUTPUT_DIR] [--save-report]
[--save-gnuplot] [--view] [--no-intro] [-v]
```

Options

`-l, --label LABEL`

SIESTA SystemLabel – reads '`<label>.ANI`'. Mutually exclusive with `--trajectory`.

`--geometry-file PATH`

Explicit path to the real SIESTA .fdf input file (`--label` mode only). MD.InitialTimeStep/MD.LengthTimeStep (the real MD timestep) and, if '<label>.XV'/'<label>.out' aren't usable, the lattice itself are read from here. Auto-detecting '`<label>.fdf`' almost never works in practice – the input file's own name is chosen by the user and is very often NOT `<label>.fdf` (e.g. SystemLabel 'siesta' with the real input called `calc.fdf`) – pass this explicitly instead of relying on that guess.

`--trajectory PATH`

Read a generic ASE-readable multi-frame trajectory instead of a SIESTA .ANI – e.g. one written by `stb-mlmd` (xsf/pdb/xyz). Mutually exclusive with `--label`.

`--dt FS`

Real time between saved frames in the `--trajectory` file, in fs (`--trajectory` mode only). Auto-detected if the file is an extended-xyz with per-frame 'Time' info (e.g. `stb-mlmd --out-format` xyz); required otherwise (xsf/pdb carry no such data). type: float

`--stride STRIDE`

Keep only every Nth frame (default: 1, every frame). type: int

`--skip SKIP`

Discard the first N frames (post-stride) as thermal equilibration, before any statistics are computed (default: 0). type: int

`--pair A-B`

Restrict the RDF to one species pair, e.g. 'O-H' (default: total RDF over all pairs).

`--r-max R_MAX`

RDF cutoff in Angstrom (default: the minimum-image limit of the smallest cell found in the trajectory).type: float

`--n-bins N_BINS`

Number of RDF bins (default: 200). type: int

`--fit-start FS`

MSD linear-fit window start, in fs (default: half the trajectory length). type: float

`--fit-end FS`

MSD linear-fit window end, in fs (default: end of trajectory). type: float

`--track-atom N`

0-based atom index whose own Cartesian displacement (from its initial position, PBC-unwrapped – same convention as the MSD) is tracked and reported over time (optional). type: int

`--track-pair I-J`

Two 0-based atom indices, e.g. '0-5', whose relative distance (minimum-image convention, same as the RDF) is tracked and reported over time (optional).

`--list-atoms`

Print a table of every atom's 0-based index, species, and Cartesian coordinates (Ang, from the first frame only) to help pick `--track-atom`/`--track-pair` indices, then exit immediately – no RDF/MSD/VDOS/report is computed. Off by default (a large structure can have hundreds of atoms). switch, off by default

`-o, --output-dir OUTPUT_DIR`
Directory to write <stem>_*.dat/.gplot (with `--save-gnuplot`) and stb_aimdAnalysis_report.txt/references.bib into (default: current directory). Created if it doesn't exist.

`--save-report`
Also persist the report to `stb_aimdAnalysis_report.txt`. Off by default. switch, off by default

`--save-gnuplot`
Also write a .dat + .gplot pair for every computed quantity (RDF, MSD, VACF, VDOS, and the atom-displacement/pair-distance series if `--track-atom`/`--track-pair` are used). Off by default – this tool previously wrote matplotlib PNGs unconditionally on every run with no gnuplot output at all. switch, off by default

`--view`
Show an interactive matplotlib preview before finishing. Off by default – this tool previously always generated (and saved) the matplotlib plots, with no way to skip it. switch, off by default

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Example usage:
  stb-aimdAnalysis --label aimd
  stb-aimdAnalysis -l aimd --pair O-H --skip 50
  stb-aimdAnalysis -l aimd --fit-start 5000 --fit-end 15000
  stb-aimdAnalysis -l aimd --track-atom 0 --track-pair 0-1
  stb-aimdAnalysis -l aimd --save-report --save-gnuplot --view
  stb-aimdAnalysis --trajectory si_bulk_md_traj.xyz --dt 10
```

8.19 Native Charges

At a glance

Menu code: 2.19 **Command:** `stb-nativecharges`
Guided example: `examples/2.19-stb-nativecharges/`
Summary: Read and compare the atomic charges SIESTA prints in its log – Mulliken, Hirshfeld, Voronoi – per atom and per species, with the spread between methods, sum checks and the point-charge dipole against SIESTA's.

8.19.1 Overview

SIESTA prints Mulliken [54] (`WriteMullikenPop 1`), Hirshfeld [55] (`Charge.Hirshfeld end`) and Voronoi [56] (`Charge.Voronoi end`) atomic charges at the end of its log. **`stb-nativecharges`** reads the last block of each and reports, for one folder, the charges per atom side by side with the spread between methods (and the spin populations of a spin-polarized run), per species with the spread over equivalent atoms, the checks – the charges add up to minus the net charge, population + charge = Z_{val} on every atom, the spin populations add up to SIESTA's spin moment –, and for a molecule or slab the dipole of the charges as point charges against SIESTA's own dipole of the density. Several folders (`-path` glob) give one table per species and method; `-csv` writes every atom. It warns when the log has none of the blocks (naming the flags) and when Mulliken and Hirshfeld disagree in sign, the signature of Mulliken's basis dependence. No new partition is computed: for iterative Hirshfeld-I [57] use **5.17**, for Bader **2.6**.

```
stb-nativecharges --path nacl                        # one folder
stb-nativecharges --path 'h2o_basis/*' --csv q.csv   # a series: one table per species and method
```

8.19.2 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/2.19-stb-nativecharges/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

At the end of a run, SIESTA can print three tables of atomic charges, each from its own way of cutting one electron density into atoms. `WriteMullikenPop 1` gives Mulliken, `Charge.Hirshfeld end` gives Hirshfeld, and `Charge.Voronoi end` gives Voronoi. `stb-nativecharges` reads them from the log and compares them:

- **per atom, side by side**, with the spread between the methods, and the spin populations of a spin-polarized run;
- per species, with the spread over equivalent atoms;
- **checks**: the charges add up to the net charge, population + charge = Z_val on every atom, the spin populations add up to SIESTA's spin moment;
- for a molecule or a slab, **the dipole of the charges as point charges** against the one SIESTA computes from the density;
- for several folders, one table per species and method, and a CSV of every atom.

It computes no new partition: the iterative Hirshfeld-I is `5.17` and Bader is `2.6`.

The example uses real SIESTA 5.4 runs: water with four basis sets, rock-salt NaCl, and the O_2 triplet. They show the facts to know before using a partial charge. Mulliken charges follow the basis, even in sign. The three methods disagree by 0.36 e on one atom of water. For an ionic solid they all give a small fraction of the formal ±1. And point charges miss half of a molecule's dipole. Everything runs in seconds; no SIESTA is needed.

8.19.2.1 Contents

1. Theory
2. The comparison at a glance
3. Libraries and external dependencies used
4. What `stb-nativecharges` does
5. Running it both ways
6. Worked example
7. What to take from the example
8. Reproducing the reference data
9. Known, deliberate limitations
10. Step-by-step: charges of your own runs
11. Files in this folder
12. Running the script
13. Flag reference
14. What's next
15. References

8.19.2.2 1. Theory

1.1 A partial charge is a partition. The electron density $\rho(r)$ is an observable; the charge "of an atom" is not. To get one, ρ has to be divided among the atoms, and every method does that its own way:

```
q_A = Z_val,A − N_A,     N_A = the share of the valence electrons given to atom A
(1)
```

SIESTA [1] works with valence electrons and pseudopotentials, so `Z_val` (4 for Si, 6 for O, 9 for Na with its 2s2p semicore) replaces the atomic number. A positive q means electrons lost. The three partitions SIESTA prints:

Method	Partition	Depends on	Flag
Mulliken [2]	in orbital space: each orbital's population goes to its own atom, each **overlap** population is split half and half between the two atoms	the basis set, strongly	`WriteMullikenPop 1`
Hirshfeld [3]	in real space, by weights: at every point, atom A gets the fraction ρ^0_A(r)/Σ_B ρ^0_B(r) of the density, ρ^0 the free neutral atoms	the free-atom densities (the "promolecule")	`Charge.Hirshfeld end`
Voronoi [4]	in real space, by cells: each point goes to the nearest atom	the geometry (all atoms equal size)	`Charge.Voronoi end`

Each partition has its known biases:

- **Mulliken** has no notion of space. A diffuse or polarization orbital on O that really describes density near H still counts as O's, so the charges change with the basis, sometimes in sign (section 6.3).
- **Hirshfeld** weights by neutral free atoms, so it pulls charges toward zero. For ionic solids the result is small (section 6.4); the iterative **Hirshfeld-I** [5] corrects this with charged references (5.17).
- **Voronoi** cells are equal for a small and a large atom, which biases ionic systems the other way.

None of them is "the" charge. Only differences computed **with one method and one basis** carry meaning: charge transfer on adsorption, the change along a reaction path.

1.2 The checks. For every method:

```
Σ_A q_A = −NetCharge          (0 for a neutral cell)
N_A + q_A = Z_val,A           (population + charge, per atom)
Σ_A Sz_A = S                   (the spin moment, spin-polarized runs)
(2)
```

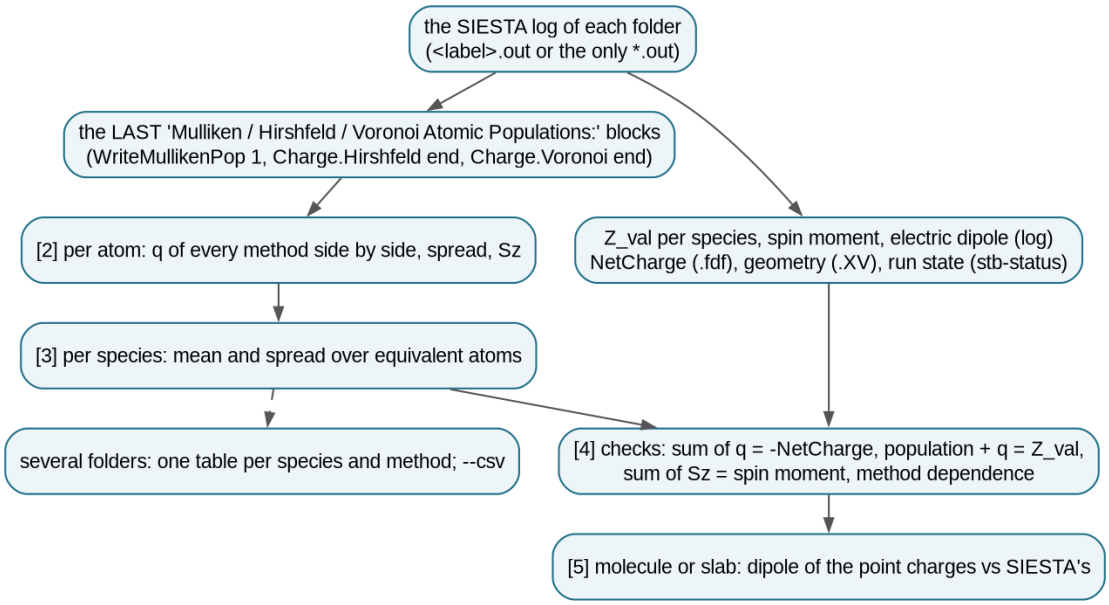
The first two hold by construction when the block is complete. A failure means a truncated log, a mismatched `Z_val`, or a run that printed its charges before converging. The third checks that the spin populations belong to the moment SIESTA reports.

1.3 The dipole of point charges. For a molecule (or a slab, along its normal), the charges define a dipole if they are placed on the nuclei:

```
p_point = Σ_A q_A (R_A − R_centre)
(3)
```

SIESTA's own dipole comes from the full density, `p = Σ Z_val R − ∫ρ r`. The two differ by every atom's **own** dipole, the shape of its share of ρ: lone pairs, polarization. Partial charges that reproduce the molecular dipole exist (fitted electrostatic-potential charges), but none of the three above is built to. The report gives the fraction of SIESTA's dipole that each method's point charges carry, along each non-periodic direction.

8.19.2.3 2. The comparison at a glance



stb-nativecharges flowchart

Figure 1. What `stb-nativecharges` *reads and reports.*

8.19.2.4 3. Libraries and external dependencies used

Library	Used for
`stb.core.siesta_log`	the three population blocks (`get_mulliken_charges`, `get_hirshfeld_charges`, `get_voronoi_charges`), Z_val, spin moment
`stb.status`	the run's state, atoms and settings
`sisl` [6]	the geometry (`.XV`) for the dipole
`numpy`	statistics, dipoles

8.19.2.5 4. What `stb-nativecharges` does

4.1 Step by step.

1. **Find the log** of each folder (`<label>.out` or the only `*.out`; the label from the `.fdf` or `-l`).
2. **Read the last** Mulliken, Hirshfeld and Voronoi blocks: the final geometry of a relaxation.
3. **Read** `Z_val` per species, the spin moment and the electric dipole from the log, `NetCharge` and the basis from the `.fdf`, the geometry from the `.XV`, the state from `stb-status`.
4. **Report** per atom, per species, the checks and the dipole; for a glob, one table per species and method.

4.2 The report, section by section.

Section	What it shows
`[0] RUN METADATA`	date, version, target, the three methods and their flags
`[1] RUN`	folder, log, state and run type, atoms, basis, spin, net charge, which methods the log has (and the flag for each missing one)
`[2] CHARGES PER ATOM`	index, element, Z_val, q of each method, spread; Sz of each method for a spin-polarized run
`[3] PER SPECIES`	mean q of each method, with the spread over the atoms of the species
`[4] CHECKS`	per method: sum of q, max |population + q − Z_val|, sum of Sz; the expected values; the largest method dependence
`[5] DIPOLE`	molecule or slab: each method's point-charge dipole along the vacuum axes, |p| and the fraction of SIESTA's dipole
`[6] SUMMARY & FILES`	how many folders have each method; the CSV
`[7] WARNINGS`	every warning, numbered

With several folders, `[1] CHARGES PER SPECIES` (one row per folder and species, one column per method) replaces sections 1–5, and `--details` adds the full report of each folder.

4.3 Guard rails. The report warns when the log has none of the three blocks (naming the flags to add), when a method's charges do not add up to the net charge, when Mulliken and Hirshfeld disagree in sign on an atom (a basis artefact), and when the run did not finish.

8.19.2.6 5. Running it both ways

From the command line:

```
stb-nativecharges                                  # this folder
stb-nativecharges --path nacl                      # another folder
stb-nativecharges --path 'h2o_basis/*'             # a series: one table per species and method
stb-nativecharges --path 'h2o_basis/*' --csv charges.csv --details
```

From the menu (`stb-suite`, code `2.19`): the folder or glob, a label, the full reports (for a glob), the CSV and the report are asked in turn.

8.19.2.7 6. Worked example

`runs/` holds SIESTA 5.4.2 runs [1] (PBE [7], PseudoDojo pseudopotentials [8]), each with the three charge flags, made by `run_example.sh` in 16 s:

- **`h2o_basis/{SZ,DZ,DZP,TZP}/`**: one water molecule at the experimental geometry (O–H 0.9572 Å, 104.52°) in a 10 Å box, 250 Ry, with four basis sizes;
- **`nacl/`**: rock-salt NaCl, 2-atom cell, a = 5.64 Å, DZP, 6×6×6 k-points (Na with its 2s2p semicore: Z_val 9);
- **`o2/`**: the O_2 triplet in an 8 Å box, spin-polarized, DZP.

Each keeps only `calc.fdf`, `calc.out` and the `.XV` (256 KB in all).

6.1 Water, DZP: the whole report (Case 1).

```
stb-nativecharges --path h2o_basis/DZP
```

```
[2] CHARGES PER ATOM
# | el | Z_val | q Mulliken | q Hirshfeld | q Voronoi | spread
1 | O  | 6     | +0.1183    | -0.2433     | -0.1934   | 0.3616
2 | H  | 1     | -0.0592    | +0.1216     | +0.0967   | 0.1808
3 | H  | 1     | -0.0592    | +0.1216     | +0.0967   | 0.1808

[4] CHECKS
method    | sum of q (e) | max |pop + q - Z_val| | sum of Sz
Mulliken  | +0.000000    | 1.1e-16               | -
Hirshfeld | -0.000003    | 0.0e+00               | -
Voronoi   | -0.000003    | 0.0e+00               | -
Method dependence : up to 0.362 e (O1) -- a partial charge is a convention, not an observable

[7] WARNINGS
  1. h2o_basis/DZP: Mulliken and Hirshfeld disagree in SIGN on O1, H2, H3 -- a basis-set artefact of
  Mulliken
     (diffuse/polarization orbitals); prefer Hirshfeld/Voronoi (or Bader, Hirshfeld-I).
```

Hirshfeld and Voronoi make O negative, as its electronegativity says. Mulliken makes it positive, and with the same density. All three pass the checks of Eq. 2, so the disagreement is not a bug: it is what each partition means.

6.2 The dipole of point charges (Case 2).

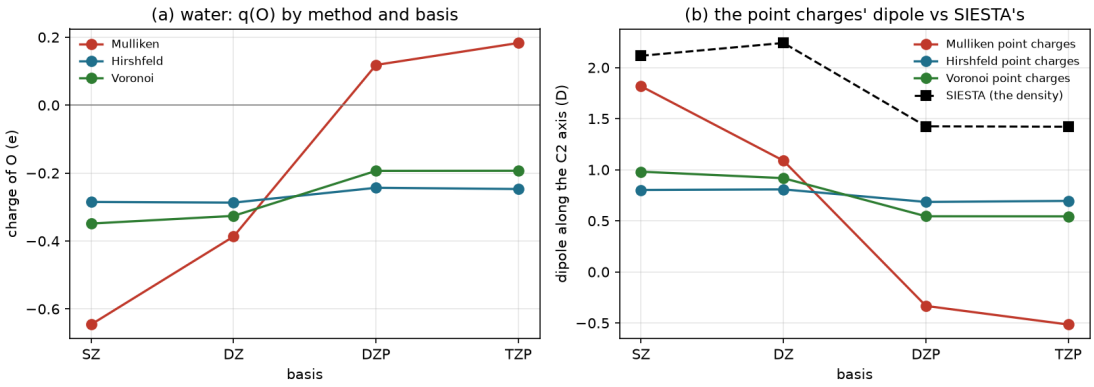
```
[5] DIPOLE
source           | along a (D) | along b (D) | along c (D) | |p| (D) | of SIESTA's
Mulliken         | +0.0000     | -0.0000     | -0.3329     | 0.3329  | 23 %
Hirshfeld        | +0.0000     | +0.0000     | +0.6846     | 0.6846  | 48 %
Voronoi          | +0.0000     | +0.0000     | +0.5444     | 0.5444  | 38 %
SIESTA (density) | +0.0000     | +0.0000     | +1.4246     | 1.4246  | 100 %
```

The molecule's C_2 axis is z. Hirshfeld point charges carry under half of the dipole, and Mulliken's point the wrong way. The missing part is in the shape of each atom's share of the density, mostly the O lone pairs (Eq. 3).

6.3 Four basis sets: Mulliken's sign flip (Case 3).

```
stb-nativecharges --path 'h2o_basis/*' --csv h2o_basis_charges.csv
```

```
folder        | basis | species | atoms | mean q Mulliken | mean q Hirshfeld | mean q Voronoi
h2o_basis/DZ  | DZ    | O       | 1     | -0.3868         | -0.2870          | -0.3260
h2o_basis/DZP | DZP   | O       | 1     | +0.1183         | -0.2433          | -0.1934
h2o_basis/SZ  | SZ    | O       | 1     | -0.6458         | -0.2850          | -0.3486
h2o_basis/TZP | TZP   | O       | 1     | +0.1831         | -0.2469          | -0.1931
...
```



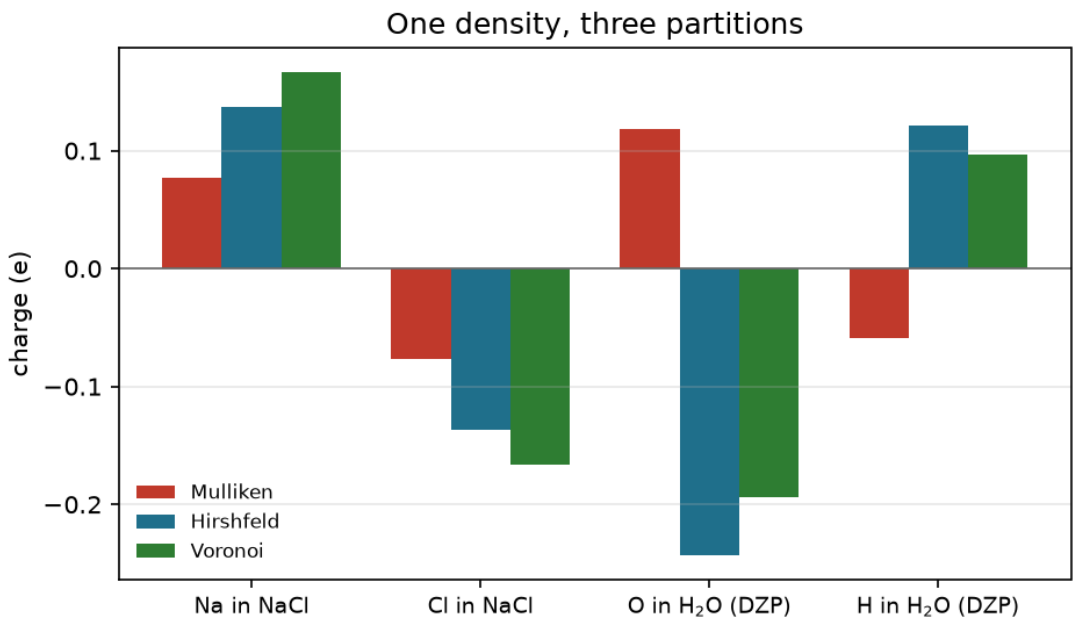
Basis dependence

Figure 2. Water. (a) The charge of O: Mulliken goes from −0.65 (SZ) to +0.18 (TZP); Hirshfeld stays within −0.29…−0.24, Voronoi within −0.35…−0.19. (b) The point charges' dipole (Eq. 3) against SIESTA's.

With a minimal basis (SZ), Mulliken's charges even reproduce 86 % of the dipole. That is a coincidence of a small basis: as soon as polarization orbitals appear (DZP), the overlap populations shift and Mulliken's O turns positive. Hirshfeld changes by 0.04 e over the same series. SIESTA's own dipole also depends on the basis: 2.1–2.2 D without polarization orbitals, 1.42 D with them (experiment 1.85 D [9]). The density changes too; the charges only magnify it.

6.4 NaCl: an ionic solid (Case 4).

```
# | el | Z_val | q Mulliken | q Hirshfeld | q Voronoi | spread
1 | Na | 9     | +0.0771    | +0.1370     | +0.1664   | 0.0893
2 | Cl | 7     | -0.0771    | -0.1370     | -0.1664   | 0.0893
```



Three partitions

Figure 3. One density, three partitions: NaCl and water (DZP).

A textbook ionic crystal gets charges of ±0.08 to ±0.17, a tenth of the formal ±1. Hirshfeld's neutral reference atoms pull every charge toward zero. The example of `5.17` computes, for its own NaCl run (a = 5.674 Å), Hirshfeld ±0.14 (the same as here), Hirshfeld-I ±0.51 and Bader ±0.86 [5, 10]. The value depends on the method more than on the chemistry. "Na loses 0.14 e" is a statement about Hirshfeld, not about sodium.

6.5 The O_2 triplet: spin (Case 5).

```
# | el | Z_val | q Mulliken | q Hirshfeld | q Voronoi | spread | Sz Mulliken | Sz Hirshfeld | Sz
Voronoi
1 | O  | 6     | +0.0000    | +0.0000     | +0.0000   | 0.0000 | +1.0000     | +1.0000      | +1.0000
2 | O  | 6     | +0.0000    | +0.0000     | +0.0000   | 0.0000 | +1.0000     | +1.0000      | +1.0000
Expected          : sum of q = +0 e; sum of Sz = spin moment 2.00000
```

By symmetry the two atoms share everything: no charge, one unpaired electron each, and the spin populations of every method add up to SIESTA's moment.

6.6 A log without the blocks; the interactive menu (Case 6). The NaCl log with the three blocks cut out, as SIESTA writes it without the flags (`reference_results/guard_nocharges/`):

```
Mulliken          : not in the log (add WriteMullikenPop 1)
Hirshfeld         : not in the log (add Charge.Hirshfeld end)
Voronoi           : not in the log (add Charge.Voronoi end)
```

`stb-suite` → `2.19` with `h2o_basis/*` gives the same table as Case 3 (`reference_results/menu/report.txt`).

8.19.2.8 7. What to take from the example

- **A partial charge belongs to its method.** On one water molecule the three methods differ by 0.36 e; compare charges only within one method and one basis.
- **Mulliken follows the basis**, here even in sign; prefer Hirshfeld or Voronoi for anything quantitative, and Hirshfeld-I or Bader for ionic systems.
- **Small charges in an ionic solid** (±0.14 for NaCl) are a feature of the partition, not of the bonding.
- **Point charges are not the dipole**: they carry about half of it here.

8.19.2.9 8. Reproducing the reference data

```
source ~/.anaconda3/etc/profile.d/conda.sh && conda activate stb   # or any environment with
stb_suite
bash make_reference.sh          # every report into reference_results/<case>/ (seconds, no SIESTA)
bash run_example.sh 4           # runs/ itself (SIESTA 5.x + mpirun, ~16 s)
python3 theory_figures.py       # figures/basis_dependence.png, methods.png
dot -Tpng -Gdpi=150 figures/charges_flowchart.dot -o figures/charges_flowchart.png
```

8.19.2.10 9. Known, deliberate limitations

- **Only what SIESTA printed**: no new partition is computed. Hirshfeld-I is `5.17`, Bader `2.6`.
- **The last block of each kind** is read: the final geometry of a relaxation. Charges along a trajectory need one log per step (or `stb-hirshfeld*`).
- **Mulliken's orbital-resolved populations** (`WriteMullikenPop 2/3`) are not tabulated; only the atomic charges.
- **The dipole needs vacuum** (a gap of at least 4 Å) and the `.XV`; along periodic directions it is not defined this way.
- **Non-collinear spin**: the Sz column is read as printed; the spin direction is not.

8.19.2.11 10. Step-by-step: charges of your own runs

1. **Add the flags** to the `.fdf`: `WriteMullikenPop 1`, `Charge.Hirshfeld end`, `Charge.Voronoi end`.
2. **Run** `stb-nativecharges --path <folder>`: read the checks, the spread and the warnings.
3. **Pick one method** for your comparison (Hirshfeld is the safest of the three) and keep the basis fixed.
4. **For a series** (adsorption, strain, doping), use a glob and `--csv`, and compare differences, not absolute values.

8.19.2.12 11. Files in this folder

File	What it is
`README.md`	this tutorial
`example_2.19.sh`	the guided walk-through (6 cases, seconds)
`runs/`	the six SIESTA 5.4.2 runs (`h2o_basis/*`, `nacl/`, `o2/`: input, log, `.XV`)
`run_example.sh`	makes `runs/` (needs SIESTA)
`make_reference.sh`	regenerates `reference_results/`
`theory_figures.py`	Figures 2 and 3

File	What it is
`figures/`	every figure of this README; `charges_flowchart.dot` is the flowchart's source
`reference_results/`	one folder per case: `report.txt`; `h2o_basis/h2o_basis_charges.csv`

8.19.2.13 12. Running the script

```
bash example_2.19.sh               # pauses between cases
yes '' | bash example_2.19.sh      # straight through
```

It needs the `stb_suite` package only (no SIESTA). It starts by wiping its own `output/`, never touches `runs/`, and takes a few seconds.

8.19.2.14 13. Flag reference

Flag	Default	What it does
`--path PATH`	.	a folder, or a quoted glob of folders
`-l, --label LABEL`	from each folder's `.fdf`	SystemLabel override
`--details`	off	with several folders, also the full report of each
`--csv FILE`	none	every atom's charges (all folders, all methods) as CSV
`--save-report`	off	also write `stb_nativecharges_report.txt` (current folder)
`--no-intro, -v`		skip the banner; print the version

8.19.2.15 14. What's next

- **`5.17` (Hirshfeld-I)** computes iterative Hirshfeld charges from SIESTA's density, with charged references.
- **`2.6` (`stb-bader`)** partitions the density by zero-flux surfaces.
- **`5.16` (charge density difference)** shows where charge moves on bonding, without assigning it to atoms.

8.19.2.16 15. References

1. J. M. Soler et al., "The SIESTA method for ab initio order-N materials simulation", *J. Phys.: Condens. Matter* **14**, 2745 (2002).
2. R. S. Mulliken, "Electronic population analysis on LCAO–MO molecular wave functions. I", *J. Chem. Phys.* **23**, 1833 (1955).
3. F. L. Hirshfeld, "Bonded-atom fragments for describing molecular charge densities", *Theor. Chim. Acta* **44**, 129 (1977).
4. C. Fonseca Guerra, J.-W. Handgraaf, E. J. Baerends, F. M. Bickelhaupt, "Voronoi deformation density (VDD) charges", *J. Comput. Chem.* **25**, 189 (2004).
5. P. Bultinck, C. Van Alsenoy, P. W. Ayers, R. Carbó-Dorca, "Critical analysis and extension of the Hirshfeld atoms in molecules", *J. Chem. Phys.* **126**, 144111 (2007).
6. N. Papior, sisl: v0.16, https://github.com/zerothi/sisl (2024).

7. J. P. Perdew, K. Burke, M. Ernzerhof, "Generalized gradient approximation made simple", *Phys. Rev. Lett.* **77**, 3865 (1996).
8. M. J. van Setten et al., "The PseudoDojo: Training and grading a 85 element optimized norm-conserving pseudopotential table", *Comput. Phys. Commun.* **226**, 39 (2018).
9. S. A. Clough, Y. Beers, G. P. Klein, L. S. Rothman, "Dipole moment of water from Stark measurements of H_2O, HDO, and D_2O", *J. Chem. Phys.* **59**, 2254 (1973).
10. W. Tang, E. Sanville, G. Henkelman, "A grid-based Bader analysis algorithm without lattice bias", *J. Phys.: Condens. Matter* **21**, 084204 (2009).

8.19.3 Command-line reference

8.19.3.1 `stb-nativecharges`

Read and compare the atomic charges SIESTA prints in its log – Mulliken (WriteMullikenPop 1), Hirshfeld (Charge.Hirshfeld end) and Voronoi (Charge.Voronoi end): per atom side by side, per species, the spread between methods, checks (sum = net charge, population + q = Z_val, sum of Sz = spin moment) and, for a molecule or slab, the dipole of the point charges vs SIESTA's. Several folders (a glob) give one table per species. No new partitioning is computed (Hirshfeld-I: 5.17, Bader: 2.6).

Synopsis.

```
usage: stb-nativecharges [-h] [--path PATH] [-l LABEL] [--details] [--csv CSV]
                         [--save-report] [--no-intro] [-v]
```

Options

`--path PATH`
: Directory to inspect, or a glob pattern (e.g. 'strain_*') matching several directories (default: current directory).

`-l, --label LABEL`
: SystemLabel override, applied to every directory. Auto-detected per directory from its .fdf file(s) if not given.

`--details`
: With several folders, also print the full report of each one. switch, off by default

`--csv CSV`
: Also write every atom's charges (all folders, all methods) as CSV.

`--save-report`
: Also write the report to `stb_nativecharges_report.txt` (current folder). Off by default. switch, off by default

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Example usage:
  stb-nativecharges                          # this folder
  stb-nativecharges --path nacl              # another folder
  stb-nativecharges --path 'h2o_basis/*'     # a series: one table per species and method
  stb-nativecharges --path 'h2o_basis/*' --csv charges.csv
```

Chapter 9

Utils

In this chapter

9.1 File Translator . . . 187
9.2 Clean File Tools . . . 203
9.3 Grid to Cube Converter . . . 216
9.4 Wantibexos Interface . . . 231
9.5 AIMD Trajectory Converter . . . 244
9.6 Status Checker . . . 257
9.7 Archive Packager . . . 268

Helpers for everyday file handling: converting structures between formats, cleaning and archiving calculation folders, converting grids to Gaussian cube files, exporting the SIESTA Hamiltonian to WanTiBEXOS [58], turning an MD trajectory into a viewer-friendly format, summarizing the state of many runs, and packaging a finished run into a verified archive.

Code	Tool	Purpose
3.1	File Translator	Convert between file formats (CIF, POSCAR, fdf, xyz...).
3.2	Clean File Tools	Reset a calc folder for a new run: keeps the inputs, archives and checks the rest, reports what the next run starts from.
3.3	Grid to Cube Converter	Write a SIESTA grid (RHO, DRHO, BADER, TOCH, VT, VH, ...) as a Gaussian cube, checked: units, integral vs expected, spin, dipole, read-back.
3.4	Wantibexos Interface	Write the SIESTA H and S (SaveHS) as WanTiBEXOS tight-binding files, with a report and a read-back check of the bands.
3.5	AIMD Trajectory Converter	Convert a SIESTA AIMD .ANI trajectory into a multi-frame file with the cell per frame, with a report on the run and a read-back check.
3.6	Status Checker	State of calc folders (finished/failed/running/stopped), progress vs tolerances, SCF, energies, gap, forces, next tools; several folders as one table.
3.7	Archive Packager	Package a calc folder into a verified .tar.gz (inputs always in, MANIFEST and SHA-256 checksums), the complement of `stb-clean`.

9.1 File Translator

At a glance

Menu code: 3.1 **Command:** `stb-translate`
Guided example: `examples/3.1-stb-translate/`
Summary: Convert between structure formats (fdf, STRUCT_OUT, POSCAR, CIF, XYZ, XSF, FHI-aims, DFTB+, LAMMPS), formats inferred from the file names, with a report of the structure and its symmetry and a check of the written file.

9.1.1 Overview

`stb-translate` converts a crystal structure between the formats used around SIESTA and reports what it read and whether what it wrote is the same structure. Both formats come from the file names when `-if`/`-of` are omitted; `-cf direct|cartesian` chooses the output coordinates where the format allows both; `-i` accepts a glob for a batch into `-out-dir`; `-dry-run` reads and reports only. The report gives the coordinates as stored and their range (a Direct/Cartesian mix-up is flagged), the structure (formula, cell, density, periodicity, shortest distance per element pair against the covalent radii), the space group at 0.001, 0.01 and 0.1 Å with the primitive cell, and reads the written file back to compare it with the input. A CIF is written symmetrized at `-cif-symprec` (0.01 Å; 0 writes P1 exactly as read), and then compared with pymatgen's `StructureMatcher`, since its cell is re-expressed in the standard setting.

```
stb-translate -i CONTCAR -o structure.fdf             # both formats from the names
stb-translate -i siesta.STRUCT_OUT -o relaxed.cif --save-report
stb-translate -i '*.cif' -of fdf --out-dir fdf/
stb-translate -i slab.fdf --dry-run                   # report only
```

9.1.2 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/3.1-stb-translate/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

`stb-translate` converts a crystal structure between the file formats used around SIESTA, and tells you what it read and whether what it wrote is the same structure. It reads and writes:

Format	Read	Write	Typical file
`fdf`	yes	yes	SIESTA input: `structure.fdf`
`siesta`	yes	—	SIESTA output: `<label>.STRUCT_OUT` (the relaxed cell and positions)
`poscar`	yes	yes	VASP: `POSCAR`, `CONTCAR`, `*.vasp`
`cif`	yes	yes	Crystallographic Information File: databases, papers, VESTA
`xyz`	yes	yes	plain XYZ: molecules, visualisers (no cell)
`xsf`	yes	yes	XCrySDen
`fhi`	yes	yes	FHI-aims `geometry.in`
`dftb`	yes	yes	DFTB+ `.gen`
`lammps`	—	yes	LAMMPS data file (`atom_style atomic`, metal units)

Both formats are taken from the file names when `-if`/`-of` are omitted. For every file it reports:

- what was read: the coordinates as stored and their range;
- the structure: composition, cell, volume, density, periodicity, shortest bonds;
- its symmetry at three tolerances;

- a check of the written file, read back and compared with the input.

The check catches the classic mistakes that a conversion makes silently: fractional and Cartesian coordinates mixed up, a wrong scale factor, a lossy format. While this example was being written, it also caught a real bug: the DFTB+ reader swapped two cell vectors. That bug is now fixed (section 6.10).

The worked example (section 6) uses real structures: bulk Si and a Si(111) slab from the `7.1/7.6` examples, and the `siesta.STRUCT_OUT` of a SIESTA relaxation of a Si adatom on that slab. It adds a few hand-made files that each show one idea — a CIF from a database, a cell in Bohr, a molecule, noisy and broken files. Everything runs in seconds.

9.1.2.1 Contents

1. Theory
2. The conversion at a glance
3. Libraries and external dependencies used
4. What `stb-translate` does
5. Running it both ways
6. Worked example
7. What to take from the example
8. Reproducing the reference data
9. Known, deliberate limitations
10. Step-by-step: converting your own files
11. Files in this folder
12. Running the script
13. Flag reference
14. What's next
15. References

9.1.2.2 1. Theory

1.1 A crystal in a file. A periodic structure is fully described by three things [1]:

1. **The cell**: three vectors **a**, **b**, **c**. Every lattice point is $n_1a + n_2b + n_3c$ with integer n_i, and the crystal repeats itself by every such translation.
2. **The species** of the atoms in one cell.
3. **The positions** of those atoms inside the cell.

Every structure format stores these three, in its own layout and units. Most store nothing else. A few add information the others cannot carry: symmetry operations (CIF), constraints (POSCAR's selective dynamics), charges, magnetic moments, velocities. A conversion keeps the three essentials and drops whatever the target format has no place for.

The cell is conveniently written as a matrix whose **rows** are the vectors:

```
    | ax  ay  az |
A = | bx  by  bz |        (Angstrom; the volume of the cell is |det A|)
    | cx  cy  cz |
```

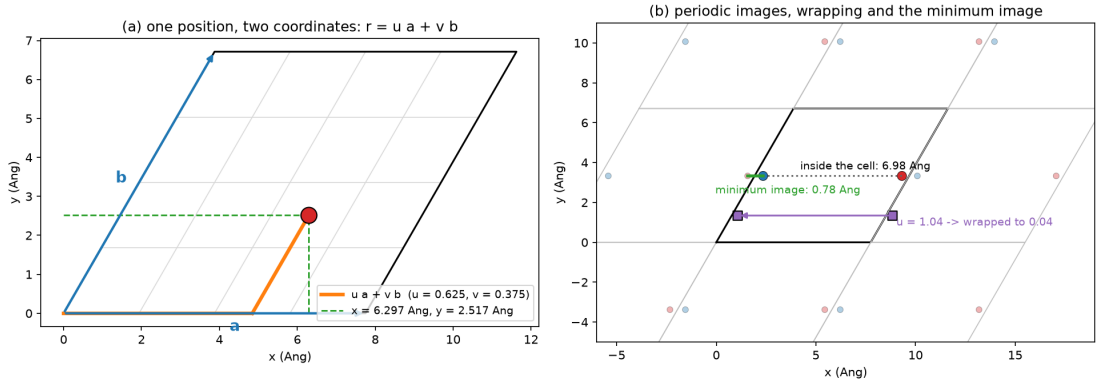
1.2 Fractional and Cartesian coordinates. A position can be given in two ways:

- **Cartesian** `r = (x, y, z)`, in Å along the fixed axes x, y, z;
- **fractional** `s = (u, v, w)`, along the cell vectors: `r = u a + v b + w c`.

In matrix form, with `r` and `s` as row vectors,

```
r = s A          s = r A^-1
```

For an orthogonal cell this is a scaling of each axis. For an oblique cell — hexagonal, monoclinic, the 60° surface cell of the example's Si(111) slab — it mixes the axes: a fractional `u` contributes to both `x` and `y` (Figure 1a). Case 4 does this sum by hand for one atom. Inside the cell every fractional coordinate lies in `[0, 1)`. A value of 4.1 is legal arithmetic but almost never what anyone meant (section 1.6).



Coordinates

Figure 1. (a) One position of the Si(111) slab's oblique surface cell (γ = 60°), as fractional coordinates `(u, v) = (0.625, 0.375)` *along* ***a*** *and* ***b****, and as Cartesian* `(x, y)`*. (b) Periodic images of two atoms near opposite edges of the cell. Inside the cell they are 6.98 Å apart; through the boundary — the minimum image — they are 0.78 Å apart. An atom at* `u = 1.04` *is the same atom as one at* `u = 0.04` *(wrapping).*

1.3 Units and scale factors. The formats agree that positions are in Å or fractional, but each stores the length scale its own way:

- **SIESTA fdf**: `%block LatticeVectors` multiplied by `LatticeConstant <value> <unit>`. The unit can be Ang, Bohr or nm. phonopy's SIESTA writer, for example, uses `LatticeConstant 1.0 Bohr` with the vectors in Bohr (1 Bohr = 0.529177 Å).
- **POSCAR**: line 2 is a *scale factor* that multiplies the three vectors (and Cartesian positions). If it is negative, it is the cell *volume* in Å^3 instead, and the factor is `(|V| / det A)^(1/3)`.
- **STRUCT_OUT, XSF, FHI-aims, DFTB+, CIF**: lengths in Å. A CIF gives the cell as `a, b, c,` α, β, γ rather than as vectors; the reader builds the vectors with **a** along x and **b** in the xy plane.

A wrong scale factor is the second classic mistake (section 1.6). It does not raise any error: it shrinks or swells the whole crystal.

1.4 Periodicity, wrapping and the minimum image. A position outside `[0, 1)` names the same atom as its image inside: `u` and `u − 1` are the same site (Figure 1b). Some programs write wrapped coordinates, others leave atoms where an MD run or a relaxation moved them.

Distances between atoms must use the **minimum image**: of all the periodic copies of atom *j*, the one closest to atom *i*. Two atoms on opposite faces of a cell are neighbours, not a cell length apart. Every distance in the report is taken this way.

A slab, a wire or a molecule is modelled in the same periodic box, with vacuum along the non-periodic directions. The report calls an axis non-periodic when the atoms leave a gap of at least 10 Å along it. From the number of such axes it calls the system a 3D bulk, a 2D slab or layer, a 1D wire or a 0D molecule in a box.

1.5 Symmetry, tolerance and the primitive cell. A crystal's symmetry is the set of operations — rotations, reflections, inversions, possibly combined with fractional translations — that map it onto itself. Their group is the **space group**: 230 types in 3D, tabulated in the International Tables [2]. The report finds it with spglib [3].

A real structure is never exactly symmetric. A relaxation, an MD snapshot or a rounded file leaves every atom a little off its ideal site. spglib therefore asks whether each operation maps every atom onto an atom of the same species within a **tolerance**, `symprec`, in Å. Figure 3 measures what that means on bulk Si:

- with noise of amplitude *A*, the full symmetry (192 operations, Fd-3m) survives while *A* stays below about 0.4 `symprec`;
- above that, the detected symmetry collapses within a factor of two, to P1.

So a tolerance much tighter than the noise sees no symmetry. One much looser than real distortions *invents* symmetry the structure does not have. The report gives the space group at 0.001, 0.01 and 0.1 Å, so the reader can see where it changes.

The **primitive cell** is the smallest cell that tiles the crystal by lattice translations. The cubic cell of Si has 8 atoms; its primitive cell has 2, so the cubic cell is 4 primitive cells. The report gives the size of the primitive cell, found at 0.01 Å.

CIF and symmetry. A CIF can list only the **asymmetric unit** — the symmetry-distinct atoms — together with the operations of the space group. The full cell is then generated by applying every operation and removing duplicates (Case 7: 2 sites of NaCl become 8 atoms). Writing a CIF the other way round means finding the symmetry first: `stb-translate` lets pymatgen's `CifWriter` [4] do that at `--cif-symprec` (0.01 Å by default). That moves every atom onto its exact symmetric position and writes the asymmetric unit in the standard setting. It is what a database or a paper expects. It also changes the data by up to the tolerance, and can rotate or re-choose the cell. `--cif-symprec 0` writes P1 instead: the cell and the positions exactly as read (Case 6).

1.6 What can go wrong silently, and how the report catches it. Most conversion errors produce a file that reads without complaint. The report looks for the symptoms.

Mistake	What it does	How the report sees it
Cartesian numbers labelled fractional (`Direct` in a POSCAR)	positions multiplied by the cell: atoms scattered, often into a wrong symmetric pattern	fractional coordinates far outside [0, 1] (> 1.25 or < −0.25)
Fractional numbers labelled Cartesian	the whole crystal squeezed into a 1 Å corner	every Cartesian coordinate between 0 and 1 Å in a cell of more than 2 Å

Mistake	What it does	How the report sees it
Wrong scale factor / unit (Bohr read as Å)	every distance scaled by the same factor	density far from normal; bonds shorter than 0.75 × the sum of the covalent radii [5]; atoms closer than 0.5 Å
A lossy or buggy writer/reader	the written file is another structure	the written file is read back and compared: composition, number of atoms, volume per formula unit, cell, largest atom displacement

The read-back check compares the input with the written file atom by atom. Each input atom is matched to the nearest output atom of the same element, by minimum image. This works when the cell is unchanged. When the writer re-expresses the cell, as a symmetrized CIF does, the check uses pymatgen's `StructureMatcher` [4] instead, which recognises the same crystal in another cell or origin.

1.7 What each format stores.

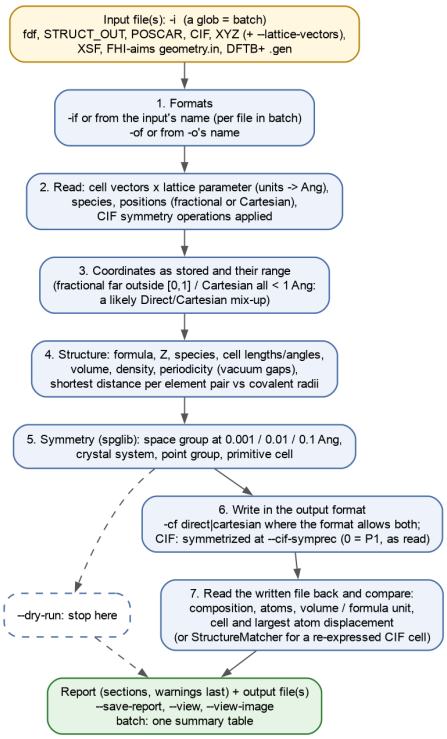
	cell	coordinates	species	extra	notes
fdf	`LatticeVectors` × `LatticeConstant` (Ang/Bohr/nm)	fractional or Cartesian (`AtomicCoordinatesFormat`)	`ChemicalSpeciesLabel` (Z < 0 = ghost)		what SIESTA reads
STRUCT_OUT	3 vectors, Å	fractional	species index + Z	—	what SIESTA writes after each relaxation step
POSCAR	3 vectors × scale (or volume)	Direct or Cartesian (first letter decides)	element line + counts	selective dynamics (read, not carried)	VASP 5 format
CIF	a, b, c, α, β, γ	fractional	element per site	symmetry operations, occupancies, labels	the archival format
XYZ	none	Cartesian, Å	element	comment line	cell from `--lattice-vectors`
XSF	`PRIMVEC`	Cartesian, Å	atomic number	—	XCrySDen
FHI-aims	`lattice_vector`	`atom` (Cartesian) or `atom_frac`	element	—	`geometry.in`
DFTB+	origin + 3 vectors	`S` (Cartesian), `F` (fractional), `C` (cluster)	type list	—	`.gen`
LAMMPS	box bounds + tilt factors	Cartesian, Å	atom types + masses	—	written by ASE [6]

LAMMPS needs a "restricted triclinic" box, with **a** along x and **b** in the xy plane. ASE rotates the cell into that form when needed, so a LAMMPS file can differ from the input by a rigid rotation.

1.8 What this tool simplifies — explicitly.

- **One structure per file**: no trajectories (that is `stb-ani2traj`).
- **What is not carried over**: constraints, magnetic moments, velocities, charges and fractional occupancies. A disordered CIF is read with its sites as listed.
- **CIF output is symmetrized** unless `--cif-symprec 0`.
- **POSCAR is read in the VASP 5 layout**, with the element line; a POSCAR without it (VASP 4) is not understood.

9.1.2.3 2. The conversion at a glance



stb-translate

9.1.2.4 3. Libraries and external dependencies used

Library	Used for
ASE [6]	`Atoms`, minimum-image distances, the LAMMPS writer and reader, the 3D views (`--view`, `--view-image`)
pymatgen [4]	CIF reading (symmetry expansion) and writing (`CifWriter`), `StructureMatcher`, compositions, the crystal system
spglib [3]	space group, number of operations, primitive cell
NumPy	coordinates, cell matrices
matplotlib	`--view-image`
`stb.core`	the `.fdf` reader/writer (`structure_io`, the only one in the suite), vacuum detection (`kspace`), report helpers (`cli`)

`stb-translate` comes with the base package (`pip install stb_suite`); it needs no ML extra and no SIESTA.

9.1.2.5 4. What `stb-translate` does

4.1 Step by step.

1. **Formats.**

- Input: `-if`, else from the name: `.fdf`, `.STRUCT_OUT`, `.vasp/POSCAR/CONTCAR`, `.cif`, `.xyz`, `.in`, `.gen`, `.xsf`. In batch mode this is decided per file.
- Output: `-of`, else from `-o`'s name: `.fdf`, `.vasp/.poscar/POSCAR`, `.cif`, `.xyz`, `.in`, `.gen`, `.xsf`, `.data/.lmp`. Batch mode needs `-of` and `--out-dir`.

2. **Read** with the format's reader. The cell is multiplied by its lattice parameter or scale factor (a negative POSCAR scale is a volume), a CIF's symmetry operations are applied, and an XYZ file gets its cell from `--lattice-vectors`.
3. **Input section**: file size; the coordinates as the file stores them (read from the file itself); their range, with the mix-up check of section 1.6.
4. **Structure section**:
   - formula and number of formula units Z;
   - species with their counts, fractions and masses;
   - cell lengths and angles, the vectors;
   - volume, volume per atom, density;
   - periodicity (section 1.4);
   - the shortest distance for every pair of elements, with its ratio to the covalent bond length.
5. **Symmetry section**: space group and number of operations at 0.001, 0.01 and 0.1 Å; crystal system and point group; size of the primitive cell.
6. **Write** (unless `--dry-run`). `-cf direct|cartesian` chooses the coordinates where the format allows both; CIF output is symmetrized at `--cif-symprec`.
7. **Output section**: file size; the coordinates as written (read from the file); the read-back check of section 1.6.
8. **Summary** (a table per file in batch mode), the files written, and **all warnings** in the last section. `--view`/`--view-image` come last, after the report.

4.2 The report, section by section.

Section	Content
`RUN METADATA`	date, versions (stb, ASE, pymatgen, spglib), input and output with their formats and whether each was given or inferred, `-cf`, CIF symmetry
`INPUT`	file and size, format, coordinates as stored, lattice parameter, coordinate range (and the mix-up warning)
`STRUCTURE`	formula and Z, species table, cell, vectors, volume, density, periodicity, shortest distance per element pair
`SYMMETRY`	space group / number / operations at three tolerances, crystal system, point group, primitive cell
`OUTPUT`	file and size, format, coordinates as written, CIF symmetry, the verification table (and a note when the cell was re-expressed)
`SUMMARY & FILES`	status, structure, files written, report, options not used; in batch mode a table: input, formula, atoms, space group, output, status
`WARNINGS`	every warning, numbered

In batch mode the `INPUT` to `OUTPUT` sections repeat for each file, titled with its name.

4.3 Guard rails.

Situation	Behaviour
no format given and an unknown extension	clean error asking for `-if`/`-of`
`-o` missing (and not `--dry-run`)	clean error
XYZ input without `--lattice-vectors`	clean error
a glob with `-o` (one name for many files)	clean error: use `--out-dir`

Situation	Behaviour
a file that cannot be read	`[FAIL]` for that file; a batch goes on with the others, exit code 1
a written file that is not the input	warning (composition, volume, displacement, or not the same crystal)
coordinates or bonds that look like a mix-up	warning

9.1.2.6 5. Running it both ways

Command line:

```
stb-translate -i CONTCAR -o structure.fdf                      # both formats from the names
stb-translate -i siesta.STRUCT_OUT -o relaxed.cif --save-report # a relaxation, for a paper or VESTA
stb-translate -i relaxed.STRUCT_OUT -o relaxed_p1.cif --cif-symprec 0   # the CIF exactly as relaxed
stb-translate -i POSCAR -o structure.fdf -cf cartesian         # choose the output coordinates
stb-translate -i water.xyz -o water.fdf --lattice-vectors 15 0 0 0 15 0 0 0 15
stb-translate -i '*.cif' -of fdf --out-dir fdf/                # batch
stb-translate -i structure.fdf --dry-run                       # report only
stb-translate -i structure.fdf -o s.xsf --view-image s.png     # and a picture
```

Interactive menu: run `stb-suite` and type `3.1`. It asks, in this order:

1. the input file and its format (`0` = detect from the name, the default);
2. the output file and its format (`0` = detect from the name, the default);
3. the output coordinates (Cartesian, Direct, or the default);
4. for an XYZ input, the nine components of the lattice vectors;
5. whether to view the structure (window and/or image), and whether to save the report.

9.1.2.7 6. Worked example

All numbers below come from `reference_results/` (`make_reference.sh`) and from the guided script.

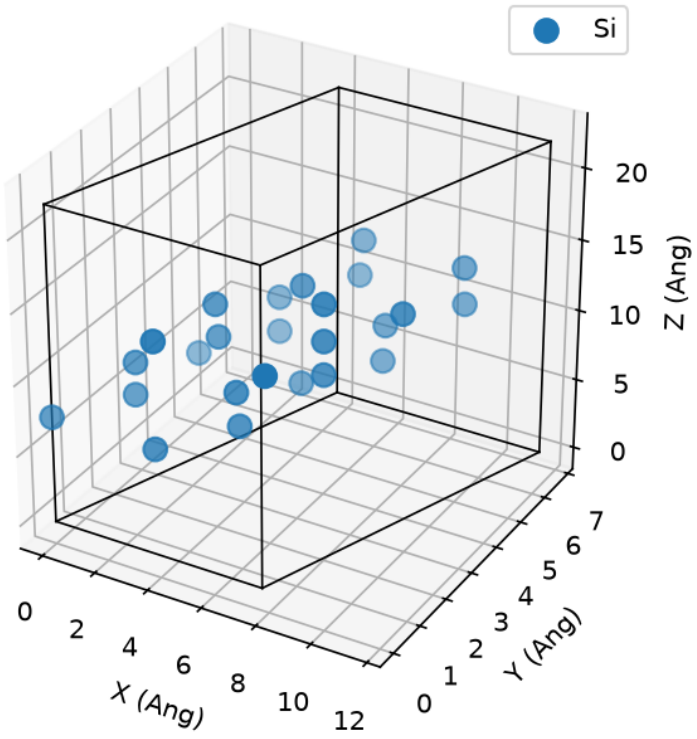
6.1 A SIESTA relaxation to CIF (Case 2). `si111_adatom_relaxed.STRUCT_OUT` is what SIESTA wrote at the end of the relaxation of a Si adatom on Si(111) in the `7.6` example: three cell vectors in Å, then one line per atom with its species index, Z and fractional coordinates.

```
stb-translate -i si111_adatom_relaxed.STRUCT_OUT -o si111_adatom.cif
```

```
Input             : si111_adatom_relaxed.STRUCT_OUT -- siesta (auto, from the name
'si111_adatom_relaxed.STRUCT_OUT')
Output            : si111_adatom.cif -- cif (auto, from the name 'si111_adatom.cif')
CIF symmetry      : 0.01 Ang (--cif-symprec)
...
Formula           : Si25 -- reduced Si, Z = 25 formula units
Cell             : a = 7.75023, b = 7.75023, c = 22.11904 Ang; alpha = 90.000, beta = 90.000, gamma =
60.000 deg
Volume            : 1150.6035 Ang^3 (46.0241 Ang^3/atom); density 1.0133 g/cm^3 (mass 702.125 u)
Periodicity       : 2D slab/layer -- vacuum (>= 10 Ang gap) along c
Si-Si | 2.3257         | Si13 - Si17 | 1.05       |
...
0.01          | P3m1        | 156    | 6
```

```
Crystal system    : trigonal; point group 3m
Primitive cell    : 25 atoms (symprec 0.01 Ang) -- the input cell is primitive
...
same crystal (StructureMatcher) |         | yes     | rms 5.7e-06 Ang
Note              : the writer re-expressed the cell (the standard setting of a symmetrized CIF):
compared as the
                    same crystal up to a change of cell/origin (pymatgen StructureMatcher)
```

The extensions chose both formats. The report sees a slab: the 22 Å cell has 13.7 Å of vacuum along c, so the density of 1.01 g/cm^3 is the density of slab plus vacuum. The relaxed adatom keeps a three-fold axis and three mirrors (P3m1, 6 operations). The CIF is written in P3m1's standard setting, and the check confirms it is the same crystal. With `--cif-symprec 0` the CIF is P1 instead, in the original cell, and the check runs atom by atom: largest displacement 1.1 × 10^{-7} Å, the precision of the numbers written.



view image

Figure 2. `--view-image si111_adatom.png` *of the same run: a 3D snapshot of the slab in its cell, written without a display.*

6.2 One crystal, every output format (Case 3). `si_bulk.fdf` (bulk Si, 8 atoms) written in all eight output formats, each file read back:

File	Format	Coordinates written	Largest shift on read-back
`si.cif`	cif	fractional (always)	0
`si.vasp`	poscar	fractional (Direct, as the input)	0
`si.xyz`	xyz	Cartesian, Å (always)	0
`si.xsf`	xsf	Cartesian, Å (always)	0
`geometry.in`	fhi	fractional (`atom_frac`)	0

File	Format	Coordinates written	Largest shift on read-back
`si.gen`	dftb	fractional (`F`)	0
`si.data`	lammps	Cartesian, Å	1.3×10^{-15} Å
`si_copy.fdf`	fdf	fractional	0

Every format reproduces the crystal. Bulk Si is already exactly symmetric, so the symmetrized CIF moves nothing. The formats differ in what they can say: CIF is always fractional, XYZ/XSF/LAMMPS always Cartesian, and the others follow the input unless `-cf` asks otherwise.

6.3 Fractional and Cartesian, by hand (Case 4). The fifth atom of the slab, written with `-cf direct` and with `-cf cartesian`:

```
fractional :   0.16666667   0.16666667   0.37483566
Cartesian  :   1.93755676   1.11864892   8.29100420
```

With the rows of the cell `a = (7.7502, 0, 0)`, `b = (3.8751, 6.7119, 0)`, `c = (0, 0, 22.1190)`:

```
x = 0.16667 × 7.7502 + 0.16667 × 3.8751 = 1.9376
y = 0.16667 × 6.7119                    = 1.1186
z = 0.37484 × 22.1190                   = 8.2910
```

The `x` coordinate collects contributions from both **a** and **b**: that is the oblique cell. The guided script also computes `s = r A`$^{-1}$ and gets the fractional coordinates back.

6.4 Units: a lattice in Bohr (Case 5). `si_bohr.fdf` is the 2-atom primitive cell of Si with `LatticeConstant 1.0 Bohr` and the vectors `(0, 5.178, 5.178)` … in Bohr. Written as a POSCAR:

```
0.529177210903
0.00000000   5.17807400   5.17807400
...
```

The lattice constant became VASP's scale factor, and the vectors were copied unchanged. The report reads the cell as a = 3.875 Å with 60° angles: the fcc primitive cell. Its volume per atom, 20.574 Å^3, is that of `si_bulk.fdf`, and its symmetry is Fd-3m with 48 operations, the point group without the fcc centring translations. Reading the vectors as Å would give a cell 1.89 times too large in every direction, 6.7 times in volume. `stb.core.structure_io` once had exactly this bug for phonopy's SIESTA files.

6.5 Symmetry and tolerance (Case 6). `si_noisy_<A>.fdf` is bulk Si with every atom displaced at random by up to A. Space group found at each tolerance:

Noise A (Å)	0.001 Å	0.01 Å	0.1 Å	primitive cell at 0.01
0.002	P1	**Fd-3m**	Fd-3m	2 atoms
0.02	P1	P1	**Fd-3m**	8 atoms
0.2	P1	P1	P1	8 atoms

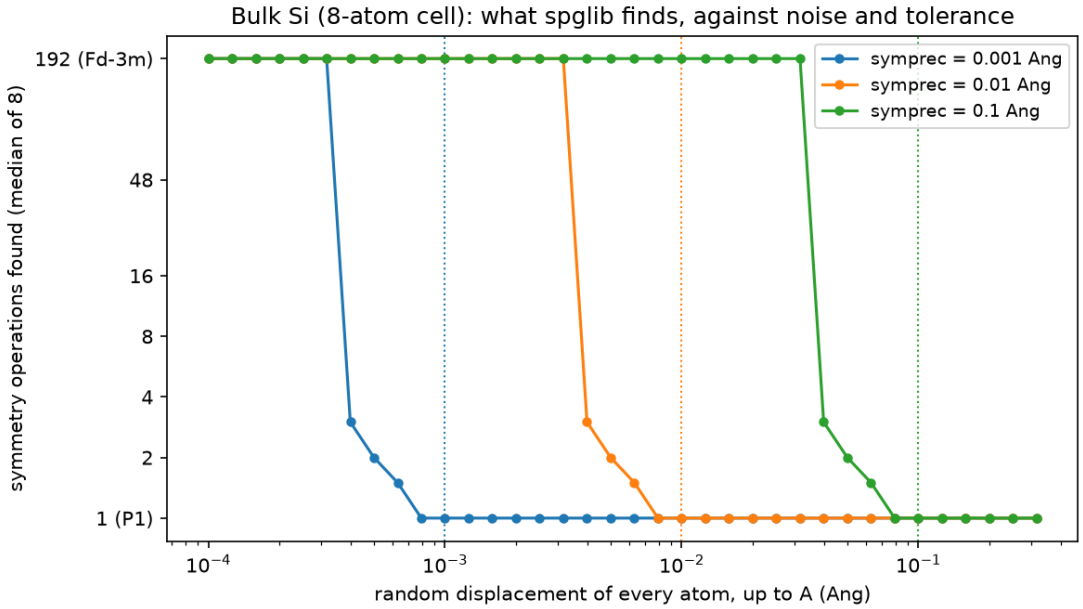


Figure 3. Symmetry operations spglib finds in bulk Si (8-atom cell) against the amplitude of random displacements, for the three tolerances of the report (median of 8 random draws; the dotted lines mark each tolerance). The full Fd-3m survives while the noise is below about 0.4 × the tolerance.

A structure relaxed to 0.01 eV/Å typically carries noise of 10^{-3} Å. 0.01 Å, the default of most tools in the suite, sees its symmetry; 0.001 Å does not.

Written as a CIF, the 0.002 Å file is symmetrized:

```
largest atom displacement (Ang) |          | 2.64e-03 | symmetrized
Symmetrized       : the CIF writer moved atoms by up to 0.0026 Ang onto their exact symmetric
positions
                    (--cif-symprec 0.01); --cif-symprec 0 keeps them as read
```

That is usually what a CIF should contain. When the small distortion is the physics — a Jahn–Teller distortion, a polar displacement, an MD snapshot — use `--cif-symprec 0`.

6.6 A CIF from a database (Case 7). `nacl_asym.cif` is rock salt as a database writes it: space group Fm-3m, a = 5.64 Å, and only two sites, each with its multiplicity:

```
  Na  Na0  4  0.00000000  0.00000000  0.00000000  1
  Cl  Cl1  4  0.00000000  0.00000000  0.50000000  1
```

```
Formula           : Cl4Na4 -- reduced NaCl, Z = 4 formula units
Atoms             : 8 (2 species)
Cl-Na | 2.8200          | Cl1 - Na5 | 1.05       |
0.01          | Fm-3m       | 225    | 192
Primitive cell    : 2 atoms (symprec 0.01 Ang) -- the input cell is 4x larger
```

The reader applied the 192 operations and kept the 8 distinct positions of the conventional cell: 4 Na, 4 Cl, Na–Cl 2.82 Å = a/2. SIESTA needs all of them. For a cheaper calculation, the 2-atom primitive cell describes the same crystal (`stb-unitcell`, `4.7`).

6.7 A molecule: XYZ has no cell (Case 8).

```
stb-translate -i water.xyz -o water.fdf                      # error: An XYZ input needs
--lattice-vectors
stb-translate -i water.xyz -o water.fdf --lattice-vectors 15 0 0 0 15 0 0 0 15
```

```
Periodicity       : 0D molecule/cluster in a box -- vacuum (>= 10 Ang gap) along a, b, c
H-O  | 0.9686         | H1 - O3 | 1.00       |
0.01          | Pmm2        | 25     | 4
```

A plain XYZ file has positions and nothing else, so the box has to be given. In it the report finds vacuum on all three axes (a 0D system). The "space group" Pmm2 of a lone molecule in a box is its point group, mm2 (C2v), plus the box's translations.

6.8 What the report catches (Case 9).

Both files below read without any error.

Cartesian numbers under `Direct` (`si_mixup.vasp`):

```
Coordinate range  : 0.000 .. 4.110 -- fractional coordinates far outside [0, 1]
0.01          | R3m         | 160    | 6
  1. si_mixup.vasp: the coordinates are read as fractional but span 0.00 .. 4.11 -- a Cartesian file
  labelled
     Direct/Fractional? The structure below is then wrong.
```

The atoms land on scattered sites that happen to form an R3m crystal. Without the warning, it is a plausible-looking wrong structure.

Scale factor 0.5 instead of 1.0 (`si_wrongscale.vasp`):

```
Volume            : 20.5735 Ang^3 (2.5717 Ang^3/atom); density 18.1345 g/cm^3 (mass 224.680 u)
Si-Si | 1.1865         | Si1 - Si2 | 0.53       | short
  1. si_wrongscale.vasp: a bond is shorter than 0.75 x the sum of the covalent radii -- check the
  geometry.
```

The result is a silicon eight times denser than real Si (2.33 g/cm^3), with half-length bonds. The symmetry is still a perfect Fd-3m, because a uniform scale changes no symmetry, so only the density and the bonds can tell.

6.9 Batch conversion (Case 10).

```
stb-translate -i 'si*.fdf' -of cif --out-dir cif
```

```
input              | formula | atoms | space group (0.01) | output             | status
si111_2x2.fdf      | Si24    | 24    | P-3m1              | si111_2x2.cif      | ok
si_bohr.fdf        | Si2     | 2     | Fd-3m              | si_bohr.cif        | ok
si_bulk.fdf        | Si8     | 8     | Fd-3m              | si_bulk.cif        | ok
si_noisy_0.002.fdf | Si8     | 8     | Fd-3m              | si_noisy_0.002.cif | ok
si_noisy_0.02.fdf  | Si8     | 8     | P1                 | si_noisy_0.02.cif  | ok
si_noisy_0.2.fdf   | Si8     | 8     | P1                 | si_noisy_0.2.cif   | ok
```

Each file gets its own `INPUT…OUTPUT` sections, and the summary collects them. A file that fails is marked `FAILED`, the others are still written, and the exit code is 1.

6.10 A bug the read-back check found. The read-back check was added while this example was being prepared, and its first run on the slab flagged the DFTB+ `.gen` file: the cell read back differed from the cell written. The `.gen` reader took the last three lines of the file — **a**, **b**, **c** — from the end, as **c**, **b**, **a**:

```
written (fdf):   a = ( 7.750, 0,     0     )    read back (.gen):  a = ( 0,     0,     22.119)
                 b = ( 3.875, 6.712, 0     )                       b = ( 3.875, 6.712, 0     )
                 c = ( 0,     0,     22.119)                       c = ( 7.750, 0,     0     )
```

With fractional coordinates, swapping two vectors builds another crystal. A cubic cell hides the swap, which is why the existing tests never saw it; any non-cubic `.gen` was silently wrong. It is fixed, and the test now converts a hexagonal slab fdf → `.gen` → fdf and requires the same cell and positions. A verification step that reads its own output is cheap; this is what it is for.

9.1.2.8 7. What to take from the example

- **A structure file is a cell, species and positions**, in some units and with fractional or Cartesian coordinates. Every format stores a subset of that in its own convention. A conversion keeps the essentials and drops the rest.
- **Let the names choose the formats, and read the report.** The coordinate range, the density and the shortest bonds catch the classic mix-ups, which read without errors, before SIESTA runs on them.
- **Symmetry depends on the tolerance.** 0.01 Å suits relaxed structures. A CIF written with a tolerance is symmetrized; `--cif-symprec 0` keeps the data exactly as it is.
- **Every written file is read back.** An unexpected shift means a lossy format or a bug, and it found one.

9.1.2.9 8. Reproducing the reference data

```
source ~/.anaconda3/etc/profile.d/conda.sh && conda activate stb   # or any environment with
stb_suite
python3 make_inputs.py        # the hand-made inputs (seconds)
bash make_reference.sh        # every case into reference_results/<case>/ (report.txt + the files
written)
python3 theory_figures.py     # figures/coordinates.png, figures/tolerance.png
dot -Tpng -Gdpi=150 figures/translate_flowchart.dot -o figures/translate_flowchart.png
```

Everything takes seconds and is deterministic: the noisy files use fixed random seeds.

9.1.2.10 9. Known, deliberate limitations

- **One structure per file.** Trajectories (`.ANI`, multi-frame XYZ) go through `stb-ani2traj`.
- **Only the cell, species and positions are carried over.** Constraints, magnetic moments, charges, velocities and partial occupancies are dropped. A disordered CIF is read with its sites as listed, not resolved.
- **POSCAR is read in the VASP 5 layout**, with the element line.
- **LAMMPS output may be a rigid rotation of the input** (ASE's restricted-triclinic convention).
- **CIF output is symmetrized by default** (`--cif-symprec`).

9.1.2.11 10. Step-by-step: converting your own files

1. **Name the files after their format** (`.fdf`, `.cif`, `.vasp`/`POSCAR`, `.STRUCT_OUT`, `.xyz`, `.in`, `.gen`, `.xsf`, `.data`) and let `stb-translate` infer both formats. Use `-if`/`-of` only for unusual names.
2. **Run `--dry-run` first on anything you did not write yourself.** Read the coordinate range, the density, the shortest bonds and the periodicity. Do they describe the material you expect?
3. **Check the symmetry at the three tolerances.** If it changes between 0.01 and 0.1 Å, the structure is slightly distorted from a higher symmetry. Decide whether that distortion is physics or noise.
4. **Convert, and read the verification table.** For a CIF, decide between the symmetrized standard setting (default) and the data as is (`--cif-symprec 0`).
5. **For SIESTA**: the fdf written here is the structure only. `stb-inputfile` (`1.1`) builds the full `calc.fdf` around it, and `stb-kgrid` (`1.3`) chooses its k-points.

9.1.2.12 11. Files in this folder

File	What it is
`README.md`	this tutorial
`example_3.1.sh`	the guided walk-through (12 cases, seconds)
`si_bulk.fdf`	bulk Si, 8-atom cubic cell, a = 5.48024 Å (the `7.1` training reference)
`si111_2x2.fdf`	Si(111) 2×2 slab, 3 bilayers, 24 atoms (from `7.6`)
`si111_adatom_relaxed.STRUCT_OUT`	SIESTA relaxation of a Si adatom on that slab (from `7.6`, T4 site)
`make_inputs.py`	writes the hand-made inputs below
`nacl_asym.cif`	NaCl, Fm-3m, asymmetric unit + symmetry operations
`si_bohr.fdf`	Si primitive cell with `LatticeConstant 1.0 Bohr`
`water.xyz`	H_2O (needs `--lattice-vectors`)
`si_noisy_0.002.fdf`, `si_noisy_0.02.fdf`, `si_noisy_0.2.fdf`	bulk Si with random displacements
`si_mixup.vasp`, `si_wrongscale.vasp`	two broken POSCARs (coordinate mix-up, wrong scale)
`make_reference.sh`	regenerates `reference_results/`
`theory_figures.py`	the figures of section 1
`figures/`	every figure of this README; `translate_flowchart.dot` is the flowchart's source
`reference_results/`	one folder per case: `report.txt` and the files written

9.1.2.13 12. Running the script

```
cd examples/3.1-stb-translate
bash example_3.1.sh            # press Enter between cases
yes '' | bash example_3.1.sh   # straight through
```

Each case writes into `output/<case>/`, which is wiped at every start. Case 1 rewrites the hand-made inputs, and the other cases run `stb-translate` on them; Case 12 drives the menu (`stb-suite`, code 3.1) through a pipe.

9.1.2.14 13. Flag reference

`stb-translate`:

- input and output: `-i/--in-file FILE|GLOB` (required) · `-if/--in-format {cif,dftb,fdf,fhi,poscar,siesta,xsf,xyz}` · `-o/--out-file FILE` · `-of/--out-format {cif,dftb,fdf,fhi,lammps,poscar,xsf,xyz}` · `--out-dir DIR` (batch);
- what is written: `-cf/--coord-format {direct,cartesian}` · `--cif-symprec Å` (0.01; 0 = P1 as read) · `--lattice-vectors AX AY AZ BX BY BZ CX CY CZ` (XYZ input);
- report and viewing: `--dry-run` · `--save-report` · `--view` · `--view-image PATH` · `--no-intro` · `-v`.

9.1.2.15 14. What's next

- **`1.1` (`stb-inputfile`)** builds a complete SIESTA input around the fdf written here.
- **`4.7` (`stb-unitcell`)** reduces a structure to its primitive or conventional cell, the cell sizes the `SYMMETRY` section reports.
- **`2.5` (`stb-symmetry`)** analyses the symmetry in depth: Wyckoff positions, layer groups for slabs, distortion from the ideal structure.
- **`1.6` (`stb-fetch`)** downloads structures from COD, the Materials Project or OPTIMADE databases, as CIFs like the one of section 6.6.

9.1.2.16 15. References

1. N. W. Ashcroft, N. D. Mermin, *Solid State Physics* (Holt, Rinehart and Winston, 1976), chapters 4 and 7.
2. *International Tables for Crystallography*, Vol. A: *Space-group symmetry*, ed. M. I. Aroyo (IUCr/Wiley, 2016).
3. A. Togo, K. Shinohara, I. Tanaka, "Spglib: a software library for crystal symmetry search", *Sci. Technol. Adv. Mater.: Methods* **4**, 2384822 (2024).
4. S. P. Ong et al., "Python Materials Genomics (pymatgen): A robust, open-source python library for materials analysis", *Comput. Mater. Sci.* **68**, 314 (2013).
5. B. Cordero et al., "Covalent radii revisited", *Dalton Trans.* 2832 (2008).
6. A. H. Larsen et al., "The atomic simulation environment — a Python library for working with atoms", *J. Phys.: Condens. Matter* **29**, 273002 (2017).
7. S. R. Hall, F. H. Allen, I. D. Brown, "The crystallographic information file (CIF): a new standard archive file for crystallography", *Acta Cryst. A* **47**, 655 (1991).
8. J. M. Soler et al., "The SIESTA method for ab initio order-N materials simulation", *J. Phys.: Condens. Matter* **14**, 2745 (2002).

9.1.3 Command-line reference

9.1.3.1 `stb-translate`

Structure file converter: reads one structure (or a glob of them) in one format and writes it in another, with a report of what was read and a check of what was written. Both formats are detected from the file names when -if/-of are omitted.

Synopsis.

```
usage: stb-translate [-h] [-if {cif,dftb,fdf,fhi,poscar,siesta,xsf,xyz}] -i IN_FILE
                     [-of {cif,dftb,fdf,fhi,lammps,poscar,xsf,xyz}] [-o OUT_FILE]
                     [--out-dir OUT_DIR] [--dry-run] [--save-report]
                     [-cf {direct,cartesian}] [--cif-symprec ANG]
                     [--lattice-vectors AX AY AZ BX BY BZ CX CY CZ] [--view]
```

```
                    [--view-image PATH] [-v] [--no-intro]
```

Options

`-if, --in-format {cif,dftb,fdf,fhi,poscar,siesta,xsf,xyz}`
Input format: poscar, cif, siesta [SIESTA STRUCT_OUT], xyz, fhi, dftb, xsf, fdf [SIESTA input]. Default: from `--in-file`'s name (.vasp/POSCAR/CONTCAR, .cif, .STRUCT_OUT, .xyz, .in, .gen, .xsf, .fdf), per file in batch mode. choices: cif, dftb, fdf, fhi, poscar, siesta, xsf, xyz

`-i, --in-file IN_FILE`
Input file, or a glob pattern (e.g. '*.cif') for batch conversion. required

`-of, --out-format {cif,dftb,fdf,fhi,lammps,poscar,xsf,xyz}`
Output format: cif, xyz, poscar, fdf, dftb, xsf, fhi, lammps [LAMMPS data, atom_style atomic, metal units]. Default: from `--out-file`'s name (.cif, .xyz, .fdf, .vasp/.poscar/POSCAR, .gen, .in, .xsf, .data/.lmp). Required in batch mode. choices: cif, dftb, fdf, fhi, lammps, poscar, xsf, xyz

`-o, --out-file OUT_FILE`
Output file. Required unless `--dry-run`, or in batch mode (`--out-dir` instead).

`--out-dir OUT_DIR`
Output directory in batch mode: each output is named after its input with the output format's extension.

`--dry-run`
Read and report only (structure, symmetry, checks); write nothing. switch, off by default

`--save-report`
Also write the report to `stb_translate_report.txt` (next to `--out-file`, or in `--out-dir`). switch, off by default

`-cf, --coord-format {direct,cartesian}`
Coordinates of the output: direct (fractional) or cartesian. Default: the input's, or the output format's own. choices: direct, cartesian

`--cif-symprec ANG`
CIF output only: tolerance with which the space group is found and written (default: 0.01 Ang). The atoms are moved onto their exact symmetric positions (by up to this much) and the asymmetric unit is written in the standard setting – what a database or a paper expects. 0 writes P1: the cell and the positions exactly as read. type: float

`--lattice-vectors AX AY AZ BX BY BZ CX CY CZ`
The 3 lattice vectors (Cartesian, Ang), needed for an XYZ input, e.g. 5.43 0 0 0 5.43 0 0 0 5.43. Applied to every file in batch mode. type: float

`--view`
Open the structure in ASE's 3D viewer at the end (needs a display). switch, off by default

`--view-image PATH`
Save a 3D snapshot to PATH (a directory in batch mode, one PNG per file). No display needed.

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Example usage:
  stb-translate -i si.cif -o si.fdf                # both formats from the names
  stb-translate -i POSCAR -o structure.xyz -cf cartesian
  stb-translate -i siesta.STRUCT_OUT -of cif -o relaxed.cif --save-report
  stb-translate -i '*.cif' -of fdf --out-dir fdf/   # batch
  stb-translate -i slab.fdf --dry-run               # report only
```

9.2 Clean File Tools

At a glance

Menu code: 3.2 **Command:** `stb-clean`
Guided example: `examples/3.2-stb-clean/`
Summary: Reset a SIESTA calc folder for a new run: keeps the inputs (and every file they read), archives and checks everything else before removing it, and reports what the next run will start from.

9.2.1 Overview

`stb-clean` prepares a calculation folder for a new run. It keeps the extensions in `–keep` (default `.psml`, `.psf`, `.fdf`, `.sh`) and removes the rest – logs, restart files, bands, grids, basis tables. Never removed: hidden files, earlier **`stb-clean`** backups and every file a kept `.fdf` reads (`%include F`, `%block X < F`, `Label < F`). Before anything goes, the files are put into a `tar.gz` next to the folder and the archive is read back and checked (`–no-archive` skips both); the tool asks once per folder (`-n` skips it) and `–dry-run` only reports. The report gives the previous run (finished or not, geometry steps, how recently it was written), what stays and why, what goes by kind and size, and a RESTART table: `.DM`, `.XV`, `.CG` next to `DM.UseSaveDM`, `MD.UseSaveXV`, `MD.UseSaveCG` – whether the next run starts from scratch or continues. Keep `.DM` and `.XV` to continue (in the guide below, 1 geometry step and 2 SCF iterations instead of 13 steps and 72). `–path` accepts a glob, one archive per folder; `–recursive` takes a whole tree as one folder (hidden folders skipped). This is a reset, not an archive of results: that is **`stb-archive`** (3.7).

```
stb-clean --dry-run                                  # this folder: report only
stb-clean --path si_run                              # report, ask, archive + check, remove
stb-clean --path si_run --keep .fdf .psf .psml .sh .DM .XV   # continue from the previous run
stb-clean --path 'strain_*' -n                       # several folders, no questions
```

9.2.2 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/3.2-stb-clean/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

`stb-clean` resets a SIESTA calculation folder for a new run. It keeps the inputs: by default the extensions `.fdf`, `.psf`, `.psml` and `.sh`, plus every file a kept `.fdf` reads. Everything else is removed: logs, restart files, bands, densities, basis tables. Before removing anything, it puts those files into a `tar.gz` next to the folder and reads the archive back to check it. Its report says:

- what the previous run was: finished or not, how many geometry steps, how long ago it was written;
- what stays, and why;
- what goes, sorted by kind and size;
- **what the next run will start from**: SIESTA's restart files (`.DM`, `.XV`, `.CG`) against the fdf flags that would read them.

The last point is the one that matters most. Whether a cleaned folder reruns *from scratch* or *continues* the previous run depends on two or three files that are easy to delete without noticing. The worked example (section 6) measures the difference on a real run: the same relaxation, run again, takes 13 geometry steps and 72 SCF iterations from scratch, or 1 step and 2 SCF iterations when `.DM` and `.XV` are kept.

`stb-clean` is for *restarting*, not archiving. To package the results of a finished run, use `stb-archive` (`3.7`).

The example uses one real, finished SIESTA 5.4 run: bulk Si, a variable-cell CG relaxation followed by bands, DOS/PDOS and the density, which leaves 48 files in its folder. Every case works on a copy of it. Everything runs in seconds; case 4 also reruns SIESTA (~25 s) when it is installed.

9.2.2.1 Contents

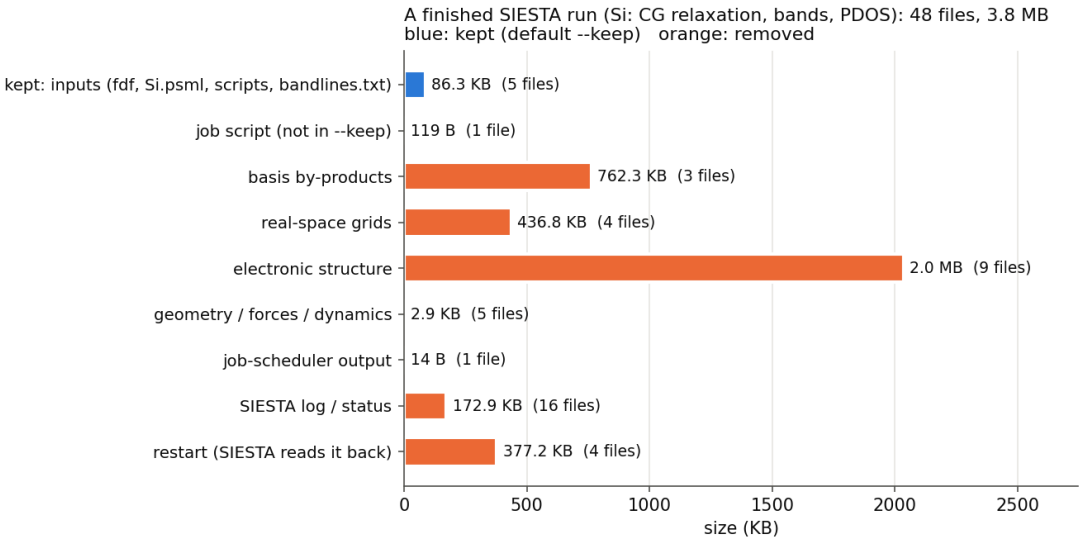
1. Theory
2. The clean at a glance
3. Libraries and external dependencies used
4. What `stb-clean` does
5. Running it both ways
6. Worked example
7. What to take from the example
8. Reproducing the reference data
9. Known, deliberate limitations
10. Step-by-step: cleaning your own folders
11. Files in this folder
12. Running the script
13. Flag reference
14. What's next
15. References

9.2.2.2 1. Theory

1.1 What a SIESTA run leaves in its folder. SIESTA [1] reads a few inputs and writes many outputs into the folder it runs in. For the run of this example:

Kind	Examples	What it is for
inputs	`calc.fdf`, `structure.fdf`, `Si.psml`, `bandlines.txt`, `run.sh`	what you wrote; needed to run again
restart files	`si.DM`, `si.XV`, `si.CG`, `si.STRUCT_NEXT_ITER`	state SIESTA can read back to continue
logs and status	`calc.out`, `MESSAGES`, `TIMES`, `CLOCK`, `0_NORMAL_EXIT`, `fdf.*.log`	what happened
geometry, forces	`si.STRUCT_OUT`, `si.ANI`, `si.FA`, `si.BONDS`	the relaxed structure, the trajectory
electronic structure	`si.EIG`, `si.bands`, `si.PDOS`, `si.HSX`	eigenvalues, bands, DOS, the Hamiltonian
real-space grids	`si.RHO`, `si.VT`, `Rho.grid.nc`	the density and the potential on the mesh
basis by-products	`Si.ion`, `Si.ion.xml`, `Si.ion.nc`	the basis orbitals SIESTA generated
job-scheduler output	`siesta.err`, `slurm-123.out`	the batch system's own logs

Most outputs are named after `SystemLabel` (`si.*` here). A few have fixed names (`MESSAGES`, `CLOCK`, ...) or carry the species label (`Si.ion`). `stb-clean` sorts every file into these kinds from SIESTA's naming conventions alone, so the report can say what goes, not only how much.



Contents of a finished SIESTA run

Figure 1. The 48 files of the finished run in `siesta_run/`*, by kind, as* `stb-clean` *sorts them. The inputs are 86 KB; the electronic structure (*`.HSX`*,* `.PDOS`*, ...) and the basis tables are most of the remaining 3.7 MB. For a real system the grids and the* `.HSX`*/*`.DM` *grow quickly with the cell and the k-points.*

1.2 Restart files: continuing instead of starting over. SIESTA can start a run from a previous one [1, 4]. It reads three files only when a flag in the fdf asks for it:

File	Holds	Read when	Without it, the run starts from
`<label>.DM`	the density matrix	`DM.UseSaveDM true`	a superposition of atomic densities
`<label>.XV`	positions (and velocities)	`MD.UseSaveXV true`	the geometry written in the `.fdf`
`<label>.CG`	the CG optimizer's history	`MD.UseSaveCG true`	a new CG history

The two conditions combine. A flag that is on reads the file **only if it is still there**, and a file left in the folder is ignored **unless its flag is on**. So "clean the folder" can mean two different things:

- **start over**: remove the restart files. The next run repeats the relaxation from the `.fdf` geometry and converges the density from atomic densities again. This is right when the input changed, for example a new basis, mesh cutoff or functional, and the old state no longer belongs to the new calculation.
- **continue**: keep `.DM` and `.XV` (`--keep ... .DM .XV`) with their flags on. The next run starts at the relaxed geometry with the converged density. This is right after a crash, a time limit, or a change that does not alter the problem, such as more output.

A `.DM` from a different calculation is a valid *starting guess* for the SCF, never part of the answer. A `.XV`, however, **is** the geometry: a rerun that reads it no longer starts from the `.fdf` structure. The `RESTART` section of the report shows each file next to its flag so that this choice is visible before the files are gone.

The final geometry of a relaxation is also in `<label>.STRUCT_OUT`. If you want to restart from it without `.XV`, convert it into an `.fdf` first (`stb-translate`, `3.1`).

1.3 What a kept input reads. An fdf file can read other files in three ways:

```
%include structure.fdf                  # the whole file, as if pasted here
%block BandLines < bandlines.txt        # the block's contents from a file
Label < file                            # a value from a file
```

SIESTA follows these when it reads its input. If `bandlines.txt` were removed because `.txt` is not in `--keep`, the next run would stop at once. `stb-clean` reads the kept `.fdf` files the same way and keeps every file they read, whatever its extension. It follows the files read by included files too, until nothing new turns up. A file that a kept input reads but that is *not in the folder* is reported, because the rerun would stop on it as well.

1.4 Safety: archive, check, then remove. Removing files cannot be undone, and SIESTA folders are often cleaned in loops over many of them. `stb-clean` follows these rules:

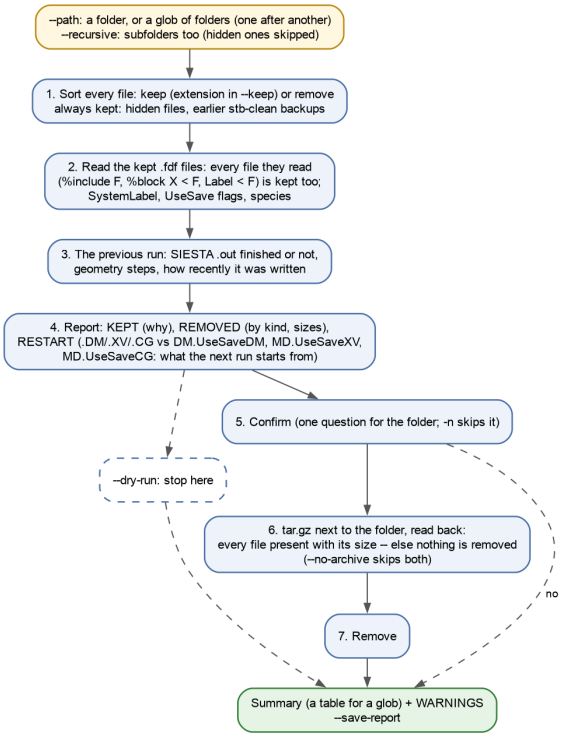
1. **Report first.** `--dry-run` stops after the report and touches nothing.
2. **Ask.** Without `-n`, it asks once per folder after the report. An answer other than `y`, including no answer at all when there is no terminal (end-of-file), cancels.
3. **Archive.** It writes everything that will go into `<folder>_backup_<date>_<time>.tar.gz` **next to** the folder, not inside it, so that a later clean of the same folder cannot remove its own backup.
4. **Check.** It reads the archive back: every file must be in it, with its size. If not, nothing is removed.
5. **Remove.**

Some files are never removed, whatever `--keep` says: hidden files (and, with `--recursive`, hidden folders such as `.git`), earlier `stb-clean` backups, and the files a kept `.fdf` reads.

1.5 What this tool simplifies — explicitly.

- The **kind** of a file comes from its name, not its contents. Only a file whose name says nothing (`submit.slurm`) is opened, to check whether it is a job script.
- The **previous run** is the SIESTA log among the `.out` files. "Finished" means the log ends with SIESTA's end-of-run line; the step count comes from its last `Begin CG/FIRE/Broyden/MD` line.
- **"Still running"** is a guess: an unfinished log written in the last two minutes. `stb-clean` cannot see the batch queue.

9.2.2.3 2. The clean at a glance



stb-clean flowchart

Figure 2. What `stb-clean` *does with one folder. With a glob, every folder goes through it in turn and the summary collects them.*

9.2.2.4 3. Libraries and external dependencies used

Library	Used for
Python standard library (`os`, `glob`, `tarfile`, `re`)	walking the folders, the archive and its check, reading the fdf files
`stb.core.cli`	the numbered report, tables, `--save-report`

Nothing else: no SIESTA, no scientific libraries. `stb-clean` works on any machine where the folders are, including a cluster's login node.

9.2.2.5 4. What `stb-clean` does

4.1 Step by step.

1. **Find the folders.** `--path` is a folder or a glob (`'strain_*'`); with a glob, each matching folder is cleaned on its own, with its own archive. `--recursive` also takes in the subfolders, as one tree.
2. **Sort every file.** Kept are files with an extension in `--keep`, hidden files and earlier backups. Every other file is to be removed.
3. **Read the kept inputs.** In the kept `.fdf` files, the first occurrence of a label wins, as in SIESTA. From them it takes the files they read (moved to *kept*), `SystemLabel`, the `UseSave` flags and the species.

4. **Find the previous run.** For each SIESTA log: finished or not, geometry steps, how long ago it was written.
5. **Report**, then **stop** for `--dry-run`.
6. **Ask** (unless `-n`), **archive and check** (unless `--no-archive`), **remove**.

4.2 The report, section by section.

Section	What it shows
`[0] RUN METADATA`	date, version, target, `--keep`, what is always kept, the mode (dry run, archive, no archive)
`[1] FOLDER`	files and sizes, kept vs removed, the SIESTA input and `SystemLabel`, the previous run, `0_NORMAL_EXIT`
`[2] KEPT`	every kept file and why: pseudopotential, input, job script, *read by calc.fdf*, hidden, backup; the species with and without a kept pseudopotential
`[3] REMOVED`	the files to go, by kind (Figure 1), with sizes and the five largest
`[4] RESTART`	`.DM`, `.XV`, `.CG`: kept or removed, the flag that reads it, what the next run starts from
`[5] ARCHIVE & REMOVAL`	the archive, its size, the check, the command that restores it, what was removed
`[6] SUMMARY & FILES`	totals; for a glob, one row per folder
`[7] WARNINGS`	every warning, numbered

With a glob, sections 1-5 repeat for each folder, so the section numbers keep counting (`[16] SUMMARY` for three folders).

4.3 Guard rails. The report warns when:

- no `.fdf` is kept: there is nothing to rerun from;
- a kept `.fdf` reads a file that is not in the folder;
- a species has no kept `.psf`/`.psml` (fine only if the job script supplies it);
- a file to be removed looks like a job script (`#!` first line, `#SBATCH`/`#PBS`/`#BSUB` lines), with the `--keep` that would keep it;
- the SIESTA log did not finish and was written in the last two minutes: the job may still be running;
- a finished run is removed with `--no-archive`, so no copy of its results is left;
- the archive check failed. In that case nothing was removed, and the exit code is 1.

9.2.2.6 5. Running it both ways

From the command line:

```
stb-clean --dry-run                                  # this folder: report only
stb-clean --path si_run                              # report, ask, archive + check, remove
stb-clean --path si_run --keep .fdf .psf .psml .sh .DM .XV   # continue from the previous run
stb-clean --path 'strain_*' -n                       # every matching folder, no questions (scripts)
stb-clean --path project --recursive --dry-run       # a whole tree
```

From the menu (`stb-suite`, code `3.2`), the same options are asked in turn: folder or glob (with Tab completion), extensions to keep, recursive, dry run, archive, confirmation and report. For a dry run, the archive and confirmation questions are skipped.

9.2.2.7 6. Worked example

`siesta_run/` is a finished SIESTA 5.4.2 run [1] of bulk Si in its 2-atom fcc cell. It is a variable-cell CG relaxation (PBE [2], DZP, 6×6×6 k-points, 200 Ry mesh) from a = 5.55 Å, 1.3 % above the PBE/DZP minimum, followed by bands along L–Γ–X, the DOS/PDOS, the density and the potential. The relaxation took 13 CG steps to a = 5.476 Å. The inputs are `calc.fdf` (it `%includes` `structure.fdf` and reads `bandlines.txt`), two job scripts (`run.sh` for PBS, `submit.slurm` for Slurm) and the PseudoDojo [3] `Si.psml`. The pseudopotential is not shipped here; each copy takes it from the suite's bundled bank. The input sets `DM.UseSaveDM true` and `MD.UseSaveXV true`, the usual choice for a calculation you may need to restart.

The guided script `example_3.2.sh` runs every case on a copy of `siesta_run/` in `output/<case>/`; the reference reports are in `reference_results/` (written by `make_reference.sh`).

6.1 What a finished run leaves behind (Case 1).

```
stb-clean --path si_run --dry-run
```

```
[1] FOLDER
Files             : 48 (3.8 MB)
Keep / remove     : 5 kept (86.3 KB), 43 to remove (3.7 MB)
SIESTA input      : calc.fdf, structure.fdf; SystemLabel si
Previous run      : calc.out -- finished normally, 13 geometry steps (last written 2.1 h ago)
Exit marker       : 0_NORMAL_EXIT present (SIESTA ended normally)

[3] REMOVED
kind                           | files | size     | e.g.
restart (SIESTA reads it back) | 4     | 377.2 KB | si.CG, si.DM, si.STRUCT_NEXT_ITER, +1
SIESTA log / status            | 16    | 172.9 KB | 0_NORMAL_EXIT, BASIS_ENTHALPY,
BASIS_HARRIS_ENTHALPY, +13
job-scheduler output           | 1     | 14 B     | siesta.err
geometry / forces / dynamics   | 5     | 2.9 KB   | si.ANI, si.BONDS, si.BONDS_FINAL, +2
electronic structure           | 9     | 2.0 MB   | si.DOS, si.EIG, si.HSX, +6
real-space grids               | 4     | 436.8 KB | Rho.grid.nc, TotalPotential.grid.nc, si.RHO, +1
basis by-products              | 3     | 762.3 KB | Si.ion, Si.ion.nc, Si.ion.xml
job script (not in --keep)     | 1     | 119 B    | submit.slurm
total                          | 43    | 3.7 MB   |
```

A 2-atom cell left 48 files. The 5 inputs are 86 KB; the other 43 files, 3.7 MB, are what SIESTA wrote. The report read the log and found that the run finished after 13 geometry steps, and it found `0_NORMAL_EXIT`. Figure 1 draws the same table.

6.2 `--dry-run`: what stays, and why (Case 2).

```
[2] KEPT
file          | size    | why it stays
Si.psml       | 84.7 KB | pseudopotential
bandlines.txt | 80 B    | read by calc.fdf
calc.fdf      | 921 B   | SIESTA input (.fdf)
run.sh        | 137 B   | job script
structure.fdf | 491 B   | SIESTA input (.fdf)
Protected         : 1 file(s) read by a kept .fdf, kept although their extension is not in --keep
Species           : Si -- pseudopotential kept for Si

[5] ARCHIVE & REMOVAL
Dry run -- nothing archived, nothing removed (43 file(s), 3.7 MB would go).
```

Nothing was touched; the folder still has its 48 files. `bandlines.txt` stays although `.txt` is not in `--keep`, because `calc.fdf` reads it (section 6.5). The species check confirms that the only species, Si, has its pseudopotential among the kept files.

6.3 The real clean: archive, check, remove; then restore (Case 3).

```
stb-clean --path si_run -n --save-report
```

```
[5] ARCHIVE & REMOVAL
Archive           : output/case3/si_run_backup_20261030_163507.tar.gz
                    43 file(s), 1.6 MB (42 % of 3.7 MB)
Verification      : read back, every file present with its size -- ok
Restore with      : tar -xzf output/case3/si_run_backup_20261030_163507.tar.gz -C output/case3/si_run
Removed           : 43 file(s), 3.7 MB freed
```

Left in the folder: `bandlines.txt`, `calc.fdf`, `run.sh`, `Si.psml`, `structure.fdf`, and the report (`stb_clean_report.txt`, written after the clean so that it does not remove itself). The archive sits next to the folder and compresses the outputs to 42 %. The script then restores it into a copy of the cleaned folder: the kept files plus the archive give back the original 48 files, one for one (`reference_results/clean/restored_files.txt`).

6.4 Start fresh, or continue from the previous run (Case 4). Two copies of the run are cleaned in two ways and run again with SIESTA (4 MPI processes):

```
stb-clean --path fresh    -n --no-archive                                  # default --keep
stb-clean --path continue -n --no-archive --keep .fdf .psf .psml .sh .DM .XV
```

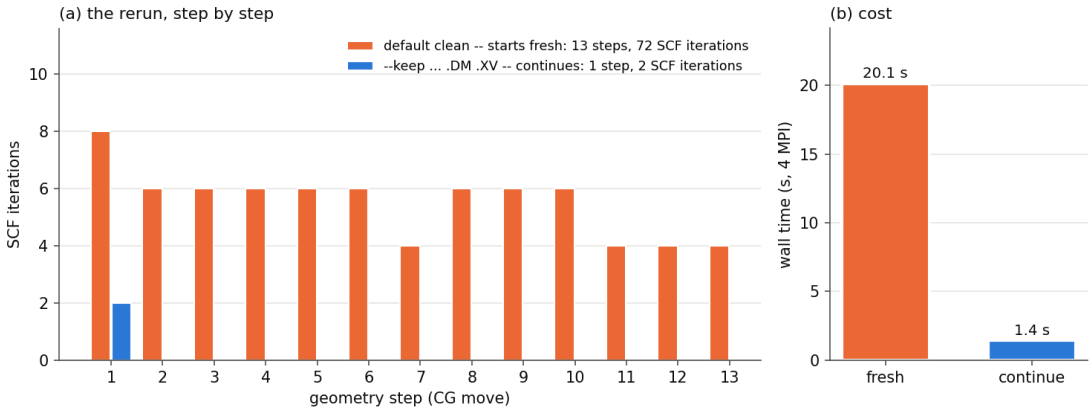
The `RESTART` section of each report says what will happen before SIESTA runs:

```
default --keep:
file  | holds                  | after cleaning | fdf flag                      | next run
si.DM | density matrix         | removed        | DM.UseSaveDM true             | starts from a
superposition of atomic densities
si.XV | positions + velocities | removed        | MD.UseSaveXV true             | starts from the
geometry in the .fdf
si.CG | CG optimizer history   | removed        | MD.UseSaveCG false (default) | starts from a new CG
history (flag off anyway)
Note              : the previous run moved the geometry (13 steps); its final geometry is in si.XV and
                    si.STRUCT_OUT, which go -- the next run starts again from the .fdf geometry. ...

--keep .fdf .psf .psml .sh .DM .XV:
si.DM | density matrix         | kept           | DM.UseSaveDM true             | next run reads it
si.XV | positions + velocities | kept           | MD.UseSaveXV true             | next run reads it
```

And the reruns (`siesta_rerun.sh`, data in `reference_results/rerun_*/`):

Clean	Geometry steps	SCF iterations	Final energy (eV)	Wall time
default `--keep` (fresh)	13	72	−230.035534	20.1 s
`--keep ... .DM .XV` (continue)	1	2	−230.035534	1.4 s



Rerun after a clean

Figure 3. (a) SCF iterations per CG step when the cleaned folder is run again. (b) Wall time on 4 MPI processes.

Both runs reach the same energy, to the last printed digit, but the costs differ by 14×. Without `.XV` the relaxation starts again from a = 5.55 Å and repeats all 13 steps (4-8 SCF iterations each, the first from atomic densities). With `.XV` and `.DM` the first geometry is already relaxed and its density already converged: 2 SCF iterations confirm both, and the run stops. For a real system the gap is hours, not seconds.

Neither choice is "right". Keep the restart files to continue a calculation, and drop them when the input changed in a way that makes the old state meaningless. A geometry relaxed with another basis, for example, is not the relaxed geometry of the new one.

6.5 Files a kept `.fdf` reads are protected (Case 5). `calc.fdf` reads two files:

```
calc.fdf:4:%include structure.fdf
calc.fdf:28:%block BandLines < bandlines.txt
```

The script adds a hidden file (`.notes`) and a line `%include missing_options.fdf` that points to a file which does not exist (`reference_results/protected/`):

```
.notes        | 17 B    | hidden file (never removed)
bandlines.txt | 80 B    | read by calc.fdf

[7] WARNINGS
  1. protected: a kept .fdf reads 'missing_options.fdf', which is not in the folder.
```

Without this, `bandlines.txt` would have been removed and the rerun would stop while reading its input. The same applies to a `%block ... < file` with coordinates, or to an included file of basis options. The missing include is caught before the job is submitted, not after it fails in the queue.

6.6 What the report warns about (Case 6). A careless `--keep .psml` (`reference_results/careless_keep/`):

```
1. si_run: no .fdf input is kept -- there is nothing to rerun from (check --keep).
2. si_run: 'run.sh' looks like a job script and will be removed -- add its extension to --keep
(--keep .psml .sh).
3. si_run: 'submit.slurm' looks like a job script and will be removed -- add its extension to --keep
(--keep .psml .slurm).
```

A run that stopped half-way, its `calc.out` cut at 60 KB and just written (`reference_results/running/`):

```
Previous run      : calc.out -- did NOT finish, 10 geometry steps (last written 0 min ago)
1. crashed: calc.out did not finish and was written 0 s ago -- is the job still running? Cleaning
would delete files it is writing.
```

`submit.slurm` is flagged in every case with the default `--keep`, because `.slurm` is not in it. The report recognises the file by its `#SBATCH` lines and lists it as *job script (not in --keep)* in `REMOVED`.

6.7 Several folders at once: a glob (Case 7). Three copies (`strain_m1`, `strain_0`, `strain_p1`), cleaned in one call that also keeps `.slurm`:

```
stb-clean --path 'strain_*' -n --keep .fdf .psf .psml .sh .slurm
```

```
[16] SUMMARY & FILES
folder    | kept | removed | size   | archive                                  | status
strain_0  | 6    | 42      | 3.7 MB | strain_0_backup_20261030_163523.tar.gz   | ok
strain_m1 | 6    | 42      | 3.7 MB | strain_m1_backup_20261030_163523.tar.gz  | ok
strain_p1 | 6    | 42      | 3.7 MB | strain_p1_backup_20261030_163524.tar.gz  | ok

Removed           : 126 file(s), 11.1 MB freed
[17] WARNINGS
  none
```

Each folder gets its own report sections and its own archive, and the summary collects them. Quote the glob, so that `stb-clean` expands it rather than the shell. A folder whose archive check fails is marked `FAILED`, keeps all its files, and makes the exit code 1.

6.8 A project tree: `--recursive` (Case 8). A `project/` holding two runs (`relax/`, `bands/`), a `.git/` folder, a hidden `.notes` and an earlier backup (`reference_results/recursive/`):

```
Files             : 98 (7.6 MB) -- including subdirectories
Keep / remove     : 12 kept (185.0 KB), 86 to remove (7.4 MB)
Previous run      : bands/calc.out -- finished normally, 13 geometry steps
Previous run      : relax/calc.out -- finished normally, 13 geometry steps

.notes                              | 22 B    | hidden file (never removed)
bands/bandlines.txt                 | 80 B    | read by bands/calc.fdf
relax/bandlines.txt                 | 80 B    | read by relax/calc.fdf
relax_backup_20260101_120000.tar.gz | 12.4 KB | earlier stb-clean backup (never removed)
```

`.git/` is never entered, so its files are not even counted. Each `bandlines.txt` is protected by the `calc.fdf` of its own subfolder. The tree is treated as one folder with one archive: for several independent runs side by side, a glob (section 6.7) gives one report and one archive per run.

6.9 The confirmation (Case 9).

```
43 file(s) will be removed:
  0_NORMAL_EXIT
  ...
  ... and 23 more

Delete these 43 file(s)? [y/N] n
Cancelled -- nothing archived, nothing removed.
```

The question comes after the report, once per folder; the default is no. In a script with no terminal, the question reads end-of-file and cancels, so a forgotten `-n` never deletes anything. Use `-n` when the deletion is intended.

6.10 The interactive menu (Case 10). `stb-suite` → `3.2` with a dry run gives the same report as the command line:

```
Mode              : dry run (nothing touched)
Dry run -- nothing archived, nothing removed (43 file(s), 3.7 MB would go).
Would remove      : 43 file(s), 3.7 MB -- rerun without --dry-run
```

9.2.2.8 7. What to take from the example

- A SIESTA folder is a few kilobytes of inputs and megabytes (or gigabytes) of outputs. `stb-clean` keeps the inputs and whatever they read, puts the rest into a checked archive next to the folder, and removes it.
- **Read the `RESTART` section before cleaning.** With `.DM`/`.XV` removed, the next run starts from scratch; with them kept and their flags on, it continues. On this run that was 13 steps against 1, at the same final energy.
- A relaxed geometry removed together with `.XV` and `.STRUCT_OUT` is lost unless the archive is kept. Convert it into an `.fdf` first if the next run should start from it.
- Use `--dry-run` on anything you care about. The warnings catch a forgotten `--keep`, a job script about to go, an include that is missing and a job that may still be running.

9.2.2.9 8. Reproducing the reference data

```
source ~/.anaconda3/etc/profile.d/conda.sh && conda activate stb   # or any environment with
stb_suite
bash make_reference.sh        # every report into reference_results/<case>/ (seconds, no SIESTA)
bash siesta_rerun.sh 4        # reference_results/rerun_fresh|rerun_continue/ (SIESTA 5.x + mpirun,
~25 s)
python3 theory_figures.py     # figures/run_contents.png, figures/rerun.png
dot -Tpng -Gdpi=150 figures/clean_flowchart.dot -o figures/clean_flowchart.png
```

The reports contain dates, archive names and "written N h ago" lines that change from run to run; everything else is deterministic. `siesta_run/` itself was produced with SIESTA 5.4.2 from `calc.fdf` + `structure.fdf` (`run.sh`).

9.2.2.10 9. Known, deliberate limitations

- **The kind of a file comes from its name.** A file with an unusual name lands in `other`. That does not change whether it is kept, which depends only on `--keep`.
- **`--recursive` treats the tree as one folder**: one report, one archive, and the `RESTART` section uses the first `SystemLabel` it finds. For several independent runs, use a glob.
- **"Still running" is a two-minute guess.** A job that has been waiting in a queue, or one that writes rarely, is not detected. Check the queue before cleaning a folder a job may use.
- **The archive goes into the parent folder**, which must be writable and have room for it (here 42 % of what is removed).
- **The tool does not judge whether a restart is physically valid.** It shows what the next run will read; whether a `.DM` or `.XV` from the previous input still belongs to the new one is your decision (section 1.2).

9.2.2.11 10. Step-by-step: cleaning your own folders

1. **Run `stb-clean --dry-run`** in the folder, or with `--path` and a glob for many folders.
2. **Read `KEPT`**: is every input there, including the files your `.fdf` reads and your job script? If not, add its extension to `--keep`.
3. **Read `RESTART` and decide**: continue (`--keep ... .DM .XV`) or start over (the default).
4. **Check the warnings**, especially a job that may still be running.
5. **Run it without `--dry-run`.** Keep the `*_backup_*.tar.gz` until the new run has finished, then delete it yourself.

9.2.2.12 11. Files in this folder

File	What it is
`README.md`	this tutorial
`example_3.2.sh`	the guided walk-through (10 cases, seconds; case 4 reruns SIESTA when available)
`siesta_run/`	the finished SIESTA 5.4.2 run (bulk Si, CG relaxation + bands + PDOS; 47 files, `Si.psml` not shipped)
`make_reference.sh`	regenerates the reports in `reference_results/`
`siesta_rerun.sh`	the reruns of section 6.4 (needs SIESTA)
`theory_figures.py`	Figures 1 and 3
`figures/`	every figure of this README; `clean_flowchart.dot` is the flowchart's source
`reference_results/`	one folder per case: `report.txt`, and the folder listings after it (`files_after.txt`, `archive_contents.txt`, `restored_files.txt`); `rerun_*/` hold the reruns' `calc.out` and `CLOCK`

9.2.2.13 12. Running the script

```
bash example_3.2.sh               # pauses between cases
yes '' | bash example_3.2.sh      # straight through
```

It needs the `stb_suite` package only; SIESTA is optional (case 4). It starts by wiping its own `output/`, never touches `siesta_run/`, and takes a few seconds.

9.2.2.14 13. Flag reference

Flag	Default	What it does
`--path PATH`	`.`	the folder, or a quoted glob of folders cleaned one after another
`--keep EXT ...`	`.psml .psf .fdf .sh`	extensions to keep; add `.DM .XV` to continue, `.slurm/.pbs` for job scripts
`--recursive`	off	the subfolders too (hidden ones are skipped)
`--dry-run`	off	report only; nothing archived or removed
`--no-archive`	off	no backup; the removed files cannot be recovered
`-n, --no-confirm`	off	do not ask before removing (for scripts)

Flag	Default	What it does
`--save-report`	off	also write `stb_clean_report.txt` (in the folder; in the current folder for a glob)
`--no-intro, -v`		skip the banner; print the version

9.2.2.15 14. What's next

- **`3.6` (`stb-status`)** summarises many run folders at once (run type, SCF, forces, energy), a quick way to see which ones finished before cleaning or rerunning them.
- **`3.7` (`stb-archive`)** packages a *finished* calculation with a manifest. Use it to keep results, and `stb-clean` to prepare a rerun.
- **`3.1` (`stb-translate`)** converts `<label>.STRUCT_OUT` into an `.fdf`, to restart from a relaxed geometry without its `.XV`.

9.2.2.16 15. References

1. J. M. Soler et al., "The SIESTA method for ab initio order-N materials simulation", *J. Phys.: Condens. Matter* **14**, 2745 (2002).
2. J. P. Perdew, K. Burke, M. Ernzerhof, "Generalized gradient approximation made simple", *Phys. Rev. Lett.* **77**, 3865 (1996).
3. M. J. van Setten et al., "The PseudoDojo: Training and grading a 85 element optimized norm-conserving pseudopotential table", *Comput. Phys. Commun.* **226**, 39 (2018).
4. A. García et al., "SIESTA: Recent developments and applications", *J. Chem. Phys.* **152**, 204108 (2020).

9.2.3 Command-line reference

9.2.3.1 `stb-clean`

Clean a SIESTA calc folder to restart a fresh run: removes every file except the extensions in `--keep` (pseudopotentials, .fdf input, submission script by default) – outputs/results too (.out, .XV, .DM, .bands, ...): it prepares a re-run, it does not archive results (that is stb-archive). Never removed: hidden files, earlier stb-clean backups and every file a kept .fdf reads (%include, '< file'). Every removed file is archived (tar.gz, next to the folder) and the archive is checked before anything is deleted, unless `--no-archive`. The report says what the previous run was, what stays and why, what goes by kind, and what the next run will start from (.DM/.XV/.CG vs the fdf UseSave flags).

Synopsis.

```
usage: stb-clean [-h] [--path PATH] [--keep KEEP [KEEP ...]] [--recursive] [--dry-run]
                 [--no-archive] [--no-confirm] [--save-report] [-v] [--no-intro]
```

Options

`--path PATH`
: Directory to clean, or a glob pattern (e.g. 'strain_*') matching several directories to clean one after another (default: current directory).

`--keep KEEP [KEEP ...]`
: Extensions to keep, i.e. NOT delete (default: .psml .psf .fdf .sh – pseudopotentials, the SIESTA input, and the submission script). Everything else, including prior run outputs, is removed. Add .DM .XV to continue from the previous run.

`--recursive`
: Also clean subdirectories (skips hidden ones, e.g. .git), not just the top level of `--path`. switch, off by default

`--dry-run`
: Report what would be kept and removed, and touch nothing (no archive either). switch, off by default

`--no-archive`
: Skip archiving removed files before deleting them. Archiving (tar.gz, next to `--path`, read back before anything is deleted) is the default safety net – turn it off only if you're sure. switch, off by default

`--no-confirm, -n`
: Don't ask for confirmation before deleting. switch, off by default

`--save-report`
: Also persist the report to `stb_clean_report.txt` (in `--path`; in the current directory for a glob). Off by default. switch, off by default

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

9.3 Grid to Cube Converter

At a glance

Menu code: 3.3 **Command:** `stb-cube`
Guided example: `examples/3.3-stb-cube/`
Summary: Write a SIESTA real-space grid (RHO, DRHO, BADER, TOCH, IOCH, RHOXC, RHOINIT, LDOS, VT, VH, VNA) as a Gaussian cube file, with a report that checks it: units, mesh and effective cutoff, geometry, integral against what it must be, spin channels, dipole of a molecule or slab, and a read-back of the cube.

9.3.1 Overview

`stb-cube` converts one of SIESTA's binary grid files, `<label>.<TYPE>` (`-t`; `-l`, which may include a folder, or the only such file in the current folder), to a Gaussian cube file for VESTA [18], VMD [20], XCrySDen [19] or a Bader code. The grid, geometry (`<label>.XV`) and spin configuration (`<label>.HSX`) are read with sisl [10]; the cube is written by the tool itself and read back with ASE. A cube has its lengths in Bohr and no unit for its values, so densities are written in e/Bohr^3 – sisl's e/Å^3 next to Bohr lengths would be 6.748 times too large (54 electrons instead of 8 in the guide) – and potentials in eV, Ry or Ha (`-potential-unit`); the unit is written in the cube's first line. The numbered report gives the mesh, the plane spacing and the effective mesh cutoff (compared with the SIESTA log), the cell and atoms (the `.XV` cell checked against the grid's), the integral against what each grid type must integrate to (valence electrons, zero, valence + core electrons, …), extremes and where they are, isosurface levels, every spin channel against SIESTA's spin moment (`-spin total/up/down/diff`; a potential's total is the average of up and down), and, for a molecule or slab, the vacuum planes of the potential and the dipole from the grid against SIESTA's. `-dry-run` reports without writing.

```
stb-cube -l si -t RHO                          # si.RHO -> si_RHO.cube
stb-cube -l run/h2o -t VH --potential-unit Ha  # a run in another folder
```

```
stb-cube -l o2 -t RHO --spin diff                # magnetization density
stb-cube -t DRHO --dry-run                       # the only *.DRHO here; report only
```

9.3.2 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/3.3-stb-cube/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

`stb-cube` writes one of SIESTA's real-space grid files (`<label>.RHO`, `.DRHO`, `.BADER`, `.TOCH`, `.IOCH`, `.RHOXC`, `.RHOINIT`, `.LDOS`, `.VT`, `.VH`, `.VNA`) as a Gaussian cube file, the format that VESTA, VMD, XCrySDen, Multiwfn and the Bader codes read. On the way it reports what the grid is:

- the quantity, its unit and the fdf flag that wrote it;
- the mesh, the spacing between grid planes and the **cutoff the mesh really has**, compared with SIESTA's log;
- the geometry the grid belongs to, checked against the grid's own cell;
- the values in the cube's unit: extremes and where they are, the **integral against what it must be** (the valence electrons for `RHO`, zero for `DRHO`, valence + core electrons for `BADER`, ...), and isosurface levels to start from;
- for a spin-polarized run, every channel (up, down, total, up − down) against SIESTA's spin moment;
- for a molecule or a slab, the potential in the vacuum and the **dipole from the grid**, compared with SIESTA's;
- a read-back of the cube that was written, with an independent reader.

Converting is the easy part. The report is there because a cube file carries no units and no provenance. A cube in the wrong unit or with a stale geometry opens without complaint and looks plausible. The worked example (section 6) shows the classic case: the same Si density written the naive way integrates to 54 electrons instead of 8.

The example uses three real, finished SIESTA 5.4 runs, each a few seconds of SIESTA: bulk Si with every grid SIESTA can write, a water molecule in a box, and the spin-polarized O_2 triplet. Everything runs in seconds; no SIESTA is needed.

9.3.2.1 Contents

1. Theory
2. The conversion at a glance
3. Libraries and external dependencies used
4. What `stb-cube` does
5. Running it both ways
6. Worked example
7. What to take from the example
8. Reproducing the reference data
9. Known, deliberate limitations
10. Step-by-step: converting your own grids
11. Files in this folder
12. Running the script
13. Flag reference

14. What's next
15. References

9.3.2.2 1. Theory

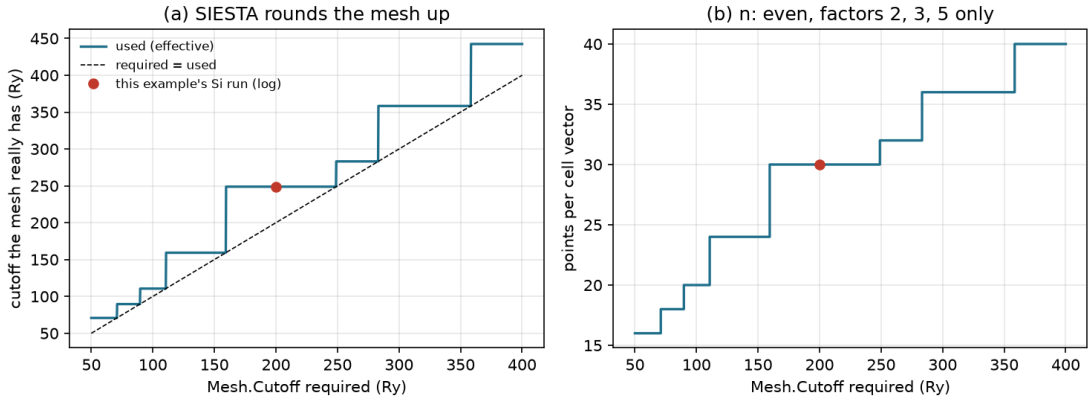
1.1 SIESTA's real-space mesh. SIESTA [1] expands the Kohn-Sham orbitals in numerical atomic orbitals, but computes the density, the Hartree and the exchange-correlation potentials on a uniform real-space mesh [1, 4]. The mesh divides each cell vector `a_i` into `n_i` intervals; a grid value `f_jkl` sits at

```
r_jkl = (j/n_1) a_1 + (k/n_2) a_2 + (l/n_3) a_3,        0 ≤ j < n_1,  0 ≤ k < n_2,  0 ≤ l < n_3
(1)
```

The grid is periodic: the point `j = n_1` is the point `j = 0` again and is not stored. The fineness is set by `Mesh.Cutoff` (`E_cut`, in Ry), the kinetic energy of the shortest plane wave the mesh can represent. With `b_i` the reciprocal vectors (2π included) in 1/Bohr, a mesh of `n_i` points along `a_i` holds waves up to `|G| = n_i |b_i| / 2`, so

```
E_mesh = min_i ( n_i |b_i| / 2 )^2   (Ry, with b_i in 1/Bohr)                    (2)
```

SIESTA takes the smallest `n_i` $\geq$ `2` $\sqrt{}$`E_cut / |b_i|` that is even and has no prime factor other than 2, 3 and 5 (for its FFTs). The mesh therefore always has **more** cutoff than requested, and SIESTA prints both (`InitMesh: Mesh cutoff (required, used)`). `stb-cube` computes `E_mesh` from the grid alone; for the three runs here it reproduces the log's value to the last digit (248.851, 122.834 Ry). The rule above, with the three runs as a check, is in `theory_figures.py`.



SIESTA's mesh against Mesh.Cutoff

Figure 1. (a) The cutoff the mesh really has against `Mesh.Cutoff`*, for this example's Si cell. (b) Points per cell vector. The required 200 Ry gives 30 points and 248.9 Ry, the red dot read from the log.*

The **spacing between grid planes** along `a_i` is `1 / (n_i |a*_i|)`, with `a*_i` the reciprocal vector without the 2π. In a non-orthogonal cell it is shorter than `|a_i| / n_i` (0.105 Å against 0.129 Å in the Si cell). The plane spacing, not the length along the vector, is the resolution that matters for an isosurface.

1.2 What each grid file holds. Each file is written by its own fdf flag:

File	fdf flag	Quantity	Integrates to
`.RHO`	`SaveRho`	valence electron density ρ	the valence electrons N
`.DRHO`	`SaveDeltaRho`	ρ minus the sum of free-atom densities	0
`.BADER`	`SaveBaderCharge`	ρ plus model core charges	N + core electrons = N + Σ (Z − Z_val)
`.TOCH`	`SaveTotalCharge`	ρ plus the ionic (pseudo-core) charge	0 for a neutral cell
`.IOCH`	`SaveIonicCharge`	the ionic charge alone, negative	$-\Sigma$ Z_val
`.RHOXC`	`SaveRhoXC`	the density XC sees (valence + partial core correction)	—
`.RHOINIT`	`SaveInitialChargeDensity`	the sum of free-atom densities	Σ Z_val
`.LDOS`	`%block LocalDensityOfStates`	local density of states in an energy window	the states in the window
`.VT`	`SaveTotalPotential`	Kohn-Sham potential: Hartree + local pseudopotential + XC	—
`.VH`	`SaveElectrostaticPotential`	electrostatic potential: Hartree + local pseudopotential	—
`.VNA`	`SaveNeutralAtomPotential`	neutral-atom potential (free atoms' electrostatic potential)	—

Charges follow SIESTA's convention: electrons positive, ions negative. Two consequences:

- **`DRHO` is the bonding picture.** The free atoms' densities are spherical, so $\Delta\rho$ shows only what bonding moved, without the large atomic background.
- **`BADER` exists for the Bader codes.** A valence pseudo-density has no maximum at the nucleus (it is small inside the core), so the zero-flux surfaces of a Bader analysis would be wrong. The model core charges put a maximum back at each atom [5].

1.3 Units: the 6.75 trap. SIESTA writes its grids in atomic units: the cell in Bohr, densities in e/Bohr^3, potentials in Ry. sisl [6], which `stb-cube` uses to read them, converts on reading to Å, e/Å^3 and eV. A cube file has its lengths in Bohr, and **its values carry no unit at all**. Every program that integrates a cube assumes values that match its lengths, so a density must be in e/Bohr^3.

Writing sisl's e/Å^3 values next to Bohr lengths gives a cube that looks right in every viewer but is too large by $1/0.529177^3 = 6.748$. Its integral is 54 electrons where there are 8 (case 2). `stb-cube` divides densities back to e/Bohr^3 and checks the integral of the cube it wrote. Potentials have no length in their unit; they are written in eV by default, or in Ry or Ha (`--potential-unit`). The unit is written in the cube's first comment line, so it travels with the file.

1.4 The cube format.

```
STB-CUBE 1.9: si.RHO -- valence electron density -- lengths in Bohr, values in e/Bohr^3   <- comment
1 (free)
OUTER LOOP: X, MIDDLE LOOP: Y, INNER LOOP: Z                                              <- comment 2
(Gaussian's)
    2     0.000000     0.000000     0.000000         <- number of atoms, origin (Bohr)
```

```
  30     0.000000     0.172469     0.172469         <- n1 and the voxel vector a1/n1 (Bohr)
  30     0.172469     0.000000     0.172469         <- n2, a2/n2
  30     0.172469     0.172469     0.000000         <- n3, a3/n3
  14    14.000000     0.000000     0.000000     0.000000    <- Z, charge, x y z (Bohr), one line per
  atom
  14    14.000000     2.587035     2.587035     2.587035
 2.24566e-03   5.13189e-03   1.40397e-02   2.82055e-02   4.39712e-02   5.58762e-02      <- values, z
  fastest
```

The values follow with x slowest and z fastest, six per line; each new (x, y) column starts a new line. The voxel vectors can be non-orthogonal, as here (the fcc primitive cell). Ghost atoms ($Z \leq 0$) are left out, since the format has no ghost species. Text cubes are about three times the size of SIESTA's binary grids (370 KB against 113 KB here).

1.5 Spin. A collinear spin-polarized run stores two components, $\rho\uparrow$ and $\rho\downarrow$. `--spin` chooses what goes into the cube:

`--spin`	density	potential
`total`	$\rho\uparrow + \rho\downarrow$	(V↑ + V↓) / 2
up, down	one channel	one channel
`diff`	m = $\rho\uparrow - \rho\downarrow$, the magnetization density	V↑ − V↓, the exchange splitting

A potential's total is the **average**: summing two potentials would double the potential. $\int$ `m` d^3r is the net spin moment SIESTA prints; $\int$ `|m|` d^3r is larger wherever some region is polarized against the net moment. Non-collinear and spin-orbit runs store four components (charge and a spin vector), which an up/down split does not describe; `stb-cube` writes their total charge and says so.

1.6 The checks. Integral. $\int$ `f` $d^3r \approx \Delta V \sum$`_jkl f_jkl`, with ΔV the voxel volume. On a periodic grid this sum is exact for the band-limited part of `f`, so `RHO` reproduces N to about 1e-5 e. `TOCH/IOCH` are an exception: their ionic charges are narrow Gaussians that the mesh samples with an error of a few 1e-3 e.

Dipole. For a molecule or a slab, the dipole along a non-periodic direction is

$$\mathtt{p} = \sum\mathtt{_I\ Z_val,I\ r_I} - \int \rho(\mathtt{r})\ \mathtt{r}\ d^3r \tag{3}$$

with positions measured from the atoms' centre by the minimum-image convention. `DRHO` gives the same `p` without the point charges, because free atoms have no dipole. Along a periodic direction a dipole is not defined this way (it depends on where the cell is cut), so `stb-cube` reports only the components normal to the periodic planes. It compares them with SIESTA's own `Electric dipole (Debye)` line.

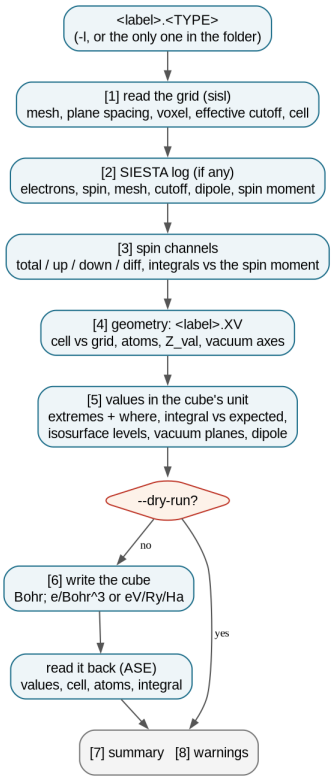
Vacuum level. Far from a neutral molecule or slab, the electrostatic potential should be flat; the plane average there is the vacuum level that a work function is measured from. A dipole across a periodic box makes it slope instead. For a slab, `Slab.DipoleCorrection` removes the slope; for a molecule in a box there is no single vacuum level. The report gives, for each vacuum axis, the plane average at the plane farthest from the atoms and how much it varies within ±1 Å.

Isosurface levels. A level `c` encloses the fraction $\int$`_{f>c} f` d^3r `/` $\int$ `f` d^3r of the charge. The report gives the levels that enclose 50, 90 and 99 % (of $\int$`|f|` for a signed grid, as ±c), in the cube's unit, as a starting point in VESTA or VMD.

1.7 What this tool simplifies — explicitly.

- **The geometry comes from `<label>.XV`**, the last one SIESTA wrote. If the grid is from an earlier step (or another run), the atoms in the cube are wrong; the cell check catches a different cell, not moved atoms.
- **The log is the only `.out` next to the grid**, or `<label>.out`. With several `.out` files and none named after the label, the checks against SIESTA's numbers are skipped.
- **The dipole needs vacuum** (a gap of at least 4 Å along an axis) and charge that does not cross the box boundary.
- **Non-collinear spin**: the total charge only (section 1.5).

9.3.2.3 2. The conversion at a glance



stb-cube flowchart

Figure 2. What `stb-cube` *does with one grid file. The numbers are the report's sections.*

9.3.2.4 3. Libraries and external dependencies used

Library	Used for
`sisl` [6]	reading SIESTA's binary grids (`.RHO`, `.VT`, ...), `.XV` and `.HSX`
`numpy`	statistics, integrals, dipole, isosurface levels
`ase` [7]	reading the written cube back, independently of the writer
`stb.core`	the SIESTA log parsers, vacuum detection, the numbered report

The cube is written by `stb-cube` itself, so that the header lines, the precision (six significant digits) and the units are under its control.

9.3.2.5 4. What `stb-cube` does

4.1 Step by step.

1. **Find the file.** `<label>.<TYPE>` from `-l` (which may include a folder), or the only `*.<TYPE>` in the current folder.
2. **Read the grid** with sisl: mesh, cell, number of spin components.
3. **Read the SIESTA log**, if one is next to the grid: electrons, spin configuration, mesh and cutoff, dipole, spin moment.
4. **Pick the spin channel** and read it.
5. **Read the geometry** from `<label>.XV`; compare its cell with the grid's.
6. **Compute** the statistics, the integral against what it must be, isosurface levels, vacuum planes and the dipole.
7. **Write the cube** (unless `--dry-run`) and **read it back** with ASE.

4.2 The report, section by section.

Section	What it shows
`[0] RUN METADATA`	date, versions, input, grid type, spin channel requested, unit, output
`[1] INPUT GRID`	file, quantity, fdf flag, stored unit, components, mesh, plane spacing, voxel volume, effective cutoff, cell
`[2] SIESTA RUN`	the log, finished or not, electrons, spin, mesh and cutoff against the grid's
`[3] SPIN`	the spin configuration and where it was read; for two components, every channel's integral against SIESTA
`[4] GEOMETRY`	the `.XV`, cell against the grid, dimensionality and vacuum axes, the atoms with Z and Z_val
`[5] VALUES`	min/max and where (nearest atom), mean, spread, integral vs expected, negative values, isosurface levels; vacuum planes and dipole when there is vacuum
`[6] OUTPUT`	the cube: size, units, header, the read-back check, the cube's own integral
`[7] SUMMARY & FILES`	the conversion in one line and the commands to open it
`[8] WARNINGS`	every warning, numbered

4.3 Guard rails. The report warns when:

- the integral differs from what the grid must integrate to (a different run, an unconverged SCF or a too coarse mesh);
- the `.XV` cell differs from the grid's (the atoms would be placed wrongly), or there is no `.XV` (no atoms);
- the log's mesh or cutoff differs from the grid's, or the log did not finish;
- `--spin` is given for an unpolarized grid, or for a non-collinear one;
- the up − down integral differs from SIESTA's spin moment;
- the dipole from the grid differs from SIESTA's (charge leaking across the box boundary, or a different run);
- a density is negative, or the density in the vacuum plane is not negligible (box too small);
- the cube read back differs from what was written.

A missing grid file stops with the list of grid files that are there and the flag that writes the one asked for.

9.3.2.6 5. Running it both ways

From the command line:

```
stb-cube -t RHO                                  # the only *.RHO in this folder -> <label>_RHO.cube
stb-cube -l si -t DRHO                            # si.DRHO -> si_DRHO.cube
stb-cube -l run/h2o -t VH --potential-unit Ha     # a run in another folder; potential in Hartree
stb-cube -l o2 -t RHO --spin diff                 # magnetization density -> o2_RHO_diff.cube
stb-cube -l si -t BADER --dry-run                 # report only
```

From the menu (`stb-suite`, code `3.3`), the same options are asked in turn: grid type (a numbered list with each type's fdf flag), label (Enter for the only file of that type), spin channel, potential unit (for a potential), output name, dry run and report.

9.3.2.7 6. Worked example

`siesta_runs/` holds three finished SIESTA 5.4.2 runs [1], PBE [2], DZP basis, PseudoDojo pseudopotentials [3]:

- **`si/`** — bulk Si in its 2-atom fcc cell at the PBE/DZP lattice constant (5.476 Å), 6×6×6 k-points, `Mesh.Cutoff 200 Ry`, a single point that saves every grid SIESTA can write (`RHO`, `DRHO`, `BADER`, `TOCH`, `IOCH`, `VT`, `VH`, `VNA`).
- **`h2o/`** — a water molecule in a 9 Å cubic box, Γ only, 100 Ry, relaxed (CG): `RHO`, `DRHO`, `VH`.
- **`o2/`** — O_2 in an 8 Å box, spin-polarized, 120 Ry: the triplet ground state, `RHO` with two components and `o2.HSX`.

Each folder keeps its `calc.fdf`, `calc.out`, `.XV` and grids (4.9 MB in all; the pseudopotentials are not shipped, `run_all.sh` copies them from the suite's bundled bank). The guided script `example_3.3.sh` runs every case on a copy in `output/<case>/`; the reference reports are in `reference_results/` (written by `make_reference.sh`).

6.1 Bulk Si: the valence density, the whole report (Case 1).

```
stb-cube -l si -t RHO
```

```
[1] INPUT GRID
Mesh              : 30 x 30 x 30 = 27000 points
Plane spacing     : 0.1054 / 0.1054 / 0.1054 Ang along a / b / c
Voxel volume      : 0.00152 Ang^3 (0.01026 Bohr^3)
Effective cutoff  : 248.851 Ry (min over a/b/c of 248.9 / 248.9 / 248.9 Ry)
Cell angles       : alpha = 60.00, beta = 60.00, gamma = 60.00 deg

[2] SIESTA RUN
Electrons         : 8.000000 (valence)
Mesh (log)        : 30 x 30 x 30 -- same as the grid
Mesh cutoff (log) : required 200.000 Ry, used 248.851 Ry -- from the grid 248.851 Ry (agrees)

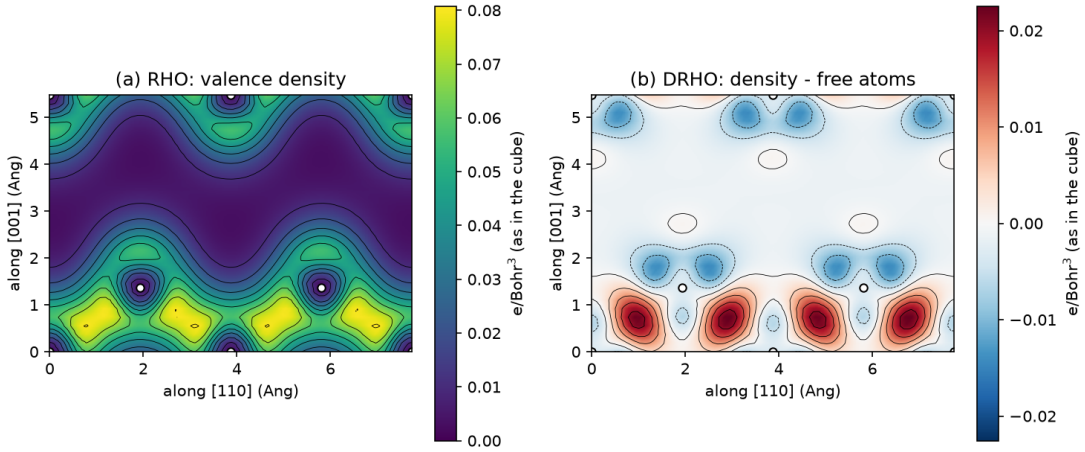
[5] VALUES
               | value (e/Bohr^3) | where
minimum        | 0.0022457        | (0.000, 0.000, 0.000) Ang, 0.000 Ang from Si1
```

```
maximum        | 0.080880         | (2.190, 2.190, 0.548) Ang, 0.948 Ang from Si1
Integral          : 7.999999 e  (positive part 8.0000, negative part 0.0000)
Expected          : 8.000000 e (valence electrons (log)) -- difference -1.21e-06 e (agrees)
Isosurface levels : 0.0456 (50 %), 0.0146 (90 %), 0.00525 (99 %) e/Bohr^3

[6] OUTPUT
File              : si_RHO.cube (369.6 KB)
Read-back check   : read again with ASE's cube reader -- ok
                    values agree to 6.2e-07 of the largest; cell, 2 atoms and atomic numbers as written
Integral (cube)   : 7.999999 e  (values x voxel volume in Bohr^3)
```

The requested 200 Ry became 248.9 Ry: 30 points per vector is the first allowed mesh above 200 Ry (Figure 1). The grid integrates to the 8 valence electrons to 1e-6, and so does the cube, integrated with its own Bohr voxels.

The extremes are physics, not noise. The **minimum** of the valence density is *at* the nucleus: a pseudopotential removes the core, and the valence pseudo-density is small there. The **maximum** is on the Si–Si bond, 0.95 Å from the atom (the bond is 2.37 Å). Figure 3(a) shows both, in a cube read back from disk.



Si density and deformation density

Figure 3. Bulk Si in the (1-10) plane, which contains the zig-zag chains of bonds, read from the cubes `stb-cube` *wrote. (a)* `RHO`*: the density is low at the atoms (white circles) and high on the bonds. (b)* `DRHO`*: charge gathered at the bond centres (red) and taken from the atoms' surroundings (blue).*

6.2 Units: what a naive conversion writes (Case 2). sisl can write a cube in one line (`grid.write("x.cube")`). Done that way, the cell goes out in Bohr and the values stay in sisl's e/Å^3 (`reference_results/units/integrals.txt`):

```
naive.cube    max  0.54581   sum x voxel (Bohr^3) =  53.9867 e
si_RHO.cube   max  0.08088   sum x voxel (Bohr^3) =   8.0000 e
```

The naive cube is 6.748 times too large (section 1.3). It shows the same isosurfaces at levels 6.75 times higher, so nothing looks wrong in a viewer. Any program that integrates it, a Bader code for example, gets 54 electrons.

6.3 Every grid of one run (Case 3). The Si run saved eight grids. Each one checked against what it must integrate to (`reference_results/si_all/`):

Grid	Integral (e)	Expected (e)	Note
RHO	7.999999	8 (valence electrons)	
DRHO	0.000000	0	+0.459 e gathered, 0.459 e taken
BADER	27.9998	28 = 8 + 2 × (14 − 4)	maximum 19.3 e/Bohr^3 at the nucleus
TOCH	−0.0058	0 (neutral)	ionic Gaussians sampled on the mesh
IOCH	−8.0058	−8	same mesh error

and the three potentials, in eV:

Grid	min	max	mean
VT	−94.17 (at the nucleus)	−5.27	−17.87
VH	−76.00 (at the nucleus)	−0.92	−8.68
VNA	−75.57 (at the nucleus)	−0.30	−8.68

DRHO shows that bonding moved 0.46 e per cell out of 8: this is the covalent bond charge of Figure 3(b). BADER has the 20 core electrons back, and its maximum sits on the nucleus, as a Bader analysis needs. VH and VNA have the same cell average because `V_H = V_NA + V_Hartree[`$\Delta\rho$`]` and the Hartree potential of $\Delta\rho$ averages to zero in SIESTA. In a bulk cell that average is a convention, not a vacuum level.

6.4 A molecule in a box: water (Case 4).

```
stb-cube -l h2o -t RHO
stb-cube -l h2o -t DRHO
stb-cube -l h2o -t VH --potential-unit Ha
```

The molecule leaves vacuum along all three axes, so the report adds the dipole and the vacuum planes:

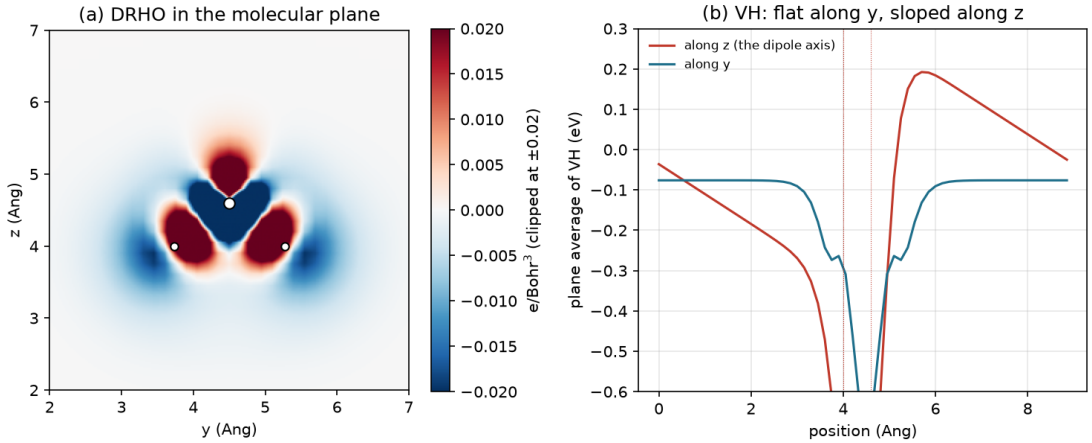
```
Dimensionality    : 0D (e.g., a molecule) -- vacuum along a, b, c (gap >= 4 Ang)
Integral          : 8.000014 e
Dipole            : from the grid -- density + valence point charges Z_val at the atoms; ...
component  | grid (Debye) | SIESTA (Debye) | difference
a: along x | 0.00000      | 0.00000        | +3.7e-09
b: along y | 0.00000      | 0.00000        | +6.4e-09
c: along z | -1.43364     | -1.43365       | +1.1e-05
```

The dipole from the grid agrees with SIESTA's to 1e-5 D. That is a check that the grid, the geometry and the log belong together. DRHO gives −1.43365 D without any point charges, and shows that bonding moved 0.478 e. Figure 4(a) shows where: into the O–H bonds and the lone-pair region above O, out of the space around the O core and behind each H.

The electrostatic potential (VH, here in Hartree):

```
axis | frac  | gap (Ang) | plane mean (Ha) | variation +-1 Ang | max |value| in plane
a    | 0.000 | 9.00      | -9.0832e-05     | 1.87e-07          | 0.00461
b    | 0.000 | 7.46      | -0.0027989      | 1.98e-07          | 0.00884
c    | 0.977 | 8.40      | -9.1786e-04     | 0.00572           | 0.00253
Not flat along c  : the plane mean changes by 0.00572 Ha within +-1 Ang -- a dipole across a periodic
box ...
```

Along x and y the plane average is flat in the vacuum. Along z, the dipole axis, it slopes by 0.156 eV per 2 Å: the periodic images of the dipole set up a field across the box (Figure 4(b)). There is no single vacuum level for this molecule. For a polar slab the same slope is what `Slab.DipoleCorrection` removes. Before reading a work function, check that this line is absent.



Water: deformation density and electrostatic potential

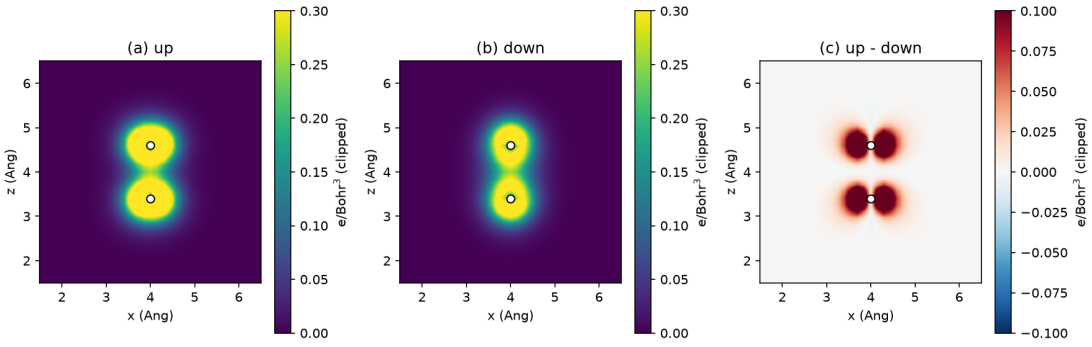
Figure 4. Water, from the cubes. (a) `DRHO` *in the molecular plane (red: gained, blue: lost). (b) Plane averages of* `VH`*: flat along y, a straight slope along z, where the dipole points.*

6.5 Spin: the O_2 triplet (Case 5).

```
stb-cube -l o2 -t RHO --spin diff
```

```
Spin configuration (from calc.out): polarized
channel           | integral (e) | SIESTA  |
total (up + down) | 12.0000      | 12.0000 |
up                | 7.0000       | 7.0000  |
down              | 5.0000       | 5.0000  |
up - down         | 2.0000       | 2.0000  | <- written
Absolute moment   : 2.0328 e (integral of |up - down|; ...)
Spin moment (log) : 2.00000 -- grid 2.00000 (agrees)
```

The SIESTA column comes from the log: the electron count and the spin moment, `N↑ = (N + S)/2`. O_2 has two unpaired electrons in its π^* orbitals, so S = 2. $\int$`|m| = 2.033 > 2`: a little density (0.016 e) is polarized *against* the moment, on the bond axis just outside the atoms. The four cubes (`total`, `up`, `down`, `diff`) get their own names, `o2_RHO_up.cube` and so on.



O2 spin density

Figure 5. O_2 in a plane containing the bond. (a) Up and (b) down densities. (c) The magnetization density: the two π lobes on each atom, perpendicular to the bond.*

6.6 What the report catches (Case 6). A grid that was not saved (`reference_results/guard_missing/`):

```
[FAIL] Input file 'si.LDOS' not found. Grid files in .: BADER, DRHO, IOCH, RHO, TOCH, VH, VNA, VT.
       SIESTA writes it only with %block LocalDensityOfStates.
```

A stale `.XV` from another run (`guard_stale/`; O_2's geometry next to the Si grid):

```
Cell vs grid      : DIFFERENT (largest difference 8.0000 Ang)
  1. lattice mismatch between the RHO grid and 'si.XV' (largest difference 8.0000 Ang) -- they may
  not belong
     to the same calculation; atom positions in the cube may be wrong.
```

No `.XV` (`guard_noxv/`): the cube is written without atoms, and the report says so. `--spin diff` on the unpolarized Si (`guard_spin/`): the total is written instead, with a warning. Two `*.RHO` files and no `-l` (`guard_ambiguous/`): it stops and lists them.

6.7 Labels and folders (Case 7). `-l` may include a folder: `stb-cube -l si/si -t VT` reads `si/si.VT`, finds `si/calc.out`, and writes `si/si_VT.cube` (and with `--save-report`, `si/stb_cube_report.txt`) next to the input (`reference_results/label_path/`). Without `-l`, the only file of that type in the current folder is used:

```
Input             : h2o.DRHO  (the only *.DRHO here)
```

6.8 The interactive menu (Case 8). `stb-suite` → `3.3`, type 10 (`VH`), label `h2o`, unit `Ha` gives the same report as the command line (`reference_results/menu/report.txt`):

```
Grid type         : VH -- electrostatic potential (Hartree + local part of the pseudopotentials)
Potential unit    : Ha in the cube
Output            : h2o_VH.cube
```

9.3.2.8 7. What to take from the example

- A cube carries no units. `stb-cube` writes densities in e/Bohr^3 (lengths are Bohr) and says so in the file. The naive conversion is 6.75 times too large and still looks right.
- **Read the integral line.** Each grid type must integrate to something known: 8, 0, 28, −8 for this Si. A mismatch means a different run, an unconverged SCF or a coarse mesh.
- The **effective cutoff** is the mesh's real resolution, always above `Mesh.Cutoff`.
- For a molecule or slab, the **dipole from the grid** agreeing with SIESTA's proves that grid, geometry and log belong together. A **sloped vacuum potential** means no well-defined vacuum level.
- For spin, write `diff` for the magnetization density; its integral is the spin moment.

9.3.2.9 8. Reproducing the reference data

```
source ~/.anaconda3/etc/profile.d/conda.sh && conda activate stb   # or any environment with
stb_suite
bash make_reference.sh          # every report into reference_results/<case>/ (seconds, no SIESTA)
bash siesta_runs/run_all.sh 4   # the three SIESTA runs (SIESTA 5.x + mpirun, ~10 s), pruned to what
is shipped
python3 theory_figures.py       # figures/mesh_cutoff.png, si_density.png, h2o.png, o2_spin.png
dot -Tpng -Gdpi=150 figures/cube_flowchart.dot -o figures/cube_flowchart.png
```

The reports' dates are replaced by <date>; everything else is deterministic.

9.3.2.10 9. Known, deliberate limitations

- **The geometry is the last `.XV`** (section 1.7): a grid saved at an earlier step of a relaxation gets the final atoms.
- **Non-collinear and spin-orbit grids**: the total charge only; no magnetization vector cubes.
- **One file per call.** Loop in the shell for many (`for t in RHO DRHO VH; do stb-cube -l si -t $t; done`).
- **Text cubes are large**: about 13 bytes per point, three times the binary grid. A 200^3 mesh gives a 100 MB cube.
- **No cube is cropped or coarsened.** The cube has exactly SIESTA's mesh, so it can be integrated; for a smaller file, lower `Mesh.Cutoff` in a dedicated run.
- **The dipole and the vacuum planes need 4 Å of vacuum** along an axis; a narrower gap is treated as periodic.

9.3.2.11 10. Step-by-step: converting your own grids

1. **Make SIESTA write the grid**: the fdf flag of section 1.2 (`SaveRho true`, ...).
2. **Run `stb-cube -l <label> -t <TYPE> --dry-run`** next to the run, with its `.XV` and log.
3. **Read `[2]` and `[5]`**: the mesh and cutoff agree with the log, the integral is what it must be, no warning about the `.XV`.
4. **Write the cube** (drop `--dry-run`) and open it. Start the isosurface at one of the levels the report gives.
5. **For a molecule or slab**, check the dipole line and the vacuum planes before using the potential's vacuum level.

9.3.2.12 11. Files in this folder

File	What it is
`README.md`	this tutorial
`example_3.3.sh`	the guided walk-through (8 cases, seconds)
`siesta_runs/`	the three finished SIESTA 5.4.2 runs (`si/`, `h2o/`, `o2/`: input, log, `.XV`, grids) and `run_all.sh`
`make_reference.sh`	regenerates `reference_results/`
`theory_figures.py`	Figures 1, 3, 4, 5 (from the cubes `stb-cube` writes)
`figures/`	every figure of this README; `cube_flowchart.dot` is the flowchart's source
`reference_results/`	one folder per case: `report.txt`, `cube_header.txt` (the first lines of a cube), `units/integrals.txt`

9.3.2.13 12. Running the script

```
bash example_3.3.sh               # pauses between cases
yes '' | bash example_3.3.sh      # straight through
```

It needs the `stb_suite` package only (no SIESTA). It starts by wiping its own `output/`, never touches `siesta_runs/`, and takes a few seconds.

9.3.2.14 13. Flag reference

Flag	Default	What it does
`-l, --label LABEL`	the only `*.<TYPE>` here	SystemLabel of the run; may include a folder (`run/si`)
`-t, --type TYPE`	required	`RHO`, `DRHO`, `BADER`, `TOCH`, `IOCH`, `RHOXC`, `RHOINIT`, `LDOS`, `VT`, `VH`, `VNA` (any case)
`-o, --output FILE`	`<label>_<TYPE>.cube`	output; `<label>_<TYPE>_<spin>.cube` for a spin channel other than total
`--spin CH`	`total`	`total`, `up`, `down`, `diff` (spin-polarized grids)
`--potential-unit U`	`eV`	`eV`, `Ry` or `Ha`, for `VT/VH/VNA`
`--dry-run`	off	report only; no cube
`--save-report`	off	also write `stb_cube_report.txt` next to the input
`--no-intro, -v`		skip the banner; print the version

9.3.2.15 14. What's next

- **`2.6` (`stb-bader`)** runs a Bader analysis on `RHO/BADER`.
- **`2.8` (`stb-density`)** draws 2D maps and planar averages of a `.RHO` directly, and density differences.
- **`2.7` (`stb-workfunction`)** reads the vacuum level of a slab from its potential (`.VT`); section 6.4 shows why the potential must be flat in the vacuum first.
- **`5.16` (charge density difference)** builds a $\Delta\rho$ between a system and its fragments, as a cube.

9.3.2.16 15. References

1. J. M. Soler et al., "The SIESTA method for ab initio order-N materials simulation", *J. Phys.: Condens. Matter* **14**, 2745 (2002).
2. J. P. Perdew, K. Burke, M. Ernzerhof, "Generalized gradient approximation made simple", *Phys. Rev. Lett.* **77**, 3865 (1996).
3. M. J. van Setten et al., "The PseudoDojo: Training and grading a 85 element optimized norm-conserving pseudopotential table", *Comput. Phys. Commun.* **226**, 39 (2018).
4. A. García et al., "SIESTA: Recent developments and applications", *J. Chem. Phys.* **152**, 204108 (2020).
5. W. Tang, E. Sanville, G. Henkelman, "A grid-based Bader analysis algorithm without lattice bias", *J. Phys.: Condens. Matter* **21**, 084204 (2009).
6. N. Papior, sisl: v0.16, https://github.com/zerothi/sisl (2024).
7. A. H. Larsen et al., "The atomic simulation environment — a Python library for working with atoms", *J. Phys.: Condens. Matter* **29**, 273002 (2017).

9.3.3 Command-line reference

9.3.3.1 `stb-cube`

Convert a SIESTA real-space grid file (`<label>.RHO`, .VT, .VH, ...) to a Gaussian cube file (VESTA, VMD, XCrySDen, Multiwfn, Bader codes), with a report on the grid: quantity and units, mesh and effective cutoff, geometry, spin channels, values (integral vs what it must be, extremes, isosurface levels), the dipole of a molecule or slab, and a read-back check of the cube. Lengths are written in Bohr, densities in e/Bohr^3, potentials in `--potential-unit`.

Synopsis.

```
usage: stb-cube [-h] [-l LABEL] -t {RHO,DRHO,BADER,TOCH,IOCH,RHOXC,RHOINIT,LDOS,VT,VH,VNA}
                [-o OUTPUT] [--spin {total,up,down,diff}] [--potential-unit {eV,Ry,Ha}]
                [--dry-run] [--save-report] [--no-intro] [-v]
```

Grid types (and the fdf flag that makes SIESTA write them).

```
RHO     valence electron density (SaveRho)
DRHO    deformation density: valence density minus the sum of free-atom densities (SaveDeltaRho)
BADER   valence density plus model core charges (input for a Bader analysis) (SaveBaderCharge)
TOCH    total charge: valence density plus the ionic (pseudo-core) charge, ions negative (SaveTotalCharge)
IOCH    ionic (pseudo-core) charge, negative in SIESTA's sign convention (SaveIonicCharge)
RHOXC   density seen by exchange-correlation (valence + partial core correction) (SaveRhoXC)
RHOINIT initial density: the sum of free-atom densities (SaveInitialChargeDensity)
LDOS    local density of states in the LocalDensityOfStates energy window (%block LocalDensityOfStates)
VT      total Kohn-Sham potential (Hartree + local pseudopotential + exchange-correlation) (SaveTotalPotential)
VH      electrostatic potential (Hartree + local part of the pseudopotentials) (SaveElectrostaticPotential)
VNA     neutral-atom potential (sum of the free atoms' electrostatic potentials) (SaveNeutralAtomPotential)
```

Options

`-l, --label LABEL`
: SystemLabel of the run; may include a folder (run/si). Default: the only `<label>.<type>` in the current folder.

`-t, --type {RHO,DRHO,BADER,TOCH,IOCH,RHOXC,RHOINIT,LDOS,VT,VH,VNA}`
: Grid file to convert (see the list above; any case). required

`-o, --output OUTPUT`
: Output file. Default: `<label>_<type>.cube` (`<label>_<type>_<spin>.cube` for a spin channel other than total), next to the input.

`--spin {total,up,down,diff}`
: Spin channel of a spin-polarized (collinear) grid: 'total' (default; up + down for a density, their average for a potential), 'up', 'down' or 'diff' (up - down: the magnetization density, or the exchange splitting of a potential). Ignored, with a warning, for an unpolarized grid and for a non-collinear/spin-orbit one. The spin configuration comes from the SIESTA log, else `<label>.HSX`. choices: total, up, down, diff

`--potential-unit {eV,Ry,Ha}`
: Unit of a potential (VT, VH, VNA) in the cube: eV (default), Ry (SIESTA's own) or Ha (atomic units). Densities are always e/Bohr^3. choices: eV, Ry, Ha

`--dry-run`
: Report only; write no cube. switch, off by default

`--save-report`
: Also write the report to `stb_cube_report.txt` (next to the input). Off by default. switch, off by default

Like every STB command it also accepts `-h`/`--help`, `-v`/`--version` and `--no-intro` (Section 3.3).

From the built-in help.

```
Example usage:
  stb-cube -t RHO                          # the only *.RHO in this folder
  stb-cube -l si -t RHO                    # si.RHO -> si_RHO.cube
  stb-cube -l run/h2o -t VH --potential-unit Ha -o h2o_vh.cube
  stb-cube -l o2 -t RHO --spin diff        # magnetization density up - down
  stb-cube -l si -t DRHO --dry-run         # report only, no cube
```

9.4 Wantibexos Interface

At a glance

Menu code: 3.4 **Command:** `stb-siesta2wtb`
Guided example: `examples/3.4-stb-siesta2wtb/`
Summary: Write the SIESTA Hamiltonian and overlap (`SaveHS true`) as the tight-binding files of WanTiBEXOS, with a report on the model and a read-back check of the bands rebuilt from the written file.

9.4.1 Overview

`stb-siesta2wtb` exports the real-space Hamiltonian and overlap of a SIESTA run to the tight-binding input of WanTiBEXOS [58], a code for excitonic and optoelectronic properties from tight-binding Hamiltonians. The input `.fdf` (`-i`) names the run; sisl [10] reads `<label>.HSX`, which SIESTA writes with `SaveHS true`. Two files are written in the current folder: `tb-<S>.ham` (the spin type, a scissor value, the Fermi level, the lattice vectors, the matrix size and number of cells, then every element as `R_x R_y R_z i j ReH ImH S`, with R in Å and H in eV) and `basis_set-<S>.basis` (index, species, position, l, m and spin of every orbital); `<S>` is `NP`, `SP` or `SOC`. The Fermi level in the header comes from the SIESTA log (`-o`) or `-f`. The numbered report gives the system and its basis per species, the spin type and matrix size, the neighbouring cells (`nsc`), sparsity and longest hopping, the spectrum of $S(\Gamma)$, the Fermi level and *the energy zero of the matrix elements* (an `.HSX` of version ≥ 1 stores E_F and sisl shifts H by $-E_F S$ on reading), the band edges from `<label>.EIG`, and a read-back check: $H(k)$ and $S(k)$ are rebuilt from the written file and their eigenvalues compared with sisl's at four k-points, judged within 5 eV of E_F. `--dry-run` reports without writing; `--no-check` skips the read-back.

```
stb-siesta2wtb -i calc.fdf -o calc.out             # Fermi level from the log
stb-siesta2wtb -i calc.fdf -f 0                    # header E_F = 0
stb-siesta2wtb -i calc.fdf -o calc.out --dry-run   # report only
```

9.4.2 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/3.4-stb-siesta2wtb/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

`stb-siesta2wtb` takes the Hamiltonian and overlap that SIESTA saves in real space (`<label>.HSX`, written with `SaveHS true`) and writes them as the tight-binding files of WanTiBEXOS [1], a code for excitonic and optical properties from tight-binding Hamiltonians:

- `tb-<S>.ham`: the spin type, a scissor shift, the Fermi level, the lattice vectors, then every element of H and S between the orbitals of the unit cell and those of each neighbouring cell;
- `basis_set-<S>.basis`: one line per orbital, with species, position, `l`, `m` and spin.

`<S>` is `NP` (no spin), `SP` (collinear spin) or `SOC` (non-collinear and spin-orbit). Its report says what went into the files:

- the system: cell, dimensionality, the basis of each species;
- the Hamiltonian: spin type, how many neighbouring cells it reaches, how sparse it is, the longest hopping, how close the basis is to linear dependence;
- the Fermi level, **what energy is the zero of the matrix elements**, the band edges from SIESTA's `.EIG`, and the number of occupied states;
- **a read-back check**: H(k) and S(k) are rebuilt from the written `.ham` and their eigenvalues compared with sisl's on the SIESTA matrices, at several k-points;
- what to put in the WanTiBEXOS input (`PARAMS_FILE`, `SYSDIM`, how many valence and conduction bands exist).

The example uses two real, finished SIESTA 5.4 runs with `SaveHS true`: an hBN monolayer (`NP`, a 2D insulator, the kind of system WanTiBEXOS is used for) and ferromagnetic bcc Fe (`SP`). The bands rebuilt from the written files reproduce SIESTA's to 0.1 meV near the Fermi level (section 6). Everything runs in about 20 seconds; no SIESTA is needed.

9.4.2.1 Contents

1. Theory
2. The conversion at a glance
3. Libraries and external dependencies used
4. What `stb-siesta2wtb` does
5. Running it both ways
6. Worked example
7. What to take from the example
8. Reproducing the reference data
9. Known, deliberate limitations
10. Step-by-step: exporting your own Hamiltonian
11. Files in this folder
12. Running the script
13. Flag reference
14. What's next
15. References

9.4.2.2 1. Theory

1.1 A SIESTA Hamiltonian is already a tight-binding model. SIESTA [2] expands the Kohn-Sham states in numerical atomic orbitals φ_μ, each centred on an atom and strictly zero beyond a cutoff radius. A Bloch state is

$$\psi_{nk}(r) = \sum_\mu c_{\mu,nk} \sum_R e^{ik\cdot R} \varphi_\mu(r - \tau_\mu - R) \quad (1)$$

and the Kohn-Sham equation becomes a generalized eigenproblem of small matrices:

$$H(k)\, c = E\, S(k)\, c, \qquad H_{\mu\nu}(k) = \sum_R e^{ik\cdot R} H_{\mu\nu}(R), \qquad S_{\mu\nu}(k) = \sum_R e^{ik\cdot R} S_{\mu\nu}(R) \quad (2)$$

with $H_{\mu\nu}(R) = \langle \varphi_\mu(0) | \hat{H} | \varphi_\nu(R) \rangle$ and $S_{\mu\nu}(R) = \langle \varphi_\mu(0) | \varphi_\nu(R) \rangle$. Because the orbitals have finite range, only a few cells `R` give non-zero elements. **Eq. 2 is a tight-binding model**, with first-principles parameters and no fitting. A Wannier-function model [3] has to be constructed from the plane-wave bands; here the model is already there. `SaveHS true` makes SIESTA write `H(R)` and `S(R)` to `<label>.HSX`, and `stb-siesta2wtb` writes them out again in WanTiBEXOS's format.

Two differences from a Wannier model follow:

- **The basis is not orthogonal.** `S(R)` is not the identity, so every element of `H` comes with its `S`, and the bands come from Eq. 2, not from `H(k) c = E c`.
- **The basis has as many orbitals as SIESTA used** (13 per atom for B or N with DZP), not just a few bands around the gap. The highest of the resulting bands lie 50 to 150 eV above the Fermi level and have no physical meaning.

1.2 The neighbouring cells: `nsc`. sisl [4] stores `H(R)` for an auxiliary supercell of `nsc = n_1 × n_2 × n_3` cells, every `R` for which some orbital of the unit cell overlaps some orbital of cell `R`. The file has one block per cell, `(dim)`^{2} lines each, zeros included:

$$\text{lines} = n_1\, n_2\, n_3 \times \text{dim}^2, \qquad \text{dim} = \begin{cases} \text{N_orb (NP)} \\ \text{or} \\ 2\ \text{N_orb (SP, SOC)} \end{cases} \quad (3)$$

The cutoff radii set `nsc`. In a 2D cell with 20 Å of vacuum it is `5 × 5 × 1`; in bulk Fe it is `9 × 9 × 9`, and the file has a million lines (section 6.2). `PAO.EnergyShift` sets the radii; a larger value gives shorter orbitals and a smaller file.

1.3 Spin.

SIESTA `Spin`	sisl components	file type	matrix	layout
`none`	1	`NP`	N × N	
`polarized`	2 (up, down)	`SP`	2N × 2N	up block (orbitals 1..N), then down block (N+1..2N); no up-down coupling
`non-collinear`	4	`SOC`	2N × 2N	up-down blocks
`spin-orbit`	8	`SOC`	2N × 2N	up-down blocks, complex

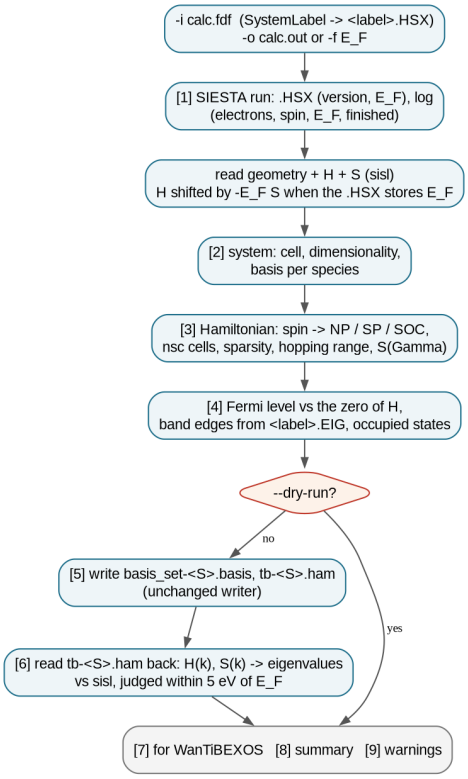
1.4 The energy zero and the Fermi level. The file's header has a Fermi level `E_F`, given with `-o calc.out` (read from SIESTA's log) or `-f`. The matrix elements have an energy zero of their own, and the two need not be the same:

- An `.HSX` of version 1 or 2 (SIESTA $\geq$ 4.1) stores the run's Fermi level, and sisl, which reads it, **shifts the Hamiltonian on reading**: `H` $\rightarrow$ `H − E_F S`. The matrix elements in the file are then referred to E_F. A state at the Fermi level has `E = 0`.
- An `.HSX` of version 0 has no Fermi level; the elements are then on SIESTA's absolute scale.

The report says which case applies (`Energy zero of H`) and warns when the header's E_F is not the zero of H. With an `.HSX` that stores E_F and `-o calc.out`, the header carries E_F although H is already referred to it. A program that subtracts the header's E_F from H then moves every band by `−E_F` (+4.02 eV for the hBN here). With `-f 0` the header matches H. Check how the version of WanTiBEXOS you use treats the header's Fermi level (section 6.3).

1.5 The read-back check. The written `.ham` is read again, and for each of four k-points (Γ, (½,0,0), ($\frac{1}{3}$,$\frac{1}{3}$,0) and a general point, with `k = 0` along vacuum axes) Eq. 2 is solved with the file's numbers. The eigenvalues are compared with sisl's, which come from the SIESTA matrices directly. The check is judged **within 5 eV of the Fermi level**. The file has 6 decimals (`%12.6f`). Where the basis is nearly linearly dependent, the smallest eigenvalue of `S(k)` is ~10^{-3}, and the rounding of `S` moves the highest, unphysical bands by tens to hundreds of meV. Near the Fermi level the change is below 0.1 meV (section 6.4). The report gives both numbers, the smallest eigenvalue of `S(Γ)`, the number of states below the Fermi level at each k, and the hermiticity of the rebuilt matrices.

9.4.2.3 2. The conversion at a glance



stb-siesta2wtb flowchart

Figure 1. What `stb-siesta2wtb` *does. The numbers are the report's sections; the files themselves are written by the same code as before this report existed.*

9.4.2.4 3. Libraries and external dependencies used

Library	Used for
`sisl` [4]	reading the `.fdf`, `.HSX` (H, S, E_F, geometry, basis) and `H.eigh(k)` for the check
`numpy`, `scipy`	building H(k), S(k) from the file, the generalized eigenproblem
`pandas`	reading the `.ham` back quickly (falls back to `numpy.loadtxt`)
`stb.core`	the SIESTA log parsers (`get_fermi_energy`, `read_eig_gap`, ...), vacuum detection, the report

WanTiBEXOS itself is not needed to write the files.

9.4.2.5 4. What `stb-siesta2wtb` does

4.1 Step by step.

1. **Find the run.** `SystemLabel` from the `.fdf` (`-i`) gives `<label>.HSX`. The log is `-o`, or else `<label>.out` or the only `*.out` (used only for the report).
2. **Read** geometry, H and S with sisl.
3. **Describe** the system, the basis, the Hamiltonian's range, the Fermi level and its reference.
4. **Write** `basis_set-<S>.basis` and `tb-<S>.ham` in the current folder (not with `--dry-run`).
5. **Check** the written `.ham` (not with `--no-check`).

4.2 The report, section by section.

Section	What it shows
`[0] RUN METADATA`	date, versions, input and `SystemLabel`, where the Fermi level comes from, the mode
`[1] SIESTA RUN`	the `.HSX` (size, date), `SaveHS` in the input, the log (version, finished, electrons, spin, E_F, spin moment), the energy zero of H
`[2] SYSTEM`	cell, dimensionality, per species: Z, atoms, orbitals, basis shells; orbitals in the cell
`[3] HAMILTONIAN`	spin → file type and matrix size, orthogonality, `nsc` and cells, stored elements, longest hopping, on-site energies, the spectrum of `S(Γ)`, the spin layout
`[4] FERMI LEVEL AND BAND EDGES`	E_F written and its source, its relation to the zero of H, VBM/CBM/gap from `<label>.EIG`, occupied states
`[5] OUTPUT FILES`	both files, sizes, write times, the header, the number of lines (or what would be written)
`[6] READ-BACK CHECK`	header and line count read back, non-zero lines, eigenvalues vs sisl per k, hermiticity, verdict
`[7] FOR WANTIBEXOS`	`PARAMS_FILE`, `SYSDIM`, the valence/conduction states available for `NBANDSV`/`NBANDSC`, the energy reference
`[8] SUMMARY & FILES`	the files and the system in one line
`[9] WARNINGS`	every warning, numbered

Section	What it shows

4.3 Guard rails. The report warns when:

- the header's Fermi level is not the zero of the matrix elements (section 1.4), or no Fermi level was given for a Hamiltonian on SIESTA's absolute scale;
- the SIESTA log did not finish;
- `S(Γ)` has an eigenvalue below 10^{-3} (nearly linearly dependent basis: the 6 decimals move the highest bands);
- for an insulator, the number of states below the Fermi level at Γ is not the number of occupied states;
- the file read back does not reproduce sisl's eigenvalues within 1 meV, 5 eV around E_F.

It stops, with the reason, when the input or the `-o` log is missing, or when there is no Hamiltonian (no `.HSX`: the run needs `SaveHS true`).

9.4.2.6 5. Running it both ways

From the command line:

```
stb-siesta2wtb -i calc.fdf -o calc.out              # Fermi level from the log
stb-siesta2wtb -i calc.fdf -f 0                     # header E_F = 0, the zero of H for a modern .HSX
stb-siesta2wtb -i calc.fdf -o calc.out --dry-run    # report only
stb-siesta2wtb -i calc.fdf -o calc.out --no-check   # skip the read-back (very large files)
```

From the menu (`stb-suite`, code `3.4`): the `.fdf`, the log (optional), a Fermi level by hand (optional), dry run, read-back check and report are asked in turn.

9.4.2.7 6. Worked example

`siesta_runs/` holds two finished SIESTA 5.4.2 runs [2], PBE [5], DZP basis, `PAO.EnergyShift 0.02 Ry`, PseudoDojo pseudopotentials [6], both with `SaveHS true` and a band structure:

- **`hbn/`** — an hBN monolayer, a = 2.51 Å, 20 Å of vacuum, 15×15×1 k-points. SIESTA gives a direct gap of 4.569 eV at K, the usual PBE value.
- **`fe/`** — ferromagnetic bcc Fe, a = 2.83 Å, 14^3 k-points, 0.01 Ry Methfessel-Paxton smearing. The spin moment is 2.22 μ_B.

Each folder keeps `calc.fdf`, `calc.out`, `.XV`, `.HSX`, `.EIG`, `.KP` and `.bands` (1.9 MB in all; the pseudopotentials are not shipped, `run_all.sh` copies them from the suite's bundled bank). The guided script `example_3.4.sh` runs every case on a copy in `output/<case>/`; the reference reports are in `reference_results/`.

6.1 hBN: the whole report (Case 1).

```
stb-siesta2wtb -i calc.fdf -o calc.out
```

```
[1] SIESTA RUN
Hamiltonian       : hbn.HSX (210.4 KB, written <date>)
Fermi level (log) : -4.022238 eV
```

```
Energy zero of H  : the run's Fermi level -- hbn.HSX (version 2) stores E_F = -4.022238 eV and sisl
shifts H by -E_F S

[2] SYSTEM
Dimensionality    : 2D (e.g., a slab or surface) -- vacuum along c
species | Z | atoms | orbitals | basis shells         | r_c (Ang)
B       | 5 | 1     | 13       | 2s(2z) 2p(2z) 3d(1z) | -
N       | 7 | 1     | 13       | 2s(2z) 2p(2z) 3d(1z) | -

[3] HAMILTONIAN
Spin (sisl)       : Spin{unpolarized} -> WanTiBEXOS type NP, 26 x 26 matrices
Orthogonal basis  : no -- the overlap S goes into the file with H
Neighbour cells   : nsc = 5 x 5 x 1 = 25 cells (all R with an orbital overlapping one of the cell)
Stored elements   : 10688 of 16900 (63.2 %) per spin component
Longest hopping   : 5.225 Ang (orbital centres; the range of the basis)
S(Gamma) spectrum : 6.56e-04 to 4.881 (smallest eigenvalue: how close the basis is to linear
dependence)

[4] FERMI LEVEL AND BAND EDGES
SIESTA .EIG       : VBM -5.5166, CBM -0.9475 eV -> gap 4.5691 eV (direct; smallest direct 4.5691 eV;
113 k-points)
Occupied states   : 4 of 26 per k (8 electrons, 2 per state)

[5] OUTPUT FILES
Hamiltonian       : tb-NP.ham (1.3 MB, 0.5 s)
Body              : 16900 lines 'R_x R_y R_z i j ReH ImH S' (R in Ang, H in eV), every i, j of every
cell, zeros included

[6] READ-BACK CHECK
k                     | states < E_F | max |dE| within 5 eV of E_F (meV) | max |dE| all (meV) | worst
band at E - E_F (eV)
(0.000, 0.000, 0.000) | 4            | 0.006                            | 161.2              | +74.0
(0.500, 0.000, 0.000) | 4            | 0.006                            | 38.9               | +173.6
(0.333, 0.333, 0.000) | 4            | 0.005                            | 28.5               | +71.7
(0.130, 0.270, 0.000) | 4            | 0.001                            | 14.7               | +159.3
Verdict           : ok -- within 5 eV of E_F the file reproduces the SIESTA bands to 0.006 meV

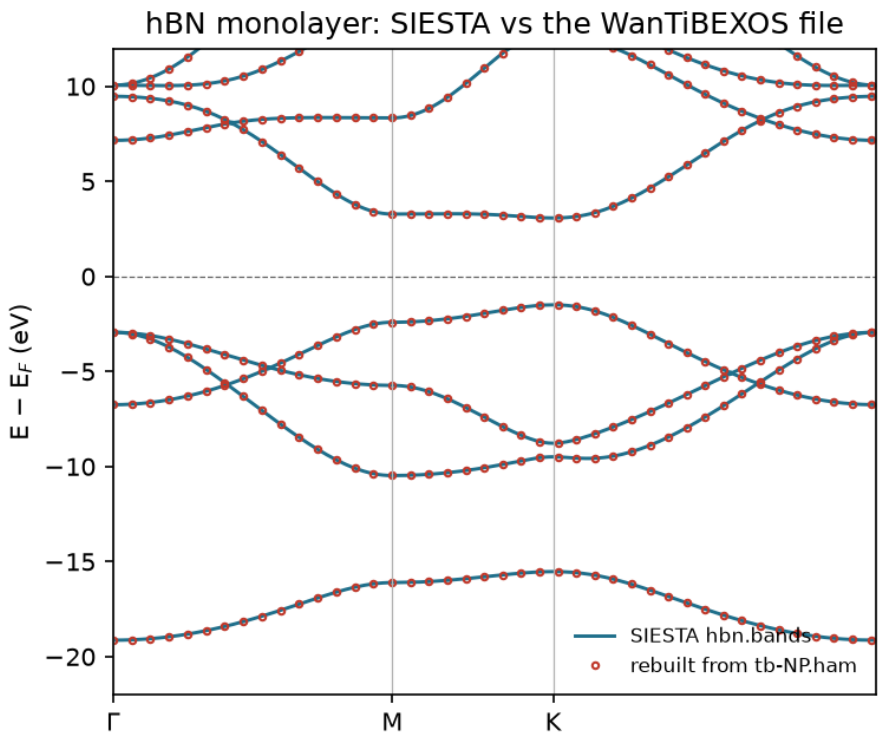
[7] FOR WANTIBEXOS
PARAMS_FILE       : tb-NP.ham
SYSDIM            : 2D (from the vacuum axes)
Valence states    : 4 occupied of 26 -- NBANDSV can be at most 4, NBANDSC at most 22
```

DZP gives 13 orbitals per atom: two radial functions (`2z`) for 2s and for 2p, and one shell of 3d polarization orbitals. The polarization shell shows as `3d(1z)`, since the `.HSX` does not mark it as such. The orbitals reach 5.2 Å, so H couples the cell to its 24 nearest cells in the plane (`5 × 5 × 1`) and none across the vacuum. The file holds 16 900 lines, 31 % of them non-zero. The 4 occupied states are the σ and π bands of the 8 valence electrons, at every k checked. The first lines of the file (`reference_results/hbn/tb_head.txt`):

```
NP                                                  <- type
    0.000000                                        <- scissor (eV)
   -4.022238                                        <- E_F written (eV)
    2.510000     0.000000     0.000000              <- a_1 (Ang)
   -1.255000     2.173724     0.000000              <- a_2
    0.000000     0.000000    20.000000              <- a_3
26                                                  <- matrix size
25                                                  <- cells
```

```
# rcell_x rcell_y rcell_z i j ReH ImH S
    0.000000     0.000000     0.000000 1 1    -9.667332     0.000000     1.000000    <- B 2s, on-site:
    E - E_F, S = 1
    0.000000     0.000000     0.000000 1 2    -7.776185     0.000000     0.934106    <- B 2s zeta 1 -
    zeta 2
```

`R` is the Cartesian position of cell `R` (Å). `i` is in the unit cell, `j` in cell `R`.



hBN bands from the file

Figure 2. hBN: SIESTA's own bands (`hbn.bands`*, lines) and the eigenvalues of Eq. 2 rebuilt from* `tb-NP.ham` *at the same k-points (circles). Within 5 eV of E_F they agree to 0.063 meV along the whole path.* `theory_figures.py` *draws it.*

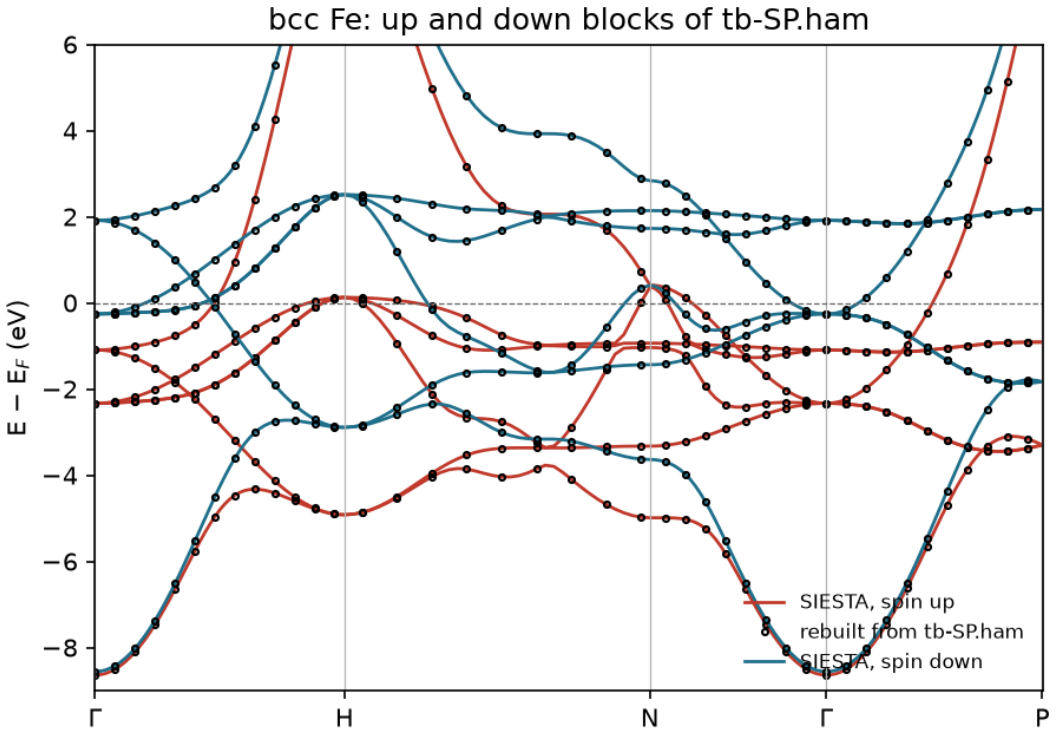
6.2 bcc Fe: two spin blocks (Case 2).

```
species | Z  | atoms | orbitals | basis shells                        | r_c (Ang)
Fe      | 26 | 1     | 19       | 3s(1z) 3p(1z) 3d(2z) 4s(2z) 4p(1z) | -
Spin (sisl)       : Spin{polarized} -> WanTiBEXOS type SP, 38 x 38 matrices
Neighbour cells   : nsc = 9 x 9 x 9 = 729 cells (all R with an orbital overlapping one of the cell)
Stored elements   : 12423 of 263169 (4.7 %) per spin component
Longest hopping   : 8.004 Ang (orbital centres; the range of the basis)
Layout            : up block (orbitals 1..N), then down block (N+1..2N); no up-down coupling
SIESTA .EIG       : metallic on the SCF k-mesh (1484 k-points; gap 0.003 eV, below the
k-mesh/smearing resolution)
Hamiltonian       : tb-SP.ham (83.9 MB, 12.1 s)
Body             : 1052676 lines 'R_x R_y R_z i j ReH ImH S' (R in Ang, H in eV), every i, j of every
cell, zeros included
Non-zero lines    : 15054 (1.4 %)
Verdict           : ok -- within 5 eV of E_F the file reproduces the SIESTA bands to 0.033 meV
```

Fe has 19 orbitals, including the 3s and 3p semicore shells of the PseudoDojo pseudopotential. Its diffuse 4s and 4p orbitals reach 8 Å, so the file spans 729 cells: one million lines, 84 MB, of which 1.4 % are non-zero. Orbital 1 is the spin-up 3s, orbital 20 the spin-down 3s:

```
0.000000  0.000000  0.000000  1  1  -87.667130  0.000000  1.000000      <- 3s up
0.000000  0.000000  0.000000 20 20  -85.145302  0.000000  1.000000      <- 3s down
```

Even the deep 3s level is split by 2.5 eV by the exchange field of the 3d electrons. For a metal, the number of states below E_F changes from one k to the next (14 to 18 here). The report does not compare it with the 16 occupied states, and says the `.EIG` "gap" of 3 meV is below the k-mesh resolution.



Fe bands from the file

Figure 3. Fe: SIESTA's up (red) and down (blue) bands, and the eigenvalues of each spin block of `tb-SP.ham` *(circles). Within 5 eV of E_F they agree to 0.12 meV along the path. The down d bands sit about 2 eV above the up ones: the exchange splitting behind the 2.22 μ_B moment.*

6.3 The Fermi level and the zero of H (Case 3). Both `.HSX` files here are version 2: they store E_F, and the matrix elements in the file are referred to it. The on-site energy of B 2s, −9.667 eV, is measured from E_F. With `-o calc.out`, the header also carries E_F = −4.022 eV:

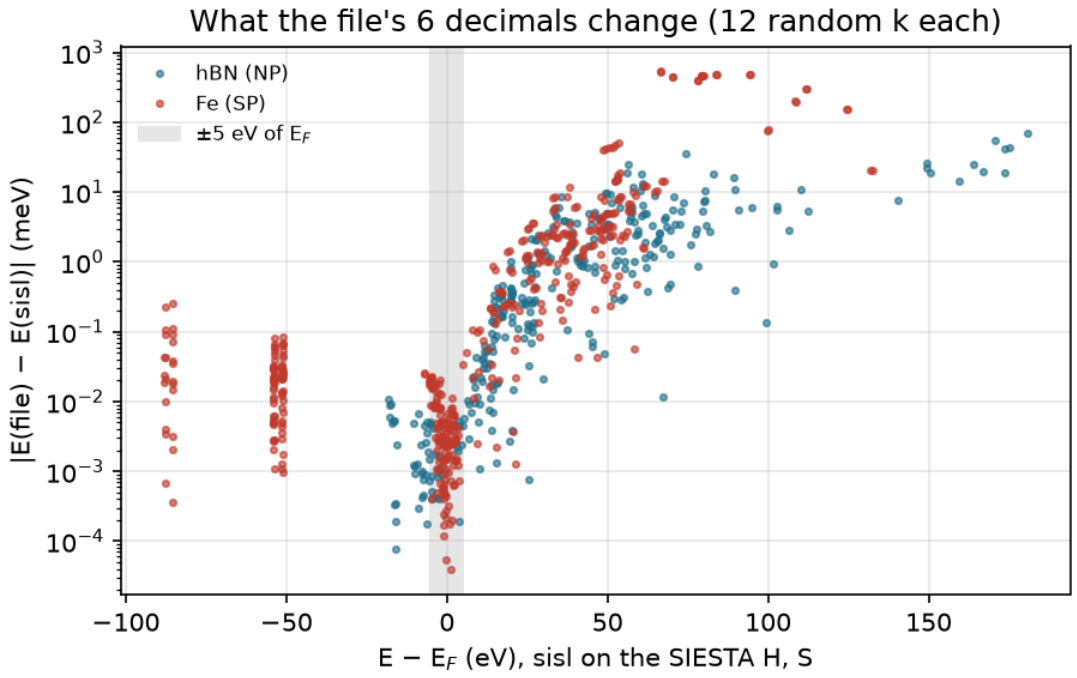
```
Reference         : H in the file is already referred to E_F (0 eV = -4.0222 eV absolute); the header
adds E_F = -4.0222 eV
  2. the matrix elements are already referred to the Fermi level (sisl shifted them by -E_F S, E_F =
  -4.0222 eV from
     hbn.HSX), and the header also carries E_F = -4.0222 eV: a reader that subtracts the header's E_F
     from H moves
     every band by +4.0222 eV (with -f 0 the header would match H).
```

With `-f 0` (`reference_results/hbn_f0/`) the header says `0.000000`, and with neither `-o` nor `-f` (`hbn_nofermi/`) 0.0 is written and the report says it matches H:

```
Reference          : header E_F = 0.0 matches H, which is already referred to E_F
```

For hBN, a shift of +4.02 eV would put the valence-band maximum at K (−1.49 eV from E_F) at +2.53 eV, above zero. A program that counts the states below zero as occupied would then see 2 of them at Γ and 3 at M and K, instead of 4. Which header is right depends on how the reading program uses it, and the report gives what is needed to decide.

6.4 What 6 decimals do (Case 4).



Precision of the file

Figure 4. For 12 random k-points, the change of each eigenvalue between the file and sisl, against its energy. Within 5 eV of E_F (grey) every change is below 0.05 meV. Tens to hundreds of meV appear only 30–150 eV above E_F.

The file has 6 decimals. Rounding `S` matters where `S(k)` is nearly singular (smallest eigenvalue 6.6×10^{-4} in hBN, 2.5×10^{-3} in Fe at Γ). Those directions are combinations of diffuse orbitals that make the highest bands. The bands an exciton calculation uses, a few eV around the gap, are reproduced to 10^{-5}–10^{-4} eV. The report's `READ-BACK CHECK` gives both numbers, and the worst band's energy, for each k.

6.5 What the report catches (Case 5). A run without `SaveHS` (`reference_results/guard_nohsx/`):

```
Hamiltonian        : no hbn.HSX next to the input -- sisl will look for another H source
[FAIL] No Hamiltonian found for 'calc.fdf' (no hbn.HSX): rerun SIESTA with SaveHS true.
```

A log that is not there (`guard_badlog/`):

```
[FAIL] SIESTA output 'siesta.out' (-o) not found.
```

And for hBN's nearly dependent basis, a warning that the 6 decimals perturb the highest bands, with the read-back check measuring by how much.

6.6 The interactive menu (Case 6). `stb-suite` → `3.4`: the `.fdf`, the log and the defaults give the same report as the command line (`reference_results/menu/report.txt`).

9.4.2.8 7. What to take from the example

- A SIESTA Hamiltonian in a localized basis is a tight-binding model as it stands; `stb-siesta2wtb` writes it with its overlap, every neighbouring cell included.
- **Read the read-back verdict**: the bands rebuilt from the file reproduce SIESTA's to ~0.1 meV within 5 eV of E_F (hBN, Fe). The large differences sit only in unphysical bands far above E_F.
- **Read the energy reference**: with a modern `.HSX` the matrix elements are already referred to E_F. Decide whether the header should carry E_F (`-o`) or 0 (`-f 0`) for the program that reads it.
- The file size grows with `nsc` × `dim`^{2}. Diffuse orbitals (bulk metals, small `PAO.EnergyShift`) make files of 100 MB and more; `--dry-run` says how large before writing.

9.4.2.9 8. Reproducing the reference data

```
source ~/.anaconda3/etc/profile.d/conda.sh && conda activate stb   # or any environment with
stb_suite
bash make_reference.sh          # every report into reference_results/<case>/ (~30 s, no SIESTA)
bash siesta_runs/run_all.sh 4   # the two SIESTA runs (SIESTA 5.x + mpirun, ~3 min), pruned to what
is shipped
python3 theory_figures.py       # figures/hbn_bands.png, fe_bands.png, precision.png (~40 s)
dot -Tpng -Gdpi=150 figures/wtb_flowchart.dot -o figures/wtb_flowchart.png
```

The reports' dates and timings are replaced by `<date>` and `<t>`; everything else is deterministic.

9.4.2.10 9. Known, deliberate limitations

- **Spin-orbit and non-collinear Hamiltonians are not part of this example.** The suite's bundled pseudopotentials are scalar-relativistic; a `SOC` run needs fully relativistic ones. The read-back check works the same way for them.
- **The writer is unchanged by the report.** It writes every element of every cell, zeros included, with 6 decimals, and it loops in Python: 12 s for Fe, about a minute for a 182-orbital cell (200 MB).
- **The files go into the current folder** with fixed names (`tb-<S>.ham`, `basis_set-<S>.basis`); a second run in the same folder overwrites them.
- **The check uses four k-points.** A problem confined to other k-points would be missed. The bands of Figures 2 and 3 check whole paths.
- **The basis shells come from the `.HSX`**: polarization orbitals appear as ordinary single-ζ shells, and cutoff radii are not stored there (`r_c` shows `-`).

9.4.2.11 10. Step-by-step: exporting your own Hamiltonian

1. **Run SIESTA with `SaveHS true`** (and the k-grid and basis you want the model to have).

2. **Run `stb-siesta2wtb -i calc.fdf -o calc.out --dry-run`**: check the spin type, the file size, the occupied states.
3. **Decide the header's Fermi level** (section 1.4): `-o calc.out`, or `-f 0` when the reading program subtracts it from an H already referred to E_F.
4. **Write and check**: drop `--dry-run`; read `[6] READ-BACK CHECK`.
5. **In WanTiBEXOS**, use `PARAMS_FILE`, `SYSDIM` and the valence/conduction counts of section `[7]`.

9.4.2.12 11. Files in this folder

File	What it is
`README.md`	this tutorial
`example_3.4.sh`	the guided walk-through (6 cases, about 20 s)
`siesta_runs/`	the two finished SIESTA 5.4.2 runs (`hbn/`, `fe/`: input, log, `.XV`, `.HSX`, `.EIG`, `.KP`, `.bands`) and `run_all.sh`
`make_reference.sh`	regenerates `reference_results/`
`theory_figures.py`	Figures 2, 3, 4 (from the files the tool writes)
`figures/`	every figure of this README; `wtb_flowchart.dot` is the flowchart's source
`reference_results/`	one folder per case: `report.txt`; `tb_head.txt` and `basis_head.txt` (the first lines of each file), `fe/down_block_line.txt`

9.4.2.13 12. Running the script

```
bash example_3.4.sh               # pauses between cases
yes '' | bash example_3.4.sh      # straight through
```

It needs the `stb_suite` package only (no SIESTA, no WanTiBEXOS). It starts by wiping its own `output/`, never touches `siesta_runs/`, and takes about 20 seconds (the Fe file is 84 MB).

9.4.2.14 13. Flag reference

Flag	Default	What it does
`-i`, `--input FDF`	required	the SIESTA input; its `SystemLabel` names the `.HSX`
`-o`, `--output LOG`	none	SIESTA log; the Fermi level written to the header is read from it
`-f`, `--fermi-level E`	none	Fermi level for the header (eV), instead of `-o`
`--dry-run`	off	report only; nothing written
`--no-check`	off	skip the read-back check
`--save-report`	off	also write `stb_wtb_report.txt` (current folder)
`-q`, `--quiet`	off	no banner
`--no-intro`, `-v`		skip the banner; print the version

With neither `-o` nor `-f`, the header's Fermi level is 0.0.

9.4.2.15 14. What's next

- **`2.1` (`stb-bands`)** plots SIESTA's own `.bands`, the reference that Figures 2 and 3 compare with.
- **`5.14` (optical properties)** computes the dielectric function with SIESTA itself (independent particles), a baseline for an excitonic calculation.
- **`3.1` (`stb-translate`)** converts the structure, if WanTiBEXOS or another code needs it in another format.

9.4.2.16 15. References

1. A. C. Dias, J. F. R. V. Silveira, F. Qu, "WanTiBEXOS: a Wannier based Tight Binding code for electronic band structure, excitonic and optoelectronic properties of solids", *Comput. Phys. Commun.* **285**, 108636 (2023).
2. J. M. Soler et al., "The SIESTA method for ab initio order-N materials simulation", *J. Phys.: Condens. Matter* **14**, 2745 (2002).
3. N. Marzari, A. A. Mostofi, J. R. Yates, I. Souza, D. Vanderbilt, "Maximally localized Wannier functions: Theory and applications", *Rev. Mod. Phys.* **84**, 1419 (2012).
4. N. Papior, sisl: v0.16, https://github.com/zerothi/sisl (2024).
5. J. P. Perdew, K. Burke, M. Ernzerhof, "Generalized gradient approximation made simple", *Phys. Rev. Lett.* **77**, 3865 (1996).
6. M. J. van Setten et al., "The PseudoDojo: Training and grading a 85 element optimized norm-conserving pseudopotential table", *Comput. Phys. Commun.* **226**, 39 (2018).

9.4.3 Command-line reference

9.4.3.1 `stb-siesta2wtb`

Convert a SIESTA Hamiltonian (`<label>.HSX`, written with SaveHS true) to the tight-binding files of WanTiBEXOS: `tb-<NP|SP|SOC>.ham` (Fermi level, lattice, then every H and S element between the orbitals of the cell and of each neighbouring cell) and `basis_set-<..>.basis` (one line per orbital: species, position, l, m, spin). The report describes the system, the basis, the Hamiltonian's range and the Fermi level, and checks the written .ham by rebuilding H(k) and S(k) from it and comparing the eigenvalues with sisl's.

Synopsis.

```
usage: stb-siesta2wtb [-h] -i INPUT [-o OUTPUT] [-f FERMI_LEVEL] [--no-check]
                      [--dry-run] [--save-report] [-q] [-v] [--no-intro]
```

Options

`-i, --input INPUT`
Input FDF file from SIESTA calculation; its SystemLabel names the `.HSX`. required

`-o, --output OUTPUT`
SIESTA output file for Fermi energy extraction.

`-f, --fermi-level FERMI_LEVEL`
Manual Fermi energy setting (optional), in eV. type: float

`--no-check`
Skip the read-back check of the written `tb-*.ham` (rebuilding H(k), S(k) from the file and comparing eigenvalues with sisl). switch, off by default

`--dry-run`
Report only: read the Hamiltonian and say what would be written; write nothing. switch, off by default

`--save-report`
Also write the report to `stb_wtb_report.txt` (current folder). Off by default. switch, off by default

`-q, --quiet`
Suppress terminal animations and colors. switch, off by default

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Example usage:
  stb-siesta2wtb -i calc.fdf -o calc.out          # Fermi level from the SIESTA log
  stb-siesta2wtb -i calc.fdf -f -4.02             # Fermi level given by hand (eV)
  stb-siesta2wtb -i calc.fdf -o calc.out --dry-run # report only, write nothing
```

9.5 AIMD Trajectory Converter

At a glance

Menu code: 3.5 **Command:** `stb-ani2traj`
Guided example: `examples/3.5-stb-ani2traj/`
Summary: Convert a SIESTA AIMD `.ANI` trajectory (no cell of its own) into a multi-frame file with the cell per frame – and, in extended XYZ, time and thermodynamics – with a report on the run and a read-back check.

9.5.1 Overview

SIESTA writes MD positions to `<label>.ANI`, a multi-frame XYZ file without the cell. **`stb-ani2traj`** adds the lattice and writes a trajectory that OVITO [21], VMD or XCrySDen can display with periodic boundaries: multi-frame XSF (default), multi-model PDB with `CRYST1`, or extended XYZ with a `Lattice=` tag and, per frame, the MD step, time, temperature, E_{KS}, E_{tot}, volume and pressure (`-of`, Section 3.7). The cell of each frame comes from the log (variable-cell runs are correct frame by frame); without one, a static cell from `<label>.XV` or the input `.fdf` (`-fdf`, found by its `SystemLabel`). The numbered report gives the ensemble and time step, which MD step each frame is (SIESTA writes a frame *after* moving the atoms, checked against `WriteCoorStep` coordinates or the input geometry, and the last frame against the `.XV`), the lattice along the run, the thermodynamics of `<label>.MDE` (temperature against the target and the canonical spread, E_{tot} drift in NVE), the motion (largest step, boundary crossings, displacement per species, closest approach), and a read-back of the written file. `-skip` drops equilibration frames, `-stride` keeps every N-th, `-unwrap` removes boundary jumps, `-dry-run` only reports. For RDF, diffusion and VDOS use **`stb-aimdAnalysis`** (2.18).

```
stb-ani2traj -l water                       # water.ANI -> water_traj.xsf
stb-ani2traj -l si -of xyz --unwrap         # extended XYZ with time and thermodynamics
stb-ani2traj -l run/si -of pdb --skip 100 --stride 5
```

9.5.2 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/3.5-stb-ani2traj/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

A SIESTA molecular-dynamics run writes its frames to `<label>.ANI` (with `WriteMDXmol true`): a plain multi-frame XYZ file, with no cell, no time and no energies. `stb-ani2traj` writes the same frames as a trajectory that viewers show with its periodic box: multi-frame XSF, multi-model PDB, or extended XYZ. The cell of each frame comes from the log, so a variable-cell run is right frame by frame. The extended XYZ also carries each frame's MD step, time, temperature, energies, volume and pressure. The report says:

- what the run was: the ensemble, the time step, how long it ran, whether it finished;
- **which MD step each frame is**: SIESTA writes a frame *after* moving the atoms, checked on every run;
- how the cell changed (variable-cell runs);
- the thermodynamics from `<label>.MDE`: the temperature against the target and the spread a canonical ensemble should have, and the drift of the total energy in NVE;
- how the atoms moved: the largest step between frames, boundary crossings, displacement per species, the closest approach of two atoms;
- a read-back of the written file.

The example uses three real SIESTA 5.4 AIMD runs: 8 water molecules with a Nosé thermostat (NVT), 8-atom Si with velocity Verlet (NVE), and 8-atom Si with Nosé–Parrinello–Rahman (NPT, a variable cell). Everything runs in seconds; no SIESTA is needed.

9.5.2.1 Contents

1. Theory
2. The conversion at a glance
3. Libraries and external dependencies used
4. What `stb-ani2traj` does
5. Running it both ways
6. Worked example
7. What to take from the example
8. Reproducing the reference data
9. Known, deliberate limitations
10. Step-by-step: converting your own run
11. Files in this folder
12. Running the script
13. Flag reference
14. What's next
15. References

9.5.2.2 1. Theory

1.1 What a SIESTA MD run writes. SIESTA [1] integrates Newton's equations for the nuclei on the Born–Oppenheimer surface. At every MD step it converges the electrons, computes the forces and moves the atoms by one time step Δt. The trajectory and its companions:

File	Written with	What it holds
`<label>.ANI`	`WriteMDXmol true`	one XYZ frame per step: species and Cartesian positions (Å); **no cell**
`<label>.MDE`	`WriteMDHistory true`	one row per step: T (K), E_KS (eV), E_tot (eV), V ($Å^3$), P (kBar)
`<label>.MD`	`WriteMDHistory true`	positions, velocities and cell per step (binary)
`<label>.XV`	always	the last geometry with velocities (the restart file)
`<label>.out` (log)	always	the MD settings (`redata:`), and per step the cell (`outcell:`), E_KS, T; the coordinates with `WriteCoorStep true`

Without `WriteMDXmol` there is no `.ANI` at all, and without `WriteMDHistory` no `.MDE`. `stb-ani2traj` stops with a message naming the flag.

1.2 Ensembles.

`MD.TypeOfRun`	Ensemble	What is conserved or controlled
`Verlet`	NVE	the total energy `E_tot = E_KS + E_kin`
`Nose`	NVT	T fluctuates around `MD.TargetTemperature` (Nosé thermostat [2])
`ParrinelloRahman`	NPH	the cell moves to balance `MD.TargetPressure` [3]
`NoseParrinelloRahman`	NPT	both
`Anneal`	—	velocities and/or cell rescaled towards a target

Two checks follow from statistical mechanics:

- **NVE**: `E_tot` must not drift. A drift of more than a few meV/atom/ps means the time step or the SCF tolerance is too large.
- **NVT**: the instantaneous temperature of N atoms in a canonical ensemble fluctuates with a relative spread of

$$\sigma_T / \langle T \rangle = \sqrt{(2 / (3N - 3))} \quad (1)$$

(3N – 3 degrees of freedom, the centre-of-mass motion removed): 0.17 for 24 atoms, 0.31 for 8. A short run, or a thermostat that has not settled, swings more.

1.3 Which MD step is each frame?. At step `s` SIESTA has the geometry `r(s)`. It converges the electrons, prints the cell, E_KS and T of that geometry, moves the atoms to `r(s+1)`, and only then writes a frame to the `.ANI` and the geometry to the `.XV`. So:

```
.ANI frame k  =  r(k+1)  =  the geometry AFTER step k
(2)
```

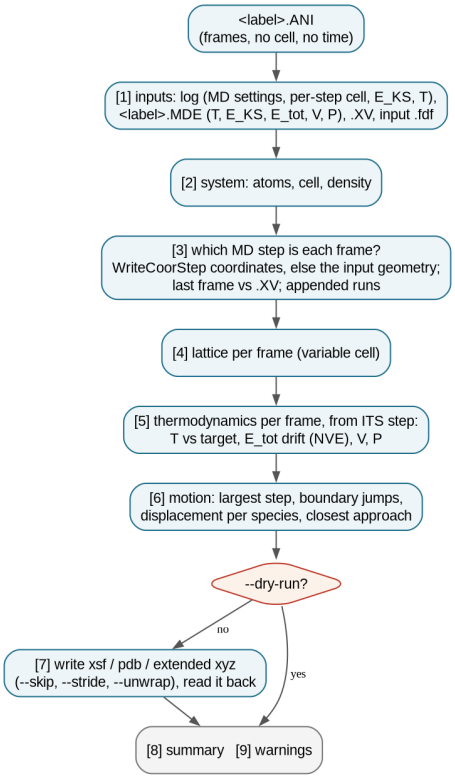
The first frame is not the input geometry, and the last frame is the `.XV`. A frame must therefore get the energy and temperature of step `k+1`, not of step `k`. Pairing frame `k` with step `k` puts every value one step late, which in a fast oscillation (Figure 1(a)) is a visible phase error. `stb-ani2traj` checks Eq. 2 on every run, against the per-step coordinates of the log when `WriteCoorStep true` (agreement to 10^{-6} Å in all three runs here), else against the input geometry, and the last frame against the `.XV`. Time is measured from the input geometry: frame `k` is at `t = k` Δ`t`.

1.4 The cell and the formats. The `.ANI` has no cell. The log prints it at every step (`outcell:`), so each frame can get the cell of its own step. For a fixed cell this is the same cell every time; for a variable cell (NPH, NPT) it is the only correct choice. Without a log, one static cell is taken from the `.XV` or the input `.fdf`, which is wrong for a variable cell, and the report says so.

Format	Cell	Per-frame values	Precision	Read by
`xsf` (AXSF)	per frame (`PRIMVEC`)	—	full	OVITO [4], VMD, XCrySDen
`pdb`	per frame (`CRYST1`)	—	3 decimals (10^{-3} Å)	VMD [5], OVITO
`xyz` (extended)	per frame (`Lattice="..."`)	MD step, time, T, E_KS, E_tot, V, P	full	OVITO, ASE; VMD ignores the cell

1.5 Wrapped and unwrapped positions. An atom that crosses a face of the cell can be written in two ways: continuing outside the cell, or wrapped back to the opposite face. A wrapped trajectory has jumps of a whole cell length, which break an MSD or a movie. `--unwrap` removes them: each frame's displacement from the previous one is taken by the minimum-image convention. SIESTA does **not** wrap the `.ANI` positions: atoms that start on a face vibrate to negative fractional coordinates and stay there (section 6.4). The report counts both the jumps and the positions outside the cell. With no jumps, `--unwrap` changes nothing.

9.5.2.3 2. The conversion at a glance



stb-ani2traj flowchart

Figure 2. What `stb-ani2traj` *does. The numbers are the report's sections.*

9.5.2.4 3. Libraries and external dependencies used

Library	Used for
`sisl` [6]	reading the `.ANI` frames, the `.XV` and the input geometry
`ase` [7]	writing and reading back xsf / pdb / extended xyz, minimum-image distances
`numpy`	cells, displacements, statistics, linear fits
`stb.core`	the SIESTA log parsers (`get_md_trajectory`, `get_mde_trajectory`), `unwrap_trajectory`, the report

9.5.2.5 4. What `stb-ani2traj` does

4.1 Step by step.

1. **Find the run**: `<label>.ANI` from `-l` (which may include a folder), or the only `*.ANI` here; the log (`--log`, `<label>.out` or the only `*.out`), the input `.fdf` (`--fdf`, `<label>.fdf`, or the `.fdf` whose `SystemLabel` is the label), `<label>.MDE` and `<label>.XV`.
2. **Read the MD settings** from the log's `redata:` block (dynamics, time step, temperatures), else the `.fdf`.
3. **Match frames to MD steps** (Eq. 2) and give each frame the cell of its step.
4. **Summarize** the lattice, the thermodynamics and the motion.

5. **Write** the frames kept (`--skip`, then `--stride`; `--unwrap`), and **read them back**.

4.2 The report, section by section.

Section	What it shows
`[0] RUN METADATA`	date, versions, input, output and format, frames kept, unwrap
`[1] INPUTS`	the `.ANI` (frames × atoms), the `.fdf`, the log (version, steps, finished), the `.MDE`, the `.XV`, the dynamics and ensemble, time step, temperatures
`[2] SYSTEM`	atoms, cell and volume of the first frame, density, dimensionality
`[3] FRAMES AND MD STEPS`	frames, log steps, `.MDE` rows, which step each frame is and how it was checked, time span, last frame vs `.XV`, frames kept
`[4] LATTICE`	where the cell comes from; for a variable cell, a, b, c and V along the run
`[5] THERMODYNAMICS`	T, E_KS, E_tot, V, P per frame kept: first, last, mean, std, min, max, drift per ps; T fluctuation vs Eq. 1, T vs target, E_tot drift in NVE
`[6] MOTION`	largest step between frames (and the speed it implies), boundary jumps, positions outside the cell, displacement per species, closest approach
`[7] OUTPUT`	the file, what each frame carries, the read-back check
`[8] SUMMARY & FILES`	the run in one line, the file, the viewer commands
`[9] WARNINGS`	every warning, numbered

4.3 Guard rails. The report warns when:

- the log did not finish;
- the `.ANI` has more frames than the log has steps (SIESTA **appends** to an existing `.ANI` when a run is restarted in the same folder);
- the last frame is not the `.XV` (another run, or a restart);
- a variable-cell run gets a static cell (no log);
- which MD step each frame is cannot be checked;
- an NVE run's total energy drifts by more than 5 meV/atom/ps;
- an atom moves more than 0.3 Å between frames (time step too large, or frames missing), or two atoms come within 0.5 Å (a run that blew up);
- the file read back differs from what was written.

It stops when the `.ANI` is missing (naming `WriteMDXmol`), when no cell can be found, or when `--skip` leaves no frame.

9.5.2.6 5. Running it both ways

From the command line:

```
stb-ani2traj -l water                          # water.ANI -> water_traj.xsf
stb-ani2traj -l si -of xyz                     # extended XYZ: cell, time, T, E_KS, E_tot, V, P per
frame
stb-ani2traj -l run/water -of pdb --skip 100 --stride 5
stb-ani2traj --dry-run                         # the only *.ANI here; report only
```

From the menu (`stb-suite`, code `3.5`): the label (Enter for the only `*.ANI`), the format, the frames to skip, the stride, `--unwrap`, the output name, dry run and report are asked in turn.

9.5.2.7 6. Worked example

`siesta_runs/` holds three SIESTA 5.4.2 AIMD runs [1], PBE [8], DZP basis (`PAO.EnergyShift 0.02 Ry`), PseudoDojo pseudopotentials [9], all with `WriteMDXmol`, `WriteMDHistory` and `WriteCoorStep`:

- **`water_nvt/`** — 8 H_2O molecules in a 6.21 Å cube (1.00 g/cm^3), Γ point, Nosé thermostat at 350 K, Δt = 0.5 fs: 288 steps, 144 fs.
- **`si_nve/`** — 8-atom cubic Si (a = 5.476 Å), 2×2×2 k-points, velocity Verlet from 600 K, Δt = 1 fs: 113 steps.
- **`si_npt/`** — the same cell, Nosé–Parrinello–Rahman at 900 K and 0 GPa, Δt = 1 fs: 111 steps.

The runs were stopped before the number of steps their inputs ask for (300 and 200), to keep the example small. Their logs therefore end mid-step, and every report carries the warning that the log did not finish, as it would for a job killed by the queue. Each folder keeps `calc.fdf`, `calc.out`, `.ANI`, `.MDE` and `.XV` (2.9 MB in all; the pseudopotentials are not shipped, `run_all.sh` copies them from the suite's bundled bank).

6.1 Water, NVT: the whole report (Case 1).

```
stb-ani2traj -l water
```

```
[1] INPUTS
Trajectory        : water.ANI (278.7 KB, 288 frames x 24 atoms)
Input .fdf        : ./calc.fdf
SIESTA log        : ./calc.out (SIESTA 5.4.2), 289 MD steps, NOT finished
Thermodynamics    : water.MDE (288 rows: T, E_KS, E_tot, V, P per step)
Dynamics          : Nose thermostat MD run -> NVT (Nose thermostat)
Time step         : 0.5 fs

[2] SYSTEM
Atoms             : 24 (O8H16)
Cell (frame 1)    : a = 6.2100, b = 6.2100, c = 6.2100 Ang, V = 239.483 Ang^3
Density           : 0.9993 g/cm^3 (0.1002 atoms/Ang^3)

[5] THERMODYNAMICS
      | unit  | first      | last       | mean       | std    | min        | max        | drift /ps
T     | K     | 336.29     | 432.89     | 351.20     | 109.59 | 175.69     | 618.20     | -1.4e+02
E_KS  | eV    | -3847.1587 | -3851.6170 | -3850.4712 | 1.2578 | -3851.6505 | -3846.9362 | -27.19
P     | kBar  | 12.77      | -6.01      | -3.98      | 8.59   | -20.07     | 13.29      | +84
T fluctuation     : 0.312 relative; canonical expectation sqrt(2/(3N-3)) = 0.170 for 24 atoms
T vs target       : mean 351.2 K vs 350 K (+1.2 K)

[6] MOTION
Largest step      : 0.0396 Ang between consecutive frames (H14, frames 18->19) -> up to 7913 m/s
Displacement, first -> last frame kept (unwrapped):
O       | 8     | 0.633     | 0.831
H       | 16    | 0.956     | 1.992
Closest approach  : 0.914 Ang (O22-H24, frame 4)

[7] OUTPUT
File              : water_traj.xsf (458.0 KB, 288 frames, xsf)
Read-back check   : read again with ASE -- ok
```

The `.fdf` is `calc.fdf`, found by its `SystemLabel`. The mean temperature is on target (351 K for 350 K), but in 144 fs it swings twice as much as Eq. 1 allows for 24 atoms (Figure 1(b)): the Nosé thermostat exchanges energy with the atoms in slow oscillations, and the run is far too short to average them out. The hydrogens move most, up to 2 Å, as the molecules rotate. The largest step, 0.04 Å per 0.5 fs, is safe. The closest approach, 0.91 Å, is an O–H bond at its most compressed. The xsf (`reference_results/water/xsf_head.txt`) starts with `ANIMSTEPS 288` and gives the cell of every frame.

6.2 Which MD step is each frame? (Case 2).

```
[3] FRAMES AND MD STEPS
Frames in .ANI    : 288
MD steps in log   : 289 (steps 1-289)
Rows in .MDE      : 288 (steps 1-288)
Frame <-> step    : frame k is the geometry AFTER MD step k (= the geometry of step k+1) -- checked
against the
                    log's per-step coordinates (WriteCoorStep), largest difference 5.0e-07 Ang
                    each frame gets the cell, energies and temperature of ITS step
Time span         : 0.5 to 144 fs (0.1435 ps; t = 0 at the input geometry)
Last frame vs .XV : 4.9e-07 Ang apart (same geometry)
```

The coordinates SIESTA prints at the start of step k+1 match frame k to 5×10^{-7} Å, the precision of the files. The last frame is the `.XV`, and the log has one more step than the `.ANI` has frames: step 289 had started when the run was stopped. The same holds for both Si runs (5×10^{-7} and 10^{-6} Å). Earlier versions of this tool gave frame k the energy and temperature of step k, one step late.

6.3 Si, NPT: a cell that changes (Case 3).

```
stb-ani2traj -l si -of xyz
```

```
[4] LATTICE
Source            : per frame, from the log's 'outcell' block of each frame's MD step
          | first    | last     | mean     | std    | min      | max
a (Ang)   | 5.4760   | 5.5186   | 5.4954   | 0.0136 | 5.4751   | 5.5186
b (Ang)   | 5.4760   | 5.5126   | 5.4927   | 0.0112 | 5.4760   | 5.5126
c (Ang)   | 5.4760   | 5.5043   | 5.4892   | 0.0082 | 5.4760   | 5.5043
V (Ang^3) | 164.2062 | 167.4343 | 165.6733 | 0.9865 | 164.2037 | 167.4343
Volume change     : +1.97 % first -> last, 0.60 % fluctuation

[7] OUTPUT
Per frame         : cell, positions as in the .ANI; E_KS, E_tot, MD_step, Pressure_kBar, Temp_ion,
Time, Volume
```

The first frame's header line (`reference_results/si_npt/xyz_head.txt`):

```
Lattice="5.475974 -1.5e-05 -3e-06 -1.5e-05 5.475999 1.5e-05 -3e-06 1.5e-05 5.476018"
Properties=species:S:1:pos:R:3
E_KS=-919.10391 Temp_ion=894.28 E_tot=-918.29475 Volume=164.206 Pressure_kBar=-0.383 MD_step=2
Time=1.0 pbc="T T T"
```

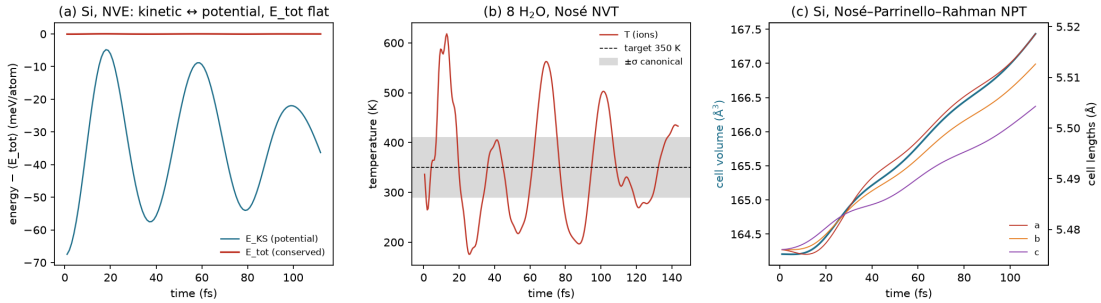
At 900 K the cell expands by 2 % in 110 fs and has not stopped (Figure 1(c)): thermal expansion, with the Parrinello–Rahman cell still on its way to the new volume. The three axes grow differently and the cell picks up small shear components, because 8 atoms are few. The temperature of this run swings from 76 to 2638 K (relative spread 0.71, against 0.31 for 8 atoms): thermostat, barostat and 8 atoms are a small, strongly coupled system. Each frame of the xyz carries its own cell and values, and `MD_step=2` on the first frame is Eq. 2 at work.

6.4 Si, NVE: energy conservation; positions and `--unwrap` (Case 4).

```
T     | K     | 596.19    | 321.20    | 292.56    | 146.05 | 42.97     | 596.19    | -3.7e+02
E_KS  | eV    | -919.1056 | -918.8564 | -918.8305 | 0.1324 | -919.1056 | -918.6042 | +0.3366
E_tot | eV    | -918.5662 | -918.5658 | -918.5658 | 0.0002 | -918.5662 | -918.5654 | +0.0005231
E_tot drift (NVE) : +0.065 meV/atom/ps; fluctuation 0.026 meV/atom

[6] MOTION
Boundary jumps    : 0 coordinate jumps of more than half a cell between consecutive frames
Outside the cell  : 290 atom-frames with a fractional coordinate outside [0, 1)
                    positions are not wrapped into the cell, so atoms move continuously already:
                    --unwrap changes nothing here
```

The run starts from the perfect crystal with velocities for 600 K. Half the kinetic energy turns into potential energy, so the temperature settles near 300 K and the two energies trade places in the vibrations of the cell (Figure 1(a)). `E_tot` stays flat to 0.03 meV/atom, and drifts by 0.065 meV/atom/ps: the 1 fs step and the 10^{-5} density-matrix tolerance conserve energy well. Four of the eight atoms start on cell faces and vibrate to negative fractional coordinates, 290 atom-frames in all, with no jump. That is direct evidence that SIESTA does not wrap the `.ANI` (section 1.5). `--unwrap` (`reference_results/si_nve_unwrap/`) gives the same positions.



Thermodynamics of the three runs

Figure 1. Read back from the extended-XYZ files `stb-ani2traj` *wrote. (a) Si NVE: E_KS oscillates, E_tot is flat. (b) Water NVT: T around the 350 K target; grey is the canonical spread of Eq. 1 for 24 atoms. (c) Si NPT: volume and cell lengths of each frame.*

6.5 Equilibration and stride, PDB (Case 5).

```
stb-ani2traj -l water -of pdb --skip 100 --stride 5
```

```
Kept              : 38 frames (indices 101..286 of 288, every 5)
Read-back check   : read again with ASE -- ok
                    38 frames; positions to 5.0e-04 Ang, cell parameters to 0.0e+00 (the pdb writer's
                    precision)
```

`--skip` drops the first frames, typically the equilibration; `--stride` then keeps every fifth. The thermodynamics of `[5]` and the motion of `[6]` describe only the frames kept. PDB has 3 decimals, so positions read back to 5×10^{-4} Å. That is fine for viewing, but use xsf or xyz for analysis.

6.6 What the report catches (Case 6). The NPT run without its log (`reference_results/guard_nolog/`):

```
Source            : one static cell (log has no usable per-step cell) -- si.XV
  1. a variable-cell run but no per-step cell in the log: every frame gets the same cell, which is
  wrong for this run.
```

The NVE `.ANI` with a copy of itself appended, as SIESTA does when a run is restarted in the same folder (`guard_appended/`):

```
Frames in .ANI    : 226
MD steps in log   : 114 (steps 1-114)
  2. si.ANI has 226 frames but the log only 114 MD steps: SIESTA APPENDS to an existing .ANI, so
  frames of an
     earlier run are probably at the start of the file.
```

A run without `WriteMDXmol` (`guard_missing/`):

```
[FAIL] Input file 'nothing.ANI' not found: SIESTA writes it only with WriteMDXmol true (and
<label>.MDE with
       WriteMDHistory true).
```

6.7 The interactive menu (Case 7). `stb-suite` → `3.5`, Enter for the label (the only `*.ANI`), format 3 (xyz) and the defaults give the same report as the command line (`reference_results/menu/report.txt`).

9.5.2.8 7. What to take from the example

- `.ANI` has no cell; the converted trajectory has the cell of every frame's own MD step, and the extended XYZ also the time and thermodynamics.
- **Frame k is the geometry after step k.** Each frame gets the values of step k+1, and the last frame is the `.XV`.
- **Read `THERMODYNAMICS` before using a run**: E_tot drift in NVE (0.065 meV/atom/ps here), T against the target and the canonical spread in NVT, and whether the volume has settled in NPT (here it had not).
- SIESTA does not wrap positions; `--unwrap` is for trajectories something else wrapped.

9.5.2.9 8. Reproducing the reference data

```
source ~/.anaconda3/etc/profile.d/conda.sh && conda activate stb   # or any environment with
stb_suite
bash make_reference.sh          # every report into reference_results/<case>/ (seconds, no SIESTA)
bash siesta_runs/run_all.sh 4   # the three AIMD runs (SIESTA 5.x + mpirun; the full inputs take ~30
min)
python3 theory_figures.py       # figures/thermo.png
dot -Tpng -Gdpi=150 figures/ani2traj_flowchart.dot -o figures/ani2traj_flowchart.png
```

`run_all.sh` runs the inputs to their last step (300 and 200). The shipped runs were stopped at 288, 113 and 111 steps, so a full rerun gives longer trajectories than the reference reports describe. The reports' dates are replaced by `<date>`; everything else is deterministic.

9.5.2.10 9. Known, deliberate limitations

- **The runs are short** (110–144 fs): long enough to show every section of the report, not for converged averages. The water temperature and the NPT volume have not equilibrated, and the report shows it.
- **Thermodynamics are summaries**: RDF, MSD/diffusion and VDOS are `stb-aimdAnalysis` (`2.18`).
- **Frame ↔ step without `WriteCoorStep`** is inferred from the first frame against the input geometry; restarts that begin from an `.XV` make that comparison meaningless, and the report says what it assumed.
- **An appended `.ANI` is flagged, not split**: cut the earlier frames with `--skip`.
- **pdb has 3 decimals**; xsf/pdb carry no per-frame values.

9.5.2.11 10. Step-by-step: converting your own run

1. **Run SIESTA with `WriteMDXmol true`, `WriteMDHistory true`** (and `WriteCoorStep true` to have the frames checked).
2. **Run `stb-ani2traj -l <label> --dry-run`** in the run folder: check the ensemble, the frame ↔ step line, the thermodynamics and the motion.
3. **Choose the frames**: `--skip` the equilibration, `--stride` for a lighter file.
4. **Choose the format**: xsf for OVITO/VMD/XCrySDen, pdb for VMD, xyz for OVITO with the per-frame values or for analysis in Python.
5. **Write it** (drop `--dry-run`) and open it with the command the report prints.

9.5.2.12 11. Files in this folder

File	What it is
`README.md`	this tutorial
`example_3.5.sh`	the guided walk-through (7 cases, seconds)
`siesta_runs/`	the three SIESTA 5.4.2 AIMD runs (`water_nvt/`, `si_nve/`, `si_npt/`: input, log, `.ANI`, `.MDE`, `.XV`) and `run_all.sh`
`make_reference.sh`	regenerates `reference_results/`
`theory_figures.py`	Figure 1 (from the extended XYZ files the tool writes)
`figures/`	every figure of this README; `ani2traj_flowchart.dot` is the flowchart's source
`reference_results/`	one folder per case: `report.txt`; `xsf_head.txt`, `xyz_head.txt`, `pdb_head.txt` (the first lines of each format)

9.5.2.13 12. Running the script

```
bash example_3.5.sh               # pauses between cases
yes '' | bash example_3.5.sh      # straight through
```

It needs the `stb_suite` package only (no SIESTA). It starts by wiping its own `output/`, never touches `siesta_runs/`, and takes a few seconds.

9.5.2.14 13. Flag reference

Flag	Default	What it does
`-l, --label LABEL`	the only `*.ANI` here	SystemLabel of the run; may include a folder
`-of, --out-format`	`xsf`	`xsf`, `pdb` or `xyz` (extended, with per-frame values)
`-o, --output FILE`	`<label>_traj.<ext>`	output file, next to the `.ANI` by default
`--skip N`	0	drop the first N frames (before `--stride`)
`--stride N`	1	keep every Nth frame
`--unwrap`	off	remove boundary jumps (minimum-image displacements)
`--fdf FILE`	`<label>.fdf`, or the `.fdf` with that SystemLabel	input `.fdf` (MD settings, fallback cell)
`--log FILE`	`<label>.out`, or the only `*.out`	SIESTA log (per-step cell, MD settings)
`--dry-run`	off	report only
`--save-report`	off	also write `stb_ani2traj_report.txt` next to the `.ANI`
`--no-intro, -v`		skip the banner; print the version

9.5.2.15 14. What's next

- **`2.18` (`stb-aimdAnalysis`)** computes the RDF, MSD/diffusion coefficient and VDOS of the same trajectory.
- **`7.2` (`stb-mlmd`)** runs MD with a MACE potential; its trajectories use the same three formats.
- **`3.6` (`stb-status`)** summarizes the state of many run folders, finished or not.

9.5.2.16 15. References

1. J. M. Soler et al., "The SIESTA method for ab initio order-N materials simulation", *J. Phys.: Condens. Matter* **14**, 2745 (2002).
2. S. Nosé, "A unified formulation of the constant temperature molecular dynamics methods", *J. Chem. Phys.* **81**, 511 (1984).
3. M. Parrinello, A. Rahman, "Polymorphic transitions in single crystals: A new molecular dynamics method", *J. Appl. Phys.* **52**, 7182 (1981).
4. A. Stukowski, "Visualization and analysis of atomistic simulation data with OVITO–the Open Visualization Tool", *Modelling Simul. Mater. Sci. Eng.* **18**, 015012 (2010).
5. W. Humphrey, A. Dalke, K. Schulten, "VMD: Visual molecular dynamics", *J. Mol. Graph.* **14**, 33 (1996).
6. N. Papior, sisl: v0.16, https://github.com/zerothi/sisl (2024).
7. A. H. Larsen et al., "The atomic simulation environment — a Python library for working with atoms", *J. Phys.: Condens. Matter* **29**, 273002 (2017).
8. J. P. Perdew, K. Burke, M. Ernzerhof, "Generalized gradient approximation made simple", *Phys. Rev. Lett.* **77**, 3865 (1996).
9. M. J. van Setten et al., "The PseudoDojo: Training and grading a 85 element optimized norm-conserving pseudopotential table", *Comput. Phys. Commun.* **226**, 39 (2018).

9.5.3 Command-line reference

9.5.3.1 `stb-ani2traj`

Convert a SIESTA AIMD trajectory (`<label>.ANI`) into a multi-frame trajectory that also carries the lattice – for OVITO, VMD, XCrySDen. .ANI is a bare multi-frame XYZ with no cell; the cell is read per frame from the SIESTA log (correct for variable-cell/NPT runs), else a single one from `<label>.XV` or the input .fdf. The report says what the run was (ensemble, time step, how long), which MD step each frame is, how the cell, temperature and energy behaved (`<label>.MDE`), how the atoms moved (largest step, boundary crossings, closest approach), and reads the written trajectory back.

Synopsis.

```
usage: stb-ani2traj [-h] [-l LABEL] [-of {pdb,xsf,xyz}] [-o OUTPUT]
                    [--skip SKIP] [--stride STRIDE] [--unwrap] [--fdf FDF]
                    [--log LOG] [--dry-run] [--save-report] [--no-intro] [-v]
```

Options

`-l, --label LABEL`
: SystemLabel of the run; may include a folder (run/si). Default: the only *.ANI here.

`-of, --out-format {pdb,xsf,xyz}`
: Output trajectory format: xsf (multi-frame AXSF; OVITO, VMD's xsf plugin, XCrySDen), pdb (multi-model PDB with CRYST1, 3 decimals; VMD), xyz (extended XYZ with a Lattice tag and per-frame E_KS, Temp_ion, E_tot, Volume, Pressure_kBar, MD_step, Time; OVITO – VMD ignores the Lattice tag).default: xsf; choices: pdb, xsf, xyz

`-o, --output OUTPUT`
: Output filename. Default: `<label>_traj.<ext>`, next to the .ANI.

`--skip SKIP`
: Drop the first N frames (equilibration) before `--stride`. type: int; default: 0

`--stride STRIDE`
: Keep only every Nth frame. type: int; default: 1

`--unwrap`
: Remove periodic-boundary jumps: each frame's positions are corrected by the minimum-image displacement relative to the previous frame, so atoms that cross the box edge move continuously. switch, off by default

`--fdf FDF`
: Input .fdf of the run (MD settings, fallback cell). Default: `<label>.fdf`, else the only .fdf here whose SystemLabel is <label>.

`--log LOG`
: SIESTA output log. Default: `<label>.out`, else the only *.out next to the .ANI.

`--dry-run`
: Report only; write no trajectory. switch, off by default

`--save-report`
: Also write the report to `stb_ani2traj_report.txt` (next to the .ANI). Off by default. switch, off by default

Like every STB command it also accepts `-h`/`--help`, `-v`/`--version` and `--no-intro` (Section 3.3).

From the built-in help.

```
Example usage:
  stb-ani2traj -l water                        # water.ANI -> water_traj.xsf
  stb-ani2traj -l si -of xyz --unwrap          # extended XYZ with E_KS, T, E_tot, P, V, time
  stb-ani2traj -l run/si -of pdb --skip 100 --stride 5
  stb-ani2traj --dry-run                       # the only *.ANI here; report only
```

9.6 Status Checker

At a glance

Menu code: 3.6 **Command:** `stb-status`
Guided example: `examples/3.6-stb-status/`
Summary: The state of SIESTA calculation folders (finished, failed with SIESTA's reason, running, stopped), their progress against the tolerances, SCF, energies, gap and forces, and the suite tools that can use their files; several folders as one table with relative energies.

9.6.1 Overview

`stb-status` reads what SIESTA leaves in a folder – the log, `MESSAGES`, `CLOCK`, `<label>.EIG`, `<label>.XV`, `<label>.MDE` – and reports the state of the run (FINISHED; FAILED with SIESTA's own reason from `MESSAGES`; RUNNING? when an unfinished log was written in the last two minutes; STOPPED otherwise), the run (version, processes, start, end, wall time), the system (atoms, cell, electrons, spin, XC, k-points, mesh cutoff), the progress (for a relaxation, SIESTA's own verdict – `Relaxed` or `Final (unrelaxed)` coordinates – and the last force and stress against `MD.MaxForceTol` and `MD.MaxStressTol`, with a table of every move; for MD, steps against those requested and the `.MDE` temperature), the SCF (iterations per step and the tolerance SIESTA actually used, from the log), the results (energies, Fermi level, band gap, spin moment, forces, pressure) and the output files with the suite tool that reads each. `-path` takes a folder or a glob; `-recursive` takes every run folder below it. Several folders give one table with each energy relative to the lowest one of the same composition and level of theory (XC, spin, mesh cutoff, basis); `-csv` saves it, `-details` adds the full report of each folder.

```
stb-status --path relax                      # one folder, the full report
stb-status --path 'strain/a_*'               # a series: one table, relative energies
stb-status --path project --recursive --csv status.csv
```

9.6.2 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/3.6-stb-status/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

After a batch of SIESTA jobs the questions are always the same: which runs finished, which failed and why, which are still running, did the SCF and the relaxation converge, and what came out. `stb-status` answers them from the files SIESTA leaves in a folder (the log, `MESSAGES`, `CLOCK`, the `.EIG`, the `.XV`, ...). For one folder it gives a full report:

- the **state**: finished, failed (with SIESTA's own reason), probably still running, or stopped;
- the run: SIESTA version, processes, start and end, wall time;

- the system: atoms, cell, electrons, spin, XC functional, k-points, mesh cutoff;
- the **progress**: steps done against those requested. For a relaxation it also gives SIESTA's own verdict, the last force and stress against their tolerances, and a table of every move;
- the SCF: iterations per step, the tolerance SIESTA actually used, failures;
- the results: energies, Fermi level, band gap, spin moment, forces, pressure;
- the output files present, each with **the suite tool that reads it**.

For several folders (a glob, or `--recursive`) it gives one table, with each energy relative to the lowest one of the same composition and level of theory. `--csv` saves that table.

The example is a small "project" of real SIESTA 5.4 runs, each finished in seconds. It covers a single point with bands, a relaxation that converges and one that runs out of moves, an SCF that fails, an MD run, a spin-polarized metal, and a series of three lattice constants. Everything runs in seconds; no SIESTA is needed.

9.6.2.1 Contents

1. Theory
2. The diagnosis at a glance
3. Libraries and external dependencies used
4. What `stb-status` does
5. Running it both ways
6. Worked example
7. What to take from the example
8. Reproducing the reference data
9. Known, deliberate limitations
10. Step-by-step: checking your own runs
11. Files in this folder
12. Running the script
13. Flag reference
14. What's next
15. References

9.6.2.2 1. Theory

1.1 What SIESTA leaves behind, and what each file says.

File	What `stb-status` reads from it
the log (`<label>.out` or the job's own name, e.g. `calc.out`)	version, `Running on N nodes`, `Start of run` / `End of run`, the `redata:` settings (spin, SCF limits and tolerance, dynamics, tolerances, number of moves), k-points, mesh, every geometry step (SCF iterations, `SCF cycle converged`, E_KS, forces, stress, pressure), the final-energy block, `Relaxed` / `Final (unrelaxed)` coordinates, warnings
`MESSAGES`	SIESTA 5's own summary: `INFO: Job completed`, or `FATAL: <reason>`

File	What stb-status reads from it
CLOCK	wall time, and an End of run line when the job ended
<label>.EIG	eigenvalues on the SCF k-mesh: the band gap
<label>.XV	the last geometry: atoms and cell
<label>.MDE	for MD: temperature and energies per step

The log's name is found as <label>.out, or the only *.out in the folder. The label comes from the folder's .fdf (SystemLabel, or --label).

1.2 The state of a run.

```
log?  no  -> NO LOG
      yes -> FATAL line in MESSAGES?        yes -> FAILED   (SIESTA's reason, e.g. SCF_NOT_CONV)
             'End of run' in log or CLOCK?   yes -> FINISHED
             log written in the last 2 min?  yes -> RUNNING?
                                          no  -> STOPPED  (killed, out of queue time, crashed silently)
```

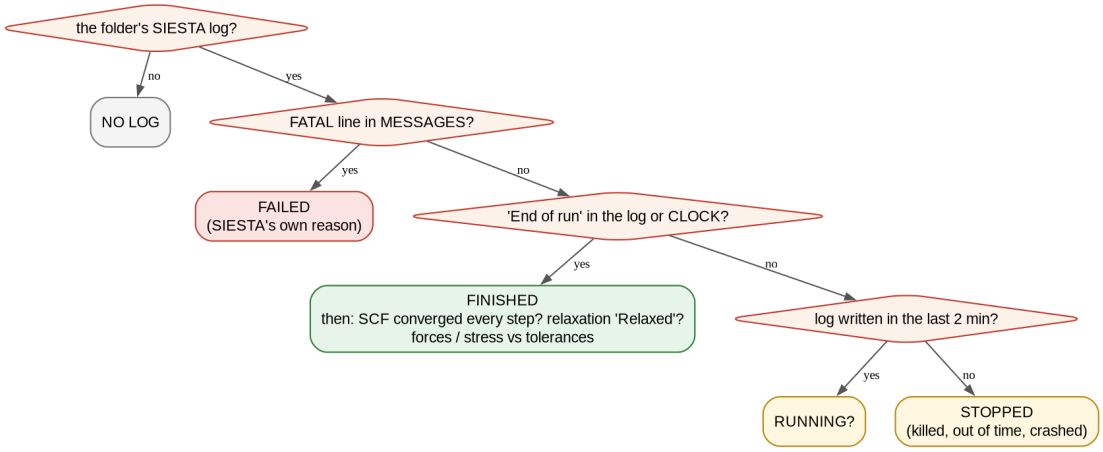
RUNNING? has a question mark because the tool cannot see the batch queue: it only knows the log is growing. A FINISHED run still needs two more checks before its result is usable:

- **SCF**: every geometry step should say SCF cycle converged after N iterations. When it does not and SCF.MustConverge is on (the default), SIESTA aborts, which gives a FAILED run. When it is off, the run goes on with an unconverged density, and the report flags every such step.
- **Relaxation**: SIESTA ends a relaxation with outcoor: Relaxed atomic coordinates when forces and stress are below MD.MaxForceTol/MD.MaxStressTol. It ends with outcoor: Final (unrelaxed) when it used up MD.NumCGsteps. The job ends normally in both cases. The report gives SIESTA's verdict, the last force (the largest atomic force) and stress (the largest component of the stress tensor), each against its tolerance.

1.3 The values SIESTA actually used. Every setting is reported as the log's redata: block gives it, not as the .fdf says. SIESTA reads the **first** occurrence of a label in the .fdf, so a value appended at the end of a file that already sets it is silently ignored. The failed run of this example has that trap (section 6.4).

1.4 Comparing runs: relative energies. Total energies of different runs can be compared only at the same level of theory and composition. In a table of several folders, dE is the energy per atom above the lowest folder with the same element fractions **and** the same XC functional, spin treatment, mesh cutoff and basis (PAO.BasisSize, PAO.EnergyShift). A different setting starts a new group with its own zero. MD runs are left out, because their energy is that of a single instant.

9.6.2.3 2. The diagnosis at a glance



stb-status flowchart

Figure 1. How the state of a folder is decided. A FINISHED run is then checked for SCF convergence in every step and, for a relaxation, for SIESTA's own verdict and the tolerances.

9.6.2.4 3. Libraries and external dependencies used

Library	Used for
Python standard library	reading the logs, MESSAGES and CLOCK, walking folders, CSV
`numpy`	statistics, forces, cells
`sisl` [1]	reading the `.XV`
`stb.core`	`siesta_log` (`find_out_file`, `read_eig_gap`, `get_spin_moment`, `get_mde_trajectory`), `structure_io`, the report

It runs on a cluster's login node next to the runs: no SIESTA, no GPU, a few seconds per hundred folders.

9.6.2.5 4. What `stb-status` does

4.1 Step by step.

1. **Find the folders**: `--path` (a folder or a glob), or every non-hidden folder below it with a `.out` or `.fdf` (`--recursive`).
2. **For each**: find the label, the log and the input; read the log in one pass, then `MESSAGES`, `CLOCK`, `.EIG`, `.XV`.
3. **Decide the state** (section 1.2) and collect the warnings.
4. **Print** the full report for one folder, or one table for several (`--details` adds the full report of each).

4.2 The report, section by section. One folder:

Section	What it shows
`[0] RUN METADATA`	date, version, target
`[1] RUN`	folder, label, input, log, **state** and reason, processes, start, end, wall time

Section	What it shows
[2] SYSTEM	atoms, last cell, electrons, spin, XC, k-points, mesh cutoff required and used
[3] PROGRESS	run type; relaxation: moves vs the maximum, variable cell, SIESTA's verdict, force and stress vs tolerances; MD: steps vs requested, T and E_tot from the .MDE; a table of every step
[4] SCF	iterations in total and per step, converged steps, the tolerance and limit SIESTA used
[5] RESULTS	total and band-structure energies, Fermi level, energy per atom, band gap, spin moment, forces (max, net), pressure, dipole, log warnings
[6] FILES & NEXT STEPS	folder size; each analysis output and the suite tool that reads it
[7] SUMMARY & FILES	the verdict
[8] WARNINGS	every warning, numbered

Several folders: [1] FOLDERS replaces sections 1–6. It holds one row per folder: state, type (with the relaxation verdict), steps, SCF, energy, dE, last max force, last pressure, gap and wall time, plus the counts of each state.

4.3 Guard rails. The report warns when a run failed (with SIESTA's reason), is probably still running or stopped, has unconverged SCF steps, ended a relaxation unrelaxed, has a net force above 0.05 eV/Å (mesh too coarse or SCF unconverged), or has no log at all.

9.6.2.6 5. Running it both ways

From the command line:

```
stb-status                                  # this folder, the full report
stb-status --path relax                     # another folder
stb-status --path 'strain/a_*'              # several folders: one table, relative energies
stb-status --path project --recursive       # every run folder below project/
stb-status --path project --recursive --csv status.csv --details
```

From the menu (stb-suite, code 3.6): the folder or glob, whether to go recursive, a label, the CSV file and the report are asked in turn.

9.6.2.7 6. Worked example

project/ holds nine SIESTA 5.4.2 runs [2] (PBE [3], DZP basis, PseudoDojo pseudopotentials [4]), all made by run_project.sh in under two minutes on 4 processes:

Folder	What it is
bulk_sp/	Si, 2-atom fcc cell, a = 5.476 Å, single point with bands and the density
relax/	Si from a = 5.55 Å, variable-cell CG relaxation (MD.MaxForceTol 0.01 eV/Ang, MD.MaxStressTol 0.1 GPa)
relax_short/	the same relaxation with MD.NumCGsteps 3
scf_fail/	Si with MaxSCFIterations 3: the SCF cannot converge
md/	Si, 8 atoms, velocity Verlet from 600 K, 20 steps
fe_spin/	bcc Fe, ferromagnetic

Folder	What it is
`strain/a_5.40/`, `a_5.476/`, `a_5.55/`	Si single points at three lattice constants

Each folder keeps its input, log, `MESSAGES`, `CLOCK` and small outputs (`.XV`, `.EIG`, `.bands`, ...); 1.6 MB in all, and the pseudopotentials are not shipped. The guided script `example_3.6.sh` runs every case on a copy in `output/`; the reference reports are in `reference_results/`.

6.1 A converged relaxation: the whole report (Case 1).

```
stb-status --path project/relax
```

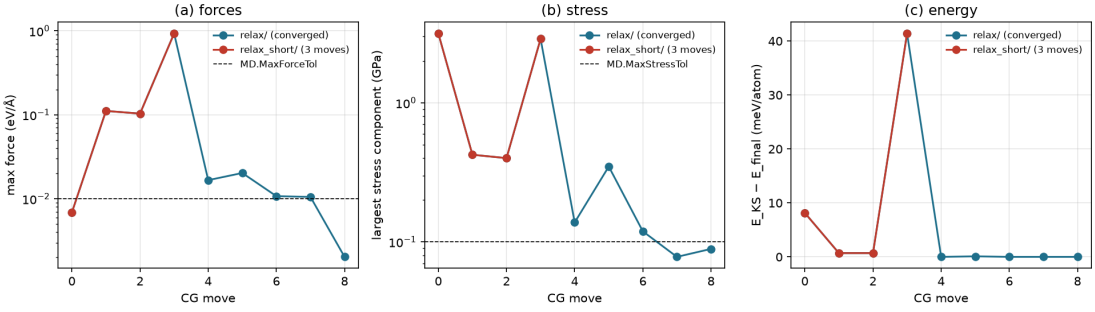
```
[1] RUN
State              : FINISHED
Processes          : 4 MPI
Started            : 30-OCT-2026  20:42:03
Ended              : 30-OCT-2026  20:42:15
Wall time          : 11.8 s (CLOCK)

[3] PROGRESS
Run type           : relaxation (CG coord. optimization)
Moves              : 8 of at most 30 (plus the input geometry)
Variable cell      : yes
Relaxation         : converged ('Relaxed atomic coordinates')
Max force (last)   : 0.0020 eV/Ang vs tolerance 0.01 (ok)
Max stress (last)  : 0.0890 GPa (largest component) vs tolerance 0.1 GPa (ok)
step | SCF its | E_KS (eV) | max F (eV/Ang) | P (kBar) | net F
0    | 8       | -230.0192 | 0.0068         | -31.51   | 0.0000
1    | 8       | -230.0341 | 0.1112         | -0.89    | 0.0001
...
8    | 4       | -230.0355 | 0.0020         | -0.89    | 0.0000

[4] SCF
Iterations         : 48 in 9 step(s); per step min 4, mean 5.3, max 8
Tolerance          : dDmax < 1e-05, at most 1000 iterations (failure aborts the run)

[5] RESULTS
Total energy       : -230.035507 eV
Fermi level        : -4.180341 eV
Band gap (.EIG)    : 0.9406 eV (indirect; VBM -4.6468, CBM -3.7062 eV; 108 k-points)
```

The input cell (a = 5.55 Å) starts under −31.5 kBar of tension. After 8 moves the cell has relaxed (a = $3.8742 \cdot \sqrt{2}$ = 5.479 Å) with both tolerances met, and SIESTA wrote `Relaxed atomic coordinates`. The first SCF needs 8 iterations; the later ones, starting from the previous density, need 4. The net force stays below 2×10^{-4} eV/Å: the mesh is fine enough.



Relaxation progress

Figure 2. The two relaxations move by move, read with `stb-status`*'s own log parser: (a) max force, (b) largest stress component, both against their tolerances (dashed), (c) energy above the final one. The CG line search overshoots at move 3;* `relax/` *recovers at move 4,* `relax_short/` *stops there.*

6.2 A single point: results and what to do next (Case 2).

```
[5] RESULTS
Total energy      : -230.035330 eV
Fermi level       : -4.177431 eV
Band gap (.EIG)   : 0.9355 eV (indirect; VBM -4.6394, CBM -3.7039 eV; 108 k-points)
Max force         : 0.006896 eV/Ang (net 0.000010)
Pressure          : 2.0864 kBar (0.2086 GPa)

[6] FILES & NEXT STEPS
output           | files                | read by
band structure   | si.bands             | stb-bands (2.1)
eigenvalues      | si.EIG               | band gap here; stb-dos (2.2)
real-space grids | si.RHO               | stb-cube (3.3), stb-bader (2.6), stb-density (2.8)
final geometry   | si.STRUCT_OUT, si.XV | stb-translate (3.1)
```

The gap from the SCF k-mesh is 0.94 eV, indirect (PBE puts silicon's gap at ~0.6 eV with converged plane waves; the DZP basis and the 6×6×6 mesh, which misses the conduction-band minimum near X, both raise it). The last table lists what the folder can be used for next.

6.3 A relaxation that ran out of moves (Case 3).

```
Moves             : 3 of at most 3 (plus the input geometry)
Relaxation        : NOT converged ('Final (unrelaxed)')
Max force (last)  : 0.9318 eV/Ang vs tolerance 0.01 (above)
Max stress (last) : 2.8790 GPa (largest component) vs tolerance 0.1 GPa (above)
Verdict           : FINISHED -- but see the warnings
  1. relaxation NOT converged: SIESTA wrote 'Final (unrelaxed)' coordinates after 3 of 3 moves --
  raise
     MD.NumCGsteps or restart from the .XV.
```

The job ended normally, with `MESSAGES` saying `Job completed`, so a check of the exit status alone would call it a success. But the structure is not relaxed. Worse, the third move was a line-search overshoot (Figure 2), so the run stopped 41 meV/atom above the minimum with forces of 0.93 eV/Å, *further* from the answer than after move 1.

6.4 A failed SCF (Case 4).

```
State             : FAILED
                    SCF_NOT_CONV: SCF did not converge in maximum number of steps (required).
[4] SCF
Iterations        : 3 in 1 step(s); per step min 3, mean 3.0, max 3
Converged steps   : 0 of 1
Tolerance         : dDmax < 1e-05, at most 3 iterations (failure aborts the run)
```

The reason is SIESTA's own, from `MESSAGES`. The tolerance line holds a lesson. This `calc.fdf` sets `SCF.DM.Tolerance 1.0d-5` and, at its end, `SCF.DM.Tolerance 1.0d-8`. SIESTA keeps the first occurrence, so the run used 10^{-5}; the report shows the value from the log, not the one at the end of the input. The run failed on `MaxSCFIterations 3` alone.

6.5 Running, or stopped? (Case 5). The script copies `relax/` and cuts its log after move 4, with no `End of run` anywhere. Written seconds ago:

```
State             : RUNNING?
                    log written 1 s ago, no end of run yet
E_KS (last step)  : -229.952700 eV (no final-energy block: the run did not finish)
```

The same folder, with the log's date set three hours back:

```
State             : STOPPED
                    no end of run, log last written 3.00 h ago -- killed, out of time, or crashed
```

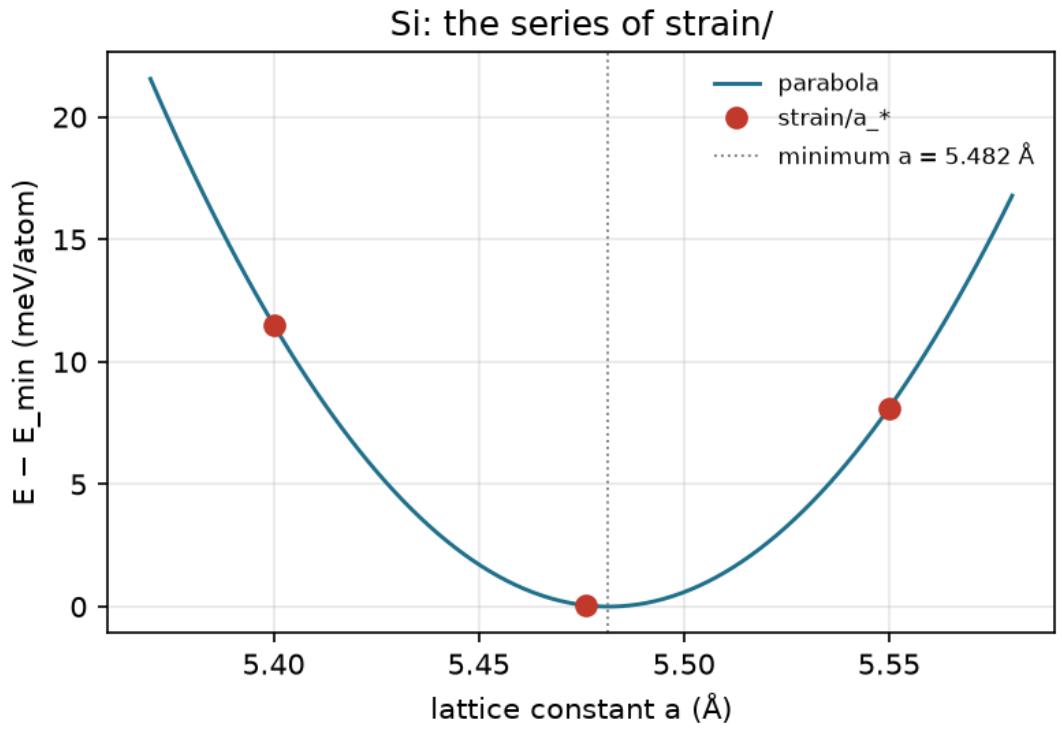
A job killed by the queue's time limit leaves exactly this: a log that stops mid-run and no `FATAL` line.

6.6 A series: relative energies (Case 6).

```
stb-status --path 'project/strain/a_*'
```

```
folder  | state    | type         | steps | SCF | E (eV)    | dE (meV/atom) | max F | P (kBar) | gap
(eV) | wall
a_5.40  | FINISHED | single point | 1     | ok  | -230.0124 | 11.5          | 0.007 | 43.4     | 0.864
| 2.0 s
a_5.476 | FINISHED | single point | 1     | ok  | -230.0353 | 0.0           | 0.007 | 2.1      | 0.935
| 2.0 s
a_5.55  | FINISHED | single point | 1     | ok  | -230.0192 | 8.1           | 0.007 | -31.5    | 1.002
| 1.8 s
```

The middle lattice constant is the lowest, and the pressure changes sign between it and 5.55 Å. A parabola through the three energies puts the minimum at a = 5.482 Å (Figure 3), and the relaxation of section 6.1 at 5.479 Å. The gap opens with the lattice constant, as silicon's indirect gap does (it closes under pressure).



E(a) of the series

Figure 3. The three single points of `strain/` *(energy per atom) and the parabola through them.*

6.7 The whole project (Case 7).

```
stb-status --path project --recursive --csv project_status.csv
```

```
folder         | state    | type                     | steps | SCF | E (eV)     | dE (meV/atom) | max F
| P (kBar) | gap (eV) | wall
bulk_sp        | FINISHED | single point             | 1     | ok  | -230.0353  | 0.1           | 0.007
| 2.1      | 0.935    | 2.0 s
fe_spin        | FINISHED | single point             | 1     | ok  | -3408.6740 | 0.0           | 0.000
| -61.5    | metal    | 32.3 s
md             | FINISHED | MD                       | 20    | ok  | -918.5653  | -             | 2.289
| 2.3      | 1.800    | 34.4 s
relax          | FINISHED | relaxation (relaxed)     | 8     | ok  | -230.0355  | 0.0           |
0.002 | -0.9     | 0.941    | 11.8 s
relax_short    | FINISHED | relaxation (not relaxed) | 3     | ok  | -229.9527  | 41.4          |
0.932 | 27.4     | 0.792    | 6.1 s
scf_fail       | FAILED   | single point             | 1     | NO  | -          | -             | -
| -        | -        | -
strain/a_5.40  | FINISHED | single point             | 1     | ok  | -230.0124  | 11.6          |
0.007 | 43.4     | 0.864    | 2.0 s
...
States            : 1 FAILED, 8 FINISHED
```

Every Si run with the same settings shares one energy zero, the relaxed structure. `fe_spin` is its own group (another element, spin-polarized, another mesh), and so would be a run with another basis. The MD row has no `dE`. Its "max F" of 2.3 eV/Å is that of a hot instant, not a sign of a problem. The failed SCF is red and the unrelaxed relaxation yellow. `project_status.csv` (`reference_results/project/`) holds the same table.

6.8 The interactive menu (Case 8). `stb-suite` → `3.6`, folder `project`, recursive yes, gives the same table (`reference_results/menu/report.txt`).

9.6.2.8 7. What to take from the example

- **FINISHED is not usable.** A relaxation can end normally and unrelaxed, here 41 meV/atom from the minimum; read the relaxation verdict and the SCF line.
- **A failure carries SIESTA's reason** (`MESSAGES`); an unfinished log is `RUNNING?` or `STOPPED` by its age.
- **The report shows what SIESTA used**, from the log; a duplicated fdf label is a silent trap.
- **Compare energies only within a group**: same composition, XC, spin, mesh, basis; the table does that for you.

9.6.2.9 8. Reproducing the reference data

```
source ~/.anaconda3/etc/profile.d/conda.sh && conda activate stb   # or any environment with
stb_suite
bash make_reference.sh          # every report into reference_results/<case>/ (seconds, no SIESTA)
```

```
bash run_project.sh 4           # project/ itself (SIESTA 5.x + mpirun, ~2 min)
python3 theory_figures.py       # figures/relax_progress.png, strain_energy.png
dot -Tpng -Gdpi=150 figures/status_flowchart.dot -o figures/status_flowchart.png
```

Dates and the age of a log in the reports are replaced by <date>/<age>; the rest is deterministic.

9.6.2.10 9. Known, deliberate limitations

- **RUNNING? is a guess** from the age of the log (2 minutes): the tool does not see the batch queue. A job that writes rarely can look stopped.
- **One log per folder**: <label>.out or the only *.out. A folder with several .out files and none named after the label is reported as having no log; rename it or pass --label.
- **Settings come from the log's redata: block**: a log cut before it (a run that died while reading its input) has none.
- **Relative energies trust the grouping** (composition, XC, spin, mesh, basis): k-points, pseudopotentials and other settings are not compared. Keep a series' other settings equal.
- **No restart files are judged**: whether a .DM or .XV still belongs to the input is stb-clean's question (3.2).

9.6.2.11 10. Step-by-step: checking your own runs

1. **One run**: stb-status --path <folder>. Read State, then PROGRESS and SCF.
2. **A batch**: stb-status --path '<pattern>' or --path <root> --recursive. Read the colours and States.
3. **Failed or stopped runs**: the reason line, then the log near its end; restart from the .XV/.DM (stb-clean --keep ... .DM .XV).
4. **A series**: read dE and P across it; save it with --csv.

9.6.2.12 11. Files in this folder

File	What it is
README.md	this tutorial
example_3.6.sh	the guided walk-through (8 cases, seconds)
project/	the nine SIESTA 5.4.2 runs (inputs, logs, MESSAGES, CLOCK, small outputs)
run_project.sh	builds and runs project/ (needs SIESTA)
make_reference.sh	regenerates reference_results/
theory_figures.py	Figures 2 and 3
figures/	every figure of this README; status_flowchart.dot is the flowchart's source
reference_results/	one folder per case: report.txt; project/project_status.csv

9.6.2.13 12. Running the script

```
bash example_3.6.sh               # pauses between cases
yes '' | bash example_3.6.sh      # straight through
```

It needs the stb_suite package only (no SIESTA). It starts by wiping its own output/, never touches project/, and takes a few seconds.

9.6.2.14 13. Flag reference

Flag	Default	What it does
`--path PATH`	`.`	a folder, or a quoted glob of folders
`--recursive`	off	every non-hidden folder below `--path` with a `.out` or `.fdf`
`-l, --label LABEL`	from each folder's `.fdf`	SystemLabel override for every folder
`--details`	off	with several folders, also the full report of each
`--csv FILE`	none	also write the summary table as CSV
`--save-report`	off	also write `stb_status_report.txt` (current folder)
`--no-intro, -v`		skip the banner; print the version

9.6.2.15 14. What's next

- **`3.2` (`stb-clean`)** resets a folder for a rerun, keeping `.DM`/`.XV` to continue a stopped one.
- **`3.7` (`stb-archive`)** packages a finished run with its essential outputs.
- **`5.15` (EOS)** turns a series like `strain/` into V0, B0 and B0′ properly.

9.6.2.16 15. References

1. N. Papior, sisl: v0.16, https://github.com/zerothi/sisl (2024).
2. J. M. Soler et al., "The SIESTA method for ab initio order-N materials simulation", *J. Phys.: Condens. Matter* **14**, 2745 (2002).
3. J. P. Perdew, K. Burke, M. Ernzerhof, "Generalized gradient approximation made simple", *Phys. Rev. Lett.* **77**, 3865 (1996).
4. M. J. van Setten et al., "The PseudoDojo: Training and grading a 85 element optimized norm-conserving pseudopotential table", *Comput. Phys. Commun.* **226**, 39 (2018).

9.6.3 Command-line reference

9.6.3.1 `stb-status`

Status of SIESTA calculation folders: state (finished / failed, with SIESTA's reason / running / stopped), run (version, processes, wall time), system (atoms, cell, spin, XC, k-points, mesh), progress (steps vs requested; a relaxation's forces and stress against its tolerances), SCF (iterations, failures), results (energies, Fermi level, gap, spin moment, forces, pressure) and which suite tools can use the files present. Several folders (a glob, or `--recursive`) give one table, with energies relative to the lowest per atom.

Synopsis.

```
usage: stb-status [-h] [--path PATH] [--recursive] [-l LABEL] [--details]
                  [--csv CSV] [--save-report] [--no-intro] [-v]
```

Options

`--path PATH`
: Directory to inspect, or a glob pattern (e.g. 'strain_*') matching several directories (default: current directory).

`--recursive`
: Every non-hidden folder under `--path` that holds a .out or .fdf. switch, off by default

`-l, --label LABEL`
: SystemLabel override, applied to every directory. Auto-detected per directory from its .fdf file(s) if not given.

`--details`
: With several folders, also print the full report of each one. switch, off by default

`--csv CSV`
: Also write the summary table to this CSV file.

`--save-report`
: Also write the report to `stb_status_report.txt` (current folder). Off by default. switch, off by default

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Example usage:
  stb-status                              # this folder, the full report
  stb-status --path relax                 # another folder
  stb-status --path 'strain/a_*'          # several folders: one table, relative energies
  stb-status --path project --recursive   # every run folder under project/
  stb-status --path project --recursive --csv status.csv
```

9.7 Archive Packager

At a glance

Menu code: 3.7 **Command:** `stb-archive`
Guided example: `examples/3.7-stb-archive/`
Summary: Package a calculation folder into a verified `.tar.gz` – inputs always in, a MANIFEST and SHA-256 checksums, never over an existing archive – for sharing or for the record; the complement of **`stb-clean`**.

9.7.1 Overview

`stb-archive` sorts every file of a SIESTA folder by kind (SIESTA's naming conventions, shared with **`stb-clean`** in `stb.core.run_files`) and decides what goes into the archive. What the input needs to run again always goes in: the `.fdf` files, every file they read (`%include`, `< file`), the pseudopotentials and the job script; the report checks that the archive can rerun the calculation. `-mode results` (default) adds logs, geometry and trajectory, bands, DOS/PDOS, eigenvalues and plots, and leaves out what SIESTA regenerates (basis tables, restart files, H/S binaries, grids), each with its reason; `inputs` keeps only the inputs, `full` everything; `-include`/`-exclude` adjust by suffix. The archive holds one folder named after the source, a `MANIFEST.txt` (the run from **`stb-status`**: state, settings, energy, gap; every file with size, kind and SHA-256; every file left out) and a `SHA256SUMS` for `sha256sum -c`; it is read back and every checksum compared before the report says ok. An existing archive is never overwritten (`_1`, `_2`, ...; `-o` needs `-force`). `-path` takes a glob (one archive per folder), `-recursive` adds subfolders, `-dry-run` only reports.

```
stb-archive --path si_relax --dry-run          # what would go in, and why
stb-archive --path si_relax --recursive        # -> si_relax_archive.tar.gz, verified
stb-archive --path si_relax --mode inputs      # only what reruns it
tar -xzf si_relax_archive.tar.gz && cd si_relax && sha256sum -c SHA256SUMS
```

9.7.2 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/3.7-stb-archive/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

`stb-archive` packages a SIESTA calculation folder into a `.tar.gz` for sharing or for the record. It sorts every file by kind and puts in what you need, with a reason for each file:

- **what the input needs to run again**, always: the `.fdf` files, every file they read (`%include`, `< file`), the pseudopotentials, the job script;
- the results, by default: logs, geometry and trajectory, bands, DOS, PDOS, eigenvalues, plots;
- not the files SIESTA regenerates (basis tables, density matrix, H/S binaries, real-space grids), unless asked.

The archive explains and checks itself. Its `MANIFEST.txt` gives what the run was (state, settings, energy, gap, from `stb-status`), and every file with its size, kind and SHA-256. Its `SHA256SUMS` is checked with `sha256sum -c` after extraction. Before the report says ok, the tool reads the archive back and compares every file's checksum. It never writes over an existing archive. `stb-archive` is the complement of `stb-clean` (`3.2`), which removes outputs to prepare a rerun.

The example uses a real SIESTA 5.4 run kept exactly as SIESTA left it: 47 files, 3 MB. The default archive takes 38 of them and compresses them to 269 KB. An inputs-only archive of 86 KB, unpacked elsewhere and run again, gives the same relaxation to the last digit. Everything runs in seconds; Case 4 also reruns SIESTA (~15 s) when it is installed.

9.7.2.1 Contents

1. Theory
2. The archive at a glance
3. Libraries and external dependencies used
4. What `stb-archive` does
5. Running it both ways
6. Worked example
7. What to take from the example
8. Reproducing the reference data
9. Known, deliberate limitations
10. Step-by-step: archiving your own runs
11. Files in this folder
12. Running the script
13. Flag reference
14. What's next
15. References

9.7.2.2 1. Theory

1.1 Three kinds of file in a SIESTA folder. A finished SIESTA [1] folder holds three kinds of file, with very different value:

Kind	Examples	Size here	Worth keeping?
inputs	`calc.fdf`, `structure.fdf`, `bandlines.txt`, `Si.psml`, `submit.slurm`	86 KB	always: without them the run cannot be repeated
results	`calc.out`, `MESSAGES`, `.XV`, `.STRUCT_OUT`, `.ANI`, `.EIG`, `.bands`, `.PDOS(.xml)`, plots	~1 MB	yes: what you look at, cite and compare
regenerable bulk	`Si.ion*` (basis tables), `.DM`, `.CG`, `.HSX`, `.RHO`, `*.grid.nc`	~2 MB	rarely: SIESTA writes them again from the inputs

In real calculations the third kind grows fastest: grids scale with the mesh, `.HSX`/`.DM` with the system and the range of the basis. That is why the default archive leaves it out, while `--include` (for example `--include .RHO` for a Bader analysis to be redone without SIESTA) or `--mode full` keeps it.

The kind of each file comes from SIESTA's naming conventions (`stb.core.run_files`, shared with `stb-clean`). A file that only its content can classify is opened: a small text file starting with `#!` or holding scheduler directives is a job script.

1.2 Rerunnable: what the input really needs. An `.fdf` can read other files:

```
%include structure.fdf                # the whole file, as if pasted here
%block BandLines < bandlines.txt      # a block's contents from a file
```

`bandlines.txt` has no telling extension, but without it the input stops at once. `stb-archive` follows these references the way SIESTA does, through included files too, and every file they read goes in, whatever its kind. It then checks that the archive *can* rerun the calculation: there is at least one `.fdf`, every file read exists, and every species of `%block ChemicalSpeciesLabel` has a `.psf`/`.psml`. Section 6.4 tests this claim by running SIESTA from the archive.

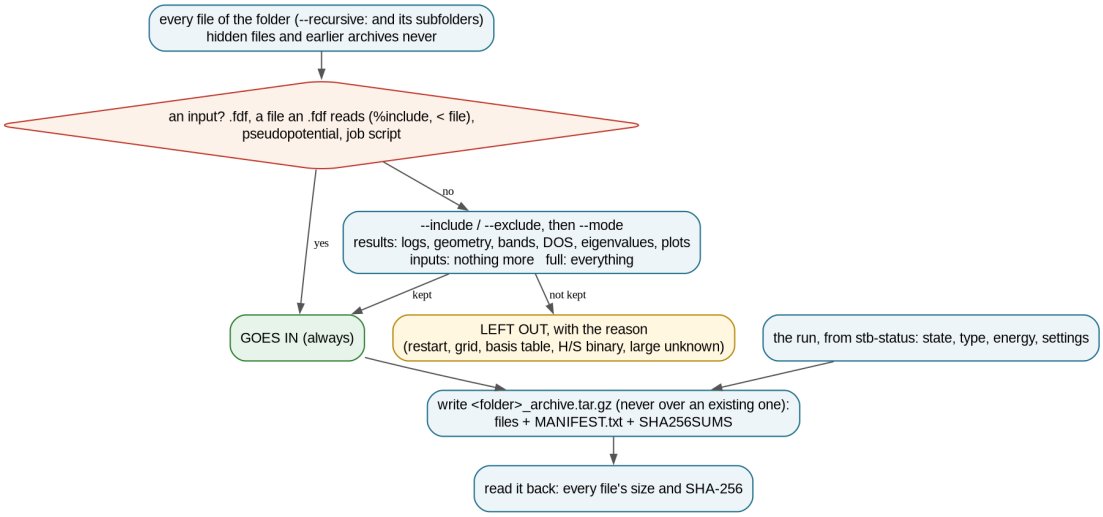
1.3 Integrity: SHA-256 and the read-back. A cryptographic hash such as SHA-256 [2] maps a file to 256 bits. Any change to the file, even one byte, changes the hash, and two different files practically never share one. The archive stores the hash of every file twice: in `MANIFEST.txt`, with sizes and kinds for a reader, and in `SHA256SUMS`, in the format of the coreutils `sha256sum -c` [3]:

```
d2f6...e1a3  calc.fdf
8773...9a28  CLOCK
```

Before reporting ok, `stb-archive` reads the written archive back and recomputes every member's hash. After extraction, anyone can repeat the check with `sha256sum -c SHA256SUMS`, which reports every file `OK`, or `FAILED` for a file that changed (section 6.2). The archive itself gets a hash in the report, to quote when the archive is shared.

1.4 Never overwrite. An archive is often the only copy left once a folder is cleaned or a scratch disk purged. So the default name `<folder>_archive.tar.gz` becomes `<folder>_archive_1.tar.gz`, `_2`, ... when it exists, and an explicit `-o` that exists is refused unless `--force`. Earlier archives and `stb-clean` backups in the folder are never packed into a new one.

9.7.2.3 2. The archive at a glance



stb-archive flowchart

Figure 1. How each file is decided, and what the archive holds.

9.7.2.4 3. Libraries and external dependencies used

Library	Used for
Python standard library (`tarfile`, `hashlib`, `os`)	the archive, SHA-256, walking the folder
`stb.core.run_files`	the kind of each file, the files an `.fdf` reads, job-script detection (shared with `stb-clean`)
`stb.status`	the run summary of the manifest (state, settings, energy, gap)
`stb.core.cli`	the numbered report

No SIESTA, no scientific libraries: it runs on a cluster's login node next to the runs.

9.7.2.5 4. What `stb-archive` does

4.1 Step by step.

1. **List the files**: the folder (`--recursive`: and its subfolders), never hidden files or earlier archives.
2. **Sort them**: inputs (always in), then `--exclude`/`--include`, then the mode (`results`, `inputs`, `full`).
3. **Check rerunnability**: `.fdf` present, every file it reads present, a pseudopotential per species.
4. **Summarize the run** with `stb-status`: state, type, settings, energy, gap, forces; warn if it did not finish, or if a relaxation did not converge.
5. **Write** `<folder>_archive.tar.gz` (one top folder named after the source; never over an existing archive), with `MANIFEST.txt` and `SHA256SUMS`.
6. **Read it back** and compare every file's size and SHA-256.

4.2 The report, section by section.

Section	What it shows
`[0] RUN METADATA`	date, version, target, mode, `--include`/`--exclude`, what always and never goes in
`[1] FOLDER <path>`	files and size, how many go in and out, the run (state, type, SIESTA, energy)
`[2] GOING IN`	every input and why it is one; the results by kind; **Rerunnable**; subfolders left out without `--recursive`
`[3] LEFT OUT`	by kind, with sizes and the reason; the largest files; the `--include` that would add them back
`[4] ARCHIVE`	the file, members, size and compression, the archive's SHA-256, the read-back verification, the restore command
`[n] SUMMARY & FILES`	the archives written; for a glob, one row per folder
`[n+1] WARNINGS`	every warning, numbered

With a glob, sections 1–4 repeat for each folder.

4.3 Guard rails. The report warns when:

- the archive could not rerun the calculation (a missing pseudopotential or a missing file read by an `.fdf`, or no `.fdf`);
- the run did not finish, or failed, or its relaxation did not converge (`stb-status`'s verdict);
- an `-o` archive exists (nothing is written);
- the read-back check fails (the exit code is then 1);
- nothing would go in.

9.7.2.6 5. Running it both ways

From the command line:

```
stb-archive --path si_relax --dry-run          # what would go in, and why
stb-archive --path si_relax --recursive        # results mode, with subfolders ->
si_relax_archive.tar.gz
stb-archive --path si_relax --include .RHO     # also the density
stb-archive --path si_relax --mode inputs      # only what reruns it
stb-archive --path 'strain/a_*'                # one archive per folder
```

Restoring and checking:

```
tar -xzf si_relax_archive.tar.gz && cd si_relax && sha256sum -c SHA256SUMS
```

From the menu (`stb-suite`, code `3.7`): the folder or glob, the mode, subfolders, extra suffixes in or out, dry run and report are asked in turn.

9.7.2.7 6. Worked example

`runs/si_relax/` is a SIESTA 5.4.2 run of bulk Si (PBE [4], DZP, PseudoDojo [5]), made by `run_example.sh`. It is a variable-cell CG relaxation from a = 5.55 Å, followed by bands, DOS/PDOS, the density and H/S (`SaveRho`, `SaveHS`). Its inputs are `calc.fdf`, which `%includes` `structure.fdf` and reads `bandlines.txt`, and the Slurm script `submit.slurm`. `analysis/` holds a band-structure plot and its data. The folder is kept exactly as SIESTA left it, 47 files and 3 MB, except for `Si.psml`. The pseudopotential is an input and must

go into the archive, but the repository does not keep it: the scripts copy it in from the suite's bundled bank. `runs/strain/a_*` are three single points (trimmed of their regenerable files for the repository).

6.1 What would go in: `--dry-run` (Case 1).

```
stb-archive --path si_relax --dry-run
```

```
[1] FOLDER si_relax
Files             : 45 (2.9 MB)
Going in          : 36 files (1003.2 KB); left out 9 (1.9 MB)
The run           : FINISHED, relaxation, SIESTA 5.4.2, E = -230.0355 eV

[2] GOING IN
input         | size    | why
Si.psml       | 84.7 KB | pseudopotential
bandlines.txt | 65 B    | read by calc.fdf
calc.fdf      | 768 B   | SIESTA input
structure.fdf | 375 B   | SIESTA input
submit.slurm  | 123 B   | job script
kind                          | files | size     | e.g.
electronic structure          | 8     | 778.0 KB | si.DOS, si.EIG, si.KP, +5
SIESTA log / status           | 16    | 134.4 KB | 0_NORMAL_EXIT, BASIS_ENTHALPY,
BASIS_HARRIS_ENTHALPY, +13
geometry / forces / dynamics  | 5     | 2.5 KB   | si.ANI, si.BONDS, si.BONDS_FINAL, +2
Rerunnable        : yes -- every .fdf, the files they read and a pseudopotential for every species
Subfolders        : analysis -- not included (--recursive takes them)

[3] LEFT OUT
kind                          | files | size     | why
basis by-products             | 3     | 762.3 KB | basis table (SIESTA regenerates it)
electronic structure          | 1     | 628.3 KB | H/S or wavefunction binary (large)
restart (SIESTA reads it back) | 3     | 376.6 KB | restart file (regenerable by rerunning)
real-space grids              | 2     | 218.4 KB | real-space grid (large)
Largest           : si.HSX (628.3 KB), si.DM (375.9 KB), Si.ion (366.3 KB)
Add back with     : --include .CG .DM .HSX .RHO .STRUCT_NEXT_ITER .grid.nc .ion .ion.nc .ion.xml  (or
--mode full)
```

`bandlines.txt` is an input because `calc.fdf` reads it, and `submit.slurm` because its content is a job script. The `.XV` (the relaxed geometry) goes in, while the other restart files stay out. Two thirds of the folder are regenerable and stay out. The subfolder is not taken without `--recursive`, and the report says so instead of dropping it silently.

6.2 The archive, the manifest and the checksums (Case 2).

```
stb-archive --path si_relax --recursive
```

```
[4] ARCHIVE
Archive           : si_relax_archive.tar.gz
                    40 members (38 files + MANIFEST.txt + SHA256SUMS), 269.4 KB (26 % of 1.0 MB)
Verification      : read back, every file's size and SHA-256 compared -- ok
Restore with      : tar -xzf si_relax_archive.tar.gz && cd si_relax && sha256sum -c SHA256SUMS
```

The manifest (`reference_results/archive/MANIFEST.txt`) starts with the run:

```
THE RUN (stb-status)
State             : FINISHED
Run type          : relaxation
SIESTA            : 5.4.2, 4 MPI processes
XC functional     : GGA Perdew, Burke & Ernzerhof 1996
k-points          : 108
Mesh cutoff       : 200 Ry required, 248.0 Ry used
Relaxation        : converged
Total energy      : -230.035507 eV
Band gap (.EIG)   : 0.9406 eV
```

and goes on with every file (SHA-256, size, kind, reason), every file left out, and how to restore. After extraction `sha256sum -c SHA256SUMS` reports `OK` for each file. With one byte appended to `si.EIG` it reports:

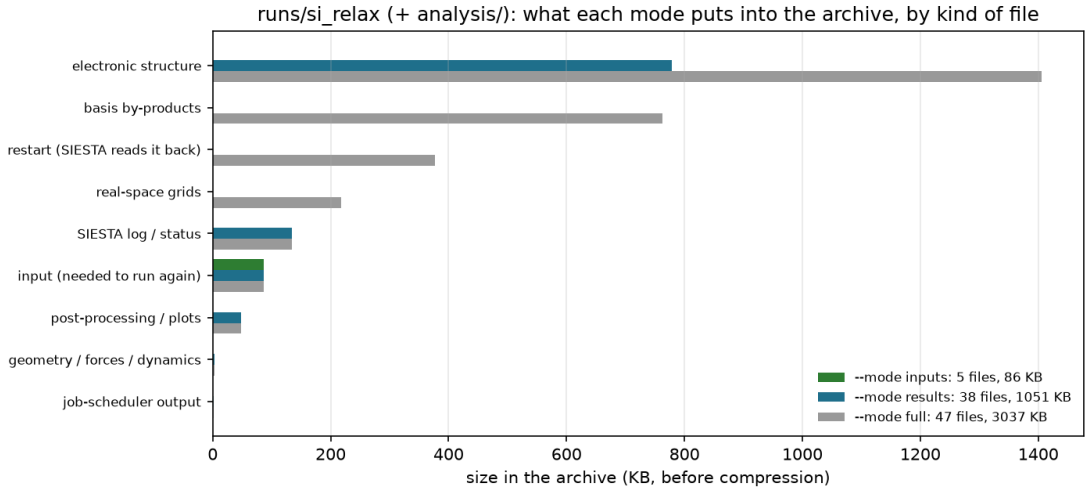
```
si.EIG: FAILED
sha256sum: WARNING: 1 computed checksum did NOT match
```

The archive is one folder, `si_relax/`, so it unpacks next to other archives without mixing files (`reference_results/archive/members.txt`).

6.3 The three modes (Case 3).

`--mode`	Files	Size (before compression)	For
`inputs`	5	86 KB	reproducing the run (section 6.4)
`results` (default)	38	1.0 MB	sharing, papers, the record
`full`	47	3.0 MB	everything, e.g. to move a folder

`--include .RHO` adds the density to the results (39 files, 1.1 MB). Figure 2 shows what each mode takes, by kind.



Contents by mode

Figure 2. `runs/si_relax` *(with* `analysis/`*) by kind of file, and what each mode puts into the archive, from* `stb-archive`*'s own selection.*

6.4 Does an inputs-only archive reproduce the run? (Case 4). `rerun_from_archive.sh` archives the folder with `--mode inputs`, unpacks it in an empty folder and runs SIESTA there (`reference_results/rerun/comparison.txt`):

```
files unpacked from the inputs-only archive: MANIFEST.txt, SHA256SUMS, Si.psml, bandlines.txt,
calc.fdf,
structure.fdf, submit.slurm
original: E_total -230.035507 eV, 8 CG moves, relaxed: True, Fermi -4.180341 eV
rerun   : E_total -230.035507 eV, 8 CG moves, relaxed: True, Fermi -4.180341 eV
difference: +0.000 meV
```

The 86 KB give back the whole calculation: the same 8 moves and the same energy to the last printed digit. The rerun used the same SIESTA build and number of processes. Another build or machine can differ in the last digits, so the manifest records the version and the processes.

6.5 Never overwritten (Case 5).

```
Name              : an archive with the default name exists -- writing si_relax_archive_1.tar.gz
(never overwritten)
Not written       : 'si_relax_archive.tar.gz' exists (pass --force to overwrite)
```

6.6 What the report catches (Case 6). Without the pseudopotential (`reference_results/guard_nopsml/`), and without the file `calc.fdf` reads (`guard_noref/`):

```
1. nopsml: no pseudopotential (.psf/.psml) for Si -- the archive cannot rerun the calculation without
it.
1. noref: an .fdf reads bandlines.txt, which is not in the folder -- the archive cannot rerun the
calculation.
```

A run that never finished (`guard_unfinished/`, the log cut after move 4 and three hours old):

```
1. unfinished: the run is STOPPED (no end of run, log last written 3.00 h ago -- killed, out of time,
or crashed)
   -- archiving an unfinished or failed run.
```

6.7 A series: one archive per folder (Case 7).

```
stb-archive --path 'strain/a_*'
```

```
folder         | run      | files | size     | archive                | archive size | check
strain/a_5.40  | FINISHED | 27    | 221.4 KB | a_5.40_archive.tar.gz  | 60.6 KB      | ok
strain/a_5.476 | FINISHED | 27    | 221.4 KB | a_5.476_archive.tar.gz | 60.6 KB      | ok
strain/a_5.55  | FINISHED | 27    | 221.4 KB | a_5.55_archive.tar.gz  | 60.7 KB      | ok
```

6.8 The interactive menu (Case 8). `stb-suite` → `3.7`, folder `si_relax`, mode 2 (`inputs`), dry run, gives the same report as the command line (`reference_results/menu/report.txt`).

9.7.2.8 7. What to take from the example

- **Inputs always go in**, including the files the `.fdf` reads and the job script; the report checks that the archive can rerun the calculation, and here an 86 KB archive did, exactly.
- The default archive keeps inputs and results (1 MB here) and leaves the regenerable two thirds out, each file with a reason; `--include` or `--mode full` changes that.
- **The archive is verified** before ok and checks itself after extraction (`sha256sum -c`). It also explains itself (`MANIFEST.txt`).
- It never overwrites an archive.

9.7.2.9 8. Reproducing the reference data

```
source ~/.anaconda3/etc/profile.d/conda.sh && conda activate stb   # or any environment with
stb_suite
bash make_reference.sh          # every report and listing into reference_results/<case>/ (seconds,
no SIESTA)
bash rerun_from_archive.sh 4    # reference_results/rerun/ (SIESTA 5.x + mpirun, ~15 s)
bash run_example.sh 4           # runs/ itself (SIESTA, ~30 s)
python3 theory_figures.py       # figures/contents.png
dot -Tpng -Gdpi=150 figures/archive_flowchart.dot -o figures/archive_flowchart.png
```

Dates, scratch paths and the hash of the archive itself (which contains a date) are replaced by `<date>`, `<work>` and `<sha>`; the per-file checksums are deterministic.

9.7.2.10 9. Known, deliberate limitations

- **The kind of a file comes from its name** (and, for job scripts, its first lines). An unknown file goes in below 1 MB and stays out above it in `results` mode, and the report lists it either way.
- **Pseudopotentials are found by species label** (`Si.psml` for species `Si`, the name SIESTA looks for by default); a pseudopotential under another name is reported missing although present.
- **Rerunnable is about files**, not about the environment: the same SIESTA version and build are needed for identical numbers (the manifest records the version and the processes).
- **One archive per folder** with a glob; for a whole project tree in one archive, use `--recursive` on its root.
- **The archive's own hash changes every time** (the manifest holds the date); the per-file hashes do not.

9.7.2.11 10. Step-by-step: archiving your own runs

1. **Check the run** (`stb-status`, `3.6`): finished, SCF converged, relaxed.
2. **`stb-archive --path <folder> --dry-run`**: read `GOING IN` (is every input there? `Rerunnable: yes`?) and `LEFT OUT` (anything you need later? add it with `--include`).
3. **Write it** (drop `--dry-run`; `--recursive` for subfolders) and keep the hash the report prints.
4. **Before deleting the folder**, unpack the archive somewhere and run `sha256sum -c SHA256SUMS`.
5. **Then clean or remove** the folder (`stb-clean`, `3.2`).

9.7.2.12 11. Files in this folder

File	What it is
README.md	this tutorial
example_3.7.sh	the guided walk-through (8 cases, seconds; case 4 reruns SIESTA when available)
runs/	si_relax/ (the full SIESTA 5.4.2 run, as SIESTA left it, minus Si.psml) and strain/a_* (trimmed)
run_example.sh	makes runs/ (needs SIESTA)
make_reference.sh	regenerates reference_results/
rerun_from_archive.sh	the rerun of section 6.4 (needs SIESTA)
theory_figures.py	Figure 2
figures/	every figure of this README; archive_flowchart.dot is the flowchart's source
reference_results/	one folder per case: report.txt; archive/ also members.txt, MANIFEST.txt, sha256sum_check.txt, sha256sum_tampered.txt; rerun/comparison.txt

9.7.2.13 12. Running the script

```
bash example_3.7.sh               # pauses between cases
yes '' | bash example_3.7.sh      # straight through
```

It needs the stb_suite package (SIESTA only for case 4). It starts by wiping its own output/, never touches runs/, and takes a few seconds (plus ~15 s with SIESTA).

9.7.2.14 13. Flag reference

Flag	Default	What it does
--path PATH	.	the folder, or a quoted glob of folders (one archive each)
--mode	results	results, inputs or full
--include SFX ...	none	also files ending with these (e.g. .RHO .HSX)
--exclude SFX ...	none	leave out files ending with these (inputs always go in)
--recursive	off	the files of subfolders too (hidden ones skipped), in the same archive
-l, --label LABEL	from the .fdf	SystemLabel for the run summary
-o, --output FILE	<folder>_archive.tar.gz	the archive's name (not with a glob)
--force	off	overwrite an existing archive of that name
--dry-run	off	report only; nothing written
--save-report	off	also write stb_archive_report.txt (current folder)
--no-intro, -v		skip the banner; print the version

9.7.2.15 14. What's next

- **3.6 (stb-status)** checks the runs before archiving them.
- **3.2 (stb-clean)** resets a folder for a rerun after its results are archived.
- **3.1 (stb-translate)** converts the relaxed .STRUCT_OUT into a CIF to publish next to the archive.

9.7.2.16 15. References

1. J. M. Soler et al., "The SIESTA method for ab initio order-N materials simulation", *J. Phys.: Condens. Matter* **14**, 2745 (2002).
2. National Institute of Standards and Technology, "Secure Hash Standard (SHS)", FIPS PUB 180-4 (2015).
3. GNU coreutils manual, "sha2 utilities: Print or check SHA-2 digests", https://www.gnu.org/software/coreutils/.
4. J. P. Perdew, K. Burke, M. Ernzerhof, "Generalized gradient approximation made simple", *Phys. Rev. Lett.* **77**, 3865 (1996).
5. M. J. van Setten et al., "The PseudoDojo: Training and grading a 85 element optimized norm-conserving pseudopotential table", *Comput. Phys. Commun.* **226**, 39 (2018).

9.7.3 Command-line reference

9.7.3.1 `stb-archive`

Package a SIESTA calculation folder into a verified .tar.gz for sharing or for the record – the complement of stb-clean (3.2). Every file is sorted by kind; what the input needs to run again (.fdf files, every file they read, pseudopotentials, the job script) always goes in. The archive holds MANIFEST.txt (the run, from stb-status, and every file with its SHA-256) and SHA256SUMS (sha256sum -c), and is read back and checked. An existing archive is never overwritten.

Synopsis.

```
usage: stb-archive [-h] [--path PATH] [--mode {results,inputs,full}]
                   [--include EXT [EXT ...]] [--exclude EXT [EXT ...]]
                   [--recursive] [-l LABEL] [-o OUTPUT] [--force] [--dry-run]
                   [--save-report] [--no-intro] [-v]
```

Modes.

```
results  inputs + logs + geometry/trajectory + small electronic outputs (bands, DOS, PDOS, eigenvalues) +
         post-processing; no grids, restart files, basis tables or H/S/wavefunction binaries
inputs   only what the run needs to be repeated: .fdf files, every file they read, pseudopotentials, job scripts
full     every file (except hidden files and earlier archives)
```

Options

`--path PATH`
: Directory to archive, or a glob pattern (e.g. 'strain_*') matching several directories, one archive each (default: current directory).

`--mode {results,inputs,full}`
: What goes in. default: results

`--include EXT [EXT ...]`
: Also include files ending with these (e.g. .RHO .HSX), whatever the mode.

`--exclude EXT [EXT ...]`
: Leave out files ending with these (inputs always go in).

`--recursive`
: Also the files of subfolders (hidden ones skipped), in the same archive. switch, off by default

`-l, --label LABEL`
: SystemLabel override for the run summary (auto-detected from the .fdf).

`-o, --output OUTPUT`
: Archive filename. Default: `<folder>_archive.tar.gz` next to the folder (_1, _2, … if it exists). Not with a glob.

`--force`
: Overwrite an existing archive of the same name. switch, off by default

`--dry-run`
: Report what would go in; write nothing. switch, off by default

`--save-report`
: Also write the report to `stb_archive_report.txt` (current folder). Off by default. switch, off by default

Like every STB command it also accepts `-h`/`--help`, `-v`/`--version` and `--no-intro` (Section 3.3).

From the built-in help.

```
Example usage:
  stb-archive --path si_relax --dry-run       # what would go in, and why
  stb-archive --path si_relax                 # results mode -> si_relax_archive.tar.gz
  stb-archive --path si_relax --include .RHO  # also the density
  stb-archive --path si_relax --mode inputs   # only what reruns it
  stb-archive --path 'strain/a_*'             # one archive per folder
```

Chapter 10

Structures

In this chapter

10.1 2D Monolayer Stacker . . . 281
10.2 Supercell Builder . . . 288
10.3 Slab Builder . . . 294
10.4 Nanotube/Nanoribbon Builder . . . 301
10.5 Point Defect Generator . . . 308
10.6 SQS Generator . . . 314
10.7 Unit Cell Finder . . . 322
10.8 Crystal Builder . . . 328
10.9 Surface Passivator . . . 335
10.10 Reference Molecule Builder . . . 341
10.11 Amorphous Structure Generator . . . 346
10.12 Random Crystal Generator . . . 354

Everything that builds, generates or transforms a structure file lives here: heterostructures of two monolayers (ZSL matching), supercells, slabs, nanotubes and nanoribbons, point defects, special quasirandom structures for alloys, reduction to primitive or conventional cells, construction from a space group and Wyckoff positions, passivation of dangling bonds, reference molecules, amorphous structures by melt-quench, and random symmetric crystals.

All of them read and write SIESTA `.fdf` files (and, where useful, other formats through **`stb-translate`**), run the structure checklist of Section 3.4 on what they produce, and can open the result in ASE's viewer with `-view`. Several can pre-relax their output with MACE (`-ml-relax`) before writing it, which is useful for a starting guess but never a substitute for the SIESTA relaxation.

Code	Tool	Purpose
4.1	2D Monolayer Stacker	Stacks two monolayers into a heterostructure using the ZSL algorithm.
4.2	Supercell Builder	Build a supercell from a structure file using a user-defined transformation matrix.
4.3	Slab Builder	Cut a Miller-index slab from a bulk structure, with vacuum.
4.4	Nanotube/Nanoribbon Builder	Roll a 2D monolayer into a nanotube or nanoribbon, given (n, m) chirality.
4.5	Point Defect Generator	Introduce a vacancy, substitution, or interstitial defect.
4.6	SQS Generator	Generate a Special Quasirandom Structure for a substitutional alloy.
4.7	Unit Cell Finder	Find the primitive or conventional unit cell of a structure.
4.8	Crystal Builder	Build a structure from a space group and Wyckoff positions.

Code	Tool	Purpose
4.9	Surface Passivator	Cap dangling bonds on a cut surface with a passivating atom (e.g.
4.10	Reference Molecule Builder	Build a reference molecule (e.g.
4.11	Amorphous Structure Generator	Melt-quench MD with MACE-MP-0 to build an amorphous starting guess (needs the optional 'ml' extra).
4.12	Random Crystal Generator	Cast random bulk/layer/rod/cluster structures (atomic or molecular, optionally ML-ranked) from a symmetry group and composition; or substitute/subgroup/supergroup an existing structure.

10.1 2D Monolayer Stacker

At a glance

Menu code: 4.1 **Command:** `stb-2Dstacking`
Guided example: `examples/4.1-stb-2Dstacking/`
Summary: Stacks two monolayers into a heterostructure using the ZSL algorithm.

10.1.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/4.1-stb-2Dstacking/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

10.1.1.1 What this tool does

Takes two 2D monolayer structures (each its own `.fdf`, periodic in-plane with vacuum along `c`) and stacks them into a single van der Waals heterostructure: a bottom layer and a top layer, separated by a chosen interlayer gap, sharing one commensurate supercell. This is the first tool in the **Structures** category (4) — everything in this category builds, generates, or transforms a structure file, as opposed to **Inputs** (1), which configures an actual SIESTA run.

10.1.1.2 Why this matters (a bit of theory)

Two independent 2D crystals almost never share an exact lattice. Even a "close" pair like graphene (a = 2.46 Å) and monolayer h-BN (a = 2.504 Å) differ by ~1.8% — enough that neither lattice tiles the other exactly at any size. Two chemically unrelated 2D materials can differ far more.

The **ZSL algorithm** (Zur & McGill, 1984 — see `references.bib` after any run) solves this by searching, up to a maximum supercell area (`--max_area`), over integer supercell transformation matrices for layer 1 and layer 2, looking for a pair whose resulting lattice vectors match each other to within a maximum strain fraction (`--max_strain`). Every match is a trade-off: forcing a match in a small cell usually needs more strain; allowing a larger cell gives the search more candidates to find a lower-strain match, at the cost of more atoms in the final structure. The search itself (the expensive part) runs once per invocation; building several stacking configurations from the same match (different gaps, shifts, twists) is cheap and reuses it.

Two independent match metrics — not just one "strain" number. Every candidate ZSL match is scored on two genuinely different quantities, both shown in the `-i`/`--interactive` candidate table and in the tool's own `[2]` report section:

- **Strain (%)** — the lattice-constant mismatch left after forcing the two supercells to the same size. This is what `--max_strain` bounds.
- **Ang. Strain (°)** — how much layer 2 has to be rigidly rotated ("un-rotated") for its supercell vectors to line up with layer 1's. ZSL matches lattice vectors by **length only**, never absolute orientation — a candidate can have ~0% length strain and still need a large rotation to actually fit. That rotation is a real geometric distortion of the input, not a bookkeeping detail, and it matters most once `--twist` is involved (see below). Matches with *both* numbers under ~1 are highlighted in green in the interactive table — a fast visual cue for "physically clean" candidates.

`match_id=0` (the default, and what `-id 0` picks) is always the **lowest-strain** match, ties broken by lowest angular strain — it does **not** consider the two symmetrically. This is almost always what you want (see `basic/` and `batch-sym/` below), with one important exception covered in the `twist/` case: a self-twisted homobilayer generates a whole family of exact ~0%-strain "trivial" matches regardless of angular strain, which can bury the genuine low-angular-strain Moiré cell far down the candidate list.

21.8° vs. the "magic angle" — a twist-angle caveat, verified live. `--twist` rotates layer 2 by a fixed angle before the ZSL search runs — the same geometric idea behind twisted-bilayer-graphene research. The demo below uses 21.8°, which is not an arbitrary choice: for a hexagonal lattice it is an **exact** commensurate angle, $\cos(\theta) = (3p^2 + 3pq + q^2/2) / (3p^2 + 3pq + q^2)$ with the smallest nontrivial case $p = q = 1$ — a small coincidence cell exists at this angle by construction. This is deliberately **not** the real ~1.1° "magic angle" of flat-band twisted bilayer graphene physics (Bistritzer–MacDonald): that one needs a supercell of several thousand atoms, far beyond what a quick CLI tutorial should build. 21.8° is chosen here specifically because it stays small enough to run in seconds while still being a genuine commensurate twist.

A real, verified gotcha follows directly from the previous point: because `-id 0` sorts on strain first, and a self-twisted homobilayer's trivial "un-rotate the whole twist away" matches also have ~0% strain, `-id 0` at 21.8° silently returns an **untwisted** structure — verified live: its symmetry table shows `p6/mmm`, identical to the plain input, at every `--max_area` from 50 up to 200. The genuine Moiré cell is in the candidate list (verified: match ID 70 of 233, at `--max_area 50`) — it's just sorted far below where the interactive table's 15-row preview stops, since its strain is ~0% but not *exactly* the same floating-point 0.0 the trivial matches get. Once picked directly (`-id 70 -a 50`), the result is genuinely twisted: symmetry drops to triclinic `p1`, a real, physically distinct structure from the naive pick. `example_4.1.sh`'s `twist/` case shows both side by side.

10.1.1.3 When you'd reach for it

- Building a realistic van der Waals heterostructure (e.g. graphene/h-BN, a 2D heterojunction) as a starting point for a real SIESTA calculation.
- Screening several high-symmetry registries (`--batch_sym`) of the same pair before committing to expensive DFT on all of them.
- Generating a set of structures at different interlayer gaps (`--gap_range`) as the first step toward an interlayer binding-energy curve.
- Building a twisted bilayer (`--twist`) of two identical (or different) monolayers, e.g. for Moiré-superlattice physics.
- Sampling an arbitrary registry point (`-tx`/`-ty`, not one of `--batch_sym`'s 6 fixed ones) — e.g. one point of a full gamma-surface (stacking-fault energy) scan.

10.1.1.4 Two ways to run it

A — direct CLI:

```
stb-2Dstacking -l1 graphene.fdf -l2 hbn.fdf -id 0
```

B — interactive `stb-suite` menu:

```
$ stb-suite
Select an option (0-7, or a tool code like 5.1.2): 4.1
```

Both paths call the exact same underlying tool and produce the exact same output — `example_4.1.sh` proves this directly at the end.

10.1.1.5 What every run does (always on)

- **A numbered report** (`[0] RUN METADATA` … `[6] SUMMARY & FILES` — `[3]` only appears with `--ml-relax`, see below) printed to the console and, with `--save-report`, also saved to `stb_2dstacking_report.txt`.
- **Structure validation** — atom proximity, lattice handedness, atomic density, each an explicit `[OK]`/`[WARNING]`/`[SKIPPED]` row — run for **every** heterostructure the invocation builds (not just one), even in `--batch_sym`/`--gap_range` batch mode.
- **A symmetry comparison table** for layer 1 / layer 2 / the heterostructure — crystal system, the 3D **space group** of the vacuum-padded cell, the **layer group** (the physically correct symmetry classification for a genuinely 2D-periodic structure — same `core.symmetry.layer_group_label` stb-fetch already uses), point group, and Hall symbol. Folded directly into the main report (no separate file — there used to be a `--sym_out symmetry_report.txt`, removed once this table moved into the report itself).
- **`references.bib`** — SIESTA (the output is a `.fdf`) plus the ZSL algorithm paper. Merges with whatever `references.bib` is already in the working directory instead of overwriting it.
- **A provenance header** written into the output `.fdf` itself: which two layer files were used, the stacking configuration name, twist angle, strain mode, the max applied linear strain for that structure, and (with `--ml-relax`) the MACE model used plus its convergence/energy.

10.1.1.6 Optional (off by default)

- **`-i`/`--interactive`** — instead of silently taking the lowest-strain match (`-id 0`), prints the top 15 candidates (area, strain, angular strain, transformation matrix) and lets you pick by hand. Rows where *both* strain and angular strain are under ~1 print in green — a quick visual shortcut for "this one's physically clean" without reading every column. Mutually exclusive with `-id`.
- **`-tx`/`-ty` (`--shift_x`/`--shift_y`)** — place layer 2 at any fractional shift instead of one of `--batch_sym`'s 6 fixed registries (ignored if `--batch_sym` is also given). Useful for sampling a full gamma-surface (stacking-fault energy landscape) point by point. An arbitrary shift usually lands on **no** special symmetry point (`p1`, no symmetry) — see `custom-shift/` below for why `--batch_sym`'s 6 points specifically are the ones a hexagonal bilayer's own symmetry can protect.
- **`--vacuum`** — overrides the vacuum thickness instead of inheriting layer 1's own.
- **`--save-report`** — also persists the full report (structure validation, the symmetry table, and, with `--ml-relax`, the full MACE simulation detail) to `stb_2dstacking_report.txt`.

- **`--view`** — opens every generated heterostructure in ASE's interactive 3D viewer (`ase-gui`), as separate, pageable frames. Needs a local display (or `ssh -X/-Y`). Never exercised by `example_4.1.sh` itself.
- **`--ml-relax`** (needs the optional `ml` extra: `pip install stb_suite[ml]`) — pre-relaxes each generated heterostructure with a MACE potential before writing it out. Positions only by default; add **`--ml-relax-cell`** to also relax the in-plane cell (a/b) — the vacuum axis (c) always stays exactly fixed, via the same `core.mace_relax.build_cell_mask` masking `stb-mlrelax` uses for a slab. `--model small/medium/large` (default `small`) or `--custom-model PATH` picks which MACE potential to use. The report shows the full simulation detail per structure: model parameter count/cutoff radius, steps used, convergence, wall time, and a **before/after table** (same convention as `1.7-stb-mlrelax/`'s own comparison table) — energy, max force, and, with `--ml-relax-cell`, lattice a/b/gamma alongside proof that c (the vacuum axis) never moves. A fast heuristic, not a substitute for a real DFT relaxation.

10.1.1.7 Files in this folder

- `graphene.fdf` / `hbn.fdf` — a primitive 2-atom hexagonal graphene cell (a = 2.46 Å) and a primitive 2-atom hexagonal h-BN cell (a = 2.504 Å), a ~1.8% lattice-mismatched pair — the classic van der Waals heterostructure test case, also used for the `--twist` demo (with itself as both layers).
- `example_4.1.sh` — the guided walkthrough (see below).
- `.gitignore` — excludes `output/`, `references.bib`, and `stb_2dstacking_report.txt`.

10.1.1.8 Running the walkthrough

```
cd examples/4.1-stb-2Dstacking
./example_4.1.sh
```

It pauses between sections (`[Press Enter to continue]`) and always starts by wiping its own `output/`. Eight self-contained cases are generated (the last one is skipped, with a hint, if the optional `ml` extra isn't installed):

Folder	What it shows
`basic/`	The mismatched graphene + h-BN pair, `-id 0`
`batch-sym/`	`--batch_sym` — all 6 high-symmetry registries in one run
`gap-range/`	`--gap_range` — the same registry at several interlayer gaps
`strain-modes/`	`-sm top/bottom/sym` side by side
`twist/`	`--twist` — a genuine 21.8° Moiré cell vs. the naive, silently-untwisted `-id 0` pick
`custom-shift/`	`-tx/-ty` + `--vacuum` — an arbitrary registry point, contrasted with `--batch_sym`'s symmetry-protected ones
`full-report/`	`--save-report` + the validation checklist + `references.bib`
`ml-relax/`	`--ml-relax/--ml-relax-cell` — MACE pre-relaxation, vacuum axis fixed

10.1.1.9 Try it yourself

```
# Pick a match by hand instead of -id 0 (shows the full candidate table)
stb-2Dstacking -l1 graphene.fdf -l2 hbn.fdf -i

# Open the result in ASE's interactive viewer
stb-2Dstacking -l1 graphene.fdf -l2 hbn.fdf -id 0 --view

# Combine batch stacking with a gap scan
```

```
stb-2Dstacking -l1 graphene.fdf -l2 hbn.fdf -id 0 --batch_sym --gap_range 3.0 4.0 6

# Pre-relax with MACE before writing, cell shape included (vacuum axis stays fixed)
stb-2Dstacking -l1 graphene.fdf -l2 hbn.fdf -id 0 --ml-relax --ml-relax-cell

# The genuine 21.8 deg Moire cell (match ID 0 alone silently un-rotates this away)
stb-2Dstacking -l1 graphene.fdf -l2 graphene.fdf -id 70 -a 50 -t 21.8
```

10.1.1.10 Flag reference

Flag	Meaning
`-l1/--layer1`, `-l2/--layer2`	Bottom / top monolayer `.fdf` (both required)
`-a/--max_area`	Max ZSL supercell area, Å^2 (default 150.0)
`-s/--max_strain`	Max ZSL strain fraction (default 0.05)
`-i/--interactive`	Show the full match table and pick by hand
`-id/--match_id`	Pick a specific match ID directly (default 0)
`--batch_sym`	Build all high-symmetry registries automatically
`-g/--gap`	Van der Waals gap in Å (default 3.2)
`--gap_range START END STEPS`	Build the same registry at several gaps (energy-curve mode)
`--vacuum`	Target vacuum thickness, Å (default: inherits layer 1's)
`-t/--twist`	Twist angle of layer 2, degrees, before the ZSL search
`-tx/--shift_x`, `-ty/--shift_y`	Fractional shift of layer 2 (ignored with `--batch_sym`)
`-sm/--strain_mode`	`top` (default) / `bottom` / `sym` — which layer absorbs strain
`-o/--output`	Output `.fdf` base name (default `stacked_structure.fdf`)
`-sp/--symprec`	Symmetry tolerance, Å (default 0.01)
`--save-report`	Persist the full report (incl. symmetry table) to disk
`--view`	Open every result in ASE's interactive 3D viewer
`--ml-relax`	Pre-relax with a MACE potential before writing (positions only)
`--ml-relax-cell`	With `--ml-relax`, also relax `a`/`b` (vacuum axis `c` stays fixed)
`--model`	MACE-MP-0 size for `--ml-relax`: `small`/`medium`/`large` (default `small`)
`--custom-model`	Path to a custom fine-tuned `.model` file for `--ml-relax`

10.1.1.11 What's next

See `1.1-stb-inputfile/` for turning a stacked heterostructure into a full SIESTA run (basis, pseudopotentials, k-grid), or `1.7-stb-mlrelax/` for a fast MACE-based pre-relaxation before committing to real DFT.

10.1.2 Implementation details

Commensurate supercells (ZSL)

Two monolayers with in-plane lattices $\{\mathbf{a}_1, \mathbf{a}_2\}$ and $\{\mathbf{b}_1, \mathbf{b}_2\}$ can share a periodic cell only if integer matrices M and N exist with $M\,\mathbf{A} \approx N\,\mathbf{B}$, i.e. two superlattices of (nearly) equal vectors. The Zur–McGill algorithm [59], implemented in pymatgen's `ZSLGenerator` [7], enumerates the superlattices of both layers up to a maximum area, reduces them, and keeps the pairs whose vector lengths differ by less than a tolerance and whose angles agree. The smaller the allowed strain, the larger the cell that may be needed.

Search and ranking.. The search uses `-max_area` (Å^2, default 150) as the area limit, `-max_strain` (fraction, default 0.05) as the length tolerance, and a fixed angle tolerance of 0.1. Every match is then evaluated on the actual supercells: the *linear strain* is the largest relative mismatch of the two supercell vector lengths, and the *angular strain* is the angle between the first vectors of the two supercells (a measure of how much layer 2 would have to be rotated to fit). Matches are sorted by linear strain, then angular strain; `ID 0` is the best and is used unless `-match_id` or `-i` (an interactive table of the 15 best, with area, strains and the transformation matrix of layer 1) chooses another. A twist (`-t`) rotates layer 2 about z before the search; when the selected match still needs more than 1° of angular correction, the report warns and suggests a larger `-max_area`.

Building the bilayer.. Layer 2 is first shifted by (`-tx`, `-ty`) in fractions of its *own* primitive cell, then both layers are expanded to their supercells. The common in-plane lattice is that of layer 1 with `-strain_mode top` (default; layer 2 absorbs the mismatch), that of layer 2 with `bottom`, or the average of the two with `sym` (both strained by half). Atoms keep their fractional coordinates on the common lattice. Layer 2 is placed so that the gap between the topmost atom of layer 1 and the lowest atom of layer 2 is `-g` (default 3.2 Å); the cell vector **c** is set along z with length equal to the total thickness plus the vacuum (`-vacuum`, or by default the vacuum of layer 1). Both input layers must lie in the xy plane with vacuum along z. The atoms of layer 1 come first in the output.

Batches.. `-batch_sym` writes the high-symmetry registries AA $(0,0)$, Bridge_X $(\frac{1}{2},0)$, Bridge_Y $(0,\frac{1}{2})$ and Center $(\frac{1}{2},\frac{1}{2})$, plus Hex_AB $(\frac{1}{3},\frac{2}{3})$ and Hex_BA $(\frac{2}{3},\frac{1}{3})$ when layer 1 is hexagonal ($a=b$, $\gamma=60°$ or $120°$); `-gap_range START END STEPS` writes one structure per gap. Combined, every registry is written at every gap, named `<output>_<registry>_gap_<g>.fdf`. Such a series is the input of a binding-energy curve; for a dense map of lateral shifts use the γ-surface workflow (**5.10**), which reuses the same matching code and runs the ZSL search only once.

Symmetry report.. A table compares, for layer 1, layer 2 and every result, the crystal system, the 3D space group of the padded cell, the layer group (spglib $\geq$ 2.1), the point group and the Hall symbol, so a registry that breaks a symmetry is visible immediately.

Optional MACE pre-relaxation.. `-ml-relax` relaxes each result with MACE (positions; with `-ml-relax-cell` also the in-plane cell, never the vacuum axis) at 0.05 eV/Å and at most 200 steps. Interlayer distances of van der Waals stacks depend strongly on dispersion; MACE-MP-0 is not a substitute for a SIESTA relaxation with an appropriate van der Waals functional or DFT-D3 [24].

10.1.3 Command-line reference

10.1.3.1 `stb-2Dstacking`

STB Monolayer Stacking Tool with Geometric Control & Symmetry Analysis.

Synopsis.

```
usage: stb-2Dstacking [-h] -l1 LAYER1 -l2 LAYER2 [-a MAX_AREA] [-s MAX_STRAIN]
                      [-i | -id MATCH_ID] [--batch_sym]
                      [-g GAP | --gap_range START END STEPS] [--vacuum VACUUM]
                      [-t TWIST] [-tx SHIFT_X] [-ty SHIFT_Y] [-sm {top,bottom,sym}]
                      [-o OUTPUT] [-sp SYMPREC] [--save-report] [--view] [--ml-relax]
                      [--ml-relax-cell] [--model {small,medium,large}]
                      [--custom-model PATH] [--custom-model-card JSON]
                      [--device {cpu,cuda}] [-v] [--no-intro]
```

Options

`-l1, --layer1 LAYER1`
Path to the bottom monolayer .fdf file — required

`-l2, --layer2 LAYER2`
Path to the top monolayer .fdf file — required

`-a, --max_area MAX_AREA`
Maximum supercell area (default: 150.0) — type: float

`-s, --max_strain MAX_STRAIN`
Maximum allowed strain fraction (default: 0.05) — type: float

`-i, --interactive`
Interactive mode: shows table to select match ID — switch, off by default; exclusive group 1

`-id, --match_id MATCH_ID`
Directly select a specific match ID (default: 0) — type: int; exclusive group 1

`--batch_sym`
Generate all high-symmetry stackings automatically — switch, off by default

`-g, --gap GAP`
Van der Waals gap in Angstroms (default: 3.2) — type: float; exclusive group 2

`--gap_range START END STEPS`
Range for energy curve. Ex: 2.5 4.5 11 — type: float; exclusive group 2

`--vacuum VACUUM`
Target vacuum space in Angstroms. Inherits L1 by default. — type: float

`-t, --twist TWIST`
Twist angle of layer 2 in degrees (default: 0.0) — type: float

`-tx, --shift_x SHIFT_X`
Fractional shift of layer 2 in X (default: 0.0) — type: float

`-ty, --shift_y SHIFT_Y`
Fractional shift of layer 2 in Y (default: 0.0) — type: float

`-sm, --strain_mode {top,bottom,sym}`
Strain distribution mode (default: top) — choices: top, bottom, sym

`-o, --output OUTPUT`
Output .fdf base file name — default: stacked_structure.fdf

`-sp, --symprec SYMPREC`
Symmetry tolerance in Angstroms (default: 0.01) — type: float

`--save-report`
Also persist the full run report (including the symmetry analysis) to `stb_2dstacking_report.txt`. Off by default.
switch, off by default

`--view`
Open an interactive 3D view of the final heterostructure(s) (via ASE) after writing the output file(s). Needs a display. Off by default. — switch, off by default

`--ml-relax`
Pre-relax each generated heterostructure with a MACE potential (needs the optional 'ml' extra: pip install stb_suite[ml]) before writing it out – positions only by default. Off by default. — switch, off by default

`--ml-relax-cell`
With `--ml-relax`, also relax the in-plane cell – the vacuum axis (c) always stays exactly fixed. Only valid together with `--ml-relax`. — switch, off by default

`--model {small,medium,large}`
MACE-MP-0 foundation model size for `--ml-relax` (default: medium). choices: small, medium, large

`--custom-model PATH`
Path to a custom fine-tuned .model file for `--ml-relax`, instead of a MACE-MP-0 foundation size.

`--custom-model-card JSON`
Model card of the `--custom-model` file: the `mlff_model_card.json` that `stb-mlffAnalysis` wrote next to it (training data, level of theory, accuracy, energy scale). Required with `--custom-model`: the card is checked against the model file and used to warn when a structure is outside what the model was trained on.

`--device {cpu,cuda}`
Device to run the MACE model on (default: cpu). choices: cpu, cuda

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Example usage:
  stb_stacking -l1 bottom.fdf -l2 top.fdf -i
  stb_stacking -l1 bottom.fdf -l2 top.fdf --batch_sym -g 3.2
```

10.2 Supercell Builder

At a glance

Menu code: 4.2 **Command:** `stb-supercell`
Guided example: `examples/4.2-stb-supercell/`
Summary: Build a supercell from a structure file using a user-defined transformation matrix.

10.2.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/4.2-stb-supercell/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

10.2.1.1 What this tool does

Takes one structure file and tiles it into a larger periodic cell, following an integer transformation matrix: either 3 numbers for a diagonal supercell (`n_a n_b n_c`, the common case — repeat the cell `n_a`/`n_b`/`n_c` times along each lattice vector) or 9 numbers for a full row-major 3×3 matrix (needed for non-orthogonal transformations, see `reconstruction/` below). Same **Structures** category (4) as `4.1-stb-2Dstacking/` — everything here builds, generates, or transforms a structure file, as opposed to **Inputs** (1), which configures an actual SIESTA run.

10.2.1.2 Why this matters (a bit of theory)

The transformation matrix. A supercell is described by a 3×3 integer matrix **M**: the new lattice vectors are `M` applied to the original ones (`pymatgen`'s own `Structure.make_supercell`). Two matrix shapes cover almost every real use:

- **Diagonal**, `diag(n_a, n_b, n_c)` — the everyday case: repeat the cell `n_a×n_b×n_c` times. The **determinant** (`n_a·n_b·n_c` here) is the atom-count multiplication factor — `stb-supercell` always reports it explicitly next to the matrix.
- **Full (non-diagonal)** — needed whenever the enlarged cell isn't simply "the same shape, bigger": a hexagonal 2D material's classic $\sqrt{3}\times\sqrt{3}$ R30° reconstruction cell (`reconstruction/` below) needs off-diagonal entries, because the new lattice vectors point along different crystallographic directions than the original ones, not just longer copies of them.

A **negative determinant** is still geometrically valid — `mirrored/` below shows it — but it is worth pausing on: mirroring a cell that has no inversion/mirror symmetry of its own (a *chiral* crystal) produces a genuinely different, enantiomeric structure, not the same one relabeled. For a centrosymmetric crystal like diamond silicon (which already contains inversion symmetry) the mirrored cell is physically indistinguishable from the original — confirmed below, its full symmetry table comes back identical, entry for entry.

Why build a supercell at all. A few recurring reasons, each pointing at a different downstream tool in this suite:

- **Diluting a point defect** (`stb-defect`) — a defect's periodic images interact with each other through the periodic boundary; a big enough supercell keeps that spurious self-interaction small and sets the effective defect *concentration* (`1 / N_atoms`).
- **Phonon force constants** (`stb-phononsCreate`/`stb-mlphonons`) — the finite-displacement method needs a real-space supercell large enough that a single atom's displacement has decayed to ~0 force on the supercell's own periodic image of itself.
- **Non-orthogonal reconstructions and Moiré-like commensurate cells** — the full 3×3 matrix path, see `reconstruction/`.
- **Matching a target k-point density** cheaply in real space instead of a denser Monkhorst-Pack grid, when a workflow specifically needs a bigger real-space cell (e.g. before `stb-sqs` places atoms across it for a disordered-alloy model).

A genuine gotcha, verified live: Space Group survives, Point Group can look like it doesn't. A well-formed supercell doesn't create a new crystal, it re-describes the same one in a bigger cell — so you'd expect the *whole* symmetry table to come back unchanged. The **Space Group**, **Layer Group**, and **Hall Symbol** columns do exactly that (verified below on both `basic/` and `reconstruction/`): they're derived from `spglib`'s own internally standardized cell, which is independent of the shape you actually handed it.

The **Point Group** column can mislead you, though. `pymatgen`'s `SpacegroupAnalyzer.get_point_group_symbol()` classifies the point group from the raw rotation matrices found for the *exact* cell you gave it, in that cell's own basis — not from the standardized space-group data. An **anisotropic** diagonal supercell (e.g. doubling only along `c`, see `nonuniform/`) genuinely restricts *which* of the crystal's true symmetry operations still map that specific, now non-cubic-shaped cell onto itself as an integer matrix. The physical crystal is still fully cubic; the *given* cell just isn't shaped like a cube any more, so the reported point group quietly drops from `m-3m` to `4/mmm`. Verified directly against `spglib`'s own symmetry dataset: `len(dataset.rotations)` drops from 192 to 128 operations found for the exact same physical crystal, purely because of the cell's shape — while `dataset.pointgroup` (a *different* spglib field, computed from the standardized cell) still correctly reports `m-3m`.

Takeaway: after building an anisotropic supercell, trust the **Space Group** row for "is this still the same crystal", not the **Point Group** row — the latter reports the point symmetry of the specific cell shape you built, which a non-uniform tiling can legitimately (not buggily) reduce.

10.2.1.3 When you'd reach for it

- Preparing a supercell for `stb-defect` (a vacancy/substitution needs periodic images far enough apart) or `stb-passivate`/`stb-slab`.
- Setting up the finite-displacement supercell a phonon calculation needs.
- Building a 2D material's reconstruction/adsorbate-superlattice cell (`reconstruction/`).
- Getting a bigger, MACE-pre-relaxed starting cell before a real SIESTA relaxation of a defect or disordered structure (`ml-relax/`).

10.2.1.4 Two ways to run it

A — direct CLI:

```
stb-supercell -f si_bulk.fdf -d 2 2 2
```

B — interactive `stb-suite` menu:

```
$ stb-suite
Select an option (0-7, or a tool code like 5.1.2): 4.2
```

Both paths call the exact same underlying tool and produce the exact same output — `example_4.2.sh` proves this directly at the end.

10.2.1.5 What every run does (always on)

- **A numbered report** (`[0] RUN METADATA` … `[9] SUMMARY & FILES` — `[4]` only appears with `--ml-relax`) printed to the console and, with `--save-report`, also saved to `stb_supercell_report.txt`.
- **Structure validation** — atom proximity, lattice handedness, atomic density, each an explicit `[OK]`/`[WARNING]`/`[SKIPPED]` row — run once on the input structure and once on the final (possibly ML-relaxed) supercell, so a mirrored cell's left-handedness or a too-small density is never silently missed.
- **A before/after symmetry comparison table** — crystal system, the 3D space group, the layer group (for a 2D-periodic structure, same `core.symmetry.layer_group_label` `stb-fetch`/`stb-2dstacking` already use), point group, and Hall symbol, for the input structure vs. the final supercell.
- **`references.bib`** — SIESTA always (the output is a `.fdf`), plus the MACE papers if `--ml-relax` was used. Merges with whatever `references.bib` is already in the working directory.
- **A provenance header** written into the output `.fdf` itself: the input file, the transformation matrix and its determinant, the atom-count change, and (with `--ml-relax`) the MACE model used plus its convergence and energy change.

10.2.1.6 Optional (off by default)

- **`-sp`/`--symprec`** — symmetry tolerance (Å) for the before/after table (default `0.01`).
- **`--ml-relax`** (needs the optional `ml` extra: `pip install stb_suite[ml]`) — pre-relaxes the *built* supercell with a MACE potential before writing it out. Positions only by default; add **`--ml-relax-cell`** to also relax the cell — any vacuum-padded axis always stays exactly fixed, same `core.mace_relax.build_cell_mask` masking `stb-mlrelax`/`stb-2dstacking` use. `--model small/medium/large` (default `small`) or `--custom-model PATH` picks which MACE potential to use. A fast heuristic, not a substitute for a real DFT relaxation.

- **`--save-report`** — also persists the full report (validation, the symmetry table, and, with `--ml-relax`, the full MACE simulation detail) to `stb_supercell_report.txt`.
- **`--view`** — opens the input structure and the final supercell side by side in ASE's interactive 3D viewer (`ase gui`), as pageable frames. Needs a local display (or `ssh -X`/`-Y`). Never exercised by `example_4.2.sh` itself.

10.2.1.7 Files in this folder

- `si_bulk.fdf` — bulk silicon, diamond cubic, the real conventional 8-atom cell (`a` = 5.431 Å, space group Fd-3m No. 227) — a clean, highly symmetric structure so the before/after symmetry table's lesson is unambiguous.
- `graphene.fdf` — a primitive 2-atom hexagonal graphene monolayer (`a` = 2.46 Å, vacuum along `c`) for the non-orthogonal reconstruction case.
- `example_4.2.sh` — the guided walkthrough (see below).
- `.gitignore` — excludes `output/`, `references.bib`, and `stb_supercell_report.txt`.

10.2.1.8 Running the walkthrough

```
cd examples/4.2-stb-supercell
./example_4.2.sh
```

It pauses between sections (`[Press Enter to continue]`) and always starts by wiping its own `output/`. Six self-contained cases are generated (the last one is skipped, with a hint, if the optional `ml` extra isn't installed):

Folder	What it shows
`basic/`	Diagonal 2×2×2 supercell of bulk Si — the everyday case, symmetry table unchanged entry for entry
`nonuniform/`	Diagonal but anisotropic 1×1×2 — the Space-Group-survives-but-Point-Group-drops gotcha, verified live
`reconstruction/`	Full row-major 3×3 matrix — graphene's $\sqrt{3}\times\sqrt{3}$ R30° reconstruction cell
`mirrored/`	A negative-determinant (mirrored) cell — still valid, and physically identical here since Si is centrosymmetric
`ml-relax/`	`--ml-relax`/`--ml-relax-cell` — MACE pre-relaxation of the built supercell
`full-report/`	`--save-report` + the validation checklist + `references.bib`

10.2.1.9 Try it yourself

```
# A general (non-diagonal) matrix, e.g. a different 2D reconstruction
stb-supercell -f graphene.fdf -d 2 0 0 0 2 0 0 0 1

# Open the result in ASE's interactive viewer, input vs. supercell
stb-supercell -f si_bulk.fdf -d 2 2 2 --view

# Pre-relax with MACE, cell included, with your own fine-tuned model
stb-supercell -f si_bulk.fdf -d 2 2 2 --ml-relax --ml-relax-cell --custom-model my_finetuned.model \
    --custom-model-card mlff_model_card.json

# A tighter symmetry tolerance for a noisier input structure
stb-supercell -f si_bulk.fdf -d 2 2 2 -sp 0.1
```

10.2.1.10 Flag reference

Flag	Meaning
`-f/--file`	Input structure file, `.fdf` (required)
`-d/--dim`	3 numbers (diagonal) or 9 numbers (full row-major matrix) (required)
`-o/--output`	Output `.fdf` file name (default `supercell.fdf`)
`-sp/--symprec`	Symmetry tolerance, Å, for the before/after table (default `0.01`)
`--ml-relax`	Pre-relax the built supercell with a MACE potential before writing
`--ml-relax-cell`	With `--ml-relax`, also relax the cell (vacuum-padded axes stay fixed)
`--model`	MACE-MP-0 size for `--ml-relax`: `small`/`medium`/`large` (default `small`)
`--custom-model`	Path to a custom fine-tuned `.model` file for `--ml-relax`
`--save-report`	Persist the full report (incl. symmetry table) to disk
`--view`	Open the input structure and final supercell in ASE's interactive viewer

10.2.1.11 What's next

See `4.1-stb-2Dstacking/` for a different kind of cell-building tool (two independent monolayers matched via the ZSL algorithm instead of one structure tiled by an integer matrix), or `1.7-stb-mlrelax/` for a closer look at the MACE pre-relaxation `--ml-relax` reuses here.

10.2.2 Implementation details

Matrix convention.. `-d` takes either three integers (a diagonal matrix, $n_1 \times n_2 \times n_3$) or nine (a full matrix, row by row), the same convention as phonopy's `DIM`. The new lattice vectors are

$$\mathbf{a}'_i = \sum_j M_{ij}\,\mathbf{a}_j,$$

so the rows of M express the new vectors in terms of the old ones, and the number of atoms is multiplied by $|\det M|$. Non-integer entries and singular matrices are refused; a negative determinant (a left-handed cell) is accepted with a warning. Common non-diagonal choices are the $\sqrt{3}\times\sqrt{3}R30^\circ$ cell of a hexagonal layer with $\gamma = 120^\circ$, `-d 2 1 0 -1 1 0 0 0 1`, and the conventional cubic cell of an fcc crystal given with the primitive vectors $\frac{a}{2}(0,1,1)$, $\frac{a}{2}(1,0,1)$, $\frac{a}{2}(1,1,0)$, `-d -1 1 1 1 -1 1 1 1 -1`.

What is checked.. The input and the result are both validated and described (formula, atom count, dimensionality, cell parameters), and a symmetry table compares them. A supercell never has *less* space-group symmetry than its parent in principle, but the point group printed for the conventional setting can look different – the guide explains this case.

Optional MACE pre-relaxation.. As in the other builders, `-ml-relax` (with `-ml-relax-cell` for the cell, vacuum axes fixed) relaxes the supercell with MACE before writing it. This is the natural step after introducing a perturbation by hand; for an unperturbed supercell of a relaxed cell it should change nothing.

Supercells and k-points.. A larger cell needs proportionally fewer k-points: when you move from a converged primitive cell to an $n_1 \times n_2 \times n_3$ supercell, divide the k-grid by n_i along each axis (Section 7.3).

10.2.3 Command-line reference

10.2.3.1 `stb-supercell`

Builds a supercell from a SIESTA FDF structure file. Give `-d` 3 numbers for a diagonal supercell, or 9 for a full row-major matrix.

Synopsis.

```
usage: stb-supercell [-h] -f FILENAME -d DIM [DIM ...] [-o OUTPUT] [-sp SYMPREC]
                     [--ml-relax] [--ml-relax-cell] [--model {small,medium,large}]
                     [--custom-model PATH] [--custom-model-card JSON]
                     [--device {cpu,cuda}] [--save-report] [--view] [-v] [--no-intro]
```

Options

`-f, --file FILENAME`
Path to the input structure file (.fdf). required

`-d, --dim DIM [DIM ...]`
Supercell dimensions: 3 numbers for a diagonal supercell (e.g. 2 2 2), or 9 numbers for a full row-major 3x3 matrix (e.g. 2 0 0 0 2 0 0 0 1). required; type: float

`-o, --output OUTPUT`
Output .fdf file name (default: `supercell.fdf`).

`-sp, --symprec SYMPREC`
Symmetry tolerance in Angstroms for the before/after symmetry analysis (default: 0.01). type: float

`--ml-relax`
Pre-relax the built supercell with a MACE potential (needs the optional 'ml' extra: pip install stb_suite[ml]) before writing it out – positions only by default. Off by default. switch, off by default

`--ml-relax-cell`
With `--ml-relax`, also relax the cell – any vacuum-padded axis always stays exactly fixed. Only valid together with `--ml-relax`. switch, off by default

`--model {small,medium,large}`
MACE-MP-0 foundation model size for `--ml-relax` (default: medium). choices: small, medium, large

`--custom-model PATH`
Path to a custom fine-tuned .model file for `--ml-relax`, instead of a MACE-MP-0 foundation size.

`--custom-model-card JSON`
Model card of the `--custom-model` file: the `mlff_model_card.json` that `stb-mlffAnalysis` wrote next to it (training data, level of theory, accuracy, energy scale). Required with `--custom-model`: the card is checked against the model file and used to warn when a structure is outside what the model was trained on.

`--device {cpu,cuda}`
Device to run the MACE model on (default: cpu). choices: cpu, cuda

`--save-report`
Also persist the full run report (including the symmetry analysis) to `stb_supercell_report.txt`. Off by default. switch, off by default

`--view`
Open an interactive 3D view (via ASE) comparing the input structure and the final supercell (page through frames in ase-gui) after writing the output file. Needs a display. Off by default. switch, off by default

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Usage examples:
  stb-supercell -f structure.fdf -d 2 2 2
  stb-supercell -f structure.fdf -d 2 0 0 0 2 0 0 0 1 -o big_cell.fdf
  stb-supercell -f structure.fdf -d 2 2 2 --ml-relax --save-report --view
```

10.3 Slab Builder

At a glance

Menu code: 4.3 **Command:** `stb-slab`
Guided example: `examples/4.3-stb-slab/`
Summary: Cut a Miller-index slab from a bulk structure, with vacuum.

10.3.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/4.3-stb-slab/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

10.3.1.1 What this tool does

Cuts a Miller-index surface out of a bulk structure and adds vacuum along the surface normal, producing a 2D-periodic slab ready for a surface-science SIESTA calculation. Same **Structures** category (4) as `4.1-stb-2Dstacking/` and `4.2-stb-supercell/` — everything here builds, generates, or transforms a structure file, as opposed to **Inputs** (1), which configures an actual SIESTA run.

10.3.1.2 Why this matters (a bit of theory)

Miller indices, terminations, and why more than one slab can come out. A Miller index `(h,k,l)` defines a family of lattice planes, but **cutting** along that family is not unique: depending on exactly where between two planes you cut, you can expose different atoms at the surface — different **terminations**. `stb-slab` (via `pymatgen`'s `SlabGenerator`) enumerates every symmetrically distinct termination for the requested `(h,k,l)`, sorts non-polar/symmetric ones first (the usual physically preferred choice), and by default keeps only the top of that sorted list (`--all` keeps every one, `-i` lets you pick, `--termination N` picks by index directly).

- **Symmetric** — the same termination on both faces of the slab (related by a mirror/inversion). Preferred: an asymmetric slab has two physically different surfaces, which usually also means a spurious residual dipole across the artificial vacuum gap in a periodic calculation.
- **Polar** — see the dedicated caveat below; `stb-slab`'s own `Polar` column needs a caveat before you trust it.

A real gotcha, verified live: the `Polar` column needs oxidation states it never gets. `pymatgen`'s `Slab.is_polar()` computes a dipole from each site's formal **oxidation state** — its own docstring says so directly: *"the Slab must be oxidation state decorated for this to work properly, otherwise the Slab will always have a dipole moment of 0."* A structure read from a SIESTA `.fdf` file never carries oxidation states (`.fdf` has no such concept) — so `stb-slab`'s `Polar` column reads **`No` unconditionally**, regardless of the real electrostatics of the cut.

Verified directly: cutting the same bulk NaCl both along `(1,0,0)` (genuinely non-polar — each atomic layer already has equal Na/Cl) and along `(1,1,1)` (the textbook **polar** rocksalt surface — alternating pure-Na and pure-Cl layers, Tasker's "Type III" classification) both report `Polar: No` through `stb-slab`. Decorating the exact same two structures with formal `Na+1`/`Cl-1` oxidation states in a Python REPL and calling `is_polar()` directly recovers the real physics: dipole per unit cell `0.0` for `(100)` vs. `26.05` (arbitrary units, e/Ang) for `(111)` — see `polar-caveat/` below for the live, side-by-side proof.

Takeaway: use `stb-slab`'s `Polar` column only as "pymatgen found no reason to flag this" — for an ionic/mixed-valence material, cross-check a surface's true polarity yourself (electronegativity difference + which species terminates each side) before trusting it blindly.

Dangling bonds and `--passivate`. Cutting a covalent structure (e.g. tetrahedral Si) necessarily leaves atoms at the new surfaces with fewer bonds than in the bulk — dangling bonds, each roughly a half-filled orbital, a strong artificial perturbation on the electronic structure. `--passivate` caps single-missing-bond sites with a passivating atom (H by default) placed along the geometrically missing-bond direction (`core.passivation.py`, shared with `stb-passivate`) — see `passivate/` below for a real, fully-resolved 8-dangling-bond case.

Surface relaxation and `--ml-relax`. A freshly cut slab's surface atoms sit at their *bulk* positions, but a real surface almost always relaxes away from them once the missing neighbors on one side change the local force balance — "surface relaxation," a real, physically expected effect (distinct from *reconstruction*, which changes the surface's periodicity/composition, not just atomic positions). `--ml-relax` lets a MACE potential estimate this relaxation in seconds, useful groundwork before committing to an expensive real SIESTA relaxation — see `ml-relax/` below for a measured example.

10.3.1.3 When you'd reach for it

- Preparing a clean surface slab for adsorption (`stb-adsorb`), STM/work -function (`stb-workfunction`), or surface-band-structure calculations.
- Comparing multiple terminations of the same Miller index before deciding which one is worth real DFT time.
- Passivating a covalent semiconductor's dangling bonds before a calculation that isn't actually studying the bare surface.
- Getting a MACE-pre-relaxed starting geometry for a defect/adsorption study built on top of a cut slab.

10.3.1.4 Two ways to run it

A — direct CLI:

```
stb-slab -f si_bulk.fdf --hkl 1 0 0
```

B — interactive `stb-suite` menu:

```
$ stb-suite
Select an option (0-7, or a tool code like 5.1.2): 4.3
```

Both paths call the exact same underlying tool and produce the exact same output — `example_4.3.sh` proves this directly at the end.

10.3.1.5 What every run does (always on)

- **A numbered report** (`[0] RUN METADATA ... [7] SUMMARY & FILES` — `[4]` only appears with `--ml-relax`) printed to the console and, with `--save-report`, also saved to `stb_slab_report.txt`.
- **Structure validation** — atom proximity, lattice handedness, atomic density, each an explicit `[OK]`/`[WARNING]`/`[SKIPPED]` row — run once on the bulk input and once per written slab.
- **A before/after symmetry comparison table** per written slab — crystal system, 3D space group, layer group, point group, Hall symbol, for the bulk structure vs. that slab. The bulk always reports `Layer Group: N/A (not 2D-periodic)`; the slab (always genuinely 2D-periodic by construction) always gets a real one — see `basic/` below.
- **`references.bib`** — SIESTA always, plus the MACE papers if `--ml-relax` was used. Merges with whatever `references.bib` is already in the working directory.
- **A provenance header** written into every output `.fdf`: the input file, Miller index, termination index (polar/non-polar, symmetric/asymmetric), passivation detail if used, and MACE convergence/energy if `--ml-relax` was used.

10.3.1.6 Optional (off by default)

- **`-i`/`--interactive`** — show every termination found (formula, atoms, thickness, vacuum, polar, symmetric) and pick one by hand, instead of the default (index 0 after sorting non-polar/symmetric first).
- **`--termination N`** — pick a specific termination index directly.
- **`--all`** — write every termination found, one file per termination.
- **`--symmetrize`** — ask `pymatgen` to symmetrize otherwise-asymmetric terminations (usually changes both the atom count and how many terminations are found — see `symmetrize/` below).
- **`--primitive`** — reduce the bulk to its primitive cell before cutting.
- **`--lll-reduce`** / **`--center-slab`** — pass straight through to `pymatgen`'s own `SlabGenerator` options.
- **`--passivate`** (+ `--passivant`/`--cutoff`/`--bond-length`) — cap dangling bonds on each written slab. See `passivate/` below.
- **`--ml-relax`** (needs the optional `ml` extra: `pip install stb_suite[ml]`) — pre-relaxes each written slab with a MACE potential. Positions only by default; add **`--ml-relax-cell`** to also relax the in-plane cell — the vacuum axis always stays exactly fixed. `--model small/medium/large` (default `small`) or `--custom-model PATH` picks which MACE potential to use.
- **`--save-report`** — also persists the full report to `stb_slab_report.txt`.
- **`--view`** — opens the bulk structure and every written slab in ASE's interactive 3D viewer (`ase-gui`), as pageable frames. Needs a local display. Never exercised by `example_4.3.sh` itself.

10.3.1.7 Files in this folder

- `si_bulk.fdf` — bulk silicon, diamond cubic, the real conventional 8-atom cell (a = 5.431 Å, space group Fd-3m No. 227) — covalent, so every cut surface leaves real dangling bonds.
- `nacl_bulk.fdf` — bulk NaCl, the real rocksalt structure (FCC Bravais lattice, 2-atom basis), conventional 8-atom cubic cell (a = 5.64 Å, space group Fm-3m No. 225) — ionic, for the polar-surface caveat.

- `example_4.3.sh` — the guided walkthrough (see below).
- `.gitignore` — excludes `output/`, `references.bib`, and `stb_slab_report.txt`.

10.3.1.8 Running the walkthrough

```
cd examples/4.3-stb-slab
./example_4.3.sh
```

It pauses between sections (`[Press Enter to continue]`) and always starts by wiping its own `output/`. Six self-contained cases are generated (the last one is skipped, with a hint, if the optional `ml` extra isn't installed):

Folder	What it shows
`basic/`	Si(100), default termination — and the bulk-has-no-layer-group/slab-does symmetry contrast
`symmetrize/`	NaCl(111) — one asymmetric termination vs. two symmetric ones with `--symmetrize`
`polar-caveat/`	NaCl(100)/(111) both report `Polar: No` through stb-slab — the oxidation-state gotcha, proven live
`passivate/`	Si(111), `--passivate` — 8/8 dangling bonds genuinely resolved with H
`ml-relax/`	Si(111), `--ml-relax` — measured surface relaxation
`full-report/`	`--save-report` + the validation checklist + `references.bib`

10.3.1.9 Try it yourself

```
# Show every termination and pick by hand
stb-slab -f si_bulk.fdf --hkl 1 1 1 -i

# Open the bulk and the cut slab in ASE's interactive viewer
stb-slab -f si_bulk.fdf --hkl 1 0 0 --view

# Pre-relax with MACE, cell included, with your own fine-tuned model
stb-slab -f si_bulk.fdf --hkl 1 1 1 --ml-relax --ml-relax-cell --custom-model my_finetuned.model \
    --custom-model-card mlff_model_card.json

# A thicker slab, symmetrized, every termination
stb-slab -f nacl_bulk.fdf --hkl 1 1 1 --min-slab-size 20 --symmetrize --all
```

10.3.1.10 Flag reference

Flag	Meaning
`-f/--file`	Input bulk structure file, `.fdf` (required)
`--hkl H K L`	Miller index of the surface to cut (required)
`--min-slab-size`	Minimum slab thickness, Å (default `10.0`)
`--min-vacuum-size`	Minimum vacuum thickness, Å (default `15.0`)
`--lll-reduce`	LLL-reduce the slab lattice
`--center-slab`	Center the slab in the vacuum
`--primitive`	Reduce the bulk to its primitive cell before cutting
`--symmetrize`	Ask pymatgen to symmetrize polar/asymmetric terminations
`--symprec`	Symmetry tolerance, Å — polar/symmetric diagnostics AND the before/after table (default `0.1`)
`-i/--interactive`	Show all terminations and pick one by hand
`--termination N`	Pick a specific termination index directly

Flag	Meaning
`--all`	Write every termination found
`--passivate`	Cap dangling bonds on each written slab
`--passivant`	Element to passivate with, only with `--passivate` (default `H`)
`--cutoff`	Neighbor-search radius, Å, only with `--passivate` (default: auto)
`--bond-length`	Passivant bond length, Å, only with `--passivate` (default: auto)
`-o/--output`	Output `.fdf` base name (default `slab.fdf`)
`--ml-relax`	Pre-relax each written slab with a MACE potential before writing
`--ml-relax-cell`	With `--ml-relax`, also relax the in-plane cell (vacuum axis stays fixed)
`--model`	MACE-MP-0 size for `--ml-relax`: `small/medium/large` (default `small`)
`--custom-model`	Path to a custom fine-tuned `.model` file for `--ml-relax`
`--save-report`	Persist the full report (incl. symmetry table) to disk
`--view`	Open the bulk structure and every written slab in ASE's interactive viewer

10.3.1.11 What's next

See `4.2-stb-supercell/` for a different structure-transform tool (an integer tiling matrix instead of a Miller-index cut), or `1.7-stb-mlrelax/` for a closer look at the MACE pre-relaxation `--ml-relax` reuses here.

10.3.2 Implementation details

Cutting.. The slab is cut with pymatgen's `SlabGenerator` [7] from the bulk structure, the Miller index `-hkl`, a minimum slab thickness `-min-slab-size` (default 10 Å) and a minimum vacuum `-min-vacuum-size` (default 15 Å). Both minima are rounded up to whole layers of the chosen plane, so the actual thickness and vacuum, printed for every result, are usually larger. The surface normal is placed along the **c** axis. Options passed straight to the generator: `-lll-reduce` (reduce the in-plane cell to the shortest vectors), `-center-slab` (put the slab in the middle of the vacuum), `-primitive` (reduce the bulk to its primitive cell first) and `-symmetrize` (ask pymatgen to make both faces equivalent, removing atoms when needed).

Terminations.. A plane (hkl) can usually be cut at several heights, giving slabs with different surface atoms. All of them are generated and sorted so that the most usable come first: non-polar before polar, then symmetric (equivalent top and bottom faces, within `-symprec`, default 0.1) before asymmetric; pymatgen's own order has no physical meaning. The first one is written by default; `-termination N` picks another, `-i` shows a table to pick from, and `-all` writes every one as `<output>_termN.fdf`.

Polar slabs.. pymatgen judges polarity from oxidation states, which a bare `.fdf` does not carry; the guide shows how the `Polar` column behaves in that case. A polar slab (net dipole across the slab) needs, in SIESTA, at least `Slab.DipoleCorrection` [22, 23], and its surface energy is not well defined from a single asymmetric slab.

Passivation.. `-passivate` caps the dangling bonds of the cut slab with `-passivant` (default H) using the algorithm of **`stb-passivate`** (Section 10.9), with optional `-cutoff` and `-bond-length`.

After the cut.. Each written slab is described (thickness, vacuum, number of atoms, polar and symmetric flags), validated, and optionally pre-relaxed with MACE (`-ml-relax`; `-ml-relax-cell` adds the in-plane cell, never the vacuum axis). A slab model is converged when its surface property no longer changes with thickness and vacuum: build two or three thicknesses and compare.

Fixing the bottom layers

For surface studies one usually relaxes only the upper layers and keeps the bottom ones at bulk positions. SIESTA does this with `atom` lines in a `%block Geometry.Constraints`; the template written by **`stb-inputfile -t relax`** contains that block, commented, with its `stress` lines as an example of the syntax.

10.3.3 Command-line reference

10.3.3.1 `stb-slab`

Cuts a Miller-index slab from a bulk SIESTA FDF structure. Adds vacuum along the surface normal; the vacuum axis lands on c.

Synopsis.

```
usage: stb-slab [-h] -f FILENAME --hkl H K L [--min-slab-size MIN_SLAB_SIZE]
                [--min-vacuum-size MIN_VACUUM_SIZE] [--lll-reduce] [--center-slab]
                [--primitive] [--symmetrize] [--symprec SYMPREC]
                [-i | --termination TERMINATION] [--all] [--passivate]
                [--passivant PASSIVANT] [--cutoff CUTOFF] [--bond-length BOND_LENGTH]
                [--ml-relax] [--ml-relax-cell] [--model {small,medium,large}]
                [--custom-model PATH] [--custom-model-card JSON] [--device {cpu,cuda}]
                [--save-report] [--view] [-o OUTPUT] [-v] [--no-intro]
```

Options

`-f, --file FILENAME`
Path to the input bulk structure file (.fdf). — required

`--hkl H K L`
Miller index of the surface to cut, e.g. `--hkl 1 0 0` — required; type: int

`--min-slab-size MIN_SLAB_SIZE`
Minimum slab thickness in Angstrom (default: 10.0). — type: float

`--min-vacuum-size MIN_VACUUM_SIZE`
Minimum vacuum thickness in Angstrom (default: 15.0). — type: float

`--lll-reduce`
LLL-reduce the slab lattice (pymatgen SlabGenerator option). — switch, off by default

`--center-slab`
Center the slab in the middle of the vacuum (pymatgen SlabGenerator option). — switch, off by default

`--primitive`
Reduce the bulk to its primitive cell before cutting the slab. Off by default so the input cell is used as given.
switch, off by default

`--symmetrize`
Ask pymatgen to symmetrize polar/asymmetric terminations. — switch, off by default

`--symprec SYMPREC`
Symmetry tolerance, Ang, used for the polar/symmetric diagnostics AND the before/after symmetry table (default: 0.1). — type: float

`-i, --interactive`
Show all terminations found and prompt for one to keep. — switch, off by default; exclusive group 1

`--termination TERMINATION`
: Directly select a termination index (0-based). Default: index 0 after sorting non-polar/symmetric terminations first. type: int; exclusive group 1

`--all`
: Write every termination found, instead of just one. switch, off by default

`--passivate`
: Cap dangling bonds on each written termination with a passivating atom (same logic as `stb-passivate`). Only single-missing-bond sites are auto-passivated; sites missing 2+ bonds are reported instead of guessed.switch, off by default

`--passivant PASSIVANT`
: Element to cap dangling bonds with, only with `--passivate` (default: H).

`--cutoff CUTOFF`
: Neighbor-search radius in Angstrom for `--passivate`. Default: auto-detected (see `stb-passivate --help`).type: float

`--bond-length BOND_LENGTH`
: Passivant bond length in Angstrom for `--passivate`. Default: auto per species pair (see `stb-passivate --help`). type: float

`--ml-relax`
: Pre-relax each written slab with a MACE potential before writing it out (needs the optional 'ml' extra: pip install stb_suite[ml]) – positions only by default. Off by default. switch, off by default

`--ml-relax-cell`
: With `--ml-relax`, also relax the in-plane cell – the vacuum axis always stays exactly fixed. Only valid together with `--ml-relax`. switch, off by default

`--model {small,medium,large}`
: MACE-MP-0 foundation model size for `--ml-relax` (default: medium). choices: small, medium, large

`--custom-model PATH`
: Path to a custom fine-tuned .model file for `--ml-relax`, instead of a MACE-MP-0 foundation size.

`--custom-model-card JSON`
: Model card of the `--custom-model` file: the `mlff_model_card.json` that `stb-mlffAnalysis` wrote next to it (training data, level of theory, accuracy, energy scale). Required with `--custom-model`: the card is checked against the model file and used to warn when a structure is outside what the model was trained on.

`--device {cpu,cuda}`
: Device to run the MACE model on (default: cpu). choices: cpu, cuda

`--save-report`
: Also persist the full run report (including the symmetry analysis) to `stb_slab_report.txt`. Off by default. switch, off by default

`--view`
: Open an interactive 3D view (via ASE) of the bulk structure and every written slab (page through frames in ase-gui) after writing the output file(s). Needs a display. Off by default. switch, off by default

`-o, --output OUTPUT`
: Output .fdf file name (default: `slab.fdf`). When more than one termination is written, it is suffixed '_term0', '_term1', ...

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Usage examples:
  stb-slab -f bulk.fdf --hkl 1 0 0
  stb-slab -f bulk.fdf --hkl 1 1 1 -i
  stb-slab -f bulk.fdf --hkl 1 1 1 --all -o surface.fdf
  stb-slab -f bulk.fdf --hkl 1 1 1 --passivate
  stb-slab -f bulk.fdf --hkl 1 1 1 --ml-relax --save-report --view
```

10.4 Nanotube/Nanoribbon Builder

At a glance

Menu code: 4.4 **Command:** `stb-nanotube`
Guided example: `examples/4.4-stb-nanotube/`
Summary: Roll a 2D monolayer into a nanotube or nanoribbon, given (n, m) chirality.

10.4.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/4.4-stb-nanotube/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

10.4.1.1 What this tool does

Rolls a 2D monolayer into a nanotube (a cylinder) or cuts it into a nanoribbon (a finite-width flat strip), given a chirality `(n, m)`. Same **Structures** category (4) as `4.1-stb-2Dstacking/`, `4.2-stb-supercell/`, `4.3-stb-slab/` — everything here builds, generates, or transforms a structure file. Unlike those three, `stb-nanotube` **reduces** dimensionality: a 2D monolayer (1 vacuum axis) always becomes a genuinely 1D-periodic tube or ribbon (2 vacuum axes) — see the symmetry table below for what that means for the reported Layer Group.

Deliberately **not hexagonal-only**: the classic carbon-nanotube `(n, m)` chirality math (gcd/`(n-m)%3d` shortcuts) only works for a honeycomb lattice. `stb-nanotube` instead searches directly over the monolayer's own lattice vectors, so it works for graphene, but equally for h-BN, a buckled/corrugated sheet (silicene, phosphorene, ...), or any other 2D material — see `corrugated/` below.

10.4.1.2 Why this matters (a bit of theory)

The chiral vector and the translation vector. A chirality `(n, m)` defines the **chiral vector** `C = n*a1 + m*a2`, using the monolayer's own in-plane lattice vectors. For **tube** mode, `C` becomes the tube's circumference (rolled so that walking all the way around `C` brings you back to an equivalent lattice point) — the diameter follows directly, `2R = |C| / pi`. For **ribbon** mode, `C` instead sets the finite **width** direction (tiled `--repeats` times), with no wrapping.

Both modes also need a second vector, the **translation vector** `T`, along which the result is periodic (tube axis, or ribbon length). `T` must be a real lattice vector of the monolayer, not parallel to `C` — but "not parallel" alone isn't enough for a *tube*: `roll_to_tube` maps the flat `(frac_C, frac_T)` cell onto a cylinder treating `C` and `T` as an **orthogonal** (circumference, axial) frame. That only preserves real bond lengths/connectivity if `T` genuinely has no component along `C` — i.e. `T` must be as close to **perpendicular** to `C` as an integer lattice vector allows, not merely non-parallel to it.

A real, serious bug, found and fixed: `T` needs to be perpendicular, not just non-parallel. This is not a hypothetical concern — it was a genuine, verified bug in an earlier version of this tool, silently producing physically wrong tubes for almost every chirality except plain zigzag. Verified live against `ase.build.nanotube` (an independent, trusted reference implementation): the old "just pick the shortest non-parallel `T`" logic gave graphene `(6, 0)` a 12-atom cell where every carbon atom had only **1** real bonded neighbor within 1.8 Ang, instead of the correct 3. The fix: every candidate `T` is first reduced modulo `C` (subtracting the integer multiple of `C` that minimizes its remaining along-`C` component — `T' = T + k*C` always generates the *same* periodic cell, since `C x C = 0`), then candidates are ranked by how close to perpendicular they land, length only as the tie-break. `basic-tube/` below reproduces the corrected, physically verified result directly.

Zigzag, armchair, chiral — and a graphene-specific electronic hint. For a genuinely **hexagonal** lattice only (equal-length vectors, 60/120 degrees apart — checked automatically, never assumed), `(n, m)` also gets the classic carbon-nanotube labels: **zigzag** (`m = 0`), **armchair** (`n = m`), or **chiral** (anything else). For a single-species hexagonal lattice with a 2-atom basis (i.e. graphene itself, not e.g. h-BN, where the two sublattices are chemically different), the well-known rule `metallic if (n - m) mod 3 == 0, else semiconducting` is reported too — see `electronic-hint/` below, verified against several chiralities.

`gcd(n, m) > 1`: a physically correct cell, just not always the smallest one. A related, subtler point, also found and documented while fixing the bug above: for a chirality where `gcd(n, m) > 1`, the shortest *exactly perpendicular* `T` this tool finds can require **more** repeating cells than the classic literature (Dresselhaus/Saito) CNT unit cell — because that classic construction additionally allows a combined **rotation+translation** ("screw") symmetry between repeats, which `roll_to_tube/build_ribbon` deliberately never use (screw symmetry is a hexagonal-lattice-specific trick, and this tool works for any 2D lattice). The cell `stb-nanotube` builds is always physically correct — fully, uniformly bonded, verified live — just occasionally larger than the absolute minimum. `basic-tube/` shows the tool's own in-report note about this directly (graphene `(6, 0)` already has `gcd = 6`).

Curvature strain: bond lengths compress on small-diameter tubes. Rolling a flat sheet without stretching along the circumference still changes real 3D bond lengths: `curvature-trend/` below shows this directly, measured, across increasing diameters — bond length rises monotonically from a compressed ~1.39 Ang at `(4, 0)` toward flat graphene's real 1.42 Ang as the tube gets wider. `ml-relax/` then shows this strain is real and relaxable: MACE finds a substantial energy drop and a large force-reduction purely from letting the initial, curvature-compressed geometry (and the axial period itself) relax.

Ribbons have real, physical edges. A **ribbon** (finite in the `C` direction, periodic only along `T`) is a genuinely different kind of object from a tube: it has two real physical edges. A **tube**, being both circumferentially closed *and* axially periodic (infinite in the idealized periodic cell), has none at all — every atom already has its full coordination by construction. `passivate/` below verifies both sides of this directly: 0 dangling bonds on a tube, real ones correctly capped with H on a ribbon's two edges.

10.4.1.3 When you'd reach for it

- Building a carbon nanotube (or a nanotube of any other 2D material) at a specific chirality for electronic-structure calculations.
- Building a graphene/h-BN/TMD nanoribbon, with a specific edge termination (`--passivate`) for a SIESTA calculation.
- Comparing metallic vs. semiconducting chiralities before committing to expensive DFT.
- Getting a MACE-pre-relaxed starting geometry for a small-diameter, strongly-curved tube before a real SIESTA relaxation.

10.4.1.4 Two ways to run it

A — direct CLI:

```
stb-nanotube -f graphene.fdf --chirality 6 0
```

B — interactive `stb-suite` menu:

```
$ stb-suite
Select an option (0-7, or a tool code like 5.1.2): 4.4
```

Both paths call the exact same underlying tool and produce the exact same output — `example_4.4.sh` proves this directly at the end.

10.4.1.5 What every run does (always on)

- **A numbered report** (`[0] RUN METADATA ... [9] SUMMARY & FILES` — `[4]` only appears with `--ml-relax`) printed to the console and, with `--save-report`, also saved to `stb_nanotube_report.txt`.
- **Structure validation** — atom proximity, lattice handedness, atomic density — run once on the input monolayer and once on the output tube/ribbon.
- **A before/after symmetry comparison table** — crystal system, 3D space group, layer group, point group, Hall symbol. The monolayer (before) always has a real Layer Group; the tube/ribbon (after) never does (it's genuinely 1D-periodic, and spglib has no 1D rod-group classification) — the opposite contrast from `4.3-stb-slab/`, where it's the *bulk* input that lacks one.
- **`references.bib`** — SIESTA always, plus the MACE papers if `--ml-relax` was used.
- **A provenance header** written into the output `.fdf`: chirality, mode, translation vector, `N_cells`, geometry (diameter/width, axial period), and passivation/MACE detail if used.

10.4.1.6 Optional (off by default)

- **`--mode ribbon`** (default `tube`) — cut a finite-width strip instead of rolling a cylinder.
- **`--repeats N`** — axial repeats (tube) or width repeats along `C` (ribbon). Default 1.
- **`--passivate`** (+ `--passivant`/`--cutoff`/`--bond-length`) — cap dangling bonds. See `passivate/` below.
- **`--ml-relax`** (needs the optional `ml` extra: `pip install stb_suite[ml]`) — pre-relaxes the built tube/ribbon with a MACE potential. Positions only by default; add **`--ml-relax-cell`** to also relax the axial period — the vacuum-padded width/thickness axes always stay exactly fixed. `--model small/medium/large` (default `small`) or `--custom-model PATH` picks which MACE potential to use.
- **`--save-report`** — also persists the full report to `stb_nanotube_report.txt`.
- **`--view`** — opens the input monolayer and the final tube/ribbon in ASE's interactive 3D viewer (`ase-gui`), as pageable frames. Needs a local display. Never exercised by `example_4.4.sh` itself.

10.4.1.7 Files in this folder

- `graphene.fdf` — a primitive 2-atom hexagonal graphene monolayer (`a` = 2.46 Å, vacuum along `c`) — the main fixture for every case below.
- `silicene_buckled.fdf` — a synthetic buckled honeycomb monolayer (`a` = 3.87 Å, 0.22 Å buckling between the 2 basis atoms) — for the `corrugated/` case.
- `example_4.4.sh` — the guided walkthrough (see below).
- `.gitignore` — excludes `output/`, `references.bib`, and `stb_nanotube_report.txt`.

10.4.1.8 Running the walkthrough

```
cd examples/4.4-stb-nanotube
./example_4.4.sh
```

It pauses between sections (`[Press Enter to continue]`) and always starts by wiping its own `output/`. Nine self-contained cases are generated (two are skipped, with a hint, if the optional `ml` extra isn't installed):

Folder	What it shows
`basic-tube/`	Zigzag `(6,0)` tube — full report, the corrected perpendicular-`T` fix, and the `gcd(n,m)>1` note
`electronic-hint/`	Metallic `(6,0)` vs. semiconducting `(7,0)` vs. always-metallic armchair `(6,6)`
`chiral-index/`	A genuinely chiral `(7,1)` tube (`gcd=1`, no cell-size caveat) — chiral angle, `CNT type: chiral`
`curvature-trend/`	Measured bond-length compression vs. diameter, `(4,0)` through `(20,0)`
`ribbon/`	Ribbon mode — finite width scaling linearly with `--repeats`
`passivate/`	A closed tube has 0 dangling bonds; a ribbon's two edges genuinely do
`ml-relax/`	MACE pre-relaxation measurably releasing curvature strain
`corrugated/`	A buckled (silicene-like) sheet rolls into a genuinely corrugated tube
`full-report/`	`--save-report` + the validation checklist + `references.bib`

10.4.1.9 Try it yourself

```
# Show the input monolayer and the final tube side by side
stb-nanotube -f graphene.fdf --chirality 6 0 --view

# A wide, nearly-strain-free tube
stb-nanotube -f graphene.fdf --chirality 20 0

# Pre-relax with MACE, cell included, with your own fine-tuned model
stb-nanotube -f graphene.fdf --chirality 6 0 --ml-relax --ml-relax-cell --custom-model
my_finetuned.model \
    --custom-model-card mlff_model_card.json

# A passivated nanoribbon, ready for a SIESTA edge-state calculation
stb-nanotube -f graphene.fdf --chirality 6 0 --mode ribbon --repeats 6 --passivate
```

10.4.1.10 Flag reference

Flag	Meaning
`-f/--file`	Input 2D monolayer structure file, `.fdf` (required)
`--chirality N M`	Chirality indices `(n, m)` (required)
`--mode`	`tube` (default) or `ribbon`
`--repeats`	Axial repeats (tube) / width repeats (ribbon), default `1`
`--vacuum-gap`	Gap, Å, to detect the input monolayer's vacuum axis (default `10.0`)
`--min-vacuum-size`	Vacuum padding around the output tube/ribbon, Å (default `15.0`)
`--lattice-tol`	Fractional-coordinate tolerance for cell membership (default `1e-6`)
`--search-bound`	Override the search range for `T` (default: auto)
`--symprec`	Symmetry tolerance, Å, for the before/after table (default `0.01`)
`--passivate`	Cap dangling bonds on the built tube/ribbon
`--passivant`	Element to passivate with, only with `--passivate` (default `H`)
`--cutoff`	Neighbor-search radius, Å, only with `--passivate` (default: auto)

Flag	Meaning
`--bond-length`	Passivant bond length, Å, only with `--passivate` (default: auto)
`-o/--output`	Output `.fdf` file name (default `nanotube.fdf`)
`--ml-relax`	Pre-relax the built tube/ribbon with a MACE potential before writing
`--ml-relax-cell`	With `--ml-relax`, also relax the axial period (vacuum axes stay fixed)
`--model`	MACE-MP-0 size for `--ml-relax`: `small/medium/large` (default `small`)
`--custom-model`	Path to a custom fine-tuned `.model` file for `--ml-relax`
`--save-report`	Persist the full report (incl. symmetry table) to disk
`--view`	Open the input monolayer and final tube/ribbon in ASE's interactive viewer

10.4.1.11 What's next

See `4.3-stb-slab/` for a different kind of cut (a finite Miller-index slab instead of a rolled/tiled monolayer), or `1.7-stb-mlrelax/` for a closer look at the MACE pre-relaxation `--ml-relax` reuses here.

10.4.2 Implementation details

Rolling a sheet

For a 2D lattice $\{\mathbf{a}_1, \mathbf{a}_2\}$ and chirality (n, m) the chiral vector $\mathbf{C} = n\mathbf{a}_1 + m\mathbf{a}_2$ becomes the circumference of the tube, of radius $R = |\mathbf{C}|/2\pi$, and the shortest lattice vector $\mathbf{T} = t_1\mathbf{a}_1 + t_2\mathbf{a}_2$ perpendicular to $\mathbf{C}$ becomes its period. An atom at fractional position (f_C, f_T) of the $\mathbf{C} \times \mathbf{T}$ rectangle, at height δ above the mean plane of the sheet, is mapped to

$$(x, y, z) = \big((R + \delta)\cos 2\pi f_C,\ (R + \delta)\sin 2\pi f_C,\ f_T\,|\mathbf{T}|\big).$$

The translation vector.. Instead of the closed formula valid only for the honeycomb lattice, $\mathbf{T}$ is found by an integer search over (t_1, t_2) within `-search-bound` (default $3(|n| + |m|) + 5$): each candidate is reduced modulo $\mathbf{C}$, and the one closest to perpendicular wins, the shortest breaking ties. The search therefore works for any oblique, rectangular or hexagonal 2D lattice (phosphorene, TMDs, …), and the cell it returns is always a valid period of the tube. When $\gcd(n, m) > 1$ the tube also has a screw symmetry, so this cell may be longer than the smallest possible description (the guide discusses it).

The sheet.. The input must be a monolayer with one vacuum axis (detected with `-vacuum-gap`). Atom heights δ are measured from the sheet's own mean plane, so buckled and multi-atom-thick layers (silicene, MoS_2) roll into correctly corrugated tubes wherever the sheet sits in its box. Every atom image inside the $\mathbf{C} \times \mathbf{T}$ cell is enumerated, with `-lattice-tol` as the boundary tolerance.

Tubes.. `-mode tube` (default) writes the rolled cell with the axis along $\mathbf{c}$, repeated `-repeats` times along it, in a square box of side $2(r_{\max} + v)$, where $r_{\max}$ is the outermost atom radius and v is `-min-vacuum-size` (default 15 Å). The report gives the radius, the diameter, bond-length statistics before and after rolling (small tubes compress bonds on the inside), and for hexagonal lattices the zigzag ($m = 0$), armchair ($n = m$) or chiral label with the graphene rule "metallic when $(n - m)$ is a multiple of 3" – valid only for carbon, as a zone-folding hint.

Ribbons.. `-mode ribbon` keeps the sheet flat and repeats the cell `-repeats` times along $\mathbf{C}$, giving a ribbon of finite width with real edges, periodic along $\mathbf{T}$ and padded with vacuum across its width and thickness. An $(n, 0)$ cut of graphene gives a zigzag-edged ribbon, an (n, n) cut an armchair-edged one. Edges have dangling bonds; `-passivate` caps them (Section 10.9).

Then relax.. Rolling bends bonds; always relax the result (`-ml-relax` gives a quick pre-relaxation, `-ml-relax-cell` relaxes the axial period only), and use a k-grid with one point along the two vacuum axes and enough points along the tube.

10.4.3 Command-line reference

10.4.3.1 `stb-nanotube`

Rolls a 2D monolayer FDF into a nanotube or nanoribbon. Chirality (n, m) picks the vector to roll around (tube) or the finite width direction (ribbon); works for any oblique 2D lattice, not just hexagonal (no gcd/(n-m)%3d hexagonal-only shortcut is used).

Synopsis.

```
usage: stb-nanotube [-h] -f FILENAME --chirality N M [--mode {tube,ribbon}]
                    [--repeats REPEATS] [--vacuum-gap VACUUM_GAP]
                    [--min-vacuum-size MIN_VACUUM_SIZE] [--lattice-tol LATTICE_TOL]
                    [--search-bound SEARCH_BOUND] [--symprec SYMPREC] [--passivate]
                    [--passivant PASSIVANT] [--cutoff CUTOFF]
                    [--bond-length BOND_LENGTH] [--ml-relax] [--ml-relax-cell]
                    [--model {small,medium,large}] [--custom-model PATH]
                    [--custom-model-card JSON] [--device {cpu,cuda}] [--save-report]
                    [--view] [-o OUTPUT] [-v] [--no-intro]
```

Options

`-f, --file FILENAME`
: Path to the input 2D monolayer structure file (.fdf). — required

`--chirality N M`
: Chirality indices (n, m), e.g. `--chirality 6 0` — required; type: int

`--mode {tube,ribbon}`
: 'tube': roll into a cylinder. 'ribbon': finite-width flat strip. Default: tube. — choices: tube, ribbon

`--repeats REPEATS`
: Axial repeats (tube) or width repeats along the chirality vector (ribbon). Default: 1. — type: int

`--vacuum-gap VACUUM_GAP`
: Minimum empty span (Angstrom) used to detect which axis of the INPUT monolayer is vacuum-padded (default: 10.0). Same meaning as `stb-kgrid/stb-kpath's --vacuum-gap`. — type: float

`--min-vacuum-size MIN_VACUUM_SIZE`
: Vacuum padding (Angstrom) added around the generated tube/ribbon (default: 15.0). Same meaning as `stb-slab's --min-vacuum-size`. — type: float

`--lattice-tol LATTICE_TOL`
: Fractional-coordinate tolerance for the C x T cell-membership test (default: 1e-6). — type: float

`--search-bound SEARCH_BOUND`
: Override the search range for the translation vector T (default: auto, 3*(|n|+|m|)+5). — type: int

`--symprec SYMPREC`
: Symmetry tolerance, Ang, for the before/after symmetry table (default: 0.01). — type: float

`--passivate`
: Cap dangling bonds on the built tube/ribbon with a passivating atom (same logic as `stb-passivate/stb-slab`). A properly closed, axially-periodic tube has no dangling bonds to begin with (every atom already has 3 real

neighbors) – this mainly matters for `--mode` ribbon, whose two finite edges are real, physical dangling bonds. Only single-missing-bond sites are auto-passivated; sites missing 2+ bonds are reported instead of guessed. switch, off by default

`--passivant PASSIVANT`

Element to cap dangling bonds with, only with `--passivate` (default: H).

`--cutoff CUTOFF`

Neighbor-search radius in Angstrom for `--passivate`. Default: auto-detected (see `stb-passivate --help`).type: float

`--bond-length BOND_LENGTH`

Passivant bond length in Angstrom for `--passivate`. Default: auto per species pair (see `stb-passivate --help`). type: float

`--ml-relax`

Pre-relax the built tube/ribbon with a MACE potential before writing it out (needs the optional 'ml' extra: pip install stb_suite[ml]) – positions only by default. Off by default. switch, off by default

`--ml-relax-cell`

With `--ml-relax`, also relax the axial period (the vacuum-padded width/thickness axes always stay exactly fixed). Only valid together with `--ml-relax`. switch, off by default

`--model {small,medium,large}`

MACE-MP-0 foundation model size for `--ml-relax` (default: medium). choices: small, medium, large

`--custom-model PATH`

Path to a custom fine-tuned .model file for `--ml-relax`, instead of a MACE-MP-0 foundation size.

`--custom-model-card JSON`

Model card of the `--custom-model` file: the `mlff_model_card.json` that `stb-mlffAnalysis` wrote next to it (training data, level of theory, accuracy, energy scale). Required with `--custom-model`: the card is checked against the model file and used to warn when a structure is outside what the model was trained on.

`--device {cpu,cuda}`

Device to run the MACE model on (default: cpu). choices: cpu, cuda

`--save-report`

Also persist the full run report (including the symmetry analysis) to `stb_nanotube_report.txt`. Off by default. switch, off by default

`--view`

Open an interactive 3D view (via ASE) comparing the input monolayer and the final tube/ribbon (page through frames in ase-gui) after writing the output file. Needs a display. Off by default. switch, off by default

`-o, --output OUTPUT`

Output .fdf file name (default: `nanotube.fdf`).

Like every STB command it also accepts `-h`/`--help`, `-v`/`--version` and `--no-intro` (Section 3.3).

From the built-in help.

```
Usage examples:
  stb-nanotube -f graphene.fdf --chirality 6 0 --mode tube
  stb-nanotube -f graphene.fdf --chirality 6 0 --mode ribbon --repeats 4
  stb-nanotube -f graphene.fdf --chirality 6 0 --mode ribbon --repeats 4 --passivate
  stb-nanotube -f graphene.fdf --chirality 6 0 --ml-relax --save-report --view
```

10.5 Point Defect Generator

At a glance

Menu code: 4.5 **Command:** `stb-defect`
Guided example: `examples/4.5-stb-defect/`
Summary: Introduce a vacancy, substitution, or interstitial defect.

10.5.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/4.5-stb-defect/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

10.5.1.1 What this tool does

Introduces a point defect — a **vacancy** (remove an atom), a **substitution** (swap an atom's species), or an **interstitial** (add a new atom) — into a structure, choosing the site by raw atom index, by the position closest to a target, or automatically at *every* symmetrically distinct site at once. Same **Structures** category (4) as `4.1-stb-2Dstacking/`, `4.2-stb-supercell/`, `4.3-stb-slab/`, `4.4-stb-nanotube/` — everything here builds, generates, or transforms a structure file.

10.5.1.2 Why this matters (a bit of theory)

Three kinds of point defect, three different physical roles.

- **Vacancy** — a missing atom. The simplest defect, and a real equilibrium feature of every crystal at finite temperature; also the starting point for diffusion (an atom hops into a vacancy) and for many color centers (e.g. diamond's nitrogen-vacancy center).
- **Substitution** — one atom's species swapped for another. The mechanism behind doping (Si:P, Si:B for n-/p-type semiconductors) and alloying.
- **Interstitial** — an extra atom squeezed in between existing ones. Common for small atoms (H, Li, N) diffusing through a host lattice — e.g. hydrogen embrittlement, or Li-ion battery intercalation.

A point defect always breaks symmetry — verified, and it's dramatic. A perfect crystal's high symmetry comes from every atom's environment being equivalent to many others under the crystal's own symmetry operations. A single point defect, embedded in an otherwise-periodic cell, breaks nearly all of that at once: the defect site is no longer equivalent to its neighbors, and any symmetry operation that used to map it onto a different (now unperturbed) atom no longer holds. `vacancy-index/` below shows this directly and quantitatively: removing **one** atom from a perfectly symmetric 8-atom cubic silicon cell (space group Fd-3m, No. 227, 48 symmetry operations) drops it straight to P-43m (No. 215, 24 operations) — losing inversion symmetry entirely, from removing a single atom out of eight.

Not every site is equivalent — `--all-inequivalent-sites`. In a crystal with more than one Wyckoff position for a given element, a vacancy/substitution at one site is **not** the same defect as one at another — different local coordination, different formation energy, different physics. Picking a site by hand (`--index`) requires already knowing which sites are distinct; `--all-inequivalent-sites` instead finds every symmetrically distinct site automatically (via spglib) and writes one output structure per site — see `all-inequivalent-sites/` below, using magnetite (Fe_3O_4), whose Fe sits on two genuinely different sites (tetrahedral and octahedral) while O sits on only one.

`--ml-rank`: which site is actually more stable?. Knowing the sites is only half the story — `--ml-rank` (only valid with `--all-inequivalent-sites`) relaxes each candidate with a MACE potential and ranks them by energy, a fast pre-screen for which site is worth real DFT time. `ml-rank/` below reproduces a genuine, physically sensible result: an Fe vacancy at magnetite's octahedral site comes out **~0.06 eV more stable** than at the tetrahedral site — a real, non-degenerate answer, not a coin flip.

`--ml-rank` vs. the generic `--ml-relax`. Both use the same MACE machinery, for two different jobs:

- **`--ml-rank`** (only with `--all-inequivalent-sites`) relaxes *every* candidate site and **ranks** them — the point is comparing sites.
- **`--ml-relax`** (any selection mode, including a single `--index` choice) just pre-relaxes whichever structure(s) you're already writing — the point is a better starting geometry, not a comparison. The two are mutually exclusive when both would apply to the same run (using `--ml-rank` already relaxes every candidate; there's nothing left for `--ml-relax` to add without picking a winner first). `ml-relax/` below uses the generic form, on a single chosen vacancy.

Diluting a defect: pair this tool with `stb-supercell`. A defect embedded in a small periodic cell interacts with its own periodic images — usually not what you want. The fix is a big enough supercell first (`4.2-stb-supercell/`): the defect *concentration* is `1 / N_atoms`, so a bigger cell means a more dilute, more physically realistic defect. This example's fixtures are deliberately small (fast, readable output); a real calculation would typically run `stb-supercell` first, then `stb-defect` on the result.

10.5.1.3 When you'd reach for it

- Building a vacancy/substitution/interstitial structure for a defect formation-energy or diffusion-barrier DFT calculation.
- Screening every symmetrically distinct site of a multi-site element before committing to expensive DFT on all of them.
- Doping a semiconductor host at a specific, hand-picked site.
- Getting a MACE-pre-relaxed starting geometry for a defect calculation.

10.5.1.4 Two ways to run it

A — direct CLI:

```
stb-defect -f si_bulk.fdf --type vacancy --index 1
```

B — interactive `stb-suite` menu:

```
$ stb-suite
Select an option (0-7, or a tool code like 5.1.2): 4.5
```

Both paths call the exact same underlying tool and produce the exact same output — `example_4.5.sh` proves this directly at the end.

10.5.1.5 What every run does (always on)

- **A numbered report** (`[0] RUN METADATA` … `[7] SUMMARY & FILES` — `[4]` only appears with `--ml-relax`/`--ml-rank`) printed to the console and, with `--save-report`, also saved to `stb_defect_report.txt`.

- **Structure validation** — atom proximity, lattice handedness, atomic density — run once on the input structure and once per output structure.
- **A before/after symmetry comparison table** per output structure — crystal system, 3D space group, layer group, point group, Hall symbol.
- **`references.bib`** — SIESTA always, plus the MACE papers if `--ml-relax`/`--ml-rank` was used.
- **A provenance header** written into every output `.fdf`: the defect type, which atom(s)/site were involved, and MACE convergence/energy detail if used.

10.5.1.6 Optional (off by default)

- **`--nearest X Y Z`** (+ `--nearest-format`, `--filter-species`) — pick the site closest to a target position instead of a raw index.
- **`--all-inequivalent-sites`** (+ `--filter-species`) — one output structure per symmetrically distinct site (vacancy/substitution only).
- **`--ml-rank`** (needs the optional `ml` extra: `pip install stb_suite[ml]`, only with `--all-inequivalent-sites`) — relax and rank every candidate site by MACE-relaxed energy.
- **`--ml-relax`** (+ `--ml-relax-cell`, `--model`/`--custom-model`) — pre-relax the output structure(s) generically, any selection mode. Mutually exclusive with `--ml-rank`.
- **`--save-report`** — also persists the full report to `stb_defect_report.txt`.
- **`--view`** — opens the input structure and every output structure in ASE's interactive 3D viewer (`ase gui`), as pageable frames. Needs a local display. Never exercised by `example_4.5.sh` itself.

10.5.1.7 Files in this folder

- `si_bulk.fdf` — bulk silicon, diamond cubic, the real conventional 8-atom cell (a = 5.431 Å, space group Fd-3m No. 227) — highly symmetric on purpose, so the symmetry-breaking demo is unambiguous.
- `magnetite.fdf` — magnetite (Fe_3O_4), 14 atoms, space group Fd-3m (No. 227) — Fe on two genuinely distinct sites (tetrahedral, octahedral), O on one.
- `example_4.5.sh` — the guided walkthrough (see below).
- `.gitignore` — excludes `output/`, `references.bib`, and `stb_defect_report.txt`.

10.5.1.8 Running the walkthrough

```
cd examples/4.5-stb-defect
./example_4.5.sh
```

It pauses between sections (`[Press Enter to continue]`) and always starts by wiping its own `output/`. Eight self-contained cases are generated (two are skipped, with a hint, if the optional `ml` extra isn't installed):

Folder	What it shows
`vacancy-index/`	A Si vacancy by `--index` — the symmetry-breaking demo, Fd-3m -> P-43m
`substitution/`	Doping, Si -> Ge by `--index`
`interstitial/`	Adding an N interstitial at a fractional position
`nearest-position/`	Selecting a site by `--nearest`, with periodic wraparound
`all-inequivalent-sites/`	Magnetite's 2 distinct Fe sites, found automatically
`ml-rank/`	Ranking those 2 Fe sites by MACE-relaxed energy (octahedral wins)
`ml-relax/`	Generic MACE pre-relaxation of a single chosen vacancy

Folder	What it shows
`full-report/`	`--save-report` + the validation checklist + `references.bib`

10.5.1.9 Try it yourself

```
# Show the input structure and the defect structure side by side
stb-defect -f si_bulk.fdf --type vacancy --index 1 --view

# Every inequivalent O site in magnetite, no MACE needed
stb-defect -f magnetite.fdf --type vacancy --all-inequivalent-sites --filter-species O

# Pre-relax with MACE, cell included, with your own fine-tuned model
stb-defect -f si_bulk.fdf --type vacancy --index 1 --ml-relax --ml-relax-cell --custom-model
my_finetuned.model \
    --custom-model-card mlff_model_card.json

# Dilute the defect first: a bigger cell, then a vacancy in it
stb-supercell -f si_bulk.fdf -d 2 2 2 -o si64.fdf --no-intro
stb-defect -f si64.fdf --type vacancy --index 1
```

10.5.1.10 Flag reference

Flag	Meaning
`-f/--file`	Input structure file, `.fdf` (required)
`--type`	`vacancy` / `substitution` / `interstitial` (required)
`--index`	Comma-separated 1-indexed atom(s), e.g. `3,7`
`--nearest X Y Z`	Select the atom closest to this position
`--nearest-format`	`fractional` (default) or `cartesian`, for `--nearest`
`--all-inequivalent-sites`	One output structure per symmetrically distinct site
`--filter-species`	Restrict `--nearest`/`--all-inequivalent-sites` to one element
`--symprec`	Symmetry precision for site-finding and the before/after table (default `1e-3`)
`--ml-rank`	Relax and rank every candidate site (needs `--all-inequivalent-sites`)
`--new-species`	Replacement element, required for `--type substitution`
`--position X Y Z`	New atom's position, required for `--type interstitial`
`--position-format`	`fractional` (default) or `cartesian`, for `--position`
`--species`	New atom's element, required for `--type interstitial`
`--ml-relax`	Pre-relax the output structure(s) with a MACE potential
`--ml-relax-cell`	With `--ml-relax`, also relax the cell (vacuum axes stay fixed)
`--model`	MACE-MP-0 size for `--ml-relax`/`--ml-rank`: `small`/`medium`/`large` (default `small`)
`--custom-model`	Path to a custom fine-tuned `.model` file
`-o/--output`	Output `.fdf` file name (default `defect.fdf`)
`--save-report`	Persist the full report (incl. symmetry table) to disk
`--view`	Open the input and output structure(s) in ASE's interactive viewer

10.5.1.11 What's next

See `4.2-stb-supercell/` for building a bigger, more dilute cell before introducing a defect into it, or `1.7-stb-mlrelax/` for a closer look at the MACE pre-relaxation `--ml-relax`/`--ml-rank` reuse here.

10.5.2 Implementation details

Selecting the site.. For vacancies and substitutions the target atom is given in one of three ways:

- `-index` – one or more 1-based indices (`3,7`), in the order of the `AtomicCoordinatesAndAtomicSpecies` block;
- `-nearest X Y Z` – the atom closest to a point, in fractional (`-nearest-format fractional`, default) or Cartesian coordinates, using the periodic minimum-image distance so a point near a cell face finds the right atom; `-filter-species` restricts the search to one element;
- `-all-inequivalent-sites` – one output per symmetrically distinct site (spglib's equivalent-atom mapping at `-symprec`, default 0.01), listed with its Wyckoff letter and multiplicity and written as `<stem>_siteN.fdf`, where `N` is the 1-based index of the representative atom; `-filter-species` keeps only the sites of one element.

Interstitials need `-position` (fractional by default, `-position-format`) and `-species`. A substitution needs `-new-species`; a new element is added to `ChemicalSpeciesLabel` with the next free index, and the block is written without gaps (Section 4.1).

Ranking sites with MACE.. With `-all-inequivalent-sites`, `-ml-rank` relaxes every candidate with MACE (positions only, cell fixed) and sorts them by relaxed total energy, reporting each relative to the lowest. All candidates have the same composition, so energy differences are meaningful as a ranking, but they are model energies of a small cell: use the ranking to decide which sites deserve a SIESTA relaxation, not as defect formation energies. `-ml-relax` instead just pre-relaxes whatever is written (optionally with the cell, `-ml-relax-cell`); the two options are exclusive.

Symmetry.. The report compares the space group before and after: a point defect in a small cell lowers the symmetry, often drastically, which also changes the number of irreducible k-points SIESTA uses.

From a defect cell to a formation energy

The formation energy of a neutral defect X is

$$E_f[X] = E_{\mathrm{tot}}[X] - E_{\mathrm{tot}}[\mathrm{bulk}] - \sum_i n_i \mu_i,$$

where n_i atoms of species i were added ($n_i > 0$) or removed ($n_i < 0$) and μ_i are their chemical potentials, which depend on the growth conditions. Both total energies must come from cells of the same size and the same settings; the defect–defect interaction through periodic images decays slowly with cell size, so build the defect in a supercell (**`stb-supercell`** first) and check convergence with two sizes. Charged defects additionally need the electron chemical potential and an electrostatic finite-size correction, which this tool does not provide.

10.5.3 Command-line reference

10.5.3.1 `stb-defect`

Introduces a point defect (vacancy, substitution, interstitial) into a SIESTA FDF structure. Select the site by raw atom index, by the position closest to a target, or automatically at every symmetrically distinct site.

Synopsis.

```
usage: stb-defect [-h] -f FILENAME --type {vacancy,substitution,interstitial}
                  [--index INDEX | --nearest X Y Z | --all-inequivalent-sites]
                  [--nearest-format {fractional,cartesian}]
                  [--filter-species FILTER_SPECIES] [--symprec SYMPREC] [--ml-rank]
                  [--new-species NEW_SPECIES] [--position X Y Z]
                  [--position-format {fractional,cartesian}] [--species SPECIES]
                  [--ml-relax] [--ml-relax-cell] [--model {small,medium,large}]
                  [--custom-model PATH] [--custom-model-card JSON]
                  [--device {cpu,cuda}] [--save-report] [--view] [-o OUTPUT] [-v]
                  [--no-intro]
```

Options

`-f, --file FILENAME`

Path to the input structure file (.fdf). required

`--type {vacancy,substitution,interstitial}`

Defect type to introduce. required; choices: vacancy, substitution, interstitial

`--index INDEX`

Comma-separated 1-indexed atom(s) to remove/substitute, e.g. '3,7'. exclusive group 1

`--nearest X Y Z`

Selects the atom closest to this position (vacancy/substitution). type: float; exclusive group 1

`--all-inequivalent-sites`

Generate one output structure per symmetrically distinct site (via spglib), instead of a single combined structure – e.g. to compare defect formation energies across all inequivalent sites rather than picking one by hand. Writes '`<output>_site<N>.fdf`' per site instead of a single `--output` file. Narrow to one species with `--filter-species`. switch, off by default; exclusive group 1

`--nearest-format {fractional,cartesian}`

How to interpret `--nearest`. Default: fractional. choices: fractional, cartesian

`--filter-species FILTER_SPECIES`

Restrict `--nearest`'s or `--all-inequivalent-sites`' search to atoms of this element.

`--symprec SYMPREC`

Symmetry precision for `--all-inequivalent-sites` and the before/after symmetry table (default: 0.01, pymatgen's own default, matches `stb-symmetry/stb-unitcell`). type: float

`--ml-rank`

Only valid with `--all-inequivalent-sites`. Quickly relaxes each candidate site's local geometry (positions only) with a MACE potential (needs the optional 'ml' extra: pip install stb_suite[ml]), then prints them ranked by relaxed energy – a fast pre-screen for which site is most likely favorable, before spending DFT time on all of them. Overwrites each site's output file with its relaxed geometry. This is a relative comparison from a fast ML potential, not an absolute DFT formation energy. Mutually exclusive with `--ml-relax` (which already relaxes every output structure, just without the ranking table). switch, off by default

`--new-species NEW_SPECIES`

Replacement element symbol (required for `--type` substitution).

`--position X Y Z`

Position of the new atom (required for `--type` interstitial). type: float

`--position-format {fractional,cartesian}`

How to interpret `--position`. Default: fractional. choices: fractional, cartesian

`--species SPECIES`
: Element symbol of the new atom (required for `--type` interstitial).

`--ml-relax`
: Pre-relax every output structure with a MACE potential before writing it out (needs the optional 'ml' extra: pip install stb_suite[ml]) – positions only by default. Off by default. Mutually exclusive with `--ml-rank`.switch, off by default

`--ml-relax-cell`
: With `--ml-relax`, also relax the cell – any vacuum-padded axis always stays exactly fixed. Only valid together with `--ml-relax`. switch, off by default

`--model {small,medium,large}`
: MACE-MP-0 foundation model size for `--ml-relax/--ml-rank` (default: medium). choices: small, medium, large

`--custom-model PATH`
: Path to a custom fine-tuned .model file for `--ml-relax/--ml-rank`, instead of a MACE-MP-0 foundation size.

`--custom-model-card JSON`
: Model card of the `--custom-model` file: the `mlff_model_card.json` that `stb-mlffAnalysis` wrote next to it (training data, level of theory, accuracy, energy scale). Required with `--custom-model`: the card is checked against the model file and used to warn when a structure is outside what the model was trained on.

`--device {cpu,cuda}`
: Device to run the MACE model on (default: cpu). choices: cpu, cuda

`--save-report`
: Also persist the full run report (including the symmetry analysis) to `stb_defect_report.txt`. Off by default. switch, off by default

`--view`
: Open an interactive 3D view (via ASE) comparing the input structure and every output structure (page through frames in ase-gui) after writing the output file(s). Needs a display. Off by default.switch, off by default

`-o, --output OUTPUT`
: Output .fdf file name (default: `defect.fdf`).

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Usage examples:
  stb-defect -f structure.fdf --type vacancy --index 5
  stb-defect -f structure.fdf --type substitution --index 5 --new-species Ge
  stb-defect -f structure.fdf --type vacancy --nearest 0.5 0.5 0.5 --filter-species O
  stb-defect -f structure.fdf --type interstitial --position 0.5 0.5 0.5 --species N
  stb-defect -f structure.fdf --type vacancy --all-inequivalent-sites --filter-species O
  stb-defect -f structure.fdf --type vacancy --index 5 --ml-relax --save-report --view
```

10.6 SQS Generator

At a glance

Menu code: 4.6 **Command:** `stb-sqs`
Guided example: `examples/4.6-stb-sqs/`
Summary: Generate a Special Quasirandom Structure for a substitutional alloy.

10.6.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/4.6-stb-sqs/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

10.6.1.1 What this tool does

Takes a structure with one ordered species and disorders it into a target substitutional alloy composition (e.g. Ni → 50:50 Ni:Fe), searching for the one small periodic cell whose local statistics best approximate the true, infinite random alloy — instead of just placing atoms at random. Same **Structures** category (4) as `4.1-stb-2Dstacking/`, `4.2-stb-supercell/`, `4.3-stb-slab/`, `4.4-stb-nanotube/`, `4.5-stb-defect/` — everything here builds, generates, or transforms a structure file.

10.6.1.2 Why this matters (a bit of theory)

The problem: DFT needs periodicity, real alloys don't have any. A real substitutional alloy or solid solution (NiFe, SiGe, a doped oxide, a high-entropy alloy, ...) has **no periodicity** — each site is occupied by one species or another following the bulk composition, with no repeating pattern. A DFT calculation, on the other hand, needs a finite, periodic cell. The naive fix — randomly assign species to a small cell's sites according to the target composition — doesn't actually solve this: a handful of atoms is nowhere near enough to statistically reproduce a truly random arrangement's **short-range order** (how often a given species finds itself next to another, at each neighbor distance). Two different random draws of the same small cell can behave quite differently from each other, and from the real, infinite random alloy.

Special Quasirandom Structures (SQS). Zunger, Wei, Ferreira & Bernard, Phys. Rev. Lett. 65, 353 (1990) proposed a different approach: instead of a literal random draw, *search* a small periodic cell for the one atomic arrangement whose **correlation functions** — how often pairs, triplets, etc. of sites at given distances are occupied by the same vs. different species — best match those of the ideal, infinitely large random alloy. The resulting cell isn't random at all (it's a specific, deterministic arrangement found by the search), but it's *quasi*-random in the sense that it statistically mimics randomness far better than an actually-random draw of the same size.

`stb-sqs` wraps icet's implementation of this search (via pymatgen's `SQSTransformation`), using the objective function described by **van de Walle et al., Calphad 42, 13 (2013)** — the same measure used internally by icet's own `generate_sqs`. Concretely, icet minimizes

```
Q = -w*L + sum_alpha |correlation_alpha - target_correlation_alpha|
```

where the sum runs over every cluster correlation the search is asked to match (see `--cluster-cutoffs` below), and `-w*L` is a bonus term that rewards a *perfect* match extending out to a larger cluster radius `w`. **More negative is better** — a lower (more negative) `Q` means a closer match to the ideal random alloy's correlation functions. `Q` is what the tool's `[5] SQS SEARCH` section reports as "Objective function".

Two search methods, cross-checked live in this example.

- `--method monte_carlo` (default) — simulated annealing: cheap, scales to large cells, but not guaranteed to find the true optimum (a stochastic search can get stuck in a local one).
- `--method enumeration` — exhaustively checks every distinct arrangement at the given cell size. Guaranteed to find the **true** global optimum, but only feasible for small cells (the number of arrangements grows combinatorially with size).

`enumeration/` in this walkthrough runs the exact same problem as `monte-carlo/` and compares the two objective functions live — a genuine cross-check that Monte Carlo really found the global optimum, not just a locally-good one.

Not every cell size fits a composition exactly. A target composition needs an **exact integer atom count per species** at whatever cell size is searched. icet auto-detects the smallest `--scaling` that allows this (`[4] SUPERCELL SIZING`, "Minimal valid scaling") from its own auto-detected primitive cell; any `--scaling` that isn't a multiple of it is rejected up front with a clear message, rather than letting icet's own search silently find zero candidates. `scaling-constraint/` below demonstrates both the rejection and the fix.

`--cluster-cutoffs`: more correlations to match = a harder search. `--cluster-cutoffs` (default icet's own `2:3, 3:2, 4:1` — pair clusters out to the 3rd neighbor shell, triplets out to the 2nd, quadruplets to the 1st) controls exactly which clusters the search tries to match. Asking for a *wider* set of correlations, with the same small cell, is a strictly harder target: `cluster-cutoffs/` below shows the objective function get measurably worse (and the search take measurably longer) when the default cutoffs are widened, on the exact same problem.

A real disordering breaks symmetry substantially. Same theme as introducing a point defect (`4.5-stb-defect/`): a perfectly ordered crystal's high symmetry comes from every site's environment being equivalent under the crystal's own symmetry operations. Disordering even a *single* species into two breaks nearly all of that — `[8] SYMMETRY ANALYSIS` compares the ordered input structure against the final SQS structure and shows this directly (verified live: FCC Ni's cubic Fm-3m, No. 225, 48 operations, routinely drops to a low-symmetry monoclinic or trigonal space group after disordering into Ni-Fe).

`--sublattice` only disorders the species you name. A structure can already have more than one species (e.g. an oxide). `--sublattice` targets exactly one of them; every other species keeps its role in the crystal. `sublattice-mgo/` below verifies this on a real (Mg,Ni)O rocksalt solid solution (a system studied for catalysis and electrocatalysis): only the Mg sites are disordered with Ni, and although icet rebuilds the cell from its own internally-detected primitive cell (so the O atoms' exact coordinates/ordering can come out different from the input — the same "equivalent, not identical" caveat noted for spglib-based tools elsewhere in this suite, e.g. `core/symmetry.py`), the underlying rocksalt bonding geometry survives: the cation-O nearest-neighbor distance comes out *exactly* unchanged.

`--ml-relax`: an SQS cell's local environment is inherently irregular. Unlike a perfect crystal, an SQS cell's atoms genuinely sit at slightly different local equilibrium positions depending on which species their neighbors happen to be. `--ml-relax` (+ `--ml-relax-cell`) pre-relaxes the found SQS structure with a MACE potential before writing it out — the same convention `stb-supercell`/`stb-slab`/`stb-nanotube`/`stb-defect` already use.

Known limitation: practically 2 species per sublattice. `--composition` with 3+ species crashes during the search itself (`ASE Atoms only supports ordered structures`) — a **verified upstream issue** in the installed pymatgen's own `SQSTransformation`/`IcetSQS` wrapper (`pymatgen/io/icet.py`), which builds its internal reference structure the same way this tool's own `minimal_scaling()` used to (fixed in this same session — see `--help` for `--composition`), but pymatgen's own copy of that pattern is out of `stb-sqs`'s reach to fix without bypassing `SQSTransformation` entirely. Not exercised by this example.

10.6.1.3 When you'd reach for it

- Building a DFT-ready cell for a substitutional alloy or solid solution (metallic alloys, doped oxides/semiconductors, ...) instead of a single hand-picked ordered arrangement.
- Comparing how a specific composition's local order changes the physics (relative to the pure end-member) in a subsequent DFT or `stb-ml*` calculation.
- Screening a composition ratio before committing to real DFT, by first checking `--ml-relax`'s MACE energy on the SQS cell.

10.6.1.4 Two ways to run it

A — direct CLI:

```
stb-sqs -f fcc_ni.fdf --sublattice Ni --composition Ni:0.5,Fe:0.5
```

B — interactive `stb-suite` menu:

```
$ stb-suite
Select an option (0-7, or a tool code like 5.1.2): 4.6
```

Both paths call the exact same underlying tool and produce the exact same output — `example_4.6.sh` proves this directly at the end.

10.6.1.5 What every run does (always on)

- **A numbered report** (`[0] RUN METADATA ... [11] SUMMARY & FILES` — `[6]` only appears with `--ml-relax`) printed to the console and, with `--save-report`, also saved to `stb_sqs_report.txt`. Earlier versions of this tool always wrote a separate `sqs_report.txt`; that file is gone — everything it used to contain is now part of this same numbered report.
- **Structure validation** — atom proximity, lattice handedness, atomic density — run once on the input structure and once on the final SQS structure.
- **A before/after symmetry comparison table** — crystal system, 3D space group, layer group, point group, Hall symbol — comparing the ordered input structure against the final (disordered) SQS structure.
- **`references.bib`** — SIESTA always, plus icet (the SQS search library itself), plus the MACE papers if `--ml-relax` was used.
- **A provenance header** written into the output `.fdf`: the disordered sublattice/composition, the scaling used, the SQS search method and objective function, and MACE convergence/energy detail if used.

10.6.1.6 Optional (off by default)

- **`--scaling N`** — override icet's auto-detected minimal supercell size (must be an integer multiple of it).
- **`--method {monte_carlo,enumeration}`** — simulated annealing (default) or exhaustive search.
- **`--instances N`** — run N independent Monte Carlo searches in parallel and keep the best (a second global-optimum cross-check).
- **`--temperature`**, **`--mc-steps`** — Monte Carlo search settings.

- **`--cluster-cutoffs SIZE:SHELL,...`** — which correlation functions the search tries to match (default icet's own `2:3, 3:2, 4:1`).
- **`--ml-relax`** (+ `--ml-relax-cell`, `--model`/`--custom-model`) — pre-relax the found SQS structure generically with MACE.
- **`--save-report`** — also persist the full report to `stb_sqs_report.txt`.
- **`--view`** — opens the input structure and the final SQS structure in ASE's interactive 3D viewer (`ase-gui`), as pageable frames. Needs a local display. Never exercised by `example_4.6.sh` itself.

10.6.1.7 Files in this folder

- `fcc_ni.fdf` — bulk FCC nickel, the conventional 4-atom cell (`a` = 3.52 Å, space group Fm-3m No. 225) — a single-species metal, disordered into a binary Ni-Fe alloy by most cases below.
- `mgo_rocksalt.fdf` — MgO, rocksalt (NaCl-type) structure, conventional 8-atom cell (`a` = 4.246 Å, also Fm-3m No. 225 — a deliberate parallel with `fcc_ni.fdf`) — already has two species, used to show `--sublattice` disordering only one of them.
- `example_4.6.sh` — the guided walkthrough (see below).
- `.gitignore` — excludes `output/`, `references.bib`, and `stb_sqs_report.txt`.

10.6.1.8 Running the walkthrough

```
cd examples/4.6-stb-sqs
./example_4.6.sh
```

It pauses between sections (`[Press Enter to continue]`) and always starts by wiping its own `output/`. Eight self-contained cases are generated (one is skipped, with a hint, if the optional `ml` extra isn't installed):

Folder	What it shows
`monte-carlo/`	The basic SQS search and its objective function
`enumeration/`	Exact search, cross-checked live against Monte Carlo's result
`scaling-constraint/`	A `--scaling` incompatible with the composition, rejected then fixed
`cluster-cutoffs/`	Wider correlation targets make the same search measurably harder
`instances/`	Parallel independent searches, a second global-optimum cross-check
`sublattice-mgo/`	(Mg,Ni)O: disordering only Mg, verifying O's bonding geometry survives
`ml-relax/`	Generic MACE pre-relaxation of the found SQS structure
`full-report/`	`--save-report` + the validation checklist + `references.bib`

10.6.1.9 Try it yourself

```
# Show the input structure and the final SQS structure side by side
stb-sqs -f fcc_ni.fdf --sublattice Ni --composition Ni:0.5,Fe:0.5 --view

# A different composition ratio
stb-sqs -f fcc_ni.fdf --sublattice Ni --composition Ni:0.25,Fe:0.75

# Pre-relax with MACE, cell included, with your own fine-tuned model
stb-sqs -f fcc_ni.fdf --sublattice Ni --composition Ni:0.5,Fe:0.5 --ml-relax --ml-relax-cell
--custom-model my_finetuned.model \
    --custom-model-card mlff_model_card.json

# A larger, more dilute SQS: build a bigger cell first
stb-supercell -f fcc_ni.fdf -d 2 2 2 -o ni32.fdf --no-intro
```

```
stb-sqs -f ni32.fdf --sublattice Ni --composition Ni:0.5,Fe:0.5
```

10.6.1.10 Flag reference

Flag	Meaning
`-f/--file`	Input structure file, `.fdf` (required)
`--sublattice`	Existing species whose sites become the disordered alloy sublattice (required)
`--composition`	Target composition on that sublattice, e.g. `Fe:0.5,Ni:0.5` (must sum to 1.0; practically 2 species, see limitation above)
`--scaling`	icet supercell-size multiplier (default: auto-detect the minimal valid one)
`--method`	`monte_carlo` (default) or `enumeration`
`--instances`	Parallel search instances (default 1)
`--temperature`	Monte Carlo starting temperature (default 1.0)
`--mc-steps`	Caps the number of Monte Carlo steps (`--method monte_carlo` only)
`--cluster-cutoffs`	`size:shell` pairs, e.g. `2:3,3:2` (default icet's own `2:3, 3:2, 4:1`)
`-sp/--symprec`	Symmetry tolerance for the before/after table (default `0.01`)
`--ml-relax`	Pre-relax the SQS structure found with a MACE potential
`--ml-relax-cell`	With `--ml-relax`, also relax the cell (vacuum axes stay fixed)
`--model`	MACE-MP-0 size for `--ml-relax`: `small/medium/large` (default `small`)
`--custom-model`	Path to a custom fine-tuned `.model` file
`-o/--output`	Output `.fdf` file name (default `sqs.fdf`)
`--save-report`	Persist the full report (incl. symmetry table) to disk
`--view`	Open the input and final SQS structure in ASE's interactive viewer

10.6.1.11 What's next

See `4.2-stb-supercell/` for building a bigger, more dilute cell before running an SQS search on it, `4.5-stb-defect/` for another example of a single missing/substituted atom breaking symmetry substantially, or `1.7-stb-mlrelax/` for a closer look at the MACE pre-relaxation `--ml-relax` reuses here.

10.6.2 Implementation details

Special quasirandom structures

A random alloy $A_{1-x}B_x$ on a lattice is characterized by its correlation functions: for a cluster figure f (pair at the k-th neighbour distance, triplet, …), $\Pi_f = \langle \sigma_{i_1}\sigma_{i_2}\cdots\rangle_f$, the average product of spin-like occupation variables $\sigma = \pm 1$. In the infinite random alloy, $\Pi_f = (2x-1)^{|f|}$. An SQS [60] is the periodic cell of a given size whose correlations of the shortest figures are closest to these random values, so it reproduces the local environments of the disordered alloy as well as a cell of that size can. The search is done by icet [13] through pymatgen's `SQSTransformation`.

Input.. `-sublattice` names the element of the input structure whose sites are disordered (the others are kept as they are, so a pseudo-binary such as $In_{1-x}Ga_xAs$ is built from GaAs by disordering only the cation sublattice), and `-composition` gives the target, e.g. `In:0.5,Ga:0.5`, summing to 1. In practice only two species per sublattice work (a limitation of the pymatgen/icet interface that the help text explains).

Cell size.. icet works on the primitive cell it detects, which may be smaller than the file you give (a 4-atom fcc cell is a 1-atom primitive). `-scaling` multiplies that primitive cell; it must make every species' count an integer, so the tool computes the smallest valid scaling, reports it, and rejects incompatible values with an explanation instead of letting the search fail obscurely.

Search..

`-method monte_carlo` (default)
simulated annealing on the occupations, from `-temperature` (default 1.0, in icet's units of the objective), capped by `-mc-steps`; `-instances` runs independent searches in parallel and keeps the best.

`-method enumeration`
exhaustive enumeration of all inequivalent cells of that size (Hart–Forcade derivative structures [61]); exact but feasible only for small cells.

`-cluster-cutoffs` sets which correlations are matched as `size:neighbour-shell` pairs (default `2:3,3:2,4:1`: pairs up to the third shell, triplets up to the second, quadruplets in the first). Matching more figures is harder and needs larger cells.

Report and output.. The report shows the composition reached, the objective value of the best cell, and the symmetry before and after (an SQS has, by construction, little symmetry). The cell is written to `-o` (default `sqs.fdf`) and can be pre-relaxed with MACE (`-ml-relax`): the local environments of an SQS are irregular, so atomic relaxation matters for its energy. Average properties over two or three independent SQS of the same size when the property is sensitive to the specific arrangement; for the configurational thermodynamics of the alloy, see GQCA (**5.13**).

10.6.3 Command-line reference

10.6.3.1 `stb-sqs`

Generates a Special Quasirandom Structure (SQS) for a substitutional alloy. Picks one sublattice (`--sublattice`) and disorders it with a target composition, then searches for the atomic arrangement that best mimics an ideal random alloy (via icet's Monte Carlo or enumeration SQS search).

Synopsis.

```
usage: stb-sqs [-h] -f FILENAME --sublattice SUBLATTICE --composition COMPOSITION
               [--scaling SCALING] [--method {monte_carlo,enumeration}]
               [--instances INSTANCES] [--temperature TEMPERATURE]
               [--mc-steps MC_STEPS] [--cluster-cutoffs CLUSTER_CUTOFFS] [-sp SYMPREC]
               [--ml-relax] [--ml-relax-cell] [--model {small,medium,large}]
               [--custom-model PATH] [--custom-model-card JSON] [--device {cpu,cuda}]
               [--save-report] [--view] [-o OUTPUT] [-v] [--no-intro]
```

Options

`-f, --file FILENAME`
Path to the input structure file (.fdf). required

`--sublattice SUBLATTICE`
Existing species whose sites become the disordered alloy sublattice. required

`--composition COMPOSITION`
Target composition on that sublattice, e.g. 'Fe:0.5,Ni:0.5' (must sum to 1.0). Practically limited to 2 species: pymatgen's own SQSTransformation/IcetSQS wrapper (pymatgen/io/icet.py) builds its internal reference structure the same way this tool's minimal_scaling() used to (a replace_species() call that leaves a site's occupancy dict merged rather than collapsed for 3+ species), and raises 'ASE Atoms only supports ordered structures' during the search itself for a 3+ `-species` composition – verified upstream in the installed pymatgen, not something this tool can work around without bypassing SQSTransformation entirely. required

`--scaling SCALING`

icet supercell-size control (positive integer). Multiplies icet's own internally-detected primitive unit, not necessarily the literal input atom count – read the actual resulting atom count from the output. Default: auto-detect the smallest scaling that gives every species in `--composition` an exact atom count (a `--scaling` that doesn't divide evenly is rejected, since icet would otherwise fail to find any candidate structure at that size). type: int

`--method {monte_carlo,enumeration}`

SQS search method (default: monte_carlo). choices: monte_carlo, enumeration

`--instances INSTANCES`

Number of parallel search instances/processes (default: 1, serial). For `--method` enumeration, icet's own enumeration of candidate structures is single `-threaded` regardless of this value – only the (comparatively cheap) scoring of already-generated candidates is split across instances, so raising it rarely helps. For monte_carlo, each instance runs an independent full search and the best is kept, which can help on a machine with real spare cores – but every instance re-pays icet/pymatgen's startup cost (imports, primitive-cell detection via spglib), so on a loaded or virtualized machine more instances can end up *slower* in wall time despite using more CPU overall. Benchmark before raising this; don't assume it helps. type: int

`--temperature TEMPERATURE`

Monte Carlo starting temperature (default: 1.0). type: float

`--mc-steps MC_STEPS`

Caps the number of Monte Carlo steps (`--method` monte_carlo only). Default: icet's own default. type: int

`--cluster-cutoffs CLUSTER_CUTOFFS`

Cluster size:shell pairs, e.g. '2:3,3:2' (default: icet's own {2:3, 3:2, 4:1}).

`-sp, --symprec SYMPREC`

Symmetry tolerance in Angstroms for the before/after symmetry analysis (default: 0.01). type: float

`--ml-relax`

Pre-relax the SQS structure found with a MACE potential (needs the optional 'ml' extra: pip install stb_suite[ml]) before writing it out – positions only by default. Off by default. switch, off by default

`--ml-relax-cell`

With `--ml-relax`, also relax the cell – any vacuum-padded axis always stays exactly fixed. Only valid together with `--ml-relax`. switch, off by default

`--model {small,medium,large}`

MACE-MP-0 foundation model size for `--ml-relax` (default: medium). choices: small, medium, large

`--custom-model PATH`

Path to a custom fine-tuned .model file for `--ml-relax`, instead of a MACE-MP-0 foundation size.

`--custom-model-card JSON`

Model card of the `--custom-model` file: the `mlff_model_card.json` that `stb-mlffAnalysis` wrote next to it (training data, level of theory, accuracy, energy scale). Required with `--custom-model`: the card is checked against the model file and used to warn when a structure is outside what the model was trained on.

`--device {cpu,cuda}`

Device to run the MACE model on (default: cpu). choices: cpu, cuda

`--save-report`

Also persist the full run report (including the symmetry analysis) to `stb_sqs_report.txt`. Off by default.switch, off by default

`--view`

Open an interactive 3D view (via ASE) comparing the input structure and the final SQS structure (page through frames in ase-gui) after writing the output file. Needs a display. Off by default. switch, off by default

`-o, --output OUTPUT`

Output .fdf file name (default: `sqs.fdf`).

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Usage examples:
  stb-sqs -f ni.fdf --sublattice Ni --composition Ni:0.5,Fe:0.5
  stb-sqs -f ni.fdf --sublattice Ni --composition Ni:0.75,Fe:0.25 --scaling 8 --method enumeration
  stb-sqs -f ni.fdf --sublattice Ni --composition Ni:0.5,Fe:0.5 --ml-relax --save-report --view
```

10.7 Unit Cell Finder

At a glance

Menu code: 4.7 **Command:** `stb-unitcell`
Guided example: `examples/4.7-stb-unitcell/`
Summary: Find the primitive or conventional unit cell of a structure.

10.7.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/4.7-stb-unitcell/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

10.7.1.1 What this tool does

Detects a structure's true crystal symmetry (via pymatgen's symmetry analyzer, spglib) and rebuilds it as the **primitive** (smallest possible), **conventional** (standardized, usually larger), or **refined** (conventional-sized, positions snapped exactly onto the detected symmetry) cell. Same **Structures** category (4) as `4.1-stb-2Dstacking/`, `4.2-stb-supercell/`, ..., `4.6-stb-sqs/` — everything here builds, generates, or transforms a structure file. Unlike most of its siblings, this tool doesn't change the crystal at all — it only changes *which* repeating cell describes it.

10.7.1.2 Why this matters (a bit of theory)

One crystal, several valid cells. A crystal's infinite periodicity can be described by more than one repeating cell, and all of them describe the *exact same* structure — they're just different choices of "the box that tiles to make it":

- **Primitive** — the smallest possible repeating unit. Fewer atoms means a cheaper DFT calculation: fewer basis functions, a smaller Hamiltonian, faster SCF — the natural choice for a real calculation, especially phonon or band-structure work where the cost scales with atom count per k-point.
- **Conventional** — a larger, standardized cell following the International Tables / IUCr convention. Easier for a human to read and compare against a database entry (most databases, e.g. Materials Project or ICSD, store this one) — at the cost of more atoms than strictly necessary for a calculation.

- **Refined** — conventional-sized, but with every atomic position snapped exactly onto the detected symmetry. Doesn't change which atoms are present or the cell size — it only cleans up numerical noise (e.g. from a DFT relaxation, or a hand-typed/CIF-derived structure) that keeps a structure just barely off its true, exact symmetry.

A real reduction, and a real surprise: cell shape ≠ crystal system. `primitive-fcc/` below reduces FCC nickel's 4-atom conventional cubic cell down to its true 1-atom primitive cell — and that primitive cell's own lattice vectors turn out to have 60° angles between them (a rhombohedral-*looking* cell), even though the crystal **system** is still reported as cubic. This isn't a contradiction: the crystal system comes from the full set of symmetry operations the space group has (cubic Fm-3m, No. 225 either way), not from how any *one* particular choice of repeating cell happens to look geometrically. `primitive-rocksalt/` verifies the same reduction preserves a compound's exact stoichiometric ratio too (NaCl's 4:4 Na:Cl → 1:1), not just a single-species metal's atom count.

Reduction is symmetry-*preserving* — the tool's own before/after table proves it. Unlike `4.2-stb-supercell/`, `4.5-stb-defect/`, or `4.6-stb-sqs/` (which all deliberately *change* a structure's symmetry), `stb-unitcell`'s job is the opposite: find a *different cell for the same symmetry*. Every run now prints the identical `[7] SYMMETRY ANALYSIS (BEFORE / AFTER)` table those other tools use — but here it's a **correctness check**: Before and After are *expected* to match exactly, and the report says so explicitly when they do (verified live in every case below).

`--symprec`: a real, dramatic consequence, not just a numeric knob. `--symprec` sets the numerical tolerance spglib uses to decide whether two positions count as symmetry-equivalent. `symprec-sensitivity/` below runs the exact same (deliberately slightly noisy) structure through two tolerances: the default (`1e-3`, looser than the ~2×10^{-5} Å noise) sees straight through it to the true Fm-3m symmetry and reduces the cell; a tolerance *tighter* than the noise itself (`1e-8`) sees only genuine, literal asymmetry — space group P1, no symmetry at all — and correctly refuses to reduce anything, since there's nothing it can prove is redundant at that tolerance. Same atoms, same physical structure, two completely different answers, purely from the tolerance.

A real bug found and fixed while building this example: the structure-validation sections' own space-group line used to ignore `--symprec` entirely (always defaulting to `1e-3` internally), while the symmetry-detection and before/after sections correctly used whatever `--symprec` was requested. At a tight `--symprec`, this meant the *same run* could show `Fm-3m` in one section and `P1` in another — confusing, and specifically misleading for this tool, since `--symprec` is not an auxiliary setting here (as it is for `4.2/4.5/4.6`'s own before/after tables) but the primary parameter controlling what "the primitive cell" even means. Fixed by threading `--symprec` through the validation sections too; verified live that all sections now agree.

The vacuum-axis and origin/relabeling caveats. This tool operates on the literal 3D periodic cell as given — it has no special vacuum-axis awareness the way `stb-slab`/`stb-supercell` do, so a 2D/slab-type input gets an explicit `[WARNING]` (`vacuum-warning/` below). The vacuum thickness itself is preserved, but — as documented for spglib-based tools elsewhere in this suite (`core/symmetry.py`) — the in-plane lattice vectors and atom ordering are **not** guaranteed to match the input in any mode: spglib rebuilds the cell from the detected symmetry operations from scratch, free to pick any symmetry-equivalent origin/ordering. `vacuum-warning/` shows this directly: the output's in-plane vectors come back re-expressed differently from the input, even though it's the exact same physical lattice.

10.7.1.3 When you'd reach for it

- Switching to the primitive cell before a real DFT calculation, to cut the atom count (and cost) without changing the physics.
- Getting the standardized conventional cell for comparison against a database entry or a paper.
- Cleaning up a relaxed or hand-built/CIF-derived structure's numerical noise (`--mode refined`) before a symmetry-sensitive analysis (e.g. group-theory-based phonon selection rules).

10.7.1.4 Two ways to run it

A — direct CLI:

```
stb-unitcell -f fcc_ni.fdf --mode primitive
```

B — interactive `stb-suite` menu:

```
$ stb-suite
Select an option (0-7, or a tool code like 5.1.2): 4.7
```

Both paths call the exact same underlying tool and produce the exact same output — `example_4.7.sh` proves this directly at the end.

10.7.1.5 What every run does (always on)

- **A numbered report** (`[0] RUN METADATA ... [10] SUMMARY & FILES` — `[5]` only appears with `--ml-relax`) printed to the console and, with `--save-report`, also saved to `stb_unitcell_report.txt`. Earlier versions of this tool always wrote a separate `unitcell_report.txt`; that file is gone — everything it used to contain is now part of this same numbered report.
- **Structure validation** — atom proximity, lattice handedness, atomic density — run once on the input structure and once on the reduced cell, both using the same `--symprec` as the reduction itself.
- **A before/after symmetry comparison table** — crystal system, 3D space group, layer group, point group, Hall symbol — expected (and verified) to match exactly, since cell reduction doesn't change the crystal.
- **`references.bib`** — SIESTA always, plus the MACE papers if `--ml-relax` was used.
- **A provenance header** written into the output `.fdf`: the mode used, the detected space group, the reduction factor, and MACE convergence/energy detail if used.

10.7.1.6 Optional (off by default)

- **`--mode {primitive,conventional,refined}`** — which cell to build (default `primitive`).
- **`--symprec`** / **`--angle-tolerance`** — the symmetry-detection tolerance, used both for the reduction itself and the before/after table.
- **`--ml-relax`** (+ `--ml-relax-cell`, `--model`/`--custom-model`) — pre-relax the reduced cell generically with MACE.
- **`--save-report`** — also persist the full report to `stb_unitcell_report.txt`.
- **`--view`** — opens the input structure and the reduced cell in ASE's interactive 3D viewer (`ase-gui`), as pageable frames. Needs a local display. Never exercised by `example_4.7.sh` itself.

10.7.1.7 Files in this folder

- `fcc_ni.fdf` — bulk FCC nickel, the conventional 4-atom cell (a = 3.52 Å, space group Fm-3m No. 225) — the classic textbook conventional-vs-primitive case.
- `nacl_rocksalt.fdf` — NaCl, rocksalt structure, conventional 8-atom cell (a = 5.64 Å, also Fm-3m No. 225 — a deliberate parallel with `fcc_ni.fdf`) — two species, used to verify the reduction preserves their exact ratio.
- `nacl_noisy.fdf` — the same NaCl crystal with a ~2×10^{-5} Å asymmetric perturbation on every position, for the `refined-noise/` and `symprec-sensitivity/` cases.
- `graphene.fdf` — 2D monolayer graphene (vacuum along c), for the `vacuum-warning/` case.
- `example_4.7.sh` — the guided walkthrough (see below).
- `.gitignore` — excludes `output/`, `references.bib`, and `stb_unitcell_report.txt`.

10.7.1.8 Running the walkthrough

```
cd examples/4.7-stb-unitcell
./example_4.7.sh
```

It pauses between sections (`[Press Enter to continue]`) and always starts by wiping its own `output/`. Eight self-contained cases are generated (one is skipped, with a hint, if the optional `ml` extra isn't installed):

Folder	What it shows
`primitive-fcc/`	FCC Ni's 4→1 reduction, and its 60° rhombohedral primitive cell
`primitive-rocksalt/`	NaCl's 8→2 reduction preserving the exact 1:1 species ratio
`conventional-noop/`	A legitimate no-op, clearly flagged instead of silently doing nothing
`refined-noise/`	Noisy positions snapped back to their exact symmetry-consistent values
`symprec-sensitivity/`	The same noisy structure at two tolerances: Fm-3m vs. P1
`vacuum-warning/`	The vacuum-axis caveat and the origin/relabeling caveat on graphene
`ml-relax/`	Generic MACE pre-relaxation of the reduced cell
`full-report/`	`--save-report` + the validation checklist + `references.bib`

10.7.1.9 Try it yourself

```
# Show the input structure and the reduced cell side by side
stb-unitcell -f fcc_ni.fdf --mode primitive --view

# The standardized conventional cell, for comparison against a database entry
stb-unitcell -f nacl_rocksalt.fdf --mode conventional

# Pre-relax with MACE, cell included, with your own fine-tuned model
stb-unitcell -f fcc_ni.fdf --ml-relax --ml-relax-cell --custom-model my_finetuned.model \
    --custom-model-card mlff_model_card.json

# Clean up a relaxed structure's numerical noise before a symmetry-sensitive analysis
stb-unitcell -f your_relaxed_structure.fdf --mode refined
```

10.7.1.10 Flag reference

Flag	Meaning
`-f/--file`	Input structure file, `.fdf` (required)
`--mode`	`primitive` (default) / `conventional` / `refined`
`--symprec`	Symmetry precision, default `1e-3` (used for both the reduction and the before/after table)
`--angle-tolerance`	Symmetry angle tolerance in degrees, default `5.0`
`--ml-relax`	Pre-relax the reduced cell with a MACE potential
`--ml-relax-cell`	With `--ml-relax`, also relax the cell (vacuum axes stay fixed)
`--model`	MACE-MP-0 size for `--ml-relax`: `small`/`medium`/`large` (default `small`)
`--custom-model`	Path to a custom fine-tuned `.model` file
`-o/--output`	Output `.fdf` file name (default `unitcell.fdf`)
`--save-report`	Persist the full report (incl. symmetry table) to disk
`--view`	Open the input and reduced-cell structure in ASE's interactive viewer

10.7.1.11 What's next

See `4.2-stb-supercell/` for the opposite direction (building a larger cell from a smaller one), or `1.7-stb-mlrelax/` for a closer look at the MACE pre-relaxation `--ml-relax` reuses here.

10.7.2 Implementation details

The three modes.. The cell is analysed with spglib [9] through pymatgen's `SpacegroupAnalyzer`, at `-symprec` (Å, default 0.01) and `-angle-tolerance` (degrees, default 5), and replaced by:

`primitive` (default)
: the standard primitive cell – the smallest cell with the full translational symmetry, in the setting of Setyawan and Curtarolo [35];

`conventional`
: the standard conventional cell of the space group (cubic for fcc and bcc, hexagonal for rhombohedral lattices, …);

`refined`
: the conventional cell with atomic positions symmetrized to the detected group – the way to remove the small numerical noise a relaxation or a CIF leaves, without changing which atoms are present.

The atom count, volume, space group, point group and crystal system are tabulated before and after; a primitive or conventional reduction never changes the space group.

Tolerance matters.. `-symprec` decides how far atoms may be from exact positions and still count as symmetric. A loose value can "find" a higher symmetry that the structure only approximately has and return a cell that is no longer the same structure; a tight value can miss a real symmetry in a relaxed structure. The guide shows a dramatic example. Read the detected space group before trusting the result.

Order and origin are not preserved.. spglib rebuilds the cell from the symmetry operations, so atom order and the choice of origin can change in every mode. The crystal is the same, relabelled: do not rely on atom indices across a reduction (for example to set constraints or pick a defect site), and recompute anything index-based afterwards.

Bulk only.. Space groups describe 3D-periodic crystals. For a slab or wire the vacuum axis would be treated as a periodic direction; the tool warns, and **`stb-symmetry`** (2.5) gives the layer or rod group instead.

10.7.3 Command-line reference

10.7.3.1 `stb-unitcell`

Finds the primitive/conventional unit cell of a structure, or refines it. Uses pymatgen's symmetry analyzer (spglib) to detect the crystal's true unit cell, which may be much smaller than the literal input cell (e.g. a 4-atom conventional FCC cell reduces to a 1-atom primitive cell).

Synopsis.

```
usage: stb-unitcell [-h] -f FILENAME [--mode {primitive,conventional,refined}]
                    [--symprec SYMPREC] [--angle-tolerance ANGLE_TOLERANCE]
                    [--ml-relax] [--ml-relax-cell] [--model {small,medium,large}]
                    [--custom-model PATH] [--custom-model-card JSON]
                    [--device {cpu,cuda}] [--save-report] [--view] [-o OUTPUT] [-v]
                    [--no-intro]
```

Options

`-f, --file FILENAME`
Path to the input structure file (.fdf). required

`--mode {primitive,conventional,refined}`
primitive: smallest possible cell (default).
conventional: standardized, usually larger, cell.
refined: conventional cell with atomic positions snapped to the detected symmetry – cleans up numerical noise from relaxations or hand-built/CIF structures without changing which atoms are present. choices: primitive, conventional, refined

`--symprec SYMPREC`
Symmetry precision (default: 0.01, pymatgen's own default, matches `stb-symmetry`). Also used as the tolerance for the before/after symmetry analysis below. Loosen further (e.g. 0.02-0.05) for a structure relaxed with a looser force tolerance – a tighter value than the relaxation's own residual numerical noise can misdetect the true space group. type: float

`--angle-tolerance ANGLE_TOLERANCE`
Symmetry angle tolerance in degrees (default: 5.0, pymatgen's own default). type: float

`--ml-relax`
Pre-relax the reduced unit cell with a MACE potential (needs the optional 'ml' extra: pip install stb_suite[ml]) before writing it out – positions only by default. Off by default. switch, off by default

`--ml-relax-cell`
With `--ml-relax`, also relax the cell – any vacuum-padded axis always stays exactly fixed. Only valid together with `--ml-relax`. switch, off by default

`--model {small,medium,large}`
MACE-MP-0 foundation model size for `--ml-relax` (default: medium). choices: small, medium, large

`--custom-model PATH`
Path to a custom fine-tuned .model file for `--ml-relax`, instead of a MACE-MP-0 foundation size.

`--custom-model-card JSON`
: Model card of the `--custom-model` file: the `mlff_model_card.json` that `stb-mlffAnalysis` wrote next to it (training data, level of theory, accuracy, energy scale). Required with `--custom-model`: the card is checked against the model file and used to warn when a structure is outside what the model was trained on.

`--device {cpu,cuda}`
: Device to run the MACE model on (default: cpu). choices: cpu, cuda

`--save-report`
: Also persist the full run report (including the symmetry analysis) to `stb_unitcell_report.txt`. Off by default. switch, off by default

`--view`
: Open an interactive 3D view (via ASE) comparing the input structure and the final reduced cell (page through frames in ase-gui) after writing the output file. Needs a display. Off by default. switch, off by default

`-o, --output OUTPUT`
: Output .fdf file name (default: `unitcell.fdf`).

Like every STB command it also accepts `-h`/`--help`, `-v`/`--version` and `--no-intro` (Section 3.3).

From the built-in help.

```
Usage examples:
  stb-unitcell -f structure.fdf
  stb-unitcell -f structure.fdf --mode conventional -o conventional.fdf
  stb-unitcell -f structure.fdf --mode refined -o clean.fdf
  stb-unitcell -f structure.fdf --ml-relax --save-report --view

Notes:
  - This operates on the literal 3D periodic cell as given. For slabs/2D materials/nanotubes with a vacuum
  direction, spot-check that the vacuum axis wasn't reoriented before using the output.
  - The output's atom order and coordinate origin are NOT guaranteed to match the input, in any mode -- spglib
  rebuilds the cell from the detected symmetry operations from scratch, free to pick any symmetry-equivalent
  origin/ordering. For highly symmetric structures (e.g. a monoatomic FCC lattice) even tiny input noise can flip
  which equivalent origin it picks. This is expected: the crystal is the same, just relabeled.
```

10.8 Crystal Builder

At a glance

Menu code: 4.8 **Command:** `stb-crystalbuilder`
Guided example: `examples/4.8-stb-crystalbuilder/`
Summary: Build a structure from a space group and Wyckoff positions.

10.8.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/4.8-stb-crystalbuilder/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

10.8.1.1 What this tool does

Builds a structure **from scratch**, given only a space group and the minimal set of symmetrically-distinct atomic sites (one per Wyckoff position) — pymatgen expands everything else using the space group's own symmetry operations. Same **Structures** category (4) as `4.1-stb-2Dstacking/`, ..., `4.7-stb-unitcell/`, but it's the **inverse** of that last one: `stb-unitcell` takes a full structure and reduces it down to its primitive/conventional cell; `stb-crystalbuilder` takes the minimal symmetry description and builds the full structure up.

10.8.1.2 Is it only for 3D systems? Yes — and this matters.

`--spacegroup` accepts one of the **230 international space groups**, which are the complete classification of symmetry groups for **3D -periodic crystals only**. There is no 2D (layer group) or 1D (rod group) analog here — pymatgen's `Structure.from_spacegroup` (what this tool wraps) is inherently a 3D-crystal builder, and every run's own `[2] STRUCTURE VALIDATION` section now prints this explicitly:

```
Dimensionality : 3D (bulk material) -- every space group is inherently
3D-periodic; use stb-slab/stb-nanotube/stb-2Dstacking on the output to cut
a 2D/1D structure from it.
```

If you want a slab, a nanotube, a monolayer, or a wire, the workflow is **two steps, not one**: build the 3D bulk crystal here, then cut it down with the tool that actually understands vacuum/lower dimensionality:

Want...	Build with `stb-crystalbuilder`, then cut with...
A surface slab	`stb-slab` (`4.3`)
An isolated 2D monolayer	`stb-slab` with a thin enough `--min-slab-size`, or `stb-2Dstacking` (`4.1`) if you're also stacking two layers
A nanotube or nanoribbon	`stb-nanotube` (`4.4`), which itself takes a 2D monolayer as input

`bulk-graphite-to-slab/` below proves this concretely: it builds real bulk graphite with `stb-crystalbuilder` (space group `P6_3/mmc`), then pipes the result straight into `stb-slab` to cut a 2D-periodic slab out of it — `stb-slab`'s own before/after table shows `Layer Group` going from `N/A (not 2D-periodic)` on the bulk input to a real, detected `p-3m1` on the cut slab, confirming the bulk crystal really was 3D-only. (`stb-crystalbuilder`'s own report never shows a `Layer Group` row at all — every space group it builds from is inherently 3D-periodic, so it would always read `N/A`, never actual information.)

10.8.1.3 Why this matters (a bit of theory)

Space groups and Wyckoff positions. A crystal's full symmetry is one of exactly 230 **space groups** — every combination of translations, rotations, mirrors, and screw/glide operations that maps an infinite 3D lattice onto itself. Rather than listing every atom's position by hand, a crystallographer only needs to give the **minimal, symmetrically-distinct set** of atoms (their **Wyckoff positions**) — the space group's own operations generate every other atom in the cell automatically. This is exactly how real crystal structures are published and stored (crystallographic databases like ICSD/COD list a space group + a handful of Wyckoff sites, not a giant explicit atom list) — `--site` mirrors that convention directly: give one entry per Wyckoff letter, not every atom.

Special positions: you can accidentally ask for *more* symmetry than requested. A Wyckoff site can land on a **general position** (no extra local symmetry) or a **special position** (sitting exactly on a mirror, axis, or inversion center of the space group). If your lattice/site combination happens to have

even more symmetry than the space group you asked for allows, the actual structure snaps to that higher symmetry — the tool detects and reports this explicitly rather than silently building something subtly different from what you requested. `special-position -polonium/` below shows a real, physical example of exactly this.

Reduction after construction: primitive vs. conventional, revisited. Space-group construction naturally produces the **conventional** cell (the one an international-tables/database entry actually lists) — often larger than strictly necessary for a DFT calculation. `--reduce {primitive,conventional,refined}` (new in this version) folds in `stb-unitcell`'s own reduction directly, so you can go from "space group + Wyckoff sites" straight to "the cheapest cell for a real calculation" in one command — see `4.7-stb-unitcell/` for the full theory of what each of these three modes means.

10.8.1.4 When you'd reach for it

- You know a crystal's space group and Wyckoff positions (from a paper, International Tables, or a database entry) and want to build the actual atomic structure rather than typing every position by hand.
- Constructing a hypothetical/idealized structure (a specific substitution pattern, a textbook example) that isn't sitting in any database as a downloadable file.
- The starting point for a lower-dimensional structure: build the 3D bulk here, then hand it to `stb-slab`/`stb-nanotube`/`stb-2Dstacking` (see above).

10.8.1.5 Two ways to run it

A — direct CLI:

```
stb-crystalbuilder --spacegroup Fm-3m --a 3.52 --site Ni 0 0 0
```

B — interactive `stb-suite` menu:

```
$ stb-suite
Select an option (0-7, or a tool code like 5.1.2): 4.8
```

Both paths call the exact same underlying tool and produce the exact same output — `example_4.8.sh` proves this directly at the end. Path B prints the bulk-only warning immediately, before asking for the space group, so it's the first thing you see even if you never read `--help`.

10.8.1.6 What every run does (always on)

- **A numbered report** (`[0] RUN METADATA` … `[10] SUMMARY & FILES` — `[4]` only with `--reduce`, `[5]` only with `--ml-relax`) printed to the console and, with `--save-report`, also saved to `stb_crystalbuilder_report.txt`.
- **Structure validation** — atom proximity, lattice handedness, atomic density, plus the dimensionality note above — run once on the raw space-group-expanded structure and once on the final structure (after any `--reduce`/`--ml-relax`).
- **Full symmetry detection** — space group, point group, crystal system, Hall symbol — with an explicit `[NOTE]` whenever the detected space group differs from the one you requested (see "special positions" above).

- **A before/after symmetry comparison table** — crystal system, space group, point group, Hall symbol — expected (and verified) to match exactly, since reduction/relaxation never changes the crystal's actual symmetry. No `Layer Group` row here (unlike `stb-unitcell`/`stb-slab`): every space group this tool builds from is inherently 3D-periodic, so it would always read `N/A (not 2D-periodic)`, never real information.
- **`references.bib`** — SIESTA always, plus the MACE papers if `--ml-relax` was used.
- **A provenance header** written into the output `.fdf`: requested vs. detected space group, lattice, sites, reduce mode and MACE convergence detail if used.

10.8.1.7 Optional (off by default)

- **`--reduce {primitive,conventional,refined}`** — reduce the built structure to a smaller/standardized/symmetry-cleaned cell before writing (reuses `stb-unitcell`'s own reduction).
- **`--symprec`** / **`--angle-tolerance`** — the symmetry-detection tolerance, used for both the post-build symmetry check and `--reduce`.
- **`--ml-relax`** (+ `--ml-relax-cell`, `--model`/`--custom-model`) — pre-relax the final structure with MACE. Since every space group is inherently 3D-periodic, `--ml-relax-cell` here always relaxes all 3 cell axes — no vacuum-axis masking is needed (unlike `stb-unitcell`, which must handle arbitrary user-supplied slabs/wires too).
- **`--save-report`** — also persist the full report to `stb_crystalbuilder_report.txt`.
- **`--view`** — opens the raw built structure and the final structure in ASE's interactive 3D viewer (`ase-gui`), as pageable frames. Needs a local display. Never exercised by `example_4.8.sh` itself.

10.8.1.8 Files in this folder

- `example_4.8.sh` — the guided walkthrough (see below). No input fixtures are needed — unlike most `2-structures` tools, `stb-crystalbuilder` builds everything from `--spacegroup`/`--site` flags, not from a structure file.
- `.gitignore` — excludes `output/`, `references.bib`, and `stb_crystalbuilder_report.txt`.

10.8.1.9 Running the walkthrough

```
cd examples/4.8-stb-crystalbuilder
./example_4.8.sh
```

It pauses between sections (`[Press Enter to continue]`) and always starts by wiping its own `output/`. Nine self-contained cases are generated (one is skipped, with a hint, if the optional `ml` extra isn't installed):

Folder	What it shows
`fcc-nickel/`	The basic case: 1 Wyckoff site -> 4 atoms, symbol/number equivalence
`magnetite-spinel/`	A real oxide with 2 distinct Fe sites + 1 O site -> 56 atoms
`special-position-polonium/`	Requesting less symmetry than the sites actually have -> the `[NOTE]`
`reduce-to-primitive/`	`--reduce primitive` on magnetite: 56 -> 14 atoms, one command
`out-of-range-error/`	The fixed bug: `--spacegroup 255` now gives a clear error message
`ml-relax/`	MACE pre-relaxation of the built + reduced structure
`full-report/`	`--save-report` + the validation checklist + `references.bib`
`bulk-graphite-to-slab/`	**The 3D-only answer, proven**: bulk graphite -> a real 2D slab via `stb-slab`

10.8.1.10 Try it yourself

```
# Build a structure from a space group you found in a paper or database
stb-crystalbuilder --spacegroup Fd-3m --a 8.396 \
    --site Fe 0.125 0.125 0.125 --site Fe 0.5 0.5 0.5 --site O 0.379 0.379 0.379

# Go straight to the cheapest cell for a real DFT calculation
stb-crystalbuilder --spacegroup Fm-3m --a 3.52 --site Ni 0 0 0 --reduce primitive

# Build a 3D bulk crystal, then cut a 2D slab out of it (the 3D-only workaround)
stb-crystalbuilder --spacegroup "P6_3/mmc" --a 2.464 --c 6.711 --gamma 120 \
    --site C 0 0 0.25 --site C 0.333333 0.666667 0.25 -o graphite.fdf
stb-slab -f graphite.fdf --hkl 0 0 1 --min-slab-size 3.0 --min-vacuum-size 15
```

10.8.1.11 Flag reference

Flag	Meaning
`--spacegroup`	International symbol (e.g. `Fm-3m`) or number 1-230 (required)
`--a/--b/--c`	Lattice constants, Ang (`--b`/`--c` default to `--a`) (required `--a`)
`--alpha/--beta/--gamma`	Lattice angles, degrees (default 90)
`--site`	`SYMBOL X Y Z`, repeatable, one per Wyckoff position (required)
`--symprec`	Symmetry precision, default `1e-3` (used for detection and `--reduce`)
`--angle-tolerance`	Symmetry angle tolerance in degrees, default `5.0`
`--reduce`	`primitive/conventional/refined`, off (no reduction) by default
`--ml-relax`	Pre-relax the final structure with a MACE potential
`--ml-relax-cell`	With `--ml-relax`, also relax the cell (all 3 axes)
`--model`	MACE-MP-0 size for `--ml-relax`: `small/medium/large` (default `small`)
`--custom-model`	Path to a custom fine-tuned `.model` file
`-o/--output`	Output `.fdf` file name (default `crystal.fdf`)
`--save-report`	Persist the full report (incl. symmetry table) to disk
`--view`	Open the built and final structures in ASE's interactive viewer

10.8.1.12 What's next

See `4.7-stb-unitcell/` for the inverse operation (reducing a full structure down) and the full theory of primitive/conventional/refined cells that `--reduce` reuses here, or `4.3-stb-slab`/`4.4-stb-nanotube` for what to do next with a bulk crystal built here when you actually need a lower-dimensional structure.

10.8.2 Implementation details

Input.. The space group is given by number (1–230) or Hermann–Mauguin symbol (`Fm-3m`, `P6_3/mmc`). The lattice is given by its parameters: `-a` is required, `-b` and `-c` default to `-a`, and the angles to $90°$. Each `-site El x y z` gives one element at one fractional *representative* position (one per Wyckoff orbit); repeat the option for every inequivalent site. The full cell is generated by applying every operation of the group with pymatgen's `Structure.from_spacegroup`, which also refuses a lattice incompatible with the group's crystal system (for example $a \neq c$ in a cubic group).

Checks.. After building, the symmetry is detected again (`-symprec`, `-angle-tolerance`) and compared with the request. A representative placed on a special position of higher symmetry than intended, or a lattice that happens to be more symmetric, produces a different detected group; the tool reports it as a

`[NOTE]` – the structure is valid, but it is not what you asked for. Two sites whose orbits overlap, or atoms that come out too close, are caught by the structure checklist.

Reduction and relaxation.. `--reduce primitive|conventional|refined` applies the reduction of **`stb-unitcell`** to the built cell, and `--ml-relax` (`--ml-relax-cell`) pre-relaxes it with MACE. The final table compares the requested, built and final symmetry.

Example.. Rock-salt NiO in its conventional cubic cell:

```
stb-crystalbuilder --spacegroup 225 --a 4.17 \
    --site Ni 0 0 0 --site O 0.5 0.5 0.5 -o NiO.fdf
```

A space group always describes a 3D crystal; cut slabs, tubes or heterostructures from the result with the tools of this chapter.

10.8.3 Command-line reference

10.8.3.1 `stb-crystalbuilder`

Builds a structure from a space group and Wyckoff positions. Give only the symmetrically-distinct sites (e.g. one per Wyckoff letter) – pymatgen expands the rest via the space group's own symmetry operations. Bulk (3D) crystals only: the 230 space groups accepted by `--spacegroup` are the international crystallographic groups for 3D-periodic crystals – there is no 2D/1D analog (layer/rod groups) here. To get a 2D or 1D structure, build the 3D bulk crystal with this tool first, then cut it down with `stb-slab` (a surface slab), `stb-nanotube` (rolled into a tube/ribbon), or stb-2Dstacking (an isolated monolayer) – e.g. build bulk graphite (space group P6_3/mmc) here, then `stb-slab --hkl 0 0 1` it into a graphene monolayer.

Synopsis.

```
usage: stb-crystalbuilder [-h] --spacegroup SPACEGROUP --a A [--b B] [--c C]
                          [--alpha ALPHA] [--beta BETA] [--gamma GAMMA] --site SYMBOL
                          X Y Z [--symprec SYMPREC]
                          [--angle-tolerance ANGLE_TOLERANCE]
                          [--reduce {primitive,conventional,refined}] [--ml-relax]
                          [--ml-relax-cell] [--model {small,medium,large}]
                          [--custom-model PATH] [--custom-model-card JSON]
                          [--device {cpu,cuda}] [--save-report] [--view] [-o OUTPUT]
                          [-v] [--no-intro]
```

Options

`--spacegroup SPACEGROUP`
: Space group as an international symbol (e.g. 'Fm-3m') or number (e.g. '225', must be between 1 and 230). required

`--a A`
: Lattice constant a (Ang). required; type: float

`--b B`
: Lattice constant b (Ang). Default: same as `--a`. type: float

`--c C`
: Lattice constant c (Ang). Default: same as `--a`. type: float

`--alpha ALPHA`
Lattice angle alpha, degrees (default: 90). type: float

`--beta BETA`
Lattice angle beta, degrees (default: 90). type: float

`--gamma GAMMA`
Lattice angle gamma, degrees (default: 90). type: float

`--site SYMBOL X Y Z`
One symmetrically-distinct Wyckoff site: element symbol and fractional x y z. Repeat `--site` for each distinct site. required

`--symprec SYMPREC`
Symmetry precision, used both for the post-build symmetry detection and for `--reduce` (default: 0.01, pymatgen's own default, matches `stb-symmetry/stb-unitcell`). type: float

`--angle-tolerance ANGLE_TOLERANCE`
Symmetry angle tolerance in degrees (default: 5.0, pymatgen's own default). type: float

`--reduce {primitive,conventional,refined}`
Reduce the space-group-expanded structure to its primitive/conventional/refined cell (via `stb-unitcell`'s own reduction) before writing. Off by default – the raw space-group-expanded structure is written as-is. choices: primitive, conventional, refined

`--ml-relax`
Pre-relax the final structure with a MACE potential (needs the optional 'ml' extra: pip install stb_suite[ml]) before writing it out – positions only by default. Off by default. switch, off by default

`--ml-relax-cell`
With `--ml-relax`, also relax the cell. Only valid together with `--ml-relax`. switch, off by default

`--model {small,medium,large}`
MACE-MP-0 foundation model size for `--ml-relax` (default: medium). choices: small, medium, large

`--custom-model PATH`
Path to a custom fine-tuned .model file for `--ml-relax`, instead of a MACE-MP-0 foundation size.

`--custom-model-card JSON`
Model card of the `--custom-model` file: the `mlff_model_card.json` that `stb-mlffAnalysis` wrote next to it (training data, level of theory, accuracy, energy scale). Required with `--custom-model`: the card is checked against the model file and used to warn when a structure is outside what the model was trained on.

`--device {cpu,cuda}`
Device to run the MACE model on (default: cpu). choices: cpu, cuda

`--save-report`
Also persist the full run report (including the symmetry analysis) to `stb_crystalbuilder_report.txt`. Off by default. switch, off by default

`--view`
Open an interactive 3D view (via ASE) comparing the raw space-group-expanded structure and the final structure (page through frames in ase-gui) after writing the output file. Needs a display. Off by default.switch, off by default

`-o, --output OUTPUT`
Output .fdf file name (default: `crystal.fdf`).

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Usage examples:
  stb-crystalbuilder --spacegroup Fm-3m --a 3.52 --site Ni 0 0 0
  stb-crystalbuilder --spacegroup 225 --a 3.52 --site Ni 0 0 0 -o fcc_ni.fdf
  stb-crystalbuilder --spacegroup Fd-3m --a 8.396 \
      --site Fe 0.125 0.125 0.125 --site Fe 0.5 0.5 0.5 \
      --site O 0.2548 0.2548 0.2548
  stb-crystalbuilder --spacegroup Fm-3m --a 3.52 --site Ni 0 0 0 \
      --reduce primitive --ml-relax --save-report --view

Notes:
  - Bulk (3D) crystals only -- see the description above for how to get
    a 2D/1D structure out of this tool's output.
```

10.9 Surface Passivator

At a glance

Menu code: 4.9 **Command:** `stb-passivate`
Guided example: `examples/4.9-stb-passivate/`
Summary: Cap dangling bonds on a cut surface with a passivating atom (e.g. H).

10.9.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/4.9-stb-passivate/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

10.9.1.1 What this tool does

Caps **dangling bonds** on a cut surface (typically a slab from `stb-slab`, `4.3`) with a passivating atom (default H), placed purely from each undercoordinated atom's own local coordination geometry — no bulk-lattice assumptions hard-coded in. Same **Structures** category (4) as `4.1-stb-2Dstacking/`, ..., `4.8-stb-crystalbuilder/`. Also available as a same-step convenience flag on `stb-slab --passivate` — this tool is the standalone version, usable on any structure, not just fresh-from-`stb-slab` output.

10.9.1.2 Why this matters (a bit of theory)

Cutting a crystal creates unphysical dangling bonds. A bulk crystal's atoms are all fully coordinated — every bond an atom "should" have (given its position in the lattice) is actually present. Cutting a slab out of that bulk (to model a surface) necessarily severs some of those bonds: the atoms right at the new surface are left with fewer neighbors than they have in the bulk. In a real material, these broken bonds get satisfied somehow — surface reconstruction, adsorbed atoms/molecules, an oxide layer. A bare, idealized DFT slab has none of that: the dangling bonds are simply unphysical half-bonds, which introduce spurious mid-gap electronic states localized at the surface — states that don't exist in the real material and can contaminate the very electronic structure (band gap, DOS, work function, ...) you built the slab to study in the first place. Capping each dangling bond with a cheap, chemically simple passivant (H is the standard choice) removes these spurious states, letting the slab's interior behave like the real bulk while the surface you actually care about (the *other* side of the slab, in an asymmetric-termination or adsorption study) stays real.

The geometry: an exact result for one missing bond, a genuine ambiguity for two or more. For an atom missing exactly one bond, the direction is **exactly determined**: the sum of the unit vectors to its existing neighbors, when negated, points precisely at the ideal position for the missing one — a direct geometric consequence of an atom's own equilibrium bonding geometry (e.g. tetrahedral for Si), not an approximation. `good-termination/` below proves this isn't just a plausible heuristic: the recovered H position sits at *exactly* 109.47°, the textbook ideal tetrahedral angle, from every one of Si's existing bonds.

For an atom missing **two or more** bonds, this breaks down: with only one (or zero) existing bond vectors left to anchor the geometry, there are infinitely many ways to place the missing atoms that still give the right coordination number — nothing pins down the actual directions without extra assumptions this tool deliberately doesn't make (it has no hard-coded knowledge of what "tetrahedral" or any other target geometry even means for the species involved). Guessing would risk silently writing an unphysical structure; instead, these sites are reported and left unpassivated for manual review. `bad-termination/` below shows this directly, with a real, physical cause: cutting the *exact same* Si(111) surface through a different atomic plane (the strongly-bonded interior of a bilayer, instead of the weakly-bonded gap between bilayers) leaves each surface atom missing 3 bonds instead of 1 — geometrically ambiguous, by the reasoning above, not a limitation of this particular structure.

Passivation should be symmetry-preserving, when it's the clean, single-missing-bond case. Since each capped bond is placed exactly along the direction the structure's own local symmetry dictates, adding a well-determined passivant never breaks the crystal's overall symmetry. Every run's own `[6] SYMMETRY ANALYSIS (BEFORE / AFTER)` table proves this concretely, not just asserts it — `symmetry-preserved/` below shows a real Si(111) slab's space group, layer group, point group, and Hall symbol all coming back *identical* before and after passivation.

`--bond-length`/`--cutoff`: a physical approximation, and how to correct it. The default bond length is the sum of the two species' pymatgen atomic radii — a fast, generic approximation (e.g. Si+H = 1.35 Ang), not the real experimental value (Si-H is actually closer to ~1.48 Ang). Good enough as a starting geometry, but `--bond-length` lets you dial in the real value for your specific system before a real DFT relaxation — or just let `--ml-relax` correct it automatically (see below).

10.9.1.3 When you'd reach for it

- Right after `stb-slab` cuts a surface with an unphysical unsaturated termination, before handing the structure to SIESTA.
- On any structure (not just a fresh `stb-slab` output) with genuinely undercoordinated atoms you want to cap.
- As a fast pre-check for whether a given termination is even a good candidate: a termination that comes back with mostly `[WARNING]` unresolved sites is telling you it's a geometrically awkward cut (see `bad-termination/`) — maybe try a different termination index instead.

10.9.1.4 Two ways to run it

A — direct CLI:

```
stb-passivate -f slab.fdf
```

B — interactive `stb-suite` menu:

```
$ stb-suite
Select an option (0-7, or a tool code like 5.1.2): 4.9
```

Both paths call the exact same underlying tool and produce the exact same output — `example_4.9.sh` proves this directly at the end.

10.9.1.5 What every run does (always on)

- **A numbered report** (`[0] RUN METADATA ... [9] SUMMARY & FILES` — `[4]` only appears with `--ml-relax`) printed to the console and, with `--save-report`, also saved to `stb_passivate_report.txt`.
- **Structure validation** — atom proximity, lattice handedness, atomic density (skipped for a vacuum-padded slab, since density isn't meaningful there) — run once before and once after passivation.
- **Full symmetry detection** via a before/after comparison table — crystal system, 3D space group, **layer group** (meaningful here, since a typical input is a genuinely 2D-periodic slab), point group, Hall symbol.
- **`references.bib`** — SIESTA always, plus the MACE papers if `--ml-relax` was used.
- **A provenance header** written into the output `.fdf`: the passivant, how many dangling bonds were found/auto-passivated/left unresolved, and MACE convergence/energy detail if used.

10.9.1.6 Optional (off by default)

- **`--passivant`** — element to cap dangling bonds with (default `H`).
- **`--cutoff`** — neighbor-search radius for coordination counting (default: auto, 1.25x the shortest pairwise distance).
- **`--bond-length`** — passivant bond length, Ang (default: auto, sum of atomic radii).
- **`--ml-relax`** (+ `--ml-relax-cell`, `--model`/`--custom-model`) — pre-relax the passivated structure with MACE. Vacuum-aware: any vacuum-padded axis (the typical slab case) always stays exactly fixed under `--ml-relax-cell`, same convention as `stb-unitcell`/`stb-slab`.
- **`--save-report`** — also persist the full report to `stb_passivate_report.txt`.
- **`--view`** — opens the input and passivated structures in ASE's interactive 3D viewer (`ase-gui`), as pageable frames. Needs a local display. Never exercised by `example_4.9.sh` itself.

10.9.1.7 Files in this folder

- `example_4.9.sh` — the guided walkthrough (see below). No committed input fixtures: like `4.8-stb-crystalbuilder/`'s own `bulk-graphite-to-slab/` case, this example builds bulk Si with `stb-crystalbuilder` and cuts both Si(111) terminations with `stb-slab` itself, live, at the start of the script — the exact same 2-step pipeline (`4.8 -> 4.3`) a real user would run before ever reaching for `stb-passivate`.
- `.gitignore` — excludes `output/`, `references.bib`, and `stb_passivate_report.txt`.

10.9.1.8 Running the walkthrough

```
cd examples/4.9-stb-passivate
./example_4.9.sh
```

It pauses between sections (`[Press Enter to continue]`) and always starts by wiping its own `output/`. Six self-contained cases are generated (one is skipped, with a hint, if the optional `ml` extra isn't installed):

Folder	What it shows
`good-termination/`	The clean case: 8 deficit-1 sites, all auto-capped at the exact 109.47° tetrahedral angle
`bad-termination/`	The *same* surface, cut one atomic plane over: 8 deficit-3 sites, all correctly left unresolved
`bond-length-override/`	The default 1.35 Ang (atomic-radii-sum) approximation vs. a dialed-in real Si-H bond length
`symmetry-preserved/`	The full report + before/after symmetry table: identical space/layer/point group
`ml-relax/`	MACE pre-relaxation of the passivated slab, vacuum axis provably fixed
`full-report/`	`--save-report` + `references.bib`

10.9.1.9 Try it yourself

```
# Build the bulk crystal and cut a slab first (see 4.8/4.3)
stb-crystalbuilder --spacegroup Fd-3m --a 5.43 --site Si 0 0 0 --reduce conventional -o si_bulk.fdf
stb-slab -f si_bulk.fdf --hkl 1 1 1 -o slab.fdf

# Passivate the dangling bonds, correcting the default bond-length approximation
stb-passivate -f slab.fdf --bond-length 1.48

# Cap with a different element entirely
stb-passivate -f slab.fdf --passivant F --bond-length 1.4

# Let MACE relax the passivated geometry before a real SIESTA calculation
stb-passivate -f slab.fdf --ml-relax --ml-relax-cell
```

10.9.1.10 Flag reference

Flag	Meaning
`-f/--file`	Input structure file, `.fdf` (required)
`--passivant`	Element to cap dangling bonds with (default `H`)
`--cutoff`	Neighbor-search radius, Ang (default: auto)
`--bond-length`	Passivant bond length, Ang (default: auto, sum of atomic radii)
`-sp/--symprec`	Symmetry tolerance for the before/after table (default `0.01`)
`--ml-relax`	Pre-relax the passivated structure with a MACE potential
`--ml-relax-cell`	With `--ml-relax`, also relax the cell (vacuum axis stays fixed)
`--model`	MACE-MP-0 size for `--ml-relax`: `small`/`medium`/`large` (default `small`)
`--custom-model`	Path to a custom fine-tuned `.model` file
`-o/--output`	Output `.fdf` file name (default `passivated.fdf`)
`--save-report`	Persist the full report (incl. symmetry table) to disk
`--view`	Open the input and passivated structures in ASE's interactive viewer

10.9.1.11 What's next

See `4.3-stb-slab/` for cutting the slab this tool is usually applied to (and its own `--passivate` convenience flag, for doing both in one step), or `4.8-stb-crystalbuilder/` for building the bulk crystal this whole pipeline starts from.

10.9.2 Implementation details

The algorithm.. The same routine (`core/passivation.py`) serves **`stb-passivate`**, `–passivate` of **`stb-slab`** and `–passivate` of **`stb-nanotube`**. It works from local geometry only, with no assumption about the parent bulk:

1. Neighbours of every atom are counted within a cutoff radius r_c (`–cutoff`; default $1.25\times$ the shortest interatomic distance of the structure, with periodic images).
2. The *reference coordination* of each species is the most frequent (modal) coordination number of that species in the structure itself.
3. An atom whose coordination is one below its reference has exactly one dangling bond. Its direction is
$$\hat{\mathbf{d}} = -\frac{\sum_j \hat{\mathbf{u}}_j}{\left|\sum_j \hat{\mathbf{u}}_j\right|},$$
where $\hat{\mathbf{u}}_j$ are unit vectors to the existing neighbours – for an ideal tetrahedral, trigonal or octahedral site this is exactly the missing bond. The passivant is placed at distance ℓ along $\hat{\mathbf{d}}$.
4. The default ℓ (`–bond-length`) is the sum of the atomic radii of the two elements (pymatgen's table), an approximation: Si–H comes out 1.35 Å against an experimental ≈1.48 Å.
5. Atoms missing two or more bonds, and atoms whose neighbour vectors cancel, are *not* capped: their missing directions are not determined by local geometry. They are listed in the report with their deficit, to be handled by hand.

When the reference coordination is wrong.. Because the reference is the modal value in the given structure, a very thin slab or a small cluster where most atoms are undercoordinated can have a modal value below the bulk one, and then nothing (or the wrong atoms) is capped. Compare the number of capped atoms in the report with what you expect for the surface (the reference value itself is not printed); if needed, cut a thicker slab first, or set `–cutoff` explicitly when the default radius misses or over-counts bonds (for example in structures with two very different bond lengths).

After passivation.. The report lists every added atom (the atom it caps and its position), every unresolved atom, and compares symmetry before and after: capping a symmetric slab with single dangling bonds on both faces preserves its symmetry. Relax the hydrogen positions (`–ml-relax` for a quick start, SIESTA for the real geometry); for pseudo-hydrogen passivation of III–V surfaces (fractional-charge H) a plain H passivant is only a geometric starting point.

10.9.3 Command-line reference

10.9.3.1 `stb-passivate`

Caps dangling bonds on a cut surface with a passivating atom. Detects undercoordinated atoms (e.g. from `stb-slab`) via local coordination geometry and places a passivant (default H) along each missing-bond direction. Only single-missing-bond sites are auto-passivated – sites missing 2+ bonds are geometrically underdetermined from local coordination alone and are reported instead of guessed.

Synopsis.

```
usage: stb-passivate [-h] -f FILENAME [--passivant PASSIVANT] [--cutoff CUTOFF]
                     [--bond-length BOND_LENGTH] [-sp SYMPREC] [--ml-relax]
```

```
[--ml-relax-cell] [--model {small,medium,large}]
[--custom-model PATH] [--custom-model-card JSON]
[--device {cpu,cuda}] [--save-report] [--view] [-o OUTPUT] [-v]
[--no-intro]
```

Options

`-f, --file FILENAME`
Path to the input structure file (.fdf), typically a cut slab. required

`--passivant PASSIVANT`
Element to cap dangling bonds with (default: H).

`--cutoff CUTOFF`
Neighbor-search radius in Angstrom, used to determine each atom's coordination number. Default: auto-detected as 1.25x the shortest pairwise distance in the structure. type: float

`--bond-length BOND_LENGTH`
Passivant bond length in Angstrom. Default: auto, per surface-species/passivant pair, as the sum of their pymatgen atomic radii (an approximation – e.g. Si+H = 1.35 Ang vs. the experimental Si-H ~1.48 Ang). type: float

`-sp, --symprec SYMPREC`
Symmetry tolerance in Angstroms for the before/after symmetry analysis (default: 0.01). type: float

`--ml-relax`
Pre-relax the passivated structure with a MACE potential (needs the optional 'ml' extra: pip install stb_suite[ml]) before writing it out – positions only by default. Off by default. switch, off by default

`--ml-relax-cell`
With `--ml-relax`, also relax the cell – any vacuum-padded axis always stays exactly fixed. Only valid together with `--ml-relax`. switch, off by default

`--model {small,medium,large}`
MACE-MP-0 foundation model size for `--ml-relax` (default: medium). choices: small, medium, large

`--custom-model PATH`
Path to a custom fine-tuned .model file for `--ml-relax`, instead of a MACE-MP-0 foundation size.

`--custom-model-card JSON`
Model card of the `--custom-model` file: the `mlff_model_card.json` that `stb-mlffAnalysis` wrote next to it (training data, level of theory, accuracy, energy scale). Required with `--custom-model`: the card is checked against the model file and used to warn when a structure is outside what the model was trained on.

`--device {cpu,cuda}`
Device to run the MACE model on (default: cpu). choices: cpu, cuda

`--save-report`
Also persist the full run report (including the symmetry analysis) to `stb_passivate_report.txt`. Off by default. switch, off by default

`--view`
Open an interactive 3D view (via ASE) comparing the input structure and the passivated structure (page through frames in ase-gui) after writing the output file. Needs a display. Off by default. switch, off by default

`-o, --output OUTPUT`
Output .fdf file name (default: `passivated.fdf`).

Like every STB command it also accepts `-h`/`--help`, `-v`/`--version` and `--no-intro` (Section 3.3).

From the built-in help.

```
Usage examples:
  stb-passivate -f slab.fdf
  stb-passivate -f slab.fdf --passivant F --bond-length 1.4 -o slab_F.fdf
  stb-passivate -f slab.fdf --ml-relax --save-report --view
```

10.10 Reference Molecule Builder

At a glance

Menu code: 4.10 **Command:** `stb-molecule`
Guided example: `examples/4.10-stb-molecule/`
Summary: Build a reference molecule (e.g. H2O, CO2) from ASE's G2 database.

10.10.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/4.10-stb-molecule/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

10.10.1.1 What this tool does

Builds an isolated reference molecule (H2O, CO2, CH4, benzene, ...) from ASE's bundled **G2 database** (162 small molecules), placed in a vacuum box ready for a SIESTA calculation. Same **Structures** category (4) as `4.1-stb-2Dstacking/`, ..., `4.9-stb-passivate/`, but the odd one out geometrically: every other tool in this category ends up with some kind of periodicity (a bulk crystal, a slab, a tube); this one deliberately builds something with **none** — a single, isolated molecule.

10.10.1.2 Why this matters (a bit of theory)

SIESTA is always periodic — an "isolated" molecule needs a trick. SIESTA (like almost every plane-wave/localized-basis DFT code) solves the electronic structure of a periodic crystal. There is no native "just one molecule, nowhere else" mode. To model an isolated molecule anyway, you put it in a large, empty box and repeat *that* periodically — the molecule's periodic images are far enough apart that they don't meaningfully interact, so the calculation behaves as if the molecule were truly alone. `--vacuum` controls how big that empty margin is (in every direction around the molecule); `stb-molecule` exists specifically to build this box correctly, so you don't have to hand-place the atoms and guess a cell size yourself.

The G2 database: a standard benchmark set, not just a random list. ASE's G2 collection is the "G2/97 test set" from computational thermochemistry (Curtiss et al.) — a standard, widely-used set of small molecules with well-characterized reference geometries and energies, originally assembled to benchmark quantum chemistry methods against experiment. That's exactly why it's a good source for this tool: every name in it (`--list` shows all 162) is a real, chemically sensible molecule with a reasonable starting geometry, not an arbitrary guess.

Point groups: a molecule's *own* symmetry, distinct from a crystal's. A crystal's symmetry is a **space group** (translations + rotations, 230 possibilities, see `4.8-stb-crystalbuilder/`); an isolated molecule has no translational symmetry to speak of, so it's classified by its **point group** instead — the set of rotations/reflections/inversions that map the molecule onto itself, in the Schoenflies notation chemists use (`C2v`, `Td`, `D6h`, ...). This is why `stb-molecule`'s own symmetry detection (`core_symmetry.point_group_label`, via pymatgen's `PointGroupAnalyzer`) is a completely different code path from the space-group detection every periodic tool in this suite uses — there is no periodic cell here for a space group to even apply to. `point-group-gallery/` below verifies 4 real, textbook point groups directly from the built geometries: water's bent `C2v`, methane's tetrahedral `Td`, benzene's hexagonal `D6h`, and CO2's linear `D*h` (pymatgen's notation for the infinite-order D∞h).

Why symmetry is expected to survive a relaxation — and what it means if it doesn't. The G2 database's own reference geometries are already close to each molecule's true equilibrium shape, so relaxing with MACE (`--ml-relax`) is expected to preserve the point group exactly — a small, real correction to the bond lengths/angles, not a change in shape. Every run's own `[5] SYMMETRY ANALYSIS (BEFORE / AFTER)` table proves this instead of just asserting it. If a real calculation on your OWN starting geometry ever shows a point-group *change* here, that's a genuine, useful signal: either the relaxation found a genuinely different (lower-symmetry, lower -energy) minimum, or the starting geometry wasn't as symmetric as it looked.

A real pitfall: `--vacuum` too small fools the tool's own dimensionality check. `stb-molecule` always builds something physically 0D (fully isolated), but the report's own "Dimensionality" line is computed generically (same `core.kspace.detect_vacuum_axes` heuristic every periodic-aware tool in this suite uses), by checking whether the gap between periodic images exceeds a fixed 10 Ang threshold on each axis. The **default** `--vacuum 10.0` sits right at that threshold, correctly reading `0D (e.g., a molecule)`. `vacuum-and-dimensionality/` below shows what happens if you turn it down: at `--vacuum 3`, the box is real and the molecule is still just as isolated physically, but the report's own dimensionality check gets fooled into reporting `3D (bulk material)` — and even prints an `Atomic density` validation row that means nothing for a lone molecule in a box. The **point group**, by contrast, is completely unaffected (it never looks at the box at all) and stays correct either way. Lesson: trust the point group here, and keep `--vacuum` at its default (or higher) for the dimensionality line to read correctly too.

10.10.1.3 When you'd reach for it

- An isolated-molecule energy reference for a workflow that needs one — e.g. `stb-cohesive`'s isolated-atom references, or a gas-phase molecule (H2, O2, H2O) used as a gas-phase reference in reaction energies.
- A quick, chemically sensible starting geometry for any small-molecule calculation, without hand-building the structure file yourself.

10.10.1.4 Two ways to run it

A — direct CLI:

```
stb-molecule --name H2O
```

B — interactive `stb-suite` menu:

```
$ stb-suite
Select an option (0-7, or a tool code like 5.1.2): 4.10
```

Both paths call the exact same underlying tool and produce the exact same output — `example_4.10.sh` proves this directly at the end.

10.10.1.5 What every run does (always on)

- **A numbered report** (`[0] RUN METADATA ... [8] SUMMARY & FILES` — `[3]` only appears with `--ml-relax`) printed to the console and, with `--save-report`, also saved to `stb_molecule_report.txt`.
- **Structure validation** — atom proximity, lattice handedness (atomic density skipped: always vacuum-padded on all 3 axes by construction) — run once before and once after any ML relax.
- **Point-group symmetry detection**, before and after, in a dedicated `[5] SYMMETRY ANALYSIS` table (see the theory above for why this is a point group, not a space group).
- **`references.bib`** — SIESTA always, plus the MACE papers if `--ml-relax` was used.
- **A provenance header** written into the output `.fdf`: the molecule name, vacuum padding, detected point group, and MACE convergence/energy detail if used.

10.10.1.6 Optional (off by default)

- **`--vacuum`** — padding in Ang around the molecule (default `10.0`; see the pitfall above before lowering it).
- **`--ml-relax`** (+ `--model`/`--custom-model`) — pre-relax the molecule's geometry with MACE. Positions only — there is no `--ml-relax-cell` option here at all (not just a no-op): an isolated molecule's vacuum box has no physically meaningful cell to relax.
- **`--save-report`** — also persist the full report to `stb_molecule_report.txt`.
- **`--view`** — opens the as-built and final molecule in ASE's interactive 3D viewer (`ase gui`), as pageable frames. Needs a local display. Never exercised by `example_4.10.sh` itself.
- **`--list`** — prints all 162 available names and exits (names are case-sensitive).

10.10.1.7 Files in this folder

- `example_4.10.sh` — the guided walkthrough (see below). No input fixtures needed — like `4.8-stb-crystalbuilder/`, everything is built from `--name`/`--vacuum` flags, not read from a structure file.
- `.gitignore` — excludes `output/`, `references.bib`, and `stb_molecule_report.txt`.

10.10.1.8 Running the walkthrough

```
cd examples/4.10-stb-molecule
./example_4.10.sh
```

It pauses between sections (`[Press Enter to continue]`) and always starts by wiping its own `output/`. Six self-contained cases are generated (one is skipped, with a hint, if the optional `ml` extra isn't installed):

Folder	What it shows
`point-group-gallery/`	4 real point groups verified directly: water `C2v`, methane `Td`, benzene `D6h`, CO2 `D*h`
`unknown-name-suggestions/`	`--list`, a case-insensitive "did you mean", and a genuine typo suggestion
`vacuum-and-dimensionality/`	The real pitfall: `--vacuum` below the detection threshold fools the dimensionality check
`ml-relax-symmetry-preserved/`	MACE pre-relaxation of methane, with `Td` proven unchanged before/after
`full-report/`	`--save-report` + `references.bib`

10.10.1.9 Try it yourself

```
# Any of the 162 G2 names -- see them all
stb-molecule --list

# A gas-phase reference molecule
stb-molecule --name H2

# Pre-relax with your own fine-tuned MACE model before a real SIESTA calculation
stb-molecule --name CH3OH --ml-relax --custom-model my_finetuned.model \
    --custom-model-card mlff_model_card.json
```

10.10.1.10 Flag reference

Flag	Meaning
`--name`	Exact, case-sensitive G2 database name (required unless `--list`)
`--list`	Print all 162 available names and exit
`--vacuum`	Vacuum padding in Ang around the molecule (default `10.0`)
`--ml-relax`	Pre-relax the molecule's geometry with a MACE potential (positions only)
`--model`	MACE-MP-0 size for `--ml-relax`: `small`/`medium`/`large` (default `small`)
`--custom-model`	Path to a custom fine-tuned `.model` file
`-o`/`--output`	Output `.fdf` file name (default `molecule.fdf`)
`--save-report`	Persist the full report (incl. symmetry table) to disk
`--view`	Open the as-built and final molecule in ASE's interactive viewer

10.10.1.11 What's next

See `5.3-cohesive/` (Workflow) for a real use of an isolated reference (atoms there, not molecules, but the same "periodic code needs an artificial vacuum box" idea), or `4.8-stb-crystalbuilder/` for the periodic, space-group side of the symmetry story this tool's point-group detection deliberately steps outside of.

10.10.2 Implementation details

Source of the geometries.. Molecules come from ASE's `g2` collection [8], the molecules of the G2 thermochemistry test set [62] with their reference geometries; `-list` prints every available name. Names are case sensitive (`CO` is carbon monoxide, `Co` is not a molecule); a near miss is answered with suggestions. The geometries are reference (experimental or MP2-optimized) structures, not relaxed with your functional: relax the molecule in SIESTA with the same settings as the system you will compare it to.

The box.. The molecule is centred in an orthorhombic box with `-vacuum` Å (default 10) between its outermost atoms and each face, so neighbouring images are 2× `-vacuum` apart. The dimensionality check of Section 4.4 then sees three vacuum axes (0D). A vacuum smaller than 5 Å makes images closer than the 10 Å threshold, and the tool would see a periodic solid – the guide shows this pitfall. For polar molecules, check convergence of the energy with the box size, or use a larger vacuum.

Symmetry.. The report gives the molecular point group (pymatgen's `PointGroupAnalyzer`), which should survive a relaxation; a lower point group after relaxing means the starting geometry was a saddle point or the relaxation was not converged.

Typical uses.. Isolated adsorbates for the adsorption workflow (5.8), reference molecules for reaction energies, and molecules for MLFF training sets. With `-ml-relax` the positions are pre-relaxed with MACE (there is no cell to relax).

SIESTA settings for an isolated molecule

Use a single k-point (`kgrid.MonkhorstPack [1 1 1]`, which **`stb-kgrid`** gives for a 0D cell), consider `Spin polarized` for radicals and O_2 (triplet), and keep the basis and mesh cutoff identical to those of the system the molecule is compared with.

10.10.3 Command-line reference

10.10.3.1 `stb-molecule`

Builds a reference molecule from ASE's G2 database. Places the molecule in a vacuum box, ready as an isolated-molecule reference (e.g. for `stb-cohesive`) or a quick starting structure.

Synopsis.

```
usage: stb-molecule [-h] [--name NAME] [--list] [--vacuum VACUUM] [--ml-relax]
                    [--model {small,medium,large}] [--custom-model PATH]
                    [--custom-model-card JSON] [--device {cpu,cuda}] [--save-report]
                    [--view] [-o OUTPUT] [-v] [--no-intro]
```

Options

`--name NAME`
: Exact (case-sensitive) name from ASE's G2 database, e.g. 'H2O', 'CO2', 'C6H6'. See `--list` for all names. Required unless `--list` is given.

`--list`
: Print all available molecule names and exit. switch, off by default

`--vacuum VACUUM`
: Vacuum padding in Angstrom, added in every direction around the molecule (default: 10.0). type: float

`--ml-relax`
: Pre-relax the molecule's geometry with a MACE potential (needs the optional 'ml' extra: pip install stb_suite[ml]) before writing it out. Positions only – there is no physically meaningful periodic cell to relax for an isolated molecule, so no `--ml-relax-cell` option exists here. Off by default. switch, off by default

`--model {small,medium,large}`
: MACE-MP-0 foundation model size for `--ml-relax` (default: medium). choices: small, medium, large

`--custom-model PATH`
: Path to a custom fine-tuned .model file for `--ml-relax`, instead of a MACE-MP-0 foundation size.

`--custom-model-card JSON`
: Model card of the `--custom-model` file: the `mlff_model_card.json` that `stb-mlffAnalysis` wrote next to it (training data, level of theory, accuracy, energy scale). Required with `--custom-model`: the card is checked against the model file and used to warn when a structure is outside what the model was trained on.

`--device {cpu,cuda}`
: Device to run the MACE model on (default: cpu). choices: cpu, cuda

`--save-report`
: Also persist the full run report (including the symmetry analysis) to `stb_molecule_report.txt`. Off by default.
switch, off by default

`--view`
: Open an interactive 3D view (via ASE) comparing the as-built and final molecule (page through frames in ase-gui) after writing the output file. Needs a display. Off by default. switch, off by default

`-o, --output OUTPUT`
: Output .fdf file name (default: `molecule.fdf`).

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Usage examples:
  stb-molecule --name H2O
  stb-molecule --name CH3OH --vacuum 12 -o methanol.fdf
  stb-molecule --name H2O --ml-relax --save-report --view
  stb-molecule --list
```

10.11 Amorphous Structure Generator

At a glance

Menu code: 4.11 **Command:** `stb-amorphize`
Guided example: `examples/4.11-stb-amorphize/`
Summary: Melt-quench MD with MACE-MP-0 to build an amorphous starting guess (needs the optional 'ml' extra).

10.11.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/4.11-stb-amorphize/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

10.11.1.1 What this tool does

`stb-amorphize` runs a melt-quench protocol with the **MACE-MP-0** machine-learned potential (or a custom model you fine-tuned yourself with `stb-mlffAnalysis`) to turn a crystalline structure into a fast, heuristic starting guess for an amorphous/glassy one — meant to give SIESTA a much better starting point than an ad-hoc random-displacement structure. **This is a heuristic, not a substitute for DFT.** NOT a substitute for a slower, production-quality quench (more MD steps, a slower cool-down) either. Bulk (3D periodic) structures only — melting a vacuum-padded slab/wire/ molecule is physically meaningless, and the input is rejected outright if a vacuum-padded axis is detected.

10.11.1.2 Why this matters (a bit of theory)

Melt-quench: why heating something up is how you make it *lose* order. An amorphous/glassy structure (real short-range chemistry — roughly the right bond lengths/angles/coordination — but no

long-range crystalline order) is hard to build directly: you can't just "un-order" atomic positions at random without creating physically absurd overlaps. The classic laboratory trick is to melt the material, then cool it faster than it can re-crystallize. It works because a liquid has already forgotten its crystalline arrangement while keeping sensible local chemistry (atoms still push each other apart at the right distance); cool it fast enough and there isn't time for atoms to find their way back into an ordered lattice before the structure "freezes" kinetically — a glass, not a crystal. `stb-amorphize` reproduces exactly this in silico: melt a crystalline structure well above its real melting point (so it melts even given the tiny simulated time available — real quenches take seconds; this one takes picoseconds), hold it there long enough to forget the crystal, then ramp the temperature back down.

NPT-Berendsen has a *finite response time* — and that has two real consequences. The MD engine underneath is ASE's `NPTBerendsen`: not plain constant-energy dynamics, but a thermostat (`--taut`) **and** a barostat (`--taup`) that actively drive the system toward a target temperature and 0 GPa pressure — a liquid/quenched structure should relax its own volume freely, not stay pinned at the crystal's original cell size. Both `--taut`/`--taup` are **response times** (in fs), not instantaneous snaps: Berendsen coupling relaxes the current temperature toward the target roughly exponentially, with `--taut` as the time constant (default 50 fs). That single fact drives two things worth knowing before you pick step counts:

1. **`--melt-steps` needs to comfortably exceed `--taut`** for the system to actually reach the melt temperature, not just start drifting toward it. Measured live on the same 8-atom `si8.fdf` (`--seed 7` fixes the initial velocities so both runs are directly comparable — see `output/too-short-vs-adequate-melt/` below):

Protocol	`--melt-steps` (vs. `--taut`=50 fs)	Bond-angle std after MD
Too short	5 (0.1x `--taut`)	3.07 deg
Adequate	100 (2x `--taut`)	8.64 deg

   Both runs' bond-angle **mean** stays close to the crystal's own ~109.5 deg tetrahedral angle (short-range chemistry survives either way) — only the **std** (long-range disorder) tracks how much actual melting happened, exactly the diagnostic's intended signature.

2. **A `--quench-steps` ramp faster than `--taut` leaves the actual, instantaneous temperature lagging behind the falling target** — real residual thermal energy/force, not a clean local minimum. This is *why the final static relax exists at all* (see below) — measured live (`--melt-steps 30 --quench-steps 60 --seed 7`, `output/ final-relax-comparison/`): after the quench ramp nominally targets 300 K, the actual instantaneous temperature is still **760 K**.

Why the final static relax is on by default. Following directly from the lag above: the quench MD alone leaves real thermal motion, not a converged geometry. `stb-amorphize` always runs a final position+cell relax afterward (`--no-final-relax` to skip it). Measured live on the same run as above:

Quantity	Right after quench MD	After final relax	Change
Energy (eV)	-41.094170	-42.966005	-1.871835 (-0.233979/atom)
Max force (eV/Ang)	1.5434	0.0093	-1.5340

A max force of ~1.5 eV/Ang is real thermal noise (atoms still mid-vibration when the MD stopped) — a handful of FIRE steps bring it down to the same ~0.01-0.05 eV/Ang order of magnitude every other stb-suite relaxation targets.

The bond-angle mean/std check: how the tool proves amorphization happened. A crystal has a sharp, well-defined bond-angle distribution (std near 0 deg — every tetrahedral Si-Si-Si angle in `si8.fdf`

is exactly 109.47 deg, since it's a perfect diamond-cubic cell). A genuine melt-quench should **broaden** that distribution while keeping roughly the same **mean** (short-range coordination survives; long-range order doesn't) — the table above is this signature measured directly, not just asserted.

Symmetry before/after: expected to differ, unlike most other stb-suite tables. `[6] SYMMETRY ANALYSIS (BEFORE / AFTER)` is deliberately the *opposite* kind of check from `stb-unitcell`'s own before/after table: there, an unexpected symmetry change is a red flag; here, losing the crystal's symmetry is the entire point. A genuinely melt-quenched structure often collapses all the way down to `P1`/`P-1` (essentially no symmetry operations left besides identity, or identity+inversion). Measured live (`--melt-steps 100 --quench-steps 200 --seed 2`, `output/full-report/`): `Fd-3m (227)` (diamond-cubic Si) → `P-1 (2)`.

Why an 8-atom cell? (and why every number above is a qualitative illustration). This example deliberately reuses one tiny 8-atom bulk Si cell throughout so every comparison finishes in seconds on a CPU. Real amorphization needs a much bigger supercell (tens to hundreds of atoms) — with only 8 atoms, a single MD trajectory is a genuinely chaotic, small-number-statistics system: re-running the exact same command *without* `--seed` gives visibly different bond-angle numbers each time, and even `--seed` only fixes the *initial* velocities, not every source of run-to-run floating-point noise. Every comparison above still shows a real, physically sensible **trend** — that's the point being demonstrated — just don't expect the precise digits to reproduce bit-for-bit on your machine, or to mean much on their own for a cell this small.

`--save-data`: keeping the MD trace that used to be thrown away. Until this feature, every intermediate MD step was discarded — only the final frame was ever written anywhere, and the only trace of temperature/energy during the run was a `\r`-overwritten progress line on stderr that vanished the moment the run ended. `--save-data` writes `<stem>_md_diagnostics.dat` (step, time, temperature, E_pot/E_kin/E_total, cell volume) with **one continuous step/time axis spanning both the melt and quench stages**, sampled every `--stride` steps, using gnuplot's own `index` block convention — the melt stage is `index 0`, the quench stage is `index 1` **in the same file**, so either stage can be plotted alone, or both together via `index 0:1`, without re-running anything. The companion `.gplot` (same `.dat+.gplot` convention used throughout the suite, e.g. `5.1-strain/analysis/`) renders both energy and temperature vs. step into one PDF, with the melt→quench transition marked and each stage's target temperature drawn as a reference line.

`--save-traj`: watching the disorder happen, not just trusting a number. `--save-traj` writes a real multi-frame trajectory of the whole MD run (same sampling as `--save-data`) — the same 3-format choice as `stb-ani2traj`/`stb-mlmd`: `xsf` (OVITO/VMD-native, default), `pdb` (VMD's own default), or `xyz` (OVITO-native). The bond-angle std number proves disorder happened; actually opening this trajectory in OVITO or VMD lets you *see* the crystal melt and re-quench, frame by frame.

10.11.1.3 When you'd reach for it

- A better starting guess than a random-displacement structure for an amorphous/glassy SIESTA calculation (a-Si, a-SiO2, glassy alloys, ...).
- `--save-data`/`--save-traj` specifically: diagnosing whether a given `--melt-steps`/`--quench-steps`/`--melt-temp` protocol is actually doing what you think, before committing to a much bigger production cell.

10.11.1.4 Two ways to run it

A — direct CLI:

```
stb-amorphize -f si8.fdf --save-data --save-traj
```

B — interactive `stb-suite` menu:

```
$ stb-suite
Select an option (0-7, or a tool code like 5.1.2): 4.11
```

Both paths call the exact same underlying tool and produce the exact same output — `example_4.11.sh` proves this directly at the end.

10.11.1.5 What every run does (always on)

- **A numbered report** (`[0] RUN METADATA … [9] SUMMARY & FILES`) printed to the console and, with `--save-report`, also saved to `stb_amorphize_report.txt`.
- **Structure validation + bond-angle mean/std**, before and after the MD.
- **Symmetry analysis before/after** (`[6]`) — expected to differ (often down to `P1`/`P-1`), unlike most other stb-suite before/after tables.
- **`references.bib`** — SIESTA + MACE (+ MACE-MP if a foundation model was used, not a custom fine-tuned one).
- **A provenance header** written into the output `.fdf`: input file, melt/quench protocol, bond-angle before/after, final-relax summary.

10.11.1.6 Optional (off by default)

- **`--save-data`** — writes `<stem>_md_diagnostics.dat` + a companion `.gplot` (step, time, temperature, E_pot/E_kin/E_total, cell volume) for the whole melt-quench process, one gnuplot `index` block per stage. Render it with `gnuplot <stem>_md_diagnostics.gplot`.
- **`--save-traj`** (+ `--traj-format xsf|pdb|xyz`, default `xsf`) — a multi-frame trajectory of the whole MD run, for viewing in OVITO/VMD.
- **`--stride`** — sampling interval (in MD steps) shared by both exports above (default `10`).
- **`--no-final-relax`** — skip the final static (position+cell) relax.
- **`--save-report`** / **`--view`** — same as every other stb-suite tool.

10.11.1.7 Files in this folder

- `si8.fdf` — an 8-atom bulk Si cell (the same fixture `test/4-structures/11-amorphize/` uses), small enough that the whole walkthrough finishes in well under a minute on CPU. See "Why an 8-atom cell?" above for what that trades away.
- `example_4.11.sh` — the guided walkthrough (**not** an automated test — see `test/4-structures/11-amorphize/test.sh` for that). Pauses between sections so you can read before moving on; safe to re-run any time.
- `.gitignore` — excludes `output/`, `references.bib`, and `stb_amorphize_report.txt`.

10.11.1.8 Running the walkthrough

```
cd examples/4.11-stb-amorphize
./example_4.11.sh
```

It pauses between sections (`[Press Enter to continue]`) and always starts by wiping its own `output/`. The whole script is skipped, with a clear message, if the optional `ml` extra isn't installed. Six self-contained cases are generated:

Folder	What it shows
`too-short-vs-adequate-melt/`	`--melt-steps` vs. `--taut`: the bond-angle-std table above, measured live
`final-relax-comparison/`	Why the final relax is on by default: the energy/force table above
`vacuum-rejected/`	Rejected outright on a `stb-slab`-built (100) slab (bulk-only)
`save-data/`	`--save-data`: the `.dat`+`.gplot` pair, rendered to a real PDF if gnuplot is installed
`save-traj/`	`--save-traj`: all 3 formats (xsf/pdb/xyz), read back and verified with ASE
`full-report/`	`--save-report`, the symmetry `Fd-3m -> P-1` collapse, `references.bib`

10.11.1.9 Try it yourself

```
# Diagnose a protocol before committing to a bigger production cell
stb-amorphize -f si8.fdf --melt-steps 2000 --quench-steps 4000 \
    --save-data --save-traj --stride 20

# Watch the melt-quench in OVITO/VMD directly
stb-amorphize -f si8.fdf --save-traj --traj-format pdb

# Pre-relax with your own fine-tuned MACE model, keeping the full trace
stb-amorphize -f si8.fdf --custom-model my_finetuned.model \
    --custom-model-card mlff_model_card.json --save-data
```

10.11.1.10 Flag reference

Flag	Meaning
`-f/--file`	Input crystalline structure file (`.fdf`, bulk only)
`--melt-temp`	Melt temperature, K (default `3000.0`)
`--melt-steps`	MD steps held at `--melt-temp` (default `500`) — see `--taut` above
`--quench-temp`	Final target temperature, K (default `300.0`)
`--quench-steps`	MD steps for the linear cooling ramp (default `1000`)
`--timestep`	MD timestep, fs (default `1.0`)
`--taut/--taup`	Berendsen thermostat/barostat response times, fs (default `50.0`/`200.0`)
`--compressibility`	Barostat compressibility, 1/bar (default `4.57e-5`, water's value — a generic placeholder; `1/(B[GPa] x 1e4)` for your material, e.g. Si `1e-6`)
`--stride`	Sample every N MD steps for `--save-data`/`--save-traj` (default `10`)
`--save-data`	Write `<stem>_md_diagnostics.dat` + `.gplot`
`--save-traj`	Write a multi-frame MD trajectory
`--traj-format`	Trajectory format for `--save-traj`: `xsf`/`pdb`/`xyz` (default `xsf`)
`--no-final-relax`	Skip the final static (position+cell) relax
`--model`	MACE-MP-0 size: `small`/`medium`/`large` (default `small`)
`--custom-model`	Path to a custom fine-tuned `.model` file
`-sp/--symprec`	Symmetry tolerance, Ang, for the before/after table (default `0.01`)
`-o/--output`	Output `.fdf` file name (default `amorphous.fdf`)
`--save-report`	Persist the full report (incl. symmetry table) to disk
`--view`	Open the input and final structure in ASE's interactive viewer

10.11.1.11 What's next

Take the amorphized `.fdf` into a real SIESTA relaxation/calculation (e.g. via `stb-inputfile`, example `1.1`) — remember this is a fast heuristic starting guess, not a DFT-verified amorphous structure. See `4.10-stb-molecule/` for another Structures-category tool with the same numbered-report/provenance-header conventions, or `1.7-stb-mlrelax/` for another MACE-driven tool in this suite (including the same FIRE/BFGS/LBFGS convergence theory `stb-amorphize`'s own final relax step also relies on). If you have your own SIESTA training data, `stb-mlffAnalysis` (Workflow menu) can fine-tune a custom MACE model for `--custom-model` that's more accurate for your specific material than the generic foundation checkpoint.

10.11.2 Implementation details

Protocol.. The structure (a bulk cell; vacuum-padded cells are refused) goes through three stages, all driven by MACE:

1. **Melt** – velocities are drawn from a Maxwell–Boltzmann distribution at $\min(T_{\mathrm{melt}}, 300\ \mathrm{K})$ (seeded by `-seed`), then `-melt-steps` (default 500) NPT steps with ASE's `NPTBerendsen` [63] at `-melt-temp` (default 3000 K) and zero pressure.
2. **Quench** – `-quench-steps` (default 1000) further NPT steps while the thermostat target is lowered linearly from `-melt-temp` to `-quench-temp` (default 300 K); the cooling rate is therefore $(T_{\mathrm{melt}} - T_{\mathrm{quench}})/(N_{\mathrm{quench}}\,\Delta t)$.
3. **Final relaxation** – positions and cell relaxed with all six strain components free (FIRE, 0.05 eV/Å, 200 steps), skipped with `-no-final-relax`.

MD runs in single precision (float32, MACE's recommendation for MD); the final relaxation in double precision. The time step is `-timestep` (fs, default 1.0); the Berendsen time constants `-taut` and `-taup` (fs, defaults 50 and 200) and the compressibility (`-compressibility`, in bar^{-1}, default 4.57×10^{-5}, the value of water) are converted to ASE's internal units. The compressibility only sets how fast the barostat responds; a value far from the material's own makes the volume respond too slowly or too violently but does not change the target pressure.

Diagnostics.. The report follows the run stage by stage (input, validation, melt–quench MD, final relaxation, validation after, symmetry before and after) and gives the mean and standard deviation of nearest-neighbour bond angles before and after the MD: a crystal has a near-zero spread, a melt-quenched network a broad one, often around the same mean. The space group is expected to collapse to $P1$. `-save-data` writes `<stem>_md_diagnostics.dat` and a gnuplot script (melt and quench as two data blocks) sampled every `-stride` steps; `-save-traj` writes the trajectory in `-traj-format` (xsf, pdb or xyz).

A starting guess, not a validated amorphous model

A realistic amorphous structure needs a large cell (hundreds of atoms), a slow cooling rate and an accurate potential; the defaults produce a quick, qualitative disordered starting point. The guide's 8-atom example illustrates the mechanics only. Check the radial distribution function and coordination statistics (**`stb-structural`**, **`stb-aimdAnalysis`**) against experiment before relying on the model, and relax it with SIESTA.

10.11.3 Command-line reference

10.11.3.1 `stb-amorphize`

Melt-quench amorphous structure generator using the MACE-MP-0 potential. Heats a crystalline structure above its melting point long enough to erase crystalline memory, then cools it back down – a fast heuristic starting guess for an amorphous/glassy structure, meant to give SIESTA a much better starting point than an ad-hoc random-displacement structure. NOT a substitute for a slower, production-quality quench (more steps, slower cooling) or DFT verification. Bulk (3D periodic) structures only – melting a vacuum-padded slab/wire/ molecule is physically meaningless. The input is rejected outright if any vacuum-padded axis is detected.

Synopsis.

```
usage: stb-amorphize [-h] -f FILENAME [--melt-temp MELT_TEMP]
                     [--melt-steps MELT_STEPS] [--quench-temp QUENCH_TEMP]
                     [--quench-steps QUENCH_STEPS] [--timestep TIMESTEP] [--taut TAUT]
                     [--taup TAUP] [--compressibility COMPRESSIBILITY]
                     [--model {small,medium,large}] [--custom-model PATH]
                     [--custom-model-card JSON] [--device {cpu,cuda}] [--seed SEED]
                     [--no-final-relax] [--stride STRIDE] [--save-data] [--save-traj]
                     [--traj-format {pdb,xsf,xyz}] [-sp SYMPREC] [--save-report]
                     [--view] [-o OUTPUT] [-v] [--no-intro]
```

Options

`-f, --file FILENAME`
: Path to the input crystalline structure file (.fdf), typically a supercell (e.g. from `stb-supercell`). Must be bulk (3D periodic, no vacuum-padded axis). required

`--melt-temp MELT_TEMP`
: Melt temperature in K (default: 3000.0). type: float

`--melt-steps MELT_STEPS`
: MD steps held at `--melt-temp` (default: 500). type: int

`--quench-temp QUENCH_TEMP`
: Final target temperature in K (default: 300.0). type: float

`--quench-steps QUENCH_STEPS`
: MD steps for a linear cooling ramp from `--melt-temp` to `--quench-temp` (default: 1000). type: int

`--timestep TIMESTEP`
: MD timestep in fs (default: 1.0). type: float

`--taut TAUT`
: Berendsen temperature-coupling time constant, fs (default: 50.0). type: float

`--taup TAUP`
: Berendsen pressure-coupling time constant, fs (default: 200.0). type: float

`--compressibility COMPRESSIBILITY`
: Compressibility in 1/bar for the Berendsen barostat (default: 4.57e-5, water's value – a generic placeholder; override with your material's real compressibility if known, 1/(B[GPa] x 1e4) – e.g. Si, B ~ 98 GPa: 1e-6). type: float

`--model {small,medium,large}`
: MACE-MP-0 model size: speed/accuracy tradeoff (default: medium). choices: small, medium, large

`--custom-model PATH`
: Path to a custom fine-tuned .model file, instead of a MACE-MP-0 foundation size.

`--custom-model-card JSON`
: Model card of the `--custom-model` file: the `mlff_model_card.json` that `stb-mlffAnalysis` wrote next to it (training data, level of theory, accuracy, energy scale). Required with `--custom-model`: the card is checked against the model file and used to warn when a structure is outside what the model was trained on.

`--device {cpu,cuda}`
: Device to run the model on (default: cpu). choices: cpu, cuda

`--seed SEED`
: Seed for the initial Maxwell-Boltzmann velocities (default: not reproducible). type: int

`--no-final-relax`
: Skip the final static (position+cell) relax. On by default: the MD quench alone leaves real residual thermal energy, not a clean local minimum. switch

`--stride STRIDE`
: Sample one diagnostics point / trajectory frame every Nth MD step, across both the melt and quench stages (default: 10). Only used if `--save-data` and/or `--save-traj` is given. type: int

`--save-data`
: Write `<stem>_md_diagnostics.dat` + a companion .gplot (step, temperature, E_pot/E_kin/E_total, volume) for the whole melt-quench process, plottable with gnuplot. Off by default. switch, off by default

`--save-traj`
: Write a multi-frame trajectory of the melt-quench MD (see `--traj-format`) for viewing in OVITO/VMD. Off by default. switch, off by default

`--traj-format {pdb,xsf,xyz}`
: Trajectory format for `--save-traj` (default: xsf). Same 3 choices as `stb-ani2traj/stb-mlmd`.choices: pdb, xsf, xyz

`-sp, --symprec SYMPREC`
: Symmetry tolerance in Angstroms for the before/after symmetry analysis (default: 0.01). type: float

`--save-report`
: Also persist the full run report (including the symmetry analysis) to `stb_amorphize_report.txt`. Off by default. switch, off by default

`--view`
: Open an interactive 3D view (via ASE) comparing the input crystalline structure and the final amorphized structure (page through frames in ase-gui) after writing the output file. Needs a display. Off by default.switch, off by default

`-o, --output OUTPUT`
: Output .fdf file name (default: `amorphous.fdf`).

Like every STB command it also accepts `-h`/`--help`, `-v`/`--version` and `--no-intro` (Section 3.3).

From the built-in help.

```
Usage examples:
  stb-amorphize -f bulk_supercell.fdf
  stb-amorphize -f bulk_supercell.fdf --melt-temp 4000 --quench-steps 2000
  stb-amorphize -f bulk_supercell.fdf --save-report --view
  stb-amorphize -f bulk_supercell.fdf --save-data --save-traj --traj-format xsf

Notes:
  - Bulk (3D periodic) structures only -- see the description above.
```

10.12 Random Crystal Generator

At a glance

Menu code: 4.12 **Command:** `stb-crystalcast`
Guided example: `examples/4.12-stb-crystalcast/`
Summary: Cast random bulk/layer/rod/cluster structures (atomic or molecular, optionally ML-ranked) from a symmetry group and composition; or substitute/subgroup/supergroup an existing structure.

10.12.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/4.12-stb-crystalcast/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

10.12.1.1 What this tool does

`stb-crystalcast` wraps **pyxtal** to generate or transform structures via symmetry groups, in 4 modes:

- **`generate`** (default, no mode flag) — give a symmetry group + a composition, pyxtal randomly assigns atoms to compatible Wyckoff positions and draws the free coordinates. This is the **inverse** of `stb-crystalbuilder` (4.8): there, you already know the exact Wyckoff sites and want to build; here, you only know the composition and want pyxtal to find valid candidate placements for you (e.g. as starting guesses for structure prediction).
- **`--substitute`** — swap elements in an existing structure while preserving the symmetry framework (every atom keeps its Wyckoff site).
- **`--subgroup`** / **`--supergroup`** — search for a related lower-/higher-symmetry structure of an existing one. The only symmetry-breaking/-restoring search anywhere in this suite.

`--analyze` (Wyckoff decomposition of an existing structure) was removed in this pass — use `stb-symmetry` or `stb-unitcell` instead to inspect an existing structure's own symmetry.

10.12.1.2 How this differs from similar-sounding tools in the suite

Several other tools also involve "symmetry" or "generating a structure from scratch" — none of them do what `stb-crystalcast` does, but it's easy to confuse them at a glance. `example_4.12.sh`'s first case builds the *exact same* real rock-salt NiO through both `stb-crystalbuilder` and `stb-crystalcast` live, side by side, as direct proof of the first row below:

Tool	What it actually does	Why it's not `stb-crystalcast`
`stb-crystalbuilder` (4.8)	Builds a structure from a space group + the exact Wyckoff sites *you already know*	The **inverse** problem: you supply the placement, it expands it. It has no composition to check against — give it the wrong site and it silently builds a different, but still "valid", structure. `stb-crystalcast` only ever needs the composition; it works out (or randomizes) the placement itself.

Tool	What it actually does	Why it's not `stb-crystalcast`
`stb-sqs` (4.6)	Fits a **Special Quasirandom Structure** on a *fixed, already-existing* sublattice, matching real-alloy short-range-order statistics (via `icet`'s cluster expansion)	Solves a completely different problem: it never touches symmetry groups or Wyckoff positions at all — it's about how to arrange species on sites you *already have* so the *local correlations* look like a real random alloy, not about generating a new cell from a group.
`stb-defect` `--all-inequivalent-sites`	Enumerates symmetrically distinct **sites in an existing structure**, to choose where to place ONE vacancy/substitution/interstitial	Uses the same underlying idea (symmetry-equivalent orbits, via `SpacegroupAnalyzer`) as pyxtal's Wyckoff search, but the goal is picking a defect site in a structure you already have, not building/transforming a whole new one.
`stb-symmetry` / **`stb-unitcell`**	Analyze an *existing* structure's own symmetry (operations, equivalent atoms, primitive/conventional cell)	This is what `--analyze` used to do here, before that mode was removed. `stb-crystalcast` **generates or transforms**; it doesn't have an inspection-only mode of its own anymore.
`stb-molecule`	Builds **one** isolated reference molecule (from ASE's G2 database) in a vacuum box	`stb-crystalcast --molecular` instead packs whole rigid molecules (from *pyxtal's own* bundled collection, not G2) **into a periodic symmetry group** — a molecular crystal candidate, not a single isolated reference. `--dim 0` (bare atoms, point group, vacuum box) is the closer analog of `stb-molecule`'s own vacuum convention, but still composition-driven, not a G2 lookup by name.

10.12.1.3 Why this matters (a bit of theory)

Wyckoff positions: why "group + composition" is enough to build a structure. A space/layer/rod/point group's own symmetry operations partition all possible atomic positions into **Wyckoff positions** — orbits of symmetry-equivalent sites. Placing one atom at a Wyckoff site automatically generates every symmetry-equivalent copy of it (the site's *multiplicity*). `generate` mode's whole job is finding a combination of Wyckoff sites whose multiplicities sum to your requested composition (e.g. 4 Ni + 8 O for space group 225), then drawing random coordinates for whichever sites still have free parameters (some Wyckoff sites are fully fixed by symmetry alone — no randomness needed there at all). `--sites` lets you pin specific Wyckoff labels yourself instead of leaving every site random — the same information `stb-crystalbuilder` needs as direct input.

`--subgroup`/`--supergroup`: symmetry-breaking and symmetry-restoring searches. No other tool in this suite does this. `--subgroup` searches for related structures with **lower** symmetry — a controlled symmetry-breaking distortion (`--group-type t/k`, translationengleiche/klassengleiche group-subgroup relations; `--eps` is the coordinate perturbation applied) — useful for generating candidate low-symmetry phases of a known high-symmetry parent (e.g. a real ferroelectric distortion of a cubic perovskite). `--supergroup` does the reverse: given a (possibly distorted) structure and a target *higher*-symmetry group, it searches for the symmetric parent structure group-subgroup theory says it should be a distortion of (`--d-tol` is the maximum displacement tolerated when testing a candidate match). Verified live that this genuinely can go either way for a real distortion: pyxtal sometimes finds a matching parent, and sometimes correctly reports "not found" instead of a wrong answer — both are legitimate outcomes of a real search, not a crash.

A real, verified pymatgen quirk this tool works around. While standardizing this report, live testing surfaced a real inconsistency in pymatgen's `SpacegroupAnalyzer`: for a `--subgroup` candidate distorted with the default `--eps 0.05`, it can report a space group that hasn't changed (`Fm-3m (225)`) *alongside* a point group that implies a genuinely different, lower symmetry (`4/mmm`, not `m-3m`) — an internally inconsistent pair from the exact same analyzer call. Since `--subgroup`/`--supergroup` candidates already carry pyxtal's own authoritative group assignment (literally what the search built towards), `stb-crystalcast` now reports **that** label for "Space Group" in its `[5] SYMMETRY ANALYSIS` table instead of re-detecting it via pymatgen — verified live that this correctly shows `I4/mmm (No. 139)` for a targeted `--subgroup` search, where the pymatgen re-detection alone would have misleadingly still said `Fm-3m (225)`.

`--ml-rank`: now with a choice of model. `--ml-rank` relaxes every generated candidate's positions with a MACE potential and ranks them by relaxed energy — a fast, relative pre-screen for which candidate(s) to prioritize for real DFT, not an absolute DFT formation energy. Previously always used the small MACE-MP-0 foundation checkpoint with no way to choose; `--model`/`--custom-model` (only valid together with `--ml-rank`) now match the same convention every other `--ml-rank`/`--ml-relax` feature in this suite already has (`stb-crystalbuilder`, `stb-defect`, `stb-amorphize`, ...).

10.12.1.4 When you'd reach for it

- A batch of symmetry-valid candidate structures for a given composition, when you don't already know (or want to hand-pick) the exact Wyckoff sites — use `stb-crystalbuilder` instead once you do.
- Screening candidate low-/high-symmetry phases related to a known structure (`--subgroup`/`--supergroup`), e.g. exploring possible ferroelectric distortions of a known high-symmetry parent.
- A quick element-substitution variant of an existing structure that keeps the same symmetry framework (`--substitute`).

10.12.1.5 Two ways to run it

A — direct CLI:

```
stb-crystalcast --group 225 --species Ni O --num-ions 4 8
```

B — interactive `stb-suite` menu:

```
$ stb-suite
Select an option (0-7, or a tool code like 5.1.2): 4.12
```

Both paths call the exact same underlying tool and produce the exact same output — `example_4.12.sh` proves this directly at the end.

10.12.1.6 What every run does (always on)

- **A numbered report** (`[0] RUN METADATA` … `[8] SUMMARY & FILES`) printed to the console and, with `--save-report`, also saved to `stb_crystalcast_report.txt`.
- **Structure validation** (atom proximity, lattice handedness, atomic density) on every candidate written.
- **A symmetry table** — before/after (`--substitute`/`--subgroup`/ `--supergroup`) or requested-vs-detected (`generate`).

- **`references.bib`** — SIESTA + PyXtal always; + MACE/MACE-MP with `--ml-rank`. Previously never written at all in any mode.
- **A provenance header** written into every output `.fdf`: requested group/composition (or input file/substitutions/target group), detected symmetry, and the `--ml-rank` outcome if used.

10.12.1.7 Optional (off by default)

- **`--count`** — generate/keep several independent candidates in one run (numbered `<output>_1.fdf`, `_2.fdf`, ...).
- **`--seed`** — reproducible batches (generation mode; with `--molecular`, only the lattice is reproduced this way, an upstream pyxtal limitation).
- **`--molecular`** — pack whole rigid molecules instead of bare atoms.
- **`--dim 0/1/2`** — point-group cluster / rod group / layer group, instead of the default `--dim 3` space group.
- **`--sites`** — pre-assign specific Wyckoff positions instead of fully random.
- **`--ml-rank`** (+ `--model`/`--custom-model`) — MACE relaxed-energy screening of generated candidates.
- **`--save-report`** / **`--view`** — same as every other stb-suite tool.

10.12.1.8 Files in this folder

- `example_4.12.sh` — the guided walkthrough (**not** an automated test — see `test/4-structures/12-crystalcast/test.sh` for that). No committed input fixtures: like `4.8-stb-crystalbuilder/`, `generate/`–`molecular/` `--dim` build everything from flags; the one structure `--substitute`/`--subgroup`/`--supergroup` need is generated by this script's own first case, not checked in.
- `.gitignore` — excludes `output/`, `references.bib`, and `stb_crystalcast_report.txt`.

10.12.1.9 Running the walkthrough

```
cd examples/4.12-stb-crystalcast
./example_4.12.sh
```

It pauses between sections (`[Press Enter to continue]`) and always starts by wiping its own `output/`. Seven or eight self-contained cases are generated (`ml-rank/` skipped, with a clear message, if the optional `ml` extra isn't installed):

Folder	What it shows
`vs-crystalbuilder/`	The exact same real NiO built two opposite ways — live proof of the comparison table above
`generate/`	Rock-salt NiO from `--group 225` + composition; `--save-report`, provenance header, `references.bib`
`count-and-seed/`	`--count 3 --seed 7`: distinct candidates, reproducible batches
`molecular-and-clusters/`	`--molecular` (water in group 19) and `--dim 0` (isolated point-group cluster)
`substitute/`	`O:S` substitution on the NiO structure, symmetry framework preserved
`subgroup-supergroup/`	Lower-symmetry search, then a higher-symmetry search back
`ml-rank/`	`--ml-rank --model small`: candidates ranked by MACE relaxed energy

10.12.1.10 Try it yourself

```
# Pre-assign specific Wyckoff sites instead of leaving everything random
```

```
stb-crystalcast --group 225 --species Na Cl --num-ions 4 4 --sites 4a 4b

# See every molecule name bundled with pyxtal (usable with --molecular)
stb-crystalcast --list-molecules

# Screen candidates with your own fine-tuned MACE model
stb-crystalcast --group 225 --species Ni O --num-ions 4 8 --count 10 \
    --ml-rank --custom-model my_finetuned.model \
    --custom-model-card mlff_model_card.json
```

10.12.1.11 Flag reference

Flag	Meaning
`--group`	Symmetry group (number or symbol); meaning depends on `--dim`
`--species/--num-ions`	Composition (generation mode)
`--dim`	3=space group (default), 2=layer, 1=rod, 0=point group
`--molecular`	Pack whole rigid molecules instead of bare atoms
`--sites`	Pre-assign specific Wyckoff positions
`--lattice`	Fix the cell instead of estimating it from `--volume-factor`
`--count/--seed`	Batch size / reproducibility (generation, `--subgroup`, `--supergroup`)
`--substitute`	`OLD:NEW` element swap(s) on an existing structure (`-f`)
`--subgroup`	Lower-symmetry search on an existing structure (`-f`)
`--supergroup`	Higher-symmetry search on an existing structure (`-f`, needs `--target-group`)
`--target-group/--group-type/--eps/--d-tol`	`--subgroup/--supergroup` search parameters
`--ml-rank`	Rank generated candidates by MACE relaxed energy
`--model/--custom-model`	MACE model for `--ml-rank` (only valid together with it)
`-o/--output`	Output `.fdf` file name (default `crystalcast.fdf`)
`--save-report`	Persist the full report to disk
`--view`	Open the written structure(s) in ASE's interactive viewer

10.12.1.12 What's next

See `4.8-stb-crystalbuilder/` for the inverse workflow (you already know the exact Wyckoff sites), or `stb-symmetry/stb-unitcell` to inspect an existing structure's own symmetry/Wyckoff decomposition (what `--analyze` used to do here). Take a promising `--ml-rank`-screened candidate into a real SIESTA relaxation next.

10.12.2 Implementation details

Generation mode.. Without a transformation option the tool casts random structures with PyXtal [14]: for a symmetry group (`-group`) of the dimensionality `-dim` – space group (3), layer group (2), rod group (1) or point group (0) – and a composition (`-species` with `-num-ions` per conventional cell), PyXtal picks a random lattice compatible with the group, a volume of `-volume-factor` (default 1.1) times the sum of atomic volumes, and distributes the atoms over Wyckoff positions whose multiplicities add up to the requested counts. `-thickness` (2D) and `-area` (1D) constrain the non-periodic extent, `-lattice` fixes the cell, `-sites` pre-assigns Wyckoff positions (e.g. `4a 4b,4c`), and `-max-attempts` (default 10) bounds the retries when a combination is hard to place. `-count` produces several candidates, written as `<stem>_<i>.fdf`; `-seed` makes the run reproducible. Point-group clusters are placed in a vacuum box

(`-vacuum`). With `-molecular`, the species are molecules placed as rigid units: a name from PyXtal's collection (`-list-molecules`), a file in `xyz`, `gjf`, `g03` or `json` format, or a SMILES string given as `'SMILES.smi'` (needs the optional `rdkit`).

Transformation modes..

`-substitute OLD:NEW . . .`
replace elements of an existing structure (`-f`) while keeping its symmetry framework, e.g. `Cl:F`;

`-subgroup`
list or build structures in a subgroup (`-target-group`; `-group-type t` for translationengleiche, `k` for klassengleiche or `t+k`; coordinate perturbation `-eps`, default 0.05) – symmetry-breaking distortions;

`-supergroup`
search for a higher-symmetry parent within a displacement tolerance `-d-tol` – useful to identify the parent phase of a distorted structure.

Checks and ranking.. Every candidate's actual symmetry is detected and compared with the request (a random placement may land on a higher-symmetry arrangement), and validated. `-ml-rank` relaxes the atomic positions of each candidate with MACE at fixed cell (0.05 eV/Å, 200 steps) and ranks them by total energy; all candidates of one run share the same composition, so the ranking is meaningful, but the cells were random and are not relaxed in this step. Use it to decide which candidates to relax fully with **`stb-mlrelax`** and then SIESTA.

Relation to other tools.. **`stb-crystalbuilder`** (4.8) builds *one* structure from positions you know; this tool explores structures you do not know. The XRD structure-solution workflow (5.6) uses the same casting engine across many groups and ranks the candidates against a measured powder pattern.

10.12.3 Command-line reference

10.12.3.1 `stb-crystalcast`

Casts one or more random structures compatible with a given symmetry group. Give a symmetry group and a composition (species + how many of each) – pyxtal places the atoms on randomly chosen, symmetry-compatible Wyckoff positions for you. This is the inverse of `stb-crystalbuilder`: use crystalbuilder when you already know the exact Wyckoff sites you want, use crystalcast when you want valid candidate structures generated for you (e.g. as starting guesses for structure prediction). `--molecular` packs whole rigid molecules (instead of bare atoms) into the symmetry group. `--ml-rank` pre-screens generated candidates by MACE relaxed energy. `--substitute`/`--subgroup`/`--supergroup` all instead operate on an existing structure read via `-f`/`--file`: `--substitute` swaps elements while preserving the symmetry framework, `--subgroup`/`--supergroup` search for related lower/higher-symmetry structures. (To inspect an existing structure's own symmetry/Wyckoff decomposition instead of generating or transforming one, use `stb-symmetry` or `stb-unitcell`.)

Synopsis.

```
usage: stb-crystalcast [-h] [--substitute OLD:NEW [OLD:NEW ...]] [--subgroup]
                       [--supergroup] [--target-group TARGET_GROUP]
                       [--group-type {t,k,t+k}] [--eps EPS] [--d-tol D_TOL] [-f FILE]
                       [--dim {3,2,1,0}] [--group GROUP] [--molecular]
                       [--list-molecules] [--species SPECIES [SPECIES ...]]
                       [--num-ions NUM_IONS [NUM_IONS ...]]
                       [--volume-factor VOLUME_FACTOR] [--thickness THICKNESS]
```

```
[--area AREA] [--vacuum VACUUM]
[--lattice A B C ALPHA BETA GAMMA] [--sites SITES [SITES ...]]
[--max-attempts MAX_ATTEMPTS] [--count COUNT] [--seed SEED]
[--ml-rank] [--model {small,medium,large}]
[--custom-model PATH] [--custom-model-card JSON]
[--device {cpu,cuda}] [--symprec SYMPREC]
[--angle-tolerance ANGLE_TOLERANCE] [--save-report] [--view]
[-o OUTPUT] [-v] [--no-intro]
```

Options

`--substitute OLD:NEW [OLD:NEW ...]`

Substitute elements in an existing structure (read via `-f/--file`), preserving its symmetry framework: one or more OLD:NEW pairs, e.g. `--substitute` Cl:F Na:K. Mutually exclusive with `--subgroup/--supergroup` and the generation options below.

`--subgroup`

Generate lower-symmetry subgroup variant(s) of an existing structure (read via `-f/--file`) – a symmetry-breaking distortion. Up to `--count` candidates are written (numbered like `--count` > 1 in generation mode). Optionally narrow the search with `--target-group/--group-type/--eps`. Mutually exclusive with `--substitute/--supergroup` and the generation options. switch, off by default

`--supergroup`

Search for higher-symmetry supergroup structure(s) of an existing structure (read via `-f/--file`) towards `--target-group` (required – the installed pyxtal doesn't support auto-searching all possible supergroups). Mutually exclusive with `--substitute/--subgroup` and the generation options below. switch, off by default

`--target-group TARGET_GROUP`

Target space group number. Optional filter for `--subgroup` (only search towards this specific subgroup); required destination for `--supergroup`. type: int

`--group-type {t,k,t+k}`

`--subgroup` only: 't' (translationengleiche), 'k' (klassengleiche), or 't+k' relation to search (default: t). choices: t, k, t+k

`--eps EPS`

`--subgroup` only: perturbation applied to atomic coordinates when breaking symmetry (default: 0.05). type: float

`--d-tol D_TOL`

`--supergroup` only: maximum atomic-displacement tolerance allowed when searching for a higher-symmetry parent (default: 1.0). type: float

`-f, --file FILE`

Input .fdf structure file. Required with `--substitute/--subgroup/--supergroup`, unused otherwise.

`--dim {3,2,1,0}`

Structure dimensionality: 3 = bulk (space group, default), 2 = layer (layer group, periodic in a/b with vacuum along c), 1 = rod/wire (rod group, periodic along c with vacuum in a/b), 0 = isolated cluster (point group, no periodicity). `--molecular` does not support `--dim` 0 (upstream pyxtal limitation). Generation mode only. choices: 3, 2, 1, 0; type: int

`--group GROUP`

Symmetry group identifier, meaning depends on `--dim`: space group number/symbol (dim 3, e.g. 225 or 'Fm-3m'), layer group number 1-80 (dim 2), rod group number 1-75 (dim 1), or point group number/Schoenflies symbol 1-32 (dim 0, e.g. 20 or 'D3d'). Required in generation mode.

`--molecular`

Treat each `--species` entry as a whole rigid molecule instead of a bare element: a name from pyxtal's bundled

collection (see `--list-molecules`), a path to a .xyz/.gjf/.g03/.json file, or a SMILES string as 'SMILES.smi' (needs the optional 'rdkit' package). switch, off by default

`--list-molecules`
Print the molecule names bundled with pyxtal (usable directly as `--species` with `--molecular`) and exit.switch, off by default

`--species SPECIES [SPECIES ...]`
Element symbols (or, with `--molecular`, molecule identifiers) in the structure, e.g. `--species` Ni O. Required in generation mode.

`--num-ions NUM_IONS [NUM_IONS ...]`
Number of atoms (or, with `--molecular`, molecules) of each `--species`, same order and count, e.g. `--num-ions` 4 8. Required in generation mode. type: int

`--volume-factor VOLUME_FACTOR`
Scales the estimated cell volume before placing atoms; raise it if generation keeps failing to fit atoms without overlap (default: 1.1, pyxtal's own default). type: float

`--thickness THICKNESS`
Layer thickness in Ang (`--dim` 2 only; default: chosen automatically by pyxtal). type: float

`--area AREA`
Rod cross-sectional area in Ang^2 (`--dim` 1 only; default: chosen automatically by pyxtal). type: float

`--vacuum VACUUM`
Vacuum padding in Ang around the cluster, on every side (`--dim` 0 only, matches `stb-molecule's --vacuum`; default: 10.0). type: float

`--lattice A B C ALPHA BETA GAMMA`
Fix the cell instead of estimating it from `--volume-factor`, e.g. for matching a known experimental cell: A B C ALPHA BETA GAMMA (Ang/degrees). Not valid with `--dim` 0 (use `--vacuum` instead). For `--dim` 2/1, only the periodic direction(s) are actually honored – the vacuum direction(s) are still sized by `--thickness/--area/pyxtal's` own default, same as without `--lattice`. type: float

`--sites SITES [SITES ...]`
Pre-assign Wyckoff positions instead of leaving every site random: one entry per `--species`, each a comma-separated list of Wyckoff labels summing to that species' `--num-ions`, e.g. `--species` Na O `--num-ions` 4 8 `--sites` 4a 4b,4c. Default: fully random assignment.

`--max-attempts MAX_ATTEMPTS`
Internal retries pyxtal allows itself per structure to find a non-overlapping placement before giving up on it (default: 10). type: int

`--count COUNT`
Number of independent random structures to generate in generation mode, or number of candidates to keep in `--subgroup/--supergroup` mode (default: 1). type: int

`--seed SEED`
Random seed. Same seed + same inputs reproduces the same batch of structures (default: not seeded, different every run). With `--molecular`, only the lattice is reproduced this way – molecule orientations are not (an upstream pyxtal limitation). Unused outside generation mode. type: int

`--ml-rank`
Generation mode only. After generating `--count` structures, quickly relaxes each one's positions with a MACE potential, prints them ranked by relaxed energy, and writes each output file with its relaxed geometry. A relative comparison from a fast ML potential, not an absolute DFT formation energy. switch, off by default

`--model {small,medium,large}`
MACE-MP-0 foundation model size for `--ml-rank` (default: medium). Only valid together with `--ml-rank`. choices: small, medium, large

`--custom-model PATH`

Path to a custom fine-tuned .model file for `--ml-rank`, instead of a MACE-MP-0 foundation size. Only valid together with `--ml-rank`.

`--custom-model-card JSON`

Model card of the `--custom-model` file: the `mlff_model_card.json` that `stb-mlffAnalysis` wrote next to it (training data, level of theory, accuracy, energy scale). Required with `--custom-model`: the card is checked against the model file and used to warn when a structure is outside what the model was trained on.

`--device {cpu,cuda}`

Device to run the MACE model on (default: cpu). choices: cpu, cuda

`--symprec SYMPREC`

Symmetry precision: for generation, used to re-detect each `--dim` 3 candidate's actual space group; for `--substitute/--subgroup/--supergroup`, the tolerance passed to pyxtal's own symmetry detection (default: 0.01, pymatgen's own default, matches `stb-symmetry/stb-unitcell`). type: float

`--angle-tolerance ANGLE_TOLERANCE`

Angle tolerance in degrees for symmetry detection (default: 5.0, matches `stb-symmetry/stb-unitcell`). type: float

`--save-report`

Also persist the full run report (including the symmetry analysis) to `stb_crystalcast_report.txt`. Off by default. switch, off by default

`--view`

Open an interactive 3D view (via ASE) of every structure written by this run (for `--substitute/--subgroup/--supergroup`, the input structure is shown first, as frame 0) – page through frames in ase-gui. Needs a display. Off by default. switch, off by default

`-o, --output OUTPUT`

Output .fdf file name (default: `crystalcast.fdf`). With `--count` > 1 (generation, `--subgroup`, or `--supergroup`), each structure is written as '`<output>_<N>.fdf`'; the tool refuses to run if any of those numbered names already exist, to avoid silently overwriting unrelated files.

Like every STB command it also accepts `-h/-help`, `-v/-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Usage examples:
  stb-crystalcast --group 225 --species Ni O --num-ions 4 8
  stb-crystalcast --group Fd-3m --species Fe O --num-ions 8 16 \
      --count 5 --seed 42 -o spinel.fdf
  stb-crystalcast --group 225 --species Ni O --num-ions 4 8 --count 5 \
      --ml-rank --model medium
  stb-crystalcast --group 225 --species Na Cl --num-ions 4 4 \
      --lattice 5.64 5.64 5.64 90 90 90
  stb-crystalcast --group 225 --species Na Cl --num-ions 4 4 --sites 4a 4b
  stb-crystalcast --dim 2 --group 65 --species C --num-ions 6 --thickness 3.4
  stb-crystalcast --dim 0 --group D3d --species C --num-ions 6 --vacuum 12
  stb-crystalcast --molecular --group 19 --species H2O --num-ions 4
  stb-crystalcast --molecular --group 14 --species aspirin --num-ions 4 -o aspirin.fdf
  stb-crystalcast --list-molecules
  stb-crystalcast --substitute Cl:F -f rocksalt.fdf -o rocksalt_f.fdf
  stb-crystalcast --subgroup -f spinel_1.fdf --count 5 -o distorted.fdf
  stb-crystalcast --supergroup --target-group 225 -f distorted_1.fdf -o parent.fdf
  stb-crystalcast --group 225 --species Ni O --num-ions 4 8 --save-report --view
```

Chapter 11

Workflows

In this chapter

11.1 Stress-Strain . . . 365
11.2 Elastic Constants . . . 379
11.3 Cohesive Energy . . . 394
11.4 Phonons . . . 406
11.5 Convergence Tests . . . 422
11.6 Structure Solution (XRD) . . . 434
11.7 Hubbard U (Linear Response) . . . 449
11.8 Adsorption . . . 464
11.9 NEB / Reaction Path . . . 490
11.10 2D Stacking Fault (Gamma-Surface) . . . 505
11.11 Raman Spectrum . . . 516
11.12 IR Spectrum . . . 534
11.13 Generalized Quasi-Chemical Approximation (GQCA) . . . 549
11.14 Optical Properties . . . 571
11.15 Equation of State . . . 595
11.16 Charge Density Difference . . . 617
11.17 Hirshfeld-I Charge Partitioning . . . 641

Each workflow computes one physical property from several SIESTA calculations, following the prepare–run–analyse cycle of Chapter 5. In the menu every workflow is one item with two or more stages (**5.1.1**, **5.1.2**, …); on the command line each stage is its own command.

Kind of property	Workflows
Mechanical	stress–strain (5.1), elastic constants (5.2), equation of state (5.15), stacking-fault γ-surface (5.10)
Energetics	cohesive energy (5.3), adsorption with BSSE and Gibbs energy (5.8), NEB barriers (5.9), alloy thermodynamics by GQCA (5.13)
Vibrational and optical	phonons (5.4), Raman (5.11), infrared (5.12), dielectric function and optical constants (5.14)
Electronic and charge	Hubbard U by linear response (5.7), charge-density difference (5.16), Hirshfeld-I charges (5.17)
Methodology	convergence tests (5.5), structure solution from powder XRD (5.6)

Two kinds of workflow live in sections of their own: the catalysis workflows (HER, OER) in Catalysis (Chapter 12), and the training of a machine-learned force field on SIESTA data, the MLFF workflow, at the top of ML Simulations (Section 13.1).

Code	Tool	Purpose
5.1	Stress-Strain	Generate strained structures, then extract stress-strain curves and mechanical properties.
5.2	Elastic Constants	Generate deformed structures, then compute the stiffness matrix and elastic moduli.
5.3	Cohesive Energy	Set up bulk + isolated-atom calculations, then compute the cohesive energy per atom.
5.4	Phonons	Generate displaced supercells via Phonopy, then post-process into thermal properties.
5.5	Convergence Tests	Sweep Mesh.CutOff, k-grid density, or PAO.EnergyShift and check energy + relaxed-structure convergence.
5.6	Structure Solution (XRD)	Cast candidate structures across a set of space groups, then rank them by similarity to an experimental powder XRD pattern.
5.7	Hubbard U (Linear Response)	Compute a first-principles Hubbard U via Cococcioni & de Gironcoli's linear-response method, ending in a ready-to-use DFT+U fdf block.
5.8	Adsorption	Prepare a clean slab, an isolated adsorbate, and candidate adsorption sites, generate a BSSE-corrected reference at each site's relaxed geometry, then compute the real DFT adsorption energy per site.
5.9	NEB / Reaction Path	Interpolate a reaction-path band between two already-relaxed endpoint structures you provide (verified same lattice + composition), optionally refined with MACE-MP-0, then compute the DFT-level energy profile and an approximate barrier.
5.10	2D Stacking Fault (Gamma-Surface)	Slide one 2D layer across a grid of lateral offsets, then compute the equilibrium stacking, corrugation energy, and full gamma-surface map.
5.11	Raman Spectrum	Self-contained: computes its own phonons, then the +/-delta Optical (dielectric-response) displacements needed for the diagonal Raman tensor, then the approximate Raman-active frequencies and spectrum.
5.12	IR Spectrum	Self-contained: computes its own phonons, then either the +/-delta dipole displacements (0D/1D/2D) or a single equilibrium Born-effective-charge run (3D bulk), then the IR-active frequencies and spectrum.
5.13	Generalized Quasi-Chemical Approximation (GQCA)	Self-contained: builds the 3 ordered pair-cluster-representative structures (AA/AB/BB) for a substitutional alloy A(1-x)Bx, then solves the mass-action equilibrium to get Delta-H_mix/Delta-S_mix/Delta-G_mix(x,T) and short-range-order parameters (Sher et al. …
5.14	Optical Properties	Self-contained: writes one SIESTA folder per Cartesian polarization direction for a linear-response Optical.Calculate run on an already-relaxed structure, then derives eps1(E)/eps2(E), n(E)/k(E), the absorption coefficient, normal-incidence reflectivity, the …
5.15	Equation of State	Generate isotropically-scaled-volume structures, then fit an E vs V equation of state (V0/E0/B0/B0') from the resulting SIESTA energies.

Code	Tool	Purpose
5.16	Charge Density Difference	Generate ghosted fragment folders for an already-relaxed combined system (bilayer, molecule-on-slab, or any N-fragment system), then compute Delta rho = rho_combined - sum(rho_fragment_i) to visualize charge accumulation/depletion regions.
5.17	Hirshfeld-I Charge Partitioning	Iteratively refined Hirshfeld atomic charges for an already-computed combined/production system, via isolated neutral/ion atomic references – corrects simple Hirshfeld's known conservative bias without needing a Bader-style density maximum at every nucleus …

11.1 Stress-Strain

At a glance

Menu code: 5.1 (a multi-stage workflow)

Code	Stage	Command
5.1.1	Stage 1 - Prep	`stb-strain`
5.1.2	Stage 2 - Analysis	`stb-strainAnalysis`

Guided example: `examples/5.1-strain/`

Summary: Generate strained structures, then extract stress-strain curves and mechanical properties.

11.1.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/5.1-strain/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

This workflow has 2 stages: **Stage 1** (`stb-strain`, code `5.1.1`) deforms a relaxed structure by a small Cartesian strain and writes ready-to-run SIESTA folders, one per strain value; **Stage 2** (`stb-strainAnalysis`, code `5.1.2`) reads the finished SIESTA runs back and fits a stress-strain curve — initial modulus, peak strength, critical strain, toughness, and optionally yield. Together they answer one physical question: **how stiff and how strong is this material along a given direction, and how far can it be deformed before it breaks?**

Both stages live in this one folder and this one tutorial — not one folder per tool — because neither stage means much without the other: Stage 1's output only exists to feed Stage 2, and Stage 2 only exists to interpret Stage 1's output.

11.1.1.1 Theory

Deforming a crystal: the strain tensor. A small, uniform deformation of a crystal lattice is described by a **deformation gradient** `F = I + ε`, where ε is the (small) **strain tensor** and `I` is the identity — every lattice vector `a` maps to `F . a`. This is the standard small-strain (linear elasticity) approximation: valid as long as ε stays small enough that the crystal's response is still reversible and the higher-order terms of a full nonlinear deformation theory are negligible. `stb-strain` builds `F` exactly this way (`apply_cartesian_strain`) and applies it to the lattice vectors only — atomic positions, given in **fractional** coordinates, scale along with the cell for free, which is exactly why `stb-strain` requires fractional input and refuses Cartesian coordinates outright (a Cartesian position would NOT follow the deformed cell automatically).

`stb-strain` supports two families of deformation:

- **Uniaxial** (`x`, `y`, `z`) — ε has exactly one nonzero diagonal component. Physically: stretching (or compressing) the crystal along one Cartesian axis only.
- **Biaxial** (`xy`, `xz`, `yz`) — ε has two equal nonzero diagonal components, the two named axes strained together by the same amount (e.g. `xy` means ε_xx = ε_yy = ε, ε_zz = 0). This is the natural choice for probing a 2D material's **in-plane** response, since a single in-plane axis alone doesn't respect the material's own in-plane symmetry the way straining both together does.

Stress and strain components throughout this workflow (and the rest of this suite) use the standard **Voigt notation**: the 6 independent components of a symmetric 3×3 tensor collapsed into one index, `1..6 = XX/YY/ZZ/YZ/XZ/XY` — the same numbering SIESTA itself uses for `%block Geometry.Constraints` and its own `Stress tensor Voigt` output line.

What the "initial slope" measures — and what it doesn't. Stage 2 fits a straight line to the stress-strain data at small strain and reports its slope as the direction's **initial modulus** — a stiffness, in the same spirit as Young's modulus (`stress = modulus × strain`, Hooke's law, in the linear-elastic small-strain limit). But it is **not**, in general, the same number as a rigorous elastic constant `C_ij` from the full elastic tensor:

- With `--relax-mode cell-fixed`, every OTHER Voigt component is held exactly fixed while only the imposed one is strained — a "clamped -cell" measurement.
- With `--relax-mode stress-constrained`, the other periodic components are left free to relax to zero stress instead — a "relaxed-transverse -stress" measurement.

Both are legitimate, standard DFT conventions (see `stb-strain`'s own `--relax-mode` docs), and both differ from the *fully coupled* response a true `C_ij` requires — that needs the sibling workflow **`stb-elasticInputs/stb-elasticAnalysis`** (elastic constants), which fits a whole system of coupled strain directions simultaneously. `stb-strainAnalysis`'s own report says so explicitly: `[INFO] Single-direction slope under a clamped transverse cell -- for the rigorous small-strain elastic-tensor Young's Modulus, use stb-elasticAnalysis.`

Think of this workflow's own modulus as **"how stiff is this material if you pull on it exactly like this, under exactly this transverse boundary condition"** — directly meaningful and reproducible in its own right, just not automatically equal to a textbook single-crystal elastic constant unless the two conventions happen to coincide (they do, for the right symmetry and relax-mode combination).

Reading a stress-strain curve: modulus, UTS, critical strain, toughness, yield. A typical stress-strain curve for a solid rises roughly linearly at small strain, curves over, reaches a peak, and then falls (or plateaus) as the material's bonding network is pushed past its ability to sustain load. Stage 2 extracts 4 numbers (and a 5th, optional one) that summarize this shape:

Quantity	What it is	Physical meaning
Initial Slope	linear fit at small strain (section 1.2)	stiffness — how much stress it takes to produce a given small strain
Peak Stress/Force (UTS)	the single largest value on the curve	the "ultimate tensile strength" — the most load-bearing capacity this direction ever exhibits in the tested range
Critical Strain	the strain at which the peak occurs	how far you can deform the material before it starts to soften/fail

Quantity	What it is	Physical meaning
Toughness	the area under the whole curve ($\int$ `stress d(strain)`)	energy absorbed per unit volume (3D) / area (2D) / length (1D) before failure — a fracture-resistance measure, distinct from stiffness or strength
Yield (0.2% offset), opt-in via `--yield`	where the curve departs from the initial-slope line by a strain offset of 0.002	a **macroscopic metallurgy concept** (onset of dislocation-glide plasticity) — reported for comparison, but a perfect, defect-free periodic crystal under affine DFT strain has no dislocation-nucleation mechanism to actually yield through; interpret with care

If the peak sits at either edge of the strain range you actually tested (`--stmin`/`--stmax`), Stage 2 flags it explicitly — the true peak may lie beyond what you scanned, and the reported UTS/critical-strain values are only a **lower bound** in that case.

Dimensionality changes the physical quantity, not just the unit. SIESTA always reports a genuine 3D stress (force per unit area), computed from the **full simulation cell**, vacuum included. That's the right quantity as-is for a bulk (3D) structure — but not for a 2D sheet or a 1D wire, where the vacuum padding is an arbitrary simulation-box choice, not a physical thickness:

- **3D (bulk)** — Stress in **GPa** (`1 kBar = 0.1 GPa`), used directly.
- **2D (a slab/sheet, vacuum along z)** — SIESTA's own stress is diluted by the *entire* cell height, vacuum included, which is arbitrary (you could always add more vacuum and get a smaller number for the same physical sheet). Multiplying back by that same cell height cancels the arbitrary dilution and recovers the physically meaningful quantity: **force per unit width, in N/m** — exactly the convention the 2D-materials literature itself uses (e.g. graphene's own in-plane stiffness is conventionally quoted in N/m, around ~340 N/m, not as a "3D stress in GPa" that depends on an arbitrarily chosen layer thickness).
- **1D (a wire/tube, vacuum along x and y)** — same problem, one dimension worse: there's no natural "cross-section" at all without assuming a wire wall thickness. Instead of a made-up stress, Stage 2 reports the raw **axial Force, in nN** (`stress × cell cross-section`, independent of how much vacuum padding was chosen). If you *do* know the wire's real physical cross-section, `--cross-section` additionally back-derives a "conventional" GPa-equivalent stress for comparison.

Stage 2 auto-detects which of these three applies (section 3.2) instead of asking you to already know it.

Symmetry reduces how many directions you actually need. By Neumann's principle, if some symmetry operation of the crystal's point group maps one direction onto another, straining either one must give the identical mechanical response — computing both would just repeat the same physics twice. `stb-strain`'s `[3] AXIS SYMMETRY ADVISORY` (uniaxial) and the interactive menu's full direction-picker (both uniaxial and biaxial, section 2.5) detect this automatically from the structure's own point group, so you only ever need to actually run the symmetry-distinct directions.

11.1.1.2 2. Stage 1: generating strained structures (`stb-strain`, code `5.1.1`)

What it does. `stb-strain` takes a relaxed structure and a reference `calc.fdf`, applies the deformation from section 1.1 to the lattice vectors for a range of strain values, and writes one ready-to-run folder per value: the deformed structure, a copy of `calc.fdf`, a new `config_extra.fdf` (see 2.3), and any linked

pseudopotentials. It does **not** run SIESTA itself — you run each folder yourself, then hand the results to Stage 2 (section 3).

2.2 The `calc.fdf` you pass in — step by step. Golden rule: `--calc` must be the exact same file used for the structure's original relaxation — same basis set, k-points, XC functional, SCF settings. `stb-strain` doesn't regenerate any of that; it only strains the lattice and adds one `%include` line (section 2.3), so the physics of the strained evaluation stays consistent with whatever produced `structure.fdf` in the first place. This example's `calc.fdf` is exactly that: a plausible relaxation input for `structure.fdf`'s 8-atom bulk Si cell (DZP basis, an 8x8x8 Monkhorst-Pack grid, PBE, standard SCF tolerances).

Two settings in that file matter specifically for Stage 1, and `stb-strain`'s own `[4]` report section checks (and warns about) both, **without ever forcing either one**:

1. **`MD.VariableCell` stays `true`.** `stb-strain` itself never disables cell relaxation — that job belongs to the `%block Geometry.Constraints` it writes into the new `config_extra.fdf` (section 2.3), which only has *any* effect while SIESTA's cell is actually variable in the first place. Turn `MD.VariableCell` off (or leave it out) and the constraints block becomes a no-op — SIESTA never even looks at which stress components are fixed, since the cell was never going to move anyway.
2. **`MD.Steps` stays non-zero** (this is the real SIESTA keyword — the older `MD.NumCGsteps` spelling is also recognized as a fallback). This is the ionic-relaxation step count; with `0`, nothing relaxes no matter which `--relax-mode` you pick.

`calc.fdf` in this folder has both right (`MD.VariableCell true`, `MD.Steps 100`) — open it and read the comment above its `STRUCTURE RELAXATION` block. `calc_missing_relax.fdf`, also in this folder, is the same file with both **deliberately wrong** (no `MD.VariableCell`, `MD.Steps 0`), used only to demonstrate the warnings below live — never use it as a real template.

The gotcha, live.

```
$ stb-strain -s structure.fdf -c calc_missing_relax.fdf --relax-mode cell-fixed --stdir x --stmin 0
--stmax 1 --no-intro
...
[4] RELAXATION MODE & CELL CONSTRAINTS
Calc template (current state, read-only -- nothing below is forced):
  MD.TypeOfRun=CG  Steps: MD.Steps=0  MD.VariableCell=(absent)
[WARNING] MD.VariableCell is not enabled in this calc.fdf (or is absent).
The %block Geometry.Constraints written below only has an effect while
the cell is actually variable -- add 'MD.VariableCell true' to your
--calc yourself; this tool does not force it.
[WARNING] Relaxation step count is MD.Steps=0 -- with 0 (or no) steps,
atomic/cell relaxation is a no-op regardless of --relax-mode. Set
MD.Steps (or MD.NumCGsteps) > 0 in your --calc for this mode to actually
relax anything.
```

Both warnings fire together, and the run still completes (they're warnings, not hard errors — `stb-strain` can't force a directive that lives entirely in *your* file, so it tells you what's wrong instead).

2.3 `--relax-mode`: two conventions, one mechanism. Both `--relax-mode` choices (introduced physically in section 1.2) are expressed through the exact same mechanism: a `%block Geometry.Constraints` with `stress N` lines (Voigt numbering, section 1.1), written into a new `config_extra.fdf` file and included via `%include config_extra.fdf` inserted right after `%include structure.fdf` in every generated `calc.fdf` copy. Nothing else in `calc.fdf` is touched. The 2 modes only differ in **which** components that block fixes:

- **`cell-fixed`** — fixes all 6 components. The cell stays exactly at the imposed strain (the same practical effect as disabling `MD.VariableCell` entirely, just expressed as constraints so both modes share one mechanism); only the ions relax, via `calc.fdf`'s own `MD.Steps`.
- **`stress-constrained`** — fixes only the imposed strain direction's own component(s); every other periodic direction is left free to relax to zero stress. (If the structure had a vacuum-padded axis — a 2D slab, say — that axis's own component would *also* be fixed automatically, to protect it from spurious relaxation; this bulk 3D example has no vacuum axis, so that rule never triggers here.)

Side by side.

```
$ stb-strain -s structure.fdf -c calc.fdf --relax-mode cell-fixed --stdir x ...
config_extra.fdf:
  %block Geometry.Constraints
    stress 1  # Fixes XX
    stress 2  # Fixes YY
    stress 3  # Fixes ZZ
    stress 4  # Fixes YZ
    stress 5  # Fixes XZ
    stress 6  # Fixes XY
  %endblock Geometry.Constraints

$ stb-strain -s structure.fdf -c calc.fdf --relax-mode stress-constrained --stdir x ...
config_extra.fdf:
  %block Geometry.Constraints
    stress 1  # Fixes XX
  %endblock Geometry.Constraints
```

`stress-constrained` is the physically softer constraint (only 1 of the 6 components fixed here) — the standard "relaxed-ion, relaxed-transverse -stress" method for elastic-constant work; `cell-fixed` is the stricter "clamped-cell, relaxed-ion" method. Both are legitimate; which one you want depends on which elastic-constant convention you're computing.

Symmetry reduces the direction list (CLI advisory). `structure.fdf` is cubic bulk silicon (point group `m-3m`) — one of the few point groups where every Cartesian axis really is mechanically equivalent (section 1.5). `stb-strain`'s `[3] AXIS SYMMETRY ADVISORY` section detects this automatically and tells you so, before you waste a calculation straining `y` or `z` too:

```
[3] AXIS SYMMETRY ADVISORY
  Detected symmetry : point group m-3m -- 48 operation(s) ...
  Axis    Status        Equivalent to
  x       REQUESTED     --
  y       EQUIVALENT    x
  z       EQUIVALENT    x
  y, z are equivalent to 'x' by symmetry -- straining them should give
  the same mechanical response; you may not need to compute both.
```

Purely advisory — it never blocks the run, and biaxial directions are out of scope for this check (see `core/symmetry.py`'s own docstring for why).

The interactive menu's full direction-picker. The `[3] AXIS SYMMETRY ADVISORY` above is CLI-only, uniaxial-only, and purely informational — it tells you `y`/`z` are redundant but doesn't stop you from running them anyway, and it has nothing to say about biaxial directions at all. The interactive menu (`stb-suite` → `5.1.1`) does the equivalent check for **both** uniaxial and biaxial directions, together with vacuum-axis

detection, and turns it into an actual choice instead of just a warning: it detects everything up front, then only offers the directions that are physically valid (non-vacuum) and not symmetry-redundant.

Biaxial equivalence is derived from the exact same axis-symmetry groups the CLI's own advisory uses, just keyed by each biaxial direction's *excluded* axis instead of its own axes — `xy` excludes `z`, so it's equivalent to another biaxial direction whenever a point-group operation maps their excluded axes onto each other. No separate biaxial symmetry computation is needed.

On this folder's own `structure.fdf` (cubic Si, `m-3m`, no vacuum), the menu collapses from 6 possible directions down to 2 independent ones:

```
------------------------------------------------------------
            STRAIN DIRECTION SELECTION
------------------------------------------------------------
  Detected symmetry : point group m-3m -- 48 operation(s) ...
------------------------------------------------------------
  Direction  Type      Status        Notes
  ----------------------------------------------------------
  x          uniaxial  INDEPENDENT   --
  y          uniaxial  REDUNDANT     equivalent to 'x'
  z          uniaxial  REDUNDANT     equivalent to 'x'
  xy         biaxial   INDEPENDENT   --
  xz         biaxial   REDUNDANT     equivalent to 'xy'
  yz         biaxial   REDUNDANT     equivalent to 'xy'
  ----------------------------------------------------------
  2 independent, non-vacuum direction(s): x, xy
------------------------------------------------------------

  1) x
  2) xy
  3) ALL independent UNIAXIAL directions (x)
  4) ALL independent BIAXIAL directions (xy)
  5) ALL independent directions -- uniaxial + biaxial (2 total)

Select a direction [1-5]:
```

Picking `5` runs Stage 1 once per independent direction (`x` and `xy` here), each into its own `strain_runs/<direction>/` subfolder (section 2.6) — with no separate y/n follow-up prompt needed, and without ever wasting a real calculation on `y`, `z`, `xz`, or `yz`. `3` and `4` are the same idea scoped to just one strain type — handy when a structure has several independent directions of each kind and you only want, say, every uniaxial one.

On a vacuum-padded structure (a 2D slab, say), any direction touching the vacuum axis is marked `VACUUM` instead and dropped from the menu entirely — `stb-strain`'s own CLI would otherwise hard-error if you asked for one of those by hand. If the structure can't be read, or the symmetry pre-check fails for any reason, the menu falls back to the old free-text prompt (`x`, `y`, `z`, `xy`, `xz`, `yz`) with no filtering.

Output layout: one subfolder per direction. Every direction gets its own `<output-dir>/<direction>/` subfolder — `strain_runs/x/strain_x_0.00/`, `strain_runs/x/strain_x_2.00/`, and so on — rather than everything flat directly under `strain_runs/`. Run a second direction (`y`) into the same `--output-dir` and it gets its own `strain_runs/y/` alongside, with no collision and no risk of one direction's `--save-report` file overwriting another's. This is exactly the layout Stage 2 (section 3) expects: point `--dir` at `strain_runs` itself and every direction found underneath is read and compared automatically.

Running it both ways. Every command below also works through the interactive menu — `stb-suite` → `5.1.1` (or type `5.1.1` directly from the main prompt) asks the same questions (structure file, calc file,

relax mode, direction, strain range, ...) instead of flags, and calls the exact same `stb-strain` underneath. `example_5.1.sh` proves this directly by running both paths with the same inputs and diffing the generated folders.

```
stb-strain -s structure.fdf -c calc.fdf \
    --relax-mode cell-fixed --stdir x --stmin 0 --stmax 2 --step 2
```

11.1.1.3 3. Stage 2: analyzing the results (`stb-strainAnalysis`, code `5.1.2`)

Report structure. Once you've run SIESTA in every `strain_runs/<direction>/strain_<direction>_<pct>/` folder from Stage 1, Stage 2 reads the finished `.out` files back and computes everything from section 1.3 (modulus, peak, critical strain, toughness, optional yield). It reports in the same numbered `[N]` section style as Stage 1's own report:

```
[0] RUN METADATA          -- the requested directory/file/dimensionality/output settings
[1] INPUT DATA            -- which strain_* folders were found, grouped by direction
[2] DIMENSIONALITY DETECTION
[3] MECHANICAL PROPERTIES -- a quick-reference table (if >1 direction) + one full block per direction
[4] OUTPUT FILES          -- what --save-gnuplot would write / did write
[5] SUMMARY & NEXT STEPS
```

Dimensionality auto-detection. `[2]` looks for a real `.fdf` structure file inside the first `strain_*` folder found (Stage 1 always writes one — the deformed structure, same basename you gave `--structure`) and classifies it via vacuum-axis detection, applying section 1.4's rules exactly like Stage 1's own `[2]` section and `stb-mlelastic`'s `--dimensionality auto`: vacuum along `z` only → 2D (`N/m`); vacuum along `x+y` → 1D (`nN`); no vacuum → 3D (`GPa`, this folder's own case). If no `.fdf` is found at all (a hand-built folder holding only a bare SIESTA `.out`) it falls back to 3D with a `[NOTE]`, not an error — pass `--dimensionality` explicitly (`auto/3d/2d/1d`) to override either way.

Multiple directions are compared automatically, no flag needed. Point `--dir` at Stage 1's top-level output directory (`strain_runs`, holding `x/`, `xy/`, ... subfolders — e.g. from picking Stage 1's menu "ALL" option) and `[1]`/`[3]` report every direction found, side by side, the same way `--dir strain_runs/x` alone still reports just that one direction on its own.

The initial-slope fit window — and a real bug fixed here. `[3]` reports not just the modulus but the fit window itself (how many points, what strain range) and the fit's standard error, e.g. `Fit window : 3 point(s), strain 0.0000% to 4.0000%`. The fit (section 1.2/1.3) prefers every point within ±2% strain; when a sweep's own step size is coarser than that (very common for a large-deformation sweep, e.g. this workflow's own default 25% range at a few-percent step), fewer than 3 points land inside ±2% — the window widens to the nearest few points instead, with an explicit `[NOTE]` saying so.

A real bug was found and fixed here, verified live: the *previous* version of this fallback fit the *entire* sweep (0% all the way to whatever `--stmax` was) whenever too few points fell in ±2%, instead of widening to just the nearest few — silently averaging the genuine small-strain elastic response together with large-strain, manifestly nonlinear/plastic behavior into one number. On a real 2D SiC monolayer dataset (3 directions, 21 steps each, 0-40% strain, 2%/step — so only the 0%/2% points ever fell in ±2%), that gave a "modulus" with R^2 as low as ~0.01 (statistically meaningless) instead of the correct near-zero-strain slope ($R^2 > 0.99$ once the window widens to the nearest 3 points instead; see the worked example, section 4). The fit line drawn on both the gnuplot and matplotlib output is now also clipped to the actual fit window (never drawn across the full strain range, which would otherwise visually claim the material follows that straight line all the way to its peak/plastic regime).

Plotting: gnuplot and matplotlib, both opt-in, points-and-line. `--save-gnuplot` writes `<direction>_curve.dat/.gplot` (or `comparison_curve.dat/.gplot` for multiple directions) — the same `.dat+.gplot` convention used throughout this suite; the `.gplot` script references its own data/output files by bare filename, so run it from *inside* `--output-dir` (`cd <output-dir> && gnuplot <script>.gplot` — Stage 2's own `[4]` section prints this exact command). `--view` opens an interactive matplotlib plot of the same curve instead — shown on screen only, never written to disk. Both draw the stress/force-strain data as connected points-and-line (not bare scatter points), the small-strain fit line (dashed, clipped to its own fit window — section 3.4), and the peak point. `--save-report` persists the numbered report itself to `stb_strainAnalysis_report.txt`. All three default off.

```
$ stb-strainAnalysis --file calc.out --dir strain_runs --save-report --save-gnuplot
...
[3] MECHANICAL PROPERTIES
------------------------------------------------------------
2 direction(s) found -- comparing automatically (no flag needed). Full
detail per direction follows; quick-reference table first:
Direction | Slope (GPa) | R^2     | Peak (GPa) | Crit. Strain (%) | Toughness (GJ/m^3) | Notes
-----------------------------------------------------------------------------------------------
X         | ...         | ...     | ...        | ...              | ...                | --
XY        | ...         | ...     | ...        | ...              | ...                | --

Direction       : X
...
  Fit window    : 3 point(s), strain 0.0000% to 4.0000%
...
```

Running it both ways. Same as Stage 1 (section 2.7): `stb-suite` → `5.1.2` asks the same questions (directory, output filename, dimensionality — `auto`/`3d`/`2d`/ `1d`, advanced settings, yield, output directory, save-report/ save-gnuplot/view) instead of flags, and calls the exact same `stb-strainAnalysis` underneath.

11.1.1.4 Worked example: a real 2D material, end-to-end

Everything above was verified live against a real, fully-converged SIESTA dataset: a 2D SiC monolayer (fetched via `stb-fetch` from an OPTIMADE provider, then run through this exact workflow), 3 directions (`x`, `y`, `xy`), 21 strain steps each, 0-40% at 2%/step, all 63 SIESTA runs converged normally.

```
$ stb-strainAnalysis --file calc.out --dir strain_runs --save-gnuplot
...
[2] DIMENSIONALITY DETECTION
[INFO] Auto-detected from a real structure file -- dimensionality: 2D
Dimensionality    : 2D (auto-detected)
[INFO] Detected cell Z-height: 20.7697 Ang (used for N/m conversion)

[3] MECHANICAL PROPERTIES
Direction | Slope (N/m) | R^2     | Peak (N/m) | Crit. Strain (%)
X         | 154.3192    | 0.9975 | 19.5423    | 20.0000
XY        | 199.1299    | 0.9968 | 15.7829    | 16.0000
Y         | 167.3003    | 0.9999 | 19.7770    | 20.0000
```

Reading this the way section 1 sets up:

- **Dimensionality auto-detected correctly** (section 3.2) — 2D, purely from the real `structure.fdf`'s vacuum gap along `z`, no `--dimensionality` flag needed. The Z-height (20.7697 Å) is this

structure's own vacuum-inclusive cell height, auto-read from the SIESTA output to un-dilute the reported stress into N/m (section 1.4).
- **All 3 directions compared automatically** (section 3.3) — no `--compare`-style flag exists or was needed.
- **The fit-window fix in action** (section 3.4): every direction here only has 2 points within the literal ±2% window (0% and 2%), so the fit widened to the nearest 3 points (0%, 2%, 4%) instead of the old, buggy full 0-40% fit — recovering R^2 of 0.9968-0.9999 (excellent linear fits) where the old code would have given R^2 as low as ~0.01.
- **Physical sanity check**: 154-199 N/m is the right *order of magnitude* for a covalent 2D monolayer's in-plane stiffness — smaller than graphene's own ~340 N/m (expected: SiC's Si-C bonds are weaker and longer than graphene's C-C bonds), and the modest anisotropy between `X`/`Y`/`XY` (not all equal) is itself informative: it means this structure's in-plane point group is lower than a fully hexagonal (isotropic) one — exactly the kind of fact this workflow is built to surface, not something you'd know just from looking at the lattice vectors.
- **Peaks around 16-20% critical strain**, well past the small-strain elastic regime the modulus describes — a reminder that "Initial Slope" and "Peak" describe two different physical regimes of the *same* curve (section 1.3), not the same number scaled differently.

`example_5.1.sh`'s own Stage 2 cases demonstrate the same mechanics against small, hand-built `calc.out` data (since this folder's own Stage 1 run doesn't invoke real SIESTA) — the numbers there are illustrative, not a real material's properties; this section is the genuine, physically verified result.

11.1.1.5 Known, deliberate limitations

- **Not a full elastic tensor** (section 1.2) — a single-direction, clamped-or-relaxed-transverse modulus, not a rigorous `C_ij`. Use `stb-elasticInputs`/`stb-elasticAnalysis` for that (or `stb-mlelastic` for a fast MACE-based preview of the same physics).
- **Biaxial "modulus" is not a true biaxial modulus** — Stage 2 reports only the stress component along the biaxial direction's first axis (e.g. `xy` → the `xx` component), flagged with a `[WARNING]`.
- **Yield strength (0.2% offset)** is a macroscopic metallurgy concept (dislocation-glide onset) — a defect-free periodic crystal under affine DFT strain has no such mechanism; report with care (section 1.3).
- **Peak/critical strain at the edge of the tested range** is only a lower bound on the true peak — widen `--stmin`/`--stmax` if you see this flag.
- **Auto-dimensionality-detection** needs a real, parseable `.fdf` in the scanned folder; a hand-built/legacy folder holding only a bare `.out` falls back to 3D (section 3.2).

11.1.1.6 Step-by-step: running this workflow on your own structure

0. **Prerequisite**: an already-relaxed structure (`structure.fdf`, fractional coordinates) and the exact `calc.fdf` used for that relaxation (the golden rule in section 2.2) — this workflow strains an existing equilibrium structure, it doesn't find one for you.
1. **Decide `--relax-mode`** (section 2.3): `cell-fixed` (clamped-cell) or `stress-constrained` (relaxed-transverse-stress) — pick the DFT convention you actually want. Stage 1 is cheap (it only writes `.fdf` files, no SCF), so it costs nothing to redo with the other mode later if you change your mind.
2. **Run Stage 1** (`stb-strain`, code `5.1.1`):
   - CLI: `stb-strain -s structure.fdf -c calc.fdf --relax-mode <mode> --stdir <direction> --stmin <min> --stmax <max> --step <step>`

- Interactive: `stb-suite` → `5.1.1` — detects vacuum axes and symmetry-equivalent directions up front (section 2.5) and offers a numbered menu of only the independent, non-redundant directions (or an "ALL" option to run several in one pass).
- Either way, this writes `strain_runs/<direction>/strain_<direction>_<pct>/` folders (section 2.6): the deformed structure, `calc.fdf` (with the new `%include` line added), `config_extra.fdf`, and any linked pseudopotentials.

3. **Check the symmetry advisory** — the CLI's own `[3] AXIS SYMMETRY ADVISORY` (uniaxial only, section 2.4) or the interactive menu's full direction-selection table (uniaxial + biaxial, section 2.5). If two directions are symmetry-equivalent, you only need to actually compute one of them.
4. **Run SIESTA yourself in every generated folder.** Stage 1 never runs SIESTA — `cd` into each `strain_<direction>_<pct>/` and run it exactly like any other SIESTA calculation using its own `calc.fdf` (e.g. `siesta calc.fdf > calc.out`, or through your own batch/queue script). Repeat for every `<direction>/` subfolder if you generated more than one.
5. **Run Stage 2** (`stb-strainAnalysis`, code `5.1.2`) once every run finishes:
   - CLI: `stb-strainAnalysis --file calc.out --dir strain_runs` — point `--dir` at the SAME top-level directory Stage 1 wrote into. Exactly one direction found underneath → a single-direction report; more than one → compared automatically, no flag needed (section 3.3).
   - Interactive: `stb-suite` → `5.1.2` — the same questions (directory, output filename, dimensionality override, yield, output options) instead of flags.
   - Dimensionality (GPa/N/m/nN) is auto-detected from the real structure file Stage 1 wrote into each folder (section 3.2) — nothing to set manually in the common case.
6. **Read the `[3] MECHANICAL PROPERTIES` report** (sections 1.3/3.1): initial modulus (with its own fit window and R^2, section 3.4), peak stress/force (UTS) and critical strain, toughness, and yield if you passed `--yield`. Pay attention to any `[WARNING]` — peak at the edge of the tested range means widen `--stmin`/`--stmax` and rerun; a biaxial direction only ever reports its first-axis component, not a true biaxial modulus (section 5).
7. **(Optional) persist or plot the result**: `--save-report` writes the full report to `stb_strainAnalysis_report.txt`; `--save-gnuplot` writes `<direction>_curve.dat/.gplot` (render with `cd <output-dir> && gnuplot <script>.gplot`, section 3.5); `--view` opens the same curve interactively in matplotlib instead.
8. **(Optional) go further**: for the rigorous full elastic tensor instead of this workflow's own single-direction modulus, see `stb-elasticInputs`/`stb-elasticAnalysis` (section 9).

11.1.1.7 Files in this folder

File	What it is
`structure.fdf`	Bulk Si, 8-atom conventional cubic cell, fractional coordinates
`calc.fdf`	The (correct) relaxation calc.fdf for `structure.fdf`
`calc_missing_relax.fdf`	Same file, deliberately misconfigured — for the gotcha demo only
`example_5.1.sh`	This walkthrough's runnable script (both stages)

11.1.1.8 Running the script

```
bash example_5.1.sh
```

11.1.1.9 What's next

- **stb-elasticInputs/stb-elasticAnalysis** (Workflow 5.2, see ../5.2-elastic/) — the rigorous full elastic tensor C_ij this workflow's own modulus is deliberately NOT a substitute for (section 1.2/5).
- **stb-mlelastic** (ML Simulations) — a fast MACE-based preview of the same stiffness-tensor physics, no SIESTA/strain_* folders needed; a good sanity check before committing to the real DFT workflow here.
- **5.3-cohesive** (stb-cohesive/stb-cohesiveAnalysis, Workflow 5.3) — a different mechanical-stability question (cohesive energy per atom) about the same kind of structure.
- **stb-eosInputs+stb-eosAnalysis** (5.15) — bulk modulus from the curvature of the energy-volume curve instead, an independent cross-check on a 3D structure's stiffness against this workflow's own stress-strain-derived value.

11.1.2 Implementation details

Stage 1 (stb-strain).. The structure file (-s, fractional coordinates required) is deformed for every strain ε from -stmin to -stmax in steps of -step (all in per cent; negative values compress) with the small-strain deformation gradient

$$\mathbf{F} = \mathbf{I} + \boldsymbol{\varepsilon}, \qquad \mathbf{a}_i' = \mathbf{F}\,\mathbf{a}_i,$$

where $\boldsymbol{\varepsilon}$ is diagonal with ε on the axis named by -stdir: x, y, z (uniaxial) or xy, xz, yz (equi-biaxial, both axes stretched by ε – not a shear). Straining a vacuum axis is refused. Only %block LatticeVectors is rewritten (in the units of the file's own LatticeConstant); fractional positions follow the cell. Every folder <output-dir>/<direction>/strain_<direction>_<pct>/ holds the strained structure, the copied calc.fdf with an %include config_extra.fdf placed right after its structure include, the config_extra.fdf and the pseudopotentials (-p).

The two relaxation modes.. config_extra.fdf contains a %block Geometry.Constraints with stress N lines (Voigt order 1–6 = XX, YY, ZZ, YZ, XZ, XY):

cell-fixed
: all six components fixed: the cell stays at the imposed strain and only atoms relax (relaxed-ion, clamped-cell stress);

stress-constrained
: only the strained component(s), and every component involving a vacuum axis, are fixed; the other periodic directions relax to zero stress, so a uniaxial strain lets the transverse directions contract (Poisson effect).

The constraints act only if calc.fdf relaxes the cell (MD.VariableCell T) and has a non-zero number of steps (MD.Steps or MD.NumCGsteps); the tool reads the template, warns when either is missing, and never forces them. The report also lists the symmetry-equivalent directions, so equivalent strains are not computed twice.

Stage 2 (stb-strainAnalysis).. For every direction found under -dir (default strain_runs) the tool reads the final stress of each run from the log named by -f (Voigt line, kBar), converts it and fits:

- *units*: 3D – stress in GPa (1 kBar = 0.1 GPa); 2D – force per length in N/m, $\sigma_{2\mathrm{D}} = \sigma\, c$ with c = -thickness, the cell height along the vacuum (default 20 Å; 1 kBar·Å = 0.01 N/m); 1D – axial force in nN, stress times the cell cross-section (which cancels the vacuum), plus a conventional stress if

`-cross-section` gives the wall area. Dimensionality is detected from a structure file in the folders, or forced with `-dimensionality`;

- *initial modulus*: linear regression of stress on strain over the points with $|\varepsilon| \leq 2\ \%$ (at least three points; the window widens otherwise), with R^2 and standard error;
- *ultimate strength* (maximum stress) and the strain where it occurs, flagged when it is the last point (the true maximum may lie beyond the scan);
- *toughness*: $\int \sigma\, d\varepsilon$ over the scan (trapezoidal);
- *yield* (only with `-yield`): the 0.2 % offset criterion of metallurgy, included for comparison only – an ideal periodic crystal under affine strain has no dislocations to yield through.

Transverse stresses are printed so the Poisson response of a stress-constrained run can be checked. All directions found are compared in one table and one plot.

Ideal strength

The maximum of the curve is the *ideal* (theoretical) strength of a perfect crystal under homogeneous strain, typically much larger than measured strengths, which are limited by defects. Small cells can also miss instabilities with a longer wavelength than the cell (phonon instabilities); check the phonons of a strained cell before reading the curve beyond its first maximum.

11.1.3 Command-line reference

11.1.3.1 `stb-strain`

Stage 1 of 2: applies strain in Cartesian coordinates to a SIESTA structure and writes one ready-to-run folder (structure + `calc.fdf` + `config_extra.fdf` + pseudopotentials) per scanned strain value. Type (uniaxial/biaxial) is inferred from direction; `--relax-mode` selects how the cell is allowed to respond (see its own help). Doesn't run SIESTA – run each folder yourself, then use `stb-strainAnalysis`.

Synopsis.

```
usage: stb-strain [-h] -s STRUCTURE -c CALC
                  --relax-mode {cell-fixed,stress-constrained} [-p PSEUDO_DIR]
                  --stdir STDIR [--stmin STMIN] [--stmax STMAX] [--step STEP]
                  [-o OUTPUT_DIR] [--vacuum-gap VACUUM_GAP] [--symprec SYMPREC]
                  [--angle-tolerance ANGLE_TOLERANCE] [--save-report] [-v]
                  [--no-intro]
```

Options

`-s, --structure STRUCTURE`
: Input `structure.fdf` – lattice + atomic coordinates. Only this file's %block LatticeVectors is rewritten per strain step; everything else is preserved verbatim. Atomic coordinates MUST be fractional (checked in [1] of the report – Cartesian positions would not scale with the strained cell). required

`-c, --calc CALC`
: `calc.fdf` template – SCF/basis/MD settings, normally %including the structure file above by name. Copied into every generated folder with one line added: '%include `config_extra.fdf`' (the Geometry.Constraints block `--relax-mode` builds, see [4] of the report) – no other directive in this file is modified. required

`--relax-mode {cell-fixed,stress-constrained}`
: How the cell responds to the imposed strain, via a %block Geometry.Constraints written to `config_extra.fdf`: 'cell-fixed' fixes all 6 Voigt stress components (cell stays exactly at the imposed strain; atomic positions still relax, via whatever MD.TypeOfRun/MD.Steps is already in `--calc`). 'stress-constrained' fixes only the imposed strain direction's own component(s) (plus anything touching a vacuum axis); every other periodic direction relaxes freely to zero stress. Either way, MD.VariableCell must be enabled in your own `--calc` for Geometry.Constraints to have any effect – checked (not forced) in [4] of the report.required; choices: cell-fixed, stress-constrained

`-p, --pseudo-dir PSEUDO_DIR`
: Pseudopotentials source (optional): a bundled bank (dojo, virtual_vault) or a folder path. If given, the required pseudopotential for every species in the structure is copied into every generated folder, so it's immediately ready for SIESTA.

`--stdir STDIR`
: Direction of strain: x, y, z for uniaxial; xy, xz, yz, etc. for biaxial. required

`--stmin STMIN`
: Minimum strain percentage (default: 0). Can be negative for compression. type: float

`--stmax STMAX`
: Maximum strain percentage (default: 25). type: float

`--step STEP`
: Strain step percentage (default: 1). type: float

`-o, --output-dir OUTPUT_DIR`
: Top-level directory (default: strain_runs). The run folders go into its own '<output-dir>/<direction>/strain_<direction>_<val>/' subfolder – so 2 different `--stdir` runs into the same `--output-dir` never mix their folders, and `stb-strainAnalysis` can point `--dir` at one direction's subfolder alone, or at <output-dir> itself with `--compare` to analyze every direction found under it.

`--vacuum-gap VACUUM_GAP`
: Vacuum gap threshold in Ang used to detect which lattice axes are periodic vs. vacuum-padded (default: 10.0), same convention as `stb-kgrid/stb-kpath`. Straining a vacuum-padded axis is physically meaningless and is refused. type: float

`--symprec SYMPREC`
: Symmetry-detection tolerance (default: 0.01, pymatgen's own default), used only for the informational note about symmetry-equivalent directions (uniaxial only) – never blocks the run. Loosen further (e.g. 0.02-0.05) for a structure relaxed with a looser force tolerance – a tighter value than the relaxation's own residual numerical noise can misdetect the true space group. type: float

`--angle-tolerance ANGLE_TOLERANCE`
: Symmetry angle tolerance in degrees (default: 5.0, pymatgen's own default), same use as `--symprec`.type: float

`--save-report`
: Also persist the report to <output-dir>/strain_stage1.txt. Off by default. switch, off by default

Like every STB command it also accepts `-h`/`--help`, `-v`/`--version` and `--no-intro` (Section 3.3).

From the built-in help.

```
Example usage:
  stb-strain -s structure.fdf -c calc.fdf -p dojo --relax-mode cell-fixed --stdir x --stmin -2 --stmax 2 --step 1
  stb-strain -s structure.fdf -c calc.fdf --relax-mode stress-constrained --stdir xy --stmax 5 --save-report
```

11.1.3.2 `stb-strainAnalysis`

Mechanical Properties from Stress-Strain Data (Stage 2 of `stb-strain`).

Synopsis.

```
usage: stb-strainAnalysis [-h] -f FILE [--dir DIR] [--dimensionality {auto,3d,2d,1d}]
                          [--vacuum-gap VACUUM_GAP] [--thickness THICKNESS]
                          [--cross-section CROSS_SECTION] [--yield] [-o OUTPUT_DIR]
                          [--save-report] [--save-gnuplot] [--view] [-v] [--no-intro]
```

Options

`-f, --file FILE`

Siesta output file (e.g., `calc.out`). required

`--dir DIR`

Directory to scan for 'strain_*' run folders (default: strain_runs). Accepts either `stb-strain`'s own output layout – point this at a single direction's own subfolder (e.g. strain_runs/x) for that direction alone, or at the top-level output directory itself to automatically compare every direction found under it – or a flat directory of 'strain_*' folders.

`--dimensionality {auto,3d,2d,1d}`

Physical dimensionality, controlling report units (GPa/N/m/nN) and how the cross-section for 1D Force mode is computed. 'auto' (default) tries to read a real, parseable .fdf structure file from the first strain folder found and classify it via vacuum-axis detection (same convention as `stb-strain/stb-mlelastic`); if none is found (the common, expected case for hand-built/legacy folders holding only a bare SIESTA .out), falls back to 3d.
choices: auto, 3d, 2d, 1d

`--vacuum-gap VACUUM_GAP`

Vacuum-gap threshold in Ang, used only when `--dimensionality` auto finds a real .fdf structure file to classify (default: 10.0, matching `stb-strain/stb-mlelastic`). type: float

`--thickness THICKNESS`

Vacuum height (Z) for 2D conversion (Angstrom). default: 20.0; type: float

`--cross-section CROSS_SECTION`

Physical cross-section area (Ang^2) of the wire/tube wall, only used when the run is 1D (`--dimensionality` 1d, or auto resolving to 1d), to additionally report a conventional Stress (GPa) – there is no way to auto-derive this (unlike the 2D vacuum height), since a wire/tube wall thickness is a physical convention, not a cell property.
type: float

`--yield`

Also report the 0.2% offset Yield – a macroscopic-plasticity concept borrowed from metallurgy; off by default since a defect-free periodic crystal under affine DFT strain has no dislocation-nucleation mechanism to actually yield through. switch, off by default

`-o, --output-dir OUTPUT_DIR`

Directory to write <direction>_curve.dat/.gplot (single direction) or comparison_curve.dat/.gplot (multiple directions) into with `--save-gnuplot`, and `stb_strainAnalysis_report.txt` into with `--save-report` (default: current directory). Created if it doesn't exist.

`--save-report`

Also persist the report to `stb_strainAnalysis_report.txt`. Off by default. switch, off by default

`--save-gnuplot`

Also write the curve data + a companion .gplot script (single direction: <direction>_curve.dat/.gplot; multiple directions: comparison_curve.dat/.gplot). Off by default. switch, off by default

`--view`
: Show an interactive matplotlib preview (the stress/force-strain curve, fit line, and peak) before finishing. Off by default. switch, off by default

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Example usage:
  stb-strainAnalysis --file calc.out
  stb-strainAnalysis --file calc.out --dir strain_runs --save-report --save-gnuplot --view
  stb-strainAnalysis --file calc.out --dimensionality 1d --cross-section 45.2
```

11.2 Elastic Constants

At a glance

Menu code: 5.2 (a multi-stage workflow)

Code	Stage	Command
5.2.1	Stage 1 - Prep	`stb-elasticInputs`
5.2.2	Stage 2 - Analysis	`stb-elasticAnalysis`

Guided example: `examples/5.2-elastic/`

Summary: Generate deformed structures, then compute the stiffness matrix and elastic moduli.

11.2.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/5.2-elastic/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

This workflow has 2 stages: **Stage 1** (`stb-elasticInputs`, code `5.2.1`) applies canonical Voigt strains to a relaxed structure and writes one ready-to-run, single-point SIESTA folder per strain value; **Stage 2** (`stb-elasticAnalysis`, code `5.2.2`) reads the finished SIESTA runs back and fits the full symmetric stiffness matrix `C_ij`, then checks whether the material is even mechanically stable. Together they answer a question `5.1-strain` deliberately does **not**: not "how stiff is this one direction", but **the complete elastic response of the material** — every normal and shear stiffness constant, at once, exploiting the crystal's own point-group symmetry to keep the DFT cost down.

Both stages live in this one folder and this one tutorial — not one folder per tool — for the same reason `5.1-strain/` does: Stage 1's output only exists to feed Stage 2, and Stage 2 only exists to interpret Stage 1's output.

11.2.1.1 Theory

1.1 From a single modulus to the stiffness tensor `C_ij`. `5.1-strain` fits one number per direction: an "initial slope", `stress = modulus × strain`, in the spirit of Young's modulus. The real, rigorous object behind that number is the full **stiffness tensor**, and Hooke's law in its general form:

```
sigma = C . epsilon
```

`sigma` (stress) and `epsilon` (strain) are symmetric 3×3 tensors; `C` is a 4th-rank tensor with, in general, 21 independent components. Everything in this workflow (and the rest of the suite) uses **Voigt notation** to collapse this into linear algebra: the 6 independent stress/strain components indexed `1..6 = XX/YY/ZZ/YZ/XZ/XY`, so `C` becomes an ordinary symmetric 6×6 matrix (3×3 for a 2D material's in-plane response — `zz`/`yz`/`xz` are physically undefined when that axis is vacuum-padded) and `sigma = C . epsilon` becomes an honest matrix-vector product. This is the exact same Voigt convention `5.1-strain` already introduced for its own `%block Geometry.Constraints`.

Point-group symmetry is what makes this tractable: for a *general* triclinic crystal you'd need all 21 constants, each from an independent DFT calculation. For anything with real symmetry — cubic, hexagonal, tetragonal, ... — most of those 21 numbers are either zero or exact multiples of each other, and the whole point of `--symmetry-method` (section 1.3) is exploiting that instead of brute-forcing every component.

Two independent physical routes to the same tensor. `stb-elasticInputs`/`stb-elasticAnalysis` support 2 structurally different ways to measure `C`, both selected via `--method`:

- **`stress`** (the default) — apply a small strain `epsilon`, read SIESTA's own stress tensor back, and fit `sigma = C . epsilon` — a **linear** fit. One strain series (several magnitudes of the *same* Voigt direction) gives a whole *column* of `C` at once, since `sigma = C . epsilon` for a single-component `epsilon` is just `C`'s corresponding column scaled by the strain.
- **`energy`** — apply the same kind of strain, read SIESTA's *total energy* back instead, and fit the **parabolic** form `E = E0 + (V0/2) * gammaˆT C gamma` (`gamma` = the Voigt engineering-strain vector, `V0` = the unstrained cell's volume/area/length). A **pure** single-component strain's energy curvature only ever determines that component's own *diagonal* constant (`C_mm`) — off-diagonal constants (`C_12`, ...) need a **combined** pattern instead, e.g. `'xx+yy'`, straining 2 Voigt components at once so their cross term `2*C_12*gamma_1*gamma_2` shows up in the curvature. `--method energy` on the Stage 1 side auto-selects exactly the pure+combined pattern set the detected point group actually needs — never a manual list.

These are **genuinely independent observables** — one is a code's stress tensor, the other its total energy — computed from the *same* DFT method, but with different numerical sensitivities (section 1.4). Comparing them is a real physical cross-check, not just re-running the same calculation twice; Stage 2's mandatory eggbox cross-check (section 3.3) does exactly this automatically, for free, using data a single `--method stress` run already collected.

1.3 Symmetry reduces how many directions you actually need — `basic` vs `full`. `--symmetry-method` (Stage 1 *and* Stage 2, and they must agree) controls **how** `--dirs all`'s 6 canonical directions get reduced to the minimal set actually run:

- **`basic`** — only pools 2 directions when a **single** point-group operation maps one exactly onto the other (e.g. a 4-fold rotation mapping `xx` onto `yy` in a cubic crystal). This is a *pairwise* test: it cannot see reductions that only follow from the *whole* tensor's symmetry at once. Hexagonal's `C11 = C22` is exactly such a case (section 4 shows this concretely: `basic` needs all 3 in-plane directions on a real hexagonal -type 2D material where `full` needs only 1).
- **`full`** — fits the *entire* stiffness matrix in one global least -squares pass, constrained a priori to the point group's own symmetry-allowed subspace (however many truly independent constants that point group has — reported directly, e.g. `"Point group -6m2 has 5 independent elastic constant(s)"`). This catches every crystal system's correct reduction uniformly (cubic, hexagonal, trigonal, ...), at a real cost: because every measured direction's data is folded jointly into one global fit, `C_12` (say) is no longer independently measurable from 2 different directions' own data — it's true by construction, not something 2 separate DFT calculations can be shown to *agree* on. That's

exactly why Stage 2's tensor-symmetry check (section 3.3) only exists for `basic` — see section 4 for the real trade-off in DFT cost vs. this lost cross-check.

`basic` is the CLI default (matching `stb-elasticInputs --help`); the interactive menu (`stb-suite` → `5.2.1`) also defaults there but offers `full` as option `2`.

Dimensionality changes the physical quantity, not just the unit. Same convention `5.1-strain` already uses, extended to a full tensor instead of one modulus, auto-detected by Stage 2 from `reference_structure.fdf`'s own vacuum padding (section 2.5):

- **3D** — `C_ij` in **GPa**, a genuine stress-strain stiffness.
- **2D** (vacuum along `z`) — `C_ij` in **N/m** (a force per unit length, not per area) — SIESTA's own volumetric stress (eV/Å^3) un-diluted by the cell's arbitrary vacuum height `Lz`, the same convention 2D-materials literature uses (e.g. graphene's ~340 N/m in-plane stiffness).
- **1D** (vacuum on 2 axes, periodic wire/tube along `z`) — `C_ij` in **nN**, a raw axial force, un-diluted by the wire's arbitrary vacuum cross-section, no assumed physical cross-section area.

`--method energy`'s `V0` in section 1.2's formula is this same dimensionality-aware quantity: cell volume (3D), in-plane area (2D), or axial length (1D) — read from `reference_structure.fdf`, never guessed.

The eggbox effect, and why Stage 2 checks for it automatically. SIESTA evaluates everything on a finite real-space grid (`Mesh.CutOff`). As atoms/cells move continuously across that fixed grid, quantities computed from *derivatives* of the energy — forces, and especially the **stress** tensor — pick up small, spurious, non-physical oscillations tied to the grid spacing itself: the "eggbox effect". Total **energy** is comparatively smooth (it's the thing being differentiated, not the derivative), so it's structurally less susceptible. This is precisely why section 1.2's 2 independent methods (stress vs. energy) make a *physically meaningful* cross -check, not a redundant one: in the exact continuum limit `sigma(epsilon) = dE/d(epsilon) / V0`, so for a perfectly converged calculation both methods must agree exactly — any real disagreement is a genuine hint of eggbox (or other numerical) contamination, concentrated on the stress side. Stage 2's `[3] NUMERICAL QUALITY DIAGNOSTICS` (section 3.3) runs exactly this check, automatically, whenever `--method stress` is used — reusing the *same* `calc.out` files, no new DFT calculations.

11.2.1.2 2. Stage 1: generating strained structures (`stb-elasticInputs`, code `5.2.1`)

What it does. `stb-elasticInputs` takes a relaxed structure and a reference `calc.fdf`, applies a canonical Voigt strain (or, for `--method energy`, a pure or combined pattern) to the lattice vectors for a range of strain values, and writes one ready-to-run folder per value: the deformed structure, a copy of `calc.fdf` (with the single-point override added, section 2.2), a new `config_extra.fdf`, and any linked pseudopotentials. It also writes ONE `reference_structure.fdf` — the *undeformed* structure — at the top of `--output-dir` (section 2.4). It does **not** run SIESTA itself — you run each folder yourself, then hand the results to Stage 2 (section 3).

Unlike `stb-strain`, there is **no `--relax-mode`**: every generated folder is unconditionally forced to a pure single-point SCF (section 2.2) — the imposed strain must stay exactly at the sampled geometry for the stress/energy fit to be physically meaningful, so there's no "which convention do you want the cell to relax under" choice to make in the first place.

The single-point-SCF precedence proof, live. `--calc` (`calc.fdf` in this folder) is the SAME file used for `structure.fdf`'s own real relaxation — DZP basis, a 12×12×1 Monkhorst-Pack grid (only 1 k-point along the vacuum axis), 320 Ry mesh cutoff, PBE-GGA, DFT-D3 dispersion, and a REAL, still-active relaxation block: `MD.TypeOfRun CG`, `MD.VariableCell true`, `MD.Steps 200`. Unlike `stb-strain` (which only

checks and warns about a bad relaxation block, section 2.2 of `5.1-strain/README.md`), `stb-elasticInputs` never trusts `--calc` to already be single-point — it **forces** one, unconditionally, regardless of what `--calc` contains:

```
$ stb-elasticInputs -s structure.fdf -c calc.fdf -p virtual_vault --dirs all --symmetry-method full
--steps 5 --no-intro
[4] SINGLE-POINT SCF ENFORCEMENT
------------------------------------------------------------
Calc template (current state, before forcing):
  MD.TypeOfRun=CG  Steps: MD.Steps=200  MD.VariableCell=true
Every generated folder is forced to a pure single-point SCF -- NO ionic relaxation, NO cell
relaxation -- regardless of the state above, since the imposed strain must stay exactly at the
sampled geometry for the stress/energy fit to be meaningful. Forced via '%include config_extra.fdf'
PREPENDED at the very top of every generated calc.fdf (before your own directives, including the
structure %include) -- SIESTA's fdf reader is first-occurrence-wins for duplicate labels, so this
ordering guarantees the forced values win even if your own template already sets any of them (the
common case for a real relaxation calc.fdf).

config_extra.fdf (written into every generated folder):
  # Auto-generated by stb-elasticInputs -- forces a pure single-point SCF (no ionic or
  # cell relaxation) at the imposed-strain geometry, regardless of --calc's own settings.
  MD.TypeOfRun       CG
  MD.Steps           0
  MD.NumCGsteps      0
  MD.VariableCell    false
```

The mechanism (`prepend_include`, not `stb-strain`'s own `insert_include_after_structure`) matters here: `config_extra.fdf` is `%include`-d at the **very top** of the generated `calc.fdf`, before your own directives (including the structure `%include`). SIESTA's fdf reader is **first-occurrence-wins** for duplicate labels — so the forced values win even though `calc.fdf`'s own `MD.Steps 200`/`MD.VariableCell true` are still sitting right there, further down, completely untouched:

```
First line of the generated calc.fdf:
%include config_extra.fdf

...calc.fdf's own MD block, still there, further down, silently overridden:
MD.TypeOfRun            CG
MD.VariableCell         true
MD.MaxForceTol          0.01 eV/Ang
MD.Steps                200
```

If the override were instead inserted *after* the structure `%include` (the convention `stb-strain` itself uses for its own `%block Geometry.Constraints` — safe there only because that block name never pre-exists in a real `calc.fdf`), it would be silently ignored whenever `--calc` already sets any of `MD.TypeOfRun`/`MD.Steps`/`MD.VariableCell` — the common case for a real relaxation template, exactly like this one.

2.3 `--method stress` vs. `energy` at generation time.

```
$ stb-elasticInputs -s structure.fdf -c calc.fdf -p virtual_vault --method energy --steps 5
--no-intro
[3] DEFORMATION DIRECTIONS / SYMMETRY REDUCTION
------------------------------------------------------------
[INFO] Point group -6m2 has 5 independent elastic constant(s); 2 strain pattern(s) needed to
determine the ones reachable without leaving the periodic plane/axis (of 6 physically valid
candidate(s), 21 in total).
```

```
  DEFORMATION DIRECTIONS (symmetry-method energy)
------------------------------------------------------------
  Detected symmetry : point group -6m2 -- 12 operation(s) ...
------------------------------------------------------------
  Direction   Status        Reconstructed from
  ----------------------------------------------------------
  xx          RUN           --
  yy          SUPPRESSED    symmetry-allowed fit
  xy          RUN           --
  xx+yy       SUPPRESSED    symmetry-allowed fit
  xx+xy       SUPPRESSED    symmetry-allowed fit
  yy+xy       SUPPRESSED    symmetry-allowed fit
  ----------------------------------------------------------
  2 of 6 run -- 4 suppressed by symmetry (exactly reconstructible by stb-elasticAnalysis)

Modes: ['xx', 'xy']
```

`--method energy` always auto-selects its own pure+combined pattern set from the detected point group — there's no `--dirs`/`--symmetry-method` choice to make for it (`--dirs` stays at its `all` default; `--symmetry-method` is read by `--method stress` only). Note that for *this specific* point group, the 2 patterns needed both happen to be pure (`xx`, `xy`) — no `'xx+yy'`-style combined pattern was actually necessary here; the auto-selection is a rank -selection over every physically valid candidate (`21` combined+pure patterns total for this material, only `2` needed), not a fixed rule.

2.4 `reference_structure.fdf` — what it is, and why Stage 2 needs it. Every `stb-elasticInputs` run writes exactly ONE `reference_structure.fdf` at the top of `--output-dir` (`elastic_runs/` by default) — a plain copy of the **undeformed** input structure, never one from inside a `strain_*/` folder. Stage 2 reads it back for 4 things:

1. **Dimensionality auto-detection** (section 1.4) — from its vacuum padding, same `--vacuum-gap` convention as Stage 1's own `[2]` section.
2. **Symmetry/point-group detection** — for `--symmetry-method basic/full`'s own pooling/reconstruction (section 1.3).
3. **`--method energy`'s reference volume/area/length** (`V0` in section 1.2's formula) — needed to convert a fitted energy curvature into an actual stiffness constant.
4. **The strain each folder actually carries** — the folder name is rounded to two decimals (`strain_xx_0.33` is really 0.3333 % on a 7-point grid of ± 1 %), so Stage 2 measures the strain from the folder's own lattice vectors against this reference cell, $\varepsilon = (F + F^{\mathsf{T}})/2 - I$ with $F = \text{cell} \cdot \text{reference}^{-1}$, and uses it whenever it agrees with the name within that rounding (otherwise the name is kept). Reading the rounded name instead biased every slope by $\sim$0.1 % (silicon C_{11} 149.93 instead of 150.03 GPa, Section 13.4.2.5).

It must genuinely be the **undeformed** structure: a single strained direction genuinely *lowers* the crystal's symmetry (e.g. this example's `-6m2` structure, strained along `xx` alone, is no longer `-6m2` — it's exactly tetragonal-like in-plane) — reading symmetry from inside a strained folder would silently misdetect it, the same reasoning `5.1-strain` gives for never straining atomic positions directly (only fractional coordinates, which scale for free with the deformed cell).

Output layout, and running it both ways. `--output-dir <dir>/<direction>/strain_<direction>_<pct>/` — same nested convention `stb-strain` itself uses, with `reference_structure.fdf` at the top level (section 2.4) so `cd <output-dir> && stb-elasticAnalysis` (or, from the parent directory, `stb-elasticAnalysis --dir <output-dir>`) finds everything with no extra flags:

```
# Direct CLI
stb-elasticInputs -s structure.fdf -c calc.fdf -p virtual_vault \
    --dirs all --symmetry-method full --steps 5 --no-intro

# Interactive
stb-suite   # then type 5.2.1, or navigate Workflow -> Elastic Constants -> Stage 1
```

Both call the exact same underlying tool and produce byte-identical output — `example_5.2.sh`'s own Case 6 proves this directly, driving both paths with the same inputs and `diff -rq`-ing the result.

11.2.1.3 3. Stage 2: analyzing the results (`stb-elasticAnalysis`, code `5.2.2`)

Report structure.

```
[0] RUN METADATA                      -- run dir, method, dimensionality, tolerances
[1] <N>D STIFFNESS MATRIX             -- the fitted C_ij (or C33 alone for 1D)
[1b] <N>D COMPLIANCE MATRIX           -- S = C^-1 (eps = S . sigma), when invertible
[2] (MAIN|RELEVANT) CONSTANTS         -- the labeled diagonal/off-diagonal values
[2b] COUPLING CONSTANTS               -- normal-shear cross terms, 3D only, shown
                                          only when numerically significant
[3] NUMERICAL QUALITY DIAGNOSTICS     -- eggbox cross-check, fit quality, tensor
                                          symmetry (--method stress only; see 2.4)
[4] STABILITY AND PROPERTIES          -- Young's/bulk modulus, Poisson ratio,
                                          anisotropy index, Born stability VERDICT
[5] SUMMARY & FILES                   -- one glanceable results table + file paths
```

`--save-report` persists the exact same report to `elastic_stage2.txt` (matching `elastic_inputs.py`'s own `elastic_stage1.txt` naming logic).

3.2 `-d/--dir` and dimensionality — both auto-detected, no `cd` required. `-d/--dir` (default `elastic_runs`, matching Stage 1's own `--output-dir` default) accepts either the nested `<direction>/strain_.../` layout Stage 1 writes, or a flat directory of `strain_*/` folders — and, crucially, can be pointed at from the **parent** directory, no `cd` needed:

```
$ stb-elasticAnalysis --dir elastic_runs --no-intro
[INFO] Detected dimensionality: 2D (e.g., a slab or surface)
[INFO] Scanning 'elastic_runs' for 'strain_*' folders...
```

`--dimensionality auto` (the default) reads `reference_structure.fdf` (section 2.4) the exact same way Stage 1's own `[2]` section does; pass `--dimensionality 3d/2d/1d` to override and skip detection entirely.

3.3 `[3] NUMERICAL QUALITY DIAGNOSTICS` — the eggbox cross-check, and 2 more. Whenever `--method stress` is used, 3 checks run automatically — zero new DFT calculations, all reusing data a single run already collected:

- **Eggbox cross-check** (the theory from section 1.5) — every diagonal constant actually measured gets refit a *second*, independent way from the *same* `calc.out` files: the usual stress-linear fit, and a parabolic fit of the total energy already sitting in the same files. `--eggbox-tolerance` (default 5.0%) sets how much disagreement is flagged.
- **Per-direction fit quality** — R^2 and residual stress at zero strain, from the *same* stress-strain fit each direction's column already needed. `--fit-quality-tolerance` (default R^2=`0.995`) flags a noisy or non-linear (too-large-`--max`) series, or a non-negligible residual stress hinting the reference structure isn't fully relaxed.

- **Tensor symmetry (`C` vs. `C^T`)** — `--symmetry-method basic` **only** (section 1.3 explains why): 2 independently-run directions can genuinely disagree on the value they each imply for the same off-diagonal constant (`C_12` from `xx`'s own column vs. from `yy`'s own column) — a real, useful internal-consistency check `full`/`energy` cannot offer, since both force `C_12 = C_21` by construction and would report an uninformative exact 0 every time.

A real, clean run (this folder's own synthetic data, Case 4 of `example_5.2.sh`, `--symmetry-method basic`) with everything passing:

```
[3] NUMERICAL QUALITY DIAGNOSTICS
------------------------------------------------------------
Tensor symmetry (C vs C^T): max |C_ij - C_ji| = 0.000 N/m (i=1, j=1)  [OK]
Direction fit quality (stress vs. strain):
  xx: R^2 = 1.000000 | residual stress =    0.00 N/m  [OK]
  xy: R^2 = 1.000000 | residual stress =    0.00 N/m  [OK]
  yy: R^2 = 1.000000 | residual stress =    0.00 N/m  [OK]
Eggbox cross-check (stress vs. energy, same data):
  xx: stress =   200.00 N/m | energy =   200.00 N/m | diff =  0.00%  [OK]
  yy: stress =   200.00 N/m | energy =   200.00 N/m | diff =  0.00%  [OK]
  xy: stress =    80.00 N/m | energy =    80.00 N/m | diff =  0.00%  [OK]
[INFO] All checked direction(s) agree within tolerance -- no indication of eggbox contamination in
the stress data.
```

The exact same data analyzed with `--symmetry-method full` instead omits the `Tensor symmetry` line entirely — not a `0.000` value, no line at all — the concrete proof this workflow's own `example_5.2.sh` (Case 5) runs live.

A real bug, found and fixed on this exact material's data during development: `eggbox_cross_check`'s energy-side conversion used to also multiply by the *stress*-side geometric dilution factor (the cell height `Lz` for 2D, a cross-section for 1D) — a factor the energy side should never need (total energy isn't diluted by vacuum the way SIESTA's raw volumetric stress is), silently inflating the reported "energy" value by an extra factor of `Lz` (a real ~20× error was observed live on this material, a spurious `~1900%` false "possible eggbox contamination" warning where the two methods actually agreed to ~4%, section 4). Fixed in `elastic_analysis.py` — 3D was never affected (its own conversion constant has no such geometric factor to begin with).

Plotting: gnuplot and matplotlib, both opt-in. Same convention as `5.1-strain`'s own Stage 2: `--save-gnuplot` writes `<mode>_fit.dat`/`.gplot` per direction/pattern plus a combined overview into `--plot-dir` (default `elastic_plots/`, render with `cd elastic_plots && gnuplot <script>.gplot`); `--view` shows the same fits as an interactive matplotlib plot instead, points-and-line, no file written. Both off by default — the `[5]` section reminds you if you skipped them.

Running it both ways.

```
# Direct CLI
stb-elasticAnalysis --dir elastic_runs --symmetry-method full --no-intro

# Interactive
stb-suite   # then type 5.2.2
```

11.2.1.4 Worked example: a real 2D material, end-to-end

Everything above was verified live against a real, fully-converged SIESTA dataset — the SAME material `structure.fdf`/`calc.fdf` in this folder are taken from (a nice continuity with `5.1-strain/`'s own worked

example, section 4 there — also a real 2D SiC monolayer): 1 Si + 1 C atom, point group `-6m2`, run all 3 ways, `+/-2.0%, 5 steps` each.

Run	Real DFT calcs	Directions run	C11	C22	C12	C66	Eggbox check	Tensor-symmetry check
`--method stress --symmetry-method full`	**5**	`xx` only	200.35	200.34	37.18	81.58 N/m	`xx`: 4.10% [OK]	not run (exact by construction)
`--method stress --symmetry-method basic`	**15**	`xx`, `yy`, `xy`	200.35	200.43	37.04	81.51 N/m	`xx`/`yy`/`xy`: 4.10/4.05/0.06% [OK]	max|C_ij-C_ji|=0.280 N/m (i=1,j=2) [OK]
`--method energy`	**10**	`xx`, `xy` patterns	192.13	192.12	29.01	81.56 N/m	n/a (energy method never runs it)	n/a

Reading this the way sections 1 and 3 set up:

- **Both `stress` runs are internally consistent**: `C66 = (C11-C12)/2` closes exactly in both (`(200.35-37.18)/2 = 81.585` vs. the reported `81.58`; `(200.35-37.04)/2 = 81.655` vs. `81.51` — small residual fit-noise, still `R^2 > 0.998` in both directions per `[3]`) — a genuine physical-consistency check the tool never enforces, just reports.
- **`full` really is 3× cheaper here** (section 1.3, section 2.3): 5 real SIESTA calculations vs. 15 for the exact same physical answer (`C11` agrees to 4 significant figures between the two) — the real-cost version of `example_5.2.sh`'s own Case 2.
- **The ~4% gap between the `stress` runs and the `energy` run** traces to a real, correctly-flagged residual, not noise or a bug: every run reports `[WARNING] Pattern 'xx': linear energy term (3.09e-01 eV) is not negligible next to the quadratic term (5.00e+01 eV)` — the reference relaxation converged forces well (~2×10^{-4} eV/Å, well under its own `MD.MaxForceTol 0.005`), but still carries a small residual cell stress that shows up proportionally larger at this workflow's small (±2%) strain range. A useful reminder that "converged forces" and "zero residual stress at this specific strain range" are related but distinct claims.
- **The eggbox cross-check (`basic`/`full` rows) independently confirms** the same ~4% gap *within* the stress-method data alone (`xx`'s own stress-vs-energy diff is 4.10% in both stress runs) — exactly the physical observable section 1.5 says it should catch, and exactly why it's mandatory rather than opt-in.
- **`basic`'s own tensor-symmetry check passes cleanly** (`0.280 N/m` out of `~37 N/m`, well inside noise) — an actual, independently-computed agreement between `C12` read off 2 different directions' own data, not assumed by construction the way `full`'s own `C12` is.

When is each method actually the better choice?

- **`stress + full`** — the cheapest default whenever you trust the detected point group (common for well-characterized materials): fewest DFT calculations for the complete tensor.
- **`stress + basic`** — costs more DFT time (here, 3×) but buys a real, independent internal-consistency check (the tensor-symmetry line) that `full` cannot offer by construction. Worth it for an unfamiliar or low-symmetry system, or whenever you don't fully trust the auto-detected point group and want the data itself to prove the assumed symmetry holds.
- **`--method energy`** — a genuinely independent physical observable (energy curvature, not stress), most valuable as a fully separate cross-check against an entire `stress` result. The mandatory eggbox check (section 3.3) already gets you most of this value *for free* from a single `stress` run's own data — reach for a full separate `--method energy` run when you want the strongest possible independent confirmation (a completely uncorrelated dataset, not just a second fit of the same

`calc.out` files), or when eggbox flags a real disagreement you want to pin down.

11.2.1.5 Known, deliberate limitations

- **No `--relax-mode`** (section 2.1) — every generated folder is always a pure single-point SCF; there is no clamped-cell-vs-relaxed-transverse-stress choice the way `stb-strain` offers, since the imposed strain must stay exactly at the sampled geometry for either fitting method to be valid.
- **`--method energy`'s DFT cost scales with symmetry, not always favorably** — a genuinely low-symmetry (triclinic/monoclinic) structure can need many more combined patterns than either `stress` variant needs canonical directions.
- **`[3]` quality checks are `--method stress`-only** — `--method energy` never runs the eggbox/fit-quality/tensor-symmetry section at all (it has no analogous "2 independent fits of the same data" structure to check).
- **Auto-dimensionality/symmetry both need a readable `reference_structure.fdf`** (section 2.4) — without it, dimensionality falls back to 3D with a `[WARNING]`, and `--symmetry-method full`/`--method energy` hard-fail (they cannot function without symmetry detection); `--symmetry-method basic` degrades gracefully to independent per-direction fitting instead.

11.2.1.6 Step-by-step: running this workflow on your own structure

0. **Prerequisite**: an already-relaxed structure (`structure.fdf`, fractional coordinates) and the exact `calc.fdf` used for that relaxation — this workflow strains an existing equilibrium structure, it doesn't find one for you.
1. **Decide `--method`** (section 1.2): `stress` (default, cheaper, needs the mandatory `[3]` checks to trust) or `energy` (independent observable, own DFT cost). If `stress`, also **decide `--symmetry-method`** (section 1.3): `basic` (default, more DFT calls, gets you the tensor-symmetry cross-check) or `full` (fewer DFT calls, loses that check).
2. **Run Stage 1** (`stb-elasticInputs`, code `5.2.1`):
   - CLI: `stb-elasticInputs -s structure.fdf -c calc.fdf -p <bank> --dirs all --symmetry-method <basic|full> --max <pct> --steps <n>` (or `--method energy` instead of `--symmetry-method`).
   - Interactive: `stb-suite` → `5.2.1` — same questions instead of flags.
   - Either way, this writes `elastic_runs/<direction>/strain_<direction>_<pct>/` folders (section 2.5) plus one top-level `reference_structure.fdf` (section 2.4).
3. **Run SIESTA yourself in every generated folder.** Stage 1 never runs SIESTA — `cd` into each `strain_<direction>_<pct>/` and run it like any other SIESTA calculation (`siesta calc.fdf > calc.out`, or through your own batch/queue script). Repeat for every `<direction>/` subfolder.
4. **Run Stage 2** (`stb-elasticAnalysis`, code `5.2.2`) once every run finishes:
   - CLI: `stb-elasticAnalysis --dir elastic_runs --symmetry-method <basic|full>` (must match Stage 1's own choice; omit for `--method energy`, which always uses its own symmetry-allowed-subspace fit).
   - Interactive: `stb-suite` → `5.2.2`.
   - `--dir` and dimensionality are both auto-detected (sections 2.4/3.2) — nothing to set manually in the common case, and no `cd` required.
5. **Read `[3] NUMERICAL QUALITY DIAGNOSTICS`** (section 3.3) before trusting the numbers: any eggbox/fit-quality `[WARNING]` is a real, specific hint (contaminated stress data, too-large `--max`, an incompletely

relaxed reference) about exactly which direction to re-examine, not a generic "something's wrong somewhere".

6. **Read `[4] STABILITY AND PROPERTIES`** — the Born mechanical-stability criteria and the final `VERDICT: STABLE/UNSTABLE`; **`[5] SUMMARY & FILES`** for the same numbers in one glanceable table plus where everything ended up on disk.
7. **(Optional) persist or plot**: `--save-report` writes `elastic_stage2.txt`; `--save-gnuplot` writes `<mode>_fit.dat/.gplot` into `--plot-dir` (section 3.4); `--view` opens the same fits interactively in matplotlib instead.
8. **(Optional) go further**: `stb-mlelastic` (ML Simulations) previews the exact same physics with a MACE potential, no SIESTA/`strain_*` folders needed — a fast sanity check before committing to this real DFT workflow, or to decide between `basic`/`full`/`energy` cheaply first (section 9).

11.2.1.7 Files in this folder

File	What it is
`structure.fdf`	Real, relaxed 2D SiC monolayer, 2 atoms/cell, point group `-6m2`
`calc.fdf`	The real relaxation `calc.fdf` for `structure.fdf` (DZP/12x12x1/PBE-D3)
`example_5.2.sh`	This walkthrough's runnable script (both stages, 6 cases)

11.2.1.8 Running the script

```
bash example_5.2.sh
```

11.2.1.9 What's next

- **`5.1-strain`** (Workflow `5.1`) — the single-direction modulus this workflow's full tensor is deliberately not needed for when you only care about one direction's stiffness/strength/toughness, not the complete `C_ij`.
- **`stb-mlelastic`** (ML Simulations, code `7.4`) — a fast MACE-based preview of the exact same stiffness-tensor physics, no SIESTA/`strain_*` folders needed; a good sanity check (or a way to cheaply try `basic` vs. `full` vs. `energy` before committing DFT time) before the real workflow here.
- **`stb-eosInputs/stb-eosAnalysis`** (Workflow `5.15`) — bulk modulus from the curvature of the energy-volume curve instead, an independent cross-check on a 3D structure's own `B` (section 3, `[4]`) against this workflow's stress-strain-derived value.
- **`5.3-cohesive`** (`stb-cohesive/stb-cohesiveAnalysis`, Workflow `5.3`) — a different mechanical-stability question (cohesive energy per atom) about the same kind of structure.

11.2.2 Implementation details

Stress–strain and energy–strain routes

In Voigt notation ($1{=}xx$, $2{=}yy$, $3{=}zz$, $4{=}yz$, $5{=}xz$, $6{=}xy$; shear strains as engineering strains $\gamma_{ij} = 2\varepsilon_{ij}$) Hooke's law reads $\sigma_i = \sum_j C_{ij}\varepsilon_j$ [64]. A strain series along a single component j gives a whole column, $C_{ij} = \partial\sigma_i/\partial\varepsilon_j$, from the six stress components of every run. The energy route

uses

$$E(\boldsymbol{\varepsilon}) = E_0 + \frac{V_0}{2}\sum_{ij} C_{ij}\varepsilon_i\varepsilon_j + O(\varepsilon^3),$$

whose curvature along a single component gives only the diagonal C_{jj}; off-diagonal constants need combined patterns (e.g. `xx+yy`), which `-method energy` generates.

Stage 1 (`stb-elasticInputs`). For each direction the strains $\pm$`-max` per cent are sampled with `-steps` points per side (defaults 2 % and 4), using the deformation $\mathbf{F} = \mathbf{I} + \boldsymbol{\varepsilon}$ with the symmetric strain tensor of that Voigt mode (shear modes set $\varepsilon_{ij} = \varepsilon_{ji} = \delta$). `-dirs` accepts the six modes plus `bi` (equi-biaxial) and `hydro`; `all` (default) keeps one representative per group of symmetry-equivalent directions – `basic` reduction maps directions pairwise by point-group operations, `full` picks the minimal set that determines the point group's independent constants (needed, e.g., for $C_{11} = C_{22}$ of hexagonal lattices). Directions along vacuum axes are excluded. Every folder is a single-point SCF: `config_extra.fdf` forces `MD.TypeOfRun`, `MD.Steps 0` and `MD.VariableCell false` and is prepended, so a relaxation template cannot move the strained cell. `reference_structure.fdf` (the undeformed cell) is written at the top of `-o` (default `elastic_runs`).

Stage 2 (`stb-elasticAnalysis`).

1. Stresses are read from every `calc.out`, converted (3D: GPa; 2D: N/m via the cell height; 1D: only the axial constant) and fitted column by column; with symmetry reduction the missing columns are reconstructed by rotating the stress response of an equivalent direction (`basic`) or the whole matrix is fitted in the symmetry-allowed subspace by least squares (`full`). `-symmetry-method` must match Stage 1.
2. **Quality diagnostics**: every direction's linear fit must reach $R^2 \geq$ `-fit-quality-tolerance` (0.995) without a large stress at zero strain; the *eggbox cross-check* recomputes each diagonal constant from the curvature of the total energies of the same runs and flags differences above `-eggbox-tolerance` (5 %) – stresses suffer more from SIESTA's grid noise than energies; the symmetrized tensor's deviation from symmetry (C_{ij} vs C_{ji}) is reported.
3. **Properties and stability** (3D): Voigt and Reuss bounds and the Hill averages [65–67]

$$B_V = \tfrac{1}{9}\left[(C_{11}{+}C_{22}{+}C_{33}) + 2(C_{12}{+}C_{13}{+}C_{23})\right],$$
$$G_V = \tfrac{1}{15}\left[(C_{11}{+}C_{22}{+}C_{33}) - (C_{12}{+}C_{13}{+}C_{23}) + 3(C_{44}{+}C_{55}{+}C_{66})\right],$$

   B_R, G_R from the compliance $S = C^{-1}$, $B_H = (B_V + B_R)/2$, $G_H = (G_V + G_R)/2$, $E = 9BG/(3B + G)$, $\nu = (3B - 2G)/[2(3B + G)]$, and the universal anisotropy index $A^U = 5G_V/G_R + B_V/B_R - 6$ [68]. Stability requires a positive definite C (all eigenvalues positive), the general form of the Born conditions [69, 70]. 2D: layer moduli $E_x = (C_{11}C_{22} - C_{12}^2)/C_{22}$, $E_y = (C_{11}C_{22} - C_{12}^2)/C_{11}$, Poisson ratios $\nu_{yx} = C_{12}/C_{22}$, $\nu_{xy} = C_{12}/C_{11}$, $G = C_{66}$, stable if $C_{11} > 0$, $C_{66} > 0$, $C_{11}C_{22} > C_{12}^2$. 1D: the axial stiffness $C_{33} > 0$.

`-method energy` replaces step 1 by quadratic fits of $E(\delta)$ for every pattern (a non-negligible linear term warns that the reference is not relaxed) and solves for the symmetry-allowed constants; it needs `-reference-structure`. Plots of every series and fit go to `-plot-dir` (gnuplot) or a window (`-view`).

Good practice.. Start from a cell relaxed with the same settings (residual stress biases every column), use strains of 0.5–2 %, and converge `Mesh.Cutoff` first: the eggbox check is there because grid noise is the most common source of wrong elastic constants in SIESTA.

Clamped-ion constants

Every folder is a single point, so the atoms follow the homogeneous strain without relaxing: the result is the *clamped-ion* tensor. In crystals where some atoms are not at centres of inversion (diamond and zinc-blende, wurtzite, most compounds with internal parameters) internal relaxation lowers some constants, often C_{44} markedly. For relaxed-ion normal-strain responses use the stress–strain workflow **5.1** in `cell-fixed` mode, which relaxes the atoms in each strained cell, and compare the initial slopes.

11.2.3 Command-line reference

11.2.3.1 `stb-elasticInputs`

Stage 1 of 2: applies one or more canonical Voigt strains to a SIESTA structure and writes one ready-to-run folder (structure + `calc.fdf` + `config_extra.fdf` + pseudopotentials) per scanned strain value, forcing a pure single-point SCF at each imposed strain. Doesn't run SIESTA – run each folder yourself, then use `stb-elasticAnalysis`.

Synopsis.

```
usage: stb-elasticInputs [-h] -s STRUCTURE -c CALC [-p PSEUDO_DIR]
                         [--method {stress,energy}] [--dirs DIRS [DIRS ...]]
                         [--symmetry-method {basic,full}] [--max MAX] [--steps STEPS]
                         [-o OUTPUT_DIR] [--vacuum-gap VACUUM_GAP] [--symprec SYMPREC]
                         [--angle-tolerance ANGLE_TOLERANCE] [--save-report] [-v]
                         [--no-intro]
```

Options

`-s, --structure STRUCTURE`

Input `structure.fdf` – lattice + atomic coordinates. Only this file's %block LatticeVectors is rewritten per strain step; everything else is preserved verbatim. Atomic coordinates MUST be fractional (checked in [1] of the report – Cartesian positions would not scale with the strained cell). required

`-c, --calc CALC`

`calc.fdf` template – SCF/basis/MD settings, normally %including the structure file above by name. Copied into every generated folder with one line added: '%include `config_extra.fdf`', prepended at the very top (see [4] of the report for why) – this forces every generated folder to a pure single-point SCF, regardless of what MD settings this template itself contains. required

`-p, --pseudo-dir PSEUDO_DIR`

Pseudopotentials source (optional): a bundled bank (dojo, virtual_vault) or a folder path. If given, the required pseudopotential for every species in the structure is copied into every generated folder, so it's immediately ready for SIESTA.

`--method {stress,energy}`

Physical quantity the generated strains are meant to feed (default: stress). 'stress': single-component canonical Voigt strains, for `stb-elasticAnalysis`'s stress-strain fit (`--method` stress there too). 'energy': a point-group-selected set of pure AND combined (two Voigt components at once, e.g. 'xx+yy') strain patterns for an energy-strain (parabolic) fit – combined patterns are physically required for off-diagonal constants there, since a pure single-component strain's energy curvature only ever determines its own diagonal constant. Always auto-selects (`--dirs` must be left at its 'all' default); needs more DFT calculations than `--method` stress

for the same crystal system, since each pattern only yields one number (a curvature) instead of a full stress column. choices: stress, energy

`--dirs DIRS [DIRS ...]`

List of directions to generate.
Options: [xx, yy, zz, xy, xz, yz, bi, hydro, all].
Example: `--dirs` xx yy xy
'all' auto-reduces to 1 representative direction per symmetry-equivalent group (e.g. just 2 of the 6 for a cubic material) – `stb-elasticAnalysis` reconstructs the rest by symmetry. Pass every direction explicitly (not 'all') to run all 6 anyway, e.g. to get `stb-elasticAnalysis's` diagnostic comparison across truly independent DFT calculations instead. default: all

`--symmetry-method {basic,full}`

How 'all' picks which directions to actually run (default: basic). 'basic': only reduces when a single point-group operation maps one canonical direction exactly onto another (e.g. cubic); misses reductions like hexagonal's C11=C22, which come from the full elastic tensor's symmetry, not a pairwise direction match. 'full': picks the minimal set of directions that fully determines the point group's independent elastic constants (greedy rank selection) – catches hexagonal/trigonal reductions too. Must match the `--symmetry-method` passed to `stb-elasticAnalysis` later. choices: basic, full

`--max MAX`

Max strain % (default: 2.0) type: float

`--steps STEPS`

Steps per direction (default: 4) type: int

`-o, --output-dir OUTPUT_DIR`

Top-level directory (default: elastic_runs). Run folders go into its own '<output-dir>/<direction>/strain_<direction>_<val>/' subfolder, same convention as `stb-strain's` own `--output-dir` – so `stb-elasticAnalysis` can just 'cd <output-dir>' and find everything (including `reference_structure.fdf`) with no extra flags.

`--vacuum-gap VACUUM_GAP`

Vacuum gap threshold in Ang used to detect which lattice axes are periodic vs. vacuum-padded (default: 10.0), same convention as `stb-kgrid/stb-strain`. Requested directions touching a vacuum-padded axis are skipped with a warning instead of generated. type: float

`--symprec SYMPREC`

Symmetry-detection tolerance (default: 0.01, pymatgen's own default), used for the symmetry-based direction reduction and the informational note about triclinic/monoclinic structures. Loosen further (e.g. 0.02-0.05) for a structure relaxed with a looser force tolerance – a tighter value than the relaxation's own residual numerical noise can misdetect the true space group. type: float

`--angle-tolerance ANGLE_TOLERANCE`

Symmetry angle tolerance in degrees (default: 5.0, pymatgen's own default), same use as `--symprec`.type: float

`--save-report`

Also persist the report to <output-dir>/elastic_stage1.txt. Off by default. switch, off by default

Like every STB command it also accepts `-h/-help`, `-v/-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Example usage:
  stb-elasticInputs -s structure.fdf -c calc.fdf -p dojo --dirs all
```

```
stb-elasticInputs -s structure.fdf -c calc.fdf --dirs xx yy xy --max 1.5 --steps 6 --save-report
```

11.2.3.2 stb-elasticAnalysis

Siesta Elastic Properties Analyzer (STB Suite)

Synopsis.

```
usage: stb-elasticAnalysis [-h] [-f FILE] [-d DIR] [--method {stress,energy}]
                           [--dimensionality {auto,3d,2d,1d}]
                           [--vacuum-gap VACUUM_GAP] [--symmetry-method {basic,full}]
                           [--reference-structure REFERENCE_STRUCTURE]
                           [--symprec SYMPREC] [--angle-tolerance ANGLE_TOLERANCE]
                           [--eggbox-tolerance EGGBOX_TOLERANCE]
                           [--fit-quality-tolerance FIT_QUALITY_TOLERANCE]
                           [--plot-dir PLOT_DIR] [--save-gnuplot] [--view]
                           [--save-report] [-v] [--no-intro]
```

Options

`-f, --file FILE`

Siesta log/output filename inside strain folders (default: `calc.out`)

`-d, --dir DIR`

Directory to scan for 'strain_*' run folders and the undeformed `reference_structure.fdf` (default: elastic_runs, matching `stb-elasticInputs`' own `--output-dir` default). Accepts either `stb-elasticInputs`' own nested output layout – point this at a single direction's own subfolder (e.g. elastic_runs/xx) for that direction alone, or at the top-level output directory itself to automatically use every direction found under it – or a flat directory of 'strain_*' folders.

`--method {stress,energy}`

Which physical quantity to fit (default: stress). 'stress': the existing stress-strain linear fit (sigma = C . epsilon), see `--symmetry-method`. 'energy': an energy-strain (parabolic) fit (E = E0 + (V0/2) gamma^T C gamma) over the pure+combined patterns written by `stb-elasticInputs --method energy` – a physically independent cross-check (total energy vs. the stress tensor), since a code's total energy and stress can have different sources of numerical error. Always uses the general symmetry-allowed-subspace fit (`--symmetry-method` doesn't apply); requires `--reference-structure`. choices: stress, energy

`--dimensionality {auto,3d,2d,1d}`

Physical dimensionality of the material, controlling report units and which constants are shown (default: auto). 'auto': detected from `--reference-structure` using the same vacuum-gap convention as `stb-elasticInputs/stb-kgrid/stb-strain` (see `--vacuum-gap`) – 0 vacuum-padded axes -> 3d (GPa), 1 -> 2d (N/m), 2 -> 1d (nN); falls back to 3d with a warning if `--reference-structure` isn't readable, or if the vacuum sits on an axis other than this workflow's assumed convention (in-plane xy for 2D, wire-along-c for 1D). '3d'/'2d'/'1d': force the mode manually and skip detection entirely. choices: auto, 3d, 2d, 1d

`--vacuum-gap VACUUM_GAP`

Vacuum gap threshold in Ang (default: 10.0) used only by `--dimensionality` auto, same convention as `stb-elasticInputs/stb-kgrid/stb-strain`. type: float

`--symmetry-method {basic,full}`

How to use detected symmetry (default: basic). 'basic': exact reconstruction/pooling only when a single point-group operation maps one canonical strain direction exactly onto another (misses reductions like hexagonal's C11=C22, which come from the FULL elastic tensor's invariance, not a pairwise direction match).

'full': fits the whole stiffness matrix at once, constrained a priori to the point group's symmetry-allowed subspace – catches every crystal system's correct independent-constant count uniformly (cubic, hexagonal, trigonal, ...), at the cost of being a global least-squares fit rather than an exact per-column one. Requires `--reference-structure` to be available. choices: basic, full

`--reference-structure REFERENCE_STRUCTURE`

UNDEFORMED structure file (default: `reference_structure.fdf` inside `--dir`, written automatically by `stb-elasticInputs` next to the strain_*/ folders), used only to detect symmetry for the diagnostic/pooling/reconstruction across equivalent directions. Must be the undeformed structure, not one from inside a strain folder – a single-axis strain genuinely lowers the symmetry (e.g. cubic strained along x alone is exactly tetragonal, not 'cubic with noise'), so reading symmetry from a strained structure would silently misdetect it. Never required – falls back to independent per-direction fitting if unavailable.

`--symprec SYMPREC`

Symmetry-detection tolerance (default: 0.01, pymatgen's own default). Loosen further (e.g. 0.02-0.05) for a structure relaxed with a looser force tolerance – a tighter value than the relaxation's own residual numerical noise can misdetect the true space group. type: float

`--angle-tolerance ANGLE_TOLERANCE`

Symmetry angle tolerance in degrees (default: 5.0, pymatgen's own default), same use as `--symprec`.type: float

`--eggbox-tolerance EGGBOX_TOLERANCE`

Relative difference (%, default: 5.0) above which the mandatory eggbox cross-check (`--method` stress only, see section [3] of the report) flags a direction. Every diagonal constant actually run is computed two independent ways from the SAME `calc.out` files – a stress-linear fit and an energy-parabolic fit – with no new DFT calculations; a disagreement beyond this tolerance is a hint of eggbox (or other numerical) contamination in the stress data. Needs `--reference-structure`; skipped with a warning (not an error) if it isn't available.type: float

`--fit-quality-tolerance FIT_QUALITY_TOLERANCE`

R^2 (default: 0.995) below which the mandatory per-direction stress-vs-strain fit-quality check (`--method` stress only, section [3] of the report) flags a direction – a clean linear elastic fit should sit well above this. Also flags a non-negligible residual stress at zero strain (same relative-contribution criterion as the energy method's linear-term check). No new DFT calculations: reuses the same stress-strain fit each direction's column already needed. type: float

`--plot-dir PLOT_DIR`

Directory (default: elastic_plots, created only if `--save-gnuplot` is given) to write the raw fitting data + gnuplot scripts into – one .dat/.gplot pair per direction (`--method` stress) or pattern (`--method` energy), plus a combined overview.

`--save-gnuplot`

Also write the raw fitting data + gnuplot scripts into `--plot-dir` (same data+.gplot convention as the rest of the suite, e.g. `stb-strainAnalysis`). Off by default – no new DFT calculations either way, reuses the data/fit already computed for the report. switch, off by default

`--view`

Show an interactive matplotlib preview (every measured direction/pattern overlaid with its own fit) before finishing. Never saved to disk – pass `--save-gnuplot` for that. Off by default. switch, off by default

`--save-report`

Also persist the report to `elastic_stage2.txt`. Off by default. switch, off by default

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

11.3 Cohesive Energy

At a glance

Menu code: 5.3 (a multi-stage workflow)

Code	Stage	Command
5.3.1	Stage 1 - Prep	`stb-cohesive`
5.3.2	Stage 2 - Analysis	`stb-cohesiveAnalysis`

Guided example: `examples/5.3-cohesive/`

Summary: Set up bulk + isolated-atom calculations, then compute the cohesive energy per atom.

11.3.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/5.3-cohesive/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

This workflow has 2 stages: **Stage 1** (`stb-cohesive`, code `5.3.1`) takes an already-relaxed structure and writes one ready-to-run, single-point SIESTA folder for the full structure, one Gamma-only folder per chemical species (each atom alone in a large vacuum box), and — by default — a second set of isolated-atom-type folders where that same atom is surrounded by "ghost" copies of its real neighbors, correcting for a systematic bias localized-basis DFT has for this exact quantity. **Stage 2** (`stb-cohesiveAnalysis`, code `5.3.2`) reads the finished SIESTA runs back and reports the cohesive energy per atom — uncorrected and (if present) BSSE-corrected — plus whether that correction has actually converged. Together they answer one physical question neither `5.1-strain` nor `5.2-elastic` does: not how the structure *responds* to a deformation, but **how strongly bound it is in the first place** — the energy per atom needed to pull the solid apart into free, isolated atoms.

Both stages live in this one folder and this one tutorial — not one folder per tool — for the same reason `5.1-strain/` and `5.2-elastic/` do: Stage 1's output only exists to feed Stage 2, and Stage 2 only exists to interpret Stage 1's output.

11.3.1.1 Theory

What "cohesive energy" means. For a structure with `N` atoms and total (Kohn-Sham) energy `E_bulk`, and each chemical species' isolated-atom energy `E_atom(species)`:

```
E_coh = (E_bulk - sum_i E_atom(species_i)) / N
```

(the sum runs over every atom in the cell, so a species with 2 atoms contributes `2 * E_atom(species)`). **Negative `E_coh` means a bound solid** — it's the energy *released* by assembling `N` free atoms into the observed structure, per atom; equivalently, `-E_coh` is the energy you'd have to *supply* to pull the solid apart back into isolated atoms. A more negative number means a more strongly bound material. This is exactly the arithmetic `stb-cohesiveAnalysis`'s own `[3] COHESIVE ENERGY RESULTS` table performs, with `E_bulk` from `structure/calc.out` and each `E_atom` from `atoms/<species>/calc.out`.

Why LCAO cohesive energies over-bind: Basis Set Superposition Error. SIESTA uses **localized atomic-like orbitals (LCAO/PAO)**, not plane waves — a finite set of basis functions centered on each atom, not a smooth, basis-independent grid filling all of space. This creates a subtle but real bias specific to cohesive-energy-type calculations: when an atom sits alone in `atoms/<species>/`, it only has *its own*

basis functions to vary the wavefunction with. When that *same* atom sits inside the real solid, it also has variational freedom from *its neighbors'* basis functions — even though those neighbors' own electrons aren't "given" to it, their orbital tails still overlap and let the atom's own wavefunction relax into a lower-energy shape than its own basis alone would allow. The isolated-atom reference energy computed with only its own basis is therefore **too high** (under-relaxed) — and since `E_coh` subtracts that reference, a too-high isolated-atom energy makes `E_coh` **too negative**: the structure looks more strongly bound than it really is. This is a basis-set artifact, not real physics — it doesn't happen in plane-wave DFT, where every calculation already uses the same basis (a uniform grid) regardless of what's actually inside the box. Section 4 shows exactly how large this bias is for a real material (spoiler: large — about 40%).

The counterpoise (ghost-atom) correction. The standard fix, adapted here from quantum chemistry's own counterpoise-correction literature, is to give the isolated atom the *same* basis-function environment it has inside the solid, **without adding any real physics**: recompute its energy while surrounding it with "ghost" copies of its actual nearest neighbors — same basis orbitals as the real element, but (SIESTA's own ghost-atom convention) **zero nuclear charge and zero valence electrons**. A ghost atom contributes basis functions for the real atom to relax into, and nothing else — no extra electron, no extra nuclear attraction, no new physics. `stb-cohesive` builds this automatically from the real structure's own local geometry (`--bsse-correction`, ON by default): for each species, it finds that species' real nearest neighbors in `structure.fdf` within a chosen radius (section 1.4) and writes a small cluster — the one real atom at the center, ghost copies of its real neighbors placed at their real relative positions — into `atoms_bsse/<species>/`. `stb-cohesiveAnalysis` then reports the cohesive energy computed both ways side by side.

1.4 The cutoff radius (`--bsse-cutoff`): what it controls, how to read it. `--bsse-cutoff` (default `4.0` Ang) is the radius, in Angstrom, around each species' real anchor atom within which *real* neighbors from the structure become *ghost* atoms in its BSSE cluster. How many coordination shells this actually reaches depends entirely on the material's bond length — for short-bonded covalent solids (graphene, diamond, ~1.4-1.5 Ang bonds) even a modest cutoff reaches several shells; for longer-bonded ionic/metallic solids it may only reach the first. `stb-cohesive`'s own `[5] BSSE (COUNTERPOISE) CORRECTION` table prints the actual **"Ghost nb."** count it found for each species/site — this is the direct, checkable signal for whether a chosen cutoff is doing anything at all (a count of `0` prints a `[WARNING]`: the "corrected" reference would be identical to the uncorrected one).

Convergence: why one cutoff is never enough on its own. Choosing a cutoff and trusting it blindly is not enough — the correction must be shown to have **stabilized** as the cutoff grows, the same methodological discipline `5.1-strain`/`5.2-elastic` apply to their own strain range/mesh choices. `stb-cohesive` offers two ways to check this, both gated behind `--bsse-convergence-check`:

- **A single before/after point** — `--bsse-convergence-increment` given *one* value (the default, `2.0`) generates exactly one extra reference at `--bsse-cutoff + increment`, written to `atoms_bsse_check/`. `stb-cohesiveAnalysis` reports the shift between the two as `BSSE cutoff convergence shift` in `[3]` — small means converged at this cutoff already; large means keep increasing `--bsse-cutoff`.
- **A full multi-cutoff scan** — give `--bsse-convergence-increment` *two or more* values (e.g. `--bsse-convergence-increment 2 4 6`) and `stb-cohesive` generates one extra reference per value, each in its own `atoms_bsse_check_<cutoff>/`. `stb-cohesiveAnalysis` auto-detects this and adds a `[3b] BSSE CUTOFF CONVERGENCE SCAN` section — a table of cohesive energy per atom at every scanned cutoff plus the change on the *last* step — and a third panel on `cohesive_correction.png` plotting the whole trend. This is the only way to actually *see* whether the correction has plateaued, rather than infer it from a single before/after gap; section 4 tells the real story of why this scan feature exists at all.

The vacuum box, periodic images, and a real bug found while building this. SIESTA is **always periodic** — there is no true "isolated atom" mode. `atoms/<species>/` and every BSSE cluster are really a sparse periodic lattice of copies of themselves, spaced apart by a large empty (`--vacuum`-sized) box, and simply relies on that spacing being large enough that neighboring copies don't meaningfully interact through their own basis orbitals' tails. If the box is too small for how far those orbitals actually reach, you get a real, physical-looking but entirely spurious energy shift from an atom "seeing" its own periodic image.

A real bug in this exact spot was found and fixed while building this workflow: the ghost-cluster box used to be a **fixed** `--vacuum`-sized cube *regardless* of `--bsse-cutoff` — so a larger convergence-check cutoff (section 1.5) placed its outermost ghost atom farther from the box center while the box itself stayed the same size, silently eating into the very buffer meant to prevent periodic self-interaction. This is exactly backward: the moment you deliberately widen the cutoff to *check* convergence is the moment the box most needs to grow too. The fix: the box side is now `2 * cutoff + --vacuum` for every BSSE-type reference — a **plain isolated atom** (`atoms/<species>/`, cutoff `0`) is unaffected (box `= --vacuum`, exactly as before), while a BSSE cluster or convergence-check cluster always keeps a guaranteed `--vacuum / 2` buffer beyond its own outermost ghost atom, no matter how large the cutoff gets. Section 4 walks through the real numbers this bug produced (or, as it turned out, mostly didn't — read on).

Multi-site weighting. A species doesn't always sit in one local environment. When a species occupies 2 or more *symmetrically distinct* crystallographic sites — real, different local coordination, not just different unit-cell copies of the same site — a single flat BSSE reference for that species would silently average over 2 physically different situations. `--bsse-multi-site` (default ON) detects this via space-group symmetry (`core/symmetry.py::find_inequivalent_sites`) and builds one ghost cluster **per distinct site**, in nested `atoms_bsse/<species>/site_<wyckoff>_x<mult>/` folders. `stb-cohesiveAnalysis` then reads all of them back and computes a single reference energy **weighted by each site's own multiplicity** (`read_bsse_energy`'s weighted average) — so a species with, say, 6 atoms in one environment and 2 in another gets a reference that's actually representative of its real population, not whichever site happened to be generated first. Section 2.5 demonstrates this on a real material where it genuinely matters: AB-stacked bilayer graphene, where carbon has 2 real, distinct local environments in the *same* structure.

Spin and k-grid conventions for isolated-atom-type calculations. Every isolated-atom-type calculation (plain atom, BSSE cluster, convergence -check cluster) is **always spin-polarized and Gamma-only**, regardless of the *full* structure's own `--spin` setting or k-point density — correct physics for the overwhelming majority of cases (an isolated atom's ground state is essentially always non-trivially spin-polarized, e.g. a lone carbon atom has 2 unpaired 2p electrons; a large vacuum box has no dispersion for a denser k-grid to sample). `stb-cohesive` prints an explicit `[NOTE]` about this in `[3] FULL STRUCTURE SETUP` whenever the full structure itself is run non-polarized: if the *bulk* material is expected to order magnetically, comparing its non-polarized bulk energy against polarized atomic references is an *inconsistent*, not just incomplete, comparison — pass `--spin` on the full structure too if in doubt.

11.3.1.2 2. Stage 1: generating the calculations (`stb-cohesive`, code `5.3.1`)

What it does. Reads one structure, then writes (all under one wrapping `-O/--output-dir`, default `cohesive_runs/`, matching `strain_runs//elastic_runs/`):

- `structure/` — the full structure, one single-point SIESTA folder.
- `atoms/<species>/` — one Gamma-only isolated-atom folder per species that actually has atoms in the cell (a species declared in `ChemicalSpeciesLabel` but never placed contributes exactly 0 to the cohesive-energy sum regardless of its own energy, so it's correctly skipped — section 2.6).

- `atoms_bsse/<species>/[site_<wyckoff>_x<mult>/]` — the BSSE-corrected reference(s), unless `--no-bsse-correction`.
- `atoms_bsse_check/` (one extra cutoff) or `atoms_bsse_check_<cutoff>/` (2+ cutoffs, a scan) — only with `--bsse-convergence-check`.

2.2 No `calc.fdf` prerequisite — and why. Unlike `stb-strain`/`stb-elasticInputs`, `stb-cohesive` takes **no** `-c/--calc-fdf` input at all — it always builds its own single-point SCF `calc.fdf` internally (DZP basis, GGA-PBE, `MeshCutoff 320 Ry`, `SCF.DM.Tolerance 1e-5 eV`), identically across the full structure and every isolated-atom-type calculation. This is deliberate: a cohesive-energy comparison is only meaningful if every energy in the sum comes from *exactly* the same basis/functional/mesh — safer to guarantee that by generating every `calc.fdf` from one template than to trust a hand-edited file to stay in sync everywhere. `-k/--k-density` (default `0.2` 1/Ang) and `--spin`/`--dispersion` are the only physics knobs Stage 1 exposes for the full structure; everything else about the SCF setup is fixed on purpose.

The vacuum-buffer guarantee, live.

```
$ stb-cohesive -s structure.fdf -p dojo --bsse-cutoff 4 \
    --bsse-convergence-check --bsse-convergence-increment 2 --no-intro

[5] BSSE (COUNTERPOISE) CORRECTION
------------------------------------------------------------
Species | Site | Ghost nb. | Cutoff  | Used | Folder
------------------------------------------------------------
Ga      | --   | 4         | 4.0 Ang | yes  | atoms_bsse/Ga/
As      | --   | 4         | 4.0 Ang | yes  | atoms_bsse/As/
...
Also generated 'atoms_bsse_check/' at cutoff 6.0 Ang (--bsse-cutoff +
--bsse-convergence-increment) -- stb-cohesiveAnalysis will report the shift
between the two as a convergence diagnostic. Its box is 32.0 Ang wide (vs.
28.0 Ang for the --bsse-cutoff reference) -- both keep the same 10.0 Ang
buffer beyond their respective outermost ghost atom, so this shift reflects
the correction's real cutoff dependence, not a box-size artifact.
```

`atoms_bsse/Ga/structure.fdf`'s own `%block LatticeVectors` is a `28.0 Ang` cube (`2*4.0 + 20.0`); `atoms_bsse_check/Ga/structure.fdf`'s is `32.0 Ang` (`2*6.0 + 20.0`) — both leave exactly `10.0 Ang` (`--vacuum/2`) of empty space beyond their own outermost ghost atom. Case 2 of `example_5.3.sh` reproduces this exact check on the real GaAs structure in this folder.

Single check vs. multi-cutoff scan, live.

```
$ stb-cohesive -s structure.fdf -p dojo --bsse-convergence-check \
    --bsse-convergence-increment 2 4 6 --no-intro

BSSE convergence check: ON, scanning 3 cutoffs (6.0, 8.0, 10.0 Ang)
...
Also generated 3 BSSE convergence-check references -- a full cutoff scan
from --bsse-convergence-increment's multiple values -- so
stb-cohesiveAnalysis can report and plot the cohesive energy vs. cutoff
trend instead of a single before/after point. Every box keeps the same
10.0 Ang buffer beyond its own outermost ghost atom (box side = 2*cutoff +
--vacuum), regardless of cutoff, so the trend reflects the correction's
real cutoff dependence, not a box-size artifact.
Cutoff   | Box side | Folder
-------------------------------------------
6.0 Ang  | 32.0 Ang | atoms_bsse_check_6.0/
```

```
8.0 Ang  | 36.0 Ang | atoms_bsse_check_8.0/
10.0 Ang | 40.0 Ang | atoms_bsse_check_10.0/
```

With **exactly one** `--bsse-convergence-increment` value (the default), naming/behavior is unchanged from before this scan feature existed — `atoms_bsse_check/`, no suffix. Case 3 of `example_5.3.sh` runs this live.

Multi-site detection and per-site folders, live. Run against `structure_multisite.fdf` in this folder (AB-stacked bilayer graphene, section 1.7):

```
[5] BSSE (COUNTERPOISE) CORRECTION
------------------------------------------------------------
[INFO] 2 symmetrically distinct site(s) detected for C: d (x2), c (x2)
Species | Site   | Ghost nb. | Cutoff  | Used | Folder
-------------------------------------------------------------------------
C       | d (x2) | 24        | 4.0 Ang | yes  | atoms_bsse/C/site_d_x2/
C       | c (x2) | 22        | 4.0 Ang | yes  | atoms_bsse/C/site_c_x2/
...
Space group(s) detected (per species, same structure -- should agree): C=P-3m1 (No. 164)
```

The 2 sites are the well-known "eclipsed"/dimer vs. "non-eclipsed" carbon environments of Bernal-stacked bilayer graphene (section 1.7's own structural explanation) — 2 real coordination environments for the exact same element, correctly kept separate rather than averaged into one reference. Case 4 of `example_5.3.sh` runs this live and shows the nested folder layout on disk.

Phantom species (declared but never placed) are correctly skipped. A `structure.fdf` can declare a species in `%block ChemicalSpeciesLabel` that never actually appears in `%block AtomicCoordinatesAndAtomicSpecies` (e.g. hand-edited from a template, or a placeholder for a future substitution). Since such a species contributes exactly `0 * E_atom` to the cohesive-energy sum no matter what its own energy would be, `stb-cohesive` correctly does **not** generate an `atoms/<species>/` (or BSSE) folder for it, and `[1] INPUT STRUCTURE` prints an explicit `[INFO] Declared but never placed (no isolated-atom calculation needed): <species>` instead of silently requiring (and blocking on) a calculation nobody needs. Case 5 of `example_5.3.sh` demonstrates this directly.

Output layout, and running it both ways.

Path	What it is
`cohesive_runs/structure/`	Full structure, single-point SCF
`cohesive_runs/atoms/<species>/`	Isolated atom, Gamma-only, spin-polarized
`cohesive_runs/atoms_bsse/<species>/[site_.../]`	BSSE-corrected reference(s)
`cohesive_runs/atoms_bsse_check[_<cutoff>]/...`	Convergence check/scan reference(s)

Both CLI and interactive menu (`stb-suite` → `5.3.1`) produce identical output for identical answers — Case 6 of `example_5.3.sh` proves this with a real `diff -rq`.

11.3.1.3 3. Stage 2: analyzing the results (`stb-cohesiveAnalysis`, code `5.3.2`)

Report structure.

```
[0] RUN METADATA
[1] INPUT STRUCTURE
[2] ENERGY EXTRACTION
```

```
[3] COHESIVE ENERGY RESULTS
[3b] BSSE CUTOFF CONVERGENCE SCAN     <- only when a multi-cutoff scan was found
[4] CORRECTION PLOT
[5] SUMMARY & FILES
```

3.2 `-d/--dir` auto-defaults to `cohesive_runs`. Matching Stage 1's own `-O/--output-dir` default, so `stb-cohesiveAnalysis -o calc.out` works with **no `--dir` at all** in the common case, run from the same parent directory Stage 1 was — no manual `cd` required either way. An explicit `[FAIL] Directory '<dir>' not found.` fires immediately if it's missing, rather than failing indirectly later.

Numerical-quality diagnostics. `[2] ENERGY EXTRACTION` checks the **full structure's own** `calc.out` (not the isolated-atom-type ones — an isolated atom has zero net force by symmetry, and a BSSE cluster's real atom only feels basis-incompleteness -driven Pulay forces from its ghosts, not a physical force worth "relaxing" away): SCF convergence, and residual force against `--force-tolerance` (default `0.05` eV/Ang) — a `[WARNING]` here means the cohesive energy may reflect a strained/off-equilibrium geometry, not the true minimum. Advisory only, never blocks the result.

3.4 Reading the `[2]`/`[3]` tables. `[2]`'s per-species table has a **Delta** column for each correction, relative to the uncorrected reference — since BSSE removes an artificial over-stabilization of the isolated atom (section 1.2), a **positive** Delta here is the expected, physically correct direction (the isolated atom's energy going *up*, closer to what it should be). `[3]`'s method table mirrors this with a **Delta vs Uncorrected** column on the final cohesive energy per atom itself — the single number that answers "how much did BSSE actually change my answer".

3.5 `[3b] BSSE CUTOFF CONVERGENCE SCAN`. Only printed when a multi-cutoff scan (section 2.4) is found. One row per scanned cutoff (plus the base `--bsse-cutoff` reference, labeled `(base)`, recovered from a small `bsse_cutoff.txt` sidecar Stage 1 writes alongside every reference — degrades gracefully to "value not recoverable" for an older run generated before this file existed, still showing the scan points themselves). The closing **"Last step change"** line is the number to actually look at: small means the last cutoff increase barely moved the answer (converged); still large means keep increasing `--bsse-cutoff`/`--bsse-convergence-increment` further. Section 4 shows this exact section with real, physically-verified numbers.

The matplotlib plot. `cohesive_correction.png` is **always saved** (no flag needed) whenever a complete BSSE-corrected reference is available — 2 panels normally (per -species correction shift relative to uncorrected; net effect on cohesive energy per atom), a 3rd panel (cohesive energy per atom vs. cutoff) added automatically whenever `[3b]`'s scan data exists. `--view` additionally pops the same figure up interactively (a no-op under a non-interactive backend like the `MPLBACKEND=Agg` this example's own script uses).

Running it both ways. Both CLI and `stb-suite` → `5.3.2` (which now also prompts for `--save-report`/`--view`, not just the required filename/directory) produce the identical report for identical answers.

11.3.1.4 Worked example: a real GaAs calculation, end-to-end

This is not illustrative data — it's the exact real investigation that motivated sections 1.5/1.6/2.4 and the multi-cutoff scan feature itself, run on the real `structure.fdf` in this folder (GaAs, zincblende, DZP/GGA -PBE, `dojo` pseudopotentials, `-k 0.2`, `--bsse-cutoff 4.0`, `--bsse-convergence-check`).

First pass (before the vacuum-buffer fix, section 1.6 — box was a fixed 20 Ang cube regardless of cutoff):

Method	Per atom (eV/atom)	Delta vs Uncorrected
Uncorrected	-4.6876	(reference)
BSSE-corrected (4.0 Ang)	-3.6479	+1.0397
BSSE check (6.0 Ang)	-3.1724	+1.5152

Reading this the way sections 1.2/1.5 set up:

- **The uncorrected result is badly over-bound** — a commonly cited experimental/reference cohesive energy for GaAs is around `~3.3` eV/atom; `-4.6876` is roughly **42% too strong**, the textbook BSSE signature (section 1.2) at essentially textbook scale.
- **BSSE-corrected (`-3.6479`) lands much closer** to that reference value (~11% high, a very ordinary residual GGA-PBE error) — strong evidence the correction is doing exactly what section 1.3 says it should.
- **BSSE check (`-3.1724`) is a large, suspicious jump** — `+0.4755 eV/atom` between a 4.0 and 6.0 Ang cutoff is about 13% of the entire cohesive energy, far too large to treat the 4.0 Ang result as converged.

That large jump is what triggered the actual investigation: was it real physics (the cutoff genuinely too small), or the vacuum-box artifact section 1.6 warns about? The evidence pointed at the box: with the *old* fixed 20 Ang cube, the 6.0 Ang-cutoff cluster's outermost ghost atom sat only `4.0` Ang from the box edge (`half_box - cutoff = 10 - 6`) — and the real PAO orbital cutoff radii read straight out of `calc.out` (`grep "rc ="`) showed Ga's own basis reaching as far as **7.12 Ang**, comfortably farther than that 4.0 Ang buffer. A real risk of periodic self-interaction, not a theoretical one.

Second pass, after fixing the vacuum-buffer bug (section 1.6: box side `= 2*cutoff + vacuum`, guaranteeing a constant `10.0 Ang` buffer at *any* cutoff) and re-running for real:

Method	Per atom (eV/atom), fixed box	Change vs. first pass
BSSE-corrected (4.0 Ang)	-3.6502	+0.0023 eV/atom
BSSE check (6.0 Ang)	-3.1741	+0.0017 eV/atom

The numbers barely moved at all — about 1-3 meV, an order of magnitude smaller than typical real-space-grid ("eggbox") noise, let alone a correction of order 1 eV. If the old fixed box really had been contaminating the result, tripling its buffer (4.0 → 10.0 Ang) should have changed the answer visibly. It didn't.

Conclusion: the large 4.0→6.0 Ang shift is **real physics, not a box -size artifact** — 4.0 Ang (4 ghost neighbors, the first coordination shell only) simply isn't a large enough cluster for GaAs's BSSE correction to have converged. The vacuum-buffer fix was still the right thing to do (it removes a real risk that, for a different material or a larger requested cutoff, absolutely could bite — sections 1.6/2.3), but it wasn't *this* material's actual problem. This is exactly what directly motivated building the multi-cutoff scan (section 1.5/2.4): a single before/after pair can't tell you *whether* a shift is converging, only that it changed. **An honest note for anyone reproducing this**: a real multi-point scan (`--bsse-convergence-increment 4 8 12`, say) has not yet been run on this real structure — a genuine next step, not a result already in hand.

One more small, secondary finding along the way, purely for illustration of how much a numbered report surfaces without being asked: `stb-cohesive` detected space group `R3m` (trigonal, No. 160) for this structure, not the ideal cubic zincblende `F-43m` (No. 216) — traced to the full structure's own SIESTA stress tensor showing a small residual shear (`-1.09` kbar alongside a `-19.03` kbar hydrostatic component), i.e. this specific relaxed geometry carries a small real trigonal-type distortion, not a symmetry -detection bug. Zero practical effect here (each species has only 1 atom in the cell, so site multiplicity is 1 regardless

of point group), but exactly the kind of thing worth noticing before trusting a multi-site weighting result (section 1.7) on a less trivial structure.

11.3.1.5 Known, deliberate limitations

- **No `calc.fdf` customization** (section 2.2) — basis/functional/mesh are fixed by `stb-cohesive`'s own template; only `-k`, `--spin`, `--dispersion` are exposed, on purpose.
- **`--bsse-cutoff`/`--vacuum` are user-chosen, never auto-optimized** — `stb-cohesive` reports the ghost-neighbor count and (with `--bsse-convergence-check`) the convergence trend, but never picks a "safe" cutoff for you; section 4's own real result shows why that would be genuinely hard to automate reliably.
- **The convergence scan is diagnostic only** — it tells you *whether* the correction has stabilized, not what cutoff to use if it hasn't; you still have to re-run with a larger `--bsse-cutoff`/wider `--bsse-convergence-increment` values yourself.
- **Isolated-atom-type spin/k-grid are fixed** (section 1.8) — always spin-polarized, always Gamma-only, not independently configurable from the full structure's own settings.
- **Dimensionality/vacuum-gap detection** shares the exact same heuristic (and the same caveats) as `stb-kgrid`/`stb-strain`/`stb-elasticInputs`.

11.3.1.6 Step-by-step: running this workflow on your own structure

0. **Prerequisite**: an already-relaxed structure (`structure.fdf`, fractional coordinates). Unlike `5.1`/`5.2`, there is **no** separate `calc.fdf` to prepare (section 2.2) — `stb-cohesive` builds its own.
1. **Decide on `--bsse-correction`** (default ON, section 1.2/1.3): keep it on unless you specifically want the (systematically over-bound) uncorrected number for some other reason.
2. **Decide `--bsse-cutoff`** (section 1.4): start at the default `4.0` Ang, check the printed "Ghost nb." count per species, and plan to verify convergence (step 5) rather than trusting one value blindly.
3. **Run Stage 1** (`stb-cohesive`, code `5.3.1`):
   - CLI: `stb-cohesive -s structure.fdf -p <bank> --bsse-convergence-check --bsse-convergence-increment <inc1> [inc2 ...]` (one increment for a single check, several for a full scan — section 2.4).
   - Interactive: `stb-suite` → `5.3.1` — same questions instead of flags (the convergence-increment prompt also accepts several space-separated values now).
   - Writes everything under `cohesive_runs/` (section 2.1/2.7).
4. **Run SIESTA yourself in every generated folder.** Stage 1 never runs SIESTA — `cd` into `structure/`, each `atoms/<species>/`, each `atoms_bsse/...` and `atoms_bsse_check.../` folder and run it like any other SIESTA calculation.
5. **Run Stage 2** (`stb-cohesiveAnalysis`, code `5.3.2`) once every run finishes:
   - CLI: `stb-cohesiveAnalysis -o calc.out` (no `--dir` needed in the common case, section 3.2).
   - Interactive: `stb-suite` → `5.3.2`.
6. **Read `[2]`'s SCF/force diagnostics** (section 3.3) before trusting the numbers, then **`[3]`/`[3b]`** (sections 3.4/3.5) for the actual cohesive-energy result and whether the BSSE correction has converged — if `[3b]`'s last step is still large, go back to step 2 with a larger cutoff/wider scan.
7. **(Optional) persist or view**: `--save-report` writes `cohesive_results.dat`; `--view` opens `cohesive_correction.png` interactively (it's always saved to disk regardless, section 3.6).

11.3.1.7 Files in this folder

File	What it is
`structure.fdf`	Real, relaxed GaAs, 2 atoms/cell — the material section 4's real numbers are for
`structure_multisite.fdf`	Purpose-built AB-stacked bilayer graphene, 4 C atoms/cell, 2 distinct sites — multi-site demo only (section 2.5)
`example_5.3.sh`	This walkthrough's runnable script (both stages, 8 cases)

11.3.1.8 Running the script

```
bash example_5.3.sh
```

11.3.1.9 What's next

- **`5.1-strain`** / **`5.2-elastic`** / **`5.4-phonons`** — a different mechanical or vibrational question (how the structure *responds* to a deformation, or how it vibrates) about the same kind of already-relaxed structure this workflow starts from.
- There is currently **no ML-preview twin tool** for this workflow (unlike `5.2-elastic`'s `stb-mlelastic`) — a real, documented gap in the suite today, not an oversight in this tutorial.

11.3.2 Implementation details

No template needed.. Unlike the other workflows, Stage 1 does not take a `calc.fdf`: every folder receives the same built-in template, so the bulk and the atoms are guaranteed to share basis (`PAO.BasisSize DZP`, `PAO.EnergyShift 0.02 Ry`), `Mesh.Cutoff` (320 Ry), functional (PBE) and SCF settings. The differences are deliberate:

	`structure/`	**`atoms/`, `atoms_bsse*`**
k-grid	from `-k` (default 0.2 Å^{-1}), 1 along vacuum axes	Γ only
spin	non-polarized, unless `-spin`	always polarized (free atoms are usually open-shell)
cell	the input cell	cube of side `-vacuum` (default 20 Å), checked to fit the cluster
dispersion	`DFTD3` with `-dispersion`	same

Edit the generated `calc.fdf` files consistently (all of them) if your production settings differ – the cohesive energy is only meaningful with identical settings everywhere.

Counterpoise-corrected cohesive energy

With N atoms in the cell,

$$E_{\mathrm{coh}} = \frac{1}{N}\Big(E_{\mathrm{bulk}} - \sum_{i=1}^{N} E^{\mathrm{atom}}_{s(i)}\Big), \qquad E^{\mathrm{CP}}_{\mathrm{coh}} = \frac{1}{N}\Big(E_{\mathrm{bulk}} - \sum_{i=1}^{N} E^{\mathrm{ghost}}_{s(i)}\Big),$$

where E^{ghost}_{s} is the energy of the isolated atom computed with the basis functions of its neighbours present as ghost atoms (Boys–Bernardi counterpoise [71]). The ghost cluster is the real atom at the centre of the box surrounded by every neighbour within `-bsse-cutoff` (default 4.0 Å) at its true relative position, each neighbour as a ghost species (`<El>_ghost`: negative Z, no electrons, the same basis and pseudopotential file as the real element).

Several inequivalent sites.. When a species occupies symmetrically distinct sites (`-symprec`), each gets its own ghost cluster (`site_<label>_x<multiplicity>/`) and the analysis weights them by multiplicity (`-no-bsse-multi-site` uses one representative).

Convergence of the correction.. The counterpoise correction grows with the cutoff as more ghost functions are included and saturates once the neighbours beyond the cutoff no longer overlap the atom's orbitals. `-bsse-convergence-check` adds clusters at cutoffs increased by `-bsse-convergence-increment` (one or several values), and Stage 2 tabulates and plots $E_{\mathrm{coh}}^{\mathrm{CP}}$ against the cutoff.

Stage 2.. **`stb-cohesiveAnalysis`** reads the energies from the log named by `-o`/`-out` in every folder under `-d`, checks SCF convergence and residual forces (`-force-tolerance`, default 0.05 eV/Å), counts atoms per species from the bulk structure, and reports the plain and corrected cohesive energies per atom, the correction per species, and the cutoff scan; results go to `cohesive_results.dat` and `cohesive_correction.png` (with `-view`).

Spin of the free atoms.. SIESTA's smeared occupations and spherical-atom starting density may not reach the true atomic ground state (Hund's rules, orbital polarization); a free-atom energy that is too high makes E_{coh} too negative. Check the spin moment of each atom in its log against the expected ground-state multiplicity.

11.3.3 Command-line reference

11.3.3.1 `stb-cohesive`

Prepare folder structure for cohesive energy calculations.

Synopsis.

```
usage: stb-cohesive [-h] -s STRUCTURE [-p PP_PATH] [-k K_DENSITY] [--spin]
                    [--vacuum VACUUM] [--vacuum-gap VACUUM_GAP] [-O OUTPUT_DIR]
                    [--bsse-correction] [--no-bsse-correction]
                    [--bsse-cutoff BSSE_CUTOFF] [--symprec SYMPREC]
                    [--bsse-multi-site] [--no-bsse-multi-site]
                    [--bsse-convergence-check]
                    [--bsse-convergence-increment BSSE_CONVERGENCE_INCREMENT [BSSE_CONVERGENCE_INCREMENT ...]]
                    [--dispersion] [--save-report] [-v] [--no-intro]
```

Options

`-s, --structure STRUCTURE`
Path to the initial `structure.fdf` file — required

`-p, --pp-path PP_PATH`
Pseudopotentials source (optional): a bundled bank (dojo, virtual_vault) or a folder path.

`-k, --k-density K_DENSITY`
K-point density in 1/Angstrom (default: 0.2) — type: float

`--spin`
Set the full structure calculation to spin polarized — switch, off by default

`--vacuum VACUUM`
Guaranteed buffer (Ang/2, see below) between the outermost atom of each isolated-atom-type calculation and the edge of its cubic simulation box (default: 20.0). For a plain isolated atom (no neighbors) the box is exactly

this value, same as before. For a BSSE ghost cluster (see `--bsse-cutoff`) or its `--bsse-convergence-check` counterpart, the box automatically GROWS with the cluster's own radius (box side = 2*cutoff + `--vacuum`) so the buffer beyond the outermost ghost atom stays exactly `--vacuum/2` regardless of how large `--bsse-cutoff/--bsse-convergence-increment` are – otherwise a bigger convergence-check cluster would eat into a fixed-size box and risk interacting with its own periodic images, contaminating the very convergence check it's meant to validate. Increase for elements with unusually diffuse basis orbitals. type: float

`--vacuum-gap VACUUM_GAP`

Vacuum gap threshold in Ang used to detect which lattice axes of the FULL structure are periodic vs. vacuum-padded (default: 10.0), same convention as `stb-kgrid/stb-strain/stb-elasticInputs`. Vacuum-padded axes are forced to a single (Gamma-only) k-grid division regardless of `--k-density`. type: float

`-O, --output-dir OUTPUT_DIR`

Root directory (default: cohesive_runs, matching `stb-strain's/stb-elasticInputs`' own <property>_runs convention) for the 'structure', 'atoms' and 'atoms_bsse' folders – so `stb-cohesiveAnalysis` can just 'cd <output-dir>' (or pass `--dir`) and find everything with no extra flags. Set this to avoid silently overwriting a previous run's setup when preparing more than one structure from the same working directory.

`--bsse-correction`

Also generate a BSSE (Basis Set Superposition Error) `-corrected` reference per species, in 'atoms_bsse/<symbol>/' (default: ON). LCAO cohesive energies are otherwise systematically over-bound: the isolated atom is computed with a smaller effective basis than it has inside the solid (no neighboring atoms' orbitals available). The correction embeds the same real atom in 'ghost' copies (SIESTA convention: negative Z, no valence charge, same basis as the real element) of its actual nearest neighbors from the full structure (see `--bsse-cutoff`), giving it the same effective basis it has in the solid without changing its physics. Doubles the number of isolated-atom-type DFT calculations needed (one plain + one BSSE-corrected per species). switch, off by default

`--no-bsse-correction`

Skip the BSSE-corrected reference (halves the isolated-atom-type calculation count, at the cost of a systematically over-bound cohesive energy). switch

`--bsse-cutoff BSSE_CUTOFF`

Radius in Ang (default: 4.0) around each species' anchor atom within which real neighbors become ghost atoms in its BSSE-corrected reference (see `--bsse-correction`). How many coordination shells this actually reaches depends on the material's bond length – for short-bonded covalent solids (e.g. graphene, diamond, ~1.4-1.5 Ang bonds) it typically reaches several shells; for longer-bonded ionic/metallic solids it may only reach the 1st. Check the printed ghost-neighbor count per species; the correction is converged once increasing `--bsse-cutoff` no longer changes the reported BSSE-corrected energy appreciably. Only used when `--bsse-correction` is on. type: float

`--symprec SYMPREC`

Symmetry-detection tolerance (default: 0.01, pymatgen's own default), used to find symmetrically distinct sites per species for `--bsse-multi-site`. Loosen further (e.g. 0.02-0.05) for a structure relaxed with a looser force tolerance – a tighter value than the relaxation's own residual numerical noise can misdetect the true space group (see `stb-symmetry --scan-symprec` to check where a given structure's symmetry call is sensitive). type: float

`--bsse-multi-site`

When a species occupies 2+ symmetrically distinct sites (e.g. octahedral vs. tetrahedral Fe), build a separate BSSE ghost-cluster reference PER site (default: ON) – `stb-cohesiveAnalysis` then weights each site's BSSE-corrected energy by its multiplicity, instead of using a single (potentially unrepresentative) site for the whole species. switch, off by default

`--no-bsse-multi-site`

Use only the first detected site as the BSSE reference for a species with multiple distinct sites (prints a warning naming the sites being skipped), instead of one cluster per site. switch

`--bsse-convergence-check`
: Also generate one or more extra BSSE references per species/site at a larger cutoff (see `--bsse-convergence-increment`) – `stb-cohesiveAnalysis` then reports how much the BSSE-corrected cohesive energy shifts as the cutoff grows, as a convergence check (default: OFF – on top of `--bsse-correction`'s own doubling, each extra cutoff adds one more isolated-atom-type calculation per species/site). Requires `--bsse-correction`.switch, off by default

`--bsse-convergence-increment BSSE_CONVERGENCE_INCREMENT [BSSE_CONVERGENCE_INCREMENT ...]`
: One or more Ang values added to `--bsse-cutoff` (default: 2.0, a single point). Pass several (e.g. `--bsse-convergence-increment 2 4 6`) to scan multiple cutoffs in one run and see the correction's full trend instead of a single before/after comparison – `stb-cohesiveAnalysis` will report and plot the whole curve. With exactly one increment (the default), the extra reference is written to 'atoms_bsse_check/' as before; with 2+, each is written to its own 'atoms_bsse_check_<cutoff>/' instead. type: float

`--dispersion`
: Enable Grimme DFT-D3 dispersion correction (SIESTA's own DFTD3 directive, default: OFF) for every calculation (full structure, isolated atoms, and BSSE references alike) – relevant for layered/molecular materials where van der Waals interactions contribute non-negligibly to the cohesive energy. Safe to apply uniformly: a genuinely isolated atom has no neighbor to form a dispersion pair with, so D3 contributes ~0 there regardless. switch, off by default

`--save-report`
: Also persist the report to <output-dir>/cohesive_stage1.txt. Off by default. switch, off by default

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Example usage:
  stb_cohesive -s structure.fdf -k 0.15
  stb_cohesive -s structure.fdf --spin -p /path/to/pseudos
```

11.3.3.2 `stb-cohesiveAnalysis`

Process cohesive energy results from SIESTA calculations.

Synopsis.

```
usage: stb-cohesiveAnalysis [-h] -o OUT_FILE [-d DIR_PATH]
                            [--force-tolerance FORCE_TOLERANCE] [--save-report]
                            [--view] [-v] [--no-intro]
```

Options

`-o, --out OUT_FILE`
: Name of the SIESTA output file (e.g., `calc.out`) required

`-d, --dir DIR_PATH`
: Path to the folder containing 'structure' and 'atoms' directories (default: cohesive_runs, matching `stb-cohesive`'s own `--output-dir` default – no `--dir` needed in the common case, even run from the parent directory).

`--force-tolerance FORCE_TOLERANCE`
: Residual atomic force in eV/Ang (default: 0.05) above which the full structure's `calc.out` is flagged as possibly not relaxed – the cohesive energy would then reflect a strained/off-equilibrium geometry rather than the true minimum. Advisory only, never blocks the result. type: float

`--save-report`
: Also persist the report to <dir>/cohesive_results.dat. Off by default. switch, off by default

`--view`
: Show an interactive matplotlib preview of the BSSE-correction figure (see [4] of the report) before finishing. The figure itself is always saved when BSSE data is available, regardless of this flag. Off by default.switch, off by default

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Example usage:
  stb_cohesive_analysis -o calc.out
  stb_cohesive_analysis -o calc.out -d /path/to/results
```

11.4 Phonons

At a glance

Menu code: 5.4 (a multi-stage workflow)

Code	Stage	Command
5.4.1	Stage 1 - Prep	`stb-phononsCreate`
5.4.2	Stage 2 - Analysis	`stb-phononsPos`

Guided example: `examples/5.4-phonons/`

Summary: Generate displaced supercells via Phonopy, then post-process into thermal properties. For a fast ML-only preview (no SIESTA) instead, including QHA, see ML Simulations > ML Phonons (`stb-mlphonons`).

11.4.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/5.4-phonons/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

This workflow has 2 stages: **Stage 1** (`stb-phononsCreate`, code `5.4.1`) takes an already-relaxed structure and its own `calc.fdf`, displaces each symmetry-inequivalent atom by a tiny amount, and writes one ready-to-run, single-point SIESTA folder per displacement. **Stage 2** (`stb-phononsPos`, code `5.4.2`) reads the finished SIESTA runs (or an ML-computed force-constant file) back and reports whether the structure is dynamically stable, its phonon band structure, density of states, and the thermal properties (free energy, entropy, heat capacity) derived from it. Together they answer a question neither `5.1-strain`, `5.2-elastic`, nor `5.3-cohesive` does: not how the structure *responds* to a deformation or how strongly it's bound, but **how it vibrates**, and whether the relaxed geometry you started from is actually a genuine energy minimum at all (a structure can be perfectly converged in energy/forces and still be dynamically unstable).

Both stages live in this one folder and this one tutorial — same reasoning as `5.1/5.2/5.3`: Stage 1's output only exists to feed Stage 2.

11.4.1.1 Theory

The harmonic approximation and the finite-displacement method. Near its equilibrium geometry, a crystal's potential energy surface is approximated as a quadratic (harmonic) function of atomic displacements — the same approximation behind a classical mass-and-spring lattice model. The curvature of that quadratic form, the **force-constant matrix** Φ `_ij = -dF_i/du_j` (how much the force on atom `i` changes per unit displacement of atom `j`), is exactly what this workflow computes, via finite differences: displace one atom a small, known amount `u` (`--distance`, default `0.01` Ang), compute the resulting forces on every *other* atom with a single-point SCF, and $\Phi \approx$ `-F/u`. Diagonalizing the resulting dynamical matrix (the mass-weighted Fourier transform of Φ) at each q-point gives the phonon frequencies ω`(q)` — the actual physical output of this workflow. This is a genuinely different kind of calculation from `5.1-strain/5.2-elastic` (both static, zero-frequency responses to a finite deformation) — phonons are the *dynamic*, frequency-dependent response of the lattice.

Symmetry-reduced displacements: why AlP only needs 2 of 96. A brute-force calculation would displace every atom in the supercell along every Cartesian direction, both `+` and `-` (Phonopy's own default when no symmetry is assumed, since a real force response isn't perfectly linear) — `N_atoms * 3 * 2` displacements: `16 * 3 * 2 = 96` for AlP's 2x2x2 = 16-atom supercell, exactly the "of 96" Section 3.3's live report quotes. But if the crystal has real symmetry (zincblende AlP: space group `F-43m`, 192 symmetry operations), most of those displacements are related to each other by a symmetry operation and give *redundant* information — Phonopy (via spglib) detects this and generates only the symmetry-inequivalent subset, reconstructing the full force-constant tensor afterward by applying the detected symmetry operations to the computed subset. For AlP this is a **97.9% reduction**: 2 real SIESTA calculations instead of 96 (Section 3.3, live).

From force constants to a band structure: the q-path. A "band structure" plot needs a specific path through reciprocal space connecting high-symmetry points (Γ, X, W, ...) appropriate to the crystal's own Bravais lattice. `stb-phononsPos` auto-detects this using **ASE's own Bravais-lattice machinery** (`Cell.get_bravais_lattice/Cell.bandpath`, the same approach `stb-kpath` already uses for electronic band structures) — deliberately **not** Phonopy's own `seekpath`-based `auto_band_structure`, since `seekpath` isn't a dependency of this suite at all (Section 2). The path itself (e.g. AlP's `Γ-X-W-K-Γ-L-U-W-L-K | U-X`) is standard for an FCC-derived lattice and is printed in every Stage 2 report that uses `--bands`.

Dynamical stability: what a negative frequency means, and why a hard zero is the wrong test. Because ω`(q)` comes from diagonalizing the dynamical matrix, its square ω^2 is the actual eigenvalue — a **negative** eigenvalue gives an **imaginary** frequency, conventionally *displayed* as a negative number (e.g. `-5.8 THz`) even though "negative frequency" isn't physically meaningful on its own. What it means: the harmonic energy actually *decreases* along that specific atomic displacement pattern at that specific q-point — the structure is not sitting in a true local energy minimum along that direction. Causes range from a genuinely unstable phase (a real soft mode, e.g. approaching a phase transition), an insufficiently-relaxed input geometry, to pure numerical noise.

That last case is exactly why **a hard `< 0` test is the wrong tool**: IEEE-754 floating point represents `-0.0` (negative zero) as distinct from, but numerically equal to, `+0.0` — so `-0.0 < 0` evaluates to **False** in Python, meaning a structure whose true acoustic-branch frequency is exactly zero can print as `"-0.0000 THz"` while a hard `< 0` check calls it *stable* — technically not wrong (`-0.0` really is zero), but with zero visible reasoning, which reads as an accident rather than a deliberate rule. Worse in the other direction: a real but tiny residual (e.g. `-0.0003 THz`, ordinary acoustic-sum-rule numerical noise, see Section 4.2)

would trip a hard `< 0` test and print a scary `[WARNING]` for a structure that is, in every physical sense, stable. `stb-phononsPos` instead:

1. Calls `phonon.symmetrize_force_constants()` right after loading — enforces the acoustic sum rule (translational invariance: the sum of forces on all atoms from a rigid whole-crystal translation must be exactly zero) — removing most of this numerical noise at the source.
2. Compares against an explicit, documented tolerance, `IMAGINARY_MODE_TOL_THZ = -0.01 THz` (`core/phonon_workflow.py`, shared with `stb-mlphonons`), instead of a bare `0` — anything less negative than this is treated as "zero, within numerical tolerance."

The report's own wording makes the distinction explicit rather than silent — compare `"No imaginary modes found on the sampled mesh."` (clean, positive/exact-zero result) against `"No imaginary modes found along the band path (within numerical tolerance)."` (a small negative residual, correctly classified stable, but now honestly labeled as such) — Section 4.2 shows this live, on this tutorial's own real data.

Thermal properties from the phonon density of states. Once $\omega(q)$ is known over a dense sampling mesh (`-m/--mesh`, default `20 20 20`), each mode is an independent quantum harmonic oscillator, and standard statistical mechanics gives the vibrational contribution to the free energy, entropy, and constant-volume heat capacity as sums over every mode at every temperature — this is what `[3] THERMAL PROPERTIES SUMMARY` reports. Two textbook cross-checks any real result should pass, both verified live on this tutorial's own data in Section 5:

- **Third law**: entropy `S -> 0` as `T -> 0 K` (a perfect, defect-free harmonic crystal has one unique, fully-ordered ground state).
- **Dulong-Petit limit**: heat capacity `Cv -> 3*N_atoms*R` (the classical equipartition limit, no longer quantum-mechanically suppressed) as `T -> infinity` — for AlP's 2-atom cell, `3*2*8.314 = 49.88 J/K/mol`.

`[3b] DENSITY OF STATES` additionally fits a **Debye model** (a free -particle approximation, `g(omega) ~ omega^2`) to the low-frequency part of the real DOS to extract a Debye frequency/temperature — a single-number summary of the "typical" phonon energy scale, and the origin of the DOS plot's own blue curve (Section 4.4).

What this workflow simplifies — explicitly.

- **Harmonic only.** No phonon-phonon (anharmonic) coupling — no intrinsic phonon lifetime/linewidth, no direct thermal-conductivity calculation. Thermal *expansion* specifically needs a beyond-harmonic treatment (quasi-harmonic approximation, QHA: computing the harmonic phonon spectrum at several *volumes* and combining with an equation of state). **`stb-mlphonons` has a `--qha` mode; this DFT-based Stage 1/2 pair does not** — a real, current gap in the suite, not an oversight in this tutorial (Section 6).
- **No non-analytic term correction (NAC).** Polar materials (AlP and GaAs both qualify — heteropolar III-V semiconductors) have long-range dipole-dipole (Born-effective-charge-mediated) contributions to the dynamical matrix that cause LO-TO splitting (a discontinuity in the optical branches exactly at Γ) — this workflow has no `BORN` file support, so that splitting is simply absent from every band structure and DOS this tutorial shows (Section 6).
- **Symmetry-based displacement reduction assumes the input structure obeys its own detected symmetry almost exactly.** Real DFT-relaxed structures always carry *some* residual numerical noise; Section 5's GaAs case is a real, worked demonstration of this assumption breaking down in practice, not just a theoretical caveat.
- **Supercell size and `--symprec` are user-chosen, never auto-optimized or auto-converged.** Unlike `stb-mlphonons`'s own `--check-convergence` (affordable there because MACE evaluations are

essentially free) — real DFT makes an automatic multi-supercell-size convergence check far too expensive to offer here (Section 6).

11.4.1.2 Libraries and external dependencies used

- **Phonopy** (`4.3.1` in this environment) is the actual engine behind almost everything in this workflow: displacement generation with symmetry reduction, force-constant construction from `FORCE_SETS`, the dynamical matrix and its diagonalization, band structure/DOS/thermal-property calculations, and acoustic-sum-rule symmetrization. `core/phonon_workflow.py` wraps the pieces shared between Stage 1/Stage 2 (and other tools, such as `stb-ramanModes` and `stb-irModes`); the rest is direct Phonopy API calls in `phonons_create.py/phonons_pos.py`.
- **ASE** (`3.29.0`) supplies the Bravais-lattice classification and q-path generation (`Cell.get_bravais_lattice/Cell.bandpath`, Section 1.3) — a deliberate choice over Phonopy's own `seekpath`-based path finder, since `seekpath` is not a dependency of this suite.
- **spglib** (`2.7.0`) does the actual space-group/symmetry-operation detection, called internally by both Phonopy (Stage 1's displacement reduction, Stage 2's `primitive_matrix= 'auto'` primitive-cell detection) and, elsewhere in the suite, by pymatgen's `SpacegroupAnalyzer` (the same underlying library, different Python wrapper).
- **numpy** / **scipy** — general array math; `scipy.constants` supplies the Planck/Boltzmann constants used for the Debye-temperature conversion (`h * freq_Debye / kB`).
- **matplotlib** (`3.11.0`) — the always-saved PNGs (`phonon_bands.png`, `thermal_properties.png`) and `--view`'s live preview, all via Phonopy's own `plot_band_structure()/plot_total_dos()/plot_projected_dos()` convenience methods, not hand-built figures.
- **gnuplot** (`6.0`, external, not a Python dependency) — only touched when `--save-gnuplot` is given; `stb-phononsPos` writes `.dat + .gplot` file pairs, gnuplot itself is never invoked by the suite.
- **MACE** (via `stb-mlphonons`) — used **only** by this tutorial's own `example_5.4.sh`, as a shortcut to get real force constants for Stage 2's demo cases without a SIESTA wait. The real DFT-based Stage 1/Stage 2 pair this tutorial documents never touches MACE or any ML dependency.

11.4.1.3 3. Stage 1: generating displacements (`stb-phononsCreate`, code `5.4.1`)

What it does. Reads a structure + `calc.fdf`, then writes into `phonon_runs/` (fixed name, no `-o/--output-dir` flag — unlike `strain_runs/elastic_runs/ cohesive_runs`):

- `disp-001/`, `disp-002/`, ... — one single-point SIESTA folder per symmetry-inequivalent displacement, each with its own `calc.fdf` (forced to single-point SCF, Section 3.2) and a copy of the structure file with that one displacement applied.
- `phonopy_disp.yaml` — Phonopy's own metadata (supercell, primitive matrix, the exact displacement vectors used) — Stage 2 needs this file to reconstruct the full force-constant tensor from the finished runs.

Single-point SCF enforcement, live. Every `calc.fdf` — even one written for a full ionic+cell relaxation, like this tutorial's own (`MD.TypeOfRun CG`, `MD.VariableCell true`, `MD.Steps 150`) — is forced to a pure single-point SCF in every `disp-*/` folder, since a finite-difference phonon calculation needs the force at *exactly* the displaced geometry; any further relaxation during the run would contaminate the very forces the whole method depends on. This uses the **same `%include config_extra.fdf` prepend mechanism** as `stb-elasticInputs` (`core/structure_io.py::prepend_include`) — SIESTA's fdf reader is first-occurrence-wins for duplicate labels, so prepending the override at the very top of the generated `calc.fdf` guarantees it wins regardless of what the template itself says later:

```
$ stb-phononsCreate -s structure.fdf -c calc.fdf -p dojo -dim 2 2 2 --no-intro

[1] SINGLE-POINT SCF ENFORCEMENT
------------------------------------------------------------
Calc template (before forcing): MD.TypeOfRun=CG  Steps: MD.Steps=150  MD.VariableCell=true
Forced to single-point SCF via '%include config_extra.fdf'.

Supercell k-grid  : 5 5 5 (auto-suggested, density=0.2 1/Ang)

config_extra.fdf (written into every generated folder):
  # Auto-generated by stb-phononsCreate.
  # Supercell k-grid (auto-suggested, density=0.2 1/Ang) -- takes precedence over --calc's own k-grid,
  # which was tuned for the smaller unit cell.
  kgrid.MonkhorstPack   [5  5  5]

  # Forces a pure single-point SCF (no ionic or cell relaxation) at each displaced
  # supercell's exact geometry, regardless of --calc's own settings.
  MD.TypeOfRun       CG
  MD.Steps           0
  MD.NumCGsteps      0
  MD.VariableCell    false
```

3.3 Symmetry reduction and `--symprec`, live.

```
[3] SYMMETRY REDUCTION
------------------------------------------------------------
Generating supercell [2, 2, 2] with 0.01 Ang displacements ...

>>> Detected space group: F-43m (216)  (symprec=0.01 Ang) <<<
Quantity                    | Value
-----------------------------------------------------------------
Symmetry precision (symprec) | 0.01 Ang
Space group                  | F-43m (216)
Point group                  | -43m
Symmetry operations          | 192
Displacements needed         | 2 (of 96 without symmetry reduction)
Reduction                    | 97.9% fewer SIESTA runs
```

`--symprec` (default `0.01` Ang) matters far more than it might look — the suite-wide fix (several tools used to default to `1e-3`, tighter than pymatgen's real own default of `0.01` and far tighter than typical DFT-relaxation noise) directly targeted this exact structure. Reproduced live, with `--symprec 1e-5` (Phonopy's own raw, unexposed internal default) instead:

```
$ stb-phononsCreate -s structure.fdf -c calc.fdf -p dojo -dim 2 2 2 --symprec 1e-5 --no-intro

[3] SYMMETRY REDUCTION
------------------------------------------------------------
>>> Detected space group: R3m (160)  (symprec=1e-05 Ang) <<<
Symmetry precision (symprec) | 1e-05 Ang
Space group                  | R3m (160)
Point group                  | 3m
Symmetry operations          | 48
Displacements needed         | 4 (of 96 without symmetry reduction)
Reduction                    | 95.8% fewer SIESTA runs
```

The *true* zincblende symmetry (F-43m, cubic) gets misdetected as the lower-symmetry R3m (trigonal) purely from the input structure's own tiny relaxation noise (its lattice vectors carry a real residual of ~8.5e-6 Ang in the position that should be exactly 0 — smaller than 1e-5 but this shows the same failure mode at an even tighter tolerance for contrast) — not just a report-line cosmetic difference: it doubles the real SIESTA calculations needed (2 -> 4) AND changes which physical symmetry operations the whole downstream force-constant reconstruction at Stage 2 relies on.

The supercell k-grid: auto-suggested or explicit. --calc's own k-grid was tuned for the small reference unit cell — using it unmodified on the much larger phonon supercell would badly *over*-sample reciprocal space (wasted computation) for no benefit. stb-phononsCreate either auto-suggests a grid from a target k-point **density** (--kgrid-density, default 0.2 1/Ang — same convention as stb-kgrid/stb-strain), or accepts an explicit --kgrid X Y Z override — written into config_extra.fdf *first*, ahead of the single-point-SCF block, so it's the very first directive SIESTA reads:

```
$ stb-phononsCreate -s structure.fdf -c calc.fdf -p dojo -dim 2 2 2 --kgrid-density 0.15 --no-intro
Supercell k-grid  : 7 7 7 (auto-suggested, density=0.15 1/Ang)

$ stb-phononsCreate -s structure.fdf -c calc.fdf -p dojo -dim 2 2 2 --kgrid 7 7 7 --no-intro
Supercell k-grid  : 7 7 7 (explicit --kgrid)
```

The interactive stb-suite menu (5.4.1) previews the auto-suggested grid directly as the bracketed default at its own k-grid prompt (Supercell k-grid [5 5 5]:), so blank == accept, matching the suite's UX convention rather than a silent "blank means auto" prompt.

Running it both ways. Both CLI and stb-suite -> 5.4.1 produce identical phonon_runs/ output for identical answers (Case 9 of example_5.4.sh demonstrates the interactive path directly).

11.4.1.4 4. Stage 2: analyzing the results (stb-phononsPos, code 5.4.2)

Report structure.

```
[0] RUN METADATA
[1] FORCE EXTRACTION
[2] DYNAMICAL STABILITY
[2b] BAND STRUCTURE        <- only with --bands
[2c] MODE FREEZE           <- only with --freeze-unstable-mode
[3] THERMAL PROPERTIES SUMMARY
[3b] DENSITY OF STATES     <- only with --dos/--pdos
[3c] THERMAL DISPLACEMENTS <- only with --thermal-displacements
[4] SUMMARY & FILES
```

-dir/--directory defaults to phonon_runs, matching Stage 1's own fixed output name — no cd or extra flag needed in the common case. [1] FORCE EXTRACTION transparently handles two different input kinds: real SIESTA .FA force files (built into FORCE_SETS via phonopy-init --siesta -f), or a phonopy_disp.yaml with force constants **already embedded** (an ML-sourced file, e.g. from stb-mlphonons — this tutorial's own Stage 2 shortcut, Section 2) — the latter skips SIESTA-specific extraction entirely.

Dynamical stability, and reading "(within numerical tolerance)". Live, on this tutorial's own real (MACE-computed, Section 2) AlP force constants:

```
$ stb-phononsPos -dir phonon_runs -m 20 20 20 --bands --no-intro
```

```
[2] DYNAMICAL STABILITY
------------------------------------------------------------
Supercell used         : 2 x 2 x 2 (4 displacements, 0.0100 Ang each)
Minimum mesh frequency : 0.2572 THz
No imaginary modes found on the sampled mesh.
[INFO] Building auto-detected high-symmetry q-path for band structure ...

[2b] BAND STRUCTURE
------------------------------------------------------------
Bravais lattice   : face-centred cubic (FCC)
Path              : Γ-X-W-K-Γ-L-U-W-L-K | U-X
Minimum band-path frequency : -0.0000 THz
No imaginary modes found along the band path (within numerical tolerance).
```

This is a **live, real reproduction of the exact "-0.00000 looks unstable but reads stable" scenario** that motivated Section 1.4's fix: the band-path minimum genuinely displays as `-0.0000 THz` (a tiny negative residual, not the sampled mesh's own clean positive `0.2572 THz` result — different q-points, different numerical noise level), and the report now says so explicitly instead of leaving the reader to wonder whether that's an oversight. Both are, physically, the same thing: an acoustic branch correctly going to (numerically) zero at Γ.

Band structure — the zero-line, explained. `phonon_bands.png` (always saved when `--bands` is given) has a thin dotted **blue** horizontal line at `Frequency = 0` running across every panel. Traced directly to Phonopy's own source (`phonopy.phonon.band_structure.BandPlot.decorate()`):

```
ax.plot([spts[0], spts[-1]], [0, 0], linestyle=":", linewidth=0.5, color="b")
```

This is Phonopy's own **built-in reference guide**, drawn on every band -structure plot it produces regardless of what tool calls it — stb-suite adds nothing here. It exists purely so a reader can visually confirm the acoustic branches actually touch zero at Γ; it is not a plotted mode, not a warning, not a sign of instability. (This is exactly the finding from a real investigation into production AlP results, made before this tutorial was written.)

DOS/PDOS — the Debye-fit blue curve, and species projection.

```
$ stb-phononsPos -dir phonon_runs -m 20 20 20 --dos --view --no-intro

[3b] DENSITY OF STATES
------------------------------------------------------------
Total DOS peak    : 10.8712 THz
Debye frequency   : 8.2113 THz (Debye temperature 394.08 K)
```

With `--dos --view`, Phonopy's own `phonon.plot_total_dos()` overlays a **second, blue** curve on top of the real (red) DOS — traced to `phonopy.phonon.dos.plot_total_dos`'s own `color_Debye: str = "blue"` default argument. This is the Debye quadratic fit (`g(omega) = Debye_fit_coef * omega^2`) Section 1.5 introduces, drawn from `omega=0` up to the fitted Debye frequency, then dropping to zero — the **other** "strange blue line", and unlike the band-structure one, this one carries real information: it shows how well the actual DOS matches ideal Debye (free-particle, low-frequency) behavior. The correct reading for a real 2-atom-basis crystal: the fit and the real DOS agree well only in the lowest few THz (the purely acoustic region) and then visibly diverge once optical branches contribute — expected, textbook behavior, not an artifact.

`--pdos` splits the same DOS per chemical species instead of (or alongside) the total:

```
$ stb-phononsPos -dir phonon_runs -m 20 20 20 --pdos --thermal-displacements --no-intro

[3b] DENSITY OF STATES
------------------------------------------------------------
  Al PDOS peak    : 10.8712 THz
  P PDOS peak     : 10.8712 THz
```

4.5 Thermal displacements (ADP / `U_iso`). `--thermal-displacements` computes each atom's isotropic mean-square displacement (the Debye-Waller factor, `U_iso`), the same quantity X-ray crystallography reports as atomic displacement parameters, and writes a `tdispmat.cif` for direct visualization:

```
[3c] THERMAL DISPLACEMENTS
------------------------------------------------------------
ADP CIF (@ 1000.0 K) : tdispmat.cif
  Al U_iso @ 1000.0 K : 0.030916 Ang^2
  P U_iso @ 1000.0 K : 0.028382 Ang^2
```

(Computed on a denser, symmetry-**unreduced** mesh with eigenvectors, reused for `--pdos` too when both are requested together — one extra pass, not two.)

4.6 `--symprec` at Stage 2 — two effects, and a real interaction to respect. `--symprec` here controls **two independent things**: the tolerance Phonopy uses to re-symmetrize the loaded force constants (Section 1.4), and the tolerance ASE uses to classify the Bravais lattice for the q-path (Section 1.3) — both threaded from the same one flag. On this tutorial's own (already very close to ideal, MACE-relaxed) structure, loosening it doesn't visibly change anything:

```
$ stb-phononsPos -dir phonon_runs -m 20 20 20 --bands --symprec 0.5 --no-intro
Symmetry precision: 0.5 Ang (force-constant symmetrization + band-path detection)
Bravais lattice   : face-centred cubic (FCC)
```

A real, found-and-verified interaction: a Stage-2 `--symprec` substantially **tighter** than what Stage 1 used to reduce the original displacement set can fail to reconstruct the force constants at all — verified live against this suite's own `test/4-workflow/ 4-phonons/analysis/test.sh` fixture (a real SIESTA `.FA`-based dataset, Stage 1 run at the default `symprec=0.01`, Stage 2 re-run with `--symprec 1e-5`):

```
[1] FORCE EXTRACTION
------------------------------------------------------------
Force files found : 42 (label 'Sn3O4')
FORCE_SETS       : generated successfully
[INFO] Initializing Phonopy API and loading FORCE_SETS ...
[ERROR] Could not load phonopy data: Input forces are not enough to
calculate force constants,or something wrong (e.g. crystal structure does
not match).
```

The mechanism: force constants for a symmetry-reduced displacement set are reconstructed by applying the *detected* symmetry operations to the computed subset — if Stage 2 detects a **different, lower** symmetry (a tighter tolerance rejects some of the operations a looser one accepted), it can no longer explain all the needed tensor elements from what's actually on disk. **Rule of thumb: loosen, never tighten blindly, if in doubt** — the CLI help text and the interactive prompt both say this explicitly now.

4.7 `--save-gnuplot` / `--view` — both opt-in.

```
$ stb-phononsPos -dir phonon_runs -m 20 20 20 --bands --dos --save-gnuplot --no-intro
Files               : phonon_runs/thermal_properties.png, phonon_runs/thermal_properties.dat,
phonon_runs/phonon_bands.png, phonon_runs/band.yaml, phonon_runs/phonon_plots/ (7 .dat/.gplot pairs)
```

Gnuplot `.dat`/`.gplot` output is opt-in via `--save-gnuplot` (default off), matching the exact convention `aimd_analysis.py`/`effmass.py`/`coop.py` already established — matplotlib PNGs (`phonon_bands.png`, `thermal_properties.png`) are **unaffected**, still always saved either way. `--view` additionally pops up the computed figures live via matplotlib (bands, thermal properties, DOS, PDOS) — a safe no-op under a non-interactive backend like `MPLBACKEND=Agg` (this tutorial's own script uses it for exactly that reason). Note: `--view` does **not** cover `--thermal-displacements` — Phonopy's `plot_thermal_displacements()` needs a *different* internal calculation (`run_thermal_displacements`, isotropic-only) than the ADP-tensor one this tool actually runs (`run_thermal_displacement_matrices`), and there is no matrices-plotting equivalent in Phonopy's own API (Section 6).

The interactive `stb-suite` menu (`5.4.2`) also offers an **`"all"`** shortcut for its additional-analyses list — expands to `bands+dos+pdos+thermal`, deliberately **excluding** mode-freeze (a different kind of action, writing a structure file, not just a plot/report):

```
Select by number (1-5), space/comma-separated, 'all' for 1-4, or blank for none: all
Selected: bands, dos, pdos, thermal
```

Running it both ways. Both CLI and `stb-suite -> 5.4.2` produce the identical report for identical answers (Case 9 of `example_5.4.sh`).

11.4.1.5 Worked example: two real materials, end-to-end (AlP clean, GaAs broken)

This is not illustrative data — both investigations happened this same session, on real production folders the user actually ran (`stb -phononsCreate` + `stb-phononsPos`, real SIESTA, DZP/GGA-PBE, `dojo` pseudopotentials).

5.1 AlP: clean, stable, and a direct confirmation of the `--symprec` fix. Stage 1's real, saved report on this exact structure (`structure.fdf`/ `calc.fdf` in this folder) already shows the correct result at the fixed default: `F-43m (216)`, 192 symmetry operations, 2 of 96 displacements needed (Section 3.3). Stage 2, run on the real SIESTA-computed force constants (not this tutorial's own MACE shortcut):

- **Every one of the 6060 sampled band-path frequencies checked directly** (not just visually from the plot): minimum `+9.95e-08 THz` — positive, essentially exact zero at Γ, **zero negative frequencies anywhere**. Genuinely, cleanly dynamically stable.
- **Heat capacity at 1000 K: 48.97 J/K/mol**, vs. the Dulong-Petit classical limit of `49.88 J/K/mol` (Section 1.5) — 98.2% of the limit, exactly the plateau the theory predicts.
- **Entropy at 0 K: exactly 0** — Third Law respected.
- **DOS peak at 12.89 THz**, matching the flat optical-branch pileup visible in the band structure near X and along L-U-W-L.
- **Lattice constant**: this structure's own relaxed cell implies a conventional cubic lattice constant of `~5.53 Ang`, vs. AlP's real experimental value of `~5.46 Ang` — a `~1.2%` overestimate, exactly the expected direction/magnitude of PBE-GGA's own well-known systematic lattice-constant bias, not a red flag.

Every one of these is an independent physical cross-check, and every one passes — strong, real evidence that both the physics *and* the `--symprec` fix (Section 3.3) are working correctly end-to-end, not just in a synthetic test fixture.

GaAs: a genuine, forensically-traced instability. The real GaAs production folder (same setup, `--symprec 0.05` explicitly at Stage 1) tells a very different story. Stage 2's own saved report:

```
[2] DYNAMICAL STABILITY
------------------------------------------------------------
Minimum mesh frequency : -5.8340 THz
[WARNING] Negative (imaginary) phonon frequencies found ...

[2b] BAND STRUCTURE
------------------------------------------------------------
Minimum band-path frequency : -5.8363 THz
[WARNING] Negative (imaginary) frequency found along the band path ...
```

This is **not** the kind of near-zero noise Section 1.4/4.2 tolerates — it's large, real, and visible directly on the band plot: the 3 branches that should be acoustic (rising from 0 at Γ) instead dive to deeply negative values away from Γ, almost mirroring the optical branches. Both SIESTA displacement runs completed normally (`0_NORMAL_EXIT`, no failed calculation feeding bad data in).

Hypothesis 1, tested and ruled out live: a Stage-1/Stage-2 `--symprec` mismatch (Section 4.6's real, documented risk). Re-running Stage 2 with `--symprec 0.05` (matching Stage 1 exactly, instead of the tighter `0.01` default) barely changes anything (`-5.816 THz` vs. the original `-5.834 THz`) — this specific instability is **not** caused by a tolerance mismatch.

Hypothesis 2, supported by direct evidence: checking the dynamical matrix eigenvalues at Γ directly (not through the CLI, via Phonopy's Python API) shows the acoustic branches are **essentially perfect there** (`~1e-8 THz`, and the acoustic-sum-rule residual before any correction is already tiny, `~1.5e-8`) — the force constants are **not** globally broken; the problem is specific to q-points away from Γ, which depend on how accurately the symmetry-based expansion (Section 1.2/4.6) reconstructed the *full* force-constant tensor from just 2 real displacements. Comparing the two structures' own residual numerical noise (the tiny near-zero term in each `%block LatticeVectors`):

Structure	Residual noise
AlP (clean, stable)	`0.00000852 Ang`
GaAs (unstable)	`0.00395996 Ang`

GaAs carries ~465x more symmetry-breaking noise than AlP in the same structural position — small enough that `spglib` still correctly *identifies* the space group as `F-43m`, but evidently not small enough for the symmetry-operation-based force-constant *expansion* (which applies those operations to the real, slightly-noisy atomic positions) to remain accurate for atom pairs far apart in the supercell — exactly consistent with "correct at Γ, broken away from it."

Conclusion and recommendation: this is a genuine data-quality/ convergence issue with the *input structure and/or supercell size* for this specific material, not a stb-suite bug — the code is working correctly and already flagged the problem with its own `[WARNING]`. Two concrete next steps, neither of which could be verified further without running new DFT (a real limitation of this tutorial, not glossed over): (1) symmetry-refine the structure to the exact detected space group before Stage 1 (e.g. `stb-unitcell --mode refined`) to remove the `0.004 Ang` -scale noise at the source, and/or (2) increase the supercell size (e.g. `3x3x3`) to check whether GaAs's real interatomic force range is simply longer than what a 16-atom supercell can capture — the same size-convergence assumption AlP happened to satisfy but GaAs, evidently, does not.

11.4.1.6 Known, deliberate limitations

- **No non-analytic term correction (NAC/Born effective charges)** — LO-TO splitting at Γ is absent from every band structure/DOS this workflow produces, for any polar material (Section 1.6).
- **Harmonic only** — no anharmonic phonon lifetimes/linewidths, no thermal expansion for this DFT-based pair (`stb-mlphonons --qha` exists for the ML path only, Section 1.6).
- **Supercell size and `--symprec` are user-chosen, never auto-converged** — Section 5.2's GaAs case is the concrete demonstration of why this matters in practice, not just a caveat.
- **A Stage-2 `--symprec` tighter than Stage 1's can break force-constant reconstruction outright** (Section 4.6) — a real, sharp-edged risk, not a gentle accuracy tradeoff.
- **`--view` has no `--thermal-displacements` coverage** — Phonopy's `plot_thermal_displacements()` needs a different internal calculation than the ADP-tensor one this tool actually runs; there's no matrices-plotting equivalent in Phonopy's own API today (Section 4.7).
- **Symmetry-based displacement reduction assumes near-exact input symmetry** — real residual relaxation noise degrades the reconstructed force constants for distant neighbors specifically (Section 5.2), with no automatic detection of when this has actually happened (the mesh/band-path stability check is the only downstream signal).
- **`--kgrid-density`'s 0.2 1/Ang default, `--distance`'s `0.01` Ang default, and the supercell dimensions themselves are never auto-converged** — same "user-chosen, not automated" limitation as `5.1`/`5.2`/`5.3`'s own strain range/mesh/cutoff choices.

11.4.1.7 Step-by-step: running this workflow on your own structure

0. **Prerequisite**: an already-relaxed structure (`structure.fdf`) and the **same `calc.fdf`** used for that relaxation (Section 3.2 — Stage 1 reads it only to know the basis/functional/mesh, then forces single-point SCF itself).
1. **Decide the supercell size** (`-dim`, e.g. `2 2 2`). Bigger captures longer-ranged interatomic forces more accurately but costs more SIESTA runs even after symmetry reduction — Section 5.2's GaAs case is a real example of a size that turned out to be too small for that specific material.
2. **Decide `--symprec`** (default `0.01`, Section 3.3) — start at the default; if the input structure is unusually noisy (freshly relaxed with loose force tolerances, say), consider checking with `stb-symmetry --scan-symprec` first (same tool `5.3`'s own tutorial points to).
3. **Run Stage 1** (`stb-phononsCreate`, code `5.4.1`):
   - CLI: `stb-phononsCreate -s structure.fdf -c calc.fdf -p <bank> -dim <nx> <ny> <nz>`.
   - Interactive: `stb-suite` -> `5.4.1`.
   - Writes `phonon_runs/disp-*/` (Section 3.1).
4. **Run SIESTA yourself in every `disp-*/` folder.** Stage 1 never runs SIESTA.
5. **Run Stage 2** (`stb-phononsPos`, code `5.4.2`) once every displacement finishes:
   - CLI: `stb-phononsPos -dir phonon_runs -m <mx> <my> <mz> --bands --dos --pdos --thermal-displacements` (mix and match the extra-analysis flags you want).
   - Interactive: `stb-suite` -> `5.4.2` (the `"all"` shortcut picks bands+dos+pdos+thermal in one go, Section 4.7).
6. **Read `[2]`/`[2b]` first** (Section 4.2) — if you see a `[WARNING]` that isn't explained by "(within numerical tolerance)", the geometry may genuinely not be a true energy minimum, or (Section 5.2) the supercell/symmetry assumptions may not hold for this material — don't trust `[3]`'s thermal

properties until this is resolved (Phonopy itself silently drops imaginary modes from those sums, so they'd be wrong, not just absent).

7. **(Optional) persist or view**: `--save-report` writes `phonon_properties.txt`; `--save-gnuplot` writes `.dat`/`.gplot` pairs (off by default, Section 4.7); `--view` pops up the computed plots live.

11.4.1.8 Files in this folder

File	What it is
`structure.fdf`	Real, relaxed AlP, zincblende, 2 atoms/cell — the material Section 5.1's real numbers are for
`calc.fdf`	The matching real calc template (DZP, PBE-GGA, 320 Ry, 11x11x11 k-grid)
`example_5.4.sh`	This walkthrough's runnable script (both stages, 9 cases)

GaAs's own structure (Section 5.2) is not checked into this folder — that investigation used the real production data directly, not a tutorial fixture; the numbers quoted above are real and independently reproducible against that structure, just not shipped here.

11.4.1.9 Running the script

```
bash example_5.4.sh
```

11.4.1.10 What's next

- **`5.1-strain`** / **`5.2-elastic`** / **`5.3-cohesive`** — the other mechanical/energetic questions about the same kind of already-relaxed structure this workflow starts from (deformation response, stiffness tensor, binding energy) — phonons answer a genuinely different one (vibrational stability and thermal behavior).
- **`stb-mlphonons`** (ML Simulations, code `7.3`) is this workflow's own ML-preview twin — same physics, MACE-MP-0 instead of SIESTA, no SIESTA wait, plus features this DFT-based pair doesn't have (`--qha` thermal expansion, `--check-convergence` supercell-size scanning, foundation -model comparison) — exactly the shortcut this tutorial's own script used to get real Stage 2 data without waiting on SIESTA.

11.4.2 Implementation details

Finite displacements

In the harmonic approximation the force constants are $\Phi_{\alpha\beta}(i,j) = \partial^2 E/\partial u_{i\alpha}\partial u_{j\beta} \approx -\Delta F_{j\beta}/\Delta u_{i\alpha}$: each atom i is displaced by a small Δu in a supercell and the forces on all atoms j are recorded. Phonopy [11, 12] uses the space group to keep only the symmetrically distinct displacements, builds Φ, and diagonalizes the dynamical matrix $D_{\alpha\beta}(ij,\mathbf{q}) = \sum_{\mathbf{R}} \Phi_{\alpha\beta}(i0, j\mathbf{R})\, e^{i\mathbf{q}\cdot\mathbf{R}}/\sqrt{m_i m_j}$ to obtain $\omega_\nu(\mathbf{q})$ [69, 72]. Thermal properties follow from the phonon DOS in the harmonic approximation.

Stage 1 (`stb-phononsCreate`).. From `-s` (default `structure.fdf`) and `-c` (default `calc.fdf`) it builds the supercell `-dim` (default $2 \times 2 \times 2$), generates the displacements of `-d`/`--distance` (default 0.01 Å) with phonopy at `--symprec` (0.01, a tolerance loose enough for relaxed structures – phonopy's own 10^{-5} misses symmetries), and writes:

- `disp-NNN/` – one displaced supercell per symmetry-distinct displacement, with the template, the pseudopotentials and a `config_extra.fdf` that forces a single-point SCF and a k-grid for the *supercell* (`--kgrid`, or computed from `--kgrid-density`, default 0.2 Å^{-1});
- `perfect_supercell/` – the undisplaced supercell with the same settings (unless `--no-perfect-supercell`); see below;
- `phonopy_disp.yaml` – phonopy's record of the supercell and displacements.

`--ml-prerelax` first relaxes the unit cell with MACE (positions, `--ml-fmax`), which reduces the residual forces of a structure that was not fully relaxed; it is no substitute for a converged SIESTA relaxation.

Residual forces.. A displacement's force difference is meaningful only if the undisplaced supercell feels no force. In practice SIESTA leaves small residual forces (finite relaxation tolerance, grid noise), and summed over a whole sublattice they distort the force constants: in a test on graphene relaxed to 0.004 eV/Å in a 5×5 supercell, the Γ-point E_{2g} mode came out at 31.9 instead of 41.0 THz. Stage 2 therefore subtracts the forces of `perfect_supercell/` from every displacement (after removing their mean, as phonopy does for each set), rewrites `FORCE_SETS` (the original is kept as `FORCE_SETS.raw`) and reports the largest residual. If the folder is missing or unfinished it warns and proceeds without the correction.

Stage 2 (`stb-phononsPos`).. It finds the `SystemLabel`, collects `disp-*/<label>.FA` into `FORCE_SETS`, loads the force constants (symmetrized, acoustic sum rule enforced) and computes on request: the band structure along a dimension-aware high-symmetry path (`--bands`, `--band-points` per segment) with group velocities, the DOS on the `-m`/`--mesh` (default 20^3) and its species projection (`--dos`, `--pdos`), thermal properties from `--tmin` to `--tmax` in steps of `--tstep` (free energy, entropy, heat capacity), and thermal displacement (Debye–Waller) tensors (`--thermal-displacements`, written to `tdispmat.cif`, with U_{iso} per species plotted against temperature). Dynamical stability is judged with a tolerance: frequencies above -0.01 THz count as zero within numerical noise; lower ones are reported as imaginary modes. With an instability, `--freeze-unstable-mode` writes `frozen_mode.fdf`, the structure modulated along the soft mode with a largest atomic displacement of `--freeze-amplitude` (default 0.05 Å, measured with the minimum-image convention), as a starting point for a lower-symmetry relaxation.

Supercell size and accuracy.. The supercell must be large enough for the force constants to decay within it (check convergence with two sizes); low-frequency acoustic branches near Γ are the most sensitive. Polar materials additionally need the non-analytic (LO–TO) correction with Born charges and the dielectric tensor, which this workflow does not apply; see the IR workflow (**5.12**) for Born charges.

11.4.3 Command-line reference

11.4.3.1 `stb-phononsCreate`

Automate SIESTA phonon displacement folders with Phonopy.

Synopsis.

```
usage: stb-phononsCreate [-h] [-dim DIM DIM DIM] [-d DISTANCE] [-s STRUCTURE]
```

```
[-c CALC] [-p PSEUDO_DIR] [--vacuum-gap VACUUM_GAP]
[--symprec SYMPREC] [--kgrid KGRID KGRID KGRID]
[--kgrid-density KGRID_DENSITY] [--ml-prerelax]
[--ml-model {small,medium,large}] [--ml-device {cpu,cuda}]
[--ml-fmax ML_FMAX] [--no-perfect-supercell] [--save-report]
[-v] [--no-intro]
```

Options

`-dim DIM DIM DIM`

Supercell dimensions (default: 2 2 2) type: int

`-d, --distance DISTANCE`

Displacement distance in Angstroms (default: 0.01) type: float

`-s, --structure STRUCTURE`

Input structure file containing the unit cell (default: `structure.fdf`)

`-c, --calc CALC`

Input calculation parameters file (default: `calc.fdf`)

`-p, --pseudo-dir PSEUDO_DIR`

Pseudopotentials source: a bundled bank (dojo, virtual_vault) or a folder path (default: current directory).

`--vacuum-gap VACUUM_GAP`

Minimum gap (Ang) along an axis to consider it vacuum-padded, for the supercell-dimension advisory (default: 10.0) type: float

`--symprec SYMPREC`

Symmetry-detection tolerance (Ang) Phonopy uses internally to reduce how many displacements are actually needed (default: 0.01, pymatgen's own default – matches the rest of the suite, and deliberately NOT Phonopy's own raw default of 1e-5, which is far too tight for a real DFT-relaxed structure and can misdetect the true space group: verified live on a real relaxed AlP zincblende structure, 1e-5 reports the wrong R3m while 0.01 correctly reports F-43m). Loosen further (e.g. 0.02-0.05) for a structure relaxed with a looser force tolerance. type: float

`--kgrid KGRID KGRID KGRID`

Explicit Monkhorst-Pack k-grid 'X Y Z' for the SUPERCELL's own single-point SCF (e.g. `--kgrid 2 2 2`). Overrides the automatic `--kgrid-density` suggestion. Written into `config_extra.fdf` (see [1]) as the first directive, so it takes precedence over `--calc`'s own k-grid, which was tuned for the smaller unit cell. type: int

`--kgrid-density KGRID_DENSITY`

K-point density (1/Ang) used to auto-suggest the supercell's own k-grid when `--kgrid` isn't given (default: 0.2, same convention as `stb-kgrid/stb-strain`). type: float

`--ml-prerelax`

Optional pre-flight check (opt-in): compute the residual force on the input structure with the MACE-MP-0 foundation potential, and if it's large, run a quick ML relax for reference. An unrelaxed reference structure is a common cause of spurious imaginary phonon modes. Diagnostic only – this run still generates displacements from the original `--structure` file unchanged; the ML-relaxed structure is written out separately for you to review/use in a follow-up run. switch, off by default

`--ml-model {small,medium,large}`

MACE-MP-0 model size for `--ml-prerelax` (default: medium). choices: small, medium, large

`--ml-device {cpu,cuda}`

Device for `--ml-prerelax` (default: cpu). choices: cpu, cuda

`--ml-fmax ML_FMAX`

Force threshold (eV/Ang) for `--ml-prerelax`: above this, a warning is shown and a relax is offered (default:

0.05). type: float

`--no-perfect-supercell`

Do not write perfect_supercell/ (the undisplaced supercell, one more SIESTA run). It is written by default: `stb-phononsPos` subtracts its forces – the reference's residual forces – from every displacement's, which a small relaxation residual otherwise turns into errors of several THz at Gamma. switch

`--save-report`

Also persist the report to <output-dir>/phonon_prep_properties.txt. Off by default. switch, off by default

Like every STB command it also accepts `-h`/`--help`, `-v`/`--version` and `--no-intro` (Section 3.3).

From the built-in help.

```
Example usage:
  stb_phonon --structure structure.fdf --calc calc.fdf --dim 2 2 2
  stb_phonon --pseudo-dir ~/pseudos --distance 0.015
```

11.4.3.2 `stb-phononsPos`

Post-processing for SIESTA phonon calculations using Phonopy.

Synopsis.

```
usage: stb-phononsPos [-h] [-dir DIRECTORY] [-l LABEL] [-m MESH MESH MESH]
                      [--tmin TMIN] [--tmax TMAX] [--tstep TSTEP]
                      [--plot-dir PLOT_DIR] [--bands] [--band-points BAND_POINTS]
                      [--vacuum-gap VACUUM_GAP] [--dos] [--pdos]
                      [--thermal-displacements] [--freeze-unstable-mode]
                      [--freeze-amplitude FREEZE_AMPLITUDE] [--symprec SYMPREC]
                      [--save-report] [--save-gnuplot] [--view] [-v] [--no-intro]
```

Options

`-dir, --directory DIRECTORY`

Directory containing the displacement folders (default: phonon_runs)

`-l, --label LABEL`

SystemLabel used in `calc.fdf` (default: auto-detected from the copied calc.fdf/structure.fdf, falling back to 'siesta')

`-m, --mesh MESH MESH MESH`

Q-point mesh for thermal properties (default: 20 20 20) type: int

`--tmin TMIN`

Minimum temperature in K (default: 0) type: float

`--tmax TMAX`

Maximum temperature in K (default: 1000) type: float

`--tstep TSTEP`

Temperature step in K (default: 10) type: float

`--plot-dir PLOT_DIR`

Directory (inside the phonon directory) for gnuplot .dat/.gplot output (default: phonon_plots)

`--bands`
: Compute and plot the phonon band structure along an auto-detected high-symmetry path switch, off by default

`--band-points BAND_POINTS`
: Number of q-points per band-structure path segment (default: 101) type: int

`--vacuum-gap VACUUM_GAP`
: Minimum gap (Ang) along an axis to consider it vacuum-padded, for `--bands`' periodicity detection (default: 10.0) type: float

`--dos`
: Compute and plot the total phonon density of states switch, off by default

`--pdos`
: Compute and plot the phonon density of states projected per chemical species switch, off by default

`--thermal-displacements`
: Compute per-atom thermal displacement (Debye-Waller) tensors and write a `tdispmat.cif` switch, off by default

`--freeze-unstable-mode`
: If the mesh has an imaginary (unstable) mode, export the structure frozen along that mode's displacement pattern as `frozen_mode.fdf` – a starting point for a real relaxation switch, off by default

`--freeze-amplitude FREEZE_AMPLITUDE`
: Target maximum atomic displacement (Ang) for `--freeze-unstable-mode` (default: 0.05) type: float

`--symprec SYMPREC`
: Symmetry-detection tolerance (Ang), used for force-constant symmetrization and the `--bands` high-symmetry path (default: 0.01, pymatgen's own default – matches the rest of the suite, and deliberately NOT Phonopy's/ASE's own raw defaults (1e-5 / 2e-4), which are far tighter than typical DFT-relaxation noise). A much TIGHTER value than what `stb-phononsCreate` used to reduce the displacements (its own `--symprec`, also default 0.01) can fail to reconstruct force constants – loosen, don't tighten, if in doubt. type: float

`--save-report`
: Also persist the report to <directory>/phonon_properties.txt. Off by default. switch, off by default

`--save-gnuplot`
: Also write a .dat + .gplot pair for every computed quantity (thermal properties, bands, group velocity, DOS, PDOS, thermal displacements). Off by default – this tool previously wrote these unconditionally on every run. switch, off by default

`--view`
: Show an interactive matplotlib preview of the computed plots (thermal properties, bands, DOS, PDOS) before finishing. Off by default. switch, off by default

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Example usage:
  phonons_post -dir phonon_runs -l siesta -m 20 20 20
  phonons_post --tmin 0 --tmax 800 --tstep 5
```

11.5 Convergence Tests

At a glance

Menu code: 5.5 (a multi-stage workflow)

Code	Stage	Command
5.5.1	Stage 1 - Prep	`stb-convergence`
5.5.2	Stage 2 - Analysis	`stb-convergenceAnalysis`

Guided example: `examples/5.5-convergence/`

Summary: Sweep Mesh.CutOff, k-grid density, or PAO.EnergyShift and check energy + relaxed-structure convergence.

11.5.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/5.5-convergence/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

Every SIESTA calculation depends on three numerical "dials" that have no universally correct setting: how fine the real-space integration grid is (`Mesh.CutOff`), how spatially complete the localized basis is (`PAO.EnergyShift`), and how densely the Brillouin zone is sampled (the k-grid). This workflow has **2 stages**: **Stage 1** (`stb-convergence`, code `5.5.1`) sweeps one or more of those three parameters, writing one **fully relaxed** (positions + cell) SIESTA folder per swept value. **Stage 2** (`stb-convergenceAnalysis`, code `5.5.2`) reads the finished runs back and reports, per parameter, the value at which **both** the total energy **and** the relaxed geometry itself stop changing beyond a chosen tolerance — not just one or the other.

Both stages live in this one folder and this one tutorial — not one folder per tool — for the same reason `5.1-strain/5.2-elastic/5.3-cohesive` do: Stage 1's output only exists to feed Stage 2, and Stage 2 only exists to interpret Stage 1's output. This workflow is also the natural **first** stop before any of those others: its entire purpose is making sure the `calc.fdf` you're about to use for a real strain/elastic/cohesive/phonon calculation isn't quietly wrong in a way that has nothing to do with physics.

11.5.1.1 Theory

What "convergence testing" means, and why every calculation needs one first. SIESTA (like every real DFT code) never solves the exact many-electron problem — it approximates it using a **finite basis** of localized atomic orbitals, a **finite real-space grid** for the electron density, and a **finite sampling** of the Brillouin zone. Each of these approximations has a tunable "fineness" parameter, and — critically — **the computed total energy, forces, and relaxed geometry all depend on the chosen fineness, not just on the physics you're trying to compute.** A calculation run at a too-coarse setting doesn't fail outright; it silently returns a number that looks like an answer but contains numerical error that can dwarf the real physical effect you're after (e.g. a strain-energy difference of a few meV, Section 4.1/4.2's whole subject). **Convergence testing** is the systematic fix: sweep a parameter, watch where the answer stops changing beyond some tolerance, and use *that* value — not a "reasonable-sounding" textbook default — for every production calculation downstream.

The three dials this tool turns, and the physics behind each.

Parameter	fdf keyword	Unit	What it actually controls	Direction that improves accuracy
`meshcutoff`	`Mesh.CutOff`	Ry	Fineness of the real-space grid SIESTA uses to compute the Hartree/XC energy and potential — the LCAO analogue of a plane-wave cutoff	**larger**
`energyshift`	`PAO.EnergyShift`	Ry	The energy increase, relative to the free orbital, at which each basis orbital's radial tail is truncated — smaller means the orbital is allowed to extend farther out, making the basis more spatially complete	**smaller**
`kgrid`	`kgrid.MonkhorstPack` (density-driven)	1/Ang	Brillouin-zone sampling density — matters most for metals/small-gap systems where bands vary rapidly near the Fermi level, far less for large-gap insulators or molecules in a big vacuum box	**smaller** density value (= a denser grid)

`--parameter` selects one, several, or `all` three; each is swept **independently** in its own subfolder (Section 1.6). `kgrid` is swept as a **density**, in 1/Ang — the same convention `stb-kgrid` itself uses — not as a raw `N x N x N` grid directly; Stage 1 converts it via `core/kspace.compute_monkhorts` (Section 2.5 shows this live).

1.3 Why every sweep point is a *full relaxation*, not a fixed-geometry single point — the eggbox effect. A finite real-space grid is not perfectly translationally invariant: moving an atom by a fraction of a grid spacing can change the computed energy by a small, spurious amount, purely because of where the atom sits *relative to the grid points* — SIESTA's own documentation calls this the **"eggbox effect."** This matters enormously for convergence testing specifically: if you compared energies at a single **fixed, arbitrary geometry** across a `Mesh.CutOff` sweep, two different settings could both look self-consistent and "energy-converged" simply because they're evaluating the same (possibly not-yet-relaxed) geometry — while the geometry SIESTA would actually *relax to* at each setting is still quietly drifting. A fixed-point comparison cannot see that drift at all.

This is exactly why Stage 1 forces `MD.TypeOfRun CG` + `MD.Steps` + `MD.VariableCell true` into **every** generated folder, regardless of what your own `--calc` template says (verified live in Section 2.2's captured `[3] RELAXATION & PARAMETER ENFORCEMENT` table) — full positions-and-cell relaxation at every single swept value. It's more expensive than a single-point sweep, but it's the only honest way to test this, and it sets up the entire reason Stage 2 needs *two* separate convergence criteria (Section 1.4).

Two independent convergence criteria: energy AND relaxed structure. Because Section 1.3's eggbox effect can make total energy *look* converged at a geometry that itself hasn't settled, Stage 2 tracks **two criteria separately, per parameter**:

- **Energy**: `|Delta E/atom|` between consecutive swept values, against `--tolerance` (default `0.01` eV/atom).
- **Structure**: `|Delta V/V|`, the percent change in the *relaxed* cell volume between consecutive values, against `--volume-tolerance` (default `0.5` %).

Both are reported side by side in `[2.N]`'s table, and `[4]`'s summary shows exactly where each one first stabilizes. Section 4's worked example shows a real, code-verified case where **they disagree** — energy looking converged two sweep steps before the relaxed geometry actually does. This is the entire reason Stage 1 always does a full relaxation (Section 1.3) instead of a cheaper single-point sweep: a single-criterion, fixed-geometry tool structurally *cannot* catch this.

1.5 Why the plateau scan checks the *entire remaining tail*, not just the next point. A single small delta between two adjacent swept values could be a coincidence — two points landing close together by chance, or ordinary SCF noise — not a genuine plateau. Stage 2's internal scan (`_scan_plateau`) only declares a value "converged" if **every later point in the sweep** also stays under tolerance (and has confirmed SCF convergence), not just the very next one. This avoids a false "converged" reading from one lucky adjacent pair while the underlying trend hasn't actually settled yet — the same discipline `5.3-cohesive`'s own BSSE-cutoff convergence check applies (one before/after point is a hint, not proof; a scan is proof).

Independent, not joint, sweeps — a real limitation worth understanding before it bites. Each of the three parameters is swept **alone**, holding the other two fixed at whatever your `--calc` template already says. The tool never performs a joint/coupled optimization across all three at once. This matters in practice whenever two parameters are physically coupled: for example, a very coarse k-grid in a metallic system can itself introduce enough energy noise that a `Mesh.CutOff` scan run *at that same coarse k-grid* looks "converged" earlier than it really would at a denser, properly-converged grid — the k-grid-driven noise floor can simply be larger than the mesh-cutoff-driven changes you're trying to measure. **Practical recommendation**: converge `kgrid` first, then re-verify `meshcutoff/energyshift` with that denser k-grid now held fixed in your `--calc` template, rather than trusting one pass's independent sweeps in isolation for a system where this coupling is likely to matter (metals, small cells).

Data-quality diagnostics are not a convergence criterion. `--force-tolerance` (default `0.05` eV/Ang) and the `[2.N]` table's **Quality** column (`SCF?`, `F>tol`, `steps`) flag a *specific run's own* numerical health — did SCF actually confirm converged, is the residual force still large (meaning the geometry might not really be at a relaxed minimum), did the relaxation exhaust its declared `MD.Steps` budget (meaning it may have been cut off before finishing). This is **completely separate machinery** from the tolerance-based plateau scan of Section 1.4/1.5 — a flagged row never blocks or alters the convergence scan itself, it's advisory only. Worth stating plainly because the two are easy to conflate: `--tolerance`/`--volume-tolerance` ask "has the *trend* settled," `--force-tolerance` asks "is *this one run* actually trustworthy in the first place." Section 3.5 demonstrates a real flagged row.

11.5.1.2 2. Stage 1: generating the calculations (`stb-convergence`, code `5.5.1`)

What it does. Reads one structure and one `calc.fdf` template, then writes, under one wrapping `-o`/`--output-dir` (default `convergence_runs/`):

```
<output-dir>/<parameter>/convergence_<parameter>_<value>/
    structure.fdf       (copied verbatim from -s)
    calc.fdf             (your -c template, with '%include config_extra.fdf' prepended)
    config_extra.fdf      (forces this folder's swept value + full relaxation)
    <pseudopotential files>   (only if -p/--pseudo-dir given)
```

One such subtree per selected parameter (Section 2.4), each an **independent** sweep (Section 1.6).

2.2 What gets forced into every folder, and why `stb-convergence` needs a real `calc.fdf`. Unlike `5.3-cohesive` (which builds its own internal single-point template), `stb-convergence` takes your **own** `-c`/`--calc` template and reuses it almost unchanged — because the entire point is testing the numerical settings of the calculation you're actually about to run, not some fixed internal reference. Only `config_extra.fdf`, prepended via `%include` (fdf is first-occurrence-wins, so this always overrides whatever your template says), forces the swept parameter's value plus full relaxation. Captured live, real tool output (`Mesh.CutOff = 250 Ry` case):

```
# Auto-generated by stb-convergence -- forces this folder's own swept parameter
# value plus full relaxation (positions + cell), regardless of --calc's own settings.
Mesh.CutOff           250.0000  Ry

MD.TypeOfRun       CG
MD.Steps           100
MD.VariableCell    true
```

`--relax-steps` (default `100`) controls the forced `MD.Steps`.

Single-parameter default sweep, live.

```
$ stb-convergence -s structure.fdf -c calc.fdf -p dojo --parameter meshcutoff --no-intro

[2] PARAMETER SELECTION
------------------------------------------------------------
Parameter  | Range source      | Sweep range              | Convergence direction
-------------------------------------------------------------------------------
meshcutoff | suggested default | 100 to 400, step 50 (Ry) | larger = more converged

[5] GENERATED CONVERGENCE FOLDERS
------------------------------------------------------------
Under 'convergence_runs/<parameter>/':
Folder                          | Parameter  | Value       | Files
-------------------------------------------------------------------------------------------------------
--
convergence_meshcutoff_100.0000 | meshcutoff | 100.0000 Ry | structure.fdf, calc.fdf,
config_extra.fdf
convergence_meshcutoff_150.0000 | meshcutoff | 150.0000 Ry | structure.fdf, calc.fdf,
config_extra.fdf
convergence_meshcutoff_200.0000 | meshcutoff | 200.0000 Ry | structure.fdf, calc.fdf,
config_extra.fdf
convergence_meshcutoff_250.0000 | meshcutoff | 250.0000 Ry | structure.fdf, calc.fdf,
config_extra.fdf
convergence_meshcutoff_300.0000 | meshcutoff | 300.0000 Ry | structure.fdf, calc.fdf,
config_extra.fdf
convergence_meshcutoff_350.0000 | meshcutoff | 350.0000 Ry | structure.fdf, calc.fdf,
config_extra.fdf
convergence_meshcutoff_400.0000 | meshcutoff | 400.0000 Ry | structure.fdf, calc.fdf,
config_extra.fdf
```

Real, verbatim output from this folder's own `structure.fdf`/`calc.fdf` (bulk silicon, diamond cubic, DZP/GGA-PBE — Section 7). Case 1 of `example_5.5.sh` reproduces this exactly.

2.4 `--parameter all`: three independent sweeps in one call.

```
$ stb-convergence -s structure.fdf -c calc.fdf -p dojo --parameter all --no-intro

[2] PARAMETER SELECTION
------------------------------------------------------------
Parameter   | Range source      | Sweep range                   | Convergence direction
-----------------------------------------------------------------------------------------
meshcutoff  | suggested default | 100 to 400, step 50 (Ry)      | larger = more converged
energyshift | suggested default | 0.001 to 0.05, step 0.01 (Ry) | smaller = more converged
kgrid       | suggested default | 0.05 to 0.3, step 0.05 (1/Ang) | smaller = more converged
```

producing 3 independent subtrees on disk:

```
convergence_runs/meshcutoff/convergence_meshcutoff_{100,150,200,250,300,350,400}.0000/
convergence_runs/energyshift/convergence_energyshift_{0.0010,0.0110,0.0210,0.0310,0.0410,0.0510}/
convergence_runs/kgrid/convergence_kgrid_{0.0500,0.1000,0.1500,0.2000,0.2500,0.3000}/
```

Case 2 of `example_5.5.sh` runs this live.

Custom ranges, and the k-grid density convention. `--meshcutoff-range`/`--energyshift-range`/`--kgrid-range MIN MAX STEP` override the suggested default range for that parameter only (harmless to pass for a parameter not selected in `--parameter` — Section 2.6 shows what happens if you do, though: it's actually an error, not a silent no-op). Remember `kgrid` sweeps a **density** (1/Ang), not a raw grid — real, verbatim `config_extra.fdf` at density `0.05` on this folder's bulk-Si structure:

```
# Auto-generated by stb-convergence -- forces this folder's own swept parameter
# value plus full relaxation (positions + cell), regardless of --calc's own settings.
kgrid.MonkhorstPack   [24  24  24]

MD.TypeOfRun       CG
```

`0.05` (1/Ang, a *dense* sampling — remember: smaller density number = denser grid, Section 1.2) resolved to a real `24x24x24` Monkhorst-Pack grid for this 2-atom, `5.43` Ang cubic cell, via the exact same `core/kspace.compute_monkhorts` machinery `stb-kgrid` itself uses.

2.6 Validation: what `stb-convergence` correctly refuses. Real, verbatim error messages (all exit with a non-zero status, same `argparse`-style convention as every other tool in the suite):

```
$ stb-convergence -s structure.fdf -c calc.fdf --parameter bogus --no-intro
stb-convergence: error: --parameter: unrecognized value 'bogus' (choose from meshcutoff, energyshift,
kgrid, or 'all').

$ stb-convergence -s structure.fdf -c calc.fdf --parameter meshcutoff --energyshift-range 0.01 0.05
0.01 --no-intro
stb-convergence: error: --energyshift-range was given but 'energyshift' is not in --parameter.

$ stb-convergence -s structure.fdf -c calc.fdf --parameter meshcutoff --meshcutoff-range 100 400 0
--no-intro
stb-convergence: error: --meshcutoff-range: step must be > 0.
```

Case 3 of `example_5.5.sh` reproduces all three.

Output layout, and running it both ways.

Path	What it is
`convergence_runs/<parameter>/convergence_<parameter>_<value>/`	One fully-relaxed SIESTA folder per swept value
`convergence_stage1.txt`	Persisted report, only with `--save-report`

Both CLI and interactive menu (`stb-suite` -> `5.5.1`) produce **byte -identical** output for identical answers — verified directly (`diff -rq`, real check, not assumed) and reproduced by Case 4 of `example_5.5.sh`.

11.5.1.3 3. Stage 2: analyzing the results (`stb-convergenceAnalysis`, code `5.5.2`)

Report structure.

```
[0] RUN METADATA
[1] DISCOVERED PARAMETER SWEEPS
[2.N] <PARAMETER> SWEEP        <- one per discovered parameter, e.g. [2.1] MESHCUTOFF SWEEP
[3] OUTPUT FILES
[4] SUMMARY & NEXT STEPS
[5] APPLY                       <- only printed with --apply
```

3.2 `--dir` auto-detects both folder layouts — including a real, verified quirk in Stage 1's own message. `--dir` (default `convergence_runs`, matching Stage 1's own `-o/--output-dir` default) auto-detects **both** the nested layout (`<dir>/<parameter>/convergence_<parameter>_<value>/`, what Stage 1 actually writes) and a flat layout (`convergence_<parameter>_<value>/` directly under `<dir>`, e.g. this folder's own test fixtures). **Worth flagging explicitly**: Stage 1's own `[6] SUMMARY & NEXT STEPS` prints a `[NOTE]` claiming *"stb-convergenceAnalysis (Stage 2) doesn't yet support this per-parameter layout -- point --dir at one parameter's own subfolder"* — but running `stb-convergenceAnalysis --dir convergence_runs` directly against Stage 1's own nested output (verified live, not assumed) discovers the parameter correctly regardless:

```
[1] DISCOVERED PARAMETER SWEEPS
------------------------------------------------------------
Parameter  | Folders found
--------------------------
meshcutoff | 7
```

This is a **stale message left in Stage 1's own report text**, not a real functional limitation — Stage 2's actual discovery code supports both layouts already. Point `--dir` at whichever level is convenient; both work.

3.3 Reading the `[2.N]` table. Real, verbatim output (Section 4's worked example):

```
[2.1] MESHCUTOFF SWEEP
------------------------------------------------------------
Folders found: 4 (skipped: 0 unreadable/incomplete)  |  Increasing meshcutoff improves accuracy: True
Value    | E/atom(eV)  | |Delta E/atom| | Volume(Ang^3) | |Delta V/V|(%) | Quality
-------------------------------------------------------------------------------
150.0000 | -250.450000 | --             | 153.1304      | --             | OK
200.0000 | -250.425000 | 0.025000       | 159.2201      | 3.9768         | OK
250.0000 | -250.422500 | 0.002500       | 161.8786      | 1.6697         | OK
300.0000 | -250.422000 | 0.000500       | 161.9677      | 0.0551         | OK
```

`E/atom` and `Volume` are absolute values at that swept value; `|Delta ...|` columns are the **change from the previous row** (Section 1.4/1.5's plateau-scan inputs) — blank (`--`) on the first row since there's no previous point to compare against.

How the "recommended value" is chosen. `[4]`'s summary table reports **both** criteria's own converged value side by side, plus a single **Recommended** value — whichever of the two is more accurate-ward (i.e. the *tighter* of the two), so both criteria end up satisfied simultaneously:

```
[4] SUMMARY & NEXT STEPS
------------------------------------------------------------
Parameter  | Energy-converged | Structure-converged | Recommended
------------------------------------------------------------
meshcutoff | 250.0000         | 300.0000            | 300.0000
```

When the two disagree, an explicit [NOTE] is printed (Section 4 shows it verbatim) — this is exactly Section 1.4's "energy can look converged before the relaxed geometry does" scenario, made concrete.

3.5 The Quality column, live: SCF?, F>tol, steps. Real, verbatim output — one deliberately poor 50 Ry run added to the same sweep (Section 1.7):

```
Value    | E/atom(eV)  | |Delta E/atom| | Volume(Ang^3) | |Delta V/V|(%) | Quality
-------------------------------------------------------------------------------------
50.0000  | -249.100000 | --             | 132.6510      | --             | SCF?,F>tol
150.0000 | -250.450000 | 1.350000       | 153.1304      | 15.4385        | OK
...
[WARNING] 1/5 run(s) flagged (SCF? = SCF convergence not confirmed, F>tol = residual force above
--force-tolerance 0.05 eV/Ang, steps = relaxation used its full declared step budget) -- their
geometry may not be a genuine relaxed minimum.
```

SCF? means the SCF cycle never printed a confirmed-converged line; F>tol means the residual force is still above --force-tolerance (default 0.05 eV/Ang) — both flag that 50.0000's own row is untrustworthy on its own terms, entirely separately from whether it fits the overall convergence trend (Section 1.7). Case 6 of example_5.5.sh reproduces this.

3.6 --apply: writing the recommended value back into your own calc.fdf.

```
$ stb-convergenceAnalysis --dir . --no-intro --apply calc.fdf

[5] APPLY
------------------------------------------------------------
[Applied] meshcutoff = 300.0000 (300.0000 Ry) -> calc.fdf
```

verified: the target file's own Mesh.CutOff line is rewritten in place (Mesh.CutOff 300.0000 Ry) — same tag-substitution machinery Stage 1 itself uses internally, applied here to *your* production calc.fdf instead of a generated one. A parameter with no recommended value (e.g. still not converged anywhere in the scanned range) is skipped, not guessed.

3.7 The plots: --save-gnuplot / --view. --save-gnuplot writes, per discovered parameter, a .dat + companion .gplot pair (same convention as elastic_analysis.py/strain_analysis.py — see the manual's developer reference): <parameter>_convergence.dat / <parameter>_convergence.gplot, a 2-panel pdfcairo plot (energy per atom on top, relaxed cell volume on the bottom — the exact two quantities Section 1.4's dual criterion tracks). --view additionally pops the same 2 panels up interactively via matplotlib (a no-op under a non-interactive backend, e.g. this example's own MPLBACKEND=Agg). Neither is written unless requested.

Running it both ways. Both CLI and stb-suite -> 5.5.2 produce the identical report for identical answers.

11.5.1.4 Worked example: energy convergence lies about geometry convergence

Bulk silicon (diamond cubic, this folder's own structure.fdf/calc.fdf — DZP basis, GGA-PBE), a Mesh.CutOff sweep at 150/200/250/300 Ry. **An honest note on the data**: these are small, hand-built calc.out files (the same convention this suite's own test fixtures use, test/5-workflow/5-convergence/analysis/) with realistic, physically -plausible bulk-silicon-like numbers — not a completed real ab initio calculation. Every number and table shown below, however, is **genuine, verbatim stb-convergenceAnalysis output**, generated by actually running the real tool against them — nothing here is hand-derived or approximated.

```
Value    | E/atom(eV)  | |Delta E/atom| | Volume(Ang^3) | |Delta V/V|(%) | Quality
-------------------------------------------------------------------------------
150.0000 | -250.450000 | --             | 153.1304      | --             | OK
200.0000 | -250.425000 | 0.025000       | 159.2201      | 3.9768         | OK
250.0000 | -250.422500 | 0.002500       | 161.8786      | 1.6697         | OK
300.0000 | -250.422000 | 0.000500       | 161.9677      | 0.0551         | OK

Energy converged at: meshcutoff = 250.0000 (or higher, |Delta E/atom| < 0.01 eV)
Structure converged at: meshcutoff = 300.0000 (or higher, |Delta V/V| < 0.5%, relaxed cell)
[NOTE] Energy and relaxed structure disagree on where meshcutoff converges -- using the more
accurate-ward of the two (300.0000) so BOTH criteria are satisfied. This is exactly the scenario
stb-convergence's own docstring warns about: the total energy can look converged before the relaxed
geometry actually is.
```

Reading this the way Section 1.4 set up:

- **Energy alone would say 200 Ry is already "close enough"** — the delta from 200->250 Ry is only `0.0025` eV/atom, well under the `0.01` eV/atom default tolerance, and it only gets smaller from there (`0.0005` eV/atom by 300 Ry). If this tool only tracked energy, you'd walk away thinking `200` Ry (or even less) was a safe production setting.
- **The relaxed cell volume tells a different story.** The cube side grows `5.35 -> 5.42 -> 5.45 -> 5.451` Ang across the same 4 points — `Delta V/V` drops `3.98% -> 1.67% -> 0.055%`. It only falls below the `0.5%` default tolerance on the **last** step, `250 -> 300` Ry. At `200` or even `250` Ry, the relaxed geometry itself is still measurably drifting even though the energy already looks flat.
- **The tool's own recommendation, `300.0000` Ry, is the honest answer** — the smallest value where *both* criteria are simultaneously satisfied, not just the cheaper one two criteria would have picked in isolation. Using `200` Ry for a real strain/elastic-constant calculation downstream (`5.1-strain`/`5.2-elastic`) would risk comparing relaxed geometries that are still shifting with the mesh — numerical noise that could easily be mistaken for a real physical trend at the sub-percent strain scale those workflows care about.

This is precisely the failure mode Section 1.3's eggbox-effect discussion warns about, made concrete with real tool output rather than left abstract: **a coarser Mesh.CutOff can absolutely look energy-converged while the geometry it would relax atoms to is still moving.**

11.5.1.5 Known, deliberate limitations

- **Independent, not joint, sweeps** (Section 1.6) — each parameter is varied alone; the true joint optimum across all three can differ, especially when two parameters are physically coupled (a coarse k-grid masking a real mesh-cutoff dependence in a metal, for example).
- **Default ranges are a fixed, evenly-spaced approximation** of standard SIESTA convergence-test scales, not the literal non-uniform point sets some tutorials quote, and are **not derived from your actual structure** — heavier elements or higher-accuracy work may need a wider `Mesh.CutOff` range than the default `100-400` Ry.
- **Full variable-cell relaxation at every single swept point, always** (Section 1.3) — the only honest way to catch the eggbox effect, but correspondingly more expensive per point than a fixed-geometry single-point sweep.
- **`--force-tolerance`/the Quality column are advisory only** (Section 1.7) — they never gate or alter the tolerance-based plateau scan itself.
- **`--apply` for `kgrid` re-derives the Monkhorst-Pack grid from only the *first* discovered folder's own `structure.fdf`** for that parameter, assuming the same structure/vacuum layout holds across the whole sweep.

- **Stage 1's own `[6]` NOTE about Stage 2's layout support is stale** (Section 3.2, verified live) — a minor, real documentation inconsistency in the tool's own printed text, not a functional limitation of Stage 2 itself.

11.5.1.6 Step-by-step: running this workflow on your own structure

0. **Prerequisite**: a `structure.fdf` **and** a real `calc.fdf` — the production settings you actually intend to use. Unlike `5.3-cohesive`, Stage 1 does **not** build its own template (Section 2.2); it tests *your* settings.
1. **Decide which parameter(s) to test** (Section 1.2): `meshcutoff`, `energyshift`, `kgrid`, or `all` three.
2. **Run Stage 1** (`stb-convergence`, code `5.5.1`) — CLI or `stb-suite -> 5.5.1`. Writes everything under `convergence_runs/` (or your own `-o`).
3. **Run SIESTA yourself** in every generated `convergence_runs/<parameter>/convergence_*/` folder.
4. **Run Stage 2** (`stb-convergenceAnalysis`, code `5.5.2`) once they finish — CLI or `stb-suite -> 5.5.2`.
5. **Read `[2.N]`'s Quality column first** (Section 3.5) — don't trust a flagged row's number until you know why it's flagged.
6. **Read `[4]`'s Energy-converged / Structure-converged / Recommended** columns (Section 3.4) — if the first two disagree, trust Recommended, not whichever one you happened to look at first.
7. **(Optional)**: `--apply` to write the recommended value straight into your own production `calc.fdf` (Section 3.6); `--save-gnuplot`/`--view` for the 2-panel energy/volume plot (Section 3.7).

11.5.1.7 Files in this folder

File	What it is
`structure.fdf`	Bulk silicon, diamond cubic, 2 atoms/cell — the standard SIESTA convergence-test fixture (same file `test/5-workflow/5-convergence/` itself uses)
`calc.fdf`	DZP/GGA-PBE template — the actual settings this example convergence-tests
`example_5.5.sh`	This walkthrough's runnable script (both stages, 7 cases)

11.5.1.8 Running the script

```
bash example_5.5.sh
```

11.5.1.9 What's next

- Once a parameter is converged here and applied (`--apply`) into your `calc.fdf`, feed that **same** `calc.fdf` into `5.1-strain`, `5.2-elastic`, `5.3-cohesive`, or `5.4-phonons` — this workflow's entire purpose is making sure those results aren't numerical-parameter noise in disguise.

11.5.2 Implementation details

The three parameters..

`-parameter`	fdf directive	default sweep	more accurate when
`meshcutoff`	`Mesh.Cutoff`	100–400 Ry, step 50	larger: finer real-space grid for ρ, V_H, V_{xc}
`energyshift`	`PAO.EnergyShift`	0.001–0.05 Ry, step 0.01	smaller: longer, softer orbitals
`kgrid`	`kgrid.MonkhorstPack`	0.05–0.30 Å^{-1}, step 0.05	smaller density value: more k-points

`all` runs the three sweeps independently; each can get its own range with `–meshcutoff-range`, `–energyshift-range` or `–kgrid-range` (`MIN MAX STEP`). The k-grid at each density is computed as in **`stb-kgrid`**, with one point along vacuum axes.

Stage 1.. For every value a folder `<output-dir>/<parameter>/convergence_<parameter>_<value>/` is written with the structure, the template, pseudopotentials and a prepended `config_extra.fdf` that sets the swept value and forces a full CG relaxation of positions *and* cell (`MD.VariableCell true`, `MD.Steps = –relax-steps`, default 100). The relaxation is the point: SIESTA's grid makes forces and stresses noisy (eggbox effect), so a setting can give a converged energy at a fixed geometry and still a wrong relaxed geometry.

Stage 2.. For every parameter found, the tool reads the final total energy and the relaxed cell (last `outcell` block of the log named by `–file`) of each run and applies two criteria along the sweep, from cheap to accurate:

- energy: $|\Delta E|$ per atom between consecutive values below `–tolerance` (default 0.01 eV/atom);
- structure: $|\Delta V/V|$ between consecutive relaxed volumes below `–volume-tolerance` (default 0.5 %).

A value is converged only if *every* remaining step beyond it satisfies the criterion and every run there reached SCF convergence; the recommendation is the stricter of the two criteria, with a note when they disagree. Runs whose residual force exceeds `–force-tolerance` (default 0.05 eV/Å) or that used the whole step budget are marked in a quality column, without affecting the criteria. `–apply CALC` writes the recommended values into the given `calc.fdf` (this is the one place where the suite edits a file of yours, and only when asked).

Limitations.. The sweeps are independent: each varies one parameter with the template's values for the others, so converge in a sensible order (mesh and basis first, then k-points) or iterate. Energy differences between related structures often converge faster than total energies; for a property, the decisive test is the convergence of that property itself.

11.5.3 Command-line reference

11.5.3.1 `stb-convergence`

Stage 1 of 2: writes one ready-to-run SIESTA folder per swept value of one or more convergence-test parameters, one subfolder per parameter, fully relaxing the structure (positions + cell) at each. Doesn't run SIESTA – run each folder yourself, then use `stb-convergenceAnalysis`.

Synopsis.

```
usage: stb-convergence [-h] [-s STRUCTURE] [-c CALC] [-p PSEUDO_DIR]
                       --parameter PARAMETER [PARAMETER ...]
                       [--meshcutoff-range MIN MAX STEP]
                       [--energyshift-range MIN MAX STEP] [--kgrid-range MIN MAX STEP]
                       [--relax-steps RELAX_STEPS] [--vacuum-gap VACUUM_GAP]
                       [-o OUTPUT_DIR] [--save-report] [-v] [--no-intro]
```

Options

`-s, --structure STRUCTURE`

Input `structure.fdf`, copied as 'structure.fdf' into every generated folder (default: `structure.fdf`).

`-c, --calc CALC`
: `calc.fdf` template (default: `calc.fdf`) – SCF/basis/MD settings, normally %including the structure file above by name. Copied into every generated folder with one line added: '%include `config_extra.fdf`', prepended at the very top (see [3] of the report) – this forces full relaxation and the swept parameter's value, regardless of what this template itself contains.

`-p, --pseudo-dir PSEUDO_DIR`
: Pseudopotentials source (optional): a bundled bank (dojo, virtual_vault) or a folder path. If given, the required pseudopotential for every species in the structure is copied into every generated folder, so it's immediately ready for SIESTA.

`--parameter PARAMETER [PARAMETER ...]`
: Which tag(s) to sweep: meshcutoff, energyshift, kgrid, or 'all' for all three (one independent sweep per parameter, each in its own subfolder). required

`--meshcutoff-range MIN MAX STEP`
: Custom Mesh.CutOff sweep range (Ry). Omit to use the suggested default (see [2] of the report). Only meaningful if 'meshcutoff' is in `--parameter`. type: float

`--energyshift-range MIN MAX STEP`
: Custom PAO.EnergyShift sweep range (Ry) (see `--meshcutoff-range`). Only meaningful if 'energyshift' is in `--parameter`. type: float

`--kgrid-range MIN MAX STEP`
: Custom k-grid density sweep range (1/Ang) (see `--meshcutoff-range`). Only meaningful if 'kgrid' is in `--parameter`. type: float

`--relax-steps RELAX_STEPS`
: Number of relaxation steps (MD.Steps) forced in every generated folder (default: 100). Both atomic positions and the cell relax (MD.VariableCell true). type: int

`--vacuum-gap VACUUM_GAP`
: `--parameter` kgrid only: vacuum-axis detection threshold in Angstrom, same meaning as `stb-kgrid's --vacuum-gap` (default: 10.0). type: float

`-o, --output-dir OUTPUT_DIR`
: Top-level directory (default: convergence_runs). Run folders go into its own '<output-dir>/<parameter>/convergence_<parameter>_<value>/' subfolder, one tree per swept parameter.

`--save-report`
: Also persist the report to <output-dir>/convergence_stage1.txt. Off by default. switch, off by default

Like every STB command it also accepts `-h/-help`, `-v/-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Usage examples:
  stb-convergence -s structure.fdf -c calc.fdf -p dojo --parameter all
  stb-convergence -s structure.fdf -c calc.fdf -p dojo --parameter all --meshcutoff-range 150 350 50
  stb-convergence -s structure.fdf -c calc.fdf -p dojo --parameter kgrid --relax-steps 200 --save-report
```

11.5.3.2 `stb-convergenceAnalysis`

Stage 2 of 2: analyzes a `stb-convergence` sweep and reports where each swept parameter's energy AND relaxed structure converge.

Synopsis.

```
usage: stb-convergenceAnalysis [-h] [--dir DIR] [--file FILE] [--tolerance TOLERANCE]
                               [--volume-tolerance VOLUME_TOLERANCE]
                               [--force-tolerance FORCE_TOLERANCE] [-o OUTPUT_DIR]
                               [--apply CALC_FDF] [--vacuum-gap VACUUM_GAP]
                               [--save-report] [--save-gnuplot] [--view] [-v]
                               [--no-intro]
```

Options

`--dir DIR`
: Directory to scan for 'convergence_*' run folders – either `stb-convergence's` default nested layout (one <parameter>/ subfolder per swept parameter under here) or a flat layout (pointing directly at one parameter's own subfolder) (default: convergence_runs).

`--file FILE`
: SIESTA output filename inside each run folder (default: `calc.out`).

`--tolerance TOLERANCE`
: Energy convergence tolerance in eV/atom (default: 0.01). type: float

`--volume-tolerance VOLUME_TOLERANCE`
: Relaxed-cell-volume convergence tolerance, as a percent change between consecutive swept values (default: 0.5). type: float

`--force-tolerance FORCE_TOLERANCE`
: Residual-force data-quality flag threshold in eV/Ang (default: 0.05) – NOT a convergence criterion, just flags a run whose CG relaxation may not have reached a genuine minimum. type: float

`-o, --output-dir OUTPUT_DIR`
: Directory to write <parameter>_convergence.dat/.gplot into with `--save-gnuplot`, and the report into with `--save-report` (default: current directory). Created if it doesn't exist.

`--apply CALC_FDF`
: Writes every parameter's converged (recommended) value into this `calc.fdf` in place (same tag substitution `stb-convergence` itself uses) – parameters with no converged value found are skipped.

`--vacuum-gap VACUUM_GAP`
: `--apply` with a kgrid sweep only: vacuum-axis detection threshold in Angstrom, same meaning as `stb-kgrid's` `--vacuum-gap` (default: 10.0). type: float

`--save-report`
: Also persist the report to <output-dir>/convergence_report.txt. Off by default. switch, off by default

`--save-gnuplot`
: Also write, per discovered parameter, curve data + a companion .gplot script (2-panel: energy + relaxed volume). Off by default. switch, off by default

`--view`
: Show an interactive matplotlib preview (energy + volume per discovered parameter) before finishing. Off by default. switch, off by default

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Usage examples:
```

```
stb-convergenceAnalysis --dir convergence_runs
stb-convergenceAnalysis --dir convergence_runs --tolerance 0.005 --save-report --save-gnuplot --view
stb-convergenceAnalysis --dir convergence_runs --apply calc.fdf
```

11.6 Structure Solution (XRD)

At a glance

Menu code: 5.6 (a multi-stage workflow)

Code	Stage	Command
5.6.1	Stage 1 - Prep	`stb-xrdsearch`
5.6.2	Stage 2 - Analysis	`stb-xrdrank`

Guided example: `examples/5.6-xrd/`

Summary: Cast candidate structures across a set of space groups, then rank them by similarity to an experimental powder XRD pattern.

11.6.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/5.6-xrd/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

This workflow has 2 stages: **Stage 1** (`stb-xrdsearch`, code `5.6.1`) takes a *composition* (not a structure — this is the one workflow pair in this suite that doesn't start from an already-built `structure.fdf`) and a list of candidate space groups, and casts one or more random atomic arrangements per group, ready to relax with real SIESTA. **Stage 2** (`stb-xrdrank`, code `5.6.2`) simulates each candidate's own powder XRD pattern and ranks them by similarity to a real experimental pattern — a fast pre-screen for *which* candidate space group/arrangement is most likely the true structure, before committing to expensive DFT on all of them.

This answers a genuinely different question from `5.1-5.5`: those all start from a structure you already believe is right and ask how it responds to load (`5.1/5.2`), how strongly it's bound (`5.3`), how it vibrates (`5.4`), or whether its total energy/relaxed geometry has converged with some numerical parameter (`5.5`) — this workflow instead asks **which structure is right in the first place**, using a real experimental measurement as the arbiter.

Both stages live in this one folder and this one tutorial, same reasoning as `5.1-5.5`: Stage 1's output only exists to feed Stage 2.

11.6.1.1 Theory

Powder XRD and Bragg's law. A powder sample contains crystallites in every orientation at once, so a monochromatic X-ray beam (this workflow defaults to Cu-K-alpha, `1.54184 Ang`) diffracts off every family of lattice planes `(hkl)` simultaneously, each at its own angle given by **Bragg's law**:

```
n * lambda = 2 * d_hkl * sin(theta)
```

`d_hkl` is the spacing between `(hkl)` planes (for a cubic cell, `d_hkl = a / sqrt(h^2 + k^2 + l^2)`). The experiment records intensity vs. `2*theta` — a 1D "fingerprint" of the 3D crystal structure: peak *positions*

come from the lattice geometry (Bragg's law above), peak *intensities* come from the structure factor (1.2). This is exactly why a 1D pattern can distinguish between different atomic arrangements sharing the same composition: different space groups give different lattice geometries and different structure factors, hence different fingerprints.

The structure factor and systematic absences. Not every `(hkl)` allowed by Bragg's law actually produces a peak — the **structure factor** `F_hkl = sum_j f_j * exp(2*pi*i*(h*x_j + k*y_j + l*z_j))` (sum over every atom `j` in the basis, `f_j` its atomic scattering factor) can be exactly zero for some `(hkl)` combinations, depending on the space group's own symmetry — **systematic absences**. This is the concrete mechanism this workflow's decisive example (Section 5) exploits: diamond- cubic silicon (`Fd-3m`, space group 227, 8 atoms/cell at Wyckoff `8a`) and a naively-relaxed simple-cubic arrangement of the same 8 Si atoms have *different* systematic absences, so their simulated patterns look nothing alike even though both are "just silicon."

From a stick pattern to a comparable profile. A calculated pattern (via `pyxtal.XRD.XRD`, Section 2) is naturally a set of discrete sticks (`2theta, d, h, k, l, intensity`) — zero width, since Bragg's law gives an exact angle for a perfect infinite crystal. Real instruments broaden every peak (finite crystallite size, instrumental resolution, ...), so before comparing to a real pattern, both this workflow's own simulated stick pattern (via `pattern.get_profile()`) and any experimental input that looks like a similarly sparse stick/peak list (Section 1.5/4.4) are Gaussian-broadened onto the same footing: `FWHM = 0.1 deg`, `res = 0.01 deg` — pyxtal's own defaults, used consistently everywhere this suite touches XRD (`stb-xrd`, `core/xrd.py::broaden_peak_list`).

The similarity metric. `stb-xrdrank` scores every candidate with `pyxtal.XRD.Similarity`, a cosine-weighted cross-correlation between the two broadened profiles `f` (simulated) and `g` (experimental):

```
value = | xCorr_fg(w) / sqrt( xCorr_ff(w) * xCorr_gg(w) ) |
```

(`xCorr_fg(w)` is a windowed cross-correlation, weighted by a cosine kernel over a shift range `l` — this is what makes the metric tolerant of a small rigid shift between the two patterns, e.g. from a slightly-off lattice constant, rather than demanding a perfect point-by-point match.) Read literally: **this is a normalized dot product** — `1.0` means the two profiles are identical up to scale, `~0` means uncorrelated. Section 4.5 traces the exact place this formula breaks (a real bug, now fixed) when one of the two profiles is degenerate (all zero).

The experimental data file: the model this workflow expects. This is the canonical model — `experimental.dat` in this folder is a real, working example of it:

```
# 2theta   intensity
   28.330    100.000
   47.110     47.434
   55.890     24.408
   ...
```

Two columns. Column 1 is `2theta` in degrees; column 2 is intensity, on **any** scale (`0-100`, `0-1`, raw counts, ...) — both sides get max-normalized before comparing, so the absolute scale never matters. Whitespace- or comma-separated; blank lines and `#`-prefixed comments (the whole header above) are skipped. `--wavelength` must match whatever source actually produced this data (default `CuKa`).

How the code actually picks the intensity column, and why it isn't just "always the last one": `core/xrd.py::read_experimental_pattern` reads *every* column on each line (not just the first and last),

then `_select_intensity_column()` picks the **last column that is not constant** among the columns after 2-theta. For a plain 2-column file that's the only candidate anyway. This matters for real instrument export formats with a 3rd (or more) column — Section 4.5 works through a real one, live, and shows exactly what used to go wrong before it was fixed.

1.6 Why this needs a *search* at all. Given only a composition and a space group, there are usually many symmetry-equivalent-but-not-identical ways to place atoms on that group's Wyckoff positions (and, for a low-symmetry group, real continuous positional freedom within them) — `stb-crystalcast` (driven internally by `stb-xrdsearch`, Section 3) samples this space randomly, `--count-per-group` times per group, which is why Stage 1 is a *search* (possibly multiple candidates per group) feeding a *ranking* (Stage 2), not a single deterministic answer.

11.6.1.2 Libraries and external dependencies used

- **PyXtal** (`1.1.4` in this environment) does essentially all of the domain-specific work: random symmetry-constrained structure casting from a space group + composition (`stb-crystalcast`, driven by Stage 1), the powder-pattern simulation itself (`pyxtal.XRD.XRD`), and the similarity metric (`pyxtal.XRD. Similarity`, Section 1.4). `core/xrd.py` wraps the pieces shared between `stb-xrd` (Analysis, code `2.9`) and this workflow's Stage 2.
- **pymatgen** (`2025.10.7`) supplies the `Structure` object every other piece of code here operates on, plus `SpacegroupAnalyzer` (via **spglib**, `2.7.0`) for the space-group label shown in Stage 2's ranking table (`core/symmetry.py::space_group_label`).
- **ASE** (`3.27.0`) reads a relaxed SIESTA `.STRUCT_OUT` file (`core/structure_io.py::read_siesta_structure`) — there is no `.fdf`-style reader for that format, it's ASE's own.
- **MACE** (via `core/mace_relax.py`, the same shared module `stb-mlrelax/stb-defect --ml-rank/stb-amorphize` use) — **optional**, only touched by Stage 1's `--mace-relax` (Section 3.4) and `--ml-rank` (a *different*, transient, ranking-only relax passed through to `stb-crystalcast` — Section 3.4 explains the distinction). Needs `pip install stb_suite[ml]`.
- **numpy** / **scipy** — general array math; `scipy.signal.find_peaks` is used only by *this tutorial's own* fixture-generation, not by the suite itself.
- **matplotlib** (`2.10.8`) — Stage 2's `--view` (Section 4.6), a two-panel figure (ranking bar chart + best-match overlay), always via a blocking `plt.show()` after the report is fully printed (same convention as `stb-xrd`/`stb-density`/`stb-workfunction`'s own `--view`).
- **gnuplot** (`6.0`, external, not a Python dependency) — only touched when `--save-gnuplot` is given (Section 4.6); the suite writes `.dat` + `.gplot` file pairs, gnuplot itself is never invoked by the suite.

11.6.1.3 3. Stage 1: casting candidates (`stb-xrdsearch`, code `5.6.1`)

What it does, and how. For each space group in `--groups`, shells out to `stb-crystalcast` once (`--count-per-group` random candidates per call), then writes each surviving candidate into its own `<output-dir>/group_<G>[_<i>]/structure.fdf` — a bare structure file, no calc setup yet. From there, **by default**, every candidate folder is made ready to run as-is:

1. **calc.fdf** — auto-generated (the exact same standard relaxation template `stb-inputfile -t relax` produces, reused directly as a library call — Section 3.2), or copied verbatim from `--calc <path>` if you already have your own template.
2. **Pseudopotentials** — copied in from `-p/--pseudo-dir` (a bundled bank name, e.g. `dojo`, or your own folder — Section 3.3), for the real species found in the *cast* structure (not `--species` directly, which `--molecular` can make a molecule name rather than an element symbol).

3. **MACE pre-relaxation** (opt-in, `--mace-relax`) — positions + cell, before calc.fdf is generated, so the k-grid calc.fdf computes already reflects the pre-relaxed cell (Section 3.4).

calc.fdf: auto-generate vs. custom template, live.

```
$ stb-xrdsearch --species Si --num-ions 8 --groups 227 --no-intro

[1] CALC.FDF SETUP
------------------------------------------------------------
Mode                  : auto-generate (standard relaxation calc.fdf, same as stb-inputfile -t relax)
DFT-D3 correction     : disabled
Spin polarization     : non-polarized
```

`--calc my_template.fdf` switches to the second mode instead — copied verbatim into every candidate folder as `calc.fdf` (always this exact name in *either* mode, even though a custom template's own basename might differ, so every folder in a batch is runnable the identical way regardless of which mode produced it).

Pseudopotentials, live.

```
[4] PSEUDOPOTENTIALS
------------------------------------------------------------
Source                : .../stb-suite/src/stb/pseudopotentials/dojo
Species | Status
----------------
Si      | found
[OK] All required pseudopotentials found -- will be copied into every generated folder.
```

Copied (`shutil.copy2`), not symlinked — a symlink into a bundled bank or an external `--pseudo-dir` breaks the moment a candidate folder is archived/rsynced/scp'd elsewhere (e.g. to an HPC cluster) without also bringing the link target; a real copy always travels with the folder (`core/pseudopotentials.py`, shared by every prep tool in this suite).

MACE pre-relaxation, live.

```
$ stb-xrdsearch --species Si --num-ions 8 --groups 227,225,229 --mace-relax -p dojo --no-intro

[3] MACE PRE-RELAXATION
------------------------------------------------------------
Model                 : MACE-MP-0 (small)
Folder    | E before (eV) | E after (eV) | Vol change | Max disp (Ang) | Steps
-------------------------------------------------------------------------------------
group_227 | -42.9479      | -42.9666     | +1.91%     | 0.0444         | 7 (converged)
group_225 | -38.7782      | -41.4365     | -24.48%    | 0.7294         | 9 (converged)
group_229 | -38.3101      | -41.4366     | +27.56%    | 0.5867         | 7 (converged)
```

Real, physically meaningful result, not illustrative: `227` (`Fd-3m`, diamond) relaxes to `-42.9666 eV`; `225` and `229` (both physically *wrong* space groups for elemental Si — a single-site FCC/BCC packing, not the tetrahedrally-bonded diamond motif) both collapse under their own imposed symmetry constraint into the same higher-energy `-41.44 eV` minimum — **already, before any XRD comparison at all, MACE's own energy shows a ~1.5 eV gap favoring the true structure.** This is *not* the same thing as `--ml-rank`: that flag (passed straight through to `stb-crystalcast`) relaxes candidates positions-only, transiently, purely to *select/rank* which of several random attempts to keep — the relaxed geometry it computes is discarded, never written to disk. `--mace-relax` relaxes (positions + cell) the *final* candidates that survive casting, and **keeps** the result, overwriting `structure.fdf` — the two are independent and can be combined.

Report structure.

```
[0] RUN METADATA
[1] CALC.FDF SETUP
[2] CANDIDATE CASTING
[3] MACE PRE-RELAXATION
[4] PSEUDOPOTENTIALS
[5] CALC.FDF & PSEUDOPOTENTIAL GENERATION
[6] SUMMARY & NEXT STEPS
[7] LIBRARY WARNINGS
```

`[3]`-`[5]` always print, even when their feature wasn't requested (e.g. `[3]` prints "Not requested" rather than being skipped) — every section number means the same thing on every run, whether or not you used that particular flag.

Running it both ways. Both CLI and `stb-suite -> 5.6.1` produce identical `xrd_search/` output for identical answers (Section 9's script exercises the CLI path; the interactive menu asks the same questions in the same order, including the MACE pre-relax model choice, Section 10's script for `5.4` shows the general pattern this suite uses for every workflow's interactive twin).

11.6.1.4 4. Stage 2: ranking candidates (`stb-xrdrank`, code `5.6.2`)

Report structure.

```
[0] RUN METADATA
[1] CANDIDATES FOUND
[2] EXPERIMENTAL PATTERN
[3] RANKING
[4] OUTPUT DATA & PLOTS
[5] SUMMARY & NEXT STEPS
[6] LIBRARY WARNINGS
```

4.2 `--input-dir` defaults to `xrd_search`. `--input-dir` is **not required** — it defaults to `xrd_search`, Stage 1's own default `--output-dir`, so the common case (Stage 2 right after Stage 1, same folder) needs zero extra flags:

```
$ stb-xrdrank --experimental experimental.dat --no-intro
...
Input directory        : xrd_search
```

If a candidate subfolder also contains a `*.STRUCT_OUT` file (you've run real SIESTA there since Stage 1), that relaxed structure is used instead of the raw `structure.fdf` — no flag needed, `find_candidates()` checks for it automatically per subfolder (this is exactly what happens for the real production data in Section 5).

The experimental pattern, live — a clean structure solution. Using this folder's own `experimental.dat` (real Si, Cu-K-alpha, the exact 2-column model Section 1.5 describes) against the 3 candidates cast in Section 3.4 (`227` correct, `225/229` wrong):

```
$ stb-xrdrank --experimental experimental.dat --no-intro

[2] EXPERIMENTAL PATTERN
```

```
------------------------------------------------------------
Points                : 11900
2-theta span (file)   : 1.000 - 119.990 deg
Peak-list detected    : no -- compared as a continuous scan, unbroadened.

[3] RANKING
------------------------------------------------------------
  -> group_225: similarity 0.0326 (Si, Pm-3m (No. 221), raw)
  -> group_227: similarity 0.9972 (Si, Fd-3m (No. 227), raw)
  -> group_229: similarity 0.0376 (Si, Pm-3m (No. 221), raw)
Rank | Name      | Similarity | Formula | Space group     | Source
------------------------------------------------------------------
1    | group_227 | 0.9972     | Si      | Fd-3m (No. 227) | raw
2    | group_229 | 0.0376     | Si      | Pm-3m (No. 221) | raw
3    | group_225 | 0.0326     | Si      | Pm-3m (No. 221) | raw
Score gap (#1 vs #2)   : 0.9596
```

A textbook-clean decisive result — `0.9972` for the true structure vs. `~0.03-0.04` for two physically wrong ones, exactly the kind of unambiguous separation Section 1.2's systematic-absences argument predicts. Compare this against Section 5's real production data, where all 3 candidates are the *same* correct structure at slightly different relaxation quality — a genuinely different, subtler question the score gap also answers correctly.

Peak-list auto-detection and broadening — the fix, demonstrated live. `experimental_peaks.dat` (this folder) is the *same* real Si pattern, but as a **sparse 9-point peak list** — the shape a literature/indexed peak table or `stb-xrd`'s own stick-pattern output has, not a dense continuous scan. `core/xrd.py::looks_like_peak_list()` auto-detects this (highly uneven spacing between sorted 2-theta points) and Gaussian-broadens it (Section 1.3) before comparing:

```
$ stb-xrdrank --experimental experimental_peaks.dat --no-intro
[2] EXPERIMENTAL PATTERN
Points                : 9000
Peak-list detected    : yes -- auto-broadened (Gaussian, FWHM=0.1 deg) before comparing...
[3] RANKING
  -> group_227: similarity 0.9765   (correct structure -- still wins decisively)
  -> group_229: similarity 0.0159
  -> group_225: similarity 0.0138
```

Now the same file, `--raw-experimental` (skip broadening):

```
$ stb-xrdrank --experimental experimental_peaks.dat --raw-experimental --no-intro
[2] EXPERIMENTAL PATTERN
Points                : 9
Peak-list detected    : yes -- but --raw-experimental was given, so it is compared unbroadened...
[3] RANKING
  -> group_225: similarity 0.3485
  -> group_229: similarity 0.3476
  -> group_227: similarity 0.2897   (correct structure -- now LOSES)
Score gap (#1 vs #2)   : 0.0009  [close call -- consider relaxing both before deciding]
```

This is a live, real demonstration of why the fix matters, not a synthetic illustration: comparing a zero-width stick list directly against a broadened simulated profile via cubic interpolation (what `pyxtal.XRD.Similarity` does internally with no broadening) draws a spurious curve through the empty gaps between peaks — the *wrong* structures happen to score higher, a complete inversion of the correct ranking. Broadening both sides the same way before comparing is not a cosmetic default; without it, this workflow gives the wrong answer.

The 3-column bug — reproduced and shown fixed, live. Real diffractometer export formats aren't always 2 columns. A `.int`-style export encountered in real production use (`xrd.int`, Section 5) has **3** columns: `2theta`, intensity, and a 3rd field (an unpopulated uncertainty/esd column) that was **exactly zero on all 11900 lines**. `read_experimental_pattern` used to always take the *last* column as intensity (a rule that happened to be correct for the only 2 formats seen before: a plain 2-column file, and `stb-xrd`'s own 6-column stick-pattern output) — for this file, that meant reading the all-zero 3rd column as "intensity."

Traced to the exact line: `pyxtal.XRD.similarity_calculate()` computes `xCorrfg_w / sqrt(aCorrff_w * aCorrgg_w)` (Section 1.4) — with an all-zero `gy` (the misread "experimental" intensity), `aCorrgg_w = 0` and `xCorrfg_w = 0` too, so the division is a literal `0/0` -> **`nan`**, with no exception and no visible cause beyond a bare `RuntimeWarning: invalid value encountered in scalar divide` — which, before the `[6] LIBRARY WARNINGS` section existed, had nowhere organized to surface at all (Section 4.7).

The fix, reproduced live in this folder — same real data (`experimental.dat`) with a synthetic bogus 3rd column appended, mimicking the exact real-world mistake:

```
$ awk '{print $1, $2, "0.00000"}' experimental.dat > experimental_3col.dat
$ stb-xrdrank --experimental experimental_3col.dat --no-intro
[3] RANKING
  -> group_227: similarity 0.9972   (identical to the clean 2-column result, Section 4.3)
```

`_select_intensity_column()` now reads every column, excludes any column that's constant across the whole file (a constant column can never be a real diffraction intensity signal), and picks the **last non-constant** one — correct for this 3-column case, and still correct (unchanged behavior) for the 2-column and 6-column stick-pattern formats, since none of *their* columns are ever constant for a real multi-peak pattern. If *every* column after 2-theta is degenerate (nothing usable at all), the tool now raises a clear error instead of silently proceeding:

```
$ printf "1 0\n2 0\n3 0\n4 0\n5 0\n" > broken.dat
$ stb-xrdrank --experimental broken.dat --no-intro
[FAIL] 'broken.dat': every column after 2-theta is constant (no usable intensity signal) -- check the
file's column order/format.
```

This same fix also benefits `stb-xrd --compare-to` (Analysis, code `2.9`), which shares the exact same `core/xrd.py::read_experimental_pattern`.

Plotting: gnuplot and matplotlib, both opt-in. `--save-gnuplot` writes **two** `.dat+.gplot` pairs, both under their own `plot/` subfolder (kept separate from the plain-text ranking/report files next to it):

```
$ stb-xrdrank --experimental experimental.dat --save-gnuplot --no-intro
[4] OUTPUT DATA & PLOTS
[OK] Ranking bar chart written to './plot/xrd_rank.dat' / './plot/xrd_rank.gplot'.
[OK] Best-match overlay written to './plot/xrd_rank_top_sim.dat' / './plot/xrd_rank_top_exp.dat' /
'./plot/xrd_rank_overlay.gplot'.
```

- **`xrd_rank.gplot`** — a bar chart, one bar per candidate, `similarity` on the y-axis.
- **`xrd_rank_overlay.gplot`** — the best candidate's own simulated pattern overlaid on the experimental one (two separate small `.dat` files, since they're generally on different grids — pyxtal's own broadened-profile resolution for one, the experimental file's own sampling for the other).

`--view` shows the **same two plots**, as one matplotlib figure (bar chart on top, overlay below) instead of two `.gplot` scripts — a live preview, not a file output, always shown **last**, after the report is fully printed/closed, so a blocking `plt.show()` window never delays it (`MPLBACKEND=Agg` makes this a safe no-op for a non-interactive script, same convention `example_5.4.sh` uses).

4.7 `[6] LIBRARY WARNINGS` — what it captures, and how. Every third-party call this tool makes (`pyxtal`'s pattern computation and `Similarity`, `pymatgen`/`spglib`'s space-group detection, the peak-list broadening) is wrapped in `capture_library_noise()` (`core/cli.py`, shared with `stb-phononsCreate`/`stb-xrdsearch`): both `stdout` prints **and** `warnings.warn()` calls made *inside* the wrapped block are captured into one collector list, instead of leaking straight to the terminal interleaved with the report. This tool's *own* `print_dual` report lines are never inside a wrapped block, so the report itself is never delayed or hidden.

A real, subtle gap, since fixed (verified against this Si example's own real data, Section 5): the space-group lookup (`core/symmetry.py::space_group_label`, called once per candidate to fill the ranking table's own column) used to sit in its *own* `try/except`, **outside** the wrapped block that computes the pattern/similarity for that same candidate. Python's own warning system shows a given warning only *once per call site* by default — depending on whether an identical warning had already fired earlier inside some *other* wrapped block in the same run, this specific one would sometimes vanish untracked and sometimes leak straight to the terminal, unpredictably. Moving the `space_group_label()` call *inside* the same `capture_library_noise` block as the rest of that candidate's comparison closes the gap regardless of ordering — reproduced live:

```
$ stb-xrdrank --experimental experimental.dat --no-intro
...
[6] LIBRARY WARNINGS
------------------------------------------------------------
Messages emitted by external libraries (pyxtal/scipy/numpy/pymatgen/spglib) during this run...
[pyxtal XRD ranking] DeprecationWarning: Set OLD_ERROR_HANDLING to false and catch the errors
directly.
```

That one line is the *only* place this message appears anywhere in the tool's output — never before the report banner, never interleaved with `[3] RANKING`'s own live progress lines.

Running it both ways. Both CLI and `stb-suite -> 5.6.2` produce the identical report for identical answers — the interactive menu prompts for the same choices (input folder, defaulting to `xrd_search`; experimental file; auto- broadening; `--top`; save-gnuplot/view/save-report) in the same order as the CLI flags above.

11.6.1.5 Worked example: real production data, end-to-end

This is not illustrative data — this is a real calculation run with this workflow: `stb-xrdsearch` cast 3 candidates for `Si8`, space group `227` (`Fd-3m`), each pre-relaxed with MACE, then **actually relaxed with real SIESTA** (GGA-PBE, DZP, `320 Ry` mesh cutoff, `6x6x6` Monkhorst-Pack — `132` irreducible k-points, force tolerance `0.01 eV/Ang`) — a genuinely different question from Section 4.3's clean right-vs-wrong-space-group case: all 3 candidates here **are** the same correct structure; the question is which SIESTA run relaxed it best.

5.1 First run: `nan` for every candidate. The real experimental file, `xrd.int`, is the exact 3-column format Section 4.5 works through (2theta, intensity, an always-zero 3rd column). Before the fix, every candidate scored `similarity nan`:

```
[3] RANKING
  -> group_227_1: similarity nan (Si, Fd-3m (No. 227), relaxed)
  -> group_227_2: similarity nan (Si, Fd-3m (No. 227), relaxed)
  -> group_227_3: similarity nan (Si, Fd-3m (No. 227), relaxed)
[6] LIBRARY WARNINGS
[pyxtal XRD ranking] RuntimeWarning: invalid value encountered in scalar divide
```

Exactly the mechanism Section 4.5 traces, on real data, in real production use — not a contrived test case.

After the fix: real numbers, and a real relaxation-quality finding.

```
Rank | Name        | Similarity | Formula | Space group     | Source
1    | group_227_2 | 0.9922     | Si      | Fd-3m (No. 227) | relaxed
2    | group_227_3 | 0.9539     | Si      | Fd-3m (No. 227) | relaxed
3    | group_227_1 | 0.9537     | Si      | Fd-3m (No. 227) | relaxed
Score gap (#1 vs #2)   : 0.0383
```

Investigating **why** `group_227_2` scored differently (SIESTA's own `calc.out` files, not this workflow's report) found a real relaxation -quality difference: `group_227_1`/`group_227_3` fully relaxed over 4 CG steps to `a = 5.4800 Ang` with a near-zero residual stress (`~2e-4 eV/Ang^3`); `group_227_2` stopped after just **1** CG step at `a = 5.4608 Ang`, with a residual stress (`~0.0058 eV/Ang^3`, `~0.92 GPa`) just under SIESTA's default stress tolerance (`MD.MaxStressTol` was left commented out in the auto-generated `calc.fdf`, Section 3.2) — **not** fully converged. Reconstructing the experimental pattern's own implied lattice constant from its 6 strongest peaks (Bragg's law, Section 1.1) gives `a_exp = 5.4561 +/- 0.0002 Ang` — closer to the *real* literature value for Si (`5.4310 Ang`) than either PBE result, but still measurably different from both, and notably close to typical published PBE-GGA silicon values (`~5.463-5.470 Ang`) — suggesting `xrd.int`, despite its name, is more likely a theoretical/simulated reference pattern than a genuine lab measurement. **`group_227_2` "won" this ranking because its under-converged cell happened to land closer to that reference value than the fully -relaxed cells did — not because it is the more correct DFT result.** A real, physically subtle finding this workflow's own numbers surface, but cannot resolve on their own (Section 6's limitations, Section 7's recommended next step: re-relax `group_227_2` with an explicit, tighter `MD.MaxStressTol` before trusting this ranking).

11.6.1.6 Known, deliberate limitations

- **Not a Rietveld refinement.** This is a fast similarity pre-screen (Section 1.4) between simulated and experimental *patterns* — it never fits atomic positions/thermal factors/preferred orientation against the data the way a real Rietveld refinement does. Use it to prioritize which candidate(s) deserve real DFT, not as final proof (every report says so explicitly, Section 4.1's own `[5]`).
- **The similarity score can't distinguish "wrong structure" from "right structure, poorly relaxed."** Section 5.2's real `group_227_2` case is the concrete demonstration — a single number conflates both effects; only reading the underlying SIESTA relaxation quality (residual stress, number of CG steps) separates them.
- **`_select_intensity_column`'s "last non-constant column" heuristic (Section 4.5) is not a general-purpose column-format detector** — it correctly handles every real format this suite has encountered so far (2-column, `stb-xrd`'s 6-column stick pattern, this `.int` 3-column export), but a hypothetical file where intensity is genuinely *not* the last varying column would still be misread. No `--intensity-column` override exists yet (would follow this suite's own "expose on first genuine need" convention if one ever comes up).
- **`--mace-relax` (Section 3.4) is a heuristic pre-screen, not a DFT replacement** — same documented limitation as every other MACE-MP-0 consumer in this suite (`stb-mlrelax` etc.).

- **Casting is random, not exhaustive** — `--count-per-group` samples the Wyckoff configuration space randomly; a low `--count-per-group` on a low-symmetry space group with real continuous positional freedom can miss the true minimum-energy arrangement entirely.
- **Wavelength/2-theta-range mismatches between the simulated and experimental sides are the user's responsibility** — `--wavelength` must match whatever source produced the experimental file; the comparison is only ever computed over the *overlap* of both ranges, with no explicit warning if that overlap is small.

11.6.1.7 Step-by-step: running this workflow on your own composition

1. **Decide the composition and candidate space group(s)** (`--species/ --num-ions/--groups`) — if you have prior knowledge (e.g. from a related compound, or a short list from indexing software), list only those; otherwise cast a broader net and let Stage 2's ranking narrow it down (Section 4.3's clean 3-way example is exactly this).
2. **Decide `--count-per-group`** and whether to add `--ml-rank` (a cheap, transient MACE-based pre-screen *during* casting, Section 3.4) if you're casting many random attempts per group and want only the most promising ones kept (`--keep-top-per-group`).
3. **Run Stage 1** (`stb-xrdsearch`, code `5.6.1`):
   - CLI: `stb-xrdsearch --species <S...> --num-ions <N...> --groups <G,...> --mace-relax -p <bank>`.
   - Interactive: `stb-suite -> 5.6.1`.
   - Writes `xrd_search/group_<G>[_<i>]/` (Section 3.1), each folder already containing `structure.fdf` + `calc.fdf` + pseudopotentials, ready to run.
4. **(Optional) run SIESTA yourself** in each `xrd_search/group_*/` folder — Stage 2 works equally well on the raw, pre-SIESTA candidates (Section 4.3) or on relaxed `*.STRUCT_OUT` results (Section 5) if you already have them; it auto-detects which is present per folder.
5. **Get (or prepare) your experimental pattern as the 2-column model** (Section 1.5/this folder's `experimental.dat`) — export it directly if your instrument software supports it, or extract just the `2theta` and intensity columns from whatever multi-column format it actually produces (Section 4.5's exact real-world lesson).
6. **Run Stage 2** (`stb-xrdrank`, code `5.6.2`) once every candidate is ready:
   - CLI: `stb-xrdrank --experimental <file> [--wavelength <src>] [--save-gnuplot] [--view]` (`--input-dir` needs no flag if it's still the default `xrd_search`, Section 4.2).
   - Interactive: `stb-suite -> 5.6.2`.
7. **Read `[2]` before trusting `[3]`** — confirm `Peak-list detected` correctly matches what your file actually is (Section 4.4); a wrongly -skipped auto-broadening can invert the ranking (Section 4.4's own live before/after).
8. **Check `[6] LIBRARY WARNINGS`** — usually harmless, but a genuine `RuntimeWarning: invalid value encountered` there (Section 4.5/5.1) is a strong signal something upstream (most likely the experimental file's own column format) needs a second look, even if `[3]` itself doesn't obviously look wrong yet.
9. **Relax the top-ranked candidate(s) with real SIESTA**, then rerun Stage 2 on the relaxed `*.STRUCT_OUT` results to confirm (Section 5's own real example is exactly this second pass) — and read the *SIESTA* convergence details (residual stress, CG steps), not just the similarity score, before picking a final winner (Section 5.2/6).

11.6.1.8 Files in this folder

File	What it is
`experimental.dat`	Real Si (diamond cubic) powder pattern, Cu-K-alpha, the canonical 2-column model (Section 1.5) -- derived from real `xrd.int` production data with its bogus 3rd column removed
`experimental_peaks.dat`	The same real pattern, as a sparse 9-point peak list (Section 4.4)
`example_5.6.sh`	This walkthrough's runnable script (both stages)

No `structure.fdf`/`calc.fdf` fixture is checked in here, unlike `5.1`- `5.4` — this workflow is the one pair in the suite that starts from a bare composition, not an existing structure (Section intro).

11.6.1.9 Running the script

```
bash example_5.6.sh
```

11.6.1.10 What's next

- **`5.1-strain`** / **`5.2-elastic`** / **`5.3-cohesive`** / **`5.4 -phonons`** / **`5.5-convergence`** — once this workflow has told you *which* structure is right, these are the natural next questions about it (mechanical response, stiffness, binding energy, vibrational stability, numerical convergence).
- **`stb-xrd`** (Analysis, code `2.9`) — the single-structure XRD tool this workflow's own `core/xrd.py` pattern-computation code is shared with; useful on its own for simulating/inspecting one structure's pattern without a full candidate search.
- **`stb-mlrelax`** (Structures, code `1.7`) — Section 5's own recommendation for a fast MACE pre-relax pass before committing to a second real-SIESTA relaxation of a borderline candidate like `group_227_2`.

11.6.2 Implementation details

The idea.. A powder pattern constrains the structure strongly, but inverting it is hard. When the composition is known and the space group is not, this workflow tries the candidate groups directly: cast random structures of that composition in each group (Stage 1), relax them with SIESTA (and optionally MACE first), and rank every candidate by the similarity of its simulated pattern to the measured one (Stage 2).

Stage 1 (`stb-xrdsearch`).. For every group in `-groups` it calls **`stb-crystalcast`** with `-species`, `-num-ions` (per conventional cell), `-volume-factor`, `-max-attempts` and `-seed`, producing `-count-per-group` candidates (default 1). With `-ml-rank` the candidates of each group are relaxed (positions only) with MACE and ranked, the folders are named `<group>_rank<N>`, and `-keep-top-per-group N` keeps only the best N. `-mace-relax` then pre-relaxes every kept candidate fully (positions and cell, `-mace-fmax`, `-mace-max-steps`). Each candidate gets its own folder `<output-dir>/<name>/` (default `xrd_search`) with `structure.fdf`, a `calc.fdf` – your template (`-c`) or the relaxation template of **`stb-inputfile`** (with `-d3` and `-s` for dispersion and spin) – and the pseudopotentials (`-p`).

Stage 2 (`stb-xrdrank`).. In every candidate folder the relaxed geometry (`*.STRUCT_OUT`) is used when SIESTA has finished, otherwise the unrelaxed `structure.fdf`; loose `*.fdf` files in the folder are accepted too. For each, the pattern is computed as in **`stb-xrd`** (same wavelength and 2θ range options) and

compared with `-experimental` through PyXtal's similarity (peak lists broadened automatically unless `-raw-experimental`). The ranking table (similarity, space group, formula, relaxed or raw) goes to `-o` (default `xrd_rank.txt`); `-top` limits it; `-save-gnuplot`/`-view` plot the ranking and the best match overlaid on the experiment. Warnings emitted by the libraries during the comparison are collected in a final report section.

Limits.. The search is only as good as the list of groups and the number of candidates per group: random casting can miss the right arrangement within the right group, so use several candidates per promising group. Similarity scores rank candidates for one measurement; a best match still needs a Rietveld refinement against the data before it is a solved structure.

11.6.3 Command-line reference

11.6.3.1 `stb-xrdsearch`

Casts candidate structures across a set of space groups (Stage 1 - Prep). Part of the Structure Solution (XRD) workflow: given a composition and a list of space groups to try, casts `--count-per-group` random candidates in each (by driving `stb-crystalcast` once per group), and puts each candidate in its own folder as '<folder>/structure.fdf'. Optionally pre-relaxes every final candidate with a MACE potential (`--mace-relax`, positions + cell) – a fast cleanup of pyxtal's raw random placement, not a substitute for a real SIESTA relaxation. By default, also generates the standard relaxation `calc.fdf` (same as `stb-inputfile -t relax`) in every folder – use `--calc` to supply your own template instead – and links pseudopotentials if `--pseudo-dir` is given, so every folder is ready for a real SIESTA relaxation as-is. Follow up with `stb-xrdrank` to score every candidate against an experimental XRD pattern (either now, on these raw candidates, or later, on your relaxed .STRUCT_OUT results).

Synopsis.

```
usage: stb-xrdsearch [-h] --species SPECIES [SPECIES ...] --num-ions NUM_IONS
                     [NUM_IONS ...] --groups GROUPS
                     [--count-per-group COUNT_PER_GROUP] [--molecular]
                     [--volume-factor VOLUME_FACTOR] [--max-attempts MAX_ATTEMPTS]
                     [--seed SEED] [--ml-rank] [--keep-top-per-group N] [--mace-relax]
                     [--mace-model {small,medium,large}] [--mace-custom-model PATH]
                     [--mace-custom-model-card JSON] [--mace-device {cpu,cuda}]
                     [--mace-fmax MACE_FMAX] [--mace-max-steps MACE_MAX_STEPS]
                     [-c PATH] [--d3] [-s] [-p PATH_OR_BANK] [--view] [--save-report]
                     [-o OUTPUT_DIR] [-v] [--no-intro]
```

Options

`--species SPECIES [SPECIES ...]`
Element symbols in the structure, e.g. `--species` Ni O. — required

`--num-ions NUM_IONS [NUM_IONS ...]`
Number of atoms of each `--species`, same order and count, e.g. `--num-ions` 4 8. — required; type: int

`--groups GROUPS`
Comma- or space-separated space group numbers to try, e.g. '225,227,229' or '225 227 229'. — required

`--count-per-group COUNT_PER_GROUP`
Random candidates to generate per space group (default: 1). — type: int

`--molecular`
: Passed through to `stb-crystalcast`: treat `--species` as whole rigid molecules instead of bare elements. switch, off by default

`--volume-factor VOLUME_FACTOR`
: Passed through to `stb-crystalcast` (default: 1.1). type: float

`--max-attempts MAX_ATTEMPTS`
: Passed through to `stb-crystalcast` (default: 10). type: int

`--seed SEED`
: Base random seed; each space group's batch uses seed+group so groups don't share identical draws (default: not seeded). type: int

`--ml-rank`
: Passed through to `stb-crystalcast`: quickly relax each candidate with MACE-MP-0 and rank by energy (needs the optional 'ml' extra: pip install stb_suite[ml]) – a fast, local pre-screen, not a substitute for the real SIESTA relaxation. Folders are then named '<group>_rank<N>' (most stable first) instead of by attempt order. switch, off by default

`--keep-top-per-group N`
: Only keep the N most stable candidates per group (requires `--ml-rank`; default: keep all `--count-per-group` candidates). type: int

`--mace-relax`
: Pre-relax every FINAL candidate structure (positions + cell) with a MACE potential before generating `calc.fdf` – a fast heuristic cleanup of pyxtal's raw random placement, not a substitute for the real SIESTA relaxation. The relaxed geometry overwrites each candidate's `structure.fdf`. Distinct from `--ml-rank`, which only relaxes (positions-only, transiently) to RANK/select candidates during casting – the geometry it uses for that is discarded, not kept. Needs the optional 'ml' extra (pip install stb_suite[ml]). switch, off by default

`--mace-model {small,medium,large}`
: MACE-MP-0 model size for `--mace-relax` (default: medium). Ignored if `--mace-custom-model` is given. Only valid together with `--mace-relax`. choices: small, medium, large

`--mace-custom-model PATH`
: Path to a custom fine-tuned .model file for `--mace-relax`, instead of a MACE-MP-0 foundation size. Only valid together with `--mace-relax`.

`--mace-custom-model-card JSON`
: Model card of the `--mace-custom-model` file: the `mlff_model_card.json` that `stb-mlffAnalysis` wrote next to it (training data, level of theory, accuracy, energy scale). Required with `--mace-custom-model`: the card is checked against the model file and used to warn when a structure is outside what the model was trained on.

`--mace-device {cpu,cuda}`
: Device to run the MACE model on (default: cpu) – used for both `--mace-relax` (here) and `--ml-rank` (passed through to the `stb-crystalcast` subprocess). choices: cpu, cuda

`--mace-fmax MACE_FMAX`
: Force convergence threshold for `--mace-relax`, eV/Ang (default: 0.05). type: float

`--mace-max-steps MACE_MAX_STEPS`
: Maximum optimizer steps for `--mace-relax` (default: 200). type: int

`-c, --calc PATH`
: Custom `calc.fdf` template, copied (as 'calc.fdf') into every candidate folder – should %include 'structure.fdf' by name. If omitted (default), the standard relaxation `calc.fdf` – the same default `stb-inputfile -t relax` produces – is auto-generated for every candidate instead (see `--d3`/`--spin-polarized`).

`--d3`
: Auto-generate mode only (`--calc` not given): enable the DFT-D3 (Grimme) van der Waals correction in the

generated `calc.fdf`. Rejected together with `--calc`. switch, off by default

`-s, --spin-polarized`
Auto-generate mode only (`--calc` not given): enable spin polarization in the generated `calc.fdf`. Rejected together with `--calc`. switch, off by default

`-p, --pseudo-dir PATH_OR_BANK`
Pseudopotentials source (optional): a bundled bank (dojo, virtual_vault) or a folder path. If given, the required pseudopotential for every species actually present in the cast candidates is copied into every candidate folder.

`--view`
After generating, open one interactive 3D view (via ASE) paging through every candidate structure. Needs a display. Off by default. switch, off by default

`--save-report`
Also persist the report to <output-dir>/xrdsearch_stage1.txt. Off by default. switch, off by default

`-o, --output-dir OUTPUT_DIR`
Folder to write candidate structures into (default: xrd_search); each candidate gets its own '<output-dir>/<name>/'. Created if it doesn't exist; refuses to run if it already has any content, to avoid mixing batches.

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Usage examples:
  stb-xrdsearch --species Ni O --num-ions 4 8 --groups 225,227,229
  stb-xrdsearch --species Na Cl --num-ions 4 4 --groups 225,221,62 \
      --count-per-group 3 -o candidates --pseudo-dir dojo
  stb-xrdsearch --species Na Cl --num-ions 4 4 --groups 225,221,62 \
      --count-per-group 5 --ml-rank --keep-top-per-group 1
  stb-xrdsearch --species Ni O --num-ions 4 8 --groups 225,227 \
      --calc my_calc_template.fdf --pseudo-dir /path/to/pps --view
  stb-xrdsearch --species Ni O --num-ions 4 8 --groups 225,227 \
      --mace-relax --mace-model medium --pseudo-dir dojo
```

11.6.3.2 `stb-xrdrank`

Ranks candidate structures by similarity to an experimental XRD pattern (Stage 2 - Analysis). Part of the Structure Solution (XRD) workflow: reads every candidate structure in `--input-dir` (default 'xrd_search', `stb-xrdsearch`'s own default output folder – one candidate per subfolder, picking up a relaxed '*.STRUCT_OUT' over the raw '`structure.fdf`' if you've already run SIESTA there), simulates each one's powder XRD pattern, and ranks them by similarity to an experimental pattern – a fast pre-screen for which candidate space group/ arrangement is most likely correct. Run it once on the raw candidates to prioritize which to relax with real DFT, then again on the relaxed results to confirm. Each comparison takes 10s of seconds (pyxtal's Similarity() has no faster mode), so this can take a while for a large `--input-dir`.

Synopsis.

```
usage: stb-xrdrank [-h] [--input-dir INPUT_DIR] --experimental PATH
                   [--raw-experimental] [--wavelength WAVELENGTH]
                   [--two-theta-range MIN MAX] [--top TOP] [-o OUTPUT]
                   [--save-gnuplot] [--view] [--save-report] [-v] [--no-intro]
```

Options

`--input-dir INPUT_DIR`

Folder of candidate structures, one per subfolder (e.g. written by `stb-xrdsearch` as '<subfolder>/structure.fdf') – defaults to 'xrd_search', `stb-xrdsearch`'s own default `--output-dir`, so Stage 2 runs against Stage 1's output with no extra flag in the common case. If a subfolder also has a '*.STRUCT_OUT' file (from running SIESTA there), the relaxed structure is used instead of the raw one. Flat .fdf files directly in `--input-dir` are also accepted.

`--experimental PATH`

Experimental XRD pattern: a plain text file with two columns (2theta intensity), whitespace- or comma-separated, '#' comments and blank lines allowed. If the file looks like a sparse list of discrete peaks (e.g. a literature peak table, or `stb-xrd`'s own stick-pattern output) rather than a continuous scan, it is automatically Gaussian-broadened before comparing – see `--raw-experimental`. required

`--raw-experimental`

Never auto-broaden a peak-list-looking `--experimental` file – compare it exactly as read. Off by default (auto-detection normally handles this correctly; matches `stb-xrd`'s own `--compare-to` flag of the same name). switch, off by default

`--wavelength WAVELENGTH`

X-ray source: a known name (CuKa, CuKa1, MoKa, CrKa, FeKa, CoKa, AgKa, ...) or a wavelength in Ang as a plain number (default: CuKa, 1.54184 Ang).

`--two-theta-range MIN MAX`

2-theta range in degrees to compute (default: 0 90). type: float

`--top TOP`

Only show/plot the N best-matching candidates (default: show all). The full ranking is always written to `--output` regardless. type: int

`-o, --output OUTPUT`

Output ranking table file name (default: `xrd_rank.txt`). Always lists every candidate, regardless of `--top`. Gnuplot/report files (`--save-gnuplot`/`--save-report`) are written alongside it, in the same directory.

`--save-gnuplot`

Also write the ranking as a gnuplot bar chart (`xrd_rank.dat` + `xrd_rank.gplot`) and the best candidate's simulated-vs-experimental pattern overlay (xrd_rank_top_sim.dat/xrd_rank_top_exp.dat + `xrd_rank_overlay.gplot`), all under a 'plot/' subfolder alongside `--output`. Off by default. switch, off by default

`--view`

Show an interactive matplotlib preview (ranking bar chart + best candidate's simulated-vs-experimental overlay, one figure) before finishing. Off by default. switch, off by default

`--save-report`

Also persist the full numbered run report to `xrd_rank_report.txt` (written alongside `--output`). Off by default. switch, off by default

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Usage examples:
  stb-xrdrank --experimental measured.dat
  stb-xrdrank --input-dir candidates --experimental measured.dat \
      --wavelength MoKa --top 5
  stb-xrdrank --experimental measured.dat --save-gnuplot --view --save-report
```

11.7 Hubbard U (Linear Response)

At a glance

Menu code: 5.7 (a multi-stage workflow)

Code	Stage	Command
5.7.1	Stage 1 - Reference Prep	`stb-hubbardu`
5.7.2	Stage 2 - Perturbation Prep	`stb-hubbarduAlphas`
5.7.3	Stage 3 - Analysis	`stb-hubbarduAnalysis`

Guided example: `examples/5.7-hubbardu/`

Summary: Compute a first-principles Hubbard U via Cococcioni & de Gironcoli's linear-response method, ending in a ready-to-use DFT+U fdf block.

11.7.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/5.7-hubbardu/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

Plain DFT (LDA/GGA) systematically gets transition-metal and rare-earth compounds wrong — wrong band gaps, wrong magnetic moments, sometimes even the wrong ground state. The standard fix is a DFT+U correction, but DFT+U needs a number, `U`, and picking it by hand (or copying one from a paper that used a different code, functional, and basis set) is exactly the kind of unverifiable shortcut this suite tries to avoid. `stb-dftu` (example `1.5`) already lets you *use* a literature `U`; this workflow computes a **first-principles** one for *your own* structure, functional, pseudopotential, and basis set, via Cococcioni & de Gironcoli's linear-response method (Phys. Rev. B 71, 035105, 2005).

Unlike every other workflow example in this suite so far, this one has **three** stages, not two: a reference calculation (Stage 1), a set of small perturbations around it (Stage 2), and a fit that turns the perturbed occupations into a number (Stage 3). None of the three run SIESTA for you — each is a real, no-shortcuts folder-generation/analysis tool; you run SIESTA yourself in between. This walkthrough proves the whole chain end to end with a synthetic dataset whose exact answer is known in advance, so you can see precisely what "correct" looks like before trusting it on your own material.

11.7.1.1 Theory

Why plain DFT gets correlated compounds wrong. LDA and GGA approximate the exchange-correlation energy as a (semi-)local functional of the density. For simple metals and covalent semiconductors this is a good approximation. It breaks down for compounds with partially filled, spatially localized `d` or `f` shells (transition-metal oxides, lanthanides, actinides) because of the **self-interaction error**: an electron in LDA/GGA feels part of its own charge through the mean-field Hartree term, an error that isn't cancelled cleanly by the approximate exchange-correlation piece. For delocalized states this error is small; for a tightly localized `d`/`f` orbital it is not, and it shows up as a spurious *curvature* of the total energy as a function of that orbital's occupation — real physics says removing/adding a fraction of an electron from a localized orbital should cost energy in a specific, sometimes discontinuous way (the derivative discontinuity of the exact functional); LDA/GGA instead gives a smooth, convex curve that favours **fractional occupation**, artificially delocalizing electrons that should stay localized. The textbook symptom is NiO/MnO/FeO predicted as metals or small-gap semiconductors with plain GGA, when they are real, wide-gap antiferromagnetic insulators.

DFT+U: penalizing fractional occupation. DFT+U (Anisimov, Zaanen & Andersen, 1991) adds a Hubbard-model-like term that penalizes fractional occupation of the chosen correlated shell and rewards integer occupation (0 or 1 per orbital) — pushing the functional's energy-vs-occupation curve back toward the physically-correct shape. In SIESTA this is the `%block LDAU.proj` mechanism `stb-dftu` (example `1.5`) already builds a snippet for, given a `U` (and `J`) you supply. The correction only ever does its job if `U` is *right* for your exact setup — functional, pseudopotential, and basis set all affect it — which is exactly why a copied literature value is a starting-point sanity check (`stb-dftu --use-reference/--suggest`), never a substitute for computing one from your own data.

The Cococcioni & de Gironcoli linear-response method. Cococcioni & de Gironcoli's insight: `U` is, by definition, the *curvature* of the total energy with respect to the correlated shell's occupation — $U = d^2E/dn^2$. Instead of computing that second derivative directly (hard), they compute it indirectly, from how the shell's **occupation itself** responds to a small external perturbation:

1. Apply a rigid potential shift `alpha` to the correlated shell only (in SIESTA: `%block LDAU.proj` with `LDAU.PotentialShift T`, setting the would-be `U` slot in that block to the perturbation strength `alpha` instead of a real Hubbard correction — the same block, reused as a *diagnostic probe* rather than a correction).
2. Measure how the shell's SCF occupation `n` changes as `alpha` is swept through a handful of small positive and negative values.
3. The slope `dn/dalpha` is a **response function**, `chi`. Two versions of it — self-consistent and frozen-density (next subsection) — combine into `U` via a formula (Section 1.4) derived from exactly this occupation response, without ever computing d^2E/dn^2 directly.

Stage 1 (`stb-hubbardu`) prepares the `alpha=0` reference point; Stage 2 (`stb-hubbarduAlphas`) prepares every other `alpha`; Stage 3 (`stb-hubbarduAnalysis`) fits the slopes and applies the formula.

1.4 Screened (`chi`) vs. bare (`chi0`) response, and the U formula. A perturbation on one shell doesn't just change that shell's own occupation — the rest of the system reorganizes (screens) in response too. Two different response slopes matter, and the difference between them is exactly what isolates the correlation-driven curvature from an ordinary, uninteresting electrostatic response:

- **`chi` — the self-consistent (screened) response**: each perturbed calculation is run to full self-consistency (`scf_alpha_*/reference` folders). This includes the "real" physical response — the rest of the electronic density relaxing around the perturbed shell.
- **`chi0` — the frozen-density (bare) response**: each perturbed calculation is capped at a handful of SCF iterations (`frozen_alpha_*` folders, `MaxSCFIterations` deliberately small) — just enough for the perturbed shell's own occupation to react to `alpha`, before the rest of the density has a chance to relax around it.

`chi` and `chi0` are each fit as the slope of a straight line, occupation `n` vs. perturbation `alpha` (Stage 3, `numpy.polyfit`, degree 1). The final formula:

```
U = 1/chi0 - 1/chi
```

Subtracting the *bare* response's reciprocal from the *screened* response's reciprocal removes exactly the part of the response that isn't a genuine electron-correlation effect, leaving the physically meaningful on-site `U`. Section 6 below fits both branches against a known, hand-constructed occupation-vs-alpha dataset and recovers the exact expected `U`, which is the cleanest way to see this formula actually work before trusting it on noisy, real SCF numbers.

Why every perturbed run needs the exact same starting point. The measured slope is only meaningful if the *only* thing that changes between runs is `alpha` — not the geometry, not the SCF starting point. Two safeguards make this true, both enforced automatically by Stage 1/2, never left to you to remember:

- **Fixed geometry**: every generated folder (reference and every perturbed `alpha`) forces a pure single-point SCF (`MD.TypeOfRun CG`, `MD.Steps 0`, `MD.NumCGsteps 0`, `MD.VariableCell false`) regardless of what your own `calc.fdf` template configures. Letting the ions (or the cell) relax under a rigid, unphysical `alpha` shift would change the geometry between perturbation points — contaminating the measured response with a geometric effect that has nothing to do with the electronic occupation response the method needs.
- **Fixed starting density**: Stage 2 physically copies Stage 1's converged `.DM` file into every perturbed folder and sets `DM.UseSaveDM T`. Without this, each perturbed SCF cycle would start from its own default initial guess and could converge to a slightly different point on the potential-energy surface (especially likely for a correlated, often multi-minima system) — again contaminating the slope with numerical noise unrelated to the real electronic response.

Isolating a single atom: species aliasing. `%block LDAU.proj` perturbs an entire species *label*, not one atom. If the target species appears more than once in the structure (e.g. two equivalent Mn atoms in a supercell), perturbing the species would perturb *both* atoms at the exact same time — measuring a **collective** response of two coupled perturbed sites, not the **single isolated-atom** response the Cococcioni-de Gironcoli supercell method actually requires (the whole point of using a big-enough supercell is that the *other* atoms of the same species stay unperturbed, acting as the "bulk" the one perturbed atom responds against). `stb-hubbardu` handles this by aliasing the one chosen atom (`--atom-index`) onto a new, same-Z species label (`<species>_pert`) behind the scenes — only that aliased label goes into `%block LDAU.proj`; every other atom of the original species is left completely alone. Section 3.3 demonstrates this live, including the `%block ChemicalSpeciesLabel`/coordinates-block edits it makes.

What this workflow simplifies — explicitly. By hand, a linear-response `U` calculation means: hand-writing 9+ nearly identical `calc.fdf` files that differ only in one perturbation-strength number; manually copying a converged `.DM` file into each one and adding `DM.UseSaveDM T`; remembering to cap `MaxSCFIterations` for exactly the "frozen" half of them and not the other half; parsing an `Occupations:` line out of 9+ `.out` files by hand (including the non-obvious factor-of-2 correction for a non-polarized calculation, Section 5.2); fitting two slopes; and finally re-deriving `U = 1/chi0 - 1/chi` correctly. This workflow does all of that mechanically and consistently, and — just as importantly — reports **when the underlying assumption (a clean, linear response) doesn't hold** (Section 5.3), rather than silently handing you a number.

11.7.1.2 Libraries and external dependencies used

- **`numpy`** — the linear fit itself (`numpy.polyfit`, degree 1) for both the `chi` and `chi0` branches, and the R^2 computed from its residuals.
- **`pymatgen`** / **`spglib`** (via `core/symmetry.py`) — only when `--atom-index` is ambiguous (Section 3.3): finds symmetry-inequivalent atoms of the target species and reports which Wyckoff site each candidate `--atom-index` belongs to, so you're not guessing blindly.
- **`matplotlib`** (optional, `--view`) — a live preview of both response branches (points + fitted lines) at Stage 3, so you can eyeball the fit instead of trusting R^2 alone.
- **`gnuplot`** (optional, `--save-gnuplot`) — writes the same response curve as a portable `.dat`/`.gplot` pair under `hubbardu_runs/plot/`, this workflow category's own older plotting convention (matching `stb-strainAnalysis`/`stb-elasticAnalysis`/`stb-convergenceAnalysis`).

- No network access, no ML/MACE dependency anywhere in this workflow — every stage is a pure, local, deterministic file operation or numerical fit.

11.7.1.3 3. Stage 1: reference prep (`stb-hubbardu`, code `5.7.1`)

What it does. Given a structure, a target species, and (optionally) a specific atom index, `stb-hubbardu` writes one folder, `reference/`, containing:

- `structure.fdf` — your structure, copied in as-is (or with one atom's species aliased, Section 3.3).
- `calc.fdf` — your own template, untouched, except for one new line at the very top: `%include config_extra.fdf`.
- `config_extra.fdf` — the DFT+U *perturbation* block (`alpha=0.0` here, the reference point) plus the single-point-SCF-enforcement lines (Section 1.5) — kept in a sidecar file rather than inlined, so nothing in your own template is ever rewritten or duplicated.

It also snapshots the (possibly aliased) structure and the raw `calc.fdf` template as `_template_structure.fdf`/`_template_calc.fdf`, and writes `run_manifest.json` — the single source of truth Stages 2 and 3 both read (species, shell quantum numbers, `J`, the SIESTA `SystemLabel`, and a `runs` dict that grows with every stage).

The single-point-SCF constraint, live.

```
$ stb-hubbardu --species Mn --no-intro
```

```
[2] DFT+U PERTURBATION SETUP
------------------------------------------------------------
SystemLabel            : siesta
Species / shell        : Mn (3d: n=3, l=2)
Perturbation           : alpha=0.0 (reference run)
Forced to single-point SCF via '%include config_extra.fdf'.

config_extra.fdf (written into every generated folder):
  # Auto-generated by stb-hubbardu -- DFT+U perturbation (Cococcioni-de Gironcoli) + single-point SCF
  enforcement.
  LDAU.PotentialShift T
  %block LDAU.proj
  Mn   1
  n=3    2
  0.000    0.000
  0.000    0.000
  %endblock LDAU.proj

  # Forces a pure single-point SCF (no ionic or cell relaxation) -- ...
  MD.TypeOfRun       CG
  MD.Steps           0
  MD.NumCGsteps      0
  MD.VariableCell    false
```

Note `n=3, l=2` — this is Mn's default correlated shell (`3d`), the exact same `core/dftu_data.py::DEFAULT_SHELL/SHELL_NAMES` tables `stb-dftu` (example `1.5`) already uses, so a shell picked automatically here means the same thing it would mean there. `reference/calc.fdf` itself is just:

```
%include config_extra.fdf

SystemLabel siesta
%include structure.fdf
...
```

— your template, byte-for-byte, with one new line prepended.

3.3 Multi-atom species and `--atom-index`, live. `structure_2mn.fdf` in this folder is the conventional (2-atom) bcc-Mn cell — space group `Im-3m` (No. 229), both atoms symmetry-equivalent. Asking to perturb `Mn` here without saying *which* one is a hard error, not a guess:

```
$ stb-hubbardu -s structure_2mn.fdf --species Mn --no-intro
```

```
[1] ATOM SELECTION
------------------------------------------------------------
Total atoms in structure : 2
Atom(s) of 'Mn' : 2 found (index(es) 1, 2)
[FAIL] --species 'Mn' appears 2 times in 'structure_2mn.fdf' -- the
linear-response perturbation must be isolated to a single atom (perturbing
every atom of the species at once measures a collective response, not the
isolated single-site response the method requires). Pass --atom-index to
pick one:
Space group: Im-3m (No. 229)
--atom-index | Wyckoff | Note
1            | a       | 2 symmetry-equivalent atom(s) -- any one of them gives the same result
```

Passing `--atom-index 1` resolves it — and aliases just that one atom:

```
$ stb-hubbardu -s structure_2mn.fdf --species Mn --atom-index 1 --no-intro
```

```
Perturbed atom : atom #1 of 'Mn', isolated via alias species 'Mn_pert'
(same Z, own pseudopotential) -- the other atom(s) of 'Mn' are left
unperturbed.
```

and the generated `reference/structure.fdf` shows exactly what changed — one new species label, same atomic number, and only atom #1 repointed to it:

```
NumberOfSpecies 2
%block ChemicalSpeciesLabel
1 25 Mn
 2   25   Mn_pert
%endblock ChemicalSpeciesLabel
...
%block AtomicCoordinatesAndAtomicSpecies
  0.00   0.00   0.00   2
0.50 0.50 0.50 1
%endblock AtomicCoordinatesAndAtomicSpecies
```

Atom #1 (originally species index `1`, i.e. `Mn`) now points at species index `2` (`Mn_pert`); atom #2 is untouched, still plain `Mn`. `%block LDAU.proj` in `config_extra.fdf` now perturbs `Mn_pert` only — exactly the isolated single-site perturbation Section 1.6 explained, made concrete.

Pseudopotentials, live. `-p/--pseudo-dir` accepts a bundled bank (same `core/pseudopotentials.py` banks every other tool in this suite shares) or a custom folder, and only copies the species your structure actually needs:

```
$ stb-hubbardu --species Mn --pseudo-dir dojo --no-intro
```

```
[4] PSEUDOPOTENTIALS
------------------------------------------------------------
[OK] Copied 1 pseudopotential file(s) from '.../pseudopotentials/dojo'
into 'hubbardu_runs/reference' (and saved for stb-hubbarduAlphas to reuse)
```

The snapshot it saves is what lets Stage 2 copy the same pseudopotential into every `scf_alpha_*/frozen_alpha_*` folder too, without you having to pass `--pseudo-dir` a second time.

Report structure.

```
[0] RUN METADATA
[1] ATOM SELECTION
[2] DFT+U PERTURBATION SETUP
[3] REFERENCE FOLDER & TEMPLATE SNAPSHOT
[4] PSEUDOPOTENTIALS
[5] SUMMARY & NEXT STEPS
```

`--save-report` persists this exact text to `hubbardu_runs/hubbardu_stage1.txt`; no `references.bib` is written by any of the three tools in this workflow (unlike `stb-inputfile`/`stb-kgrid`/ `stb-kpath`) — the Cococcioni & de Gironcoli citation is instead printed directly in Stage 3's `[3] COMPUTED U` section, right next to the number it justifies.

Running it both ways. A — direct CLI: every command above.

B — interactive `stb-suite` menu:

```
stb-suite
# at the main prompt, type: 5.7.1
```

`5.7.1` asks for the structure file, the calc.fdf template, the species, an optional atom index, an optional shell override, `J`, a pseudopotential source (bundled bank / custom path / skip), the output directory, and finally whether to save a report — the exact same flags as the CLI, in the exact same order. `example_5.7.sh` proves both paths agree.

11.7.1.4 4. Stage 2: perturbation prep (`stb-hubbarduAlphas`, code `5.7.2`)

What it does. Reads `reference/` and `run_manifest.json` from Stage 1, and writes one folder per perturbation strength in `--alphas` (default `-0.10 -0.05 0.05 0.10`, eV):

- `scf_alpha_<v>/` — full self-consistency, for the **screened** (`chi`) branch.
- `frozen_alpha_<v>/` — `MaxSCFIterations` capped (`--frozen-iterations`, default `1`), for the **bare** (`chi0`) branch — one per `alpha` **plus** `alpha=0.0`, evaluated by the exact same frozen recipe as every other point (not just re-using `reference`'s own fully-converged value), so every point on the frozen-response line is mutually consistent with every other.

Auto-copying the reference DM, live.

```
$ stb-hubbarduAlphas --dir hubbardu_runs --no-intro
```

```
[1] VALIDATION & REFERENCE CHECK
------------------------------------------------------------
Species / shell        : Mn (n=3, l=2), J=0.000 eV
Reference DM           : hubbardu_runs/reference/siesta.DM (found, will be copied into every folder)
Template snapshot      : OK (no pre-existing DFT+U block)
```

If `reference/siesta.DM` doesn't exist yet, Stage 2 refuses to proceed — by design: without it, every perturbed folder would start from its own independent SCF guess, defeating the entire point of Section 1.5's fixed-starting-density requirement. (This walkthrough hasn't actually run SIESTA, so a placeholder file stands in for it below — see Section 6 for why that's fine for proving the mechanics and the analysis math, but not a substitute for a real converged density.)

4.3 `scf_*` vs. `frozen_*` folders.

```
$ cat hubbardu_runs/scf_alpha_0.0500/config_extra.fdf
```

```
LDAU.PotentialShift T
%block LDAU.proj
Mn   1
n=3    2
0.050    0.000
0.000    0.000
%endblock LDAU.proj
DM.UseSaveDM T
MD.TypeOfRun       CG
...
```

```
$ cat hubbardu_runs/frozen_alpha_0.0500/config_extra.fdf
```

```
LDAU.PotentialShift T
%block LDAU.proj
Mn   1
n=3    2
0.050    0.000
0.000    0.000
%endblock LDAU.proj
MaxSCFIterations 1
DM.UseSaveDM T
MD.TypeOfRun       CG
...
```

Identical perturbation, identical starting density (`DM.UseSaveDM T`) — the *only* difference is `MaxSCFIterations 1` in the frozen folder. That one line is the entire mechanism separating the bare response from the screened one: one SCF iteration is barely enough for the perturbed orbital's own occupation to react, nowhere near enough for the rest of the density to relax around it.

Report structure.

```
[0] RUN METADATA
[1] VALIDATION & REFERENCE CHECK
[2] PSEUDOPOTENTIALS
[3] PERTURBATION FOLDERS
[4] SUMMARY & NEXT STEPS
```

Running it both ways. A — direct CLI: `stb-hubbarduAlphas --dir hubbardu_runs`.

B — interactive `stb-suite` menu:

```
stb-suite
# at the main prompt, type: 5.7.2
```

`5.7.2` asks for the stage-1 directory, the perturbation strengths (space-separated, blank = default), the frozen-iteration cap, and whether to save a report. `example_5.7.sh` proves both paths agree.

11.7.1.5 5. Stage 3: analysis (`stb-hubbarduAnalysis`, code `5.7.3`)

Report structure.

```
[0] RUN METADATA
[1] READING RUNS
[2] LINEAR RESPONSE FIT
[3] COMPUTED U
[4] OUTPUT FILES
[5] SUMMARY & NEXT STEPS
[6] LIBRARY WARNINGS
```

`[1]` prints one table row per folder (kind, alpha, parsed occupation, SCF status); `[2]` the two fitted slopes/intercepts/R^2; `[3]` the formula and the final `U`, plus a same-element literature-value comparison (from `core/dftu_data.py::REFERENCE_U`, the same table `stb-dftu --list-reference/--suggest` expose) as an *informational* sanity check only — never a validation, since that table is itself just a Materials-Project-tabulated GGA+U convenience value for oxides, not a ground truth for your exact functional/pseudopotential/basis.

The non-polarized occupation-doubling fix. SIESTA's own `Occupations:` line means something different depending on whether the calculation is spin-polarized. For a **non-polarized** (`nspin=1`) run, SIESTA prints only the *per-spin-channel* occupation — literally half the real total, because the code internally divides by `(3 - nspin)`. `stb-hubbarduAnalysis` detects this from the *shape* of the parsed line (exactly 2 numbers, vs. 3+ for a polarized run) and doubles it back:

```
total = float(nums[-1])
if len(nums) == 2:
    total *= 2.0
```

This was verified against a real non-polarized Mn run: a raw printed value of `~2.47` is the correct `~4.94` total once doubled — silently trusting the raw number would have measured a response curve at roughly *half* the real occupation, propagating straight into a wrong `chi/chi0` and hence a wrong `U`. The worked example in Section 6 deliberately uses a 3-number (already-total) format specifically so its analytic ground truth stays simple to verify by hand; a real non-polarized SIESTA run would hit this doubling path automatically and correctly.

Sanity checks: R^2, intercept deviation, and sign checks. A single number, `U`, can look confident and still be meaningless if the occupation-vs-`alpha` data isn't actually linear. Three independent checks guard against that, all demonstrated live in Section 6.3:

- **R^2 per branch** (default tolerance `0.98`, `--r2-tolerance`): warns when either fit doesn't explain the data well — usually a sign the `--alphas` range in Stage 2 was too wide for the linear regime.
- **Intercept deviation**: each fit's *intercept* (extrapolated occupation at `alpha=0`) is compared against the occupation *actually measured* at the real `alpha=0` point; a difference above `0.01` warns that the response may not be as linear as R^2 alone suggests — R^2 can look deceptively good even when the intercept has drifted.
- **Sign / negativity checks**: `U` is a repulsive on-site term and must be positive; a negative result, or `chi`/`chi0` with opposite signs, is flagged as a strong indicator of a bad fit rather than a real (if surprising) physical result.

5.4 `[6] LIBRARY WARNINGS.` `numpy.polyfit` silently emits a `RankWarning` when the fit is badly conditioned (near-duplicate `alpha` values, or too few points for the requested degree). Rather than letting that warning interleave with the report mid-print — confusing at best — `stb-hubbarduAnalysis` traps it via `core/cli.py::capture_library_noise` and reports it in its own dedicated, always-present final section, `No library warnings.` when the fit was well-conditioned (as in every case in this walkthrough).

Running it both ways. A — direct CLI: `stb-hubbarduAnalysis --dir hubbardu_runs`.

B — interactive `stb-suite` menu:

```
stb-suite
# at the main prompt, type: 5.7.3
```

`5.7.3` asks for the stage-1/2 directory, the SIESTA output filename inside each folder, the R^2 tolerance, an optional output filename, then three yes/no questions (`--save-gnuplot`, `--view`, `--save-report`). `example_5.7.sh` proves both paths agree.

11.7.1.6 Worked example: a known analytic ground truth, end-to-end

Real SIESTA output isn't available inside this walkthrough (there's no SIESTA binary to invoke), so — exactly like `stb-eosAnalysis`'s own verification against a synthetic exact Birch-Murnaghan curve — Section 6 fabricates `calc.out` files whose occupation-vs-`alpha` response follows an **exact, hand-chosen linear law**, so the correct answer is known *before* running Stage 3, not just plausible after the fact:

- Reference occupation `N0 = 5.0`.
- Self-consistent (screened) slope `chi = 0.30` 1/eV: `n(alpha) = 5.0 + 0.30 * alpha`.
- Frozen-density (bare) slope `chi0 = 0.20` 1/eV: `n(alpha) = 5.0 + 0.20 * alpha`.
- Expected `U = 1/chi0 - 1/chi = 1/0.20 - 1/0.30 = 5.0 - 3.333333 = 1.666667 eV`.

The script writes these nine occupation values (reference + 4 `scf_alpha_*`

- 5 `frozen_alpha_*`, matching Stage 2's own default `--alphas`) as real `Occupations: x x TOTAL` lines into the exact folders Stage 1/2 just generated, then runs Stage 3 for real:

```
[2] LINEAR RESPONSE FIT
------------------------------------------------------------
Branch                    | chi (1/eV) | Intercept | R^2
```

```
Self-consistent (screened) | 0.300000   | 5.000000  | 1.0000
Frozen-density (bare)      | 0.200000   | 5.000000  | 1.0000

[3] COMPUTED U
------------------------------------------------------------
Formula                : U = 1/chi0 - 1/chi (Cococcioni & de Gironcoli, PRB 71, 035105, 2005)
Computed U             : 1.6667 eV
```

Both fits recover the exact input slopes (R^2 = 1.0000, since the data is exactly linear by construction) and `U` matches the hand-computed `1.666667 eV` to the 4th decimal — confirming the entire pipeline (folder scan → occupation parsing → linear fit → `U` formula) end to end, the same way `stb-eosAnalysis`'s own synthetic-EOS verification confirms *its* fitting pipeline.

6.1 What a bad fit looks like: negative `U`. Swapping which branch is steeper (`chi = 0.20`, `chi0 = 0.30` — the bare response now *larger* than the screened one) reproduces the exact same formula honestly, and it comes out negative:

```
[3] COMPUTED U
------------------------------------------------------------
[WARNING] Computed U is NEGATIVE -- the Hubbard U is a repulsive on-site
term and should always be positive. This strongly suggests a bad fit or
bad data rather than a real result.
Computed U             : -1.6667 eV
```

Nothing here is a bug: the arithmetic is correct, and the tool tells you, plainly, that a negative on-site repulsion isn't physical — a real material would never converge to this sign combination in practice.

What a bad fit looks like: low R^2 and intercept drift. Replacing just one of the five `scf_alpha_*` occupations with an outlier (everything else left exactly on the same line as Section 6's clean dataset) triggers both remaining safety nets at once:

```
[2] LINEAR RESPONSE FIT
------------------------------------------------------------
Self-consistent (screened) | 0.870000   | 4.943000  | 0.2497
[WARNING] The self-consistent response fit has R^2=0.2497, below the 0.98
tolerance -- the perturbation range may be too wide for the linear regime.
[WARNING] The self-consistent fit's intercept (4.943000) differs from the
actually-measured alpha=0 occupation (5.000000) by more than 0.01 -- the
response may not be as linear as the R^2 alone suggests.
```

One bad SCF point (a common real-world failure: a run that converged to the wrong local minimum, or simply didn't converge) is enough to wreck the fit — exactly why Stage 3 checks this instead of reporting `chi/U` unconditionally.

11.7.1.7 Known, deliberate limitations

- **No SIESTA run is ever performed by this suite.** All three stages generate/read input and output files; you run SIESTA in `reference/` and every `scf_alpha_*/frozen_alpha_*` folder yourself, in between stages, exactly as the real workflow requires.
- **`J` is fixed for the whole workflow, not fit.** Only `U` comes out of the linear-response fit; `J` (default `0.0`) is whatever you passed to Stage 1 throughout.

- **One species/one atom per run.** Computing `U` for two different species (e.g. a spinel with two transition-metal sites) means running the whole three-stage workflow twice, independently, into two separate `--output-dir`/`--dir` folders.
- **The `REFERENCE_U` comparison in `[3] COMPUTED U` is informational only** — a literature GGA+U oxide value from a different code, functional, pseudopotential, and basis set is a sanity check on the right order of magnitude, never a validation of your own computed value.
- **The linear-response approximation itself assumes small `alpha`.** Section 6.2 shows what happens when a single point is inconsistent with linearity; a genuinely too-wide `--alphas` range (Section 1.4) would show the same symptom (low R^2) even with otherwise-perfect SCF convergence in every folder.

11.7.1.8 Step-by-step: running this workflow on your own structure

1. Have a structure (`.fdf`) and a working `calc.fdf` template — one that does **not** already configure `%block LDAU.proj`/`DFTU.*` (Stage 1 refuses to proceed on a template that does, to avoid silently stacking perturbations on top of an existing DFT+U setup).
2. Run Stage 1: `stb-hubbardu --species <El> [--atom-index N] [--shell 3d|4d|5d|4f|5f] [--j <val>] [--pseudo-dir <bank-or-path>]`. If your species appears more than once, follow the printed Wyckoff-site table to choose `--atom-index`.
3. Run SIESTA yourself inside `hubbardu_runs/reference/`. Confirm it converges (writes a `.DM` file) — if it doesn't, `stb-hubbardu --help` itself lists concrete `calc.fdf` settings to try (a lower `DM.MixingWeight`, higher `DM.NumberPulay`, looser `SCF.H.Tolerance`).
4. Run Stage 2: `stb-hubbarduAlphas --dir hubbardu_runs [--alphas ...] [--frozen-iterations N]`.
5. Run SIESTA yourself inside every generated `scf_alpha_*/` and `frozen_alpha_*/` folder.
6. Run Stage 3: `stb-hubbarduAnalysis --dir hubbardu_runs [--save-gnuplot] [--view] [--save-report]`. Read `[2]`/`[3]` for any `[WARNING]` before trusting the computed `U` — don't just read the final number.
7. Copy the `%block LDAU.proj` from the written `<species>_LDAU.fdf` into your **production** `calc.fdf` (this file has no `LDAU.PotentialShift` line — it's the real correction, not another perturbation probe).

11.7.1.9 Files in this folder

File	Purpose
`structure.fdf`	Single-atom simplified bcc-Mn cell — the main Stage 1/2 mechanics demo. Not a physically converged structure, purely for exercising the tools.
`structure_2mn.fdf`	Conventional 2-atom bcc-Mn cell (space group `Im-3m`), both atoms symmetry-equivalent — demonstrates the `--atom-index`/species-aliasing requirement (Section 3.3).
`calc.fdf`	Shared `calc.fdf` template for both structures above (non-polarized, modest basis/mesh — mechanics only).
`example_5.7.sh`	The guided walkthrough (**not** an automated test — see `test/5-workflow/7-hubbardu/{prep,alphas,analysis}/test.sh` for that). Pauses between sections so you can read before moving on; safe to re-run.
`output/`	Created by `example_5.7.sh` when you run it (git-ignored, not checked in). See below.

11.7.1.10 Running the script

```
./example_5.7.sh
```

Case	Command(s)	What it shows
`output/stage1/`	`stb-hubbardu --species Mn -o output/stage1`	Stage 1 mechanics, single-atom structure
`output/stage1_2mn/`	`stb-hubbardu -s structure_2mn.fdf --species Mn [--atom-index 1] -o output/stage1_2mn`	`--atom-index`/aliasing, both the ambiguous-error and resolved paths
`output/stage1_pp/`	`stb-hubbardu --species Mn --pseudo-dir dojo -o output/stage1_pp`	pseudopotential bank copy
`output/workflow/`	Stage 1 → Stage 2 → (synthetic `calc.out` injection) → Stage 3	the full 3-stage chain, known-answer worked example (Section 6)
`output/workflow_badU/`, `output/workflow_noisy/`	same chain, deliberately inverted/noisy occupations	the warning paths (Section 6.1/6.2)
(no folder — a diff only)	Stages 1/2/3 via `printf ... | stb-suite`	proof the interactive menu (`5.7.1`/`5.7.2`/`5.7.3`) agrees with the CLI

11.7.1.11 What's next

- **`1.5-stb-dftu`** — once you have a computed `U`, this is what you'd reach for again to regenerate the exact same `%block LDAU.proj` snippet from that number for a *different* structure/species combination, or to compare it against the literature `REFERENCE_U` table this workflow's own Stage 3 already checks against informally.
- **`1.1-stb-inputfile`** — the production `calc.fdf` this `U` ultimately belongs in; paste the `%block LDAU.proj` from `<species>_LDAU.fdf` straight into it (no `LDAU.PotentialShift` line to remove — Stage 3 already omits it).
- Every other `4.x` workflow example in this suite generates real candidate/perturbed geometries and expects a real SIESTA run in between stages, the same two-way (direct CLI / interactive menu) split, and the same numbered-report convention — if this is your first workflow example, `5.1-strain` is the shortest two-stage one to start with.

11.7.2 Implementation details

Linear-response U

Following Cococcioni and de Gironcoli [41], a rigid potential shift α is applied to the localized shell of one atom I, and the change of that shell's occupation n_I is measured twice: *self-consistently* (the electrons screen the perturbation), $\chi = dn_I/d\alpha$, and *bare* (the response before any self-consistent rearrangement, from the first Kohn–Sham step starting from the converged unperturbed density), $\chi_0 = dn_I/d\alpha$. The Hubbard parameter that removes the spurious curvature of the DFT energy with respect to n_I is

$$U = \chi_0^{-1} - \chi^{-1}.$$

Both responses are slopes of $n_I(\alpha)$ obtained from a few small α of both signs.

Stage 1 (`stb-hubbardu`).. Writes `reference/` from your structure and template: a single-point SCF (forced through `config_extra.fdf`, so every later run uses exactly this geometry) with a `%block LDAU.proj` for the perturbed shell and `LDAU.PotentialShift T` at $\alpha = 0$ – with that flag SIESTA treats the value in the block as a rigid shift and prints the shell occupation, which the analysis needs. The shell defaults to the standard one of the element (`-shell`). A template that already contains a DFT+U block is refused, because the two would conflict. SIESTA applies DFT+U to whole species, so when the species has several atoms the perturbed one (`-atom-index`, 1-based) is given its own species label `<El>_pert` (same element and

pseudopotential, copied under the new name); the structure snapshot and a `run_manifest.json` record the setup for the next stages.

Stage 2 (`stb-hubbarduAlphas`).. After the reference has run, it writes for every α in `-alphas` (default $-0.10, -0.05, +0.05, +0.10$ eV) two folders, both starting from a copy of the reference density matrix (`DM.UseSaveDM`) and both with `LDAU.PotentialShift T`, which makes SIESTA treat the value in `LDAU.proj` as the rigid shift α:

- `scf_alpha_<a>/` – converged normally: gives χ;
- `frozen_alpha_<a>/` – stopped after `-frozen-iterations` (default 1) SCF step(s) through `MaxSCFIterations`: gives χ_0.

Stage 3 (`stb-hubbarduAnalysis`).. It reads the occupation of the perturbed shell from every log (`-file`, default `calc.out`) – doubling it for non-spin-polarized runs, where SIESTA reports one spin channel – fits $n(\alpha)$ linearly for both sets (including the reference point $\alpha = 0$), and reports χ, χ_0, their R^2 (warning below `-r2-tolerance`, default 0.98), the deviation of each intercept from the reference occupation, sign checks (opposite signs of χ and χ_0, or a negative U, indicate a broken calculation), and U with a ready-to-use `%block LDAU.proj` (`-o` saves it). The literature value of the element (**`stb-dftu`** table) is shown for comparison only. Plots: occupation against α for both responses (`-save-gnuplot`, `-view`).

Caveats.. The U obtained belongs to SIESTA's projectors and your basis; it is not transferable to plane-wave codes. Responses must be linear: keep $|\alpha|$ small and check R^2. A supercell large enough that the perturbed atom does not see its own images is needed for a converged value; and a self-consistent U (recomputing with the obtained U included) is beyond this workflow.

11.7.3 Command-line reference

11.7.3.1 `stb-hubbardu`

Stage 1 of 3: generates the 'reference' folder for a Hubbard U linear-response calculation (Cococcioni & de Gironcoli, Phys. Rev. B 71, 035105, 2005). Workflow: (1) this tool generates 'reference/' – run SIESTA in it yourself; (2) once it's converged, run `stb-hubbarduAlphas` to generate the perturbed 'scf_alpha_*'/'frozen_alpha_*' folders (it copies the reference .DM in for you); (3) run SIESTA in those, then `stb-hubbarduAnalysis` to compute U. If 'reference/' doesn't converge: DFT+U SCF on correlated oxides is often slow/oscillatory with default settings. Things worth trying in your own `calc.fdf` template: a lower DM.MixingWeight (~0.05), a higher DM.NumberPulay (~6), and a looser SCF.H.Tolerance (~0.01 eV). If `--species` appears more than once in `--structure`, `--atom-index` is required: SIESTA's %%block LDAU.proj perturbs an entire species label at once, so with no `--atom-index` every atom of that species would be perturbed simultaneously – measuring a collective response instead of the single isolated-atom response the Cococcioni-de Gironcoli supercell method requires. When needed, the chosen atom is isolated onto its own species label (same Z, own pseudopotential file) behind the scenes; `--species` stays the real chemical species everywhere else (the final recommended %%block LDAU.proj, the REFERENCE_U sanity check, ...).

Synopsis.

```
usage: stb-hubbardu [-h] [-s STRUCTURE] [-c CALC] --species SPECIES [--atom-index N]
                    [--shell {3d,4d,4f,5d,5f}] [--j J] [-p DIR] [-o OUTPUT_DIR]
                    [--save-report] [-v] [--no-intro]
```

Options

`-s, --structure STRUCTURE`
Input `structure.fdf`, copied as 'structure.fdf' into every run folder (default: `structure.fdf`).

`-c, --calc CALC`
Existing `calc.fdf` template (default: `calc.fdf`), with %include `structure.fdf` and everything else already configured; the DFT+U perturbation lines (and single-point SCF enforcement) are added via a `config_extra.fdf` %include, not embedded inline.

`--species SPECIES`
Species label to apply the Hubbard U correction to (must be present in `--structure`). required

`--atom-index N`
1-based index (counting ALL atoms in `--structure`'s coordinate block, in file order – same convention as `stb-defect --index`) of the specific atom to perturb. Required only if `--species` appears more than once in `--structure`; omit it when `--species` matches a single atom. type: int

`--shell {3d,4d,4f,5d,5f}`
Correlated shell (3d/4d/5d/4f/5f). Default: the standard shell for `--species` (transition metals -> (n)d, lanthanides -> 4f, actinides -> 5f). choices: 3d, 4d, 4f, 5d, 5f

`--j J`
Exchange J (eV), fixed across the whole workflow (default: 0.0). type: float

`-p, --pseudo-dir DIR`
Where to get the pseudopotential files (.psf/.psml) SIESTA needs – a bundled bank (dojo, virtual_vault) or a folder path. Only the species actually present in `--structure` are copied into 'reference/' now, and into every scf_alpha_*/frozen_alpha_* folder later by `stb-hubbarduAlphas`. If omitted, you must copy them into every run folder yourself.

`-o, --output-dir OUTPUT_DIR`
Output directory for the run folders (default: hubbardu_runs).

`--save-report`
Also persist the report to <output-dir>/hubbardu_stage1.txt. Off by default. switch, off by default

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Usage example:
  stb-hubbardu --species Mn
  stb-hubbardu --species Fe --shell 3d --j 0.5
  stb-hubbardu --species Mn --pseudo-dir ./pseudos
  stb-hubbardu --species Mn --pseudo-dir dojo --save-report
  stb-hubbardu --species Mn --atom-index 1  # Mn appears twice in structure.fdf
```

11.7.3.2 `stb-hubbarduAlphas`

Stage 2 of 3: generates the perturbed run folders for a Hubbard U linear-response calculation. Reads the 'reference/' folder and template snapshot written by `stb-hubbardu` (stage 1) – 'reference/' must already be converged, since its .DM is copied into every generated folder here. Writes one 'scf_alpha_<v>' folder per perturbation strength (full self-consistent response) and one 'frozen_alpha_<v>' folder per strength INCLUDING v=0 (single-iteration, frozen-density response) – the v=0 frozen point is evaluated

by the exact same recipe as the others rather than reusing reference's own fully-converged value, so every point on the frozen-response line is mutually consistent. Also copies in any pseudopotentials `stb-hubbardu` was given via `--pseudo-dir`. Doesn't run SIESTA itself – run each folder's calculation yourself, then use `stb-hubbarduAnalysis`.

Synopsis.

```
usage: stb-hubbarduAlphas [-h] [--dir DIR] [--alphas ALPHAS [ALPHAS ...]]
                          [--frozen-iterations FROZEN_ITERATIONS] [--save-report] [-v]
                          [--no-intro]
```

Options

`--dir DIR`
: Directory containing stage 1's 'reference/' folder and manifest – defaults to 'hubbardu_runs', `stb-hubbardu`'s own default `--output-dir`, so stage 2 runs against stage 1's output with no extra flag in the common case.

`--alphas ALPHAS [ALPHAS ...]`
: Perturbation strengths in eV (default: [-0.1, -0.05, 0.05, 0.1]). type: float

`--frozen-iterations FROZEN_ITERATIONS`
: MaxSCFIterations for the frozen-density runs (default: 1; raise this only if `stb-hubbarduAnalysis` finds nothing to parse in those folders). type: int

`--save-report`
: Also persist the report to <dir>/hubbardu_stage2.txt. Off by default. switch, off by default

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Usage example:
  stb-hubbarduAlphas --dir hubbardu_runs
  stb-hubbarduAlphas --dir hubbardu_runs --alphas -0.2 -0.1 0.1 0.2
  stb-hubbarduAlphas --dir hubbardu_runs --save-report
```

11.7.3.3 `stb-hubbarduAnalysis`

Stage 3 of 3: analyzes a `stb-hubbardu` linear-response sweep and computes the Hubbard U. Fits the self-consistent (screened) and frozen-density (bare) occupation responses to the perturbation strength, then applies the standard Cococcioni & de Gironcoli formula: U = 1/chi0 - 1/chi (Phys. Rev. B 71, 035105, 2005). Reports a convergence summary, both fits' R^2 and intercepts (flagging a fit whose zero point drifted from what was actually measured there), and warns if chi/chi0 have opposite signs or U comes out negative – all signs of a bad fit rather than a real result. Writes a ready-to-use %%block LDAU.proj snippet with the computed U, plus (opt-in) a gnuplot data+script pair and/or a live matplotlib preview to inspect the fit visually instead of trusting R^2 alone.

Synopsis.

```
usage: stb-hubbarduAnalysis [-h] [--dir DIR] [--file FILE]
                            [--r2-tolerance R2_TOLERANCE] [-o OUTPUT] [--save-gnuplot]
                            [--view] [--save-report] [-v] [--no-intro]
```

Options

`--dir DIR`
: Directory containing the run folders and `run_manifest.json` written by `stb-hubbardu/stb-hubbarduAlphas` (stages 1/2) – defaults to 'hubbardu_runs', their own default `--output-dir/--dir`, so stage 3 runs against their output with no extra flag in the common case.

`--file FILE`
: SIESTA output filename inside each run folder (default: `calc.out`).

`--r2-tolerance R2_TOLERANCE`
: Warn if either fit's R^2 falls below this (default: 0.98) – a poor linear fit means the perturbation range is probably too wide. type: float

`-o, --output OUTPUT`
: Output .fdf snippet filename (default: `<species>_LDAU.fdf`).

`--save-gnuplot`
: Also write the response curve as a gnuplot data+script pair (<species>_hubbardu_response.dat/.gplot), under a 'plot/' subfolder inside `--dir`. Off by default. switch, off by default

`--view`
: Show an interactive matplotlib preview of the response curve (both branches + fitted lines) before finishing. Off by default. switch, off by default

`--save-report`
: Also persist the report to <dir>/hubbardu_stage3.txt. Off by default. switch, off by default

Like every STB command it also accepts `-h/-help`, `-v/-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Usage example:
  stb-hubbarduAnalysis --dir hubbardu_runs --file calc.out
  stb-hubbarduAnalysis --dir hubbardu_runs --save-gnuplot --view --save-report
```

11.8 Adsorption

At a glance

Menu code: 5.8 (a multi-stage workflow)

Code	Stage	Command
5.8.1	Stage 1 - Prep	`stb-adsorb`
5.8.2	Stage 2 - BSSE Prep	`stb-adsorbBsse`
5.8.3	Stage 3 - Analysis	`stb-adsorbAnalysis`
5.8.4	Stage 4 - Gibbs Free Energy	`stb-adsorbGibbs`

Guided example: `examples/5.8-adsorption/`

Summary: Prepare a clean slab, an isolated adsorbate, and candidate adsorption sites, generate a BSSE-corrected reference at each site's relaxed geometry, then compute the real DFT adsorption energy per site.

11.8.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/5.8-adsorption/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

Computing a real DFT adsorption energy means comparing three numbers — the combined slab+adsorbate system, the bare slab, and the isolated adsorbate — each from its own SIESTA calculation:

```
E_ads = E_site - E_clean_slab - E_adsorbate
```

Getting an honest number out of that comparison depends on a handful of details that are easy to get subtly wrong by hand: whether the cell is allowed to relax, whether the slab's own periodic images interact with the adsorbate, and whether the basis-set mismatch between the three separate calculations is corrected for. This item has **four** stages:

- **Stage 1 (`stb-adsorb`, code `5.8.1`)** generates every folder the uncorrected `E_ads` needs (`clean_slab/`, `adsorbate*/`, one folder per candidate `sites/site_*/`), with the fixed-cell/self-interaction details below already handled correctly.
- **Stage 2 (`stb-adsorbBsse`, code `5.8.2`)** generates the Boys-Bernardi counterpoise (BSSE) correction — but only once Stage 1's own sites have actually finished relaxing in SIESTA, since the correction is only meaningful at the real, relaxed geometry (Section 2).
- **Stage 3 (`stb-adsorbAnalysis`, code `5.8.3`)** reads everything Stage 1/2 produced and reports `E_ads`, with and without the BSSE correction, ranked most stable first — plus a per-site physical-diagnostics table (dipole, magnetic moment, bond-length change), a "what to run next" pointer at other tools in the suite, and an opt-in `--compute-gibbs` step that preps the vibrational-Hessian folders Stage 4 needs (Section 13).
- **Stage 4 (`stb-adsorbGibbs`, code `5.8.4`)** turns those Hessian folders, once you've run SIESTA in them, into an actual Gibbs free energy of adsorption, `DG(T) = E_ads + DZPE - T*DS` (Section 13).

`example_5.8.sh` is a guided, runnable walkthrough of all four stages — **including a worked numeric example (Section 8) where the BSSE correction changes which site you'd conclude is the most stable one, a second one (Section 13.4) where accounting for entropy turns a favorable adsorption unfavorable above room temperature, and a third one (Section 13.6/13.7) that turns that same `DG(T)` sweep into a rough desorption *timescale* — and shows the degenerate case where that timescale estimate stops being informative at all.** Those are the three most important things to take from this page: an uncorrected adsorption-energy ranking is not just "a little too negative," it can point at the wrong site outright; even a BSSE-corrected `E_ads` is only the electronic part of the story, not the full thermodynamic answer; and even `DG(T)` itself only tells you *whether* something desorbs, not *how fast* — a question Stage 4's `[3c]` section answers only very roughly, and sometimes (Section 13.7) not usefully at all.

11.8.1.1 Why the cell stays fixed

Every folder `stb-adsorb`/`stb-adsorbBsse` write (`clean_slab/`, `adsorbate*/`, every `sites/site_*/`, and the BSSE ghost-fragment folders) gets a `config_extra.fdf` sidecar forcing `MD.VariableCell false`, `%include`-d ahead of your own `--calc` template (same sidecar-override convention `stb-hubbardu`/`stb-elasticInputs`/`stb-phononsCreate` already use, since SIESTA's fdf reader is first-occurrence-wins for a duplicate directive — the override has to physically precede anything your own template already sets).

Two independent reasons this has to be fixed, not left to whatever your `--calc` template happens to say:

- **The lattice constant is a bulk property.** The slab's in-plane lattice vectors come from the underlying bulk crystal's own equilibrium geometry — a single adsorbed atom or molecule has no physical business changing it. If the cell were allowed to relax, the "adsorption energy" you'd compute would be contaminated by a spurious change in the substrate's own lattice constant, not just the adsorbate's binding.
- **The vacuum gap is a numerical parameter, not a physical one.** The gap along `c` exists purely so the slab doesn't interact with its own image above/below it (Section 3.2 below is the in-plane analogue of this same idea). Letting `MD.VariableCell` touch that axis would change a number that was never meant to have a physical value in the first place.

Atomic positions are a different story: whether the slab, adsorbate, and site should relax their **positions** is entirely up to your own `--calc` template's `MD.TypeOfRun`/`MD.NumCGsteps` (a real adsorption study normally does want that) — `stb-adsorb` never touches those. The one exception is the BSSE ghost-fragment folders, next section.

11.8.1.2 BSSE — the counterpoise correction, and why it needs a stage of its own

The problem. SIESTA is a localized-basis (LCAO) code. A fragment computed **alone** only has its own atoms' basis functions available to describe its electrons. In the **combined** slab+adsorbate calculation, each fragment additionally has the *other* fragment's basis functions sitting nearby, even though those orbitals belong to atoms that, from that fragment's own point of view, aren't really part of it. This extra, unearned variational freedom systematically **lowers** the combined system's energy relative to either fragment's own smaller-basis calculation — Basis Set Superposition Error (BSSE) — making every LCAO adsorption energy look more strongly bound than it really is, purely as a basis-set artifact, not a physical effect. Section 4 below shows exactly how misleading that artifact can be: not just "a bit too strong," but capable of flipping which candidate site looks best.

The fix: Boys-Bernardi counterpoise, both directions. The standard correction (Boys & Bernardi, 1970) re-evaluates each fragment's energy **in the full combined basis**, by adding "ghost" atoms standing in for the atoms that are physically absent from that fragment — same real basis functions and pseudopotential, but zero nuclear/valence charge, so they add basis freedom without adding any real physics. `stb-adsorbBsse` (Stage 2) writes both directions, at the exact RELAXED geometry of the site they belong to:

- `bsse_slab/` — the real **substrate**, with the adsorbate replaced by ghosts → the substrate's own energy, evaluated in the full dimer basis.
- `bsse_adsorbate/` — the real **adsorbate**, with the substrate replaced by ghosts → the adsorbate's own energy, evaluated in the full dimer basis.

`stb-adsorbAnalysis` (Stage 3) then applies:

```
E_ads(BSSE-corrected) = E_site - E_bsse_slab - E_bsse_adsorbate
```

Ghost species use SIESTA's own ghost-atom convention: a new species label (`<element>_ghost`) with a **negative** atomic number in `%block ChemicalSpeciesLabel`, resolving to the *same* pseudopotential file as the real element (`core/pseudopotentials.py::copy_pseudo`'s `dest_label` parameter) but contributing zero charge to the calculation.

Because both fragments here — a finite slab, a finite adsorbate — are already complete, self-contained systems, this counterpoise is **exact**: there's no truncation choice to make in either direction. Contrast this with `stb-cohesive`'s own BSSE correction for a *periodic bulk* crystal, which has no natural "edge" to stop at and needs an arbitrary neighbor-shell cutoff (`--bsse-cutoff`) instead.

Why BSSE needs its own stage, AFTER the sites relax. A counterpoise correction only means what it's supposed to mean if every fragment is evaluated at the **same geometry** — the geometry of the actual, combined, relaxed site. That geometry doesn't exist yet at Stage 1 (`stb-adsorb`) time: `sites/site_*/` is a real relaxation (whatever `MD.TypeOfRun` your `--calc` template requests), and its atoms only settle into their true adsorption geometry once SIESTA has actually finished running there. An earlier version of this workflow wrote the BSSE ghost-fragment folders directly from `stb-adsorb`'s own pre- relaxation initial guess — which, for a real chemisorption bond, can be substantially wrong: a live test case saw an O-Si bond shrink from a 2.0 Ang initial guess to ~1.62 Ang after real relaxation (plus ~0.46 Ang of substrate reconstruction), and BSSE grows quickly as atoms get closer together, so evaluating it at the wrong (too-separated) geometry under-estimates the true correction.

`stb-adsorbBsse` (Stage 2) fixes this by running strictly AFTER Stage 1's sites have finished relaxing: for each `sites/site_*/` with a finished `siesta.XV`, it reads that RELAXED geometry directly (skipping, with a clear report, any site that hasn't finished yet) and writes `bsse/site_*/bsse_slab/+bsse/site_*/bsse_adsorbate/` at that exact geometry — with `config_extra.fdf` additionally forcing single-point SCF on top of the fixed cell (`MD.TypeOfRun CG`, `MD.Steps 0`, `MD.NumCGsteps 0` — the same single-point-enforcement idea `stb-hubbardu` uses for its own perturbation probes, for the analogous reason: a counterpoise correction, like a linear-response slope, is only meaningful when every point being compared shares the exact same geometry). If `bsse_slab//bsse_adsorbate/` were instead allowed to relax further, each ghost-fragment calculation would drift to its *own* locally-optimal geometry instead of staying at the site's, and the three energies being subtracted would no longer refer to the same physical configuration.

11.8.1.3 Is the slab big enough? Two separate self-interaction checks

A periodic slab calculation has *two* independent directions in which a badly-sized supercell can make the adsorbate spuriously interact with a copy of itself, and `stb-adsorb` checks both. `example_5.8.sh` exercises both live.

Out-of-plane: the isolated-adsorbate reference box. The isolated-adsorbate reference (`adsorbate/`) sits in a cubic `--vacuum-box` (default 20 Ang). If the molecule's own extent exceeds half that box, it can reach its own periodic image in *that* separate, throwaway cell — `stb-adsorb` already warns about this (`molecule_extent` vs. `--vacuum-box`).

In-plane: the slab's own lateral supercell. This is a different, easy-to-miss failure mode: the **site** folders use the slab's *own* in-plane periodicity (vectors `a`/`b`), which is fixed by whatever structure you gave `stb-adsorb` — not something this tool controls. If that in-plane supercell is small (a bare primitive cell, for instance), the adsorbate's periodic images in neighboring cells can sit right next to each other, turning what was meant to be an isolated adsorption event into an artificial 2D "monolayer" of adsorbates at full coverage instead.

`stb-adsorb` checks this directly: `min_adsorbate_image_distance` computes the minimum distance between the adsorbate and the nearest copy of itself across the slab's `a`/`b` lattice vectors (translation- invariant — it only depends on the lattice and the adsorbate's own internal geometry, not which site it ends up on, so it's checked once per adsorbate, in `[1] REFERENCE FOLDERS`, before any site is even placed). Below **8 Ang** (a common rule-of-thumb separation for avoiding lateral adsorbate-adsorbate interaction — dipole coupling, steric contact — in the literature), it prints a `[WARNING]` recommending a larger in-plane supercell (`stb-supercell`).

This is exactly the failure mode `structure.fdf`'s bare 2-atom graphene primitive cell (lattice constant 2.46 Ang) demonstrates on purpose in `example_5.8.sh`'s `output/stage1_lateral_warning/` section — *far* below the 8 Ang recommendation, since it's only meant to exercise `stb-adsorb`'s mechanics quickly, not to be a physically meaningful adsorption calculation on its own. `output/stage1_fixed/` immediately follows it

up by tiling the same cell 4x4 in-plane with `stb-supercell` and re-running `stb-adsorb`, confirmed live to make the warning disappear — that bigger cell is what the rest of the walkthrough (Section 4) actually uses.

11.8.1.4 Libraries and external dependencies used

- **`pymatgen`** (`pymatgen.analysis.adsorption.AdsorbateSiteFinder`) — finds candidate on-top/bridge/hollow sites and reduces them by symmetry (`--symprec`). Note: for `structure.fdf`'s graphene basis, pymatgen's finder never returns a `hollow` candidate at any cell size tried — ontop and bridge are what the worked example below compares; this is a property of the site-finding algorithm on this exact basis, not a limitation `stb-adsorb` itself imposes.
- **`sisl`** — reads each site's relaxed `siesta.XV` in Stage 2 (`stb-adsorbBsse`).
- **`matplotlib`** — the site-layout plot (`adsorption_sites.png`, Stage 1) and the adsorption-energy ranking plot (`adsorption_ranking.png`, Stage 3, `--view-plots` to preview live).
- **`gnuplot`** (optional, always written by Stage 3) — `adsorption_curve.dat/ .gplot`, a portable per-site `E_ads`/`E_ads_BSSE` scatter.
- **MACE** (optional, `pip install stb_suite[ml]`) — only touched by `--ml-prerelax` (Stage 1, pre-relax the isolated adsorbate) and `--ml-rank` (Stage 1, pre-screen candidate sites before committing to real SIESTA folders). Neither is used in this walkthrough.

11.8.1.5 5. Stage 1 mechanics, live (`stb-adsorb`, code `5.8.1`)

```
$ stb-adsorb --adsorbate O -O output/stage1 --no-intro
```

writes `clean_slab/`, `adsorbate/` (forced spin-polarized — many single atoms/molecules, like O, N, or NO, have a non-zero ground-state spin), and one `sites/site_1_ontop/` (the default site type/index). By default (`--force-spin`, ON unless you pass `--no-force-spin`), every `sites/site_*/` folder is ALSO forced spin-polarized — the physically relevant quantity, `E_ads`, comes from the **combined** slab+adsorbate calculation, and a single adsorbate atom bonded to a slab commonly leaves that combined system with a net magnetic moment too (most simply, an odd total valence-electron count, which a spin-restricted calculation cannot represent at all). `clean_slab/` itself is left alone (no adsorbate present, no clear universal reason to force it). Every folder also gets `Slab.DipoleCorrection T` forced on `clean_slab/` and every `sites/site_*/` (never the isolated `adsorbate/` reference, which is a molecule in an all-around vacuum box, not a slab) — adsorbing on only one face breaks the slab's inversion/mirror symmetry along the vacuum axis, giving the cell a net dipole along a PERIODIC direction that would otherwise contaminate the total energy with a spurious periodic-image field. `clean_slab/`'s `config_extra.fdf` is exactly:

```
# Auto-generated by stb-adsorb -- keeps the cell fixed (see the example's
# README.md for why).
MD.VariableCell false
```

while a `sites/site_*/`'s `config_extra.fdf` additionally has:

```
# Auto-generated -- forces spin-polarized SCF (see write_reference_folder's
# force_spin docstring for why).
Spin                polarized
```

Report structure (`[0]`–`[6]`, `--save-report` persists it to `adsorption_prep_report.txt`):

```
[0] RUN METADATA
[1] REFERENCE FOLDERS
[2] ADSORPTION SITES: FINDING & COUNT
[3] ML PRE-SCREENING
[4] WRITING SITE FOLDERS
[5] SUMMARY & NEXT STEPS
[6] LIBRARY WARNINGS
```

`--site-type all --all-sites` writes one folder per symmetrically-distinct site of every type instead of just one; `--both-sides` additionally mirrors each selected site onto the bottom face for a free-standing 2D material like this graphene fixture (verified: `site_1_ontop_bothsides/structure.fdf` has `NumberofAtoms 4` — 2 substrate + 1 adsorbate per face); `--height-sweep <min> <max> <step>` writes an approach curve instead of one fixed height.

A caution worth knowing before you script this yourself: calling `stb-adsorb` twice into the *same* `-O` output directory with two different single `--site-type` values (e.g. once for `ontop`, once for `bridge`) overwrites `sites/adsorption_sites.txt` — the machine-readable site table `stb-adsorbAnalysis` reads for each site's adsorbate/height — silently dropping the first call's row. Use a single `--site-type all --all-sites` call instead (as Section 4 below does) if you want more than one site type in the same study.

Interactive path: `stb-suite`, type `5.8.1` at the main prompt — same flags, same order, as the CLI above. `example_5.8.sh` proves the two agree.

11.8.1.6 6. Stage 2 mechanics, live (`stb-adsorbBsse`, code `5.8.2`)

```
$ stb-adsorbBsse --dir adsorption_run --save-report
```

Report structure (`[0]`–`[4]`, `--save-report` persists it to `adsorption_bsse_report.txt`):

```
[0] RUN METADATA
[1] SITE SCAN
[2] WRITING BSSE FOLDERS
[3] SUMMARY & NEXT STEPS
[4] LIBRARY WARNINGS
```

`[1]` prints a table with one row per `sites/site_*/`: `ready` (has a `siesta.XV`, gets a BSSE folder pair written), `SKIP (no .XV yet -- not relaxed)` (not fatal — re-run this stage later once it's finished), or a mismatched/stale-folder warning if a site's atom count no longer matches its own `structure.fdf`. `[2]` writes `bsse/site_*/bsse_slab/` and `bsse/site_*/bsse_adsorbate/` only for the `ready` sites, reusing that site's own already-copied pseudopotentials directly (no `-p` flag on this stage). Interactive path: `stb-suite`, type `5.8.2`.

11.8.1.7 7. Stage 3 mechanics, live (`stb-adsorbAnalysis`, code `5.8.3`)

```
$ stb-adsorbAnalysis --dir adsorption_run --save-report --apply production.fdf
```

Report structure (`[0]`–`[5]`, `--save-report` persists it to `adsorption_report.txt`):

```
[0] RUN METADATA
[1] REFERENCE ENERGIES
[2] SITE RESULTS: CONFIGURATION COUNT & TABLE
[3] SUMMARY & PLOT
[4] APPLY
[5] LIBRARY WARNINGS
```

[2] prints one row per site (E_ads, and E_ads_BSSE once Stage 2 has run there, plus an SCF-convergence/residual-force quality flag — a folder missing calc.out entirely is SKIPped, not fatal to the rest of the batch) and a BSSE coverage: complete N, incomplete N, absent N summary. [3] reports the most stable site **twice** — once by the uncorrected ranking, once by the BSSE-corrected ranking — plus the BSSE correction's exact magnitude at the corrected winner. [4] (--apply <path>) copies the most stable site's structure.fdf to <path>, preferring the BSSE-corrected ranking whenever it's available. Interactive path: stb-suite, type 5.8.3.

11.8.1.8 Worked example: watching the BSSE correction flip the ranking

This is the section example_5.8.sh's output/workflow/ walks through line by line, and the reason this whole workflow needs a Stage 2. No real SIESTA binary is available inside this walkthrough, so — same convention 5.7-hubbardu's own worked example uses for its linear-response fit — every calc.out below is a **hand-chosen, exact** number, picked in advance to make one very concrete point.

Two candidate sites on the 4x4-supercell-fixed structure (Section 3.2), site_1_ontop and site_2_bridge:

Quantity	Value (eV)
E_clean_slab	-200.000000
E_adsorbate (isolated O)	-13.500000
E_site (ontop)	-214.100000
E_site (bridge)	-214.150000
E_bsse_slab (ontop)	-200.050000
E_bsse_adsorbate (ontop)	-13.520000
E_bsse_slab (bridge)	-200.090000
E_bsse_adsorbate (bridge)	-13.560000

Uncorrected E_ads = E_site - E_clean_slab - E_adsorbate:

```
E_ads(ontop)  = -214.100000 - (-200.000000) - (-13.500000) = -0.600000 eV
E_ads(bridge) = -214.150000 - (-200.000000) - (-13.500000) = -0.650000 eV   <- more negative, "wins"
```

Bridge looks more stable by 0.050 eV. Now the BSSE-corrected E_ads(BSSE) = E_site - E_bsse_slab - E_bsse_adsorbate:

```
E_ads_BSSE(ontop)  = -214.100000 - (-200.050000) - (-13.520000) = -0.530000 eV
E_ads_BSSE(bridge) = -214.150000 - (-200.090000) - (-13.560000) = -0.500000 eV   <- now LESS negative
than ontop
```

Ontop is now more stable by 0.030 eV — the ranking flips. Both corrections make the binding weaker (less negative), exactly the direction Section 2.1 predicts (BSSE always over-binds); the bridge site simply happened to have the larger correction (+0.150 eV vs. ontop's +0.070 eV — plausible physically, since a bridge adsorbate typically sits closer to more substrate atoms at once, borrowing more basis functions). stb-adsorbAnalysis's own report says it directly:

```
Most stable site (uncorrected):   site_2_bridge  (E_ads = -0.650000 eV, exothermic (favorable))
Most stable site (BSSE-corrected): site_1_ontop  (E_ads = -0.530000 eV, exothermic (favorable))
BSSE correction at that site: +0.070000 eV (uncorrected LCAO adsorption energies
systematically over-bind -- expect this to make the energy less negative)
```

--apply best_production.fdf in this same run copies **site_1_ontop** (the BSSE-corrected winner) forward — proof the tool itself treats the correction as authoritative once it's available, not just an FYI number next to the "real" one.

The remaining two candidate sites from Stage 1 (`site_3_bridge`, `site_4_bridge`) are deliberately left without a `siesta.XV/calc.out` in this walkthrough, at no extra scripting cost demonstrating that neither Stage 2 nor Stage 3 treats an unfinished site as fatal — both simply skip it and report exactly which one, and why (`SKIP (no .XV yet -- not relaxed)` / `SKIP (missing calc.out)`).

11.8.1.9 Known, deliberate limitations

- **No SIESTA run is ever performed by this suite.** All four stages generate/read input and output files; you run SIESTA yourself in `clean_slab/`, every `adsorbate*/`, every `sites/site_*/`, every `bsse/site_*/bsse_slab/+bsse_adsorbate/`, and every `gibbs/*/disp_*/`, in between stages.
- **`--no-force-spin` leaves the combined slab+adsorbate calculation's `Spin` setting up to you.** By default (Section 5) both the isolated reference AND every `sites/site_*/` folder are forced spin-polarized; if you disable that, verify by hand whether `Spin polarized` is still needed for an open-shell adsorbate — adsorption often (not always) quenches the adsorbate's own spin.
- **The BSSE correction assumes each finished `sites/site_*/`'s `siesta.XV` really is the converged, relaxed geometry** — Stage 2 has no way to distinguish a genuinely converged run from one that stopped early for other reasons; check your own SIESTA convergence before trusting the correction.
- **`stb-adsorb`'s own site-finder is pymatgen's `AdsorbateSiteFinder`, not a physically-informed search** — as Section 4/8 notes, it can miss a real candidate site type entirely (no `hollow` site found for this graphene basis at any tested cell size); always sanity-check `adsorption_sites.png` against what you'd expect chemically.
- **Calling `stb-adsorb` more than once into the same `-O` directory with different single `--site-type` values silently drops the earlier call's site-table row** (Section 5) — use one `--site-type all --all-sites` call, or separate output directories, instead.
- **Stage 4's Gibbs free energy is a harmonic-approximation treatment, not an anharmonic one** (Section 13) — `ZPE`/entropy come from a harmonic Hessian (site) and `ase.thermochemistry.IdealGasThermo` (isolated reference, full translational+rotational+vibrational — no longer vibrational-only), both standard, but neither captures real anharmonic effects (thermal expansion of the well, mode coupling). The isolated reference's standard-state pressure is also fixed at 1 bar, not user-configurable.
- **`--zpe-mode local`'s site-side treatment is a decoupled-oscillator approximation** — only the adsorbate's own atoms are displaced (substrate frozen), ignoring any coupling between the adsorbate's vibrations and the substrate's own phonons. `--zpe-mode full` removes this approximation (a real phonon calculation of the whole structure) at the cost of far more SIESTA jobs.
- **Orientation deduplication (Section 14.2) doesn't correct for periodic wraparound** — RMSD is a direct Cartesian comparison, no minimum-image convention. Not expected to matter in practice (every orientation for one candidate starts at the same site coordinate and relaxes independently), but a documented simplification, not a rigorous guarantee.
- **`[3c]`'s desorption kinetics (`tau_desorb`) is a *lower bound*, not a kinetic prediction** (Section 13.6) — it assumes desorption has literally no barrier beyond `-DG_ads(T)` and a bare, uncorrected TST prefactor. A real barrier only makes the true residence time longer than the number printed here, sometimes by many orders of magnitude. **Never quote `tau_desorb` on its own as "the desorption time"** — it is only ever a floor on that time.
- **When `DG_ads(T)` is already positive at `--tmin` itself, `[3c]` degenerates into "instant" at every scanned temperature** (Section 13.7) — a real, observed failure mode (not a hypothetical one), not a bug. It means the barrierless-TST assumption has nothing left to bound: there is no thermodynamic well anywhere in the scanned range for it to measure a lifetime against. Treat this pattern as a screening signal ("this site does not bind at any temperature you scanned"), not as literal femtosecond/nanosecond kinetics.

11.8.1.10 Step-by-step: running this workflow on your own structure

1. Have a slab/2D structure (`.fdf`, vacuum along c) and a working `calc.fdf` template.
2. Run Stage 1: `stb-adsorb -s <structure.fdf> -c <calc.fdf> --adsorbate <El-or-G2-molecule> --site-type all --all-sites`. Read `[1]`'s self-interaction checks (Section 3) before proceeding — tile with `stb-supercell` first if either warns.
3. Run SIESTA yourself in `adsorption_run/clean_slab/`, every `adsorption_run/adsorbate*/`, and every `adsorption_run/sites/site_*/`.
4. Run Stage 2: `stb-adsorbBsse --dir adsorption_run`. Re-run it later for any site reported `SKIP (no .XV yet -- not relaxed)` once that site finishes.
5. Run SIESTA yourself in every generated `bsse/site_*/bsse_slab/` and `bsse/site_*/bsse_adsorbate/`.
6. Run Stage 3: `stb-adsorbAnalysis --dir adsorption_run --save-report --apply production.fdf`. Read `[3]` for **both** rankings before trusting a number — if they disagree, trust the BSSE-corrected one (Section 8). Add `--compute-gibbs` if you also want a Gibbs free energy (Section 13) — it needs both the winning site AND its isolated -adsorbate reference to already be relaxed.
7. If you passed `--compute-gibbs`: run SIESTA yourself in every `adsorption_run/gibbs/*/disp_*/` folder, then run Stage 4: `stb-adsorbGibbs --dir adsorption_run --save-report --tmin <T1> --tmax <T2>` covering whatever temperature range you actually care about. Read `D0` (the electronic+ZPE part) alongside `DG(T)` (the full answer, including entropy) — don't assume they agree (Section 13.4); check the estimated desorption temperature (`[3b]`) against your own operating conditions.

11.8.1.11 Files in this folder

File	Purpose
`structure.fdf`	Bare 2-atom graphene primitive cell, free-standing (10 Ang vacuum along c) — small and fast, deliberately laterally too small to exercise Section 3.2's self-interaction check.
`calc.fdf`	Shared `calc.fdf` template (non-polarized, Gamma-only, modest basis/mesh — mechanics only, not physically converged).
`example_5.8.sh`	The guided walkthrough (**not** an automated test — see `test/5-workflow/8-adsorption/{prep,bsse,analysis,gibbs}/test.sh` for that). Pauses between sections so you can read before moving on; safe to re-run.
`output/`	Created by `example_5.8.sh` when you run it (git-ignored, not checked in). See below.

11.8.1.12 Running the script

```
./example_5.8.sh
```

Case	Command(s)	What it shows
`output/stage1/`	`stb-adsorb --adsorbate O -O output/stage1`	Stage 1 mechanics: fixed cell, spin-polarization note
`output/stage1_lateral_warning/`	same command	Section 3.2's lateral self-interaction `[WARNING]`, live
`output/stage1_fixed/`	`stb-supercell -d 4 4 1` then `stb-adsorb --site-type all --all-sites`	the warning fixed by tiling; the `[2]` candidate-count table
`output/stage1_bothsides/`	`stb-adsorb --adsorbate H --both-sides`	2D free-standing material, both faces

Case	Command(s)	What it shows
`output/workflow/`	Stage 1 → (fabricated `.XV/calc.out`) → Stage 2 → (fabricated BSSE `calc.out`) → Stage 3 → Stage 3b (`--compute-gibbs`) → (fabricated Hessian `.FA`) → Stage 4	**the worked examples (Section 8: BSSE flips the ranking; Section 13.4: entropy flips DG unfavorable above ~331 K; Section 13.6: the same crossing, in a rough desorption timescale)**
(no folder — a diff only)	Stage 1 via `printf . . . | stb-suite`	proof the interactive menu (`5.8.1`) agrees with the CLI

11.8.1.13 13. Stage 3 diagnostics/suggestions, and Stage 4 — Gibbs free energy (`stb-adsorbGibbs`, code `5.8.4`)

Stage 3 does more than just rank `E_ads` values now:

- **`[2b] PHYSICAL DIAGNOSTICS`** — per site, the electric dipole magnitude and net magnetic moment SIESTA printed (both come free from `Slab.DipoleCorrection`/`Spin polarized` already being forced by Stage 1), plus how much the closest adsorbate-slab distance changed between the pre-relaxation guess and the relaxed geometry — a quick sanity check that the adsorbate actually moved toward the surface during relaxation, not away from it.
- **`[2c] BSSE BREAKDOWN`** — once Stage 2 has run, the raw ghost-fragment energies (`E_bsse_slab`/`E_bsse_adsorbate`) behind every site's `E_ads_BSSE`, plus the correction's shift in both eV and as a percentage of the uncorrected `E_ads` — for *every* site with BSSE data, not just the winner Section 8's summary line already reports. Useful for spotting a site where BSSE is a disproportionately large fraction of the whole binding energy (a sign the LCAO basis may be too small for that particular geometry).
- **`[5] SUGGESTED NEXT ANALYSES`** — points you at other tools in the suite worth running on the winning site (`stb-bader`, `stb-dos`, `stb-workfunction`, `stb-coop`), each conditional on a SIESTA output file Stage 1 doesn't force on its own (e.g. `.RHO`, `.PDOS.xml`).

The DG formula, and what it physically means. A plain `E_ads` is an electronic-energy-only number — it ignores that the adsorbate actually vibrates in its adsorption well (zero-point energy) and that vibrating degrees of freedom carry entropy. The full correction:

```
DG(T) = E_ads (BSSE-corrected if available, else raw)
        + [ZPE(site) - ZPE(isolated reference)]
        - T * [S(site) - S(isolated reference)]
      = D0 - T * DS
```

`ZPE(site)`/`S(site)` come from a vibrational Hessian of the adsorbate's own atoms at the winning site (substrate frozen — a decoupled-oscillator approximation, `--zpe-mode local`, the default; `--zpe-mode full` instead runs a real Phonopy phonon calculation of the entire site structure minus the clean slab's own phonons, far more SIESTA jobs but no decoupling approximation). These are standard harmonic-oscillator ZPE/entropy sums over whichever normal modes come out with a genuinely positive frequency — a substrate-frozen adsorbate has no exact zero modes to worry about, so any negative eigenvalue found here is a real red flag (an unconverged geometry), not a symmetry artifact.

`ZPE(isolated reference)`/`S(isolated reference)` always come from a full-molecule Hessian of the isolated-adsorbate reference alone — there's no periodic substrate to freeze/subtract for a molecule in a vacuum box, so `--zpe-mode` only ever changes the SITE side. **This is where a real, quantitatively significant bug was fixed this session.** A free molecule/atom has 3 translational + (3 for nonlinear, 2 for linear, 0 for monatomic) rotational rigid-body zero modes that carry no vibrational ZPE/entropy

of their own — but the finite-difference Hessian construction has no way to make these land at exactly zero; roughly half typically come out numerically negative ("imaginary") and half numerically positive, purely from floating-point noise. The isolated reference now selects exactly the `3N-6/3N-5/0` **highest-frequency** modes as genuine vibrations (`ase.thermochemistry.IdealGasThermo`'s own count-based selection, mirrored here explicitly so the split is visible in the report) — not by sign. Treating the isolated reference as a genuine ideal gas this way also means it now gets its **real translational (and, for a polyatomic adsorbate, rotational) entropy** (Sackur-Tetrode + rigid-rotor formulas, at a fixed 1 bar standard-state pressure, symmetry-number-aware via `core/symmetry.py::point_group_details`) — previously it contributed vibrational entropy only, silently missing the dominant entropy term for any small, weakly-bound adsorbate. Verified directly against gas-phase water's known experimental standard entropy (188.8 J/(mol·K) at 298.15 K, matched to <0.1%) before being trusted for a real calculation.

Physically, this matters because a free gas-phase species carries a lot of entropy (freedom to translate and rotate in 3D) that it mostly loses upon binding to a surface (it can now only vibrate in place). `DTS = T*[S(site) - S(isolated reference)]` is therefore usually strongly negative and grows in magnitude with `T` — meaning `DG` becomes progressively **less** favorable than the raw electronic `E_ads` as temperature rises, exactly the trend Section 13.4's worked example below shows numerically.

Input parameters. `stb-adsorbAnalysis --compute-gibbs` (Stage 3's opt-in Gibbs-prep step):

Flag	Meaning
`--compute-gibbs`	Writes the Hessian/phonon displacement folders for the BSSE-corrected (or, if unavailable, uncorrected) winning site. Off by default; needs the winning site AND its isolated-adsorbate reference to already have a finished `siesta.XV` — a clean `[ERROR]`, not a silent fallback to the pre-relaxation guess, if either isn't relaxed yet.
`--zpe-mode {local,full}`	`local` (default): a partial-Hessian, decoupled-oscillator approximation — only the SITE's adsorbate atoms are displaced, substrate frozen, 6 folders per atom. `full`: a real Phonopy phonon calculation of the ENTIRE site structure and the clean slab (so Stage 4 can subtract the clean slab's own phonon ZPE/entropy) — physically more rigorous, far more SIESTA jobs (`6*3*N_atoms` displacements for an `N`-atom structure, vs. `6*N_adsorbate_atoms` for `local`). The isolated reference always uses `local` regardless — a molecule in vacuum has no periodic substrate phonon to subtract, so `full`'s extra machinery buys nothing there.
`--displacement`	Finite-difference displacement, in Ang (default 0.015) — same physical role as `stb-phononsCreate`'s own displacement, smaller values are more harmonic but noisier, larger values sample more anharmonicity.
`--gibbs-supercell`	`--zpe-mode full` only: supercell dimensions for the site+clean-slab phonon calculations (default `1 1 1`, appropriate once the slab's own in-plane supercell — Section 3.2 — is already large enough).

`stb-adsorbGibbs` (Stage 4 itself):

Flag	Meaning
`--dir`	Root directory containing `gibbs//sites//clean_slab/` (default `adsorption_run`).
`--file`	SIESTA output filename inside each folder (default `calc.out`).
`--tmin/--tmax/--tstep`	Temperature sweep, in K (defaults 200/400/25) — `DG/DS` are recomputed at every step (vibrational entropy is temperature-dependent even though `ZPE` itself isn't); the sweep's first point (`--tmin`) is also what `[FINAL RESULT]/D0` report.
`--force-tolerance`	Residual atomic force, eV/Ang (default 0.05), above which a folder is flagged as possibly not fully converged/relaxed. Advisory only.
`--save-report`	Persists the full report to `adsorption_gibbs_report.txt`. Off by default.
`--view-plots`	Also shows the `DG/D0/DS` plot on screen instead of only saving it as a PNG.

Interpreting the output.

- **D0** — the ZPE-corrected binding energy, E_ads + DZPE. Temperature-independent (unlike DG); the electronic+ZPE part of the story, before any entropy is considered. Printed once, alongside E_ads (raw)/E_ads (BSSE).
- **DZPE** — ZPE(site) - ZPE(isolated reference). Usually positive: binding to a surface generally adds real, nonzero-frequency vibrational modes that didn't exist (or existed as free rigid-body motion, contributing no ZPE) before adsorption.
- **DTS**/**DS** — DS is DTS / T, reported in both eV/K and J/(mol·K) (the conventional surface-science unit) — the actual entropy change of adsorption. Typically strongly negative (Section 13.1) — literature values for losing most of a small molecule's translational/rotational freedom upon adsorption are commonly in the 100-150 J/(mol·K) range, a useful sanity-check scale for your own result.
- **DG(T)** — the actual Gibbs free energy of adsorption at each scanned temperature; the physically meaningful "is adsorption favorable" answer, not E_ads alone.
- **Estimated desorption temperature ([3b])** — linear interpolation (or, flagged [EXTRAPOLATED], extrapolation) of the DG(T) sweep to its DG = 0 crossing. A rough estimate from a harmonic model, not a kinetic prediction (real desorption also depends on a barrier/attempt-frequency, not just the sign of DG) — but a useful single number for "is this still bound at the temperature I actually care about."
- **Desorption kinetics ([3c])** — a rough Eyring-TST estimate of the desorption rate k_desorb(T) and timescale tau_desorb(T) at every temperature already swept in [3]. Always printed (console **and** --save-report file) directly under an explicit [APPROXIMATION] warning — see Section 13.6 for the full derivation, the barrierless assumption behind it, and why tau_desorb is a **lower bound**, not a prediction. If every row reads ~0 (instant), see Section 13.7 — that's the degenerate case where DG_ads(T) is already positive at --tmin, and [3c] stops being informative at all.
- **adsorption_gibbs.png** — a 2-panel plot: top, DG(T) against two temperature-independent reference lines (E_ads and D0); bottom, DS(T).
- **adsorption_mode_spectrum.png** — a stick-spectrum comparison of the site's and isolated reference's vibrational frequencies (site above the axis, isolated reference below), genuine vibrational modes as full-height solid stems, excluded rigid-body/imaginary modes as short red stems — the isolated reference's excluded modes cluster near 0 THz, well separated from any genuine vibration. This is exactly the kind of plot that would have visually surfaced the mode-counting bug (Section 13.1) on sight, rather than requiring a manual literature cross-check to notice.
- **adsorption_desorption_kinetics.png** / **plot/desorption_kinetics.{dat,gplot}** — tau_desorb(T), log-scale, from the same [3c] sweep. Both are written every run, not either/or: the PNG (matplotlib) is for immediately looking at the result; the .dat/.gplot pair (gnuplot, WORKFLOW_TOOLS convention — see the manual's developer reference) lives in its own plot/ subfolder rather than next to the two PNGs above, since it's the only gnuplot-convention output this stage produces (cd plot && gnuplot desorption_kinetics.gplot renders desorption_kinetics.pdf there).

Worked example: watching entropy turn a favorable adsorption unfavorable. example_5.8.sh's output/workflow/ chain continues past Stage 3 (Section 8) into a Stage 4 example on the exact same site_1_ontop (the BSSE-corrected winner, E_ads_BSSE = -0.530000 eV) — a single adsorbed O atom, so a particularly clean case: a monatomic species has **zero** possible vibrational modes by definition (there's nothing to vibrate against), so its isolated reference's ZPE must come out **exactly zero** regardless of any Hessian data fed to it, while its entropy is **purely translational** (no rotation for a point particle either) — the simplest possible demonstration of that fix.

Two fabricated .FA (force) datasets, hand-chosen the same "no real SIESTA available" way Section 8's calc.out values are:

- **Site** (site_1_ontop's adsorbed O, substrate frozen): an isotropic harmonic restoring force, k = 10

eV/Ang^2 in all 3 directions — a plausible ~400 cm^{-1} adatom-surface vibration.
- **Isolated reference** (O_isolated): **exactly zero** force at every displacement — nothing restores a truly free atom, so its local Hessian is identically zero.

Running it (stb-adsorbAnalysis --dir . --compute-gibbs --zpe-mode local, then stb-adsorbGibbs --dir . --tmin 200 --tmax 400 --tstep 100) gives, verbatim:

```
Isolated reference (1-atom, monatomic) -- ideal-gas treatment (translation + rotation + vibration):
    -0.00 THz (imaginary) -- excluded as a rigid-body translation/rotation mode
    -0.00 THz (imaginary) -- excluded as a rigid-body translation/rotation mode
    -0.00 THz (imaginary) -- excluded as a rigid-body translation/rotation mode
Site (1-atom local Hessian) vibrational modes:
    412.27 cm^-1  ( 12.36 THz)  E = 0.0511 eV   (x3, isotropic)
ZPE(site)  = 0.0767 eV   ZPE(isolated ref) = 0.0000 eV   DZPE = +0.0767 eV
D0 (ZPE-corrected binding energy) = E_ads + DZPE = -0.4533 eV

T =  200.00 K   DZPE = +0.0767 eV   DTS = -0.2690 eV   DS = -1.3452 meV/K (-129.80 J/(mol*K))   DG =
-0.1843 eV
T =  300.00 K   DZPE = +0.0767 eV   DTS = -0.4102 eV   DS = -1.3672 meV/K (-131.92 J/(mol*K))   DG =
-0.0432 eV
T =  400.00 K   DZPE = +0.0767 eV   DTS = -0.5483 eV   DS = -1.3708 meV/K (-132.26 J/(mol*K))   DG =
+0.0950 eV

[FINAL RESULT] DG(adsorption) = -0.1843 eV at T = 200.0 K (D0 = -0.4533 eV)
T_desorption ~= 331.2 K (interpolated within the scanned range)
```

ZPE(isolated ref) = 0.0000 eV exactly, as expected — but note DTS is very much **not** zero: the isolated O atom's real translational entropy (~130 J/(mol·K), right in the literature ballpark quoted in Section 13.3) dominates the whole calculation. The electronic+BSSE picture alone (E_ads_BSSE = -0.530000 eV) says this adsorption is comfortably favorable at any reasonable temperature; **DG tells a different story** — favorable at 200 K, only barely favorable at 300 K, and outright unfavorable by 400 K, crossing zero at an estimated **331 K**, just above room temperature. This is the whole reason Stage 4 exists: a purely electronic E_ads (even BSSE-corrected) cannot tell you this on its own.

The Gibbs-prep folders also demonstrate the config_extra.fdf mechanism — gibbs/site_1_ontop/disp_001/calc.fdf is now just %include config_extra.fdf plus your original --calc template, unmodified:

```
%include config_extra.fdf

SystemLabel gibbs_site
...
```

with the single-point-SCF/level-of-theory overrides collected in config_extra.fdf itself — the same %include-based split stb-adsorb/ stb-adsorbBsse already use (Sections 1/2), instead of editing calc.fdf's own text in place as earlier versions of this tool did:

```
# Auto-generated -- keeps the cell fixed and forces a single-point SCF
# (0 CG steps) so this displaced geometry isn't relaxed away.
MD.VariableCell false
MD.TypeOfRun       CG
MD.Steps           0
MD.NumCGsteps      0
# Auto-generated -- forces spin-polarized SCF (see write_reference_folder's
# force_spin docstring for why).
Spin               polarized
```

```
# Auto-generated -- forces the slab dipole correction (see write_reference_folder's
# force_dipole docstring for why).
Slab.DipoleCorrection      .true.
# Auto-generated -- forces the Grimme DFT-D3 dispersion correction (see
# write_reference_folder's force_vdw docstring for why).
DFTD3                  .true.
```

(A handful of `[WARNING] Pseudopotential 'O.psml' or 'O.psf' not found` lines also appear in this walkthrough's own Stage 4 output — expected and harmless: this example never had real pseudopotential files to copy in the first place, same "mechanics only, not physically converged" caveat the whole walkthrough already carries.)

Running it.

```
stb-adsorbAnalysis --dir adsorption_run --compute-gibbs --zpe-mode local
# run SIESTA in every adsorption_run/gibbs/*/disp_*/ folder, then:
stb-adsorbGibbs --dir adsorption_run --save-report --tmin 200 --tmax 400 --tstep 25
```

`stb-adsorbAnalysis --compute-gibbs` needs the winning site AND its isolated-adsorbate reference to already be relaxed (a finished `siesta.XV` in both) — it refuses with a clear `[ERROR]`, not a silent fallback to the pre-relaxation guess, if either isn't. `stb-adsorbGibbs` auto-detects which single site's folder set exists under `gibbs/` — you never need to pass a site label explicitly, since `--compute-gibbs` only ever preps one site's worth of folders per run.

13.6 Rough desorption kinetics (`[3c]`) — how fast, not just whether. Section 13.4 stopped at *whether* `site_1_ontop` desorbs — `DG(T)` crosses zero around 331 K, so it's bound below that and unbound above it. That's a thermodynamic answer, not a kinetic one: it says nothing about **how fast** desorption actually happens at a given temperature, which is what you'd need to compare against, say, an experiment's dwell time or a device's operating lifetime. `[3c]` gives a genuinely rough answer to that question, computed directly from the same `DG(T)` sweep `[3]` already produced — no new SIESTA calculation, no transition-state search:

```
k_desorb(T)   = (kB*T/h) * exp(DG_ads(T) / (kB*T))
tau_desorb(T) = 1 / k_desorb(T)
```

This is the Eyring/transition-state-theory rate law under the **barrierless -desorption approximation**: it assumes the desorption activation free energy `DG_double_dagger` equals `-DG_ads(T)` exactly — i.e. that there is *no* extra kinetic barrier beyond the thermodynamic well depth this workflow already computes. A real desorption pathway almost always has *some* additional barrier (the adsorbate has to pass through an actual transition state on its way out, not just teleport from the bound minimum to a free particle) — this tool has no way to compute that barrier, since it would require a genuine transition-state search (e.g. `stb-neb`, Section 9), not just the endpoint energies this workflow already has. The prefactor `kB*T/h` is also the bare TST value, not corrected by a Vineyard prefactor (the ratio of the bound state's and the transition state's own vibrational frequencies) — that correction needs the transition state's vibrational spectrum too, equally unavailable here. Both approximations point the same direction: **`tau_desorb` printed here is a lower bound on the true desorption time, not a prediction of it** — a real barrier can only make desorption slower/`tau` longer than shown.

This caveat is not a footnote — it is printed as an explicit `[APPROXIMATION]` warning immediately above the `[3c]` table on **every** run, in the live console output and in `--save-report`'s persisted file alike (same `print_dual` mechanism that keeps every other section of this report identical between the two), precisely so the rough numbers below are never read in isolation from what they assume.

Continuing the exact same `site_1_ontop` scenario as Section 13.4 (same `E_ads_BSSE = -0.530000 eV`, same isotropic ~400 cm^{-1} site Hessian, same free monatomic-O isolated reference) — `stb-adsorbGibbs --dir . --tmin 200 --tmax 400 --tstep 100`, output continuing directly after `[3b]`, verbatim:

```
[3c] DESORPTION KINETICS -- ROUGH ESTIMATE (Eyring-TST)
------------------------------------------------------------
  [APPROXIMATION] Assumes desorption has NO kinetic barrier beyond the thermodynamic
  well depth already computed above (DG_double_dagger = -DG_ads(T)) and a bare TST
  prefactor kB*T/h (no Vineyard correction from a transition-state vibrational
  spectrum -- this workflow does not run a NEB/transition-state search). tau_desorb
  below is therefore a LOWER BOUND on the real desorption time, not a kinetic
  prediction -- a real barrier can only make desorption SLOWER. See Section 13.6 of
  examples/5.8-adsorption/README.md for the full derivation and caveats.
  T =  200.00 K   DG = -0.1843 eV   k_desorb =  9.467e+07 1/s   tau_desorb = 10.6 ns
  T =  300.00 K   DG = -0.0432 eV   k_desorb =  1.178e+12 1/s   tau_desorb = 849 fs
  T =  400.00 K   DG = +0.0950 eV   k_desorb =  1.311e+14 1/s   tau_desorb = 7.63 fs

  At T = 200.0 K (rough estimate, lower bound): tau_desorb ~= 10.6 ns
[Saved] ./adsorption_gibbs.png
[Saved] ./adsorption_mode_spectrum.png
[Saved] ./adsorption_desorption_kinetics.png (APPROXIMATION -- see [3c] above)
[Saved] ./plot/desorption_kinetics.dat / ./plot/desorption_kinetics.gplot (run
'gnuplot desorption_kinetics.gplot' inside './plot' -- APPROXIMATION, see [3c] above)
```

`tau_desorb` collapses from `10.6 ns` at 200 K (where `DG` is still negative — the adsorbate is thermodynamically bound, and this rough model already predicts it sits there for a while) down to `7.63 fs` at 400 K (where `DG` has gone positive — essentially instantaneous on this model, once desorption is even thermodynamically favorable). That eight-order-of-magnitude collapse across a 200 K window, driven entirely by the `exp(DG/(kB*T))` factor, is the actual qualitative payoff of `[3c]`: `[3b]`'s single crossing temperature tells you *where* the sign flips; `[3c]` tells you the timescale falls off a cliff right around it, not gradually — useful context even though every number in it is a lower bound. **Read literally, `9.467e+07/1.178e+12/1.311e+14 1/s` and their corresponding `tau` values are not measured rates** — they are what the barrierless-TST approximation gives from `DG(T)` alone; a real experimental `k_desorb` at any of these temperatures could be many orders of magnitude smaller once an actual barrier is included.

`desorption_kinetics.dat` (the gnuplot pair's data file, `plot/` subfolder):

```
# Desorption kinetics for site_1_ontop -- ROUGH Eyring-TST estimate
# (barrierless-desorption assumption: DG_double_dagger = -DG_ads(T),
# bare kB*T/h prefactor, no Vineyard correction) -- see 'examples/
# 5.8-adsorption/README.md' Section 13.6 or the report's [3c] section
# for the full caveat. tau_desorb is a LOWER BOUND, not a prediction.
# 1:T(K) 2:DG_ads(eV) 3:k_desorb(1/s) 4:tau_desorb(s)
200.0000  -0.184300  9.467271e+07  1.056304e-08
300.0000  -0.043200  1.177914e+12  8.489499e-13
400.0000  0.095000  1.311132e+14  7.627628e-15
```

`cd plot && gnuplot desorption_kinetics.gplot` renders `desorption_kinetics.pdf` in the same folder — a log-scale `tau_desorb(T)` line plot, same `pdfcairo`/`linespoints` styling as every other gnuplot-convention plot in this suite (e.g. `stb-nebAnalysis`'s energy profile). `adsorption_desorption_kinetics.png` is the same data as a matplotlib figure, for viewing directly without needing gnuplot installed at all.

13.7 The degenerate case: `DG_ads(T)` already positive at `--tmin` — a real example. Section 13.6's worked example is the "nice" case: `DG(T)` starts negative (bound) and crosses to positive (unbound) *inside*

the scanned window, so `tau_desorb` has something to measure — it falls from a genuinely long time to a genuinely short one as `T` sweeps across that crossing. That is not the only shape this can take. **When `DG_ads(T)` is already positive at `--tmin` itself — i.e. the system is thermodynamically unbound at *every* temperature you scanned — `[3c]` degenerates into `tau_desorb ~= ~0 (instant)` at every single row, not just near the crossing.** This is not a hypothetical corner case; it is what an actual finished SIESTA calculation in this suite produced (weakly-bound CO2 physisorption, run outside this example folder — the numbers below are a real, code-verified `stb-adsorbGibbs` run, not a hand-fabricated illustration like Sections 8/13.4):

```
[1] ELECTRONIC ENERGIES
------------------------------------------------------------
  E_ads (raw)  =    -0.532653 eV
  E_ads (BSSE) =    -0.096923 eV
  E_ads used for DG below: BSSE-corrected = -0.096923 eV

[2] VIBRATIONAL/THERMAL TERMS
------------------------------------------------------------
  ZPE(site)  = 0.3218 eV   ZPE(isolated ref) = 0.3043 eV   DZPE = +0.0175 eV
  D0 (ZPE-corrected binding energy) = E_ads + DZPE = -0.0794 eV

[3] GIBBS FREE ENERGY vs. TEMPERATURE
------------------------------------------------------------
  T =  200.00 K   DZPE = +0.0175 eV   DTS = -0.2907 eV   DS = -1.4534 meV/K (-140.24 J/(mol*K))   DG =
  +0.2113 eV
  T =  300.00 K   ...                                                                              DG =
  +0.3520 eV
  T =  400.00 K   ...                                                                              DG =
  +0.4911 eV

[FINAL RESULT] DG(adsorption) = +0.2113 eV at T = 200.0 K (E_ads BSSE-corrected, D0 = -0.0794 eV)

[3b] ESTIMATED DESORPTION TEMPERATURE
------------------------------------------------------------
  [EXTRAPOLATED] T_desorption ~= 49.0 K -- DG stays positive throughout the scanned
  [200.0, 400.0] K range; this is a linear extrapolation beyond it, not an
  interpolation -- widen --tmin/--tmax for a direct estimate.

[3c] DESORPTION KINETICS -- ROUGH ESTIMATE (Eyring-TST)
------------------------------------------------------------
  T =  200.00 K   DG = +0.2113 eV   k_desorb =  8.790e+17 1/s   tau_desorb = ~0 (instant)
  T =  300.00 K   DG = +0.3520 eV   k_desorb =  5.129e+18 1/s   tau_desorb = ~0 (instant)
  T =  400.00 K   DG = +0.4911 eV   k_desorb =  1.285e+19 1/s   tau_desorb = ~0 (instant)
```

Why this happens (theory): `D0` — the electronic+ZPE binding energy, `E_ads + DZPE`, the `T = 0` limit of the whole calculation — is still favorable here (`-0.0794 eV`, a genuine although weak minimum). What flips `DG` positive already at 200 K is `DTS`: `DS = -140.24 J/(mol*K)`, i.e. this adsorbate loses almost its *entire* gas-phase translational+rotational entropy on binding (squarely in, in fact toward the high end of, the 100-150 J/(mol·K) literature range quoted in Section 13.3), while the electronic well is only `~0.08 eV` deep — nowhere near enough to outweigh a `-T*DS` penalty of that size even at 200 K, let alone 300-400 K. Physically: **a shallow physisorption well can lose the fight to gas-phase entropy before you ever reach a temperature worth running SIESTA at.**

> **(!) READ THIS BEFORE TRUSTING `tau_desorb` IN THIS REGIME.** Once `DG_ads(T) > 0` at every scanned `T`, `k_desorb(T) = (kB*T/h) * exp(DG_ads(T)/(kB*T))` (Section 13.6) has a positive exponent at every row — it is mathematically *guaranteed* to sit above the bare

attempt frequency `kB*T/h` (~1e13 1/s), which is exactly why `tau_desorb` floors out at "instant" (femtoseconds or less) everywhere. **This does not mean the real adsorbate desorbs in femtoseconds.** It means the barrierless -TST model (Section 13.6: `DG_double_dagger = -DG_ads(T)`, no real transition-state search) has run out of anything to measure — there is no thermodynamic well left in this window for "how long does it sit in the well" to even be a meaningful question. Section 13.6's own "lower bound" framing is technically still true here (a real barrier can only make desorption slower), but it is **not useful** in this regime: "at least instant" carries no information. **Do not report a `tau_desorb` value from this regime as a kinetic result.**

What to do instead:

- **Trust `[3b]`'s extrapolated crossing (`T_desorption ~= 49.0 K` above), not `[3c]`, as the actionable number** — and note it is flagged `[EXTRAPOLATED]`, a linear extrapolation *below* `--tmin`, not an interpolation. Re-run with a lower `--tmin` (e.g. `--tmin 50`) to turn that extrapolation into a direct interpolation instead — cheap to do, since it reuses the same finished Hessian folders, no new SIESTA runs.
- **Treat "`DG` positive throughout the whole scan" as a screening result, not a kinetics result**: this site/adsorbate pair is not going to stay bound at any temperature you actually scanned, full stop — the femtosecond `tau_desorb` numbers add nothing beyond confirming that.
- **If you actually need a real kinetic answer** (a genuine desorption rate, not a thermodynamic screening result), this workflow cannot give you one in *any* regime, degenerate or not — that requires an actual transition-state search (`stb-neb`, Section 9) with the adsorbate's real escape pathway and its own vibrational spectrum for a proper Vineyard prefactor, neither of which Stage 4 computes.
- Before concluding the *site* is simply weakly bound, double check the basics the diagnostics above depend on: was `DFTD3` actually applied (`config_extra.fdf`, Section 14.1) — van der Waals is often the dominant term for a weak physisorption well, and a missing/misconfigured dispersion correction can produce exactly this "too-shallow-to-matter" `D0`.

11.8.1.14 Van der Waals correction (always on), and systematic orientation sampling

DFT-D3, forced on every folder. `stb-adsorb` forces the Grimme DFT-D3 dispersion correction (`DFTD3 .true.`, SIESTA's own default parameters — PBE-parametrized, BJ damping) on `clean_slab/`, the isolated -adsorbate reference, and every `sites/site_*/` — unconditionally, no flag to turn it off. Standard GGA functionals miss dispersion forces entirely, which matters for a physisorption-like or weakly-bound site; without it, a comparison across candidate sites can be systematically biased toward whichever one happens to be least dispersion-dependent. Stage 2 (`stb-adsorbBsse`) and Stage 3's `--compute-gibbs` Hessian folders both inherit this the same way they already inherit `Spin polarized/ Slab.DipoleCorrection` — every fragment being compared against `E_site` needs to be at the *same level of theory*, not just the same geometry, or the correction being computed stops meaning what it's supposed to.

Systematic orientation sampling for a molecular adsorbate. For a single-atom adsorbate, there's only one way to place it. For a molecule (H2O, CO2, ...), the molecule's own default orientation (from ASE's G2 database) picks *which atom touches the surface* — for water, whether it's the O or an H isn't something you control by default; it's just whatever pymatgen's `AdsorbateSiteFinder` happens to do with the molecule's as-loaded geometry.

`--ml-rank` can now sample this systematically:

```
stb-adsorb -s structure.fdf -c calc.fdf --adsorbate H2O --site-type all --all-sites \
    --ml-rank --n-orientations-polar 4 --n-orientations-azimuthal 2 \
    --orientation-top-k 3
```

For every candidate `(site, height)`, this relaxes `n_polar * n_azimuthal` (here 4×2=8) systematically-generated starting orientations with MACE-MP-0 (substrate fixed, same as `--ml-rank` already does for a single orientation), deduplicates the relaxed results (two orientations that converge to the same geometry AND the same energy collapse to one), and keeps up to `--orientation-top-k` unique survivors — each written as its own real SIESTA folder, `site_<N>_<type>_orient<k>/`, exactly the way `--ml-rank` already writes one folder per site.

This is systematic, not random or exhaustive: `--n-orientations-polar` picks that many directions (in the molecule's own frame) to point at the surface, evenly spread over a sphere; `--n-orientations-azimuthal` spins each of those about the surface normal. The same flags always produce the exact same orientation set — no `--seed` needed, unlike `stb-mladsorb --n-orientations`'s fully random sampling (a different tool, doing a cheaper single-point-only pre-screen instead of a full relax of each orientation).

Cost scales linearly with `n_polar * n_azimuthal` — one full MACE relax per orientation per candidate. A modest `4 2` (8 orientations) is a reasonable starting point; `--orientation-rmsd-tol` (default 0.3 Ang) controls how aggressively near-duplicate orientations get merged before `--orientation-top-k` is applied.

14.3 A single site instead of `--all-sites`. Orientation sampling also works on one chosen site (`--site-index N`, no `--all-sites`) — useful when you already know which site you care about and just want to explore how the molecule sits there:

```
stb-adsorb -s structure.fdf -c calc.fdf --adsorbate H2O --site-type ontop --site-index 0 \
    --ml-rank --n-orientations-polar 4 --n-orientations-azimuthal 2 \
    --orientation-top-k 3
```

Without orientation sampling, `--ml-rank` needs `--all-sites` (ranking one site against itself is meaningless); with it, the single chosen site's own sampled orientations are what gets ranked, and every generated configuration lands on that same site instead of spreading across the other symmetric sites.

14.4 Progress feedback and the full-exploration `.xyz` trajectory. Each orientation's MACE relax prints a one-line progress update as it runs (a self-overwriting counter on an interactive terminal, or a periodic line every ~10% of the total when output is redirected to a file) instead of staying silent until the whole batch finishes — useful once `n_polar * n_azimuthal` gets into the dozens.

Every relaxed orientation actually generated — not just the unique/`--orientation-top-k` survivors written as SIESTA folders — is also saved to `sites/orientation_trajectory<_adsorbate>.xyz`, a multi-frame extended-XYZ file openable directly as a trajectory/video in VMD or OVITO. Each frame records its site, site type, height, orientation index, MACE energy, and convergence status as per-frame metadata, so the whole exploration (including near-duplicates and higher-energy attempts that got discarded) can be reviewed visually, not just read off the ranking table/plot.

11.8.1.15 What's next

- **`stb-supercell`** — tile a small cell (like the graphene primitive cell here) into a laterally larger one before running `stb-adsorb`, if Section 3.2's check warns you.
- **`stb-cohesive`** — the sibling BSSE correction for a periodic bulk crystal (needs a neighbor-shell cutoff, unlike this exact one).
- **`stb-mladsorb`** — a pure-MACE, no-SIESTA extraction of the same site-screening idea, for a fast pre-DFT ranking of candidate sites before committing to the real workflow above.

11.8.2 Implementation details

Adsorption energy, counterpoise and free energy

With the energies of the relaxed site, the clean slab and the isolated adsorbate,

$$E_{\mathrm{ads}} = E_{\mathrm{site}} - E_{\mathrm{slab}} - E_{\mathrm{adsorbate}}, \qquad E_{\mathrm{ads}}^{\mathrm{CP}} = E_{\mathrm{site}} - E_{\mathrm{slab}}^{\mathrm{ghost}} - E_{\mathrm{adsorbate}}^{\mathrm{ghost}},$$

where the counterpoise energies [71] are those of each fragment, at the geometry it has in the relaxed site, computed with the basis functions of the other fragment present as ghost atoms. Negative values mean binding. The Gibbs energy of adsorption adds the vibrational zero-point energy and entropy of the adsorbed species and the zero-point energy and ideal-gas entropy of the free reference,

$$\Delta G(T) = E_{\mathrm{ads}} + [\mathrm{ZPE}_{\mathrm{site}} - \mathrm{ZPE}_{\mathrm{gas}}] - T\,[S_{\mathrm{site}}(T) - S_{\mathrm{gas}}(T)]\,,$$

with harmonic-oscillator sums $\mathrm{ZPE} = \sum_\nu \frac{1}{2}\hbar\omega_\nu$ and $S = k_B \sum_\nu \left[\frac{x_\nu}{e^{x_\nu}-1} - \ln(1-e^{-x_\nu})\right]$, $x_\nu = \hbar\omega_\nu / k_B T$.

Stage 1 (`stb-adsorb`).. Input: a relaxed slab (`-s`, vacuum along **c**; a slab with its vacuum on another axis is relabelled, and a slab touching the cell boundary is re-centred in the vacuum) and a template (`-c`). The adsorbate is an element or an ASE G2 molecule name (`-list`); several, comma-separated, run the same search once per adsorbate. Candidate sites are found with pymatgen's `AdsorbateSiteFinder` [7]: `-site-type ontop`, `bridge`, `hollow` or `all`, symmetry-reduced at `-symprec`; `-site-index` picks one, `-all-sites` keeps all of them. `-position X Y` or `-positions-file` replace the search with your own sites. The adsorbate is placed `-height` Å (default 2.0) above the site, or at every height of `-height-sweep` for an approach curve; `-both-sides` adsorbs on the two faces of a free-standing 2D layer. The run folder (`-O`, default `adsorption_run`) contains:

`clean_slab/`	the slab, fixed cell, dipole correction, D3
`adsorbate/`	the isolated adsorbate in a cube of `-vacuum-box` Å (default 20), spin-polarized, D3
`sites/site_<n>_<type>/`	slab plus adsorbate: fixed cell, dipole correction, D3, spin-polarized unless `-no-force-spin`
`sites/adsorption_sites.txt`	the machine-readable site table read by Stage 3

All forced settings live in each folder's `config_extra.fdf` (Section 4.2). The cell is kept fixed everywhere: the adsorption energy is a difference of energies at the *same* cell, and relaxing the slab's cell with an adsorbate would mix in a strain energy.

MACE pre-screening.. `-ml-rank` relaxes every candidate with MACE (substrate fixed, `-ml-fmax`, `-ml-steps`) with D3(BJ) dispersion added to match the DFT-D3 of the SIESTA folders, ranks them, and with `-top-k` writes folders only for the best sites. For molecules, `-n-orientations-polar` × `-n-orientations-azimuthal` orientations per site are sampled systematically, duplicates within `-orientation-rmsd-tol` removed and the best `-orientation-top-k` kept. `-ml-prerelax` pre-relaxes the written geometries. The MACE step runs on CPU if CUDA is not usable, with a warning (Section 6.4).

Stage 2 (`stb-adsorbBsse`).. After the sites have relaxed, it reads each site's final geometry (`.XV`), splits it into substrate and adsorbate (atom order is known from Stage 1), and writes `bsse_slab/` (real substrate, ghost adsorbate) and `bsse_adsorbate/` (real adsorbate, ghost substrate) next to it – single points at that geometry, with the spin, dipole and D3 settings read from the site's own `config_extra.fdf`. Ghosts are species `<El>_ghost` with negative atomic number and the real element's pseudopotential and basis. Sites that did not converge (SCF or forces above `-force-tolerance`) are reported and skipped.

Stage 3 (`stb-adsorbAnalysis`). It reads all energies (`-file`, default `calc.out`), checks convergence and residual forces, and tabulates E_{ads} and, where Stage 2 has run, $E_{\mathrm{ads}}^{\mathrm{CP}}$ and the size of the correction; the most stable site is reported by both rankings (they can differ). A height sweep gives the binding curve in `-o` (default `adsorption_curve.dat`); `-apply PATH` copies the best structure. A table of physical diagnostics gives, per site, the dipole and magnetic moment read from its log and the change of the closest adsorbate–slab distance during relaxation (negative: the adsorbate moved closer), followed by a per-fragment BSSE breakdown and suggested next analyses. With `-compute-gibbs` it writes, for the winning site and the isolated reference, the finite-displacement folders of Stage 4 under `gibbs/` (`-displacement`, default 0.015 Å): with `-zpe-mode local` (default) only the adsorbate atoms are displaced, the substrate frozen; with `full` a phonopy calculation of the whole site (and of the clean slab, subtracted), in a `-gibbs-supercell`. This is the one analysis stage of the suite that writes new calculations.

Stage 4 (`stb-adsorbGibbs`). It builds the mass-weighted Hessian from the forces, diagonalizes it, and evaluates ZPE and vibrational entropy from `-tmin` to `-tmax` (steps `-tstep`). For the free reference, ASE's `IdealGasThermo` is used: exactly $3N-6$ ($3N-5$ for linear, none for an atom) of the highest frequencies are kept as vibrations, and translational (Sackur–Tetrode) and rotational entropy at 1 bar are added, with the rotational symmetry number from the molecule's point group. The report gives $\Delta G(T)$, the temperature at which it changes sign (desorption becomes favourable; flagged `[EXTRAPOLATED]` outside the scan), and a rough desorption time from Eyring theory with no barrier beyond $-\Delta G$, $k_{\mathrm{des}} = \frac{k_B T}{h} e^{\Delta G / k_B T}$, $\tau = 1/k_{\mathrm{des}}$ – a lower bound, since a real barrier only slows desorption.

About the built-in help of `stb-adsorbGibbs`

The description printed by `-help` (reproduced in the reference below) still states that the isolated reference has vibrational entropy only. The current code adds the ideal-gas translational and rotational entropy as described above, and its report says so.

Good practice. Converge the slab thickness and lateral size (the adsorbate must not interact with its own images), relax the clean slab and the adsorbate with the same settings before Stage 1, and treat sites with large BSSE corrections with caution: SIESTA's localized basis makes the counterpoise correction substantial, and it can reorder sites.

11.8.3 Command-line reference

11.8.3.1 `stb-adsorb`

Prepares SIESTA folders for an adsorption-energy study: a clean slab/2D material, an isolated adsorbate, and one folder per candidate adsorption site.

Synopsis.

```
usage: stb-adsorb [-h] [-s STRUCTURE] [-c CALC] [-p PSEUDO_DIR]
                  [--adsorbate ADSORBATE] [--list]
                  [--site-type {ontop,bridge,hollow,all}] [--site-index SITE_INDEX]
                  [--all-sites] [--position X Y] [--positions-file PATH]
                  [--height HEIGHT] [--height-sweep MIN MAX STEP] [--both-sides]
                  [--symprec SYMPREC] [--force-spin] [--no-force-spin]
                  [--vacuum-gap VACUUM_GAP] [--vacuum-box VACUUM_BOX] [--ml-prerelax]
                  [--ml-rank] [--ml-model {small,medium,large}]
                  [--ml-custom-model PATH] [--ml-custom-model-card JSON]
                  [--ml-device {cpu,cuda}] [--ml-fmax ML_FMAX] [--ml-steps ML_STEPS]
                  [--top-k TOP_K] [--n-orientations-polar N_ORIENTATIONS_POLAR]
                  [--n-orientations-azimuthal N_ORIENTATIONS_AZIMUTHAL]
```

```
[--orientation-top-k ORIENTATION_TOP_K]
[--orientation-rmsd-tol ORIENTATION_RMSD_TOL] [-O OUTPUT_DIR]
[--save-report] [--view] [--view-plots] [-v] [--no-intro]
```

Options

`-s, --structure STRUCTURE`

Input slab/2D `structure.fdf` (vacuum along c). Default: `structure.fdf`.

`-c, --calc CALC`

`calc.fdf` template already configured for the slab (kgrid, basis, XC, %include `structure.fdf`, etc.) – copied as-is into 'clean_slab/' and every 'sites/site_*/' folder (same lattice, so the same k-grid applies to all of them); a Gamma-only, spin-polarized derivative goes into 'adsorbate/'. A `config_extra.fdf` sidecar is %included on top, forcing MD.VariableCell false (see [1]/[4] in the report). Default: `calc.fdf`.

`-p, --pseudo-dir PSEUDO_DIR`

Pseudopotentials source (optional): a bundled bank or a folder path.

`--adsorbate ADSORBATE`

Element symbol (single atom) or ASE G2 molecule name (e.g. H2O, CO2, CH4). Comma-separated for more than one (e.g. 'O,N,C') – the same site search runs once per adsorbate, useful for comparing binding affinity across species at the same site(s).

`--list`

List the available G2 molecule names and exit. switch, off by default

`--site-type {ontop,bridge,hollow,all}`

Which family of adsorption sites to search (default: ontop). choices: ontop, bridge, hollow, all

`--site-index SITE_INDEX`

Which symmetrically distinct site of `--site-type` to use, 0-based (default: 0). Ignored with `--all-sites` or `--position`. type: int

`--all-sites`

Write one folder per symmetrically distinct site of `--site-type`, instead of just `--site-index`. switch, off by default

`--position X Y`

Manual Cartesian (X, Y) position in Ang, overriding automatic site-finding entirely (`--site-type`/`--site-index`/`--all-sites` are ignored when this is given). type: float

`--positions-file PATH`

'.dat' file with one or more adsorption site positions, as in-plane fractional (a, b) coordinates relative to the file's own LATTICE_A/LATTICE_B/LATTICE_C lines ('number type frac_a frac_b' per site line; '#' comments and blank lines ignored) – used INSTEAD of automatic site-finding (`--site-type`/`--symprec` are ignored). `--all-sites`/`--site-index` then select among the file's positions exactly as they would among auto-found ones. `stb-adsorb` always writes 'sites/site_positions.dat' with the lattice and positions actually used this run (auto-found or manual alike), in the same format, so it can be inspected, hand-edited, and fed back here for a future run. Mutually exclusive with `--position` and `--both-sides`.

`--height HEIGHT`

Adsorption distance in Ang above the surface (default: 2.0). Ignored if `--height-sweep` is given. type: float

`--height-sweep MIN MAX STEP`

Generate one site per height in this range (inclusive), instead of a single fixed `--height` – an approach-curve scan. `stb-adsorbAnalysis` will report E_ads vs. height for it. Same min/max/step convention as `stb-convergence`. Incompatible with `--both-sides`. type: float

`--both-sides`

Adsorb on both exposed faces (for a free-standing 2D material with vacuum on both sides of the c-axis).

Requires a concrete `--site-type` (not 'all'); incompatible with `--position`, `--ml-rank` and `--height-sweep`. switch, off by default

`--symprec SYMPREC`

Symmetry-reduction tolerance for site-finding (default: 0.01, pymatgen's own default). type: float

`--force-spin`

Force Spin polarized (via `config_extra.fdf`, overriding `--calc`'s own Spin tag if any) in every 'sites/site_*/' folder too, not just the isolated-adsorbate reference (default: ON) – a single adsorbate atom (or a molecule containing one) bonded to a slab commonly leaves the combined system with a net magnetic moment (most simply, an odd total valence-electron count, which a spin-restricted calculation cannot represent at all). Costs nothing for a genuinely closed-shell site (converges to zero moment). Does not affect 'clean_slab/' (no adsorbate present). switch, off by default

`--no-force-spin`

Leave `--calc`'s own Spin tag untouched in 'sites/site_*/' (the isolated-adsorbate reference is still always forced polarized). switch

`--vacuum-gap VACUUM_GAP`

Vacuum-axis detection threshold in Ang, used to validate the input's vacuum axis is c (default: 10.0), same convention as `stb-kgrid/stb-slab`. type: float

`--vacuum-box VACUUM_BOX`

Cubic box side (Ang) for the isolated-adsorbate reference calculation (default: 20.0). type: float

`--ml-prerelax`

Relax the isolated adsorbate's geometry (positions only) with MACE-MP-0 before writing its reference folder – same idea as `stb-phononsCreate --ml-prerelax`. Mainly useful for adsorbates whose starting geometry isn't already at equilibrium (e.g. a hand-built one rather than an ASE G2 entry, which already is). Needs the optional 'ml' extra. switch, off by default

`--ml-rank`

Pre-screen candidate sites with a MACE-MP-0 relax (substrate fixed, only the adsorbate moves) before writing SIESTA folders, ranking by relaxed energy – a fast pre-screen for which site(s) to prioritize for real DFT, not a replacement for `stb-adsorbAnalysis`. Needs the optional 'ml' extra. Only valid with `--all-sites`.switch, off by default

`--ml-model {small,medium,large}`

MACE-MP-0 model size for `--ml-prerelax/--ml-rank`: speed/accuracy tradeoff (default: medium). Ignored if `--ml-custom-model` is given. choices: small, medium, large

`--ml-custom-model PATH`

Use a custom MACE model file for `--ml-prerelax/--ml-rank` instead of the MACE-MP-0 foundation potential – e.g. one fine-tuned on your own SIESTA data via `stb-mlffAnalysis`. Overrides `--ml-model`.

`--ml-custom-model-card JSON`

Model card of the `--ml-custom-model` file: the `mlff_model_card.json` that `stb-mlffAnalysis` wrote next to it (training data, level of theory, accuracy, energy scale). Required with `--ml-custom-model`: the card is checked against the model file and used to warn when a structure is outside what the model was trained on.

`--ml-device {cpu,cuda}`

default: cpu; choices: cpu, cuda

`--ml-fmax ML_FMAX`

default: 0.05; type: float

`--ml-steps ML_STEPS`

Max MACE-MP-0 optimizer steps per relax, for both `--ml-prerelax` and `--ml-rank` (default: 200). Raise it if a candidate is reported as not converged; lower it to speed up screening a large number of sites/orientations at the cost of less-relaxed rankings. type: int

`--top-k TOP_K`

With `--ml-rank`: only write SIESTA folders for the N best-ranked sites, instead of all of them. Counts SITES, not orientations – if orientation sampling is also on, every already `--orientation-top-k-kept` orientation of each of the N selected sites is written, so the number of folders can exceed N. type: int

`--n-orientations-polar N_ORIENTATIONS_POLAR`

With `--ml-rank` and a multi-atom `--adsorbate`: systematically sample this many initial molecule orientations PER SITE before relaxing – which part of the molecule ends up facing the surface (e.g. O-down vs. H-down for H2O) is otherwise fixed by the molecule's default G2 geometry alone. Default 1 = current behavior, no sampling. Combined with `--n-orientations-azimuthal` below (total orientations per site = polar x azimuthal); a modest 4x2=8 is a reasonable starting point – cost scales linearly with this product (one full MACE relax per orientation per site). Works with a single `--site-index` too, not just `--all-sites` – every sampled orientation of the SAME site is then what gets ranked. Every relaxed orientation (not just the kept/unique ones) is saved to sites/orientation_trajectory.xyz as a multi-frame trajectory (viewable in VMD/OVITO). type: int

`--n-orientations-azimuthal N_ORIENTATIONS_AZIMUTHAL`

With `--ml-rank`: this many evenly-spaced in-plane rotations sampled per polar direction (see `--n-orientations-polar`). Default 1. type: int

`--orientation-top-k ORIENTATION_TOP_K`

With `--ml-rank` and orientation sampling: keep only the N best unique (post-deduplication) orientations per site, instead of all surviving ones. Unset (default) keeps every unique orientation found. type: int

`--orientation-rmsd-tol ORIENTATION_RMSD_TOL`

With `--ml-rank` and orientation sampling: two relaxed orientations at the same site are treated as duplicates (only the better-energy one kept) when their adsorbate-atom RMSD is below this (Ang) AND their energy differs by less than 0.01 eV. Default 0.3. type: float

`-O, --output-dir OUTPUT_DIR`

Root directory (default: adsorption_run) for 'clean_slab', 'adsorbate' and 'sites' – everything this study needs lives under one folder, same convention as `stb-hubbardu`'s hubbardu_runs.

`--save-report`

Also persist the full report narrative to `adsorption_prep_report.txt` (under `--output-dir`). Off by default. 'sites/adsorption_sites.txt' (the SITE_TABLE `stb-adsorbAnalysis` reads) is always written regardless. switch, off by default

`--view`

Open every generated structure (clean_slab, isolated adsorbate(s), and every written site) in ASE's interactive 3D viewer as a multi-frame browser, after everything else has finished. Needs a display. switch, off by default

`--view-plots`

Also show the matplotlib plot(s) this run generates – the candidate-site layout, and the ML pre-screen ranking chart if `--ml-rank` is used – on screen (one blocking window per plot), instead of only saving them as PNGs. Off by default. A separate flag from `--view` since that one already means the ASE structure viewer here.switch, off by default

Like every STB command it also accepts `-h`/`--help`, `-v`/`--version` and `--no-intro` (Section 3.3).

From the built-in help.

```
Usage examples:
  stb-adsorb -s slab.fdf -c calc.fdf --adsorbate O --site-type ontop
  stb-adsorb -s slab.fdf -c calc.fdf --adsorbate H2O --all-sites --site-type all
  stb-adsorb -s slab.fdf -c calc.fdf --adsorbate H --all-sites --ml-rank --top-k 3
  stb-adsorb -s slab.fdf -c calc.fdf --adsorbate O,N,C --site-type ontop
  stb-adsorb -s slab.fdf -c calc.fdf --adsorbate O --height-sweep 1.5 3.0 0.5
```

11.8.3.2 stb-adsorbBsse

Stage 2 of the Adsorption workflow: generates the BSSE (counterpoise) ghost-fragment folders at each site's RELAXED geometry, once stb-adsorb's own sites/site_*/ have finished running in SIESTA.

Synopsis.

```
usage: stb-adsorbBsse [-h] [--dir DIR] [--file FILE]
                      [--force-tolerance FORCE_TOLERANCE] [--save-report] [-v]
                      [--no-intro]
```

Options

`--dir DIR`
: Root directory containing 'clean_slab/' and 'sites/' (default: adsorption_run – stb-adsorb's own default --output-dir).

`--file FILE`
: SIESTA output filename inside each site folder (default: calc.out).

`--force-tolerance FORCE_TOLERANCE`
: Residual atomic force in eV/Ang (default: 0.05) above which a site's relaxation is flagged as possibly incomplete – advisory only, never blocks BSSE folder generation. type: float

`--save-report`
: Also persist the report to adsorption_bsse_report.txt. Off by default. switch, off by default

Like every STB command it also accepts -h/–help, -v/–version and –no-intro (Section 3.3).

From the built-in help.

```
Usage examples:
  stb-adsorbBsse
  stb-adsorbBsse --dir adsorption_run --file calc.out
```

11.8.3.3 stb-adsorbAnalysis

Computes adsorption energies from an stb-adsorb sweep: E_ads = E_site - E_clean_slab - E_adsorbate.

Synopsis.

```
usage: stb-adsorbAnalysis [-h] [--dir DIR] [--file FILE] [-o OUTPUT]
                          [--apply STRUCTURE_FDF] [--force-tolerance FORCE_TOLERANCE]
                          [--save-report] [--view] [--view-plots] [--compute-gibbs]
                          [--zpe-mode {local,full}] [--displacement DISPLACEMENT]
                          [--gibbs-supercell GIBBS_SUPERCELL GIBBS_SUPERCELL GIBBS_SUPERCELL]
                          [-v] [--no-intro]
```

Options

`--dir DIR`
: Root directory containing 'clean_slab/', 'adsorbate*/' and 'sites/' (default: adsorption_run – `stb-adsorb`'s own default `--output-dir`, so this stage runs against that one's output with no extra flag in the common case).

`--file FILE`
: SIESTA output filename inside each folder (default: `calc.out`).

`-o, --output OUTPUT`
: Output data file name (default: `adsorption_curve.dat`).

`--apply STRUCTURE_FDF`
: Copy the most stable site's `structure.fdf` (BSSE-corrected ranking when available for every site, else the uncorrected ranking) to this path.

`--force-tolerance FORCE_TOLERANCE`
: Residual atomic force in eV/Ang (default: 0.05, same as `stb-cohesiveAnalysis`) above which a site's `calc.out` is flagged as possibly not relaxed – E_ads would then reflect a strained/off-equilibrium geometry rather than the true adsorption minimum. Advisory only, never blocks the result. type: float

`--save-report`
: Also persist the report to `adsorption_report.txt`. Off by default. switch, off by default

`--view`
: Open clean_slab, every adsorbate reference and every site (preferring each folder's own `<label>.XV` – the actual relaxed geometry, if SIESTA has been run there – falling back to its input `structure.fdf` otherwise) in ASE's interactive 3D viewer as a multi-frame browser, after everything else has finished. Needs a display. switch, off by default

`--view-plots`
: Also show the matplotlib adsorption-ranking plot on screen (one blocking window), instead of only saving it as a PNG. Off by default. switch, off by default

`--compute-gibbs`
: Also write the vibrational-Hessian displacement folders (under '<dir>/gibbs/') `stb-adsorbGibbs` (Stage 4) needs to compute a Gibbs free energy of adsorption, for the winning site (BSSE-ranked if complete, else uncorrected) and its isolated-adsorbate reference. Off by default – this is the one exception in this tool to 'analysis never generates new folders', by explicit design (see the manual): the alternative would be a whole extra prep-only stage duplicating everything [0]-[4] already do. switch, off by default

`--zpe-mode {local,full}`
: Vibrational Hessian mode for the SITE side, when `--compute-gibbs` (default: local). 'local': partial Hessian, only the adsorbate's own atoms displaced (substrate frozen) – fast, a decoupled-oscillator approximation, no Phonopy involved. 'full': a real Phonopy phonon calculation of the ENTIRE site structure, minus the clean slab's own (also full-structure) phonon calculation – rigorous, but far more SIESTA jobs. The isolated-adsorbate reference ALWAYS uses the 'local' (full-Hessian-of-the-whole-molecule) method regardless of this flag – there's no periodic substrate phonon to subtract for an isolated molecule in a vacuum box, so Phonopy's supercell machinery buys nothing there. choices: local, full

`--displacement DISPLACEMENT`
: Finite-difference displacement in Ang for the Hessian/Phonopy displacements (default: 0.015). type: float

`--gibbs-supercell GIBBS_SUPERCELL GIBBS_SUPERCELL GIBBS_SUPERCELL`
: Supercell dimensions for `--zpe-mode full`'s site+clean-slab phonon calculations (default: 1 1 1 – appropriate when the slab's own in-plane supercell is already large enough). type: int

Like every STB command it also accepts `-h`/`--help`, `-v`/`--version` and `--no-intro` (Section 3.3).

From the built-in help.

```
Usage examples:
  stb-adsorbAnalysis
  stb-adsorbAnalysis --dir adsorption_run --file calc.out
  stb-adsorbAnalysis --apply best_structure.fdf
```

11.8.3.4 `stb-adsorbGibbs`

Stage 4 of the Adsorption workflow: combines the vibrational Hessian/phonon folders `stb-adsorbAnalysis --compute-gibbs` (Stage 3) wrote into a Gibbs free energy of adsorption, DG = E_ads(BSSE if available) + DZPE - T*DS. DZPE/DS = [ZPE/S(site) - ZPE/S(isolated adsorbate reference)], both computed with the SAME method (local partial-Hessian, or full Phonopy for the site side when Stage 3 used `--zpe-mode` full) – the isolated reference always uses the local, full-molecule Hessian (there's no periodic substrate phonon to subtract for a molecule in a vacuum box, so Phonopy's supercell machinery buys nothing there; for a single-atom adsorbate this correctly gives ZPE=S=0, since a free atom has no vibrational modes at all once its 3 translational modes are excluded). KNOWN LIMITATION, always printed in the report: the isolated-reference term above is VIBRATIONAL/HARMONIC ONLY – it does NOT include the translational/rotational entropy a real gas molecule has from being free to move/rotate in 3D (an ideal-gas partition-function term this tool does not compute). For a single-atom adsorbate this is exact (no rotation/translation to begin with beyond the excluded trivial modes); for a polyatomic adsorbate it is an approximation.

Synopsis.

```
usage: stb-adsorbGibbs [-h] [--dir DIR] [--file FILE] [--tmin TMIN] [--tmax TMAX]
                       [--tstep TSTEP] [--force-tolerance FORCE_TOLERANCE]
                       [--save-report] [--view-plots] [-v] [--no-intro]
```

Options

`--dir DIR`
: Root directory containing 'gibbs/', 'sites/', 'clean_slab/' etc. (default: adsorption_run – `stb-adsorb's` own default `--output-dir`).

`--file FILE`
: SIESTA output filename inside each folder (default: `calc.out`).

`--tmin TMIN`
: Sweep start, Kelvin (default: 200). type: float

`--tmax TMAX`
: Sweep end, Kelvin (default: 400). type: float

`--tstep TSTEP`
: Sweep step, Kelvin (default: 25). type: float

`--force-tolerance FORCE_TOLERANCE`
: Residual atomic force in eV/Ang (default: 0.05) above which a folder is flagged as possibly not converged/relaxed. Advisory only. type: float

`--save-report`
: Also persist the report to `adsorption_gibbs_report.txt`. Off by default. switch, off by default

`--view-plots`
Also show the DG-vs-T plot on screen, instead of only saving it as a PNG. Off by default.switch, off by default

Like every STB command it also accepts `-h`/`--help`, `-v`/`--version` and `--no-intro` (Section 3.3).

From the built-in help.

```
Usage example:
  stb-adsorbGibbs --dir adsorption_run --tmin 100 --tmax 500
```

11.9 NEB / Reaction Path

At a glance

Menu code: 5.9 (a multi-stage workflow)

Code	Stage	Command
5.9.1	Stage 1 - Prep	`stb-neb`
5.9.2	Stage 2 - Analysis	`stb-nebAnalysis`

Guided example: `examples/5.9-neb/`

Summary: Interpolate a reaction-path band between two already-relaxed endpoint structures you provide (verified same lattice + composition), optionally refined with MACE-MP-0, then compute the DFT-level energy profile and an approximate barrier.

11.9.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/5.9-neb/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

A reaction or diffusion barrier is the energy cost of the **highest point** along the path connecting two already-relaxed structures — an adsorbate hopping to a neighboring site, a vacancy migrating, a bond breaking and reforming. The Nudged Elastic Band (NEB) method finds that path, and its saddle point, without you having to guess where it is by hand. This item has **two** stages, plus one CLI-only tool that sits between them:

- **Stage 1 (`stb-neb`, code `5.9.1`)** takes two already-relaxed endpoint structures you provide directly, interpolates a chain of images between them, optionally shapes that chain with a real climbing-image NEB on the MACE-MP-0 foundation model, and writes single-point SIESTA folders for every image.
- **`stb-nebCycle`** (CLI-only, **not** in the interactive `stb-suite` menu — see Section 6) takes one real-DFT NEB step at a time, meant to be called from your own cluster submission script's loop, alternating with real SIESTA runs.
- **Stage 2 (`stb-nebAnalysis`, code `5.9.2`)** reads every image's SIESTA energy back, estimates the barrier (both from the highest discrete image and from a spline fit), and re-validates the band as it actually exists on disk right now — not just what Stage 1 reported at prep time.

`example_5.9.sh` is a guided, runnable walkthrough of all of this — **including two live demonstrations of exactly the things this README was asked to make clear: what stb-neb requires of your input files (Section 2), and what happens when those requirements aren't met yet (the same section, live).**

11.9.1.1 The theory, briefly

Plain linear interpolation between two endpoints is not a reaction path — it is one **specific, usually wrong** guess at one. Two well-known failure modes motivate everything the rest of this document explains:

- **Corner-cutting.** A straight-line interpolation between two Cartesian endpoints can walk an atom directly through where another atom already sits, if the true reaction path has to bend around it.
- **Uneven spacing / kinks.** Nothing keeps successive images evenly spread out along a curved path — they can bunch up in easy regions and leave gaps in the interesting (high-energy) one, wasting SIESTA time and under-resolving the actual barrier.

NEB fixes the second problem directly: it connects neighboring images with virtual springs (spring constant `k`, `--ml-k`/`--k` below) and relaxes every image **perpendicular** to the path (the springs' own force along the path keeps the spacing even; nothing but the springs acts along it). The **climbing-image** refinement (Henkelman & Jónsson, 2000) then turns this off for exactly one image — the current highest-energy one — and lets it climb **uphill** along the path direction too, so it converges onto the true saddle point instead of staying at whichever quantized image happened to sample it best. `stb-neb`/`stb-nebCycle` run this in two stages, always: stage 1 with climbing disabled (shape the whole band first), stage 2 (or, for the cluster loop, cycles past `--climb-after`) with it enabled — climbing too early risks locking onto the wrong image as "the" saddle before the band has even found its overall shape.

`--idpp` (Image Dependent Pair Potential — Smidstrup, Pedersen, Vegge, Jónsson, 2014) is a cheaper, purely classical/geometric alternative that runs **before** any of the above, directly on the plain linear interpolation: it smooths the path by minimizing pairwise interatomic distance changes, with no notion of chemistry or energy at all. It fixes corner-cutting far more reliably than linear interpolation alone, at essentially no cost — reach for it whenever the endpoints differ by more than just the reacting atom(s) themselves (Section 4 explains the advisory that tells you when).

11.9.1.2 Input file requirements — read this before running your own study

This is the part of the workflow the tool can only partially check for you. Two of the four requirements below are **hard, enforced, [ERROR]-and-exit checks**; the other two are **entirely your own responsibility** — `stb-neb` has no way to verify them and does not try to.

2.1 Enforced: `--initial`/`--final` must already be relaxed geometries. `--initial`/`--final` each accept either a bare `.fdf` file (used exactly as given — no relaxation-status concept applies to a bare file) or a **directory**, in which case `stb-neb` looks for that directory's own finished `siesta.XV` and prefers it over the pre-relaxation `structure.fdf` guess sitting next to it (`structure_io.read_relaxed_or_input`, the same helper `stb-adsorbBsse` uses for the analogous "read the RELAXED geometry, not the guess" need). If no `siesta.XV` exists yet, `stb-neb` still runs — useful to preview the path — but prints a `[WARNING]`:

```
[WARNING] init_dir, final_dir: no finished 'siesta.XV' found in this directory yet --
using the pre-relaxation structure.fdf guess instead. stb-neb officially expects
already-relaxed endpoints ...; the band below is still generated, useful to preview
the path, but re-run once SIESTA has actually relaxed this endpoint.
```

`example_5.9.sh`'s `output/endpoints_unrelaxed/` section triggers this warning live, then fabricates a `siesta.XV` and re-runs to show it go away (`output/endpoints_relaxed/`).

Why this matters, not just as a formality: every image folder Stage 1 writes is a **single-point** evaluation (Section 3) — nothing ever relaxes an endpoint further once it's inside this workflow. If `--initial`/`--final` are not actually at their true relaxed geometry, every energy along the "path" is measured relative to the wrong starting and ending points, and the reported barrier is not a real reaction barrier at all, just a number computed on an arbitrary pair of structures.

2.2 Your own responsibility: `calc.fdf` must match the level of theory `--initial`/`--final` were actually relaxed at. `stb-neb` has no way to know what basis set, k-point density, mesh cutoff, or XC functional you used to relax `--initial`/`--final` in the first place — it only reads their **geometry**. Whatever `--calc` template you pass here is copied into every `image_NN/`, with `MD.TypeOfRun`/`MD.Steps` forced to a single-point evaluation (unconditionally, Section 3) — but **everything else in that template is used exactly as you wrote it**, including basis size, k-grid, and XC functional.

If those settings **don't** match what actually produced `--initial`/ `--final`'s relaxed geometry, the energies `stb-nebAnalysis` compares are not on a consistent footing with each other — you'd be comparing a single-point energy computed with (say) a DZP basis against a geometry that was only ever actually a minimum at SZP. This is exactly the same "level-of-theory propagation" concern `stb-adsorbBsse`'s BSSE folders and `stb-hubbardu`'s perturbation probes already have to get right (the manual calls this out as a repeated, previously-buggy pattern across this suite) — here, it's on you to get it right by hand, since `--initial`/ `--final` can come from **any** prior calculation, not necessarily one this suite produced (so there's no `config_extra.fdf`/report file for `stb-neb` to read the prior level of theory back out of, unlike those other workflows).

A practical checklist, before trusting any barrier this workflow reports:

- Same `PAO.BasisType`/`PAO.BasisSize`/`PAO.EnergyShift` as the calculation that relaxed `--initial`/`--final`.
- Same `kgrid.MonkhorstPack` density (or at least dense enough that the energy is converged with respect to it — Section 4.1's own `stb-kgrid` walkthrough covers how to check this).
- Same `Mesh.CutOff`/`FilterCutoff`.
- Same `XC.Functional`/`XC.Authors`.
- Same `DFTD3`/`Spin` settings, if either mattered for the original relaxation.

`calc.fdf` in this folder documents this directly at the top: its `MD.TypeOfRun CG` / `MD.Steps 100` represent whatever relaxation you'd have used to converge `initial.fdf`/`final.fdf` in the first place — and are the one part of the file `stb-neb` silently ignores and overrides, since only the geometry-determining settings above actually matter to a single-point evaluation.

Enforced: same composition, same lattice.

```
[ERROR] Initial and final structures have different composition (H: 1 vs 0) --
NEB requires the exact same atoms on both endpoints (a reaction that creates or
destroys atoms is not a reaction path in this sense).
```

```
[ERROR] Initial and final structures have different lattices (largest component
difference: 0.1200 Ang, tolerance: 0.001 Ang) -- ase.mep.neb.NEB and pymatgen's
interpolation both require every image to share one exact cell (no variable-cell
NEB support in ASE). stb-neb requires --initial/--final to already share the same
lattice ... re-relax --final with --initial's own cell ... before retrying.
```

Both are hard `[ERROR]`s, exit 1, nothing written — `example_5.9.sh` triggers both live (`output/composition_mismatch/`, `output/lattice_mismatch/`). Composition must match exactly (a reaction that creates or

destroys atoms is a different kind of problem — thermodynamics, not a reaction path). The lattice check has a **1e-3 Ang** tolerance for ordinary floating-point/rounding noise between two independently-converged calculations (below it, `--final` is silently rebuilt bit-for-bit onto `--initial`'s own lattice matrix, no warning needed); above it, `stb-neb` refuses outright rather than guessing which endpoint's cell is "right" — **this used to silently override one with the other**, which is worse than refusing, since a real physical difference in relaxed lattice constant between two calculations usually means something is actually wrong (not fully converged, or a genuinely different structure), not that one should just be discarded.

This check exists because no downstream library involved supports a per-image cell change at all: `ase.mep.neb.NEB/idpp_interpolate` raise `NotImplementedError` on any per-image cell mismatch, and pymatgen's `Structure.interpolate` raises `ValueError` on unequal lattices unless told to interpolate them too (which `stb-neb` deliberately never does, for the same reason Section 2.1 of `examples/5.8-adsorption/README.md` keeps a slab's cell fixed during adsorption: a single reacting/adsorbing atom has no physical business changing a bulk lattice constant that's common to every image on the band).

Atom correspondence between the two endpoints. A subtler, related requirement: `--initial`/`--final` must list their atoms in an order pymatgen can match up correctly — `--autosort-tol` (default 0.5 Ang, pymatgen's own suggested value) matches atom `i` in one endpoint to whichever atom in the other sits within that distance. If both files were derived from the same base structure (the common case — e.g. one is the other with a single atom moved), they already share the exact same atom order; pass `--autosort-tol 0` to use it directly instead of re-matching by distance (more robust for a small/densely-packed cell, where several atoms of the same species can sit within any reasonable distance of each other). If matching fails at the requested tolerance, `stb-neb` automatically retries once at `--autosort-tol 0` (a `[WARNING]`, not a dead end) before giving up.

11.9.1.3 Why every image is forced to a single-point evaluation

Every `image_NN/config_extra.fdf` Stage 1 writes contains, unconditionally (no opt-out flag — this is the one thing about an image folder that isn't configurable):

```
MD.TypeOfRun        CG
MD.Steps             0
```

`%include`-d ahead of your own `--calc` template (same `config_extra.fdf`

- `%include` convention used everywhere else in this suite — SIESTA's fdf reader is first-occurrence-wins for a duplicate directive, so the override has to physically precede anything your own template already sets). Each image is one independent, uncoupled sample point **on** the path — if SIESTA were allowed to relax it, it would walk straight downhill off the path and back toward whichever endpoint is closer, destroying the whole point of having a chain of images in the first place. The NEB coupling between images (the spring force, and the climbing-image correction) is applied by `stb-neb`'s own MACE relax (modes 1/2) or by `stb-nebCycle` (the real-DFT refinement loop) — never by SIESTA's own internal relaxation.

`--force-spin`/`--force-vdw`/`--force-dipole` (all default **ON**, same `config_extra.fdf` block constants `core/adsorption_sites.py` already defines) additionally force `Spin polarized`/`DFTD3 .true.`/`Slab.DipoleCorrection .true.` on every image, for the same reasons `stb-adsorb`'s own reference folders do (a reacting/adsorbing system commonly has a non-zero net spin; dispersion matters for any physisorption-like point on the path; a one-sided slab reaction breaks inversion symmetry along the vacuum axis). `--no-force-*` opts out of any of the three individually if your system genuinely doesn't need it (e.g. `--no-force-dipole` for a bulk/fully-periodic reaction path with no vacuum gap at all).

11.9.1.4 The three modes

```
$ stb-neb -i initial.fdf -f final.fdf -c calc.fdf --mode {1,2,3}
```

Mode	Path engine	Output	SIESTA's role
1	100% MACE-MP-0 (climbing-image NEB), then ONE single-point read	`image_NN/`	one round, energy only
2 (default)	MACE-MP-0 shapes the path, then a FEW real-DFT refinement cycles	`cycle_00/image_NN/` + printed loop	several rounds, real forces
3	100% real-DFT NEB from a plain interpolated path, no MACE at all	`cycle_00/image_NN/` + printed loop	every round

A pure MACE-MP-0 path with **no** SIESTA at all — useful for a fast, purely-ML barrier estimate, e.g. screening many candidate diffusion events before committing to any real DFT — is **not** a mode here at all; that capability lives exclusively in `stb-mlneb` (ML Simulations menu, code `7.5`), which already existed as a dedicated, pure-MACE extraction of the same underlying interpolation/NEB mechanics `stb-neb` itself reuses (`neb.py`'s own `check_composition_match`, `wrap_into_cell`, `linear_interpolate_images`, `idpp_refine_images`, `cumulative_reaction_coordinates`, `check_path_quality`, `write_path_trajectory`, `write_ml_preview_plot` are all shared, not duplicated, between the two tools).

Choosing between 1/2/3:

- **Mode 1** is the cheapest way to get a real reaction-path estimate: MACE-MP-0 does all the path-shaping work, SIESTA is only asked to confirm each image's energy once. Good for a quick look, or for a system MACE-MP-0's foundation training already covers reasonably well.
- **Mode 2** (the default) is the right choice for a production barrier: MACE-MP-0 gives the real-DFT refinement loop a physically sensible starting shape (fewer `stb-nebCycle` rounds needed to converge), and `--climb-after 0` is safe from the very first cycle, since the band already has a sensible overall shape by the time SIESTA sees it.
- **Mode 3** skips MACE-MP-0 entirely — appropriate when you don't trust (or don't have) a MACE-MP-0 potential for your chemistry, at the cost of needing more real-DFT refinement cycles, and a later `--climb-after` (the printed cluster snippet suggests 5, vs. mode 2's 0) — climbing too early on a raw interpolated path risks locking onto the wrong image as the saddle before the band has found its shape.

`--ml-freeze-substrate` (default ON, modes 1/2 only) freezes any atom whose position barely differs between `--initial`/`--final` (`--ml-freeze-threshold`, default 0.3 Ang) during the MACE relax — fewer degrees of freedom for the optimizer, and it stops a spectator atom from spuriously wandering during a fast/loose ML relax. `--ml-prerelax-endpoints` (independent of `--mode`) additionally relaxes both endpoints' own positions with MACE-MP-0 before interpolating — a cheap safety net, not a substitute for Section 2.1's real requirement.

Interactive path: `stb-suite`, type `5.9.1` — same questions, same defaults, as the flags above. `example_5.9.sh` proves the two agree.

11.9.1.5 5. Stage 1 mechanics, live (`stb-neb`, code `5.9.1`)

```
$ stb-neb -i initial.fdf -f final.fdf -c calc.fdf -n 5 --mode 3
```

writes everything under a self-contained `neb_run/` subfolder of `--output-dir` (matching `stb-adsorb`'s own `sites/`-nesting convention), never `--output-dir` directly — `stb-nebCycle`/`stb-nebAnalysis --dir`

must point at <output-dir>/neb_run, not <output-dir> itself (both default to exactly that, so most of the time you never need to think about it).

Report structure ([0]–[9], always saved to neb_run/neb_setup.txt):

```
[0] RUN METADATA
[1] ML PRE-RELAX ENDPOINTS      (only with --ml-prerelax-endpoints)
[2] ENDPOINT CHECKS
[3] PATH INTERPOLATION
[4] MACE-MP-0 PATH SHAPING      (modes 1/2 only)
[5] PATH QUALITY
[6] IMAGE FOLDERS
[7] CLUSTER SUBMISSION          (modes 2/3 only)
[8] SUMMARY
[9] LIBRARY WARNINGS
```

[5] always writes neb_run/neb_path.xyz — every image, one multi-frame extended-XYZ file, viewable directly in VESTA/OVITO/ASE-GUI — **before** any real-DFT refinement has happened (it's the pre-refinement, MACE -shaped-or-not path). [5] also runs check_path_quality: a [WARNING] if two neighboring images are nearly identical (step below 0.05 Ang — consider fewer -n) or if one step is unusually large relative to the mean (usually a sign of a bad --autosort-tol match, not a genuinely non-uniform path, unless atom correspondence is already proven — Section 7). check_endpoint_displacement (in [2]) separately recommends --idpp whenever a broad fraction of the structure (not just the reacting atom itself) moved between the two endpoints.

Interactive path: stb-suite, type 5.9.1.

11.9.1.6 6. stb-nebCycle mechanics, live (CLI-only — modes 2/3 only)

```
$ stb-nebCycle --dir neb_run --fmax 0.05 --climb-after 0
```

Deliberately **not** registered in the interactive stb-suite menu (see "CLI-only tools" in the manual's developer reference) — it's meant to be called once per real -DFT NEB step, from inside your own cluster submission script's loop (Stage 1's own printed snippet, Section 4 above, is exactly that loop), not from an interactive session. Each call:

1. Reads the **latest** cycle_NN/image_*/ with finished SIESTA results (calc.out energy + <SystemLabel>.FA forces).
2. Takes exactly **one** climbing-image NEB step, using ASE's FIRE optimizer — its velocity/timestep state is persisted to neb_cycle_state.json between calls (ase.optimize.FIRE's own native restart mechanism), since each call is a separate, short-lived process with no live optimizer to keep running in memory between real SIESTA jobs that may finish hours apart.
3. Either writes cycle_{N+1}/ (the next geometry — structure.fdf/ calc.fdf only, no calc.out yet, waiting for you to run SIESTA there) or, once the max residual force drops below --fmax, writes a NEB_CONVERGED sentinel file and stops — your submission-script loop should check for this file and break out (the printed snippet already does).

--climb-after N only enables the climbing-image correction once the **current** cycle number is >= N — 0 (mode 2's suggestion) is safe once MACE-MP-0 has already shaped the band; a later value (mode 3's suggestion: 5) gives a raw interpolated path a few cycles to find its overall shape first, same reasoning core/mace_relax.py::relax_neb's own two-stage climb=False-then-True approach uses, just spread across separate cluster-queued cycles instead of one live process.

example_5.9.sh's output/mode2_cycle/ section fabricates two rounds of .FA/calc.out (a smooth harmonic "pull toward a fixed target" force field — **not** random noise, which was found live to trigger a real ASE improvedtangent 0/0 tangent edge case for two near-degenerate images) and shows both outcomes: a normal step (cycle_01/ written) and, with a looser --fmax, convergence (NEB_CONVERGED written).

11.9.1.7 7. Stage 2 mechanics, live (stb-nebAnalysis, code 5.9.2)

```
$ stb-nebAnalysis --dir neb_run --save-report --save-gnuplot --save-path-xyz
```

Report structure ([0]–[7], --save-report persists it to neb_run/neb_report.txt):

```
[0] RUN METADATA
[1] IMAGE ENERGIES
[2] CONSISTENCY CHECKS
[3] BARRIER ANALYSIS
[3b] BARRIER VS. CYCLE          (modes 2/3 only, once 2+ complete cycles exist)
[4] ENERGY PROFILE PLOT
[5] SUMMARY
[6] APPLY                       (only with --apply)
[6b] PATH EXPORT                (only with --save-path-xyz)
[7] LIBRARY WARNINGS
```

--dir defaults to neb_run (auto-detected as . instead if you're already standing inside it — e.g. right after Stage 1's own printed cluster-submission snippet's cd "$run_root"). For modes 2/3, it auto-discovers the **right** cycle to analyze: not necessarily the highest-numbered cycle_NN/ — while not yet converged, that folder is always stb-nebCycle's freshly-written **next** geometry guess, with zero SIESTA results yet (a real, previously-live bug reproduced against a real 30+-cycle run: analyzing the highest-numbered folder between two stb-nebCycle calls used to hard-fail with "No valid image results found" instead of reporting the previous, real cycle's numbers). Once NEB_CONVERGED exists, the highest-numbered cycle correctly **is** the converged one.

[2] CONSISTENCY CHECKS re-validates the band **as it exists on disk right now** — not just what Stage 1 reported at prep time: declared vs. found image count, reaction-coordinate monotonicity, cross-image composition, a recomputed path length cross-checked against Stage 1's own neb_setup.txt record, and a plausibility check on the spline-fitted transition state (warns if it sits within 5% of either endpoint). [3] reports both the discrete highest-energy image's barrier and a cubic-spline-fitted one (needs >= 4 images with distinct reaction coordinates) — a smoother, less grid-dependent estimate, the DFT-side analogue of stb-neb's own NEBTools.get_barrier(fit=True) for the MACE band. --apply <path> copies the highest-energy image's structure.fdf (the approximate transition-state guess) forward; --save-path-xyz writes the **currently**-analyzed band (whichever cycle [0] reports, converged or not) as neb_run/neb_path_current.xyz — distinct from Stage 1's own neb_path.xyz, which is the pre-refinement path and is never updated afterwards.

Interactive path: stb-suite, type 5.9.2.

11.9.1.8 Worked example: a symmetric hop, live numbers

example_5.9.sh's output/mode3/ section fabricates calc.out for all 5 images of a mode-3 (no MACE, fastest) band, hand-chosen (same "no real SIESTA binary available" convention every example in this suite uses) so that image_00 and image_04 — physically **equivalent** carbons, by the lattice's own symmetry — share the exact same energy:

Image	E (eV)	Max force (eV/Ang)
image_00	-300.000000	0.010000
image_01	-299.700000	0.015000
image_02	-299.500000	0.400000
image_03	-299.700000	0.015000
image_04	-300.000000	0.010000

stb-nebAnalysis's report, verbatim:

```
Highest-energy image (approx. TS) : image_02  (E = -299.500000 eV)
Forward barrier  (TS - initial)   : 0.500000 eV
Backward barrier (TS - final)     : 0.500000 eV
Reaction energy  (final - initial) : 0.000000 eV, endothermic (unfavorable)
Spline-fitted barrier (smoothed)  : 0.500000 eV at reaction coordinate 0.7101 Ang (cubic spline
through the 5 energies)
```

Forward and backward barriers coming out **identical**, and the spline-fitted transition state sitting at **exactly 50%** of the path ([2]'s plausibility check confirms this explicitly), is exactly what a genuinely symmetric hop between two equivalent sites should produce — were example_5.9.sh to have injected an asymmetric pair of energies instead, this cross-check is what would have caught it. (The [3] wording "endothermic (unfavorable)" at a reaction energy of exactly 0 is a labeling edge case worth knowing about, not a bug: the check is a plain < 0/>= 0 split, so an exactly thermoneutral reaction — the 0.000000 eV here — prints on the "unfavorable" side of that split by convention, rather than a separate "thermoneutral" label.)

The deliberately large residual force on image_02 (0.400000 eV/Ang, well above --force-tolerance's default 0.05) also fires the expected advisory:

```
[WARNING] 1 image(s) have residual force above --force-tolerance (0.05 eV/Ang), possibly
not single-point-converged: image_02.
```

— never fatal to the barrier estimate, just a flag that this particular image's own SCF/relaxation quality is worth a second look before trusting the number too far.

11.9.1.9 Known, deliberate limitations

- **No SIESTA run is ever performed by this suite.** stb-neb and stb-nebCycle generate/read input and output files; you run SIESTA yourself in every image_NN/, alternating with stb-nebCycle calls for modes 2/3.
- **stb-neb cannot verify Section 2.2's level-of-theory requirement.** Unlike stb-adsorbBsse/stb-hubbardu, which read a PRIOR stage's own config_extra.fdf to inherit the right settings automatically, --initial/ --final can come from any prior calculation at all — there is no report file for stb-neb to read a "what level of theory was this relaxed at" answer back out of. Getting calc.fdf right is entirely on you (the checklist in Section 2.2).
- **The lattice-mismatch tolerance (1e-3 Ang) is a hard-coded constant**, not user-configurable — appropriate for ordinary floating-point noise between two independently-converged calculations, not for a genuinely different (even if small) relaxed lattice constant, which is refused outright rather than silently rebuilt.
- **A single fixed lattice for the whole band is a hard requirement, not a simplification this tool chose** — no variable-cell NEB exists in either ase.mep.neb or pymatgen's interpolation, so there is no "relax the cell too" option to offer even if it were physically desirable for your reaction

(e.g. a reconstruction that genuinely changes the lattice constant along the way is out of scope entirely).

- **The reported barrier is only as good as the path.** Mode 3's plain interpolated-then-real-DFT-refined path, without a MACE-shaped starting guess, may need many more `stb-nebCycle` rounds to reach a physically meaningful saddle point than modes 1/2 — `[3b] BARRIER VS. CYCLE`'s convergence-history plot (once 2+ complete cycles exist) is the honest way to check whether the barrier has actually settled down, rather than trusting a single early cycle's number.
- **`--ml-freeze-substrate`'s threshold (0.3 Ang default) is a blunt, global cutoff** — every atom below it is frozen during the MACE stage, regardless of whether it's truly a spectator or just happens to move a little; for a reaction with genuine, broad substrate relaxation, lower the threshold or use `--no-ml-freeze-substrate`.
- **Mode 1's single-point SIESTA read has no coupling between images at the DFT level at all** — Section 4's caveat table calls this out directly (`stb-nebAnalysis`'s own `[3]` prints a matching `[NOTE]` for this exact case): a real chemical effect MACE-MP-0's foundation training doesn't capture well can still produce a wrong path shape that SIESTA never gets a chance to correct, since nothing relaxes the images further at the DFT level in this mode.
- **A pure MACE-MP-0 path with no SIESTA at all is not available here** (Section 4) — use `stb-mlneb` (ML Simulations menu) if that's genuinely what you want; forcing it through this workflow's modes isn't possible by design, not an oversight.

11.9.1.10 Step-by-step: running this workflow on your own structures

1. Have two **already-relaxed** endpoint structures (Section 2.1) sharing the exact same composition and lattice (Section 2.3), and a `calc.fdf` template using the **same level of theory** that relaxed them (Section 2.2).
2. Run Stage 1: `stb-neb -i <initial> -f <final> -c <calc.fdf> --mode 2` (or `1/3` — Section 4). Read `[2]`/`[5]`'s advisories before proceeding — consider `--idpp` if recommended.
3. For mode 1: run SIESTA yourself in every `neb_run/image_NN/`, then skip to step 5.
4. For modes 2/3: run SIESTA yourself in every `neb_run/cycle_00/image_NN/`, then loop `stb-nebCycle --dir neb_run --fmax <target> --climb-after <N>` (Stage 1's own printed snippet is ready to paste into a real submission script), running SIESTA in each newly-written `cycle_{N+1}/image_NN/` in between, until `neb_run/NEB_CONVERGED` appears.
5. Run Stage 2: `stb-nebAnalysis --dir neb_run --save-report --save-gnuplot --save-path-xyz`. Read `[2]` for the re-validated consistency checks and `[3]` for the barrier — both the discrete and spline-fitted estimates. For modes 2/3, check `[3b]`'s convergence history once available.
6. `--apply <path>` if you want the approximate transition-state structure carried forward for a separate, dedicated TS-refinement calculation.

11.9.1.11 Files in this folder

File	Purpose
`structure.fdf`	Bare 2-atom graphene primitive cell (same as `examples/5.8-adsorption/structure.fdf`) — not itself fed to `stb-neb`; kept as the documented starting point `initial.fdf`/`final.fdf` were built from (Section 10 of this file explains how, via `stb-supercell`/`stb-adsorb`).
`initial.fdf` / `final.fdf`	A 3x3 graphene supercell (18 C) with a single H atom adsorbed ontop of two nearest-neighbor carbons — a small, genuine surface-diffusion hop, not an arbitrary displacement. Treated by this walkthrough as already-relaxed (Section 2.1).
`calc.fdf`	Shared `calc.fdf` template (non-polarized, Gamma-only, modest basis/mesh — mechanics only, not physically converged; documents Section 2.2's level-of-theory point directly at the top).

File	Purpose
`example_5.9.sh`	The guided walkthrough (**not** an automated test — see `test/5-workflow/9-neb/{prep,cycle,analysis}/test.sh` for that). Pauses between sections so you can read before moving on; safe to re-run.
`output/`	Created by `example_5.9.sh` when you run it (git-ignored, not checked in). See below.

11.9.1.12 Running the script

```
./example_5.9.sh
```

Case	Command(s)	What it shows
`output/endpoints_unrelaxed/`	`stb-neb -i init_dir -f final_dir --mode 3`	Section 2.1's `[WARNING]`, live, when no `siesta.XV` exists yet
`output/endpoints_relaxed/`	same, after fabricating `siesta.XV` in both	the warning gone, same command otherwise
`output/composition_mismatch/`	`stb-neb` against a deliberately-mismatched `final.fdf`	Section 2.3's hard `[ERROR]`, composition
`output/lattice_mismatch/`	`stb-neb` against a deliberately-rescaled `final.fdf`	Section 2.3's hard `[ERROR]`, lattice
`output/mode3/`	`stb-neb --mode 3` then `stb-nebAnalysis`	Stage 1/2 mechanics, single-point folders, **the worked example (Section 8)**
`output/mode1/` (needs the `ml` extra)	`stb-neb --mode 1 --ml-max-steps 30`	a real MACE-MP-0 climbing-image relax shaping the path
`output/mode2_cycle/`	`stb-neb --mode 2` then `stb-nebCycle` x2	the cluster-submission snippet, one real-DFT NEB step, then convergence
(no folder — a diff only)	Stage 1 via `printf ... | stb-suite`	proof the interactive menu (`5.9.1`) agrees with the CLI

11.9.1.13 What's next

- **`stb-mlneb`** (`7.5`, ML Simulations menu) — the same interpolation/climbing-image-NEB machinery with every force from a MACE model, for a fast pre-DFT barrier estimate. The band needs no `--calc`/pseudopotentials. With a fine-tuned model it can also check the band with SIESTA single points (`--siesta-check`), send it to the MLFF training set (`--to-mlff`), and write the images (`--save-images`) to start this workflow from. Its guided example, `examples/7.5-mlneb/` (Section 13.5), shows why the check matters: a vacancy hop in silicon at 0.72 eV with the model, against 0.25 eV with SIESTA on the same path.
- **`stb-kgrid`/`stb-mlrelax`** — useful before this workflow even starts, for getting `--initial`/`--final` themselves converged and relaxed in the first place (Section 2.1/2.2).

11.9.2 Implementation details

Nudged elastic band

A band of N images $\mathbf{R}_0, \ldots, \mathbf{R}_{N-1}$ joins two minima. Each interior image feels the true force perpendicular to the path and a spring force along it,

$$\mathbf{F}_i = -\nabla E(\mathbf{R}_i)\big|_{\perp} + k\left(|\mathbf{R}_{i+1} - \mathbf{R}_i| - |\mathbf{R}_i - \mathbf{R}_{i-1}|\right)\hat{\boldsymbol{\tau}}_i,$$

with the tangent $\hat{\boldsymbol{\tau}}_i$ of Henkelman and Jónsson [73, 74]. In the climbing-image variant [75] the

highest image feels no spring and the force component along the tangent inverted, so it climbs to the saddle point; the barrier is $E_{\mathrm{saddle}} - E_0$. IDPP interpolation [76] gives a better initial band than a straight line when atoms move past each other.

Endpoints.. `-i/-f` are two *already relaxed* structures (a `.fdf`, or a folder whose finished `.XV` is preferred). They must have the same composition and the same lattice within 10^{-3} Å; otherwise the tool stops with an `[ERROR]` rather than guessing. Atom correspondence is proven by a `neb_manifest.json` on both sides when present, otherwise found by distance matching with `-autosort-tol` (default 0.5 Å), retried once with the given order (`-autosort-tol 0`) if matching fails. Coordinates are wrapped into the cell first, and a warning is printed when a large part of the structure moves between the endpoints without `-idpp`. `-ml-prerelax-endpoints` re-relaxes both endpoints with MACE when you are unsure they are minima.

Folders.. Everything goes to `<output-dir>/neb_run/`: the report `neb_setup.txt` (with a `# MODE: N` marker the analysis reads), the initial band `neb_path.xyz`, and either `image_NN/` (mode 1) or `cycle_00/image_NN/` (modes 2 and 3). Every image is a SIESTA single point (forced in its `config_extra.fdf`), with spin polarization, DFT-D3 and the slab dipole correction forced on by default (`-no-force-spin`, `-no-force-vdw`, `-no-force-dipole` to leave the template's own settings).

The three modes.. Mode 1 shapes the whole path with a MACE climbing-image NEB (spring `-ml-k`, default 0.1 eV/Å^2; first without, then with climbing; atoms that barely move between endpoints – less than `-ml-freeze-threshold`, 0.3 Å – frozen) and SIESTA only evaluates each image once. Mode 2 (default) uses the MACE path as the starting band of a real-DFT NEB; mode 3 starts the real-DFT NEB from the interpolated band. A pure-MACE NEB is **`stb-mlneb`** (7.5); its `-save-images` gives a starting band for this workflow, and its `-siesta-check` measures the model against SIESTA on that band first.

The real-DFT loop (`stb-nebCycle`).. For modes 2 and 3 the report prints a shell loop for your job script: run SIESTA in every `cycle_NN/image_MM/`, then call **`stb-nebCycle -dir neb_run`**, repeat. One call is one NEB step: it reads each image's energy (`-file`) and forces (`<label>.FA`), wraps them in ASE single-point calculators, takes one FIRE step [45] of the band (springs `-k`; climbing from cycle `-climb-after`, default 0), keeps the optimizer state on disk between calls, checks convergence against the forces of the step just evaluated (`-fmax`, default 0.05 eV/Å), and either writes `cycle_NN+1/` or reports convergence. Endpoints never move. The loop parameters (`-max-cycles`, `-siesta-exe`, `-mpirun-np`, `-conda-env`, `-cycle-fmax`) only shape the printed snippet.

Stage 2 (`stb-nebAnalysis`).. It reads the mode marker, analyses the latest complete cycle (modes 2 and 3) or the image folders (mode 1), checks SCF convergence and residual forces, and reports the energy profile against the cumulative reaction coordinate, the forward and reverse barriers and the highest image, with a convergence plot of the barrier over cycles once two cycles exist. Outputs: `neb_curve.dat` (`-o`), gnuplot files, the path as XYZ (`-save-path-xyz`), the highest image (`-apply`). In mode 1 the profile is a set of DFT energies on an ML path: the barrier is an estimate, not a converged saddle point.

11.9.3 Command-line reference

11.9.3.1 `stb-neb`

Prepares SIESTA folders for a NEB (Nudged Elastic Band) reaction-path study: interpolates images between two already-relaxed endpoint structures, optionally refining the path with MACE-MP-0 before writing single-point SIESTA folders per image.

Synopsis.

```
usage: stb-neb [-h] -i INITIAL -f FINAL -c CALC [-p PSEUDO_DIR] [--force-spin]
               [--no-force-spin] [--force-vdw] [--no-force-vdw] [--force-dipole]
               [--no-force-dipole] [-n N_IMAGES] [--autosort-tol AUTOSORT_TOL]
               [--idpp] [--idpp-fmax IDPP_FMAX] [--idpp-steps IDPP_STEPS]
               [--mode {1,2,3}] [--ml-model {small,medium,large}]
               [--ml-device {cpu,cuda}] [--ml-fmax ML_FMAX] [--ml-k ML_K]
               [--ml-max-steps ML_MAX_STEPS] [--ml-freeze-substrate]
               [--no-ml-freeze-substrate] [--ml-freeze-threshold ML_FREEZE_THRESHOLD]
               [--ml-prerelax-endpoints] [-O OUTPUT_DIR] [--max-cycles MAX_CYCLES]
               [--siesta-exe SIESTA_EXE] [--mpirun-np N] [--conda-env CONDA_ENV]
               [--cycle-fmax CYCLE_FMAX] [-v] [--no-intro]
```

Options

`-i, --initial INITIAL`
: Initial (already-relaxed) endpoint: a `structure.fdf` file, or a directory containing one – a directory's own finished `siesta.XV`, if present, is preferred over its `structure.fdf` guess. required

`-f, --final FINAL`
: Final (already-relaxed) endpoint – same file-or-directory convention as `--initial`, same composition. required

`-c, --calc CALC`
: `calc.fdf` template (kgrid, basis, XC, %include `structure.fdf`, etc.) – copied into every image_NN/ folder with MD.TypeOfRun/MD.Steps forced to a single-point evaluation. required

`-p, --pseudo-dir PSEUDO_DIR`
: Pseudopotentials source (optional): a bundled bank or a folder path.

`--force-spin`
: Force Spin polarized (via `config_extra.fdf`, overriding `--calc`'s own Spin tag if any) in every image_NN/ (default: ON) – a reacting atom commonly leaves the system with a net magnetic moment, which a spin-restricted calculation cannot represent at all; costs nothing for a genuinely closed-shell path (converges to zero moment). switch, off by default

`--no-force-spin`
: Leave `--calc`'s own Spin tag untouched in every image_NN/. switch

`--force-vdw`
: Force the Grimme DFT-D3 dispersion (van der Waals) correction (default: ON) in every image_NN/'s `config_extra.fdf`. switch, off by default

`--no-force-vdw`
: Leave `--calc`'s own dispersion settings untouched. switch

`--force-dipole`
: Force the slab dipole correction (default: ON) in every image_NN/'s `config_extra.fdf` – only meaningful for a genuinely one-sided slab/surface reaction path; harmless (evaluates near zero) otherwise. switch, off by default

`--no-force-dipole`
: Leave `--calc`'s own dipole settings untouched. switch

`-n, --n-images N_IMAGES`
: Total images along the band, endpoints included (default: 7). type: int

`--autosort-tol AUTOSORT_TOL`
: Atom-correspondence tolerance (Ang) for linear interpolation (default: 0.5, pymatgen's own suggested value for two endpoints that may have a different atom order). Pass 0 to skip distance -based matching entirely and use `--initial/--final`'s atom order directly – the right choice (and more robust for a small/densely-packed

cell) when both endpoints already share a guaranteed matching order (e.g. a neb_manifest.json-proven pair). If matching fails at this tolerance, `stb-neb` automatically retries once with `--autosort-tol 0` (a [WARNING], not an [ERROR]) before giving up – this is usually already the right correspondence, so most runs never need this flag set explicitly at all. type: float

`--idpp`

Refine the interior images with ASE's IDPP method after linear interpolation – a better initial guess than plain linear, no MACE needed. switch, off by default

`--idpp-fmax IDPP_FMAX`

default: 0.1; type: float

`--idpp-steps IDPP_STEPS`

default: 100; type: int

`--mode {1,2,3}`

How to build the path and what to write (default: 2). A pure MACE-MP-0 path with no SIESTA at all lives exclusively in `stb-mlneb` (ML Simulations menu), not as a mode here:
1 = 100% MACE-MP-0, then ONE single-point SIESTA per image -> image_NN/ (this is the old `--ml-neb` behavior).
2 = MACE-MP-0 shapes the path, then a FEW real-DFT NEB refinement cycles (via the separate `stb-nebCycle` CLI tool, run in your own submission-script loop) -> cycle_00/image_NN/ + a printed loop snippet.
3 = 100% real-DFT NEB from a plain interpolated path, no MACE at all -> cycle_00/image_NN/ + a printed loop snippet (same `stb-nebCycle` tool as mode 2, more cycles expected). choices: 1, 2, 3; type: int

`--ml-model {small,medium,large}`

default: medium; choices: small, medium, large

`--ml-device {cpu,cuda}`

default: cpu; choices: cpu, cuda

`--ml-fmax ML_FMAX`

default: 0.05; type: float

`--ml-k ML_K`

NEB spring constant, eV/Ang^2 (default: 0.1, ASE's own default). type: float

`--ml-max-steps ML_MAX_STEPS`

default: 200; type: int

`--ml-freeze-substrate`

With `--mode` 1/2: freeze atoms whose position barely differs between `--initial` and `--final` (default: ON) – fewer degrees of freedom for the MACE optimizer and avoids spurious drift of atoms that are spectators to the reaction. See `--ml-freeze-threshold`. switch, off by default

`--no-ml-freeze-substrate`

Let every atom relax during the MACE path-shaping stage (modes 1/2), even ones that don't move between `--initial` and `--final`. switch

`--ml-freeze-threshold ML_FREEZE_THRESHOLD`

Displacement threshold in Ang (default: 0.3) below which an atom is considered a spectator and frozen by `--ml-freeze-substrate`. type: float

`--ml-prerelax-endpoints`

Relax both endpoints' positions (cell fixed) with MACE-MP-0 before interpolating – independent of `--mode` (a cheap safety net when you're not fully certain the endpoints are already relaxed). Needs the optional 'ml' extra. switch, off by default

`-O, --output-dir OUTPUT_DIR`

Root directory (default: current directory) for every image_NN/.

`--max-cycles MAX_CYCLES`

Cluster-submission snippet only (--mode 2/3): max number of stb-nebCycle refinement cycles the printed loop will attempt before giving up (default: 30). Has no effect on this run itself. type: int

--siesta-exe SIESTA_EXE
Cluster-submission snippet only (--mode 2/3): SIESTA executable name or path for the printed loop to call (default: 'siesta', assumed already on PATH).

--mpirun-np N
Cluster-submission snippet only (--mode 2/3): prefix the SIESTA call in the printed loop with 'mpirun -np N'. Omit for no mpirun (default). type: int

--conda-env CONDA_ENV
Cluster-submission snippet only (--mode 2/3): conda environment name to activate at the top of the printed loop. Omit to print a generic placeholder reminder instead (default).

--cycle-fmax CYCLE_FMAX
Cluster-submission snippet only (--mode 2/3): the looped stb-nebCycle call's own --fmax (real-DFT NEB force-convergence threshold, eV/Ang, default: 0.05) – distinct from this tool's own --ml-fmax (the MACE-MP-0 path-shaping target). type: float

Like every STB command it also accepts -h/-help, -v/-version and -no-intro (Section 3.3).

From the built-in help.

```
Usage examples:
  stb-neb -i initial.fdf -f final.fdf -c calc.fdf -n 7
  stb-neb -i initial.fdf -f final.fdf -c calc.fdf --idpp
  stb-neb -i initial.fdf -f final.fdf -c calc.fdf --mode 1 --idpp --ml-k 0.2
  stb-neb -i initial.fdf -f final.fdf -c calc.fdf --mode 3
```

11.9.3.2 stb-nebAnalysis

Computes a NEB energy profile from an stb-neb band: reads each image_NN/'s SIESTA single-point energy and estimates the reaction barrier from the highest-energy image.

Synopsis.

```
usage: stb-nebAnalysis [-h] [--dir DIR] [--file FILE] [-o OUTPUT]
                       [--apply STRUCTURE_FDF] [--force-tolerance FORCE_TOLERANCE]
                       [--save-report] [--save-gnuplot] [--view] [--view-path]
                       [--save-path-xyz] [-v] [--no-intro]
```

Options

--dir DIR
Root directory containing every 'image_NN/' (default: 'neb_run', the self-contained run folder stb-neb --Stage 1 always writes its output into – auto-detected as '.' instead when the current directory already has its own neb_setup.txt, e.g. right after stb-neb's own printed cluster-submission snippet left you inside neb_run/ already).

--file FILE
SIESTA output filename inside each folder (default: calc.out).

`-o, --output OUTPUT`
Base filename for the gnuplot data file (default: `neb_curve.dat`), written under '<dir>/plot/' – only relevant with `--save-gnuplot`.

`--apply STRUCTURE_FDF`
Copy the highest-energy image's `structure.fdf` (the approximate transition-state guess) to this path.

`--force-tolerance FORCE_TOLERANCE`
Residual atomic force in eV/Ang (default: 0.05, same as `stb-adsorbAnalysis/stb-cohesiveAnalysis`) above which an image's `calc.out` is flagged as possibly not relaxed. Advisory only, never blocks the result.type: float

`--save-report`
Also persist the report to <dir>/neb_report.txt. Off by default. switch, off by default

`--save-gnuplot`
Also save the energy-profile curve as gnuplot .dat + .gplot scripts, under '<dir>/plot/'. Off by default. switch, off by default

`--view`
View the energy-profile plot interactively via matplotlib now, in addition to always saving it as a PNG. Off by default. Needs a display. switch, off by default

`--view-path`
Open every image's structure (in band order) in ASE's interactive multi-frame 3D viewer, to step through the reaction path – after everything else has finished. Needs a display. Independent of `--view` (which shows the energy-vs-reaction-coordinate plot, not structures). switch, off by default

`--save-path-xyz`
Write the currently-analyzed band (the same images used for [1] IMAGE ENERGIES above – the latest complete cycle for `--mode` 2/3, converged or not) as a single multi-frame XYZ trajectory, '<dir>/neb_path_current.xyz'. Off by default. Distinct from `stb-neb`'s own '`neb_path.xyz`', which is written once at prep time from the pre-refinement path and never updated afterwards. switch, off by default

Like every STB command it also accepts `-h`/`–help`, `-v`/`–version` and `–no-intro` (Section 3.3).

From the built-in help.

```
Usage examples:
  stb-nebAnalysis                              # analyzes ./neb_run by default
  stb-nebAnalysis --dir . --file calc.out
  stb-nebAnalysis --apply ts_guess.fdf
  stb-nebAnalysis --save-gnuplot --view
  stb-nebAnalysis --view-path
  stb-nebAnalysis --save-path-xyz
```

11.9.3.3 `stb-nebCycle`

One real-DFT NEB step: reads the latest cycle_NN/image_*/ (finished SIESTA single-points), takes exactly one climbing-image NEB step (ASE's own FIRE optimizer, restart-persisted between calls), and either writes cycle_{N+1}/ or declares convergence. Meant to be called in a loop from a cluster submission script – see `stb-neb --mode` 2/–mode 3's own printed example. Not available in the interactive `stb-suite` menu.

Synopsis.

```
usage: stb-nebCycle [-h] [--dir DIR] [--file FILE] [--k K] [--fmax FMAX]
                    [--climb-after N] [--save-report] [-v] [--no-intro]
```

Options

`--dir DIR`
Root directory containing every 'cycle_NN/' (default: current directory).

`--file FILE`
SIESTA output filename inside each image folder (default: `calc.out`).

`--k K`
NEB spring constant, eV/Ang^2 (default: 0.1, ASE's own default). type: float

`--fmax FMAX`
Force convergence threshold, eV/Ang (default: 0.05). type: float

`--climb-after N`
Only enable the climbing-image correction once the CURRENT cycle number is >= N (default: 0, climb from the start – appropriate once a MACE-shaped path already has a sensible shape, e.g. `stb-neb --mode 2`; a larger value is recommended for `--mode 3`, which starts from a raw interpolated path). type: int

`--save-report`
Also persist this cycle's report to <dir>/neb_cycle_report.txt. Off by default. switch, off by default

Like every STB command it also accepts `-h`/`--help`, `-v`/`--version` and `--no-intro` (Section 3.3).

From the built-in help.

```
Usage examples:
  stb-nebCycle --dir .
  stb-nebCycle --dir . --fmax 0.03 --climb-after 5
```

11.10 2D Stacking Fault (Gamma-Surface)

At a glance

Menu code: 5.10 (a multi-stage workflow)

Code	Stage	Command
5.10.1	Stage 1 - Prep	`stb-stackingfault`
5.10.2	Stage 2 - Analysis	`stb-stackingfaultAnalysis`

Guided example: `examples/5.10-stacking_fault/`

Summary: Slide one 2D layer across a grid of lateral offsets, then compute the equilibrium stacking, corrugation energy, and full gamma-surface map.

11.10.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/5.10-stacking_fault/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

Slide one layer of a van der Waals bilayer (or heterostructure) laterally over the other, recompute the total energy at a grid of offsets, and you get a 2D energy landscape — the **generalized stacking-fault energy surface**, or **gamma surface**. This item has **two** stages:

- **Stage 1 (`stb-stackingfault`, code `5.10.1`)** rigidly slides layer 2 across a grid (or a 1D line) of lateral offsets and writes one SIESTA folder per point, with a choice of 3 strategies for the interlayer gap.
- **Stage 2 (`stb-stackingfaultAnalysis`, code `5.10.2`)** reads every finished energy back and reports the equilibrium stacking, the highest-energy registry, the corrugation, and the full gamma-surface map.

`example_5.10.sh` is a guided, runnable walkthrough of all of this, including a real 30-point energy landscape (Case 5 / Section 4 below) and a live proof that the CLI and the interactive `stb-suite` menu produce byte-identical output (Case 7).

A 3rd stage used to exist here (`stb-stackingfaultBsse`, a BSSE/ counterpoise correction). It was removed — Section 1.2 and Section 5 explain why in detail, since it's the single most important thing to understand about this workflow if you're coming from `stb-adsorbBsse` or a cohesive-energy calculation and expect a similar correction step here.

11.10.1.1 Theory

The gamma-surface and its two headline numbers. For every sampled lateral offset (`shift_x`, `shift_y`) (fractional coordinates of layer 2's in-plane lattice vectors), Stage 1 writes a single structure with layer 1 fixed and layer 2 rigidly translated, and Stage 2 reads back its total energy `E(shift_x, shift_y)`. Two numbers summarize the resulting landscape:

- **Equilibrium stacking**: the offset with the lowest energy — e.g. graphene's `AB` (Bernal) registry, one of the 2 local minima a honeycomb bilayer has (the other, symmetric one is `BA`).
- **Corrugation**: `max(E) - min(E)` over the sampled grid — the energy barrier resisting interlayer sliding/shear. Directly relevant to interlayer friction, and (for a 3D crystal's actual stacking fault, as opposed to a 2D vdW bilayer) to dislocation dissociation energetics.

Reported in **meV/Å^2** (millielectronvolt per square Ångström of interface area) — the standard, cell-size-independent unit for this kind of areal energy density, computed as `(E - E_min) * 1000 / area`, where `area` is the in-plane cell area (`|a × b|` of the structure's own lattice vectors — see Section 3.2).

1.2 Why there is no BSSE-correction stage here (read this if you know `stb-adsorbBsse`). Basis Set Superposition Error (BSSE) correction is necessary when you compare energies of systems that have **genuinely different atom counts and basis sizes** — e.g. adsorption energy (slab+adsorbate vs. slab alone vs. adsorbate alone) or cohesive energy (bulk crystal vs. isolated atom). In those cases, the "combined" system's SCF benefits from more basis functions than either isolated reference does on its own, artificially lowering it — counterpoise correction (real+ghost single-point references at the same geometry) fixes that asymmetry.

A stacking-fault scan has no such asymmetry. Every single point in the whole grid has the *exact same atoms, the exact same basis set* — only positions change. Write `E(shift) = E_layer1_isolated + E_layer2_isolated + E_interaction(shift)`. Since neither layer's own internal geometry changes with `shift` (only layer 2's *position* changes, not its internal structure), `E_layer1_isolated` and `E_layer2_isolated` are **constants**, identical at every grid point. They cancel exactly in `max(E) - min(E)` — the raw total energy's corrugation **is already, exactly**, the true interaction-energy corrugation. No correction needed, or even meaningful, to apply here.

Applying the `stb-adsorbBsse`-style formula anyway (`E_corrected = E - e_layer1(ghost) - e_layer2(ghost)`) doesn't just do nothing useful — it makes things *worse*: `e_layer1(ghost)`/`e_layer2(ghost)` (real+ghost single-points) are **lower** than the true isolated references by exactly the BSSE amount, so subtracting them *adds the BSSE signal back into the result* (`E_corrected(shift) = E_interaction(shift) + BSSE_total(shift)`, provably, see Section 5). This was verified live, on a real 30-point scan, before the correction stage was removed from this workflow: the "corrected" corrugation curve showed spurious kinks the raw energy curve didn't have, traced to exactly this effect.

11.10.1.2 2. Stage 1: generating the grid (`stb-stackingfault`, code `5.10.1`)

2.1 Sweep shape: `--scan surface` (default) vs. `x/y/xy`. `--scan surface` (default) samples the full 2D grid, `-nx`/`-ny` points along each in-plane lattice vector independently (an anisotropic lattice often warrants a different density per axis). `--scan x/y/xy` trade that for a cheaper 1D line — along `shift_x` only, `shift_y` only, or the `shift_x = shift_y` diagonal — `-n`/`--scan-points` points. Same `shift_II_JJ` folder-naming/manifest scheme either way (one index just stays `00`) — Stage 2 auto-detects which shape it's looking at from `sf_manifest.json`, no flag needed.

`example_5.10.sh`'s Case 1 runs a real `5x5 --scan surface` grid; Case 4 runs all 3 of `--scan x/y/xy`.

2.2 The 3 interlayer-gap strategies (`--mode`).

Mode	Strategy	Cost	Accuracy
1	SIESTA relaxes z for real: a restricted CG relaxation per point (x,y frozen for every atom via `%block Geometry.Constraints`, only z free, `--relax-z-steps` steps)	highest (a real relaxation per point)	highest (same level of theory as the reported energy)
2	MACE-MP-0 relaxes z first (same x/y-frozen definition, cheap ML), then a plain SIESTA single-point at the result	middle	middle (mixes a cheap ML pre-relaxation into an otherwise-DFT number)
3	fixed `--gap` (default 3.2 Å), plain SIESTA single-point, no relaxation at all	lowest	lowest (every point uses the same interlayer distance, right for none of them exactly)

`example_5.10.sh`'s Case 2 generates the same 2-point sweep 3 times, once per mode, and shows each mode's own `config_extra.fdf` side by side — mode 1's `Geometry.Constraints` block, mode 2/3's plain single-point (`MD.Steps 0`). Mode 2 additionally needs the optional `ml` extra (`pip install stb_suite[ml]`); if `--d3` is also on (the default), MACE-MP-0's own D3(BJ) dispersion needs the separate optional `torch-dftd` package too — the script degrades gracefully (falls back to `--no-d3` for that one demonstration, or skips mode 2 entirely) if either is missing, same as it will for you.

Known caveat, worth reading before trusting a `--mode 1` production run: see Section 5.1.

ZSL matching — same-file sliding vs. real heterostructures. Pass the **same file** for `--layer1`/`--layer2` for the canonical case: a material sliding against itself (e.g. graphite `ABA` vs. `ABC` stacking). This is a trivial `1×1`, 0-strain match by construction — no supercell search needed.

Pass **different files** to study an interlayer/heterostructure sliding landscape instead — `stb-stackingfault` then runs the Zur & McGill Zero-Strain-Layer (ZSL) algorithm to find the lowest-strain **commensurate supercell** that fits both lattices, exactly the same matching logic `stb-2Dstacking` uses. `--max_area`/`--max_strain`/`--match_id` control the search; `-sm`/`--strain_mode` (`top`/`bottom`/`sym`) controls how the mismatch strain is distributed between the two layers; `-t`/`--twist` applies a *fixed* twist angle (the same physical system throughout the whole grid — not swept).

`example_5.10.sh`'s Case 3 runs `graphene.fdf` (a=2.46 Å) against `hbn.fdf` (a=2.504 Å, ~1.8% mismatched)

and finds a real, non-trivial match: **26.20 Å^2, 1.76% strain, 20 atoms per grid point** (10 per layer) — vs. Case 1's trivial 5.24 Å^2, 0% strain, 4-atom homobilayer case.

2.4 D3 dispersion, ML pre-relax/preview, and `sf_manifest.json`. `--d3` (default **on**) forces the Grimme DFT-D3 dispersion correction in every generated `config_extra.fdf` — interlayer binding here is van der Waals-dominated, which plain GGA misses. `--ml-prerelax-layers` optionally relaxes each monolayer's own positions with MACE-MP-0 before stacking (a safety net if you're not fully sure your inputs are already at equilibrium); `--ml-preview` evaluates the whole grid's single-point energy on MACE-MP-0 first (no SIESTA) and writes a quick PNG heatmap/line sanity check of the gamma-surface's shape before committing to the real DFT grid. Both need the optional `ml` extra.

Every run always writes `sf_manifest.json` (grid shape + per-point shift/gap, machine-readable) under `sf_run/` — the primary source Stage 2 reads, independent of whether `--save-report`'s narrative `stackingfault_setup.txt` was also requested.

11.10.1.3 3. Stage 2: analyzing the results (`stb-stackingfaultAnalysis`, code `5.10.2`)

Report structure.

```
[0] RUN METADATA        -- directory, grid shape, mode
[1] GRID ENERGIES        -- one row per point: E(eV), dE(meV/A2), SCF, MaxF, GapFinal (mode 1 only)
[2] STACKING FAULT ANALYSIS  -- equilibrium/highest-energy registry, corrugation
[3] SUMMARY               -- gnuplot data (if --save-gnuplot), the always-on animation
[4] APPLY                 -- only if --apply was given
```

`--dir` (default `sf_run`, auto-detected as `.` if you're already inside it) points at Stage 1's own run folder; `--file` (default `calc.out`) is the SIESTA output filename expected inside every `positions/shift_II_JJ/`.

The meV/Å^2 scale. `[2]`'s leading `[INFO] Interface area` line reports the in-plane cell area used for the conversion — always computed fresh from whichever grid point's `structure.fdf` was read first, `|a × b|` of its own lattice vectors (Å^2). Every relative energy in the report (`[1]`'s `dE(meV/A2)` column, `[2]`'s equilibrium/highest/corrugation lines) is `(E - E_min) * 1000 / area` — the standard "millielectronvolt per square Ångström" convention used throughout the 2D-materials literature for this kind of areal energy density. The absolute per-point `E(eV)` column stays in raw eV (diagnostic only — an absolute SCF total energy in meV would just be an unwieldy 6-7 digit number).

3.3 `--save-gnuplot` and `plot/`, `--view`, and the always-on animation. Gnuplot `.dat+.gplot` output is **opt-in** (`--save-gnuplot`, off by default), written under `<dir>/plot/` — a `pm3d` heatmap for `--scan surface`, a line plot for `--scan x/y/xy`. A gnuplot `pm3d` map needs **every** point of the grid present to render at all (a single missing point renders the entire map blank, verified live) — an incomplete grid prints a `[WARNING]` and skips the plot files instead of writing a broken one; the numeric analysis in `[1]`/`[2]` still works fine on a partial grid regardless.

`--view` shows the *same* landscape as an interactive matplotlib preview (a heatmap or line plot, `plt.show()`) — independent of, and more forgiving than, `--save-gnuplot`'s file output: it renders even on an incomplete grid (a `NaN` gap in the array, not a blank plot). Following this suite's own convention (`WORKFLOW_TOOLS` writes gnuplot pairs; matplotlib here is preview-only), it is **never** written to disk as a PNG.

An **extended-XYZ animation** (`stackingfault_animation.xyz`, one frame per analyzed point, `Lattice=`/per-frame `.info` with `label`, `shift_x`/`shift_y`, `energy_eV`, `dE_meV_per_A2`) is **always** written, unconditionally — viewable directly in VESTA/OVITO/ASE-GUI, or interactively via `--view-animation` (ASE's own 3D viewer, needs a display).

3.4 `--apply`. Copies the equilibrium (lowest-energy) point's `structure.fdf` to a path of your choosing — e.g. as the starting geometry for a follow-up calculation at that specific registry.

11.10.1.4 Worked example: a real 30-point energy landscape

`example_5.10.sh`'s Case 5 fabricates `calc.out` for a real `--scan x -n 30` sweep using **real `siesta: FreeEng` values** from an actual bilayer-graphene DFT calculation (`PAO.BasisSize DZP`, `XC.Functional GGA/PBE`, real SIESTA 5.4.2 single-points) — not invented numbers, just written onto this folder's own small, unrelaxed `graphene.fdf` pair (area 5.2408 Å^2, vs. the original run's own relaxed lattice) so the exact meV/Å^2 figure below is this fixture's own.

```
$ stb-stackingfault -l1 graphene.fdf -l2 graphene.fdf -c calc.fdf --scan x -n 30 --mode 3 --no-intro
$ stb-stackingfaultAnalysis --dir sf_run --save-gnuplot --view --view-animation --no-intro
```

```
[1] GRID ENERGIES
------------------------------------------------------------
Point           Shift(x,y)          E(eV)           dE(meV/A2)    SCF   MaxF(eV/A)
-------------------------------------------------------------------------------------
shift_00_00     (0.000,0.000)       -654.900928     3.188         OK    0.0100
shift_01_00     (0.033,0.000)       -654.901136     3.148         OK    0.0100
shift_02_00     (0.067,0.000)       -654.901574     3.065         OK    0.0100
shift_03_00     (0.100,0.000)       -654.902258     2.934         OK    0.0100
shift_04_00     (0.133,0.000)       -654.903136     2.767         OK    0.0100
shift_05_00     (0.167,0.000)       -654.904736     2.461         OK    0.0100
shift_06_00     (0.200,0.000)       -654.906246     2.173         OK    0.0100
shift_07_00     (0.233,0.000)       -654.907901     1.857         OK    0.0100
shift_08_00     (0.267,0.000)       -654.909639     1.526         OK    0.0100
shift_09_00     (0.300,0.000)       -654.911400     1.190         OK    0.0100
shift_10_00     (0.333,0.000)       -654.913058     0.873         OK    0.0100
shift_11_00     (0.367,0.000)       -654.914659     0.568         OK    0.0100
shift_12_00     (0.400,0.000)       -654.915805     0.349         OK    0.0100
shift_13_00     (0.433,0.000)       -654.916873     0.145         OK    0.0100
shift_14_00     (0.467,0.000)       -654.917482     0.029         OK    0.0100
shift_15_00     (0.500,0.000)       -654.917635     0.000         OK    0.0100
shift_16_00     (0.533,0.000)       -654.917482     0.029         OK    0.0100
shift_17_00     (0.567,0.000)       -654.916873     0.145         OK    0.0100
shift_18_00     (0.600,0.000)       -654.915805     0.349         OK    0.0100
shift_19_00     (0.633,0.000)       -654.914659     0.568         OK    0.0100
shift_20_00     (0.667,0.000)       -654.913058     0.873         OK    0.0100
shift_21_00     (0.700,0.000)       -654.911400     1.190         OK    0.0100
shift_22_00     (0.733,0.000)       -654.909639     1.526         OK    0.0100
shift_23_00     (0.767,0.000)       -654.907901     1.857         OK    0.0100
shift_24_00     (0.800,0.000)       -654.906246     2.173         OK    0.0100
shift_25_00     (0.833,0.000)       -654.904736     2.461         OK    0.0100
shift_26_00     (0.867,0.000)       -654.903136     2.767         OK    0.0100
shift_27_00     (0.900,0.000)       -654.902258     2.934         OK    0.0100
shift_28_00     (0.933,0.000)       -654.901574     3.065         OK    0.0100
shift_29_00     (0.967,0.000)       -654.901136     3.148         OK    0.0100
-------------------------------------------------------------------------------------

[2] STACKING FAULT ANALYSIS
[INFO] Interface area (in-plane): 5.2408 Ang^2
Equilibrium stacking (min)    : shift_15_00  (shift 0.5000, 0.0000)  (E = -654.917635 eV, dE = 0.000
meV/Ang^2)
Highest-energy registry (max) : shift_00_00  (shift 0.0000, 0.0000)  (E = -654.900928 eV, dE = 3.188
meV/Ang^2)
```

> Corrugation (stacking-fault) energy : 3.188 meV/Ang^2 (max - min over the sampled grid; 0.016707 eV over a 5.2408 Ang^2 cell)

Reading this physically: the energy decreases smoothly and **perfectly symmetrically** from `shift_x=0` (eclipsed/highest energy) down to `shift_x=0.5` (equilibrium, lowest energy) and back up — exactly the shape expected for a graphene bilayer sliding along one lattice direction, with the true minimum-energy `AB`-like registry sitting at the midpoint of this particular path. No BSSE column anywhere in this table (Section 1.2) — the raw energy already is the physically correct comparison.

11.10.1.5 Known, deliberate limitations

5.1 `--mode 1`'s CG relaxation can converge inconsistently across the grid. Verified live, on a real 30-point `--mode 1` scan: with a loose `SCF.DM.Tolerance`/`MD.MaxForceTol` (e.g. `0.01 eV/Å`), SIESTA's CG relaxation can converge in as few as **2-4 steps** — satisfying the force tolerance, but with very different margins from point to point (some points land at a residual force of `~0.002 eV/Å`, others at `~0.009 eV/Å`, both technically "converged"). The resulting small interlayer-gap inconsistency (a few hundredths of an Å) is nearly invisible in the raw total energy (which varies slowly/quadratically near an already-close geometry) but can produce a visible kink in `GapFinal(A)` between neighboring grid points. **Fix**: tighten `MD.MaxForceTol` in your `calc.fdf` (e.g. `0.001-0.005 eV/Å`) if you see a non-smooth `GapFinal(A)` progression in `[1]`'s table — don't just increase `--relax-z-steps`, since the step *count* usually isn't the limiting factor (2-4 steps is typically enough to reach even a tight tolerance on a system this small).

5.2 `--mode 3`'s fixed gap is an approximation. Every point uses the *same* nominal `--gap`, regardless of whether that's close to the true equilibrium distance at that specific registry (an eclipsed/high-energy point's true relaxed gap is typically *larger* than an offset/low-energy point's). `[2]`'s report always prints a `[NOTE]` reminding you the true, fully-relaxed corrugation may be somewhat lower — `--mode 1/2` exist specifically to remove this approximation, at increasing cost.

No BSSE correction — by design, not by omission. Covered in full in Section 1.2 — repeated here because it's the most likely point of confusion for anyone who has used `stb-adsorbBsse` or a cohesive-energy workflow and expects an analogous stage here. There isn't one, and there shouldn't be: the raw energy's corrugation is already the exact, correct physical answer for this specific kind of same-atom-count comparison.

11.10.1.6 Step-by-step: running this workflow on your own structure(s)

1. Prepare 1 or 2 monolayer `.fdf` files (same file for a material sliding against itself; 2 different files for a heterostructure) and a `calc.fdf` template (basis, k-grid, XC — Stage 1 forces the single-point/z-relaxation behavior itself via `config_extra.fdf`, independent of whatever `MD.TypeOfRun`/`MD.Steps` your template has).
2. `stb-stackingfault -l1 <layer1.fdf> -l2 <layer2.fdf> -c calc.fdf --mode 1` (most accurate — start here unless you have a specific reason to trade accuracy for speed) — pick `-nx`/`-ny` (surface) or `--scan x/y/xy -n` (1D) depending on whether you need the full 2D map or a cheaper profile first.
3. Run SIESTA in every `sf_run/positions/shift_II_JJ/` folder.
4. `stb-stackingfaultAnalysis --dir sf_run --save-report --save-gnuplot --view` — inspect `[1]`'s `GapFinal(A)` column for the smoothness issue in Section 5.1 before trusting the corrugation number if you used `--mode 1`.

11.10.1.7 Files in this folder

```
graphene.fdf        Primitive 2-atom graphene monolayer (a=2.46 Ang)
hbn.fdf              Primitive 2-atom h-BN monolayer (a=2.504 Ang, ~1.8% mismatched)
calc.fdf             Shared calc.fdf template (DZP, GGA-PBE) -- Stage 1 overrides its
                      MD block per-point regardless (Section 2.2)
example_5.10.sh      This walkthrough
README.md            This file
.gitignore           Ignores output/ (regenerated by the script every run)
```

11.10.1.8 Running the script

```
cd examples/5.10-stacking_fault
./example_5.10.sh
```

Needs no real SIESTA binary for Stage 1 (writes `.fdf` files only, plus a real MACE-MP-0 z-relaxation for `--mode 2` if the optional `ml` extra is installed). Stage 2's worked example (Case 5) uses fabricated `calc.out` data (Section 4) since this walkthrough doesn't invoke real SIESTA either. Every case writes into its own `output/case*/` subfolder; `output/` itself is gitignored and safe to delete/regenerate any time.

11.10.1.9 What's next

Once you have a real gamma-surface for your own material:

- The equilibrium `structure.fdf` (via `--apply`) is a natural starting point for a follow-up calculation at that specific registry — an NEB path between two symmetry-equivalent minima (workflow `5.9`), or a phonon calculation (workflow `5.4`) to check its dynamical stability.
- The corrugation energy is directly comparable to literature values for interlayer sliding/friction barriers in the same material family.

11.10.2 Implementation details

Generalized stacking-fault energy

Sliding one layer rigidly over the other by an in-plane vector $\mathbf{u}$ changes the energy; per unit area,

$$\gamma(\mathbf{u}) = \frac{E(\mathbf{u}) - E_{\min}}{A},$$

the γ-surface [77]. Its minimum is the equilibrium stacking, its maximum minus minimum the *corrugation energy*, and the lowest barrier between equivalent minima governs interlayer sliding and friction. Reported in meV/Å^2 (1 meV/Å^2 = 16.02 mJ/m^2).

Stage 1 (`stb-stackingfault`).. The bilayer is built exactly as in **`stb-2Dstacking`** (4.1, same ZSL options; the same file twice gives a homobilayer), but the ZSL search runs once and the shift is swept: `-scan surface` (default) a grid of `-nx`×`-ny` shifts (default 7 × 7) of layer 2 in fractions of its own cell, or a line along `x`, `y` or `xy` with `-n` points (default 15). Each point is a folder `sf_run/positions/shift_II_JJ/` under `-O` with DFT-D3 forced (`-no-d3` to disable) and one of three gap strategies (`-mode`): 1 – SIESTA relaxes only the z coordinates (`Geometry.Constraints` freezes x, y of every atom; `-relax-z-steps`); 2 – MACE relaxes z first, SIESTA does a single point; 3 – fixed `-g` gap, single point. `-ml-preview` gives a MACE

γ-surface before any SIESTA run, `-ml-prerelax-layers` relaxes the monolayers first. A `sf_manifest.json` records the grid.

Stage 2 (`stb-stackingfaultAnalysis`).. It reads every point's energy, checks convergence and forces, converts to meV/Å^2 with the in-plane cell area, and reports the minimum (equilibrium stacking), maximum, corrugation energy and, for line scans, the barrier profile. Outputs: `stackingfault_surface.dat` (`-o`), a gnuplot map, a sliding animation, and `-apply` to copy the most stable stacking.

No counterpoise stage.. All points have the same atoms and the same basis: only the lateral registry changes. The basis superposition error is nearly constant across the surface and largely cancels in γ, so no BSSE folders are generated (see the guide).

11.10.3 Command-line reference

11.10.3.1 `stb-stackingfault`

Prepares SIESTA folders for a 2D stacking-fault (generalized stacking fault energy / gamma-surface) study: rigidly slides one layer of a bilayer across a 2D grid of lateral offsets and writes one SIESTA folder per grid point – a plain single-point by default, or a real/ML-assisted z-only relaxation, see `--mode`.

Synopsis.

```
usage: stb-stackingfault [-h] -l1 LAYER1 -l2 LAYER2 -c CALC [-p PSEUDO_DIR]
                         [-a MAX_AREA] [-s MAX_STRAIN] [-id MATCH_ID] [-g GAP]
                         [--vacuum VACUUM] [-sm {top,bottom,sym}] [-t TWIST] [--d3]
                         [--no-d3] [--mode {1,2,3}] [--relax-z-steps RELAX_Z_STEPS]
                         [--scan {surface,x,y,xy}] [-n SCAN_POINTS] [-nx GRID_NX]
                         [-ny GRID_NY] [--ml-prerelax-layers] [--ml-preview]
                         [--ml-model {small,medium,large}] [--ml-custom-model PATH]
                         [--ml-custom-model-card JSON] [--ml-device {cpu,cuda}]
                         [--ml-fmax ML_FMAX] [-O OUTPUT_DIR] [--save-report] [-v]
                         [--no-intro]
```

Options

`-l1, --layer1 LAYER1`
Bottom monolayer .fdf (stays fixed). required

`-l2, --layer2 LAYER2`
Top monolayer .fdf (rigidly slid across the grid). Pass the same file as `--layer1` for a material sliding against itself. required

`-c, --calc CALC`
`calc.fdf` template (kgrid, basis, XC, %include `structure.fdf`, etc.) – copied into every shift_II_JJ/ folder; single-point SCF enforcement (and the optional `--d3` dispersion correction) is added via a `config_extra.fdf` %include, not embedded inline. required

`-p, --pseudo-dir PSEUDO_DIR`
Pseudopotentials source (optional): a bundled bank or a folder path.

`-a, --max_area MAX_AREA`
Maximum ZSL commensurate-supercell area, Ang^2 (default: 150.0, same as stb-2Dstacking). type: float

`-s, --max_strain MAX_STRAIN`
Maximum allowed ZSL match strain fraction (default: 0.05). type: float

`-id, --match_id MATCH_ID`
Which ZSL match to use, 0-based, best (lowest strain) first (default: 0). type: int

`-g, --gap GAP`
Interlayer gap in Ang (default: 3.2, same as stb-2Dstacking). Under `--mode` 3 (default) this is the FIXED value used across the whole grid – a rigid-shift protocol, standard in the literature (see the [INFO] closest-contact line in the report for how close high-energy registries got at this fixed distance). Under `--mode` 1/2 this is only the STARTING guess – SIESTA (mode 1) or MACE-MP-0 (mode 2) relaxes it per grid point from here. type: float

`--vacuum VACUUM`
Target vacuum space in Ang. Inherits `--layer1's` by default. type: float

`-sm, --strain_mode {top,bottom,sym}`
Strain distribution mode for the ZSL match (default: top). choices: top, bottom, sym

`-t, --twist TWIST`
Twist angle of layer 2 in degrees (default: 0.0), FIXED for the whole grid – a different twist is a different physical system, not swept here. type: float

`--d3`
Force the Grimme DFT-D3 dispersion correction (via `config_extra.fdf`) in every generated `calc.fdf` – physically relevant here since the interlayer binding swept across the grid is a van-der-Waals-dominated interaction that plain GGA misses (same correction `stb-adsorb/stb-neb` force by default elsewhere in the suite). ON by default. Applies in every `--mode`. switch, off by default

`--no-d3`
Disable the DFT-D3 dispersion correction (not recommended for this workflow – see `--d3`). switch

`--mode {1,2,3}`
Interlayer-gap strategy, per grid point (default: 1):
1 = SIESTA relaxes z for real: writes one restricted CG relaxation per grid point (x,y frozen for every atom, only z free – see build_z_relax_block/–relax-z-steps). Most accurate (same level of theory as the reported energy), most expensive (a real relaxation, not one SCF, per point).
2 = MACE-MP-0 relaxes z first (same x,y-frozen/z-free definition as mode 1, cheap ML engine), THEN writes a plain single-point SIESTA folder at the resulting geometry – middle ground between cost and accuracy. Needs the optional 'ml' extra.
3 = fixed `--gap`, plain single-point SIESTA, no relaxation of any kind (today's behavior, the cheapest of the three). choices: 1, 2, 3; type: int

`--relax-z-steps RELAX_Z_STEPS`
With `--mode` 1: MD.Steps for the restricted SIESTA z-relaxation (default: 200). Ignored in modes 2/3. type: int

`--scan {surface,x,y,xy}`
Sweep shape (default: surface):
surface = full 2D grid (`-nx` x `-ny` points), the gamma-surface – output is the pm3d map/heatmap already documented above.
x = 1D sweep along shift_x only (shift_y fixed at 0) – `-n` points, output is a line plot (E vs. distance along x).
y = 1D sweep along shift_y only (shift_x fixed at 0) – `-n` points, output is a line plot (E vs. distance along y).
xy = 1D sweep along the diagonal shift_x == shift_y – `-n` points, output is a line plot (E vs. distance along the diagonal).
`-nx/-ny` only apply to 'surface'; `-n/--scan-points` only applies to 'x'/'y'/'xy'. choices: surface, x, y, xy

`-n, --scan-points SCAN_POINTS`
With `--scan` x/y/xy: number of points along the 1D sweep (default: 15). Ignored under `--scan` surface (see `-nx/-ny` instead). type: int

`-nx, --grid-nx GRID_NX`
With `--scan` surface: grid resolution along shift_x, this many lateral shifts covering [0, 1) (default: 7). Indepen-

dent of --grid-ny – an anisotropic lattice (a != b, e.g. a rectangular or low-symmetry 2D cell) often warrants a different sampling density per axis rather than forcing a square grid. Ignored under --scan x/y/xy (see -n/--scan-points instead). type: int

-ny, --grid-ny GRID_NY

With --scan surface: grid resolution along shift_y (default: 7). See --grid-nx. Ignored under --scan x/y/xy. type: int

--ml-prerelax-layers

Relax each monolayer's positions (cell fixed) with MACE-MP-0 before stacking – a cheap safety net when you're not fully certain --layer1/--layer2 are already at equilibrium. Needs the optional 'ml' extra.switch, off by default

--ml-preview

Evaluate the whole grid's single-point energy on MACE-MP-0 (fast, no SIESTA) and write stackingfault_ml_preview.png – a quick sanity check of the gamma-surface's shape before committing to the full Nx x Ny DFT single-points. Written alongside the normal SIESTA folders, not instead of them. Needs the optional 'ml' extra. switch, off by default

--ml-model {small,medium,large}

MACE-MP-0 model size (default: medium). Ignored if --ml-custom-model is given.choices: small, medium, large

--ml-custom-model PATH

Use a custom MACE model file instead of the MACE-MP-0 foundation potential – e.g. one fine-tuned on your own SIESTA data via stb-mlffAnalysis. Overrides --ml-model.

--ml-custom-model-card JSON

Model card of the --ml-custom-model file: the mlff_model_card.json that stb-mlffAnalysis wrote next to it (training data, level of theory, accuracy, energy scale). Required with --ml-custom-model: the card is checked against the model file and used to warn when a structure is outside what the model was trained on.

--ml-device {cpu,cuda}

default: cpu; choices: cpu, cuda

--ml-fmax ML_FMAX

Force convergence for --ml-prerelax-layers and --mode 2's z-only MACE relaxation (default: 0.05).type: float

-O, --output-dir OUTPUT_DIR

Root directory (default: current directory) – everything lives under its 'sf_run/' subfolder (see --save-report): every 'shift_II_JJ/' grid folder under 'sf_run/positions/', the manifest, and the optional report directly under 'sf_run/' – same self-contained-run-folder convention as stb-adsorb's 'sites/' and stb-neb's 'neb_run/'.

--save-report

Also persist the full run narrative to <output-dir>/sf_run/stackingfault_setup.txt. Off by default – the machine-readable sf_manifest.json that stb-stackingfaultAnalysis actually reads is always written regardless of this flag. switch, off by default

Like every STB command it also accepts -h/-help, -v/-version and -no-intro (Section 3.3).

From the built-in help.

```
Usage examples:
  stb-stackingfault -l1 graphene.fdf -l2 graphene.fdf -c calc.fdf -nx 7 -ny 7
  stb-stackingfault -l1 graphene.fdf -l2 hbn.fdf -c calc.fdf -nx 9 -ny 5 -g 3.4
  stb-stackingfault -l1 graphene.fdf -l2 graphene.fdf -c calc.fdf --scan x -n 15

Pass the SAME file for -l1/-l2 for the canonical use case (a material sliding against itself, e.g. graphite ABA
vs. ABC stacking) -- already the precedented 'identical layers' case for stb-2Dstacking's own ZSL match.
Different files study an interlayer/heterostructure sliding energy landscape instead.
```

```
-nx/-ny may differ (an asymmetric grid) -- useful for an anisotropic 2D lattice where one in-plane direction
needs finer sampling than the other (--scan surface only).

--scan x/y/xy trade the full 2D gamma-surface for a cheaper 1D line through it (along shift_x, shift_y, or the
shift_x=shift_y diagonal) -- useful for a quick high-symmetry-line profile before committing to the full surface.
```

11.10.3.2 `stb-stackingfaultAnalysis`

Computes a stacking-fault (gamma-surface) energy landscape from an `stb-stackingfault` grid: reads each positions/shift_II_JJ/'s SIESTA energy and reports the equilibrium stacking, the highest-energy registry, and the corrugation energy.

Synopsis.

```
usage: stb-stackingfaultAnalysis [-h] [--dir DIR] [--file FILE] [-o OUTPUT]
                                 [--apply STRUCTURE_FDF]
                                 [--force-tolerance FORCE_TOLERANCE] [--save-report]
                                 [--save-gnuplot] [--view] [--view-animation] [-v]
                                 [--no-intro]
```

Options

`--dir DIR`
: Root directory containing 'positions/shift_II_JJ/' (default: 'sf_run', the self-contained run folder `stb-stackingfault` Stage 1 always writes its output into – auto-detected as '.' instead when the current directory already has its own `sf_manifest.json`).

`--file FILE`
: SIESTA output filename inside each folder (default: `calc.out`).

`-o, --output OUTPUT`
: Base filename for the gnuplot data file (default: `stackingfault_surface.dat`), written under '<dir>/plot/' – only relevant with `--save-gnuplot`.

`--apply STRUCTURE_FDF`
: Copy the equilibrium (lowest-energy) grid point's `structure.fdf` to this path.

`--force-tolerance FORCE_TOLERANCE`
: Residual atomic force in eV/Ang (default: 0.05, same as `stb-adsorbAnalysis/stb-nebAnalysis`) above which a grid point's `calc.out` is flagged as possibly not single-point-converged. Advisory only, never blocks the result. type: float

`--save-report`
: Also persist the report to <dir>/stackingfault_report.txt. Off by default. switch, off by default

`--save-gnuplot`
: Also save the gamma-surface (or line-scan) data as gnuplot .dat + .gplot scripts, under '<dir>/plot/'. Off by default. switch, off by default

`--view`
: View the gamma-surface heatmap (or 1D line, for `--scan` x/y/xy) interactively via matplotlib now. Off by default. Needs a display. Independent of `--save-gnuplot` – matplotlib here is only an on-screen preview, never written to disk. switch, off by default

`--view-animation`
: Open every analyzed grid point's structure (same order as the always-saved animation XYZ) in ASE's

interactive multi-frame 3D viewer. Off by default. Needs a display. Independent of `--view` (which shows the energy plot, not structures). switch, off by default

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Usage examples:
  stb-stackingfaultAnalysis --dir . --file calc.out
  stb-stackingfaultAnalysis --dir . --apply equilibrium.fdf
  stb-stackingfaultAnalysis --save-gnuplot --view
  stb-stackingfaultAnalysis --view-animation
```

11.11 Raman Spectrum

At a glance

Menu code: 5.11 (a multi-stage workflow)

Code	Stage	Command
5.11.1	Stage 1 - Phonon Displacements	`stb-raman`
5.11.2	Stage 2 - Modes & Optical Displacements	`stb-ramanModes`
5.11.3	Stage 3 - Analysis	`stb-ramanAnalysis`

Guided example: `examples/5.11-raman/`

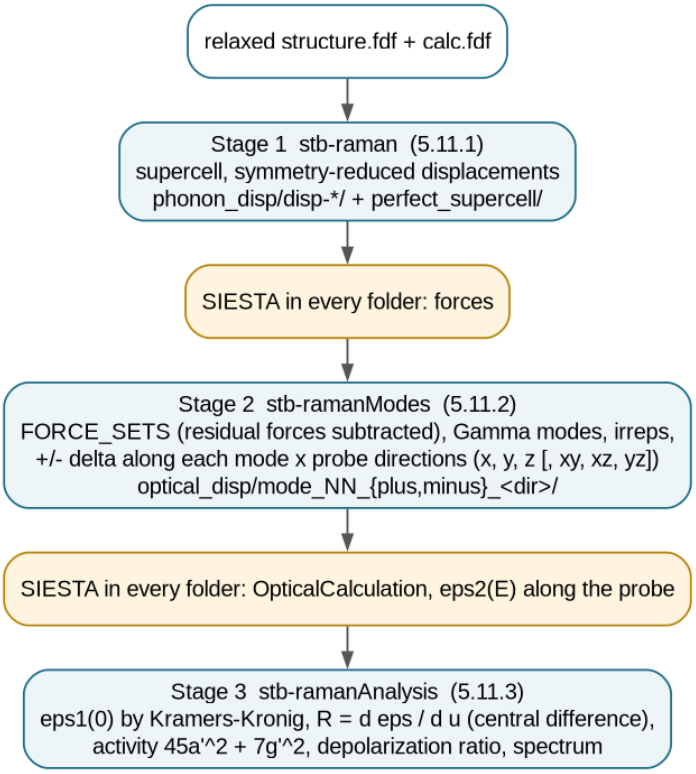
Summary: Self-contained: computes its own phonons, then the +/-delta optical (dielectric-response) displacements along each mode, then the Raman tensor (diagonal, or full with `-full-tensor`), the activities, depolarization ratios and the spectrum.

11.11.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/5.11-raman/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

The non-resonant, first-order Raman spectrum of a crystal or molecule, entirely from SIESTA, in three stages: **Stage 1** (`stb-raman`, code `5.11.1`) displaces atoms to get the phonons; **Stage 2** (`stb-ramanModes`, `5.11.2`) finds the Γ-point modes, classifies them by symmetry, and displaces the cell by $\pm\delta$ along each mode, with an optical calculation in every folder; **Stage 3** (`stb-ramanAnalysis`, `5.11.3`) turns the change of the dielectric constant into a Raman tensor per mode, an activity, a depolarization ratio and a spectrum. SIESTA runs in between.

This folder ships the finished SIESTA runs (`runs/`, made by `run_example.sh`) of **bulk silicon** — one Raman line, T_2g at about 520 cm^{-1} in experiment [4], with a tensor that has only off-diagonal entries — and of the **graphene monolayer**, where the selection rules can be checked mode by mode. No SIESTA is needed to follow the tutorial.



The three stages of the Raman workflow

11.11.1.1 Contents

1. Theory
2. The workflow at a glance
3. Libraries and external dependencies used
4. Stage 1: phonon displacements (`stb-raman`, `5.11.1`)
5. Stage 2: modes and optical folders (`stb-ramanModes`, `5.11.2`)
6. Stage 3: tensor, activity, spectrum (`stb-ramanAnalysis`, `5.11.3`)
7. Worked example
8. What to take from the example
9. Reproducing the reference data
10. Known, deliberate limitations
11. Step by step: your own structure
12. Files in this folder
13. Running the script
14. What's next
15. References

11.11.1.2 1. Theory

1.1 Raman scattering. Light of frequency ω_L scattered by a crystal comes out mostly at ω_L (Rayleigh), and a small part at ω_L − ω_ν (Stokes) or ω_L + ω_ν (anti-Stokes), having created or absorbed a vibration of frequency ω_ν. The photon carries almost no momentum on the scale of the Brillouin zone, so in **first-order** Raman only $q \approx 0$ phonons — the **Γ-point modes** — take part, one at a time. Overtones and the D/2D bands of graphene are second-order processes and are not computed here.

1.2 The Raman tensor (Placzek approximation). When $\hbar\omega$_L is well below the electronic gap, the electrons follow the nuclei adiabatically and the scattered field comes from the change of the electronic polarizability along the mode [1]:

$$\alpha_ij(Q_\nu) = \alpha_ij(0) + (\partial\alpha_ij/\partial Q_\nu)\ Q_\nu + \ldots \qquad R_ij(\nu) \propto \partial\varepsilon\infty_ij\ /\ \partial Q_\nu \tag{1}$$

In a crystal α is the electronic susceptibility, $\chi = (\varepsilon\infty - 1)/4\pi$, so the **Raman tensor** of mode ν is the derivative of the high-frequency (clamped-ion) dielectric tensor along the mode. It is a symmetric 3×3 tensor, and the light polarizations select its components: a line is seen with incident polarization **e**_L and analysed polarization **e**_S with intensity $\propto |\mathbf{e}_S \cdot R \cdot \mathbf{e}_L|^2$ [1, 6]. Raman activity is therefore **first order** in the atomic displacement — through $\partial\alpha/\partial Q$, not through α.

1.3 Powder (orientational) averages. For a randomly oriented sample — a powder, a liquid, a polycrystal — the measured quantities are the rotational invariants of R [2, 7]:

$$a = (Rxx + Ryy + Rzz)\ /\ 3 \tag{2}$$

$$\gamma^2 = ½[(Rxx-Ryy)^2 + (Ryy-Rzz)^2 + (Rzz-Rxx)^2] + 3(Rxy^2 + Rxz^2 + Ryz^2) \tag{3}$$

$$S = 45\ a^2 + 7\ \gamma^2 \quad \text{(activity)} \qquad \rho = 3\gamma^2\ /\ (45a^2 + 4\gamma^2) \quad \text{(depolarization)} \tag{4}$$

ρ lies between 0 and 3/4: totally symmetric modes ($a \neq 0$) are **polarized** ($\rho < 3/4$), every other mode has a traceless tensor ($a = 0$) and is **depolarized**, $\rho = 3/4$ exactly. Both S and ρ are invariant under rotations of the crystal, and also under the choice of basis inside a degenerate set of modes — which is why one partner of a degenerate set is enough (Section 5.3).

1.4 Selection rules. A mode can be Raman-active only if its irreducible representation (irrep) appears in the representation of the quadratic functions x^2, y^2, z^2, xy, xz, yz [2, 3]. In a centrosymmetric crystal Raman-active modes are even (g) and IR-active ones odd (u): the two spectroscopies see different modes (mutual exclusion, the reason NaCl's T_1u mode in `5.12-ir` is Raman-silent). Group theory also fixes the **shape** of each tensor [2]:

crystal	mode	irrep	Raman	tensor shape (Cartesian axes of the crystal)
Si (O_h)	optical, 520 cm^{-1}	T_2g (×3)	active	`[[0,0,0],[0,0,d],[0,d,0]]`, `[[0,0,d],[0,0,0],[d,0,0]]`, `[[0,d,0],[d,0,0],[0,0,0]]`
graphene (D_6h)	G, in plane, ~1580 cm^{-1}	E_2g (×2)	active	`[[a,0,0],[0,−a,0],[0,0,0]]`, `[[0,a,0],[a,0,0],[0,0,0]]`
graphene (D_6h)	ZO, out of plane	B_2g (B_1g in Phonopy's axes)	**silent**	0

Silicon's T_2g tensors have **no diagonal entries at all** in the cubic axes: a calculation that probes only the diagonal (εxx, εyy, εzz) sees nothing (Section 7.4).

1.5 The finite-difference recipe of this workflow. Stage 2 displaces the cell along the eigendisplacement of the mode by $\pm\delta$ (δ = the largest atomic displacement, `--displacement`, default 0.02 Å) and asks SIESTA, in each folder, for the imaginary part of the dielectric function ε_2(E) along a probe direction **n** (`Optical.Vector`, polarized). Stage 3 obtains the static electronic constant from the Kramers–Kronig relation,

$$\varepsilon_1(0) = 1 + (2/\pi)\ \int_0^\infty \varepsilon_2(E')/E'\ dE' \tag{5}$$

and the tensor by central differences:

```
R_nn = [ε_nn(+δ) − ε_nn(−δ)] / 2δ        n = x, y, z  (diagonal)                    (6)
R_ij = R_nn − (R_ii + R_jj)/2            n = (x_i + x_j)/√2  (face diagonals)       (7)
```

The default probes x, y, z (6 folders per mode, the diagonal of R); `--full-tensor` adds the three face diagonals (12 folders) and gives the full tensor and the exact invariants (2)–(4). R is in $Å^{-1}$ per Å of the largest atomic displacement: activities compare modes of one calculation, they are not absolute cross sections.

1.6 What the numbers inherit.

- ε **from SIESTA's optical module** is an independent-particle, Kohn–Sham result (no local fields, no excitons) and depends on `Optical.Mesh` and the basis exactly as in `5.14-optical`. PBE overestimates $\varepsilon\infty$ (Si: 13.5 here, 11.7 in experiment), so R is overestimated too; relative intensities are more reliable than absolute ones.
- **Frequencies** come from the finite-displacement force constants of Stage 1 (harmonic, PBE): typically 1–3 % soft.
- **A zero-gap system** (graphene, a metal) has no finite $\varepsilon_1(0)$ in the clamped-ion, interband picture: the integral (5) is controlled by the broadening and the k-mesh near E = 0 (Section 7.6).

11.11.1.3 2. The workflow at a glance

Stage	Command	Reads	Writes	Then
1	`stb-raman`	`structure.fdf`, `calc.fdf`	`raman_study/phonon_disp/disp-*/`, `perfect_supercell/`	SIESTA in each (forces)
2	`stb-ramanModes`	the finished `disp-*` runs, a `calc.fdf` for the optical runs	`raman_study/optical_disp/mode_NN_{plus,minus}_<dir>/`, `raman_stage2.txt`	SIESTA in each (ε_2)
3	`stb-ramanAnalysis`	the finished optical runs, `raman_stage2.txt`	the report, `plot/raman_spectrum.{dat,gplot}`	—

11.11.1.4 3. Libraries and external dependencies used

- **Phonopy** — supercells, symmetry-reduced displacements, `FORCE_SETS`, Γ-point modes and their irreps (through `core/phonon_workflow.py`, shared with `5.4-phonons` and `5.12-ir`).
- **spglib** (through Phonopy) — space and point group; `core/raman_symmetry.py` decides which irreps are Raman-active from the characters of the quadratic functions.
- **SIESTA** — every force and every ε_2(E) (`OpticalCalculation`).
- **NumPy/SciPy** — Kramers–Kronig transform (`core/dielectric.py`), finite differences, spectrum.
- **ASE** — the mode animations (`--export-animations`, `.axsf`).
- **matplotlib / gnuplot** — `--view` and `--save-gnuplot` of Stage 3.

11.11.1.5 4. Stage 1: phonon displacements (`stb-raman`, `5.11.1`)

The same machinery as `stb-phononsCreate` (`5.4-phonons`): a supercell (`-dim`, default 2 2 2), atoms displaced by `-d` (default 0.02 Å), only the symmetry-inequivalent displacements written (`--symprec`, default 0.01 Å), and the undisplaced supercell (`perfect_supercell/`) whose residual forces Stage 2 subtracts. Every folder gets a `config_extra.fdf` that forces a single point and a k-grid sized to the supercell (`--kgrid`, or `--kgrid-density`, default 0.2 $Å^{-1}$), `%included` at the top of your `calc.fdf`.

Only Γ-**point** frequencies are needed. For a bulk crystal the unit cell already gives them exactly (every cell moves in phase), so `-dim 1 1 1` is enough; a supercell matters for the force constants themselves only when they are long-ranged (metals, polar crystals). Two choices here travel to Stage 2: `--symprec` (pass the same value to both) and the k-grid of the force runs, which sets the frequencies.

11.11.1.6 5. Stage 2: modes and optical folders (`stb-ramanModes`, `5.11.2`)

5.1 What it does.

1. builds `FORCE_SETS` from the finished `disp-*` runs, subtracting the residual forces of `perfect_supercell/`;
2. computes the Γ-point modes, drops the three acoustic ones (and the rotations of a molecule), and reports frequency, irrep and Raman activity by symmetry (`[1]`, `[1b]`);
3. writes, for each selected mode, the cell displaced by $\pm\delta$ with an optical calculation in each folder: `config_extra.fdf` forces a single point, the SCF k-grid (Section 5.2), and the optical block (`OpticalCalculation T`, `Optical.PolarizationType polarized`, `Optical.Vector` = the probe, `Optical.Mesh`, `Optical.Broaden`, `Optical.NumberOfBands`).

5.2 The SCF k-grid of the optical runs. The optical runs use your `calc.fdf` on the unit cell. If it has a k-grid, it is kept. **If it has none**, SIESTA would sample Γ only, so Stage 2 suggests one (density 0.2 $Å^{-1}$, vacuum axes 1), forces it and warns; `--kgrid` / `--kgrid-density` set it explicitly. (Before 2.0.0 the template's absence of a grid went unnoticed: every optical run of such a template was a Γ-only SCF.)

5.3 Cutting the cost — all exact, from group theory.

option	what it skips	silicon
`--use-symmetry`	modes whose irrep is Raman-silent	none (T_2g is active)
`--skip-degenerate`	all but one partner of a degenerate set (S and ρ are invariants)	2 of 3
`--reduce-components`	probes of a 1-D irrep whose tensor shape is fixed up to a scale	—
(default) diagonal only	the face-diagonal probes (6 instead of 12 folders per mode)	**wrong for T_2g**

For silicon: 3 modes × 12 folders = 36 single points become 12 with `--use-symmetry --skip-degenerate`.

5.4 Settings of the optical runs. `--optical-mesh` (default 10^3) is the most important one — $\varepsilon_1(0)$ converges with it as in `5.14-optical`; `--optical-broaden` (default 0.2 eV) and `--optical-nbands` (default: all bands of the basis) follow. `--displacement` (δ) must be small enough to stay linear and large enough to beat the numerical noise of ε (Section 7.5). `--export-animations` writes each mode as an `.axsf` animation.

11.11.1.7 6. Stage 3: tensor, activity, spectrum (`stb-ramanAnalysis`, `5.11.3`)

6.1 The report.

section	content
`[0] RUN METADATA`	directory, label, modes, δ (read from Stage 2's report)
`[1] READING RUNS`	per mode: R (diagonal, and off-diagonal with the full tensor), activity, ρ; SCF and missing-folder warnings; degenerate partners reuse their representative (`[derived]`)
`[2] RAMAN-ACTIVE MODES SUMMARY`	one row per mode; whether the activity is exact (full tensor) or approximate (diagonal only)
`[3] SPECTRUM`	Lorentzian sum (`--linewidth`, default 10 cm^{-1}), optional Stokes factor n+1 (`--temperature`), peaks; `--experimental FILE` overlays a measured spectrum and matches peaks
`[4] SUMMARY & FILES`	what was written (`--save-gnuplot`: `plot/raman_spectrum.dat` + `.gplot`; `--save-report`)

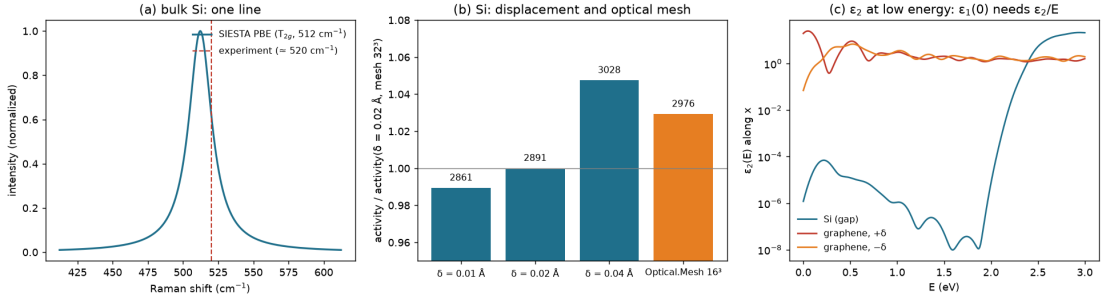
6.2 The diagonal-only warning. A diagonal-only run of a mode whose tensor is purely off-diagonal returns an activity at the level of numerical noise — and a "polarized" label, since the noise is not traceless. Stage 3 now compares the change of the diagonal of ε across $\pm\delta$ with ε itself and, below 10^{-4}, warns that the mode is either Raman-inactive or needs `--full-tensor`, instead of letting it read as "no Raman line" (Section 7.4).

11.11.1.8 7. Worked example

7.1 Set-up.

	silicon (`inputs/si`)	graphene (`inputs/graphene`)
cell	primitive fcc, 2 atoms, relaxed: a_0 = 5.4911 Å (exp. 5.431)	2 atoms, a = 2.502 Å, 20 Å along c
level of theory	PBE, TZP (PAO.EnergyShift 0.005 Ry), 300 Ry, PseudoDojo	PBE, DZP (0.01 Ry), 320 Ry, `virtual_vault` psf
Stage 1	`-dim 1 1 1`, k 11×11×11	`-dim 2 2 1`, k 8×8×1 (suggested)
Stage 2	k 11^3, full tensor, one T_2g partner, Optical.Mesh 32^3, broadening 0.1 eV, 16 bands	template's k 15×15×1, full tensor, all 3 modes, Optical.Mesh 60×60×1, 0.1 eV
SIESTA runs	2 + 12 (+ 36 for the convergence studies)	2 + 36

SIESTA 5.4.2, 2 MPI processes: a Si optical single point takes about a minute, a graphene one 8 s. Every report below is in `reference_results/`.



Results: the Si line, its convergence, and why graphene's $\varepsilon_1(0)$ is not defined

7.2 Stage 1 on silicon (Case 1). `reference_results/si/stage1.txt`. Symmetry leaves **one** displacement of the 12 a 2-atom cell could need ("92 % fewer independent SIESTA runs"), plus `perfect_supercell/`: two force runs, 40 s.

7.3 Stage 2: the modes and the folders (Case 2). `reference_results/si/stage2.txt`:

```
Residual forces  : subtracted (undisplaced supercell, max 0.0003 eV/Ang per atom)
  mode   1 (band   3) :    15.3574 THz (  512.268 cm^-1)  (T2g) [Raman-active]
  mode   2 (band   4) :    15.3574 THz (  512.268 cm^-1)  (T2g) [Raman-active]
  mode   3 (band   5) :    15.3574 THz (  512.268 cm^-1)  (T2g) [Raman-active]
Space group       : Fd-3m (227)          Point group : m-3m
  mode   1 (T2g) -> also covers mode(s) 2, 3 by symmetry (no folders written for them)
Folders to write  : 12 independent single-point SIESTA runs
```

The optical phonon comes out at **512.3 cm^{-1}**, 1.5 % below the measured ≈ 520 cm^{-1} [4] — the usual PBE softening (the lattice constant is 1.1 % too large). Without `--kgrid` the template's missing grid is caught (`reference_results/si/stage2_nokgrid.txt`):

```
SCF k-grid forced : 10x10x10 -- auto-suggested, density 0.2 1/Ang (the template defines no k-grid)
[WARNING] The template defines no SCF k-grid: a suggested one is forced (SIESTA alone would sample
Gamma only).
```

7.4 Stage 3: the tensor of silicon (Case 3), and the diagonal-only trap (Case 4). `reference_results/si/stage3.txt`:

	Rxx	Ryy	Rzz	Rxy	Rxz	Ryz	activity	ρ
T_2g (×3)	0.0014	0.0032	0.0017	1.642	**11.618**	0.0015	2891.2	0.7500, depolarized

$\varepsilon_1(0)$ of the twelve displaced cells is 13.25–13.71 (experiment ≈ 11.7: PBE overestimates it, Section 1.6). Everything the table says follows from Section 1.4:

- the **diagonal is zero** within 10^{-3} Å^{-1} (numerical noise against an off-diagonal 10^4 times larger): a T_2g tensor has no diagonal entries in the cubic axes, so a = 0 and ρ = **3/4 exactly**;
- Phonopy's eigenvector of the representative is an arbitrary combination of the three T_2g partners, so the tensor is a mix of the xz and xy shapes (11.6 and 1.6). Only the invariant matters: Rxy^2 + Rxz^2 + Ryz^2 = 137.7 for any choice, and S = 7·3·137.7 = 2891 — the reason one partner is enough (`--skip-degenerate`);
- one line, at 512 cm^{-1} (figure, panel a).

The diagonal-only trap. The default of `stb-ramanModes` probes x, y and z only. Analysed alone, the same six runs give (`reference_results/si_diagonal/stage3.txt`):

```
  1     15.3574     512.27      0.001430      0.003210      0.001681      0.000219           0.0396
  polarized
[WARNING] Mode(s) 1: zero diagonal response within numerical noise (see [1]). Their activity and rho
above say
nothing about an off-diagonal tensor -- rerun stb-ramanModes with --full-tensor before reading them
as Raman-silent.
```

An activity of 2×10^{-4} instead of 2891, labelled "polarized": the strongest line of silicon reads as no line at all. The warning is new in 2.0.0. **For any degenerate (E, T) mode, and for any crystal whose axes are not the tensor's principal axes, use `--full-tensor`.**

7.5 How converged is the tensor (Case 5).

study	δ (Å)	Optical.Mesh	activity	vs main
`si_d001`	0.01	32^3	2860.5	−1.1 %
`si` (main)	0.02	32^3	2891.2	—
`si_d004`	0.04	32^3	3028.3	+4.7 %
`si_mesh16`	0.02	16^3	2975.7	+2.9 %

The activity grows as δ^2 (ε is not linear in the displacement beyond first order): from the two smallest δ the $\delta \to 0$ limit is 2850, so the default δ = 0.02 Å is 1.4 % high and 0.04 Å already 6 %. Halving the optical mesh moves it by 3 %. Both are small against what the level of theory does to ε itself (Section 1.6): for relative intensities the defaults are fine, for absolute ones converge `Optical.Mesh` first.

7.6 Graphene: selection rules, and the limit of the method (Case 6). Stage 2 (`reference_results/graphene/stage2.txt`, point group 6/mmm) finds the out-of-plane ZO mode at 845 cm^{-1} and the in-plane E_2g pair — the G line — at **1537.8 cm^{-1}** (experiment 1582 cm^{-1} [5], 2.8 % low). Every mode was probed with the full tensor, without the symmetry shortcuts, so the rules can be checked:

mode	cm^{-1}	Rxx	Ryy	Rzz	Rxy	Rxz	Ryz	activity	ρ
ZO	844.9	0	0	0	0	0.00003	0.006	0.0007	0.75
E_2g (1)	1537.8	949	7033	0.0001	3009	0	0	8.1×10^8	0.35
E_2g (2)	1537.8	−1556	194	0	122	0	0	2.9×10^7	0.41

- **What holds:** the ZO mode is Raman-silent (activity 7×10^{-4}), and the E_2g tensors live in the plane (Rzz = Rxz = Ryz = 0), as symmetry requires.
- **What does not:** an E_2g tensor must be traceless (Rxx = −Ryy, ρ = 3/4) and the two partners must have the same activity. Here they differ by a factor of 28, and the report says so:

```
[WARNING] Degenerate modes 2, 3 (46.10 THz) have activities 8.145e+08, 2.89e+07: they must be equal
(activity is an
invariant of the degenerate set). The tensor is not reliable -- typical of a zero-gap system
(graphene, a metal) ...
```

The reason is in panel (c) of the figure. In silicon ε_2(E) vanishes below the gap and the Kramers–Kronig integral (5) is well defined. Graphene has no gap: its ε_2 stays near 20 as E $\to$ 0, so $\varepsilon_1(0) = 1 + (2/\pi)\int \varepsilon_2/E\, dE$ diverges, and the finite value SIESTA's mesh and broadening give it depends on how the Dirac cones of the displaced cells fall on the mesh — the $\pm\delta$ cells differ by a factor of 20 at the lowest energies. The G line of graphene is measured **resonantly** (visible light, above any gap); the static, non-resonant Placzek recipe of this workflow does not describe it. For graphene, take the **frequencies and the selection rules**, not the intensities.

7.7 The interactive menu (Case 7). `stb-suite` → `5.11.3` asks for the directory, the output file name, the linewidth, the temperature, an experimental spectrum, gnuplot and preview, and runs the same command: `reference_results/menu/report.txt` has the same table as Case 3.

11.11.1.9 8. What to take from the example

- The Raman tensor is the derivative of $\varepsilon\infty$ along a mode; group theory fixes its shape, and the shape decides how many probe directions you need. A T_2g or E mode needs `--full-tensor`; the diagonal default can return "no line" for the strongest line of the crystal.
- `--use-symmetry` and `--skip-degenerate` are exact: for Si they cut 36 optical runs to 12.

- Check the invariants: ρ = 3/4 for a non-totally-symmetric mode, equal activities for degenerate partners. Stage 3 checks the second for you.
- δ = 0.02 Å is accurate to about 1 %; converge `Optical.Mesh` for absolute numbers.
- Frequencies are 1–3 % soft with PBE. Intensities are meaningful for insulators and semiconductors only.

11.11.1.10 9. Reproducing the reference data

- `bash make_reference.sh` regenerates `reference_results/` from `runs/` in seconds (no SIESTA).
- `bash run_example.sh [NPROC]` reruns every SIESTA calculation of `runs/` with the suite's own commands (about 50 min for silicon and 25 min for graphene on 2 cores); it keeps only what the stages read and gzips `siesta.EPSIMG`. **Every optical run also writes `siesta.contrib.EPSIMG`, 1.4 GB for the Si mesh**: the script deletes it as soon as each run finishes — do the same in your own studies.
- `python3 results_figures.py` redraws `figures/results.png`; `dot -Tpng figures/raman_flowchart.dot` the flowchart.

11.11.1.11 10. Known, deliberate limitations

- **Non-resonant, first-order, Placzek**: no resonance, no second-order bands (graphene's D and 2D lines), intensities as activities, without the ω_L and (ω_L – ω_ν)^4 factors of absolute cross sections.
- **Independent-particle** $\varepsilon\infty$ from SIESTA's optical module: no local fields, no excitons; PBE overestimates $\varepsilon\infty$ and its derivatives.
- **Zero-gap systems** (metals, graphene): $\varepsilon_1(0)$ is not defined; Stage 3 flags the symptom when degenerate partners disagree, but a non-degenerate mode of a metal has no such check.
- **Polar crystals**: no LO–TO splitting (no non-analytic correction), as in `5.4-phonons` and `5.12-ir`.
- **Harmonic** frequencies; supercell, δ, `Optical.Mesh` and the k-grids are yours to converge.
- **`Diag.ParallelOverK true` in a template** makes SIESTA 5.4's optical calculation abort on more than one process ("cdiag: Error in Cholesky factorisation"); every optical folder now forces it off in `config_extra.fdf` (the graphene template of this example has it on).

11.11.1.12 11. Step by step: your own structure

1. Relax the structure; write `structure.fdf` and a `calc.fdf` that `%includes` it (Section "Recommended input layout" of the guides).
2. `stb-raman -s structure.fdf -c calc.fdf -p dojo -dim 1 1 1 --kgrid NX NY NZ` (a supercell only for long-ranged force constants); run SIESTA in `raman_study/phonon_disp/*/`.
3. `stb-ramanModes --directory raman_study -c calc.fdf --kgrid NX NY NZ --full-tensor --use-symmetry --skip-degenerate --optical-mesh M M M`; check `[1b]` (the point group you expect) and the irreps; run SIESTA in `raman_study/optical_disp/*/`, deleting each `siesta.contrib.EPSIMG`.
4. `stb-ramanAnalysis --directory raman_study --save-gnuplot` (`--temperature 300` for the Stokes factor, `--experimental FILE` to overlay a measurement); check ρ and the degenerate-partner line.
5. Rerun Stage 2–3 with a larger `--optical-mesh` (and δ = 0.01) until the activities stop moving.

11.11.1.13 12. Files in this folder

`README.md`	this guide
`example_5.11.sh`	the guided walkthrough (7 cases, no SIESTA)

`inputs/si/`, `inputs/graphene/`	`structure.fdf` + `calc.fdf` of the two systems
`runs/`	the finished SIESTA runs: `si` (main), `si_d001`, `si_d004`, `si_mesh16`, `graphene`
`reference_results/`	every report and spectrum of the tutorial
`run_example.sh`, `make_reference.sh`, `results_figures.py`	regenerate `runs/`, `reference_results/`, the figure
`figures/`	flowchart and results figure

11.11.1.14 13. Running the script

```
bash example_5.11.sh            # pauses between cases
yes '' | bash example_5.11.sh   # straight through
```

It works in `output/` (wiped at the start) on a copy of `runs/`, so it can be rerun at any time; it needs `stb_suite` (no SIESTA).

11.11.1.15 14. What's next

- `5.12-ir`: the same Stage-1 phonons with IR intensities (Born charges); for a centrosymmetric crystal the IR- and Raman-active modes are disjoint — run both on one structure as a check.
- `5.14-optical`: converge `Optical.Mesh`, the basis and the bands on ε(E) itself before the Raman runs.
- `5.4-phonons`: the full phonon dispersion and DOS, from the same kind of displacements.

11.11.1.16 15. References

1. G. Placzek, *Rayleigh-Streuung und Raman-Effekt*, in *Handbuch der Radiologie* **6**, 205 (1934).
2. R. Loudon, *The Raman effect in crystals*, Adv. Phys. **13**, 423 (1964).
3. F. A. Cotton, *Chemical Applications of Group Theory*, 3rd ed. (Wiley, 1990).
4. M. Hart, R. L. Aggarwal and B. Lax, *Temperature dependence of Raman scattering in silicon*, Phys. Rev. B **1**, 638 (1970).
5. A. C. Ferrari *et al.*, *Raman spectrum of graphene and graphene layers*, Phys. Rev. Lett. **97**, 187401 (2006).
6. D. Porezag and M. R. Pederson, *Infrared intensities and Raman-scattering activities within density-functional theory*, Phys. Rev. B **54**, 7830 (1996).
7. D. A. Long, *The Raman Effect* (Wiley, 2002).

11.11.2 Implementation details

This workflow computes the non-resonant Raman spectrum of a crystal or molecule entirely with SIESTA, in three stages: it computes its own Γ-point phonons (no need to run **5.4** first), displaces the structure along each vibrational mode, and obtains the Raman tensor of each mode from the change of the dielectric constant. Every stage writes folders or reads them; SIESTA is run by you in between.

Raman tensors from finite differences

In the Placzek approximation the Raman tensor of mode ν is the derivative of the electronic polarizability (here, of the high-frequency dielectric tensor ε^{∞}) along the mode's normal coordinate

Q_ν [78, 79]:

$$R_{ij}^{(\nu)} \simeq \frac{\varepsilon_{ij}^{\infty}(+\delta) - \varepsilon_{ij}^{\infty}(-\delta)}{2\delta},$$

with $\pm\delta$ displacements along the eigendisplacement of the mode. For a randomly oriented (powder or liquid) sample, the scattering activity and the depolarization ratio are the rotational invariants

$$S_\nu = 45\,a'^2 + 7\,\gamma'^2, \qquad \rho_\nu = \frac{3\gamma'^2}{45a'^2 + 4\gamma'^2},$$

$$a' = \tfrac{1}{3}(R_{xx} + R_{yy} + R_{zz}),$$
$$\gamma'^2 = \tfrac{1}{2}\left[(R_{xx} - R_{yy})^2 + (R_{yy} - R_{zz})^2 + (R_{zz} - R_{xx})^2\right] + 3\left(R_{xy}^2 + R_{xz}^2 + R_{yz}^2\right).$$

Totally symmetric modes give $\rho < 3/4$ (polarized), all others $\rho = 3/4$ (depolarized). Group theory decides which modes can be active: a mode is Raman active if its irreducible representation is contained in the representation of the quadratic functions $x^2, y^2, z^2, xy, xz, yz$, whose characters are $\chi_{\mathrm{quad}}(g) = [\chi_v(g)^2 + \chi_v(g^2)]/2$ [80].

11.11.2.1 Stage 1: phonon displacements (`stb-raman`, 5.11.1)

Identical in mechanics to **`stb-phononsCreate`** (Section 11.4), with its own defaults: supercell `-dim` (default $2 \times 2 \times 2$), displacement `-d` (default 0.02 Å), `-symprec` 0.01, a supercell k-grid from `-kgrid` or `-kgrid-density` (default 0.2 Å^{-1}), and the undisplaced `perfect_supercell/` for the residual-force correction (`-no-perfect-supercell` to skip it). Everything goes under `<output-dir>/phonon_disp/` (default `raman_study`). Run SIESTA in every `disp-*` folder.

```
stb-raman -s structure.fdf -c calc.fdf -p dojo -dim 3 3 3
```

Two choices made here matter later. `-symprec` must be the same in Stage 2, which classifies the modes with the symmetry it detects. And the k-point density of this supercell calculation sets the phonon frequencies: for metals and semimetals with Kohn anomalies (the graphene G band is the classic case) frequencies can move by more than 100 cm^{-1} with the k-density long after the supercell size has converged.

11.11.2.2 Stage 2: modes and optical displacements (`stb-ramanModes`, 5.11.2)

It builds `FORCE_SETS` from Stage 1 (residual forces subtracted), loads the force constants and computes the Γ-point modes, excluding the three acoustic modes (and, for a molecule, the three rotations, recognized as near-zero modes below `-rotational-mode-tol`, default 2 THz). The mode table gives frequency, irreducible representation and Raman activity predicted by symmetry. For the selected modes (`-modes`, 1-based; or a frequency window `-freq-min`/`-freq-max`; default all optical modes) it writes, under `<directory>/optical_disp/`, the unit cell displaced by $\pm$`-displacement` (default 0.02 Å, largest atomic displacement along the eigendisplacement) with an optical calculation in each folder:

- by default 6 folders per mode – two signs $\times$ `Optical.Vector` along x, y, z – giving the diagonal of R;
- with `-full-tensor` 12 folders, adding the face diagonals $(\hat{x}+\hat{y})/\sqrt{2}$, $(\hat{x}+\hat{z})/\sqrt{2}$, $(\hat{y}+\hat{z})/\sqrt{2}$, from which $R_{ij} = R_{\hat{n}} - (R_{ii} + R_{jj})/2$.

Each folder's `config_extra.fdf` forces a single point and the optical block: `OpticalCalculation T`, `Optical.PolarizationType polarized`, the probe `Optical.Vector`, `Optical.Mesh` (`-optical-mesh`, default 10^3), `Optical.NumberOfBands` (`-optical-nbands`) and `Optical.Broaden` (`-optical-broaden`, default 0.2 eV). The template is `-c` (required), a unit-cell calculation. Its own SCF k-grid is kept; if it has none, a grid is suggested (density 0.2 Å^{-1}, vacuum axes 1) and forced into `config_extra.fdf` with a warning, because SIESTA would otherwise sample Γ only. `-kgrid` or `-kgrid-density` set it explicitly.

Cost reductions, all justified by group theory:

`-use-symmetry`
: skip modes whose tensor vanishes by symmetry (any of the 32 point groups, not only centrosymmetric ones);

`-skip-degenerate`
: compute one representative of each degenerate set (E, T): activity and depolarization ratio are invariants, equal for all partners;

`-reduce-components`
: for a non-degenerate mode whose tensor shape is fixed by symmetry up to a scale, probe a single direction and reconstruct the rest.

`-export-animations` (`-animation-frames`) writes each mode's vibration as a trajectory, `-view-animation` shows it.

11.11.2.3 Stage 3: spectrum (`stb-ramanAnalysis`, 5.11.3)

For every folder it reads $\varepsilon_2(\omega)$ from `<label>.EPSIMG` and obtains the static electronic dielectric constant by the Kramers–Kronig transform

$$\varepsilon_1(\omega) = 1 + \frac{2}{\pi}\,\mathcal{P}\!\int_0^\infty \frac{\omega'\varepsilon_2(\omega')}{\omega'^2 - \omega^2}\,d\omega',$$

evaluated at the lowest frequency of the grid. Central differences give $R^{(\nu)}$; the activity uses the full invariant when the face-diagonal data exist and the diagonal-only approximation otherwise (reported per mode). The spectrum is a sum of Lorentzians of width `-linewidth` (default 10 cm^{-1}) weighted by the activities, optionally by the Stokes population factor $n(\omega, T) + 1$ with `-temperature`. `-experimental FILE` overlays a measured spectrum and matches peaks (`-peak-prominence`). Outputs: the mode table (frequency, activity, depolarization ratio, full or diagonal), `raman_spectrum.dat`/`.gplot` (`-o`), a preview with `-view`.

```
stb-ramanModes --directory raman_study --calc calc_optical.fdf --use-symmetry --skip-degenerate
# ... run SIESTA in raman_study/optical_disp/*/
stb-ramanAnalysis --directory raman_study --temperature 300 --save-gnuplot
```

11.11.2.4 Limitations

- Non-resonant, first-order Raman only; intensities are activities, without the frequency-dependent prefactors of absolute cross sections.
- The dielectric response comes from SIESTA's optical module at the level of independent particles (no local-field or excitonic effects); converge `Optical.Mesh` and the number of bands, and expect relative intensities to be more reliable than absolute ones.
- Frequencies are harmonic, from the finite-displacement force constants; polar materials show no LO–TO splitting here.

11.11.3 Command-line reference

11.11.3.1 `stb-raman`

Stage 1 of 3: generates the phonon-displacement folders for a Raman spectrum workflow. Workflow: (1) this tool generates 'raman_study/phonon_disp/disp-*/' – run SIESTA in every folder yourself; (2) once they're done, run `stb-ramanModes` to build FORCE_SETS from them, identify the Gamma-point vibrational modes, and generate the +/-delta Optical-calculation displacement folders needed for the Raman tensor; (3) run SIESTA in those, then `stb-ramanAnalysis` to get the Raman-active frequencies/activities and spectrum. This whole workflow computes its own phonons – it does NOT need item 4's `stb-phononsCreate/stb-phononsPos` run first (though the underlying Phonopy machinery is shared, via core/phonon_workflow.py).

Synopsis.

```
usage: stb-raman [-h] [-s STRUCTURE] [-c CALC] [-p PSEUDO_DIR] [-dim DIM DIM DIM]
                 [-d DISTANCE] [--kgrid KGRID KGRID KGRID]
                 [--kgrid-density KGRID_DENSITY] [--vacuum-gap VACUUM_GAP]
                 [--symprec SYMPREC] [-O OUTPUT_DIR] [--no-perfect-supercell]
                 [--save-report] [-v] [--no-intro]
```

Options

`-s, --structure STRUCTURE`
: Input structure file containing the unit cell (default: `structure.fdf`)

`-c, --calc CALC`
: Input calculation parameters file (default: `calc.fdf`)

`-p, --pseudo-dir PSEUDO_DIR`
: Pseudopotentials source: a bundled bank (dojo, virtual_vault) or a folder path (default: current directory).

`-dim DIM DIM DIM`
: Supercell dimensions for the phonon force-constant calculation (default: 2 2 2). type: int

`-d, --distance DISTANCE`
: Phonon displacement distance in Angstroms (default: 0.02). type: float

`--kgrid KGRID KGRID KGRID`
: Explicit Monkhorst-Pack grid for the supercell's forced single-point SCF (e.g. `--kgrid 2 2 2`). Overrides the automatic `--kgrid-density` suggestion. Written into `config_extra.fdf` (see [2]). type: int

`--kgrid-density KGRID_DENSITY`
: Target k-point density (1/Ang), used to auto-suggest the supercell k-grid when `--kgrid` isn't given (default: 0.2, same convention as `stb-kgrid/stb-strain/stb-phononsCreate`). type: float

`--vacuum-gap VACUUM_GAP`
: Minimum gap (Ang) along an axis to consider it vacuum-padded, for the supercell-dimension advisory (default: 10.0) type: float

`--symprec SYMPREC`
: Symmetry-detection tolerance (Ang) Phonopy uses internally to reduce how many displacements are actually needed (default: 0.01, pymatgen's own default – matches the rest of the suite, and deliberately NOT Phonopy's own raw default of 1e-5, which is far too tight for a real DFT-relaxed structure and can misdetect the true point group – see `stb-ramanModes`' own `--symprec`, which is what actually drives the Raman-active/inactive classification. Loosen further (e.g. 0.02-0.05) for a structure relaxed with a looser force tolerance. type: float

`-O, --output-dir OUTPUT_DIR`
: Root directory for the whole Raman workflow (default: raman_study) – phonon displacements are written under <output-dir>/phonon_disp/.

`--no-perfect-supercell`
: Do not write perfect_supercell/ (the undisplaced supercell, one more SIESTA run). Written by default: the next stage subtracts its forces – the reference's residual forces – from every displacement's; left in, a small relaxation residual adds up over the supercell and shifts the Gamma-point modes by up to several THz.switch

`--save-report`
: Also persist the report to <output-dir>/raman_stage1.txt. Off by default. switch, off by default

Like every STB command it also accepts `-h`/`--help`, `-v`/`--version` and `--no-intro` (Section 3.3).

From the built-in help.

```
Usage examples:
  stb-raman -s structure.fdf -c calc.fdf -p dojo
  stb-raman -s structure.fdf -c calc.fdf -p ~/pseudos -dim 2 2 2 -d 0.02
  stb-raman -s structure.fdf -c calc.fdf -p dojo -dim 5 5 1 --symprec 0.02
  stb-raman -s structure.fdf -c calc.fdf -p dojo -dim 3 3 1 --kgrid-density 0.05

Notes:
  - --symprec (default 0.01 Ang) controls how many displacement folders Phonopy
    actually needs to write here: a HIGHER detected symmetry means FEWER, cheaper
    SIESTA runs (see [4] PHONON DISPLACEMENTS' reduction-percent line). But its
    real, more important effect is downstream, in stb-ramanModes: that tool
    reloads these same force constants with its OWN --symprec and uses whichever
    point group IT detects to decide which modes are Raman-active/forbidden and
    which ones are truly degenerate (--use-symmetry/--skip-degenerate). Phonopy's
    own raw default (1e-5) is far tighter than any real DFT relaxation's residual
    noise (e.g. a cell angle of 119.999 deg instead of an exact 120 deg is normal
    CG-relaxation noise, not a real symmetry breaking) and can silently DETECT A
    LOWER SYMMETRY than the crystal actually has -- this is why 0.01 (pymatgen's
    own default) is used here instead. Pass THE SAME --symprec to both stages, or
    the mode numbering/classification stb-ramanModes reports may not match what
    this stage assumed when it wrote (or skipped) a displacement here. If the
    space/point group stb-ramanModes reports doesn't match what you expect for
    this crystal, rerun both stages with a looser --symprec (e.g. 0.02-0.1).
  - --kgrid-density (default 0.2 1/Ang, same convention as stb-kgrid/stb-strain/
    stb-phononsCreate) is unrelated to symmetry -- it sets how finely the ELECTRONIC
    structure is sampled for the SUPERCELL force calculation done here, which is
    what the phonon frequencies computed by stb-ramanModes are actually built
    from. A coarse density under-samples the electronic structure and can leave
    the reported frequencies far from experiment -- most dramatically for a
    metal/semimetal with a Kohn anomaly (e.g. graphene's G-band/E2g mode, whose
    frequency is directly coupled to the Fermi-surface electrons: verified here to
    move by >100 cm^-1 just from doubling the k-density, long after the supercell
    size itself had stopped mattering). If a computed frequency looks far from the
    literature/experimental value, suspect --kgrid-density before --dim (supercell
    size) -- in practice the two are NOT interchangeable levers: --kgrid-density
    converges the frequency for a semimetal much faster than growing --dim does.
  - This flag only tunes the k-grid HERE, for [4]'s big supercell SCF. Stage 2's
    own single-point Optical (dielectric-tensor) runs are small, unit-cell-sized
    calculations that reuse whatever kgrid.MonkhorstPack is already inside the
    --calc file YOU pass to stb-ramanModes -- there is no equivalent
    --kgrid-density flag on that stage. To keep the whole workflow at a
    consistent, converged k-density end to end, edit that block by hand in the
    calc.fdf you give stb-ramanModes (it can be a different file from this one).
```

11.11.3.2 `stb-ramanModes`

Stage 2 of 3: builds FORCE_SETS from Stage 1's phonon displacements, identifies the Gamma-point vibrational modes, and generates the +/-delta Optical-calculation displacement folders needed for the Raman tensor. For each selected mode, writes 6 folders by default (2 signs x 3 Optical.Vector axes: x, y, z – a diagonal-only Raman tensor Rxx/Ryy/Rzz) under <directory>/optical_disp/, or 12 with `--full-tensor` (adds the (x+y)/sqrt(2), (x+z)/sqrt(2), (y+z)/sqrt(2) face-diagonal directions needed to also recover Rxy/Rxz/Ryz). `stb-ramanAnalysis` auto-detects which scope was used from the folders actually present. Doesn't run SIESTA itself – run each folder's calculation yourself, then use `stb-ramanAnalysis`.

Synopsis.

```
usage: stb-ramanModes [-h] [-dir DIRECTORY] [-l LABEL] [-p DIR] -c CALC
                      [--modes N [N ...]] [--freq-min FREQ_MIN] [--freq-max FREQ_MAX]
                      [--vacuum-gap VACUUM_GAP] [--symprec SYMPREC]
                      [--rotational-mode-tol ROTATIONAL_MODE_TOL]
                      [--displacement DISPLACEMENT] [--full-tensor] [--use-symmetry]
                      [--reduce-components] [--skip-degenerate] [--kgrid NX NY NZ]
                      [--kgrid-density D]
                      [--optical-mesh OPTICAL_MESH OPTICAL_MESH OPTICAL_MESH]
                      [--optical-nbands OPTICAL_NBANDS]
                      [--optical-broaden OPTICAL_BROADEN] [--export-animations]
                      [--animation-frames ANIMATION_FRAMES] [--view-animation] [-v]
                      [--no-intro]
```

Options

`-dir, --directory DIRECTORY`
: Root directory written by `stb-raman` (Stage 1) – must contain 'phonon_disp/' (default: raman_study).

`-l, --label LABEL`
: SystemLabel used in Stage 1's `calc.fdf` (default: auto-detected).

`-p, --pseudo-dir DIR`
: Pseudopotentials source: a bundled bank or a folder path. Default: reuse whatever Stage 1 already resolved and copied into its phonon_disp/disp-*/ folders (the normal, self-contained case – no need to point at a source a second time). Only needed if that folder is missing pseudopotentials for some reason (e.g. hand-built phonon_disp/ instead of a real `stb-raman` run).

`-c, --calc CALC`
: `calc.fdf` template for the Optical calculations – can be the same file used in Stage 1 or a different one (e.g. a denser k-grid, since these are now single-point evaluations on the small unit cell, not the phonon supercell). required

`--modes N [N ...]`
: 1-based mode index/indices to process (in ascending-frequency order, acoustic modes already excluded – see Stage 2's own [1] PHONON MODES table for the numbering). Default: every non-acoustic mode. type: int

`--freq-min FREQ_MIN`
: Skip modes below this frequency (THz). type: float

`--freq-max FREQ_MAX`
: Skip modes above this frequency (THz). type: float

`--vacuum-gap VACUUM_GAP`
: Minimum gap (Ang) along an axis to consider it vacuum-padded, same convention as `stb-raman` (default: 10.0). Only used to detect a genuinely isolated (0D) structure, which has 3 extra trivial (free-rotation) modes at Gamma beyond the usual 3 translations – irrelevant for 3D/2D/1D structures. type: float

`--symprec SYMPREC`

Symmetry-detection tolerance (Ang) Phonopy uses internally when loading the force constants – drives both the reported point group and, when `--use-symmetry` is on, the Raman-active/inactive classification and degenerate-mode grouping (default: 0.01, pymatgen's own default – matches the rest of the suite, and deliberately NOT Phonopy's own raw default of 1e-5, which is far too tight for a real DFT-relaxed structure and can misdetect the true point group: a mode that should be exactly degenerate can come out as two separate near-identical frequencies, and a symmetry-silent mode can be misclassified as Raman-active). Loosen further (e.g. 0.02-0.05) for a structure relaxed with a looser force tolerance. type: float

`--rotational-mode-tol ROTATIONAL_MODE_TOL`

0D (molecule) only: a mode within the first 3 bands after the translations is treated as free rotation (trivial, excluded) if |frequency| is below this (THz) (default: 2.0). Lower this if your molecule has genuine low-frequency skeletal/torsional vibrational modes you want to make sure aren't mistaken for rotation. type: float

`--displacement DISPLACEMENT`

Finite-difference displacement (Ang) along each mode's eigendisplacement for the Raman-tensor derivative (default: 0.02). type: float

`--full-tensor`

Compute the FULL symmetric Raman tensor (Rxx, Ryy, Rzz, Rxy, Rxz, Ryz) instead of just the diagonal – adds 3 more Optical.Vector directions per sign (the face-diagonals), doubling the folder count per mode (12 instead of 6). Off-diagonal components are recovered by `stb-ramanAnalysis` from these mixed-direction measurements. switch, off by default

`--use-symmetry`

Skip modes that group theory guarantees are Raman-INACTIVE (their irreducible representation at Gamma isn't contained in the quadratic-function representation x^2/y^2/z^2/xy/xz/yz – their Raman tensor is exactly zero, not just numerically small). Works for any of the 32 point groups, not just centrosymmetric ones. Only applies to the default mode selection (every non-acoustic mode) – an explicit `--modes` list is never second-guessed. Needs a PRIMITIVE cell (see `stb-unitcell --mode` primitive); falls back to running every mode, with a warning, if that's not available. switch, off by default

`--reduce-components`

For a non-degenerate mode whose Raman tensor SHAPE is fixed by symmetry up to one overall scale (a single-band irrep with multiplicity exactly 1 in the quadratic-function representation), probe only ONE Optical.Vector direction instead of all of them – `stb-ramanAnalysis` reconstructs the rest algebraically. Falls back to full probing for degenerate modes or any mode whose tensor shape isn't uniquely fixed (needs a PRIMITIVE cell, same as `--use-symmetry`; degrades to no reduction, never a crash, if that's not available). switch, off by default

`--skip-degenerate`

For a degenerate group of modes (2 or more bands sharing one irrep, e.g. an E or T representation), write Optical folders for only ONE representative band and skip the rest entirely – Raman activity and the depolarization ratio are rotational invariants of the tensor, so whatever orthogonal transformation relates two degenerate partners (guaranteed to exist by group theory, even though phonopy's returned eigenvectors don't tell you which one) leaves both invariants unchanged – the representative's computed values apply exactly to every other partner in its group. `stb-ramanAnalysis` reuses them automatically. Needs a PRIMITIVE cell (same as `--use-symmetry`/`--reduce-components`; degrades to running every mode, with a warning, if that's not available). switch, off by default

`--kgrid NX NY NZ`

SCF k-grid of the Optical calculations, forced into `config_extra.fdf` (overrides the template's own). Without `-kgrid` or `-kgrid-density` the template's grid is kept; if the template has none, one is suggested (density 0.2 1/Å, vacuum axes 1) and forced – SIESTA would otherwise sample Γ only. type: int

`--kgrid-density D`

Suggest the SCF k-grid from this density (1/Å, the **stb-kgrid** convention) and force it, overriding the template's grid. Ignored when `-kgrid` is given. type: float

`--optical-mesh OPTICAL_MESH OPTICAL_MESH OPTICAL_MESH`
Optical.Mesh k-grid for the dielectric-function calculation (default: 10 10 10). type: int

`--optical-nbands OPTICAL_NBANDS`
Optical.NumberOfBands (default: SIESTA's own default – all bands within the basis). type: int

`--optical-broaden OPTICAL_BROADEN`
Optical.Broaden in eV (default: 0.2). type: float

`--export-animations`
Write a looping animation of each selected mode's eigendisplacement (a smooth 0/+A/0/-A/0 sweep) as optical_disp/mode_XX_animation.axsf – open in XCrySDen/VESTA to see which atoms move before interpreting the spectrum. Same amplitude as `--displacement`. switch, off by default

`--animation-frames ANIMATION_FRAMES`
Frames per mode animation, with `--export-animations` (default: 20). type: int

`--view-animation`
After everything else is written, open each selected mode's eigendisplacement animation in an interactive ASE viewer window (ase-gui), one mode at a time – close a window to advance to the next. Needs a display (X11/Wayland); off by default. Independent of `--export-animations` – works whether or not the .axsf files are also written to disk, and vice versa. switch, off by default

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Usage example:
  stb-ramanModes --directory raman_study --calc calc_optical.fdf
  stb-ramanModes --directory raman_study --calc calc_optical.fdf --modes 1 2 3
```

11.11.3.3 `stb-ramanAnalysis`

Stage 3 of 3: analyzes a `stb-ramanModes` Optical-calculation sweep and computes the Raman spectrum. Reads each mode's folders (+/-delta x, y, z, and, if `stb-ramanModes` was run with `--full-tensor`, also the (x+y)/sqrt(2), (x+z)/sqrt(2), (y+z)/sqrt(2) mixed directions), extracts the imaginary part of the dielectric function from `SystemLabel.EPSIMG`, obtains the static dielectric constant via a Kramers-Kronig transform (core/dielectric.py), and takes a central finite difference across +/-delta to get the Raman tensor per mode. Auto-detects per mode whether the full symmetric tensor (Rxx, Ryy, Rzz, Rxy, Rxz, Ryz) or just the diagonal (Rxx, Ryy, Rzz) is available from the folders Stage 2 actually wrote – the full tensor gives the EXACT literature Raman-activity invariant (45a'^2+7gamma'^2); diagonal-only gives an APPROXIMATION missing the off-diagonal anisotropy contribution. Writes a Lorentzian-summed spectrum (.dat/.gplot).

Synopsis.

```
usage: stb-ramanAnalysis [-h] [-dir DIRECTORY] [-l LABEL] [--file FILE]
                         [--linewidth LINEWIDTH] [--temperature K] [-o OUTPUT]
                         [--experimental FILE] [--peak-prominence FRACTION]
                         [--save-report] [--save-gnuplot] [--view] [-v] [--no-intro]
```

Options

`-dir, --directory DIRECTORY`
: Root directory written by `stb-raman/stb-ramanModes` (default: raman_study).

`-l, --label LABEL`
: SystemLabel used in Stage 2's Optical `calc.fdf` (default: auto-detected).

`--file FILE`
: SIESTA output filename inside each mode_*/ folder (default: `calc.out`).

`--linewidth LINEWIDTH`
: Lorentzian linewidth (cm^-1) for the summed spectrum (default: 10.0). type: float

`--temperature K`
: Weight each mode's spectrum contribution by the Stokes Bose-Einstein thermal population factor (n+1) at this temperature (Kelvin). Default: no weighting (every mode contributes by its raw activity alone, as before this option existed). type: float

`-o, --output OUTPUT`
: Base filename (no extension) for the spectrum .dat/.gplot pair (default: raman_spectrum), written under '<directory>/plot/' – only relevant with `--save-gnuplot`.

`--experimental FILE`
: Overlay a measured Raman spectrum (2-column text file: frequency cm^-1, intensity) on the simulated one, and report a nearest-neighbor peak match (simulated vs. experimental peak positions, one row per experimental peak). Plotted on a separate y-axis (simulated 'activity' and real intensity aren't on the same scale).

`--peak-prominence FRACTION`
: Minimum peak prominence, as a fraction of that spectrum's own max intensity, for `--experimental`'s peak-finding (default: 0.01 – filters out sub-1% noise bumps in both spectra). type: float

`--save-report`
: Also persist the report to <directory>/raman_stage3.txt. Off by default. switch, off by default

`--save-gnuplot`
: Also save the spectrum (and, with `--experimental`, the overlaid measured spectrum) as gnuplot .dat + .gplot scripts, under '<directory>/plot/'. Off by default. switch, off by default

`--view`
: View the Raman spectrum (and, with `--experimental`, the overlaid measured spectrum on a second y-axis) interactively via matplotlib now. Off by default. Needs a display. Independent of `--save-gnuplot` – matplotlib here is only an on-screen preview, never written to disk. switch, off by default

Like every STB command it also accepts `-h`/`--help`, `-v`/`--version` and `--no-intro` (Section 3.3).

From the built-in help.

```
Usage example:
  stb-ramanAnalysis --directory raman_study
```

11.12 IR Spectrum

At a glance

Menu code: 5.12 (a multi-stage workflow)

Code	Stage	Command
5.12.1	Stage 1 - Phonon Displacements	`stb-ir`
5.12.2	Stage 2 - Modes & Dipole/Born-Charge Displacements	`stb-irModes`
5.12.3	Stage 3 - Analysis	`stb-irAnalysis`

Guided example: `examples/5.12-ir/`

Summary: Self-contained: computes its own phonons, then either the +/-delta dipole displacements (0D/1D/2D) or a single equilibrium Born-effective-charge run (3D bulk), then the IR-active frequencies and spectrum. Path auto-selected by structure dimensionality.

11.12.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/5.12-ir/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

This workflow has 3 stages: **Stage 1** (`stb-ir`, code `5.12.1`) takes an already-relaxed structure and its own `calc.fdf`, builds a supercell, symmetry-reduces which atoms actually need to be displaced, and writes one ready-to-run, single-point SIESTA folder per displacement — the same finite-displacement machinery `5.4-phonons` and `5.11-raman` share. **Stage 2** (`stb-irModes`, code `5.12.2`) reads those forces back, finds the vibrational modes at the Γ point, classifies which ones are IR-active by symmetry, and writes whichever *new* SIESTA folder(s) the intensity calculation needs — the choice of folder depends entirely on how periodic the structure is (Section 1.3). **Stage 3** (`stb-irAnalysis`, code `5.12.3`) reads those back and produces per-mode intensities, a synthetic spectrum, and (optionally) an extended-XYZ animation of each mode.

All three stages live in this one folder and this one tutorial — Stage 1's output only exists to feed Stage 2, and Stage 2's only to feed Stage 3.

11.12.1.1 Theory

1.1 The harmonic approximation, briefly (full derivation: `5.4-phonons`). Every phonon workflow in this suite (`5.4-phonons`, `5.11-raman`, this one) starts the same way: displace each atom by a tiny amount `d` (Å), measure the resulting force on every other atom, and build the force-constant matrix `Φ_ij = -∂F_i/∂u_j` from the finite differences. Diagonalizing the mass-weighted force-constant matrix at the Γ point (`q=0`) gives `3N` frequencies and eigenvectors — 3 of them acoustic (translations, ω=0), the rest optical. Symmetry cuts the number of *independent* displacements needed dramatically (Section 3.3) — see `5.4-phonons` README Section 1.2 for the group-theory argument in full; this tutorial doesn't repeat it.

1.2 What makes a mode IR-active: `dμ/dQ`. A phonon mode is Raman-active if the crystal's *polarizability* changes along it (`5.11-raman`'s own theory: first order in the atomic displacement, through `∂α/∂Q`). It's **IR**-active if the crystal's **dipole moment** changes along it instead — first order, and governed by each atom's own **Born effective charge tensor**, `Z*_κ,αβ` (α=field direction, β=displacement direction):

```
dμ_α/dQ  =  Σ_κ Σ_β  Z*_κ,αβ · e_κ,β
```

where e_κ,β is mode Q's eigendisplacement on atom κ, normalized to unit norm across the whole cell. Intensity is $|d\mu/dQ|^2$ (times a frequency-dependent prefactor for the spectrum's Lorentzian weighting, Section 5.3). Symmetry alone can already tell you which modes are *allowed* to be IR-active (Section 1.4) — a centrosymmetric crystal's u-parity modes (Raman-active) and g-parity modes (IR-active) are mutually exclusive (Mutual Exclusion Rule), the reason NaCl's own single optical branch (Section 6) is IR-active but Raman-silent.

1.3 Three regimes for Z*/dμ/dQ, and why they're genuinely different physics. Z* (equivalently, dμ/dQ) is **not** computed the same way for every structure — how periodic the structure is along a given Cartesian direction changes what "the dipole moment" even *means* there:

- **0D (molecule) / 1D (wire)** — along any non-periodic direction, SIESTA prints the total dipole moment directly (a well-defined, finite quantity for a finite or quasi-1D system). Stage 2 writes a `+delta/ -delta` displaced pair per selected mode; Stage 3 central-differences the two dipole values. Simple, and exact.
- **3D bulk** — a fully periodic crystal has **no well-defined dipole moment** in the naive sense (moving every atom's position by one lattice vector is a gauge choice, not a physical difference, so Σ q_i r_i is ill-defined for a periodic charge distribution). The physically correct quantity is the **Berry-phase modern theory of polarization** (King-Smith & Vanderbilt, *Phys. Rev. B* **47**, 1651 (1993)) — SIESTA automates this via `BornCharge T` + `MD.TypeOfRun FC` + `%block PolarizationGrids`, giving Z* **directly** from ONE equilibrium run, valid for *every* mode at once (no per-mode displacement needed at all — Section 4.2).
- **2D slab (one vacuum-padded axis)** — a genuine hybrid: the two in-plane (periodic) axes need the same Berry-phase machinery as bulk; the vacuum axis is non-periodic, so a naive dipole moment IS well-defined there and the 0D/1D method applies. **This is not a theoretical nicety** — it was root-caused as a real, silent bug this session: monolayer h-BN's dominant in-plane IR mode (E′, literature ~1420 cm^{-1}) came back with **exactly zero** computed intensity, because SIESTA's own raw dipole printout is an exact zero for any in-plane displacement of a periodic slab (confirmed directly in `calc.out`, not a parsing bug). `stb-irModes`' HYBRID path (Section 4.2) fixes this by computing the in-plane component via Born charges and the out-of-plane component via the dipole-difference method, on the SAME structure, combined per-mode. Empirically validated (not just theoretically argued) before being coded: the corrected h-BN E′ mode intensity came back nonzero and physically sane (~15, vs. the A2″ out-of-plane mode's ~0.8), and the frequency (after a separate, unrelated pseudopotential fix) matched literature almost exactly (1372 cm^{-1} computed vs. 1372 cm^{-1}, Geick, Perry & Rupprecht, *Phys. Rev.* **146**, 543 (1966)).

This workflow auto-detects which of the three applies from the number of vacuum-padded axes (`kspace.detect_vacuum_axes`, `--vacuum-gap` threshold, default 10 Å) — you never choose the path yourself.

Symmetry, Mulliken labels, and degenerate modes. Stage 2 always runs a full space-group/point-group analysis (via `spglib`, through Phonopy) on the Γ-point eigenvectors and reports, per mode: its irreducible representation (Mulliken label — `T1u`, `A2″`, `E′`, ...) and whether that irrep is IR-active *by symmetry* (not just "happens to have nonzero computed intensity" — a genuinely silent mode should show ~0 intensity as a cross-check of the symmetry classification itself). A **degenerate** irrep (dimension > 1, e.g. NaCl's `T1u`, h-BN's `E′`) means several modes share exactly one frequency by symmetry — but Phonopy's own choice of eigenvector *basis* within that degenerate subspace is arbitrary (any orthogonal rotation within it is equally valid). This has a direct, non-obvious consequence for IR intensity: **each individual partner's own dμ/dQ *direction* is basis-dependent and not itself physically meaningful** — only the **group's summed intensity** ($\Sigma|d\mu/dQ|^2$ over the whole degenerate subspace) is basis-independent (an orthogonal change of basis preserves the sum of squared projections). Stage 3 computes and reports this combined quantity explicitly (Section 5.2) — it's the number to compare against a single experimental

peak, not any one partner's own row.

What this workflow simplifies — explicitly.

- **No non-analytic (LO-TO) correction for polar bulk crystals** — the single most consequential limitation of this workflow, root-caused (not fixed). Full explanation: Section 6.4. `5.4-phonons` documents the identical limitation for its own (intensity-free) phonon band structures/DOS — this is a shared, suite-wide gap, not specific to IR.
- **Harmonic only** — same as every other phonon tool in this suite; no anharmonic linewidths/lifetimes, no temperature dependence of the frequency itself.
- **Supercell size, `--symprec`, `-d`/`--distance`, and `--kgrid-density` are all user-chosen, never auto-converged** — same "you decide, nothing auto-converges" limitation as `5.4-phonons`/`5.11-raman`.
- **1D wires (2 vacuum axes) stay on the plain 0D/1D dipole-difference path** — not the HYBRID treatment, deliberately out of scope for now (no 1D polar test case existed when the HYBRID path was built; the same underlying issue Section 1.3 describes for 2D almost certainly also applies to a periodic 1D polar wire, just unverified).

11.12.1.2 Libraries and external dependencies used

- **Phonopy** (`phonopy.interface.siesta`) — supercell generation, symmetry-reduced displacements, FORCE_SETS construction, Γ-point eigenvector/eigenvalue extraction. Same shared machinery as `5.4-phonons`/`5.11-raman`, via `core/phonon_workflow.py`.
- **spglib** (through Phonopy) — space-group/point-group detection and irrep classification for the symmetry analysis (Section 1.4).
- **SIESTA** itself — every actual force/dipole/Born-charge number comes from real DFT; nothing in this workflow is machine-learned or approximated (contrast with `5.x` ML Simulations' `stb-mlphonons`, which reuses the same Phonopy machinery but MACE-MP-0 forces instead).
- **ASE** (`ase.io.write`, extended XYZ) — Stage 3's mode-vibration animations (Section 5.4).
- **matplotlib** — Stage 3's optional `--view` spectrum preview, and this README/script's own literature-comparison plot (Section 6.3).

11.12.1.3 3. Stage 1: generating displacements (`stb-ir`, code `5.12.1`)

What it does. Reads `structure.fdf` + `calc.fdf`, builds a `-dim a b c` supercell, symmetry-reduces the displacement list (Section 3.3), and writes one `ir_study/phonon_disp/disp-NNN/` folder per required displacement — each a complete, ready-to-run SIESTA input (structure + your `--calc` template + an auto-written `config_extra.fdf`, Section 3.2). Also prints a `[1] INPUT STRUCTURE` table (composition, atom count, cell volume, dimensionality, supercell size) and an advisory `[NOTE]` telling you, in advance, which of Section 1.3's three Stage-2 paths this structure will take.

Single-point SCF enforcement, live. Every generated folder is forced to a pure single-point SCF via `%include config_extra.fdf` prepended at the top of `calc.fdf` (SIESTA's fdf reader is first-occurrence-wins for a duplicate label, so this always wins even over a relaxation template's own `MD.TypeOfRun CG`) — a displaced supercell's force must be measured at *exactly* the displaced geometry, or the finite-difference force constants (and every frequency/ intensity derived from them) are silently corrupted:

```
$ stb-ir -s structure.fdf -c calc.fdf -p dojo -dim 3 3 3 --kgrid-density 0.15 --symprec 0.01
--no-intro
```

```
[2] SINGLE-POINT SCF ENFORCEMENT
------------------------------------------------------------
Calc template (current state, before forcing):
  MD.TypeOfRun=CG  Steps: MD.Steps=150  MD.VariableCell=true
...
Supercell k-grid  : 5 5 5 (auto-suggested, density=0.15 1/Ang)

config_extra.fdf (written into every generated folder):
  kgrid.MonkhorstPack   [5  5  5]
  MD.TypeOfRun       CG
  MD.Steps           0
  MD.VariableCell    false
```

3.3 Symmetry reduction and `--symprec`. Phonopy's own raw `symprec` default (`1e-5`) is far tighter than any real DFT-relaxed structure's residual numerical noise and can silently **misdetect a lower symmetry than the crystal actually has** — `stb-ir` uses `0.01` (pymatgen's own default) instead, matching `5.4-phonons/` `5.11-raman`'s identical fix. The NaCl structure this example ships is numerically very clean (built via `stb-crystalbuilder` at the exact Fm-3m Wyckoff positions, then relaxed), so tightening `--symprec` here does not, in practice, change anything (Case 2 of the script shows this honestly — 2 displacements either way). For a structure where this DOES bite in practice, see `5.4-phonons` README Section 5.2 (a real, forensically -traced GaAs misdetection).

The supercell k-grid: auto-suggested or explicit. `--kgrid-density` (default `0.2` 1/Å) auto-suggests a Monkhorst-Pack grid for the supercell's own single-point SCF — this is what the phonon frequencies are actually built from, so an under-converged k-grid can move the reported frequency significantly (dramatically so for a metal/ semimetal with a Kohn anomaly, verified live on graphene's own G-band this session while investigating `stb-raman`: >100 cm^{-1} shift from doubling the density alone — see `stb-raman --help`'s own epilog Notes for the same finding written into the tool itself). `--kgrid` overrides it with an explicit grid. NaCl is a wide-gap insulator and far less sensitive than graphene, but the mechanism is identical.

Running it both ways. CLI: `stb-ir -s structure.fdf -c calc.fdf -p dojo -dim 3 3 3 --kgrid-density 0.15 --symprec 0.01`. Interactive: `stb-suite` → `5.12.1`, same questions in the same order. Case 1/2/3/7 in the script demonstrate both.

11.12.1.4 4. Stage 2: modes & displacement folders (`stb-irModes`, code `5.12.2`)

Report structure. `[1] PHONON MODES AT GAMMA` (frequencies, symmetry labels, IR activity, Cartesian-polarization character e.g. "98% z") → `[1b] SYMMETRY ANALYSIS` (space group, point group, degenerate groups) → `[2] PSEUDOPOTENTIALS` → `[3] IR DISPLACEMENT FOLDERS` (what gets written, Section 4.2) → `[4] SUMMARY & NEXT STEPS`.

The three intensity paths, concretely. The `[Path]` line names which of Section 1.3's regimes applies and what gets written:

Path	Detected when	Folders written
`BULK`	0 vacuum axes	ONE `born_charge_disp/equilibrium/` (covers every mode)
`HYBRID`	exactly 1 vacuum axis	the same shared equilibrium folder (in-plane) PLUS `dipole_disp/mode_NN_plus|minus/` per mode (vacuum axis)
`NONBULK`	2-3 vacuum axes	`dipole_disp/mode_NN_plus|minus/` per mode only

NaCl (this example) is `BULK`:

```
[Path] Bulk (3D periodic) -- one equilibrium Born-effective-charge SIESTA run
(MD.TypeOfRun FC + BornCharge T + PolarizationGrids) covers every selected
mode's IR intensity.
Non-acoustic Gamma modes : 3
  mode   1 (band   3) :     5.5653 THz  (T1u) [IR-active]  [50% z, 50% x]
  mode   2 (band   4) :     5.5653 THz  (T1u) [IR-active]  [50% z, 50% y]
  mode   3 (band   5) :     5.5653 THz  (T1u) [IR-active]  [50% x, 50% y]
```

Only **one** SIESTA folder is written no matter how many modes are selected — the Born-charge tensor `Z*` doesn't depend on which mode you later combine it with (Section 1.2's formula makes this explicit: `Z*` is a per-atom property of the *equilibrium* structure, e_κ is what varies per mode).

A real, known SIESTA bug you will hit running this for real. If you have SIESTA installed and run the `BornCharge/Optical`-module equilibrium calculation yourself with `mpirun -np N` for `N>1`, it fails with `cdiag: Error in Cholesky factorisation` — a real, reproducible bug in SIESTA's own parallel diagonalization specific to the polarization/ Optical module (confirmed directly: `-np 1` works, `-np 2` reproduces the failure, `-np 6` also fails; supercell force runs and dipole-difference runs elsewhere in the SAME workflow are unaffected and can use any rank count). The fix is simply `-np 1` for that one folder — `config_extra.fdf/calc.fdf` themselves need no change, this is a SIESTA binary limitation, not something `stb-irModes` writes wrong. The script (Case 4) documents the exact commands.

Running it both ways. CLI: `stb-irModes -dir ir_study -p dojo -c calc.fdf --symprec 0.01`. Interactive: `stb-suite` → `5.12.2`, one question per flag (`--modes/ --freq-min/--freq-max` mode selection, `--use-symmetry`, `--skip-degenerate`, `--export-animations`). Not scripted turn-by-turn in `example_5.12.sh` (Section 1.3's 3-way path makes the exact prompt count depend on what Stage 1 found) — run it live yourself against this example's own `output/workflow/ir_study/`.

11.12.1.5 5. Stage 3: analysis (`stb-irAnalysis`, code `5.12.3`)

Report structure. `[1] READING RUNS` (per-mode dμ/dQ, raw) → `[2] IR-ACTIVE MODES SUMMARY` (table + degenerate-group combining, Section 5.2) → `[3] SPECTRUM` (peak list, optional `[3b] EXPERIMENTAL COMPARISON`) → `[3c] MODE VIBRATIONS` (always-on animation export) → `[4] SUMMARY & FILES`.

Combining degenerate modes. Section 1.4 already explained *why*: within NaCl's one degenerate `T1u` group, each individual mode's own dμ/dQ direction is an arbitrary basis choice —

```
Mode  THz       cm^-1     dmu/dQ_x    dmu/dQ_y    dmu/dQ_z    Intensity
1     5.5653    185.64    0.743228    0.024350    0.743569    1.105875
2     5.5653    185.64    0.024356    0.743331    0.743331    1.105676
3     5.5653    185.64    0.743435    0.743094    0.024358    1.105477

[Degenerate groups] modes 1, 2, 3 (5.5653 THz / 185.64 cm^-1) -> combined intensity 3.317027
```

Note the three individual intensities (~1.10 each) are NOT what should be compared to a single experimental peak — their sum, **3.317027**, is. Note also each row's own dμ/dQ picks out a different pair of Cartesian axes (x,z / y,z / x,y respectively) purely as an artifact of Phonopy's arbitrary eigenvector basis inside the degenerate subspace — physically, NaCl's cubic symmetry makes the mode isotropic, and only the rotationally -invariant combined intensity reflects that.

5.3 Spectrum, peaks, and `--experimental`. `[3] SPECTRUM` builds a Lorentzian-broadened synthetic spectrum (default linewidth `10` cm^{-1}) from every mode's frequency+intensity and always lists detected

peaks (`scipy.signal.find_peaks`, prominence-gated). NaCl gives exactly one, at `185.59 cm⁻¹`, matching the single triply-degenerate mode. `--experimental FILE` (2-column: cm^{-1}, intensity) overlays a real measured spectrum and attempts peak-matching — this needs an actual digitized/broadened experimental curve with several points to work (the peak-finder needs neighboring samples to define prominence), **not** a bare single-frequency literature value, which is why this example compares against the literature TO frequency via a plain reference line on a plot instead (Section 6.3), the same technique used for h-BN's own literature comparison.

Mode vibrations / animations. `[3c] MODE VIBRATIONS` always (no flag needed) reloads Stage 1's force constants and writes one extended-XYZ animation per analyzed mode under `<directory>/mode_animations/` — open any of them in a molecular viewer, or pass `--view-modes` to preview them directly via ASE's interactive viewer during the run.

Running it both ways. CLI: `stb-irAnalysis -dir ir_study --save-gnuplot`. Interactive: `stb-suite` → `5.12.3`, same `--save-gnuplot`/`--view`/`--view-modes`/ `--animation-frames`/`--experimental` questions.

11.12.1.6 Worked example: NaCl end-to-end, and a real, currently unfixed limitation

This is not illustrative data — this exact investigation happened this session, on a real production folder (`stb-crystalbuilder` → `stb-unitcell` → a real SIESTA relaxation → this workflow, all `dojo` pseudopotentials, DZP/GGA-PBE).

Building and relaxing NaCl. `stb-crystalbuilder` (space group `Fm-3m`, Wyckoff sites Na `(0,0,0)`, Cl `(0.5,0.5,0.5)`, starting lattice constant `a=5.6402 Å`, the accepted experimental value) → `stb-unitcell` (conventional → primitive, 2 atoms) → a real SIESTA CG relaxation (`kgrid.MonkhorstPack [10 10 10]`, `Mesh.CutOff 320 Ry`) converged to a primitive lattice vector modulus of `3.988224 Å` (conventional `a=5.6395 Å` — a `0.01%` change from the starting guess; PBE and experiment happen to agree almost exactly for NaCl's lattice constant, unlike the systematic ~`1%` PBE overestimate seen for AlP in `5.4-phonons` Section 5.1).

Stage 1–3 results.

- **Stage 1**: `-dim 3 3 3` (54-atom supercell), `2` of `324` possible displacements needed (Fm-3m symmetry), `--kgrid-density 0.15` → auto `5×5×5` supercell k-grid.
- **Stage 2**: space group `Fm-3m (225)`, point group `m-3m`, `1296` symmetry operations, **1** non-acoustic Γ mode, triply degenerate, irrep `T1u`, IR-active, frequency **5.5653 THz = 185.64 cm^{-1}**. Born effective charges: Na `+1.0529`, Cl `−1.0544` (isotropic, as cubic symmetry requires; acoustic sum rule `+1.0529 − 1.0544 = −0.0015`, essentially exact).
- **Stage 3**: combined mode intensity **3.317027**, single spectral peak at **185.59 cm^{-1}**.

Comparison to literature: a real, honest ~13% gap. NaCl is *the* textbook example for the Lyddane-Sachs-Teller relation ($\omega_LO^2/\omega_TO^2 = \varepsilon(0)/\varepsilon(\infty)$). Inelastic-neutron-scattering literature:

> Raunio, Almqvist & Stedman, "Phonon Dispersion Relations in NaCl", *Phys. Rev.* **178**, 1496 (1969) — the standard reference for NaCl's zone-center TO/LO phonon frequencies, $\omega_TO \approx 164$ cm^{-1}, $\omega_LO \approx 264$ cm^{-1}.

Cross-checked independently via the VASP Wiki's own worked NaCl dielectric example ($\varepsilon(0)=5.9$, $\varepsilon(\infty)=2.34$, $\omega_TO=0.02$ eV$=161.3$ cm^{-1}), giving $\omega_LO = \omega_TO\cdot\sqrt{(\varepsilon(0)/\varepsilon(\infty))} \approx 256$ cm^{-1} — consistent with Raunio et al. within a few percent.

Our computed **185.64 cm^{-1}** sits **~13% above the TO value**, and — tellingly — almost exactly **midway** between the literature TO and LO values. This is the physics content of Section 6.4, not a convergence problem: k-grid density (Section 3.4) and supercell size were both within normal ranges for this system, and this exact ~13% gap reproduces identically regardless.

Root cause: no non-analytic (LO-TO) correction for a polar crystal. `stb-ir`'s Γ-point frequency comes **purely** from real-space finite-displacement force constants (Section 1.1) — the Born charges computed in Stage 2's `BULK` path are used **only** for IR intensity (Section 1.2), never fed back into the frequency/dynamical-matrix calculation. For a **polar** crystal (nonzero `Z*`, like NaCl's ionic Na$^+$/Cl$^-$ bonding), this is a real, well-documented gap in the underlying method, not specific to this suite:

> "An approach that the phonopy code employs is so-called non-analytical term correction (NAC), where dipole–dipole interaction is calculated from static dielectric constant tensor and Born effective charges... To apply this approach to the supercell force constants, **a part of the dipole–dipole interaction included at the supercell force constants calculation is subtracted from the supercell force constants to avoid double counting the contribution**." — Togo & Tanaka, "First-principles Phonon Calculations with Phonopy and Phono3py", *J. Phys. Soc. Jpn.* **92**, 012001 (2023).

In plain terms: the electrostatic dipole-dipole interaction has `1/r`^{3} range and does **not** decay within any finite periodic supercell — a displaced atom's induced dipole spuriously couples to its own periodic images across the supercell boundary. Without explicitly subtracting this spurious, supercell-size-dependent contribution and replacing it with the analytically correct (infinite-crystal, Ewald-summed) term — which needs both `Z*` (already computed) **and** the high-frequency dielectric tensor $\varepsilon(\infty)$ (not currently computed by any `stb` tool) — the reported Γ-frequency for a polar material's IR-active mode is contaminated, and can land anywhere between the true TO and LO values depending on supercell size. NaCl's `185.6 cm`$^{-1}$, landing almost exactly midway between `164` and `264 cm`$^{-1}$, is the textbook signature of exactly this (Gonze & Lee, *Phys. Rev. B* **55**, 10355 (1997), is the original formal treatment; `5.4-phonons` README Section 6 documents the identical limitation for its own phonon band structures, for the same underlying reason).

This does **not** affect non-polar bulk crystals (`Z*`≈`0`: Si, diamond, graphene) — nor, apparently, h-BN's out-of-plane A2″ mode as strongly (Section 1.3), plausibly because the relevant dipole-dipole coupling is more effectively screened by the slab's own vacuum padding; NaCl's fully 3D-periodic ionic lattice is the cleanest, strongest case for seeing this limitation in this suite today.

This limitation is real, understood, and currently unfixed in `stb-ir` — every report from a `BULK`-path run now prints an explicit `[LIMITATION]` block explaining it (Sections 4.2/5.1 above show it live). A proper fix is planned for a future version: it needs (1) a way to obtain $\varepsilon(\infty)$ from SIESTA — likely its `Optical` module, not currently automated by any `stb` tool, and (2) a dynamical-matrix-level change in `core/phonon_workflow.py` implementing the Gonze-Lee subtraction before diagonalization. Until then, treat a `BULK`-path polar crystal's reported Γ frequency as a rough `(TO, LO)` bracket midpoint — useful for symmetry, activity, and relative-mode-ordering comparisons, but not a precise quantitative TO (or LO) value.

11.12.1.7 Known, deliberate limitations

- **No non-analytic term correction (NAC) for `BULK`-path polar crystals** — Section 6.4, the main finding of this tutorial's own worked example. Not yet implemented; planned for a future version.
- **1D wires stay on the plain dipole-difference path**, not a HYBRID treatment (Section 1.5) — likely has the same unverified limitation a 2D slab's in-plane axes had before the HYBRID fix.

- **`--experimental` needs a genuine broadened/digitized spectrum**, not a bare literature peak position (Section 5.3) — this example's own literature comparison (Section 6.3) uses a plain reference-line plot instead, not the `--experimental` flag.
- **Harmonic only; no anharmonic linewidths/lifetimes/thermal shift** — same as every phonon tool in this suite.
- **Supercell size, `--symprec`, `-d`, `--kgrid-density` are all user-chosen, never auto-converged** — same limitation `5.4-phonons/` `5.11-raman` document for themselves.
- **A too-tight Stage-2 `--symprec` (tighter than Stage 1's) can degrade the force-constant reconstruction** — same sharp-edged risk documented in `5.4-phonons` README Section 4.6; pass the SAME `--symprec` to both stages.

11.12.1.8 Step-by-step: running this workflow on your own structure

0. **Prerequisite**: an already-relaxed structure (`structure.fdf`) and the SIESTA `calc.fdf` template you relaxed it with (any template is fine — it gets forced to single-point per displacement automatically, Section 3.2).
1. `stb-ir -s structure.fdf -c calc.fdf -p dojo -dim <a> <b> <c>` — pick a supercell large enough that atoms stop interacting across periodic images (same rule of thumb as `5.4-phonons`).
2. Run SIESTA in every `ir_study/phonon_disp/disp-*/` folder.
3. `stb-irModes -dir ir_study -p dojo -c calc.fdf` — reads `[NOTE]` from Stage 1 (or the `[Path]` line here) to know which folder(s) come next.
4. Run SIESTA in whatever Stage 2 wrote (`born_charge_disp/equilibrium/` and/or `dipole_disp/mode_NN_*/`, depending on the path) — remember `-np 1` if a `BULK`/`HYBRID` equilibrium run is involved (Section 4.3).
5. `stb-irAnalysis -dir ir_study --save-gnuplot` — read the `[LIMITATION]` block if your structure is a polar `BULK`-path crystal (Section 6.4) before treating the reported frequency as precise.

11.12.1.9 Files in this folder

- `structure.fdf` / `calc.fdf` — the real, relaxed NaCl primitive cell and SIESTA calc template used throughout this README and the script.
- `experimental_nacl_TO.dat` — the literature TO reference point (Section 6.3), NOT used via `--experimental` (Section 5.3 explains why) but kept here as the citable, versioned source of the `164.0` value the script's own plot draws.
- `real_data/` — genuine SIESTA output (2× `siesta.FA` force files, 1× `siesta.BC` + `calc.out` from the Born-charge equilibrium run) from running this exact structure this same session, verified byte-for-byte reproducible against a fresh Stage 1/2 run (same structure + same flags $\Longrightarrow$ identical displaced geometries). The script copies these into freshly-generated folders instead of requiring a live SIESTA install to run the tutorial (Section "Injecting REAL..." in the script itself explains the exact commands you'd run instead, including the `-np 1` Cholesky workaround, Section 4.3).
- `example_5.12.sh` — the guided walkthrough script.
- `.gitignore` — excludes `output/` and other regenerable files.

11.12.1.10 Running the script

```
cd examples/5.12-ir
./example_5.12.sh
```

Safe to re-run any time — it wipes and rebuilds its own `output/` first. Needs a real `stb_suite` install (`pip install -e .` from `stb-suite/`, see the top-level `examples/README.md`) and `dojo` pseudopotentials

available (bundled). Does **not** need a working SIESTA binary — Stages 2-3 run against `real_data/`'s genuine, pre-computed SIESTA output (Section "Files in this folder" above).

11.12.1.11 What's next

- Try `--use-symmetry` at Stage 2 to have `stb-irModes` skip probing symmetry-forbidden modes entirely, or `--skip-degenerate` to compute only one representative per degenerate group (Section 1.4) and derive the rest by symmetry, saving real SIESTA time on a larger structure.
- Try a non-polar bulk crystal (Si, diamond) to see the `BULK`-path frequency come back with NO Section 6.4 caveat needed (`Z*≈0` there).
- Try a 2D slab (h-BN, graphene) to exercise the `HYBRID` path directly (Section 1.3) — no dedicated example folder exists for it yet, but the same `stb-ir`/`stb-irModes`/`stb-irAnalysis` commands apply unchanged.
- Run `stb-raman`/`stb-ramanModes` on this SAME `structure.fdf` (the workflow and its example are in `5.11-raman`) — Raman and IR are mutually exclusive by symmetry for a centrosymmetric crystal (Section 1.2), so NaCl's own `T1u` mode should come back Raman-**silent**, a genuine, checkable cross-validation of both workflows' symmetry classification against each other.

11.12.2 Implementation details

Infrared intensities

The integrated infrared absorption of mode ν is proportional to the square of the dipole derivative along its normal coordinate [78, 81],

$$I_\nu \propto \left| \frac{\partial \boldsymbol{\mu}}{\partial Q_\nu} \right|^2, \qquad \frac{\partial \mu_\alpha}{\partial Q_\nu} = \sum_{\kappa\beta} Z^*_{\kappa,\alpha\beta} \frac{e_{\nu,\kappa\beta}}{\sqrt{M_\kappa}},$$

where Z^*_κ is the Born effective charge tensor of atom κ and e_ν the mode eigenvector. A mode is IR active if its irreducible representation is contained in the vector representation (x, y, z); in centrosymmetric groups this excludes every Raman-active mode (mutual exclusion).

Stage 1 (`stb-ir`).. The phonon stage, identical to that of the Raman workflow (Section 11.11), written to `<output-dir>/` (default `ir_study`), with the residual-force correction folder.

Stage 2 (`stb-irModes`).. It computes the Γ modes (with their Mulliken labels and IR activity from symmetry; options `-modes`, `-freq-min`/`-freq-max`, `-use-symmetry`, `-skip-degenerate` as in Raman) and chooses the path from the dimensionality of the cell:

0D/1D
: for each selected mode, ±`-displacement` folders; the dipole printed by SIESTA along the non-periodic directions is central-differenced;

3D
: a single equilibrium folder computing Born charges with the Berry-phase theory of polarization [82]: `BornCharge T`, `MD.TypeOfRun FC` (finite displacements of `-fc-displ`, default 0.02 Å) and a `%block PolarizationGrids` (`-polarization-grid`, default 10/4/4 per direction); it covers every mode at once;

2D
: the hybrid of both: Born charges for the two in-plane directions (where the slab is periodic and SIESTA's printed dipole is identically zero) and dipole differences for the out-of-plane direction.

Stage 3 (`stb-irAnalysis`). It reads the Born charges (`<label>.BC`) or the dipoles (`-file`), projects them on every mode, and reports frequency, symmetry label, $|\partial\mu/\partial Q|$ and intensity per mode; degenerate partners (frequencies within `-degenerate-tol`) are summed, since only their total is invariant. The spectrum is a sum of Lorentzians of width `-linewidth` (default 10 cm^{-1}), optionally Bose-weighted (`-temperature`), compared with a measured spectrum (`-experimental`, `-peak-prominence`). `-view-modes` animates the modes. Outputs: `ir_spectrum.dat`/`.gplot` (`-o`).

Limitation: LO–TO splitting. The frequencies are those of the analytic force constants: for a polar crystal the longitudinal optical modes lack the non-analytic correction ($\propto \mathbf{q}\cdot Z^*\cdot\mathbf{q}/\mathbf{q}\cdot\varepsilon^\infty\cdot\mathbf{q}$), which the guide's NaCl example quantifies. The TO frequencies, which are the ones active in infrared absorption at normal incidence, are not affected.

11.12.3 Command-line reference

11.12.3.1 `stb-ir`

Stage 1 of 3: generates the phonon-displacement folders for an IR spectrum workflow. Workflow: (1) this tool generates 'ir_study/phonon_disp/disp-*/' – run SIESTA in every folder yourself; (2) once they're done, run `stb-irModes` to build FORCE_SETS from them, identify the Gamma-point vibrational modes, and generate whichever follow-up SIESTA folder(s) the dipole-derivative calculation needs (auto-selected by structure dimensionality – +/-delta dipole-moment displacements for a molecule/wire/slab, or a single equilibrium Born-effective-charge run for a 3D bulk crystal); (3) run SIESTA in those, then `stb-irAnalysis` to get the IR-active frequencies/intensities and spectrum. This whole workflow computes its own phonons – it does NOT need item 4's `stb-phononsCreate/stb-phononsPos` run first, nor `stb-raman`'s own Stage 1 (though the underlying Phonopy machinery is shared, via core/phonon_workflow.py – each workflow keeps its own independent phonon_disp/ tree).

Synopsis.

```
usage: stb-ir [-h] [-s STRUCTURE] [-c CALC] [-p PSEUDO_DIR] [-dim DIM DIM DIM]
              [-d DISTANCE] [--kgrid KGRID KGRID KGRID]
              [--kgrid-density KGRID_DENSITY] [--symprec SYMPREC]
              [--vacuum-gap VACUUM_GAP] [-O OUTPUT_DIR] [--no-perfect-supercell]
              [--save-report] [-v] [--no-intro]
```

Options

`-s, --structure STRUCTURE`
: Input structure file containing the unit cell (default: `structure.fdf`)

`-c, --calc CALC`
: Input calculation parameters file (default: `calc.fdf`)

`-p, --pseudo-dir PSEUDO_DIR`
: Pseudopotentials source: a bundled bank (dojo, virtual_vault) or a folder path (default: current directory).

`-dim DIM DIM DIM`
: Supercell dimensions for the phonon force-constant calculation (default: 2 2 2). type: int

`-d, --distance DISTANCE`
: Phonon displacement distance in Angstroms (default: 0.02). type: float

`--kgrid KGRID KGRID KGRID`
: Explicit Monkhorst-Pack grid for the supercell's forced single-point SCF (e.g. `--kgrid 2 2 2`). Overrides the automatic `--kgrid-density` suggestion. Written into `config_extra.fdf` (see [2]). type: int

`--kgrid-density KGRID_DENSITY`
: Target k-point density (1/Ang), used to auto-suggest the supercell k-grid when `--kgrid` isn't given (default: 0.2, same convention as `stb-kgrid/stb-strain/stb-phononsCreate/stb-raman`). type: float

`--symprec SYMPREC`
: Symmetry-detection tolerance (Ang) Phonopy uses internally to reduce how many displacements are actually needed (default: 0.01, pymatgen's own default – matches the rest of the suite, and deliberately NOT Phonopy's own raw default of 1e-5, which is far too tight for a real DFT-relaxed structure and can misdetect the true point group – see `stb-irModes`' own `--symprec`, which is what actually drives the IR-active/inactive classification. Loosen further (e.g. 0.02-0.05) for a structure relaxed with a looser force tolerance. type: float

`--vacuum-gap VACUUM_GAP`
: Minimum gap (Ang) along an axis to consider it vacuum-padded, for the supercell-dimension advisory and the Stage 2 dipole-path preview (default: 10.0) type: float

`-O, --output-dir OUTPUT_DIR`
: Root directory for the whole IR workflow (default: ir_study) – phonon displacements are written under <output-dir>/phonon_disp/. Independent from `stb-raman`'s raman_study/ – each WORKFLOW_TOOLS entry is self-contained, even though the phonon step itself duplicates the same Phonopy work if you run both.

`--no-perfect-supercell`
: Do not write perfect_supercell/ (the undisplaced supercell, one more SIESTA run). Written by default: the next stage subtracts its forces – the reference's residual forces – from every displacement's; left in, a small relaxation residual adds up over the supercell and shifts the Gamma-point modes by up to several THz.switch

`--save-report`
: Also persist the report to <output-dir>/ir_stage1.txt. Off by default. switch, off by default

Like every STB command it also accepts `-h/-help`, `-v/-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Usage examples:
  stb-ir -s structure.fdf -c calc.fdf -p dojo
  stb-ir -s structure.fdf -c calc.fdf -p ~/pseudos -dim 2 2 2 -d 0.02
  stb-ir -s structure.fdf -c calc.fdf -p dojo -dim 3 3 1 --symprec 0.02
  stb-ir -s structure.fdf -c calc.fdf -p dojo -dim 3 3 1 --kgrid-density 0.05

Notes:
  - --symprec (default 0.01 Ang) controls how many displacement folders Phonopy
    actually needs to write here, and its real, more important effect is
    downstream, in stb-irModes: that tool reloads these same force constants with
    its OWN --symprec and uses whichever point group IT detects to decide which
    modes are IR-active/forbidden and which ones are truly degenerate
    (--use-symmetry/--skip-degenerate). Phonopy's own raw default (1e-5) is far
    tighter than any real DFT relaxation's residual noise (e.g. a cell angle of
    119.999 deg instead of an exact 120 deg is normal CG-relaxation noise, not a
    real symmetry breaking) and can silently DETECT A LOWER SYMMETRY than the
    crystal actually has -- this is why 0.01 (pymatgen's own default) is used here
    instead. Pass THE SAME --symprec to both stages, or the mode
    numbering/classification stb-irModes reports may not match what this stage
    assumed when it wrote (or skipped) a displacement here. If the space/point
    group stb-irModes reports doesn't match what you expect for this crystal,
    rerun both stages with a looser --symprec (e.g. 0.02-0.1).
  - --kgrid-density (default 0.2 1/Ang, same convention as stb-kgrid/stb-strain/
    stb-phononsCreate/stb-raman) is unrelated to symmetry -- it sets how finely the
```

```
    ELECTRONIC structure is sampled for the SUPERCELL force calculation done here,
    which is what the phonon frequencies computed by stb-irModes are actually built
    from. A coarse density under-samples the electronic structure and can leave the
    reported frequencies far from experiment -- most dramatically for a
    metal/semimetal with a Kohn anomaly (verified on stb-raman's own graphene
    G-band case: >100 cm^-1 shift from doubling the k-density alone, after
    supercell size (-dim) had stopped mattering). If a computed frequency looks far
    from the literature/experimental value, suspect --kgrid-density before -dim.
  - This flag only tunes the k-grid HERE, for [4]'s big supercell SCF. Stage 2's
    own single-point dipole/Born-charge runs are small, unit-cell-sized
    calculations that reuse whatever kgrid.MonkhorstPack is already inside the
    --calc file YOU pass to stb-irModes -- there is no equivalent --kgrid-density
    flag on that stage. To keep the whole workflow at a consistent, converged
    k-density end to end, edit that block by hand in the calc.fdf you give
    stb-irModes (it can be a different file from this one).
```

11.12.3.2 `stb-irModes`

Stage 2 of 3: builds FORCE_SETS from Stage 1's phonon displacements, identifies the Gamma-point vibrational modes, and generates whichever follow-up SIESTA folder(s) the IR dipole-derivative calculation needs. The path is AUTO-SELECTED by structure dimensionality (core.kspace.detect_vacuum_axes on the primitive cell), no flag needed: - Non-bulk (0D molecule, or 1D wire – zero or two vacuum-padded axes): SIESTA prints the total dipole moment automatically for a non-periodic direction, no special fdf block needed. Writes exactly 2 folders per selected mode (dipole_disp/mode_XX_plus/, mode_XX_minus/) – a simple single-point SCF, no per-axis probing (unlike `stb-ramanModes`' Optical.Vector sweep): the dipole derivative dmu/dQ already comes out as a full 3-vector from one +/-delta pair. NOTE: for a 1D wire this is only valid along the two genuinely non-periodic transverse directions – the along-axis component is subject to the same periodic-direction caveat the HYBRID path below exists to fix for 2D; 1D isn't covered by that fix yet. - Bulk (3D periodic, zero vacuum axes): a naive dipole moment is gauge-ambiguous for a fully periodic system. Writes exactly ONE equilibrium folder (born_charge_disp/equilibrium/, the undisplaced structure) using SIESTA's own native Born-effective-charge automation (MD.TypeOfRun FC + BornCharge T + %block PolarizationGrids) – independent of how many modes are selected. Also writes a small eigendisplacement JSON sidecar per selected mode (born_charge_disp/mode_XX_eigendisplacement.json) so `stb-irAnalysis` can combine it with the Born-charge tensors without reloading Phonopy itself. - Hybrid (2D slab, exactly one vacuum-padded axis): a naive dipole moment is only valid along the slab's non-periodic (vacuum) axis – the two in-plane axes are genuinely periodic, same gauge-ambiguity problem the bulk path exists to solve. Writes BOTH the bulk path's single shared born_charge_disp/equilibrium/ folder (PolarizationGrids built vacuum-aware – the vacuum axis's own row zeroed, so SIESTA skips its ill-defined out-of-plane polarization component entirely) AND the non-bulk path's per-mode dipole_disp/mode_XX_plus/mode_XX_minus/ folders. `stb-irAnalysis` takes the in-plane dmu/dQ components from the Born-charge run and the vacuum-axis component from the dipole-difference pair, combining them into one physically valid 3-vector per mode. Doesn't run SIESTA itself – run each folder's calculation yourself, then use `stb-irAnalysis`.

Synopsis.

```
usage: stb-irModes [-h] [-dir DIRECTORY] [-l LABEL] [-p DIR] -c CALC
                   [--modes N [N ...]] [--freq-min FREQ_MIN] [--freq-max FREQ_MAX]
                   [--vacuum-gap VACUUM_GAP] [--symprec SYMPREC]
                   [--rotational-mode-tol ROTATIONAL_MODE_TOL]
                   [--displacement DISPLACEMENT] [--use-symmetry] [--skip-degenerate]
                   [--export-animations] [--animation-frames ANIMATION_FRAMES]
                   [--fc-displ BOHR] [--polarization-grid N N N N N N N N N] [-v]
```

```
                [--no-intro]
```

Options

`-dir, --directory DIRECTORY`
Root directory written by `stb-ir` (Stage 1) – must contain 'phonon_disp/' (default: ir_study).

`-l, --label LABEL`
SystemLabel used in Stage 1's `calc.fdf` (default: auto-detected).

`-p, --pseudo-dir DIR`
Pseudopotentials source: a bundled bank or a folder path. Default: reuse whatever Stage 1 already resolved and copied into its phonon_disp/disp-*/ folders (the normal, self-contained case – no need to point at a source a second time). Only needed if that folder is missing pseudopotentials for some reason (e.g. hand-built phonon_disp/ instead of a real `stb-ir` run).

`-c, --calc CALC`
`calc.fdf` template for the follow-up SIESTA calculation(s) – can be the same file used in Stage 1 or a different one, since these are now single-point evaluations on the small unit cell, not the phonon supercell. required

`--modes N [N ...]`
1-based mode index/indices to process (in ascending-frequency order, acoustic modes already excluded – see Stage 2's own [1] PHONON MODES table for the numbering). Default: every non-acoustic mode. On the bulk path this only controls which modes appear in the report/MODE_TABLE and get an eigendisplacement sidecar – the single equilibrium folder is written regardless. type: int

`--freq-min FREQ_MIN`
Skip modes below this frequency (THz). type: float

`--freq-max FREQ_MAX`
Skip modes above this frequency (THz). type: float

`--vacuum-gap VACUUM_GAP`
Minimum gap (Ang) along an axis to consider it vacuum-padded, same convention as `stb-ir` (default: 10.0). Used both for the 0D extra-trivial-mode detection AND to auto-select the bulk vs. non-bulk IR path.type: float

`--symprec SYMPREC`
Symmetry-detection tolerance (Ang) Phonopy uses internally when loading the force constants – drives both the reported point group and, when `--use-symmetry` is on, the IR-active/inactive classification and degenerate-mode grouping (default: 0.01, pymatgen's own default – matches the rest of the suite, and deliberately NOT Phonopy's own raw default of 1e-5, which is far too tight for a real DFT-relaxed structure and can misdetect the true point group: a mode that should be exactly degenerate can come out as two separate near-identical frequencies, and a symmetry-silent mode can be misclassified as IR-active). Loosen further (e.g. 0.02-0.05) for a structure relaxed with a looser force tolerance. type: float

`--rotational-mode-tol ROTATIONAL_MODE_TOL`
0D (molecule) only: a mode within the first 3 bands after the translations is treated as free rotation (trivial, excluded) if |frequency| is below this (THz) (default: 2.0). Lower this if your molecule has genuine low-frequency skeletal/torsional vibrational modes you want to make sure aren't mistaken for rotation. type: float

`--displacement DISPLACEMENT`
Finite-difference displacement (Ang) along each mode's eigendisplacement for the non-bulk dipole derivative, and the default amplitude for `--export-animations` (default: 0.02). Has no effect on the bulk path's folder (it's the equilibrium structure, never displaced). type: float

`--use-symmetry`
Skip modes that group theory guarantees are IR-INACTIVE (their irreducible representation at Gamma isn't

contained in the vector (x,y,z) representation – their dipole derivative is exactly zero, not just numerically small). Works for any of the 32 point groups. Only applies to the default mode selection (every non-acoustic mode) – an explicit `--modes` list is never second-guessed. On the bulk path this only reduces what's listed/sidecar'd, never the folder count (always 1). Needs a PRIMITIVE cell (see `stb-unitcell --mode primitive`); falls back to running every mode, with a warning, if that's not available. switch, off by default

`--skip-degenerate`

Non-bulk path only: for a degenerate group of modes (2 or more bands sharing one irrep), write dipole-displacement folders for only ONE representative band and skip the rest – the IR intensity |dmu/dQ|^2 is a rotational invariant, identical for every degenerate partner regardless of phonopy's arbitrary eigenvector choice within the subspace. `stb-irAnalysis` reuses the representative's value automatically. No effect on the bulk path (already exactly 1 folder regardless of mode count – nothing to save by skipping). Needs a PRIMITIVE cell, same as `--use-symmetry`. switch, off by default

`--export-animations`

Write a looping animation of each selected mode's eigendisplacement (a smooth 0/+A/0/-A/0 sweep) as a mode_XX_animation.axsf file – open in XCrySDen/VESTA to see which atoms move before interpreting the spectrum. Same amplitude as `--displacement`. switch, off by default

`--animation-frames ANIMATION_FRAMES`

Frames per mode animation, with `--export-animations` (default: 20). type: int

`--fc-displ BOHR`

Bulk path only: MD.FCDispl for SIESTA's own native Born-effective-charge finite-displacement automation, in Bohr (default: 0.02). Has no effect on the non-bulk path. type: float

`--polarization-grid N N N N N N N N N`

Bulk path only: the 9 integers (row-major 3x3, one row per lattice vector) for SIESTA's %block PolarizationGrids – diagonal entries are 1D line-integral k-point counts, off-diagonal entries are the 2D surface-integral mesh (default: 10 4 4 4 10 4 4 4 10). Manual, not auto-generated from a density heuristic – consult the SIESTA manual for your structure. Has no effect on the non-bulk path. type: int

Like every STB command it also accepts `-h`/`--help`, `-v`/`--version` and `--no-intro` (Section 3.3).

From the built-in help.

```
Usage example:
  stb-irModes --directory ir_study --calc calc.fdf
  stb-irModes --directory ir_study --calc calc.fdf --modes 1 2 3
```

11.12.3.3 `stb-irAnalysis`

Stage 3 of 3: analyzes a `stb-irModes` displacement sweep and computes the IR spectrum. Auto-detects, per mode, which path Stage 2 used (from the MODE_TABLE's own 'path' column, never inferred from which files happen to exist): - NONBULK: reads the total dipole moment (SIESTA's own "Electric dipole (a.u.)" line) from each mode's +/-delta dipole_disp/mode_*/ folder, central-differences to get dmu/dQ directly as a 3-vector. - BULK: reads the Born effective charge tensors (`SystemLabel.BC`) from the single shared born_charge_disp/equilibrium/ folder, combines with each mode's own eigendisplacement (a JSON sidecar Stage 2 already wrote) via dmu_beta/dQ = sum_atom sum_tau Z*[atom,tau,beta] * e[atom,tau]. - HYBRID (2D slab): both of the above at once – the in-plane dmu/dQ components come from the same Born-charge x eigendisplacement formula as BULK (computed with a vacuum-aware PolarizationGrids that only covers the two periodic axes), the vacuum-axis component comes from the same dipole-difference formula as NONBULK. A naive dipole is only valid along a non-periodic

direction, and Born-effective-charge/Berry-phase is only valid along a periodic one – neither method alone covers all 3 axes of a slab correctly, so this combines one component from each. Either way, IR intensity is |dmu/dQ|^2 – same formula, only the derivation differs. Builds a Lorentzian-summed spectrum and always lists its detected peaks; the gnuplot .dat/.gplot pair itself is opt-in (`--save-gnuplot`, under <directory>/plot/) and an on-screen matplotlib preview is available too (`--view`). Also always reloads Stage 1's force constants to write an extended-XYZ vibration animation per analyzed mode under <directory>/mode_animations/ (skipped gracefully if phonon_disp/ is gone), optionally opened one at a time in ASE's interactive viewer (`--view-modes`). Near-degenerate mode groups get a combined, basis-independent intensity alongside their individual (basis-dependent) values.

Synopsis.

```
usage: stb-irAnalysis [-h] [-dir DIRECTORY] [-l LABEL] [--file FILE]
                      [--linewidth LINEWIDTH] [--temperature K] [-o OUTPUT]
                      [--experimental FILE] [--peak-prominence FRACTION]
                      [--degenerate-tol THZ] [--save-report] [--save-gnuplot] [--view]
                      [--view-modes] [--animation-frames ANIMATION_FRAMES]
                      [--symprec SYMPREC] [-v] [--no-intro]
```

Options

`-dir, --directory DIRECTORY`
: Root directory written by `stb-ir/stb-irModes` (default: ir_study).

`-l, --label LABEL`
: SystemLabel used in Stage 2's `calc.fdf` (default: auto-detected).

`--file FILE`
: SIESTA output filename inside each displacement folder (default: `calc.out`).

`--linewidth LINEWIDTH`
: Lorentzian linewidth (cm^-1) for the summed spectrum (default: 10.0). type: float

`--temperature K`
: Weight each mode's spectrum contribution by the Stokes Bose-Einstein thermal population factor (n+1) at this temperature (Kelvin). Default: no weighting (every mode contributes by its raw intensity alone, as before this option existed). type: float

`-o, --output OUTPUT`
: Base filename (no extension) for the spectrum .dat/.gplot pair (default: ir_spectrum).

`--experimental FILE`
: Overlay a measured IR spectrum (2-column text file: frequency cm^-1, intensity) on the simulated one, and report a nearest-neighbor peak match (simulated vs. experimental peak positions, one row per experimental peak). Plotted on a separate y-axis (simulated intensity and real intensity aren't on the same scale).

`--peak-prominence FRACTION`
: Minimum peak prominence, as a fraction of that spectrum's own max intensity, for peak detection (both the standalone [3] SPECTRUM peak list and `--experimental`'s matching) (default: 0.01 – filters out sub-1% noise bumps). type: float

`--degenerate-tol THZ`
: Two modes within this frequency tolerance (THz) are reported as a degenerate group with a combined intensity, in addition to their individual values – |dmu/dQ|^2 is not itself basis-independent within a degenerate subspace (Phonopy's returned eigenvectors are an arbitrary orthogonal basis of it), but the GROUP's summed intensity is the physically meaningful, basis-independent quantity (default: 0.001 THz, comfortably above float round-trip noise through the MODE_TABLE's 6-decimal text format, comfortably below any real frequency splitting

between genuinely distinct modes). type: float

`--save-report`
Also persist the report to <directory>/ir_stage3.txt. Off by default. switch, off by default

`--save-gnuplot`
Also save the spectrum (and, with `--experimental`, the overlaid measured spectrum) as gnuplot .dat + .gplot scripts, under '<directory>/plot/'. Off by default. switch, off by default

`--view`
View the IR spectrum (and, with `--experimental`, the overlaid measured spectrum on a second y-axis) interactively via matplotlib now. Off by default. Needs a display. Independent of `--save-gnuplot` – matplotlib here is only an on-screen preview, never written to disk. switch, off by default

`--view-modes`
Open each analyzed mode's eigendisplacement animation interactively in ASE's 3D viewer, one at a time (close a window to advance to the next) – needs a display and Stage 1's 'phonon_disp/' folder to still be present (reloads the same force constants Stage 2/this stage's own always-on XYZ export already use). Off by default. switch, off by default

`--animation-frames ANIMATION_FRAMES`
Frames per mode animation, for both `--view-modes` and the always-on extended-XYZ export below (default: 20). type: int

`--symprec SYMPREC`
Symmetry-detection tolerance (Ang), only used when reloading the phonon force constants for mode animations (`--view-modes` and the always-on XYZ export) – same default as every other stage of this workflow (0.01, pymatgen's own default). Does not affect the IR-active/inactive classification (that was already decided in Stage 2) or the spectrum itself. type: float

Like every STB command it also accepts `-h`/`--help`, `-v`/`--version` and `--no-intro` (Section 3.3).

From the built-in help.

```
Usage examples:
  stb-irAnalysis --directory ir_study
  stb-irAnalysis --directory ir_study --save-gnuplot --view
  stb-irAnalysis --directory ir_study --view-modes --animation-frames 30
```

11.13 Generalized Quasi-Chemical Approximation (GQCA)

At a glance

Menu code: 5.13 (a multi-stage workflow)

Code	Stage	Command
5.13.1	Stage 1 - Structure Generation	`stb-gqca`
5.13.2	Stage 2 - Analysis	`stb-gqcaAnalysis`

Guided example: `examples/5.13-gqca/`

Summary: Self-contained: builds the 3 ordered pair-cluster-representative structures (AA/AB/BB) for a substitutional alloy A(1-x)Bx, then solves the mass-action equilibrium to get Delta-H_mix/Delta-S_mix/Delta-G_mix(x,T) and short-range-order parameters (Sher et al., PRB 36, 4279, 1987). Works on 2D and 3D structures alike.

11.13.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/5.13-gqca/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

This workflow has 2 stages: **Stage 1** (`stb-gqca`, code `5.13.1`) takes the structure of a host crystal and a `calc.fdf`, picks one sublattice, and writes **three** ready-to-relax SIESTA folders — the two pure end-members (AA, BB) and one ordered 50:50 compound (AB). **Stage 2** (`stb-gqcaAnalysis`, code `5.13.2`) reads the three finished relaxations back and, from just their three energies, reports the mixing enthalpy, entropy and free energy of the substitutional alloy **A(1–x)B(x) at every composition x and temperature T**, a short-range-order (SRO) parameter, an equivalent regular-solution interaction parameter, and a set of sanity checks, tables, plots and relaxed structures.

It answers a question none of `5.1–6.2` does: not how a *given* structure responds, but **how an alloy of two species will arrange itself** — does it prefer unlike neighbours (ordering), like neighbours (clustering/phase separation), or neither (an ideal solution) — at the cost of only three DFT relaxations, whatever the composition you eventually care about. The worked example (Section 5) runs the whole thing for real, with SIESTA, on two textbook III-V alloys that behave in opposite ways: Ga(1–x)In(x)As (size mismatched, tends to cluster) and Ga(1–x)Al(x)As (lattice matched, nearly ideal).

Both stages live in this one folder and this one tutorial — same reasoning as `5.1–5.5`: Stage 1's output exists only to feed Stage 2.

11.13.1.1 Theory

Symbols at a glance (all energies are per alloy-sublattice site unless stated):

Symbol	Meaning
A, B	the two alloy species on the chosen sublattice (`--sublattice`, `--species-b`)
`x`	alloy composition: fraction of the sublattice occupied by B
`N`	cluster size; here `N = 2` (nearest-neighbour pairs)
`j`, `n_j`	pair type: number of B atoms in the pair (0 = AA, 1 = AB, 2 = BB)
`g_j`	degeneracy of pair type `j`: `[1, 2, 1]`
`e_j`	energy per sublattice site of reference structure `j` (`cluster_n{j}`)
Δ`_j`	excess of `e_j` over the linear interpolation of the end-members, eq. (2)
`x_j`	fraction of pairs that are of type `j` (the pair populations)
η	Lagrange multiplier ("chemical activity" of B) fixing the composition, eq. (1)
Δ`H`, Δ`S`, Δ`G`	mixing enthalpy, entropy, free energy per site, eqs. (4)–(6)
`SRO`	short-range-order parameter, eq. (7): `> 0` clustering, `< 0` ordering
Ω, `T_c`	regular-solution interaction parameter and critical temperature, eqs. (9)–(10)

Why alloys are hard, and where GQCA sits. A substitutional alloy A(1–x)B(x) is a crystal whose lattice sites are fixed but whose *occupation* is not: each site of one sublattice holds an A or a B. With `N` sites there are `C(N, xN)` arrangements — astronomically many — and the real, finite-temperature crystal samples all of them with Boltzmann weights. Two exact-in-principle routes exist and both are expensive: compute the energy of an enormous number of supercells with DFT, or fit those energies to a **cluster expansion** [9] and run Monte Carlo on the fit. A cheaper route in the same spirit is a **special quasirandom structure** (SQS) [10]: a single, well-chosen supercell whose correlation functions mimic a random alloy at one composition, giving the *enthalpy of the random alloy* — but nothing about how the alloy actually orders or clusters.

The **quasi-chemical approximation** sits at the other end of the cost scale. Its lineage is old: Bethe [5] treated a binary alloy as a set of nearest-neighbour pairs; Fowler and Guggenheim [4, 3] turned

that into a mass-action statistical theory; Sher, van Schilfgaarde, Chen and Chen [1, 2] generalized it to clusters of any size and to *first-principles* cluster energies — hence **Generalized** Quasi-Chemical Approximation. That is what this workflow implements in its simplest form, the **pair cluster** (N = 2). Its appeal is economy: all the DFT you need is the energy of three small, ordered structures.

The physical picture: a gas of independent pairs. Look at the alloy not as a lattice of atoms but as a collection of nearest-neighbour **pairs** (bonds). Every pair is in one of three states, labelled by its number of B atoms `n_j`:

```
n_j = 0  (AA)        n_j = 1  (AB)        n_j = 2  (BB)
degeneracy g = [1, 2, 1]
```

(`g_j = C(N, j)`: an AB pair can be read in two orders, AA and BB in one [1].) The central approximation is that these pairs are **statistically independent "molecules"** whose populations `x_j` (fractions of pairs in each state, Σ`_j x_j = 1`) are set by thermal equilibrium, subject to one constraint — the alloy contains the right overall amount of B:

```
Σ_j (n_j / N) x_j = x                                                    (1)
```

The chemistry is exactly that of a reaction between molecules:

```
2 AB  <==>  AA + BB
```

If forming unlike (AB) pairs is energetically costly the equilibrium shifts right (AA and BB pairs dominate: **clustering**); if AB pairs are favourable it shifts left (**ordering**); if it is energetically free the populations are the purely statistical (binomial) ones — a **random** alloy.

The equations. (a) Excess energy of each pair. Let `e_j` be the energy of the reference structure that realizes pair type `j`, normalized per alloy-sublattice site (Stage 2 does this normalization — the three cells have different sizes). Subtract the linear interpolation between the two pure end-members:

```
Δ_j = e_j − (1 − n_j/N) e_0 − (n_j/N) e_N                                (2)
```

By construction Δ`_0 = `Δ`_N = 0`; the whole alloy is then controlled by one number for N = 2, Δ`_1` (Sher et al. [1]; restated as Eq. 4 of Ferreira et al. [33]). Note the interpolation is in the **local cluster composition** `n_j/N`, not in the target alloy composition `x`.

(b) Mass-action equilibrium. The thermal weight of a pair cluster is set by the energy of the **whole N-site cluster**, `N` Δ`_j` (the Δ`_j` of (2) is per site). Minimizing the free energy of the cluster gas subject to (1) with a Lagrange multiplier η (the "chemical activity" of B) gives the Boltzmann-like populations

```
x_j(η, T) = g_j η^{n_j} exp(−N Δ_j / k_B T) / Z,    Z = Σ_j g_j η^{n_j} exp(−N Δ_j / k_B T)   (3)
```

and η is fixed by inserting (3) into (1) (an independent open-source implementation of the same formalism is described in [34]). `stb-gqcaAnalysis` solves this root problem with Brent's method [30] in `ln` η (the root is unique because the mean composition is monotonic in η). When all Δ`_j = 0`, η `= x/(1−x)` and (3) collapses to the binomial distribution `x_j = C(N, j) x^j (1−x)^{N−j}` — the random-alloy reference used for SRO and as an analytic test of the code.

(c) Thermodynamic functions.

```
ΔH_mix = Σ_j x_j Δ_j                                  (per site)        (4)
ΔS_mix = −(k_B / N) Σ_j x_j ln( x_j / g_j )           (per site)        (5)
ΔG_mix = ΔH_mix − T ΔS_mix                            (per site)        (6)
```

(The sum in (5) is the entropy of the cluster gas **per cluster**; dividing by `N` makes it per site, like (4). For `Δ_j = 0` it reduces *exactly* to the ideal-solution entropy `−k_B [x ln x + (1−x) ln(1−x)]` per site — see Section 1.7-ii for why this matters.) The populations (3) are precisely the minimizer of `ΔH_mix − T ΔS_mix` under constraint (1); `Δ_1/k_B T` therefore enters as `N Δ_1 / k_B T`.

(d) Short-range order. The fraction of unlike pairs, `x_1`, compared with its random value `x_1^rand = 2x(1−x)` gives a Warren–Cowley-style parameter [7, 8]:

```
SRO = ( x_1^rand − x_1 ) / x_1^rand                                   (7)
```

`SRO = 0` is a random alloy; **`SRO > 0`: fewer unlike pairs than random (clustering)**; **`SRO < 0`: more (ordering)** — the sign convention of Cowley's α parameters. (The normalization is provisional, Section 1.7.)

Reading the equations. The equilibrium constant. From (3) with `Δ_0 = Δ_2 = 0`:

```
K(T) ≡ x_0 x_2 / x_1^2  =  (g_0 g_2 / g_1^2) exp(2NΔ_1 / k_B T)  =  ¼ exp(4Δ_1 / k_B T)      (8)
```

independent of `x` — the law of mass action for `2 AB ⇌ AA + BB`. At `Δ_1 = 0`, `K = ¼` (random). `Δ_1 > 0` raises K above ¼ (clustering); `Δ_1 < 0` lowers it (ordering). The only energy scale that matters is `N Δ_1 / k_B T` (cluster energy over `k_B T`), which is why the report prints that ratio at each temperature.

Limits. `T → ∞`: every `exp(−N Δ_j/k_B T) → 1`, the populations become binomial and `SRO → 0`: thermal disorder always wins eventually. `T → 0`: for `Δ_1 > 0` the AB fraction `x_1 → 0` (the alloy segregates into AA and BB domains); for `Δ_1 < 0` and `x = ½`, `x_1 → 1` (a perfectly ordered AB compound).

The regular-solution equivalent. In the random limit, `x_1 = 2x(1−x)` and (4) gives

```
ΔH_mix^rand(x) = 2 Δ_1 x (1−x)  ≡  Ω x (1−x),      Ω = 2 Δ_1                  (9)
```

i.e. the familiar *regular solution* [35] with interaction parameter `Ω = 2Δ_1` (`Ω > 0`: unlike atoms repel). A regular solution phase-separates below the critical temperature

```
T_c = Ω / (2 k_B) = Δ_1 / k_B                                         (10)
```

`stb-gqcaAnalysis` prints (9)–(10) as an *estimate* — a regular-solution statement derived from `Δ_1`, **not** a result of the GQCA free energy itself, which cannot show a gap (Section 1.7). Historically this is the language of III-V alloy miscibility work, where Stringfellow derived Ω for the ternary and quaternary III-V systems from a strain-energy model [14] (Ho and Stringfellow applied the same regular-solution reasoning to the nitride alloy GaInN [15]).

1.5 What `Δ_1` contains, and why the cells must really relax. `Δ_1` bundles two contributions. A **chemical** one (are Ga–As–In bonds intrinsically more or less favourable than Ga–As–Ga and In–As–In?) and an **elastic** one: A and B atoms want different bond lengths, and mixing them forces the crystal to strain. For isovalent semiconductor alloys the elastic part usually dominates and scales roughly with the *square* of the size mismatch, as in Keating's valence-force-field picture [12] and the bond-length

studies of Martins and Zunger [13]. This is why Ga(1−x)In(x)As (≈ 7 % mismatch) is expected to have a sizeable positive Δ_1 and Ga(1−x)Al(x)As (≈ 0.1 %) a tiny one — the two cases of Section 5.

The consequence for the calculation is non-negotiable: **each of the three reference structures must be relaxed at its own equilibrium lattice constant**. The three generally have different cells (GaAs 5.65 Å, InAs 6.06 Å, and something in between for the ordered compound). If you evaluated all three at one fixed lattice constant you would add a large, spurious strain energy to `e_0` or `e_2` and corrupt Δ_1. Stage 1 therefore forces `MD.TypeOfRun CG` and `MD.VariableCell T` for bulk inputs (Section 3.2). Vegard's law [16] — the lattice constant of an alloy interpolates linearly between its end-members — is the natural benchmark for the relaxed volumes, and Stage 2 reports the deviation of the AB cell from it.

Choosing the three structures. The end-members are trivial: every site of the chosen sublattice is set to A (`cluster_n0`) or to B (`cluster_n2`). The AB structure is the interesting one. A pair cluster with exactly one A and one B has the *only* interior composition a 2-site cluster has — 50 % B — so `cluster_n1` is always built at 50:50, **whatever alloy composition you ultimately care about**: the target `x` is swept entirely in Stage 2. The three DFT runs are the complete input to the whole `x–T` surface.

But 50:50 composition is not enough: a *random* 50:50 mixture has, on average, as many AA and BB bonds as AB ones. The structure must realize the **local** AB environment — every nearest-neighbour pair unlike. `stb-gqca` enumerates all symmetrically distinct ordered 50:50 arrangements at the smallest cell that reaches exactly 50 % B (Hart–Forcade derivative-structure enumeration [18] as implemented in icet [19]) and keeps the one whose nearest-neighbour **pair correlation** is closest to −1 (the value for "every pair unlike"; 0 for a random alloy at 50:50).

Whether −1 is reachable depends on the lattice, and this is a genuine piece of geometry, not a software limit:

Sublattice	Connectivity	Best AB pair correlation
simple cubic	bipartite (2-colourable: a checkerboard)	−1 exactly
FCC (zincblende/rocksalt cations), triangular, HCP	contains triangles (odd cycles)	> −1: **frustrated**

Triangles cannot be 2-coloured so that all three bonds are unlike. This is the same frustration that stops the antiferromagnetic Ising model on a triangular lattice from reaching perfect Néel order [17]. For GaAs's FCC cation sublattice the best 50:50 cell reaches a pair correlation of only −1/3 (deviation 0.667 from the ideal). The consequence is stated plainly in every report: `cluster_n1`'s environment is *not purely* AB, so Δ_1 is a less clean estimate than on a bipartite lattice — qualitatively meaningful, not a rigorous bound. `stb-gqca` also runs a bounded probe at twice the cell size and reports whether a larger cell would improve the ordering (`[NOTE] Delta_1 sensitivity check`).

2D inputs (a vacuum-padded monolayer such as MoS2) are detected automatically from the vacuum gap along one axis; the enumeration is restricted in-plane so the vacuum direction is never tiled. Cell relaxation is left **off** for 2D (positions only) — see Section 7 for why.

What the model assumes, and where it is not (yet) trustworthy. (i) The three energies are per sublattice site. Stage 2 divides each `FreeEng` by that folder's own count of sublattice (A + B) atoms, because the three cells differ in size. This is `e_j` in (2).

(ii) The entropy normalization — a bug found and fixed while writing this tutorial. Equation (5)'s sum is the Boltzmann entropy of the cluster gas per **cluster**. An earlier version of the solver used it as if it were per **site** while the energies (2)/(4) are per site, and used the per-site Δ_j as if it were the cluster energy in the weights (3). The giveaway: for a *random* alloy (all `Δ_j = 0`) the old `ΔS_mix` was exactly **2×**

the ideal-solution entropy per site (at `x = 0.5`: 0.1195 meV/K instead of 0.0597 meV/K), and the old results at temperature `T` were those of the correct model at `2T` — i.e. it understated clustering/ordering at any given `T`. The fix (weights with `N Δ_j`, entropy divided by `N`) is checked in the test-suite two ways: the random-alloy entropy equals the exact ideal value per site (`x = 0.25, 0.5, 0.75`), and the closed-form populations coincide with an independent numerical minimization of `ΔH − TΔS` per site. `Δ_j`, `Ω`, `T_c`, lattice constants and volumes were never affected. What remains `[UNVERIFIED]` is the *closed form beyond the random limit* (an independent-cluster entropy; no Guggenheim-type overlap correction, [3, 4]) — flagged in the code, the CLI help and every report — so `ΔS_mix`, `ΔG_mix` and the temperature dependence of `x_j`/SRO are semi-quantitative, while `Δ_j`, `Ω`, `T_c` and the geometry are direct.

(iii) The model cannot show a miscibility gap. With (4)–(5), the free energy as a function of composition is the minimum, over the pair populations `{x_j}` subject to the linear constraints `Σ x_j = 1` and `Σ (n_j/N) x_j = x`, of the function `Σ_j x_j Δ_j + (k_B T / N) Σ_j x_j ln(x_j/g_j)`. That function is jointly convex in `{x_j}`, and minimizing a jointly convex function over linear constraints yields a function that is convex in the constraint parameters (Boyd and Vandenberghe [36], §3.2.5). Hence `ΔG_mix(x)` is **provably convex for any `Δ_j`**: the common-tangent search in the report's `[7]` will essentially never find a gap, however repulsive the alloy. That is an intrinsic property of an independent-cluster model, not a physical claim that the alloy is miscible. The model captures *short-range order*, not genuine phase separation, which needs correlations between clusters (cluster-variation, cluster-expansion + Monte Carlo — Kikuchi [6], Sanchez, Ducastelle and Gratias [9]). The regular-solution `T_c` of (10) is offered precisely to fill that gap *as an estimate*.

(iv) SRO normalization. Equation (7) is a provisional Warren–Cowley-style definition; its sign convention is Cowley's [7] but its normalization has not been pinned to a primary source (`[UNVERIFIED]`).

(v) Frustrated lattices (Section 1.6) make `Δ_1` a less clean estimate.

(vi) Everything is equilibrium thermodynamics at 0 K energies. There are no vibrational contributions to `Δ_j`, and epitaxial or kinetically frozen samples need not follow the equilibrium populations.

What this workflow simplifies — explicitly.

- **Pair clusters only (N = 2).** Larger clusters (the tetrahedral cluster natural for zincblende, N = 4 [1, 2]) capture more of the local environment but need many more reference structures.
- **One reference structure for AB**, and a frustrated one on FCC/triangular lattices (Section 1.6).
- **No gap, no true phase diagram** (Section 1.7-iii).
- **Cluster energies come from a single relaxed ordered cell**, not from an average over disordered configurations — an SQS [10] or a cluster expansion [9] would give the *random-alloy* energy directly and is the natural cross-check.
- **Spin, magnetism and charged defects are not treated**; the template's own settings (e.g. `Spin`) are applied identically to all three folders.

A calculation you can check by hand. Take the InGaAs numbers of Section 5.2 — `Δ_1 = 58.60 meV` per site, composition `x = ½`, temperature `T = 600 K` — and reproduce one row of the report's table (Section 4.6) with a pocket calculator. `k_B T = 51.70 meV` and the cluster energy is `N Δ_1 = 117.20 meV`.

1. **Equilibrium constant**, eq. (8): `K = ¼ exp(2 N Δ_1 / k_B T) = ¼ exp(4.534) = ¼ × 93.1 = 23.27`. (Compare the random alloy, `K = ¼`: here AA + BB pairs are favoured ~93-fold over AB pairs.)
2. **Populations.** At `x = ½` the composition constraint (1) forces `x_0 = x_2 = (1 − x_1)/2`, so `K = x_0 x_2 / x_1² = [(1 − x_1)/(2 x_1)]²`. Hence `(1 − x_1)/(2 x_1) = √K = 4.824` and `x_1 = 1 / (1 + 2 × 4.824) = 0.0939`, `x_0 = x_2 = 0.4530`.
3. **Short-range order**, eq. (7): `x_1^rand = 2 × ½ × ½ = 0.5`, so `SRO = (0.5 − 0.0939) / 0.5 = +0.812`.

4. **Enthalpy**, eq. (4): ΔH = x_1 Δ_1 = 0.0939 × 58.60 = 5.50 meV/site (only the unlike pairs cost energy).
5. **Entropy**, eq. (5): Σ x_j ln(x_j/g_j) = 2 × 0.4530 ln 0.4530 + 0.0939 ln(0.0939/2) = −0.7174 − 0.2872 = −1.0046, so ΔS = (k_B/2) × 1.0046 = 0.0433 meV/K and $-T\Delta S$ = −25.97 meV.
6. **Free energy**, eq. (6): ΔG = 5.50 − 25.97 = −20.47 meV/site.

Every number above is the T = 600 row of the x = 0.500 table in Section 4.6 (x_1 = 0.0939, SRO = +0.8122, H_mix = +5.504, S_mix = 0.0433, -T*S = -25.973, G_mix = -20.469). Seeing the tool's output follow from three lines of algebra is the best way to trust it — and to understand what to expect from your own alloy: a large N Δ_1 / k_B T empties the AB column and drives the SRO toward +1; a small one leaves everything near the random values.

Physical reading. At 600 K only 9 % of the nearest-neighbour cation pairs are In–Ga although a random alloy would have 50 %: the lattice has *segregated locally*, forming In-rich and Ga-rich patches, because each unlike pair costs the elastic energy of squeezing a large In atom next to a small Ga atom. Heat reduces the penalty relative to k_B T, so the patches dissolve gradually (SRO 0.98 at 300 K, 0.42 at 1500 K) — but never completely, which is why the alloy has a finite critical temperature T_c below which a *macroscopic* miscibility gap opens (Section 1.4, equation (10)).

11.13.1.2 Libraries and external dependencies used

- **icet** [19] is the engine of Stage 1: it builds the cluster space of the chosen sublattice, enumerates every symmetrically distinct ordered 50:50 arrangement (Hart–Forcade derivative structures [18]) and evaluates each candidate's nearest-neighbour **cluster vector**, the pair correlation used to pick cluster_n1 (Section 1.6). It is an optional dependency (core.deps.require_icet): Stage 1 prints an install hint if it is missing; Stage 2 does not need it.
- **pymatgen** [26] and **ASE** [27] — structure conversion (.fdf ↔ pymatgen ↔ ASE), formula/lattice bookkeeping, AseAtomsAdaptor, CIF/XSF/XYZ/VASP writers for --export-relaxed, and the interactive 3D viewer behind --view.
- **spglib** [28] — symmetry analysis used internally by icet/pymatgen when reducing the host cell to its primitive cell.
- **sisl** [32] — reads SIESTA's .XV file (relaxed positions **and** cell) for --view / --export-relaxed.
- **numpy** / **SciPy** [29] — array maths; scipy.optimize.brentq solves the mass-action root η(x, T) (Section 1.3, [30]).
- **matplotlib** [31] — the 6-panel dashboard (--png / --show). Gnuplot (external, never invoked by the suite) is only used if you run the generated .gplot yourself.
- **SIESTA** [20, 21] itself — *not* called by the suite. You run it on the three folders; Stage 2 only parses its output (calc.out, .XV). The reference data of Section 5 was produced with SIESTA 5.4.2, GGA-PBE [22], PseudoDojo norm-conserving pseudopotentials [23], and Monkhorst–Pack k-grids [24].

11.13.1.3 3. Stage 1: generating the three cluster structures (stb-gqca, code 5.13.1)

What it does. Reads the host structure (-f, .fdf) and the calc.fdf template (-c, **default calc.fdf** in the current directory), and writes, under -O (default gqca_study):

```
gqca_study/
+- gqca_stage1.txt        <- the full Stage 1 report (also printed on screen)
+- cluster_n0/            <- AA: the whole sublattice set to species A
+- cluster_n1/            <- AB: the ordered 50:50 structure (Section 1.6)
+- cluster_n2/            <- BB: the whole sublattice set to species B
        structure.fdf  calc.fdf  config_extra.fdf  <pseudopotentials>
```

It never runs SIESTA. `-p/--pseudo-dir` takes a bundled bank (`dojo`) or a folder of your own `.psml/.psf` files and copies the right ones into each folder.

3.2 Forcing the relaxation: `config_extra.fdf`. Every folder gets a `config_extra.fdf` and a `calc.fdf` whose **first line** is `%include config_extra.fdf`; the rest of `calc.fdf` is your template, byte for byte. This is the suite's standard mechanism (same as `5.1`–`5.4`): SIESTA's fdf reader is *first-occurrence-wins* for a repeated label, so a directive in `config_extra.fdf` beats the same directive anywhere in your template — without editing the template. Live, for this tutorial's own InGaAs `cluster_n1`:

```
$ cat gqca_study/cluster_n1/config_extra.fdf
# Auto-generated by stb-gqca -- forces a real relaxation of this cluster.
# %included at the top of calc.fdf, so it wins over any same-name directive in the
# template (SIESTA's fdf reader is first-occurrence-wins). The template is untouched.
MD.TypeOfRun          CG
MD.NumCGsteps         150
MD.VariableCell       T
kgrid.MonkhorstPack   [8  8  6]
```

What is forced and why:

Directive	Value	Reason
`MD.TypeOfRun`	`CG`	a real relaxation, never a single point (Section 1.5)
`MD.NumCGsteps`	your template's value if `> 0`, else `200`	SIESTA's default is **0** (no relaxation): a template left at 0 would silently skip the relaxation
`MD.VariableCell`	`T` (3D input only)	each cluster relaxes to its own lattice constant; left unforced for a 2D input (Section 7)
`kgrid.MonkhorstPack`	recomputed **per folder** from `--kdensity`	the three cells differ in size and shape

A hostile template shows the mechanism at work — `kgrid [1 1 1]`, `MD.NumCGsteps 0`, `MD.TypeOfRun Broyden` all lose (Case 2 of the script):

```
Directive in config_extra.fdf | Value                | Template had | Effect
MD.TypeOfRun                  | CG                   | Broyden      | overrides
MD.NumCGsteps                 | 200                  | 0            | raised (a value of 0 / absent =
no relaxation; default 200)
MD.VariableCell               | T                    | (absent)     | added
kgrid.MonkhorstPack           | per folder (see [4]) | present      | overrides, density 0.2 1/Ang
```

(An earlier version of the tool *appended* the k-grid and run type to the end of `calc.fdf`; with first-occurrence-wins semantics a template that already had a `kgrid` kept its own grid. The `%include`-on-top mechanism fixes that.)

The report, live.

```
$ stb-gqca -f structure.fdf --sublattice Ga --species-b In -p dojo --no-intro

[1] INPUT STRUCTURE
Formula         : Ga4 As4  (8 atoms)
Lattice (Ang)   : a=5.6533  b=5.6533  c=5.6533
Cell volume     : 180.6783 Ang^3
Species | Z  | Atoms | Role
Ga      | 31 | 4     | alloy sublattice (A)
As      | 33 | 4     | host (kept fixed)
```

```
Detected (input) : 3D (bulk material)
Cell treatment   : MD.VariableCell T (full cell relaxes)

[2] PAIR ORBIT
Shortest sublattice-sublattice distance : 3.9975 Ang
Orbit radius    : 1.9987 Ang
Pair states      : n_j=0 (AA), n_j=1 (AB), n_j=2 (BB), A=Ga, B=In

Written structures:
Folder     | Pair       | Formula   | Atoms | A/B sites | x_B(sub) | a b c (Ang)       | V (Ang^3) |
V/site | k-grid
cluster_n0 | AA (Ga-Ga) | Ga4As4    | 8     | 4/0       | 0.00     | 5.653 5.653 5.653 | 180.68    |
45.17  | 6x6x6
cluster_n1 | AB (Ga-In) | In1Ga1As2 | 4     | 1/1       | 0.50     | 3.997 3.997 5.653 | 90.34     |
45.17  | 8x8x6
cluster_n2 | BB (In-In) | In4As4    | 8     | 0/4       | 1.00     | 5.653 5.653 5.653 | 180.68    |
45.17  | 6x6x6

AB ordering (cluster_n1):
  Cell size searched     : 2x primitive (primitive cell = 2 atom(s); smallest size reaching exactly
  50% B)
  Candidates evaluated   : 2 symmetrically distinct
  Pair correlation       : -0.3333  (ideal AB = -1.0000)
  Deviation from ideal   : 0.6667  (geometrically frustrated sublattice -- Delta_1 less clean)
```

How to read it. The 50:50 cell has **4 atoms** (1 In + 1 Ga + 2 As) — half the size of the 8-atom end-member cells, because it comes from icet's enumeration on the *primitive* cell; this is why Stage 2 normalizes per sublattice site. The two enumerated candidates are the layered (001) CuAu-I-type ordering and one other; the best has pair correlation −1/3 (each cation has 8 unlike and 4 like nearest cation neighbours), a deviation of 0.667 from the ideal −1, the frustration of Section 1.6. The k-grids (`6×6×6`, `8×8×6`) come from the same density (`--kdensity 0.2`) applied to different cells.

3.4 `--kdensity`: how dense a k-grid. The default `0.2` 1/Å is the suite's usual starting point (same convention as `stb-kgrid`; `stb-phononsCreate`, `stb-ir`, `stb-raman` and `stb-cohesive` use the same). The earlier default of `0.04` (28×28×28 for this 8-atom cell) was five times too dense. Case 3 of the script shows the three grids for 0.5, 0.2 and 0.1:

```
--kdensity 0.5   3x3x3 | 4x4x3 | 3x3x3
--kdensity 0.2   6x6x6 | 8x8x6 | 6x6x6        (default)
--kdensity 0.1   12x12x12 | 16x16x12 | 12x12x12
```

Whether `0.2` is dense enough for *your* mixing energy is a convergence question, answered in Section 5.4 with a real number.

Three lattices, three behaviours. Case 4 of the script runs Stage 1 on three fixtures and compares the `Pair correlation` line:

Fixture	Sublattice	AB pair correlation	Cell treatment
`cubic.fdf` (Na → K)	simple cubic, bipartite	**−1.0000** (ideal reached)	3D, full cell
`structure.fdf` (Ga → In)	FCC, frustrated	−0.3333	3D, full cell
`mos2.fdf` (Mo → W)	triangular, frustrated, 2D	−0.3333	**positions only**

For the 2D MoS2 monolayer a `[KNOWN LIMITATION]` note is printed: cell relaxation is left off, so the in-plane lattice mismatch between the two end-members is *not* captured (Section 7).

Validation. `stb-gqca` refuses, with a clear `[ERROR]` and exit status 1: a sublattice species not present in the structure, `--sublattice` equal to `--species-b`, a missing structure file, and a missing calc template (the `calc.fdf` default not being in the current directory — the message says so and points to `-c`).

Running it both ways. `stb-suite` → `5.13.1` asks the same questions as guided prompts; its calc.fdf prompt now accepts a blank answer for the default, and its k-density advanced prompt also defaults to `0.2`. Case 6 of the script drives it with piped answers and diffs the generated folders against the direct CLI run.

11.13.1.4 4. Stage 2: analyzing the results (`stb-gqcaAnalysis`, code `5.13.2`)

Before running Stage 2: run SIESTA in the three folders.

```
for c in n0 n2 n1; do (cd gqca_study/cluster_$c && mpirun -np 4 siesta calc.fdf > calc.out); done
```

Each is a short bulk relaxation (a few CG steps; minutes on a laptop for 8-atom cells). Stage 2 expects `calc.out` in each folder (`--file` changes the name) plus the `structure.fdf`/`calc.fdf`/`config_extra.fdf` Stage 1 wrote.

Report structure.

```
[0] RUN METADATA
[1] LEVEL OF THEORY               <- the 3 folders compared, setting by setting
[2] CLUSTER ENERGIES, CONVERGENCE & RELAXED GEOMETRY
[3] MIXING ENERGIES               <- Delta_j in eV/meV/kJ/mol/kcal/mol; Omega, T_c
[4] COMPOSITION / TEMPERATURE GRID  <- tables at --x-target compositions
[5] SANITY DIAGNOSTICS
[6] INTERPRETATION                <- plain-language reading of the numbers
[7] MISCIBILITY GAP (COMMON-TANGENT)
[8] SUMMARY & FILES
```

No arguments are required: `stb-gqcaAnalysis` alone reads `./gqca_study`, the `calc.out` files, and sweeps 300–1500 K in 5 points (`--temp` for one temperature, `--temp-min`/`--temp-max`/`--temp-points` for your own sweep). `--x-target 0.25 0.5 0.75` adds one detailed table per composition.

4.3 `[1]` — is the comparison legitimate?. Mixing energies only mean something if the three clusters were computed **identically**. Stage 2 reads each folder's `config_extra.fdf` + `calc.fdf` (first-occurrence-wins, exactly as SIESTA would) and tabulates the settings that matter:

```
Setting               | cluster_n0    | cluster_n1    | cluster_n2    | Consistency
XC functional         | GGA           | GGA           | GGA           |
XC authors            | PBE           | PBE           | PBE           |
Basis size            | DZP           | DZP           | DZP           |
Mesh cutoff           | 250 Ry        | 250 Ry        | 250 Ry        |
Variable cell         | T             | T             | T             |
k-grid                | 6x6x6         | 8x8x6         | 6x6x6         | differs (k-grid follows each
cell)
All 3 folders share the same level of theory (k-grid aside, which is density-based per cell).
```

If any other setting differs, that row turns yellow with `DIFFERS` and a `[WARNING]` follows (Case 10b of the script edits `Mesh.Cutoff` in one folder to show it). The k-grid is *expected* to differ.

4.4 `[2]` — converged? relaxed to what?. Live, on this tutorial's real InGaAs data:

```
Cluster    | Job completed | SCF converged | Geom. moves | Max force (eV/Ang) | Stress xx yy zz (kbar)
cluster_n0 | yes           | yes (7 it.)   | 4           | 0.0000             | +0.10 +0.10 +0.10
cluster_n1 | yes           | yes (10 it.)  | 10          | 0.0120             | +0.46 +0.63 +0.63
cluster_n2 | yes           | yes (8 it.)   | 10          | 0.0001             | -0.06 -0.06 -0.07

Cluster    | Input cell a b c  | Relaxed cell a b c | V/site relaxed | dV vs input
cluster_n0 | 5.653 5.653 5.653 | 5.791 5.791 5.791  | 48.557         | +7.50 %
cluster_n1 | 3.997 3.997 5.653 | 4.258 4.258 5.995  | 54.333         | +20.29 %
cluster_n2 | 5.653 5.653 5.653 | 6.231 6.231 6.231  | 60.485         | +33.91 %

Effective lattice mismatch  (V_BB/V_AA)^(1/3) - 1 : +7.60 %
AB volume vs Vegard (linear) average          : -0.34 %  (V_AB = 54.333 vs 54.521 Ang^3/site)
```

What to check: forces below the `MD.MaxForceTol` you asked for (here 0.02 eV/Å; a `[WARNING]` appears above `--force-tolerance`, default 0.05), residual stresses of a fraction of a kbar (the relaxation really reached the cell minimum), SCF confirmed. The **effective mismatch** (cube root of the volume-per-site ratio) is the number to compare with the experimental lattice-constant mismatch; the **AB volume versus Vegard** [16] tells you the alloy cell follows the linear law (–0.34 % is excellent).

4.5 `[3]` — the three numbers that carry the method.

```
j | Pair       | Delta_j (eV) | Delta_j (meV) | kJ/mol | kcal/mol
0 | AA (Ga-Ga) | +0.000000    | +0.00         | +0.00  | +0.00
1 | AB (Ga-In) | +0.058598    | +58.60        | +5.65  | +1.35
2 | BB (In-In) | +0.000000    | +0.00         | +0.00  | +0.00

Regular-solution equivalent (random-mixing limit of this model, H = Omega x (1-x)):
  Omega = 2*Delta_1            : +117.20 meV/site  (+2.70 kcal/mol, +11.31 kJ/mol)
  H_mix(x=0.5, random limit)   : +29.30 meV/site
  T_c = Omega / (2 kB)         : 680 K
```

Δ_0 and Δ_2 are zero by definition (they are the reference); everything rests on Δ_1. Ω and `T_c` are equations (9)–(10).

4.6 `[4]` — what happens along the temperature axis.

```
  x = 0.500   [random-alloy reference: x_0=0.250, x_1=0.500, x_2=0.250]
T (K) | x_0    | x_1    | x_2    | H_mix   | S_mix  | -T*S    | G_mix   | SRO     | N*Delta_1/kT
300   | 0.4947 | 0.0106 | 0.4947 | +0.623  | 0.0324 | -9.721  | -9.098  | +0.9787 | 4.53
600   | 0.4530 | 0.0939 | 0.4530 | +5.504  | 0.0433 | -25.973 | -20.469 | +0.8122 | 2.27
900   | 0.4096 | 0.1808 | 0.4096 | +10.593 | 0.0502 | -45.204 | -34.611 | +0.6384 | 1.51
1200  | 0.3782 | 0.2435 | 0.3782 | +14.271 | 0.0538 | -64.541 | -50.270 | +0.5129 | 1.13
1500  | 0.3562 | 0.2877 | 0.3562 | +16.858 | 0.0557 | -83.580 | -66.723 | +0.4246 | 0.91
```

At 300 K only **1 %** of the pairs are unlike (a random alloy would have 50 %): **SRO = +0.98**, essentially fully clustered; as T rises `x_1` climbs toward 0.5 and SRO falls toward 0. `H_mix` rises because more unlike (costly) pairs appear, and `S_mix` tends to the ideal value `k_B ln 2 = 0.0597 meV/K` at `x = ½` (0.0557 meV/K at 1500 K). The `S_mix`, `-T*S`, `G_mix` columns and the *speed* of the SRO decay depend on the independent-cluster entropy flagged in Section 1.7-ii (semi-quantitative); `H_mix` for a given set of populations, and all of `[3]`, do not.

4.7 [5]–[7] — sanity checks, interpretation, and the gap search. [5] checks the pure-A and pure-B limits, Σ x_j = 1, the composition mass balance of equation (1), Δ_0 = Δ_2 = 0, and that all clusters converged. [6] turns the signs and magnitudes into sentences (clustering vs ordering, N Δ_1/kT, SRO vs T, the regular-solution T_c). [7] is the common-tangent search, which always reports "fully miscible" by Section 1.7-iii.

The optional extras.

Flag	What you get
--png	<dir>/<output>_summary.png: the 6-panel dashboard (ΔH(x), ΔG(x), pair fractions vs random, SRO(x), pair excess energies, relaxed volume vs Vegard)
--show	the same dashboard in a window (needs a display; with none it says so and still saves with --png)
--plot-temp T	temperature of the single-T panels (nearest sampled T)
--view	each relaxed structure in ASE's 3D viewer, one after the other
--export-relaxed [cif|xsf|xyz|vasp]	the 3 relaxed structures (positions **and** cell, from .XV) in <dir>/relaxed/
--save-report	the text report as <dir>/gqca_stage2.txt
-o NAME	base name of the .csv/.dat/.gplot/.png outputs

Always written: gqca_results.csv (every (x, T) point), gqca_results_clusters.csv (one row per cluster), gqca_results.dat + .gplot (all temperatures, solid = ΔH, dashed = ΔG; run gnuplot gqca_results.gplot).

Guard rails. Case 10 of the script shows three realistic failures: results missing (clean [ERROR], exit 1), one folder computed with a different Mesh.Cutoff (DIFFERS row + [WARNING] in [1]), and a calc.out that never confirmed SCF convergence ([WARNING] in [2] and WARN in [5]). All advisory except the first: the numbers are still computed, but you are told not to trust them.

Running it both ways. stb-suite → 5.13.2 asks for the directory, output name, temperature mode (single / custom sweep / default sweep), target compositions and the optional extras above. Case 11 of the script drives it with piped answers and compares the resulting CSV with the CLI's, byte for byte.

11.13.1.5 Worked example: two real alloys, end-to-end, with the problems left in

This is **not illustrative data**. Everything below was produced by running this folder's own structure.fdf/calc.fdf through stb-gqca, SIESTA 5.4.2 and stb-gqcaAnalysis (Section 6). Two alloys of the GaAs host were studied, chosen because they sit at opposite ends of the size-mismatch scale — and the exercise also exposed one result that does **not** match expectation, which is left in deliberately, because finding out *why* is part of using this workflow.

Set-up.

Host / sublattice	GaAs zincblende, 8-atom cubic cell; alloy on the Ga (cation) sublattice
Alloys	Ga(1−x)In(x)As (--species-b In), Ga(1−x)Al(x)As (--species-b Al)
DFT	SIESTA 5.4.2, GGA-PBE [22], PseudoDojo [23] (Ga, In with semicore d), DZP basis (TZP for the convergence runs), Mesh.Cutoff 250 Ry, 25 meV electronic temperature, k-density 0.2 1/Å (6×6×6 / 8×8×6 grids)
Relaxation	MD.TypeOfRun CG, MD.VariableCell T, force tolerance 0.02 eV/Å, stress tolerance 0.1 GPa, forced by config_extra.fdf
Cost	3 relaxations per alloy; ~15 min per alloy (DZP) on 2 cores

Ga(1–x)In(x)As: a size-mismatched alloy (the clean case). The three relaxations all converged (SCF confirmed, residual force ≤ 0.012 eV/Å, residual stresses < 1 kbar). Relaxed lattice constants and the effective mismatch (Section 4.4 shows the full tables):

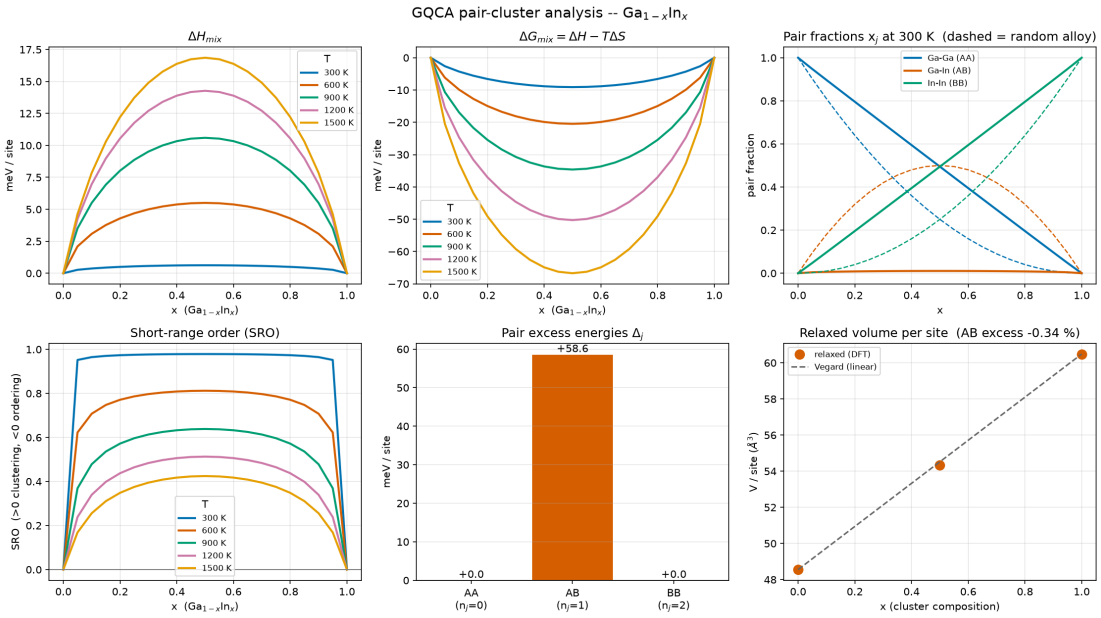
	DZP	TZP	experiment [25]
a(GaAs)	5.791 Å (+2.4 %)	5.739 Å (+1.5 %)	5.653 Å
a(InAs)	6.231 Å (+2.9 %)	6.192 Å (+2.2 %)	6.058 Å
mismatch (InAs/GaAs − 1)	**+7.6 %**	+7.9 %	+7.2 %
AB cell vs Vegard [16]	−0.34 %	—	—

The absolute lattice constants are overestimated, as expected for a GGA with a modest basis (a plane-wave PBE calculation gives 5.668 Å for GaAs [37]); the **mismatch** — the physically relevant quantity for the strain part of Δ_1 — is close to experiment, and the ordered alloy cell sits within 0.4 % of Vegard's law.

The mixing energies (DZP, from the Stage 2 report):

```
Delta_1 = +58.60 meV/site     (TZP: +51.7)
Omega   = 2*Delta_1 = 117.2 meV/site = 2.70 kcal/mol     (TZP: 103.3 meV)
T_c     = Omega/(2 kB) = 680 K                           (TZP: 600 K)
```

Δ_1 > 0: unlike In–Ga neighbours are *unfavourable* → a tendency to clustering. At x = 0.5 and 300 K only 1 % of the pairs are In–Ga (50 % in a random alloy), SRO = +0.98, relaxing toward random as the temperature rises (table of Section 4.6). The dashboard summarizes it:



InGaAs dashboard

Top row: ΔH_mix(x) for the five temperatures (always positive, symmetric because $\Delta_0 = \Delta_2 = 0$); ΔG_mix(x); the pair fractions at 300 K against the random reference (dashed): AA and BB pairs over-represented, AB suppressed. *Bottom row:* SRO vs x; the only non-zero excess energy Δ_1 (the bar); and the relaxed volume per site against Vegard's line (three DFT points on the line).

Ga(1–x)Al(x)As: the "ideal" alloy that is not — and what was tried. AlAs and GaAs are lattice matched to ~0.1 % experimentally [25], so there is almost no elastic energy to pay and Ga(1–x)Al(x)As is the textbook **near-ideal solid solution** (Chen and Sher [2]; Stringfellow [14]). The expectation is a Δ_1 of at most a few meV. The workflow gives something else:

Run	basis / mesh	a(GaAs)	a(AlAs)	mismatch	Δ_1 (meV/site)
`algaas`	DZP / 250 Ry	5.791	5.716	−1.29 %	**39.9**
`algaas_tzp`	TZP / 250 Ry	5.739	5.716	−0.40 %	**29.0**
`algaas_mesh450`	DZP / 450 Ry	5.787	5.714	−1.25 %	**43.0**

(experiment: AlAs is 0.14 % *larger* than GaAs.) Δ_1 is clearly **not** near zero, nor converged. The investigation, with what each test did to Δ_1 of AlGaAs unless stated:

Hypothesis	Test	Result
elastic (strain) energy	coherent-strain estimate, $\varepsilon \approx 0.65$ %, biaxial modulus $\approx 1.2\times10^{11}$ Pa, 48 Å^3/cation (an order-of-magnitude estimate, not a calculation)	$\lesssim$ 2 meV: **cannot explain** 40 meV
k-point sampling	InGaAs, density 0.15 → 0.2 (8×8×8 → 6×6×6): Δ_1 57.8 → 58.6 meV	**0.8 meV** (Section 5.4)
cell shape / grid mismatch between the 4-atom and 8-atom cells	GaAs and AlAs *inside the 4-atom* `cluster_n1` *cell* vs the 8-atom cell	per-site energies agree to **−3.0 / −2.5 meV**
integration mesh (egg-box)	`Mesh.Cutoff` 250 → 450 Ry	**+3.1 meV** (wrong direction)
basis incompleteness	DZP → TZP	**−10.9 meV** (and GaAs `a`: 5.791 → 5.739 Å)

So three suspects are excluded to within a few meV, and the basis is responsible for about a quarter of the effect. **About 29 meV/site remains unexplained** at TZP. It cannot be attributed from the data in hand; plausible (untested) candidates are the localized-basis description of the mixed Ga–As–Al environment with semicore-d pseudopotentials, or a genuine deficiency of this level of theory for the Ga–Al interaction. The recommended follow-up is an independent check of Δ_{1}`(AlGaAs)` with a plane-wave code, or an SQS [10] / cluster-expansion [9] estimate of the random-alloy enthalpy, before using the workflow's Δ_1 for AlGaAs. For InGaAs the TZP value changed Δ_1 by −7 meV (−12 %) — the qualitative conclusion (clustering, `T_c` of several hundred K) survives, the digits do not.

> **The lesson of this section.** A workflow can run flawlessly and print a self-consistent report and still hand you a number that a known physical fact contradicts. Δ_1 is a ~10^{-5} relative difference of three total energies: converge it with respect to **basis, mesh and k-points** (Cases 3 and 9 of the script), compare with a *known* system first (a nearly ideal alloy is a good null test), and trust only what survives.

5.4 k-point convergence of Δ_1. With the older default (density 0.15, grids 8×8×8 / 11×11×8) the InGaAs run gave Δ_1 `= 57.78 meV`; with the current default (0.2; 6×6×6 / 8×8×6) it gives `58.60 meV` — a **0.8 meV (1.4 %)** change for a ~40 % cheaper calculation. The default is adequate for a semiconductor; for a metal, tighten `--kdensity` and repeat. (The 0.15 run used the same settings but the earlier, append-to-`calc.fdf` mechanism of Stage 1.)

How far to trust each number.

Quantity	Reliability
relaxed lattice constants, mismatch, volumes, Vegard deviation, convergence status	direct DFT outputs; compare with experiment as in 5.2
Δ_1, Ω, T_c	quantitative **only after convergence** (basis/mesh/k, Section 5.3) and for a lattice where the AB structure is acceptable (Section 1.6); here ±10 meV set-up dependent
ΔS_mix, ΔG_mix, rate of SRO/x_j relaxation with T	**semi-quantitative**: consistent per-site normalization (Section 1.7-ii) but an independent-cluster closed form still flagged [UNVERIFIED]
miscibility gap from the common-tangent search	**not available** in this model (Section 1.7-iii); use T_c

Comparison with the literature. What can be said, and what cannot:

- **Lattice constants** are compared with experiment [25] in 5.2 and with a plane-wave PBE value for GaAs [37]; the basis-set overestimate (DZP +2.4 %, TZP +1.5 % vs +0.4 % for plane waves) is typical of SIESTA at this level.
- **Qualitative behaviour of InGaAs** — positive mixing enthalpy, tendency to clustering, a critical temperature of several hundred kelvin for the equivalent regular solution — is the picture of strain-driven III-V miscibility gaps developed by Stringfellow [14] and by first-principles work of the Zunger group [10, 11, 13]. A few kcal/mol is the order of magnitude of strain-energy estimates of Ω for III-V ternaries [14]; this workflow gives 2.4–2.7 kcal/mol.
- **No quantitative comparison is made**: a primary-source value of Ω or of ΔH_mix(x = ½) for Ga(1–x)In(x)As could not be retrieved while writing this tutorial, and none is quoted from memory. The numbers to compare with are Stringfellow's Ω for the Ga–In–As system [14] and SQS/cluster-expansion mixing enthalpies [10, 11, 38] — compare Ω x(1−x) of equation (9) with them.
- **AlGaAs** contradicts the established near-ideal behaviour ([2, 14]); this is reported as an open problem (Section 5.3), not as a result.

11.13.1.6 Reproducing the reference data

reference_results/ holds finished SIESTA runs, so Stage 2 works out of the box without a SIESTA installation. Each subfolder contains, per cluster, the exact structure.fdf, calc.fdf, config_extra.fdf, the full calc.out and the relaxed siesta.XV (pseudopotentials omitted to keep the repository small; they are regenerated by -p dojo).

Folder	Alloy	Basis	Notes
ingaas/	Ga(1–x)In(x)As	DZP	main worked example
algaas/	Ga(1–x)Al(x)As	DZP	the "ideal-solution" case that is not (Section 5.3)
ingaas_tzp/	Ga(1–x)In(x)As	TZP	basis convergence (Section 5.3)
algaas_tzp/	Ga(1–x)Al(x)As	TZP	basis convergence (Section 5.3)
algaas_ mesh450/	Ga(1–x)Al(x)As	DZP, Mesh.Cutoff 450 Ry	mesh convergence (Section 5.3)

To regenerate any of them yourself (this folder's structure.fdf and calc.fdf are the inputs; replace In by Al for the other alloy; edit PAO.BasisSize for TZP, Mesh.Cutoff for the mesh test):

```
stb-gqca -f structure.fdf --sublattice Ga --species-b In -p dojo -O ingaas
for c in n0 n2 n1; do
  (cd ingaas/cluster_$c && mpirun -np 2 siesta calc.fdf > calc.out 2> calc.err)
done
```

```
stb-gqcaAnalysis --directory ingaas --x-target 0.25 0.5 0.75 --png
```

Computed with SIESTA 5.4.2 (conda-forge build, GNU 14.3, MPI), `mpirun -np 2` on a 2-core machine: about 15 minutes per alloy with DZP; TZP and the 450 Ry mesh run took similar order of time. The numbers in this README were produced from exactly these files.

11.13.1.7 Known, deliberate limitations

- **The independent-cluster entropy is still `[UNVERIFIED]`** (Section 1.7-ii): per-site normalization is now consistent and exact in the random limit, but the closed form beyond it (no Guggenheim overlap correction) is not checked against the primary source; treat Δ`S_mix`, Δ`G_mix` and the temperature dependence of `x_j`/SRO as semi-quantitative.
- **No miscibility gap is possible in this model** (Section 1.7-iii); use the regular-solution `T_c` of equation (10) as an estimate, or a cluster expansion + Monte Carlo [9, 11, 38] / the cluster-variation method [6] for a real phase diagram.
- **One reference structure for AB, frustrated on FCC/triangular lattices** (Section 1.6). Under a simple pair-bond counting, the CuAu-I-type cell of GaAs's FCC cation sublattice has 8 unlike and 4 like neighbour bonds per cation (pair correlation −1/3), so Δ`_1` measures the energy of a structure that is only 2/3 "AB", not a pure AB pair; the report states the deviation but does not rescale Δ`_1`.
- **2D inputs: positions-only relaxation.** For a vacuum-padded cell, `MD.VariableCell` is not forced because the exact SIESTA syntax for freezing only the vacuum axis could not be confirmed. The in-plane lattice mismatch between the two end-members is therefore *not* captured for 2D alloys — an accuracy limitation, not a crash.
- **Basis set and pseudopotentials matter at the tens-of-meV level** (Section 5.3): Δ`_1` is a small difference of large energies. Always check convergence with respect to basis size and mesh cut-off before quoting it.
- **Zero-temperature electronic energies only.** No phonon (vibrational) free-energy contribution; equilibrium populations only (kinetically frozen or epitaxial samples need not follow them).
- **Spin-polarized and magnetic alloys** are not special-cased: whatever the template says is applied identically to the three folders.

11.13.1.8 Step-by-step: running this workflow on your own alloy

1. **Choose the host and the sublattice.** Prepare a *relaxed* host structure (`structure.fdf`) containing the sublattice whose sites will be randomly occupied by A (the species already there, `--sublattice`) and B (`--species-b`). Substitution on the anion sublattice works the same way.
2. **Write a production-quality `calc.fdf`** (functional, basis, mesh cut-off, SCF settings; a `MD.NumCGsteps` of 100–200 and the force/stress tolerances). Test basis and mesh convergence on the *end-member* first (`5.5-convergence`). Do not set the run type, `VariableCell` or the k-grid: Stage 1 forces them.
3. **Generate the three folders:** `stb-gqca -f structure.fdf --sublattice A --species-b B -p dojo`. Read the report: the pair-correlation line (is the AB ordering ideal?), the cluster table, and any `[NOTE]`/`[WARNING]`.
4. **Run SIESTA in `cluster_n0`, `cluster_n2`, `cluster_n1`** (Section 4.1).
5. **Inspect `[1]` and `[2]` of the Stage 2 report first:** same level of theory? SCF converged? forces/stresses small? A relaxed cell that follows Vegard?
6. **Read the physics:** Δ`_1` and its sign (clustering or ordering), the regular- solution Ω and `T_c`, and the tables at your target compositions (`--x-target`).

7. **Check convergence of Δ_1** against the k-density (`--kdensity`), the mesh cut-off and the basis (Sections 5.3–5.4) before drawing conclusions.
8. **Cross-check** against an SQS or cluster-expansion calculation [10, 9, 38] and against experiment when available (Section 5.6).

11.13.1.9 Files in this folder

File	What it is
`README.md`	this tutorial
`example_5.13.sh`	guided, runnable walk-through (12 cases)
`compare_runs.py`	helper of the script: side-by-side summary of several Stage 2 runs
`structure.fdf`	GaAs zincblende host (the example's main input)
`calc.fdf`	the SIESTA template used for all reference runs
`cubic.fdf`, `mos2.fdf`	extra Stage 1 fixtures (bipartite lattice; 2D monolayer), Section 3.5
`reference_results/`	real finished SIESTA runs (Section 6)
`figures/ingaas_dashboard.png`	the dashboard of Section 5.2 (`--png` output on `reference_results/ingaas`)
`output/`	created by the script (git-ignored)

11.13.1.10 Running the script

```
bash example_5.13.sh
```

It pauses between cases (`[Press Enter to continue]`); `yes '' | bash example_5.13.sh` runs it straight through. It starts by wiping its own `output/`. Cases 1–6 run Stage 1 (no SIESTA needed); Cases 7–12 run Stage 2 on `reference_results/`. It sets `MPLBACKEND=Agg`, so `--show`/`--view` are no-ops in the script; try them yourself on a desktop:

```
stb-gqcaAnalysis --directory reference_results/ingaas --show --view
```

11.13.1.11 Flag reference

`stb-gqca` — `-f FILE` host structure · `--sublattice SPECIES` · `--species-b SPECIES` · `-c CALC` (default `calc.fdf`) · `-p PSEUDO` (bank or folder) · `--kdensity D` (default 0.2 1/Å) · `--vacuum-gap Å` (2D detection, default 10) · `-O DIR` (default `gqca_study`) · `--no-intro` · `-v`.

`stb-gqcaAnalysis` — `--directory DIR` (default `gqca_study`) · `--file NAME` (default `calc.out`) · `--x-min`/`--x-max`/`--x-points` · `--x-target X [X ...]` · `--temp T` or `--temp-min`/`--temp-max`/`--temp-points` (default sweep 300–1500 K, 5 points) · `--force-tolerance eV/Å` · `-o NAME` · `--png` · `--show` · `--plot-temp T` · `--view` · `--export-relaxed [cif|xsf|xyz|vasp]` · `--save-report` · `--no-intro` · `-v`.

11.13.1.12 What's next

- Section 5's lesson applies to every energy-difference workflow in the suite: converge basis and mesh on the *difference*, not just the total energy (`5.5-convergence`).
- For a quantitative random-alloy enthalpy, build an SQS with `4.6-stb-sqs` and compare with $\Omega\,x(1-x)$ from this workflow.
- To follow a mixed system's lattice with composition, `5.15`'s equation-of-state workflow and `5.1`/`5.2` (strain, elastic constants) apply to any alloy structure you generate.

11.13.1.13 References

1. A. Sher, M. van Schilfgaarde, A.-B. Chen, W. Chen, *Quasichemical approximation in binary alloys*, Phys. Rev. B **36**, 4279 (1987). doi:10.1103/PhysRevB.36.4279
2. A.-B. Chen, A. Sher, *Semiconductor Alloys: Physics and Materials Engineering*, Plenum Press, New York (1995).
3. E. A. Guggenheim, *Mixtures*, Oxford University Press (1952).
4. R. H. Fowler, E. A. Guggenheim, *Statistical Thermodynamics*, Cambridge University Press (1939).
5. H. A. Bethe, *Statistical theory of superlattices*, Proc. R. Soc. Lond. A **150**, 552 (1935).
6. R. Kikuchi, *A theory of cooperative phenomena*, Phys. Rev. **81**, 988 (1951).
7. J. M. Cowley, *An approximate theory of order in alloys*, Phys. Rev. **77**, 669 (1950).
8. B. E. Warren, *X-ray Diffraction*, Addison-Wesley (1969).
9. J. M. Sanchez, F. Ducastelle, D. Gratias, *Generalized cluster description of multicomponent systems*, Physica A **128**, 334 (1984).
10. A. Zunger, S.-H. Wei, L. G. Ferreira, J. E. Bernard, *Special quasirandom structures*, Phys. Rev. Lett. **65**, 353 (1990).
11. L. G. Ferreira, S.-H. Wei, A. Zunger, *First-principles calculation of alloy phase diagrams: The renormalized-interaction approach*, Phys. Rev. B **40**, 3197 (1989).
12. P. N. Keating, *Effect of invariance requirements on the elastic strain energy of crystals with application to the diamond structure*, Phys. Rev. **145**, 637 (1966).
13. J. L. Martins, A. Zunger, *Bond lengths around isovalent impurities and in semiconductor solid solutions*, Phys. Rev. B **30**, 6217 (1984).
14. G. B. Stringfellow, *Calculation of ternary phase diagrams of III-V systems*, J. Phys. Chem. Solids **33**, 665 (1972); *Miscibility gaps in quaternary III/V alloys*, J. Cryst. Growth **27**, 21 (1974).
15. I.-H. Ho, G. B. Stringfellow, *Solid phase immiscibility in GaInN*, Appl. Phys. Lett. **69**, 2701 (1996). doi:10.1063/1.117683
16. L. Vegard, *Die Konstitution der Mischkristalle und die Raumfüllung der Atome*, Z. Phys. **5**, 17 (1921).
17. G. H. Wannier, *Antiferromagnetism. The triangular Ising net*, Phys. Rev. **79**, 357 (1950).
18. G. L. W. Hart, R. W. Forcade, *Algorithm for generating derivative structures*, Phys. Rev. B **77**, 224115 (2008).
19. M. Ångqvist *et al.*, *ICET – a Python library for constructing and sampling alloy cluster expansions*, Adv. Theory Simul. **2**, 1900015 (2019).
20. J. M. Soler *et al.*, *The SIESTA method for ab initio order-N materials simulation*, J. Phys.: Condens. Matter **14**, 2745 (2002).
21. A. García *et al.*, *Siesta: Recent developments and applications*, J. Chem. Phys. **152**, 204108 (2020).
22. J. P. Perdew, K. Burke, M. Ernzerhof, *Generalized gradient approximation made simple*, Phys. Rev. Lett. **77**, 3865 (1996).
23. M. J. van Setten *et al.*, *The PseudoDojo: Training and grading a 85 element optimized norm-conserving pseudopotential table*, Comput. Phys. Commun. **226**, 39 (2018). doi:10.1016/j.cpc.2018.01.012
24. H. J. Monkhorst, J. D. Pack, *Special points for Brillouin-zone integrations*, Phys. Rev. B **13**, 5188 (1976).
25. I. Vurgaftman, J. R. Meyer, L. R. Ram-Mohan, *Band parameters for III–V compound semiconductors and their alloys*, J. Appl. Phys. **89**, 5815 (2001).
26. S. P. Ong *et al.*, *Python Materials Genomics (pymatgen)*, Comput. Mater. Sci. **68**, 314 (2013).
27. A. H. Larsen *et al.*, *The atomic simulation environment*, J. Phys.: Condens. Matter **29**, 273002 (2017).
28. A. Togo, I. Tanaka, *Spglib: a software library for crystal symmetry search*, arXiv:1808.01590 (2018).
29. P. Virtanen *et al.*, *SciPy 1.0*, Nat. Methods **17**, 261 (2020).
30. R. P. Brent, *Algorithms for Minimization without Derivatives*, Prentice-Hall (1973).
31. J. D. Hunter, *Matplotlib: A 2D graphics environment*, Comput. Sci. Eng. **9**, 90 (2007).
32. sisl — `https://zerothi.github.io/sisl/` (N. Papior).
33. Ferreira *et al.*, extended GQCA (EGQCA) restatement of the Sher *et al.* formalism, Mater. Today

Phys. (2024), as cited in `core/gqca_solver.py` (Eqs. 3, 4, 13–15 of that paper are the ones the solver was checked against).
34. Rao *et al.*, Acta Mater. (2022), and the open-source reference implementation WMD-group/GQCA_alloys, as cited in `core/gqca_solver.py`.
35. D. R. Gaskell, *Introduction to the Thermodynamics of Materials*, Taylor & Francis (regular solutions).
36. S. Boyd, L. Vandenberghe, *Convex Optimization*, Cambridge University Press (2004), §3.2.5.
37. A PAW study comparing the LDA, PBE and PBEsol lattice constants of GaAs (PBE 5.668 Å vs. experiment 5.648 Å at 0 K) — `https://www.nanotechnology.hu/reprint/MatSciEngB_91_83.pdf` (found in a web search; full bibliographic details not verified).
38. A. van de Walle, M. Asta, G. Ceder, *The Alloy Theoretic Automated Toolkit*, Calphad **26**, 539 (2002).

11.13.2 Implementation details

The generalized quasi-chemical approximation

The alloy $A_{1-x}B_x$ (on one sublattice) is treated as a mixture of independent clusters of N sites, each with $j = 0, \dots, N$ atoms B [83, 84]. Here $N = 2$ (pair clusters AA, AB, BB) with degeneracies $g_j = \binom{N}{j} = (1, 2, 1)$. With e_j the energy per sublattice site of a structure representing cluster type j, the mixing energies are

$$\Delta_j = e_j - \Big(1 - \frac{j}{N}\Big) e_0 - \frac{j}{N} e_N,$$

and the cluster fractions follow the mass-action law

$$x_j(\eta, T) = \frac{g_j\, \eta^j\, e^{-N\Delta_j / k_B T}}{\sum_k g_k\, \eta^k\, e^{-N\Delta_k / k_B T}}, \qquad \sum_j \frac{j}{N}\, x_j = x,$$

where η is fixed by the composition constraint (solved numerically). Per sublattice site,

$$\Delta H_{\mathrm{mix}} = \sum_j x_j \Delta_j, \qquad \Delta S_{\mathrm{mix}} = -\frac{k_B}{N} \sum_j x_j \ln \frac{x_j}{g_j}, \qquad \Delta G_{\mathrm{mix}} = \Delta H_{\mathrm{mix}} - T\, \Delta S_{\mathrm{mix}}.$$

With all $\Delta_j = 0$ the entropy reduces to the ideal $-k_B[x \ln x + (1-x)\ln(1-x)]$; the closed form beyond that limit is marked `[UNVERIFIED]` in the code. The formalism follows the restatement of Ferreira *et al.* [85].

Stage 1 (`stb-gqca`). From the parent structure (`-f`), the sublattice element (`-sublattice`) and the substituting element (`-species-b`), icet's Hart–Forcade enumeration [13, 61] supplies the reference structures: pure A (AA), pure B (BB), and the 50:50 cell whose nearest-neighbour pair correlation is closest to -1 (AB, every bond mixed). On frustrated sublattices (fcc, triangular) -1 is unreachable; the reached value is reported. Each structure is written to `<output-dir>/` (default `gqca_study`) with your template (`-c`, default `calc.fdf`) and a prepended `config_extra.fdf` forcing a CG relaxation (`MD.TypeOfRun CG`, a positive `MD.NumCGsteps`, `MD.VariableCell T` for 3D cells) and a k-grid matched to each cell at `-kdensity` (default 0.2 Å^{-1}). Works for 2D and 3D structures.

Stage 2 (`stb-gqcaAnalysis`). It reads the free energies (`-file`), divides them by the number of sublattice sites of each cell, checks convergence, forces (`-force-tolerance`) and that all folders share the level of theory and the k-sampling, and computes Δ_1 (with $\Delta_0 = \Delta_2 = 0$ by construction). Results:

- the equivalent regular solution $\Omega = 2\Delta_1$ (random limit $\Delta H = \Omega x(1-x)$) and, for $\Omega > 0$, the critical temperature $T_c = \Omega / 2k_B$ of the miscibility gap;

- x_j, ΔH, ΔS, ΔG on a composition grid (`-x-min`, `-x-max`, `-x-points`) at the temperatures `-temp` or `-temp-min`/`-temp-max`/`-temp-points`, tables at `-x-target`;
- miscibility gaps from the common-tangent (lower convex hull) construction on $\Delta G(x)$ at each T, and a short-range-order indicator (deviation of the mixed-cluster fraction from random; provisional, marked `[UNVERIFIED]`);
- a dashboard figure (`-show`, `-png`), data files (`-o`), the relaxed structures (`-export-relaxed`).

Scope.. A pair-cluster model describes nearest-neighbour chemistry and ignores strain coupling between clusters; it is a good first description of mixing thermodynamics, not a replacement for a cluster expansion with Monte Carlo. The example of the repository uses real SIESTA results (InGaAs, AlGaAs) and documents an unresolved anomaly in AlGaAs's Δ_1; do not take those AlGaAs numbers as validated.

11.13.3 Command-line reference

11.13.3.1 `stb-gqca`

Stage 1 of 2: generates the 3 ordered pair-cluster-representative structures (AA/AB/BB) for a substitutional alloy A(1-x)Bx. Implements the pair-cluster (N=2) special case of the Generalized Quasi-Chemical Approximation (GQCA, Sher, van Schilfgaarde, Chen, Chen, Phys. Rev. B 36, 4279, 1987) – the classic Guggenheim quasi-chemical approximation, applicable to ANY lattice/dimensionality (2D or 3D alike; verified on both a 3D zincblende cell and a 2D transition-metal-dichalcogenide cell during this tool's development). Disorders one existing sublattice (`--sublattice`) with a second species (`--species-b`, typically absent from the input structure) and writes 3 folders: the two pure end-members (cluster_n0/, cluster_n2/) and the ordered 50:50 compound that most purely realizes an "every neighbor pair is unlike" (AB) local environment along the shortest sublattice-sublattice bond (cluster_n1/) – NOT a random 50:50 mixture, whose bonds would average to a mix of AA/AB/BB. All 3 folders are written for a REAL relaxation (a `config_extra.fdf` %included on top of `calc.fdf` forces MD.TypeOfRun CG, MD.NumCGsteps > 0, the folder's own k-grid, and MD.VariableCell T too for a 3D/bulk input; your `--calc` template itself is never edited) – the 3 representative structures generally have different equilibrium lattice constants (atomic size mismatch between A and B), and single-point evaluation at a fixed lattice constant would contaminate the mixing energy with spurious strain. [KNOWN LIMITATION] For a 2D (vacuum-padded) input, cell relaxation is left OFF entirely (positions-only, at the input's own in-plane lattice constant) rather than risk an unconfirmed SIESTA cell-constraint syntax for excluding the vacuum axis – see build_config_extra's docstring. Doesn't run SIESTA – run each folder yourself, then use `stb-gqcaAnalysis`. [WHY ONLY 3 STRUCTURES / ONLY EVER 50:50] The pair cluster (N=2) has exactly 3 possible local compositions (0, 1, or 2 B atoms per pair) – there is no "user-chosen x" knob here, and cluster_n1/ is always built at the cluster-internal 50:50 composition (the ONLY interior point a 2-site cluster has), regardless of the alloy composition you actually care about. Your real target composition x (e.g. Mo0.7W0.3S2) is swept entirely in Stage 2 (`stb-gqcaAnalysis --x-min/--x-max`), which combines these 3 fixed reference energies via the mass-action law to get x_j(x,T) at ANY x in [0,1] – these 3 DFT calculations are the complete, sufficient input for the whole x/T thermodynamic surface, not a coarse sample of it.

Synopsis.

```
usage: stb-gqca [-h] -f FILE --sublattice SUBLATTICE --species-b SPECIES_B [-c CALC]
                [-p PSEUDO_DIR] [--kdensity KDENSITY] [--vacuum-gap VACUUM_GAP]
                [-O OUTPUT_DIR] [-v] [--no-intro]
```

Options

`-f, --file FILE`
Input `structure.fdf` (2D or 3D). required

`--sublattice SUBLATTICE`
Existing species whose sites become the disordered alloy sublattice. required

`--species-b SPECIES_B`
Second alloy species (typically absent from the input structure). required

`-c, --calc CALC`
`calc.fdf` template for the 3 cluster relaxations (default: `calc.fdf`).

`-p, --pseudo-dir PSEUDO_DIR`
Pseudopotentials source: a bundled bank or a folder path.

`--kdensity KDENSITY`
k-point density (1/Ang) for the per-folder Monkhorst-Pack grid, same convention as `stb-kgrid` and the other workflows (default: 0.2). type: float

`--vacuum-gap VACUUM_GAP`
Vacuum-axis detection threshold in Ang (default: 10.0). type: float

`-O, --output-dir OUTPUT_DIR`
Root directory for the whole GQCA workflow (default: gqca_study).

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Usage example:
  stb-gqca -f mos2.fdf --sublattice Mo --species-b W -p dojo        # uses ./calc.fdf
  stb-gqca -f mos2.fdf --sublattice Mo --species-b W -c my_calc.fdf -p dojo
```

11.13.3.2 `stb-gqcaAnalysis`

Stage 2 of 2: solves the GQCA pair-cluster mass-action equilibrium and reports alloy mixing thermodynamics. Reads the 3 relaxed cluster_n{0,1,2}/ folders written by `stb-gqca` (AA/AB/BB pair-cluster representatives; SIESTA output read from <folder>/calc.out by default, level of theory read back from each folder's own `calc.fdf` + `config_extra.fdf`), normalizes each to a per-sublattice-site energy (the 3 folders generally have different cell sizes – see read_cluster_energy's own docstring), and solves the mass-action equilibrium (Sher, van Schilfgaarde, Chen, Chen, Phys. Rev. B 36, 4279, 1987) over a composition x / temperature T grid to report Delta-H_mix, Delta-S_mix, Delta-G_mix, and a short-range-order (SRO) parameter. The report covers: level-of-theory consistency across the 3 folders, per-folder convergence and relaxed geometry (cell, volume per site, residual force/stress), pair excess energies in several units, an equivalent regular-solution estimate (Omega, T_c), tables at chosen target compositions, sanity checks, and a plain-language interpretation. Optional on-screen/PNG dashboard (`--show`/`--png`), interactive 3D viewer of the 3 relaxed structures (`--view`) and CIF/XSF/... export (`--export-relaxed`). [UNVERIFIED] The closed form of the mixing entropy (-(kB/N) * Sum_j x_j*ln(x_j/g_j) per site, the standard "gas of independent clusters" form – exact ideal-solution entropy in the random limit) and the SRO-parameter normalization could not be pinned to a fully confirmed closed form from the primary literature during this tool's development – see core/gqca_solver.py's own docstrings (which also document the per-site vs per-cluster normalization). The mixing-enthalpy and mass-action-law equations ARE confirmed against a secondary source that restates the original 1987 formalism. Every report/CSV

column derived from an [UNVERIFIED] quantity is documented as such below. Also runs a discrete-grid common-tangent (miscibility gap) search on Delta-G_mix(x) at every sampled T – kept for completeness, but [STRUCTURAL LIMITATION] Delta-G_mix(x) from this independent-cluster formalism is PROVABLY CONVEX for any Delta_j (a convex-analysis theorem, not a numerical coincidence), so a gap is essentially never found here regardless of the input energies – this model captures short-range order, not genuine phase separation. See core/gqca_solver.py::common_tangent_gaps' own docstring for the full argument.

Synopsis.

```
usage: stb-gqcaAnalysis [-h] [--directory DIRECTORY] [--file FILE] [--x-min X_MIN]
                        [--x-max X_MAX] [--x-points X_POINTS]
                        [--x-target X_TARGET [X_TARGET ...]] [--temp TEMP]
                        [--temp-min TEMP_MIN] [--temp-max TEMP_MAX]
                        [--temp-points TEMP_POINTS]
                        [--force-tolerance FORCE_TOLERANCE] [-o OUTPUT] [--show]
                        [--png] [--plot-temp PLOT_TEMP] [--view]
                        [--export-relaxed [FORMAT]] [--save-report] [-v] [--no-intro]
```

Options

`--directory DIRECTORY`
: Root directory written by `stb-gqca` (default: gqca_study).

`--file FILE`
: SIESTA output filename inside each cluster_n* folder (default: `calc.out`).

`--x-min X_MIN`
: Composition grid minimum (default: 0.0). type: float

`--x-max X_MAX`
: Composition grid maximum (default: 1.0). type: float

`--x-points X_POINTS`
: Number of composition grid points (default: 21). type: int

`--x-target X_TARGET [X_TARGET ...]`
: Composition(s) x for the detailed per-temperature tables (default: 0.5). type: float

`--temp TEMP`
: Single fixed temperature in K. Mutually exclusive with `--temp-min/--temp-max/--temp-points`. type: float

`--temp-min TEMP_MIN`
: Temperature sweep minimum (K). Without any temperature option the sweep 300..1500 K is used. type: float

`--temp-max TEMP_MAX`
: Temperature sweep maximum (K). type: float

`--temp-points TEMP_POINTS`
: Number of temperature sweep points (default: 5). type: int

`--force-tolerance FORCE_TOLERANCE`
: Residual atomic force in eV/Ang (default: 0.05) above which a cluster folder's `calc.out` is flagged as possibly not relaxed. Advisory only, never blocks the result. type: float

`-o, --output OUTPUT`
: Base filename (no extension) for the .csv/.dat/.gplot/.png outputs (default: gqca_results).

`--show`
: Open the analysis dashboard (matplotlib) on screen. Needs a display. switch, off by default

`--png`
Save the dashboard as <directory>/<output>_summary.png. switch, off by default

`--plot-temp PLOT_TEMP`
Temperature (K) used by the single-T dashboard panels (default: the lowest sampled T; the nearest sampled T is used). type: float

`--view`
Open each relaxed cluster structure in ASE's interactive 3D viewer, one after the other (needs a display). switch, off by default

`--export-relaxed [FORMAT]`
Write the 3 relaxed structures (positions + relaxed cell) to <directory>/relaxed/ as cif (default), xsf, xyz or vasp. choices: cif, xsf, xyz, vasp

`--save-report`
Also persist the report to <directory>/gqca_stage2.txt. Off by default. switch, off by default

Like every STB command it also accepts `-h`/`--help`, `-v`/`--version` and `--no-intro` (Section 3.3).

From the built-in help.

```
Usage examples:
  stb-gqcaAnalysis                                  # defaults: ./gqca_study, T = 300..1500 K (5 points)
  stb-gqcaAnalysis --directory gqca_study --temp 800
  stb-gqcaAnalysis --temp-min 300 --temp-max 1500 --temp-points 5 --x-target 0.25 0.5 0.75
  stb-gqcaAnalysis --show --view                    # dashboard + 3D viewer on screen
  stb-gqcaAnalysis --png --export-relaxed cif       # figure + relaxed structures to files
```

11.14 Optical Properties

At a glance

Menu code: 5.14 (a multi-stage workflow)

Code	Stage	Command
5.14.1	Stage 1 - Direction Folders	`stb-optical`
5.14.2	Stage 2 - Analysis	`stb-opticalAnalysis`

Guided example: `examples/5.14-optical/`

Summary: Self-contained: writes one SIESTA folder per Cartesian polarization direction for a linear-response Optical.Calculate run on an already-relaxed structure, then derives eps1(E)/eps2(E), n(E)/k(E), the absorption coefficient, normal-incidence reflectivity, the energy-loss function, and the real optical conductivity from the resulting `SystemLabel.EPSIMG` file(s).

11.14.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/5.14-optical/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

This workflow has 2 stages: **Stage 1** (`stb-optical`, code `5.14.1`) takes an **already relaxed** structure and a `calc.fdf` template and writes one ready-to-run SIESTA folder per requested polarization direction (`xx`, `yy`, `zz`, and the biaxial `xy`, `xz`, `yz`), each asking SIESTA for its linear-response optical module. **Stage 2** (`stb-opticalAnalysis`, code `5.14.2`) reads the finished runs back and turns SIESTA's imaginary dielectric

function into the full set of linear optical properties — real part by Kramers–Kronig, refractive index and extinction coefficient, absorption coefficient, reflectivity, energy-loss function, optical conductivity — plus the quantities that tell you whether to trust them: sum rules, absorption edge, plasmon, anisotropy checked against the crystal symmetry, a what-if scissor shift, and a 9-panel dashboard.

It answers a question none of `5.1–5.13` does: not how a structure is deformed, bound, or ordered, but **how it responds to light** — what it absorbs, reflects, and at which photon energies. The worked example (Section 5) runs the whole thing for real, with SIESTA, on bulk silicon (where the textbook answer is known) and on a graphene monolayer (to show what changes in two dimensions, and in a semimetal), including the convergence study that decides whether a peak position can be believed.

Both stages live in this one folder and this one tutorial — same reasoning as `5.1–5.5` and `5.13`: Stage 1's output exists only to feed Stage 2.

11.14.1.1 Theory

Symbols at a glance

Symbol	Meaning
$\hbar\omega$, `E`	photon energy (eV)
ε`(E) =` ε_1 `+ i`ε_2	complex relative dielectric function (a 3×3 tensor in general)
`N = n + ik`	complex refractive index, `N`^{2} `=` ε
α	absorption coefficient (cm^{-1})
`R`	normal-incidence reflectivity
`L = −Im(1/`ε`)`	energy-loss function
σ_1	real part of the optical conductivity (S/m)
Ω	cell volume
$\hbar\omega$`_p`	plasma energy of the valence electrons
`N_eff(E)`	number of electrons accounted for by transitions up to `E` (f-sum rule)
`e`	unit polarization vector of the light (`Optical.Vector`)

What light does to a solid, and what is measured. In the linear regime a solid answers an oscillating electric field with an induced polarization proportional to it. The frequency-dependent proportionality is the **complex dielectric function**

$$\varepsilon(E) = \varepsilon_1(E) + i\,\varepsilon_2(E) \tag{1}$$

ε_2 measures **absorption** (energy taken from the light by exciting electrons) and ε_1 measures **dispersion** (how much the light is slowed and polarizes the medium). The two are not independent: causality ties them by the Kramers–Kronig relations (Section 1.3). Every other linear optical observable is a function of ε (Section 1.4) — which is why a single quantity, ε_2`(E)`, is all SIESTA has to compute.

Experimentally, ε is obtained by ellipsometry or by reflectance/transmittance spectroscopy (Aspnes and Studna's 1983 tables for Si and the III-V semiconductors [1] are the standard reference). Optical properties are the fingerprint of the electronic structure: the position of the absorption edge gives the (direct) gap, the sharp peaks are interband critical points (van Hove singularities of the joint density of states), the zero of ε_1 with a peak of the loss function marks a collective **plasmon**.

1.2 From the band structure to ε_2: independent-particle interband absorption. SIESTA's optical module computes ε_2 in the **independent-particle (random-phase, no local fields) approximation**: a photon of energy `E` and polarization `e` lifts an electron from an occupied band `v` to an empty band `c` at the same crystal momentum `k` (vertical transitions), with a rate given by Fermi's golden rule:

$$\varepsilon_2(E) = (\pi\ e^2\ /\ \varepsilon_0\ m^2\ \omega^2\ \Omega)\ \Sigma_k\ w_k\ \Sigma_{v,c}\ |\langle ck|\ e\cdot p\ |vk\rangle|^2\ \delta(\ E_c(k) - E_v(k) - E\ ) \qquad (2)$$

(spin degeneracy included in the sum over states; p is the momentum operator, ω = E/$\hbar$.) Two ingredients, both from the ground-state DFT calculation:

- the **energies** E_c(k) − E_v(k): the Kohn–Sham band gaps, which fix *where* the peaks fall;
- the **matrix elements** $|\langle c|e\cdot p|v\rangle|^2$ (transition probabilities, also selection rules: a transition not allowed for the chosen polarization contributes nothing), which fix *how strong* they are.

The delta function becomes a Gaussian of width Optical.Broaden (the experimental/lifetime broadening stand-in), and the k sum runs over the mesh Optical.Mesh — **independent of the SCF k-grid** (Section 1.10). The peaks of ε_2 are the interband critical points: where E_c − E_v is stationary in k the joint density of states has a van Hove singularity (in silicon, $E_1 \approx 3.4$ eV along Λ and $E_2 \approx 4.3$ eV near X and Σ [1]).

1.3 From ε_2 to ε_1: Kramers–Kronig. Causality (the response cannot precede the field) forces

$$\varepsilon_1(E) = 1 + (2/\pi)\ P \int_0^\infty\ E'\ \varepsilon_2(E')\ /\ (E'^2 - E^2)\ \ dE' \qquad (3)$$

(P is the principal value). stb-opticalAnalysis evaluates (3) by numerical quadrature on SIESTA's energy grid (0.01 eV steps up to 136 eV by default). Two consequences matter in practice. The **static dielectric constant**,

$$\varepsilon_1(0) = 1 + (2/\pi)\ \int_0^\infty \varepsilon_2(E')/E'\ dE' \qquad (4)$$

weights low-energy transitions most, so it is controlled by the **gap** (a too-small DFT gap overestimates it). And because the integral extends to infinity, an ε_2 truncated at a finite energy or by too few bands gives a slightly wrong ε_1 everywhere — one reason Stage 2 reports a sum-rule check (Section 1.5).

1.4 Everything else follows from ε. With $N^2 = \varepsilon$:

$$n = \mathrm{sqrt}(\ (|\varepsilon| + \varepsilon_1)\ /\ 2\ ),\quad k = \mathrm{sqrt}(\ (|\varepsilon| - \varepsilon_1)\ /\ 2\ ) \qquad (5)$$

$$\alpha(E) = 4\pi\ k\ E\ /\ (hc) \qquad \text{(Beer–Lambert, } I \propto e^{-\alpha x}\text{; } hc = 1.2398\times10^{-4} \text{ eV·cm)} \qquad (6)$$

$$R(E) = ((n-1)^2 + k^2)\ /\ ((n+1)^2 + k^2) \quad \text{(normal incidence, vacuum/solid)} \qquad (7)$$

$$L(E) = -\mathrm{Im}(1/\varepsilon) = \varepsilon_2\ /\ (\varepsilon_1^{\,2} + \varepsilon_2^{\,2}) \qquad (8)$$

$$\sigma_1(E) = \varepsilon_0\ \omega\ \varepsilon_2 \qquad (9)$$

R is the fraction of normally incident light reflected at a clean flat surface — it is *not* the reflectance of a thin film or a rough surface (Section 1.11). L is what fast electrons (EELS) lose energy to: it peaks at **volume plasmons**, where ε_1 crosses zero while ε_2 is small. σ_1 is the optical conductivity. For a non-metal, $\varepsilon_2 = 0$ below the gap, so $\alpha = 0$ and the crystal is transparent there.

Sum rules, the plasmon, and the Penn model. The **f-sum rule** says that the total oscillator strength of a system equals its number of electrons. Applied to the valence electrons of density N_val/Ω:

$$\int_0^\infty E\ \varepsilon_2(E)\ dE = (\pi/2)\ (\hbar\omega_p)^2\ , \qquad \hbar\omega_p = \hbar\ \mathrm{sqrt}(\ N_val\ e^2\ /\ (\varepsilon_0\ m\ \Omega)\ ) \qquad (10)$$

Define the cumulative N_eff(E) = (2 ε_0 m Ω / π $\hbar^2$) $\int_0^E E'\ \varepsilon_2(E')\ dE'$: how many electrons' transitions are already included up to E. A complete calculation has $N_eff(\infty) \to N_val$; the shortfall measures a **truncated** spectrum (too few bands, too small an energy range, or — the usual limit in LCAO

codes — an incomplete basis). SIESTA prints its own version of the check (`Optical: Checking f-sum rule`, 1 = complete) and Stage 2 recomputes `N_eff` independently from ε_2 so the two can be compared.

$\hbar\omega$_p of (10), computed for the valence-electron density, is the **free-electron plasma energy**: in a simple sp metal or semiconductor the volume plasmon (maximum of `L`) sits close to it. And a quick one-oscillator estimate links the static constant to an average gap, the **Penn model** [2]:

$$\varepsilon_1(0) \approx 1 + (\hbar\omega_p \,/\, E_P)^2 \quad \Rightarrow \quad E_P = \hbar\omega_p \,/\, \mathrm{sqrt}(\,\varepsilon_1(0) - 1\,) \tag{11}$$

(`E_P`, the "Penn gap", is not a band gap but a measure of where the transition weight sits; ~4–5 eV for silicon.)

The dielectric tensor, polarization directions, and symmetry. In an anisotropic crystal ε is a symmetric 3×3 tensor. SIESTA computes the response $\mathbf{e}^{\mathsf{T}}\,\varepsilon\,\mathbf{e}$ along **one** polarization vector `e` per run (`Optical.Vector`), so the full tensor needs several runs:

- the diagonal components ε_xx, ε_yy, ε_zz come directly from `e = x, ŷ, ẑ`;
- an off-diagonal component is reached with the **bisector trick**: along `e = (x + ŷ)/`$\sqrt{2}$ the response is `(`ε`_xx + `ε`_yy)/2 + `ε`_xy`, hence

$$\varepsilon_xy = \varepsilon_(\text{bisector } xy) - (\varepsilon_xx + \varepsilon_yy)/2 \tag{12}$$

and the same for `xz` and `yz`. This is what Stage 1's biaxial directions and Stage 2's automatic reconstruction do.

Crystal symmetry reduces the number of runs. The tensor must be invariant under every symmetry operation of the crystal, which leaves:

Crystal system	Independent components	Form (axes along the symmetry axes)
cubic	1	ε_xx = ε_yy = ε_zz, off-diagonal 0
tetragonal, hexagonal, trigonal	2	ε_xx = ε_yy ($\perp$ axis), ε_zz ($\parallel$ axis)
orthorhombic	3	ε_xx, ε_yy, ε_zz distinct
monoclinic	4	three diagonal + one off-diagonal
triclinic	6	all

`stb-optical` finds this for *your cell in its own Cartesian orientation* (it projects the tensor onto the subspace invariant under the detected space group, so it also works when the axes are not aligned) and tells you the minimal direction set. `stb-opticalAnalysis` then **checks the finished spectra against it**: components that symmetry makes equal must agree, and components it forbids must vanish — an end-to-end test of the calculation (Section 5.2 shows silicon passing it to 10^{-8}). A powder or polycrystal measures the **isotropic average** (`avg`, a third of the trace), which Stage 2 also builds.

The absorption edge: onset, Tauc, direct vs indirect. Below the gap there is no absorption; the edge's shape depends on the type of transition. For **direct allowed** transitions between parabolic bands

$$\alpha(E)\;E \propto (E - E_g)\hat{}\{1/2\} \quad \Rightarrow \quad (\alpha E)^2 \text{ is linear in } E \text{ and extrapolates to } E_g \tag{13}$$

(the **Tauc plot**; for indirect gaps `(`α`E)^{1/2}` is used instead). Stage 2 reports the **onset** (the energy where ε_2 reaches 5 % of its maximum) and the direct Tauc estimate. Two cautions. (i) Real band edges are not parabolic (silicon's edge is steep and structured), so the Tauc number is **indicative**. (ii) In the independent-particle approximation only **vertical** transitions exist: **indirect gaps are invisible** — silicon's 1.17 eV indirect gap, which needs a phonon, does not appear; the first absorption is its direct gap, about 2.5 eV in this calculation (Section 5.2).

What DFT-level optics gets right, and what it does not. Equation (2) fed by Kohn–Sham bands is a very good *qualitative* and often good *semi-quantitative* description of the optical response of sp semiconductors and metals, but it has well-known limitations, each flagged in the report:

1. **The gap problem.** Semilocal DFT (LDA/GGA) underestimates band gaps (silicon: ~0.6 eV against 1.17 eV). Since ε_2 peaks sit at the transition energies, the whole spectrum is **red-shifted** by roughly the gap error and ε_1`(0)` is **overestimated** (equation (4)). The fix is a quasiparticle (GW) correction; a cheap stand-in is the **scissor operator**: a rigid upward shift Δ of the conduction bands. With position-matrix elements kept it amounts to ε_2`'(E) =` ε_2`(E −` Δ`)` (Levine and Allan [3]; see also Godby, Schlüter and Sham [4]), which Stage 2 offers as `--scissor` — a what-if, not a prediction.
2. **No excitons.** Equation (2) treats the excited electron and the hole as non-interacting. Their attraction moves spectral weight toward lower energy, enhances the lowest peaks (silicon's E_1) and creates bound exciton states *below* the gap — decisive for low-dimensional materials. Capturing it needs the Bethe–Salpeter equation (Rohlfing and Louie [5]; Onida, Reining and Rubio [6]), outside this workflow.
3. **No local-field effects.** The macroscopic response also includes the response to the microscopic fluctuations of the field (Adler [7], Wiser [8]); neglecting them (the independent-particle approximation) matters for anisotropic and layered systems and for ε_1`(0)`.
4. **No intraband (Drude) term.** Only interband transitions are computed, so a metal's low-energy free-carrier response is absent (ε_2 is non-zero down to 0 eV only through interband tails).
5. **No phonon-assisted absorption, no temperature, no spin–orbit coupling.** Indirect edges and band-edge tails, thermal broadening and spin–orbit splittings are not included.
6. **The velocity operator with non-local pseudopotentials.** The velocity is `i[H, r]/`$\hbar$, equal to `p/m` only for a local potential; with non-local (norm-conserving) pseudopotentials the momentum matrix elements need a commutator correction (Read and Needs [9]). How the code treats it is a detail of SIESTA's implementation (see its manual).
7. **A finite, localized basis.** Unoccupied bands are only as good as the basis (SIESTA's LCAO set). Practically: the f-sum rule cannot reach 1 once the basis is exhausted, and a confined basis (large `PAO.EnergyShift`) can shift peaks (Section 5.4).
8. **Normal-incidence reflectivity of a perfect flat surface**, nothing else (no angle, no film interference, no roughness).

Two-dimensional and zero-dimensional systems. A slab or monolayer is simulated as a periodic supercell with vacuum. SIESTA's ε`_SC` is then the response of the **whole cell**, material plus vacuum, so it is diluted by the vacuum (and the result scales with the arbitrary cell height). The intrinsic response of the layer is restored by treating cell and layer as a capacitor stack. For a layer of thickness `t` in a cell of height `c` (Laturia, Van de Put and Vandenberghe [10]; Yang and Gao [11]):

$$\begin{aligned} &\text{in-plane (E continuous):} && \varepsilon\text{_2D},\parallel(E) = 1 + (c/t)\ (\ \varepsilon\text{_SC},\parallel(E) - 1\ ) \\ &\text{out-of-plane (D continuous):} && \varepsilon\text{_2D},\perp(E) = [\ 1 + (c/t)\ (\ 1/\varepsilon\text{_SC},\perp(E) - 1\ )\ ]^{-1} \end{aligned} \qquad (14)$$

`t` is **not** auto-estimated, because the "thickness" of a 2D material is not one well-defined number (atomic span? plus van der Waals radii? the bulk interlayer distance?) — `--thickness` is yours to choose, and the corrected values scale with it. For an isolated **molecule** (vacuum on all three axes) the box is the dilute limit: the molecular polarizability follows from α`(E) =` ε_0 Ω `(`ε`_SC(E) − 1)`, reported in Å^3. **1D** systems (wires) are not corrected: the literature approach is not a simple generalization of (14) and has not been verified for this tool.

The numerical parameters, and what each one controls.

Parameter	Where	What it controls	How to converge
SCF k-grid	`kgrid.MonkhorstPack` (Stage 1 forces one if the template has none)	the ground-state density and bands	density ~0.2 1/Å for semiconductors; check with a denser grid
`Optical.Mesh`	`--optical-mesh`	the k sum of (2): **smoothness of ε_2 and peak heights/positions**	the most important: double it until peaks stop moving (Section 5.3); spacing $\lesssim$ 0.07 1/Å for silicon
`Optical.Broaden`	`--optical-broaden`	Gaussian width of every transition	0.05–0.2 eV; smaller than the mesh can resolve gives spikes
`Optical.NumberOfBands`	`--optical-nbands`	conduction bands entering the sum: **energy range**	more than the occupied number; check the f-sum
Basis size and `PAO.EnergyShift`	`calc.fdf`	quality of the **unoccupied** bands	TZP and a small shift ($\leq$ 0.01 Ry); check peaks
`Mesh.Cutoff`	`calc.fdf`	real-space integration grid	little effect here (Section 5.3)
Scissor	`--scissor` (Stage 2)	rigid conduction-band shift	a what-if, never a convergence knob

The **SCF k-grid is not the optical mesh**: the SCF only needs enough points to converge the density; the optical sum (2) is a much more demanding integral of a sharply structured function and needs a mesh typically several times denser.

What this workflow simplifies — explicitly.

- **Independent-particle only**: no excitons, no local fields, no quasiparticle correction (only the post-processing scissor).
- **Linear, vertical interband response of a perfect crystal**: no Drude term, no phonons, no temperature, no spin–orbit, no disorder.
- **One polarization per folder**, repeating the same ground-state SCF in each (an economical use of the SCF would reuse it; here independence is the point).
- **Normal-incidence reflectivity only.**
- **2D/0D via a user-chosen thickness / the dilute limit** (Section 1.9); no 1D.
- **A single relaxed geometry**: nothing here relaxes the structure (that is why Stage 1 requires a relaxed input).

A calculation you can check by hand. The silicon numbers of Section 5.2 let you re-derive a row of the report with a pocket calculator. Take the 4.3 eV row: $\varepsilon_1 = -16.121$, $\varepsilon_2 = 15.042$.

1. $|\varepsilon| = \sqrt{(16.121^2 + 15.042^2)} = 22.05$.
2. Equation (5): $n = \sqrt{((22.05 - 16.121)/2)} = \sqrt{2.965} = 1.722$, $k = \sqrt{((22.05 + 16.121)/2)} = \sqrt{19.09} = 4.369$.
3. Equation (6): $\alpha = 4\pi\, k\, E / hc = 4\pi \times 4.369 \times 4.3 / 1.2398\times10^{-4} = 1.904\times10^{6}$ cm^{-1}.
4. Equation (7): $R = ((1.722-1)^2 + 4.369^2) / ((1.722+1)^2 + 4.369^2) = (0.521 + 19.09)/(7.41 + 19.09) = 0.740$.
5. Static limit: $n(0) = \sqrt{\varepsilon_1(0)} = \sqrt{13.441} = 3.666$, so $R(0) = (2.666/4.666)^2 = 0.3265$.
6. Penn gap, equation (11): $\hbar\omega_p = 16.32$ eV (8 valence electrons in 41.39 Å^3), $E_P = 16.32/\sqrt{(13.441 - 1)} = 4.63$ eV.

Each is the corresponding entry of the Stage 2 tables ($n = 1.7216$, $k = 4.3686$, $\alpha = 1.904e{+}06$, $R = 0.7401$, $R(0) = 0.3265$, Penn gap 4.63 eV): the tool is nothing more mysterious than equations (3)–(11) applied to ε_2.

11.14.1.2 Libraries and external dependencies used

- **SIESTA** [12, 13] itself is *not* called by the suite. It does the real work: the ground-state calculation and, with `OpticalCalculation T`, the interband optical module that writes `<SystemLabel>.EPSIMG` (the energy grid and ε_2) and, in `calc.out`, the `Optical:` block (polarization, bands, energy range, broadening, scissor, mesh and the **f-sum check**). The reference data of Section 5 was produced with SIESTA 5.4.2, GGA-PBE [14], PseudoDojo norm-conserving pseudopotentials [15] and Monkhorst–Pack k-grids [16].
- **numpy** / **SciPy** [17] — the Kramers–Kronig quadrature, the f-sum integral, Gaussian smoothing and peak finding (`scipy.signal.find_peaks`), constants.
- **pymatgen** [18] / **spglib** [19] — the space-group and symmetry operations that turn the crystal into the list of independent tensor components (Section 1.6).
- **ASE** [20] and **sisl** [21] — structure I/O; sisl reads SIESTA's `.XV` in this tutorial's helper scripts.
- **matplotlib** [22] — the dashboard (`--png` / `--show`). The `.dat/.gplot` pairs are for gnuplot, which the suite never invokes.

11.14.1.3 3. Stage 1: generating the direction folders (`stb-optical`, code `5.14.1`)

What it does. Reads an **already relaxed** structure (`-f`, `.fdf`) and the `calc.fdf` template (`-c`, default `calc.fdf`), and writes, under `-O` (default `optical_study`), one folder per requested direction:

```
optical_study/
+- optical_stage1.txt        <- the Stage 1 report (with --save-report)
+- dir_xx/ dir_yy/ dir_zz/   <- diagonal components (Optical.Vector = x, y, z)
+- dir_xy/ dir_xz/ dir_yz/   <- biaxial bisectors (for eps_ij, Section 1.6)
        structure.fdf  calc.fdf  config_extra.fdf  <pseudopotentials>
```

It never runs SIESTA and never moves an atom: every folder holds the *same* structure. `--directions` selects which folders (default `xx yy zz`); `-p` copies PseudoDojo (or your own) pseudopotentials.

3.2 `config_extra.fdf`: everything mandatory, the template untouched. Each folder's `calc.fdf` is your template with `%include config_extra.fdf` added on the first line; the forced directives live in `config_extra.fdf`. SIESTA's fdf reader is *first-occurrence-wins*, so they override whatever the template says. Live, for the biaxial `xy` folder of silicon:

```
$ cat optical_study/dir_xy/config_extra.fdf
# Ground-state SCF k-grid (explicit --kgrid) -- takes precedence over --calc's own grid.
kgrid.MonkhorstPack   [11  11  11]

# Auto-generated -- forces a single-point evaluation (no relaxation): this
# folder is one independent, uncoupled sample point, not something SIESTA
# should be moving on its own.
MD.TypeOfRun          CG
MD.Steps              0
MD.NumCGsteps         0
MD.VariableCell       false

OpticalCalculation T
Optical.PolarizationType    polarized
Optical.Broaden       0.1 eV
%block Optical.Mesh
  32  32  32
%endblock Optical.Mesh
```

```
%block Optical.Vector
  0.7071  0.7071  0.0000
%endblock Optical.Vector
Optical.NumberOfBands  16
```

Directive	Forced value	Why
`kgrid.MonkhorstPack`	the template's own grid is kept; **forced** when the template has none, or with `--kgrid X Y Z` / `--kgrid-density D`	the SCF needs a k-grid — SIESTA would otherwise sample Γ only; suggested at density 0.2 1/Å, **1 division on a vacuum axis**
`MD.TypeOfRun CG`, `MD.Steps 0`, `MD.NumCGsteps 0`, `MD.VariableCell false`	single point	the geometry is evaluated as given — verified with SIESTA: a template with `MD.NumCGsteps 40` and `MD.VariableCell T` still runs one single point
`OpticalCalculation T`	—	switches the optical module on
`Optical.PolarizationType polarized`	—	without it SIESTA defaults to `polycrystal` and **silently ignores `Optical.Vector`**: all directions would return the identical isotropic result
`Optical.Broaden`, `Optical.Mesh`, `Optical.NumberOfBands`	`--optical-broaden` (0.2 eV), `--optical-mesh` (10 10 10), `--optical-nbands` (SIESTA default: all)	Section 1.10
`Optical.Vector`	one per folder	the polarization direction

The report, live.

```
$ stb-optical -f structure.fdf -p dojo --directions xx yy zz xy xz yz --kgrid 11 11 11 \
      --optical-mesh 32 32 32 --optical-broaden 0.1 --optical-nbands 16 --no-intro

[1] INPUT STRUCTURE
Lattice (Ang)   : a=3.8828  b=3.8828  c=3.8828
Angles (deg)    : alpha=60.00  beta=60.00  gamma=60.00
Species | Z  | Atoms | Valence e- | Total e- | Valence from
Si      | 14 | 2     | 4          | 8        | Si.psml
Valence electrons : 8  ->  4 occupied band(s) in a spin-unpolarized calculation
Detected : 3D (bulk material)

[3] OPTICAL PARAMETERS (applied identically to every direction)
Optical.Mesh | Divisions | Spacing (1/Ang) | Assessment
axis a       | 32        | 0.0619          | fine
axis b       | 32        | 0.0619          | fine
axis c       | 32        | 0.0619          | fine
Optical k-points  : 32768 total (independent of the SCF k-grid)
Bands included    : 16 = 4 occupied + 12 unoccupied

[5] DIRECTIONS & DIELECTRIC-TENSOR MAPPING
  Space group           : Fd-3m (No. 227), cubic -- 48 operations at symprec 0.01 Ang
  Independent components: 1 of 6
  Non-zero components   : eps_xx=eps_yy=eps_zz
  Zero by symmetry      : eps_xy, eps_xz, eps_yz
  Minimal direction set : xx
[NOTE] yy zz xy xz yz follow from symmetry, so they are redundant; computing them is a
consistency check (identical results expected), not new information.
```

How to read it. **[1]** reads the valence electron count from the pseudopotential (`z-pseudo` of the `.psml`), so the number of occupied bands is known and `Optical.NumberOfBands` can be checked against it (16 bands

= 4 occupied + 12 conduction). **[3]** assesses the optical mesh per axis by its k-point *spacing* $|b_i|/n_i$ (fine ≤ 0.07, coarse > 0.12 1/Å — a rule of thumb from converging this very crystal, Section 5.3) and counts the optical k-points. **[5]** applies Section 1.6: silicon needs one direction; this tutorial computes all six on purpose, as a consistency test.

The advisories. Printed where they apply and summarized at the end: an `Optical.Mesh` coarse along some axis; `Optical.NumberOfBands` not exceeding the occupied bands (**no conduction states: no absorption at all**) or leaving only a few; a broadening below 0.05 eV (a forest of spikes unless the mesh is very dense) or above 0.3 eV (washes out structure); a vacuum axis with more than one mesh division (no dispersion there, wasted time); a missing pseudopotential; a spin-polarized template (not special-cased) and a minimal basis (poor conduction bands).

Bulk, slab, molecule. A vacuum-padded cell is detected from the gap along each axis (`--vacuum-gap`, default 10 Å): one vacuum axis → 2D, two → 1D, three → 0D, each with its own limitation note (Section 1.9) and a hint for Stage 2. A vacuum axis gets a single SCF k-point division. No direction is ever skipped for a 2D input, including out-of-plane.

Validation. `stb-optical` refuses, with a clear `[ERROR]`: a missing structure, a missing calc template (the `calc.fdf` default not being in the current directory — the message says so), and an unknown direction (argparse: `invalid choice`).

Running it both ways. `stb-suite` → `5.14.1` asks the same questions as guided prompts (its advanced step now also asks for the SCF k-grid). Case 7 of the script drives it with piped answers and diffs the generated folders against the direct CLI run.

11.14.1.4 4. Stage 2: analyzing the results (`stb-opticalAnalysis`, code `5.14.2`)

Before running Stage 2. Run SIESTA in each folder:

```
for d in optical_study/dir_*; do (cd $d && mpirun -np 4 siesta calc.fdf > calc.out); done
```

Each folder must end with `<SystemLabel>.EPSIMG` (the energy grid and ε_2) and a `calc.out` whose `Optical:` block carries the f-sum check. Stage 2 detects each folder's direction from its **own `Optical.Vector`**, not from its name, so renamed folders work; folders without an `.EPSIMG` are skipped with a warning.

Report structure.

```
[0] RUN METADATA
[1] SYSTEM & CALCULATION      <- structure, ground state, per-folder Optical settings, symmetry
[2] DATA QUALITY & CONVERGENCE <- SCF, SIESTA's f-sum check, N_eff(E) from eps2
[3] STATIC RESPONSE & ABSORPTION EDGE   <- eps1(0), n, R, onset, Tauc, Penn gap, Kohn-Sham gap
[4] SPECTRAL FEATURES         <- eps2 peaks, alpha/R/eps1 maxima, plasmon, eps1 zeros
[5] ANISOTROPY & SYMMETRY CHECKS
[6] OFF-DIAGONAL DIELECTRIC TENSOR (eps_ij RECONSTRUCTION)
[7] OPTICAL CONSTANTS AT SELECTED ENERGIES
[8] DIMENSIONALITY CORRECTION        <- only with --dimensionality-correction
[9] SCISSOR WHAT-IF                  <- only with --scissor
[10] EXPERIMENTAL COMPARISON         <- only with --experimental
[11] INTERPRETATION
[12] SUMMARY & FILES
```

Numbering follows what is present. **No arguments beyond `--directory`** are needed (default `optical_study`). Warnings are printed where they apply and repeated in a count at the end.

4.3 [1]–[2]: is the calculation what you think, and is it complete?. [1] reads the structure, the ground state (functional, basis, mesh cut-off, k-grid) and, for **each** folder, the optical settings *as SIESTA ran them* — mesh, broadening, bands, scissor — from `calc.out`. If the folders were not run with identical optical settings a warning says the comparison between directions (anisotropy, ε_ij) is no longer legitimate (Case 14c of the script).

[2] is the **convergence section**: per folder the SCF status, SIESTA's own f-sum check, and N_eff(E_max) recomputed from ε_2 (Section 1.5):

```
Dir | SCF ok | f-sum (SIESTA) | N_eff(E_max) | N_eff/N_val | E range (eV) | Points | Step (eV)
xx  | yes    | 0.8703         | 6.962        | 0.870       | 0.00-136.0   | 13606  | 0.0100
```

The independent estimate `6.962 = 0.8703 × 8` agrees with SIESTA's number to the printed digits — a check that the energy grid, the volume and the constants are all consistent. The [WARNING] below 0.9 says what it means: the transition sum is incomplete, and — an important refinement found while building this tutorial — *more bands help only while fewer than all are used; beyond that the basis is the limit* (Section 5.4).

4.4 [3]: static response, edge, and the Kohn–Sham gap. For each direction and the isotropic average: ε_1(0), n(0), R(0), the ε_2 **onset**, the direct-gap **Tauc** estimate and the **Penn gap** (Section 1.5/1.7). From the folder's .EIG it also reads the **Kohn–Sham gap on the SCF k-mesh**, fundamental and smallest direct, and uses it to say what the edge means:

```
Kohn-Sham band gap (SCF k-mesh, 666 points): fundamental 0.644 eV (indirect); smallest direct gap
2.527 eV.
Reference spectrum 'avg': the absorption edge starts at 2.52 eV (wavelength 492 nm). It matches the
smallest direct Kohn-Sham gap (2.53 eV): the edge is direct, and the indirect gap is invisible here.
```

For a **metal or semimetal** (zero Kohn–Sham gap) the tool refuses to call the onset a gap: it is set by the smallest transition the optical mesh samples and by the broadening, and the Tauc/Penn columns are blanked (Section 5.6).

4.5 [4]–[7]: features, symmetry, tensor, constants. [4] lists the peaks of ε_2 (position, height, FWHM, and ε_1, n, k, α, R at each), the maxima of α, R and ε_1, the zero crossings of ε_1, and the loss-function maximum compared with the free-electron plasmon (only for a 3D cell: in a vacuum-padded cell the electron density is not the material's). Peaks are located on a Gaussian-smoothed curve (--smoothing, 0.05 eV) so a spiky spectrum does not fool the finder. [5] compares directions with the symmetry (Section 1.6), [6] reconstructs ε_ij and checks it vanishes when symmetry demands, [7] tabulates ε_1, ε_2, n, k, α, R, L at the energies you choose (--energies, default 0 1 2 3 4 5 10, with the wavelength).

The optional extras.

Flag	What you get
--png / --show	the 9-panel dashboard (ε, n/k, α with its onset, Tauc, R, loss, σ_1, f-sum accumulation, scissor/off-diagonal/anisotropy) saved / opened with matplotlib
--scissor EV	a what-if: ε_2'(E) = ε_2(E − EV), ε_1 by Kramers–Kronig, with a before/after table and an overlay (Section 5.5)
--dimensionality-correction (+ --thickness Å)	the intrinsic 2D function (equation 14) or the molecular polarizability
--experimental PATH	overlay and peak matching of a measured spectrum (single-valued --plot-quantity)

Flag	What you get
`--energy-min/--energy-max`	trim the data used by every analysis
`--energies`, `--n-peaks`, `--smoothing`, `--plot-emax`	table energies, peaks listed, peak-finding smoothing, dashboard range
`--plot-quantity {...,all}`	which `.gplot` is written (default `eps`)

Always written: `optical_results.csv` (every direction and energy), `optical_results_summary.csv` (one row per direction with the static numbers, the `N_eff` and the Kohn–Sham gaps), a `.dat` per direction and per reconstructed component, and the gnuplot scripts.

Guard rails. Case 14 of the script shows what Stage 2 flags from the files alone: folders with no `.EPSIMG` (skipped; with none usable it stops), an SCF that never confirmed convergence, folders run with different optical settings, and a symmetry-forbidden ε_ij that is not zero. The unit tests also cover the Kohn–Sham gap reading for an insulator and a semimetal.

Running it both ways. `stb-suite` → `5.14.2` asks the directory, the quantity to plot, the optional extras (screen/PNG dashboard, scissor, selected energies) and the advanced settings. Case 14 drives it with piped answers and checks it writes the same dashboard and scissor spectrum as the CLI run.

11.14.1.5 Worked example: bulk silicon and graphene, end to end

Everything below was produced by running this folder's own inputs through `stb-optical`, SIESTA 5.4.2 and `stb-opticalAnalysis` (Section 6). Silicon is the textbook case, with a symmetry check built in; graphene shows a 2D semimetal.

Set-up.

	Silicon	Graphene
Structure	diamond, 2-atom primitive cell, relaxed with SIESTA	2 atoms, 20 Å cell along c
DFT	GGA-PBE, TZP, `PAO.EnergyShift` 0.005 Ry, `Mesh.Cutoff` 300 Ry, 25 meV	GGA-PBE, TZP, 0.01 Ry, 300 Ry
Pseudopotentials (Section 7)	PseudoDojo (`-p dojo`)	`virtual_vault` PSF carbon (`-p virtual_vault`)
SCF k-grid	11×11×11 (`--kgrid`)	24×24×1 (`--kgrid`)
Optical	mesh 32^3 (32 768 k), broaden 0.1 eV, 16 bands	mesh 72×72×1, broaden 0.1 eV, 24 bands
Directions	all six (a consistency test; 1 would do)	xx, zz (hexagonal)
Cost	~1–2 min per folder	~17 s per folder

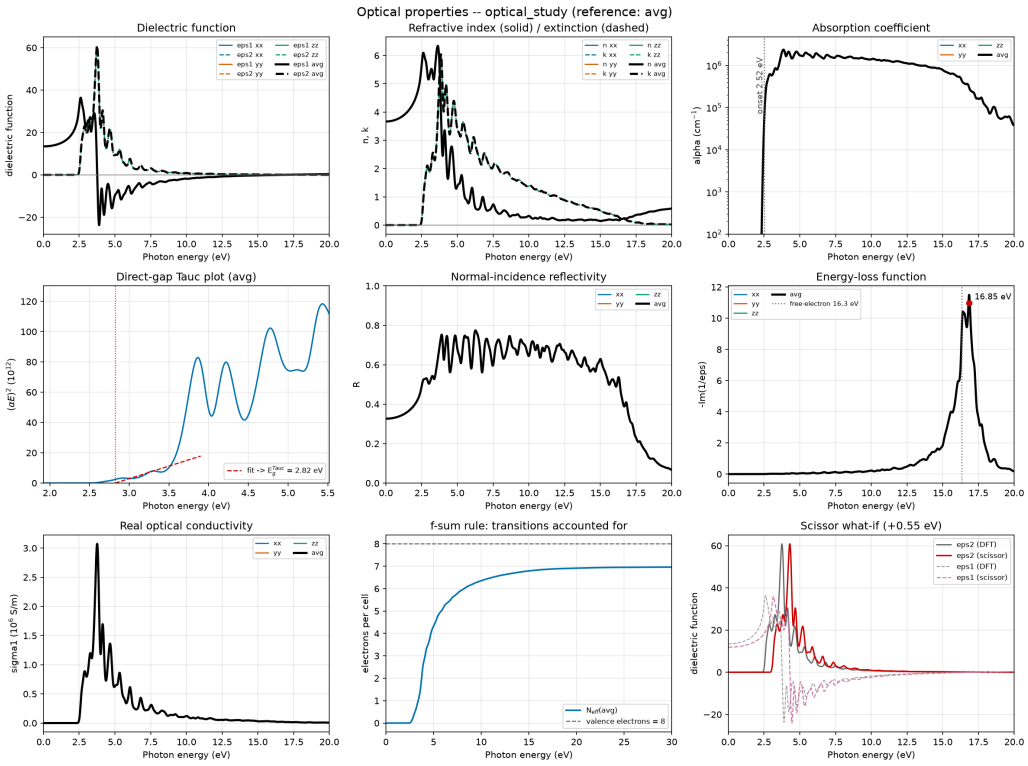
Step 1 (relaxation). Silicon: variable-cell CG from the experimental 5.431 Å gives $\mathbf{a_0}$ = **5.4911 Å** (+1.1 %; PBE overestimates), residual stress < 0.5 kbar. Graphene: the two atoms are fixed by symmetry, so only the lattice constant needs finding — an energy scan (`reference_results/graphene/a0_scan.csv`) puts the minimum at **2.4815 Å**, shipped as 2.48 Å.

Silicon: the textbook case, and a built-in test. All six folders ran in a total of a few minutes. First the **consistency test** of Section 1.6 (cubic Fd-3m): the three diagonal spectra agree to 5×10^{-8} of their norm ($\varepsilon_1(0) = 13.4410$ in all), and the reconstructed off-diagonal components are 5.6×10^{-5} of the maximum ε_2 — zero to the noise. This validates the folders, the settings, the bisector trick and the reconstruction at once.

The static and edge numbers (isotropic average):

Quantity	This calculation	Reference
ε_1(0)	13.44	≈ 11.7 (experiment, from memory — not verified here)
n(0), R(0)	3.666, 0.3265	—
Kohn–Sham gap	0.644 eV indirect, 2.527 eV smallest direct	1.17 eV indirect (experiment); PBE ≈ 0.6 eV
ε_2 onset	**2.52 eV** (492 nm)	matches the 2.53 eV direct KS gap
Penn gap	4.63 eV	~4–5 eV
strongest ε_2 peak	**3.75 eV** (ε_2 = 58, FWHM 0.47)	E_2 ≈ 4.2–4.3 eV [1]
E_1	a shoulder at ≈ 3.2–3.4 eV	≈ 3.4 eV [1]
α maximum	2.4×10^6 cm^{-1} at 3.86 eV	—
loss-function maximum	**16.85 eV** (FWHM 1.3 eV)	≈ 16.7 eV (experiment, from memory); free-electron 16.32 eV
ε_1 zero crossings	3.76 eV (falling), 16.53 eV (rising)	—
f-sum	N_eff = 6.962 = 0.870 × 8, SIESTA 0.8703	1

How to read it. **The edge**: the first absorption is at 2.52 eV, equal to the smallest *direct* Kohn–Sham gap, and the indirect 0.64 eV gap is invisible (Section 1.7). **The red shift**: the strongest peak is at 3.75 eV against ≈ 4.2–4.3 eV measured, a deficit of about 0.5 eV — almost exactly the gap error (1.17 − 0.64 = 0.53 eV), the expected behavior of independent-particle optics on a semilocal ground state; ε_1(0) is correspondingly too large. **The plasmon**: the loss maximum sits at 1.03× the free-electron value, the signature of a collective valence-electron plasmon. The dashboard summarizes it:



Silicon dashboard

Top row: the dielectric function, n/k, and α on a log scale with the onset; *middle:* the direct-gap Tauc plot (the fit extrapolates to 2.82 eV, 0.3 eV above the onset — a reminder that silicon's steep edge is not parabolic), R, and the loss function with the plasmon marked against the free-electron value (dotted); *bottom:* σ_1, the f-sum accumulation (flattening at 7.0 of 8 electrons) and the scissor what-if of Section 5.5.

Convergence: which knob moves which number. The most important lesson of this tutorial. Eleven re-runs of `dir_xx`, each changing **one** setting of the production calculation (all analysed with `--energy-max 30`; the production row too):

run	what differs from production	eps1(0)	main eps2 peak (eV)	loss peak (eV)	f-sum
production	(baseline)	13.441	3.75 (58.1)	16.85	0.870
mesh16	Optical.Mesh = 16	13.460	3.66 (53.1)	16.91	0.871
mesh24	Optical.Mesh = 24	13.441	3.83 (52.9)	16.71	0.870
mesh48	Optical.Mesh = 48	13.441	3.74 (56.2)	16.74	0.870
shift020	EnergyShift = 0.02 Ry	12.994	3.78 (54.6)	16.78	0.859
shift010	EnergyShift = 0.01 Ry	13.317	3.76 (56.5)	16.84	0.868
shift002	EnergyShift = 0.002 Ry	13.428	3.74 (58.6)	16.81	0.871
dzp	basis = DZP	13.319	3.75 (58.0)	16.55	0.867
allbands	bands = 26	13.442	3.75 (58.1)	16.84	0.880
broaden005	Broaden (eV) = 0.050	13.438	3.75 (62.4)	16.84	0.870
broaden020	Broaden (eV) = 0.200	13.455	3.74 (48.4)	16.82	0.870
cut450	Mesh.Cutoff = 450 Ry	13.441	3.75 (58.1)	16.85	0.870

What each row teaches:

- **The optical mesh sets the peak position and smoothness.** At 16 and 24 divisions the strongest peak lands at 3.66 and 3.83 eV (and its height drops from 58 to 53); at 32 and 48 it is 3.75 and 3.74 eV — converged to 0.01 eV. A coarse mesh can move a peak by 0.1 eV or more, and nothing in a single run tells you so: you have to repeat it. (In earlier exploratory runs on a 20×20×20 mesh the same peak came out between 3.8 and 4.0 eV depending on the basis — an artifact of the mesh, which is why those numbers are not quoted.)
- **The broadening sets the heights, not the positions**: 0.05 eV gives a peak of 62, 0.2 eV of 48; positions agree to 0.01 eV.
- **The basis moves ε_1(0) and the f-sum, not the peak positions** (at a converged mesh): a confined basis (`PAO.EnergyShift` 0.02 Ry) lowers ε_1(0) by 3 % and the f-sum from 0.871 to 0.859; the result is converged by 0.005 Ry. DZP is within 1 % of TZP on ε_1(0).
- **The number of bands barely matters once there are enough**: all 26 bands raise the f-sum from 0.870 to only 0.880 and leave ε_1(0) unchanged — the remaining deficit is the basis' (Section 5.4).
- **`Mesh.Cutoff`** (300 → 450 Ry) changes nothing visible.

The SCF k-grid is a separate matter: 11×11×11 here (spacing 0.18 1/Å); the optical mesh is 32×32×32, about three times denser.

The f-sum rule: bands or basis?. SIESTA reports 0.870 (6.96 of 8 electrons) — the transition sum is incomplete. The natural reaction is to add bands. Section 5.3 shows the answer: with all 26 bands the check only reaches 0.880. In a localized basis the number of states is finite (26 here for two atoms in TZP), and once they are all used the sum rule is bounded by what that basis can represent in the continuum. The way to raise it is a richer basis (more zetas, polarization, a smaller `PAO.EnergyShift`), at the price of cost. The practical consequence: ε_1(0) computed at an incomplete sum is slightly **underestimated at high energy weight** — small next to the gap error, but worth knowing, and why Stage 2 prints the check for every folder.

Comparing with experiment, and the scissor what-if. The honest comparison of Section 5.2: edge at the direct Kohn–Sham gap, a peak 0.5 eV low, ε_1(0) too high. The scissor what-if shifts ε_2 rigidly by the gap error (here +0.55 eV) and re-derives ε_1:

```
Quantity                | DFT    | scissor +0.55 eV
eps1(E->0)              | 13.441 | 11.869
n(E->0)                 | 3.666  | 3.445
R(E->0)                 | 0.3265 | 0.3026
eps2 onset (eV)         | 2.52   | 3.07
strongest eps2 peak (eV) | 3.75   | 4.30
loss peak (eV)          | 16.85  | 17.78
```

The peak moves to 4.30 eV and ε_1(0) to 11.9 — both in the experimental range ($\approx$ 4.2–4.3 eV and $\approx$ 11.7). It also pushes the plasmon to 17.8 eV, *away* from the measured $\approx$ 16.7 eV: a rigid shift is right for the low-energy interband structure and wrong at high energy (the self-energy correction is energy dependent). That asymmetry is the reminder of what the scissor is: a diagnostic of *where* the gap error puts the peaks, not a substitute for GW — and, for silicon, not a cure for the missing excitonic enhancement of E_1 (Section 1.8).

Graphene: a 2D semimetal, and what the correction does. Graphene is a zero-gap semimetal with a Dirac cone at K: the Kohn–Sham bands touch the Fermi level (gap 2×10^{-5} eV on the 24×24 SCF mesh). The report says so, and calls the ε_2 onset (0.25 eV) *not a gap*: it is the smallest transition the 72×72 optical mesh samples around the Dirac point (the nearest k-points to K are not at K) — and the sharp low-energy peak (0.44 eV in this calculation) is the same sampling artifact, which is why the report does not list sub-1 eV peaks as strongest structure for a metal. A converged Dirac region needs a much denser mesh or a dedicated treatment.

What the calculation gets right is the structure of the spectrum:

Feature	This calculation	Reference
in-plane $\pi \rightarrow \pi*$ interband peak (σ_1, ε_2)	**4.02 eV**	$\approx$ 4.6 eV measured (from memory, check [25]), with an excitonic origin (a saddle-point exciton, [24, 25]); independent-particle DFT puts it near 4 eV
in-plane loss maximum (the π plasmon)	**5.02 eV**	$\approx$ 5 eV (EELS, from memory)
$\sigma*$ onset and peak	$\approx$ 11 eV, 13.6 eV	$\sigma \rightarrow \sigma*$ region
out-of-plane ε_zz	structureless below the σ threshold ($\approx$ 12 eV)	π orbitals are perpendicular: a transition with e $\parallel$ z between the two is symmetry forbidden at low energy

The raw supercell numbers are diluted by the vacuum (ε_1(0)_xx = 6.77, ε_1(0)_zz = 1.23 in a 20 Å cell); the correction restores the intrinsic 2D function, and its dependence on the thickness you supply is the central caveat. In-plane the correction is linear: $\varepsilon_{\parallel,2D}(0) = 1 + (c/t)(\varepsilon_{SC} - 1) = 35.4$ for t = 3.35 Å (the graphite interlayer distance) — a number that scales with t and is not a property of the sheet alone. Out-of-plane it is a complex *inversion* and ill conditioned:

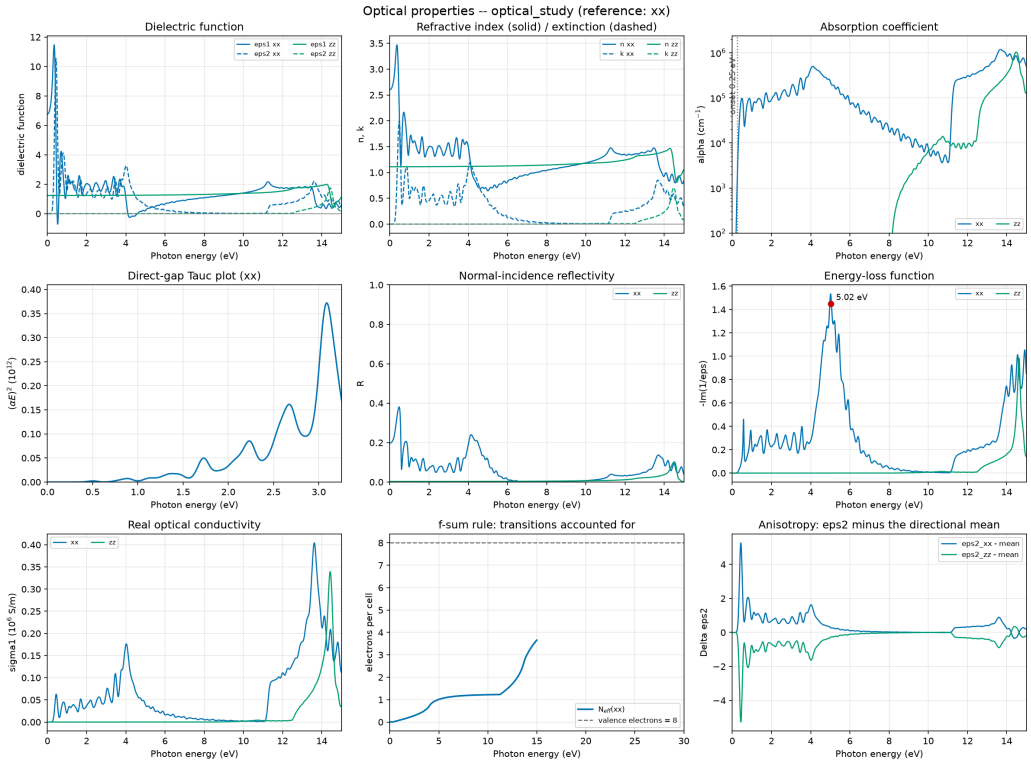
```
--thickness 3.35   Direction zz (perpendicular) -- eps1_2D(E->0) = -7.6335 (raw supercell: 1.2337)
--thickness 6.7    Direction zz (perpendicular) -- eps1_2D(E->0) =  2.3015 (raw supercell: 1.2337)
```

With t = 3.35 Å the corrected static ε_zz is **negative** (unphysical): the cell's out-of-plane polarizability is larger than a 3.35 Å slab of *any* dielectric could give, i.e. the π cloud extends beyond that nominal thickness. With t = 6.7 Å the number is sensible (2.3). That is precisely why --thickness is left to you (Section 1.9).

A final check against a known number. For a free-standing sheet in the thin-sheet limit the absorbance is A(E) = E t $\varepsilon_{2,2D}$(E) / $\hbar$c, and for graphene in the visible the measured value is the universal $\pi\alpha$ = 2.3 % (Nair et al. [23]):

```
E = 1.0 eV   A = 1.55 %
E = 1.5 eV   A = 2.07 %
E = 2.0 eV   A = 2.76 %
E = 3.0 eV   A = 5.04 %
universal value pi*alpha = 2.29 %
```

The calculation brackets the universal value around 1.5–2 eV (4.1–2.8 %) and rises toward the $\pi \rightarrow \pi*$ peak — what experiment shows, though experiment is flatter and the low-energy side here is depressed by the same Dirac-sampling limitation. The dashboard:



Graphene dashboard

How far to trust each number.

Quantity	Reliability
the consistency tests (symmetry, `N_eff`, ε`_ij` $\approx$ `0`), edge = direct KS gap	solid: they test the calculation itself
peak **positions**	converge with `Optical.Mesh` (verify: 32 and 48 agree); then a systematic red shift $\approx$ the DFT gap error
peak **heights**, α, `R` maxima	depend on the broadening and the mesh; qualitative
ε_1`(0)`, `n(0)`, `R(0)`	overestimated by the small gap (13.4 vs $\approx$ 11.7); semi-quantitative
scissor results	a what-if, right for the low-energy interband structure, wrong at the plasmon
2D corrected values	scale with the thickness you choose; the out-of-plane one can be unphysical
anything below the first few tenths of eV of a metal/semimetal	a mesh artifact

What to take from the example. Three habits the numbers above justify: **converge the optical mesh first**, by doubling and looking at where the peaks go; **read the sum rule and the Kohn–Sham**

gap before the pretty plots; and **compare with experiment keeping the independent-particle red shift and the missing excitons in mind** — the same discipline that turned a plausible-looking number into a checked one in 5.13.

11.14.1.6 Reproducing the reference data

`reference_results/` holds finished SIESTA runs, so Stage 2 works out of the box without a SIESTA installation.

Folder	What	Contents
`si/relax/`	step 1: variable-cell relaxation of bulk Si	`structure.fdf` (start), `calc.fdf`, `calc.out`, `siesta.XV`
`si/optical_study/`	the 6 direction folders of Case 1	per `dir_*`: `structure.fdf`, `calc.fdf`, `config_extra.fdf`, `calc.out`, `siesta.EPSIMG` (`dir_xx` also `siesta.EIG`, the Kohn–Sham eigenvalues)
`si/convergence/<run>/dir_xx/`	11 single-change re-runs of `dir_xx`	same files; `siesta.EPSIMG` **trimmed to 30 eV**
`graphene/a0_scan.csv`	step 1: energy vs lattice constant of graphene (minimum at 2.4815 Å)	5 points
`graphene/optical_study/`	the xx and zz folders	per `dir_*`: as above; `dir_xx` also has `siesta.EIG`

Pseudopotentials are omitted to keep the repository small (`-p dojo` / `-p virtual_vault` regenerate them). To regenerate everything yourself (the folder `optical_study/` is created by Stage 1):

```
# --- silicon ------------------------------------------------------------
mkdir relax && cp si_start.fdf relax/structure.fdf && cp calc_relax.fdf relax/calc.fdf
(cd relax && stb-pseudo -f structure.fdf -p dojo -o . && mpirun -np 2 siesta calc.fdf > calc.out)
#  ... turn relax/siesta.XV into structure.fdf (a0 = 5.4911 Ang; shipped as structure.fdf)
stb-optical -f structure.fdf -p dojo --directions xx yy zz xy xz yz --kgrid 11 11 11 \
            --optical-mesh 32 32 32 --optical-broaden 0.1 --optical-nbands 16
for d in optical_study/dir_*; do (cd $d && mpirun -np 2 siesta calc.fdf > calc.out); done
# --- graphene monolayer (virtual_vault carbon: Section 7) ------------------
stb-optical -f graphene.fdf -c calc_graphene.fdf -p virtual_vault --directions xx zz --kgrid 24 24 1 \
            --optical-mesh 72 72 1 --optical-broaden 0.1 --optical-nbands 24
for d in optical_study/dir_*; do (cd $d && mpirun -np 2 siesta calc.fdf > calc.out); done
```

Computed with SIESTA 5.4.2 (conda-forge build, GNU 14.3, MPI), `mpirun -np 2` on a 2-core machine: about 1–2 minutes per silicon direction folder, 17 s per graphene folder. The convergence runs change exactly one setting of `dir_xx` each (listed in Section 5.4). Their `siesta.EPSIMG` files are cut at 30 eV to keep the repository small, so every convergence row is analysed with `--energy-max 30` (the production row too).

11.14.1.7 Known, deliberate limitations

Most limitations are those of the method (Section 1.8); what to remember when using the numbers:

- **Independent-particle optics.** No excitons, no local fields, no quasiparticle correction: the spectrum is red-shifted by about the DFT gap error and $\varepsilon_1(0)$ is overestimated. The `--scissor` what-if is a post-processing shift of ε_2, not a GW calculation; it moves peaks rigidly and shifts high-energy features too far (Section 5.5).

- **Low-dimensional materials are the worst case**: the excitonic binding of a 2D semiconductor is hundreds of meV and the independent-particle ε_2 misses the bound exciton peaks entirely (Section 5.6). The 2D correction restores the intrinsic ε, not the physics that is absent.
- **No intraband (Drude) term, no phonon-assisted/indirect absorption, no temperature, no spin–orbit coupling.** Indirect gaps are invisible; metals lack their free-carrier response.
- **The optical mesh is the dominant numerical parameter**, and its convergence is *not* reported by the tool (it cannot know): a spiky ε_2 is a hint, but only re-running with a denser mesh proves it (Section 5.3). Peak positions from a coarse mesh can be off by tenths of an eV.
- **The basis limits the unoccupied bands and the sum rule.** SIESTA's f-sum check cannot reach 1 once the basis is exhausted: more bands do not help beyond that (the number of bands is not the limit — silicon with all 26 bands only goes from 0.870 to 0.880), a larger basis / smaller `PAO.EnergyShift` does.
- **Pseudopotentials and the relaxed geometry.** Spectra inherit the quality of the ground state; check the relaxed structure against a known value before trusting them. In this example silicon uses the PseudoDojo set, whose relaxed lattice constant (5.49 Å, +1.1 %) is in line with PBE, while for graphene the `virtual_vault` PSF carbon gives a_0 = 2.48 Å (PBE $\approx$ 2.47 Å) where the PseudoDojo carbon put the minimum near 2.50 Å with the same basis; in a separate test on another layered material a PseudoDojo pseudopotential gave a clearly wrong bond length in this SIESTA set-up (not investigated further). Bank quality is not guaranteed — verify.
- **`R` is normal-incidence reflectivity of a perfect flat surface**; α assumes a homogeneous bulk absorber.
- **2D thickness is yours to choose** and the corrected values scale with it; **1D is not corrected**; **0D** gives the dilute-limit polarizability only.
- **The input must be relaxed**: the tool evaluates the geometry as given and has no way to know the structure is not at equilibrium (a stretched cell shifts the whole spectrum).
- **Spin-polarized/magnetic systems** are not special-cased.
- **Each folder repeats the same SCF.** Independent and simple, but wasteful for many directions; the symmetry analysis tells you which ones you can skip.

11.14.1.8 Step-by-step: running this workflow on your own crystal

1. **Relax the structure** at the level of theory you will use for the optics (functional, basis, pseudopotentials). Optical spectra are extremely sensitive to the cell: a few-percent lattice error shifts every peak.
2. **Write a `calc.fdf` with the physics only**: functional, basis (TZP with `PAO.EnergyShift` $\leq$ 0.01 Ry is a good start for the unoccupied bands), `Mesh.Cutoff`, `Spin`. No k-grid, no MD directives: Stage 1 handles them (or give `--kgrid`).
3. **Run Stage 1** with the directions your symmetry requires (`stb-optical` tells you the minimal set) and read its report: the optical mesh assessment, the bands included, every advisory.
4. **Run SIESTA** in each `dir_*` folder.
5. **Run Stage 2** and read, in this order: *data quality* (SCF, the f-sum check), *system & calculation* (same settings in every folder?), then the physics.
6. **Check the symmetry consistency** of the tensor: if it fails, something differs between folders.
7. **Converge** the optical mesh first (double it until peaks stop moving), then the basis (`PAO.EnergyShift`), then bands and broadening. Re-run Stage 1 with the new value and compare the Stage 2 numbers as Section 5.4 does.
8. **Interpret with the limitations in mind**: compare edge and peak positions with experiment *including the expected DFT red shift*, use `--scissor` to see where they would land, and do not quote ε_1`(0)` or exciton-sensitive features as predictions.

11.14.1.9 Files in this folder

File	What it is
`README.md`	this tutorial
`example_5.14.sh`	guided, runnable walk-through (14 cases)
`convergence_table.py`	helper of the script: side-by-side summary of several Stage 2 runs
`structure.fdf`	relaxed bulk silicon — the main input of Stage 1
`calc.fdf`	ground-state template for the silicon optical folders
`si_start.fdf`, `calc_relax.fdf`	step 1: the experimental-lattice starting structure and the relaxation input
`graphene.fdf`, `calc_graphene.fdf`	graphene monolayer (the 2D case) and its template
`h2o.fdf`	water molecule in a box (0D fixture, Stage 1 only)
`reference_results/`	real finished SIESTA runs (Section 6)
`figures/`	the dashboards of Sections 5.2 and 5.6
`output/`	created by the script (git-ignored)

11.14.1.10 Running the script

```
bash example_5.14.sh
```

It pauses between cases (`[Press Enter to continue]`); `yes '' | bash example_5.14.sh` runs it straight through. It starts by wiping its own `output/`. Cases 1–7 run Stage 1 (no SIESTA needed); Cases 8–14 run Stage 2 on `reference_results/`. It sets `MPLBACKEND=Agg`, so `--show` is a no-op in the script; try it on a desktop:

```
stb-opticalAnalysis --directory reference_results/si/optical_study --show --scissor 0.55
```

11.14.1.11 Flag reference

`stb-optical` — `-f FILE` relaxed structure · `-c CALC` (default `calc.fdf`) · `-p PSEUDO` (bank or folder) · `--directions {xx,yy,zz,xy,xz,yz} ...` (default `xx yy zz`) · `--kgrid NX NY NZ` · `--kgrid-density D` (1/Å) · `--optical-mesh NX NY NZ` (default 10 10 10) · `--optical-broaden EV` (default 0.2) · `--optical-nbands N` · `--vacuum-gap Å` (default 10) · `-O DIR` (default `optical_study`) · `--save-report` · `--no-intro` · `-v`.

`stb-opticalAnalysis` — `--directory DIR` (default `optical_study`) · `--file NAME` (default `calc.out`) · `-l LABEL` · `--energy-min/--energy-max EV` (trim the data) · `--plot-quantity {eps,n_k,alpha,R,L,sigma1,all}` · `--experimental PATH` (+ `--peak-prominence F`) · `--dimensionality-correction` (+ `--thickness Å` for 2D) · `--energies EV ...` · `--n-peaks N` · `--smoothing EV` · `--scissor EV` · `--show` · `--png` · `--plot-emax EV` · `-o NAME` · `--save-report` · `--no-intro` · `-v`.

11.14.1.12 What's next

- For a quantitative comparison with experiment, add a quasiparticle (GW) gap or at least a scissor, and for bound excitons a Bethe–Salpeter calculation — outside this suite.
- `2.2-stb-dos` and `2.1-stb-bands` show the band structure and DOS behind the critical points that appear as peaks here; `5.5-convergence` is the same convergence discipline applied to the ground state.
- The Raman workflow (`5.11`) uses the same `Optical.*` machinery to get the dielectric derivatives of the Raman tensor.

11.14.1.13 References

1. D. E. Aspnes, A. A. Studna, *Dielectric functions and optical parameters of Si, Ge, GaP, GaAs, GaSb, InP, InAs, and InSb from 1.5 to 6.0 eV*, Phys. Rev. B **27**, 985 (1983).
2. D. R. Penn, *Wave-number-dependent dielectric function of semiconductors*, Phys. Rev. **128**, 2093 (1962).
3. Z. H. Levine, D. C. Allan, *Linear optical response in silicon and germanium including self-energy effects*, Phys. Rev. Lett. **63**, 1719 (1989).
4. R. W. Godby, M. Schlüter, L. J. Sham, *Self-energy operators and exchange-correlation potentials in semiconductors*, Phys. Rev. B **37**, 10159 (1988).
5. M. Rohlfing, S. G. Louie, *Electron-hole excitations and optical spectra from first principles*, Phys. Rev. B **62**, 4927 (2000).
6. G. Onida, L. Reining, A. Rubio, *Electronic excitations: density-functional versus many-body Green's-function approaches*, Rev. Mod. Phys. **74**, 601 (2002).
7. S. L. Adler, *Quantum theory of the dielectric constant in real solids*, Phys. Rev. **126**, 413 (1962).
8. N. Wiser, *Dielectric constant with local field effects included*, Phys. Rev. **129**, 62 (1963).
9. A. J. Read, R. J. Needs, *Calculation of optical matrix elements with nonlocal pseudopotentials*, Phys. Rev. B **44**, 13071 (1991).
10. A. Laturia, M. L. Van de Put, W. G. Vandenberghe, *Dielectric properties of hexagonal boron nitride and transition metal dichalcogenides: from monolayer to bulk*, npj 2D Mater. Appl. **2**, 6 (2018).
11. Yang and Gao, npj 2D Mater. Appl. **5**, 78 (2021) (method to restore the intrinsic dielectric functions of 2D materials in periodic calculations), as cited in the tool's `--help`.
12. J. M. Soler *et al.*, *The SIESTA method for ab initio order-N materials simulation*, J. Phys.: Condens. Matter **14**, 2745 (2002).
13. A. García *et al.*, *Siesta: Recent developments and applications*, J. Chem. Phys. **152**, 204108 (2020).
14. J. P. Perdew, K. Burke, M. Ernzerhof, *Generalized gradient approximation made simple*, Phys. Rev. Lett. **77**, 3865 (1996).
15. M. J. van Setten *et al.*, *The PseudoDojo: Training and grading a 85 element optimized norm-conserving pseudopotential table*, Comput. Phys. Commun. **226**, 39 (2018). doi:10.1016/j.cpc.2018.01.012
16. H. J. Monkhorst, J. D. Pack, *Special points for Brillouin-zone integrations*, Phys. Rev. B **13**, 5188 (1976).
17. P. Virtanen *et al.*, *SciPy 1.0*, Nat. Methods **17**, 261 (2020).
18. S. P. Ong *et al.*, *Python Materials Genomics (pymatgen)*, Comput. Mater. Sci. **68**, 314 (2013).
19. A. Togo, I. Tanaka, *Spglib: a software library for crystal symmetry search*, arXiv:1808.01590 (2018).
20. A. H. Larsen *et al.*, *The atomic simulation environment*, J. Phys.: Condens. Matter **29**, 273002 (2017).
21. sisl — `https://zerothi.github.io/sisl/` (N. Papior).
22. J. D. Hunter, *Matplotlib: A 2D graphics environment*, Comput. Sci. Eng. **9**, 90 (2007).
23. R. R. Nair *et al.*, *Fine structure constant defines visual transparency of graphene*, Science **320**, 1308 (2008).
24. L. Yang, J. Deslippe, C.-H. Park, M. L. Cohen, S. G. Louie, *Excitonic effects on the optical response of graphene and bilayer graphene*, Phys. Rev. Lett. **103**, 186802 (2009).
25. K. F. Mak, J. Shan, T. F. Heinz, *Seeing many-body effects in single- and few-layer graphene: observation of two-dimensional saddle-point excitons*, Phys. Rev. Lett. **106**, 046401 (2011).
26. A. H. Castro Neto, F. Guinea, N. M. R. Peres, K. S. Novoselov, A. K. Geim, *The electronic properties of graphene*, Rev. Mod. Phys. **81**, 109 (2009).
27. M. Fox, *Optical Properties of Solids*, Oxford University Press (2010); F. Wooten, *Optical Properties of Solids*, Academic Press (1972) — the textbook relations (5)–(9).

11.14.2 Implementation details

From ε_2 to the optical constants

SIESTA computes $\varepsilon_2(\omega)$ for light polarized along a direction $\hat{\mathbf{n}}$ from interband transitions (independent particles). The tool obtains ε_1 by Kramers–Kronig (as in Section 11.11) and, with $N = n + ik = \sqrt{\varepsilon_1 + i\varepsilon_2}$ [86, 87],

$$\alpha(\omega) = \frac{2\omega k}{c}, \qquad R(\omega) = \left|\frac{N-1}{N+1}\right|^2, \qquad L(\omega) = \mathrm{Im}\left[-\frac{1}{\varepsilon(\omega)}\right], \qquad \sigma_1(\omega) = \varepsilon_0\,\omega\,\varepsilon_2(\omega).$$

The f-sum rule counts the electrons taking part: $N_{\mathrm{eff}}(E) \propto V\int_0^E E'\,\varepsilon_2(E')\,dE'$, normalized so that it tends to the number of valence electrons per cell when all transitions are included.

Stage 1 (`stb-optical`). The input is an *already relaxed* structure (`-f`) and a template (`-c`). One folder per direction in `-directions` (default `xx yy zz`; also the bisectors `xy`, `xz`, `yz`) is written to `<output-dir>/` (default `optical_study`), each with a `config_extra.fdf` that forces: a single point (`MD.Steps 0`, `MD.NumCGsteps 0`, `MD.VariableCell false`); `Optical.Calculate T`; `Optical.PolarizationType polarized` (mandatory – SIESTA's default `polycrystal` ignores `Optical.Vector`); the direction in `Optical.Vector`; `Optical.Mesh` (`-optical-mesh`, default 10^3), `Optical.Broaden` (`-optical-broaden`, default 0.2 eV) and `Optical.NumberOfBands` (`-optical-nbands`). The SCF k-grid is forced only if the template has none (SIESTA would otherwise use Γ alone) or if `-kgrid`/`-kgrid-density` is given. The report analyses the symmetry of the dielectric tensor in the cell's own Cartesian frame – which components are independent, equal or zero – and lists the smallest set of directions needed.

Stage 2 (`stb-opticalAnalysis`). For each folder it reads `<label>.EPSIMG` and the optical block of the log, and computes ε_1, n, k, α, R, L, σ_1 on the window `-energy-min`/`-energy-max`. Off-diagonal components come from the bisectors, $\varepsilon_{ij} = \varepsilon_{\hat{n}} - (\varepsilon_{ii} + \varepsilon_{jj})/2$. Diagnostics:

- the f-sum check printed by SIESTA and $N_{\mathrm{eff}}(E)$ recomputed from ε_2;
- static constants $\varepsilon_1(0)$ and $n(0)$, values at `-energies`;
- the absorption onset, a Tauc estimate [88] and the Penn gap [89], compared with the Kohn–Sham gap read from `<label>.EIG`; metals are recognized and their low-energy "onset" labelled a k-mesh artefact;
- the `-n-peaks` main peaks (smoothing `-smoothing`) and the plasmon from L;
- anisotropy compared with the symmetry-allowed form of the tensor;
- `-scissor`: a rigid shift of ε_2 to a target gap (a what-if, not a many-body correction);
- `-dimensionality-correction` with `-thickness` for 2D layers (converting the supercell response to a sheet; ill-conditioned for the out-of-plane component and dependent on the thickness chosen).

A nine-panel dashboard (`-show`, `-png`, `-plot-quantity`, `-plot-emax`), data files (`-o`) and an optional comparison with a measured curve (`-experimental`) complete the output.

Lessons from the example. The optical k-mesh (`Optical.Mesh`, independent of the SCF grid) sets peak positions – converge it first; the broadening sets peak heights; once all bands are included, the f-sum is limited by the basis set rather than by the number of bands. Independent-particle spectra lack excitonic and local-field effects, and semilocal functionals underestimate the gap, so peaks appear at too low energies.

11.14.3 Command-line reference

11.14.3.1 `stb-optical`

Stage 1 of 2: writes one SIESTA folder per requested Cartesian/biaxial polarization direction, ready for a linear-response Optical.Calculate run. Takes an ALREADY-RELAXED structure (`--file`) and a `calc.fdf` template (`--calc`, default: `calc.fdf` in the current directory), and for each requested direction (`--directions`, default xx yy zz) writes '<output-dir>/dir_<axis>/' with the SAME structure (no new geometry – unlike this suite's other WORKFLOW_TOOLS prep stages, there's nothing to deform/search/enumerate here). Every forced directive – MD.TypeOfRun CG / MD.Steps 0 / MD.NumCGsteps 0 / MD.VariableCell false (single-point, no relaxation), that folder's own Optical.Mesh/Optical.Vector/Optical.Broaden/Optical.NumberOfBands block (SIESTA's own linear-response/RPA dielectric-function machinery – see the SIESTA manual's Optical.* directives) and, when it is REQUIRED, the ground-state SCF k-grid – is written into a `config_extra.fdf` sidecar, %include'd at the very top of `calc.fdf`, never edited into your own `--calc` template text directly (this suite's standard `config_extra.fdf` convention – see the manual). The SCF k-grid is forced when your template defines none (SIESTA would sample Gamma only; a grid is suggested at density 0.2 1/Ang, vacuum axes getting 1 division) or when you ask for one (`--kgrid` X Y Z, or `--kgrid-density` D); a template that already has a grid keeps it. Doesn't run SIESTA – run each folder yourself, then use `stb-opticalAnalysis` (which aggregates every direction folder present into one combined report/spectrum, so run it once against the whole `--output-dir` root, not once per folder). [DIAGONAL vs. BIAXIAL DIRECTIONS] 'xx'/'yy'/'zz' set Optical.Vector to the pure Cartesian unit axis (e_x/e_y/e_z) – the diagonal dielectric-tensor component eps_ii comes directly from that one folder. 'xy'/'xz'/'yz' set Optical.Vector to the NORMALIZED BISECTOR of the two axes involved (e.g. xy -> (e_x+e_y)/sqrt(2)) – SIESTA's Optical.Vector always computes a single-direction response n^T.eps.n, never a genuine off-diagonal component directly, so the bisector is the standard trick to reach one: for a symmetric dielectric tensor, response along that bisector expands to (eps_ii + eps_jj)/2 + eps_ij, so once eps_ii, eps_jj AND the bisector's own response are all known, eps_ij = eps_(bisector ij) - (eps_ii + eps_jj) / 2 recovers the true off-diagonal component. Requesting 'xy' without also requesting 'xx'/'yy' in the same (or an earlier) run still writes a valid folder – it's simply not reconstructible into eps_xy until the 2 diagonal folders it needs also exist; the Stage 1 report flags this per direction. Stage 2 (`stb-opticalAnalysis`) auto-detects each folder's direction from its own Optical.Vector value (not the folder name) and, whenever a biaxial direction's 2 matching diagonal folders are ALSO present in the same run, applies the reconstruction formula above automatically – eps_ij(E) is written to its own dedicated output alongside the raw per-direction eps_ii(E)/eps_jj(E) results, no extra flag needed. A biaxial folder without its matching diagonal pair is still read normally by Stage 2 (its raw eps_(bisector)(E) response is real data), just not reconstructible into eps_ij yet. [KNOWN LIMITATION, 2D/slab inputs] No direction is skipped or specially treated for a vacuum-padded input – every requested direction (including out-of-plane and any biaxial one involving it) is still written and computed. For a slab, the periodic-supercell dielectric response along ANY direction is diluted by the vacuum region (a well-known methodological caveat for slab dielectric/polarizability DFT calculations) – no vacuum-thickness rescaling is applied here. Interpret out-of-plane 2D results with this in mind.

Synopsis.

```
usage: stb-optical [-h] -f FILE [-c CALC] [-p PSEUDO_DIR]
                   [--directions {xx,yy,zz,xy,xz,yz} [{xx,yy,zz,xy,xz,yz} ...]]
                   [--optical-mesh NX NY NZ] [--optical-broaden EV]
                   [--optical-nbands N] [--kgrid NX NY NZ] [--kgrid-density D]
                   [--vacuum-gap VACUUM_GAP] [-O OUTPUT_DIR] [--save-report] [-v]
                   [--no-intro]
```

Options

`-f, --file FILE`
Already-relaxed input `structure.fdf`. required

`-c, --calc CALC`
`calc.fdf` template for the Optical calculations (default: `calc.fdf`).

`-p, --pseudo-dir PSEUDO_DIR`
Pseudopotentials source: a bundled bank or a folder path.

`--directions {xx,yy,zz,xy,xz,yz} [{xx,yy,zz,xy,xz,yz} ...]`
Direction(s) to compute (default: xx yy zz). 'xx'/'yy'/'zz' are the diagonal Cartesian components (Optical.Vector = a unit axis, eps_ii read directly). 'xy'/'xz'/'yz' are BIAXIAL directions (Optical.Vector = the normalized bisector of the 2 axes, e.g. (1,1,0)/sqrt(2) for xy) – the off-diagonal eps_ij component is only reconstructible from these once the matching xx/yy/zz pair also exists; see the description above for the exact formula. choices: xx, yy, zz, xy, xz, yz

`--optical-mesh NX NY NZ`
Optical.Mesh k-point sampling for the optical-transition integral (default: 10 10 10). type: int

`--optical-broaden EV`
Optical.Broaden Gaussian broadening in eV (default: 0.2). type: float

`--optical-nbands N`
Optical.NumberOfBands (default: SIESTA's own default – all bands). type: int

`--kgrid NX NY NZ`
Ground-state SCF k-grid forced into every folder's `config_extra.fdf` (e.g. `--kgrid 8 8 8`). Overrides the template's own grid. Without `--kgrid` or `--kgrid-density` the template's grid is kept; if the template has none, a grid is suggested automatically (density 0.2 1/Ang). type: int

`--kgrid-density D`
Suggest the SCF k-grid from a target density in 1/Ang (same convention as `stb-kgrid`; vacuum axes get 1 division) and force it into `config_extra.fdf`, overriding the template's grid. Ignored when `--kgrid` is given. type: float

`--vacuum-gap VACUUM_GAP`
Vacuum-axis detection threshold in Ang (vacuum axes get a single SCF k-point division and drive the dimensionality notes; default: 10.0). type: float

`-O, --output-dir OUTPUT_DIR`
Root directory for the whole optical-properties workflow (default: optical_study).

`--save-report`
Also persist the report to <output-dir>/optical_stage1.txt. Off by default. switch, off by default

Like every STB command it also accepts `-h`/`--help`, `-v`/`--version` and `--no-intro` (Section 3.3).

From the built-in help.

```
Usage examples:
  stb-optical -f relaxed.fdf -p dojo
  stb-optical -f relaxed.fdf -p dojo --directions xx yy zz xy xz yz
```

11.14.3.2 `stb-opticalAnalysis`

Stage 2 of 2: aggregates every direction folder written by `stb-optical` and derives the full set of linear optical properties. Scans `--directory` for every subfolder containing a `calc.fdf`, identifies each one's direction from its own %block Optical.Vector (not the folder name – robust to renamed/reorganized folders; 'xx'/'yy'/ 'zz' diagonal or 'xy'/'xz'/'yz' biaxial, the same convention `stb-optical` writes), reads its `SystemLabel.EPSIMG`, and derives eps1(E)/eps2(E) (Kramers-Kronig transform), n(E)/k(E) (refractive index/extinction coefficient), the absorption coefficient, normal-incidence reflectivity, the electron energy-loss function, and the real optical conductivity – all standard textbook relations (Wooten/Fox, "Optical Properties of Solids"), not flagged [UNVERIFIED]. Missing or incomplete direction folders are skipped with a warning, never block the directions that ARE ready (advisory only, same convention as this suite's other multi-folder analysis stages). [OFF-DIAGONAL RECONSTRUCTION] Whenever a biaxial direction ('xy'/'xz'/'yz') AND its 2 matching diagonal directions are all present, also reconstructs the true off-diagonal dielectric-tensor component eps_ij(E) = eps_(bisector ij)(E) - (eps_ii(E) + eps_jj(E)) / 2 (same bisector-based formula raman_analysis.py's own raman_tensor_full uses for the Raman tensor's Rxy/Rxz/Ryz) and writes it to its own dedicated output – eps1_ij/eps2_ij only, NOT n/k/alpha/R/L/sigma1 (those textbook relations assume a principal-axis/diagonal response and have no standalone meaning for a genuine off-diagonal tensor component). [ASSUMPTION] Reflectivity R(E) is normal-incidence, vacuum(n0=1)/material interface – no angle- dependent Fresnel formula or thin-film interference is computed. [KNOWN LIMITATION, 2D/1D/0D inputs] The raw supercell values above are diluted by the vacuum region and NOT corrected by default – pass `--dimensionality-correction` (see its own `--help` entry) to additionally restore the intrinsic 2D dielectric function or extract a 0D molecular polarizability, written ALONGSIDE the raw values, never replacing them (1D is not yet implemented – see `--dimensionality-correction's` own help text; this correction is only ever applied to the 3 diagonal directions, never to a biaxial/reconstructed one). When all 3 of xx/yy/zz are present, also computes and reports/plots an isotropic ('avg') average – eps2 averaged across the 3 directions first, then every other quantity (including eps1) re-derived from that averaged eps2 – the standard convention for comparing against a polycrystalline/powder experimental measurement, which doesn't correspond to any single crystallographic direction. [REPORT] System/ground state and per-folder Optical settings read back from the folders; data quality (SCF, SIESTA's f-sum check, the f-sum rule recomputed from eps2 as N_eff(E)); static eps1/n/R, absorption onset, Tauc direct gap and Penn gap; peaks of eps2 and the plasmon; anisotropy and a symmetry-consistency check of the tensor; optical constants at chosen energies; a plain-language interpretation. Optional: `--show/--png` (9-panel matplotlib dashboard), `--scissor` (post-processing what-if for the DFT gap error), `--energies`, `--n-peaks`.

Synopsis.

```
usage: stb-opticalAnalysis [-h] [--directory DIRECTORY] [--file FILE] [-l LABEL]
                           [--energy-min EV] [--energy-max EV]
                           [--plot-quantity {eps,n_k,alpha,R,L,sigma1,all}]
                           [--experimental PATH] [--peak-prominence FRACTION]
                           [--dimensionality-correction] [--thickness ANG] [-o OUTPUT]
                           [--save-report] [-v] [--no-intro] [--show] [--png]
                           [--plot-emax EV] [--energies EV [EV ...]] [--n-peaks N]
                           [--smoothing EV] [--scissor EV]
```

Options

`--directory DIRECTORY`

Root directory written by `stb-optical` (default: optical_study).

`--file FILE`
: SIESTA output filename inside each direction folder, used only for the SCF-convergence advisory check (default: `calc.out`).

`-l, --label LABEL`
: SystemLabel override (default: auto-detected per folder).

`--energy-min EV`
: Trim the reported/plotted energy range from below (default: no trim). type: float

`--energy-max EV`
: Trim the reported/plotted energy range from above (default: no trim). type: float

`--plot-quantity {eps,n_k,alpha,R,L,sigma1,all}`
: Which quantity the combined .gplot overlays across every direction present (default: eps – plots eps1 AND eps2 together). 'all' writes one .gplot per quantity in a single run (mutually exclusive with `--experimental` – pick one specific quantity for a direct comparison instead). choices: eps, n_k, alpha, R, L, sigma1, all

`--experimental PATH`
: 2-column (energy_eV, value) experimental spectrum (e.g. UV-Vis-NIR absorbance/reflectance) to overlay on the plot – only valid with a single-valued `--plot-quantity` (alpha/R/L/sigma1), not eps/n_k/all. Peak positions are matched against the isotropic average (if all 3 directions are present) or the first available direction otherwise.

`--peak-prominence FRACTION`
: Minimum peak prominence, as a fraction of that spectrum's own max value, for `--experimental`'s peak-finding (default: 0.01). type: float

`--dimensionality-correction`
: Restore the intrinsic (vacuum-independent) response from the raw vacuum-padded supercell result. 2D (1 vacuum axis): rescales eps(E) via the D-field/E-field continuity argument (Laturia et al., npj 2D Mater. Appl. 2, 6, 2018; Yang & Gao, npj 2D Mater. Appl. 5, 78, 2021) – requires `--thickness`. 0D (3 vacuum axes, an isolated molecule): extracts the molecular polarizability alpha(E) via the dilute-limit Clausius-Mossotti relation – no extra input needed (uses the cell volume). 1D (2 vacuum axes): NOT implemented – the literature approach (Maxwell-Garnett effective medium) is not a simple generalization of the 2D formula and has not been verified for this tool; prints a [NOTE] and reports only the raw supercell values. 3D (bulk): no-op, nothing to correct. Off by default – always written ALONGSIDE (never replacing) the raw supercell .dat/.csv files. switch, off by default

`--thickness ANG`
: Physical thickness of the 2D material in Ang, required together with `--dimensionality-correction` for a 2D (1 vacuum axis) input. Deliberately not auto-estimated – 'thickness' of a 2D material is not a single well-defined quantity (atomic span? + van der Waals radii? bulk interlayer spacing?), left to the user's own judgment.type: float

`-o, --output OUTPUT`
: Base filename (no extension) for the .csv/.dat/.gplot outputs (default: optical_results).

`--save-report`
: Also persist the report to <directory>/optical_stage2.txt. Off by default. switch, off by default

`--show`
: Open the analysis dashboard (matplotlib, 9 panels) on screen. Needs a display. switch, off by default

`--png`
: Save the dashboard as <directory>/<output>_summary.png. switch, off by default

`--plot-emax EV`
: Upper energy of the dashboard's x axes (default: the data range, at most 20 eV). `--energy-min/--energy-max` instead trim the DATA used by every analysis. type: float

`--energies EV [EV ...]`
: Photon energies (eV) of the 'optical constants at selected energies' table (0 = static limit; default: 0 1 2 3 4 5 10). type: float

`--n-peaks N`
: Number of eps2 peaks listed in the spectral-features table (default: 6). type: int

`--smoothing EV`
: Gaussian sigma (eV) used only to locate peaks/zero crossings in a spiky spectrum (default: 0.05). type: float

`--scissor EV`
: What-if scissor shift (eV) applied in post-processing to the reference spectrum: eps2'(E) = eps2(E - EV), eps1 re-derived by Kramers-Kronig; reports the effect on the static constant, edge, peaks and plasmon and overlays it on the dashboard. type: float

Like every STB command it also accepts `-h`/`--help`, `-v`/`--version` and `--no-intro` (Section 3.3).

From the built-in help.

```
Usage examples:
  stb-opticalAnalysis --directory optical_study --plot-quantity alpha
  stb-opticalAnalysis --directory optical_study --plot-quantity alpha --experimental uvvis.dat
  stb-opticalAnalysis --directory optical_study --plot-quantity all
  stb-opticalAnalysis --directory optical_study --dimensionality-correction --thickness 3.13
```

11.15 Equation of State

At a glance

Menu code: 5.15 (a multi-stage workflow)

Code	Stage	Command
5.15.1	Stage 1 - Prep	`stb-eosInputs`
5.15.2	Stage 2 - Analysis	`stb-eosAnalysis`

Guided example: `examples/5.15-eos/`

Summary: Generate isotropically-scaled-volume structures, then fit an E vs V equation of state (V0/E0/B0/B0') from the resulting SIESTA energies.

11.15.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/5.15-eos/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

This workflow has 2 stages: **Stage 1** (`stb-eosInputs`, code `5.15.1`) takes an **already relaxed** bulk structure and a `calc.fdf` template and writes one ready-to-run SIESTA folder per scanned volume — the cell scaled isotropically, your template untouched, a `config_extra.fdf` that fixes the cell and makes every folder a single point, and the pseudopotentials. **Stage 2** (`stb-eosAnalysis`, code `5.15.2`) reads the finished runs back and fits an **equation of state** `E(V)` — equilibrium volume `V0`, energy `E0`, bulk modulus `B0` and its pressure derivative `B0'` — plus the quantities that tell you whether to trust it: SIESTA's own pressure (from the stress tensor) against `−dE/dV` of the fit, a second bulk modulus from the stress alone, leave-one-out uncertainties, residuals, the consistency of the level of theory across folders, the suggested new lattice constant, a pass/warn table of checks, and a 6-panel dashboard.

It answers a question none of 5.1–7.1 does in this form: not how a structure responds to a small strain (that is 5.2 elastic constants), nor what a single relaxed cell costs (5.3 cohesive energy), but **how the energy of the crystal depends on its volume over a finite range** — the curve from which the lattice constant, the compressibility, the pressure at any volume and the volume at any pressure follow. The worked example (Section 5) runs the whole thing for real, with SIESTA, on bulk silicon: a 14-setup convergence study (k-points, mesh, basis) and the comparison with an all-electron calculation, which turns out to teach more about how to judge a basis set than about silicon.

Both stages live in this one folder and this one tutorial — same reasoning as 5.1–5.5, 5.13 and 5.14: Stage 1's output exists only to feed Stage 2.

11.15.1.1 1. Theory

Symbols at a glance

Symbol	Meaning
`V`, `V0`	cell volume; its value at the energy minimum (Å^3; per atom when stated)
`E(V)`, `E0`	total energy of the cell at volume `V`; its minimum (eV)
`P = −dE/dV`	pressure of the static lattice (GPa; `1 eV/Å³ = 160.2177 GPa`)
`B = −V dP/dV = V d²E/dV²`	bulk modulus at volume `V`; `B0 = B(V0)` (GPa)
`B0' = dB/dP`	pressure derivative of the bulk modulus at `P = 0` (dimensionless)
σ	SIESTA's stress tensor (eV/Å^3): positive means the cell is under tension
`P_SIESTA = −tr(σ)/3`	the pressure each run reports through its own stress
`x = V0/V`	compression ratio
`f = [(V0/V)^(2/3) − 1] / 2`	Eulerian finite strain
Δ	Delta-test distance between two equations of state (meV/atom)
`N`	number of atoms of the cell

1.1 Why an equation of state. At zero temperature and with the nuclei held at their lattice sites (the *static lattice*), a crystal at volume `V` has the energy `E(V)` obtained by relaxing everything else. That one function holds:

- the **equilibrium volume** `V0`, hence the lattice constant of the material at the level of theory used;
- the **cohesive-energy minimum** `E0` (a reference for energy differences between phases);
- the **bulk modulus** `B0` — how hard it is to compress — and its pressure derivative `B0'` — how fast it hardens;
- the **pressure** at any volume, `P = −dE/dV`, and by inversion the **volume under any pressure**.

It is the input of many other calculations: the quasi-harmonic approximation (`stb-mlphonons --qha` scans exactly this volume range), thermal expansion, the transition pressure between two phases (the crossing of their enthalpies, Section 1.10), the volume at which a high-pressure experiment is to be simulated. And it is the cheapest honest test of a SIESTA setup: pseudopotential, basis, mesh cutoff and k-sampling all leave a mark on `V0` and `B0` that can be compared with experiment and with other codes (Section 1.8).

One SIESTA run gives one point `E(V)`. A scan of 9–11 volumes and a fit gives the curve.

1.2 Thermodynamics in four lines. For the static lattice the pressure, the bulk modulus and its derivative are

```
P(V)  = −dE/dV                                                    (1)
```

```
B(V)  = −V dP/dV = V d²E/dV²                                                    (2)
B0'   = dB/dP at P = 0                                                          (3)
H(P)  = E(V) + P V      (enthalpy; at T = 0 it is the Gibbs energy)             (4)
```

At zero external pressure the equilibrium volume satisfies `dE/dV = 0`; under an external pressure `P` it satisfies `dE/dV = −P`, i.e. it minimizes `H`. Near the minimum a parabola is the first approximation,

```
E(V) ≈ E0 + (B0 / 2 V0) (V − V0)²          so       B0 ≈ V0 · d²E/dV²          (5)
```

and its failure — the curve is stiffer when compressed than when expanded — is exactly what `B0'` measures (about 4 for most solids: Section 5.2 finds 4.1–4.3 for silicon). A solid's `B` rises with pressure; the energy is therefore *not* symmetric about `V0`, and a fit that forgets it returns a wrong `B0`.

1.3 What the SIESTA runs give, and how the cell is scaled. Each folder is one SCF calculation at a fixed cell: the total (free) energy `E`, the atomic forces and the stress tensor σ come out of the same run. This workflow scales the cell **isotropically**,

```
L = L0 · (1 + ε_V / 100)^(1/3)          on every lattice vector                 (6)
```

with `ε_V` the volume strain in percent (from `−strain_range` to `+strain_range`), the same convention as `stb-mlphonons --qha`. The *shape* of the cell (its angles and axis ratios) is kept, and the atomic positions are fractional, so they scale for free: for a crystal whose atoms have no free internal coordinates (diamond, fcc, bcc, rocksalt...) every folder is already the relaxed structure at its volume, and one SCF each is enough (**single point**, the default). If the crystal has free internal coordinates (wurtzite, rutile, quartz...), `--relax-positions` lets the ions relax at each volume, *with the cell fixed*: letting the cell relax would undo the scan.

Two routes to the pressure exist inside every run: the derivative of the energy across the scan (Eq. 1) and the stress tensor of each run,

```
P_SIESTA = −tr(σ) / 3                                                           (7)
```

(σ `> 0`: the cell is under tension, it wants to shrink; so a cell *larger* than `V0` has σ `> 0` and `P < 0`). Stage 2 compares them (Section 1.6).

1.4 Equation-of-state forms. An equation of state is a function `E(V; E0, V0, B0, B0')` with four parameters, fitted by least squares to the scanned points. The ones `stb-eosAnalysis` offers (via ASE's `EquationOfState`, [15]):

Murnaghan [1] — assumes the bulk modulus is linear in pressure, `B(P) = B0 + B0' P`:

```
E(V) = E0 + (B0 V / B0') [ (V0/V)^B0' / (B0' − 1) + 1 ] − B0 V0 / (B0' − 1)      (8)
P(V) = (B0 / B0') [ (V0/V)^B0' − 1 ]
```

Birch–Murnaghan, third order [2] — the finite-strain (Eulerian) expansion of the energy carried to third order in `f`:

```
E(V) = E0 + (9 V0 B0 / 16) { [(V0/V)^(2/3) − 1]³ B0' + [(V0/V)^(2/3) − 1]² [6 − 4 (V0/V)^(2/3)] }   (9)
     = E0 + (9/2) V0 B0 f² + (9/2) V0 B0 (B0' − 4) f³
P(V) = (3 B0 / 2) (x^(7/3) − x^(5/3)) { 1 + (3/4)(B0' − 4)(x^(2/3) − 1) },        x = V0/V
```

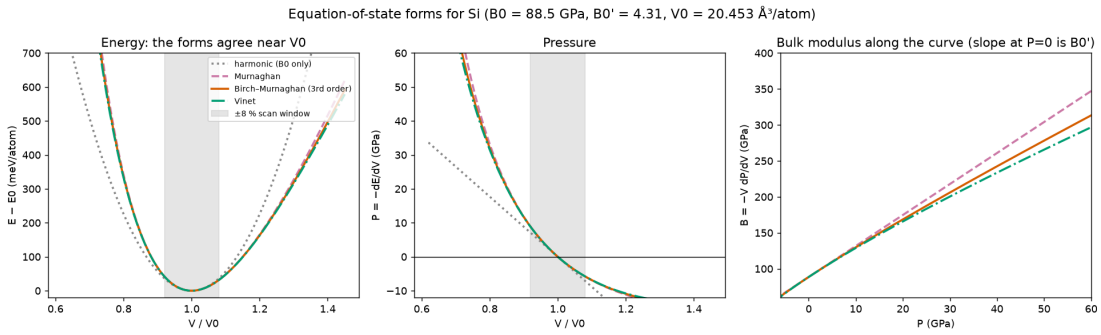
It is the default of the suite and of the Delta test [7]. Setting `B0' = 4` removes the cubic term: the *second-order* Birch–Murnaghan form.

Vinet [3] — the "universal" form derived from an exponential interatomic potential, better behaved than Birch–Murnaghan for strong compression:

> `E(V) = E0 + (9 B0 V0 / ξ²) { 1 + [ξ (1 − X) − 1] exp[ξ (1 − X)] },  X = (V/V0)^(1/3),  ξ = (3/2)(B0' − 1)` (10)

Stabilized jellium (sj) [5] — a polynomial in `V^(−1/3)`, motivated by the stabilized-jellium model; ASE's fit returns `V0`, `E0` and `B0` but no `B0'`, which the report then prints as `-`.

Near the minimum all four coincide to within the numerical noise of the SCF: with the parameters of silicon, Birch–Murnaghan and Vinet differ by at most 0.05 meV/atom inside a ±8 % window of volume (Murnaghan, the stiffest, by 0.11), by 0.8 and 1.7 meV/atom at ±15 %, by 3 and 6 meV/atom at ±20 %, and by tens of meV beyond ±30 % (Fig. 1). That is why `--eos all` is a *robustness check of the fit*, not a physical choice: if the forms agree on `B0` (the report tests 1 %), the result does not depend on the functional form; if they do not, the scan is too wide or too narrow. `B0'` is the poorly determined parameter: it controls the *asymmetry* of the curve, which a ±8 % window barely samples.



Equation-of-state forms for silicon: energy, pressure and bulk modulus

Fig. 1 — The four equation-of-state forms for silicon's parameters (all-electron reference `V0 = 20.453 Å³/atom`, `B0 = 88.5 GPa`, `B0' = 4.31`; `theory_figures.py`). The grey band is the ±8 % scan of the example. The harmonic approximation (Eq. 5) is the dotted parabola.

1.5 Fitting, and how sure the numbers are. The fit is a non-linear least squares in the **energies**, `min Σ [E_i − E(V_i; E0, V0, B0, B0')]²`. Energies — not pressures — are fitted because they are the quantity SIESTA converges to the tightest tolerance; the pressure of each run is kept as an independent check (Section 1.6). Four parameters need at least 4 volumes; the tool asks for 5 and defaults to 9. A scan has to satisfy three practical conditions:

- it must **bracket the minimum**: points on both sides of `V0`, at least two each — otherwise `V0` is an extrapolation and `B0` is dominated by one branch of the curve;
- it must be **wide enough** that the curvature stands above the SCF noise (a few percent in volume is the usual lower bound), and **narrow enough** to stay where the chosen form is accurate (up to ~10 % is typical; Section 1.4);
- an **odd number** of volumes keeps the input volume itself in the scan.

Uncertainty. The report gives the **leave-one-out** (jackknife) standard error of `V0` and `B0`: the fit is repeated with each point removed in turn, and

$$\sigma_\text{jack} = \text{sqrt}\left((n-1)/n \cdot \Sigma_i \, (\theta_i - \theta)^2 \right) \qquad (11)$$

with θ_i the parameter without point `i` [21]. It measures how much a single run can move the answer — it is a statement about the *scan*, not about the physics: it says nothing about the basis, the pseudopotential or the functional, which usually dominate (Section 5.5). A second diagnostic, the **window sensitivity**, refits with the central 60 % of the points only: a `B0` that moves by more than ~2 % means the answer depends on how far from `V0` you look, i.e. the curve is not described by the chosen form over the whole range.

The parabolic limit (Eq. 5) is the by-hand check of Section 1.12.

1.6 Two routes to the pressure: the stress and `−dE/dV`. SIESTA prints the stress tensor of every run. For an atom-centred basis whose orbitals keep their *shape and range* when the cell is scaled, the stress is the exact derivative of the energy with respect to the strain — apart from the real-space mesh (Section 1.7) and the SCF tolerance — so `P_SIESTA` (Eq. 7) must coincide with `−dE/dV` (Eq. 1) evaluated on the fitted curve. Stage 2 prints both for every volume and flags the maximum difference. This is a strong test: two different observables of the same SCF calculation, one of them not used in the fit.

It is also worth contrasting with plane-wave codes: there the number of plane waves changes with the volume at a fixed cutoff, so the stress and `−dE/dV` differ by the *Pulay stress* (finite-basis-set correction [6]) unless the cutoff is converged or the number of plane waves is held constant. SIESTA's orbitals are tied to the atoms, so this particular inconsistency is absent; the residual disagreement points to the mesh, the SCF or a poor fit.

The stress gives a **second bulk modulus** from the same runs: fit `P_SIESTA(V)` with a cubic polynomial (a quadratic over ±8 % truncates the curvature of `P(V)` and overestimates `B` by ~2 %) and take `B = −V dP/dV` at `V0` (the report calls it *B0 from the stress alone*). It uses no energy at all; a `B0` that disagrees by more than a few percent with the energy route is a diagnostic, not a competition.

1.7 What moves `V0` and `B0`: the numerical settings. Everything below is a *convergence parameter* of the SIESTA calculation, not a physical one; the point of the example (Section 5) is to measure how much each moves the answer.

- **K-point sampling.** A semiconductor with a gap converges quickly (smooth integrand); a metal needs a much denser grid and a smearing (`ElectronicTemperature`), and its `E(V)` is correspondingly noisier. The grid is the *same* in every folder: a scaled cell with the same divisions samples the scaled Brillouin zone consistently.
- **Mesh cutoff.** SIESTA evaluates the Hartree and exchange-correlation terms on a real-space grid whose spacing is set by `Mesh.Cutoff`. A scaled cell at the same cutoff gets a grid of *another integer size*, so `E(V)` carries a small, irregular noise (the egg-box effect [10]) that a finer mesh reduces. It acts on the energy and, slightly differently, on the stress: it is the main source of any residual `P_SIESTA` $\neq$ `−dE/dV`.
- **Basis set.** Numerical atomic orbitals (NAOs) of finite range. Their size (`PAO.BasisSize`: SZ < DZ < DZP < TZP...) and, above all, their **radii**, set by `PAO.EnergyShift` (the energy a confined orbital is allowed to rise: a *smaller* shift means *longer* orbitals) change the equilibrium volume and the stiffness. Short orbitals overlap little: the basis functions of neighbouring atoms lend variational freedom to one another more when the atoms are compressed (the basis-set superposition error), which favours small volumes. Hence a poor or short basis tends to give a `V0` that is too small and a `B0` that is too large; a longer basis moves both toward the complete-basis value, and — as the example shows — may overshoot. Basis convergence is the slowest of the three.
- **Pseudopotential and functional.** Norm-conserving PseudoDojo pseudopotentials [13] with PBE [12] are used here. Any other choice changes `V0` and `B0` systematically: that is the physics of the approximation, not a convergence error.

1.8 Comparing with experiment and with an all-electron calculation. Two comparisons, with different meanings.

Against experiment. A static-lattice DFT calculation at 0 K differs from a laboratory measurement by (i) the functional's own error — PBE overestimates lattice constants by about 1 % and underestimates bulk moduli by about 10 % for silicon (Section 5.6) — and (ii) the thermal and zero-point motion the experiment contains and the static lattice does not. For silicon the zero-point correction is small; for light elements (diamond) it matters. So agreement within a few percent of experiment says the setup is sane, not that it is converged.

Against an all-electron calculation of the *same functional.* This isolates the numerical error of the pseudopotential-plus-basis machinery from the physics of PBE. The standard tool is the **Delta test** [7, 8, 9]: the root-mean-square energy difference between the two equations of state, integrated over ±6 % of the mean volume,

$$\Delta = \mathrm{sqrt}\Big(\int [E_a(V) - E_b(V)]^2 \, dV \;/\; (0.12\, V0) \Big), \qquad V0 = (V0,a + V0,b) \;/\; 2 \tag{12}$$

with each curve referred to its own minimum (this is how ASE's `deltacodesdft.delta` and the `calcDelta` package evaluate it). Δ in meV/atom condenses `V0`, `B0` and `B0'` into one number. In the Delta test, the reference is the all-electron WIEN2k calculation; for silicon its numbers are `V0 = 20.4530 Å³/atom`, `B0 = 88.545 GPa` and `B0' = 4.31` ([9], file `WIEN2k.txt`). The example (Section 5.6) uses exactly this reference.

Why `B0` alone is a poor figure of merit. `B` depends strongly on the volume at which it is evaluated: `dB/dV` $\approx$ `−B0 B0'/V0`, so a `V0` that is wrong by 1 % moves `B0` by `B0'` % $\approx$ 4 % (about 4 GPa for silicon) *in the other direction.* A basis that makes `V0` too large and `B` at a given volume unchanged will look as though it fixed `B0`. `V0`, `B0` at the same volume and Δ must be read together.

1.9 Non-cubic crystals and internal coordinates. The isotropic scaling (Eq. 6) keeps the shape of the cell. For a cubic crystal that is the whole story. For a hexagonal, tetragonal or lower-symmetry one it is a *constrained path*: `c/a` (and any angle) stays at the input's value, although the fully relaxed solid under pressure would change it. The `B0` obtained is the modulus along that path, not the hydrostatic bulk modulus — the two differ by an amount that depends on how soft `c/a` is (large for layered materials). The report prints a note whenever the crystal is not cubic. To get the hydrostatic curve, relax the shape at each volume (SIESTA's `MD.VariableCell` with `Constant.Volume`, outside this workflow), or at least relax `c/a` at `V0` first and scan from there.

Internal coordinates are a separate issue: with free coordinates a single point is not the relaxed energy at that volume, so use `--relax-positions`; the report's residual-force check flags a scan that needs it.

1.10 Target pressure, enthalpy and phase stability. The fit is also a pressure–volume relation. `--target-pressure P1 P2 ...` inverts `P(V) = −dE/dV` of the fit and reports, for each pressure, the volume and the linear scale `L/L(V0) = (V/V0)^(1/3)` of the cell — for example the lattice constant to use for a calculation at 5 GPa. Pressures outside the range the scan covered (`P(V)` at the ends of the scanned volumes) are flagged **EXTRAPOLATED**: there the equation of state's *form* decides the answer, not the data.

A transition between two phases at `T = 0` happens where their enthalpies cross, `H_1(P) = E_1(V_1) + P V_1 = H_2(P)`: run this workflow for each phase with the same settings and compare `H(P)` — the common tangent construction in `E(V)`.

1.11 What this workflow simplifies — explicitly.

- **Static lattice**: no zero-point energy, no thermal pressure; it is the 0 K curve. Finite temperature needs phonons (`5.4`, `stb-mlphonons --qha`).

- **Isotropic scaling only**: Section 1.9.
- **Four-parameter empirical forms**: none of them is exact; their agreement is evidence, not proof (and `B0'` is the weakest parameter).
- **The jackknife** quantifies the scan's sensitivity to single points, not the systematic error of the setup.
- **Single point by default**: valid only when atoms have no free internal coordinates.
- **No automatic convergence**: the tool reports what a setting does (Section 5); you decide how far to push it.
- **One SIESTA output file per folder** (`calc.out`) is the source of energy, volume and stress.

1.12 A calculation you can check by hand. Take three consecutive rows of the table of the example's base scan (Section 5.2) — the volume of the input cell and its two neighbours:

V (Å^3)	E (eV)
40.2401	−229.937172
40.8944	−229.938064
41.5487	−229.933014

With a step `h = 0.6543 Å`^{3}, the second finite difference is

$$E'' \approx (E_- + E_+ - 2\,E_0) / h^2 = (-229.937172 - 229.933014 + 2\cdot 229.938064) / 0.6543^2 = 0.013879 \text{ eV/Å}^6$$
$$B(40.894) = V\,E'' = 40.894 \cdot 0.013879 \cdot 160.2177 = 90.9 \text{ GPa}$$

and the central difference of the energy gives the pressure,

$$P(40.894) = -(E_+ - E_-)/(2h) \cdot 160.2177 = -0.509 \text{ GPa}$$

For comparison, the full Birch–Murnaghan fit of the 11 points gives `B = 92.0 GPa` and `P = −0.54 GPa` at that volume, SIESTA's own stress gives `P_SIESTA = −0.52 GPa`, and the parabola through the three points above gives `V0 = 40.666 Å`^{3} against `40.659 Å`^{3} of the fit. Three numbers, no software: `B` is right to ~1 %, the pressure to 0.03 GPa, and the equilibrium volume to 0.02 %. The `1 %` in `B` is the asymmetry of the curve (`B0'`) that the parabola ignores.

11.15.1.2 2. Libraries and external dependencies used

- **SIESTA** [10, 11] itself is *not* called by the suite. It does the real work: the SCF calculation of each volume, writing `calc.out` with the free energy, the cell (`outcell:` block), the forces and the stress tensor. The reference data of Section 5 was produced with SIESTA 5.4.2, GGA-PBE [12], PseudoDojo norm-conserving pseudopotentials [13] and Monkhorst–Pack k-grids [14].
- **ASE** [15] — `ase.eos.EquationOfState` fits the four forms (`stb.core.eos_fit`); `ase.utils.deltacodesdft.delta` evaluates Eq. 12 in `convergence_table.py`; `ase.neighborlist` gives the shortest bond.
- **numpy** / **SciPy** [16] — the cubic splines that turn the fitted curve into `P(V)` and `B(V)`, the polynomial fits of the pressure, the leave-one-out refits.
- **pymatgen** [17] / **spglib** [18] — space group and crystal system, density, structure reading.
- **matplotlib** [19] — the dashboard (`--png` / `--show`). The `.dat/.gplot` pair (`--save-gnuplot`) is for gnuplot, which the suite never invokes.

11.15.1.3 3. Stage 1: generating the volume folders (`stb-eosInputs`, code `5.15.1`)

3.1 What it does. `stb-eosInputs` takes an **already relaxed** structure (`-f`, fractional coordinates, bulk only) and your `calc.fdf` template (`-c`, default `./calc.fdf`) and, for each of `--n-volumes` volumes between `---strain-range` and `+--strain-range` percent, writes

```
<output-dir>/vol_<pct>/
    structure.fdf        the structure, ONLY %block LatticeVectors scaled (Eq. 6)
    calc.fdf             '%include config_extra.fdf' + your template, untouched
    config_extra.fdf     the forced directives (Section 3.2)
    <species>.psml       the pseudopotentials (-p: a bundled bank or a folder)
<output-dir>/eos_scan.json   the plan: structure, volumes, forced directives
```

It does not run SIESTA. Run each folder (`mpirun -np N siesta calc.fdf > calc.out`) and call Stage 2. Pseudopotentials follow the suite's convention (`-p dojo`, `-p virtual_vault` or a folder path; the same as `stb-optical`, `stb-inputfile`, ...), and the interactive menu asks for them with the same prompt.

3.2 `config_extra.fdf`: everything mandatory, the template untouched. The scan only makes sense if every folder is *one* SCF at *its* fixed cell. Those requirements are written into each folder's `config_extra.fdf`, and `calc.fdf` starts with `%include config_extra.fdf` followed by your template byte for byte. SIESTA's fdf reader is first-occurrence-wins, so the forced values beat anything the template says — even a template meant for a variable-cell relaxation:

Directive	Forced	Why
`MD.VariableCell`	`false`	the cell is the scan variable: it must never relax
`MD.TypeOfRun`	`CG`	no molecular dynamics
`MD.Steps` / `MD.NumCGsteps`	`0` (single point) or `--relax-steps`	no relaxation, or an ionic budget with `--relax-positions`
`MD.MaxForceTol`	`0.01 eV/Ang`	only with `--relax-positions`
`kgrid.MonkhorstPack`	from `--kgrid`/`--kgrid-density`	only when the template has no grid (SIESTA would sample Gamma only) or you ask for one; a template's own grid is kept

Only these directives are forced: basis, mesh cutoff, functional, SCF settings are whatever the template says, identical in all folders. The same convention as every other prep stage of the suite (`stb-optical`, `stb-gqca`, `stb-adsorb`...): your `calc.fdf` is never edited.

Special cases: if no `calc.fdf` exists and `-c` was not given, only the scaled structures are written (the earlier behavior of the tool); if `-c` is the same file as `--file` (a self-contained input), the scaled copy of it is the run input, with the `%include` on top.

3.3 The report, live.

```
[1] INPUT STRUCTURE
------------------------------------------------------------
Formula         : Si2  (2 atom(s), 1 species)
Lattice (Ang)   : a=3.8672  b=3.8672  c=3.8672
Angles (deg)    : alpha=60.00  beta=60.00  gamma=60.00
Volume          : 40.8944 Ang^3  (20.4472 Ang^3/atom)
Density         : 2.2809 g/cm^3
Shortest bond   : 2.3681 Ang
Detected        : 3D (bulk material)
Space group     : Fd-3m (No. 227), cubic system
Species | Z  | Atoms | Valence e- | Total e- | Valence from
```

```
------------------------------------------------------------
Si      | 14 | 2     | 4          | 8        | Si.psml
Valence electrons : 8

[2] LEVEL OF THEORY READ FROM THE TEMPLATE
------------------------------------------------------------
XC functional ... GGA / PBE        Basis ... split DZP, PAO.EnergyShift 0.02 Ry
Mesh cutoff ... 320 Ry             SCF k-grid ... 10x10x10 (spacing 0.199 1/Ang)
SCF k-grid forced : no -- the template's own grid is kept (--kgrid / --kgrid-density override it)
[NOTE] PAO.EnergyShift 0.02 Ry: V0 and B0 depend on the orbital cutoff radii; converge them ...

[3] SCAN DEFINITION
------------------------------------------------------------
Volume strain   : -8.00 % .. +8.00 %, 11 points (step 1.600 %)
Cell scaling    : L = L0 * (1 + strain/100)^(1/3) on every lattice vector (shape kept); fractional
positions unchanged
Folder     | dV/V0 (%) | V (Ang^3) | V/atom  | L/L0    | a b c (Ang)          | d_min (Ang)
-------------------------------------------------------------------------------------------
vol_m8.00/ | -8.00     | 37.6228   | 18.8114 | 0.97259 | 3.7612 3.7612 3.7612 | 2.3032
 ...
vol_0.00/  | +0.00     | 40.8944   | 20.4472 | 1.00000 | 3.8672 3.8672 3.8672 | 2.3681
 ...
vol_8.00/  | +8.00     | 44.1659   | 22.0830 | 1.02599 | 3.9677 3.9677 3.9677 | 2.4297

[4] DIRECTIVES FORCED THROUGH config_extra.fdf (your template is not edited)
------------------------------------------------------------
Directive           | Forced value | Template had | Effect
kgrid.MonkhorstPack | (not forced) | 10x10x10     | the template's own grid
MD.TypeOfRun        | CG           | (absent)     | no molecular dynamics
MD.Steps            | 0            | (absent)     | single point, no relaxation
MD.NumCGsteps       | 0            | (absent)     | older spelling of the same budget, forced for
templates that still use it
MD.VariableCell     | false        | (absent)     | the cell is the scan variable: it must NEVER relax

[5] FOLDERS WRITTEN        (one line per folder: structure.fdf, calc.fdf, config_extra.fdf, Si.psml)

[6] SUMMARY & NEXT STEPS
------------------------------------------------------------
11 volume folder(s) written under 'eos_runs' (calc.fdf is the SIESTA input).
Scan record          : eos_runs/eos_scan.json (read by stb-eosAnalysis)
Cost                 : 11 independent runs (single-point SCF each)
Next steps:
  1. Run SIESTA in every folder, e.g.:
       for d in eos_runs/vol_*; do (cd $d && mpirun -np 4 siesta calc.fdf > calc.out); done
  2. Check that every run finished (SCF converged), then:
       stb-eosAnalysis --dir eos_runs --eos all --show
```

([0] is the run metadata; the `a b c` of the primitive fcc cell are the primitive vectors, `a_conventional / √2`.) Every warning of the run is repeated in a list at the end of the report.

3.4 The advisories. Printed where they apply and repeated at the end of the report:

- the template does not `%include` the structure file (SIESTA would use the unscaled cell in every folder: eleven copies of the same calculation);
- the template defines a lattice of its own (`LatticeVectors`, `LatticeParameters`, `LatticeConstant`) next to the included structure — whichever SIESTA reads first wins;
- a non-cubic crystal (Section 3.5), a small basis (SZ/DZ/SZP), no `Mesh.Cutoff`, Cartesian coordinates

(which would not scale), a `--strain-range` below 2 % (curvature lost in SCF noise) or above 15 %, and an even number of volumes (no point at the input volume).

3.5 Bulk only, and the crystal system. A vacuum-padded structure (slab, wire, molecule) has no bounded cell volume: it is refused with an error. For a non-cubic crystal the report adds the *constrained-path* note of Section 1.9 and records the crystal system in `eos_scan.json`, from where Stage 2 repeats it.

3.6 Validation. `--n-volumes` below 5 and a `--strain-range` outside (0, 40) % are rejected; a missing `--file`, an explicit `-c` that does not exist, an unknown `-p` source, and a vacuum-padded structure all exit with a clear `[ERROR]`.

3.7 Running it both ways.

```
stb-eosInputs -f structure.fdf -p dojo --n-volumes 11 --strain-range 8.0 --save-report
stb-suite           # then: 5 (Workflow) -> 15 (Equation of State) -> 1 (Stage 1)  (or type 5.15.1)
```

11.15.1.4 4. Stage 2: analyzing the results (`stb-eosAnalysis`, code `5.15.2`)

4.1 Before running Stage 2. Every `vol_*` folder must hold the SIESTA output (`calc.out` by default, `--file` to change it). A folder without it, or with an unparseable one, is *skipped, not fatal*, and reported. Stage 2 reads each run's free energy, the **last** `outcell:` block (the cell SIESTA actually used), the SCF convergence line, the forces and the stress tensor. `eos_scan.json` next to the folders is optional but recommended: it enables the planned-versus-actual volume check and the suggested lattice constant.

4.2 Report structure.

Section	Content
`[0]`	run metadata
`[1]`	the scan (from `eos_scan.json`) and the **level of theory of every folder**, checked for being identical
`[2]`	one line per folder: read, skipped, SCF not confirmed
`[3]`	the table: `V`, `V/atom`, `dV`, `E`, `E/atom`, `E − E_min`, `P_SIESTA`, SCF, force
`[4]`	the fit(s): `V0`, `V0/atom`, `E0`, `B0`, `B0'`, R^2, residual rms; uncertainty, window sensitivity, form spread; suggested `LatticeConstant`
`[5]`	the **pressure cross-check**: `P_SIESTA` vs `−dE/dV` per volume, the second `B0` from the stress, the pressure at `V0`
`[6]`	`--target-pressure`: volume and cell scale at each pressure
`[7]`	the **checks table** (OK/WARN) with hints
`[8]`	summary and the files written
`[9]`	next steps
`[10]`	every warning again

4.3 `[1]`–`[3]`: is the scan what you think?. `[1]` reads `config_extra.fdf`, `calc.fdf` and the pseudopotential files of *each* folder, compares the settings (functional, basis, mesh cutoff, spin, smearing, D3, k-grid, pseudopotential hash) and warns on any difference — `E(V)` is one curve only if they are all identical. `[2]` compares each run's cell with the plan (a folder edited after Stage 1, or a `%include` problem, shows as "SIESTA ran a cell of ... but the plan says ..."). `[3]` prints the data with the pressure of each run's own stress tensor.

4.4 [4]: the fit. With `--eos all` the four forms are fitted on the same data and listed side by side, with `Bracket ok` when `V0` lies in the scanned range; the reference fit (Birch–Murnaghan) feeds everything below. Besides the parameters:

- *Uncertainty (leave-one-out)* — Eq. 11 for `V0` and `B0`;
- *Window sensitivity* — `B0` from the central 60 % of the points;
- *Spread between EOS forms* — `B0` and `V0` range over the four forms;
- *Fit quality* — R^2 and the residuals, in meV/atom;
- *Compressibility* `1/B0` in TPa^{-1} and `B` of the curve at `V0`;
- *Versus the input structure* — `V0/V_input`, the linear scale, and the **suggested `LatticeConstant`** (the input's `LatticeConstant` times the scale: the equilibrium volume at the input's shape).

4.5 [5]: the pressure cross-check. The table of `P_SIESTA`, `−dE/dV` and the difference per volume; the maximum difference; the SIESTA pressure at the fitted `V0` (it should be ~0); and the **second `B0`**, from the stress alone (Section 1.6). See Section 5.3 for what these numbers look like on real data and what moves them.

4.6 [7]: the checks.

Check	Passes when	If it fails
All planned runs available	every folder of the plan was read	run the missing folders
SCF converged in every run	each `calc.out` has the convergence line	rerun with more iterations / smaller mixing weight
`V0` inside the scanned range	the fitted minimum lies within the data	widen `--strain-range`
Points on both sides of `V0` (≥ 2 each)	`n_below` $\geq$ 2 and `n_above` $\geq$ 2	re-centre the scan on `V0` (suggested `LatticeConstant`)
Fit $R^2 \geq 0.9999$	—	change the strain range; check the SCF settings
Residual rms $\leq$ 0.5 meV/atom	—	same
`B0'` within 2.5–7	physically plausible	too narrow a scan, or a noisy one
EOS forms agree on `B0` (< 1 %)	only with `--eos all`	the functional form matters: the scan is too wide or narrow
`B0` uncertainty (leave-one-out) < 1 %	—	a single run is moving the fit: inspect the table
`B0` stable against the window (< 2 %)	—	the curve is not described by the form over the whole range
`P_SIESTA` agrees with `−dE/dV` (< 0.5 GPa)	—	raise `Mesh.Cutoff`, tighten `DM.Tolerance`, widen the scan
Residual pressure at `V0` small (< 0.2 GPa)	—	same
`B0` from stress agrees with energy (< 5 %)	—	same
Residual forces $\leq$ 0.05 eV/Å	atoms are at equilibrium in every folder	use `--relax-positions`
Same level of theory in every folder	—	a folder was edited: rerun with identical settings

The thresholds are conventions of the tool (they are constants at the top of `eos_analysis.py`), not laws; a WARN says *look here*.

4.7 Plots and data on request. Nothing is written unless asked for:

Option	Writes
`--show`	opens the 6-panel matplotlib dashboard (needs a display; without one it warns and only saves with `--png`)
`--png`	`<output>_summary.png`, the same dashboard
`--save-gnuplot`	`<output>.dat` (data block and fitted curve) + `<output>.gplot` (E(V) and P(V), `gnuplot <output>.gplot` renders `<output>.pdf`)
`--save-data`	`<output>_table.csv` (one row per volume) + `<output>_results.json` (every fit, check and number)
`--save-report`	`stb_eosAnalysis_report.txt`

The dashboard has six panels: `E(V)` with the data and every fit, the fit residuals, the pressure from the stress and from the fit, the bulk modulus along the curve `B(P)` (its slope at `P = 0` is `B0'`) with the stress-derived `B0` marked, `B0` by EOS form with the leave-one-out band, and a text summary.

4.8 Guard rails. A missing or unparseable run is skipped and the fit uses the rest (fewer than 4 valid volumes is a hard error); a scan that does not bracket the minimum, a different level of theory in one folder, an unconverged SCF and a cell that is not the planned one are each reported where they apply, in `[7]`, and in `[10]`.

4.9 Running it both ways.

```
stb-eosAnalysis --dir eos_runs --eos all --png --save-gnuplot --save-data --save-report
stb-eosAnalysis --dir eos_runs --target-pressure 0 5 10 --show
stb-suite            # then: 5 -> 15 -> 2  (or type 5.15.2): it asks for the options above
```

11.15.1.5 5. Worked example: bulk silicon, end to end

5.1 Set-up. Bulk silicon (diamond, Fd-3m), the 2-atom primitive fcc cell (`structure.fdf`, `a = 5.469` Å, the conventional-cell constant), PBE [12], the PseudoDojo norm-conserving pseudopotential [13], DZP basis with the default `PAO.EnergyShift 0.02 Ry`, `Mesh.Cutoff 320 Ry`, a 10×10×10 Monkhorst–Pack grid, 300 K electronic temperature, `DM.Tolerance 1e-5` (`calc.fdf`). Eleven volumes from −8 % to +8 %, SIESTA 5.4.2, `mpirun -np 2`: about 6 s per volume. The reference data are **14 such scans** (Section 6): this *base* setup and 13 others that change exactly one thing — the k-grid (4^3, 6^3, 8^3, 14^3), the `Mesh.Cutoff` (150, 200, 450, 600 Ry), the basis (DZP and TZP with `PAO.EnergyShift` 0.02, 0.01, 0.005 Ry). Note that the 2-atom cell reports `a = b = c = 3.867` Å and angles of 60°: those are the primitive vectors (`a_conventional` / $\sqrt{2}$); the conventional lattice constant is `a0 = (8 V0/atom)^(1/3)`, which `convergence_table.py` prints.

5.2 The base scan.

```
stb-eosInputs -f structure.fdf -p dojo --n-volumes 11 --strain-range 8.0 --save-report
for d in eos_runs/vol_*; do (cd $d && mpirun -np 2 siesta calc.fdf > calc.out); done
stb-eosAnalysis --dir eos_runs --eos all --png --save-gnuplot --save-data --save-report
```

The 11 energies (Section 4's report `[3]`, shortened) follow a smooth curve with a minimum between the −1.6 % and +0 points, and a pressure that changes sign there:

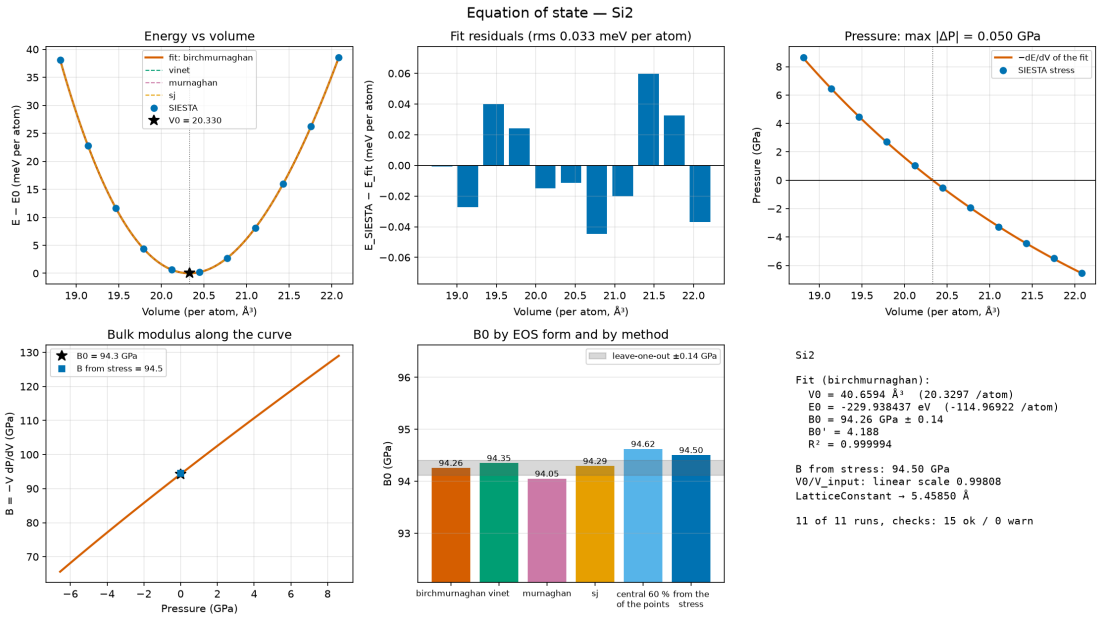
`dV` (%)	`V` (Å^3)	`E − E_min` (meV/atom)	`P_SIESTA` (GPa)
−8.0	37.6228	37.95	+8.650
−4.8	38.9315	11.49	+4.450
−1.6	40.2401	0.45	+1.023
0.0	40.8944	0.00	−0.518

dV (%)	V (Å^3)	E − E_min (meV/atom)	P_SIESTA (GPa)
+1.6	41.5487	2.53	−1.918
+4.8	42.8573	15.83	−4.462
+8.0	44.1660	38.32	−6.530

The fit ([4]):

EOS form	V0 (Å^3)	B0 (GPa)	B0'	R^2	residual rms (meV/atom)
Birch–Murnaghan	40.6594	94.26	4.188	0.999994	0.0325
Vinet	40.6594	94.35	4.186	0.999995	0.0306
Murnaghan	40.6593	94.05	4.193	0.999992	0.0378
stabilized jellium	40.6593	94.29	—	0.999995	0.0317

so V0 = 40.659 Å^3 (20.330 Å^3/atom, a0 = 5.4585 Å), E0 = −229.938437 eV and B0 = 94.26 ± 0.14 GPa (leave-one-out), B0' = 4.19. The four forms agree on B0 to 0.32 % and on V0 to 0.0001 Å^3; the central 7 of 11 points give B0 = 94.62 (+0.37 GPa); the residuals are 0.03 meV/atom — the SCF noise. V0 lies 0.57 % below the volume of the input cell: the report suggests LatticeConstant 5.4585 for the next scan. **All 15 checks of [7] are OK.** Statistically this is as clean a fit as one gets; Sections 5.4–5.6 show that "clean" is not "accurate".



The dashboard of stb-eosAnalysis on the base scan

Fig. 2 — The dashboard (--png) of the base scan: E(V) with the four fits, residuals, P(V) from the stress and from the fit, B(P), B0 by EOS form and by method, and the summary.

5.3 The pressure cross-check. [5] compares the pressure of each run's stress tensor with −dE/dV of the fit:

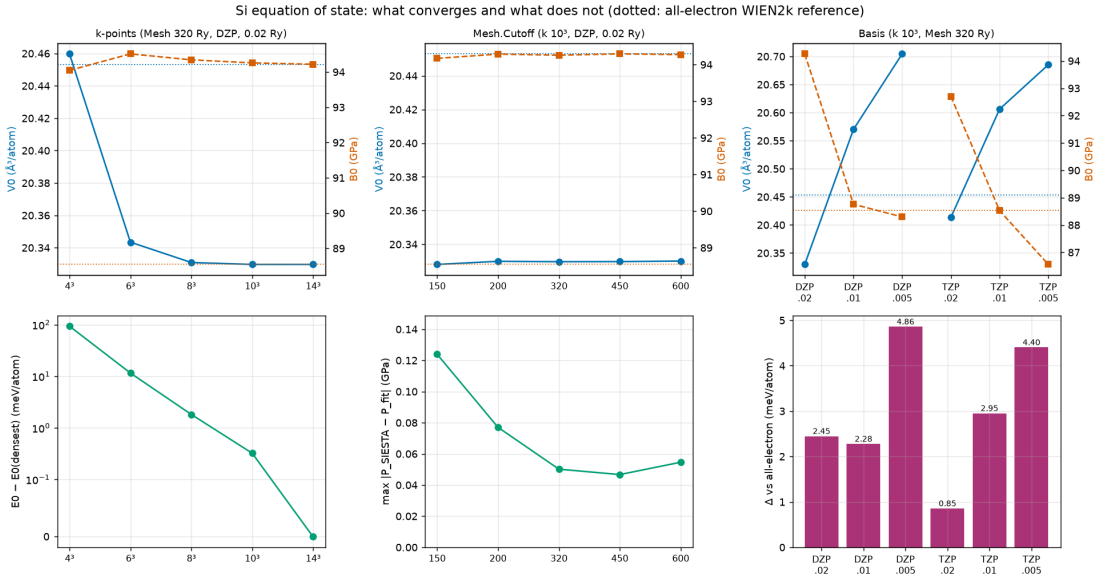
V (Å^3)	P_SIESTA (GPa)	P_fit (GPa)	difference
37.6228	+8.6495	+8.6097	+0.040
39.5858	+2.7179	+2.6678	+0.050
40.8944	−0.5181	−0.5368	+0.019
42.2030	−3.2870	−3.2488	−0.038

V (Å^3)	P_SIESTA (GPa)	P_fit (GPa)	difference
44.1660	−6.5302	−6.5593	+0.029

The largest difference over the 11 volumes is **0.050 GPa** (0.3 % of the pressure range), the SIESTA pressure at the fitted V0 is +0.004 GPa, and the bulk modulus from the stress alone (a cubic polynomial of P_SIESTA(V), B = −V dP/dV) is **94.50 GPa** against 94.26 from the energies: two independent observables of the same runs agree to 0.3 %. (A *quadratic* fit of the pressure gives 96.25 GPa, +2.1 %: over a ±8 % window it truncates the curvature of P(V); the tool uses a cubic for that reason.)

What moves this agreement is the **mesh**, not the basis or the k-points. At Mesh.Cutoff 150, 200, 320, 450, 600 Ry the largest abs(P_SIESTA − P_fit) is 0.124, 0.077, 0.050, 0.047, 0.055 GPa, while the *energy* curve does not move at all (V0 20.328–20.330 Å^3/atom, E0 identical to 0.02 meV/atom): the stress feels the real-space grid before the energy does, and ~0.05 GPa is the floor set by the SCF tolerance. A mesh cutoff that is fine for E(V) can still be too coarse for a pressure.

5.4 Convergence: k-points and mesh.



What converges and what does not: k-points, mesh, basis

Fig. 3 — V0 and B0 (top), and below the total-energy convergence (k), the stress/energy agreement (mesh) and the distance to the all-electron reference (basis). Dotted: the all-electron values. python3 convergence_table.py --png.

k-grid	V0 (Å^3/atom)	B0 (GPa)	B0'	E0 above the densest (meV/atom)
4×4×4	20.460	94.04	4.160	92.6
6×6×6	20.343	94.51	4.191	11.2
8×8×8	20.331	94.34	4.188	1.5
10×10×10 (base)	20.330	94.26	4.188	0.32
14×14×14	20.330	94.22	4.190	0

Mesh.Cutoff (Ry)	V0 (Å^3/atom)	B0 (GPa)	B0'	max abs(P_SIESTA − P_fit) (GPa)
150	20.328	94.17	4.252	0.124
200	20.330	94.29	4.173	0.077
320 (base)	20.330	94.26	4.188	0.050

Mesh.Cutoff (Ry)	V0 (Å^3/atom)	B0 (GPa)	B0'	max abs(P_SIESTA − P_fit) (GPa)
450	20.330	94.29	4.185	0.047
600	20.330	94.27	4.165	0.055

- **The EOS properties converge faster than the total energy.** E0 falls 92.6 → 11.2 → 1.5 → 0.3 meV/atom from 4^3 to 14^3, but the error of a sparse grid is nearly the same at every volume and cancels in E(V) differences: V0 is within 0.06 % from 6^3 and B0 within 0.3 GPa. The 10^3 base grid is converged for this purpose; 6^3 would have sufficed — for a *semiconductor*.
- **The mesh does nothing to V0 or B0 above 150 Ry here** (the spread, 0.12 GPa, is the noise of the fit); it shows only in the stress agreement (Section 5.3) and in B0', which wanders between 4.17 and 4.25: it is determined to ±0.05 at best — do not read a difference of 0.1 in B0' between two setups as physics.
- **Convergence is not monotone in the one number you look at.** The 4×4×4 row has a V0 closer to the all-electron value (20.460 vs 20.453) than the converged grids (20.330), and the *smallest* Delta distance (0.60 meV/atom) of all the k-point runs — with a total energy 93 meV/atom off. A good agreement obtained from an unconverged setup is a coincidence; check the total energy before believing it.

5.5 The basis: the knob that matters.

Basis, PAO.EnergyShift	V0 (Å^3/atom)	a0 (Å)	B0 (GPa)	B0'	E0 vs base (meV/atom)
DZP, 0.02 Ry (base)	20.330	5.4585	94.26	4.19	0
DZP, 0.01 Ry	20.570	5.4799	88.76	4.33	−48.2
DZP, 0.005 Ry	20.705	5.4919	88.30	4.30	−58.3
TZP, 0.02 Ry	20.413	5.4660	92.70	4.08	−28.6
TZP, 0.01 Ry	20.606	5.4831	88.53	4.22	−69.7
TZP, 0.005 Ry	20.685	5.4901	86.56	4.24	−78.1

- **The basis moves V0 by 1.8 % and B0 by 8 GPa (8.5 %)** — 60 times the 0.14 GPa leave-one-out uncertainty of any single scan, and 20 times the effect of any k-grid or mesh above. The fit's uncertainty is a statement about the scan, not about the answer (Section 1.5).
- **Longer orbitals make a larger, softer crystal.** In both DZP and TZP, every halving of PAO.EnergyShift raises V0 and lowers B0. This is the direction expected from Section 1.7 (short orbitals favour compression through the basis-superposition error), but the table cannot prove the mechanism: it only shows the trend.
- **The total energy is not converged.** From 0.02 to 0.005 Ry it drops 58 meV/atom (DZP) and 50 meV/atom (TZP), and TZP lies another 20–29 meV/atom below DZP at the same radii — two orders of magnitude above the k-point or mesh error, and still falling. No setting of this tutorial is a "converged basis": the question is how much each moves the *properties*.

5.6 Compared with an all-electron calculation and with experiment. The all-electron PBE reference of the Delta test [9] — V0 = 20.453 Å^3/atom (a0 = 5.4695 Å), B0 = 88.545 GPa, B0' = 4.31 — is the benchmark for the *numerical* error of the SIESTA setup (same functional, so the physics of PBE cancels). Δ is Eq. 12; B(V_AE) is the bulk modulus of *our* curve at the all-electron volume.

Setup	V0 vs AE	B0 (GPa)	B(V_AE) (GPa)	Δ (meV/atom)
all-electron WIEN2k	20.453	88.55	88.55	0
DZP, 0.02 Ry (base)	−0.6 %	94.26	91.9	2.45
DZP, 0.01 Ry	+0.6 %	88.76	91.0	2.28
DZP, 0.005 Ry	+1.2 %	88.30	93.0	4.86
TZP, 0.02 Ry	−0.2 %	92.70	92.0	**0.85**
TZP, 0.01 Ry	+0.7 %	88.53	91.4	2.95

Setup	V0 vs AE	B0 (GPa)	B(V_AE) (GPa)	Δ (meV/atom)
TZP, 0.005 Ry	+1.1 %	86.56	90.8	4.40

Read in this order:

1. **B0 alone would have picked the wrong setups.** The DZP 0.01 and TZP 0.01 rows reproduce the reference B0 (88.76, 88.53 vs 88.55) — because their V0 is 0.6–0.7 % too large, which softens the crystal (dB ≈ −B0 B0' dV/V0). At the *same volume* every setup has B = 91–93 GPa: the curvature agrees across the basis sets to ±1 GPa and is 2.5–5 % *above* the all-electron value. The B0 differences between setups are mostly a volume effect.
2. **Δ ranks them differently:** TZP at the default shift (0.85 meV/atom) is the closest to the all-electron curve, and the two longest-orbital DZP/TZP runs the farthest (4.4–4.9). Neither end of the shift scan is "right": V0 is too small at 0.02 Ry and too large at 0.005 Ry, and Δ is smallest where the two errors balance. The agreement of the best row (0.85) therefore comes from a cancellation as much as from a converged basis — the energy of the same TZP basis falls another 49 meV/atom at 0.005 Ry.
3. **The all-electron V0 is bracketed, the remainder is not understood.** With the longest orbitals V0 overshoots the reference by 1.1–1.2 %. A good norm-conserving pseudopotential in a complete basis should reproduce the all-electron V0 of silicon to a fraction of a percent, so that residue points to the NAO bases of this tutorial (no extra polarization or diffuse functions) or to the SIESTA implementation of the pseudopotential — a question these runs cannot settle (not tested: shift 0.01 with a larger split, another pseudopotential).
4. **Against experiment** (a = 5.43 Å, B ≈ 97.9 GPa as quoted in [23]; compilations differ): every setup gives a0 0.5–1.1 % too large (5.459–5.492 Å) and B0 86.6–94.3 GPa, 4–12 % too small — the PBE signature (the all-electron PBE itself is 0.7 % and 10 % off, and the literature PBE modulus for Si is 88.0 GPa [22]). It confirms that the setup is a sensible PBE calculation; it says nothing about whether the basis is converged.

5.7 How far to trust each number.

Quantity	Spread across this tutorial's setups	Verdict
V0, a0	1.8 % across the basis (0.06 % across k ≥ 6^3 and mesh)	±1 % unless the basis is tested; converged in k and mesh
B0	8.5 % across the basis (0.3 GPa across k ≥ 6^3 and mesh)	±5 %: do not over-read; compare at a fixed volume
B0'	5.1–4.3, noise ±0.05	two figures at most
E0	tens of meV/atom with the basis, ~100 with k = 4^3	meaningless in absolute value: differences between setups of the *same* basis only
P(V) inside the scan	0.05 GPa between stress and energy	reliable to ~0.05 GPa for ≥ 320 Ry
V(P) outside the scan	not testable	--target-pressure flags it as EXTRAPOLATED: do not use
the leave-one-out uncertainty (0.14 GPa)	—	the precision of the *scan*, 60× smaller than the basis effect

5.8 What to take from the example.

1. A statistically perfect fit (R^2 = 0.99999, 15 checks OK, 0.03 meV/atom residuals) can still sit 6 % from the all-electron B0. The checks table says *the scan is clean*, not *the number is right.*
2. The stress is a free second opinion: it agrees with −dE/dV to 0.05 GPa and gives B0 again to 0.3 %. It is sensitive to the mesh before the energy is.

3. k-points and mesh converge fast for silicon; the basis does not, and it is the one that decides `V0` and `B0`.
4. Judge a basis with a reference (all-electron or experiment) using `V0`, `B` at a fixed volume and Δ, together with the convergence of the total energy — never `B0` alone. A good number from an unconverged setting is a coincidence.
5. Use `--eos all`, `--png` and the checks as a habit: they cost nothing and catch bracketing, noise and inconsistent folders.

11.15.1.6 6. Reproducing the reference data

`reference_results/si/` holds the finished SIESTA runs of the 14 setups of Section 5, so Stage 2 works out of the box without a SIESTA installation.

Folder	What	Contents
`base/`	Case 1's scan: DZP, `PAO.EnergyShift 0.02 Ry`, `Mesh.Cutoff 320 Ry`, k 10×10×10	`calc_template.fdf`, `eos_scan.json`, `stb_eosInputs_report.txt`, per `vol_*`: `structure.fdf`, `calc.fdf`, `config_extra.fdf`, `calc.out`
`k04`, `k06`, `k08`, `k14`	k-grid 4, 6, 8, 14 (all else as `base`)	same
`m150`, `m200`, `m450`, `m600`	`Mesh.Cutoff` 150, 200, 450, 600 Ry	same
`dzp_s010`, `dzp_s005`	DZP with `PAO.EnergyShift` 0.01, 0.005 Ry	same
`tzp_s020`, `tzp_s010`, `tzp_s005`	TZP with `PAO.EnergyShift` 0.02, 0.01, 0.005 Ry	same

Pseudopotentials and SIESTA's by-products (`.DM`, `.EIG`, ...) are omitted to keep the repository small (`-p dojo` regenerates the pseudopotential). `make_reference.sh` regenerates everything: for each setup it edits the template (`calc.fdf` of this folder), runs `stb-eosInputs` and SIESTA in each of the 11 folders:

```
bash make_reference.sh                # ~30 minutes on 2 cores; writes reference_run/ and
reference_results/si/
```

The single commands of the base setup are those of Case 1 of the script:

```
stb-eosInputs -f structure.fdf -p dojo --n-volumes 11 --strain-range 8.0 --save-report
for d in eos_runs/vol_*; do (cd $d && mpirun -np 2 siesta calc.fdf > calc.out); done
stb-eosAnalysis --dir eos_runs --eos all --png --save-gnuplot --save-data --save-report
```

Computed with SIESTA 5.4.2 (conda-forge build, GNU 14.3, MPI), `mpirun -np 2` on a 2-core machine: 5–25 seconds per volume depending on the setup (the denser k-grid, finer mesh and TZP basis are the slow ones).

11.15.1.7 7. Known, deliberate limitations

- **Static lattice, 0 K.** No zero-point energy and no thermal pressure; comparing `V0` and `B0` with a room-temperature experiment includes that omission (small for Si, larger for light elements). Section 1.8.
- **Isotropic scaling.** For non-cubic crystals the result is the modulus along the constrained path; the tool says so but does not relax the shape. Section 1.9.

- **Bulk only.** A vacuum-padded cell is refused; layered materials with van der Waals gaps are bulk — use `--d3` in the template and expect `B0'` and the c-axis to need care.
- **`B0'` is poorly determined** by a narrow scan (it describes the asymmetry of the curve); do not quote it to more than two figures and do not over-interpret differences between setups of 0.1–0.2.
- **The reported uncertainty is the scan's**, not the setup's: leave-one-out says nothing about the basis, the pseudopotential or the functional (Section 5.5 shows that these are an order of magnitude larger).
- **Basis sets are not converged by construction.** The default (DZP, `PAO.EnergyShift 0.02 Ry`) is a *starting point*; Section 5.5 measures a ±1 % spread in `V0` and ±4 % in `B0` between reasonable choices. Treat SIESTA's `B0` as good to about ±5 % unless the basis has been tested against a reference.
- **A pseudopotential's quality is not guaranteed** (the suite's usual caution): compare the relaxed geometry with a known value before trusting an EOS built on it.
- **The all-electron comparison is for one element** with the PseudoDojo pseudopotential and the plain DZP/TZP bases of this tutorial; the Delta-test value of an optimized SIESTA basis can be much smaller.
- **Metals** need a denser k-grid and a sensible smearing than silicon; the checks flag the symptoms (noisy residuals, `B0` unstable against the window) but not the cause.
- **Energy vs stress agreement** is a test of the numerics, not of the physics: it passes for a wrong functional as well as for a right one.

11.15.1.8 8. Step-by-step: running this workflow on your own crystal

1. **Relax the structure** (cell and atoms) at the level of theory you will scan with. The scan is centred on the input volume; one that starts within a few percent of equilibrium needs the fewest points.
2. **Write a `calc.fdf` with the physics only**: functional, basis, `PAO.EnergyShift`, `Mesh.Cutoff`, `Spin`, smearing, `%include structure.fdf`. A k-grid is optional: Stage 1 suggests one at density 0.2 $Å^{-1}$ if absent.
3. **Run Stage 1** with `-p <bank>` and an *odd* `--n-volumes` (9–11), `--strain-range` 6–8 %. Read the report: the level of theory, the crystal-system note, every advisory. Add `--relax-positions` if atoms have free coordinates.
4. **Run SIESTA** in each `vol_*` folder (all folders take about the same time).
5. **Run Stage 2** with `--eos all --png` and read, in this order: `[1]` (same settings in every folder? did SIESTA run the planned cells?), `[3]` (SCF, forces), then `[4]`, `[5]` and the checks `[7]`.
6. **Re-centre** if `V0` is not near the middle (the report gives the new `LatticeConstant`): rerun Stage 1 from there.
7. **Converge** what shapes the curve, one change at a time, comparing `V0`, `B0` *and* the volume-independent figures: k-grid, `Mesh.Cutoff`, then basis size and `PAO.EnergyShift` (Section 5 shows the order of effects). Judge a basis by `V0` and by a reference (experiment or all-electron Δ), never by `B0` alone.
8. **Interpret with the limitations in mind**: compare with experiment *including* the functional's known bias and the static-lattice omission; use `--target-pressure` to get the geometry at the pressure of interest.

11.15.1.9 9. Files in this folder

File	What it is
`README.md`	this tutorial
`example_5.15.sh`	guided, runnable walk-through (11 cases)
`convergence_table.py`	helper of the script: fits every setup of `reference_results/si/`, prints the table and draws `figures/convergence.png`

File	What it is
`theory_figures.py`	draws `figures/eos_forms.png` (Fig. 1)
`make_reference.sh`	regenerates `reference_results/si/` with SIESTA (Section 6)
`structure.fdf`	bulk silicon, 2-atom primitive cell, `a = 5.469` Å — the input of Stage 1
`calc.fdf`	the template: PBE, DZP, `PAO.EnergyShift 0.02 Ry`, `Mesh.Cutoff 320 Ry`, k 10×10×10
`reference_results/si/`	real finished SIESTA runs, 14 setups × 11 volumes (Section 6)
`figures/`	`eos_forms.png`, `si_dashboard.png`, `convergence.png`
`output/`	created by the script (git-ignored)

11.15.1.10 10. Running the script

```
bash example_5.15.sh
```

It pauses between cases (`[Press Enter to continue]`); `yes '' | bash example_5.15.sh` runs it straight through. It starts by wiping its own `output/`. Cases 1–5 run Stage 1 (no SIESTA needed); Cases 6–11 run Stage 2 on `reference_results/`. It sets `MPLBACKEND=Agg`, so `--show` is replaced by `--png` in the script; try it on a desktop:

```
stb-eosAnalysis --dir reference_results/si/base --eos all --show --target-pressure 0 5 10
```

11.15.1.11 11. Flag reference

`stb-eosInputs` — `-f FILE` relaxed structure (fractional, bulk) · `-c CALC` (default `./calc.fdf`; absent and not given = scaled structures only; the same file as `-f` = self-contained input) · `-p PSEUDO` (bank or folder) · `--n-volumes N` (default 9, ≥ 5) · `--strain-range %` (default 5.0) · `--relax-positions` · `--relax-steps N` (default 100) · `--kgrid NX NY NZ` · `--kgrid-density D` (1/Å) · `--vacuum-gap Å` (default 10) · `-o DIR` (default `eos_runs`) · `--save-report` · `--no-intro` · `-v`.

`stb-eosAnalysis` — `--dir DIR` (default `eos_runs`) · `--file NAME` (default `calc.out`) · `--eos {birchmurnaghan,vinet,murnaghan,sj,all}` · `--target-pressure GPA ...` · `-o NAME` (default `eos_curve`) · `--show` · `--png` · `--save-gnuplot` · `--save-data` · `--save-report` · `--no-intro` · `-v`.

11.15.1.12 12. What's next

- `5.2-elastic` computes the full elastic tensor by stress–strain; its bulk modulus is the independent cross-check of `B0` here. With a MACE model fine-tuned on your SIESTA settings (`7.1`), `stb-mlelastic` (`7.4`) gives the same bulk modulus in seconds.
- `5.4-phonons` and `stb-mlphonons --qha` use this same volume scan to get thermal expansion and the finite-temperature equation of state.
- `5.5-convergence` is the same convergence discipline applied to a ground-state total energy; this tutorial's Section 5 applies it to the properties derived from `E(V)`.
- For the fully relaxed hydrostatic modulus of a non-cubic crystal, relax the shape at each volume (`MD.VariableCell` + `Constant.Volume`) and feed the energies to `stb-eosAnalysis` by hand.

11.15.1.13 13. References

1. F. D. Murnaghan, *The compressibility of media under extreme pressures*, Proc. Natl. Acad. Sci. USA **30**, 244 (1944).

2. F. Birch, *Finite elastic strain of cubic crystals*, Phys. Rev. **71**, 809 (1947).
3. P. Vinet, J. Ferrante, J. R. Smith, J. H. Rose, *A universal equation of state for solids*, J. Phys. C **19**, L467 (1986); P. Vinet, J. H. Rose, J. Ferrante, J. R. Smith, *Universal features of the equation of state of solids*, J. Phys.: Condens. Matter **1**, 1941 (1989).
4. O. L. Anderson, *Equations of State of Solids for Geophysics and Ceramic Science*, Oxford University Press (1995).
5. A. B. Alchagirov, J. P. Perdew, J. C. Boettger, R. C. Albers, C. Fiolhais, *Energy and pressure versus volume: Equations of state motivated by the stabilized jellium model*, Phys. Rev. B **63**, 224115 (2001).
6. G. P. Francis, M. C. Payne, *Finite basis set corrections to total energy pseudopotential calculations*, J. Phys.: Condens. Matter **2**, 4395 (1990).
7. K. Lejaeghere *et al.*, *Reproducibility in density functional theory calculations of solids*, Science **351**, aad3000 (2016).
8. K. Lejaeghere, V. Van Speybroeck, G. Van Oost, S. Cottenier, *Error estimates for solid-state density-functional theory predictions: an overview by means of the ground-state elemental crystals*, Crit. Rev. Solid State Mater. Sci. **39**, 1 (2014).
9. The Delta-codes DFT repository, `https://github.com/molmod/DeltaCodesDFT` — file `WIEN2k.txt`: all-electron PBE reference (WIEN2k 13.1) of the elemental crystals; Si: `V0 = 20.4530 Å³/atom`, `B0 = 88.545 GPa`, `B1 = 4.31`.
10. J. M. Soler *et al.*, *The SIESTA method for ab initio order-N materials simulation*, J. Phys.: Condens. Matter **14**, 2745 (2002).
11. A. García *et al.*, *Siesta: Recent developments and applications*, J. Chem. Phys. **152**, 204108 (2020).
12. J. P. Perdew, K. Burke, M. Ernzerhof, *Generalized gradient approximation made simple*, Phys. Rev. Lett. **77**, 3865 (1996).
13. M. J. van Setten *et al.*, *The PseudoDojo: Training and grading a 85 element optimized norm-conserving pseudopotential table*, Comput. Phys. Commun. **226**, 39 (2018). doi:10.1016/j.cpc.2018.01.012
14. H. J. Monkhorst, J. D. Pack, *Special points for Brillouin-zone integrations*, Phys. Rev. B **13**, 5188 (1976).
15. A. H. Larsen *et al.*, *The atomic simulation environment*, J. Phys.: Condens. Matter **29**, 273002 (2017).
16. P. Virtanen *et al.*, *SciPy 1.0*, Nat. Methods **17**, 261 (2020).
17. S. P. Ong *et al.*, *Python Materials Genomics (pymatgen)*, Comput. Mater. Sci. **68**, 314 (2013).
18. A. Togo, I. Tanaka, *Spglib: a software library for crystal symmetry search*, arXiv:1808.01590 (2018).
19. J. D. Hunter, *Matplotlib: A 2D graphics environment*, Comput. Sci. Eng. **9**, 90 (2007).
20. P. Pernot, B. Civalleri, D. Presti, A. Savin, *Prediction uncertainty of density functional approximations for properties of crystals with cubic symmetry*, J. Phys. Chem. A **119**, 5288 (2015) (supplementary tables: PBE and experimental lattice constants).
21. B. Efron, R. J. Tibshirani, *An Introduction to the Bootstrap*, Chapman & Hall (1993) (the jackknife).
22. PBE bulk modulus of Si, 88.0 GPa, and the weak role of zero-point corrections outside carbon: *Lattice dynamics and structural phase stability of group-IV elemental solids with the r²SCAN functional*, arXiv:2601.11177.
23. Experimental lattice constant (10.26 bohr = 5.43 Å) and bulk modulus (97.9 GPa) of Si as quoted in *Iterative minimization in reduced density matrix functional theory for periodic systems*, arXiv:2607.27679; compilations of the experimental modulus differ (about 98–99 GPa), and the original measurements were not consulted.

11.15.2 Command-line reference

11.15.2.1 `stb-eosInputs`

Stage 1 of 2 (equation of state): writes one SIESTA folder per scanned volume. Takes an ALREADY-RELAXED structure (`--file`, fractional coordinates) and your `calc.fdf` template (`--calc`, default `./calc.`

fdf), and writes '<output-dir>/vol_<pct>/' with the cell scaled isotropically (volume strain from -strain-range to +strain-range), your template untouched except for a '%include config_extra.fdf' at its top, the config_extra.fdf sidecar with the forced directives (fixed cell, single point – or ionic relaxation with --relax-positions –, and the SCF k-grid when the template defines none or you ask for one) and the pseudopotentials (-p). Also writes eos_scan.json (the plan, read by stb-eosAnalysis). Does not run SIESTA: run each folder, then use stb-eosAnalysis.

Synopsis.

```
usage: stb-eosInputs [-h] -f FILE [-c FDF] [-p PSEUDO_DIR] [--n-volumes N_VOLUMES]
                     [--strain-range STRAIN_RANGE] [--relax-positions]
                     [--relax-steps RELAX_STEPS] [--kgrid NX NY NZ]
                     [--kgrid-density D] [-o OUTPUT_DIR] [--vacuum-gap VACUUM_GAP]
                     [--save-report] [-v] [--no-intro]
```

Options

`-f, --file FILE`
Input structure (.fdf), already relaxed. Atomic coordinates must be in fractional format (same requirement as stb-strain). It is written, scaled, under the same name into every folder, and your calc.fdf must %include it. required

`-c, --calc FDF`
SIESTA input template (default: calc.fdf). Copied into every folder with '%include config_extra.fdf' prepended; not edited otherwise. If it does not exist and -c was not given, only the scaled structures are written (the earlier behavior). If it is the same file as --file (a self-contained input), the scaled copy of it is the run input.

`-p, --pseudo-dir PSEUDO_DIR`
Pseudopotential source: a bundled bank (dojo, virtual_vault) or a folder. The .psml/.psf of each species is copied into every folder.

`--n-volumes N_VOLUMES`
Number of volumes to scan (default: 9). At least 5 for the 4-parameter fit of stb-eosAnalysis; an odd number includes the input volume itself. type: int

`--strain-range STRAIN_RANGE`
Max isotropic volume strain, % (default: 5.0), spanning [-strain-range, +strain-range]. type: float

`--relax-positions`
Let the ions relax at every volume (cell fixed): needed for crystals with free internal coordinates. Default: single point (positions as given). switch, off by default

`--relax-steps RELAX_STEPS`
Max CG steps per volume with --relax-positions (default: 100). type: int

`--kgrid NX NY NZ`
Force this SCF k-grid in every folder (config_extra.fdf), over the template's. type: int

`--kgrid-density D`
Force an SCF k-grid of this density, 1/Ang (as stb-kgrid), over the template's. Without --kgrid/--kgrid-density the template's own grid is kept; if it has none, one of density 0.2 is forced (SIESTA would sample Gamma only). type: float

`-o, --output-dir OUTPUT_DIR`
Directory for the 'vol_<pct>' folders (default: eos_runs).

`--vacuum-gap VACUUM_GAP`
: Gap (Ang) used to detect vacuum-padded axes – bulk only, a vacuum-padded structure is rejected (default: 10.0). type: float

`--save-report`
: Also persist the report to <output-dir>/`stb_eosInputs_report.txt`. Off by default. switch, off by default

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Example usage:
  stb-eosInputs --file structure.fdf -p dojo
  stb-eosInputs --file structure.fdf --calc calc.fdf --n-volumes 11 --strain-range 8.0 -p dojo
  stb-eosInputs --file structure.fdf --kgrid-density 0.15 --relax-positions -o eos_runs
```

11.15.2.2 `stb-eosAnalysis`

Stage 2 of 2 (equation of state): aggregates the 'vol_*' folders written by `stb-eosInputs` and fits an equation of state (Birch-Murnaghan by default) to get V0/E0/B0 (bulk modulus)/B0'. Also reads each run's SIESTA pressure (stress tensor) and checks it against -dE/dV of the fit, gives a second B0 from the stress alone, the uncertainty of V0/B0 (leave-one-out), the residuals, the consistency of the level of theory across folders, and the suggested LatticeConstant; every warning is repeated in the last section. `--show` / `--png` draw a dashboard, `--save-gnuplot` writes the .dat/.gplot pair, `--save-data` a CSV + JSON.

Synopsis.

```
usage: stb-eosAnalysis [-h] [--dir DIR] [--file FILE]
                       [--eos {birchmurnaghan,birch_murnaghan,vinet,murnaghan,sj,all}]
                       [--target-pressure GPA [GPA ...]] [-o OUTPUT] [--show] [--png]
                       [--save-gnuplot] [--save-data] [--save-report] [-v]
                       [--no-intro]
```

Options

`--dir DIR`
: Directory containing the 'vol_*' run folders (default: eos_runs).

`--file FILE`
: SIESTA output filename inside each run folder (default: `calc.out`).

`--eos {birchmurnaghan,birch_murnaghan,vinet,murnaghan,sj,all}`
: Equation-of-state form to fit (default: birchmurnaghan). 'sj' = stabilized jellium (no B0' reported). 'all' fits every form on the same data and reports them side by side (the reference fit for everything else is then birchmurnaghan); mutually exclusive with `--target-pressure`. switch, off by default

`--target-pressure GPA [GPA ...]`
: One or more target external pressures (GPa) – inverts the fitted EOS (P = -dE/dV) to report the predicted equilibrium volume and linear lattice-parameter scale factor at each one. Not available with `--eos all`. type: float

`-o, --output OUTPUT`
: Base filename (no extension) of the output files (default: eos_curve).

`--show`
: Open the analysis dashboard (matplotlib, 6 panels) on screen. Needs a display. switch, off by default

`--png`
: Save the dashboard as <output>_summary.png. switch, off by default

`--save-gnuplot`
: Also write <output>.dat and <output>.gplot (E(V) and P(V), data + fit); 'gnuplot <output>.gplot' renders <output>.pdf. Off by default. switch, off by default

`--save-data`
: Also write <output>_table.csv (one row per volume) and <output>_results.json (every fit, check and number of the report). Off by default. switch, off by default

`--save-report`
: Also persist the report to stb_eosAnalysis_report.txt. Off by default. switch, off by default

Like every STB command it also accepts -h/-help, -v/-version and -no-intro (Section 3.3).

From the built-in help.

```
Example usage:
  stb-eosAnalysis --dir eos_runs --show
  stb-eosAnalysis --dir eos_runs --eos all --png --save-gnuplot --save-report
  stb-eosAnalysis --dir eos_runs --eos vinet --target-pressure 0 5 10
```

11.16 Charge Density Difference

At a glance

Menu code: 5.16 (a multi-stage workflow)

Code	Stage	Command
5.16.1	Stage 1 - Prep	stb-chargediffPrep
5.16.2	Stage 2 - Analysis	stb-chargediffAnalysis

Guided example: examples/5.16-chargediff/

Summary: Generate ghosted fragment folders for an already-relaxed combined system (bilayer, molecule-on-slab, or any N-fragment system), then compute Delta rho = rho_combined - sum(rho_fragment_i) to visualize charge accumulation/depletion regions.

11.16.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/5.16-chargediff/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

This workflow has 2 stages: **Stage 1** (`stb-chargediffPrep`, code `5.16.1`) takes a **combined structure** — two molecules, a molecule on a surface, two layers, any system made of `N` $\geq$ `2` fragments — and a fragment split, and writes `N + 1` ready-to-run SIESTA folders: one with the whole system and one per fragment, each fragment keeping the atoms of all the others as **ghost atoms**, every folder with the same cell, mesh and level of theory, the template untouched and everything mandatory in a `config_extra.fdf`. **Stage 2** (`stb-chargediffAnalysis`, code `5.16.2`) reads the finished runs back and computes the **charge density difference**

```
Δρ(r) = ρ_combined(r) − Σ_i ρ_fragment_i(r)
```

with what you need to read it and to trust it: the consistency of the runs (grid, cell, mesh SIESTA actually used, level of theory, pseudopotentials, SCF, electron count), the **counterpoise-corrected interaction energy** from the same runs, the statistics of $\Delta\rho$, the charge gained or lost by each fragment and atom, the charge that crossed the plane between two fragments, the **dipole** the rearrangement induces (checked against SIESTA's own dipoles) and the work-function shift it causes, a 2D map, a planar profile, a Gaussian cube for isosurfaces and a 4-panel dashboard.

It answers a question none of `5.1`–`5.15` does: not how strongly two things bind (that is `5.8` adsorption, `5.3` cohesive energy) but **where the electrons go when they meet** — which regions gain and lose charge, how much moves, how much crosses from one fragment to the other, and what dipole that leaves behind. The worked example (Section 5) runs the whole thing for real, with SIESTA, on three systems that span the chemistry of an interface — a hydrogen bond (the water dimer), a dative bond (ammonia borane) and an alkali atom on a 2D sheet (Li on graphene) — plus three studies on the water dimer (basis set, mesh, distance) that show what the numbers depend on.

Both stages live in this one folder and this one tutorial — same reasoning as `5.1`–`5.5`, `5.13`–`5.15`: Stage 1's output exists only to feed Stage 2.

11.16.1.1 1. Theory

Symbols at a glance

Symbol	Meaning
ρ_AB(r), ρ_A(r)	electron density of the combined system and of fragment A (e/Å^3)
$\Delta\rho$(r)	charge density difference; > 0 where the electrons accumulate
Q_moved = ½ $\int$ abs($\Delta\rho$)	the charge moved by the interaction (e)
Δq_A	net charge change of fragment A (e; > 0: it gained electrons)
ρ(z), λ(z)	planar average of $\Delta\rho$, and its planar integral λ = A ρ (e/Å)
ΔQ(z)	charge below the plane at z, $\int$ λ dz' (e)
$\Delta\mu$	dipole induced by the rearrangement (e Å, or Debye: 1 e Å = 4.803 D)
$\Delta\Phi$	work-function shift caused by $\Delta\mu$ (eV)
E_X(Y)	energy of fragment X computed in the basis of the set of atoms Y (eV)
E_int, E_def, E_bind	interaction, deformation and binding energies (eV or kcal/mol; 1 eV = 23.06 kcal/mol)
BSSE	basis-set superposition error
A	cell area normal to the profile axis (Å^2)

1.1 What the difference shows. Bring two fragments `A` and `B` together, keep their geometries, and let the electrons relax. The density of the pair is not the sum of the densities of the parts: some electrons moved. The difference

```
Δρ(r) = ρ_AB(r) − ρ_A(r) − ρ_B(r)                                   (1)
```

evaluated with every density **at the same atomic positions**, isolates that rearrangement from everything that was already there. Its integral is zero — nothing is created, only moved:

```
∫ Δρ d³r = N_AB − N_A − N_B = 0                                     (2)
```

What the map shows depends on the interaction:

- **Polarization** — each fragment's cloud is distorted by the field of the other: gain and loss side by side *inside the same fragment*, a dipole-like pattern. Present in every interaction.
- **Charge transfer** — electrons cross from one fragment to the other: a net gain on one side and a net loss on the other. Large for a dative bond or an alkali on a surface, a few hundredths of an electron for a hydrogen bond.
- **Covalent bonding** — accumulation between the atoms that bond, depletion from the lone pair or orbital that formed it.
- **Pauli repulsion** — depletion where two closed shells overlap, pushing density back onto each fragment.

The magnitude of the effect is condensed in the **charge moved**,

$$Q_{moved} = \int_{\Delta\rho>0} \Delta\rho \, d^3r = -\int_{\Delta\rho<0} \Delta\rho \, d^3r = \tfrac{1}{2} \int |\Delta\rho| \, d^3r \qquad (3)$$

which counts polarization and transfer together, and in the **net charge change of each fragment** Δq_A, which only counts the transfer (Section 1.5) but needs a rule to decide which part of space belongs to whom (Section 1.6).

$\Delta\rho$ of fragments is different from the *deformation density* $\rho - \Sigma \rho_{atom}$ of crystallography, which subtracts spherical free atoms and shows every bond of the system. This workflow subtracts the fragments you choose, so only the interaction between them remains; split the system into single atoms (`--fragment-sizes 1,1,1,...`) and you get the deformation density back.

1.2 Three conditions for a meaningful subtraction. Eq. 1 is evaluated **point by point** on SIESTA's real-space grid. That only works if:

1. **The same geometry.** The fragments are not relaxed apart: ρ_A is computed with the atoms of `A` exactly where they are in `AB`. (The energy of relaxing them is a separate quantity, Section 1.4.)
2. **The same grid.** Every density must sit on the same grid points. SIESTA builds its grid from the cell and `Mesh.Cutoff`; the same cell and the same cutoff give the same grid. Stage 1 forces `MeshCutoff` into every folder, and Stage 2 checks the mesh SIESTA actually used — which is not the requested one: SIESTA rounds up to a grid that fits the cell (320 Ry requested, 361 Ry used for the 14 Å box of the example). A different grid is refused (it cannot be subtracted); a different cutoff with the same grid is flagged.
3. **The same basis.** SIESTA expands the orbitals in numerical atomic orbitals (NAOs) centred on the atoms. A fragment computed alone has only its own orbitals; in the complex it can also use its partner's. Section 1.3.

There is also the **egg-box effect**: the integration grid is fixed in space while the atoms are not, so energies and densities depend slightly on where the atoms sit relative to the grid points. Because every folder has the atoms at the same place on the same grid, this error is the same in every term of Eq. 1 and largely cancels — one more reason the three conditions matter.

1.3 Ghost atoms and the basis-set superposition error. In a finite atom-centred basis the energy of a fragment is lowered when extra basis functions are nearby, whether or not they belong to anything physical. In the complex, `A` borrows `B`'s orbitals (and vice versa) and is artificially stabilized; the energy difference `E_AB − E_A − E_B` then includes a spurious attraction, the **basis-set superposition error** (BSSE). The **counterpoise** correction of Boys and Bernardi [1] computes every fragment in the *full* basis of the complex:

$$E_{int(CP)} = E_{AB}(AB) - E_A(AB) - E_B(AB) \qquad (4)$$

```
E_int(raw) = E_AB(AB) − E_A(A) − E_B(B)
BSSE       = E_int(CP) − E_int(raw) = [E_A(A) − E_A(AB)] + [E_B(B) − E_B(AB)] ≥ 0   (5)
```

where `E_X(Y)` is the energy of `X` in the basis of the atoms `Y`. The partner's orbitals are supplied by **ghost atoms**: in SIESTA, a species with a *negative* atomic number in `%block ChemicalSpeciesLabel` places its orbitals at the atom's position with no nucleus and no electrons. Stage 1 always does this — every fragment folder contains every other fragment as ghosts — so

- the **density** of each fragment is computed in the same basis as the combined system, and $\Delta\rho$ contains no basis artefact near the partner;
- the three energies of the runs **are** the counterpoise interaction energy (Eq. 4): Stage 2 reports it at no extra cost.

The BSSE shrinks as the basis grows (Section 5.3 measures it); in the complete-basis limit it vanishes and the two energies of Eq. 4 coincide. Whether the full counterpoise *over*corrects has been debated for decades [2, 3]; for density functionals a systematic assessment [4] finds that counterpoise-corrected interaction energies with a double-ζ basis are close to the complete-basis values — and SIESTA's DZP is a double-ζ basis.

1.4 Interaction energy and binding energy. `E_int` of Eq. 4 is evaluated at the geometry of the complex. The fragments are not at their own equilibrium there: their geometries are *deformed* by the interaction. The energy needed to deform them is

```
E_def = Σ_X [ E_X(X; geometry in the complex) − E_X(X; relaxed alone) ]  ≥ 0      (6)
```

computed in the fragments' own basis [5], and the **binding energy** — what you would measure to separate the complex into relaxed fragments — is

```
E_bind = E_int(CP) + E_def,          D_e = −E_bind                                (7)
```

For a hydrogen bond `E_def` is a small correction; for a dative bond it can be a large fraction of `E_int` (Section 5.5). Stage 2 reports `E_int` and says so; the example computes `E_def` from relaxed monomers.

1.5 Planar averages, the charge across a plane, and the induced dipole. For a layered system — a slab and an adsorbate, two layers, two molecules facing each other — the natural view of $\Delta\rho$ is its **planar average** along the normal `z`:

```
λ(z) = ∫∫_cell Δρ(x, y, z) dx dy                    (e/Å)          (8)
ΔQ(z) = ∫_0^z λ(z') dz'                             (e)            (9)
```

`ΔQ(z)` is the charge that sits below the plane at height `z`: if the two fragments are separated along `z`, its value at a plane between them is the **net charge transferred**, with no partition of space needed — only the choice of the plane, which Stage 2 puts halfway between the two fragments' closest atoms along the axis. Where λ is non-zero at that plane the number depends on the plane; it is unambiguous when λ vanishes in a vacuum gap between the fragments.

The first moment of the profile is the **dipole induced** by the rearrangement,

```
Δμ_z = −∫ z λ(z) dz                  (electrons counted positive; e Å)          (10)
```

measured from the middle of the vacuum so the cell boundary does not cut the system. Since the nuclei are the same in the combined system and in the fragments, $\Delta\mu$ must equal the difference of the total dipoles SIESTA prints in each run, μ_AB − Σ μ_X: Stage 2 checks it (Section 4.3), a strong test of the whole chain (grids, units, signs). For a **periodic 2D system** a dipole layer of density $\Delta\mu$/A makes the electrostatic potential jump across the layer (Helmholtz):

$$\Delta\Phi = -e\,\Delta\mu_z \;/\; (\varepsilon_0\,A) = -180.95\ \text{eV Å}\cdot\Delta\mu_z(\text{e Å})\;/\;A(\text{Å}^2) \qquad (11)$$

the change of the work function on the +z side relative to the −z side. An alkali atom that gives its electron to the surface points $\Delta\mu$ away from the surface and *lowers* the work function on its side — the classic alkali-adsorption effect [6].

1.6 Charges of fragments and atoms: a partition is a choice. The *net* charge of a fragment inside a complex is not an observable: it needs a rule that assigns each point of space to a fragment or an atom. Every rule gives a different number [7]:

- **Voronoi / nearest atom** — each grid point belongs to the nearest atom. Simple, parameter-free, works on $\Delta\rho$ directly. The one Stage 2 uses (`[5]` of the report).
- **Bader** (QTAIM) — the zero-flux surfaces of the total density [8] (`stb-bader`, 2.6).
- **Hirshfeld** — shares each point in proportion to free-atom densities [9] (`stb-hirshfeld`).
- **Mulliken** — partitions the basis functions, not space; strongly basis-dependent.

Two things make the numbers of this workflow more robust than an absolute charge: they are *differences* (of the same partition applied to ρ_AB and to Σ ρ_X), so much of the arbitrariness cancels; and the plane-integrated value of Section 1.5 needs no partition at all. In the example they agree to 1e-4 e for the water dimer, whose fragments are well separated along the axis (Section 5.2). For a partition into atoms there is no such cross-check: compare atoms within one system, and systems with the same rule.

1.7 What SIESTA's density is, and the units. The `.RHO` file is the **valence** pseudo-density on the real-space grid: the core electrons of the pseudopotential are not in it, so the total is the number of valence electrons (16 for the water dimer; 75 for Li on 3×3 graphene, whose PseudoDojo Li has its 1s in the valence: 18 × 4 + 3), and near a nucleus the density is the smooth pseudo-density, not the all-electron cusp. Features of $\Delta\rho$ within a few tenths of an ångström of a nucleus are real *pseudo*-valence rearrangements but should not be compared with an all-electron density there.

Units: SIESTA's grid, read by sisl, is in e/Å^3; the report and the `.dat` files use e/Å^3. A Gaussian **cube** file carries its lattice in bohr and, by convention, its values in **e/bohr^3** — `1 e/bohr`^{3} `= 6.748 e/`Å^3 — and Stage 2 writes it that way, so isosurface levels typed in VESTA or VMD and charges integrated there are right. The report prints a first isosurface level in that unit.

Spin. For spin-polarized runs (`--force-spin`), $\Delta\rho$ is the difference of the total densities (up + down); `--spin` gives instead the difference of the spin densities `m =` $\rho\uparrow - \rho\downarrow$, which shows where a moment appears or disappears (Section 5.6).

1.8 Where the electrons can go: what the map is not.

- $\Delta\rho$ is the response **at a fixed geometry**: it does not contain the rearrangement caused by the atoms moving (that is in `E_def`, not in $\Delta\rho$).
- It depends on the **functional**: semilocal functionals (PBE) delocalize electrons too easily and overestimate charge transfer between fragments with very different electronegativity (the self-interaction error); hybrid functionals reduce it. The example's numbers are PBE numbers.
- It depends on the **basis** a little even with ghosts (a richer basis polarizes more freely; Section 5.3) and on the **mesh** very little once the three conditions of Section 1.2 hold (Section 5.4).

- Dispersion (van der Waals) does not change the density in a semilocal functional with an additive D3 correction: `--force-vdw` changes energies and, through relaxation, geometries — not $\Delta\rho$ at a given geometry.

1.9 A calculation you can check by hand. From the report of the water dimer (Section 5.2) — three energies and nothing else:

```
E_AB(AB)  = −962.477198 eV       E_A(AB) = −481.249919 eV       E_B(AB) = −481.035089 eV
E_int(CP) = −962.477198 + 481.249919 + 481.035089 = −0.192190 eV = −4.43 kcal/mol
```

and from the runs without ghost atoms (`fragmentN_alone/`, Section 5.3), `E_A(A) = −481.004966` and `E_B(B) = −481.002416` eV, Eq. 5 gives the BSSE: `0.244953 + 0.032673 = 0.277626 eV = 6.40 kcal/mol`. The dipole check is just as direct: SIESTA prints `Electric dipole (Debye)` in each `calc.out`; subtract the fragments' from the combined one and compare with the `Induced dipole` line of `[5]`.

11.16.1.2 2. Libraries and external dependencies used

- **SIESTA** [10, 11] itself is *not* called by the suite. It does the real work: one SCF calculation per folder, writing the density on the real-space grid (`<label>.RHO`, because `SaveRho` is forced) and the log (`calc.out`) with the energy, the electron count, the mesh used, the SCF convergence and the dipole. The reference data of Section 5 was produced with SIESTA 5.4.2, GGA-PBE [12], PseudoDojo norm-conserving pseudopotentials [13] and Monkhorst–Pack k-grids [14].
- **sisl** [15] — reads the `.RHO` grids (any spin layout) and writes the cube file.
- **numpy** — the subtraction, the integrals, the nearest-atom partition (on fractional coordinates, minimum image), the planar profile and its moments.
- **ASE** [16] / **pymatgen** [17] — structure reading, closest contact between fragments, dimensionality (Stage 1), the `--view` window.
- **matplotlib** [18] — the fragment figure of Stage 1 and the dashboard of Stage 2 (`--png` / `--show`). The `.dat`/`.gplot` pair (`--save-gnuplot`) is for gnuplot, which the suite never invokes.

11.16.1.3 3. Stage 1: generating the folders (`stb-chargediffPrep`, code `5.16.1`)

3.1 What it does. `stb-chargediffPrep` takes the **combined structure** (`-s`, default `structure.fdf`; the real geometry — relax it first), a **fragment split** and your **template** (`--calc`, default `calc.fdf`), and writes

```
<output-dir>/                          (default chargediff_run)
    combined/      structure.fdf (all atoms real), calc.fdf, config_extra.fdf, one pseudopotential
    per label
    fragment1/     structure.fdf (fragment 1 real, every other fragment as ghosts), calc.fdf,
    config_extra.fdf, ...
    fragment2/     ...
    chargediff_manifest.json          the split, read by Stage 2
    chargediff_fragments.png          with --png: the system projected on xy/xz/yz, atoms coloured by
    fragment
    chargediff_prep_report.txt        with --save-report
    references.bib
```

Three ways to split:

Option	Split	Use it for
`--fragment-sizes N1,N2,...`	by atom count, in file order	molecules, any system whose atoms are listed fragment by fragment
`--fragment-species 'C;B,N'`	by element, `;` between fragments	heterostructures whose fragments share no element
`--fragment-manifest [PATH]`	the slab/adsorbate split `stb-adsorb` (5.8) records next to each site	an adsorption site

Every species of every fragment gets **its own label**, as in the adsorption workflow: `O_frag1`, `H_frag2`, and in the folders where they are ghosts `O_frag2_ghost`, with the same pseudopotential file renamed after the label (`-p dojo`, `-p virtual_vault` or a folder). A ghost label has a negative atomic number in `%block ChemicalSpeciesLabel` (Section 1.3). The atom order of the structure is kept in every folder, so atom `i` is the same atom everywhere.

3.2 `config_extra.fdf`: everything mandatory, the template untouched. Every folder's `calc.fdf` is `%include config_extra.fdf` followed by your template byte for byte. SIESTA's fdf reader is first-occurrence-wins, so the forced values beat the template's — even a template written for a relaxation:

Directive	Forced	Why
`MD.VariableCell`	`false`	the cell is shared by all folders
`MD.TypeOfRun` / `MD.Steps`	`CG` / `0`	single point: the geometry is the one you gave (Section 1.2)
`MeshCutoff`	`--mesh-cutoff` (default 400 Ry)	the same real-space grid in every folder (Section 1.2)
`SaveRho`	`true`	SIESTA must write the `.RHO` Stage 2 reads
`Spin polarized`	inherited, or `--force-spin` / `--no-force-spin`	the same spin treatment everywhere
`Slab.DipoleCorrection`	inherited, or `--force-dipole` / `--no-force-dipole`	for a slab with a dipole
`DFTD3 .true.`	inherited, or `--force-vdw` / `--no-force-vdw`	dispersion (energies only, Section 1.8)

"Inherited" means: whatever the combined system's own `config_extra.fdf` next to `-s` says (for example a site of `stb-adsorb`), so the fragments run at the level of theory of the calculation they came from. Everything else — functional, basis, k-grid, SCF settings — is the template's, identical in all folders.

3.3 The report, live.

```
[2] FRAGMENT DEFINITION
------------------------------------------------------------
  Split by        : atom counts 3,3 (in file order)
  Fragments       : 2
Folder     | Formula | Atoms | Atom indices | Species labels (real) | Valence e-
fragment1/ | H2O     | 3     | 1-3          | H_frag1, O_frag1      | 8
fragment2/ | H2O     | 3     | 4-6          | H_frag2, O_frag2      | 8
  Valence electrons: 16 in total (the combined system's SCF must find this number; each fragment
  folder finds only its
  own, ghost atoms carry no electrons).
  Closest contact between fragments: 1.810 Ang (O1 - H5)

[3] LEVEL OF THEORY AND FORCED SETTINGS
------------------------------------------------------------
  Read from the template (identical in every folder):
XC functional ... GGA / PBE    Basis ... split DZP, PAO energy shift 0.01 Ry    SCF k-grid ... 1x1x1
```

```
  Every folder's config_extra.fdf (%include'd on top of calc.fdf, which is the template untouched):
Directive               | Forced value | Template had        | Why
MD.VariableCell         | false        | (absent)            | the cell is shared by all folders
MD.TypeOfRun / MD.Steps | CG / 0       | (absent) / (absent) | single point: the geometry is the one
you gave
MeshCutoff              | 320 Ry       | 320 Ry              | the SAME real-space grid in every
folder, or the densities cannot be subtracted
SaveRho                 | true         | (absent)            | SIESTA must write the .RHO that Stage 2
reads
Spin polarized          | no           | non-polarized       | inherited from the combined folder's
config_extra.fdf
...

[4] FOLDERS WRITTEN
------------------------------------------------------------
Folder     | Real atoms | Ghost atoms | Species labels (SystemLabel entries)           | Files
combined/  | all 6 real | none        | O, H                                           | 5
fragment1/ | 3 real     | 3 ghost     | O_frag1, H_frag1, O_frag2_ghost, H_frag2_ghost | 7
fragment2/ | 3 real     | 3 ghost     | O_frag1_ghost, H_frag1_ghost, O_frag2, H_frag2 | 7
```

([0] is the run metadata, [1] the combined structure — formula, lattice, dimensionality —, [5] the figure, [6] the references, [7] the summary with the next steps and every warning again.)

3.4 Validation. A split that does not cover every atom exactly once (sizes that do not add up, a species missing from `--fragment-species` or in two groups, a manifest whose counts disagree with the structure) is refused before anything is written; so is a missing structure or template. `--fragment-species` cannot separate two fragments that share an element (sulfur in both layers of MoS_2/WS_2, hydrogen in both halves of ammonia borane): use `--fragment-sizes`. A pseudopotential that cannot be found is warned about, per label.

3.5 Running it both ways.

```
stb-chargediffPrep -s water_dimer.fdf --fragment-sizes 3,3 --calc calc.fdf -p dojo --mesh-cutoff 320
--png --save-report
stb-suite            # then: 5 (Workflow) -> 16 (Charge Density Difference) -> 1 (Stage 1)   (or type
5.16.1)
```

11.16.1.4 4. Stage 2: analyzing the results (`stb-chargediffAnalysis`, code `5.16.2`)

4.1 Before running Stage 2. Run SIESTA in every folder (`combined/` and each fragment):

```
for d in chargediff_run/*/; do (cd $d && mpirun -np 4 siesta calc.fdf > calc.out); done
```

Stage 2 needs, in each folder, the `.RHO` (forced by Stage 1) and one `*.out` log (for the energy, the electron count, the mesh, the SCF convergence and the dipole; without it those parts are skipped, not fatal). `--dir` is the folder Stage 1 wrote into; the split is read from `chargediff_manifest.json`.

4.2 Report structure.

Section	Content
[0]	run metadata

Section	Content
`[1]`	the system: cell, grid (shape, spacing, voxel), SIESTA version, one line per folder (atoms real/ghost, $\int\rho$, SIESTA's electron count, energy)
`[2]`	the **consistency checks** (Section 4.3)
`[3]`	the **counterpoise interaction energy** (Eq. 4) in eV, kcal/mol, kJ/mol
`[4]`	statistics of $\Delta\rho$: integral, charge moved (Eq. 3), rms, the extremes with the nearest atom, an isosurface level
`[5]`	charge change by **atom and fragment** (nearest-atom partition), the **planar view**: separating plane, charge across it, induced dipole, work-function shift
`[6]`	the map (slice, profile or 3D) and the cube
`[7]`	references
`[8]`	summary and files
`[9]`	every warning again

4.3 `[2]`: is the subtraction legitimate?.

Check	Passes when	If it fails
Same real-space grid in every folder	the `.RHO` shapes agree (otherwise Stage 2 stops: nothing to subtract)	rerun with the same `MeshCutoff` and cell
Same cell in every folder	within 1e-3 Å (otherwise it stops)	the folders come from different structures
Same level of theory	functional, basis, smearing, k-grid, spin, D3 ... read from each folder's `config_extra.fdf` + `calc.fdf` in SIESTA's reading order	a folder was edited: rerun it
Same pseudopotential files	the set of pseudopotential contents (md5) is the same	labels differ by design, contents must not
Same mesh actually used by SIESTA	the `Mesh cutoff (required, used)` lines agree	rerun with the same `MeshCutoff`
SCF converged in every run	each log has `SCF cycle converged`	more iterations, smaller mixing
Electron count of the combined `.RHO` equals the sum over fragments	`abs(ΔN) < 1e-3 e`	a wrong electron count in one run (charge, pseudopotential)
Net $\Delta\rho$ vanishes	`abs(`$\int\Delta\rho$`) < 1e-3 e` (Eq. 2)	same
Induced dipole equals SIESTA's own dipoles	within 0.005 e Å or 5 % (Eq. 10)	a run without the dipole line, or a profile axis without vacuum

A grid integral of a folder's density that differs from SIESTA's electron count by more than 5e-3 e is flagged too.

4.4 `[5]`: where the charge went. The nearest-atom partition (Section 1.6) of $\Delta\rho$, atom by atom and summed per fragment, then the **planar view** along the profile axis — by default the lattice axis along which the fragments' centres are farthest apart; `--axis` chooses it:

- **Separating plane** — halfway across the gap between the two fragments' closest atoms along the axis (only for two fragments that do not overlap along it), and the net charge on each side (Eq. 9): the partition-free charge transfer, printed next to the partition's;
- **Induced dipole** (Eq. 10) when the axis has vacuum, with SIESTA's value from the logs;
- **Work-function shift** (Eq. 11) when the two other axes are periodic (a slab).

4.5 Maps and files.

Option	Writes
(default) slice	<dir>_chargediff_slice.dat: a plane normal to --axis (default z) at --pos (default: the plane with the largest $\int$ abs($\Delta\rho$))
--profile	<dir>_chargediff_profile.dat: the planar average along --axis
--3d [--iso-min X]	<dir>_chargediff_3d.dat: the grid points with abs($\Delta\rho$) $\geq$ X
(default) cube	<dir>_chargediff.cube, in e/bohr^3 (--no-cube to skip)
--show / --png	the 4-panel dashboard on screen / <dir>_chargediff_summary.png (without a display --show saves the PNG)
--save-gnuplot	a .gplot next to the .dat (--contour overlays contours, --vmin/--vmax fix the colour range)
--save-data	<dir>_chargediff_atoms.csv (charge change per atom) + <dir>_chargediff_results.json (every number of the report)
--save-report	chargediff_analysis_report.txt

The **dashboard**: the slice with the atoms near it (zoomed on the atoms of an isolated system); the planar integral λ(z) and the cumulative charge ΔQ(z) with every atom and the separating plane marked; the charge change by atom, coloured by fragment; a summary (integral, charge moved, extremes, E_int, charge by fragment and across the plane, dipole, work-function shift, checks).

4.6 Guard rails. A grid or cell that differs between folders stops the run with an [ERROR] (the subtraction is impossible); a missing chargediff_manifest.json too; --spin on runs that are not all spin-polarized too. Everything else — a different setting, an unconverged SCF, a missing log, an electron count that does not add up — is a **warning** in [2] and again in [9], and the analysis goes on.

4.7 Running it both ways.

```
stb-chargediffAnalysis --dir chargediff_run --png --save-data --save-report
stb-chargediffAnalysis --dir chargediff_run --profile --axis 2 --save-gnuplot --show
stb-suite           # then: 5 -> 16 -> 2  (or type 5.16.2): it asks for the mode, axis, position and
outputs
```

11.16.1.5 5. Worked example: three bonds, end to end

5.1 Set-up. Three systems, chosen to span the strength and the nature of an interaction:

System	Bond	Fragments	Cell	Template
water dimer $(H_2O)_2$	hydrogen bond	acceptor H_2O / donor H_2O (--fragment-sizes 3,3)	14 Å cube	calc.fdf
ammonia borane H_3N–BH_3	dative bond	NH_3 / BH_3 (--fragment-sizes 4,4)	12 Å cube	calc.fdf
Li on 3×3 graphene	alkali on a 2D sheet	C_{18} / Li (--fragment-sizes 18,1)	7.40 × 7.40 Å, 20 Å along c	calc_graphene.fdf

All with SIESTA 5.4.2, PBE [12], PseudoDojo pseudopotentials [13] (Li with its 1s in the valence: 3 electrons), DZP basis with PAO.EnergyShift 0.01 Ry. The molecules: Γ only, MeshCutoff 320 Ry (SIESTA uses 361 Ry for the 14 Å box), 300 K smearing. Li on graphene: a 6×6×1 k-grid, MeshCutoff 300 Ry (322 Ry used), spin-polarized (the isolated Li has one unpaired electron) and the slab dipole correction, both forced in every folder by --force-spin --force-dipole; PAO.Polarization.NonPerturbative true, because SIESTA cannot build the perturbative polarization orbital of this Li. Each complex was relaxed first (CG, fixed cell, 0.01 eV/Å), and the monomers H_2O, NH_3, BH_3 alone (for E_def). The geometries are

sensible: O···O = 2.777 Å (Section 5.4 explains why it is short), B–N = 1.642 Å against ≈ 1.67 Å from the microwave spectrum [19], Li 1.743 Å above a sheet that stays flat (0.003 Å of corrugation).

```
stb-chargediffPrep -s water_dimer.fdf --fragment-sizes 3,3 --calc calc.fdf -p dojo --mesh-cutoff 320
-o water
for d in water/*/; do (cd $d && mpirun -np 2 siesta calc.fdf > calc.out); done
stb-chargediffAnalysis --dir water --png --save-data --save-report
```

Every set also ran each fragment **without** its ghost atoms (`fragmentN_alone/`, `build_inputs.py alone`) — the uncorrected energy of Eq. 4 — so the BSSE is measured, not assumed.

5.2 The water dimer: a hydrogen bond. Stage 2 on the three finished runs (`[1]`–`[3]`, shortened):

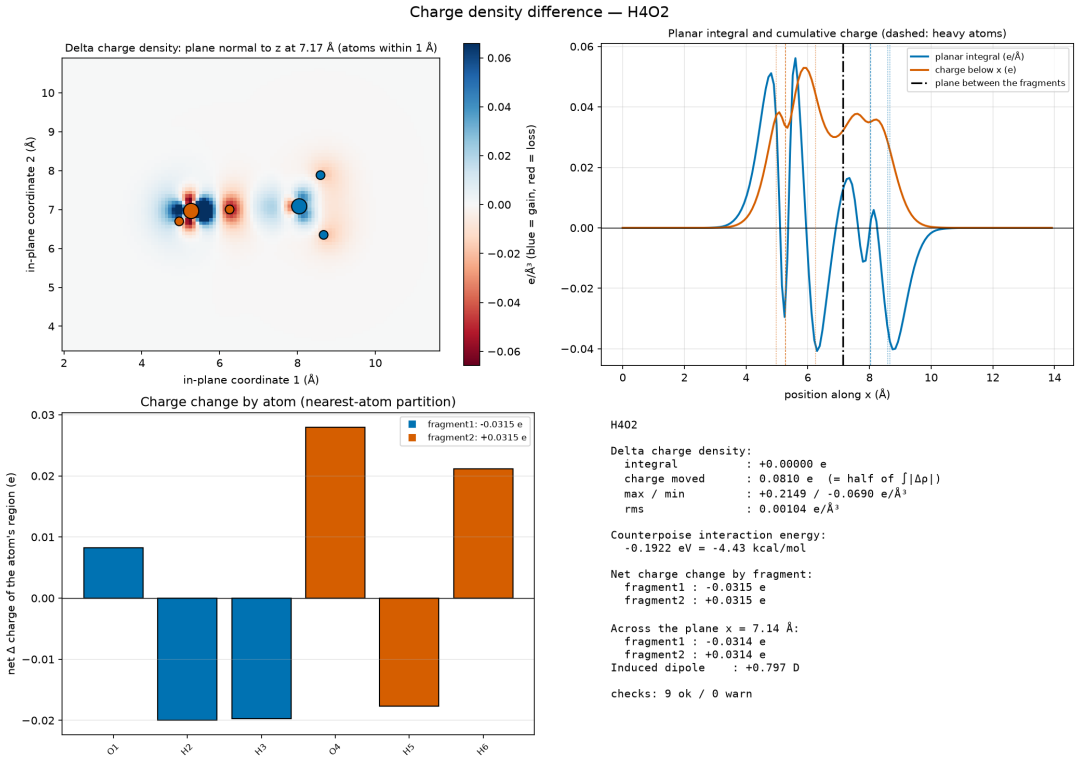
```
Folder     | Atoms | Basis            | ∫rho (e) | SIESTA electrons | E_KS (eV)
combined/  | 6     | all atoms real   | 16.0000  | 16.0000          | -962.477198
fragment1/ | 3     | 3 real + 3 ghost | 8.0000   | 8.0000           | -481.249919
fragment2/ | 3     | 3 real + 3 ghost | 8.0000   | 8.0000           | -481.035089

Same real-space grid in every folder                                          | OK     | 160x160x160
Same mesh actually used by SIESTA                                             | OK     | 361.0 Ry
Electron count of the combined .RHO equals the sum over fragments (< 0.001 e) | OK     | 16.0000 vs
16.0000 e
Net Delta rho vanishes (|integral| < 0.001 e)                                 | OK     | +6.70e-07 e
Induced dipole from Delta rho equals SIESTA's own dipoles (along x)           | OK     | +0.797 vs
+0.797 D
   (and the 4 other checks: cell, level of theory, pseudopotentials, SCF -- all OK)

E_int (CP)     : -0.1922 eV = -4.43 kcal/mol = -18.54 kJ/mol
```

and what moved (`[4]`, `[5]`):

```
Charge moved   : 0.0810 e gained = 0.0810 e lost (half of the integral of |Delta rho|)
maximum (gain) | +0.21494        | 5.51 7.00 7.17 | O4 (O_frag2), 0.25 Ang
minimum (loss) | -0.06902        | 5.25 6.74 7.17 | O4 (O_frag2), 0.24 Ang
fragment1     : -0.03148 e (loses electrons)          <- the acceptor
fragment2     : +0.03148 e (gains electrons)          <- the donor
Separating plane: x = 7.140 Ang, halfway across the 1.79 Ang between the two fragments' closest atoms
along x
Charge across  : fragment1 side -0.03145 e; fragment2 side +0.03145 e
Induced dipole : +0.1660 e Ang = +0.797 D along +x (electrons pushed toward -x); SIESTA's dipoles
give +0.797 D
```



Dashboard of the water dimer

Fig. 1 — The dashboard of the water dimer (`--png`). The donor (orange) is on the left, its bonding H at $x \approx 6.3$ Å; the acceptor (blue) on the right. Slice through the molecular plane; planar integral and cumulative charge along x with the separating plane; charge change per atom; summary.

Reading it:

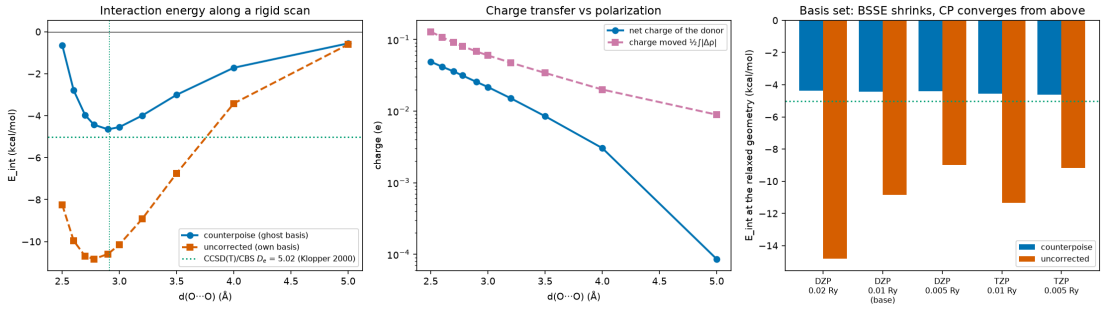
- **The pattern is the textbook one.** Along the O–H···O line: the bonding H of the donor *loses* density (red, x ≈ 6.3 Å), the region between it and the acceptor oxygen *gains* (the acceptor's lone pair pushed toward the proton, x ≈ 7.3 Å), and the donor oxygen gains strongly — the O–H bond polarizes toward O as the proton is pulled away. The extremes sit within 0.25 Å of the donor oxygen, where the (pseudo-)valence density is large; their magnitude says little (Section 1.7), the 1e-3 e/Å^3 contour over 1 % of the box says where the effect lives.
- **Net transfer: 0.031 e from the acceptor to the donor** — the direction of the charge-transfer (n → σ^*) picture of the hydrogen bond: the acceptor's lone pair donates into the donor's O–H antibond. The partition-free value across the plane between the molecules is the same to 3e-5 e: the two molecules are well separated along x and both rules see the same thing.
- **The charge moved (0.081 e) is 2.6× the net transfer**: most of what happens is polarization *inside* each molecule (Section 1.1).
- **The induced dipole, 0.797 D, is SIESTA's own number** to the last digit: the dipole of the dimer minus those of the two ghosted monomers. The grids, the units, the signs and the planar integration are all checked by this one line.
- **Energies:** `E_int(CP) = −4.43 kcal/mol` at the relaxed geometry. With the deformation of the monomers (0.08 kcal/mol, from `relax/h2o`) the binding energy is `−4.35 kcal/mol`, against the CCSD(T) complete-basis `D_e = 5.02 ± 0.05 kcal/mol` [20]. Section 5.4 shows the geometry is part of that difference.

5.3 The basis-set superposition error, measured. `python3 study_table.py` collects every set; `[A]` and `[B]`:

System	`E_int` CP	`E_int` raw	BSSE	`E_def`	`E_bind`	charge moved (e)	fragments (e)
$(H_2O)_2$	−4.43	−10.83	+6.40	+0.08	−4.35	0.081	−0.031 / +0.031
H_3N–BH_3	−47.57	−55.58	+8.01	+12.46	−35.11	0.447	−0.204 / +0.204
Li / graphene 3×3	−17.16	−22.63	+5.47	+0.33	−16.83	0.458	+0.204 / −0.204

(kcal/mol; "raw" is Eq. 4 with each fragment in its own basis, `E_bind = E_int(CP) + E_def`.)

Water dimer, basis (same geometry)	`E_int` CP	`E_int` raw	BSSE	charge moved (e)	donor (e)
DZP, 0.02 Ry	−4.36	−14.80	+10.44	0.0804	+0.0308
DZP, 0.01 Ry (base)	−4.43	−10.83	+6.40	0.0810	+0.0315
DZP, 0.005 Ry	−4.41	−9.00	+4.59	0.0817	+0.0318
TZP, 0.01 Ry	−4.55	−11.34	+6.79	0.0822	+0.0312
TZP, 0.005 Ry	−4.62	−9.17	+4.55	0.0833	+0.0317



Water dimer studies

Fig. 2 — The water dimer: interaction energy along a rigid O···O scan, with and without the counterpoise correction (left); net charge transfer and charge moved along the same scan, log scale (middle); the five basis sets (right). Dotted green: the CCSD(T)/CBS `D_e` *and* `R_e` *[20].* `python3 study_table.py --png`.

- **The BSSE is larger than the interaction itself** with SIESTA's default orbitals: 6.4 kcal/mol for a 4.4 kcal/mol bond, 10.4 with the shorter orbitals of `PAO.EnergyShift 0.02 Ry`. Without the ghost atoms the water dimer would come out bound by 9–15 kcal/mol — two to three times too much — and the number would depend mostly on the orbital radii.
- **It is not shared equally.** Of the 0.278 eV, 0.245 eV is the acceptor's (its energy drops that much when the donor's orbitals are added) and 0.033 eV the donor's: the donor's hydrogen sits 1.79 Å from the acceptor's oxygen, right in its lone-pair region, and its orbitals are what the acceptor borrows. Ammonia borane shows the same asymmetry (0.277 eV for NH_3, 0.070 eV for BH_3).
- **Longer orbitals reduce it** (10.4 → 3.4 → 5.6 kcal/mol from 0.02 to 0.005 Ry, for both DZP and TZP), and a larger basis does not by itself: TZP at 0.01 Ry has as much BSSE as DZP. In NAO bases the *range* of the orbitals matters more than their number for this error.
- **The counterpoise value hardly moves**: −4.36 to −4.62 kcal/mol over five bases, approaching the reference from above as the basis grows — the behavior expected of a CP-corrected DFT interaction energy [4]. The density does not move either: the net transfer varies by 3 % and the charge moved by 4 % across the bases. With the ghosts in place, $\Delta\rho$ is a basis- converged quantity well before the energy is.

5.4 The mesh, and the distance. `[C]` repeats the base set at five `MeshCutoff` values:

MeshCutoff (Ry)	used	E_int CP	$\int \Delta\rho$ (e)	charge moved (e)	donor (e)	max (e/Å^3)
150	164.5	−4.43	+1.1e-6	0.0809	+0.0319	0.2045
200	203.1	−4.43	−2.7e-7	0.0810	+0.0319	0.2041
320	361.0	−4.43	+6.7e-7	0.0810	+0.0315	0.2149
450	456.9	−4.43	−4.9e-7	0.0810	+0.0315	0.2174
600	657.9	−4.43	+2.1e-7	0.0810	+0.0317	0.2142

Because every term of Eq. 1 is computed with the atoms at the same place on the same grid, the egg-box error cancels: `E_int` does not change in the second decimal from 150 to 600 Ry, nor does the charge moved. What changes is the *peak* value (by 6 %), a point property sampled on a grid of spacing 0.14 → 0.07 Å near a nucleus, and the per-fragment charge in the fourth decimal. The column *used* is the cutoff SIESTA actually used (the grid has to fit the cell): two runs with "the same" requested cutoff in different cells can differ, which is why Stage 2 compares the used value. Case 7 of the script edits one log to another mesh and shows the warning.

`[D]`: the donor moved rigidly along O···O (the monomers keep their geometry in the dimer):

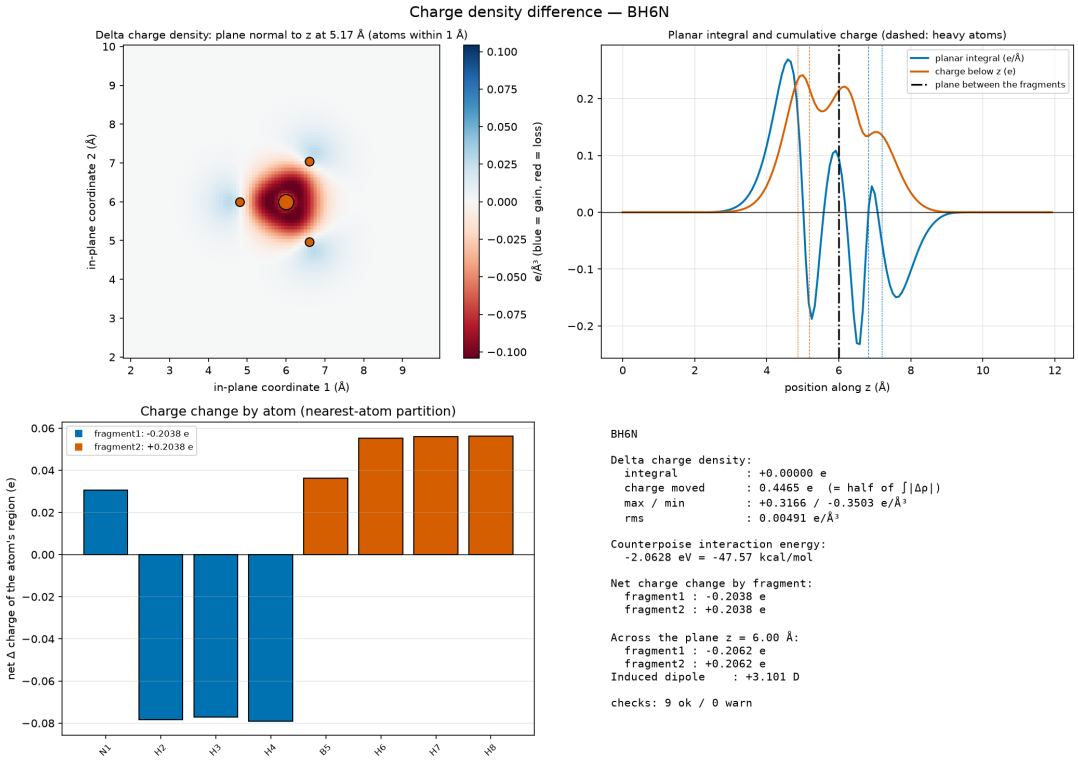
d(O···O) (Å)	E_int CP	E_int raw	−BSSE	donor (e)	charge moved (e)
2.50	−0.64	−8.25	−7.61	+0.0491	0.1288
2.60	−2.79	−9.96	−7.17	+0.0417	0.1077
2.70	−3.97	−10.69	−6.72	+0.0359	0.0912
2.777	−4.43	−10.83	−6.40	+0.0315	0.0810
2.90	**−4.65**	−10.59	−5.94	+0.0256	0.0681
3.00	−4.54	−10.14	−5.60	+0.0217	0.0600
3.20	−3.99	−8.91	−4.92	+0.0151	0.0475
3.50	−3.01	−6.75	−3.74	+0.0085	0.0342
4.00	−1.72	−3.42	−1.70	+0.0030	0.0200
5.00	−0.55	−0.61	−0.05	+0.0001	0.0089

- **The counterpoise curve has its minimum at 2.90 Å** (−4.65 kcal/mol), next to the CCSD(T) equilibrium distance of 2.912 Å [20]; the uncorrected curve has it at 2.78 Å, which is exactly where the *uncorrected* relaxation of Section 5.1 stopped. The BSSE grows as the molecules approach (from 0.05 kcal/mol at 5 Å to 7.6 at 2.5 Å) and so pulls the geometry in: a relaxation with a small NAO basis and no counterpoise gives intermolecular distances that are too short. The geometry of the base case is therefore 0.13 Å too short, and its `E_int` (−4.43) is not the minimum of the CP surface (−4.65).
- **Charge transfer is short-ranged, polarization is not.** The net charge of the donor decays exponentially, `q(R)` $\propto$ `exp(−R/0.36 Å)` beyond 2.9 Å — it needs the orbitals of the two molecules to overlap — and is 1e-4 e at 5 Å. The charge moved falls from 0.13 to 0.009 e but much more slowly (Fig. 2, middle): at 5 Å, where nothing crosses any more, each molecule is still polarized by the other's dipole. This is the separation of polarization and charge transfer of Section 1.1, made visible.

5.5 Ammonia borane: a dative bond. The same workflow on H_3N–BH_3 (Case 9 of the script):

```
E_int (CP)     : -2.0628 eV = -47.57 kcal/mol = -199.03 kJ/mol
Charge moved   : 0.4465 e gained = 0.4465 e lost
maximum (gain) | +0.31659        | 6.00 6.00 5.75 | B5 (B_frag2), 0.57 Ang
minimum (loss) | -0.35030        | 6.00 6.00 6.58 | N1 (N_frag1), 0.24 Ang
fragment1     : -0.20375 e (loses electrons)          <- NH3
fragment2     : +0.20375 e (gains electrons)          <- BH3
Separating plane: z = 5.999 Ang, halfway across the 1.64 Ang between the two fragments' closest atoms
along z
```

```
Charge across  : fragment1 side -0.20621 e; fragment2 side +0.20621 e
Induced dipole : +0.6456 e Ang = +3.101 D along +z; SIESTA's dipoles give +3.101 D
```



Dashboard of ammonia borane

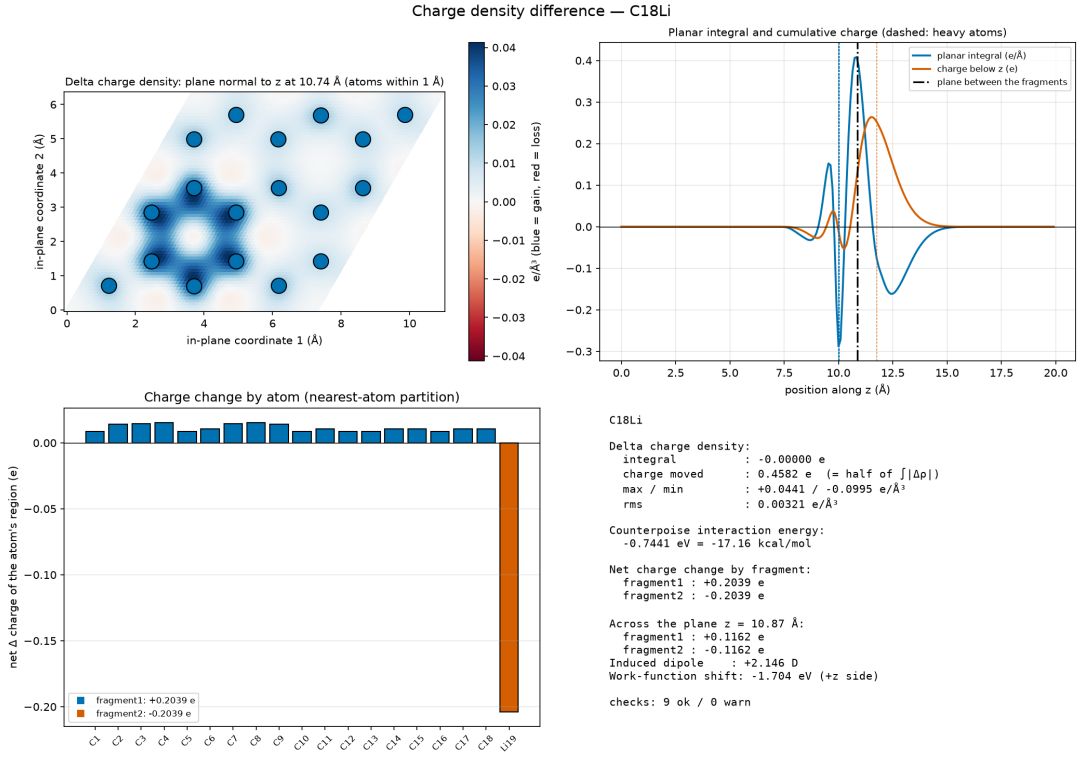
Fig. 3 — Ammonia borane: BH_3 (orange) below, NH_3 (blue) above along z. The planar integral shows the depletion on the nitrogen's side and the accumulation on boron's side of the B–N bond.

- **The donation is visible on the B–N axis**: the largest loss is 0.24 Å from N *on the boron side* (the lone pair leaves), the largest gain 0.57 Å from B *toward N* (it arrives in boron's empty p orbital and builds the B–N bond). 0.21 e crosses the plane between the fragments — seven times the hydrogen bond's — and it is the hydrogens of NH_3 that lose most in the partition (−0.078 e each) while those of BH_3 gain (+0.056 e each): the whole NH_3 becomes more positive, the whole BH_3 more negative, as in the familiar zwitterionic picture $H_3N^+–BH_3^-$.
- **The deformation energy is a large part of the story.** BH_3 is planar alone and pyramidal in the complex: 12.5 kcal/mol to bend it (NH_3: 0.05). `E_int(CP) = −47.6` but `E_bind = −35.1 kcal/mol`. For a dative bond, an interaction energy at the complex geometry is *not* the bond strength; the report says so in `[3]`, the example computes the difference.
- **Against the literature.** The binding energy, 35.1 kcal/mol, is close to the MP4 `D_e` of 34.7 kcal/mol [21]; the best ab initio binding *enthalpy* is −27.5 ± 0.5 kcal/mol [22], which includes zero-point and thermal terms that reduce the electronic `D_e` by several kcal/mol, so the two are not directly comparable. PBE is expected to do reasonably for this bond; the B–N distance (1.642 Å, ≈ 1.67 Å measured [19]) is slightly short.

5.6 Li on graphene: an interface, a profile, a dipole. Case 10 of the script, with `--profile --axis 2`:

```
Same mesh actually used by SIESTA                                | OK      | 322.4 Ry
```

```
Electron count of the combined .RHO equals the sum over fragments (< 0.001 e) | OK     | 75.0000 vs
75.0000 e
Induced dipole from Delta rho equals SIESTA's own dipoles (along z)           | OK     | +2.146 vs
+2.146 D
E_int (CP)     : -0.7441 eV = -17.16 kcal/mol = -71.79 kJ/mol
19 | Li      | Li_frag2 | fragment2 | -0.20395
Separating plane: z = 10.873 Ang, halfway across the 1.74 Ang between the two fragments' closest
atoms along z
Charge across  : fragment1 side +0.11618 e; fragment2 side -0.11618 e
                 (nearest-atom partition above: fragment1 +0.20395 e; fragment2 -0.20395 e)
Induced dipole : +0.4467 e Ang = +2.146 D along +z (electrons pushed toward -z); SIESTA's dipoles
give +2.146 D
Work function  : the dipole layer shifts the vacuum level by -1.704 eV on the +z side relative to the
-z side
```



Dashboard of Li on graphene

Fig. 4 — Li on 3×3 graphene: the slice 0.74 Å above the sheet (the hexagon under Li gains, in blue), the planar integral and the cumulative charge along z with the separating plane, the charge change per atom.

- **Electrons go from Li to the sheet, and mostly to the hexagon under it** (Fig. 4, slice). The induced dipole points away from the sheet (2.15 D, again equal to SIESTA's), and the Helmholtz estimate (Eq. 11) gives a work function **1.70 eV lower** on the Li side at this coverage (one Li per 47.4 Å^2) — the alkali effect [6]. To compute the vacuum-level difference itself, use `stb-workfunction` on the combined run's electrostatic potential.
- **How much charge was transferred depends on what you ask.** The nearest-atom partition gives 0.204 e, the plane halfway between Li and the sheet 0.116 e. Neither is wrong: the 2s orbital of a free Li atom is so diffuse that a good part of it already lies below that plane in the *isolated* fragment, so the plane counts only the change, while the partition counts the change inside Li's

Voronoi cell. A Bader analysis of the total density would give yet another number — the common statement that Li on graphene is "almost fully ionized" refers to that partition of the total density, not to a difference. SIESTA's Mulliken analysis of a similar system gave 0.39 e [24]. Quote the number together with its definition (Section 1.6).

- **Energies.** `E_int(CP) = −0.744 eV`, the deformation of the sheet is negligible (0.014 eV), so `E_bind = −0.73 eV`; without the counterpoise correction the same runs give −0.97 eV. Other GGA calculations report about 1.0–1.1 eV for Li at the hollow site of graphene [23, 24] (other codes, bases and coverages; the SIESTA one [24] does not say it applies a counterpoise correction). Our two numbers bracket the range; with a diffuse 2s electron and a DZP basis this is a case where the counterpoise correction may overshoot, and a larger basis or a plane-wave check would settle it.
- **Spin, and a caveat.** The isolated Li carries 1 μB, the sheet with the Li ghost none, so the spin-density difference (`--spin`) integrates to the disappearance of the Li moment. Here it integrates to −1.17 μB, not −1.00: the single point of the combined system converged to a weakly polarized state (−0.17 μB) that lies 10 meV *below* the non-magnetic state the relaxation ended in. This metallic 3×3 cell with a 6×6 k-grid may have a weak spin instability, or the two SCF runs found different local minima; these runs cannot tell which. It shifts `E_int` by ≈ 0.01 eV and leaves the charge density difference essentially unchanged, but it is the kind of thing `--spin` is for: check the moments of every run (the script prints them) before reading a spin-density map.

5.7 How far to trust each number.

Quantity	What moves it (this example)	Verdict
$\int \Delta\rho$, electron counts	nothing (1e-6 e)	a check, not a result
induced dipole	nothing: equal to SIESTA's to 0.001 D	reliable whenever the axis has vacuum
`E_int` (CP)	basis ±0.13 kcal/mol, mesh < 0.01, geometry 0.2 (water)	good to a few tenths of kcal/mol for a given geometry
`E_int` (raw)	basis: 9 to 15 kcal/mol for the same bond	do not use with NAO bases
`E_bind`	`E_def` from relaxed monomers; the geometry of the complex (relaxed without CP: too short)	relax with care; for dative bonds `E_def` is essential
charge moved	basis 4 %, mesh 0.1 %	robust
net transfer, plane-integrated	basis 3 %, mesh 1 %	robust *as defined*
net transfer, partitioned	the partition rule (factor 2 for Li/graphene)	quote with its definition
extreme values of $\Delta\rho$	mesh 6 %, near the nuclei (pseudo-valence)	locate features, do not quote
work-function shift (Eq. 11)	coverage, depolarization	an estimate; compute it with `stb-workfunction`

5.8 What to take from the example.

1. The ghost atoms are not optional with a NAO basis: the BSSE of the water dimer is larger than its interaction energy, and it pulls relaxed geometries in. The same runs that give $\Delta\rho$ give the counterpoise energy for free.
2. Read `[2]` first. The checks are cheap and they catch every mistake that makes a difference map meaningless — another grid, another mesh, another level of theory, a wrong electron count. The dipole check closes the loop: one number computed two independent ways.
3. Polarization and charge transfer are different things: the charge moved counts both, the charge across a plane only the transfer, and their ratio (2.6 for the hydrogen bond at equilibrium, ~90 at 5 Å) says which dominates.
4. A net charge is a definition. For two well-separated fragments the partition and the plane agree

(water, ammonia borane); for a diffuse donor on a surface they differ by a factor of two (Li). Report the definition with the number.

5. An interaction energy at the complex geometry is not a binding energy: add the deformation of the fragments (12.5 kcal/mol for BH_3).

11.16.1.6 6. Reproducing the reference data

`reference_results/` holds every finished SIESTA run of Section 5, so Stage 2 works out of the box without a SIESTA installation.

Folder	What	Contents
`relax/water_dimer`, `relax/nh3bh3`, `relax/li_graphene`	CG relaxations (fixed cell) of the three complexes	`calc.fdf`, `template.fdf`, start `structure.fdf`, `relaxed.fdf`, `calc.out`
`relax/h2o`, `relax/nh3`, `relax/bh3`	the monomers relaxed alone (for `E_def`)	same
`graphene_flat/`	the flat 3×3 sheet before Li (for the sheet's `E_def`)	`calc.fdf`, `template.fdf`, `structure.fdf`, `calc.out`
`water/`, `nh3bh3/`, `li_graphene/`	the three cases	`chargediff_manifest.json`, the prep report, per folder `structure.fdf`, `calc.fdf`, `config_extra.fdf`, `calc.out`, **`siesta.RHO.xz`**; `fragmentN_alone/` (no ghosts); `analysis/` (report, CSV, JSON)
`water_mesh150` … `water_mesh600`	the dimer at `MeshCutoff` 150, 200, 450, 600 Ry	as above, without the `.RHO`
`water_dzp_s002`, `water_dzp_s0005`, `water_tzp_s001`, `water_tzp_s0005`	the dimer with four other basis sets	same
`water_r2.50` … `water_r5.00`	the donor moved rigidly to `d(O···O)` = 2.5–5.0 Å	same

The density grids of the three main cases are stored compressed (`xz`); `example_5.16.sh` unpacks them into `output/` (`xz -d siesta.RHO.xz` by hand). The studies keep their analysis results (report, CSV, JSON) and the SIESTA logs but not the grids, to keep the repository small; `make_reference.sh` regenerates everything:

```
bash make_reference.sh                # ~2 h on 2 cores; writes reference_run/ and reference_results/
```

It relaxes the three complexes and the monomers, runs `stb-chargediffPrep` for every set (the commands of Section 5), SIESTA in each folder, `build_inputs.py alone` for the fragments without ghosts, and `stb-chargediffAnalysis --save-data --save-report`. Computed with SIESTA 5.4.2 (conda-forge build, GNU 14.3, MPI), `mpirun -np 2`.

11.16.1.7 7. Known, deliberate limitations

- **Frozen geometry.** $\Delta\rho$ and `E_int` are at the geometry of the complex; the relaxation of the fragments is in `E_def` (Section 1.4), which Stage 2 does not compute (it needs relaxed fragments: separate runs).
- **The partition is geometric.** The per-atom and per-fragment charges of `[5]` use the nearest-atom rule; they are not Bader or Hirshfeld charges (Section 1.6). The plane-integrated value is partition-free but needs the fragments to be separable along an axis.
- **Valence pseudo-density.** No core electrons, a smooth density near the nuclei (Section 1.7).
- **One functional.** PBE delocalizes charge; the charge transferred between very different fragments is overestimated (Section 1.8). The workflow reports what the functional gives.

- **Same grid required.** The fragments must be computed in the same cell with the same cutoff; a fragment computed in a smaller box cannot be used.
- **The dipole and the work-function shift** need vacuum along the profile axis; the work-function shift is the Helmholtz estimate for a periodic layer, not a computed vacuum-level difference.
- **No convergence by itself.** The tool reports what the mesh, the basis and the geometry do (Section 5); you decide how far to push them.

11.16.1.8 8. Step-by-step: running this workflow on your own system

1. **Relax the combined system** at the level of theory you will use (and the fragments alone, if you want `E_def`).
2. **Write a `calc.fdf` with the physics only** — functional, basis, k-grid, smearing, `Spin` — and `%include structure.fdf`. List the atoms fragment by fragment if you will split by count.
3. **Run Stage 1** with `-p <bank>` and a `--mesh-cutoff` you have converged (Section 5.4). Read `[2]` (is the split what you meant? the electron counts?) and `[3]` (what is forced, what is inherited). Use `--png` to see the split.
4. **Run SIESTA** in `combined/` and every fragment folder.
5. **Run Stage 2** with `--png --save-data`. Read `[2]` first (every check OK?), then `[3]`, `[4]`, `[5]`. For an interface, add `--profile --axis N`.
6. **Visualize** the cube in VESTA (isosurface level from `[4]`, in e/bohr^3) — positive (gain) and negative (loss) lobes in two colours.
7. **Test what the picture depends on** (Section 5): the mesh, the basis, the distance. A feature that survives them is physics.

11.16.1.9 9. Files in this folder

File	What it is
`README.md`	this tutorial
`example_5.16.sh`	guided, runnable walk-through (11 cases)
`study_table.py`	helper of the script: collects every reference set (energies, BSSE, deformation, charges) into the tables of Section 5 and draws `figures/water_studies.png`
`build_inputs.py`	helper of `make_reference.sh`: start structures, the distance scan, the fragments without ghosts, the monomers
`make_reference.sh`	regenerates `reference_results/` with SIESTA (Section 6)
`water_dimer.fdf`	the water dimer (acceptor O H H, donor O H H), 14 Å box
`nh3bh3.fdf`	ammonia borane, staggered start geometry, 12 Å box
`li_graphene.fdf`	Li above a hollow site of 3×3 graphene, 20 Å along c
`calc.fdf`, `calc_graphene.fdf`	the templates: PBE, DZP, `PAO.EnergyShift 0.01 Ry`; the second spin-polarized with a 6×6 k-grid
`reference_results/`	real finished SIESTA runs (Section 6)
`figures/`	the dashboards of the three cases and the water-dimer studies
`output/`	created by the script (git-ignored)

11.16.1.10 10. Running the script

```
bash example_5.16.sh
```

It pauses between cases (`[Press Enter to continue]`); `yes '' | bash example_5.16.sh` runs it straight through. It starts by wiping its own `output/`. Cases 1–3 run Stage 1 (no SIESTA needed); Cases 4–11 run

Stage 2 on `reference_results/`. It sets `MPLBACKEND=Agg`, so `--show` is replaced by `--png` in the script; try it on a desktop:

```
cp -r reference_results/water /tmp/w && xz -d /tmp/w/*/siesta.RHO.xz && stb-chargediffAnalysis --dir
/tmp/w --show
```

11.16.1.11 11. Flag reference

`stb-chargediffPrep` — `-s STRUCTURE` (default `structure.fdf`) · one of `--fragment-sizes N1,N2,...`, `--fragment-species 'El,El;El'`, `--fragment-manifest [PATH]` · `--calc CALC` (default `calc.fdf`) · `--mesh-cutoff RY` (default 400) · `-p PSEUDO` (bank or folder; default: the structure's folder) · `--force-spin`/`--no-force-spin` · `--force-dipole`/`--no-force-dipole` · `--force-vdw`/`--no-force-vdw` · `--output-dir DIR` (default `chargediff_run`) · `--show` · `--png` · `--view` · `--save-report` · `--no-intro` · `-v`.

`stb-chargediffAnalysis` — `--dir DIR` (default `chargediff_run`) · `-o OUTPUT_DIR` (default `.`) · `--profile` | `--3d` · `-a`/`--axis {0,1,2}` · `-p`/`--pos ANG` · `--spin` · `--iso-min RHO` · `--no-cube` · `--vmin`/`--vmax` · `--contour` · `--show` · `--png` · `--save-gnuplot` · `--save-data` · `--save-report` · `--no-intro` · `-v`.

11.16.1.12 12. What's next

- `5.8-adsorption` gives the adsorption energy of a molecule on a surface with its BSSE correction; its sites carry the `fragment_manifest.json` that `--fragment-manifest` reads, so a site's $\Delta\rho$ is one command away.
- `stb-bader` (2.6) and the Hirshfeld tools give atomic charges by other partitions (Section 1.6): compare.
- `stb-density` (2.8) maps a single density (or the difference of two `.RHO` files with `--rho2`, without ghosts or checks); `stb-workfunction` (2.7) computes the work function from the electrostatic potential — the direct counterpart of Eq. 11.
- `5.9-neb`: follow $\Delta\rho$ along a reaction path by running this workflow on the images.

11.16.1.13 13. References

1. S. F. Boys, F. Bernardi, *The calculation of small molecular interactions by the differences of separate total energies. Some procedures with reduced errors*, Mol. Phys. **19**, 553 (1970).
2. J. Collins *et al.*, *The full versus the virtual counterpoise correction for basis set superposition error in self-consistent field calculations*, Chem. Phys. Lett. (1986). doi:10.1016/0009-2614(86)87014-2
3. M. Gutowski *et al.*, *Does the Boys and Bernardi function counterpoise method actually overcorrect the basis set superposition error?*, Chem. Phys. Lett. (1986). doi:10.1016/0009-2614(86)80221-4
4. M. Gray *et al.*, *Systematic evaluation of counterpoise correction in density functional theory*, J. Chem. Theory Comput. (2022). doi:10.1021/acs.jctc.2c00883
5. I. Mayer *et al.*, *Monomer geometry relaxation and the basis set superposition error*, Chem. Phys. Lett. (1992). doi:10.1016/0009-2614(92)85415-7
6. N. D. Lang, *Theory of work-function changes induced by alkali adsorption*, Phys. Rev. B **4**, 4234 (1971).
7. K. B. Wiberg, P. R. Rablen, *Comparison of atomic charges derived via different procedures*, J. Comput. Chem. **14**, 1504 (1993). doi:10.1002/jcc.540141213
8. R. F. W. Bader, *Atoms in Molecules: A Quantum Theory*, Oxford University Press (1990).
9. F. L. Hirshfeld, *Bonded-atom fragments for describing molecular charge densities*, Theor. Chim. Acta **44**, 129 (1977).

10. J. M. Soler *et al.*, *The SIESTA method for ab initio order-N materials simulation*, J. Phys.: Condens. Matter **14**, 2745 (2002).
11. A. García *et al.*, *Siesta: Recent developments and applications*, J. Chem. Phys. **152**, 204108 (2020).
12. J. P. Perdew, K. Burke, M. Ernzerhof, *Generalized gradient approximation made simple*, Phys. Rev. Lett. **77**, 3865 (1996).
13. M. J. van Setten *et al.*, *The PseudoDojo: Training and grading a 85 element optimized norm-conserving pseudopotential table*, Comput. Phys. Commun. **226**, 39 (2018). doi:10.1016/j.cpc.2018.01.012
14. H. J. Monkhorst, J. D. Pack, *Special points for Brillouin-zone integrations*, Phys. Rev. B **13**, 5188 (1976).
15. sisl — https://zerothi.github.io/sisl/ (N. Papior).
16. A. H. Larsen *et al.*, *The atomic simulation environment*, J. Phys.: Condens. Matter **29**, 273002 (2017).
17. S. P. Ong *et al.*, *Python Materials Genomics (pymatgen)*, Comput. Mater. Sci. **68**, 314 (2013).
18. J. D. Hunter, *Matplotlib: A 2D graphics environment*, Comput. Sci. Eng. **9**, 90 (2007).
19. L. R. Thorne, R. D. Suenram, F. J. Lovas, *Microwave spectrum, torsional barrier, and structure of BH_3NH_3*, J. Chem. Phys. **78**, 167 (1983) (B–N ≈ 1.67 Å).
20. W. Klopper *et al.*, *Computational determination of equilibrium geometry and dissociation energy of the water dimer*, Phys. Chem. Chem. Phys. **2**, 2227 (2000). doi:10.1039/a910312k — CCSD(T): `R_e = 2.912 ± 0.005 Å`, `D_e = 21.0 ± 0.2 kJ/mol` (5.02 kcal/mol).
21. J. Binkley *et al.*, *A theoretical study of the properties of BH_3NH_3*, J. Chem. Phys. (1983). doi:10.1063/1.446117 — MP4 B–N dissociation energy 34.7 kcal/mol.
22. J. A. Plumley *et al.*, *Covalent and ionic nature of the dative bond and account of accurate ammonia borane binding enthalpies*, J. Phys. Chem. A (2007). doi:10.1021/jp074937z — binding enthalpy −27.5 ± 0.5 kcal/mol.
23. C. Uthaisar *et al.*, *Lithium adsorption on zigzag graphene nanoribbons*, J. Appl. Phys. (2009). doi:10.1063/1.3265431 — Li at the hollow site of graphene: 1.04 eV (PBE), 1.55 eV (LSDA).
24. G. Kaur *et al.*, *Structures and energetics of lithium adatom and its dimer on graphene — a DFT study*, Appl. Surf. Sci. (2015). doi:10.1016/j.apsusc.2014.07.108 — SIESTA, GGA: ≈ 1.07 eV, Mulliken transfer ≈ 0.39 e.

11.16.2 Command-line reference

11.16.2.1 `stb-chargediffPrep`

Stage 1 of the Charge Density Difference workflow: writes a 'combined/' SIESTA input folder (every atom real, no ghosting) plus one fragment folder per fragment of an N-fragment system (bilayer, molecule-on-slab, etc.) – ALL of them sharing the identical cell/mesh/level of theory, forced by this tool itself. Every fragment folder ALWAYS includes every other fragment as ghost atoms (no opt-out) so every calculation shares the same basis set.

Synopsis.

```
usage: stb-chargediffPrep [-h] [-s STRUCTURE]
                          (--fragment-manifest PATH | --fragment-sizes N1,N2,...,NK | --fragment-species
                          El_a,El_b;El_c;...)
                          [--calc CALC] [--mesh-cutoff RY] [-p PSEUDO_DIR]
                          [--force-spin] [--no-force-spin] [--force-dipole]
                          [--no-force-dipole] [--force-vdw] [--no-force-vdw]
                          [-o OUTPUT_DIR] [--view] [--show] [--png] [--save-report]
                          [--no-intro] [-v]
```

Options

`-s, --structure STRUCTURE`
: Path to the combined system's own .fdf structure file – taken directly as the REAL, authoritative geometry to be split into fragments (structure_io.read_fdf, no directory scanning, no automatic preference for a relaxed '*.XV' that might sit alongside it – point this at whichever file you consider the real geometry, e.g. a fresh `structure.fdf` written from a converged '*.XV' via `stb-translate`). Default: '`structure.fdf`'.

`--fragment-manifest PATH`
: Reuse an existing `fragment_manifest.json` (written by `stb-adsorb`) to recover its slab/adsorbate split – exactly 2 fragments. Pass an empty value ('') to default to '`fragment_manifest.json`' next to `--structure` itself. exclusive group 1

`--fragment-sizes N1,N2,...,NK`
: General split by ATOM COUNT: comma-separated atom counts, in the order fragments appear in the structure file (fragment 1 = first N1 atoms, fragment 2 = next N2, ...). Must sum to the combined structure's total atom count. K >= 2. exclusive group 1

`--fragment-species El_a,El_b;El_c;...`
: General split by CHEMICAL SPECIES, independent of atom order: ';'-separated groups, each a comma-separated list of element symbols belonging to that fragment (e.g. 'C;B,N' for a graphene/h-BN heterostructure). Every species present in the structure must appear in exactly one group. LIMITATION: cannot be used when two fragments physically share a species (e.g. MoS2 on WS2, sulfur in both layers) – use `--fragment-sizes` or `--fragment-manifest` instead in that case. exclusive group 1

`--calc CALC`
: SIESTA input template (.fdf) used for every fragment folder – same basis/SCF/k-grid as the combined system (default: '`calc.fdf`').

`--mesh-cutoff RY`
: Explicit MeshCutoff (Ry) to force in every fragment folder – the SAME value on all of them, never auto-detected from `--calc`/the combined system's own `calc.fdf` (a per-run SIESTA default can silently differ between calculations with different atoms/pseudopotentials, which would corrupt the point-by-point subtraction). Default: 400 Ry – raise it for a finer grid (recommended for a production CDD map) or type a different value here. default: 400; type: float

`-p, --pseudo-dir PSEUDO_DIR`
: Pseudopotentials source: a bundled bank (dojo, virtual_vault) or a folder path (same convention as `stb-adsorb`/`stb-her`/`stb-hubbardu`). Default: reuse the directory `--structure` lives in, which already has every pseudopotential the combined calculation needed.

`--force-spin`
: Force spin-polarized SCF in every fragment folder (default: inherited from the combined system's own `config_extra.fdf`, if any). switch, off by default

`--no-force-spin`
: Force spin-restricted SCF, overriding the combined system's own setting. switch, off by default

`--force-dipole`
: Force the slab dipole correction (default: inherited from the combined system's own `config_extra.fdf`, if any). switch, off by default

`--no-force-dipole`
: Disable the slab dipole correction, overriding the combined system's own setting. switch, off by default

`--force-vdw`
: Force the DFT-D3 dispersion correction (default: inherited from the combined system's own `config_extra.fdf`, if any). switch, off by default

`--no-force-vdw`
: Disable the DFT-D3 dispersion correction, overriding the combined system's own setting. switch, off by default

`-o, --output-dir OUTPUT_DIR`
Root directory to write the fragment folders into (default: chargediff_run).

`--view`
After writing them, open ONE ASE interactive 3D viewer window with the combined structure followed by every fragment as navigable movie frames – ghost atoms render as mostly-hollow circles (see structure_to_view_atoms's docstring). Needs a display. Off by default. switch, off by default

`--show`
Open a matplotlib figure on screen: the combined system projected on the xy/xz/yz planes, atoms coloured by fragment. Needs a display. Off by default. switch, off by default

`--png`
Save that figure as <output-dir>/`chargediff_fragments.png`. Off by default. switch, off by default

`--save-report`
Also persist the full run report to `chargediff_prep_report.txt`. Off by default. switch, off by default

Like every STB command it also accepts `-h`/`--help`, `-v`/`--version` and `--no-intro` (Section 3.3).

From the built-in help.

```
Example usage:
  stb-chargediffPrep -s water_dimer.fdf --fragment-sizes 3,3 --calc calc.fdf -p dojo --mesh-cutoff 320 --png
  stb-chargediffPrep -s bilayer.fdf --fragment-species "C;B,N" -p dojo -o bilayer_cdd
  stb-chargediffPrep -s li_graphene.fdf --fragment-sizes 18,1 --force-spin --force-dipole -p dojo
  stb-chargediffPrep -s adsorption_run/sites/site_1/structure.fdf --fragment-manifest
  adsorption_run/sites/site_1/fragment_manifest.json
```

11.16.2.2 `stb-chargediffAnalysis`

Stage 2 of the Charge Density Difference workflow: given just `--dir` (the same folder `stb-chargediffPrep` wrote into), auto-discovers the combined system and every fragment from `chargediff_manifest.json`, reads each one's own .RHO (once SIESTA has been run in every folder), and computes Delta rho = rho_combined - sum(rho_fragment_i) – the standard way to visualize charge accumulation/depletion regions. Besides the map (2D slice / planar profile / 3D point cloud / Gaussian cube in e/bohr^3) the report checks the consistency of the folders (grid, cell, mesh cutoff, level of theory, SCF, electron count), gives the counterpoise-corrected interaction energy from the same runs, statistics of Delta rho and the charge change of every fragment and atom (nearest-atom partition). `--show` / `--png` draw a 4-panel dashboard; `--save-gnuplot` writes the .gplot; `--save-data` a CSV and a JSON. `--spin` computes the spin-density difference instead (only if the combined system and every fragment are spin-polarized).

Synopsis.

```
usage: stb-chargediffAnalysis [-h] [--dir DIR] [-o OUTPUT_DIR] [--3d | --profile]
                              [-a {0,1,2}] [-p POS] [--spin] [--iso-min RHO]
                              [--no-cube] [--vmin VMIN] [--vmax VMAX] [--contour]
                              [--show] [--png] [--save-gnuplot] [--save-data]
                              [--save-report] [--no-intro] [-v]
```

Options

`--dir DIR`
: Directory `stb-chargediffPrep` wrote into – must contain 'chargediff_manifest.json' plus the 'combined/' and fragment subfolders it lists, each already run in SIESTA. Default: chargediff_run (`stb-chargediffPrep`'s own default `--output-dir`).

`-o, --output-dir OUTPUT_DIR`
: Directory to write the data/gnuplot/cube/report files and references.bib into (default: current directory). Created if it doesn't exist.

`--3d`
: Export full 3D volume — switch, off by default; exclusive group 1

`--profile`
: Write the planar average of Delta rho along `--axis` as a 1D profile (average taken over the other two axes) instead of a 2D/3D map – useful for bilayers, slabs, and interfaces — switch, off by default; exclusive group 1

`-a, --axis {0,1,2}`
: For a slice/profile: the axis normal to the cut plane (slice) or the axis the profile varies along (profile) (0=X, 1=Y, 2=Z). Default: Z for the slice; for the profile (and the dashboard's profile panel) the lattice axis along which the fragments are farthest apart. — choices: 0, 1, 2; type: int; default: automatic

`-p, --pos POS`
: Position (Angstrom) of the 2D cut along `--axis` (default: the plane where the planar integral of |Delta rho| is largest; ignored in `--profile` mode) — type: float

`--spin`
: Compute the SPIN density difference instead of the charge density difference – only available if the combined system AND every fragment are spin-polarized. — switch, off by default

`--iso-min RHO`
: For –3d: only export/preview points with |Delta rho| >= this threshold (e/Ang^3) – keeps the point cloud a manageable size and visually meaningful instead of dominated by near-zero points — type: float

`--no-cube`
: Skip writing Delta rho's full 3D grid as a Gaussian .cube file (in e/bohr^3) – written by default (needs a '*.XV' or '`structure.fdf`' in the combined folder for the geometry), for real 3D isosurface rendering in VESTA/VMD/Avogadro. — switch, off by default

`--vmin VMIN`
: Fix the colorbar/palette lower bound manually (e/Ang^3), e.g. to compare several slices on the same scale. — type: float

`--vmax VMAX`
: Fix the colorbar/palette upper bound manually (e/Ang^3). — type: float

`--contour`
: Overlay contour lines on a 2D slice map/dashboard (ignored for –3d/`--profile`) — switch, off by default

`--show`
: Open the 4-panel dashboard (slice with atoms, planar profile, charge change per atom, summary) on screen with matplotlib. Needs a display. Off by default (`--view` is the older name of this option). — switch, off by default

`--png`
: Save the dashboard as <output-dir>/<name>`_summary.png`. Off by default. — switch, off by default

`--save-gnuplot`
: Also write a .gplot script next to each .dat file this run generates. Off by default. — switch, off by default

`--save-data`
: Also write <name>`_atoms.csv` (charge change per atom) and <name>`_results.json` (every number of the report). Off by default. — switch, off by default

`--save-report`
Also persist the full run report to `chargediff_analysis_report.txt`. Off by default. switch, off by default

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Example usage:
  stb-chargediffAnalysis --dir chargediff_run --png
  stb-chargediffAnalysis --dir chargediff_run --profile --axis 2 --save-gnuplot
  stb-chargediffAnalysis --dir chargediff_run --axis 2 --pos 7.1 --contour --show --save-data
  stb-chargediffAnalysis --dir chargediff_run --3d --iso-min 0.01 --no-cube
```

11.17 Hirshfeld-I Charge Partitioning

At a glance

Menu code: 5.17 (a multi-stage workflow)

Code	Stage	Command
5.17.1	Stage 1 - Neutral Prep	`stb-hirshfeldPrep`
5.17.2	Stage 2 - Ion Prep	`stb-hirshfeldIons`
5.17.3	Stage 3 - Analysis	`stb-hirshfeldAnalysis`

Guided example: `examples/5.17-hirshfeld/`

Summary: Iteratively refined Hirshfeld atomic charges for an already-computed combined/production system, via isolated neutral/ion atomic references – corrects simple Hirshfeld's known conservative bias without needing a Bader-style density maximum at every nucleus (see Utils > Native Charges for plain SIESTA-native Hirshfeld/Voronoi output instead).

11.17.1 Overview

How many electrons does each atom of a molecule or a crystal own? The density $\rho(\mathbf{r})$ is an observable; the share of it that belongs to one atom is not, and every answer to the question is a *rule* for cutting the density into atoms. Hirshfeld's *stockholder* rule [55] shares the density at every point in proportion to what free reference atoms would put there, and Hirshfeld-I [57] makes those references consistent with the atoms they produce. The workflow builds the references with SIESTA itself — same pseudopotentials, basis and functional as the system — and needs three stages with two rounds of SIESTA runs in between (Fig. 11.1).

Hirshfeld-I in five equations

Stockholder partition. Place a spherical pro-atom ρ_A^0 on every nucleus (and on every periodic image $\mathbf{T}$ of it, in a crystal [90]) and give atom A the fraction w_A of the density at each point:

$$w_A(\mathbf{r}) = \frac{\sum_{\mathbf{T}} \rho_A^0(|\mathbf{r} - \mathbf{R}_A - \mathbf{T}|)}{\sum_B \sum_{\mathbf{T}} \rho_B^0(|\mathbf{r} - \mathbf{R}_B - \mathbf{T}|)}, \qquad N_A = \int_{\text{cell}} w_A(\mathbf{r})\, \rho(\mathbf{r})\, d^3r, \qquad q_A = Z_A^{\text{val}} - N_A.$$

The weights add up to one, so the charges always add up to the charge of the cell. The pieces $\rho_A = w_A \rho$ are the ones closest to the pro-atoms in the information-theoretic sense: they minimize $\sum_A \int \rho_A \ln(\rho_A/\rho_A^0)$ under $\sum_A \rho_A = \rho$ [91, 92].

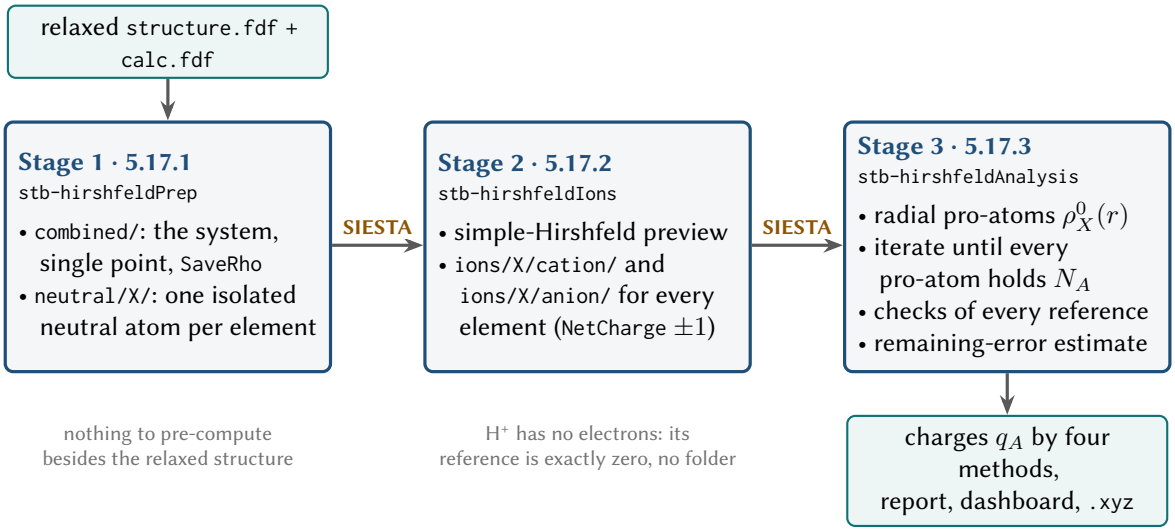


Figure 11.1: The three stages of the Hirshfeld-I workflow. Every SIESTA run between the stages is a single point with the mesh, functional, basis and pseudopotentials of the system; only Stage 3 does the partition, and it can be re-run in seconds with another `-tol`.

Self-consistent pro-atoms. Neutral pro-atoms pull every charge toward zero. Hirshfeld-I asks instead that each pro-atom hold as many electrons as the atom it defines, interpolating between the neutral atom and the ion of the same sign:

$$\int \rho^0_A \, d^3r = N_A, \qquad \rho^0_A(q) = (1-f)\,\rho^0_X + f\,\rho^{\pm}_X, \quad f = \min(|q_A|, 1),$$

solved by iteration from the simple-Hirshfeld charges until $\Delta q_n = \max_A |q_A^{(n)} - q_A^{(n-1)}| < \text{tol}$. The solution does not depend on the starting point [93].

How far from the fixed point. Near the solution each step is a fixed fraction r of the previous one, so the charges are still

$$e_n \simeq \Delta q_n \, \frac{r}{1-r}$$

away when the loop stops — about $2\times$tol for water ($r \approx 0.67$), $3\times$tol for NaCl ($r \approx 0.76$). Stage 3 measures r and prints e_n.

Before you quote a Hirshfeld-I charge

- **Read the error estimate, not `-tol`.** Tighten `-tol` until the estimated remaining error is small; Stage 3 suggests the value, and re-running it needs no SIESTA.
- **Fix the basis.** In SIESTA the anion reference is only as diffuse as the orbitals allow: `PAO.EnergyShift` from 0.02 to 0.005 Ry moved $q(\mathrm{O})$ of water from -0.60 to -0.77. Compare charges computed with one basis.
- **Past ± 1 the reference stops at the ion** (MgO: ± 1.09, flagged *capped*): read it as a lower bound.
- **Quote the method with the number.** On one density Voronoi, Hirshfeld, Hirshfeld-I and Bader differed by factors of 2–4 (Section 11.17.2.5).

11.17.2 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/5.17-hirshfeld/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

This workflow computes **Hirshfeld-I** (iterative Hirshfeld) atomic charges [5] from a SIESTA density, in 3 stages. **Stage 1** (`stb-hirshfeldPrep`, code `5.17.1`) takes your relaxed structure and template and writes the system as a single point that saves its density, plus one isolated **neutral atom** per element. **Stage 2** (`stb-hirshfeldIons`, code `5.17.2`), after SIESTA ran there, writes the **cation and the anion** of every element. **Stage 3** (`stb-hirshfeldAnalysis`, code `5.17.3`) builds a spherical reference from each isolated atom and iterates the stockholder partition until every reference holds as many electrons as the atom it defines:

> `q_A = Z_A − ∫ w_A(r) ρ(r) d³r,  w_A = ρ_A⁰ / Σ_B ρ_B⁰,  ρ_A⁰ = (1 − |q_A|) ρ_neutral + |q_A| ρ_ion`

It reports the charges next to Voronoi, SIESTA's own Hirshfeld and simple Hirshfeld (neutral references) of the same density, with what you need to trust them: a check of every reference atom (electrons, radius, SCF), the history of the iteration and an estimate of **how far the charges still are from the fixed point** when it stops, the charge balance, a dipole check, and an optional dashboard.

It answers a question the rest of section 4 does not: not an energy or a spectrum, but **how many electrons each atom owns** — the number behind "the oxygen of water carries −0.7 e", oxidation-state arguments, charge transfer at an interface, or the point charges of a force field. Section 1 explains why that number needs a definition and what this one is; the worked example (Section 5) runs all three stages for real with SIESTA on water, N_2O (two atoms of one element with opposite charges), the NaCl crystal and MgO (charges beyond ±1), plus mesh, box and basis studies of water that show what the numbers depend on.

The three stages live in this one folder and this one tutorial — same reasoning as `5.8`, `7.1`: each stage's output exists only to feed the next.

11.17.2.1 1. Theory

Symbols at a glance

Symbol	Meaning
ρ(r)	electron density of the system (the valence pseudo-density SIESTA writes, e/Å³)
ρ_A^0(r)	the **pro-atom**: a spherical reference density of atom A, centred on its nucleus
ρ^0(r) = Σ_A ρ_A^0	the **promolecule**: all pro-atoms superposed at the atomic positions (every periodic image included)
w_A(r) = ρ_A^0/ρ^0	Hirshfeld weight of atom A at r, 0 ≤ w_A ≤ 1, Σ_A w_A = 1
N_A, q_A	population of atom A (e) and its charge q_A = Z_A − N_A (> 0: it lost electrons)
Z_A	valence charge of A's pseudopotential (Z_val: O 6, H 1, Na 9, Mg 10, Cl 7)
ρ_X^{0}, ρ_X^{+}, ρ_X^{−}	spherically averaged density of the isolated neutral atom, cation and anion of element X
f = min(abs(q), 1)	the fraction of the ion mixed into a pro-atom
Δq_n	max_A abs(q_A^(n) − q_A^(n−1)), the step of iteration n (e)
r	the **contraction ratio** of the iteration, Δq_n / Δq_{n−1}
μ	dipole moment (Debye; 1 e Å = 4.803 D)

1.1 An atomic charge is a partition, not an observable. The density ρ(r) of a molecule or a crystal is an observable; the charge of one of its atoms is not. To say that the oxygen of water carries −0.7 e

you must first say **which electrons belong to the oxygen**, that is, cut the density into atomic pieces $\rho_A(r)$ that add up to the whole:

$$\rho(r) = \Sigma_A\ \rho_A(r), \qquad N_A = \int \rho_A(r)\ d^3r, \qquad q_A = Z_A - N_A \tag{1}$$

There are many ways of cutting, and each gives its own numbers [1, 2]:

- **Space is divided into regions**, one per atom, with sharp boundaries: Voronoi cells (each point goes to the nearest nucleus; SIESTA's `Charge.Voronoi`), Bader basins (the zero-flux surfaces of ρ [3]; `stb-bader`, 2.6).
- **The basis set is divided**, not space: Mulliken and Löwdin populations. They need no grid but depend strongly on the basis.
- **Every point is shared** among all atoms with smooth weights $w_A(r)$, $\rho_A = w_A\ \rho$: the **stockholder** family — Hirshfeld [4], Hirshfeld-I [5], iterated stockholder (ISA) [6], MBIS [7], DDEC [8].

This workflow computes **Hirshfeld-I** charges, a member of the last family, and reports next to them the simple Hirshfeld, Voronoi and (in the example) Bader charges of the same density, so the numbers can be compared partition by partition.

1.2 The Hirshfeld (stockholder) partition. Hirshfeld's idea [4] is to share the density at each point **in proportion to what the free atoms would have put there**. Place a spherical reference atom ρ_A^0 on every nucleus, add them up into the promolecule ρ^0, and give atom A the fraction

$$w_A(r) = \rho_A^0(|r - R_A|) \,/\, \rho^0(r), \qquad \rho^0(r) = \Sigma_B\ \rho_B^0(|r - R_B|) \tag{2}$$

$$N_A = \int w_A(r)\ \rho(r)\ d^3r \tag{3}$$

of the real density there — like the shareholders of a company, each gets a share of the profit proportional to its investment, hence the name. The weights add up to one at every point, so $\Sigma_A\ N_A = \int\rho$ exactly, whatever the pro-atoms are: the charges always sum to the total charge.

The partition has a clean justification: the Hirshfeld atoms are the pieces ρ_A that **stay as close as possible to the free atoms**, in the sense that they minimize the information distance (Kullback–Leibler divergence) to the pro-atoms,

$$\Sigma_A \int \rho_A(r)\ \ln[\ \rho_A(r) \,/\, \rho_A^0(r)\ ]\ d^3r \qquad \text{subject to} \quad \Sigma_A\ \rho_A = \rho \tag{4}$$

and Eq. 2 is the exact minimizer [9]. The atoms overlap smoothly (no sharp boundary), are transferable between similar environments, and need nothing but the density and the pro-atoms.

What they need **is a choice of pro-atoms**, and that is the weak point. The usual choice is the **neutral** free atoms. It is known to make charges too small [5, 10]: the weight function is built from neutral atoms, so a region where the oxygen of water has gained density is still shared as if the oxygen were neutral; the partition is *biased toward neutrality*. It is also arbitrary (why neutral? why not the ions, or a different neutral state?) and breaks for a charged system, where $\int\rho^0 \neq \int\rho$.

1.3 Hirshfeld-I: make the pro-atoms consistent with the atoms they produce. Bultinck *et al.* [5] removed the arbitrariness with one condition: **each pro-atom must hold as many electrons as the atom it defines**,

$$\int \rho_A^0\ d^3r = N_A \tag{5}$$

A pro-atom with a non-integer population is built by interpolating linearly between the isolated atoms of the two integer charge states that bracket it. This workflow keeps the states −1, 0 and +1 of every element and builds, for an atom with charge q,

$$\rho_A^0(q) = (1 - f)\,\rho_X^{0} + f\,\rho_X^{\pm}, \qquad f = \min(|q|, 1), \quad \pm = \text{sign of } q \tag{6}$$

which integrates to $Z_A - q$ exactly for $|q| \le 1$ — the linear interpolation is what keeps Eq. 5 exact. Eq. 5 is then solved by iteration:

$$\begin{aligned} &q^{(0)} = \text{simple Hirshfeld (neutral pro-atoms, } f = 0) \\ &q^{(n+1)} = \text{Hirshfeld partition (Eqs. 2-3) with the pro-atoms } \rho_A^0(q_A^{(n)}) \text{ of Eq. 6} \end{aligned} \tag{7}$$

$$\text{stop when } \Delta q_n = \max_A |q_A^{(n)} - q_A^{(n-1)}| < \text{tol}$$

Each atom chooses its **own** ion — its cation when it is positive, its anion when negative — so two atoms of the same element can lean in opposite directions (the two nitrogens of N_2O do, Section 5.4). That is why Stage 2 writes both ions of every element.

The self-consistent charges do not depend on the starting point [11] (no promolecule to choose), are only slightly basis dependent, work for charged systems, are larger in magnitude than the simple Hirshfeld ones, and reproduce the molecular electrostatic potential about as well as charges fitted to it [12]. Whether the solution is always unique is not proven in general [13]; a counterexample with symmetry-equivalent atoms getting different charges has been reported for some materials [8]. The workflow starts every atom from simple Hirshfeld, so symmetry-equivalent atoms start equal and stay equal.

1.4 How far from converged: the contraction ratio. Near its solution the iteration of Eq. 7 behaves like a linear map: the error of every round is a fixed fraction r of the previous one (the largest eigenvalue of the map's Jacobian),

$$e_n \approx r\, e_{n-1}, \qquad \Delta q_n = e_{n-1} - e_n \approx e_n\,(1 - r)\,/\,r \tag{8}$$

$$e_n \approx \Delta q_n \cdot r\,/\,(1 - r) \qquad \text{(the remaining error after round n)} \tag{9}$$

So the usual stopping rule $\Delta q_n < \text{tol}$ does **not** bound the error of the charges: it bounds the last step, and at $r = 0.68$ (water) the charges are still $0.68\,/\,0.32 \approx 2.1 \times \text{tol}$ away from the fixed point when it fires. Stage 3 measures r from the last three steps, prints the estimate of Eq. 9, and warns — with the `--tol` that would make the *error* small — when the estimate exceeds the tolerance (not below 0.001 e). The estimate needs no new SIESTA run, and Section 5.3 checks it against the converged answer. r itself says how hard the partition is: it approaches 1 when the charge transfer is large and the anion reference is very different from the neutral atom.

1.5 Periodic systems. Nothing in Eqs. 2–7 needs a molecule: in a crystal the promolecule is the sum of the pro-atoms of **every periodic image** of every atom,

$$\rho^0(r) = \sum_A \sum_T \rho_A^0(|r - R_A - T|), \qquad w_A(r) = \sum_T \rho_A^0(|r - R_A - T|)\,/\,\rho^0(r) \tag{10}$$

and N_A is integrated over one cell [14]. Stage 3 sums every image within the reach of each pro-atom (the radius beyond which it vanishes, a few Å for SIESTA's confined orbitals, Section 1.6). For a molecule in a large box that is only the nearest image; in the 2-atom cell of rock salt a point near a Cl sees six Na neighbours at 2.8 Å, each a different image of the same atom. (The first version of the workflow kept only the nearest image; Section 5.5 shows what that did to NaCl.)

1.6 Reference atoms with SIESTA. The pro-atoms are computed, not tabulated: one SIESTA run per element and charge state, a single atom in a cubic box (`--vacuum`, default 20 Å), at the level of theory of the system. Several consequences are worth knowing:

- **Valence pseudo-densities.** SIESTA's `.RHO` holds only the electrons of the pseudopotential's valence, so `Z_A` is `Z_val` and the core is in neither ρ nor ρ^0. Semicore states are valence when the pseudopotential includes them: the PseudoDojo Na has 9 electrons ($2s^2\ 2p^6\ 3s^1$), Mg 10; their cations are then Na^+ with 8 and Mg^+ with 9. Using pseudo-valence densities for Hirshfeld-I in solids was shown to reproduce the all-electron charges to good accuracy [15].
- **The same mesh, basis and functional** as the system (Stage 1 forces the same `MeshCutoff` into every folder, the template supplies the rest). The isolated atoms are run **spin-polarized** with Γ only.
- **Confined orbitals confine the reference.** SIESTA's numerical atomic orbitals are strictly zero beyond a cutoff radius set by `PAO.EnergyShift` (Section 5.7), so the density of every reference atom vanishes beyond it too — a few Å, far inside the box. A free anion is a delicate object (the extra electron of O^- is weakly bound; O^{2-} is not bound at all in vacuum), and in a plane-wave code its density spreads over the whole box [14]; here the basis keeps it on the atom. The anion reference is therefore as confined as the basis, and the charges inherit a basis dependence through it.
- **The bare proton.** H has `Z_val = 1`, so H^+ has no electrons: its density is exactly zero and no SIESTA run is needed (SIESTA could not converge it anyway). Stage 2 skips the folder; Stage 3 uses `ρ_H^{+} = 0`, so a positive hydrogen's pro-atom is `(1 − q) ρ_H^{0}`.
- **Spherical averages.** An open-shell atom (O, N, Cl) is not spherical in a spin-polarized SCF: Stage 3 averages each reference over angles in shells 0.25 grid spacings wide (empty shells dropped) and checks that the profile still holds `Z_val`, `Z_val − 1` and `Z_val + 1` electrons (`[2]` of the report).

1.7 Charges beyond ±1. Eq. 6 stops at the ions: when `abs(q) > 1`, the pro-atom is the ion itself (`f = 1`). The partition still returns whatever charge the density gives, but Eq. 5 no longer holds (`N_A` $\neq \int \rho_A^0$), and the true Hirshfeld-I value would need the next ion, Mg^{2+} or O^{2-}. Mg^{2+} is an ordinary closed shell; O^{2-} is not bound in vacuum and must be stabilized artificially, and different stabilizations give different charges — the main known ambiguity of Hirshfeld-I for oxides, which motivated the Hirshfeld-E scheme [16]. Stage 3 flags every such atom (`(capped)` in `[5]` and a warning). MgO in Section 5.6 is the case.

1.8 What point charges can and cannot reproduce. Atomic charges are often used to build the electrostatics of a model (force fields, the dipole of a molecule). The dipole of a molecule is exactly

$$\mu = \Sigma_A\ q_A\ R_A \ + \ \Sigma_A\ \mu_A, \qquad \mu_A = -\int\ (r - R_A)\ \rho_A(r)\ d^3r \tag{11}$$

— the point charges plus the dipole of each atomic piece (the asymmetry of its cloud around its own nucleus). Any partition satisfies Eq. 11 with its own `q_A` and μ_A; the point-charge term alone depends on how much of the dipole the partition put into the charges. Stage 3 prints the point-charge dipole of every method next to SIESTA's dipole of the full density, along the axes where a dipole is defined (non-periodic ones). Neither "closer is better" nor "larger is better" is a law: Hirshfeld charges are known to be too small to give the dipole by themselves, Hirshfeld-I charges are larger and leave a smaller, opposite atomic-dipole term [10, 12]. The comparison says which charges carry the electrostatics in a given molecule (Section 5.2).

1.9 A calculation you can check by hand. The water molecule of Section 5.2: `[4]` of its report (and `hirshfeld_iterations.dat`) lists the step of every round, and the last three are

```
Δq_8 = 0.008415     Δq_9 = 0.005665     Δq_10 = 0.003820 e
r ≈ 0.005665/0.008415 = 0.673,   0.003820/0.005665 = 0.674
```

```
e_10 ≈ 0.003820 × 0.673 / (1 − 0.673) = 0.0079 e              (Eq. 9; the report prints 0.0079)
```

so `q(O) = −0.7002 e` after round 10 should be ≈ `−0.708 e` at the fixed point — the run at `--tol 1e-6` gives `−0.7082` (Section 5.3). The charges add up: `q(O) + 2 q(H) = −0.7002 + 2 × 0.3501 = 0.0000 e`. And the reference of the oxygen at the end, `0.70 anion` in `[5]`, is Eq. 6 with `f = 0.70`: `0.30 ρ_O^0 + 0.70 ρ_O^−`, which holds `0.30 × 6 + 0.70 × 7 = 6.70` electrons — its population, as Eq. 5 demands.

11.17.2.2 2. Libraries and external dependencies used

- **SIESTA** [18, 19] itself is *not* called by the suite. It does the real work: one SCF per folder — the system, and every element as an isolated neutral atom, cation and anion — writing the density on its real-space grid (`<label>.RHO`, because `SaveRho` is forced) and the log (`calc.out`) with `Z_val`, the SCF convergence, its own Hirshfeld and Voronoi charges and the dipole. The reference data of Section 5 was produced with SIESTA 5.4.2, GGA-PBE [20], PseudoDojo norm-conserving pseudopotentials [21] and Monkhorst–Pack k-grids [22].
- **sisl** [23] — reads the `.RHO` grids (spin up + down).
- **numpy** — the radial profiles, the promolecule on the grid with its periodic images, the weights and the integrals of Eqs. 2–3, the iteration.
- **pymatgen** [24] / **ASE** [25] — structure reading and the dimensionality of the cell (which axes have vacuum, for the dipole check).
- **matplotlib** [26] — the 6-panel dashboard of Stage 3 (`--png` / `--show`).

11.17.2.3 3. Stages 1 and 2: the folders

The reference atoms cannot all be written at once: whether an element needs its ions at all, and how each ion behaves, is only known after the neutral atoms ran. So there are two writing stages, with SIESTA between them:

```
stb-hirshfeldPrep  ->  SIESTA in combined/, neutral/*  ->  stb-hirshfeldIons  ->  SIESTA in ions/*/*
->  stb-hirshfeldAnalysis
   (5.17.1)                                                  (5.17.2)
   (5.17.3)
```

3.1 Stage 1 (`stb-hirshfeldPrep`, code `5.17.1`). Takes the **relaxed** structure (`-s`, default `structure.fdf`) and your template (`--calc`, default `calc.fdf`; it must `%include structure.fdf` and use an implicit basis such as `PAO.BasisSize DZP`, which works for any subset of the elements) and writes

```
<output-dir>/                          (-O, default hirshfeld_study)
    combined/        structure.fdf, calc.fdf, config_extra.fdf, the pseudopotentials
    neutral/<El>/    one isolated atom per element, in a cubic box of side --vacuum
    hirshfeld_manifest.json            what Stage 2 reads (paths RELATIVE to the study: copy or move
    it freely)
    hirshfeld_prep_report.txt          with --save-report
```

Every `calc.fdf` is `%include config_extra.fdf` above your template, untouched; SIESTA's fdf reader is first-occurrence-wins, so the forced settings beat the template's:

Directive	`combined/`	`neutral/<El>/` (and the ions)	Why
`MD.VariableCell false`, `MD.TypeOfRun CG`, `MD.Steps 0`	forced	forced	a single point at the geometry given
`MeshCutoff`	`--mesh-cutoff` (400 Ry)	the same	every density on a comparably fine grid
`SaveRho true`	forced	forced	Stage 2/3 read the `.RHO`
`Charge.Hirshfeld end`, `Charge.Voronoi end`	forced	—	SIESTA's own charges of the same density, for comparison
`kgrid.MonkhorstPack`	the template's	`[1 1 1]`	one atom in a box needs Γ only
`Spin`	the template's	`polarized` (also written into `calc.fdf`)	an isolated atom is usually an open shell

The box (`--vacuum`, 20 Å) only has to hold the atom: SIESTA's atom densities vanish beyond the orbital radii (≈ 2–4 Å here), so 10 Å would do (Section 5.7); Stage 3 checks it.

3.2 Stage 2 (`stb-hirshfeldIons`, code `5.17.2`). After SIESTA ran in `combined/` and every `neutral/<El>/`, Stage 2 (`-O` the same study) reads `Z_val` of every element from the log of `combined/` (`--ref` for a log not named `*.out`), the neutral densities, and writes

```
ions/<El>/cation/    NetCharge +1  (Z_val - 1 electrons)
ions/<El>/anion/     NetCharge -1  (Z_val + 1 electrons)
hirshfeld_ions_manifest.json
```

for **every** element — both ions, because in Stage 3 each atom chooses its own (Section 1.3). The exception is an element with `Z_val` $\leq$ `1` (H): its cation has no electrons at all, an exactly zero density, so no folder is written and Stage 3 uses zero. Its report also previews the simple-Hirshfeld charges of `combined/` (pass 0, what Stage 3 starts from) next to SIESTA's own, and counts how many atoms of each element lean positive or negative.

3.3 Running them.

```
stb-hirshfeldPrep -s structure.fdf --calc calc.fdf -p dojo --save-report         # Stage 1
for d in hirshfeld_study/combined hirshfeld_study/neutral/*; do (cd $d && mpirun -np 4 siesta
calc.fdf > calc.out); done
stb-hirshfeldIons -O hirshfeld_study -p dojo --save-report                        # Stage 2
for d in hirshfeld_study/ions/*/*; do (cd $d && mpirun -np 4 siesta calc.fdf > calc.out); done
stb-suite            # or: 5 (Workflow) -> 17 (Hirshfeld-I) -> 1 / 2   (5.17.1, 5.17.2)
```

The isolated-atom runs are small (one atom, Γ only): a few seconds to a few minutes each. An anion can need more SCF iterations than the neutral atom (Na^- took 25); `[2]` of Stage 3 reports each one.

11.17.2.4 4. Stage 3: the analysis (`stb-hirshfeldAnalysis`, code `5.17.3`)

4.1 What it does. Reads every reference folder, builds its radial profile (Section 1.6), reads `combined/`'s density, runs simple Hirshfeld (pass 0) and iterates Hirshfeld-I (Eq. 7) until `max|Δq| < --tol` (default 0.005 e) or `--max-iter` (20). The iteration is cheap — seconds for the molecules of Section 5 — and needs no new SIESTA run, so `--tol` can be tightened at will (Section 5.3).

4.2 Report structure.

Section	Content
`[0]`	run metadata
`[1]`	the combined system: periodicity, grid, `Z_val` per element, electrons in the density, SCF
`[2]`	every **reference atom**: target electrons, on the grid, in the radial profile, the radius holding 99.9 % against the box, grid spacing, SCF
`[3]`	the iteration, round by round, with the atom that moved most
`[4]`	**convergence history**: the step and its ratio to the previous one, the contraction ratio, the estimated remaining error (Eq. 9)
`[5]`	per atom: Voronoi and Hirshfeld (SIESTA), simple Hirshfeld, Hirshfeld-I, their difference, the population, the reference used (`0.70 anion` = 30 % neutral + 70 % anion; `(capped)` past ±1)
`[6]`	per element: mean, spread, how many atoms are cation- or anion-like, means of the other methods, the Hirshfeld-I / Hirshfeld ratio
`[7]`	charge balance: total charge by every method, electrons assigned, simple Hirshfeld here vs SIESTA's
`[8]`	the **dipole check** along the non-periodic axes (Section 1.8)
`[9]`	summary and files written
`[10]`	every warning

4.3 The checks.

Check	Warns when	What to do
electrons in the combined density vs Σ `Z_val`	they differ by > 0.02 e	a charged run (then the charges add up to it) or a `Z_val` that does not match
SCF of `combined/` and of every reference	not confirmed in the log	converge it: the charges are only as good as the density
electrons of each reference on its grid vs `Z_val`, `Z_val` ∓ 1	> 0.02 e	a wrong charge state or pseudopotential
the radial profile vs the grid	> max(0.02 e, 0.5 %)	a mesh too coarse to average the atom
the radius holding 99.9 % vs the box	> 80 % of the inradius	a larger `--vacuum`
electrons assigned to no atom	> 0.02 e	density where no reference reaches
`abs(q) > 1`	always	the reference stops at ±1 (Section 1.7)
the estimated remaining error	> `--tol` (and > 0.001 e)	it prints the `--tol` that would make the error small
not converged within `--max-iter`	always	more rounds

A missing reference `.RHO` stops it with an `[ERROR]` naming the folder: a reference cannot be guessed.

4.4 Output files (all opt-in).

Option	Writes
`--save-data`	`hirshfeld_charges.dat` (per atom: every method, population, reference), `hirshfeld_iterations.dat` (every atom's charge per round), `hirshfeld_charges.xyz` (extended XYZ with the charges as per-atom columns; `ase gui` colours atoms by them)
`--png` / `--show`	the 6-panel dashboard `hirshfeld_analysis.png`: convergence with the tolerance and the estimated error, each atom's charge per round, the mean charge per element by method, Hirshfeld-I vs simple Hirshfeld per atom, the reference profiles $4\pi r^2\rho(r)$, the structure coloured by charge (`--show` without a display saves the PNG)
`--save-report`	`hirshfeld_analysis_report.txt`

4.5 Running it both ways.

```
stb-hirshfeldAnalysis -O hirshfeld_study --save-data --png --save-report
stb-hirshfeldAnalysis -O hirshfeld_study --tol 1e-4 --max-iter 100          # tighter, seconds more
```

```
stb-suite           # then: 5 -> 17 -> 3  (5.17.3): it asks for the folder, tolerance and outputs
```

11.17.2.5 5. Worked example: four systems, end to end

5.1 Set-up.

System	Why	Cell	Template
H_2O	a polar molecule; the base case of every study	14 Å cube	`calc.fdf`
N_2O	two atoms of one element with opposite charges	14 Å cube	`calc.fdf`
NaCl	an ionic crystal: periodic images, semicore Na	2-atom fcc primitive cell	`calc_bulk.fdf`
MgO	an oxide whose charges pass ±1	2-atom fcc primitive cell	`calc_bulk.fdf`

All with SIESTA 5.4.2, PBE [20], PseudoDojo pseudopotentials [21] (`Z_val`: H 1, N 5, O 6, Na 9, Mg 10, Cl 7 — Na and Mg with their 2s 2p semicore), DZP basis with `PAO.EnergyShift 0.01 Ry`, `MeshCutoff 400 Ry`, 300 K smearing; the molecules at Γ, the crystals with an 8×8×8 Monkhorst–Pack grid [22]. Each structure was relaxed first with the same template (the molecules at fixed cell, 0.01 eV/Å; the crystals with the cell, 0.05 GPa): O–H 0.975 Å and 104.4° (experiment 0.958 Å, 104.5° [28]), N–N 1.163 and N–O 1.203 Å (1.128 and 1.184 Å [28]), a = 5.674 Å for NaCl and 4.250 Å for MgO (5.64 and 4.21 Å measured; PBE overestimates lattice constants by about this much). The isolated atoms sit in 20 Å boxes (the default).

```
stb-hirshfeldPrep -s relaxed.fdf --calc calc.fdf -p dojo -O h2o --mesh-cutoff 400 --save-report
for d in h2o/combined h2o/neutral/*; do (cd $d && mpirun -np 4 siesta calc.fdf > calc.out); done
stb-hirshfeldIons -O h2o -p dojo --save-report
for d in h2o/ions/*/*; do (cd $d && mpirun -np 4 siesta calc.fdf > calc.out); done
stb-hirshfeldAnalysis -O h2o --save-data --png --save-report
```

5.2 Water: the three stages. **Stage 1** writes `combined/`, `neutral/O/` and `neutral/H/`; the isolated atoms get, in their `config_extra.fdf`,

```
MD.VariableCell false       MD.TypeOfRun CG    MD.Steps 0      kgrid.MonkhorstPack [1 1 1]
MeshCutoff 400 Ry           SaveRho true       Spin polarized
```

Stage 2, after SIESTA, reads `Z_val` from the log, previews pass 0 and writes the ions:

```
Idx | Elem | Z_val | Population(e-) | Charge(e-) | Native Hirshfeld(e-)
1   | O    | 6.0   | 6.2555         | -0.2555    | -0.2421
2   | H    | 1.0   | 0.8723         | +0.1277    | +0.1211
  [OK] O: ions/O/cation/ (NetCharge +1.0), ions/O/anion/ (NetCharge -1.0)
  [OK] H: ions/H/anion/ (NetCharge -1.0); cation SKIPPED (Z_val=1 -- NetCharge +1.0 would leave 0
  electrons ...)
```

Stage 3 first checks the five references SIESTA computed (and the exact zero of H^+):

```
Elem | State   | Target N | N on grid | N radial  | Radial-target | r99.9%(Ang) | Box inradius(Ang) |
Spacing(Ang) | SCF conv.
O    | neutral | 6        | 6.0000    | 6.0053    | +0.0053       | 1.91        | 10.00             |
0.0800       | yes (9)
O    | cation  | 5        | 5.0000    | 5.0052    | +0.0052       | 1.81        | 10.00             |
0.0800       | yes (9)
```

```
O    | anion   | 7        | 7.0000    | 7.0054    | +0.0054       | 1.97        | 10.00            |
0.0800       | yes (13)
H    | neutral | 1        | 1.0000    | 1.0015    | +0.0015       | 2.39        | 10.00            |
0.0800       | yes (6)
H    | cation  | 0        | 0 (exact) | 0 (exact) | -             | -           | -                | -
| -
H    | anion   | 2        | 2.0000    | 2.0012    | +0.0012       | 2.49        | 10.00            |
0.0800       | yes (12)
```

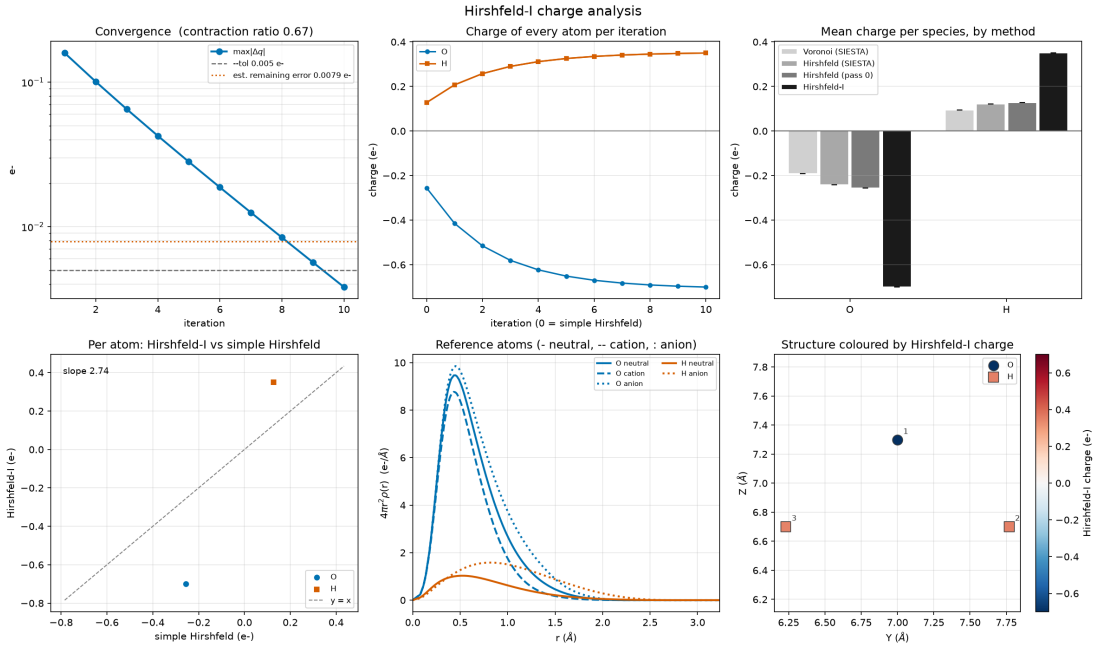
Every reference holds exactly its electrons (`NetCharge -1` gives 7.0000 on O: the sign convention is right), the spherical averages hold them to 0.1 %, and 99.9 % of each atom lies within 2.5 Å — a fifth of its box. Then the iteration:

```
Iteration | max|Delta q| (e-) | Ratio to previous | Atom that moved most
1         | 0.159092          | -                 | #1 (O)
2         | 0.100813          | 0.634             | #1 (O)
...
9         | 0.005665          | 0.673             | #1 (O)
10        | 0.003820          | 0.674             | #1 (O)
Converged after 10 iteration(s) (max|Delta q| < 0.005 e-).
Contraction ratio      : 0.673 (each step ~67 % of the previous)
Est. remaining error   : 0.0079 e- on the slowest atom, distance to the converged charges
```

and the charges, with the dipole check:

```
Idx | Elem | Voronoi(e-) | Hirsh. SIESTA(e-) | Simple Hirshfeld(e-) | Hirshfeld-I(e-) | HI - H(e-) |
Pop. HI(e-) | Reference
1   | O    | -0.1905     | -0.2421           | -0.2555              | -0.7002         | -0.4447    |
6.7002      | 0.70 anion
2   | H    | +0.0953     | +0.1211           | +0.1277              | +0.3501         | +0.2224    |
0.6499      | 0.35 cation
3   | H    | +0.0953     | +0.1211           | +0.1277              | +0.3501         | +0.2224    |
0.6499      | 0.35 cation

Component (Debye) | Voronoi (SIESTA) | Hirshfeld (SIESTA) | Simple Hirshfeld (pass 0) | Hirshfeld-I |
SIESTA (density)
|mu|              | 0.547           | 0.695             | 0.733                     | 2.010       | 1.430
```

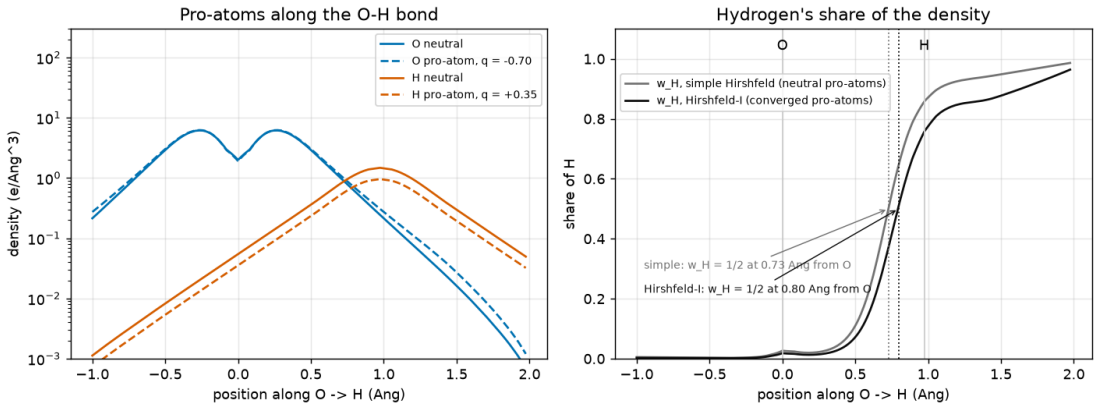


Dashboard of water

Fig. 1 — The dashboard of water (`--png`): the step per round with the tolerance and the estimated remaining error; the charge of each atom per round (0 = simple Hirshfeld); the mean charge per element by method; Hirshfeld-I against simple Hirshfeld per atom (slope 2.74); the reference atoms $4\pi r^2\rho(r)$ (neutral, cation, anion); the molecule coloured by charge.

Reading it:

- **Hirshfeld-I is 2.7× simple Hirshfeld**: O −0.70 against −0.26. The iteration moves the oxygen steadily away from neutral (Fig. 1, second panel) and stops at a reference that is 70 % O^-. Fig. 2 shows why the numbers change: with the oxygen's reference made more negative (more diffuse) and the hydrogens' smaller (`(1 − 0.35)` ρ_H^0, H^+ has no density), the point where the hydrogen's share of the density falls to one half moves from 0.73 to 0.80 Å along the 0.975 Å bond — the oxygen owns more of the bond.
- **Simple Hirshfeld here and SIESTA's own agree** (−0.256 and −0.242): both use neutral references, SIESTA its own free-atom densities from the basis, this tool the SCF atoms of `neutral/`; the 0.013 e difference is that choice. Voronoi gives less (−0.19), a different partition altogether (the cell of each nucleus, [27]).
- **The dipole:** SIESTA's dipole of the density is 1.43 D (the experiment gives 1.855 D [28]: DZP is short of diffuse functions, Section 5.7). Point charges alone give 0.55–0.73 D with the neutral-reference methods and 2.01 D with Hirshfeld-I: by Eq. 11, the Hirshfeld atoms carry a large atomic-dipole term in the same direction (+0.70 D), the Hirshfeld-I atoms a smaller one in the opposite direction (−0.58 D). Neither set of point charges *is* the dipole; Hirshfeld-I's are the ones that put most of the electrostatics into the charges [12].

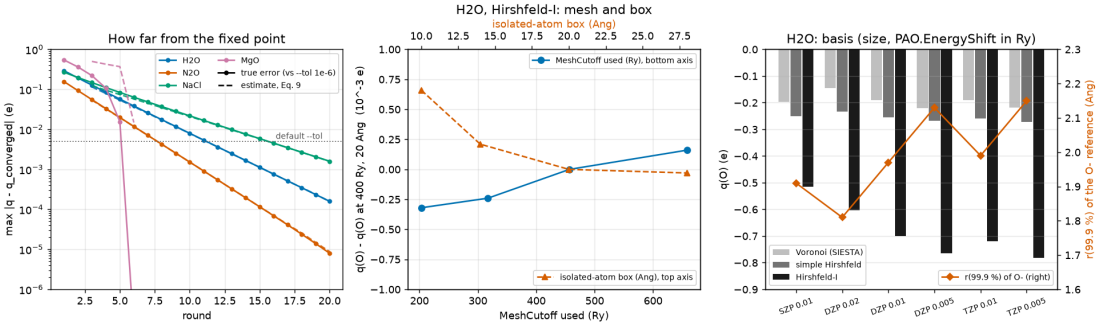


Hirshfeld weights along the O-H bond

Fig. 2 — Left: the pro-atoms along the O→H bond, neutral (solid) and at the converged charges (dashed); SIESTA's pseudo-density of O has a dip at the nucleus, where the core was removed. Right: the hydrogen's weight `w_H` *(Eq. 2) with neutral pro-atoms and with the converged Hirshfeld-I ones.* `python3 study_table.py --png`.

5.3 How converged is "converged". The report's warning on water: *"The last step was 0.0038 e-, but at a contraction ratio of 0.67 the charges are still ~0.0079 e- from the converged Hirshfeld-I values (1.6 x --tol). For the error itself to be below 0.005 e-, rerun with --tol 0.0024"*. Each system was also run at `--tol 1e-6` (`analysis_tight/`), which gives the true error of every round (`study_table.py [C]`):

Round	H_2O step	estimate (Eq. 9)	true error	NaCl step	estimate	true error
8	0.00842	0.0171	0.0174	0.01143	0.0348	0.0371
10	**0.00382**	**0.0079**	**0.0080**	0.00661	0.0210	0.0218
12	0.00174	0.0036	0.0036	**0.00387**	**0.0126**	**0.0129**
14	0.00080	0.0017	0.0017	0.00228	0.0075	0.0076



Water studies

Fig. 3 — Left: the true distance of the charges from the fixed point per round (solid) and the estimate of Eq. 9 (dashed) for the four systems. Middle: q(O) of water against the mesh and the isolated-atom box, in 10^{-3} e. Right: q(O) for six basis sets, with the radius that holds 99.9 % of the O^- reference. `python3 study_table.py --png`.

- **The stopping rule is not the error.** At the default `--tol 0.005` the charges of water are 0.008 e from the fixed point, those of NaCl 0.013 e (2.6 × tol): the iteration contracts slowly (ratio 0.67 and 0.76) and the last step is only a fraction `1 − r` of the error.

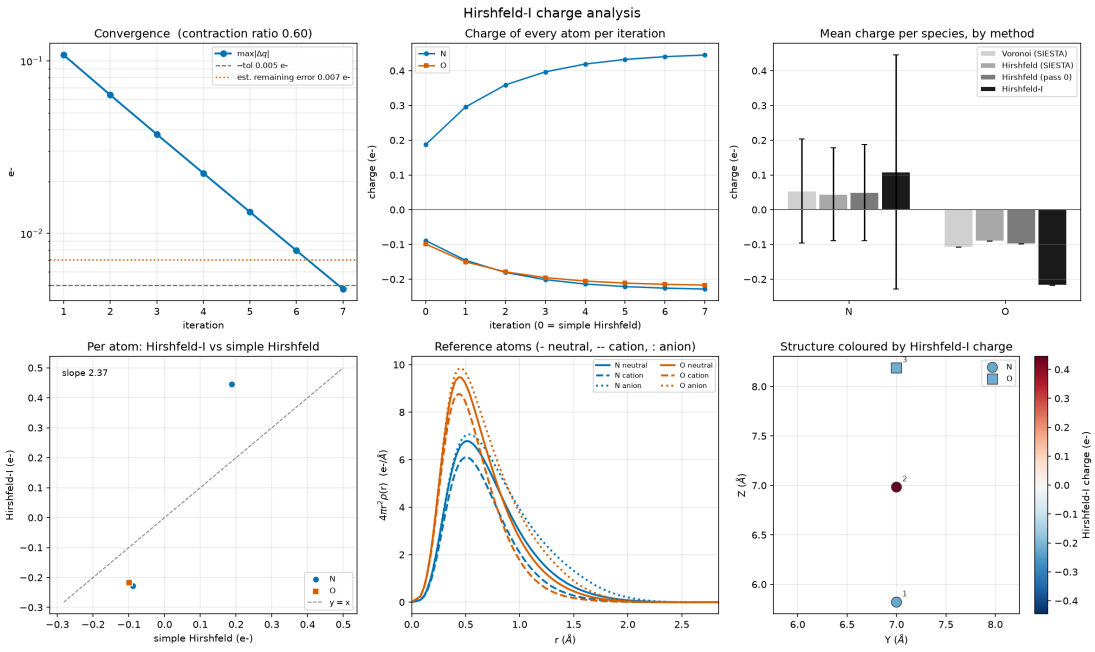
- **Eq. 9 predicts it.** For water and N_2O the estimate agrees with the true error to 1 % from round 3 on. For NaCl it is 2–7 % low, because the ratio itself still creeps up (0.70 → 0.77): the iteration is not yet in its linear regime, and a ratio that grows makes the geometric sum an underestimate. The suggested `--tol` (0.0015 for NaCl) gets the error below 0.005 e anyway.
- **It costs nothing to ask for more.** The iteration needs no SIESTA run: water to 10^{-6} e in 32 rounds, N_2O in 24, NaCl in 44 — seconds each. The converged charges: H_2O O −0.7082, H +0.3541; NaCl ±0.5143.
- **MgO breaks the assumption** (Fig. 3, the steep line): its charges pass ±1 in round 5, the references stop changing (Section 1.7) and the iteration lands on its fixed point in one step. The ratio and the estimate of the earlier rounds meant nothing; the report gives the ratio of the last rounds (0.17) and an estimate of 0.

5.4 N_2O: two nitrogens of opposite sign.

```
Idx | Elem | Z(Ang) | Voronoi(e-) | Hirsh. SIESTA(e-) | Simple Hirshfeld(e-) | Hirshfeld-I(e-) |
Reference
1   | N    | 5.8200 | -0.0953     | -0.0886           | -0.0886              | -0.2284         | 0.23
anion
2   | N    | 6.9834 | +0.2037     | +0.1787           | +0.1873              | +0.4452         | 0.45
cation
3   | O    | 8.1862 | -0.1084     | -0.0902           | -0.0987              | -0.2169         | 0.22
anion
Species highlighted above converged with atoms genuinely leaning toward BOTH ion states ...

Component (Debye) | Voronoi (SIESTA) | Hirshfeld (SIESTA) | Simple Hirshfeld (pass 0) | Hirshfeld-I |
SIESTA (density)
along c           | -0.094           | -0.026             | -0.075                    | +0.023      | +0.269
```

- **The central N is positive, the terminal N negative**, by every method: the familiar resonance picture $N^-{=}N^+{=}O \leftrightarrow N{\equiv}N^+{-}O^-$ puts the positive formal charge on the central atom. Each atom chose its own ion — the terminal N mixes in N^-, the central N mixes in N^+ — which is exactly what a one-ion-per-element shortcut cannot do and why Stage 2 writes both. `[6]` flags such an element. Converged: −0.2323, +0.4523, −0.2200.



Dashboard of N2O

Fig. 4 — N_2O (`--png`): the two nitrogens leave simple Hirshfeld in opposite directions from the first round on (second panel), which is why the per-species bar of N carries a large spread; the per-atom panel shows the central N far above the diagonal and the terminal N and O below it.

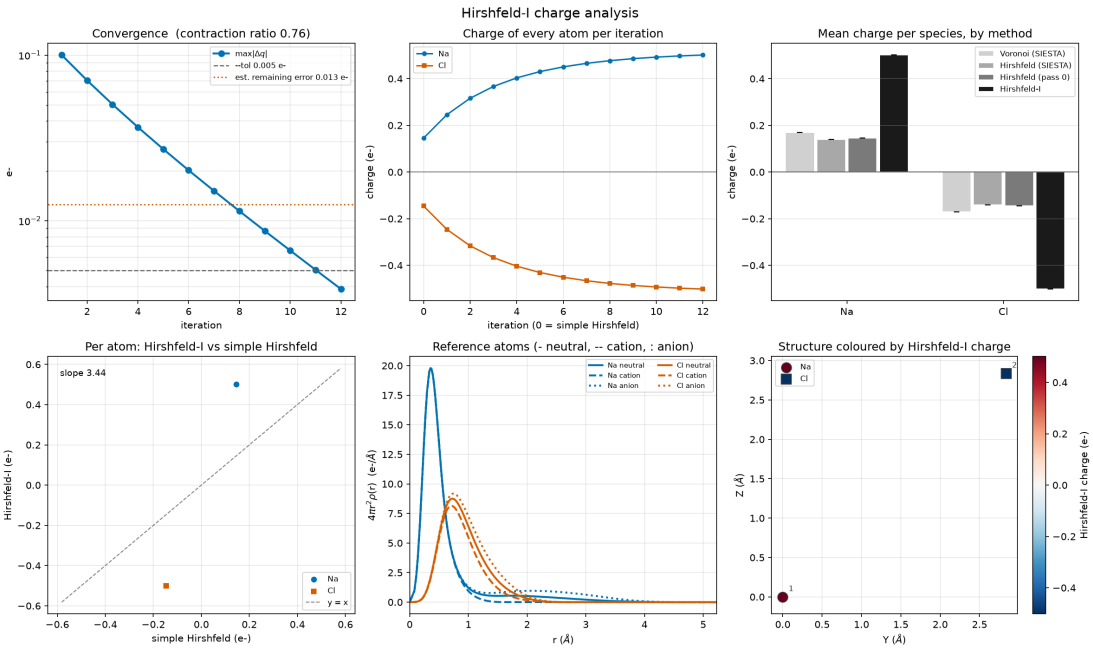
- **No point-charge set gives the dipole.** SIESTA's dipole is 0.27 D (0.161 D measured [28]); the point charges give between −0.09 and +0.02 D — not even the sign agrees across methods. In a linear molecule with nearly balanced charges the dipole is a small difference of atomic dipoles (Eq. 11), and charges alone cannot represent it. This is a limit of point charges, not of the partition; use `[8]` to see when it applies.

5.5 NaCl: a crystal, and why every periodic image counts.

```
Periodicity    : 3D (non-periodic axes: none)
Na   | neutral | 9  | 9.0000  | 9.0156  | +0.0156 | 3.81 | ...
Na   | cation  | 8  | 8.0000  | 8.0155  | +0.0155 | 1.45 | ...
Na   | anion   | 10 | 10.0000 | 10.0157 | +0.0157 | 4.02 | ...

Idx | Elem | Voronoi(e-) | Hirsh. SIESTA(e-) | Simple Hirshfeld(e-) | Hirshfeld-I(e-) | Reference
1   | Na   | +0.1702     | +0.1400           | +0.1457              | +0.5014         | 0.50 cation
2   | Cl   | -0.1702     | -0.1400           | -0.1457              | -0.5014         | 0.50 anion
```

- **The semicore electrons are valence here.** The Na reference holds 9 electrons ($2s^2\ 2p^6\ 3s^1$), its cation 8; the 3s electron makes the neutral atom large (99.9 % within 3.81 Å), the cation small (1.45 Å). Hirshfeld-I gives Na +0.51 (converged +0.514), 3.5× simple Hirshfeld.



Dashboard of NaCl

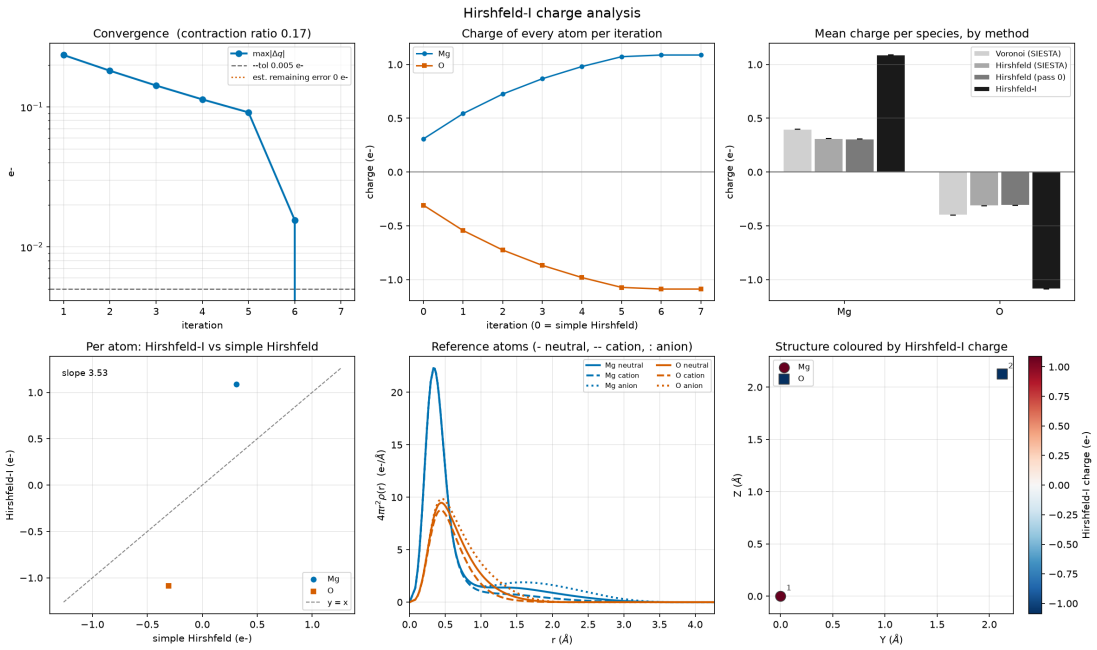
Fig. 5 — NaCl (`--png`): a slower iteration (ratio 0.76) whose last step is below `--tol` while the estimated error is 0.013 e; the reference atoms show the Na 3s tail of the neutral atom (and of Na^-) that the cation has lost.

- **Bader gives ±0.86** on the same density (`stb-bader`, Section 5.8). Different partitions, different numbers (Section 1.1): in a crystal the Bader basin of Na^+ is bounded by the density minimum between the ions, while the Hirshfeld weights share the overlap region smoothly. Neither is "the" ionic charge; compare like with like.
- **Every image matters.** The primitive cell is 4 Å across and each Cl has 6 Na neighbours at 2.84 Å, all images of the same atom. The first version of Stage 3 counted only the nearest image of each atom at every grid point. On this density it gave simple Hirshfeld ±0.333 — more than twice SIESTA's own ±0.140 — and Hirshfeld-I ±0.555. Summing every image within the reach of each reference (Eq. 10) gives ±0.146, next to SIESTA's, and ±0.514. For a molecule in a box the two are the same; `test/5-workflow/17-hirshfeld/analysis/check_core_numerics.py` now checks a small periodic cell with an exact answer.

5.6 MgO: beyond ±1.

```
Idx | Elem | Voronoi(e-) | Hirsh. SIESTA(e-) | Simple Hirshfeld(e-) | Hirshfeld-I(e-) | Pop. HI(e-) |
Reference
1   | Mg   | +0.3990     | +0.3139           | +0.3082              | +1.0880         | 8.9120      |
1.00 cation (capped)
2   | O    | -0.3990     | -0.3139           | -0.3082              | -1.0880         | 7.0880      |
1.00 anion (capped)
[WARNING 1] 2 atom(s) ended with |q| > 1 e- (#1 Mg +1.088, #2 O -1.088): the reference is capped at
the +-1 ion, so these
charges are not fully self-consistent -- a +-2 ion reference would be needed for them.
```

- **The charge passes 1 and the reference stops.** With Mg^+ and O^- as references the partition still gives ±1.088, but the oxygen's reference holds 7 electrons and the oxygen 7.088: Eq. 5 is not satisfied. A Mg^{2+} / O^{2-} reference would move the charge further; how far depends on how O^{2-} — unbound in vacuum — is stabilized, the known ambiguity of Hirshfeld-I for oxides [14, 16]. Read ±1.09 as a lower bound of the Hirshfeld-I value, and compare oxides with each other at the same setting.



Dashboard of MgO

Fig. 6 — MgO (`--png`): the step shrinks slowly until round 5, then drops to zero once both charges pass 1 and the references are capped (first panel); the Mg reference atoms show the semicore 2p peak at 0.3 Å and the 3s shell beyond 1 Å that the cation loses.

- Bader gives ±1.74 on the same density, Voronoi ±0.40: the spread between partitions is larger than in NaCl, and the ordering (Voronoi < Hirshfeld < Hirshfeld-I < Bader) is the same in both crystals.

5.7 What the charges depend on: mesh, box, basis. Eleven more complete studies of water (every folder recomputed, `study_table.py [D]–[F]`, Fig. 3):

`MeshCutoff` (Ry, used)	grid spacing (Å)	simple Hirshfeld	Hirshfeld-I	box (Å)	Hirshfeld-I
200 (203)	0.117	−0.2557	−0.7006	10	−0.6996
300 (317)	0.093	−0.2556	−0.7005	14	−0.7000
400 (457)	0.078	−0.2555	**−0.7002**	**20**	**−0.7002**
600 (658)	0.065	−0.2554	−0.7001	28	−0.7003

Basis, `PAO.EnergyShift`	Voronoi	SIESTA Hirshfeld	simple Hirshfeld	Hirshfeld-I	r99.9 % of O^- (Å)	μ HI points (D)	μ SIESTA (D)
SZP, 0.01 Ry	−0.197	−0.250	−0.250	−0.515	1.91	1.48	1.26
DZP, 0.02 Ry	−0.146	−0.210	−0.234	−0.603	1.81	1.73	1.30
DZP, 0.01 Ry	−0.191	−0.242	−0.255	**−0.700**	1.97	2.01	1.43
DZP, 0.005 Ry	−0.220	−0.262	−0.268	−0.765	2.13	2.20	1.51
TZP, 0.01 Ry	−0.190	−0.246	−0.260	−0.720	1.99	2.07	1.43
TZP, 0.005 Ry	−0.219	−0.266	−0.272	−0.783	2.15	2.25	1.51

- **The mesh and the box do not matter.** From 200 to 600 Ry the charge moves by 5×10^{-4} e; from a 10 to a 28 Å box by 7×10^{-4} e (the box only shifts the grid points relative to the atom). SIESTA's atoms vanish beyond their orbital radii, ≈ 2.5 Å, so any box that holds them is the same box. The defaults (400 Ry, 20 Å) are generous; 300 Ry and 14 Å would do.
- **The basis does — through the reference atoms.** Hirshfeld-I goes from −0.52 (SZP) to −0.78 (TZP, 0.005 Ry), simple Hirshfeld only from −0.23 to −0.27. What moves it most is `PAO.EnergyShift`, the confinement of the orbitals: from 0.02 to 0.005 Ry the O^- reference grows from 1.81 to 2.13 Å and q(O) from −0.60 to −0.77; at the same shift DZP → TZP changes it by only 0.02. In SIESTA the anion reference is exactly as diffuse as the basis allows (Section 1.6), so the iterative scheme inherits a basis dependence that is small for Hirshfeld-I in Gaussian-basis codes [11]. The density itself improves along the same path (SIESTA's dipole 1.26 → 1.51 D, toward the 1.855 D measured): the charges of the longer orbitals are the better ones, and they are not converged at the default `0.01 Ry`.
- **Compare charges only at one basis.** A trend across molecules or sites computed with the same basis is robust; an absolute Hirshfeld-I charge carries an uncertainty of ±0.1 e from the basis alone.

5.8 Why not Bader on SIESTA's density?. `stb-bader` on the `combined/` densities gives NaCl ±0.86 and MgO ±1.74, but for water **O −2.000, H +1.000**: the hydrogens get no electrons at all. That is not a bug of the partition, it is the density. Along the O–H bond SIESTA's valence pseudo-density rises to 5.5 e/Å^3 in the oxygen's shell, 0.3 Å from O, and then *decreases monotonically* through the proton (1.77 e/Å^3 at the H nucleus) — the pseudo-hydrogen has no density maximum of its own once it sits in the tail of the oxygen. Bader needs one maximum per atom; without it the hydrogen's basin is empty. (Plane-wave codes add the core charge back for the same reason before running Bader [29].) The stockholder partition needs no maximum: it shares every point by the reference atoms, and the hydrogens get 0.65 electrons each. For light atoms next to heavy ones on a pseudo-density, Hirshfeld-I is the real-space partition that works.

5.9 How far to trust each number.

Quantity	What moves it (this example)	Verdict
electrons of every reference, the charge balance	nothing (to 10^{-4} e)	a check, not a result
Hirshfeld-I at the default `--tol`	0.008 (water) – 0.013 e (NaCl) from the fixed point	read the estimate, tighten `--tol` (seconds)
Hirshfeld-I vs mesh, box	$< 10^{-3}$ e	converged at the defaults
Hirshfeld-I vs basis	±0.1 e (EnergyShift 0.02–0.005 Ry)	compare at one basis
Hirshfeld-I past ±1	the reference stops at the ion	a lower bound (MgO)
simple Hirshfeld vs SIESTA's own	0.006–0.013 e (another neutral reference)	consistent
point-charge dipole	atomic dipoles: −0.6 D (water), all of it (N_2O)	not the molecule's dipole
Bader on the pseudo-density	no basin for H next to O	ionic solids only

5.10 What to take from the example.

1. Hirshfeld-I charges are 2.2–3.5× the simple Hirshfeld ones in these four systems: the neutral reference really does pull the stockholder charges toward zero, and the iteration removes the arbitrariness.
2. `max|Δq| < tol` is not the error. Read `[4]`: the contraction ratio and the estimated remaining error tell how far the charges still are, and tightening `--tol` costs seconds.
3. Read `[2]` before the charges: every reference must hold its electrons and fit in its box.
4. In SIESTA the references are confined by the basis, so the basis — mostly `PAO.EnergyShift` — moves Hirshfeld-I charges by ±0.1 e. Compare charges at one basis.
5. Two atoms of one element can have opposite charges (N_2O): each atom chooses its own ion, which is why every element gets both. Past ±1 (MgO) the reference stops: a lower bound.
6. A charge is a definition: Voronoi, Hirshfeld, Hirshfeld-I and Bader differ by factors of 2–4 on the same density. Quote the method with the number.

11.17.2.6 6. Reproducing the reference data

`reference_results/` holds every finished SIESTA run of Section 5, so Stage 3 works out of the box without a SIESTA installation.

Folder	What	Contents
`relax/h2o`, `relax/n2o`, `relax/nacl`, `relax/mgo`	the relaxations	`calc.fdf`, `template.fdf`, start `structure.fdf`, `relaxed.fdf`, `calc.out`
`h2o/`, `n2o/`, `nacl/`, `mgo/`	the four studies	both manifests and the Stage 1/2 reports; per folder (`combined/`, `neutral/<El>/`, `ions/<El>/<cation|anion>/`) `structure.fdf`, `calc.fdf`, `config_extra.fdf`, `calc.out`, **`siesta.RHO.xz`**; `analysis/` (report, `.dat`, `.xyz`, dashboard), `analysis_tight/` (`--tol 1e-6`), `bader/`
`h2o_mesh200`, `h2o_mesh300`, `h2o_mesh600`	water at `MeshCutoff` 200, 300, 600 Ry	as above, without the `.RHO`
`h2o_box10`, `h2o_box14`, `h2o_box28`	the isolated atoms in 10, 14, 28 Å boxes	same
`h2o_szp_s001`, `h2o_dzp_s002`, `h2o_dzp_s0005`, `h2o_tzp_s001`, `h2o_tzp_s0005`	five other basis sets	same

The density grids of the four studies are stored compressed (xz): SIESTA's density is exactly zero beyond the orbital radii, so a 20 Å box compresses to ~100–600 kB. `example_5.17.sh` unpacks them into `output/` (`xz -d siesta.RHO.xz` by hand). The studies keep their inputs, logs and analysis results but not the grids; `make_reference.sh` regenerates everything:

```
bash make_reference.sh ~/scratch/hi_run      # ~1.5 h on 12 cores; then trims into reference_results/
NP=4 JOBS=3 bash make_reference.sh           # MPI ranks per SIESTA run, runs at a time
```

It relaxes the four structures, runs the three stages for each study (the commands of Section 5.1), `stb-bader` on the four `combined/` densities and Stage 3 again at `--tol 1e-6`. Computed with SIESTA 5.4.2 (conda-forge build, MPI), 3 runs of `mpirun -np 4` at a time. One run (the H atom at 600 Ry) aborted with `double free` after a converged SCF on 4 ranks and finished on 2; the script retries a failed run once on 2 ranks.

11.17.2.7 7. Known, deliberate limitations

- **References up to ±1.** Charges past ±1 are flagged and are a lower bound (Section 1.7); there is no ±2 reference.
- **Linear interpolation between neutral and ion.** The pro-atom of Eq. 6, as in the original method [5]; other choices (fractional-occupation atoms [12], Hirshfeld-E [16], ISA/MBIS [6, 7]) give other charges.
- **Spherical references, spin-summed.** Each reference is the angular average of a spin-polarized atom; spin populations are not partitioned [17].
- **Valence pseudo-densities.** Charges are relative to `Z_val`; the core is in neither density (Section 1.6).
- **Basis-confined references** (Section 5.7): the charges inherit the orbital range.
- **Point-charge dipoles** leave out the atomic dipoles (Section 1.8); the check is along non-periodic axes only.
- **Uniqueness** of the fixed point is not proven in general [13]; every run starts from simple Hirshfeld.

11.17.2.8 8. Step-by-step: running this workflow on your own system

1. **Relax the structure** with the template you will use; Stage 1 does not relax.
2. **Write a `calc.fdf` with the physics only**, `%include structure.fdf`, an implicit basis (`PAO.BasisSize`), your functional, k-grid, spin.
3. **Run Stage 1** with `-p <bank>` (`--mesh-cutoff` 300–400 Ry is plenty, Section 5.7); run SIESTA in `combined/` and every `neutral/<El>/`.
4. **Run Stage 2**; run SIESTA in every `ions/<El>/<cation|anion>/` (anions may need more SCF iterations).
5. **Run Stage 3** with `--save-data --png`. Read `[2]` (every reference OK?), `[4]` (the estimated remaining error; rerun with the suggested `--tol` if asked), `[5]`–`[7]`, then `[10]`.
6. **Test the basis** if absolute charges matter (Section 5.7): `PAO.EnergyShift` first.
7. **Quote the method** with the number, and compare charges computed the same way.

11.17.2.9 9. Files in this folder

File	What it is
`README.md`	this tutorial
`example_5.17.sh`	guided, runnable walk-through (10 cases)

File	What it is
`study_table.py`	collects the reference runs into the tables of Section 5 (`[A]`–`[F]`) and draws `figures/h2o_studies.png` and `figures/stockholder_weights.png`
`build_inputs.py`	helper of `make_reference.sh`: the start structures (a G2 molecule in a box, a rock-salt primitive cell)
`make_reference.sh`	regenerates `reference_results/` with SIESTA (Section 6)
`h2o.fdf`, `n2o.fdf`, `nacl.fdf`, `mgo.fdf`	the start structures (`reference_results/relax/*/relaxed.fdf` are the relaxed ones)
`calc.fdf`, `calc_bulk.fdf`	the templates: PBE, DZP, `PAO.EnergyShift 0.01 Ry`; the second with an 8×8×8 k-grid
`reference_results/`	real finished SIESTA runs (Section 6)
`figures/`	the dashboards of the four systems and the two figures of `study_table.py`
`output/`	created by the script (git-ignored)

11.17.2.10 10. Running the script

```
bash example_5.17.sh
```

It pauses between cases (`[Press Enter to continue]`); `yes '' | bash example_5.17.sh` runs it straight through. It starts by wiping its own `output/`. Cases 1–2 run Stages 1 and 2 (no SIESTA needed); Cases 3–10 run Stage 3 on `reference_results/`. It sets `MPLBACKEND=Agg`, so `--show` is replaced by `--png` in the script; try it on a desktop:

```
cp -r reference_results/h2o /tmp/h2o && find /tmp/h2o -name '*.xz' -exec xz -d {} \; &&
stb-hirshfeldAnalysis -O /tmp/h2o --show
```

11.17.2.11 11. Flag reference

`stb-hirshfeldPrep` — `-s/--structure` (default `structure.fdf`) · `--calc` (default `calc.fdf`) · `--mesh-cutoff RY` (default 400) · `--vacuum ANG` (isolated-atom box, default 20) · `-p/--pseudo-dir` (bank or folder; default: the structure's folder) · `-O/--output-dir` (default `hirshfeld_study`) · `--save-report` · `--no-intro` · `-v`.

`stb-hirshfeldIons` — `-O/--output-dir` (the study, default `hirshfeld_study`) · `-p/--pseudo-dir` · `--ref OUT` (the log to read `Z_val` from) · `--save-report` · `--no-intro` · `-v`.

`stb-hirshfeldAnalysis` — `-O/--output-dir` (the study) · `--tol E` (default 0.005 e) · `--max-iter N` (default 20) · `--ref OUT` · `--save-data` · `--png` · `--show` · `--save-report` · `--no-intro` · `-v`.

11.17.2.12 12. What's next

- `stb-bader` (2.6) partitions the same density by zero-flux surfaces — fine for ionic solids, not for H on a pseudo-density (Section 5.8); `stb-nativecharges` reads SIESTA's own Hirshfeld, Voronoi and Mulliken charges.
- `5.16-chargediff` shows *where* the electrons moved between two fragments, partition-free in the planar view — a complement to atomic charges at an interface.
- `5.8-adsorption`: run this workflow on an adsorption site to quantify the charge transfer to the adsorbate.

11.17.2.13 13. References

1. K. B. Wiberg, P. R. Rablen, *Comparison of atomic charges derived via different procedures*, J. Comput. Chem. **14**, 1504 (1993). doi:10.1002/jcc.540141213
2. F. Heidar-Zadeh *et al.*, *Information-theoretic approaches to atoms-in-molecules: Hirshfeld family of partitioning schemes*, J. Phys. Chem. A (2017). doi:10.1021/acs.jpca.7b08966
3. R. F. W. Bader, *Atoms in Molecules: A Quantum Theory*, Oxford University Press (1990).
4. F. L. Hirshfeld, *Bonded-atom fragments for describing molecular charge densities*, Theor. Chim. Acta **44**, 129 (1977).
5. P. Bultinck, C. Van Alsenoy, P. W. Ayers, R. Carbó-Dorca, *Critical analysis and extension of the Hirshfeld atoms in molecules*, J. Chem. Phys. **126**, 144111 (2007). doi:10.1063/1.2715563
6. T. C. Lillestolen, R. J. Wheatley, *Redefining the atom: atomic charge densities produced by an iterative stockholder approach*, Chem. Commun. 5909 (2008). doi:10.1039/b812691g
7. T. Verstraelen *et al.*, *Minimal basis iterative stockholder: atoms in molecules for force-field development*, J. Chem. Theory Comput. **12**, 3894 (2016). doi:10.1021/acs.jctc.6b00456
8. T. A. Manz, N. Gabaldon Limas, *Introducing DDEC6 atomic population analysis: part 1*, RSC Adv. **6**, 47771 (2016). doi:10.1039/c6ra04656h
9. R. F. Nalewajski, R. G. Parr, *Information theory, atoms in molecules, and molecular similarity*, Proc. Natl. Acad. Sci. USA **97**, 8879 (2000). doi:10.1073/pnas.97.16.8879
10. E. R. Davidson, S. Chakravorty, *A test of the Hirshfeld definition of atomic charges and moments*, Theor. Chim. Acta **83**, 319 (1992). doi:10.1007/bf01113058
11. P. Bultinck, P. W. Ayers, S. Fias, K. Tiels, C. Van Alsenoy, *Uniqueness and basis set dependence of iterative Hirshfeld charges*, Chem. Phys. Lett. **444**, 205 (2007). doi:10.1016/j.cplett.2007.07.014
12. S. Van Damme, P. Bultinck, S. Fias, *Electrostatic potentials from self-consistent Hirshfeld atomic charges*, J. Chem. Theory Comput. **5**, 334 (2009). doi:10.1021/ct800394q
13. L. Pujal *et al.*, *Constrained iterative Hirshfeld charges: a variational approach*, J. Chem. Phys. (2022). doi:10.1063/5.0089466
14. D. E. P. Vanpoucke, P. Bultinck, I. Van Driessche, *Extending Hirshfeld-I to bulk and periodic materials*, J. Comput. Chem. **34**, 405 (2013). doi:10.1002/jcc.23088
15. D. E. P. Vanpoucke, I. Van Driessche, P. Bultinck, *Reply to "Comment on 'Extending Hirshfeld-I to bulk and periodic materials'"*, J. Comput. Chem. (2013). doi:10.1002/jcc.23193 — pseudo-valence densities reproduce the all-electron Hirshfeld-I charges; NaF, NaCl, MgO, $SrTiO_3$.
16. T. Verstraelen *et al.*, *Hirshfeld-E partitioning: AIM charges with an improved trade-off between robustness and accurate electrostatics*, J. Chem. Theory Comput. (2013). doi:10.1021/ct4000923
17. D. Geldof *et al.*, *An extension of the Hirshfeld method to open shell systems using fractional occupations*, J. Chem. Theory Comput. (2011). doi:10.1021/ct100743h
18. J. M. Soler *et al.*, *The SIESTA method for ab initio order-N materials simulation*, J. Phys.: Condens. Matter **14**, 2745 (2002).
19. A. García *et al.*, *Siesta: Recent developments and applications*, J. Chem. Phys. **152**, 204108 (2020).
20. J. P. Perdew, K. Burke, M. Ernzerhof, *Generalized gradient approximation made simple*, Phys. Rev. Lett. **77**, 3865 (1996).
21. M. J. van Setten *et al.*, *The PseudoDojo: Training and grading a 85 element optimized norm-conserving pseudopotential table*, Comput. Phys. Commun. **226**, 39 (2018). doi:10.1016/j.cpc.2018.01.012
22. H. J. Monkhorst, J. D. Pack, *Special points for Brillouin-zone integrations*, Phys. Rev. B **13**, 5188 (1976).
23. sisl — `https://zerothi.github.io/sisl/` (N. Papior).
24. S. P. Ong *et al.*, *Python Materials Genomics (pymatgen)*, Comput. Mater. Sci. **68**, 314 (2013).
25. A. H. Larsen *et al.*, *The atomic simulation environment*, J. Phys.: Condens. Matter **29**, 273002 (2017).
26. J. D. Hunter, *Matplotlib: A 2D graphics environment*, Comput. Sci. Eng. **9**, 90 (2007).
27. C. Fonseca Guerra, J.-W. Handgraaf, E. J. Baerends, F. M. Bickelhaupt, *Voronoi deformation density (VDD) charges: assessment of the Mulliken, Bader, Hirshfeld, Weinhold, and VDD methods for charge*

analysis, J. Comput. Chem. **25**, 189 (2004). doi:10.1002/jcc.10351
28. NIST Computational Chemistry Comparison and Benchmark Database (CCCBDB), experimental geometries and dipole moments (H_2O: 1.855 D, r(OH) 0.958 Å, 104.5°; N_2O: 0.161 D, r(NN) 1.128 Å, r(NO) 1.184 Å). https://cccbdb.nist.gov
29. W. Tang, E. Sanville, G. Henkelman, *A grid-based Bader analysis algorithm without lattice bias*, J. Phys.: Condens. Matter **21**, 084204 (2009).

11.17.3 Command-line reference

11.17.3.1 `stb-hirshfeldPrep`

Stage 1 of the Hirshfeld-I (iterative) charge-partitioning workflow: writes a combined/ folder (the full system, forced to a single-point SaveRho-true evaluation) plus one isolated-atom-in-vacuum-box folder per chemical species – everything needed before `stb-hirshfeldIons` (Stage 2) can decide each species' ion sign. Nothing needs to be pre-computed.

Synopsis.

```
usage: stb-hirshfeldPrep [-h] [-s STRUCTURE] [--calc CALC] [--mesh-cutoff RY]
                         [--vacuum ANG] [-p PSEUDO_DIR] [-O OUTPUT_DIR]
                         [--save-report] [--no-intro] [-v]
```

Options

`-s, --structure STRUCTURE`
: Path to the combined/production system's own .fdf structure file (structure_io.read_fdf). Default: '`structure.fdf`'.

`--calc CALC`
: SIESTA input template (.fdf) – same basis/XC/pseudopotential directives used for combined/ and every isolated-atom folder (default: '`calc.fdf`'). Cell, single-point/MD, MeshCutoff and SaveRho are all overridden by this tool's own `config_extra.fdf` regardless of what's in this template; k-grid/Spin/dipole/vdW are left untouched for combined/, and forced Gamma-only + spin-polarized for every isolated-atom folder.

`--mesh-cutoff RY`
: Explicit MeshCutoff (Ry) to force in combined/ and every isolated-atom folder. Default: 400 Ry. type: float

`--vacuum ANG`
: Cubic vacuum-box side (Ang) for every isolated atom – must be large enough that the atom's own density has decayed to ~0 well before the box edge. Default: 20 Ang. type: float

`-p, --pseudo-dir PSEUDO_DIR`
: Pseudopotentials source: a bundled bank (dojo, virtual_vault) or a folder path. Default: reuse the directory `--structure` lives in, which already has every pseudopotential the combined calculation needed.

`-O, --output-dir OUTPUT_DIR`
: Root directory to write combined/ and neutral/<species>/ into (default: hirshfeld_study).

`--save-report`
: Also persist the report to `hirshfeld_prep_report.txt`. Off by default. switch, off by default

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Usage example:
  stb-hirshfeldPrep --structure structure.fdf --calc calc.fdf -p dojo --output-dir hirshfeld_study
```

11.17.3.2 `stb-hirshfeldIons`

Stage 2 of the Hirshfeld-I workflow: reads back every neutral atom folder `stb-hirshfeldPrep` wrote (now run through SIESTA) and writes BOTH a cation and an anion reference folder for EVERY species – Stage 3 needs both, since individual atoms of the same species can converge toward opposite ion states (see module docstring).

Synopsis.

```
usage: stb-hirshfeldIons [-h] [-O OUTPUT_DIR] [-p PSEUDO_DIR] [--ref REF]
                         [--save-report] [--no-intro] [-v]
```

Options

`-O, --output-dir OUTPUT_DIR`
: Same root directory `stb-hirshfeldPrep` wrote into (default: hirshfeld_study) – reads `hirshfeld_manifest.json` from here.

`-p, --pseudo-dir PSEUDO_DIR`
: Pseudopotentials source for the ion folders. Default: reuse the combined system's own directory (recorded in the manifest).

`--ref REF`
: Path to a specific SIESTA output file to read each species' Z_val (valence charge) and native Hirshfeld/Voronoi cross-check from – same convention as `stb-bader`'s own `--ref`. Overrides auto-detection of combined/'s own '`*.out`' (which assumes that log was left with a .out extension; use `--ref` if yours wasn't).

`--save-report`
: Also persist the report to `hirshfeld_ions_report.txt`. Off by default. switch, off by default

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

11.17.3.3 `stb-hirshfeldAnalysis`

Stage 3 of the Hirshfeld-I workflow: reads back every neutral+ion isolated-atom folder, then iterates the Hirshfeld-I reference-density refinement against the combined/production system's real density until convergence.

Synopsis.

```
usage: stb-hirshfeldAnalysis [-h] [-O OUTPUT_DIR] [--tol E-] [--max-iter N]
                             [--ref REF] [--save-report] [--save-data] [--png]
                             [--show] [--no-intro] [-v]
```

Options

`-O, --output-dir OUTPUT_DIR`
: Same root directory `stb-hirshfeldPrep`/Ions wrote into (default: hirshfeld_study).

`--tol E-`
: Convergence tolerance on max|Delta q| between rounds, in electrons (default: 0.005 e-, matching the method's own prototype). The report also estimates how far the charges still are from the fixed point, which can be 2-3 x this value when the iteration converges slowly. type: float

`--max-iter N`
: Maximum number of refinement rounds (default: 20; the method's own prototype converged in 8). type: int

`--ref REF`
: Path to a specific SIESTA output file to read each species' Z_val (valence charge) from – same convention as `stb-bader`'s own `--ref`. Overrides auto-detection of combined/'s own '`*.out`' (which assumes that log was left with a .out extension; use `--ref` if yours wasn't).

`--save-report`
: Also persist the report to `hirshfeld_analysis_report.txt`. Off by default. switch, off by default

`--save-data`
: Write `hirshfeld_charges.dat` (per-atom charges by every method, populations, reference used), `hirshfeld_iterations.dat` (every atom's charge per iteration) and `hirshfeld_charges.xyz` (extended XYZ with the charges as per-atom columns; 'ase gui' colours atoms by the Hirshfeld-I charge). Off by default. switch, off by default

`--png`
: Save the 6-panel dashboard (convergence, charge per iteration, methods per species, Hirshfeld-I vs Hirshfeld per atom, reference profiles, structure coloured by charge) as `hirshfeld_analysis.png`. Off by default. switch, off by default

`--show`
: Open the dashboard on screen (matplotlib). Needs a display. Off by default. switch, off by default

Like every STB command it also accepts `-h`/`–help`, `-v`/`–version` and `–no-intro` (Section 3.3).

Chapter 12

Catalysis

In this chapter

This section holds the catalysis workflows: the hydrogen evolution reaction (HER) and the oxygen evolution reaction (OER).

Its tools are not part of this package: they come from an extension package installed next to it (Section 14.4). The section is always shown in the menu; without the extension, opening it only says where its tools come from. When the extension is installed, its items appear here as **6.1**, **6.2**, … and are documented by the extension itself.

Chapter 13

ML Simulations

In this chapter

13.1 Custom ML Force Field (MACE fine-tuning) . . . 667
13.2 ML Molecular Dynamics . . . 708
13.3 ML Phonons . . . 741
13.4 ML Elastic Constants . . . 773
13.5 ML NEB . . . 800
13.6 ML Adsorption-Site Screening . . . 827

The first item, the MLFF workflow (**7.1**), *makes* a potential: it fine-tunes a committee of MACE models [15] on SIESTA energies, forces and stresses, validates them on a held-out test set and picks new configurations by active learning. Like the DFT workflows it has stages and writes SIESTA folders to run between them.

The other tools *run* a simulation with a MACE potential instead of driving SIESTA: the energies and forces come from the MACE-MP-0 foundation model [16] or from a model fine-tuned with the MLFF workflow. Everything happens in memory in one command, so these tools are self-contained – no folders to run, no separate analysis stage – and write their figures directly as matplotlib PNG files.

Several are fast previews of a SIESTA workflow (phonons, elastic constants, NEB, adsorption sites); comparing the two is a useful check of both. Others have no DFT counterpart in the suite (MD). Chapter 6 describes model choice, devices, dispersion and the caveats that apply to all of them.

All need the optional `ml` extra (`pip install "stb_suite[ml]"`).

Code	Tool	Purpose
7.1	Custom ML Force Field (MACE fine-tuning)	Generate training configurations (rattled, strained and/or AIMD-sampled) and a held-out test set, fine-tune a committee of MACE-MP models on the resulting SIESTA energy/forces/stress, validate each round against the test set and pick new configurations where …
7.2	ML Molecular Dynamics	NVE/NVT/NPT molecular dynamics driven by a MACE potential – fast exploration with the foundation model or your own fine-tuned one.
7.3	ML Phonons	Standalone phonon bands/DOS/thermal properties driven entirely by a MACE potential – no SIESTA, no separate analysis tool needed.
7.4	ML Elastic Constants	Standalone stiffness matrix (stress-strain method) driven entirely by a MACE potential – no SIESTA, no separate strain_*/ folders needed.
7.5	ML NEB	Climbing-image NEB (reaction barrier) driven by a MACE potential; optional SIESTA check of the band and export to the MLFF training set.

Code	Tool	Purpose
7.6	ML Adsorption-Site Screening	Enumerates candidate ontop/bridge/hollow sites, relaxes each with MACE, and ranks them by a genuine ML adsorption energy.

13.1 Custom ML Force Field (MACE fine-tuning)

At a glance

Menu code: 7.1 (a multi-stage workflow)

Code	Stage	Command
7.1.1	Stage 1 - Prep	`stb-mlff`
7.1.2	Stage 2 - Train	`stb-mlffAnalysis`
7.1.3	Stage 3 - Validate	`stb-mlffValidate`
7.1.4	Stage 4 - Active Learning	`stb-mlffActiveLearning`

Guided example: `examples/7.1-mlff/`

Summary: Generate training configurations (rattled, strained and/or AIMD-sampled) and a held-out test set, fine-tune a committee of MACE-MP models on the resulting SIESTA energy/forces/stress, validate each round against the test set and pick new configurations where the committee disagrees – stages linked by `mlff_workflow.json`. Optionally exports a LAMMPS-loadable model.

13.1.1 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/7.1-mlff/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

This workflow turns SIESTA single points into a **machine-learned force field** that you can run a thousand times faster than SIESTA itself, and tells you how far to trust it. It has four stages that talk to each other through one state file, `mlff_workflow.json`:

Stage	Command	Code	What it does
1 · Prep	`stb-mlff`	`7.1.1`	writes the SIESTA folders of the training set (rattled and strained copies of a relaxed structure) and of a held-out test set
2 · Train	`stb-mlffAnalysis`	`7.1.2`	fine-tunes a **committee** of MACE models (MACE-MP foundation model) on the finished SIESTA runs, one *round* at a time
3 · Validate	`stb-mlffValidate`	`7.1.3`	scores every round on the test set — forces, energies, stress, phonons against SIESTA, equation of state, an MD check — and says STOP / CONTINUE / PLATEAU / REGRESSED
4 · Active learning	`stb-mlffActiveLearning`	`7.1.4`	lets the committee find the configurations it is unsure of (with MD, rattles or strains *run with the model*) and writes them as the next SIESTA folders

It answers a question none of `5.1`–`5.14` does: not one property of one structure, but **a potential-energy surface** you can then use for everything the `5.x` ML Simulations tools do — molecular dynamics (`stb-mlmd`), phonons (`stb-mlphonons`), elastic constants (`stb-mlelastic`), barriers (`stb-mlneb`), adsorption sites (`stb-mladsorb`) — at the level of theory of *your* SIESTA calculation instead of the foundation model's.

The worked example (Section 8) runs the whole loop for real on **graphene**, in a 3×3 supercell (18 atoms):

30 SIESTA single points, a committee trained and validated against a SIESTA phonon calculation, one active-learning round driven by molecular dynamics, a second round, and an MD check labelled by SIESTA. Section 9 keeps a **real failure** — active learning that sampled too hard and made the model worse — because it teaches more than the success. Before either, Section 8.1 shows that the SIESTA labels themselves needed a convergence check.

All four stages live in this one folder and this one tutorial — same reasoning as `5.1–5.14`: each stage's output exists only to feed the next.

13.1.1.1 Theory

Symbols at a glance

Symbol	Meaning
`R = {r_i}`	the atomic positions of one configuration (plus its cell)
`E(R)`	the potential-energy surface (PES): total energy as a function of `R`
`F_i = −∂E/∂r_i`	force on atom `i`
σ `= (1/V) ∂E/∂`ε	stress (derivative with respect to a homogeneous strain ε)
ε`_i`	energy contribution of atom `i` in a local (atomic) decomposition
`r_c`	cutoff radius of an atomic environment
`E0_Z`	reference ("isolated atom") energy of element `Z`
`N`	number of atoms of a configuration; `N_train` = training configurations
σ`_F`	committee spread: standard deviation of the predicted force over the models
ν	novelty of a configuration: distance of its environments to the training set
`k_B`	Boltzmann's constant, 0.08617 meV/K

Why a machine-learned force field. Every SIESTA single point solves the Kohn–Sham equations: the cost grows roughly as N^3 with the number of atoms, and one 18-atom graphene cell with a 10×10 k-grid takes ~30–60 s on two cores (Section 8). A molecular-dynamics run needs 10^5–10^6 steps, a converged phonon calculation hundreds of displaced supercells, a melting or diffusion study both — orders of magnitude beyond what direct DFT can afford.

A **machine-learned interatomic potential** (MLIP) is a fitted function `E_ML(R)` that reproduces the DFT potential-energy surface `E_DFT(R)` — energies, forces and stresses — at a small fraction of the cost (here ~0.1 s for 18 atoms on a CPU, a few hundred times cheaper than SIESTA, much more on a GPU, and linear in `N` instead of cubic) [23, 28]. It does **not** replace the DFT level of theory: it *interpolates* it. Everything the MLIP knows comes from the DFT labels it was trained on, so the central questions of this workflow are *which configurations to compute with SIESTA*, *how good the fit is*, and *where it can be trusted*.

The locality ansatz. All modern MLIPs (Behler–Parrinello [1], GAP [2], ACE [3], NequIP [4], MACE [5]) start from the same decomposition:

$$E(R) = \sum_i \varepsilon(\text{ environment of atom } i \text{ within } r_c) \qquad (1)$$

$$F_i = -\partial E/\partial r_i\,, \qquad \sigma_{\alpha\beta} = (1/V)\,\partial E/\partial \varepsilon_{\alpha\beta} \qquad (2)$$

Three consequences matter in practice:

1. **Extensivity and transferability in size.** Because `E` is a sum of local terms, a model trained on 18-atom cells can be run on 10^4 atoms. It is only reliable there if the *local environments* of the large system look like those in the training set.

2. **Forces and stresses come for free** as exact derivatives of the same function, so the dynamics conserves energy (Section 1.10). They are also by far the richest labels: one configuration gives one energy but `3N` force components (18 atoms → 54 numbers) and 6 stress components.
3. **What a periodic training cell can teach is limited by its periodicity.** In a cell of `n×n` primitive cells, every displacement pattern is periodic with that supercell: only phonons with wavevectors commensurate with it (`q = (m₁/n, m₂/n)`) are ever sampled. The 2-atom graphene cell sees only Γ; the 3×3 cell of this example also contains **K = (1/3, 1/3)**, where graphene's highest optical branch has its Kohn anomaly [24]; M = (1/2, 0) needs an even `n`. The cutoff `r_c` is not the limit — the *training data* are.

MACE: an equivariant message-passing network. MACE [5] builds `ε_i` in two message-passing layers. In each layer, every atom collects, from each neighbour within `r_c` (6 Å for MACE-MP), a radial function of the distance times spherical harmonics of the bond direction, and combines them into **many-body** features (products of up to four of them: body order up to 5 per layer, as in the atomic cluster expansion [3]). Features transform under rotations as tensors of definite order, so the energy is exactly invariant and the forces exactly equivariant under rotations, translations, reflections and permutations of identical atoms (E(3) symmetry). After two layers an atom "sees" up to `2 r_c` = 12 Å, but its energy remains a smooth function of its environment.

Foundation models, and what fine-tuning must fix. **MACE-MP-0** [6] is a MACE model trained once on ~1.5 million DFT configurations of the Materials Project (MPtrj [7], PBE/PBE+U with PAW pseudopotentials in a plane-wave code). It knows "chemistry in general" — carbon bonds, graphitic layers — and is a good starting point. It is not a SIESTA model:

- **Different level of theory.** Basis (plane waves vs SIESTA's numerical atomic orbitals), pseudopotentials, `Mesh.Cutoff`, k-sampling, smearing: all change the PES slightly. On this example the raw foundation model has a force error of hundreds of meV/Å against SIESTA (Section 8.4).
- **A different energy zero.** A total energy contains a large per-element constant: SIESTA's absolute energies and the foundation model's differ by ~10^5 meV/atom. Only *energy differences* are physical. We write

  $$E_{SIESTA}(R) = \sum_Z n_Z E0_Z + E_{int}(R) \qquad (3)$$

  and fine-tuning has to put `E0_Z` on SIESTA's scale. Stage 2's default (`--e0s foundation-residual`) fits `E0_Z` by least squares to $E_{SIESTA} - E_{int,foundation}$ over the training set *before* training, so the first epoch starts with the foundation model's own error on energy differences (tens of meV/atom) instead of the whole offset (the `average` option assumes `E_int = 0` and starts eV/atom off). **For a one-element system the `E0` and the mean interaction energy cannot be separated** (every configuration has the same stoichiometry): only their sum is fixed, which is harmless — energy differences between configurations of that composition are all that is predicted.

Fine-tuning continues training the pretrained weights on the SIESTA labels with a small learning rate (`--lr 0.001`; 0.01 destroys what pretraining learned in the first epochs). The result is a model on SIESTA's PES that keeps the foundation's general shape where the training set is silent — the reason why even 21 configurations give a usable model.

The loss function. Training minimizes a weighted sum over the training configurations:

$$L = w_E \sum_c \left((E_{ML} - E_{DFT})/N_c\right)^2 + w_F \sum_c \sum_i |F_{ML,i} - F_{DFT,i}|^2 / (3N_c) + w_\sigma \sum_c |\sigma_{ML} - \sigma_{DFT}|^2 \qquad (4)$$

Energies per atom (eV/atom), forces (eV/Å) and stresses (eV/Å^3) live on very different numerical scales: a 1 GPa stress is 0.006 eV/Å^3, so with `w_σ` = 100 the stress term is ~2 % of the loss and has no effect. Stage 2 uses `--loss stress` with `--stress-weight 1e4` by default (measured on a bulk-Si benchmark). Two practical consequences of (4):

- **Large forces dominate.** The force term is a sum of *squared* errors, and a model's error grows with the force magnitude. One configuration with 15 eV/Å forces weighs as much as many near-equilibrium ones. This is why adding very distorted configurations can make a model *worse* near equilibrium (Section 9).
- **A validation split chooses the epoch.** Each committee member keeps a random fraction of the training configurations (`--valid-fraction`) out of the fit and keeps the epoch with the lowest validation loss; with `--cv K` the configurations are dealt into `K` folds, so every configuration is predicted once by a model that never saw it (the **out-of-sample** error of Stage 2).

Designing the training set: displacement, energy and temperature. A rattled configuration (Gaussian displacements of standard deviation `s` on every Cartesian component of every atom) is a crude stand-in for a thermal snapshot. In the harmonic approximation each vibrational degree of freedom holds `k_BT/2` of potential energy on average, so per atom

$$\langle E - E_0 \rangle / N \;\approx\; (3/2)\, k_B\, T \qquad \Leftrightarrow \qquad T \approx \Delta E / (1.5\, k_B) \tag{5}$$

This turns the energy above the reference — which Stage 1 measures with the foundation model (`--ml-screen`) and Stage 4 with the committee — into a **temperature you can judge**: for graphene 50 meV/atom ↔ 390 K, 200 meV/atom ↔ 1550 K, and graphene melts near 4500 K (~580 meV/atom). Graphene's atoms at 300 K move ~0.05 Å in the plane and more out of it, so rattles of 0.015–0.06 Å per component span roughly 100–1700 K (Section 8.2's table measures it). A 0.15 Å rattle of the 2-atom cell compresses bonds to 1.19 Å (a triple bond) at 1–2 eV/atom, ~10^4 K: no simulation near room temperature will ever visit it (Section 9).

Three axes of the training set:

Axis	Stage 1 option	Teaches	Without it
Displacement amplitude	`--stdev` (several values)	the curvature and anharmonicity of the PES around the structure	one temperature only
Cell volume / shape	`--volume-strains`, Stage 4 `--sampling-method strain`	`E(V)`, bulk modulus, pressure, thermal expansion	a model that knows one volume (its stress is noise)
Cell size	the `-s` supercell, Stage 4 `--supercell`	phonons at more q-points, longer-range correlations	only Γ (2-atom graphene)

For a 2D material the volume strain is an **area strain**: only the periodic axes are scaled, the vacuum keeps its size (Stage 1 detects the vacuum axis).

Measuring the error: what each number means. For a set of configurations (train, validation or test):

$$\begin{aligned} F\ \mathrm{RMSE} &= \sqrt{\textstyle\sum_c \sum_i |\Delta F_i|^2 \,/\, (3 \sum_c N_c)} \quad (\mathrm{meV/Å}) \\ E\ \mathrm{RMSE} &= \sqrt{\langle\, (\Delta E/N)^2 \,\rangle} \quad (\mathrm{meV/atom}) \end{aligned} \tag{6}$$

- **E abs vs E rel.** With one element, a constant energy offset per atom is not determined by the data (Section 1.4). Stages 3 and 4 report the absolute error and the error after removing that constant (*E rel*): only *E rel* affects energy differences, reaction energies or `E(V)`.

- **F rel** = F RMSE / RMS SIESTA force: the error relative to the forces actually present. 30 meV/Å means a lot on a configuration with 0.1 eV/Å forces, little on one with 3 eV/Å.
- **The error of the error.** A test set of `n` configurations measures an RMSE with a statistical uncertainty; Stage 3 estimates it by **bootstrap** (resampling the test configurations): the `+/-` column. Two rounds whose difference is below it are indistinguishable on this test set.
- **Train < out-of-sample < test.** The training error measures the fit, the out-of-sample error (Stage 2) the interpolation inside the training distribution, the **test** error (Stage 3, never-trained configurations) the generalization. The ratio test / out-of-sample is the generalization gap.
- **Per kind, per element, per config.** An average hides a failure mode: Stage 3 breaks the error down by configuration kind (`rattle`, `strain`, active-learning picks, MD frames), element and configuration, and correlates it with the force magnitude and the novelty.

Committees and uncertainty. A single model cannot tell you where it is wrong. A **committee** of `M` models trained on the same data with different random seeds (different weights at the start of fine-tuning, different train/valid splits) agrees where the data constrain it and disagrees where they do not ("query by committee" [8, 9]). The spread

$$\sigma_{F,i} = \sqrt{\frac{1}{M}\sum_m |F_{m,i} - F_i|^2}, \qquad F_i = \frac{1}{M}\sum_m F_{m,i} \tag{7}$$

is an **uncertainty indicator**, not an error bar. Its scale has to be calibrated: Stage 3 computes the ratio *true error / spread* on the test set (a calibrated committee gives ~1; fine-tuned seeds of one foundation model are often over-confident, ratio > 1 [10]) and the correlation between spread and error, per configuration and per atom. Stage 4 uses that ratio to turn a spread into an estimated error. The committee **mean** is also a better model than any member (averaging cancels part of the independent errors: "+29 % from averaging" in Section 9).

Active learning. Instead of guessing more configurations, ask the model where it is uncertain [9, 11, 12]:

1. **Generate candidates** cheaply with the model itself: rattles, MD trajectories at chosen temperatures, strained cells — thousands of evaluations, no SIESTA.
2. **Score** each with the committee spread (`--score force|energy|both`).
3. **Filter**: drop the unphysical ones (`--max-force`, `--max-energy`), and those whose atomic environments are already in the training set (`--min-novelty`). Novelty ν is measured with a model-independent fingerprint (the sorted distances of each atom to its 16 nearest neighbours) in units of the training set's own typical spacing: $\nu \approx 1$ is as far as two independent training configurations are from each other.
4. **Select** a batch (`--top-k`): `greedy` takes the highest scores; `diverse` (default) multiplies the score by `min(1, ν/D)` against the training set *and the picks so far*, so it does not take five copies of the same distortion.
5. **Label** the picks with SIESTA, retrain, validate — and repeat.

The trap: uncertainty grows with distortion. A committee always disagrees most on the most distorted geometries, so "pick the most uncertain" alone drifts towards ever higher energies — geometries no simulation of interest visits, which then dominate the loss (Section 1.5). Stage 4 therefore caps every pick's predicted energy above the reference at `min(--max-energy, --frontier × the training set's 90th-percentile energy)`: each round extends the labelled range by one step (default ×1.5) instead of jumping. Section 9 shows the trap on real data: two rounds of 0.15 Å rattles of the 2-atom cell (picks at 1.2–2.4 eV/atom) made the test force error grow from 21 to 37 meV/Å.

Physical validation. Low force errors on configurations like the training ones are necessary, not sufficient. Stage 3 adds the checks that matter for use:

- **Phonons** (`--phonon-ref`): the committee-mean model on the *same* displaced supercells as a finished SIESTA phonon calculation (`5.4`), frequencies compared q-point by q-point. The harmonic force constants $\Phi_{ij} = -\partial F_i/\partial u_j \approx -\Delta F_i/\Delta u_j$ (finite displacements, phonopy [13]) give the dynamical matrix $D(q) = \Sigma_j \Phi_{0j} e^{iq\cdot R_j}/\sqrt{(m\ m')}$, whose eigenvalues are ω^2. The **residual forces** of the undisplaced supercell — tiny per atom, but summed over a sublattice — are subtracted on both sides (`perfect_supercell/`).
- **Equation of state** of the test strains: a Birch–Murnaghan fit [14] (equation (8) below) compares `V0` and `B0` of SIESTA, every round and the foundation model. For a 2D cell `V` is proportional to the area (the vacuum is fixed), so `B0` is a 3D number of the cell including vacuum — compare it between models, not with a bulk modulus.
- **MD stability** (Stage 2): a short NVT then NVE run with each model; blow-up criteria (atoms closer than 0.6× their starting distance) and the **NVE energy drift** (forces are exact derivatives, so a good model conserves the total energy to < ~1 meV/atom/ps at 1 fs).
- **MD check** (`--md-check`): an NVT trajectory run *with the model*, frames labelled by SIESTA — the error along the trajectory is the test closest to real use.
- **Learning curve**: test error against `N_train` across rounds, fitted as $F = a\ N^{-b}$ (a power law is the usual empirical form [15]). A curve that does not fall says the *sampling* limits the model, not the amount of data; one that **rises** says the new data pull the model away from the test region (Section 9).

The Birch–Murnaghan form used by Stage 3:

$$E(V) = E0 + (9\ V0\ B0 / 16)\ \{\ [(V0/V)^{2/3} - 1]^3\ B0' + [(V0/V)^{2/3} - 1]^2\ [6 - 4\ (V0/V)^{2/3}]\ \} \quad (8)$$

When to stop. Stage 3 gives a verdict on the history of rounds: **STOP** (`--target-force` met), **REGRESSED** (the latest round is worse than the best earlier one by more than the plateau threshold: use the earlier models), **PLATEAU** (the last round gained less than `--plateau` %, default 5 %: change the sampling, not the amount) or **CONTINUE**. Stage 4 adds its own: if even the most uncertain candidate's *estimated* error is below the target, another round will not help in that region.

What this workflow simplifies — explicitly.

- **One level of theory.** All labels come from one SIESTA setup; the model cannot be better than that (basis, functional, pseudopotential). Stage 2 checks that every folder really used the same settings.
- **MACE fine-tuning only**, starting from MACE-MP (small/medium/large) or from the previous round; no training from scratch, no other architecture.
- **Committee uncertainty**, not a Bayesian or ensemble-of-architectures one; members fine-tuned from the same foundation model are correlated (Section 8.5).
- **Novelty is geometric** (neighbour distances), not chemical; it does not see charge or magnetic states.
- **Single points only.** Every folder is a single point (`config_extra.fdf`), never a relaxation or an AIMD run; `--from-aimd` reuses an existing SIESTA trajectory.
- **No long-range electrostatics or dispersion beyond the receptive field** (~12 Å) other than what the model absorbs. Dispersion enters only through the labels: `stb-mlff --d3` adds Grimme's DFT-D3 [27] to every SIESTA folder and the workflow keeps it in every later one (Section 11); the model is then PBE+D3.

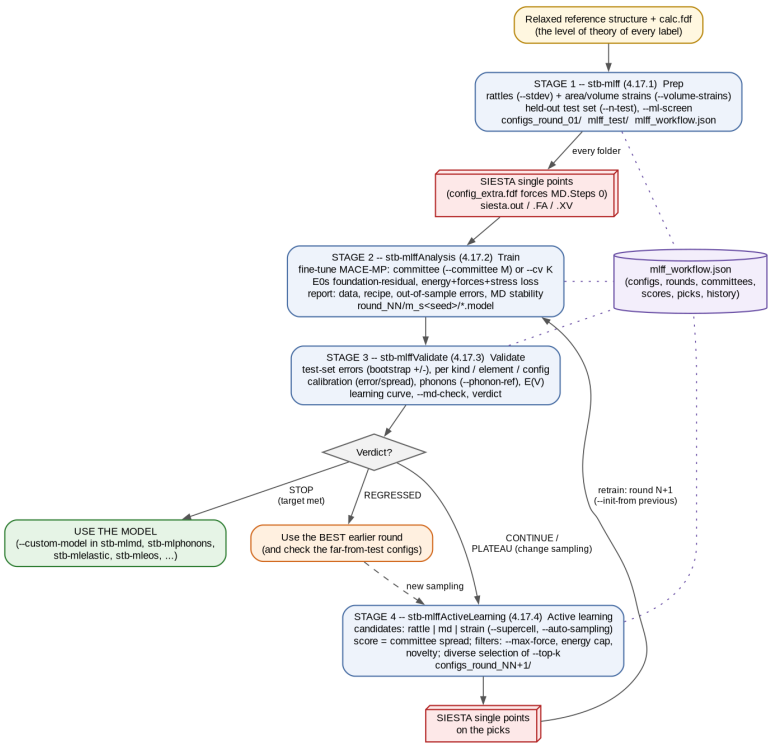
A calculation you can check by hand. Take the numbers of Section 8.

1. **Rattle → temperature.** Stage 1's foundation screen gives `mlff_config_002` (0.03 Å) +52 meV/atom. Equation (5): `T = 52 / (1.5 × 0.08617) = 402 K`.
2. **Area strain.** `--volume-strains +3` scales the two in-plane axes by `(1.03)^{1/2} = 1.01489`: `a = 7.4504 × 1.01489 = 7.561 Å` for the 3×3 cell, the vacuum stays at 20.000 Å (Stage 1's table: V = 990.24 Å^3 = 1.03 × 961.40).
3. **Cost.** 30 SIESTA single points × ~35 s ≈ 18 min on two cores; one committee evaluation of the same 18 atoms ~0.1 s (Stage 4: 99 ms per model evaluation), so a 50-candidate active-learning screen costs less than two SIESTA runs.
4. **Bootstrap.** Stage 3 gives round 1 a test F RMSE of 37.3 ± 8.2 meV/Å from 9 configurations: a relative uncertainty of 22 %, close to the `1/√(2n)` = 24 % expected for an RMSE over `n` = 9 independent configurations. A later round must beat 37 by more than ~8 meV/Å before the difference means anything.

13.1.1.2 Libraries and external dependencies used

- **SIESTA** [16, 17] is *not* called by the suite: you run it in every folder the stages write. It provides the labels — total energy (`siesta.out`), forces (`siesta.FA`), stress (`siesta.out`) and the geometry it actually used (`siesta.XV`). The reference data of Section 8 were produced with SIESTA 5.4.2, GGA-PBE [18], a DZP basis and the bundled `virtual_vault` carbon pseudopotential.
- **mace-torch** [5, 6] — the MACE architecture, the MACE-MP foundation checkpoints (`small`, `medium`, `large`), `mace_run_train` (the training driver Stage 2 calls once per committee member) and `MACECalculator` (an ASE calculator for every evaluation in Stages 2–4). It needs **PyTorch**; both come with the optional `ml` extra (`pip install stb_suite[ml]`).
- **ASE** [19] — `Atoms`, the extended-XYZ training files, Langevin/Velocity-Verlet MD (stability tests, Stage 4 `md` sampling, `--md-check`), `rattle`, `EquationOfState`, the viewer.
- **phonopy** [13] — the force constants and band structures of Stage 3's phonon comparison, on the displacements of a finished `5.4` (`stb-phononsCreate`) folder.
- **numpy / SciPy** [20] — statistics (bootstrap, correlations, least squares of the E0s, the power-law learning curve), neighbour lists.
- **pymatgen** [21] — structure handling shared with the rest of the suite (`.fdf` ↔ `Atoms`).
- **matplotlib** [22] — the PNG summary figures (one per stage; `--show` also opens them). Stage 1 additionally writes gnuplot `.dat/.gplot` coverage plots, like the other `4.x` prep stages.

13.1.1.3 The workflow at a glance



MLFF workflow flowchart

```
flowchart TD
    REF["Relaxed reference structure + calc.fdf<br/>(the level of theory of every label)"] --> S1
    S1["<b>Stage 1 — stb-mlff (7.1.1) · Prep</b><br/>rattles (--stdev), area/volume strains
    (--volume-strains)<br/>held-out test set (--n-test), --ml-screen"] -->|configs_round_01/ +
    mlff_test/| SI1
    SI1[["SIESTA single point in every folder<br/>siesta.out · .FA · .XV"]] --> S2
    S2["<b>Stage 2 — stb-mlffAnalysis (7.1.2) · Train</b><br/>fine-tune MACE-MP: committee or --cv
    K<br/>report: data, recipe, out-of-sample errors, MD stability"]
    -->|round_NN/m_s&lt;seed&gt;/*.model| S3
    S3["<b>Stage 3 — stb-mlffValidate (7.1.3) · Validate</b><br/>test-set errors ±, calibration,
    phonons, E(V),<br/>learning curve, --md-check"] --> V{Verdict}
    V -->|STOP: target met| USE(["Use the model:<br/>stb-mlmd, stb-mlphonons, stb-mlelastic ...
    --custom-model"])
    V -->|REGRESSED| BEST(["Use the best earlier round;<br/>check configs far above the test set"])
    V -->|CONTINUE / PLATEAU| S4
    BEST -.->|new sampling| S4
    S4["<b>Stage 4 — stb-mlffActiveLearning (7.1.4) · Active learning</b><br/>candidates: rattle · md
    · strain (--supercell, --auto-sampling)<br/>score = committee spread; --max-force, energy cap,
    novelty<br/>diverse selection of --top-k"] -->|configs_round_NN+1/| SI2
    SI2[["SIESTA single point on the picks"]] -->|retrain round N+1, --init-from previous| S2
    J[("mlff_workflow.json<br/>configs · rounds · committees · scores · picks · history")]
    S1 -.- J
    S2 -.- J
    S3 -.- J
    S4 -.- J
```

The four stages never call each other: they communicate through **one state file**, `mlff_workflow.json`, inside the run folder (`mlff_runs/` by default). Every stage reads what the earlier ones did from it and

appends what it did, so the defaults "just work" (Stage 2 knows which folders to train on, Stage 3 which rounds and which test set to score, Stage 4 which committee and which reference to use). Every stage still accepts explicit paths (`--path`, `--model`, `--test-path`, `-s`/`-c`) and works without the file.

The run folder.

```
mlff_runs/
+- mlff_workflow.json            <- the state file (paths relative to it; .bak kept)
+- stb_mlff_report.txt           <- Stage 1 report (--save-report)
+- coverage/                     <- Stage 1 plots: gnuplot .dat/.gplot + mlff_coverage.png
+- configs_round_01/             <- configs that train round 1 (Stage 1)
|   +-- mlff_config_001/ ...      <- calc.fdf  structure.fdf  config_extra.fdf  C.psf  (+ SIESTA
output)
+- configs_round_02/             <- Stage 4's picks that train round 2 (numbering continues)
|   +- mlff_config_022/ ...
+- mlff_test/                    <- held-out test set (Stage 1, never trained on)
|   +- test_001/ ...
+- round_01/                     <- Stage 2, round 1
|   +- m_s123/ m_s124/ m_s125/   <- one committee member per seed (mace_run_train work dir)
|   |   +- mlff_model_compiled.model
|   +- mlff_training.png         <- 6-panel training summary
|   +- stb_mlffAnalysis_report.txt
+- round_02/ ...
+- md_check/rNN_<T>K/frame_NNN/  <- Stage 3 --md-check frames (their own set)
+- mlff_validation.png           <- Stage 3 figure
+- stb_mlffValidate_report.txt
+- mlff_al_rNN_<run>.png         <- Stage 4 figure (+ _pool.dat with --save-data)
+- stb_mlffActiveLearning_report.txt
```

The `configs_round_NN/` layout answers the practical question *"where do I run SIESTA now?"*: always in the newest `configs_round_NN/` (and, after Stage 1, in `mlff_test/`). Every stage prints that folder in its warnings section, and Stage 2 refuses to train a new round while the folder meant for it has unfinished runs (`--skip-unlabelled` trains without them).

What the state file holds.

```
{ "format": "stb-mlff-workflow", "version": 1,
  "reference": { "file": "../structure.fdf", "calc": "../calc.fdf", "pseudo_dir": "virtual_vault",
  ... },
  "configs":   [ { "path": "configs_round_01/mlff_config_001", "set": "train", "kind": "rattle",
                   "round": 0, "origin": "rattle 0.015 Ang", "geometry": {...}, "ml_screen": {...} },
                   ... ],
  "rounds":    [ { "round": 1, "dir": "round_01", "configs": [...], "committee": [ {seed, model}, ...
  ],
                   "settings": {...}, "training_errors": {...}, "model_checks": {...},
                 "validation": {...}, "active_learning": [ {...picks, pool statistics...} ] }, ... ],
  "history":   [ { "time": ..., "stage": "prep|train|validate|active_learning", "argv": [...] }, ...
  ] }
```

`set` is `train`, `test` or `md_check`; `kind` is `rattle`, `strain`, `aimd`, `al-rattle`, `al-md`, `al-strain`; `round` is the round that *proposed* the config (0 = Stage 1). Writes are atomic (temp file + rename), and `history` keeps the exact command line of every call — the provenance of every model.

13.1.1.4 4. Stage 1: training and test configurations (`stb-mlff`, code `7.1.1`)

What it does. Reads the reference — the suite's usual pair `-s structure.fdf -c calc.fdf` (`calc.fdf` must `%include` the structure file; it is copied verbatim and only the structure file is rewritten per folder) or one self-contained `--file` — and writes:

- `--n-configs` **rattled** copies, the `--stdev` values cycled round-robin (Gaussian displacement of every Cartesian component, seed `--seed + i` for config `i`);
- one **strained** copy per `--volume-strains` value (cell scaled by `(1 + p/100)^{1/d}` on the `d` periodic axes, atoms rattled with the cycled `--stdev`);
- `--n-test` **held-out test** configurations drawn the same way with an independent seed (`--seed + 777000`) into `mlff_test/` — never trained on;
- optionally frames of an existing SIESTA AIMD run (`--from-aimd LABEL`).

Every folder gets a `config_extra.fdf`, `%included` on top of `calc.fdf`, that forces a single point (`MD.TypeOfRun CG`, `MD.Steps 0`, `MD.NumCGsteps 0`, `MD.VariableCell false`): SIESTA's fdf reader is first-occurrence-wins, so a relaxation template cannot move the sampled geometry.

The report. `[0]` metadata · `[1]` the reference (formula, cell, dimensionality, shortest distances) · `[2]` the calculation settings every folder inherits (and the k-grid as `k × a`) · `[3]` the sampling plan (with the RMS shift each `--stdev` means, and whether the test set allows Stage 3's `E(V)` fit) · `[4]` generation · `[5]` every written folder **read back** (volume, displacement from the reference `u_rms`/`u_max`, shortest distance `d_min`, novelty vs the earlier configs, and with `--ml-screen` the foundation model's energy above the reference and its largest force) · `[6]` coverage of train vs test, close contacts, near-duplicates · `[7]` files · `[8]` next steps · `[9]` **all** warnings.

Why the checks.

Check	Catches
`d_min` < 80 % (yellow) / 65 % (red) of the reference	close contacts: SCF trouble, irrelevant geometries
novelty < `--dup-threshold` (0.25)	near-duplicates: SIESTA time that adds nothing
test config outside the training V/u range	a test set that measures extrapolation
`--ml-screen`: `Fmax > 15 eV/Å` or Δ`E > 1000 meV/atom`	unphysical configurations before paying SIESTA (one shared definition across the 4 stages: `UNPHYSICAL_FMAX`, `UNPHYSICAL_DE`)
fewer than 20 training configs	a quick test, not a production model
fewer than 5 test volumes	Stage 3 cannot fit Birch–Murnaghan (4 parameters)

13.1.1.5 5. Stage 2: training a committee (`stb-mlffAnalysis`, code `7.1.2`)

What it does.

1. Collects the training configurations (from the JSON: every `set = train` folder of the rounds so far, else `--path` / `mlff_config_*`), reads energy, forces and stress, and checks **the level of theory** of every folder (settings in SIESTA's reading order, the k-sampling as `k × a` in reference cells, the `PAO.Basis` block and the md5 of each pseudopotential file).
2. Splits train/valid **in Python** for each member (seed `--seed + m`; `mace_run_train --valid_fraction` is silently ignored once a `--valid_file` is given), or deals `--cv K` folds.
3. Fits the **foundation-residual E0s** (Section 1.4) and writes them explicitly.
4. Runs `mace_run_train` once per member, fine-tuning from the MACE-MP checkpoint (`--foundation-model`, default `medium`) or from the previous round (`--init-from previous`), with energy + forces

+ stress loss (`--stress-weight 1e4`), `--lr 0.001`, `--epochs 100`. **Not** `--multiheads_finetuning` / `--E0s foundation`: they keep the foundation's energy scale, which is not SIESTA's.
5. Evaluates every member on every configuration: train/valid errors, the **out-of-sample pool** (each configuration scored by the members that did not train on it), per kind, against the raw foundation model; committee spread vs held-out error.
6. Runs an **MD stability test** per model (NVT then NVE in an auto supercell of the reference, blow-up criteria, NVE drift, ms/step and ns/day).
7. Records the round in the JSON.
8. Writes the **model card** of every member, `round_NN/m_s<seed>/mlff_model_card.json`, next to the model file (below).

The report. `[0]` metadata · `[1]` training data per kind (energy spread, forces, pressure, volume; SCF convergence read from each log) and the level-of-theory check · `[2]` the training setup (recipe, E0s, splits) · `[3]` training, one line per member (learning history from `results/*_train.txt`, kept epoch, time) · `[4]` errors on the data: every model on every config, the out-of-sample pool per kind vs the foundation, the worst configs · `[5]` committee (spread vs held-out error) · `[6]` comparison with earlier rounds · `[7]` model checks: MD stability and speed · `[8]` summary and files · `[9]` next steps · `[10]` **all** warnings. Figure `round_NN/mlff_training.png` (6 panels); `--view-worst N` writes and opens the worst configs with the per-atom force error as colour.

The model card: a model is always used with its JSON. A `.model` file holds weights and nothing else. So each member gets a card next to it that records where those weights come from and where they can be trusted:

Section	What it holds
`model`	file names and **SHA-256** of the plain and the compiled `.model`, seed, architecture (cutoff, parameters, layers), foundation model and starting weights
`committee`	size, round, the sibling models and cards, the member with the lowest validation force error, committee spread
`elements`, `energy`	the elements trained on; the per-element `E0` (eV) and the statement that energies are SIESTA absolute energies — only differences are physical
`level_of_theory`	SIESTA version, XC, basis, `Mesh.Cutoff`, k-sampling, spin, smearing, md5 of the `PAO.Basis` block and of the pseudopotentials, and **whether the labels include DFT-D3**
`reference_structure`, `training_data`	the reference cell and dimensionality; configurations per kind, atoms per element, which labels (stress) were used, SCF failures, every configuration folder
`domain`	the **trained range**: elements, area or volume per atom of the periodic axes, shortest distance, largest SIESTA force, energy and pressure ranges
`recipe`, `metrics`, `checks`	epochs, kept epoch, learning rate, loss, stress weight, device and the exact `mace_run_train` command; train, validation, out-of-sample and foundation errors; MD stability and speed
`validation`	filled by `stb-mlffValidate`: test-set errors, E(V), phonons, calibration, verdict
`usage`, `provenance`	recommended precision, notes ("trained without stress", "PBE+D3 already", "failed the stability check"), the workflow file, round, command line and software versions

Every tool that takes `--custom-model` now **also requires** `--custom-model-card` (`--ml-custom-model-card` in `stb-adsorb` and `stb-stackingfault`, `--mace-custom-model-card` in `stb-xrdsearch`):

```
stb-mlmd --file si.fdf --custom-model round_03/m_s124/mlff_model_compiled.model \
         --custom-model-card round_03/m_s124/mlff_model_card.json
```

The pair is checked before anything runs: a card of another model — or of the same model retrained — is refused (SHA-256). Then the card is used while the model runs: the tool prints the summary above its report, never adds a second D3 correction to a model whose labels already include it, stops with a

clear message for an element the model was not trained on, and warns (once) when the cell size or the shortest bond is outside the trained range or the predicted forces exceed 1.5 times the largest training force. Models trained before the cards existed get theirs with `stb-mlffAnalysis --make-cards [ROUND]` (no training; the numbers come from the workflow file, the ranges from the SIESTA folders), and `stb-mlffValidate` refreshes the `validation` section of every card of the rounds it scores.

13.1.1.6 6. Stage 3: validation on the held-out test set (`stb-mlffValidate`, code `7.1.3`)

What it does. Scores the committee mean — and every member — of every round on the test set: energy (absolute and offset-removed), forces, stress, per kind, per element, per config; the bootstrap `+/-`; the calibration of the spread; a comparison of the test set with the training set of the detailed round (level of theory, range, the configs far above the test set in energy). Optionally:

Option	Adds
`--phonon-ref DIR`	phonons vs a finished `5.4` folder, q-point by q-point (Section 1.10)
(test strains with ≥ 5 volumes)	Birch–Murnaghan `V0`, `B0` of SIESTA, every round, the foundation
(3+ rounds of different size)	learning curve `F = a N^-b`, configs needed for `--target-force`
`--compare A B`	two rounds config by config and per kind
`--md-check`	NVT frames with the best model → SIESTA folders in `md_check/` (the only option that writes folders)
`--target-force F`	the STOP criterion

The best model. Every run also picks the one model to use and copies it, with its card, to `<run folder>/best_model/` (`mlff_model_compiled.model`, `mlff_model.model`, `mlff_model_card.json`), replacing what the previous run left there. *Best* is the round with the lowest committee-mean force RMSE among the rounds scored in that run and, inside it, the single member with the lowest such RMSE (a member that failed Stage 2's MD stability check only wins if all did). The RMSE is over the test set; once `--md-check` frames are labelled and scored for every round, it is pooled over the test set and those frames, every SIESTA force component weighted alike — the test set is drawn like the training set (often one small cell) and can miss what an MD visits. On bulk Si the 8-atom test set preferred round 1 (23 vs 29 meV/Å) while on 64-atom frames of an 800 K MD round 2 was four times better (35 vs 133 meV/Å); the pooled RMSE picks round 2, and `[13]` says when the two choices differ. The card there carries a `selection` section (round, seed, why, the rounds compared, test scores, the verdict of the latest round) and the JSON state file a `best` entry, so `--custom-model best_model/mlff_model_compiled.model --custom-model-card best_model/mlff_model_card.json` is always the current answer. Run without `--rounds` to compare every round (a restricted run says so in `[13]`).

The report. `[0]` metadata · `[1]` test set (and its comparison with the training set) · `[2]` the models scored · `[3]` test errors of every round (the table to read first) · `[4]` the detailed round per kind / element / config, with error-vs-force and error-vs-novelty correlations and the generalization gap · `[5]` committee (members, averaging gain, calibration) · `[6]` phonons · `[7]` equation of state · `[8]` MD check · `[9]` round comparison · `[10]` history, learning curve and verdict · `[11]` summary · `[12]` next steps · `[13]` **all** warnings. Scores are stored per round (and per config) in the JSON; `mlff_validation.png` has 6 panels plus a third row with whichever of learning curve / E(V) / MD check / comparison is available.

13.1.1.7 7. Stage 4: active learning (`stb-mlffActiveLearning`, code `7.1.4`)

What it does.

1. Loads the committee of the latest round (or `--round K`), the reference, the pseudopotential source and the training set from the JSON; reads the stored quality (Stage 2 out-of-sample, Stage 3 test

error, verdict, calibration).
2. Evaluates the reference cell and the training set with the committee: the **energy scale** of the round (the training set's 90th-percentile energy above the reference, as a temperature).
3. Builds a candidate pool with the model — `--sampling-method rattle` (stdev `--stdev`), `md` (one NVT trajectory per `--md-temperature`, a frame every `--stride` steps; `--md-ensemble npt` for bulk) or `strain` (random symmetric strain up to `--strain-max` on the periodic axes, rattled) — optionally in a `--supercell NA NB NC` (written with the reference k-grid divided by the supercell). `--auto-sampling` first scans `--scan-values` and chooses the setting (Section 1.9).
4. Scores every candidate (`--score force|energy|both`; the energy spread after removing each member's constant offset), filters (`--max-force`, the energy cap, `--min-novelty`) and selects `--top-k` (`--selection diverse|greedy`).
5. Writes the picks into `configs_round_NN+1/` (wrapped into the cell, single-point `config_extra.fdf`, a strained pick's own cell) and records them in the JSON as training configs proposed by this round. `--dry-run` writes nothing.

Seeds. The effective seed is `--seed + 1000 × (round the picks will train)` — every round screens a fresh pool — and rattle candidates add their own +30000, so they never repeat Stage 1's displacement directions (Stage 1 rattles config `i` with seed `42 + i`; with a shared seed, candidate `i` used to be training config `i` along the very same direction).

The report. `[0]` metadata (committee, sampling, score, selection, output folder) · `[1]` reference, committee quality, calibration, training range and **energies** · `[2]` the pool (auto-sampling scan, per setting max|F| / spread / novelty / d_min, energy above the reference, spread-vs-force correlation, new territory, beyond training) · `[3]` the selection funnel (skipped by force, energy cap, novelty) and the comparison with a greedy pick · `[4]` every pick (estimated error, novelty, d_min, pool rank, worst atom, beyond-training flags F/d/E, energies as temperatures) · `[5]` is another round worth it (`--target-force`: STOP / SMALL / CONTINUE / UNSURE) · `[6]` the active-learning history of the workflow · `[7]` SIESTA cost estimate (median training wall time scaled as N^3) · `[8]` summary and files · `[9]` next steps · `[10]` **all** warnings.

13.1.1.8 Worked example: graphene, end to end, with real SIESTA labels

The goal: a SIESTA-quality force field for **graphene up to ~1500 K**, good enough for its phonons and for molecular dynamics, with a force error below **30 meV/Å** (`--target-force 30`) on configurations it never saw.

Choice	Value	Why
Level of theory	PBE, DZP, `PAO.EnergyShift 0.005 Ry`, `Mesh. Cutoff 320 Ry`, 300 K smearing, `virtual_vault` C pseudopotential	one SIESTA setup for every label; `calc.fdf`
k-points	**30×30 per primitive cell** (`kgrid 10 10 1` in the 3×3 cell)	Section 8.1 — 15×15 gives wrong phonons
Training cell	**3×3 supercell, 18 atoms** (`stb-supercell -d 3 3 1`)	contains the K-point phonons; the 2-atom cell only Γ (Sections 1.2, 9)
Model	MACE-MP **medium**, committee of 3, 100 epochs	defaults of Stage 2
Machine	2 CPU cores, no GPU	everything below is reproducible on a laptop

Reference data (everything in `reference_results/`): 30 single points for round 1, 6 active-learning picks for round 2, 6 MD-check frames, and the SIESTA phonon calculation (`5.4`, 5×5 supercell). SIESTA 5.4.2, `mpirun -np 2`, 30–60 s per 18-atom single point, ~2 min per 50-atom phonon supercell.

Before any training: are the labels right?. A model trained on SIESTA can only be as good as SIESTA's own answer at the chosen settings, so the level of theory comes first. Graphene has a trap here.

It is a **semimetal**: its valence and conduction bands touch at the Dirac point K. The highest optical phonon at Γ (the E2g mode, the Raman G band, 1580 cm^{-1} = 47.4 THz measured) couples strongly to the electrons near K — the **Kohn anomaly** [24, 25] — and that coupling is exactly the part of the electronic structure a coarse k-grid gets wrong.

The check (`reference_results/kgrid_e2g/`, `e2g_frozen_phonon.py`): displace the two atoms of the primitive cell by +u and −u along x — the E2g pattern — and read the restoring force; for this two-atom mode $\omega^2 = -(F_1 - F_2)/(m \cdot 2u)$. Eight SIESTA single points (a few seconds each):

```
$ python e2g_frozen_phonon.py reference_results/kgrid_e2g
k-grid (primitive) | u (Ang) | E2g (THz) | (cm^-1) | contains K?
-----------------------------------------------------------------
          15 x 15 |   0.005 |     43.38 |    1447 | yes
          15 x 15 |   0.010 |     45.24 |    1509 | yes
          16 x 16 |   0.005 |     47.42 |    1582 | no
          16 x 16 |   0.010 |     47.33 |    1579 | no
          30 x 30 |   0.005 |     46.46 |    1550 | yes
          30 x 30 |   0.010 |     46.84 |    1562 | yes
          45 x 45 |   0.005 |     46.96 |    1566 | yes
          45 x 45 |   0.010 |     47.08 |    1570 | yes
```

- A Monkhorst–Pack grid divisible by 3 contains K **exactly**. With a coarse one (15×15) and 300 K of electronic smearing, the few k-points at and next to the Dirac point carry far too much weight: the E2g frequency is **softened by up to 4 THz and depends on the amplitude** (a harmonic frequency must not). A finite supercell calculation with an equivalent sampling — the 5×5 phonon supercell with a 3×3 grid, where one displaced atom is a much weaker Γ perturbation — gave **41.0 THz** at Γ, in line with the trend of the amplitude (43.4 THz at 0.005 Å). With the 30×30 sampling the same 5×5 calculation gives **45.9 THz** (`reference_results/phonons/`).
- A grid that misses K (16×16) looks converged at once — for the wrong reason: it misses the anomaly instead of resolving it.
- 30×30 and 45×45 agree within 0.5 THz and lose most of the amplitude dependence: **converged, ~47 THz (1565 cm^{-1})**, in line with other PBE calculations and with experiment.

So this example uses **30×30 per primitive cell** everywhere: `kgrid 10 10 1` in the 3×3 training cell (`calc.fdf`), `6 6 1` in the 5×5 phonon supercell. The primitive-cell run of Section 9 used 15×15 — its phonon "errors" are partly the reference's. Stage 2's level-of-theory check measures the k-sampling as `k × a` in reference cells (here 75 Å), so a folder written with a different grid is flagged.

Stage 1: the configurations.

```
$ stb-supercell -f graphene.fdf -d 3 3 1 -o structure.fdf        # 18-atom reference
$ stb-mlff -s structure.fdf -c calc.fdf --pseudo-dir virtual_vault --n-configs 16 \
      --stdev 0.015 0.03 0.045 0.06 --volume-strains -3 -1.5 1.5 3 4.5 --n-test 9 \
      --ml-screen --export-traj --save-report
```

```
[1] REFERENCE STRUCTURE
Formula           : C18 (18 atoms: C 18)
Cell a, b, c      : 7.4503  7.4503  19.9998 Ang
Dimensionality    : 2D (e.g., a slab or surface) -- vacuum along c
Shortest distances: C-C 1.434 Ang

[2] CALCULATION SETTINGS (inherited by every folder)
k-grid               : 10 x 10 x 1  (k x a = 74.5  74.5  20.0 Ang)
```

```
Run type             : single point in every folder (config_extra.fdf)
Pseudopotentials     : bundled bank 'virtual_vault' -> C.psf (140 KiB)
Dispersion           : none (no DFT-D3; --d3 adds it -- for layered or vdW-bonded systems)

[3] SAMPLING PLAN
Kind   | Count | Parameters
rattle | 16    | 0.015 Ang x4, 0.03 Ang x4, 0.045 Ang x4, 0.06 Ang x4 (RMS shift ~ 0.026, 0.052,
0.078, 0.104 Ang)
strain | 5     | volume -3, -1.5, +1.5, +3, +4.5 % (+ rattle)
test   | 9     | same kinds, cycled, independent seed
Volume strains    : periodic axes only (the vacuum keeps its size) -- an area strain for a 2D cell
Test-set E(V)     : yes -- Stage 3 fits it on the test strains
```

The foundation-model screen of `[5]` turns each amplitude into an energy, and equation (5) into a temperature (medians of the four configurations of each):

`--stdev` (Å)	RMS shift (Å)	`dE_ML` (meV/atom)	≈ T (K)	max force (eV/Å)
0.015	0.026	10–16	80–120	2–3
0.03	0.052	37–52	290–400	4
0.045	0.078	87–141	670–1100	6–7
0.06	0.104	166–221	1300–1700	8–17

So this training set covers graphene from ~100 K to ~1700 K. One configuration (`mlff_config_012`, 0.06 Å) is flagged: `Fmax_ML` 17.4 eV/Å, above the workflow's "unphysical" line of 15 eV/Å, although its energy (+209 meV/atom) is ordinary — one bond compressed to 1.21 Å. It was kept (SIESTA converged it in 13 SCF iterations); the first attempt with `--stdev` up to 0.08 Å had **four** such configurations (Case 2 of the script) and was redone with smaller amplitudes before any SIESTA run — exactly what `--ml-screen` is for.

The 21 training folders go to `mlff_runs/configs_round_01/`, the 9 test folders to `mlff_runs/mlff_test/`; `mlff_configs.xsf` holds all 31 structures for VESTA/OVITO.

SIESTA.

```
$ for d in mlff_runs/configs_round_01/mlff_config_* mlff_runs/mlff_test/test_*; do
      (cd $d && mpirun -np 2 siesta < calc.fdf > siesta.out); done
```

All 30 converged, in 10–16 SCF iterations. Every folder's `calc.fdf` is the template with one line on top, `%include config_extra.fdf`, which forces the single point:

```
$ cat mlff_runs/configs_round_01/mlff_config_001/config_extra.fdf
# Auto-generated -- forces a single-point evaluation (no relaxation): this
# folder is one independent, uncoupled sample point, not something SIESTA
# should be moving on its own.
MD.TypeOfRun          CG
MD.Steps              0
MD.NumCGsteps         0
MD.VariableCell       false
```

Stage 2: round 1.

```
$ stb-mlffAnalysis --committee 3 --device cpu --save-data --save-report
```

```
[1] TRAINING DATA
Labels            : 21 energies, 1134 force components, 126 stress components
Kind   | N  | E - E_min (meV/atom) | F_rms (eV/Ang) | F_max (eV/Ang) | P (GPa)        | V/atom (Ang^3)
rattle | 16 | 0.0 .. 265.4         | 1.44 .. 7.52   | 2.53 .. 19.58  | +0.01 .. +1.22 | 53.411 .. 53.411
strain | 5  | 16.5 .. 274.6        | 1.33 .. 6.04   | 2.46 .. 11.58  | -2.65 .. +2.18 | 51.809 .. 55.814
Level of theory   : ElectronicTemperature 300 K, Mesh.Cutoff 320 Ry, PAO.Basis block none,
PAO.BasisSize DZP
                    PAO.EnergyShift 0.005 Ry, SCF tolerance 1.0e-5 eV, Spin non-polarized, XC.authors
                    PBE
                    XC.functional GGA, k x a (Ang) 73 73 G
[OK] Same level of theory in all 21 folders (settings, k-sampling, basis, pseudopotential files).

[3] TRAINING
Member 1/3 -- seed 123: 19 train / 2 valid (valid: mlff_config_007, mlff_config_008)
  E0s             : C -154.1633 eV ('average' -154.8587); start E RMSE 18.2 meV/atom
Seed | Train/valid | Start E (meV/at) | Val F start | Val F kept | Kept epoch | Time (s) | Model
123  | 19/2        | 18.2             | 739         | 55         | 93/99      | 485      | ok
124  | 19/2        | 18.6             | 641         | 44         | 90/99      | 446      | ok
125  | 19/2        | 19.4             | 611         | 32         | 96/99      | 445      | ok

[4] ERRORS ON THE DATA (independent evaluation, every model on every config)
Out-of-sample pool: 5/21 configs were held out by at least one member
Kind   | N | F RMSE fine-tuned | E RMSE fine-tuned* | F RMSE foundation | F gain
rattle | 4 | 43.2              | 0.7                | 678.1             | +94 %
strain | 1 | 23.0              | 0.7                | 463.4             | +95 %
ALL    | 5 | 40.0              | 0.7                | 640.9             | +94 %

[5] COMMITTEE
Spread vs error   : correlation +0.96 over 5 held-out configs (error / spread ~ 1.1x)

[7] MODEL CHECKS: MD STABILITY & SPEED
MD cell           : reference structure x 2x2x1 = 72 atoms; NVT 400 fs at 300 K, then NVE 200 fs (1 fs
steps)
Seed | Status | T mean (K) | d_min/start | F_max (eV/Ang) | NVE drift (meV/atom/ps) | ms/step
123  | OK     | 271        | 0.94        | 5.44           | +0.01                   | 202.9
```

How to read it:

- **The E0s.** The fitted `E0(C)` = −154.163 eV puts the foundation model's own interaction energy on SIESTA's scale: the starting error on energy differences is 18 meV/atom *before any epoch.* The `average` alternative (−154.859 eV) would start 0.7 eV/atom off (Section 1.4).
- **Training works.** Each member's validation force error falls from 610–740 meV/Å (the foundation model) to 32–55 meV/Å; the kept epoch is near the end (90–96 of 100), so more epochs might still help a little.
- **Fine-tuning is the point.** The raw MACE-MP medium has a force error of ~640 meV/Å against these SIESTA labels: a different basis, pseudopotential and smearing make a different PES. After fine-tuning: 40 meV/Å, energies to 0.7 meV/atom.
- **Few out-of-sample configs.** With a 10 % validation split and 3 members, only 5 of 21 configs were ever held out; `--cv 5` would give all 21 an out-of-sample prediction. The test set (Stage 3) is the real judge.
- **MD stability.** NVT at 300 K then NVE in a 72-atom cell: no blow-up, NVE drift 0.01 meV/atom/ps (exact forces conserve energy), ~0.2 s per step for 72 atoms on the CPU.

Stage 3: round 1 on the test set, phonons, E(V).

```
$ stb-mlffValidate --phonon-ref phonons/phonon_runs --target-force 30 --save-data --save-report
```

```
[3] TEST ERRORS (committee mean, every round)
Model      | E abs   | E rel | F RMSE | +/- | F MAE | F max | F rel % | S RMSE | vs found.
round 1    | 1.14    | 0.71  | 37.3   | 8.2 | 24.5  | 193   | 1.7     | 0.076  | +92 %
foundation | (scale) | 17.99 | 484.9  | -   | 343.5 | 2061  | 21.7    | 0.622  | -

[4] ROUND 1 IN DETAIL
Error vs force    : corr(config F RMSE, SIESTA max|F|) = +0.86 -- the error grows with how far the
config is from equilibrium
Error vs novelty  : corr(config F RMSE, Stage-1 novelty) = +0.96 -- configs unlike the training set
are predicted worse
Generalization    : test F 37.3 vs Stage-2 out-of-sample 40.0 meV/Ang (0.93x)

[5] COMMITTEE (round 1)
Averaging         : committee mean 37.3 vs mean member 39.0, best member 38.5 meV/Ang (+4 % from
averaging)
True error/spread : 3.5x (median over configs with spread >= 1 meV/Ang) -- an error bar from the
spread is ~3.5 x spread
Correlation       : per config +0.95; per atom +0.68

[6] PHONONS vs SIESTA
Residual forces  : subtracted (undisplaced supercell, max 0.0049 eV/Ang per atom) -- SIESTA's and
each model's own
Round   | RMSE | Acoustic | Optical | Shift | Max |dev| | Top (ML/ref)  | Min (ML/ref)  | Imag. q
round 1 | 0.48 | 0.33     | 0.60    | -0.22 | 2.15      | 48.33 / 48.31 | +0.37 / +0.93 | 0

[10] ROUND HISTORY, LEARNING CURVE & VERDICT
[CONTINUE] First scored round -- run an active-learning round (7.1.4), label it, train (7.1.2) and
validate again to see the trend.
```

- **Test error 37.3 ± 8.2 meV/Å**, 1.7 % of the RMS SIESTA force, energy differences to 0.7 meV/atom, stress to 0.08 GPa. The test error matches the out-of-sample one (0.93×): no overfitting. 37 is above the 30 meV/Å target: **CONTINUE**.
- **Where the error is.** It grows with the force (corr +0.86) and above all with novelty (+0.96): the two worst test configs are the most distorted ones (`test_004`, 0.06 Å, 16.6 eV/Å; `test_008`, +3 % strain with 0.06 Å). This is what more data in that region should fix.
- **The committee is over-confident**: its spread ranks the configurations correctly (corr +0.95 per config) but is **3.5× smaller** than the true error. Three seeds fine-tuned from the same foundation model are nearly the same model (Section 1.8). Stage 4 uses this calibration to turn spreads into error estimates.
- **Phonons: 0.48 THz RMSE** over the whole Brillouin zone (8×8 q-mesh, frequencies up to 48 THz: 1 %), no imaginary mode, the top of the optical branch exact (48.33 vs 48.31 THz), a small systematic softening (–0.22 THz). The model never saw a phonon calculation: this is a genuine test of its curvature.
- **E(V) is not meaningful here** — and the report says why. Stage 1 rattles the strained configurations too (with the cycled `--stdev`), and in graphene the rattle energy (up to 260 meV/atom) dwarfs the area-strain energy, so the Birch–Murnaghan fit has R^2 0.67 and a negative `B0`, for SIESTA as for the models. Read it only as "the models reproduce SIESTA on the same configs" (V0 within 0.01 %, `B0` within 0.2 % of SIESTA's fit; the foundation 16 % off). For a clean E(V), run a separate Stage 1 with one small `--stdev` (e.g. 0.005 Å) and strains only.

Stage 4: active learning with molecular dynamics.

```
$ stb-mlffActiveLearning --sampling-method md --auto-sampling --top-k 6 --save-data --save-report
```

```
[1] REFERENCE, COMMITTEE & TRAINING SET
Committee quality : out-of-sample F 40.0 meV/Ang (7.1.2), test F 37.3 meV/Ang (7.1.3), verdict
CONTINUE
Calibration       : true error ~ 3.5 x spread on the test set (7.1.3) -- used below to turn a spread
into an error estimate
Training energies : 90 % within 263, all within 289 meV/atom above the reference (model-predicted;
~2036 / ~2232 K as 3/2 kT per atom)
Energy cap        : picks at most 395 meV/atom above the reference (~3054 K as 3/2 kT; --frontier 1.5
x the training set's 90th percentile 263)

[2] CANDIDATE POOL
Auto-sampling scan (12 candidates per setting, a fresh seed):
Setting K | N  | Score med/max | max|F| med | E med (meV/atom) | New % | Close % | MD
300       | 12 | 0.012 / 0.019 | 3.32       | 37 (~288 K)      | 100   | 0       | ok
600       | 12 | 0.019 / 0.025 | 4.69       | 76 (~589 K)      | 100   | 0       | ok
900       | 12 | 0.025 / 0.033 | 5.44       | 111 (~861 K)     | 100   | 0       | ok
1200      | 12 | 0.029 / 0.042 | 5.90       | 144 (~1114 K)    | 100   | 0       | ok
1500      | 12 | 0.031 / 0.046 | 6.79       | 184 (~1423 K)    | 100   | 0       | ok
Chosen            : 1500 K -- highest median score among the settings whose median energy stays
within --frontier 1.5 x
                    the training set's 90th-percentile energy (263 -> 395 meV/atom above the
                    reference), ...
T target (K) | Steps run | Frames | T inst (K)   | Status
1500         | 500/500   | 50     | 1240 +/- 324 | completed
Spread vs force   : corr +0.52 -- the spread carries information beyond the force magnitude
Energy above ref. : median 168, max 264 meV/atom (model-predicted; median ~1300 K as 3/2 kT); 0 above
the cap (395)

[4] PICKED CONFIGURATIONS
# | Origin            | Score | Est. err | E spr | max|F| | Novelty | d_min | Rank | Worst atom | Range
1 | md 1500 K step 470 | 0.058 | 201      | 0.07  | 9.61   | 2.27    | 1.291 | 1    | C12        | in
2 | md 1500 K step 130 | 0.042 | 147      | 0.08  | 7.25   | 1.96    | 1.323 | 2    | C3         | in
...
Energy above ref. : picks 162 .. 264 meV/atom (~1256 .. 2040 K as 3/2 kT); training set up to 289
Diversity         : each pick vs the other picks, novelty min 1.73, median 2.12
```

- **MD with the model, not with SIESTA.** The whole screen — five short scan trajectories, a 500-step NVT run at the chosen temperature, 110 committee evaluations — took 3.6 min on the CPU, about what SIESTA needs for five of these single points.
- **The temperatures are physical by construction**: the scan's median energies (37 → 184 meV/atom) are, within ~5 %, the `(3/2) k_B T` of equation (5) at the thermostat temperature — the MD samples what graphene really visits at 300–1500 K.
- **The energy cap did not bind** here (picks 162–264 meV/atom, cap 395): MD at 1500 K stays inside the training range (`Range: in` for every pick) while sampling **correlated, collective** distortions a rattle never produces (novelty 1.5–3.0: 100 % of the pool is new territory). Compare Section 9, where the cap is what stops a runaway.
- **The spread is informative** (corr +0.52 with the force, not ~1): MD frames differ in *which* environments they contain, not only in how distorted they are.
- With the calibration, the most uncertain candidate has an estimated error of ~200 meV/Å on its worst atom — far above the 30 meV/Å target, so a round is worth it.

The 6 picks are written to `mlff_runs/configs_round_02/` (`mlff_config_022–027`, the numbering continues), recorded in the JSON as training configs proposed by round 1.

Round 2: label, retrain, compare.

```
$ for d in mlff_runs/configs_round_02/mlff_config_*; do (cd $d && mpirun -np 2 siesta < calc.fdf >
siesta.out); done
$ stb-mlffAnalysis --committee 3 --device cpu --init-from previous --save-data --save-report
$ stb-mlffValidate --phonon-ref phonons/phonon_runs --target-force 30 --compare 1 2 --save-data
--save-report
```

```
[3] TEST ERRORS (committee mean, every round)
Model      | E abs   | E rel | F RMSE | +/- | F MAE | F max | F rel % | S RMSE | vs found.
round 1    | 1.14    | 0.71  | 37.3   | 8.2 | 24.5  | 193   | 1.7     | 0.076  | +92 %
round 2    | 2.81    | 0.50  | 26.8   | 6.2 | 17.2  | 173   | 1.2     | 0.058  | +94 %
foundation | (scale) | 17.99 | 484.9  | -   | 343.5 | 2061  | 21.7    | 0.622  | -

[5] COMMITTEE (round 2)
True error/spread : 1.7x (median over configs with spread >= 1 meV/Ang)
Correlation       : per config +0.85; per atom +0.57

[6] PHONONS vs SIESTA
Round   | RMSE | Acoustic | Optical | Shift | Max |dev| | Top (ML/ref)  | Min (ML/ref)  | Imag. q
round 1 | 0.48 | 0.33     | 0.60    | -0.22 | 2.15      | 48.33 / 48.31 | +0.37 / +0.93 | 0
round 2 | 0.43 | 0.31     | 0.52    | -0.25 | 2.16      | 48.03 / 48.31 | +0.42 / +0.93 | 0

[9] ROUND COMPARISON
Overall          : F 37.3 -> 26.8 meV/Ang (-28.2 %), E rel 0.71 -> 0.50 meV/atom, S 0.076 -> 0.058 GPa
Configs           : 8 better, 0 worse, 1 within +/-10 % (of 9)
Kind   | N | F round 1 | F round 2 | Change  | Better | Worse
rattle | 4 | 37.6      | 26.2      | -30.5 % | 4      | 0
strain | 5 | 37.1      | 27.3      | -26.3 % | 4      | 0

[10] ROUND HISTORY, LEARNING CURVE & VERDICT
Round | Train cfgs | F RMSE | +/- | vs prev | E rel | S RMSE | Phonons | Spread
1     | 21         | 37.3   | 8.2 | -       | 0.71  | 0.076  | 0.48    | 27.2
2     | 27         | 26.8   | 6.2 | -28.2 % | 0.50  | 0.058  | 0.43    | 32.0
[STOP] Target met: test F RMSE 26.8 <= 30 meV/Ang -- stop here and use the model.
```

Six MD-sampled configurations — 29 % more data — cut the test force error by **28 %**, improved **every** test configuration but one (which stayed equal), the energy differences by 30 %, the stress by 24 % and the phonons by 10 %. Note that the test set contains **no MD frames**: the MD picks improved rattles and strains alike, because they taught the model environments (collective distortions) that both contain. The difference, 10.5 meV/Å, is larger than either bootstrap error bar (8.2, 6.2): it is real. The committee also became better calibrated (true error / spread 3.5 $\rightarrow$ 1.7). The verdict is **STOP**: 26.8 $\leq$ 30 meV/Å.

The MD check: the model on its own trajectory. The test set was drawn like the training set. The test closest to real use is a trajectory **run with the model**, whose frames SIESTA then labels:

```
$ stb-mlffValidate --md-check --md-check-temperature 600 --md-check-frames 6 --target-force 30
#   -> mlff_runs/md_check/r02_600K/frame_000 .. frame_005 (recorded as their own set, never the test
set)
$ for d in mlff_runs/md_check/*/frame_*; do (cd $d && mpirun -np 2 siesta < calc.fdf > siesta.out);
done
$ stb-mlffValidate --phonon-ref phonons/phonon_runs --target-force 30 --compare 1 2 --save-report
```

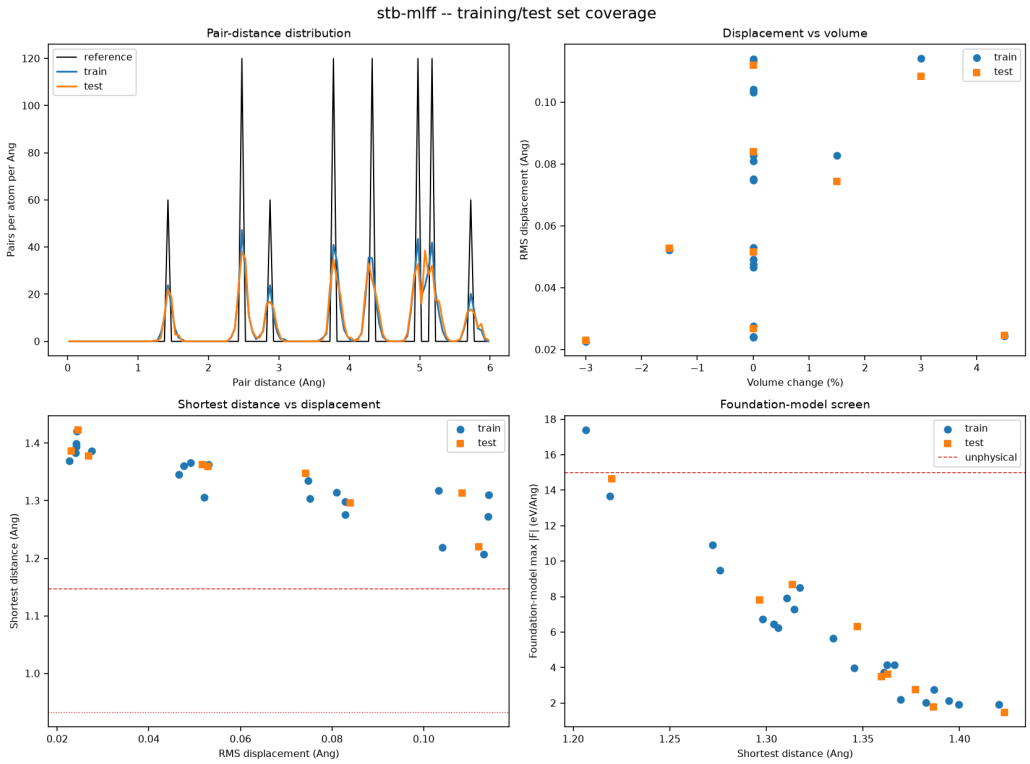
```
[8] MD CHECK (error along a trajectory run with the model)
Run r02_600K: NVT 600.0 K with round 2's model (round_02/m_s123/mlff_model_compiled.model), 6
frame(s) of 18 atoms
```

```
t (fs) | T (K) | Fmax ref | F round 2 | Found. F | dE    | Spread
333    | 552   | 3.63     | 16.9      | 326.0    | +0.02 | 18.6
667    | 422   | 5.07     | 15.7      | 368.7    | -0.26 | 26.4
1000   | 477   | 5.06     | 19.3      | 376.3    | -0.36 | 23.4
1333   | 710   | 5.02     | 17.4      | 383.7    | +0.17 | 26.2
1667   | 604   | 4.43     | 19.0      | 390.8    | +0.06 | 24.3
2000   | 468   | 3.96     | 13.3      | 325.4    | +0.37 | 24.9
Trend            : first third 16.3 -> last third 16.2 meV/Ang (1.0x), corr(t, F) -0.23; test set 26.8
```

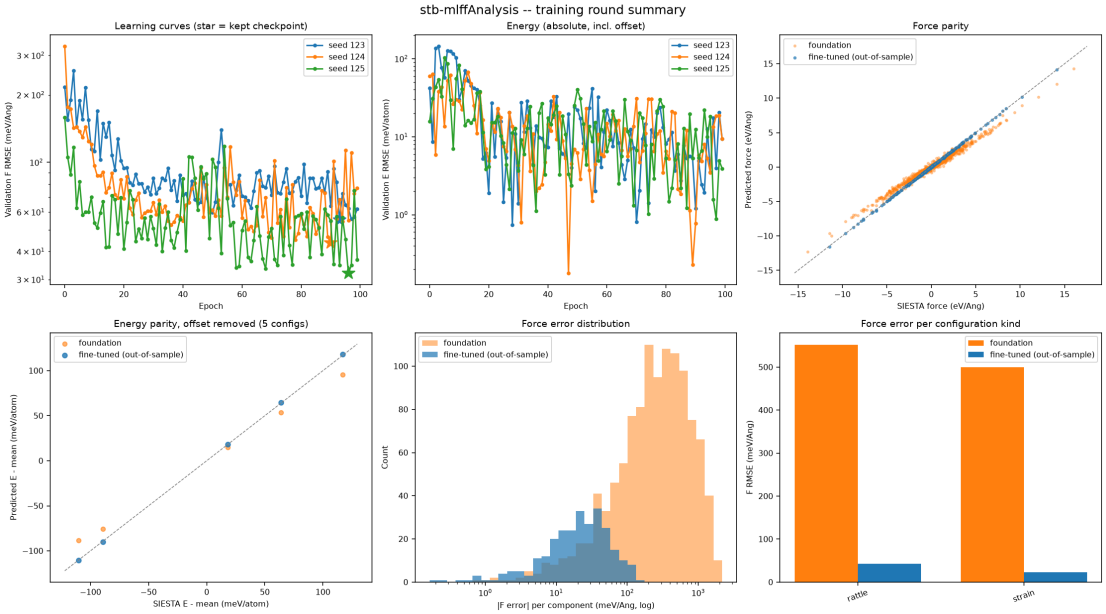
Over 2 ps at 600 K the force error stays at **13–19 meV/Å** with **no growth** along the trajectory (first third 16.3, last third 16.2), below the test-set value — the MD visits milder configurations than the test set's largest rattles — and 20× below the foundation model's 330–390 meV/Å; energies within 0.4 meV/atom. A model drifting into a region it does not know would show the error rising with `t`; this one does not. `--md-check-to-train` would move these six labelled frames into the training set of a next round.

The figures of the run (all written by the stages themselves):

Stage 1 coverage	Stage 2 round 1

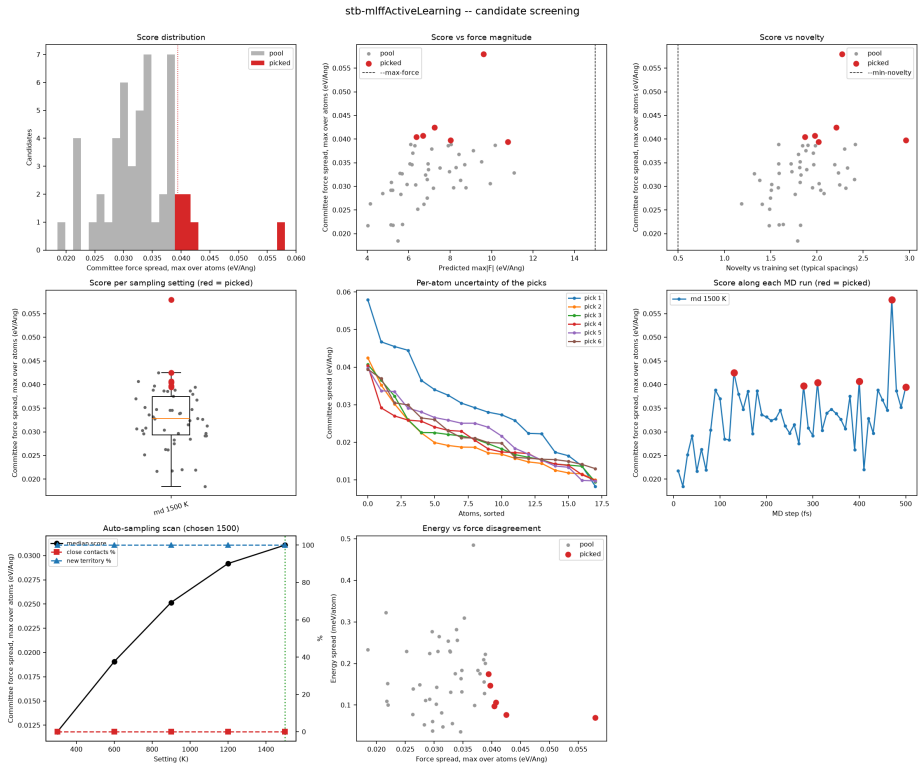


Stage 1 coverage



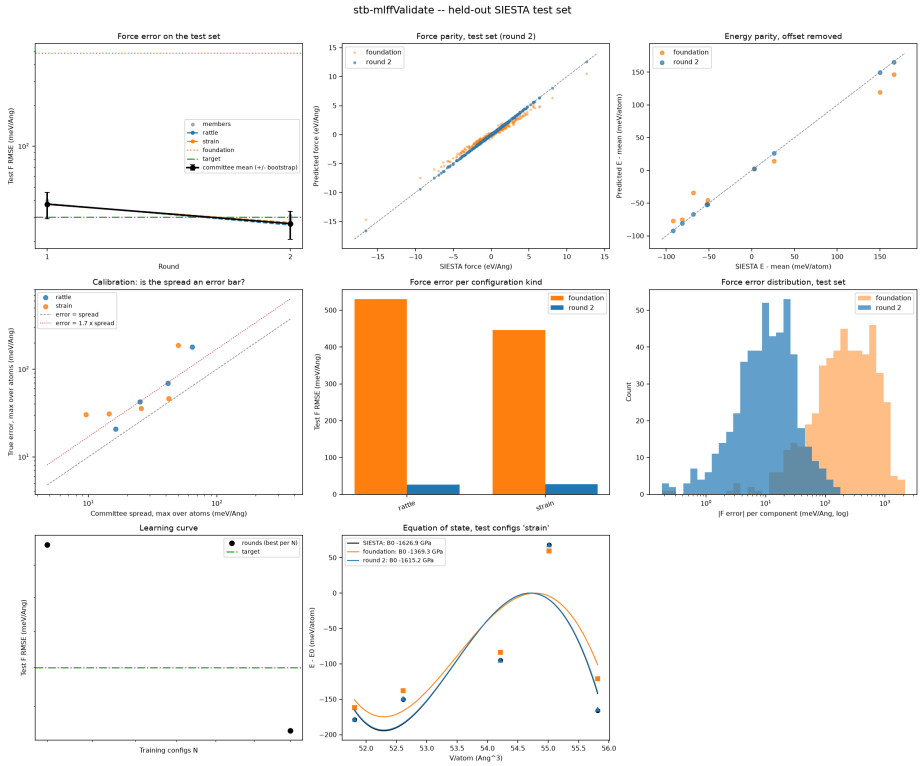
Stage 2 training summary

| | **Stage 4 screening (round 1 → 2)** | **Stage 3 validation (rounds 1 and 2)** | |



Stage 4 active learning

|



Stage 3 validation

|

How far to trust each number.

Quantity	Round 2	Trust it for
Forces	26.8 ± 6.2 meV/Å (1.2 % of the RMS force)	MD and relaxations of graphene up to ~1500 K
Energy differences	0.5 meV/atom	relative energies of distorted configurations of this phase
Stress	0.06 GPa (of a cell with 20 Å vacuum)	in-plane stress trends; not a bulk property
Phonons	0.43 THz RMSE, no imaginary modes	the full dispersion, at the SIESTA level of Section 8.1
E(V)	not tested cleanly (Section 8.5)	— run a strain-only Stage 1 first
Outside the training range	**nothing**	defects, edges, other phases, bilayers, > ~2000 K

The model is exactly as good as SIESTA at these settings, *in the region it was trained on*. For a bilayer or graphite the interlayer interaction needs dispersion in the labels (`stb-mlff --d3`, Section 11) and stacked configurations in the training set.

What to take from the example.

1. **Converge the labels first** (Section 8.1): the model inherits every error of the reference calculation.
2. **Choose the cell for the physics you need**: the 3×3 cell gives K-point phonons that the 2-atom cell can never teach.
3. **Translate amplitudes into temperatures** (equation (5)) and sample the range you will use — no more.

4. **Trust the test set, not the training error**, and its error bar: 37.3 ± 8.2 vs 26.8 ± 6.2 is a real improvement; 37 vs 35 would not be.
5. **Let the model propose its own data with MD** at the temperatures of interest — one round of six configurations reached the target here.
6. **Validate physics, not only RMSEs**: the phonons against an independent SIESTA calculation, and an MD check.

13.1.1.9 A real failure: active learning that sampled too hard

`reference_results/primitive_too_hard/` is a complete run on the **2-atom primitive cell** of graphene, with the older Stage 4 rule for `--auto-sampling` ("the highest committee spread that is still physical"). It is kept because every number in it illustrates a section of the theory.

What was done.

Step	Command (abridged)	Result
Stage 1	`stb-mlff --n-configs 10 --stdev 0.05 --volume-strains -4 -2 2 4 --n-test 8`	14 training + 8 test configs, 2 atoms each, 0–373 meV/atom
Stage 2, round 1	`--cv 5` (5 fold models)	out-of-sample F 18.5 meV/Å
Stage 4 → round 2	`--auto-sampling` (rattle) → **0.15 Å** chosen, 5 picks	picks **1.26–1.92 eV/atom** above the lowest config
Stage 4 → round 3	again 0.15 Å, 5 picks	picks **0.59–2.39 eV/atom**
Stage 3	all three rounds	**REGRESSED**

The SIESTA labels of the picks (`reference_results/primitive_too_hard/mlff_runs/configs_round_0{2,3}/`): C–C distances down to **1.19 Å** (a triple bond; graphene's is 1.434 Å), the two sublattices shifted by up to **0.74 Å** out of the plane — a uniformly buckled sheet, since in a 2-atom cell every neighbour of an A atom is the same B atom — and energies corresponding, by equation (5), to **~9 700–18 500 K**. Graphene melts near 4500 K.

What Stage 3 says about it.

```
Model      | E abs   | E rel | F RMSE | +/- | F MAE | F max | F rel % | S RMSE | vs found.
round 1    | 11.18   | 0.65  | 21.3   | 3.3 | 15.5  | 58    | 0.7     | 0.087  | +98 %
round 2    | 19.65   | 1.05  | 31.1   | 5.9 | 22.4  | 85    | 1.0     | 0.173  | +96 %
round 3    | 56.11   | 2.49  | 37.5   | 7.9 | 25.1  | 117   | 1.2     | 0.214  | +96 %
foundation | (scale) | 37.51 | 852.1  | -   | 655.5 | 1975  | 28.4    | 0.697  | -

Far from the test : 9 training config(s) lie more than one test-set energy spread above the highest
                    test energy: mlff_config_020 (+2016), mlff_config_015 (+1545), mlff_config_021
                    (+1519), ...
[REGRESSED] Round 3 (F 37.5 meV/Ang) is 76 % WORSE than round 1 (21.3): ... Use round 1's models
Learning curve    : F = 1.31 N^+1.06 over 3 training-set sizes (R2 0.99 ...)
                    the test error GROWS with the training-set size -- the added configs pull the
                    model away from the test region
```

Every column got worse — forces, energy differences, stress — and the error bars do not overlap: the regression is real, not noise. The phonons went the same way: against a SIESTA 5×5 phonon calculation at the same level of theory, RMSE 1.60 THz for round 1, 2.39 THz for round 3 (that reference used this run's 15×15 k-grid, itself questionable: Section 8.1). The stage-2 out-of-sample error (18.5 → 42.1 → 107 meV/Å) had already shown it: the new configurations were not like the old ones.

Why.

1. **The score rewards distortion.** The committee spread correlated **+0.93** with the largest predicted force over the candidate pool: "most uncertain" meant "most distorted" (Section 1.9). The scan made it explicit: median spread 0.026 eV/Å at 0.03 Å, 0.19 at 0.15 Å, 0.31 at 0.3 Å (with one candidate at 3780: a blow-up) — and 0.15 Å was the largest setting still under the force and close-contact limits.
2. **The loss is dominated by large forces** (Section 1.5). Five configurations with 8–14 eV/Å forces outweigh fourteen with 3–8 eV/Å; the model spent its capacity there.
3. **The 2-atom cell cannot be helped by more rattles.** Every rattle of it is a Γ-point distortion; larger amplitudes only climb the same Γ modes higher. What the model was missing — K and M phonons, local bond disorder — needs a **supercell** (Section 1.2), which is what Section 8 does.

What the workflow does about it now.

- **An energy cap on every pick**: at most `min(--max-energy, --frontier` × the training set's 90th-percentile energy) above the reference. On this run's round-1 committee (Case 8 of the script; its output is below this list), the picks now sit at 0.28–0.55 eV/atom — one step beyond the labelled range, not 1–2 eV/atom away from it.
- **Stage 3 names the cause**: the *Far from the test* line in `[1]`, a REGRESSED verdict that points at it, and a learning curve written `N^+b` when it rises.
- **Every energy is also given as a temperature**, the number to judge it by.

What Stage 4 prints for it:

```
Training energies : 90 % within 371, all within 393 meV/atom above the reference (~2871 / ~3038 K)
Energy cap        : picks at most 557 meV/atom above the reference (~4306 K as 3/2 kT;
                    --frontier 1.5 x the training set's 90th percentile 371)
Setting Ang (stdev) | ... | E med (meV/atom) | ...
0.03                | ... | 23 (~180 K)      |
0.05                | ... | 64 (~493 K)      |
0.1                 | ... | 253 (~1959 K)    |
0.15                | ... | 562 (~4344 K)    |
0.2                 | ... | 981 (~7589 K)    |
0.3                 | ... | 2175 (~16824 K)  |
Chosen            : 0.1 Ang -- highest median score among the settings whose median energy stays
                    within --frontier 1.5 x the training set's 90th-percentile energy (371 -> 557 ...)
Energy above ref. : picks 281 .. 545 meV/atom (~2177 .. 4220 K as 3/2 kT); training set up to 393
```

The lesson is the same for any material: **decide the temperature (or strain) range the model is for, and keep the training set — and the active-learning picks — inside it, plus a margin.**

13.1.1.10 Reproducing the reference data

Everything in `reference_results/` was computed with the inputs of this folder, SIESTA 5.4.2 (`mpirun -np 2`) and stb 1.9.28x on 2 CPU cores; nothing in it is fabricated. To regenerate it:

```
# --- the reference structure (already relaxed with 30x30 k: forces < 0.005 eV/Ang) -------
stb-supercell -f graphene.fdf -d 3 3 1 -o structure.fdf

# --- 8.1: the E2g k-grid study (8 primitive-cell single points, seconds each) -----------
python e2g_frozen_phonon.py --write e2g --structure graphene.fdf --calc calc_primitive.fdf \
    --pseudo <path to C.psf> --kgrids 15 16 30 45 --amplitudes 0.005 0.01
```

```
for d in e2g/k*; do (cd $d && mpirun -np 2 siesta < calc.fdf > siesta.out); done
python e2g_frozen_phonon.py e2g

# --- the SIESTA phonon reference (5.4): 5x5 supercell, 6x6x1 k = 30x30 per primitive cell --
mkdir phonons && cp graphene.fdf phonons/structure.fdf && cp calc_phonons.fdf phonons/calc.fdf
(cd phonons && stb-phononsCreate -dim 5 5 1 -s structure.fdf -c calc.fdf -p virtual_vault --kgrid 6 6
1)
for d in phonons/phonon_runs/disp-* phonons/phonon_runs/perfect_supercell; do
    (cd $d && mpirun -np 2 siesta < calc.fdf > siesta.out); done

# --- the workflow: Sections 8.2-8.8, in order ------------------------------------------
stb-mlff -s structure.fdf -c calc.fdf --pseudo-dir virtual_vault --n-configs 16 \
    --stdev 0.015 0.03 0.045 0.06 --volume-strains -3 -1.5 1.5 3 4.5 --n-test 9 \
    --ml-screen --export-traj --save-report
#   SIESTA in mlff_runs/configs_round_01/* and mlff_runs/mlff_test/*
stb-mlffAnalysis --committee 3 --device cpu --save-data --save-report
stb-mlffValidate --phonon-ref phonons/phonon_runs --target-force 30 --save-data --save-report
stb-mlffActiveLearning --sampling-method md --auto-sampling --top-k 6 --save-data --save-report
#   SIESTA in mlff_runs/configs_round_02/*
stb-mlffAnalysis --committee 3 --device cpu --init-from previous --save-data --save-report
stb-mlffValidate --phonon-ref phonons/phonon_runs --target-force 30 --compare 1 2 --save-data
--save-report
stb-mlffValidate --md-check --md-check-temperature 600 --md-check-frames 6 --target-force 30
#   SIESTA in mlff_runs/md_check/*/frame_*
stb-mlffValidate --phonon-ref phonons/phonon_runs --target-force 30 --compare 1 2 --save-report
```

Training is stochastic (seeds fix the splits and the initial order, but threads on a CPU do not reproduce bit-for-bit): rerunning Stage 2 gives errors within a few meV/Å of the ones quoted, and Stage 4 then picks different (equivalent) frames. That is why `reference_results/graphene_3x3/mlff_runs/` keeps two snapshots of the state file — `mlff_workflow.after_stage1.json` and `mlff_workflow.after_stage4.json` — with which the script relabels exactly the reference run's folders. The trained models are not shipped (~160 MB per round); the reports of every stage are.

13.1.1.11 Known, deliberate limitations

- **One level of theory, chosen by you.** The model reproduces SIESTA at the settings of `calc.fdf`; it cannot correct the basis, the functional or an unconverged k-grid (Section 8.1). Stage 2 refuses to hide a mixed data set: the level-of-theory check lists every folder that differs.
- **Dispersion is in the labels or not at all.** `stb-mlff --d3` writes `DFTD3 .true.` into every folder's `config_extra.fdf` and records it in the JSON; Stage 4's picks and Stage 3's MD-check frames inherit it, Stages 2–3 check it. The model is then PBE+D3 — never add a D3 correction on top of it (`dispersion=True` in MACE, or torch-dftd), it would count twice. Its range is limited by the model's receptive field (~12 Å for MACE-MP: 2 layers × 6 Å), shorter than D3's own: fine for interlayer distances, approximate for long-range adsorption energies.
- **The test set is drawn like the training set** (same kinds, independent seed). It measures interpolation; MD-check frames and phonons are the tests closer to use. An MD trajectory far outside the training range is not caught by any of them — watch the energies (Stage 4 gives them as temperatures).
- **Rattled strains** (Stage 1): the strain configurations are rattled too, which makes the test-set E(V) noisy for a soft deformation (Section 8.5).
- **Committee uncertainty from seeds** of one foundation model: correlated members, over-confident spreads before calibration (3.5× here in round 1).
- **One element, one composition** in this example: the E0 and the mean interaction energy are then not separable (only energy differences are defined), which the reports state as `E abs` vs `E rel`.

- **CPU training is slow** (~8 min per member per round here, two cores); Stage 2 auto-detects a GPU (`--device cuda`).
- **2D stress and E(V) include the vacuum**: compare models with SIESTA, not with a bulk modulus.

13.1.1.12 Step-by-step: building a force field for your own material

1. **Decide the use** — which phase(s), which temperatures, which properties (phonons? diffusion? melting?) — and therefore the region of configuration space the model must cover. Everything below follows from it.
2. **Converge the SIESTA settings for that property first** (`5.5-convergence`; for a metal or semimetal check the k-grid on a sensitive quantity as in Section 8.1). Relax the reference structure at those settings.
3. **Choose the training cell**: large enough to contain the phonons and local environments you need (a 2×2×2 diamond cell, a 3×3 or 4×4 graphene cell), small enough for SIESTA.
4. **Stage 1** with several `--stdev` (from ~0.3× to ~1× the thermal amplitude at your highest temperature), 5+ `--volume-strains` for a bulk material, `--n-test` for a test set, `--ml-screen` to catch unphysical configs, and `--d3` if dispersion matters. Read `[5]`: turn `dE_ML` into temperatures.
5. **SIESTA** in every folder Stage 1 names.
6. **Stage 2**: the committee (3 members, or `--cv 5` with few configs). Read `[1]` (one level of theory?), `[3]` (did training converge?), `[4]` (out-of-sample errors), `[7]` (stable MD?).
7. **Stage 3** with `--target-force` and, if you can afford one, a SIESTA phonon reference (`--phonon-ref`). Read `[3]` with its `+/-`, `[4]` (where the error is), `[5]` (calibration), `[10]` (the verdict).
8. **Stage 4** with `--sampling-method md` at the temperatures of interest (or `--auto-sampling`), `--supercell` if you need longer wavelengths; check `[1]` (energy cap), `[4]` (picks' energies as temperatures). Run SIESTA in the new `configs_round_NN/`.
9. **Repeat 6–8** (`--init-from previous`) until STOP — or until PLATEAU tells you to change the sampling. If REGRESSED: go back to the best round and read the *Far from the test* line.
10. **Final checks**: `stb-mlffValidate --md-check` (an MD run with the model, labelled by SIESTA) at your use temperature; then use the model that Stage 3 left in `best_model/`: `--custom-model best_model/mlff_model_compiled.model --custom-model-card best_model/mlff_model_card.json` in `stb-mlmd`, `stb-mlphonons`, `stb-mlelastic`, `stb-mlneb`, `stb-mladsorb`

13.1.1.13 Files in this folder

File	What it is
`README.md`	this tutorial
`example_7.1.sh`	guided, runnable walk-through (10 cases)
`graphene.fdf`	relaxed graphene primitive cell (a = 2.4835 Å, 20 Å vacuum)
`structure.fdf`	its 3×3 supercell, 18 atoms — the reference of Stages 1 and 4
`calc.fdf`	SIESTA template of every training/test folder (k 10×10×1 = 30×30 per primitive cell)
`calc_phonons.fdf`	template of the SIESTA phonon reference (primitive cell, k 30×30)
`calc_primitive.fdf`	the primitive-cell template of Section 9 and of the k-grid study (k 15×15, relaxation settings)
`e2g_frozen_phonon.py`	helper: Γ E2g frequency by frozen phonon (Section 8.1)
`figures/`	the flowchart (`mlff_flowchart.png`, from `mlff_flowchart.dot`) and the summary figures of Section 8
`reference_results/graphene_3x3/`	the run of Section 8: every SIESTA folder (outputs only), the stage reports, the state file and two snapshots of it
`reference_results/phonons/`	the finished SIESTA phonon reference (5×5 supercell)
`reference_results/kgrid_e2g/`	the 8 single points of the k-grid study

File	What it is
`reference_results/primitive_too_hard/`	the run of Section 9 (2-atom cell, 3 rounds)
`output/`	created by the script (git-ignored)

13.1.1.14 Running the script

```
bash example_7.1.sh
```

It runs in **about a minute** and never runs SIESTA. Cases 1, 3, 9 and 10 are live and take seconds: Stage 1 on the graphene 3×3 cell, its folders labelled with the finished runs of `reference_results/` after checking the geometries are identical, the k-grid study, and the menu path. Case 8 runs one whole turn of the loop — train, validate, active learning (`--dry-run`) — on the 2-atom primitive-cell data with a **tiny recipe** (2 models, 5 epochs, ~45 s): its errors mean nothing, its mechanics and the energy cap are the point. Cases 2 and 4–7 show the real reports of Section 8's full run (~1 h of CPU training), stored in `reference_results/graphene_3x3/mlff_runs/`. Case 8 needs the `ml` extra (`pip install stb_suite[ml]`) and is skipped without it.

It pauses between cases (`[Press Enter to continue]`); `yes '' | bash example_7.1.sh` runs it straight through. It starts by wiping its own `output/`, and leaves `output/run/mlff_runs/` labelled and ready: `cd output/run && stb-mlffAnalysis --committee 3` trains the real round 1 yourself (~25 min on 2 cores). It sets `MPLBACKEND=Agg`, so `--show` is a no-op; on a desktop:

```
cd output/primitive && stb-mlffValidate --show    # the 6-panel validation figure (Case 8's models)
stb-mlffValidate --view-worst 3                   # the worst test configs, coloured by error
cd ../run && stb-mlff -s structure.fdf -c calc.fdf --pseudo-dir virtual_vault -o view_only --view
```

13.1.1.15 Flag reference

`stb-mlff` (Stage 1) — `-s FILE` (default `structure.fdf`) · `-c CALC` (default `calc.fdf`) · `--file FDF` (one self-contained .fdf instead) · `--pseudo-dir SRC` (bank or folder) · `--n-configs N` (10) · `--stdev Å ...` (0.05) · `--volume-strains % ...` · `--n-test N` (0) · `--from-aimd LABEL` + `--n-samples N` · `--d3` · `-o DIR` (`mlff_runs`) · `--workflow JSON` · `--seed N` (42) · `--ml-screen` + `--ml-model` + `--ml-device` · `--dup-threshold X` (0.25) · `--export-traj [FILE]` · `--no-plots` · `--show` · `--view` · `--save-report` · `--no-intro` · `-v`.

`stb-mlffAnalysis` (Stage 2) — `--workflow JSON` · `--path GLOB` · `--skip-unlabelled` · `--committee M` (3) · `--make-cards [ROUND]` (cards only, no training) · `--cv K` · `--foundation-model {small,medium,large}` (medium) · `--init-from foundation|previous|PATH` · `--epochs N` (100) · `--batch-size N` · `--lr X` (0.001) · `--e0s {foundation-residual,average}` · `--stress-weight W` (1e4) · `--no-stress-training` · `--valid-fraction F` · `--device {cpu,cuda}` (auto) · `--name` · `--seed N` (123) · `-o DIR` · `--skip-foundation-comparison` · `--no-stability` · `--stability-temperature K` · `--stability-steps N` · `--view-worst N` · `--show` · `--save-data` · `--export-lammps` · `--save-report` · `--no-intro` · `-v`.

`stb-mlffValidate` (Stage 3) — `--workflow JSON` · `--rounds K ...` · `--detail-round K` · `--model PATH ...` · `--test-path GLOB` · `--phonon-ref DIR` + `--phonon-mesh NA NB NC` · `--target-force meV/Å` · `--plateau %` (5) · `--foundation-model` · `--skip-foundation-comparison` · `--device` (cpu) · `--png FILE` · `--show` · `--save-data` · `--view-worst N` · `--compare A B` · `--md-check` + `--md-check-temperature K` (300) + `--md-check-steps N` (2000) + `--md-check-frames N` (8) + `--md-check-supercell NA NB NC` · `--md-check-to-train` · `--save-report` · `--no-intro` · `-v`.

stb-mlffActiveLearning (Stage 4) — `--workflow JSON` · `--round K` · `--model PATH ...` · `-s/-c` or `--file` · `--pseudo-dir` · `--supercell NA NB NC` · `--sampling-method {rattle,md,strain}` · `--stdev Å ...` · `--md-temperature K ...` · `--md-steps N` (500) · `--stride N` (10) · `--md-ensemble {nvt,npt}` + `--md-pressure GPa` · `--strain-max %` (3) · `--auto-sampling` + `--scan-values V ...` + `--scan-n N` (12) · `--score {force,energy,both}` · `--selection {diverse,greedy}` + `--diversity D` (1) · `--n-candidates N` · `--top-k N` (5) · `--min-novelty X` · `--train-path GLOB` · `--max-force eV/Å` (15) · `--max-energy meV/atom` (1000) · `--frontier F` (1.5) · `--target-force meV/Å` · `--d3` · `--seed N` (42) · `--device` (cpu) · `-o DIR` · `--dry-run` · `--show` · `--save-data` · `--view-picks` · `--save-report` · `--no-intro` · `-v`.

13.1.1.16 What's next

- Use the model: `7.2 stb-mlmd` (MD at the temperatures it was trained for), `7.3 stb-mlphonons` (phonons in large supercells, thermal properties), `7.4 stb-mlelastic`, `7.5 stb-mlneb`, `7.6 stb-mladsorb` — each with `--custom-model` and its `--custom-model-card`, and with the comparison against the raw foundation model built in.
- `5.4-phonons` is the SIESTA phonon workflow used here as the reference; `5.5-convergence` the discipline of Section 8.1 for the ground state.
- For a bilayer, graphite or an adsorbate, start Stage 1 with `--d3` and include the stackings/sites in the training set; for a defect or a surface, put them in the reference cell — the model only knows the environments it was shown.

13.1.1.17 References

1. J. Behler, M. Parrinello, *Generalized neural-network representation of high-dimensional potential-energy surfaces*, Phys. Rev. Lett. **98**, 146401 (2007).
2. A. P. Bartók, M. C. Payne, R. Kondor, G. Csányi, *Gaussian approximation potentials: the accuracy of quantum mechanics, without the electrons*, Phys. Rev. Lett. **104**, 136403 (2010).
3. R. Drautz, *Atomic cluster expansion for accurate and transferable interatomic potentials*, Phys. Rev. B **99**, 014104 (2019).
4. S. Batzner *et al.*, *E(3)-equivariant graph neural networks for data-efficient and accurate interatomic potentials*, Nat. Commun. **13**, 2453 (2022).
5. I. Batatia, D. P. Kovács, G. N. C. Simm, C. Ortner, G. Csányi, *MACE: Higher order equivariant message passing neural networks for fast and accurate force fields*, Adv. Neural Inf. Process. Syst. **35** (NeurIPS 2022).
6. I. Batatia *et al.*, *A foundation model for atomistic materials chemistry*, arXiv:2401.00096 (2023).
7. B. Deng *et al.*, *CHGNet as a pretrained universal neural network potential for charge-informed atomistic modelling*, Nat. Mach. Intell. **5**, 1031 (2023) (the MPtrj data set).
8. H. S. Seung, M. Opper, H. Sompolinsky, *Query by committee*, Proc. 5th Annual Workshop on Computational Learning Theory (COLT), 287 (1992).
9. E. V. Podryabinkin, A. V. Shapeev, *Active learning of linearly parametrized interatomic potentials*, Comput. Mater. Sci. **140**, 171 (2017).
10. F. Musil, M. J. Willatt, M. A. Langovoy, M. Ceriotti, *Fast and accurate uncertainty estimation in chemical machine learning*, J. Chem. Theory Comput. **15**, 906 (2019).
11. J. S. Smith, B. Nebgen, N. Lubbers, O. Isayev, A. E. Roitberg, *Less is more: sampling chemical space with active learning*, J. Chem. Phys. **148**, 241733 (2018).
12. C. Schran, K. Brezina, O. Marsalek, *Committee neural network potentials control generalization errors and enable active learning*, J. Chem. Phys. **153**, 104105 (2020).
13. A. Togo, I. Tanaka, *First principles phonon calculations in materials science*, Scr. Mater. **108**, 1 (2015).
14. F. Birch, *Finite elastic strain of cubic crystals*, Phys. Rev. **71**, 809 (1947).

15. C. Cortes, L. D. Jackel, S. A. Solla, V. Vapnik, J. S. Denker, *Learning curves: asymptotic values and rate of convergence*, Adv. Neural Inf. Process. Syst. **6**, 327 (1994).
16. J. M. Soler *et al.*, *The SIESTA method for ab initio order-N materials simulation*, J. Phys.: Condens. Matter **14**, 2745 (2002).
17. A. García *et al.*, *Siesta: Recent developments and applications*, J. Chem. Phys. **152**, 204108 (2020).
18. J. P. Perdew, K. Burke, M. Ernzerhof, *Generalized gradient approximation made simple*, Phys. Rev. Lett. **77**, 3865 (1996).
19. A. H. Larsen *et al.*, *The atomic simulation environment*, J. Phys.: Condens. Matter **29**, 273002 (2017).
20. P. Virtanen *et al.*, *SciPy 1.0*, Nat. Methods **17**, 261 (2020).
21. S. P. Ong *et al.*, *Python Materials Genomics (pymatgen)*, Comput. Mater. Sci. **68**, 314 (2013).
22. J. D. Hunter, *Matplotlib: A 2D graphics environment*, Comput. Sci. Eng. **9**, 90 (2007).
23. V. L. Deringer, M. A. Caro, G. Csányi, *Machine learning interatomic potentials as emerging tools for materials science*, Adv. Mater. **31**, 1902765 (2019).
24. S. Piscanec, M. Lazzeri, F. Mauri, A. C. Ferrari, J. Robertson, *Kohn anomalies and electron-phonon interactions in graphite*, Phys. Rev. Lett. **93**, 185503 (2004).
25. M. Lazzeri, F. Mauri, *Nonadiabatic Kohn anomaly in a doped graphene monolayer*, Phys. Rev. Lett. **97**, 266407 (2006).
26. J. Maultzsch, S. Reich, C. Thomsen, H. Requardt, P. Ordejón, *Phonon dispersion in graphite*, Phys. Rev. Lett. **92**, 075501 (2004).
27. S. Grimme, J. Antony, S. Ehrlich, H. Krieg, *A consistent and accurate ab initio parametrization of density functional dispersion correction (DFT-D) for the 94 elements H-Pu*, J. Chem. Phys. **132**, 154104 (2010).
28. O. T. Unke *et al.*, *Machine learning force fields*, Chem. Rev. **121**, 10142 (2021).
29. H. J. Monkhorst, J. D. Pack, *Special points for Brillouin-zone integrations*, Phys. Rev. B **13**, 5188 (1976).

13.1.2 Implementation details

The state file.. The four stages communicate through `mlff_workflow.json` inside the run folder (`mlff_runs/` by default). It records the reference structure and template, the pseudopotential source and whether DFT-D3 is on; every configuration (folder, set `train`/`test`/`md_check`, kind, origin, geometry descriptors); every training round (committee members with seed, model path and validation metrics, settings, test scores, active-learning picks); and the history of every call. Paths are relative to the file, so the run folder can be moved to a cluster; writes are atomic. Later stages find it at `mlff_runs/mlff_workflow.json`, or `./mlff_workflow.json` inside the folder, and every stage still accepts explicit paths without it. Layout:

```
mlff_runs/
  configs_round_01/mlff_config_001 ...   training configurations (Stage 1)
  mlff_test/test_001 ...                 held-out test set (Stage 1)
  round_01/m_s123/ ...                   committee of round 1 (Stage 2)
  configs_round_02/mlff_config_011 ...   picks of active learning (Stage 4)
  md_check/rNN_<T>K/frame_000 ...        MD-check frames (Stage 3)
  mlff_workflow.json
```

One definition of an unphysical configuration is shared by all stages: a force above 15 eV/Å or an energy more than 1000 meV/atom above the reference.

Stage 1 – Prep (`stb-mlff`).. From the reference (`-s`/`-c`, or one self-contained `-file`) it writes `-n-configs` rattled configurations (Gaussian displacements of standard deviation `-stdev`, one or several values), optionally strained cells (`-volume-strains`, isotropic on the periodic axes, rattled – without them the

model sees one volume only and cannot learn $E(V)$, bulk moduli or pressure), frames sampled from an existing AIMD run (`-from-aimd`, `-n-samples`), and `-n-test` held-out test configurations of the same kinds with an independent seed. Every folder is a single point (`config_extra.fdf`); `-d3` forces DFT-D3 everywhere and is recorded. The report reads every folder back and tabulates volume, displacement from the reference and shortest distance, flags close contacts, near-duplicates (environment fingerprints, `-dup-threshold`) and test configurations outside the training range; `-ml-screen` evaluates each configuration with MACE-MP before SIESTA to catch unphysical ones. Coverage plots go to `coverage/`; `-export-traj` writes all structures as one trajectory.

Stage 2 – Train (`stb-mlffAnalysis`). After SIESTA has labelled the configurations, it fine-tunes a committee of `-committee` (default 3) MACE-MP models with seeds `-seed`, `-seed+1`, …, each with its own train/validation split (`-valid-fraction`), in `round_NN/m_s<seed>/`. `-cv K` replaces the committee by K-fold cross-validation. The recipe defaults come from a three-seed benchmark on bulk Si against a held-out SIESTA test set:

- reference energies E0 from `foundation-residual` – a least-squares fit of the SIESTA energy minus the foundation model's interaction energy – because SIESTA's absolute energies are not on MACE-MP's scale and the `average` choice starts the model eV/atom off;
- learning rate `-lr` 0.001 (0.01 wrecks the pretrained weights in the first epochs), `-epochs` 100, `-batch-size` 5;
- stress in the loss with `-stress-weight` 10^4 (in eV/Å^3 the stress is otherwise a negligible part of the loss), unless `-no-stress-training`.

`-init-from previous` continues from the last round's model. The report covers the dataset per kind (energy spread, forces, pressure, volume, SCF convergence), the level of theory across folders (settings in SIESTA's reading order, $k \times a$, basis block, pseudopotential checksums), the learning curve of each member, every model evaluated on every configuration (train, validation and an out-of-sample pool scored by the members that did not train on it) against the foundation model (energies compared after removing the constant offset), committee spread against error, the worst configurations (`-view-worst`), and an MD stability test of each model (NVT then NVE in a supercell, `-stability-temperature`, `-stability-steps`) with its speed in ns/day. `-export-lammps` also writes a model LAMMPS can load. All warnings are collected in a final section.

Stage 3 – Validate (`stb-mlffValidate`). It scores every round's committee mean and members on the test set – energies (absolute and offset-removed), forces and stress, per kind and per element, with bootstrap uncertainties – the calibration of the committee (true error divided by spread), $E(V)$ fits of the test kinds (Birch–Murnaghan V_0, B_0 against SIESTA), a power-law learning curve $F = aN^{-b}$ across rounds, phonons against a finished **`stb-phononsCreate`** folder (`-phonon-ref`: the model on the same displacements, frequencies compared point by point), and a verdict: `STOP` (target force error `-target-force` reached), `REGRESSED` (the latest round is worse than an earlier one by more than `-plateau` per cent – it names the round to use), `PLATEAU` (gain below `-plateau`) or `CONTINUE`. `-md-check` is its only writing option: an NVT run with the best model whose frames become SIESTA folders (set `md_check`); after they are labelled, the error along the trajectory is reported, and `-md-check-to-train` moves them into training.

Stage 4 – Active learning (`stb-mlffActiveLearning`). It generates a pool of candidates – `-sampling-method rattle`, `md` (NVT, or NPT for bulk with `-md-ensemble npt`, at the `-md-temperature` values) or `strain` (random symmetric strains up to `-strain-max`) – optionally in a `-supercell` of the reference (its k-grid divided accordingly), scores them by committee disagreement (`-score force`: the largest per-atom force spread; `energy`: spread after removing each member's constant offset; `both`), and

selects `-top-k` picks. Selection (`-selection diverse`, default) multiplies the score by a novelty factor (model-independent environment fingerprints relative to the training set and earlier picks, `-diversity`, `-min-novelty`) and rejects candidates beyond `-max-force` or more than $\min$(`-max-energy`, `-frontier`$\times$ the training set's 90th-percentile energy) above the reference – without that energy cap the highest-spread candidates are always the most distorted ones, and training on them made a graphene model worse. `-auto-sampling` scans the sampling amplitude (`-scan-values`, `-scan-n`) and keeps the most informative physical setting. The picks become new training folders of the next round; `-dry-run` writes nothing. When the committee is calibrated by Stage 3, spreads are converted to estimated errors and a verdict against `-target-force` is given.

Committees of fine-tuned models

Members fine-tuned from the same foundation model are nearly the same model: on Si their spread tracked the error per configuration but not per atom and was 3–15 times smaller than the true error. Treat spreads as a ranking of candidates, and rely on the test set (Stage 3) for the error.

Using the model.. The final `.model` works with every tool that takes `-custom-model` (or a model path), and with LAMMPS when exported. It is valid for the level of theory of the labels: a model trained on DFT-D3 labels already contains dispersion.

13.1.3 Command-line reference

13.1.3.1 `stb-mlff`

Stage 1 (Prep) of the Custom ML Force Field workflow: generates training configurations (rattled displacements and/or AIMD-sampled snapshots) from a reference SIESTA calculation, ready for the user to run through SIESTA, plus an optional held-out test set (`--n-test`). Creates `mlff_workflow.json`, through which the next stages – Train (`stb-mlffAnalysis`), Validate (`stb-mlffValidate`), Active learning (`stb-mlffActiveLearning`) – find what the earlier ones did.

Synopsis.

```
usage: stb-mlff [-h] [-s STRUCTURE] [-c CALC] [--file FILE] [--n-configs N_CONFIGS]
                [--stdev STDEV [STDEV ...]] [--volume-strains PCT [PCT ...]]
                [--pseudo-dir PSEUDO_DIR] [--from-aimd LABEL] [--n-samples N_SAMPLES]
                [--n-test N_TEST] [-o OUTPUT_DIR] [--workflow JSON] [--seed SEED]
                [--d3] [--save-report] [--ml-screen] [--ml-model ML_MODEL]
                [--ml-device {cpu,cuda}] [--dup-threshold DUP_THRESHOLD]
                [--export-traj [FILE]] [--no-plots] [--show] [--view] [--no-intro]
                [-v]
```

Options

`-s, --structure STRUCTURE`
: Reference structure (default: `structure.fdf`): lattice, species and coordinates, rewritten in every generated configuration.

`-c, --calc CALC`
: Calculation template (default: `calc.fdf`), with '%include <structure>' and the basis/mesh cutoff/SCF settings, copied verbatim into every folder (a single-point `config_extra.fdf` is %included on top).

`--file FILE`
: Instead of `-s`/`-c`: one self-contained reference .fdf (structure and settings in the same file), reused verbatim with new coordinates.

`--n-configs N_CONFIGS`

Number of rattled configurations to generate (default: 10). Ignored if `--from-aimd` is given without also wanting rattled configs (see `--n-configs 0` to disable rattling entirely). type: int

`--stdev STDEV [STDEV ...]`

One or more rattling displacement standard deviations, in Angstrom (default: 0.05). Cycled round-robin across `--n-configs`. type: float

`--volume-strains PCT [PCT ...]`

Also write one configuration per volumetric strain, in % (e.g. -5 -3 -1 1 3 5): the cell scaled isotropically by (1 + PCT/100)^(1/3), atoms rattled with `--stdev`. Without them every configuration has the reference volume and the fitted model cannot learn the volume response (E(V), bulk modulus, pressure). Default: none. Bulk only – in a slab this also scales the vacuum. type: float

`--pseudo-dir PSEUDO_DIR`

Pseudopotentials source: a bundled bank (dojo, virtual_vault) or a folder with <element>.psf/.psml (default: current directory).

`--from-aimd LABEL`

Also sample starting geometries from an existing AIMD trajectory ('`<LABEL>.ANI`' + '.out'), in addition to (or instead of, with `--n-configs 0`) rattled configurations – more physically realistic diversity than rattling alone.

`--n-samples N_SAMPLES`

Number of evenly-spaced AIMD frames to sample when `--from-aimd` is given (default: 5). type: int

`--n-test N_TEST`

Also write N held-out test configurations to mlff_test/test_NNN/ (default: 0): drawn like the training ones (each `--stdev`, each `--volume-strains` entry, cycled) with an independent seed, never trained on. Stage 3 (`stb-mlffValidate`) scores every round against them – needed to know when to stop. type: int

`-o, --output-dir OUTPUT_DIR`

Run folder (default: mlff_runs): the training configs go to <run>/configs_round_01/mlff_config_NNN/ (they train round 1; Stage 4's picks go to configs_round_02/, ...), the test set to <run>/mlff_test/, plus the workflow file and the next stages' round_NN/ folders.

`--workflow JSON`

Workflow state file shared by the 4 stages (default: <output-dir>/mlff_workflow.json). Created here; an existing one is copied to .bak first.

`--seed SEED`

Random seed for rattling (default: 42). type: int

`--d3`

Add Grimme's DFT-D3 dispersion correction (SIESTA's DFTD3 .true., its own defaults) to every folder's `config_extra.fdf`, so the labels – and the model – are PBE+D3. Recorded in the workflow file: Stage 4's picks and Stage 3's `--md-check` frames get it too, and Stage 2 checks every folder has it. For vdW-bonded systems (layers, molecular crystals, physisorption). Default: whatever the template says (off if nothing). switch, off by default

`--save-report`

Also persist the report to <run folder>/stb_mlff_report.txt. Off by default. switch, off by default

`--ml-screen`

Before any SIESTA run, evaluate every config with a MACE-MP foundation model (needs the 'ml' extra): its max force and energy above the reference flag unphysical geometries, and show how far off the base model starts. Informational only – nothing is filtered. switch, off by default

`--ml-model ML_MODEL`

Model for `--ml-screen`: small/medium/large or a .model file (default: medium).

`--ml-device {cpu,cuda}`

Device for `--ml-screen` (default: cpu). choices: cpu, cuda

`--dup-threshold DUP_THRESHOLD`

Flag a config whose atomic environments are all closer than this to an earlier one (units of the set's typical spacing; default: 0.25; 0 disables). type: float

`--export-traj [FILE]`

Also write reference + every config as one multi-frame file in the run folder (default name: `mlff_configs.xsf`; .xsf, .pdb or .xyz – extended XYZ carrying folder/set/kind per frame), e.g. for VESTA/OVITO.

`--no-plots`

Do not write the coverage plots (gnuplot .dat + .gplot in <run folder>/coverage/). switch

`--show`

Also display the coverage figure on screen (it is always saved as <run folder>/coverage/mlff_coverage.png, unless `--no-plots`). switch, off by default

`--view`

Open the written configurations in ASE's interactive viewer (frame 0 = reference, then training, then test configs, in the report's order). Needs a display. switch, off by default

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Example usage:
  stb-mlff -s structure.fdf -c calc.fdf --n-configs 10 --stdev 0.05 0.1
  stb-mlff --n-configs 30 --stdev 0.05 0.1 --volume-strains -5 -3 -1 1 3 5
  stb-mlff --n-configs 6 --from-aimd aimd --n-samples 6
  stb-mlff --n-configs 30 --stdev 0.05 0.1 --volume-strains -3 3 --n-test 8
  stb-mlff --file si.fdf --n-configs 10     # one self-contained .fdf instead
```

13.1.3.2 `stb-mlffAnalysis`

Stage 2 (Train) of the Custom ML Force Field workflow: aggregates the finished SIESTA calculations (energy + forces, best-effort stress), builds a MACE-format training set, and fine-tunes a MACE-MP foundation model on it via 'mace_run_train' – by default a committee of 3 models (different seeds), recorded as a new round in `mlff_workflow.json` for Validate (`stb-mlffValidate`) and Active learning (`stb-mlffActiveLearning`). Reports the dataset, each member's training history and errors (train, validation, out-of-sample per kind, vs the raw foundation model), the committee agreement, and every warning in one final section.

Synopsis.

```
usage: stb-mlffAnalysis [-h] [--path PATH] [--workflow JSON] [--skip-unlabelled]
                        [--committee COMMITTEE] [--cv K]
                        [--foundation-model {small,medium,large}]
                        [--init-from foundation|previous|PATH] [--epochs EPOCHS]
                        [--batch-size BATCH_SIZE] [--lr LR]
                        [--e0s {foundation-residual,average}]
                        [--stress-weight STRESS_WEIGHT] [--no-stress-training]
                        [--valid-fraction VALID_FRACTION] [--device {cpu,cuda}]
                        [--name NAME] [--seed SEED] [-o WORK_DIR]
                        [--skip-foundation-comparison] [--no-stability]
                        [--stability-temperature K]
                        [--stability-steps STABILITY_STEPS] [--view-worst N]
```

```
                    [--make-cards [ROUND]] [--show] [--save-data]
                    [--export-lammps] [--save-report] [--no-intro] [-v]
```

Options

`--path PATH`

Glob pattern matching the config folders to train on (default: the training configs listed in `--workflow`, else every mlff_config_* here or in mlff_runs/, in configs_round_*/ subfolders or directly).

`--workflow JSON`

Workflow state file shared by the 4 stages (default: mlff_runs/mlff_workflow.json, or ./mlff_workflow.json when run inside that folder). Read for the config list, updated with this round (committee models and metrics); created next to the config folders if missing.

`--skip-unlabelled`

Train even when the configs meant for this round (<run>/configs_round_NN/, e.g. Stage 4's picks) have no finished SIESTA run yet – they are left out and train a later round. Without it the stage stops and says where to run SIESTA. switch, off by default

`--committee COMMITTEE`

Number of models to train (default: 3), seeds `--seed`, `--seed+1`, ...: each gets its own train/validation split and training seed. Their force disagreement is what `stb-mlffActiveLearning` ranks candidates by, and `stb-mlffValidate` scores the committee mean. 1 trains a single model (that score then falls back to the largest predicted force). type: int

`--cv K`

K-fold cross-validation instead of a random `--valid-fraction` per member: the configs are dealt into K folds and member k validates on fold k, so EVERY config gets one out-of-sample prediction (the committee is then the K fold models; overrides `--committee`). Best with few configs. Default: off. type: int

`--foundation-model {small,medium,large}`

MACE-MP foundation checkpoint (default: medium; small is ~2x faster). It is the starting point unless `--init-from` says otherwise, and always the baseline of the comparison. choices: small, medium, large

`--init-from foundation|previous|PATH`

Starting weights: 'foundation' (default, the MACE-MP checkpoint), 'previous' (member k continues from member k of the latest round in `--workflow`: faster active-learning rounds) or a .model file.

`--epochs EPOCHS`

Max training epochs (default: 100). type: int

`--batch-size BATCH_SIZE`

Training batch size (default: 5). type: int

`--lr LR`

Learning rate (default: 0.001; 0.01 wrecks the pretrained model in the first epochs). type: float

`--e0s {foundation-residual,average}`

Per-element reference energies (default: foundation-residual):
foundation-residual - fit so that the foundation model's own interaction
energy is on the SIESTA scale from the start
average - mace_run_train's own fit, which assumes zero interaction
energy (starts eV/atom off; the earlier behavior) choices: foundation-residual, average

`--stress-weight STRESS_WEIGHT`

Weight of the stress term in the training loss, used when every configuration has a SIESTA stress (default: 10000; forces weigh 100, energy 1). Stress is what teaches the model the volume response – pair it with `stb-mlff --volume-strains`. type: float

`--no-stress-training`
: Train on energy and forces only, even if stress is available. switch, off by default

`--valid-fraction VALID_FRACTION`
: Fraction of configurations held out for validation (default: 0.1). type: float

`--device {cpu,cuda}`
: Training device (default: auto-detect, cuda if available). choices: cpu, cuda

`--name NAME`
: Experiment/model name (default: mlff_model).

`--seed SEED`
: Training random seed of the first committee member (default: 123). type: int

`-o, --work-dir WORK_DIR`
: Working directory for this round (default: round_NN/ next to `--workflow` when that file exists, else the current directory). With `--committee` > 1 each member goes to <work-dir>/m_s<seed>/.

`--skip-foundation-comparison`
: Skip evaluating the raw (non-fine-tuned) foundation model on the data for comparison. On by default (comparison runs unless this is passed). switch, off by default

`--no-stability`
: Skip the MD stability test (on by default: a short NVT then NVE run with each model, in a supercell of the reference, checking it does not blow up and how much the NVE energy drifts; it also times the model).switch

`--stability-temperature K`
: Temperature of the stability MD (default: 300 K). type: float

`--stability-steps STABILITY_STEPS`
: NVT steps of the stability MD, 1 fs each, followed by half as many NVE steps (default: 400). type: int

`--view-worst N`
: Write the N worst-predicted configs to <round>/worst_configs.xyz (per-atom force error as 'F_error' and as the initial charge, so ASE's viewer can color by it: View > Colors > By initial charge) and open them. Default: 0. type: int

`--make-cards [ROUND]`
: Do not train: (re)write the model card (`mlff_model_card.json`) of every member of ROUND (default: the latest) from the workflow file and its SIESTA folders – for rounds trained before the cards existed, or after a folder moved. Existing cards are overwritten, validation results are re-attached from the workflow file. type: int

`--show`
: Also display the round summary figure on screen (it is always saved). switch, off by default

`--save-data`
: Also write <round>/mlff_training_errors.dat: per config, its kind, SIESTA max force, out-of-sample and foundation F RMSE and committee spread. Off by default. switch, off by default

`--export-lammps`
: Also compile the fine-tuned model into a TorchScript file loadable by LAMMPS's 'pair_style mace' (via mace_create_lammps_model, part of mace-torch) – '<model>-lammps.pt'. Off by default.switch, off by default

`--save-report`
: Also persist the report to `stb_mlffAnalysis_report.txt` (in the round directory). Off by default. switch, off by default

Like every STB command it also accepts `-h`/`--help`, `-v`/`--version` and `--no-intro` (Section 3.3).

From the built-in help.

```
Example usage:
  stb-mlffAnalysis                      # configs from mlff_workflow.json, 3-model committee
  stb-mlffAnalysis --show               # also display the summary figure
  stb-mlffAnalysis --path 'mlff_config_*' --epochs 100 --device cpu --committee 1
  stb-mlffAnalysis --foundation-model small --device cuda
  stb-mlffAnalysis --export-lammps
```

13.1.3.3 `stb-mlffValidate`

Stage 3 (Validate) of the Custom ML Force Field workflow: scores every training round recorded in `mlff_workflow.json` (its committee mean and each member) against the held-out SIESTA test set from `stb-mlff --n-test` – energy, force and stress errors, overall, per kind and per config, with a bootstrap error bar –, compares the test set with the training data (level of theory, range covered), checks whether the committee's disagreement tracks the real error, optionally compares phonons against a finished `stb-phononsCreate` run, and says whether another active-learning round is worth it. Every warning goes to one final section. Reads finished calculations only – except `--md-check`, which writes SIESTA folders of MD frames to score on the next run.

Synopsis.

```
usage: stb-mlffValidate [-h] [--workflow JSON] [--rounds K [K ...]] [--detail-round K]
                        [--model PATH [PATH ...]] [--test-path GLOB]
                        [--phonon-ref DIR] [--phonon-mesh NA NB NC]
                        [--target-force MEV_ANG] [--plateau PCT]
                        [--foundation-model {small,medium,large}]
                        [--skip-foundation-comparison] [--device {cpu,cuda}]
                        [--png FILE] [--show] [--save-data] [--view-worst N]
                        [--compare A B] [--md-check] [--md-check-temperature K]
                        [--md-check-steps MD_CHECK_STEPS]
                        [--md-check-frames MD_CHECK_FRAMES]
                        [--md-check-supercell NA NB NC] [--md-check-to-train]
                        [--save-report] [--no-intro] [-v]
```

Options

`--workflow JSON`
: Workflow state file (default: mlff_runs/mlff_workflow.json, or ./mlff_workflow.json inside that folder). Rounds and test set are read from it; each round's metrics are stored back.

`--rounds K [K ...]`
: Rounds to score (default: all rounds in `--workflow`). type: int

`--detail-round K`
: Round shown per kind, per config and per member, and in the figure (default: the latest scored round). type: int

`--model PATH [PATH ...]`
: Score these model files as one committee instead of the workflow's rounds (nothing is stored back).

`--test-path GLOB`
: Test config folders (default: the test set in `--workflow`, else 'mlff_test/test_*'). Quote the pattern.

`--phonon-ref DIR`
: A finished `stb-phononsCreate` folder (phonopy_disp.yaml + disp-*/ with SIESTA forces): the committee-mean model is run on the SAME displacements and its frequencies compared q-point by q-point.

`--phonon-mesh NA NB NC`
: q-point mesh for `--phonon-ref` (default: 8 8 8). type: int

`--target-force MEV_ANG`
: Test F RMSE (meV/Ang) that is good enough for the intended use: the verdict says stop once the latest round reaches it. type: float

`--plateau PCT`
: A round that lowers the test F RMSE by less than this, in %, is reported as a plateau (default: 5). type: float

`--foundation-model {small,medium,large}`
: Foundation model for the baseline row (default: the one the latest round was trained from, else medium). choices: small, medium, large

`--skip-foundation-comparison`
: Do not score the raw foundation model on the test set. switch, off by default

`--device {cpu,cuda}`
: Device to run the models on (default: cpu). choices: cpu, cuda

`--png FILE`
: Summary figure (default: `mlff_validation.png`, next to `--workflow`).

`--show`
: Also display the summary figure on screen (it is always saved). switch, off by default

`--save-data`
: Also write `mlff_validation_errors.dat` (next to `--workflow`): per test config of the detailed round, its kind, SIESTA max force, model and foundation F RMSE, energy error, committee spread, novelty. Off by default. switch, off by default

`--view-worst N`
: Write the N worst-predicted test configs to `mlff_test_worst.xyz` (per-atom force error as 'F_error' and as the initial charge, so ASE's viewer can color by it: View > Colors > By initial charge) and open them. Default: 0. type: int

`--compare A B`
: Compare two rounds config by config (which test configs got better or worse from round A to round B, per kind too). Both are scored even if `--rounds` leaves them out. type: int

`--md-check`
: Run an NVT MD with the best round's model and write evenly spaced frames as SIESTA single-point folders in <run folder>/md_check/ (recorded in the workflow file as their own set, not the test set). Run SIESTA in them; every later `stb-mlffValidate` then reports the error along the trajectory – the test closest to real use. The one option that writes folders. switch, off by default

`--md-check-temperature K`
: MD check temperature (default: 300 K). type: float

`--md-check-steps MD_CHECK_STEPS`
: MD check length in 1 fs steps (default: 2000). type: int

`--md-check-frames MD_CHECK_FRAMES`
: Frames written for SIESTA, evenly spaced (default: 8). type: int

`--md-check-supercell NA NB NC`
: Run the MD check in this supercell of the reference (default: 1 1 1); the k-grid is divided accordingly, as in `stb-mlffActiveLearning --supercell`. type: int

`--md-check-to-train`
: Move the finished MD-check frames into the training set (they are already SIESTA-labeled): the next `stb-mlffAnalysis` trains on them. switch, off by default

`--save-report`
: Also persist the report to `stb_mlffValidate_report.txt` (next to `--workflow`). Off by default. switch, off by default

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Example usage:
  stb-mlffValidate                        # every round in mlff_workflow.json
  stb-mlffValidate --show --save-data     # display the figure, write per-config errors
  stb-mlffValidate --target-force 30 --phonon-ref phonon_runs
  stb-mlffValidate --view-worst 3         # worst test configs in ASE's viewer
  stb-mlffValidate --model a.model b.model --test-path 'mlff_test/test_*'
```

13.1.3.4 `stb-mlffActiveLearning`

Stage 4 (Active Learning) of the Custom ML Force Field workflow: uses already fine-tuned model(s) (`stb-mlffAnalysis`) to screen a fresh batch of candidate configurations (independent rattled snapshots, or frames from short NVT trajectories driven by the model itself via `--sampling-method` md, reusing `stb-mlmd`'s own dynamics), optionally in a `--supercell` of the reference, and writes the most informative ones as new mlff_config_NNN/ folders, ready for the user to label with real SIESTA and feed back into another round of `stb-mlffAnalysis`. With 2+ `--model` (a committee, e.g. the same data trained with different `--seed`) the score is the committee's force disagreement – an actual uncertainty signal; with one model it falls back to the largest predicted force, a cheap heuristic that favours the most distorted geometries. Picks are filtered for novelty: one whose atomic environments are all close to the training set or to an earlier pick is skipped. The report describes the pool, the selection funnel, each pick against the training set (extrapolation, close contacts, estimated error from Validate's calibration) and the SIESTA cost; every warning goes to one final section.

Synopsis.

```
usage: stb-mlffActiveLearning [-h] [--workflow JSON] [--round K]
                              [--model PATH [PATH ...]] [-s STRUCTURE] [-c CALC]
                              [--file FILE] [--pseudo-dir PSEUDO_DIR]
                              [--supercell NA NB NC]
                              [--sampling-method {rattle,md,strain}]
                              [--strain-max PCT] [--md-ensemble {nvt,npt}]
                              [--md-pressure GPA] [--score {force,energy,both}]
                              [--selection {diverse,greedy}] [--diversity D]
                              [--auto-sampling] [--scan-values V [V ...]]
                              [--scan-n SCAN_N] [--target-force MEV_ANG]
                              [--n-candidates N_CANDIDATES]
                              [--stdev STDEV [STDEV ...]] [--md-steps MD_STEPS]
                              [--md-temperature MD_TEMPERATURE [MD_TEMPERATURE ...]]
                              [--stride STRIDE] [--top-k TOP_K]
                              [--min-novelty MIN_NOVELTY] [--train-path GLOB]
                              [--max-force EV_ANG] [--d3] [--max-energy MEV_ATOM]
                              [--frontier FACTOR] [--seed SEED] [--device {cpu,cuda}]
                              [-o OUTPUT_DIR] [--dry-run] [--show] [--save-data]
                              [--view-picks] [--save-report] [--no-intro] [-v]
```

Options

`--workflow JSON`
: Workflow state file (default: mlff_runs/mlff_workflow.json, or ./mlff_workflow.json inside that folder): gives the committee, the reference .fdf, the training set for the novelty filter and the output directory when those options are omitted; the picked folders are recorded in it.

`--round K`
: Take the committee of round K from `--workflow` (default: the latest). type: int

`--model PATH [PATH ...]`
: Fine-tuned MACE model file(s), overriding `--workflow`'s committee. Two or more form a committee: the score becomes their force disagreement (`stb-mlffAnalysis --committee` N trains one). With one model the score is its largest predicted force.

`-s, --structure STRUCTURE`
: Reference structure .fdf (default: the one in `--workflow`), rewritten with each pick's coordinates; with `-c/--calc`, the `calc.fdf` template (default: `calc.fdf`) that %includes it, copied into every folder.

`-c, --calc CALC`
: Calculation template that %includes `--structure` (default: `calc.fdf` with `--structure`, else the one in `--workflow`).

`--file FILE`
: Instead of `-s/-c`: one self-contained reference .fdf – basis/pseudos/SCF settings are reused verbatim in every candidate written out.

`--pseudo-dir PSEUDO_DIR`
: Directory (or bundled bank) to search for <element>.psf/.psml pseudopotentials (default: the one `stb-mlff` recorded in `--workflow`, else the current directory).

`--supercell NA NB NC`
: Sample candidates in an NA x NB x NC supercell of `--file` (default: 1 1 1, the reference cell itself). A small training cell only constrains the interactions its own periodicity allows – e.g. an 8-atom diamond cell sees the Gamma and X phonons but not L – and a supercell adds the rest. Each folder gets the reference k-grid divided by the supercell size in its `config_extra.fdf`. SIESTA cost grows steeply with the atom count.type: int

`--sampling-method {rattle,md,strain}`
: How to generate candidate configurations (default: rattle):
rattle - independent, static Gaussian-displaced snapshots (cheap: 1 evaluation per candidate and model).
md - short NVT trajectories with the (first) model itself (`stb-mlmd`'s own dynamics, reused directly); more physically representative of the configuration space the system actually explores, at the cost of running real MD steps (`--md-ensemble` npt lets the cell breathe too).
strain - random cell deformations (volume and shear, up to `--strain-max`, periodic axes only), rattled: the volume/stress response. choices: rattle, md, strain

`--strain-max PCT`
: Largest strain component, % (`--sampling-method` strain only, default: 3). type: float

`--md-ensemble {nvt,npt}`
: MD ensemble for `--sampling-method` md (default: nvt); npt (Berendsen, bulk only, `--md-pressure`) samples the cell as well, so the picks carry their own lattice. choices: nvt, npt

`--md-pressure GPA`
: Pressure for `--md-ensemble` npt, GPa (default: 0). type: float

`--score {force,energy,both}`
: What the committee's disagreement is measured on (default: force):
force - max over atoms of the per-atom force spread
energy - spread of the energy per atom (catches configs whose forces agree but whose energy does not, e.g. another volume)
both - force and energy spreads, each divided by its pool median, summed choices: force, energy, both

`--selection {diverse,greedy}`
How the top-k batch is chosen (default: diverse):
diverse - score x min(1, novelty/–diversity), novelty vs the training set and the picks so far: no near-duplicate picks
greedy - highest score first (only `--min-novelty` filters) choices: diverse, greedy

`--diversity D`
Novelty (typical training spacings) at which a candidate keeps its full score in `--selection` diverse (default: 1). type: float

`--auto-sampling`
First scan several settings of the sampling method (stdev / temperature / strain, `--scan-values`) with a few candidates each, then sample with the one of highest median score that is still physical (few close contacts, stable MD, `--max-force`, `--max-energy`) and whose median energy stays within `--frontier` x the training set's – one step beyond the labelled range, not its edge. switch, off by default

`--scan-values V [V ...]`
Settings for `--auto-sampling` (default: stdev 0.03-0.3 Ang, 300-1500 K, or strain 1-8 %). type: float

`--scan-n SCAN_N`
Candidates per scanned setting (default: 12). type: int

`--target-force MEV_ANG`
Error that is good enough (meV/Ang, as in `stb-mlffValidate`): the report says whether any candidate is still expected above it – if none is, another round on this sampling adds little. type: float

`--n-candidates N_CANDIDATES`
Number of candidate configurations to screen (`--sampling-method` rattle only, default: 50). type: int

`--stdev STDEV [STDEV ...]`
Rattling standard deviation(s) in Angstrom for the candidate batch (`--sampling-method` rattle only, default: 0.15 – typically larger than the amplitude(s) used for the original training set, to probe configuration space not yet covered). type: float

`--md-steps MD_STEPS`
NVT steps per temperature (`--sampling-method` md only, default: 500). type: int

`--md-temperature MD_TEMPERATURE [MD_TEMPERATURE ...]`
NVT temperature(s), K – one trajectory each (`--sampling-method` md only, default: 800 – deliberately higher than a typical production temperature, purely to cover more configuration space per step; not a claim about the material's real operating conditions). type: float

`--stride STRIDE`
Keep one frame every Nth MD step as a candidate (`--sampling-method` md only, default: 10). type: int

`--top-k TOP_K`
Number of candidates to write out (default: 5). type: int

`--min-novelty MIN_NOVELTY`
Skip a candidate whose atomic environments are all closer than this to the training set or to an earlier pick (default: 0.5; 0 disables). Measured on a model-independent fingerprint (sorted distances to the 16 nearest neighbours) in units of the training set's own typical spacing – 1 means as far as two independent training configs are from each other. type: float

`--train-path GLOB`
Folders the novelty filter compares against (default: the training configs in `--workflow`, else every mlff_config_* under <output-dir>, in configs_round_*/ or directly in it – labelled or not). Quote the pattern.

`--max-force EV_ANG`
Skip candidates whose predicted max|F| exceeds this, eV/Ang – geometries too distorted to matter for the intended use (default: 15, the workflow's 'unphysical' line, the same Stage 1 and 2 warn at; 0 = no limit).type:

float

`--d3`

Add DFT-D3 (DFTD3 .true.) to every pick's `config_extra.fdf`. Not needed with a workflow file: the picks follow Stage 1's `--d3` recorded there; for explicit `--model/-s/-c` runs without one. switch, off by default

`--max-energy MEV_ATOM`

Skip candidates whose predicted energy lies more than this above the reference's, meV/atom (committee mean; default: 1000, the workflow's 'unphysical' line; 0 = no limit). The report also gives each energy as a temperature (3/2 kT per atom). type: float

`--frontier FACTOR`

`--auto-sampling`: choose the highest-score setting whose median predicted energy stays within FACTOR x the training set's 90th-percentile energy above the reference (default: 1.5), so each round extends the training range a step instead of jumping to the most distorted physical setting. 0 = off (the highest score among the physical settings). type: float

`--seed SEED`

Base random seed for rattling / straining / initial MD velocities (default: 42). The seed used is `--seed` + 1000 x the round the picks will train, so each round screens new candidates; rattles also get their own offset, so they never repeat Stage 1's displacement directions. type: int

`--device {cpu,cuda}`

Device to run the model(s) on (default: cpu). choices: cpu, cuda

`-o, --output-dir OUTPUT_DIR`

Run folder (default: `--workflow`'s directory, else the current one). With a workflow file the picks go to <output-dir>/configs_round_NN/ – NN = the round they will train, the one after the latest –, else to <output-dir> itself; the mlff_config_NNN numbering continues project-wide.

`--dry-run`

Screen and report (figure and data files too) but write no folder and leave the workflow file untouched – to tune `--stdev/--md-temperature/--supercell` before committing to SIESTA runs. switch, off by default

`--show`

Also display the screening figure on screen (it is always saved). switch, off by default

`--save-data`

Also write the whole pool, one line per candidate (origin, score, spread, max|F|, novelty, shortest distance, picked), to `mlff_al_<tag>_pool.dat`. Off by default. switch, off by default

`--view-picks`

Write the picks to `mlff_al_<tag>_picks.xyz` (per-atom committee spread as 'spread' and as the initial charge, so ASE's viewer colors by it: View > Colors > By initial charge) and open them. switch, off by default

`--save-report`

Also persist the report to `stb_mlffActiveLearning_report.txt` (in the output directory). Off by default.switch, off by default

Like every STB command it also accepts `-h/--help`, `-v/--version` and `--no-intro` (Section 3.3).

From the built-in help.

```
Example usage:
  stb-mlffActiveLearning                   # latest round's committee from mlff_workflow.json
  stb-mlffActiveLearning --dry-run --show   # screen and look, write nothing
  stb-mlffActiveLearning --model m_s1.model m_s2.model m_s3.model \
                         -s structure.fdf -c calc.fdf --n-candidates 50 --stdev 0.15 --top-k 5
  stb-mlffActiveLearning --model m_s1.model m_s2.model m_s3.model --file si.fdf \
                         --sampling-method md --md-temperature 300 600 900 \
```

```
                    --supercell 2 2 2
```

13.2 ML Molecular Dynamics

At a glance

Menu code: 7.2 **Command:** `stb-mlmd`
Guided example: `examples/7.2-mlmd/`
Summary: NVE/NVT/NPT molecular dynamics driven by a MACE potential – the foundation model or your own fine-tuned one, always with its model card – with averages and error bars, ensemble checks, structure and diffusion, and a check of the run against the model's trained range.

13.2.1 Overview

`stb-mlmd` integrates Newton's equations for every atom of a periodic structure with the forces of a MACE potential [15]: the MACE-MP-0 foundation model [16] (`-model`) or a model fine-tuned on your own SIESTA calculations with the MLFF workflow (**7.1**; `-custom-model` with `-custom-model-card`). A force evaluation that costs SIESTA hours for a few hundred atoms takes tens of milliseconds on a GPU, so trajectories of hundreds of atoms over tens of picoseconds become routine — at the level of theory of the training data, as long as the run stays where the model was trained. The report says what the trajectory means and whether to trust it (Fig. 13.1).

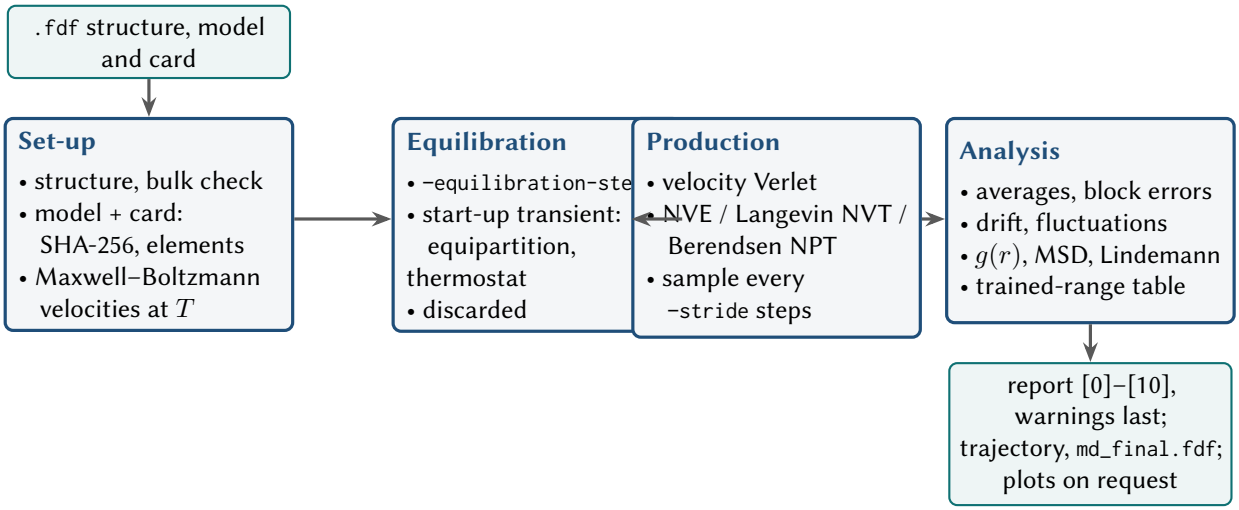


Figure 13.1: What one **`stb-mlmd`** run does. Only the production run enters the statistics; the card is checked before the first step and compared with the trajectory after the last.

Molecular dynamics in seven equations

Integration. Velocity Verlet [94, 95] is time-reversible and symplectic: it conserves a shadow energy $\tilde{E} = E + O(\Delta t^2)$, so the energy error oscillates, with an amplitude $\approx (\omega\Delta t)^2/4$ for a mode ω, instead of drifting; it is unstable for $\omega\Delta t > 2$.

$$\mathbf{v}_i^{n+\frac{1}{2}} = \mathbf{v}_i^{n} + \frac{\Delta t}{2m_i}\mathbf{F}_i^{n}, \quad \mathbf{r}_i^{n+1} = \mathbf{r}_i^{n} + \Delta t\,\mathbf{v}_i^{n+\frac{1}{2}}, \quad \mathbf{v}_i^{n+1} = \mathbf{v}_i^{n+\frac{1}{2}} + \frac{\Delta t}{2m_i}\mathbf{F}_i^{n+1}.$$

Temperature and thermostat. $T = 2K/(N_{\mathrm{dof}}k_B)$; the Langevin thermostat [96] adds friction and a random force tied by the fluctuation–dissipation relation, which makes it sample the canonical ensemble exactly:

$$m_i\dot{\mathbf{v}}_i = \mathbf{F}_i - \gamma m_i\mathbf{v}_i + \mathbf{R}_i(t), \qquad \langle R_{i\alpha}(t)R_{j\beta}(t')\rangle = 2\gamma m_i k_B T\,\delta_{ij}\delta_{\alpha\beta}\delta(t-t').$$

Fluctuations are thermodynamics [97] (canonical ensemble):

$$\frac{\sigma_T}{\langle T\rangle} = \sqrt{\frac{2}{N_{\mathrm{dof}}}}, \qquad C_V = \frac{\langle E^2\rangle - \langle E\rangle^2}{k_B T^2} \quad (3k_B \text{ per atom for a classical harmonic solid}).$$

Error bars from n_b block means [98], because consecutive frames are correlated: $\sigma_{\langle A\rangle} \approx \mathrm{std}(\bar{A}_b)/\sqrt{n_b}$. **Diffusion and melting:**

$$D = \lim_{t\to\infty}\frac{\langle|\mathbf{r}_i(t_0+t) - \mathbf{r}_i(t_0)|^2\rangle}{6t}, \qquad \delta_L = \frac{\sqrt{\langle|\mathbf{u}_i - \langle\mathbf{u}_i\rangle|^2\rangle}}{d_{\mathrm{nn}}} \quad (\gtrsim 0.1\text{–}0.15\text{: melting [99]}).$$

Before you trust an ML molecular-dynamics run

- **Read `[10]` `WARNINGS` and the domain table of `[8]` first.** Outside the trained range a run can be stable, thermostatted and wrong: the silicon model of the example, trained on crystals only, swelled by 27 % at 2500 K and sank 0.23 eV/atom *below* its own crystal (Section 13.2.2.5). Check a frame with SIESTA when anything says `OUTSIDE`.
- **Check the timestep with a short NVE run** (drift well below 1 meV/atom/ps) and equilibrate before production.
- **In NPT give the barostat your material's compressibility**, $1/(B[\mathrm{GPa}]\times 10^4)$ bar^{-1}; the default is water's.
- **The nuclei are classical:** below the Debye temperature C_V, vibration amplitudes and thermal expansion differ from the real material (silicon at 300 K: C_V 2.4, not 3 k_B).
- **A foundation model is uncalibrated** for your level of theory: plausible trajectories with errors you cannot see from inside the run.

13.2.2 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/7.2-mlmd/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

`stb-mlmd` runs **molecular dynamics** (MD) of a periodic structure with a MACE machine-learned potential: either the MACE-MP-0 foundation model, or a model you fine-tuned on your own SIESTA calculations with the MLFF workflow (`7.1`), which always comes with its **model card**. It integrates Newton's equations in one of three ensembles: NVE (constant energy), NVT (Langevin thermostat) or NPT (Berendsen thermostat and barostat). It reports what the trajectory says about the material, and whether to trust it:

- thermodynamic averages with statistical error bars;
- checks that the ensemble was sampled as intended;
- the structure along the run: shortest distance, `g(r)`, coordination, mean-square displacement and diffusion, Lindemann ratio;
- whether the run stayed inside the range of structures the model was trained on.

Plots, gnuplot files and data files are written only when you ask for them.

It answers questions that a static calculation cannot: what a crystal does **at a finite temperature** — how much its atoms vibrate, how it expands, how its heat capacity and pressure compare with the harmonic picture, when it stops behaving like a solid. One SIESTA force calculation of 216 silicon atoms took 1.8 hours on 6 cores in this example; the same step with a fine-tuned MACE model takes 75 ms on a mid-range GPU (RTX 3060). That speed-up is what makes MD of hundreds of atoms over tens of picoseconds possible at the level of theory of your own SIESTA setup. It is also the source of the main risk: the model is only as good as the structures it was trained on, which is why this tool reads the model card and checks the run against it.

The worked example (section 5) runs it for real, on a GPU, for bulk silicon, using the model trained in the `7.1` example's silicon run (shipped in `model/`). The runs:

- a 10 ps reference run at 300 K;
- energy conservation against the timestep;
- the equipartition of energy;
- finite-size effects;
- thermal expansion from 300 to 1500 K, compared with experiment;
- a melt at 2500 K, far outside the training data, judged against a SIESTA calculation of one of its frames;
- the same runs with the raw foundation model, for comparison.

13.2.2.1 1. Theory

Symbols at a glance

Symbol	Meaning
`N`	number of atoms in the cell
`r_i`, `v_i`, `p_i = m_i v_i`, `m_i`	position, velocity, momentum and mass of atom `i`
`U(r_1 . . . r_N)`	potential energy (the model's energy), eV
`F_i = −∂U/∂r_i`	force on atom `i`, eV/Å
`K = Σ p_i²/2m_i`	kinetic energy; `E = K + U` total energy
Δ`t`	MD timestep, fs
`N_dof`	number of degrees of freedom: `3N − 3` once the net momentum is removed
`T = 2K / (N_dof k_B)`	instantaneous (kinetic) temperature, K
`k_B`	Boltzmann constant, 8.617×10^{-5} eV/K
γ	Langevin friction, fs^{-1}
τ`_T`, τ`_P`	Berendsen thermostat and barostat coupling times, fs
β`_T`	isothermal compressibility, 1/bar (`= 1/B`, with `B` the bulk modulus)
`P`, `V`, ρ	pressure (GPa), volume (Å^3), number density (atoms/Å^3)
$\langle$`A`$\rangle$, σ`_A`	time average of a quantity `A` over the production run, and its standard deviation
`C_V`	heat capacity at constant volume, in `k_B` per atom
`g(r)`, `N(r)`	radial distribution function and running coordination number
`MSD(t)`, `D`	mean-square displacement and self-diffusion coefficient
δ`_L`	Lindemann ratio, the RMS vibration amplitude over the nearest-neighbour distance
θ`_D`	Debye temperature (645 K for silicon)

1.1 What molecular dynamics computes. MD follows the motion of every atom by integrating Newton's equations,

$$m_i \, d^2r_i/dt^2 = F_i = -\partial U/\partial r_i \qquad (1)$$

for a potential energy `U` that depends only on the positions of the nuclei. This is the Born–Oppenheimer picture: the electrons are assumed to adjust instantly to the nuclei, and `U` is their ground-state energy at each geometry. In ab initio MD (SIESTA's `MD.TypeOfRun Verlet/Nose/...`), `U` and `F_i` come from a full DFT calculation at every step. Here they come from a machine-learned model of that same energy surface (section 1.2).

A trajectory of `M` steps is a sequence of microscopic states. Statistical mechanics turns it into **thermodynamics**: under the ergodic hypothesis, the time average of a quantity along a long enough trajectory equals its average over the statistical ensemble the dynamics samples [3, 4]:

$$\langle A \rangle_{\text{time}} = \frac{1}{M} \sum_k A(t_k) \quad \to \quad \langle A \rangle_{\text{ensemble}} \qquad \text{(for a long run)} \tag{2}$$

So MD gives what static calculations cannot:

- finite-temperature averages such as the thermal pressure, the thermal expansion and the heat capacity;
- the size of the thermal vibrations;
- anharmonic effects, to every order the potential contains, with no harmonic or quasi-harmonic approximation;
- transport, such as diffusion;
- phase changes, such as melting.

It is also **classical**: the nuclei move as point particles, not wave packets. Section 1.14 explains what that leaves out, which matters for silicon at room temperature.

1.2 Where the forces come from: a MACE potential and its model card. A machine-learned interatomic potential writes the energy as a sum of atomic contributions, each a learned function of the atom's neighbourhood within a cutoff radius `r_max`:

$$U = \sum_i \varepsilon(\text{environment of atom } i \text{ within } r_{\max}) \qquad F_i = -\partial U / \partial r_i \quad \text{(exact derivative)} \tag{3}$$

The forces are the exact derivatives of one energy function, so the dynamics conserves energy exactly as in the continuous equations: a requirement for NVE (section 1.9). MACE [15] builds ε from equivariant message passing:

- each layer passes information between atoms closer than `r_max` (6 Å here);
- two layers give each atom a receptive field of about 12 Å.

The model is **local**, which is why a model trained on 8- and 64-atom cells can drive a 216- or 512-atom cell: every atom's environment in the big cell looks like environments the model has seen, as long as the material is in the same state.

Two kinds of model can be used:

- **MACE-MP-0 foundation models** (`--model small|medium|large`) [16]. They are trained on about 1.6 million PBE/PBE+U structures from the Materials Project, mostly near-equilibrium crystals. They cover 89 elements and need no training of your own. Their accuracy for a given material and level of theory is unknown until it is measured: for the silicon of this example, the medium model's force error is about 450 meV/Å against our SIESTA labels.
- **A model fine-tuned with `7.1`** (`--custom-model` + `--custom-model-card`). It starts from a foundation model and is retrained on your SIESTA single points, so it reproduces *your* level of theory: functional, basis, pseudopotential, mesh and k-sampling. The example's model has a force error of

29 meV/Å on held-out SIESTA configurations, against 517 meV/Å for the raw foundation model on the same configurations.

A fine-tuned model is always handed over with its **model card**, `mlff_model_card.json`, which `stb-mlmd` checks and prints in full in report section `[2]`. It contains:

- the model's SHA-256: the card must belong to this exact file;
- the level of theory of its labels;
- the energy scale (SIESTA's absolute energies, so never mix them with another model's);
- the **trained domain**: elements, volume per atom, shortest interatomic distance, force, energy and pressure ranges of the training set;
- the training recipe;
- every error measured against SIESTA: train, validation, held-out test set, phonons, MD frames;
- an MD stability test and the speed.

During the run the tool compares the trajectory with that domain (report section `[8]`). A structure outside it is not necessarily predicted badly, but nothing was measured there. Section 5.8 shows what happens when the run leaves it: a melt at 2500 K, far from the crystals the model was trained on.

1.3 Integrating the equations: velocity Verlet and the timestep. `stb-mlmd` (through ASE [25]) integrates Eq. 1 with the **velocity Verlet** algorithm [1, 2]:

```
v_i(t + Δt/2) = v_i(t) + (Δt / 2m_i) F_i(t)
r_i(t + Δt)   = r_i(t) + Δt · v_i(t + Δt/2)                                  (4)
v_i(t + Δt)   = v_i(t + Δt/2) + (Δt / 2m_i) F_i(t + Δt)
```

It needs one force evaluation per step, is accurate to second order in Δt, and has two properties that matter more than its order:

- it is **time-reversible**;
- it is **symplectic**: it preserves phase-space volume.

Because of them, Verlet does not conserve the true energy `E` exactly. It conserves a nearby "shadow" energy $\tilde{E}$ `= E + O(`Δt^2`)` [5]. The error of `E` therefore oscillates with an amplitude that grows as Δt^2, but does not accumulate: an integrator without these properties (Runge–Kutta, for example) lets the energy drift away steadily.

Fig. 1 shows it for a single harmonic mode with silicon's highest phonon frequency, 15.5 THz (period 64.5 fs):

- the relative energy oscillation is $(\omega\ \Delta t)^2/4$;
- the integration becomes **unstable** at $\omega\ \Delta t = 2$, about 20 fs here.

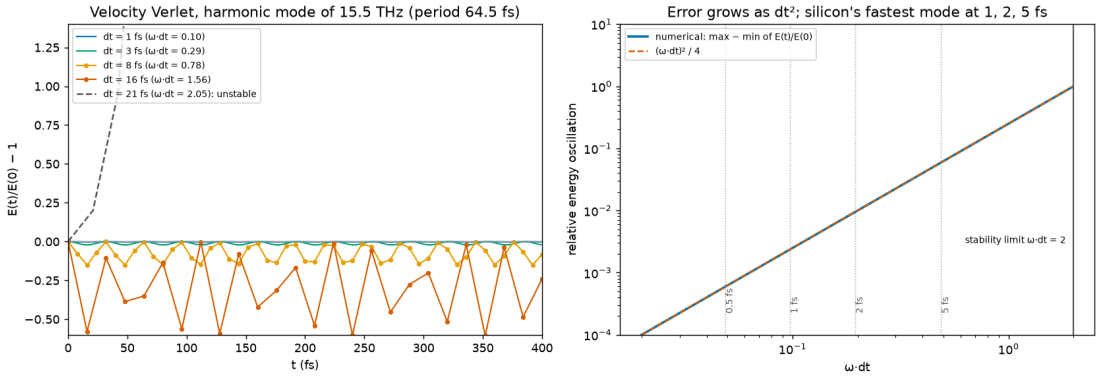
The practical rule is 10–20 steps or more per period of the fastest vibration:

Material	Fastest vibration	Usual timestep
silicon (mass 28)	64.5 fs	1–2 fs is comfortable; 5 fs is near the limit of accuracy (section 5.4)
hydrogen-containing systems	about 10 fs (O–H, C–H stretches)	0.5 fs

In a real run with a machine-learned potential, a slow energy drift can also come from:

- float32 arithmetic (MD runs in float32, the card's recommendation);
- small non-smoothnesses of the model.

The NVE check of section 1.9 measures all of these together.



Velocity Verlet on a harmonic mode of silicon's highest frequency

Fig. 1 — Velocity Verlet on a harmonic oscillator of 15.5 THz (`theory_figures.py`). Left: the energy error is bounded and oscillates; at $\omega\ \Delta t > 2$ (dt = 21 fs) the integration explodes. Right: the amplitude of the energy oscillation follows $(\omega\ \Delta t)^2/4$; the dotted lines mark silicon's fastest mode at 0.5, 1, 2 and 5 fs.

1.4 Temperature, degrees of freedom and equipartition. The temperature of an MD run is the **kinetic temperature**, from the equipartition theorem — every quadratic degree of freedom carries `k_B T / 2` on average:

```
T(t) = 2 K(t) / (N_dof k_B)          N_dof = 3N − 3  when the net momentum is zero          (5)
```

It is an instantaneous quantity that fluctuates (section 1.8): only its average is the thermodynamic temperature.

Degrees of freedom are counted as follows:

- The initial velocities are drawn from a **Maxwell–Boltzmann** distribution at `--temperature`, and the net momentum is removed, so the centre of mass does not drift. That leaves `3N − 3` degrees of freedom; the report prints the number.
- ASE's temperature divides by `3N`. In NVT its Langevin thermostat compensates: the random force is scaled so that the reported temperature matches the target.
- In NVE the reported temperature is `(3N − 3)/3N` of the true one. That is 0.5 % for 216 atoms, but 12.5 % for 8 atoms: one more reason not to run NVE on a tiny cell.

Equipartition at the start of a run. If the atoms start exactly at their lattice sites with velocities drawn at T_0, all the energy is kinetic. For a harmonic solid, equipartition then shares it equally between kinetic and potential energy (the virial theorem: $\langle K \rangle = \langle U - U_{min} \rangle$). Without a thermostat (NVE), the temperature therefore drops to $T_0/2$ within a few vibrational periods (section 5.5). With a thermostat, the system has to absorb that energy from the bath, which takes a few coupling times. Either way the first part of the run is a **transient** that is not the equilibrium state. `--equilibration-steps` runs it and throws it away before the production run, whose statistics are reported.

1.5 Ensembles and thermostats. The ensembles:

Ensemble	Held fixed	Integrator in `stb-mlmd`	Use it for
NVE (microcanonical)	number of atoms, volume, energy	velocity Verlet	checking energy conservation (model + timestep), dynamics with no artificial forces (vibrational spectra, diffusion)
NVT (canonical)	number of atoms, volume, temperature	Langevin [6]	averages and fluctuations at a given temperature in a given cell: heat capacity, thermal pressure, structure
NPT (isothermal–isobaric)	number of atoms, pressure, temperature	Berendsen thermostat + barostat [7]	the equilibrium volume at a temperature and pressure: thermal expansion, density, melting at constant pressure

Langevin thermostat (NVT). Each atom feels a friction and a random force, besides the force from the potential:

$$m_i\, dv_i/dt = F_i - \gamma\, m_i\, v_i + R_i(t) \qquad \langle R_i(t)\, R_j(t')\rangle = 2\,\gamma\, m_i\, k_B\, T\, \delta_{ij}\, \delta(t - t') \tag{6}$$

The fluctuation–dissipation relation on the right fixes the size of the random force so that the dynamics samples the **canonical** distribution exactly, fluctuations included. That is why the report can compare the temperature fluctuation and compute `C_V` in NVT. The friction (`--friction`, default 0.01 fs^{-1}) sets the coupling time $1/\gamma$ = 100 fs:

- the temperature relaxes to its target on that time scale;
- the dynamics is perturbed less the smaller γ is. Diffusion coefficients, for example, are slightly reduced when $1/\gamma$ is comparable to the velocity correlation time.

For transport properties, equilibrate in NVT and measure in NVE.

Berendsen thermostat and barostat (NPT). Berendsen's "weak coupling" rescales the velocities and the cell at every step so that `T` and `P` relax exponentially towards their targets:

$$\lambda_T = [1 + (\Delta t/\tau_T)(T_0/T - 1)]^{1/2} \qquad \text{(velocities} \times \lambda_T\text{)} \tag{7}$$

$$\mu = [1 - \beta_T\, (\Delta t/\tau_P)(P_0 - P)]^{1/3} \qquad \text{(cell vectors and positions} \times \mu\text{)} \tag{8}$$

It reaches the right **averages** quickly and robustly, which is what an equilibrium volume or density needs. It does not sample the right **fluctuations** [7]: it suppresses them. The report says so and does not compute `C_V` or a compressibility from an NPT run.

The compressibility in Eq. 8. It controls how fast the barostat moves the cell. The pressure responds to a volume change as `dP = −B dV/V`, so the pressure relaxes with an effective time constant

$$\tau_{P,\mathrm{eff}} = \tau_P \cdot (\beta_{T,\mathrm{material}} / \beta_{T,\mathrm{input}}) \tag{9}$$

The default `--compressibility` is water's, 4.57 × 10^{-5} bar^{-1}, a generic placeholder:

- Silicon's is 45 times smaller, about 1.0 × 10^{-6} bar^{-1} ($1/(B[\mathrm{GPa}] \times 10^4)$ with $B \approx 98$ GPa).

- With the placeholder, the barostat acts on silicon with τ_P,eff $\approx$ 11 fs instead of 500 fs: a coupling stiffer than the lattice vibrations themselves.
- The report warns when the placeholder is used. Section 5.7 compares both settings.

A thermostat that samples the canonical ensemble with a global rescaling (Bussi's stochastic velocity rescaling [8]), and barostats that sample the isothermal–isobaric ensemble (Martyna–Tobias–Klein [9], Parrinello–Rahman), exist but are not offered by this tool.

1.6 Pressure in MD. The pressure is the trace of the stress tensor, which has two parts:

- a kinetic, ideal-gas part N k_B T / V;
- the **virial** part from the forces, (1/3V) Σ r_i · F_i (computed by the model as the derivative of U with respect to the cell).

$$P = N_{dof}\, k_B\, T / (3V) + (1/3V)\, \langle \Sigma_i\, r_i \cdot F_i \rangle \qquad \text{(periodic form: } -(1/3V)\ \mathrm{tr}(\partial U/\partial \varepsilon)\text{)} \tag{10}$$

stb-mlmd reports the full pressure, kinetic part included. In an NVT run at the static equilibrium volume (SIESTA's relaxed lattice), the pressure is not zero: it is the **thermal pressure**, the pressure the crystal would relieve by expanding. For a solid it is about α_V B T, with α_V the volume thermal-expansion coefficient. Section 1.15 checks the 300 K value by hand.

1.7 Statistical errors: correlated samples and block averaging. Consecutive MD frames are **correlated**: a vibration lasts tens of femtoseconds, a thermostat relaxes in 100 fs. The standard error of a mean of M correlated samples is therefore not $\sigma/\sqrt{M}$ but

$$\sigma_{\langle A \rangle} = \sigma_A / \sqrt{M_{eff}}, \qquad M_{eff} = M / (2\, \tau_{int} / \Delta t_{sample}) \tag{11}$$

where τ_int is the integrated autocorrelation time. **Block averaging** [11] estimates it without computing τ_int:

1. Split the series into n_b blocks, each much longer than τ_int.
2. Average each block: the block means are nearly independent.
3. Take the standard error of the mean of those block means.

$$\sigma_{\langle A \rangle} \approx \mathrm{std(block\ means)} / \sqrt{n_b} \tag{12}$$

stb-mlmd uses n_b = 5 blocks for every ± of the report. With only five blocks the error estimate itself is uncertain by about ±35 %: read it as "the second digit is not reliable", not as an exact figure. Section 5.2 shows how the naive $\sigma/\sqrt{M}$ underestimates the error in the reference run.

Stationarity. The same idea gives a drift test: the report compares the mean of the last third of the run with the first third, with their combined block errors. A difference above three errors means the run was still relaxing — a too-short equilibration, a barostat still moving the cell, or something happening, such as melting.

1.8 Fluctuations are physics. In the canonical ensemble the size of the fluctuations is a thermodynamic property, not noise [3, 10]:

$$\sigma_T / \langle T \rangle = \sqrt{(2 / N_{dof})} \qquad \text{(kinetic temperature, NVT)} \tag{13}$$

$$C_V = (\langle E^2 \rangle - \langle E \rangle^2) / (k_B\, T^2) \qquad \text{(total-energy fluctuations, NVT)} \tag{14}$$

Temperature fluctuation (Eq. 13). For 216 atoms at 300 K, $\sigma_T = 300 \times \sqrt{(2/645)} = 16.7$ K; for 8 atoms it is 93 K. A small cell is not "at" its temperature at any instant. The report compares the measured σ_T with Eq. 13: a ratio near 1 means the thermostat samples the canonical distribution. Berendsen's ratio is well below 1 (section 1.5); in NVE the ratio is not expected to be 1 either, since the total energy is fixed.

Heat capacity (Eq. 14). For a classical harmonic solid it is exactly $3\ k_B$ per atom, the Dulong–Petit value. Anharmonicity adds a small positive correction at high temperature. Eq. 14 converges slowly: several picoseconds for a few percent. The report gives C_V only for NVT runs of 8 atoms or more, in a condensed system.

1.9 Energy conservation: the NVE drift. In NVE the total energy must stay constant up to the bounded Verlet oscillation (section 1.3). `stb-mlmd` fits a straight line to $E(t)$ per atom and reports its slope:

$$\text{drift} = d(E/N)/dt \quad \text{in meV/atom/ps} \qquad (\text{warns above 1 meV/atom/ps}) \tag{15}$$

A drift is caused by one of three things:

- a timestep too long for the fastest vibration;
- forces that are not the exact derivative of the energy (not the case for MACE);
- numerical noise: float32, discontinuities at the cutoff.

For reference, 1 meV/atom/ps is about 8 K/ps of heating: a 10 ps run would end 80 K off if a thermostat did not remove it. The report also gives the ratio of the total-energy fluctuation to the potential-energy fluctuation. A small ratio (10^{-3} at 1 fs for silicon) means the integrator follows the motion closely.

The drift used to be reported relative to the absolute total energy. With SIESTA's energy scale (−24 826 eV for 216 silicon atoms) that was always 0.0000 %, so it could never warn; it is now the slope per atom above.

1.10 Structure: `g(r)`, coordination and the shortest distance. The **radial distribution function** is the density of atoms at a distance r from an average atom, relative to the mean density ρ:

$$g(r) = \frac{1}{4\pi r^2 \rho N} \left\langle \sum_i \sum_{j \neq i} \delta(r - |r_{ij}|) \right\rangle \qquad N(r) = 4\pi\rho \int_0^r g(r')\, r'^2\, dr' \tag{16}$$

Its peaks are the coordination shells; $N(r)$ up to the first minimum is the coordination number:

Silicon phase	`g(r)` features	Coordination
crystal (diamond)	first peak at the bond length (2.373 Å for this level of theory), second shell of 12 at $a/\sqrt{2} \approx 3.88$ Å, a deep minimum between them	4
liquid	first peak near 2.45–2.5 Å, shallow minimum	5–6.5, depending on temperature and measurement [18, 19]: a metallic liquid, denser than the crystal [26]

`stb-mlmd` averages `g(r)` over up to 100 frames, out to $r = 8$ Å or half the shortest cell width.

The **shortest interatomic distance** along the run is the most direct check of the model's domain: two atoms closer than anything in the training set are in a region where the model has never seen a SIESTA force.

1.11 Dynamics: mean-square displacement and diffusion. The mean-square displacement over a time lag `t`, averaged over atoms and over every time origin, is

```
MSD(t) = ⟨ |r_i(t₀ + t) − r_i(t₀)|² ⟩_{i, t₀}                                   (17)
D      = lim_{t→∞} MSD(t) / 6t            (Einstein relation [13], 3D)          (18)
```

Unwrapping. It needs positions that do not jump when an atom crosses the cell boundary. `stb-mlmd` unwraps the saved frames by the minimum-image convention and removes the drift of the centre of mass.

The two phases look different:

- **In a solid** the MSD rises during the first vibrational period and then levels off at a plateau of twice the mean-square vibration amplitude. A slope fitted to that plateau is noise, not diffusion; the report says so when the Lindemann ratio is solid-like.
- **In a liquid** the MSD grows linearly, and `D` is its slope over 6.

A diffusion coefficient from a periodic cell of side `L` is smaller than the infinite-system value by about `2.837 k_B T / (6π η L)` (with η the viscosity) [14]: a few percent for a few hundred atoms. The coefficient needs runs much longer than the time an atom takes to move one bond length.

1.12 The Lindemann ratio and melting. Lindemann's criterion [12]: a crystal melts when the RMS vibration amplitude reaches about 10–15 % of the nearest-neighbour distance. `stb-mlmd` reports

```
δ_L = √⟨|u_i − ⟨u_i⟩|²⟩ / d_nn                                                  (19)
```

It is computed from the unwrapped trajectory, as the RMS displacement about each atom's mean position, divided by the first peak of `g(r)`. A value below about 0.1 is solid-like (atoms vibrating about fixed sites). A value well above it, together with a growing MSD and a coordination that is no longer 4, means the material is liquid or disordered.

Superheating. A perfect periodic crystal has no surface or defect from which melting can start, so MD superheats it well above the melting point before it melts [4]. Heating a crystal until it melts in a small cell gives an upper bound, not the melting temperature; the coexistence method is the proper tool. For orientation:

- the experimental melting point of silicon is 1687 K;
- PBE puts it near 1500 K [21].

1.13 Thermal expansion from NPT runs. At constant pressure the equilibrium volume rises with temperature because the potential is anharmonic. The linear thermal-expansion coefficient is

```
α(T) = (1/a) da/dT          α_V = 3α (cubic)                                    (20)
```

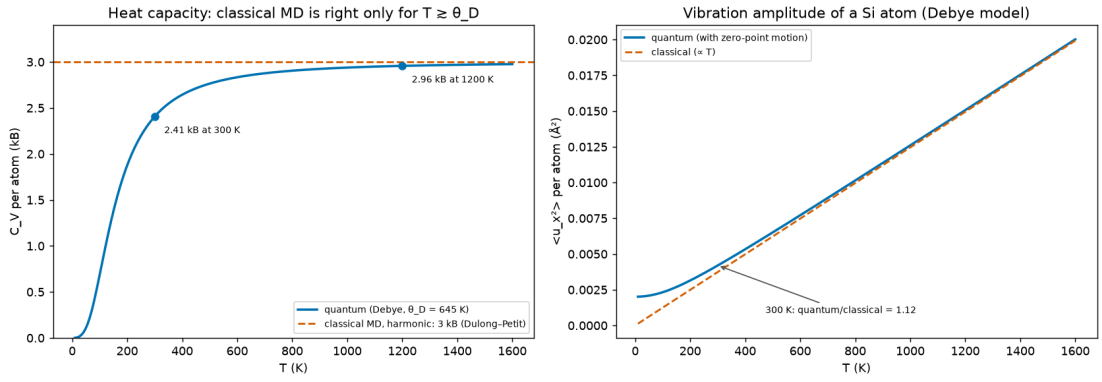
and the fractional change of the lattice constant between two temperatures is $\int \alpha \, dT$. The NPT runs of section 5.7 give the average lattice constant at each temperature. Their differences give α, which is compared with experiment:

- Okada and Tokumaru measured silicon's lattice constant from 300 to 1500 K to a few ppm [17]: $\alpha(T) = \{3.725[1 - \exp(-5.88 \times 10^{-3} (T - 124))] + 5.548 \times 10^{-4} T\} \times 10^{-6}$ K^{-1}.
- Only *differences* are compared: PBE's lattice constant is 0.9 % larger than experiment at every temperature (5.4802 Å for this level of theory, against 5.4307 Å for the real crystal at 273 K [17]).

1.14 What classical nuclei leave out. MD treats the nuclei classically. A vibrational mode of frequency ν is classical only when $h\nu \ll$ k_B T. For silicon the phonons extend to 15.5 THz ($h\nu$/k_B = 745 K) and the Debye temperature is θ_D = 645 K [23]. At room temperature the quantum character of the vibrations is still large (Fig. 2):

- **Heat capacity.**
  - Classical MD gives 3 k_B per atom at every temperature.
  - The quantum Debye value at 300 K is 2.41 k_B; the experimental C_p is 20.0 J mol^{-1} K^{-1} = 2.41 k_B [22].
  - MD's C_V is right only above θ_D.
- **Vibration amplitude.**
  - Quantum mechanics adds zero-point motion. At 300 K the quantum mean-square amplitude is 12 % larger than the classical one; at 100 K it is 87 % larger.
  - The classical Lindemann ratio, and every quantity that depends on the vibration amplitude, is too small at low temperature.
- **Thermal expansion.**
  - Below about 120 K real silicon *contracts* on heating: negative α, from the transverse acoustic modes and their Grüneisen parameters [27]. Classical MD cannot reproduce it.
  - Near and above room temperature the classical expansion is close to the real one (section 5.7), because the modes that dominate α there are nearly classical.

Path-integral MD, or a quantum thermostat [24], adds the quantum nuclear effects; that is outside this tool. Above θ_D (silicon above about 600 K), classical MD is a good approximation for every quantity in this report.



Heat capacity and vibration amplitude of silicon: classical vs quantum

Fig. 2 — What classical MD leaves out, in the Debye model of silicon (θ_D = 645 K, theory_figures.py). Left: heat capacity, quantum (Debye) against the classical 3 k_B. Right: mean-square vibration amplitude along one axis, quantum (with zero-point motion) against classical.

1.15 A calculation you can check by hand. The reference run of section 5.2 (216 Si atoms, NVT at 300 K, the fine-tuned model) reports $\langle$T$\rangle$ = 300.18 K, σ_T = 16.20 K, $\langle$E_kin$\rangle$ = 0.038802 eV/atom, σ(E_tot) = 2.940 meV/atom, $\langle$P$\rangle$ = +0.331 GPa. Each can be predicted on paper:

- **Kinetic energy:**

  - $\langle$E_kin$\rangle$ / N = (3/2) k_B $\langle$T$\rangle$ = 1.5 × 8.6173 × 10^{-5} × 300.18 = 0.038802 eV. It matches to the last digit, because ASE's temperature is defined with 3N (section 1.4).

- **Temperature fluctuation (Eq. 13):**
  - σ_T = 300 × $\sqrt{}$(2/645) = 16.7 K.
  - Measured: 16.2 K (ratio 0.97, within the few-percent uncertainty of a 10 ps run's fluctuation).

- **Heat capacity (Eq. 14):**
  - A harmonic solid has σ(E)/N = k_B T $\sqrt{}$(C_V/(N k_B)), so C_V = 3 k_B gives σ(E)/N = 0.025852 × $\sqrt{}$(3/216) = 3.05 meV/atom.
  - Measured: 2.94 meV, which is C_V = 3 × (2.94/3.05)^2 = 2.79 k_B.

- **Thermal pressure:**
  - The NPT run at 300 K (section 5.7) expands the lattice by +0.137 % relative to SIESTA's static a = 5.48024 Å, so ΔV/V = 0.41 %.
  - With B ≈ 80–90 GPa, holding the cell at the static volume costs P ≈ B ΔV/V = 0.33–0.37 GPa.
  - Measured: +0.331 GPa.

- **Lindemann ratio:**
  - The classical Debye model gives $\langle$u_x$^2\rangle$ = 3$\hbar^2$T/(m k_B θ_D^2) = 0.0037 Å^2 at 300 K, a 3D RMS amplitude of 0.106 Å, so δ_L = 0.106/2.37 = 0.045.
  - Measured: 0.054. The Debye model with the calorimetric θ_D is a crude picture of silicon's phonons, and the model's own phonons are on average 0.3 THz softer than SIESTA's (card: phonon mean shift); both make the measured value larger.

1.16 What this tool simplifies — explicitly.

- **Classical nuclei** (section 1.14): no zero-point motion, classical C_V.
- **One thermostat per ensemble**:
  - Langevin in NVT: canonical, but it perturbs the dynamics;
  - Berendsen in NPT: right averages, wrong fluctuations;
  - no Nosé–Hoover, Bussi or MTK (section 1.5).
- **Isotropic NPT**. ASE's NPTBerendsen scales the whole cell by one factor from the mean pressure: the cell's shape (angles and axis ratios) never changes. Anisotropic expansion and phase transitions that need a change of shape cannot happen.
- **No electronic temperature effects beyond those in the labels.** The model reproduces the SIESTA energy at the electronic temperature of the training data, 300 K here.
- **Statistics from five blocks** (section 1.7): honest error bars, but rough.
- **Domain checks against the card's ranges, not a committee.** The tool runs one model, so it cannot estimate the error frame by frame. 7.1.4 (stb-mlffActiveLearning --sampling-method md) runs the whole committee along an MD and picks the frames worth labelling with SIESTA.

13.2.2.2 2. Libraries and external dependencies used

Library	Used for
ASE [25]	Atoms, the MD drivers (VelocityVerlet, Langevin, NPTBerendsen), Maxwell–Boltzmann velocities, neighbour lists for g(r), trajectory writers (xsf, pdb, extended xyz)

Library	Used for
mace-torch [15, 16]	the MACE calculator, foundation models (downloaded once and cached) and custom `.model` files
PyTorch	runs the model on `cpu` or `cuda` (`--device`)
NumPy	statistics, block averages, linear fits, MSD
matplotlib	the optional dashboard (`--png`, `--show`)
gnuplot (optional)	renders the optional `.gplot` file written by `--save-gnuplot`
`stb.core`	`.fdf` reading (`structure_io`), the model card (`model_card`), the calculator and device checks (`mace_relax`), the PBC unwrap (`md_traj`), library-noise capture (`cli`)

Install the `ml` extra: `pip install stb_suite[ml]`. The first use of a foundation model downloads it (tens of MB) into `~/.cache/mace/`.

13.2.2.3 3. What `stb-mlmd` does

3.1 The run.

1. **Read** the structure (`--file`, `.fdf`), determine its dimensionality (vacuum-padded axes, from `--vacuum-gap`) and whether it is a condensed system or a molecule/slab in a box (any empty gap of 5 Å or more, the model card's criterion). With `--supercell NA NB NC` the cell is then repeated NA × NB × NC times (`ase.Atoms.repeat`: the same cell and atom order as `stb-supercell -d NA NB NC`); everything after this step — degrees of freedom, statistics, `g(r)`, the domain table, the trajectory and `md_final.fdf` — refers to the supercell.
2. **Load the model**: a foundation model (`--model`) or a custom model with its card (`--custom-model` + `--custom-model-card`). The card is checked: it must exist, be readable, and its SHA-256 must match the model file. Then the elements: the structure may contain only elements the model was trained on, otherwise it stops with a clean error. The model runs in float32 for MD (the card's recommendation); a foundation model gets no card checks.
3. **Draw the initial velocities**: Maxwell–Boltzmann at `--temperature`, net momentum removed, reproducible with `--seed`.
4. **Equilibrate** for `--equilibration-steps` (discarded), then run the **production** steps (`--n-steps`). Every `--stride` steps the tool samples:
   - temperature, potential/kinetic/total energy, pressure (with the ideal-gas term), volume, cell lengths;
   - shortest distance, largest force;
   - a trajectory frame.
5. **Analyse** the samples and write the report, the trajectory, the final structure (`md_final.fdf`, wrapped into the cell) and, on request, the dashboard, data and gnuplot files.

Output from the libraries (mace's dtype notices, torch's deprecation warnings, ASE's messages) is captured during loading and integration: it is counted in the last section and never mixed into the report.

3.2 The report, section by section.

Section	Content
`[0] RUN METADATA`	date, software versions (stb, mace-torch, torch, ase), device (GPU name), input, supercell, ensemble, temperature, timestep and steps, model
`[1] STRUCTURE`	atoms (and, with `--supercell`, input → supercell atom count), formula, dimensionality, cell, volume per atom and density, shortest and median nearest-neighbour distance, degrees of freedom; a note when the cell is a molecule/slab in a box

Section	Content
`[2] MODEL`	the **model card**, printed in full (next table), the elements check, load time and dtype; or the foundation model's identity
`[3] MD SETUP`	integrator and its parameters (friction / coupling times / compressibility and the bulk modulus it implies), initial velocities and seed, length in steps and picoseconds, sampling
`[4] EQUILIBRATION`	energy and temperature at the end of the discarded part
`[5] PRODUCTION RUN`	a progress table every 10 % of the run (T, `E_pot`/atom, P, V/atom, max force), initial and final state, number of frames, **speed** (ms/step, μs/atom/step, ns/day)
`[6] THERMODYNAMICS`	table of means with block-average errors, standard deviations and ranges (T, `E_pot`, `E_kin`, `E_tot`, P, and in NPT V and density); NPT: mean volume and lattice parameters vs the input; NVT: heat capacity from Eq. 14
`[7] ENSEMBLE CHECKS`	NVE: drift (Eq. 15), total- vs potential-energy fluctuation, start→end; NVT/NPT: $\langle$T$\rangle$ vs target, σ_T vs Eq. 13, $\langle$P$\rangle$ vs target; stationarity (last vs first third) of T, `E_pot` and, in NPT, the volume
`[8] STRUCTURE ALONG THE RUN & MODEL DOMAIN`	shortest distance (minimum, when, mean), `g(r)` first peak/minimum and coordination, MSD and `D`, RMS displacements, Lindemann ratio with its reading; with a card, the **domain table**: each trained range against this run (`inside`, `inside (within the tolerance)`, `OUTSIDE`, or `larger` for a bigger cell, which is fine for a local model). A potential energy *below* the lowest training energy (by more than 20 meV/atom) is flagged too: the model calls those structures more stable than anything in its SIESTA data, the signature of a hole in the learned surface (section 5.8)
`[9] SUMMARY & FILES`	status, the headline numbers, time, every file written, and what else could have been written
`[10] WARNINGS`	every warning of the run in one place (setup, domain, model card, stationarity, drift), numbered; then the count of distinct library messages captured

The model card in `[2]` is printed in five blocks:

Block	Content
Identity	card file and format, model name and file, SHA-256 verified against the card, committee membership and spread, architecture (MACE, `r_max`, layers, parameters, foundation it was fine-tuned from)
Level of theory	code and version, every SIESTA setting of the labels, whether DFT-D3 is in them, pseudopotentials (md5), energy scale and E0s
Training data and trained domain	number and kinds of configurations, labels, and the trained-range table
Recipe and accuracy	epochs, learning rate, loss weights; errors vs SIESTA for train, validation, out-of-sample committee, held-out test set, and the raw foundation model; fine-tuning gain; per-kind test errors; phonons; MD-check frames; committee calibration; the `7.1.3` verdict
Stability, speed and usage	the `7.1.2` MD stability test, speed, recommended dtypes, usage notes, software it was trained with

The card's own alarms go to `[10] WARNINGS`:

- no validation;
- a verdict other than STOP;
- a failed stability test;
- training warnings about this model.

So do the domain warnings raised while the model runs (`model_card.collect_warnings`).

3.3 Plots and data, on request. Nothing but the report, the trajectory and `md_final.fdf` is written by default. On request:

Flag	Writes
`--png [FILE]`	`<stem>_md_dashboard.png`: six panels: T(t), energies per atom, P(t), volume (NPT) or shortest distance (NVT/NVE), `g(r)` with `N(r)`, MSD
`--show`	the same dashboard on screen (needs a display; without one it is a warning, not an error)
`--save-data`	`<stem>_md_thermo.dat` (12 columns per sampled frame: t, T, `E_pot`, `E_kin`, `E_tot` per atom, P, V/atom, a, b, c, `d_min`, `F_max`), `<stem>_md_rdf.dat` (r, g, N), `<stem>_md_msd.dat` (lag, MSD)
`--save-gnuplot`	the data files plus `<stem>_md.gplot`: `gnuplot <stem>_md.gplot` draws the same six panels into `<stem>_md.pdf`
`--save-report`	the report as `stb_mlmd_report.txt`

`<stem>` is the input file name without `.fdf`.

3.4 Guard rails.

Situation	Behaviour
`--custom-model` without `--custom-model-card`, a missing/unreadable card, or a card whose SHA-256 does not match the file	clean error before anything runs
an element of the structure is not in the card	clean error naming the missing elements
`--supercell` with a value below 1	clean error before anything runs
`--supercell` repeating a vacuum-padded axis	warning: the vacuum grows with it and adds only empty space
`--ensemble npt` on a structure with a vacuum-padded axis	refused: a barostat on a slab squeezes the vacuum
a molecule or slab in a box (gap $\geq$ 5 Å, below `--vacuum-gap`)	runs, but the bulk-only analyses (density, pressure, `g(r)`, `C_V`, Lindemann) are skipped with a note; NPT warns
`--compressibility` left at the placeholder in NPT	warning (section 1.5)
`--timestep` > 2 fs	warning: check the NVE drift
no `--equilibration-steps` in NVT/NPT	warning: the start-up transient is inside the averages
`--device cuda` with no usable GPU	clean error (`core.mace_relax.resolve_device`)
`--show` without a display	warning; the run is not affected

13.2.2.4 4. Running it both ways

Command line:

```
stb-mlmd --file si8.fdf --supercell 3 3 3 \
         --custom-model model/mlff_model_compiled.model --custom-model-card
         model/mlff_model_card.json \
         --ensemble nvt --temperature 300 --equilibration-steps 1000 --n-steps 10000 \
         --device cuda --seed 1 --png --save-gnuplot --save-report
```

Interactive menu: run `stb-suite` and type `7.2`. It asks, in this order:

1. the structure;
2. a custom model path, or Enter for a foundation model and then its size. With a custom model it asks for the card and offers `mlff_model_card.json` next to the model as the default;
3. the supercell `NA NB NC` (Enter keeps the input cell, `1 1 1`);

4. the ensemble;
5. the temperature, and the pressure for NPT;
6. the equilibration steps, production steps and timestep;
7. optionally the advanced settings (coupling times, compressibility, device);
8. the trajectory name;
9. one question per optional output (PNG, show, gnuplot, data only, report).

The menu builds the same command line and calls `stb-mlmd`; the guided script (Case 9) drives it with piped answers.

13.2.2.5 5. Worked example: bulk silicon on a GPU

5.1 Set-up. The structure:

- `si8.fdf`, the 8-atom conventional diamond cell at SIESTA's relaxed lattice constant (5.48024 Å);
- `stb-supercell -d N N N` makes the 64-, 216- and 512-atom cells (`make_reference.sh`). `stb-mlmd --file si8.fdf --supercell N N N` builds the same cells on the fly (Case 3 of the guided script); the only difference is the output stem, `si8_md_*` instead of `si216_md_*`.

The model, in `model/`:

- `best_model/` of the `7.1` silicon run: round 2, seed 125;
- fine-tuned from MACE-MP-0 medium on 57 SIESTA single points (8 and 64 atoms: rattled, strained, and 1400 K MD frames);
- level of theory: PBE, DZP, PAO.EnergyShift 0.01 Ry, Mesh.Cutoff 300 Ry, k × a ≈ 32 Å;
- its card reports, against SIESTA:

Error measure	Value
test-set force RMSE	28.7 meV/Å (1.5 % of the RMS force)
test-set energy error	1.1 meV/atom
stress error	0.18 GPa
phonons	1.10 THz RMSE
64-atom MD frames at 800 K	30–39 meV/Å

The runs:

- every run uses `--device cuda` on one NVIDIA RTX 3060, float32, `--seed` fixed;
- NVT and NVE use the defaults (1 fs, friction 0.01 fs^{-1});
- NPT uses `--compressibility 1.0e-6` unless stated;
- `make_reference.sh` lists every command. Section 6 explains how to regenerate them.

Speed (runs alone on the GPU):

Atoms	ms/step	μs/atom/step	ns/day at 1 fs
8	20.4	2556	4.2
64	27.2	425	3.2
216	74	343	1.17
512	164	321	0.53

- Small cells are dominated by fixed overheads; the cost per atom settles near 320 μs from a few hundred atoms on.
- NPT costs about twice as much per step: the stress is computed at every step.

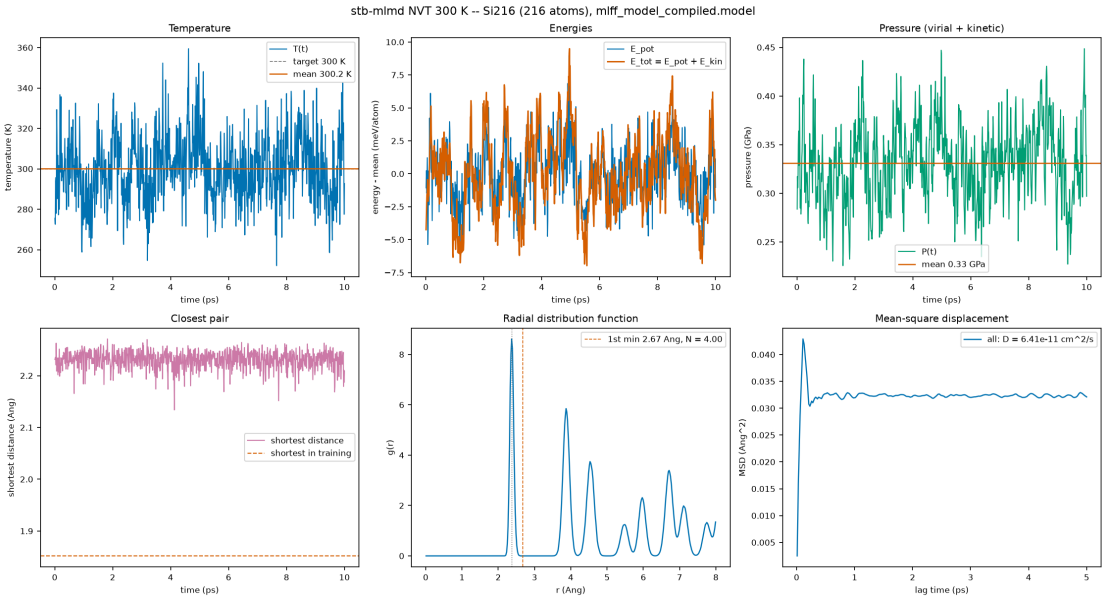
- The 216-atom NVT reports in `reference_results/` show about 150 ms/step: those runs shared the GPU with a second run, which halves the speed of each (and gains nothing overall).

5.2 The reference run: 216 atoms, NVT at 300 K, 10 ps.

```
stb-mlmd --file si216.fdf --custom-model model/mlff_model_compiled.model --custom-model-card
model/mlff_model_card.json \
        --ensemble nvt --temperature 300 --equilibration-steps 1000 --n-steps 10000 --device cuda
        --seed 1 \
        --png --save-gnuplot --save-report
```

Report sections `[6]`–`[8]`, abridged (`reference_results/nvt300/stb_mlmd_report.txt`):

```
Temperature (K) | 300.18 +/- 1.31          | 16.20        | 252.1 .. 359.4
E_pot (eV/atom) | -114.976395 +/- 0.000109 | 2.175 meV    | -114.98225 .. -114.96742
E_kin (eV/atom) | 0.038802 +/- 0.000169    | 2.094 meV    | 0.03259 .. 0.04646
E_tot (eV/atom) | -114.937594 +/- 0.000275 | 2.940 meV    | -114.94457 .. -114.92810
Pressure (GPa)  | 0.331 +/- 0.005          | 0.039        | 0.23 .. 0.45
Heat capacity     : 2.79 kB/atom from the total-energy fluctuations (classical harmonic solid: 3, ...)
T fluctuation     : 16.20 K vs canonical T sqrt(2/N_dof) = 16.71 K (ratio 0.97)
Shortest distance : min 2.1339 Ang at t = 4.130 ps, mean 2.2303 Ang (start 2.3730)
g(r)              : first peak 2.370 Ang (g = 8.61), first minimum 2.670 Ang, coordination 4.00
Lindemann ratio   : 0.054 (RMS about the mean / nearest-neighbour distance 2.370 Ang) -- solid-like
volume per atom (Ang^3) | 18.722 .. 22.425       | 20.574 .. 20.574       | inside
shortest distance (Ang) | >= 1.852               | min 2.134              | inside
max force (eV/Ang)      | <= 14.70               | max 4.10               | inside
[10] WARNINGS: None.
```



Dashboard of the reference run

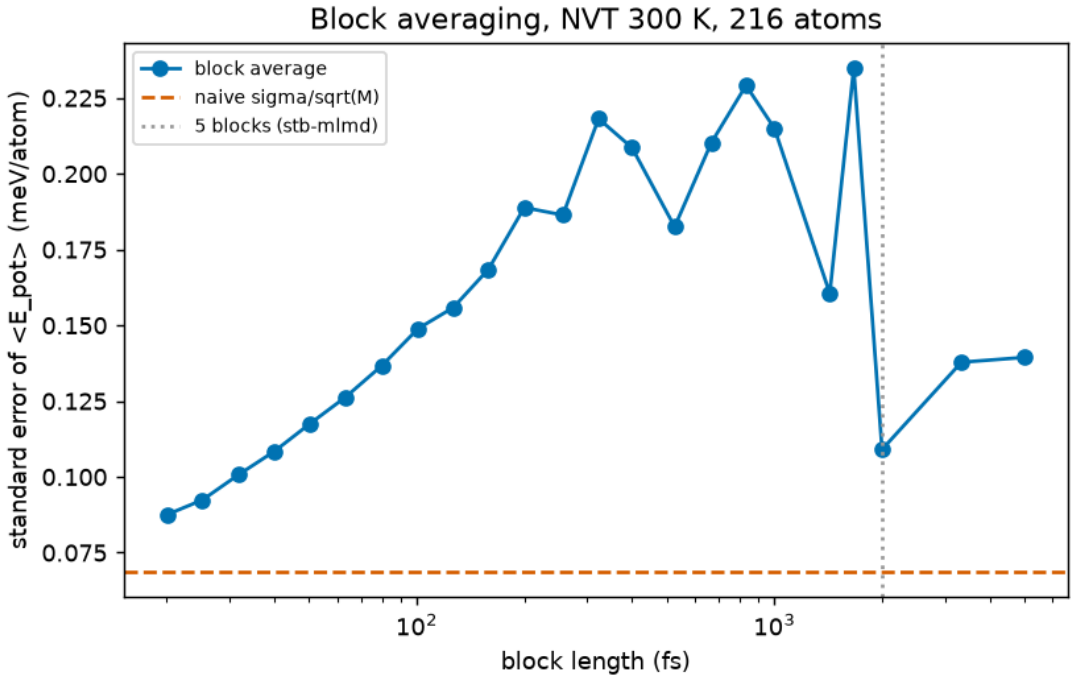
Fig. 3 — The dashboard of the reference run (`--png`). The `g(r)` is the diamond lattice broadened by thermal motion. The MSD jumps within the first vibrational period and then stays on a plateau of 0.032 Å², twice the mean-square vibration amplitude: atoms vibrate, nothing diffuses.

How to read it, number by number (section 1.15 does the arithmetic):

- **Temperature.** ⟨T⟩ = 300.2 ± 1.3 K, and the fluctuation 16.2 K is 0.97 of the canonical value: Langevin samples the canonical ensemble.
- **Heat capacity.** C_V = 2.79 k_B per atom.
  - The classical harmonic value is 3. From a 10 ps run, the variance in Eq. 14 is uncertain by about 20 % (with a correlation time near $1/\gamma$ = 100 fs, 10 ps hold about 50 independent samples), so 2.79 is consistent with 3.
  - Real silicon at 300 K has 2.41 (section 1.14): this is the classical answer, not the real one.
- **Pressure.** +0.33 GPa at the static lattice: the thermal pressure (sections 1.6 and 5.7).
- **Structure.** Four neighbours at 2.37 Å; the Lindemann ratio of 0.054 is far from melting. The shortest distance reached, 2.13 Å, is well above the shortest in the training set (1.85 Å). Every quantity of the domain table is inside; the cell (216 atoms) is larger, which is fine for a local model.

Error bars. The ± comes from five blocks of 2 ps (Eq. 12). Fig. 4 repeats the block analysis of E_pot for every block length:

- the naive $\sigma/\sqrt{M}$ = 0.069 meV/atom is three times too small;
- the error estimate rises with the block length and levels off near 0.2 meV/atom from blocks of about 300 fs on;
- the five-block value printed in the report, 0.11, falls below that plateau. It is a scatter of the expected size for an estimate from five blocks (section 1.7), but a reminder to read the ± of a short run as indicative only. Doubling it is a safe habit.



Block averaging of the potential energy

Fig. 4 — Standard error of ⟨E_pot⟩ against block length (results_figures.py). Short blocks are correlated and underestimate the error; the plateau (~0.2 meV/atom) is the honest value; the five-block estimate of the report (dotted line) is noisy.

5.3 Is the model right here? SIESTA on MD frames. Inside the MD, nothing compares the model with SIESTA. To check, the final frame of a run was taken (`md_final.fdf`, 216 atoms) and a SIESTA single point was run on it at the level of theory of the training data (`siesta_calc.fdf` + `siesta_config_extra.fdf`: the reference's k-grid divided by the 3×3×3 supercell, 2×2×2). Then both models were evaluated on the same structure (`siesta_check.py`, float64; `reference_results/siesta_check/`).

The energy error ΔE is model minus SIESTA per atom, after removing each model's constant offset measured on the 300 K frame (only energy differences are meaningful). Each SIESTA single point took 1.8 hours on 6 cores; the melt frame needed 59 SCF iterations (about 8 hours) instead of 14.

Frame	SIESTA RMS force	Fine-tuned: F RMSE (max)	Fine-tuned: ΔE	MACE-MP medium: F RMSE (max)	MACE-MP medium: ΔE
crystal, 300 K (`nvt300`)	576	**25.1** (90)	0 (reference)	242.0 (660)	0 (reference)
crystal, 1500 K (`npt_T1500`)	1224	**70.9** (259)	−1.5	405.5 (1256)	−52.8
melt, fine-tuned model (`melt_ft`)	3377	**3663** (8628)	**−1639**	518.6 (2496)	−301

Forces in meV/Å, energies in meV/atom.

300 K: the model is SIESTA, to within 4 %.

- Force RMSE 25 meV/Å, 4.4 % of the RMS force, close to the card's 29 meV/Å on its held-out test set.
- The model's absolute energy agrees with SIESTA's to 2 meV/atom: the E0s were fitted to SIESTA's scale.
- The foundation model is 10 times worse on the same frame.

1500 K: still good, a little worse.

- 71 meV/Å (5.8 % of the larger forces), against 30–39 meV/Å on the card's 800 K frames. The superheated crystal vibrates beyond anything in the training set (its 1400 K frames were 64-atom cells).
- The energy difference between the 300 and 1500 K frames is right to 1.5 meV/atom; the foundation model misses it by 53 meV/atom.
- Errors grow smoothly as the run moves away from the training data, and the domain table still said `inside`.

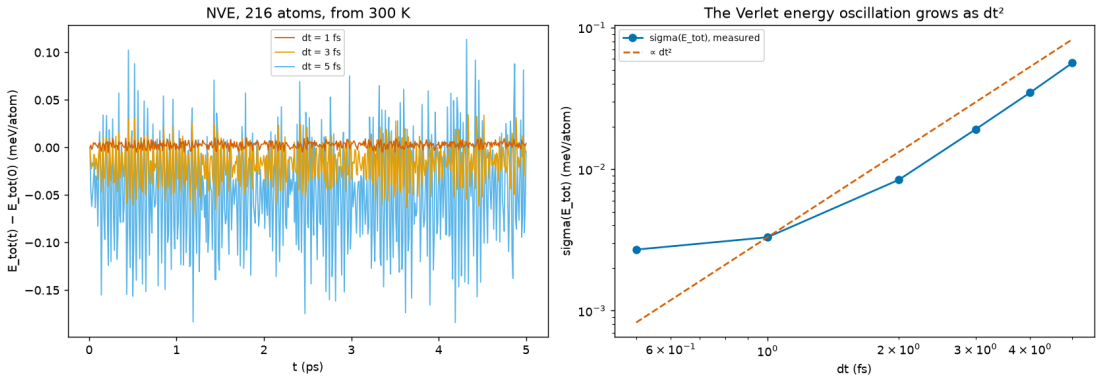
The melt: the model is meaningless there.

- The force error is 3.7 eV/Å, larger than the forces themselves. Its worst atom is off by 8.6 eV/Å.
- The energy is 1.64 eV/atom too low. By SIESTA, this frame lies 1.41 eV/atom *above* the 300 K crystal (−113.57 against −114.98 eV/atom); the model put it 0.23 eV/atom *below*.
- That is the hole of section 5.8, measured.
- The raw foundation model, never trained on this SIESTA setup, does far better on the same frame (519 meV/Å): broad training data is what protects a model outside a narrow domain.

The lesson of the table. A fine-tuned model is excellent where it was trained, degrades gradually at the edges, and fails without limit beyond them. The domain table and the warnings of section 5.8 were the only indication in the MD output. The single point here is the proof, and it is the check to run before believing any MD that leaves the trained range.

5.4 Energy conservation and the timestep. The structure at the end of the reference run, with fresh Maxwell–Boltzmann velocities at 300 K, was run in NVE for 5 ps at each timestep (`reference_results/nve_dt*`):

dt (fs)	drift (meV/atom/ps)	σ(E_tot) (meV/atom)	σ(E_tot)/σ(E_pot)	$\langle$T$\rangle$ (K)
0.5	+0.0001	0.0027	0.002	297.2
1	+0.0002	0.0033	0.002	297.1
2	+0.0003	0.0084	0.006	296.6
3	+0.0007	0.0191	0.012	295.8
4	+0.0011	0.0347	0.021	294.6
5	+0.0011	0.0562	0.034	293.1



NVE energy conservation against the timestep

Fig. 5 — Left: total energy along the NVE runs at 1, 3 and 5 fs. Right: its fluctuation against the timestep, with a $\mathtt{dt}^2$ *line through the 1 fs point.*

- **No drift at any timestep.** All drifts are a thousand times below the 1 meV/atom/ps warning. The model is a smooth, conservative force field, and silicon (mass 28, fastest mode 64.5 fs) is gentle on the integrator: even 5 fs is 13 steps per period.
- **The oscillation grows as about $\mathtt{dt}^2$** (Fig. 5), as the shadow-energy picture predicts (section 1.3):
  - from 1 to 5 fs it grows 17 times, against 25 for a pure $\mathtt{dt}^2$;
  - **below 1 fs it stops shrinking**: 0.0027 at 0.5 fs. That is the float32 floor: the rounding of a sum of 216 atomic energies of −115 eV each.
- **The temperature drifts with `dt`.** It is 4 K lower at 5 fs than at 1 fs from the same initial conditions. The velocities of a Verlet trajectory carry an $\mathtt{O(dt}^2\mathtt{)}$ error, so the kinetic energy it reports does too.

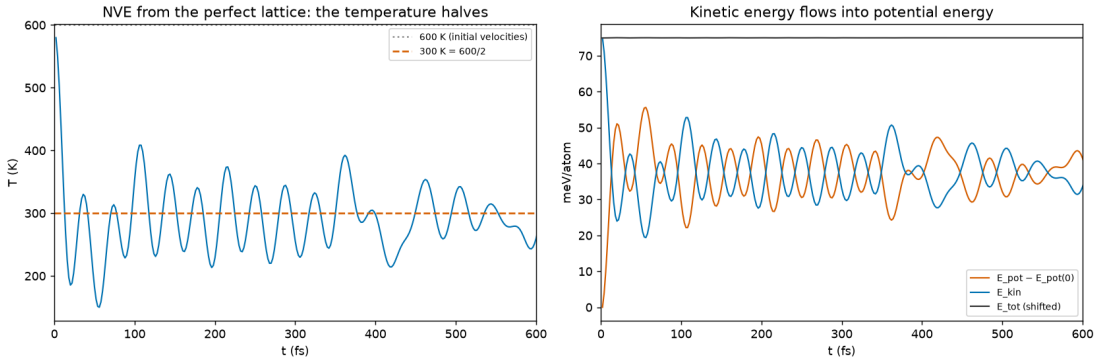
Practical reading:

- 1–2 fs is safe for silicon;
- 3 fs costs 1 % in temperature and is still clean;
- for light atoms, the same analysis would put the limit at 0.5–1 fs.

One short NVE run like this checks a new model or material before any long production.

5.5 Equipartition: the start-up transient. The perfect 216-atom crystal, velocities drawn at 600 K (the actual draw was 590 K), NVE for 2 ps (`reference_results/nve_equipartition`):

t (fs)	0	10	20	40	60	100	200	500	1000
T (K)	590	385	185	314	175	354	228	331	326



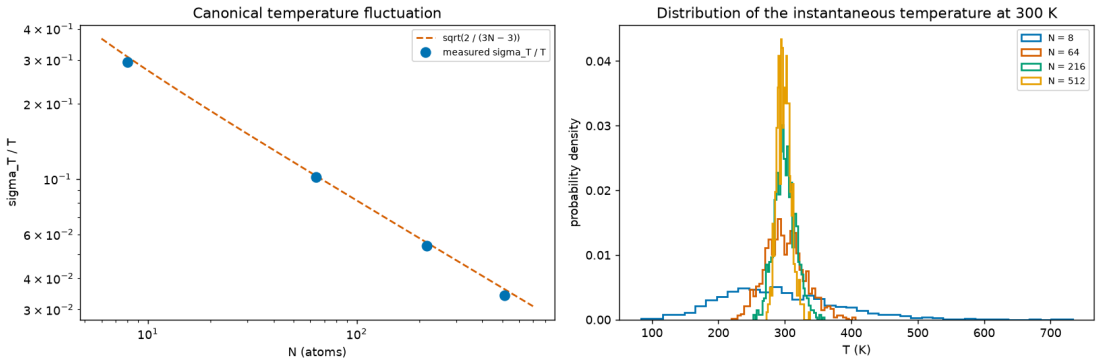
Equipartition from the perfect lattice

Fig. 6 — NVE from the perfect lattice. Left: the temperature falls from 590 K to about half within 20 fs and then oscillates about 300 K. Right: kinetic energy flows into potential energy and back, while the total stays constant.

- **The halving.** All the energy starts as kinetic. Within a quarter of a vibrational period (20 fs) half of it has become potential energy. The mean over the second half of the run is 291.7 K, close to 590/2 = 295 K. The small difference is anharmonicity: the potential energy rises faster than a harmonic one.
- **The oscillation** has a period of about 36 fs. Kinetic energy oscillates at twice the frequency of the mode, so this is a mode of about 14 THz: silicon's optical phonons, which dominate the vibrational density of states. Every atom starts at its lattice site, so the modes start in phase; anharmonic coupling dephases them over a few hundred femtoseconds.
- **The lesson.** The first few hundred femtoseconds of any run that starts from a perfect lattice are not equilibrium. With a thermostat the energy deficit is refilled over a few coupling times. That is what `--equilibration-steps` discards, and why the report warns when there is none.

5.6 Finite size. NVT at 300 K, the same model, cells of 8 to 512 atoms (`reference_results/size_N*`, `nvt300`):

N	⟨T⟩ (K)	σ_T (K)	√(2/N_dof)·T (K)	ratio	C_V (k_B)	⟨P⟩ (GPa)	σ_P (GPa)	Lindemann
8	290.2 ± 5.4	88.2	92.6	0.95	3.56	+0.197 ± 0.016	0.239	0.047
64	300.3 ± 2.8	30.4	30.9	0.99	2.70	+0.330 ± 0.004	0.070	0.052
216	300.2 ± 1.3	16.2	16.7	0.97	2.79	+0.331 ± 0.005	0.039	0.054
512	298.4 ± 1.5	10.2	10.8	0.95	2.66	+0.328 ± 0.003	0.023	0.054



Temperature fluctuations against system size

Fig. 7 — Left: relative temperature fluctuation against the number of atoms, with Eq. 13. Right: distribution of the instantaneous temperature for each size.

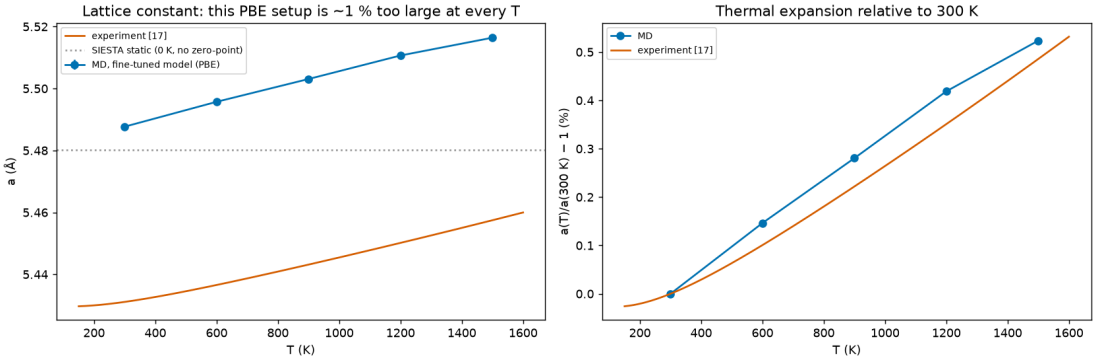
- **Fluctuations shrink as `1/√N`, exactly as Eq. 13 says.**
  - At 8 atoms the instantaneous temperature spans 80–730 K; at 512 atoms ±10 K.
  - The pressure fluctuates in the same way: 0.24 GPa for 8 atoms, 0.023 for 512.
  - A small cell is not wrong, but every average needs `N` times more steps for the same error bar.
- **Averages converge by 64 atoms.**
  - The 8-atom cell has a lower thermal pressure (0.20 against 0.33 GPa) and smaller vibrations (Lindemann 0.047 against 0.054). Its periodicity only admits phonons whose wavelength fits the 5.5 Å cell (Γ and X); the long-wavelength acoustic modes, which carry the largest amplitudes, are missing.
  - From 64 atoms on, pressure and Lindemann ratio agree within their errors.
- **Heat capacity.**
  - It scatters between 2.66 and 3.56. That is the slow convergence of Eq. 14, not physics: 5–20 ps are not enough for a variance to better than about 20 %.
  - For `C_V`, run long, or take `dE/dT` from runs at several temperatures.
- **Short-run warnings.** The 64- and 512-atom runs end with stationarity warnings (the last third 12.5 K and 2.7 K warmer than the first). With 5 ps and five blocks the error bars are optimistic (section 5.2), and the warning errs on the side of caution.

5.7 Thermal expansion (NPT) and the barostat. NPT at 0 GPa, 216 atoms, `--compressibility 1.0e-6` (silicon's), 2 ps of equilibration and 6 ps of production at each temperature (`reference_results/npt_T*`). The lattice constant is the mean cell length over 3:

T (K)	a, MD (Å)	a, experiment [17] (Å)	Δa/a from 300 K, MD	Δa/a from 300 K, experiment	Lindemann	volume stationary?
300	5.48769 ± 0.00005	5.43110	—	—	0.052	yes
600	5.49572 ± 0.00021	5.43656	+0.146 %	+0.101 %	0.075	yes
900	5.50308 ± 0.00025	5.44315	+0.280 %	+0.222 %	0.096	yes
1200	5.51067 ± 0.00036	5.45016	+0.419 %	+0.351 %	0.113	drifting

T (K)	a, MD (Å)	a, experiment [17] (Å)	Δa/a from 300 K, MD	Δa/a from 300 K, experiment	Lindemann	volume stationary?
1500	5.51642 ± 0.00032	5.45747	+0.524 %	+0.486 %	0.134	drifting

Interval (K)	α, MD (10^{-6} K^{-1})	α, experiment at the midpoint (10^{-6} K^{-1})
300–600	4.87	3.43
600–900	4.46	4.05
900–1200	4.59	4.29
1200–1500	3.48	4.47



Thermal expansion of silicon: MD vs experiment

Fig. 8 — Left: lattice constant from NPT MD and from experiment [17]; the dotted line is SIESTA's static (0 K) value. Right: expansion relative to 300 K.

The absolute lattice constant is about 1 % above experiment at every temperature: 5.4877 against 5.4311 Å at 300 K. That is the level of theory, which the model reproduces: PBE with this DZP basis relaxes silicon to 5.4802 Å (section 1.13). No MD can fix it; a better functional or basis would.

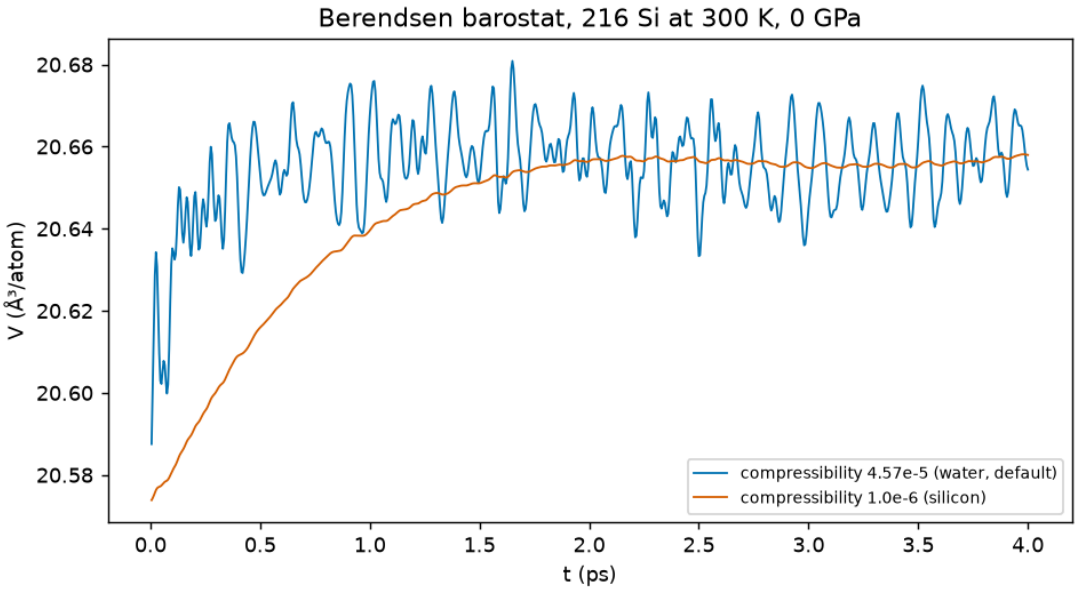
The expansion is what MD adds, and it is close to experiment:

- 300 → 1200 K: +0.42 % against +0.35 %.
- From 300 to 600 K, MD's α is 40 % too large. This is the expected signature of classical nuclei (section 1.14): at those temperatures the real crystal's modes are not fully excited (`C_V` = 2.4–2.8 instead of 3 `k_B`), so they expand it less.
- From 600 to 1200 K, where the modes are nearly classical, the two agree within 10 %.
- The 1200 and 1500 K volumes were still drifting at the end of the run (stationarity warning), so the last interval is not reliable. Near 1500 K the crystal is also superheated: Lindemann 0.134, above PBE's melting point (about 1500 K [21]).

Thermal pressure, revisited. The 300 K lattice is 0.137 % larger than the static one (`ΔV/V = 0.41 %`). The NVT run of section 5.2, held at the static lattice, felt +0.331 GPa. The ratio, `0.331/0.0041` ≈ `80 GPa`, is the crystal's isothermal bulk modulus at 300 K in this model: a little below the static value of this level of theory (about 90 GPa, `5.15` example). It closes the loop between the NVT and NPT runs.

The barostat's compressibility. Two NPT runs at 300 K start from the static cell, 4 ps each (`reference_results/npt_T300_waterbeta`, `npt_T300_sibeta`):

`--compressibility` (bar^{-1})	B the barostat assumes	⟨V⟩ after 1 ps (Å^3/atom)	σ_V (Å^3/atom)	σ_P (GPa)
4.57 × 10^{-5} (water, the default)	2.2 GPa	20.6575	0.0087	0.034
1.0 × 10^{-6} (silicon)	100 GPa	20.6543	0.0039	0.087



Volume under the Berendsen barostat with two compressibilities

Fig. 9 — Volume against time. With water's compressibility the cell jumps to its new volume within 0.2 ps and then oscillates; with silicon's it relaxes smoothly over about 1.5 ps (τ_P = 500 fs) and stays put.

The mean volume is the same either way (0.015 % apart): the placeholder does not bias the average. What it does (section 1.5) is make the barostat respond as if silicon were 45 times softer:

- the cell overreacts to every pressure fluctuation;
- the volume fluctuates twice as much and the pressure half as much, both unphysical;
- the volume's time series carries the barostat's own oscillation.

With the right compressibility the cell relaxes on the time scale you chose with `--taup`. That takes longer: equilibrate for at least four τ_P. The report warns about the placeholder for that reason.

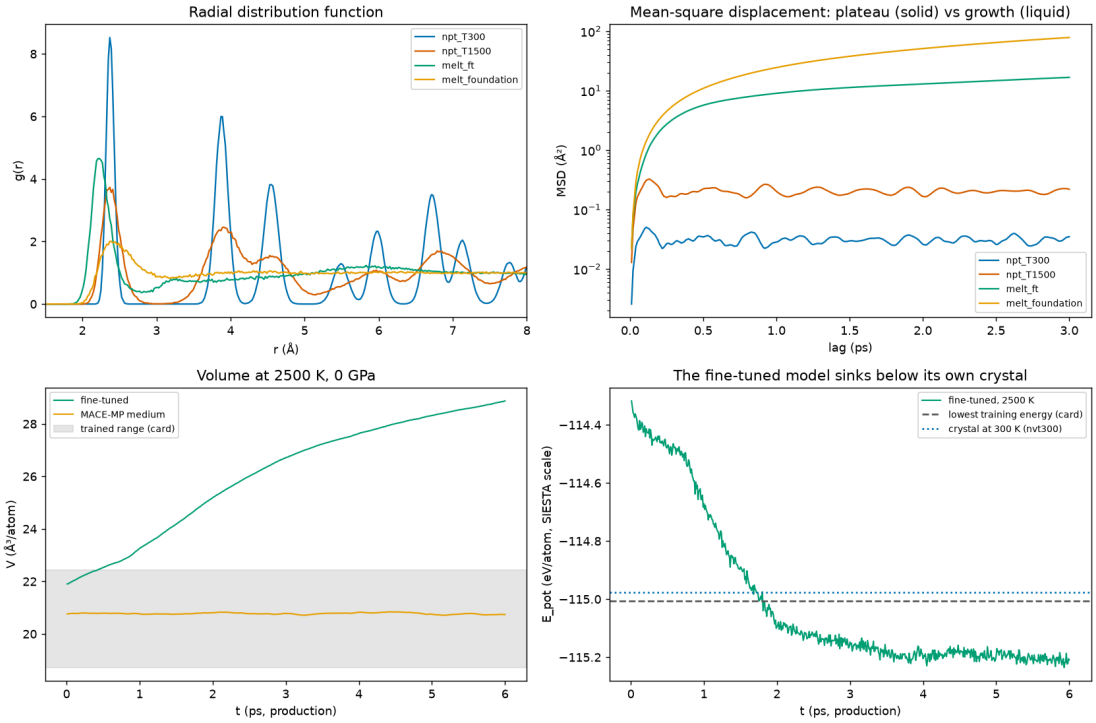
5.8 Far outside the training data: a melt at 2500 K. Both models, NPT at 2500 K and 0 GPa, 3 ps of equilibration and 6 ps of production, from the 216-atom crystal (`reference_results/melt_ft`, `melt_foundation`). Silicon melts at 1687 K. The fine-tuned model's training data holds crystals only: rattled, strained, and 64-atom MD frames at 1400 K, all still diamond.

run	density (g/cm^3)	`g(r)` first peak	coordination	D (cm^2/s)	Lindemann	OUTSIDE the trained range
crystal, 300 K (`npt_T300`)	2.258	2.370 Å	4.00	— (plateau)	0.052	none
crystal, 1500 K (`npt_T1500`)	2.223	2.370 Å	4.00	— (plateau)	0.134	none
fine-tuned, 2500 K	**1.80, still falling**	2.230 Å	4.11	6 × 10^{-5}	1.12	volume per atom, E_pot per atom

run	density (g/cm^3)	`g(r)` first peak	coordination	D (cm^2/s)	Lindemann	OUTSIDE the trained range
MACE-MP medium, 2500 K	2.245	2.390 Å	6.08	4.5×10^{-4}	2.11	(no card)

For reference, experiment gives:

- the liquid just above the melting point: 2.57 g/cm^3 [26], about 10 % denser than the crystal;
- coordination 5–6.4 from X-ray diffraction [18, 19];
- ab initio MD near the melting point: D ≈ 2×10^{-4} cm^2/s and coordination 6.5 [20].



Structure and dynamics of the 2500 K runs

Fig. 10 — Top: `g(r)` *and MSD of the crystal at 300 and 1500 K and of the two 2500 K runs. Bottom: volume (the grey band is the card's trained range) and, for the fine-tuned model, potential energy against time.*

The foundation model gives a liquid, and a reasonable one:

- six neighbours at 2.39 Å, a broad first peak, an MSD growing linearly;
- a diffusion coefficient of the right order for a hotter liquid;
- no warnings: the run is stationary and the barostat holds 0 GPa.

Its density (2.245 g/cm^3 at 2500 K) is below the 2.57 g/cm^3 measured at the melting point. Part of that is the 800 K higher temperature; this run is too short, and the model unvalidated in the liquid, to say more.

The fine-tuned model does not.

- The crystal melts during equilibration, but instead of densifying the cell swells and keeps swelling: +27 % in volume on average over the production run, +40 % at its end, still growing (Fig. 10).
- The potential energy falls by 0.9 eV/atom during the run and ends **0.23 eV/atom below the crystal**.

- The structure is an open network with four short bonds (2.23 Å), nothing like liquid silicon.

The model has found a region of configuration space that its SIESTA data never constrained: low density, short bonds. There it predicts an energy far too low, a hole in the learned energy surface. The dynamics then falls into the hole and keeps going. Nothing in the run itself (temperature, pressure, a stable integration) says that this is wrong.

The report does say it. `[8]` and `[10]` give:

```
volume per atom (Ang^3) | 18.722 .. 22.425       | 21.903 .. 28.873       | OUTSIDE
shortest distance (Ang) | >= 1.852               | min 1.781              | inside (within the 5%
tolerance)
E_pot per atom (eV)     | -115.0069 .. -114.2504 | -115.2353 .. -114.3175 | OUTSIDE (below the lowest
trained energy)
 1. Barostat: <P> = 2.30 GPa is off the 0 GPa target -- the cell is still relaxing ...
 2.-4. Stationarity: Temperature / E_pot / Volume still drifts between the first and last third of
 the run ...
 5. Model domain: the volume per atom (Ang^3) reached 21.903-28.873, outside the trained
 18.722-22.425 ...
 6. Model domain: the potential energy per atom fell to -115.2353 eV, 228 meV/atom below the
 lowest-energy training
    configuration (-115.0069 eV) -- the model calls these structures more stable than anything in its
    SIESTA data,
    typically a hole in the learned energy surface: check a frame with SIESTA before believing any of
    it.
 7. Model card check: volume per atom (Ang^3) is 22.887, outside the trained range ... (first of 138
 such warnings, ...)
```

This run exposed two weaknesses of `stb-mlmd`, which were fixed:

- The card's run-time check used to print one warning per new cell size: 153 near-identical lines in `[10]`. It now groups them by kind, with a count (warning 7).
- The domain table only checked the energy against the *top* of the trained range. An energy below the *bottom*, which for a crystal-trained model means below the relaxed crystal, is now flagged (warning 6).

What it takes to simulate liquid silicon with a fine-tuned model: put the liquid in the training data. Run `7.1.4` with `--sampling-method md` at temperatures above melting, label the picked frames with SIESTA, retrain, and validate on held-out liquid frames (`7.1.3 --md-check`). Section 5.3 measures how wrong the current model is on a frame of this run.

5.9 The raw foundation model at 300 K. The reference run of section 5.2 repeated with `--model medium` (`reference_results/nvt300_foundation`):

	fine-tuned (with card)	MACE-MP-0 medium (raw)
⟨T⟩, σ_T (K)	300.2, 16.2	298.4, 15.9
⟨E_pot⟩ (eV/atom)	−114.9764 (SIESTA's scale)	−5.3029 (the Materials Project's scale)
⟨P⟩ at SIESTA's lattice (GPa)	+0.331	−0.462
C_V (k_B)	2.79	2.93
`g(r)` first peak (height)	2.370 Å (8.61)	2.370 Å (6.78)
RMS vibration, Lindemann	0.127 Å, 0.054	0.155 Å, 0.066
forces vs SIESTA (card's test set / section 5.3)	29 meV/Å	517 meV/Å

Both runs are stable and look like silicon at 300 K. Without a reference, nothing in the foundation run would raise a flag. Against this SIESTA setup, though:

- **Pressure.** At the same cell the pressure has the opposite sign: the foundation model's own lattice is smaller than SIESTA's DZP lattice, so the cell wants to shrink.
- **Vibrations.** The atoms vibrate 22 % more (mean-square amplitude 50 % larger, a lower and broader `g(r)` peak). Its phonons are softer, by about `0.127/0.155` $\approx$ `18 %` in frequency. MACE-MP-0's softening of phonons is a known trait of the foundation models.
- **Energy scale.** Its energies live on a different scale altogether: never compare the absolute energies of two models.

That is the case for fine-tuning: the foundation model gives a plausible trajectory at another level of theory, with errors you cannot see from inside the run.

5.10 What to take from the example.

1. **Inside its training domain, a fine-tuned model gives MD at the level of theory of your own SIESTA setup**, for a cost 10^4–10^5 times lower. Here:
   - forces within about 4 % of SIESTA's on an MD frame at 300 K (section 5.3);
   - a thermal expansion within 10 % of experiment wherever classical nuclei are a fair approximation;
   - textbook fluctuations, equipartition and energy conservation.
2. **Classical MD gives classical answers.** `C_V` $\approx$ `3 k_B` and a too-large α at 300 K are not model errors (section 1.14). Know which of your quantities need quantum nuclei.
3. **The level of theory sets the absolute numbers.** The lattice constant is 1 % too large at every temperature because PBE/DZP puts it there. MD adds the temperature dependence, not a correction of the functional.
4. **Check the run before trusting it**, in this order:
   - the NVE drift for the timestep (section 5.4);
   - equilibration and stationarity (section 5.5);
   - the size: 64+ atoms for silicon (section 5.6);
   - the compressibility in NPT (section 5.7);
   - the error bars, read generously (section 5.2).
5. **Outside the training domain anything can happen, and the trajectory will not tell you.** The 2500 K run is stable, thermostatted and wrong (section 5.8). The model card, the domain table and the warnings are what catch it, and a SIESTA single point on a frame is what proves it (section 5.3).
6. **A foundation model is a plausible, uncalibrated guess.** It is stable and qualitatively right here, even for the liquid, but at another level of theory, with softer phonons and the opposite sign of thermal pressure. Its errors are invisible without a reference.

13.2.2.6 6. Reproducing the reference data

`reference_results/` holds what `make_reference.sh` copies out of its work folder:

- each run's report, data files (`*_md_thermo.dat`, `*_md_rdf.dat`, `*_md_msd.dat`), gnuplot script and final structure. The trajectories (7 MB each) are left out;
- `siesta_check/`: each SIESTA frame check (structure, input, `siesta.out`, `siesta.FA`) with its `forces_compare.json`.

To regenerate:

```
bash make_reference.sh ~/scratch/mlmd_ref A &      # lane A: NVT 300 K, NVE timestep series,
equipartition, sizes, 1500 K, melt
bash make_reference.sh ~/scratch/mlmd_ref B &      # lane B: foundation 300 K, NPT 300-1200 K,
barostat test, foundation melt
wait
bash make_reference.sh ~/scratch/mlmd_ref collect  # reports + data into reference_results/
bash make_reference.sh ~/scratch/mlmd_ref siesta   # SIESTA single points on three final frames, then
siesta_check.py
python3 results_figures.py                         # tables of section 5 and the figures
```

Step	Cost	Notes
MD lanes	about 4 h on one RTX 3060 run one after the other, about 2.5 h side by side	each case is skipped when its report exists, so an interrupted run resumes where it stopped
SIESTA checks	about 2 h per frame on 6 cores (`NP=6`)	SIESTA 5.4.2; the pseudopotential is PseudoDojo's `Si.psml`, the one the model's card records (md5 `878e1f00`)

Seeds are fixed (`--seed`), but GPU arithmetic is not bit-reproducible: a rerun gives the same numbers within their error bars, not to the last digit.

13.2.2.7 7. Known, deliberate limitations

- **Classical nuclei** (section 1.14). Below the Debye temperature, heat capacity, vibration amplitudes and thermal expansion differ from the real material. That is physics left out on purpose, not a model error.
- **Thermostats** (section 1.5). Langevin perturbs the dynamics; measure transport in NVE after an NVT equilibration. Berendsen does not sample fluctuations; do not read `C_V` or compressibilities from NPT.
- **NPT keeps the cell shape.** One isotropic scaling factor: no anisotropic expansion, no shape-changing phase transitions.
- **One model, no error bar per frame.** The domain checks compare with the card's ranges, which is necessary but not sufficient. Inside every range, an unusual local environment can still be predicted badly. A committee (`7.1.4`, `--sampling-method md`) or a SIESTA single point on a frame (section 5.3) is the real check.
- **Statistics from five blocks.** The error bars are honest about the correlation but rough, about ±35 % on the error itself.
- **The foundation model's energies are on its own scale.** Compare energy differences only, and never mix them with a fine-tuned model's SIESTA-scale energies.
- **Melting in a periodic cell superheats** (section 1.12). Heating until it melts is not a melting temperature.

13.2.2.8 8. Step-by-step: running MD on your own material

1. **Get a model you can trust for the job.**
    - For a qualitative look, the foundation model (`--model medium`) needs nothing.
    - For your level of theory, train one with `7.1` on configurations like the ones the MD will visit: rattled, strained and MD-sampled at the temperatures you care about. Validate it with `7.1.3`, ideally with `--md-check` frames.

- Keep the model and its card together.

2. **Build a cell large enough.** At least a few hundred atoms for fluctuation-based quantities; `--supercell NA NB NC` repeats your cell on the fly, or `stb-supercell` (`4.2`) writes the supercell file. The model is local, so a bigger cell than the training cells is fine (the domain table says `larger`).
3. **Check the timestep with a short NVE.** Equilibrate briefly, then run NVE for 1–2 ps. The drift should be well below 1 meV/atom/ps and the total-energy fluctuation a small fraction of the potential-energy one (section 5.4).
4. **Equilibrate, then produce.**
   - `--equilibration-steps` of a few coupling times ($\geq$ 1 ps); in NPT, until the volume is stationary.
   - Then a production run long enough for the quantity you want: picoseconds for averages, tens of picoseconds for heat capacities or diffusion.
5. **NPT: give the barostat your material's compressibility**, `--compressibility 1/(B[GPa] × 10^4)`.
6. **Read the report from the bottom.**
   - `[10] WARNINGS` first: drift, stationarity, domain, card alarms.
   - Then `[8]`: did the run stay inside the trained range, and is the phase what you think it is (coordination, Lindemann, MSD)?
   - Then the numbers of `[6]`–`[7]` with their error bars.
7. **Check one frame against SIESTA** when the result matters, especially if `[8]` says `OUTSIDE` anywhere: a single point at the same level of theory (`siesta_calc.fdf` and `siesta_check.py` here show how).
8. **Keep the data:** `--save-data` or `--save-gnuplot` writes the time series behind every number, and `--save-report` writes the report.

13.2.2.9 9. Files in this folder

File	What it is
`README.md`	this tutorial
`example_7.2.sh`	the guided walk-through (CPU, about 10 minutes; `DEVICE=cuda` for a GPU)
`si8.fdf`	bulk silicon, 8-atom conventional cell at SIESTA's relaxed lattice constant (5.48024 Å)
`model/mlff_model_compiled.model`	the fine-tuned MACE model (`best_model/` of the `7.1` silicon run: round 2, seed 125)
`model/mlff_model_card.json`	its model card (SHA-256, level of theory, trained range, errors, phonons, selection)
`make_reference.sh`	regenerates `reference_results/` (GPU MD lanes, collect, SIESTA checks)
`siesta_calc.fdf`, `siesta_config_extra.fdf`	input of the SIESTA single points on MD frames (same level of theory as the training data)
`siesta_check.py`	evaluates both models on those frames against SIESTA → `forces_compare.json` (needs the `ml` extra)
`results_figures.py`	reads `reference_results/`, prints the tables of section 5, draws `figures/*.png` (numpy + matplotlib only)
`theory_figures.py`	draws the figures of section 1
`figures/`	every figure of this README
`reference_results/`	reports, data and SIESTA checks of the GPU runs (section 6)

13.2.2.10 10. Running the script

```
cd examples/7.2-mlmd
bash example_7.2.sh               # CPU, ~10 min; press Enter between cases
DEVICE=cuda bash example_7.2.sh   # on a GPU
```

Each case writes into `output/<case>/`, which is wiped at every start. Case 7 downloads the MACE-MP-0 medium model on first use. Case 10 needs no MACE: it only reads `reference_results/`.

13.2.2.11 11. Flag reference

`stb-mlmd` — `--file FILE` (`.fdf`, required) · `--supercell NA NB NC` (repeat the cell before the run, default `1 1 1`) · `--model {small,medium,large}` (default medium) · `--custom-model PATH` + `--custom-model-card JSON` (always both) · `--ensemble {nve,nvt,npt}` (default nvt) · `--temperature K` (default 300; NVE: initial velocities only) · `--pressure GPa` (NPT, default 0) · `--timestep fs` (default 1.0) · `--n-steps N` (production, default 1000) · `--equilibration-steps N` (default 0) · `--friction` fs^{-1} (NVT Langevin, default 0.01) · `--taut fs` (NPT, default 100) · `--taup fs` (NPT, default 500) · `--compressibility 1/bar` (NPT, default 4.57e-5, water's placeholder) · `--stride N` (default 10) · `--vacuum-gap Å` (default 10) · `-of {xsf,pdb,xyz}` · `-o FILE` (trajectory, default `<stem>_md_traj.<ext>`) · `--output-structure FILE` (default `md_final.fdf`) · `--device {cpu,cuda}` · `--seed N` · `--png [FILE]` · `--show` · `--save-gnuplot` · `--save-data` · `--save-report` · `--no-intro` · `-v`.

13.2.2.12 12. What's next

- **`7.1` (MLFF workflow)** trains the model this tutorial uses. Its Stage 3 `--md-check` runs exactly the check of section 5.3 on frames of an MD, and Stage 4 `--sampling-method md` picks MD frames for the next training round.
- **`7.3` (`stb-mlphonons`)** gives the harmonic and quasi-harmonic picture (`--qha`) that the MD of section 5.7 goes beyond: compare its thermal expansion with the NPT runs.
- **`2.18` (`stb-aimdAnalysis`)** analyses any trajectory: RDF, MSD and the vibrational density of states. It reads the `.xsf`/`.xyz` written here as well as SIESTA's own `.ANI`.
- **`4.11` (`stb-amorphize`)** uses the same MD engine for a melt-quench.

13.2.2.13 13. References

1. L. Verlet, "Computer 'experiments' on classical fluids. I.", *Phys. Rev.* **159**, 98 (1967).
2. W. C. Swope, H. C. Andersen, P. H. Berens, K. R. Wilson, "A computer simulation method for the calculation of equilibrium constants…", *J. Chem. Phys.* **76**, 637 (1982).
3. M. P. Allen, D. J. Tildesley, *Computer Simulation of Liquids*, 2nd ed. (Oxford University Press, 2017).
4. D. Frenkel, B. Smit, *Understanding Molecular Simulation*, 3rd ed. (Academic Press, 2023).
5. E. Hairer, C. Lubich, G. Wanner, "Geometric numerical integration illustrated by the Störmer–Verlet method", *Acta Numerica* **12**, 399 (2003).
6. T. Schneider, E. Stoll, "Molecular-dynamics study of a three-dimensional one-component model for distortive phase transitions", *Phys. Rev. B* **17**, 1302 (1978).
7. H. J. C. Berendsen, J. P. M. Postma, W. F. van Gunsteren, A. DiNola, J. R. Haak, "Molecular dynamics with coupling to an external bath", *J. Chem. Phys.* **81**, 3684 (1984).
8. G. Bussi, D. Donadio, M. Parrinello, "Canonical sampling through velocity rescaling", *J. Chem. Phys.* **126**, 014101 (2007).

9. G. J. Martyna, D. J. Tobias, M. L. Klein, "Constant pressure molecular dynamics algorithms", *J. Chem. Phys.* **101**, 4177 (1994).
10. J. L. Lebowitz, J. K. Percus, L. Verlet, "Ensemble dependence of fluctuations with application to machine computations", *Phys. Rev.* **153**, 250 (1967).
11. H. Flyvbjerg, H. G. Petersen, "Error estimates on averages of correlated data", *J. Chem. Phys.* **91**, 461 (1989).
12. F. A. Lindemann, "Über die Berechnung molekularer Eigenfrequenzen", *Phys. Z.* **11**, 609 (1910).
13. A. Einstein, "Über die von der molekularkinetischen Theorie der Wärme geforderte Bewegung von in ruhenden Flüssigkeiten suspendierten Teilchen", *Ann. Phys.* **17**, 549 (1905).
14. I.-C. Yeh, G. Hummer, "System-size dependence of diffusion coefficients and viscosities from molecular dynamics simulations with periodic boundary conditions", *J. Phys. Chem. B* **108**, 15873 (2004).
15. I. Batatia, D. P. Kovács, G. N. C. Simm, C. Ortner, G. Csányi, "MACE: Higher order equivariant message passing neural networks for fast and accurate force fields", *Advances in Neural Information Processing Systems* **35** (2022).
16. I. Batatia et al., "A foundation model for atomistic materials chemistry", arXiv:2401.00096 (2023).
17. Y. Okada, Y. Tokumaru, "Precise determination of lattice parameter and thermal expansion coefficient of silicon between 300 and 1500 K", *J. Appl. Phys.* **56**, 314 (1984), doi:10.1063/1.333965.
18. Y. Waseda, K. Shinoda, K. Sugiyama, S. Takeda, K. Terashima, J. M. Toguri, "High temperature X-ray diffraction study of melt structure of silicon", *Jpn. J. Appl. Phys.* **34**, 4124 (1995), doi:10.1143/jjap.34.4124.
19. T. H. Kim et al., "In situ high-energy x-ray diffraction study of the local structure of supercooled liquid Si", *Phys. Rev. Lett.* **95**, 085501 (2005), doi:10.1103/physrevlett.95.085501.
20. I. Štich, R. Car, M. Parrinello, "Bonding and disorder in liquid silicon", *Phys. Rev. Lett.* **63**, 2240 (1989).
21. D. Alfè, M. J. Gillan, "Exchange-correlation energy and the phase diagram of Si", *Phys. Rev. B* **68**, 205212 (2003).
22. M. W. Chase, *NIST-JANAF Thermochemical Tables*, 4th ed., *J. Phys. Chem. Ref. Data Monograph* **9** (1998).
23. C. Kittel, *Introduction to Solid State Physics*, 8th ed. (Wiley, 2005).
24. M. Ceriotti, G. Bussi, M. Parrinello, "Nuclear quantum effects in solids using a colored-noise thermostat", *Phys. Rev. Lett.* **103**, 030603 (2009).
25. A. H. Larsen et al., "The atomic simulation environment — a Python library for working with atoms", *J. Phys.: Condens. Matter* **29**, 273002 (2017).
26. Y. Sato, T. Nishizuka, K. Hara, T. Yamamura, Y. Waseda, "Density measurement of molten silicon by a pycnometric method", *Int. J. Thermophys.* **21**, 1463 (2000).
27. K. G. Lyon, G. L. Salinger, C. A. Swenson, G. K. White, "Linear thermal expansion measurements on silicon from 6 to 340 K", *J. Appl. Phys.* **48**, 865 (1977), doi:10.1063/1.323747.

13.2.3 Command-line reference

13.2.3.1 `stb-mlmd`

Molecular dynamics driven by a MACE potential – the MACE-MP-0 foundation model, or a custom model fine-tuned on your own SIESTA data via `stb-mlffAnalysis` (`--custom-model`, always with its model card). NVE (microcanonical), NVT (Langevin thermostat), or NPT (Berendsen thermostat+barostat, bulk-only) ensembles. The report covers the model and its card, thermodynamic averages with error bars, ensemble checks (NVE drift in meV/atom/ps, temperature fluctuations, pressure), the structure along the run (shortest distance, g(r), MSD and diffusion, Lindemann ratio) and whether the run stayed inside the model's trained range; every warning goes to one final section. A fast heuristic exploration

tool, not a substitute for a real ab initio MD run.

Synopsis.

```
usage: stb-mlmd [-h] --file FILE [--model {small,medium,large}]
                [--custom-model PATH] [--custom-model-card JSON]
                [--ensemble {nve,nvt,npt}] [--temperature TEMPERATURE]
                [--pressure PRESSURE] [--timestep TIMESTEP]
                [--n-steps N_STEPS]
                [--equilibration-steps EQUILIBRATION_STEPS]
                [--friction FRICTION] [--taut TAUT] [--taup TAUP]
                [--compressibility COMPRESSIBILITY] [--stride STRIDE]
                [--supercell NA NB NC] [--vacuum-gap VACUUM_GAP]
                [-of {pdb,xsf,xyz}] [-o OUTPUT]
                [--output-structure OUTPUT_STRUCTURE] [--device {cpu,cuda}]
                [--seed SEED] [--png [FILE]] [--show] [--save-gnuplot]
                [--save-data] [--save-report] [--no-intro] [-v]
```

Options

`--file FILE`
: Input structure (.fdf).

`--model {small,medium,large}`
: MACE-MP-0 model size (default: medium). Ignored if `--custom-model` is given. choices: small, medium, large

`--custom-model PATH`
: Use a custom MACE model file instead of the MACE-MP-0 foundation potential – e.g. one fine-tuned via `stb-mlffAnalysis`. Overrides `--model`.

`--custom-model-card JSON`
: Model card of the `--custom-model` file: the `mlff_model_card.json` that `stb-mlffAnalysis` wrote next to it (training data, level of theory, accuracy, energy scale). Required with `--custom-model` – the card is checked against the model file and used to warn when a structure is outside what the model was trained on.

`--ensemble {nve,nvt,npt}`
: MD ensemble (default: nvt). npt requires a bulk (no vacuum-padded axis) structure – cell dynamics on a slab/wire/molecule would be physically meaningless. choices: nve, nvt, npt

`--temperature TEMPERATURE`
: Target temperature, K (default: 300). In NVE only the initial velocities. type: float

`--pressure PRESSURE`
: Target pressure, GPa (npt only, default: 0.0 – ambient). type: float

`--timestep TIMESTEP`
: MD timestep, fs (default: 1.0). type: float

`--n-steps N_STEPS`
: Number of PRODUCTION MD steps, after `--equilibration-steps` (default: 1000). type: int

`--equilibration-steps EQUILIBRATION_STEPS`
: Steps to run and discard BEFORE production (default: 0). Verified live that a freshly Maxwell-Boltzmann-seeded structure relaxes for ~100-200 steps before settling – letting that transient into the saved trajectory/statistics biases them towards the initial condition. type: int

`--friction FRICTION`
: Langevin friction coefficient, fs^-1 (nvt only, default: 0.01). type: float

`--taut TAUT`
: Thermostat coupling time, fs (npt only – Berendsen, default: 100). type: float

`--taup TAUP`
: Barostat coupling time, fs (npt only, default: 500). type: float

`--compressibility COMPRESSIBILITY`
: Compressibility, 1/bar (npt only, default: 4.57e-5 – water's value, a generic placeholder; override with something closer to 1/bulk-modulus for the actual material, 1/(B[GPa] x 1e4) – e.g. Si, B ~ 98 GPa: 1e-6 – for a better-calibrated barostat response). type: float

`--stride STRIDE`
: Save one trajectory frame (and one sample of every quantity) every Nth MD step (default: 10). type: int

`--supercell NA NB NC`
: Repeat the input cell NA x NB x NC times along a, b, c before the run (default: 1 1 1). A small cell constrains the dynamics to its own periodicity (no long-wavelength modes) and gives poor statistics; use ~64-500 atoms for thermodynamics, g(r) and diffusion. type: int

`--vacuum-gap VACUUM_GAP`
: Gap (Ang) used to detect vacuum-padded axes (default: 10.0, matches `stb-kgrid`/`stb-mlrelax`). type: float

`-of, --out-format {pdb,xsf,xyz}`
: Trajectory format (default: xsf). Same 3 choices as `stb-ani2traj`. choices: pdb, xsf, xyz

`-o, --output OUTPUT`
: Trajectory filename. Default: <input stem>_md_traj.<ext>

`--output-structure OUTPUT_STRUCTURE`
: Final-frame structure output (default: `md_final.fdf`).

`--device {cpu,cuda}`
: Device to run the model on (default: cpu). choices: cpu, cuda

`--seed SEED`
: Random seed for initial velocities. type: int

`--png FILE`
: Save the run's dashboard (temperature, energies, pressure, volume or shortest distance, g(r), MSD) as a PNG (default name: <input `stem`>_md_dashboard.png). Off by default.

`--show`
: Display the dashboard on screen (needs a display). Off by default. switch, off by default

`--save-gnuplot`
: Write the data files (as `--save-data`) plus <input stem>_md.gplot, the same six panels for gnuplot ('gnuplot <stem>_md.gplot' -> <stem>_md.pdf). Off by default. switch, off by default

`--save-data`
: Write the data behind the report: `<stem>_md_thermo.dat` (time series: T, energies per atom, P, volume, cell, shortest distance, max force), `<stem>_md_rdf.dat` and `<stem>_md_msd.dat`. Off by default. switch, off by default

`--save-report`
: Also persist the report to `stb_mlmd_report.txt`. Off by default. switch, off by default

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Example usage:
  stb-mlmd --file structure.fdf --ensemble nvt --temperature 300 --n-steps 5000
  stb-mlmd --file structure.fdf --ensemble nvt --equilibration-steps 200 --n-steps 5000 --png --save-gnuplot
  stb-mlmd --file structure.fdf --custom-model mlff_model.model --custom-model-card mlff_model_card.json
 --ensemble npt \
                          --temperature 500 --pressure 0.0 --compressibility 1e-6 --n-steps 10000 --device cuda
```

13.3 ML Phonons

At a glance

Menu code: 7.3 **Command:** `stb-mlphonons`
Guided example: `examples/7.3-mlphonons/`
Summary: Phonons from a MACE potential by finite displacements – the foundation model or your own fine-tuned one, always with its model card: band structure with symmetry labels, sound velocities, DOS, quantum thermodynamics; with `-qha` the thermal expansion, $B(T)$ and Grüneisen parameters; with `-check-convergence` the supercell, q-point by q-point.

13.3.1 Overview

`stb-mlphonons` computes the vibrational normal modes of a crystal with a MACE potential [15]: the MACE-MP-0 foundation model [16] (`-model`) or a model fine-tuned on your own SIESTA calculations with the MLFF workflow (7.1; `-custom-model` with `-custom-model-card`). It is the finite-displacement method of 5.4 [11, 100] with the SIESTA single points replaced by the model: phonopy moves one atom of a supercell, the model gives the forces on every atom, and the force constants, the band structure, the density of states and the harmonic thermodynamics follow in seconds. That speed opens what is too expensive with DFT: supercells of hundreds of atoms, a convergence check in every run, the quasi-harmonic volume scan, structures without symmetry. With a fine-tuned model the raw foundation model runs the same calculation, so the report shows what fine-tuning changed (Fig. 13.2).

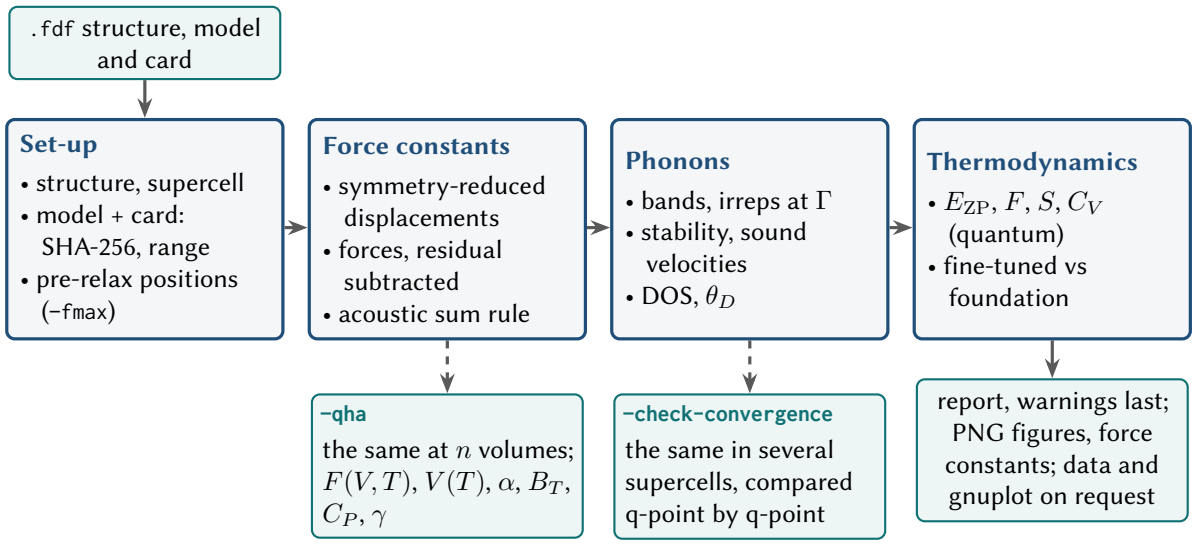


Figure 13.2: What one **`stb-mlphonons`** run does. The two other modes repeat the pipeline over volumes (QHA) or over supercells (convergence check).

Phonons in eight equations

Harmonic crystal [69, 72]. The force constants are the second derivatives of the energy, obtained from finite differences of the forces, and must obey the acoustic sum rule:

$$\Phi_{i\alpha,j\beta} = \frac{\partial^2 E}{\partial u_{i\alpha}\partial u_{j\beta}} \approx -\frac{F_{j\beta}(u_{i\alpha}=u) - F_{j\beta}(0)}{u}, \qquad \sum_j \Phi_{i\alpha,j\beta} = 0.$$

Phonons. The mass-weighted Fourier transform of Φ is the dynamical matrix; its eigenvalues are the squared frequencies of the $3n$ branches (a negative ω^2 is an imaginary, "negative", frequency):

$$D_{\kappa\alpha,\kappa'\beta}(\mathbf{q}) = \frac{1}{\sqrt{m_\kappa m_{\kappa'}}} \sum_l \Phi_{0\kappa\alpha,l\kappa'\beta}\, e^{i\mathbf{q}\cdot\mathbf{R}_l}, \qquad D(\mathbf{q})\,\mathbf{e}_s(\mathbf{q}) = \omega_s^2(\mathbf{q})\,\mathbf{e}_s(\mathbf{q}).$$

A supercell gives the exact ω at the wavevectors commensurate with it and an interpolation in between. **Sound.** The acoustic slopes are elastic constants: $\rho v^2 = C_{11}$ (LA[100]), C_{44} (TA[100]), …**Thermodynamics** (quantum, per mode $x = h\nu/k_BT$):

$$F_{\text{vib}} = \sum_{\mathbf{q}s} \left[\tfrac{1}{2} h\nu_{\mathbf{q}s} + k_BT \ln\!\left(1 - e^{-x_{\mathbf{q}s}}\right)\right], \qquad C_V = k_B \sum_{\mathbf{q}s} \frac{x_{\mathbf{q}s}^2 e^{x_{\mathbf{q}s}}}{(e^{x_{\mathbf{q}s}} - 1)^2} \xrightarrow{T\to\infty} 3k_B \text{ per atom.}$$

Quasi-harmonic approximation [101]. The phonons at several volumes and an equation of state [102] give $V(T)$; the expansion is the Grüneisen relation [103]:

$$F(V,T) = E_{\text{static}}(V) + F_{\text{vib}}(V,T), \qquad \gamma_{\mathbf{q}s} = -\frac{\partial \ln \nu_{\mathbf{q}s}}{\partial \ln V}, \qquad \alpha_V = \frac{\gamma\, C_V}{B_T\, V}.$$

Before you trust ML phonons

- **Foundation models are soft.** Universal potentials underestimate the curvature of the energy surface [104]; for silicon every MACE-MP-0 size puts the optical modes 25–36 % too low, the largest model most (Section 13.3.2.5). Fine-tune (**7.1**) for numbers you will use, and read the `FINE-TUNED vs FOUNDATION` section.
- **Compute the phonons at the model's own minimum**: relax the cell with the same model first (**`stb-mlrelax`** `-relax-cell`), and symmetrize a structure that should have symmetry — a slightly distorted input stays P1, with split degeneracies (Section 13.3.2.5).
- **No imaginary mode does not prove a minimum**: the residual forces are subtracted, so a slightly displaced structure can still look stable.
- **Converge the supercell** with `-check-convergence`: special points depend on commensurability, sound velocities on size.
- **The QHA is a harder test** than the bands: it needs the volume dependence of the force constants. Look at the shape of $\alpha(T)$ and $B_T(T)$ — a foundation model gave a bulk modulus that rises on heating.
- **No LO–TO splitting**: a MACE model has no Born charges, so polar optical modes near Γ are incomplete.
- **When comparing with DFT phonons, check the DFT force set**: a single bad SIESTA force produced all the imaginary modes of the silicon reference (Section 13.3.2.5).

13.3.2 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/7.3-mlphonons/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

`stb-mlphonons` computes the **phonons** of a crystal — its vibrational normal modes — with a MACE machine-learned potential: either the MACE-MP-0 foundation model, or a model you fine-tuned on your own SIESTA calculations with the MLFF workflow (`7.1`), which always comes with its **model card**. It uses the finite-displacement method through phonopy: move one atom of a supercell by a

hundredth of an ångström, compute the forces on every atom, build the force constants, and diagonalize the dynamical matrix. From that one calculation it reports:

- the phonon band structure, the frequencies and symmetry labels at the special points, dynamical stability;
- the sound velocities, which are elastic constants in disguise;
- the density of states (total and per species) and the Debye temperature;
- the harmonic thermodynamics — zero-point energy, free energy, entropy, heat capacity — with quantum statistics;
- optionally, a vibration as an animated trajectory, or the structure displaced along an unstable mode.

Two more modes:

- `--qha`, the **quasi-harmonic approximation**: the same calculation at several volumes, which gives the thermal expansion, the bulk modulus against temperature, the heat capacity at constant pressure and the Grüneisen parameters;
- `--check-convergence`: the same calculation in several supercells, compared q-point by q-point.

With a fine-tuned model it repeats the calculation with the raw foundation model, so you see what fine-tuning changed. Plots, gnuplot files and data files are written only when you ask for them.

Why use a potential for phonons? A phonon calculation is many force calculations on a supercell. For the silicon of this example, one SIESTA force calculation of the 64-atom supercell takes 2 min 15 s on 6 cores; the fine-tuned model takes about 3 s on one CPU, including building and diagonalizing everything. For crystalline silicon that hardly matters: its symmetry reduces the job to one displaced supercell, and the SIESTA phonons of `5.4` are cheap. It matters as soon as the calculation multiplies:

Job	SIESTA (6 cores)	Fine-tuned model (CPU)
silicon, 2×2×2 supercell (64 atoms, 1 displacement + the undisplaced cell)	≈ 4.5 min	14 s
the same in a 4×4×4 supercell (512 atoms)	hours to a day (cost grows as N^2 to N^3)	36 s
quasi-harmonic scan, 7 volumes	≈ 30 min	26 s
a structure with no symmetry: 48 displacements (section 5.8)	≈ 2 h	87 s
a defect, an alloy, a surface: hundreds of displacements	days	minutes

The price is the same as for every machine-learned quantity: the model is only as good as the structures it was trained on, and phonons are second derivatives of its energy surface, a demanding test (section 1.11). This tool prints the model card and the comparison with the foundation model so you can judge.

The worked example (section 5) runs it for bulk silicon, with the fine-tuned model of the `7.2` example (trained in the `7.1` example's silicon run), against experiment and against SIESTA:

- the band structure against inelastic neutron scattering;
- sound velocities against measured elastic constants, and against the same model's stress-strain constants;
- heat capacity and entropy against calorimetry;
- three foundation models, all too soft;
- supercell size and displacement size;
- a structure that is not at a minimum;
- the SIESTA reference phonons of `7.1`, whose imaginary modes come from one bad force;
- the quasi-harmonic thermal expansion against experiment and against the MD of `7.2`.

13.3.2.1 1. Theory

Symbols at a glance

Symbol	Meaning
N, n	atoms in the supercell; atoms in the primitive cell (2 for silicon)
u_iα	displacement of atom i along Cartesian direction α, Å
F_iα = −∂E/∂u_iα	force on atom i along α, eV/Å
Φ_iα,jβ	force constant: second derivative of the energy, eV/Å^2
m_i	mass of atom i
q, ω(q) = 2πν(q)	phonon wavevector; angular frequency; ν frequency in THz (1 THz = 33.36 cm^{-1} = 4.136 meV)
D(q)	dynamical matrix, 3n × 3n
e_s(q)	eigenvector (polarization) of branch s
g(ν)	phonon density of states, normalized to 3n per primitive cell
ħ, h, k_B	reduced and full Planck constant, Boltzmann constant
E_ZP, F_vib, S, C_V	zero-point energy, vibrational free energy, entropy, heat capacity at constant volume
θ_D	Debye temperature (645 K for silicon from calorimetry)
C_ij, ρ, v	elastic constants (Voigt), mass density, sound velocity
V, B, B′	volume, bulk modulus, its pressure derivative
γ_s(q), γ	mode Grüneisen parameter; thermodynamic Grüneisen parameter
α_V, α_L	volume and linear thermal-expansion coefficients (α_L = α_V / 3 for a cubic crystal)
u (in section 1.5)	the finite-displacement amplitude, --distance, Å

1.1 The harmonic crystal. At low temperature the atoms of a crystal vibrate about their equilibrium positions by a few hundredths of an ångström. Expand the potential energy — here, the model's energy — in the displacements u_iα from equilibrium:

$$E = E_0 + \Sigma_{i\alpha}\,(\partial E/\partial u_{i\alpha})\,u_{i\alpha} + \tfrac{1}{2}\,\Sigma_{i\alpha,j\beta}\,\Phi_{i\alpha,j\beta}\,u_{i\alpha}\,u_{j\beta} + O(u^3) \quad (1)$$

$$\Phi_{i\alpha,j\beta} = \partial^2 E \,/\, \partial u_{i\alpha}\,\partial u_{j\beta} = -\partial F_{j\beta} \,/\, \partial u_{i\alpha}$$

At equilibrium the first-order term vanishes: every force is zero. Stopping at second order is the **harmonic approximation**: the crystal is a set of masses joined by springs, and Φ is the matrix of spring constants [1–4]. Φ_iα,jβ answers one question: if atom i moves by 1 Å along α, what force appears on atom j along β? Newton's equations become linear:

$$m_i\,\ddot{u}_{i\alpha} = -\Sigma_{j\beta}\,\Phi_{i\alpha,j\beta}\,u_{j\beta} \quad (2)$$

The harmonic approximation is exact in the limit of small vibrations and good for most solids well below their melting point. What it leaves out — the terms of third and higher order — gives phonons a finite lifetime, makes the frequencies drift with temperature and makes the crystal expand. Section 1.10 brings back the part needed for thermal expansion.

1.2 Phonons: waves in a periodic crystal. In a crystal every unit cell is the same, so the solutions of Eq. 2 are waves. Atom κ of the cell at lattice vector R_l moves as

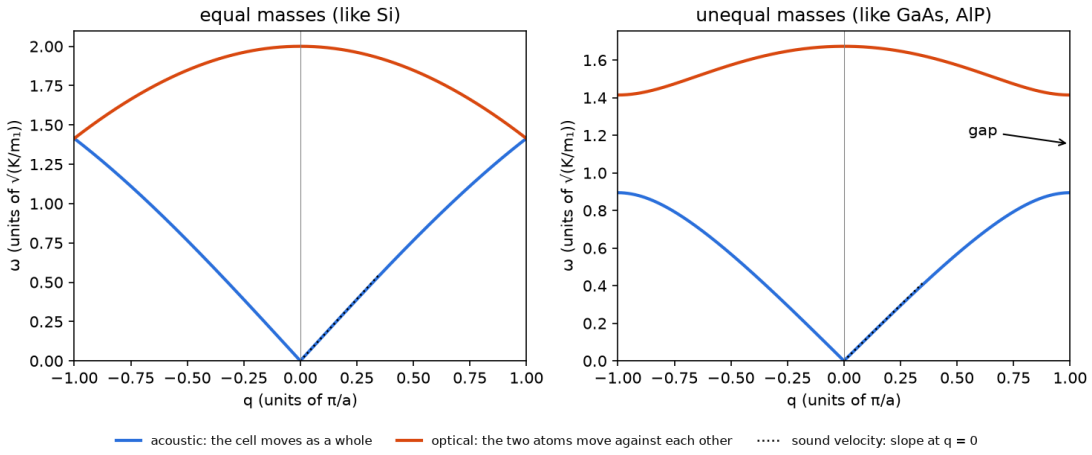
$$u_{l\kappa}(t) = (1/\sqrt{m_\kappa})\,e_\kappa(q)\,\exp[i(q\cdot R_l - \omega t)] \quad (3)$$

Substituting into Eq. 2 leaves a 3n × 3n eigenvalue problem for each wavevector q, with n the number of atoms in the primitive cell:

$$D_{\kappa\alpha,\kappa'\beta}(q) = \frac{1}{\sqrt{m_\kappa m_{\kappa'}}} \sum_l \Phi_{0\kappa\alpha,l\kappa'\beta} e^{iq\cdot R_l} \qquad D(q)\, e_s(q) = \omega_s^2(q)\, e_s(q) \tag{4}$$

$D(q)$ is the **dynamical matrix**: the mass-weighted Fourier transform of the force constants. Its $3n$ eigenvalues $\omega_s^2(q)$ are the squared frequencies of the $3n$ **branches**; the eigenvectors $e_s(q)$ say how each atom moves in that mode. A plot of $\omega_s(q)$ along lines between high-symmetry points of the Brillouin zone is the **phonon band structure**. Fig. 1 shows the simplest case, a one-dimensional chain with two atoms per cell:

- **Acoustic branches** (3 of them in 3D): at $q \to 0$ the whole cell moves together. The frequency goes to zero linearly, $\omega = v|q|$: these are sound waves, and v is the speed of sound (section 1.7).
- **Optical branches** ($3n - 3$): the atoms of a cell move against each other. Their frequency stays finite at $q = 0$. In an ionic crystal they couple to light, hence the name.
- With **two different masses** a gap opens between the branches at the zone boundary. With equal masses (silicon: two identical atoms in the cell) the two branches meet there.



Acoustic and optical branches of a diatomic chain

Fig. 1 — The 1D diatomic chain with nearest-neighbour springs (`theory_figures.py`*). Left: equal masses, like silicon. Right: unequal masses, like GaAs or AlP: a gap between the branches. The dotted line is the sound velocity, the slope of the acoustic branch at* $q = 0$.

Silicon has $n = 2$, so it has 6 branches: 3 acoustic, 3 optical. Its primitive cell is face-centred cubic; the path `stb-mlphonons` follows through the FCC Brillouin zone is Γ–X–W–K–Γ–L–U–W–L–K | U–X (Γ is the zone centre, X the centre of the square face, L the centre of the hexagonal face, K and U the middle of an edge — equivalent points).

1.3 Finite displacements: from forces to force constants. There are two ways to get Φ from first principles:

- density-functional perturbation theory, which computes the second derivatives analytically [7];
- the **finite-displacement** (direct, "frozen-phonon supercell") method [5, 6], which differentiates the forces numerically. It needs nothing but forces, so it works with any code and any potential — SIESTA (`5.4`) or MACE (this tool).

Finite displacements work as follows [5]:

1. Build a **supercell** of the primitive cell (`--dim`, default `2 2 2` of the input cell).

2. Move one atom `i` by a small `u` along α (`--distance`, default 0.01 Å).
3. Compute the forces on every atom of the supercell. To first order in `u`:

```
Φ_iα,jβ ≈ −[F_jβ(u_iα = u) − F_jβ(0)] / u
(5)
```

Two refinements matter in practice:

- **Symmetry.** Displacements related by a symmetry operation of the crystal give no new information. phonopy (through spglib [26]) generates only the symmetry-inequivalent ones and reconstructs every other force constant by symmetry. For diamond-structure silicon (space group `Fd-3m`, 1536 operations in the 2×2×2 supercell), **one** displaced supercell gives every force constant — instead of `64 atoms × 3 directions × 2 signs = 384`. When the site symmetry does not already relate `+u` to `−u`, phonopy adds the opposite displacement, which turns Eq. 5 into a central difference.
- **Residual forces.** Eq. 5 subtracts `F(0)`, the forces on the undisplaced supercell. They should be zero, but a structure relaxed to `--fmax 0.05 eV/Å` still has small forces, and summed over a sublattice even 0.005 eV/Å per atom can shift an optical mode by several THz (graphene, `7.1`). `stb-mlphonons` always computes and subtracts them (report `[3]`).

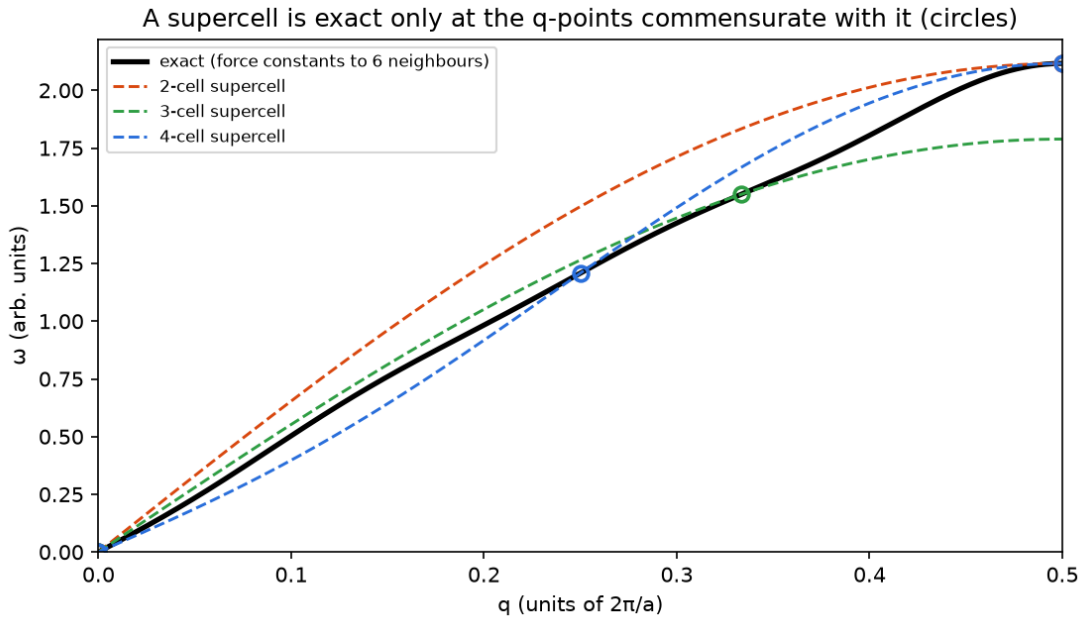
The force constants then satisfy one exact condition of physics, the **acoustic sum rule**: translating the whole crystal rigidly costs no energy, so

```
Σ_j Φ_iα,jβ = 0          for every i, α, β                    (6)
```

Numerical force constants violate it slightly, which shows up as small imaginary or non-zero acoustic frequencies at Γ. `stb-mlphonons` reports the violation and then imposes the rule (`symmetrize_force_constants`).

1.4 What a supercell can see. A supercell of `L` cells along an axis sees an atom and all its periodic images together: the force on atom `j` is the sum of the force constants to `j` and to every image of `j`, `Σ_k Φ(R_j + k L a)`. phonopy assigns that sum to the nearest image and interpolates the dispersion from it. Two consequences (Fig. 2):

- At wavevectors **commensurate** with the supercell (`q = m/L` in units of the reciprocal lattice vector), the phase `exp(iq·kLa)` is 1 for every image, so the sum is exactly what Eq. 4 needs: **these frequencies are exact**, whatever the range of the force constants.
- Everywhere else the dispersion is an interpolation. It converges as the supercell grows past the range of the force constants.



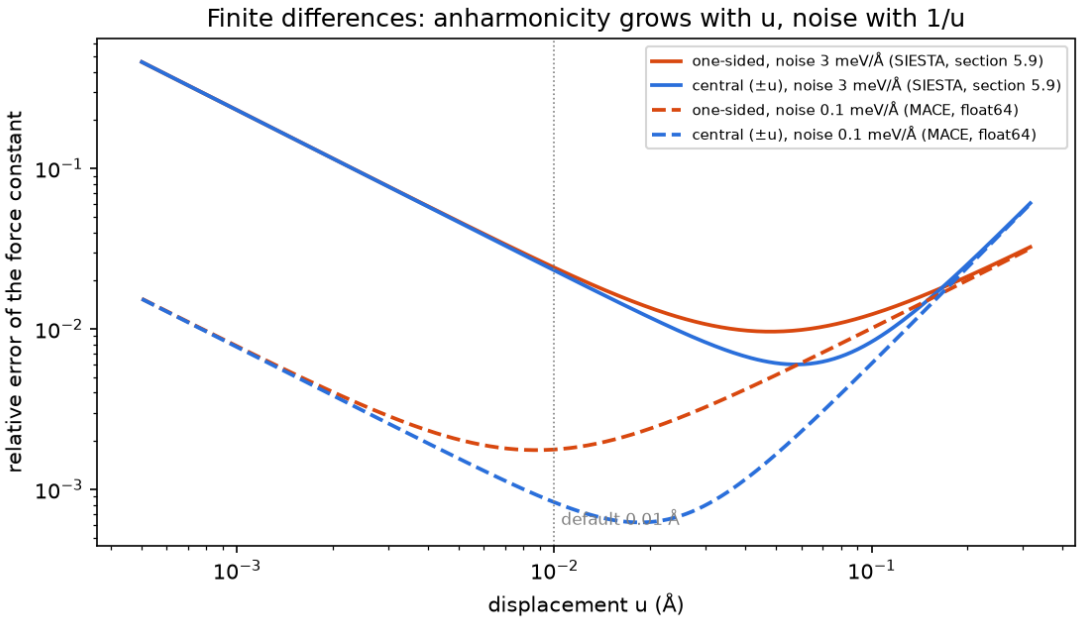
A supercell is exact at the commensurate q-points

Fig. 2 — A 1D chain whose force constants reach 6 neighbours, computed in supercells of 2, 3 and 4 cells (`theory_figures.py`*). Each supercell is exact at its commensurate points (circles; those of the 2-cell supercell,* `q = 0` *and ½, coincide with the 4-cell ones) and interpolates in between.*

For silicon's 8-atom cubic input cell, Γ and `X` are commensurate even with the 1×1×1 cell; `L` needs 2×2×2. So the convergence check (section 5.6) must look at the whole band path, not at Γ (where every supercell agrees by construction). The range of the force constants of a MACE model is bounded: each of its 2 message-passing layers reaches `r_max = 6` Å, so a displaced atom can change forces up to 12 Å away (report `[2]`, "Model range"). A supercell much wider than 12 Å gains nothing from the model's side; the physics may still need it in metals and polar materials, whose real force constants are long-ranged.

1.5 The displacement size. Eq. 5 is a finite difference, and it has two errors of opposite trend (Fig. 3):

- **Anharmonicity** grows with `u`. The cubic term of the energy biases a one-sided difference by an amount proportional to `u`; a central difference cancels it and leaves the quartic term, proportional to u^2.
- **Noise** in the forces is divided by `u`. A force error σ gives a force-constant error σ/u.



The finite-difference trade-off

Fig. 3 — Relative error of a force constant against the displacement, for a silicon-like stiffness (13 eV/Å^2) (`theory_figures.py`). Solid: force noise of 3 meV/Å, the size SIESTA's forces show in section 5.9; dashed: 0.1 meV/Å. One-sided (orange) and central (blue) differences.

The default 0.01 Å is a good compromise for DFT forces. A MACE model in float64 has almost no noise — its forces are the exact derivative of a smooth function — so for it any displacement from 0.001 to 0.03 Å gives the same result (section 5.7). Its weak point is not noise but a smooth, **systematic** error: the curvature of its energy surface may differ from SIESTA's.

1.6 Reading a band structure: degeneracies, symmetry labels, LO–TO.

- **Degeneracies.** Symmetry forces branches together. In silicon at Γ the three optical modes are one triply degenerate mode; at `X` the branches come in pairs. A degeneracy that is split by more than numerical noise means the structure has lost the symmetry you think it has (section 5.8).
- **Irreducible representations at** Γ (report `[4]`). phonopy labels each Γ mode by the irreducible representation of the point group it transforms as. The acoustic modes transform as a vector (`T1u` in `m-3m`). An optical mode of the same representation as a vector is **infrared-active**; the report marks it so. Silicon's optical mode is `T2g`: symmetric under inversion, so infrared-inactive but **Raman-active**. It is the 520 cm^{-1} (15.6 THz) Raman line every silicon wafer shows.
- **LO–TO splitting.** In a polar crystal (GaAs, AlP, oxides) the longitudinal optical mode at `q` $\to$ `0` carries a macroscopic electric field that raises its frequency above the transverse one. This needs Born effective charges and the dielectric constant (the non-analytic correction). A MACE model has neither, so **this tool shows no LO–TO splitting**: for polar materials the optical modes near Γ are those of the short-range force constants only. Silicon is non-polar, so nothing is missing here.

1.7 Sound velocities and elastic constants. Near Γ the acoustic branches are straight lines, ω = `v|q|`. Their slopes are the speeds of sound, and the speed of sound is set by the elastic constants (the Christoffel equation, ρ `v`^{2} = `C_eff` for each polarization) [3]. For a cubic crystal, along the three high-symmetry directions:

Direction	Mode (polarization)	ρ v^2
[100] (Γ→X)	LA	C11
[100]	TA (degenerate)	C44
[110] (Γ→K)	LA	(C11 + C12 + 2C44)/2
[110]	TA1 (along [1-10])	(C11 − C12)/2
[110]	TA2 (along [001])	C44
[111] (Γ→L)	LA	(C11 + 2C12 + 4C44)/3
[111]	TA (degenerate)	(C11 − C12 + C44)/3

`stb-mlphonons` takes each slope at `|q| = 0.01` Å^{-1} from every special point back towards Γ (report `[5]`). This gives the **relaxed** elastic constants, the ones that include the relaxation of the atoms inside the cell under strain — exactly what a stress-strain calculation with relaxed internal coordinates gives (`stb-mlelastic`, `7.4`). Section 5.3 compares the two.

1.8 Dynamical stability and imaginary modes. ω^2 is the eigenvalue; if it is negative, ω is imaginary, and by convention it is printed as a **negative frequency**. It means the energy *decreases* along that mode: the structure is not at a minimum. Three causes, in order of how often they occur:

1. **The structure is not at a minimum of this model.** It was relaxed with another method (SIESTA, another model), or not at all. The default pre-relax (positions only, `--fmax 0.05`) removes most of it.
2. **Numerical errors** in the force constants: noise (section 1.5), too small a supercell, a broken acoustic sum rule. Usually small imaginary acoustic frequencies near Γ.
3. **A real instability**: the crystal wants to distort — a soft mode near a phase transition, a charge-density wave, a high-symmetry structure that is not the ground state.

The report does not call `−0.00 THz` unstable: it uses a tolerance of −0.01 THz (shared with `5.4`), and leaves the three acoustic modes at Γ out of the mesh count. For case 3, `--freeze-unstable-mode` writes the structure displaced along the softest mode (`frozen_mode.fdf`): relax it and see whether the energy drops.

1.9 Thermodynamics of the harmonic crystal. Each phonon is a quantum harmonic oscillator of frequency ν. Its energy levels are `(n + ½) hν`, so even at 0 K it keeps a **zero-point energy** ½ hν. Summing over every branch `s` and every wavevector `q` on a uniform mesh (`--mesh`, default 20×20×20) gives, per primitive cell [3, 4]:

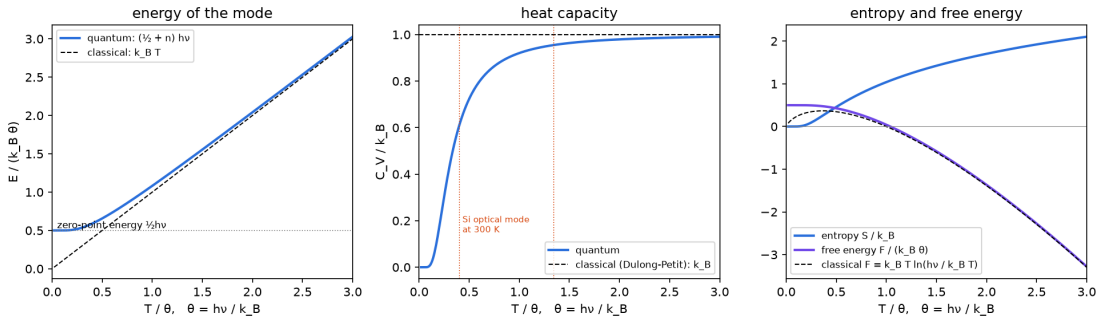
```
E_ZP  = Σ_qs ½ h ν_qs                                                    (7)
F_vib = Σ_qs [ ½ h ν_qs + k_B T ln(1 − e^(−hν_qs / k_B T)) ]
(8)
S     = Σ_qs k_B [ x_qs / (e^(x_qs) − 1) − ln(1 − e^(−x_qs)) ]        x_qs = h ν_qs / k_B T
(9)
C_V   = Σ_qs k_B x_qs² e^(x_qs) / (e^(x_qs) − 1)²
(10)
```

each divided by the number of q-points. Fig. 4 shows one mode:

- the **heat capacity** rises from 0 to the classical `k_B` (per mode) around `T` ≈ `hν/(3k_B)`;
- at high temperature `F_vib` and `S` reach their classical forms, but the zero-point energy never disappears from the energy.

A crystal of `N` atoms has `3N` modes, so the classical limit is `C_V = 3 k_B` per atom (**Dulong–Petit**). Silicon's phonons reach 15.5 THz, `hν/k_B = 745 K`: at room temperature its highest modes are still largely frozen,

and `C_V(300 K) = 2.41 k_B`, not 3. This is what classical MD (`7.2`) cannot reproduce, and what phonons give for free.



Thermodynamics of one quantum harmonic oscillator

Fig. 4 — One harmonic mode of frequency ν, against `T`/θ with θ = hν/`k_B` (`theory_figures.py`). The dotted lines in the middle panel mark silicon's optical mode at 300 and 1000 K.

Debye temperature. The report gives θ`_D = (h/k_B)` $\sqrt{(5/3\ \langle\nu^2\rangle)}$, from the second moment of the DOS (Eq. 11). It is the Debye model whose $\langle\nu^2\rangle$ matches, which makes the high-temperature heat capacity right. The calorimetric value (645 K for silicon) is fitted to the low-temperature heat capacity instead; the two differ a few percent.

$$\theta_D = (h / k_B)\ \sqrt{(5/3 \cdot \langle\nu^2\rangle)} \qquad \langle\nu^2\rangle = \int \nu^2\ g(\nu)\ d\nu\ /\ \int g(\nu)\ d\nu \tag{11}$$

Units. phonopy works in the primitive cell (2 atoms for silicon, even when the input is the 8-atom cubic cell). Its thermal properties are per mole of primitive cells; the report gives them that way **and per atom** (`kB/at`, `meV/at`), which is what you compare with experiment (J K^{-1} mol^{-1} of atoms = `kB/at × 8.314`).

1.10 Thermal expansion: the quasi-harmonic approximation. A purely harmonic crystal does not expand: its energy surface is symmetric about the minimum. Real crystals expand because their frequencies depend on volume. The **quasi-harmonic approximation** (QHA) keeps the harmonic form of Eq. 8 at each volume but lets the frequencies change with `V` [15, 16, 25]:

$$F(V,\ T) = E_static(V) + F_vib(V,\ T) \qquad V(T) = \text{the } V \text{ that minimizes } F(V,\ T) \text{ at } P = 0 \tag{12}$$

`--qha` computes the phonons at `--n-volumes` isotropically scaled volumes (`--strain-range`, ± % of the input volume), fits an equation of state (`--eos`, Vinet by default [17]) to `F(V, T)` at every temperature, and follows the minimum. That gives `V(T)`, α`_V(T) = (1/V) dV/dT`, `B_T(T)`, the heat capacity at constant pressure `C_P` and the Gibbs energy.

How much a mode feels the volume is measured by its **Grüneisen parameter**:

$$\gamma_s(q) = -d\ \ln\ \nu_s(q)\ /\ d\ \ln\ V \tag{13}$$

Most modes stiffen under compression (γ `> 0`). The thermal expansion follows from the **Grüneisen relation** [14, 25]:

```
α_V = γ C_V / (B_T V)                γ = Σ_qs γ_qs c_qs / Σ_qs c_qs   (c_qs: heat capacity of each
mode)   (14)
```

The expansion is the heat-capacity-weighted average of the mode Grüneisen parameters, divided by the stiffness. Two consequences that silicon shows (section 5.10):

- **Negative thermal expansion.** Silicon's transverse acoustic modes near the zone boundary have $\gamma \approx -1.4$: they *soften* under compression, a sign of the open diamond structure. Being the lowest-frequency modes, they are the first to be excited, so between about 20 and 120 K the weighted γ is negative and silicon **contracts** on heating [27]. Only quantum statistics can show this: in classical MD every mode is excited equally at any temperature.
- **Zero-point expansion.** Even at 0 K the zero-point energy depends on volume and pushes outward with a pressure `−dE_ZP/dV`. The 0 K volume is larger than the static minimum (report `[5]`, "Zero-point motion").

The QHA is good up to roughly half to two thirds of the melting temperature. Above that, the frequencies change with temperature at fixed volume (intrinsic anharmonicity), which only MD (`7.2`) or methods beyond this tool capture.

1.11 Phonons from a machine-learned potential. A MACE model writes the energy as a sum of atomic contributions, each a smooth learned function of the atom's neighbourhood within `r_max`; its forces are exact derivatives (`7.2`, section 1.2). For phonons that has three consequences:

- **No numerical noise.** The forces are smooth functions of the positions, computed in float64 here, so Eq. 5 is essentially exact. The noise problem of DFT forces (section 5.9) disappears.
- **Second derivatives are demanding.** A model is trained on energies, forces and stresses of distorted configurations: first derivatives. The force constants are the slope of the forces against the displacements, a quantity the training only constrains indirectly. A model with a small force error can still have the wrong curvature.
- **Foundation models are soft.** Universal models trained on near-equilibrium structures of the Materials Project systematically **underestimate the curvature** of the energy surface — energies, forces and phonon frequencies too low away from the minimum [20]. For silicon this is large (section 5.5): every MACE-MP-0 size puts the optical modes 25–35 % too low. Fine-tuning on SIESTA data of rattled, strained and MD-sampled cells fixes it.

What the tool checks, and what it cannot:

- The **model card** (report `[2]`) is printed in full: level of theory of the labels, trained range, errors against SIESTA — including a phonon comparison when `7.1.3` ran one —, recipe, stability, provenance. The structure is checked against the card's elements and ranges.
- With a fine-tuned model the **foundation model** runs the same calculation (report section `FINE-TUNED vs FOUNDATION`): the difference is what fine-tuning taught the model.
- No error bar on each frequency: one model, no committee. A SIESTA phonon calculation (`5.4`) on the same supercell, or `stb-mlffValidate --phonon-ref` (`7.1.3`), is the real test (section 5.9).

1.12 A calculation you can check by hand. The reference run of section 5.2 (silicon, 2×2×2, fine-tuned model) and the QHA of section 5.10 report numbers that can be predicted from each other:

- **Zero-point energy (Eq. 7).** Per atom it is `(3/2) ħ⟨ν⟩`. The report gives a mean frequency of 9.885 THz: `1.5 × 9.885 × 4.1357 = 61.32 meV/atom`. Reported: 61.32.

- **Heat capacity at high temperature.** Expanding Eq. 10 for x ≪ 1: C_V/3k_B ≈ 1 − ⟨x²⟩/12, and ⟨x²⟩ = (h/k_BT)² ⟨ν²⟩ = (3/5)(θ_D/T)² by Eq. 11. With θ_D = 666 K at 1000 K: 1 − 0.6 × 0.4436/12 = 0.978, so C_V = 2.933 k_B. Reported: 2.935.
- **Free energy.** F = U − TS, per atom at 300 K: U = 18.415 kJ/mol / 2 = 95.4 meV, TS = 300 × 2.321 × 0.08617 = 60.0 meV, so F = 35.4 meV. Reported: 35.43.
- **Bulk modulus from sound (section 1.7).** From the 4×4×4 velocities and ρ = 2266.8 kg/m³: C11 = ρ v_LA[100]² = 148.0 GPa, C44 = ρ v_TA[100]² = 65.1 GPa, C12 = 2ρ v_LA[110]² − C11 − 2C44 = 70.2 GPa, B = (C11 + 2C12)/3 = 96.1 GPa. The QHA's static equation of state, a different calculation with the same model, gives 93.5 GPa (3 % apart).
- **Grüneisen relation (Eq. 14).** At 300 K the QHA reports γ = 0.575, C_P ≈ C_V = 2.406 k_B per atom, B_T = 88.07 GPa, V = 20.695 Å³ per atom. α_V = 0.575 × 2.406 × 1.3806 × 10⁻²³ / (88.07 × 10⁹ × 20.695 × 10⁻³⁰) = 1.048 × 10⁻⁵ K⁻¹. Reported: 1.0465 × 10⁻⁵.
- **Zero-point expansion.** Across the scan the zero-point energy falls from 63.56 to 58.81 meV/atom as the volume grows from 19.75 to 21.40 Å³/atom: a pressure −dE_ZP/dV = 2.89 meV/Å³ = 0.46 GPa, which expands the crystal by ΔV/V = P/B₀ = 0.46/93.5 = 0.49 %. Reported: +0.516 %.

1.13 What this tool simplifies — explicitly.

- **Harmonic, or quasi-harmonic, only.** No phonon–phonon scattering: no lifetimes, linewidths or thermal conductivity; no temperature-dependent frequencies at fixed volume.
- **No LO–TO splitting** (section 1.6): no Born charges, no dielectric constant. For polar materials the optical modes near Γ are incomplete.
- **Isotropic volume scan.** `--qha` scales the whole cell by one factor. Right for cubic crystals; for a hexagonal or layered crystal the axes expand differently, which needs a scan per lattice parameter.
- **Positions relaxed, cell fixed.** The pre-relax moves atoms only, so the single-volume phonons are at the input volume. If the model's own equilibrium volume is different (the foundation model's is 1.35 % smaller here), the phonons are at a slightly strained lattice; `stb-mlrelax --relax-cell` fixes that first.
- **One model, no error bars** on the frequencies (section 1.11).
- **No electronic contribution** to the free energy and heat capacity: fine for an insulator like silicon, not for a metal at high temperature.

13.3.2.2 2. Libraries and external dependencies used

Library	Used for
phonopy [5, 16]	displacements, force constants, symmetrization, band structure, mesh, DOS/PDOS, thermal properties, irreducible representations, modulations (animation, mode freeze), the QHA (`PhonopyQHA`)
spglib [26]	symmetry of the crystal and its supercell (through phonopy)
ASE [22]	`Atoms`, the FCC/BCC/... Bravais lattice and its band path (`Cell.bandpath`), trajectory writers (xsf, pdb, xyz)
mace-torch [18, 19]	the MACE calculator, the foundation models (downloaded once, cached in `~/.cache/mace/`) and custom `.model` files
PyTorch	runs the model on `cpu` or `cuda` (`--device`)
pymatgen	structure conversion
NumPy	everything numerical
matplotlib	the PNG figures (always written), `--show`
gnuplot (optional)	renders the `.gplot` files written by `--save-gnuplot`
`stb.core`	`.fdf` reading (`structure_io`), the model card (`model_card`), the calculator, the pre-relax and device checks (`mace_relax`), displacements and force constants (`mace_phonons`), the band path (`phonons_pos`), library-noise capture (`cli`)

Install the `ml` extra: `pip install stb_suite[ml]`.

13.3.2.3 3. What `stb-mlphonons` does

3.1 One volume (the default).

1. **Read** the structure (`--file`, `.fdf`), find its dimensionality (vacuum-padded axes, from `--vacuum-gap`) and the supercell (`--dim`). A supercell thinner than 10 Å along a periodic axis, or repeated along a vacuum axis, gives a warning.
2. **Load the model**: a foundation model (`--model small|medium|large`) or a custom model with its card (`--custom-model` + `--custom-model-card`). The card must exist, be readable and match the model file's SHA-256; then it is printed in full and the structure is checked against it.
3. **Pre-relax** the atomic positions with the model (cell fixed, `--fmax`, default 0.05 eV/Å), unless `--no-relax`.
4. **Generate the displacements** (`--distance`) with phonopy, reduced by symmetry.
5. **Compute the forces** on every displaced supercell and on the undisplaced one, subtract the latter (Eq. 5), build the force constants, report the acoustic-sum-rule violation and impose the rule.
6. **Band structure** along the path of the lattice's Bravais type (`--band-points` per segment); frequencies and irreducible representations at the special points; stability on the band path and on the mesh; sound velocities.
7. **Mesh** (`--mesh`): DOS, projected DOS per species, thermal properties from `--tmin` to `--tmax` every `--tstep`.
8. Optionally **freeze** the softest imaginary mode (`--freeze-unstable-mode`, `--freeze-amplitude`) and **animate** a mode (`--animate-mode`: a band index, or no value for the first optical mode; `--animate-qpoint`, `--animate-dim`, `--animate-amplitude`, `--animate-frames`, `--animate-format`).
9. With a custom model, repeat 3–7 with the **foundation model** (`--model`) unless `--skip-foundation-comparison`, and compare.

The force constants are always saved, ASR-corrected, in `phonopy_disp.yaml`: `phonopy-load phonopy_disp.yaml` (or `phonopy.load` in Python) reuses them for anything phonopy can do.

3.2 `--qha`. Steps 1–5 and 7 at `--n-volumes` volumes (≥ 4: the equations of state have 4 parameters), scaled isotropically over `±` `--strain-range` `%` of the input volume, positions relaxed at each; then `PhonopyQHA` with the chosen `--eos`. phonopy's per-primitive-cell free energies are converted to the input cell, the cell whose volumes and energies are fitted (they were once mixed up; `--qha` then gave thermal expansions 4 times too small). Each volume's force constants are kept in `vol_NN/phonopy_disp.yaml`. The mode Grüneisen parameters at the special points come from a straight-line fit of `ln` ν against `ln V` over the scan.

3.3 `--check-convergence`. Steps 1–7 (quietly) in every supercell of `--convergence-dims` (default: `--dim` and `--dim + 1` on every periodic axis), compared with the largest along the band path (RMSE, largest difference) and at the special points; plus ZPE, `C_V` and `S` at 300 K. The smallest supercell within `--convergence-tol` (default 0.1 THz) of the largest is reported as converged.

3.4 The report, section by section. Sections are numbered in order; optional sections that do not apply are left out, so the numbers after them shift. The table numbers every section as if all were present; the reference run of section 5.2 has no `MODE FREEZE`, so its `MODE ANIMATION` is `[6]` and its `WARNINGS` `[11]`. Sections `[0]`–`[5]` never move.

One volume:

Section	Content
`[0] RUN METADATA`	date, software versions (stb, phonopy, mace-torch, torch, ase), device, input, mode, supercell, displacement, mesh and path density, temperatures, model, the comparison
`[1] STRUCTURE`	atoms, dimensionality, formula, cell, volume per atom and density, shortest distance, supercell size and its face-to-face widths
`[2] MODEL`	the **model card** in full (identity and SHA-256, level of theory, trained range, recipe and every error against SIESTA, stability and usage), or the foundation model's identity; the model's range (`r_max` × layers); load time
`[3] DISPLACEMENTS & FORCE CONSTANTS`	per model: input forces, pre-relax (forces before/after, steps, largest shift), space group and operations, number of displacements against the brute-force count, residual forces subtracted, acoustic-sum-rule violation, time; the primitive cell; the saved force constants
`[4] FREQUENCIES AT HIGH-SYMMETRY POINTS`	every branch at every special point, for each model; the Γ modes with their irreducible representations, in THz, cm^{-1} and meV, marked acoustic / optical / IR-active
`[5] BAND STRUCTURE & STABILITY`	Bravais lattice and path; per model: frequency range of the path, imaginary modes on the path and on the mesh (softest mode and where), gaps between branches, the sound-velocity table
`[6] MODE FREEZE`	`--freeze-unstable-mode` only: the softest mode, the achieved displacement, the structure written
`[7] MODE ANIMATION`	`--animate-mode` only: q-point, band and its frequency, frames, file
`[8] DENSITY OF STATES`	per model: highest and mean frequency, Debye temperature (Eq. 11), the DOS integral against `3n`, the projected DOS per species (states, share, mean frequency, upper edge)
`[9] THERMAL PROPERTIES`	per model: zero-point energy; F, S, C_V, U at 100–1000 K per primitive cell and per atom, and `C_V` as a fraction of Dulong–Petit
`[10] FINE-TUNED vs FOUNDATION`	with a custom model: frequency differences on the path and the mesh (acoustic, optical), highest frequency, `C_V`, `S`, `F` at 300 K and at `--tmax`
`[11] SUMMARY & FILES`	status, headline numbers, every file written, the optional outputs not asked for, wall time
`[12] WARNINGS`	every warning in one place, numbered; the count of library messages captured

`--qha`: `[0]`–`[2]` as above, then `[3] VOLUME SCAN` (per volume: strain, volume, lattice ratio, static energy, lowest and highest frequency, ZPE, `F_vib` at 300 K, time), `[4] EQUILIBRIUM FIT` (static E_0, V_0, B_0, B'), `[5] THERMAL EXPANSION SUMMARY` (zero-point expansion; `V`, a/a_0, α_V, α_L, B_T, C_P, γ, `G` at 0–2000 K; the values at 300 K and `--tmax`; the negative-expansion range), `[6] MODE GRUENEISEN PARAMETERS`, `[7] SUMMARY & FILES`, `[8] WARNINGS`.

`--check-convergence`: `[0]`–`[2]`, `[3] SUPERCELL CONVERGENCE CHECK` (per supercell: atoms, displacements, width, time, RMSE and largest difference against the largest, ZPE, `C_V` and `S` at 300 K, imaginary q-points; the special points per supercell; the verdict), `[4] SUMMARY & FILES`, `[5] WARNINGS`.

3.5 Files.

Written	When
`bands.png`, `dos.png`, `thermal.png` (both models overlaid)	always (one volume)
`phonopy_disp.yaml` (force constants)	always (one volume)
`qha.png`, `vol_NN/phonopy_disp.yaml`	`--qha`
`convergence.png`	`--check-convergence`
`bands.dat`, `bands_foundation.dat`, `dos.dat`, `thermal.dat` / `qha_thermal_expansion.dat` / `convergence.dat`	`--save-data`
`plot/*.dat`, `plot/*.gplot`, and `plot/*.pdf` when gnuplot is installed	`--save-gnuplot`

Written	When
`stb_mlphonons_report.txt`	`--save-report`
`mode_<band>.<xsf/pdb/xyz>`	`--animate-mode`
`frozen_mode.fdf`	`--freeze-unstable-mode`, when there is an imaginary mode
the figures on screen	`--show` (needs a display; without one it is a warning)

Everything goes into `-o/--output-dir` (default `mlphonons_out`).

3.6 Guard rails.

Situation	Behaviour
`--custom-model` without `--custom-model-card`, a missing or unreadable card, or a card whose SHA-256 does not match	clean error before anything runs
an element or a geometry outside the card's trained range	warning (`Model card check`), and run-time domain warnings from the calculator
`--qha` with fewer than 4 volumes; `--qha` with `--check-convergence`	clean error
`--animate-mode` or `--freeze-unstable-mode` with `--qha` or `--check-convergence`	clean error
`--convergence-dims` not a multiple of 3, or fewer than 2 supercells	clean error
`--animate-mode` with a negative index or an unknown word	clean error
`--animate-mode` index out of range, or the first optical mode of a 1-atom primitive cell (there is none)	warning, animation skipped
`--no-relax`	warning: expect spurious modes if the structure is not at a minimum
a supercell thinner than 10 Å along a periodic axis; `--dim` repeating a vacuum axis	warning
imaginary modes on the mesh	warning naming the softest mode and the three usual causes
QHA: an imaginary mode at some volume, the static minimum at the edge of the scan, the model's V_0 more than 1 % from the input, `V(T)` leaving the scan or in its top 15 %	warning
`--device cuda` with no usable GPU; a missing input file	clean error
`--show` without a display	warning; the PNGs are written anyway
an isolated molecule (every axis vacuum)	the band structure is skipped with a note; DOS and thermal properties are computed

13.3.2.4 4. Running it both ways

Command line:

```
stb-mlphonons --file si8.fdf --dim 2 2 2 \
              --custom-model model/mlff_model_compiled.model --custom-model-card
              model/mlff_model_card.json \
              --animate-mode --save-data --save-gnuplot --save-report
stb-mlphonons --file si8.fdf --custom-model ... --custom-model-card ... --qha --n-volumes 7
--strain-range 4 --tmax 1500
```

```
stb-mlphonons --file si8.fdf --custom-model ... --custom-model-card ... --check-convergence
--convergence-dims 1 1 1 2 2 2 3 3 3
```

Interactive menu: run `stb-suite` and type `7.3`. It asks, in this order:

1. the structure;
2. a custom model path, or Enter for a foundation model and then its size. With a custom model it asks for the card and offers `mlff_model_card.json` next to the model as the default;
3. the device (cpu/cuda; it checks the GPU);
4. the supercell (Enter: `2 2 2`) and the output folder;
5. the mode: 1 single volume, 2 QHA (then the number of volumes, the strain range and the equation of state), 3 convergence (then the supercells);
6. single volume only: whether to compare with the foundation model (custom model only), whether to freeze an unstable mode, whether to animate one (band — Enter for the first optical mode —, q-point, modulation supercell, amplitude, frames, format);
7. one question per optional output (show, gnuplot, data, report).

The menu builds the same command line and calls `stb-mlphonons`; the guided script (Case 11) drives it with piped answers.

13.3.2.5 5. Worked example: bulk silicon

5.1 Set-up. The structure, `si8.fdf`: the 8-atom conventional diamond cell at SIESTA's relaxed lattice constant, 5.48024 Å (PBE, DZP; experiment 5.4310 Å at 300 K — PBE is 0.9 % too large). `si8_rattled.fdf` is the same cell with every atom moved at random by about 0.03 Å (section 5.8).

The model, `../7.2-mlmd/model/` (shared with the `7.2` example, not copied):

- `best_model/` of the `7.1` silicon run: round 2, seed 125, fine-tuned from MACE-MP-0 medium on 57 SIESTA single points (8 and 64 atoms: rattled, strained, 1400 K MD frames);
- level of theory: PBE, DZP, PAO.EnergyShift 0.01 Ry, Mesh.Cutoff 300 Ry, k × a ≈ 32 Å;
- its card: test-set force error 28.7 meV/Å, energy 1.1 meV/atom, stress 0.18 GPa; phonons against SIESTA 1.10 THz RMSE (acoustic 1.51, optical 0.37) — section 5.9 explains why the acoustic number says more about SIESTA than about the model.

Every calculation ran on the CPU (one desktop, 12 cores); `make_reference.sh` lists them, and section 6 explains how to regenerate them. Wall times: 14 s for the reference run with both models, 36 s for 4×4×4, 26 s for a 7-volume QHA, 49 s for the 1×1×1–4×4×4 convergence check.

Experimental references: phonon frequencies from inelastic neutron scattering at 296 K [8]; elastic constants at 298 K [9]; heat capacity and entropy at 298 K [12, 13]; thermal expansion [10, 27].

5.2 The reference run: bands against experiment.

```
stb-mlphonons --file si8.fdf --custom-model ../7.2-mlmd/model/mlff_model_compiled.model \
              --custom-model-card ../7.2-mlmd/model/mlff_model_card.json --animate-mode \
              --save-data --save-gnuplot --save-report
```

Report section `[3]` (`reference_results/base/stb_mlphonons_report.txt`):

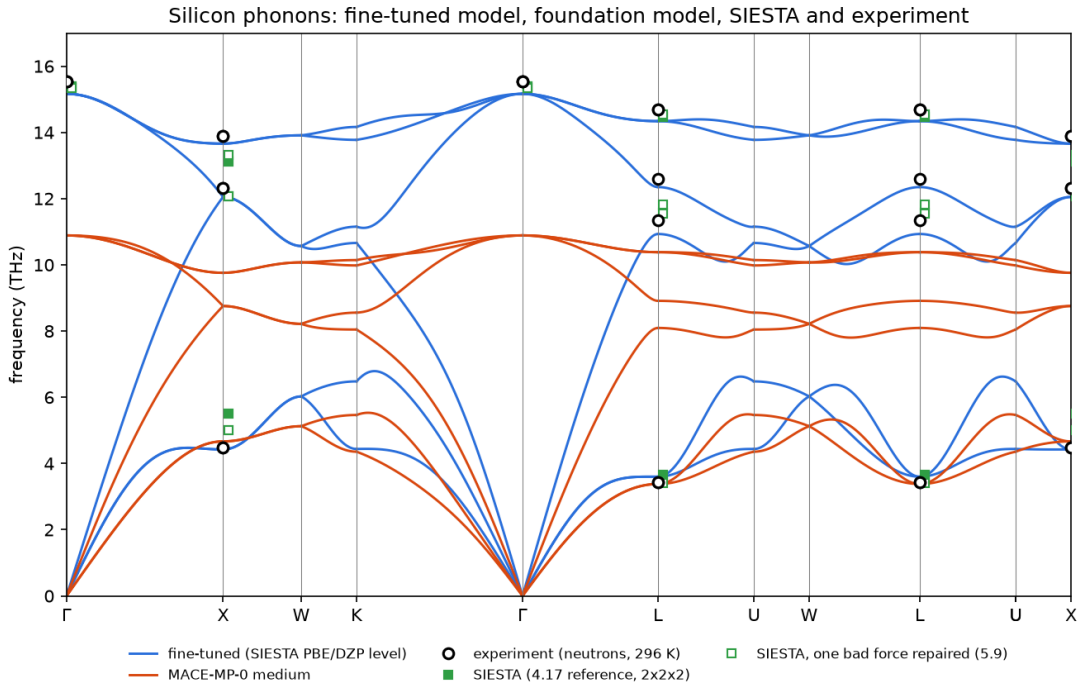
```
Input forces      : max |F| 0.0000 eV/Ang
Pre-relax         : max |F| 0.0000 eV/Ang after 0 step(s) (converged, --fmax 0.05); largest atomic
shift 0.0000 Ang
```

```
Symmetry          : Fd-3m (227) (point group m-3m, 1536 operations in the supercell)
Displacements     : 1 of 0.01 Ang (vs. 384 without symmetry: 99.7 % fewer evaluations)
Residual forces   : max 8.10e-15 eV/Ang per atom in the undisplaced supercell -- subtracted from
every displacement's
Acoustic sum rule : largest violation 6.91e-14 eV/Ang^2 before symmetrization (imposed afterwards)
Primitive cell    : 2 atoms (phonopy, spglib); supercell 64 atoms
```

The model puts the atoms exactly at their sites (zero force: the structure is at its minimum), the symmetry reduces the job to one displaced supercell, and the acoustic sum rule holds to 10^{-13} before it is even imposed — the forces are noise-free (section 1.11).

Section `[4]` against experiment [8], THz:

q	experiment	fine-tuned	MACE-MP-0 medium
Γ (optical)	15.53	15.16	10.89
X	4.49 / 12.32 / 13.90	4.42 / 12.05 / 13.66	4.67 / 8.76 / 9.76
L	3.43 / 11.35 / 12.60 / 14.68	3.60 / 10.93 / 12.35 / 14.34	3.38 / 8.09 / 8.91 / 10.38



Silicon phonon bands against experiment and SIESTA

Fig. 5 — The band structure of both models (`reference_results/base/`), the experimental frequencies at Γ, X, L (circles), and SIESTA's frequencies from the `7.1` reference calculation, as it is and with one bad force repaired (section 5.9) (`results_figures.py`).

- **The fine-tuned model** is within −3.7 to +5.0 % of experiment over the 15 measured modes (mean −1.2 %, RMS 2.8 %). Most modes are 1–3 % low: PBE's lattice is 0.9 % too large and its bonds slightly too soft, and the model reproduces PBE. The Γ optical mode, 15.16 THz = 505.8 cm^{-1}, is the Raman line (520 cm^{-1} measured).
- **At Γ**: the acoustic triplet `T1u` at zero, the optical triplet `T2g` (Raman-active, infrared-inactive), as section 1.6 says.
- **No imaginary mode** on the path or on the 8000-point mesh; softest non-zero mode 0.37 THz, the acoustic branch next to Γ.

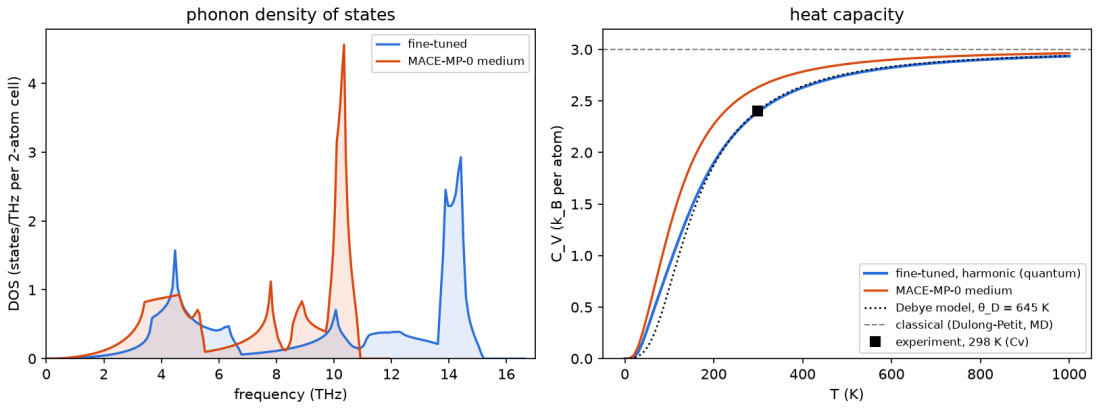
- **The foundation model** gets the acoustic modes at the zone boundary about right and every optical mode 25–35 % too low (section 5.5).

5.3 Sound velocities and elastic constants. Section `[5]` gives the slopes of the acoustic branches. From the 2×2×2 and 4×4×4 runs (`base`, `base_444`), m/s, with the experimental values computed from the measured elastic constants [9] (section 1.7):

	Γ→X [100] TA / LA	Γ→K [110] TA1 / LA	Γ→L [111] TA / LA
fine-tuned, 2×2×2	5345 / 8199	4239 / 8812	4632 / 9020
fine-tuned, 4×4×4	5361 / 8081	4122 / 8768	4566 / 8994
MACE-MP-0 medium, 2×2×2	3681 / 6506	3410 / 6647	3501 / 6698
experiment	5843 / 8433	4673 / 9134	5093 / 9356

- **Supercell.** The slopes change by up to 3 % from 2×2×2 to 4×4×4: the acoustic branches near Γ are an interpolation (section 1.4), so sound velocities need a larger supercell than the zone-boundary frequencies.
- **Elastic constants.** With the model's density (2266.8 kg/m^3) the 4×4×4 slopes give `C11 = 148.0`, `C12 = 70.2`, `C44 = 65.1 GPa`, and the check `C11 − C12 = 77.8` against $2\rho v^2$`_TA1[110] = 77.0` closes (section 1.12).
- **Cross-check with a different method.** `stb-mlelastic` (`7.4`) fits the same model's stresses under small strains, with the atoms relaxed: `C11 = 144.8`, `C12 = 68.0`, `C44 = 65.0 GPa`, `B = 93.6 GPa` (`reference_results/elastic/`). Phonon slopes and stress-strain agree within 2–3 %. That is a consistency check of the phonon machinery, not of the physics: both measure the same model.
- **Against experiment** (`C11 = 165.6`, `C12 = 63.9`, `C44 = 79.5 GPa`): the model's `C11` is 11 % low and `C44` 18 % low. Part of this is PBE at its own, larger lattice constant; part may be the model — the training set had only 6 strained cells. The foundation model's constants are much lower still (`C11 = 103`, `C44 = 29 GPa` from `stb-mlelastic`).

5.4 Density of states and thermal properties.



Phonon DOS and heat capacity

Fig. 6 — Left: the DOS of both models. Right: the heat capacity per atom, with the Debye model at the calorimetric θ`_D = 645 K`, the classical limit, and experiment at 298 K (`results_figures.py`*).*

Section `DENSITY OF STATES`: highest frequency 15.16 THz, mean 9.885 THz, Debye temperature 666 K (second moment, Eq. 11), the DOS integrates to 5.98 of 6 states. The two sharp peaks are the flat regions of the band structure: the TA modes near the zone boundary (4–4.5 THz) and the TO band (14–14.5 THz).

Section `THERMAL PROPERTIES`, per atom:

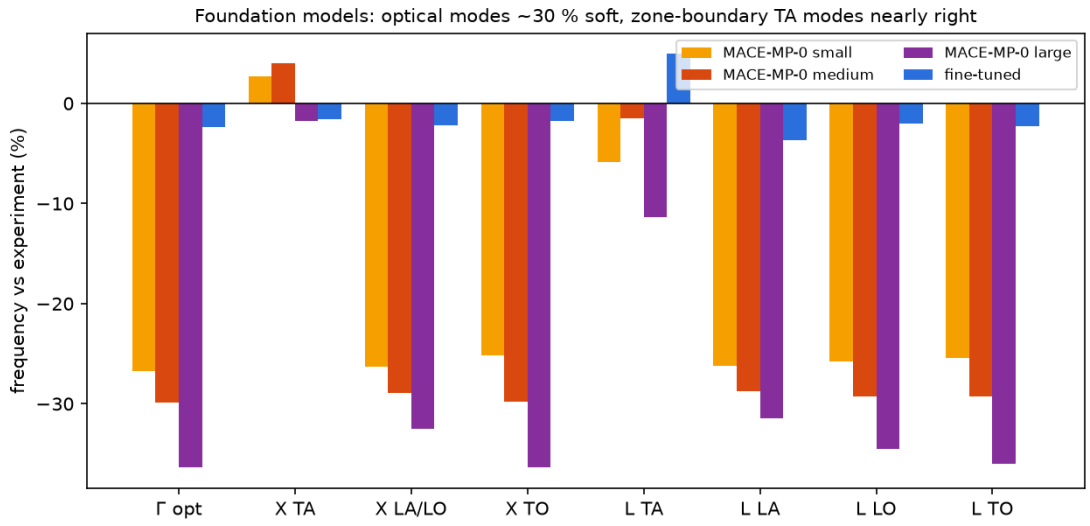
T (K)	C_V (k_B)	C_V (J K^{-1} mol^{-1})	S (k_B)	S (J K^{-1} mol^{-1})	F (meV)
100	0.904	7.51	0.483	4.01	60.04
300	2.395	19.91	2.321	19.30	35.43
500	2.753	22.89	3.647	30.33	−16.71
1000	2.935	24.40	5.632	46.82	−221.08

- **Heat capacity at 300 K**: 19.91 J K^{-1} mol^{-1}. The measured `C_P` is 20.00 [13]; for a solid `C_P − C_V = α_V² B V T` is only 0.02 here, so `C_V ≈ 19.98`. The model is within 0.4 %. Classical MD (`7.2`) gives 24.9 (3 `k_B`), 25 % too high.
- **Entropy at 300 K**: 19.30 against 18.81 measured [13], 2.6 % high: the model's frequencies are slightly low, and lower frequencies mean more entropy.
- **Debye temperature**: 666 K from the DOS against 645 K calorimetric [12]; the two are defined differently (section 1.9), and the heat-capacity curve follows the 645 K Debye model closely (Fig. 6).
- **Zero-point energy**: 61.3 meV/atom, 1.3 % of silicon's cohesive energy (4.6 eV/atom). It shifts relative energies of polymorphs and defects, and it expands the lattice (section 5.10).

The foundation model, with frequencies 30 % low, has the right high-temperature limit but reaches it too early (Fig. 6), and its entropy at 300 K is 2.95 `k_B` instead of 2.32 (section `FINE-TUNED vs FOUNDATION`): **a phonon error is a free-energy error** — 24 meV/atom at 300 K here, enough to reverse the order of two polymorphs.

5.5 The foundation models are soft. The same calculation with the three MACE-MP-0 sizes (`foundation_small/medium/large`), THz:

model	Γ optical	X TA	X TO	L TA	L TO	v_LA[100] (m/s)	θ_D (K)
MACE-MP-0 small	11.38	4.61	10.40	3.23	10.95	6311	517
MACE-MP-0 medium	10.89	4.67	9.76	3.38	10.38	6506	495
MACE-MP-0 large	9.88	4.41	8.85	3.04	9.39	5978	457
fine-tuned	15.16	4.42	13.66	3.60	14.34	8199	666
experiment	15.53	4.49	13.90	3.43	14.68	8433	645



Foundation models against experiment

Fig. 7 — Frequency error against experiment at the measured modes, for the three foundation models and the fine-tuned one (`results_figures.py`).

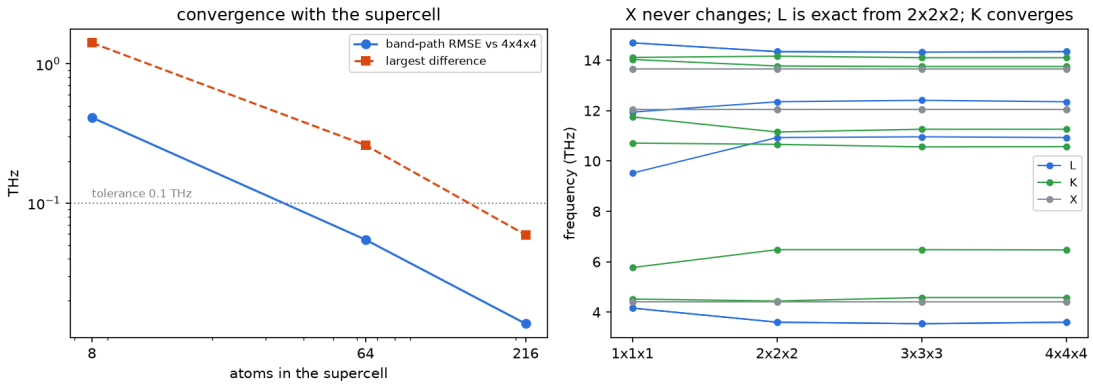
- Every optical mode is 25–36 % low in every size; **the largest model is the softest**. A bigger foundation model is not a better one for a given material.
- The transverse acoustic modes at `X` and `L` are nearly right (−11 to +4 %). Those modes bend bonds, the optical ones stretch them: the foundation models underestimate the bond-stretching curvature most.
- The foundation models are trained on PBE, the same functional as our SIESTA labels. The softening is a property of the models — the systematic softening of universal potentials trained mostly on near-equilibrium structures [20] — not a difference of level of theory. The card records it in another form: the raw foundation model's force error on the SIESTA training configurations is 448 meV/Å, against 46 for the fine-tuned model.
- An independent check confirms the tool: a hand-made finite difference of the medium model on the 2-atom cell gives 10.89 THz at Γ, the same as the report.

Fine-tuning on 57 SIESTA configurations brought every mode to within a few percent of experiment. For phonons this is the strongest argument for `7.1`.

5.6 The supercell size.

```
stb-mlphonons --file si8.fdf --custom-model ... --custom-model-card ... --check-convergence \
              --convergence-dims 1 1 1 2 2 2 3 3 3 4 4 4
```

supercell	atoms	band-path RMSE vs 4×4×4 (THz)	largest difference (THz)	ZPE (meV/atom)	C_V(300 K) (k_B/atom)	S(300 K) (k_B/atom)
1×1×1	8	0.412	1.415	60.95	2.3938	2.4025
2×2×2	64	0.055	0.260	61.32	2.3948	2.3209
3×3×3	216	0.014	0.059	61.31	2.3948	2.3235
4×4×4	512	—	—	61.31	2.3948	2.3236



Supercell convergence

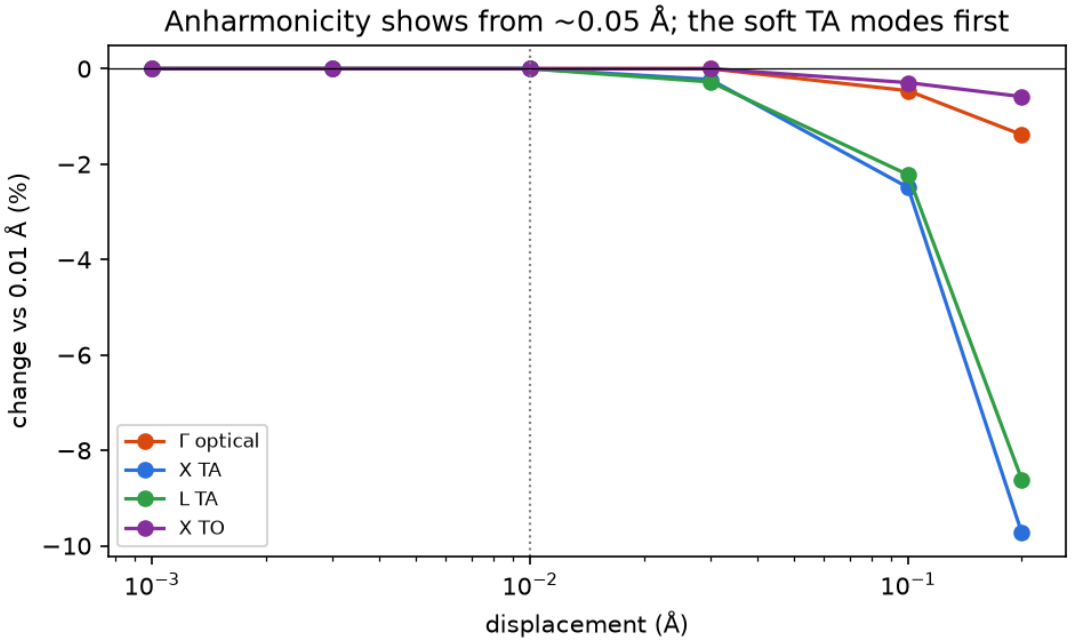
Fig. 8 — Left: band-path difference against the 4×4×4 supercell. Right: frequencies at `L`, `K` and `X` per supercell (`results_figures.py`).

- **`X` never changes**: it is commensurate with the 8-atom cubic cell (section 1.4). Neither does Γ.
- **`L` is exact from 2×2×2** and the 3×3×3 supercell, which is not commensurate with it, is slightly *worse* there (3.54 against 3.60 THz) — commensurability, not size, decides the special points.
- **`K` converges smoothly** (it is commensurate with none of these).
- The 1×1×1 cell is wrong by up to 1.4 THz at `L`; 2×2×2 is within the 0.1 THz tolerance (RMSE 0.055); 3×3×3 is within 0.015.
- **Thermal properties converge faster than the bands**: `C_V` at 300 K is the same to 0.05 % in every supercell, and the entropy to 3.5 % even in the 1×1×1 cell. They average over the whole zone, where the errors partly cancel.

The model's force constants reach 12 Å (two layers × 6 Å); the 4×4×4 supercell (22 Å wide) holds all of them. For a DFT calculation the same test would cost a 512-atom SIESTA run (section 1); here it costs 25 seconds, which is why the tool offers it.

5.7 The displacement size. Six runs, `--distance` 0.001 to 0.2 Å (`dist_*`), THz:

u (Å)	Γ optical	X TA	L TA	X TO
0.001	15.16	4.42	3.60	13.66
0.003	15.16	4.42	3.60	13.66
0.01	15.16	4.42	3.60	13.66
0.03	15.16	4.41	3.59	13.66
0.1	15.09	4.31	3.52	13.62
0.2	14.95	3.99	3.29	13.58



Frequencies against the displacement

Fig. 9 — Change of each frequency relative to the 0.01 Å run (`results_figures.py`).

- **From 0.001 to 0.03 Å nothing changes**, to the last printed digit. Even 0.001 Å, where a DFT force set would be dominated by noise, works: the model's forces are smooth (section 1.5).
- **From 0.1 Å the anharmonicity shows**, and the soft transverse acoustic modes feel it first: −2.5 % at 0.1 Å, −10 % at 0.2 Å for X-TA, against −1.4 % for the optical mode. The same modes have

the most negative Grüneisen parameters (section 5.10): both are signs of a strongly anharmonic bond-bending potential.
- The default 0.01 Å is safe for any model. With a model, prefer it to the larger values sometimes used to fight DFT noise.

5.8 A structure that is not at a minimum. `si8_rattled.fdf` has every atom moved by about 0.03 Å; the largest force is 0.85 eV/Å. Three runs:

pre-relax	max |F| before → after (eV/Å)	steps	space group	displacements	Γ optical triplet (THz)	imaginary q
none (`--no-relax`)	0.850	—	P1	48	14.08 … 15.81	0
default (`--fmax 0.05`)	0.850 → 0.034	16	P1	48	15.14 … 15.18	0
tight (`--fmax 0.001`)	0.850 → 0.0008	46	P1	48	15.16 … 15.16	0

Three lessons, one of them unexpected:

- **No imaginary mode, even without the pre-relax.** Subtracting the residual forces (Eq. 5) makes the force constants the curvature *at* the given geometry, and for a small rattle of a stiff crystal that curvature is still positive. So a phonon spectrum free of imaginary modes does **not** prove the structure is relaxed. A larger distortion, or a structure near a saddle point, would give imaginary modes.
- **But the frequencies are wrong.** Without the pre-relax, silicon's triply degenerate optical mode is spread over 1.7 THz, and every degeneracy is split: these are the phonons of a disordered snapshot, not of the crystal. The default pre-relax brings the triplet within 0.04 THz; the tight one restores it exactly.
- **The symmetry does not come back.** phonopy finds symmetry with a tolerance of 10^{-5} Å, and the relaxed atoms are still ~10^{-4} Å from their ideal sites (`F/k` ≈ `0.001/13` Å). The structure stays P1: 48 displacements instead of 1, the 8-atom cell treated as primitive (24 branches folded into the band structure), 87 s instead of 14. For a structure that should have symmetry, **symmetrize it first** (`stb-unitcell`, `4.7`, with a tolerance) and the cost and the band structure come back.

`--freeze-unstable-mode` was on in the unrelaxed run; with no imaginary mode it reports "nothing to freeze".

5.9 Against SIESTA: one bad force and its imaginary modes. The `7.1` example computed silicon's phonons with SIESTA itself (`5.4`: `stb-phononsCreate`, 2×2×2 supercell, one displacement of 0.01 Å, the training level of theory; `reference_results/siesta_phonons/`). `siesta_compare.py` evaluates both models on the *same* displaced supercell, so the three force sets differ only in where the forces come from.

	Γ opt	X TA	X TO	L TA	lowest f (THz)	imaginary q (12^3 mesh)	RMSE vs SIESTA, acoustic / optical (THz)
SIESTA	15.38	5.50	13.13	3.66	−2.93	148 / 864	—
SIESTA, one atom repaired	15.33	5.01	13.33	3.41	0.37	0 / 864	1.27 / 0.23
fine-tuned	15.16	4.42	13.66	3.60	0.61	0 / 864	1.50 / 0.35
MACE-MP-0 medium	10.89	4.67	9.76	3.38	0.46	0 / 864	1.94 / 3.61
experiment	15.53	4.49	13.90	3.43	—	0	—

SIESTA's phonons are unstable — transverse acoustic modes down to −2.9 THz near Γ on 148 of 864 q-points, which real silicon does not have — while the model trained on SIESTA's own data is stable and closer to experiment. The model's card reports "phonons vs SIESTA: 1.10 THz RMSE, acoustic 1.51": most of that is this. Tracking it down:

1. **Forces by distance** from the displaced atom (RMS per component, meV/Å):

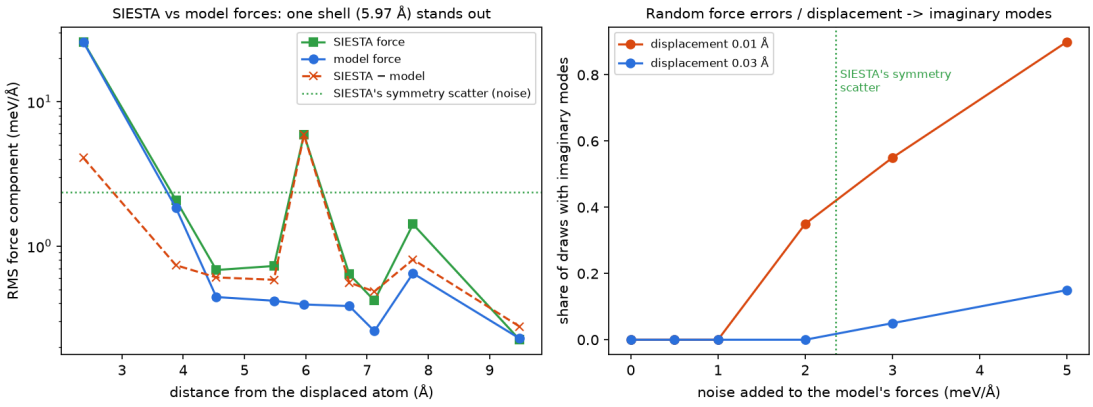
r (Å)	atoms	SIESTA	model	difference
0	1	75.66	76.65	1.00
2.37	4	25.98	25.83	4.07
3.88	12	2.08	1.84	0.74
4.54	12	0.68	0.44	0.61
5.48	3	0.73	0.42	0.58
5.97	**12**	**5.92**	**0.39**	**5.90**
6.71	12	0.64	0.38	0.56

The model reproduces SIESTA's large forces within 1–4 meV/Å. Beyond the second shell the true forces are below 2 meV/Å — and one shell, at 5.97 Å, is 15 times larger in SIESTA.

2. **Symmetry.** The displacement along `x` keeps 4 operations of the site symmetry, so symmetry-equivalent atoms must feel symmetry-equivalent forces. Within that shell, **one atom (#28) carries (1.9, 30.0, −18.0) meV/Å**; its 11 partners carry 1.3 meV/Å (median). The model's force sets obey the symmetry to 10^{-13} meV/Å.
3. **The net force.** SIESTA's own forces (`disp-001.FA`) sum to (8.1, 40.1, −9.2) meV/Å — a 41 meV/Å violation of Newton's third law, which the `Tot` line of `siesta.out` shows directly; the undisplaced supercell's sum to 0.6. phonopy removes the mean force from each force set, spreading the error over all 64 atoms (0.6 meV/Å each, as large as the real forces at long range).
4. **Repair.** Replacing atom 28's force by the value its symmetry partners imply removes **every** imaginary mode (table, "one atom repaired"); L-TA moves from 3.66 to 3.41 THz (experiment 3.43).

How sensitive is a single 0.01 Å displacement? `siesta_compare.py` adds Gaussian noise of a given size to the model's clean forces, 20 random draws each:

displacement	noise 1 meV/Å	2 meV/Å	3 meV/Å	5 meV/Å
0.01 Å: draws with imaginary modes	0 %	35 %	55 %	90 %
0.01 Å: X-TA (THz)	4.36 ± 0.68	4.20 ± 1.83	2.79 ± 2.80	1.28 ± 4.26
0.03 Å: draws with imaginary modes	0 %	0 %	5 %	15 %
0.03 Å: X-TA (THz)	4.44 ± 0.14	4.41 ± 0.31	4.33 ± 0.55	4.31 ± 0.71



SIESTA's forces and the noise experiment

Fig. 10 — Left: forces by distance from the displaced atom. Right: share of noisy force sets with imaginary modes, at two displacements (`results_figures.py`*).*

Random errors of 2 meV/Å on every force already make a 0.01 Å calculation unstable a third of the time; a single error of 35 meV/Å does it every time. Three times the displacement makes it three times more robust (Fig. 3). Where the bad force comes from was not traced: the SCF converged normally (8 iterations, `DM.Tolerance 1e-5`), and an egg-box force from the real-space mesh (Mesh.Cutoff 300 Ry) would sit on the displaced atom, not 6 Å away. What it teaches holds anyway:

- **Check a DFT phonon force set before trusting its imaginary modes**: the net force (`Tot` in `siesta.out`) should be close to that of the undisplaced supercell; symmetry-equivalent atoms should carry equivalent forces.
- **For DFT, 0.01 Å is fragile**: use 0.02–0.03 Å, tight SCF tolerances, and ± displacements.
- **A model trained on DFT can be cleaner than the DFT phonons it is compared with**: training on many configurations averages out errors like this one. Its own error is different — smooth and systematic, like the 11–18 % low elastic constants of section 5.3 — and the remaining difference to the repaired SIESTA phonons (X-TA 4.42 against 5.01 THz; experiment 4.49) cannot be settled with one SIESTA displacement. A repeated SIESTA calculation with ± 0.03 Å displacements would.

5.10 Thermal expansion: the quasi-harmonic approximation.

```
stb-mlphonons --file si8.fdf --custom-model ... --custom-model-card ... --qha --n-volumes 7
--strain-range 4 --tmax 1500
```

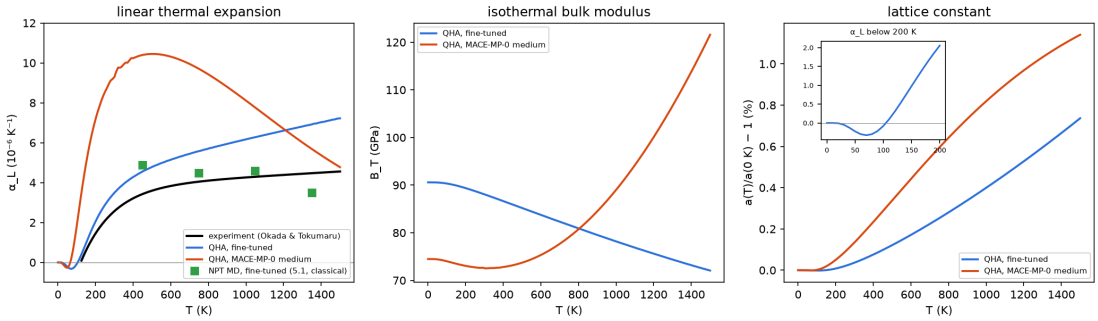
Report `[3]`, the scan (per atom; `reference_results/qha/`):

```
 Strain%   V (Ang^3)   V/atom   a/a_in    E-Emin    f_min    f_max      ZPE F_vib(300K)   time
   -4.00    158.0045  19.7506  0.98648    10.069    0.361   15.748    63.56        37.84    4.0
   -1.33    162.3935  20.2992  0.99554     1.034    0.366   15.360    62.10        36.33    3.1
    0.00    164.5880  20.5735  1.00000     0.000    0.366   15.157    61.32        35.43    3.2
    1.33    166.7825  20.8478  1.00442     1.104    0.366   14.947    60.49        34.43    3.2
    4.00    171.1716  21.3964  1.01316     9.184    0.364   14.523    58.81        32.27    2.7
```

(7 volumes; 4 shown.) Then the results, with the foundation model for comparison (`qha_foundation_v0`: its cell first relaxed with `stb-mlrelax --relax-cell`, so that the scan is centred on its own minimum):

	V_0 (Å^3/atom)	B_0 (GPa)	zero-point ΔV	α_L(300 K) (10^{-6} K^{-1})	α_L(1000 K)	B_T 300 / 1000 K (GPa)	γ (300 K)
fine-tuned	20.566	93.5	+0.52 %	3.49	6.17	88.1 / 78.2	0.58
MACE-MP-0 medium	20.296	74.7	+0.87 %	9.29	8.11	72.6 / 89.0	1.14
experiment	20.02 (300 K)	97.8	—	2.57	4.26	97.8 (298 K)	0.46

The experimental γ follows from Eq. 14: `3 × 2.57 × 10⁻⁶ × 97.8 GPa × 20.02 Å³ / 2.40 k_B = 0.46`.



Thermal expansion, bulk modulus and lattice constant

Fig. 11 — QHA of both models against experiment [10] and the NPT molecular dynamics of the `7.2` *example (same fine-tuned model, classical nuclei). Inset: the fine-tuned model's negative expansion below 100 K (*`results_figures.py`*).*

The fine-tuned model.

- **V_0 and B_0.** The static minimum is 0.04 % from SIESTA's relaxed cell. B_0 = 93.5 GPa is 4 % below experiment, but 5 % above SIESTA's own 89.0 GPa at this level of theory: the `E(V)` of the perfect cell, `7.4` section 5.11. (The 94.4 of the `7.1` benchmark is fitted over its strained test cells, which are also rattled.)
- **Negative thermal expansion below 100 K**, minimum -0.32×10^{-6} K^{-1} at 70 K. Real silicon contracts between about 20 and 120 K [27]. The cause is in report `[6]`: the transverse acoustic modes have negative Grüneisen parameters.

mode	γ, fine-tuned	γ, experiment [11]
Γ optical	+1.01	$\approx$ +1.0
X TA	−1.74	$\approx$ −1.4
L TA	−1.04	$\approx$ −1.3

- **The expansion is 36–45 % above experiment** (3.49 against 2.57×10^{-6} K^{-1} at 300 K). Eq. 14 says where it comes from: the model's γ is 26 % higher (0.58 against 0.46) and its `B_T` 10 % lower (88 against 98 GPa). Both are properties of the model's energy surface at this level of theory, not of the method.
- **QHA against MD with the same model** (green squares in Fig. 11): from 600 to 1200 K classical MD gives α_L $\approx 4.5 \times 10^{-6}$, QHA 5.2–6.6. MD is classical but includes every anharmonic term; above θ_D it is the better method, and the difference is what QHA leaves out (section 1.10). Below about 400 K it is the other way round: MD's classical nuclei overestimate the expansion (`7.2`, section 5.7), and only the QHA reproduces the negative expansion.
- **Hand checks** (section 1.12): the Grüneisen relation reproduces α_V at 300 K to 0.2 %, and the zero-point pressure the zero-point expansion to 5 %.

The foundation model gives an unphysical QHA: α_L peaks at 500 K and then *falls*, and `B_T` *rises* from 73 to 122 GPa between 300 and 1500 K. No real solid stiffens on heating like that. The scan explains it: across ± 4 % its highest frequency drops by 0.29, 0.30, 0.30, 0.29, 0.25, 0.20 THz per step (`qha_foundation_v0`, report `[3]`), so its γ falls with volume, and that volume dependence adds a large positive curvature to `F_vib(V)` that grows with T. The fine-tuned model's steps are uniform (0.19–0.21 THz). Centring the scan on the foundation model's own minimum (`qha_foundation_v0`) gives the same result as using the fine-tuned model's volumes (`qha_foundation`), with two warnings:

```
1. QHA: the model's static equilibrium volume is -1.35 % from the input structure. The scan is
centred on the input;
   relax the cell with the same model first for a scan centred on V0.
2. QHA: at 1500 K V(T) is in the top 15 % of the scanned range; widen --strain-range to keep the fit
away from the edge.
```

So the QHA tests a model harder than the phonons at one volume: it needs the *volume derivative* of the force constants, a third derivative of the energy. Before trusting a QHA, look at the shape of α(T) and B_T(T): α should rise and level off, B_T should fall.

5.11 A vibration as a movie. The reference run animated the first optical mode at Γ (--animate-mode with no value; report section MODE ANIMATION):

```
Mode              : first optical mode (band 3; bands 0-2 are acoustic)
q-point           : [0.0, 0.0, 0.0]
Band index        : 3 (15.1639 THz)
Frames            : 30 (xsf format)
[OK] Saved ./mode_3.xsf
```

reference_results/base/mode_3.xsf opens in VESTA, XCrySDen, OVITO or ase gui: one period in 30 frames, the two atoms of the primitive cell moving against each other along a bond direction, amplitude 0.5 Å (exaggerated to be seen). Bands 0–2 at Γ are rigid translations of the whole crystal and make dull movies, which is why they are not the default. A zone-boundary mode needs a modulation supercell commensurate with its wavevector: X-TA of the FCC cell, --animate-qpoint 0.5 0 0.5 --animate-dim 2 2 2 --animate-mode 0.

5.12 What to take from the example.

- A model fine-tuned on 57 SIESTA configurations gives silicon's phonons within a few percent of experiment, its heat capacity within 0.5 % and its entropy within 3 %, in seconds. The foundation model alone is 30 % too soft in the optical modes; the largest foundation model is the softest.
- Check the supercell with --check-convergence: special points depend on commensurability, sound velocities and interpolated regions on size; thermal properties converge first.
- With a model, the displacement size does not matter between 0.001 and 0.03 Å; above 0.05 Å anharmonicity shows.
- No imaginary mode does not mean relaxed, and relaxed does not mean symmetric: pre-relax, and symmetrize a structure that should have symmetry.
- A DFT force set can be wrong in one atom; the net force and the symmetry of the forces show it. The model trained on that DFT gave the cleaner phonons.
- The QHA is a harder test than the phonons themselves: a foundation model can give plausible bands and an absurd B_T(T). The fine-tuned model reproduces the negative expansion of silicon and overestimates α by 36 %, the combination of its γ and B.

13.3.2.6 6. Reproducing the reference data

reference_results/ holds what make_reference.sh copies out of its work folder: each case's report and data files (--save-data), the gnuplot scripts of the base, conv and qha cases, base/phonopy_disp.yaml (the force constants of the reference run) and base/mode_3.xsf; elastic/ (stb-mlelastic); siesta_phonons/ (the SIESTA reference of 7.1: phonopy_disp.yaml, FORCE_SETS, FORCE_SETS.raw, the .FA force files of the displaced and undisplaced supercells and the SIESTA input); siesta_compare.json.

```
source ~/.anaconda3/etc/profile.d/conda.sh && conda activate stb   # or any environment with
stb_suite[ml]
bash make_reference.sh                  # every case into ./reference_run/, then collect into
reference_results/
python3 results_figures.py              # the tables of section 5 and figures/*.png (numpy +
matplotlib only)
python3 theory_figures.py               # the figures of section 1
```

Step	Cost
`make_reference.sh`	about 10 minutes on a 12-core CPU (`DEVICE=cuda` for a GPU); each case is skipped when its report exists
`siesta_compare.py` (called by it)	about 1 minute: two models on two displacements, 240 noisy phonon calculations
the SIESTA reference itself	not rerun: it is the `7.1` example's phonon calculation (SIESTA 5.4.2, 6 cores, 2 min 15 s per supercell)

The foundation models are downloaded on first use. The numbers are deterministic on the CPU; on a GPU they agree to the printed digits.

13.3.2.7 7. Known, deliberate limitations

- **Harmonic and quasi-harmonic only** (section 1.13): no lifetimes, no thermal conductivity, no temperature-dependent frequencies at fixed volume. For strongly anharmonic crystals use MD (`7.2`) and its vibrational spectrum (`2.18`, `stb-aimdAnalysis`).
- **No LO–TO splitting** for polar materials: MACE has no Born charges.
- **Isotropic QHA**: one scaling factor for the whole cell.
- **The phonons are those of the model**, not of SIESTA. The card's numbers tell you how far to trust it; one SIESTA phonon calculation on the same supercell, done carefully (section 5.9), is the real test.
- **Symmetry is detected with phonopy's tight default tolerance**: a slightly distorted input stays low-symmetry (section 5.8).

13.3.2.8 8. Step-by-step: phonons of your own material

1. **Choose the model.**
   - For a quick, qualitative look, the foundation model (`--model medium`) needs nothing — but expect frequencies too low, possibly by tens of percent (section 5.5).
   - For numbers you will use, train a model with `7.1` on configurations around your structure: rattled cells (they teach the curvature), strained cells (they teach the volume dependence the QHA needs), and MD frames. Validate it with `7.1.3`, ideally with `--phonon-ref` against one SIESTA phonon calculation (`5.4`). Keep the model and its card together.
2. **Relax the structure with the same model.** `stb-mlrelax --relax-cell` with the same `--custom-model`: the phonons must be computed at the model's own minimum, not at SIESTA's or at experiment's. If the structure should have symmetry, symmetrize it (`stb-unitcell`) afterwards.
3. **Check the supercell**: `--check-convergence` with two or three sizes. Take the smallest within 0.1 THz; use a larger one for sound velocities.
4. **Run the single-volume calculation** with `--save-report --save-data`, and read the report from the bottom: `WARNINGS` (the last section) first, then `[5]` (imaginary modes, and whether they could be any of the three causes of section 1.8), then `[4]`, `DENSITY OF STATES`, `THERMAL PROPERTIES`. With a custom model, `FINE-TUNED vs FOUNDATION` says how much the fine-tuning matters.

5. **For thermal expansion, run `--qha`** with 7 or more volumes over ± 3–5 %, and `--tmax` well below the melting point. Check `[4]` (is V_0 inside the scan?), the warnings, and the shapes of α`(T)` and `B_T(T)`.
6. **Compare with SIESTA where it matters**: one `5.4` calculation on the same supercell. If its imaginary modes and the model's disagree, check the DFT force set (section 5.9) before blaming either.

13.3.2.9 9. Files in this folder

File	What it is
`README.md`	this tutorial
`example_7.3.sh`	the guided walk-through (CPU, about 3 minutes; `DEVICE=cuda` for a GPU)
`si8.fdf`	bulk silicon, 8-atom conventional cell at SIESTA's relaxed lattice constant (5.48024 Å)
`si8_rattled.fdf`	the same with every atom moved by about 0.03 Å (section 5.8)
`make_reference.sh`	regenerates `reference_results/` (every case of section 5, CPU, about 10 minutes)
`siesta_compare.py`	the SIESTA phonons of `7.1` against both models on the same supercell, the force analysis and the noise experiment → `siesta_compare.json` (needs the `ml` extra)
`results_figures.py`	reads `reference_results/`, prints the tables of section 5, draws `figures/*.png` (numpy + matplotlib only)
`theory_figures.py`	draws the figures of section 1
`figures/`	every figure of this README
`reference_results/`	reports, data and the SIESTA reference (section 6)
`../7.2-mlmd/model/`	the fine-tuned model and its card, shared with `7.2`

13.3.2.10 10. Running the script

```
cd examples/7.3-mlphonons
bash example_7.3.sh               # CPU, ~3 min; press Enter between cases
DEVICE=cuda bash example_7.3.sh   # on a GPU
```

Each case writes into `output/<case>/`, which is wiped at every start. Cases 1 and 9 download the MACE-MP-0 medium model on first use. Case 12 needs no MACE: it only reads `reference_results/`.

13.3.2.11 11. Flag reference

`stb-mlphonons` — `--file FILE` (`.fdf`, required) · `--model {small,medium,large}` (default medium) · `--custom-model PATH` + `--custom-model-card JSON` (always both) · `--skip-foundation-comparison` · `--dim NA NB NC` (default `2 2 2`) · `--distance Å` (default 0.01) · `--no-relax` · `--fmax eV/Å` (pre-relax, default 0.05) · `--vacuum-gap Å` (default 10) · `--band-points N` (per segment, default 101) · `--mesh NA NB NC` (default `20 20 20`) · `--tmin`, `--tmax`, `--tstep` K (default 0, 1000, 10) · `--qha` · `--n-volumes N` ($\geq$ 4, default 5) · `--strain-range %` (default 3) · `--eos {vinet,murnaghan,birch_murnaghan}` (default vinet) · `--check-convergence` · `--convergence-dims N...` (triples) · `--convergence-tol THz` (default 0.1) · `--freeze-unstable-mode` · `--freeze-amplitude Å` (default 0.05) · `--animate-mode [BAND|optical]` (no value: the first optical mode) · `--animate-qpoint QA QB QC` (default Γ) · `--animate-dim NA NB NC` (default `1 1 1`) · `--animate-amplitude Å` (default 0.5) · `--animate-frames N` (default 30) · `--animate-format {xsf,pdb,xyz}` · `--device {cpu,cuda}` · `-o/--output-dir DIR` (default `mlphonons_out`) · `--show` · `--save-gnuplot` · `--save-data` · `--save-report` · `--no-intro` · `-v`.

13.3.2.12 12. What's next

- **`5.4` (Phonons workflow)** computes the same phonons with SIESTA: the reference to validate a model against, and the method for a material you have no model for. Section 5.9 shows what to check in its force sets.
- **`7.1` (MLFF workflow)** trains the model. Its Stage 3 `--phonon-ref` compares the committee's phonons with a finished `5.4` calculation q-point by q-point.
- **`7.2` (`stb-mlmd`)** gives the full anharmonic, classical picture: compare its NPT thermal expansion with the QHA (section 5.10), and its heat capacity with the quantum one of section 5.4.
- **`7.4` (`stb-mlelastic`)** gives the elastic constants by stress-strain: compare them with the sound velocities (section 5.3).
- **`5.15` (equation of state)** fits SIESTA's static `E(V)`, the curve the QHA adds `F_vib` to.

13.3.2.13 13. References

1. M. Born, K. Huang, *Dynamical Theory of Crystal Lattices* (Oxford University Press, 1954).
2. A. A. Maradudin, E. W. Montroll, G. H. Weiss, I. P. Ipatova, *Theory of Lattice Dynamics in the Harmonic Approximation*, 2nd ed. (Academic Press, 1971).
3. N. W. Ashcroft, N. D. Mermin, *Solid State Physics* (Saunders, 1976), chapters 22, 23 and 25.
4. M. T. Dove, *Introduction to Lattice Dynamics* (Cambridge University Press, 1993).
5. A. Togo, I. Tanaka, "First principles phonon calculations in materials science", *Scr. Mater.* **108**, 1 (2015).
6. K. Parlinski, Z. Q. Li, Y. Kawazoe, "First-principles determination of the soft mode in cubic ZrO_2", *Phys. Rev. Lett.* **78**, 4063 (1997).
7. S. Baroni, S. de Gironcoli, A. Dal Corso, P. Giannozzi, "Phonons and related crystal properties from density-functional perturbation theory", *Rev. Mod. Phys.* **73**, 515 (2001).
8. G. Nilsson, G. Nelin, "Study of the homology between silicon and germanium by thermal-neutron spectrometry", *Phys. Rev. B* **6**, 3777 (1972).
9. J. J. Hall, "Electronic effects in the elastic constants of n-type silicon", *Phys. Rev.* **161**, 756 (1967).
10. Y. Okada, Y. Tokumaru, "Precise determination of lattice parameter and thermal expansion coefficient of silicon between 300 and 1500 K", *J. Appl. Phys.* **56**, 314 (1984).
11. B. A. Weinstein, G. J. Piermarini, "Raman scattering and phonon dispersion in Si and GaP at very high pressure", *Phys. Rev. B* **12**, 1172 (1975).
12. P. Flubacher, A. J. Leadbetter, J. A. Morrison, "The heat capacity of pure silicon and germanium and properties of their vibrational frequency spectra", *Philos. Mag.* **4**, 273 (1959).
13. M. W. Chase Jr., *NIST-JANAF Thermochemical Tables*, 4th ed., *J. Phys. Chem. Ref. Data, Monograph* **9** (1998).
14. E. Grüneisen, "Theorie des festen Zustandes einatomiger Elemente", *Ann. Phys.* **344**, 257 (1912).
15. S. Baroni, P. Giannozzi, E. Isaev, "Density-functional perturbation theory for quasi-harmonic calculations", *Rev. Mineral. Geochem.* **71**, 39 (2010).
16. A. Togo, L. Chaput, I. Tanaka, G. Hug, "First-principles phonon calculations of thermal expansion in Ti_3SiC_2, Ti_3AlC_2, and Ti_3GeC_2", *Phys. Rev. B* **81**, 174301 (2010).
17. P. Vinet, J. R. Smith, J. Ferrante, J. H. Rose, "Temperature effects on the universal equation of state of solids", *Phys. Rev. B* **35**, 1945 (1987).
18. I. Batatia, D. P. Kovács, G. N. C. Simm, C. Ortner, G. Csányi, "MACE: Higher order equivariant message passing neural networks for fast and accurate force fields", *Adv. Neural Inf. Process. Syst.* **35**, 11423 (2022).
19. I. Batatia et al., "A foundation model for atomistic materials chemistry", arXiv:2401.00096 (2023).
20. B. Deng, Y. Choi, P. Zhong, J. Riebesell, S. Anand, Z. Li, K. Jun, K. A. Persson, G. Ceder, "Systematic softening in universal machine learning interatomic potentials", *npj Comput. Mater.* **11**, 9 (2025).

21. J. M. Soler et al., "The SIESTA method for ab initio order-N materials simulation", *J. Phys.: Condens. Matter* **14**, 2745 (2002).
22. A. H. Larsen et al., "The atomic simulation environment — a Python library for working with atoms", *J. Phys.: Condens. Matter* **29**, 273002 (2017).
23. A. García et al., "Siesta: Recent developments and applications", *J. Chem. Phys.* **152**, 204108 (2020).
24. A. Togo, L. Chaput, T. Tadano, I. Tanaka, "Implementation strategies in phonopy and phono3py", *J. Phys.: Condens. Matter* **35**, 353001 (2023).
25. D. C. Wallace, *Thermodynamics of Crystals* (Wiley, 1972).
26. A. Togo, K. Shinohara, I. Tanaka, "Spglib: a software library for crystal symmetry search", *Sci. Technol. Adv. Mater.: Methods* **4**, 2384822 (2024).
27. K. G. Lyon, G. L. Salinger, C. A. Swenson, G. K. White, "Linear thermal expansion measurements on silicon from 6 to 340 K", *J. Appl. Phys.* **48**, 865 (1977).

13.3.3 Command-line reference

13.3.3.1 `stb-mlphonons`

Standalone phonon calculation driven entirely by a MACE potential – the MACE-MP-0 foundation model, or a custom model fine-tuned on your own SIESTA data via `stb-mlffAnalysis` (`--custom-model`). One shot: displacements, force constants, band structure, DOS (total + per-species), and thermal properties – or, with `--qha`, a quasi-harmonic volume scan (thermal expansion, bulk modulus vs. T). No SIESTA and no separate analysis tool needed. A fast heuristic preview, not a substitute for the real SIESTA-based `stb-phononsCreate/stb-phononsPos` workflow – ML potentials are markedly less reliable for phonons (imaginary modes especially) than DFT.

Synopsis.

```
usage: stb-mlphonons [-h] --file FILE [--model {small,medium,large}]
                     [--custom-model PATH] [--custom-model-card JSON]
                     [--skip-foundation-comparison] [--dim NA NB NC]
                     [--distance DISTANCE] [--no-relax] [--fmax FMAX]
                     [--vacuum-gap VACUUM_GAP] [--band-points BAND_POINTS]
                     [--mesh NA NB NC] [--tmin TMIN] [--tmax TMAX] [--tstep TSTEP]
                     [--qha] [--n-volumes N_VOLUMES] [--strain-range STRAIN_RANGE]
                     [--eos {vinet,murnaghan,birch_murnaghan}] [--check-convergence]
                     [--convergence-dims N [N ...]]
                     [--convergence-tol CONVERGENCE_TOL] [--freeze-unstable-mode]
                     [--freeze-amplitude FREEZE_AMPLITUDE]
                     [--animate-mode [BAND_INDEX]] [--animate-qpoint QA QB QC]
                     [--animate-dim NA NB NC] [--animate-amplitude ANIMATE_AMPLITUDE]
                     [--animate-frames ANIMATE_FRAMES]
                     [--animate-format {pdb,xsf,xyz}] [--device {cpu,cuda}]
                     [-o OUTPUT_DIR] [--show] [--save-gnuplot] [--save-data]
                     [--save-report] [--no-intro] [-v]
```

Options

`--file FILE`
Input structure (.fdf). required

`--model {small,medium,large}`
MACE-MP-0 model size (default: medium). Ignored if `--custom-model` is given. choices: small, medium, large

`--custom-model PATH`
: Use a custom MACE model file instead of the MACE-MP-0 foundation potential – e.g. one fine-tuned via `stb-mlffAnalysis`. Overrides `--model`.

`--custom-model-card JSON`
: Model card of the `--custom-model` file: the `mlff_model_card.json` that `stb-mlffAnalysis` wrote next to it (training data, level of theory, accuracy, energy scale). Required with `--custom-model` – the card is checked against the model file and used to warn when a structure is outside what the model was trained on.

`--skip-foundation-comparison`
: If `--custom-model` is given, also evaluate the same phonon pipeline with the raw (non-fine-tuned) foundation model (`--model`) and overlay both on the bands/DOS/thermal plots, to show whether fine-tuning actually changed the phonon prediction. On by default; pass this to skip it (`--qha` never does this comparison, regardless of this flag). switch, off by default

`--dim NA NB NC`
: Supercell dimensions for the finite-displacement method (default: 2 2 2). type: int

`--distance DISTANCE`
: Displacement distance, Angstrom (default: 0.01). type: float

`--no-relax`
: Skip the MACE pre-relax (positions only) before generating displacements. On by default – a phonon calculation assumes ~zero net force at the reference geometry; skipping this on an unrelaxed structure is a common cause of spurious imaginary modes. switch

`--fmax FMAX`
: Force convergence threshold for the pre-relax, eV/Ang (default: 0.05). type: float

`--vacuum-gap VACUUM_GAP`
: Gap (Ang) used to detect vacuum-padded axes (default: 10.0). type: float

`--band-points BAND_POINTS`
: q-points per band-structure segment (default: 101). type: int

`--mesh NA NB NC`
: q-point mesh for DOS/thermal properties (default: 20 20 20). type: int

`--tmin TMIN`
: Min temperature, K (default: 0). type: float

`--tmax TMAX`
: Max temperature, K (default: 1000). type: float

`--tstep TSTEP`
: Temperature step, K (default: 10). type: float

`--qha`
: Quasi-harmonic approximation instead of a single-volume calculation: scans `--n-volumes` isotropically-scaled volumes and fits an equation of state (thermal expansion, bulk modulus, Cp and Grueneisen parameters vs. T, mode Grueneisen parameters). No foundation-model comparison in this mode. Off by default. switch, off by default

`--n-volumes N_VOLUMES`
: Number of volume points for `--qha`, >= 4 (default: 5). type: int

`--strain-range STRAIN_RANGE`
: Volume range around equilibrium for `--qha`, +/- % (default: 3.0). type: float

`--eos {vinet,murnaghan,birch_murnaghan}`
: Equation of state for `--qha` (default: vinet). choices: vinet, murnaghan, birch_murnaghan

`--check-convergence`

Instead of a single-volume calculation, run the same pipeline at multiple supercell sizes (--convergence-dims) and compare them with the largest one along the band path and at the special points (RMSE, largest difference), plus heat capacity, entropy and ZPE – MACE is cheap enough to check --dim convergence directly, unlike DFT. Not supported together with --qha. Off by default. switch, off by default

--convergence-dims N [N ...]

Flat list of supercell sizes for --check-convergence, given as consecutive NA NB NC triples (e.g. 1 1 1 2 2 2 3 3 3 tests 1x1x1, 2x2x2, and 3x3x3) – must be a multiple of 3 values. Default: --dim itself and --dim+1 on every non-vacuum axis (two points). type: int

--convergence-tol CONVERGENCE_TOL

Band-path RMSE (THz) against the largest supercell under which a smaller one counts as converged (default: 0.1). type: float

--freeze-unstable-mode

If the phonon mesh has a genuine imaginary (unstable) mode, write a new structure displaced along that mode's eigenvector – a common trick to escape a saddle point and find the true minimum. Same feature stb-phononsPos already has for the SIESTA path. Not supported together with --qha/--check-convergence. Off by default. switch, off by default

--freeze-amplitude FREEZE_AMPLITUDE

Max atomic displacement (Angstrom) for --freeze-unstable-mode (default: 0.05). type: float

--animate-mode BAND_INDEX

Animate a normal mode's vibration and write it as a multi-frame trajectory (xsf/pdb/xyz, same convention as stb-ani2traj/stb-mlmd) for viewing in OVITO/VMD – BAND_INDEX selects which mode (0 = lowest frequency) at --animate-qpoint; with no value, or 'optical', the first optical mode (band 3: bands 0-2 are acoustic, at Gamma a rigid translation). Not supported together with --qha/--check-convergence. Off by default.

--animate-qpoint QA QB QC

Fractional q-point to animate a mode at (default: 0 0 0, Gamma). type: float

--animate-dim NA NB NC

Supercell size used to build the animated modulation – must make --animate-qpoint commensurate (q x dim ~ integer vector); the default 1 1 1 only works exactly at Gamma (default: 1 1 1). type: int

--animate-amplitude ANIMATE_AMPLITUDE

Max atomic displacement for the animation, Angstrom – deliberately larger than --freeze-amplitude since this is for visualization, not a physical perturbation (default: 0.5). type: float

--animate-frames ANIMATE_FRAMES

Number of frames in one full vibration cycle (default: 30). type: int

--animate-format {pdb,xsf,xyz}

Output trajectory format for --animate-mode, same convention as stb-ani2traj/stb-mlmd (default: xsf).choices: pdb, xsf, xyz

--device {cpu,cuda}

Device to run the model on (default: cpu). choices: cpu, cuda

-o, --output-dir OUTPUT_DIR

Output directory for all plots/data (default: mlphonons_out).

--show

Open the figures on screen at the end (needs a display; without one it is a warning, not an error). The PNGs are written either way. switch, off by default

--save-gnuplot

Write every figure as .dat + .gplot into <output-dir>/plot/ and run gnuplot there (one PDF per figure) when it is installed. Off by default. switch, off by default

`--save-data`
Also write the raw numeric data (.dat) behind each plot. Off by default. switch, off by default

`--save-report`
Also persist the report to `stb_mlphonons_report.txt`. Off by default. switch, off by default

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Example usage:
  stb-mlphonons --file structure.fdf --dim 2 2 2
  stb-mlphonons --file structure.fdf --custom-model mlff_model.model --custom-model-card mlff_model_card.json
 --dim 3 3 3
  stb-mlphonons --file structure.fdf --qha --n-volumes 7 --strain-range 5 --save-gnuplot
  stb-mlphonons --file structure.fdf --check-convergence --convergence-dims 1 1 1 2 2 2 3 3 3 --show
```

13.4 ML Elastic Constants

At a glance

Menu code: 7.4 **Command:** `stb-mlelastic`
Guided example: `examples/7.4-mlelastic/`
Summary: The elastic stiffness tensor from a MACE potential by the stress–strain method – the foundation model or your own fine-tuned one, always with its model card: the constants the crystal's symmetry allows, Born stability, Voigt/Reuss/Hill moduli and the usual ratios, clamped- vs relaxed-ion constants, Christoffel sound velocities, the Debye temperature and the directional moduli; in-plane constants in N/m for a 2D sheet; with `-check-convergence`, the constants against the strain amplitude.

13.4.1 Overview

`stb-mlelastic` computes the elastic constants of a crystal with a MACE potential [15]: the MACE-MP-0 foundation model [16] (`-model`) or a model fine-tuned on your own SIESTA calculations with the MLFF workflow (7.1; `-custom-model` with `-custom-model-card`). It is the stress–strain method of the SIESTA workflow 5.2 [105] with the SIESTA runs replaced by the model: the cell is relaxed to the model's own minimum, strained along each of the six Voigt directions at a few amplitudes up to ± 1 %, the atoms are relaxed inside every strained cell, and the slopes of stress against strain are the columns of C. The strain matrices and the fit are the ones **`stb-elasticAnalysis`** uses, so a strain called `xy` means the same deformation in both tools. A MACE stress costs milliseconds and has none of the real-space mesh noise of a SIESTA stress, so the whole tensor — with the clamped-ion scan and, for a fine-tuned model, the same calculation with the foundation model — takes well under a minute (Fig. 13.3).

Elastic constants in eight equations

Hooke's law in Voigt form [64]. Stress is linear in strain; with the engineering shear strains $\varepsilon_{4,5,6} = 2\varepsilon_{yz}, 2\varepsilon_{xz}, 2\varepsilon_{xy}$ the 21 independent constants form a symmetric 6×6 matrix, the second derivative of the energy:

$$\sigma_m = \sum_n C_{mn}\varepsilon_n, \qquad \frac{E}{V} = \frac{1}{2}\sum_{mn} C_{mn}\varepsilon_m\varepsilon_n, \qquad C_{mn} = \frac{\partial \sigma_m}{\partial \varepsilon_n} \text{ (a linear fit per direction).}$$

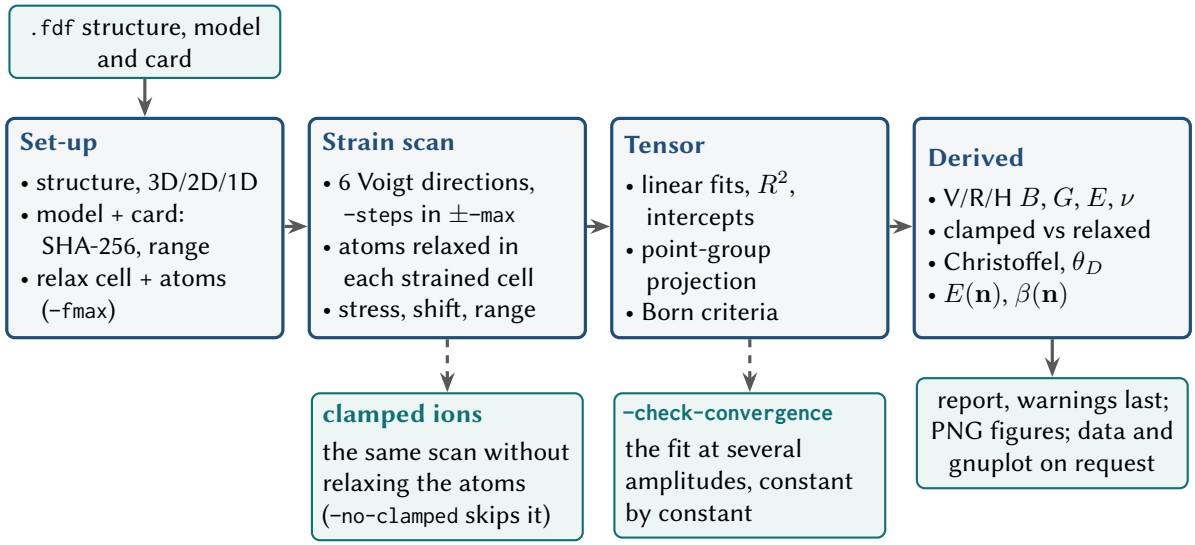


Figure 13.3: What one **stb-mlelastic** run does. With a fine-tuned model the foundation model repeats the set-up, scan and fit about its own minimum; -check-convergence repeats scan and fit at several strain amplitudes.

Internal relaxation [106, 107]. At fixed strain the atoms can still move along an internal coordinate u; minimizing over it gives the relaxed-ion constant from the clamped-ion one:

$$\frac{E}{V} = \tfrac{1}{2}C^0\varepsilon^2 - D\,\varepsilon u + \tfrac{1}{2}Ku^2 \;\Rightarrow\; u^* = \frac{D}{K}\,\varepsilon, \qquad C = C^0 - \frac{D^2}{K}.$$

Symmetry (Neumann's principle). The tensor is projected on the point group G found by spglib, in the cell's own Cartesian frame: $C_{\mathrm{sym}} = |G|^{-1}\sum_{R\in G} R \star C$. **Stability** [69, 70]: all eigenvalues of C positive; for a cubic crystal $C_{11} - C_{12} > 0$, $C_{11} + 2C_{12} > 0$, $C_{44} > 0$. **Polycrystal moduli** [65–67]. Uniform strain (Voigt) and uniform stress (Reuss) bracket the isotropic moduli; Hill takes the mean:

$$B_V = \frac{(C_{11}+C_{22}+C_{33}) + 2(C_{12}+C_{13}+C_{23})}{9}, \quad G_V = \frac{(C_{11}+C_{22}+C_{33}) - (C_{12}+C_{13}+C_{23}) + 3(C_{44}+C$$

B_R, G_R the same with the compliance $S = C^{-1}$; $E = 9BG/(3B{+}G), \nu = (3B{-}2G)/[2(3B{+}G)]$. **Sound** [108, 109]. The Christoffel equation gives three velocities per direction $\mathbf{n}$; the isotropic velocities give the elastic Debye temperature:

$$\Big(\sum_{jl} C_{ijkl}n_j n_l - \rho v^2\delta_{ik}\Big)p_k = 0, \qquad \frac{3}{v_m^3} = \frac{1}{v_L^3} + \frac{2}{v_T^3}, \qquad \theta_D = \frac{\hbar}{k_B}\,(6\pi^2 n)^{1/3}\,v_m.$$

Directions. $1/E(\mathbf{n}) = \sum s_{ijkl}n_i n_j n_k n_l$ and $\beta(\mathbf{n}) = \sum s_{ijkk}n_i n_j$; a 2D sheet reports its in-plane constants per area ($\sigma \times L_z$, N/m).

Before you trust ML elastic constants

- **Foundation models are soft** [104]: for silicon every MACE-MP-0 size puts C_{44} and G about 60 % below experiment (Section 13.4.2.5). Fine-tune (**7.1**) with **strained cells** in the training set (-volume-strains, Stage 4 -sampling-method strain).
- **Compare the model with its own level of theory first.** On silicon, SIESTA at the training settings showed that the model's shear constants are 12–18 % low and its C_{12} 15–21 % high, while

PBE/DZP itself is 6–12 % below experiment (Section 13.4.2.5). The card's stress error (0.18 GPa) did not reveal it.

- **Relax the atoms tightly**: the internal relaxation is a third of silicon's C_{44}, and `-fmax` 0.02 eV/Å leaves it 3 % low with scattered points; the default is 0.001.
- **Check the strain amplitude** with `-check-convergence`: the constants drift with it (third-order response), shear constants most.
- **Start from a symmetric minimum**: without the pre-relax a distorted structure keeps a residual stress and no symmetry is found to check the tensor against.
- **Ratios amplify errors**: a 20 % low G turned silicon "ductile" (Pugh) and "metallic" (Cauchy).
- **0 K, static, linear**: no temperature, no third-order constants; one model gives no error bars.

13.4.2 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/7.4-mlelastic/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

`stb-mlelastic` computes the **elastic constants** of a crystal — its stiffness tensor `C` — with a MACE machine-learned potential: either the MACE-MP-0 foundation model, or a model you fine-tuned on your own SIESTA calculations with the MLFF workflow (`7.1`), which always comes with its **model card**. It uses the stress-strain method: relax the cell, strain it along each of the six independent directions at a few amplitudes up to ±1 %, relax the atoms inside every strained cell, and fit stress against strain. From `C` it reports:

- the constants the crystal's symmetry allows, and how far the raw fit is from that symmetry;
- mechanical stability (Born criteria);
- polycrystal bulk, shear and Young's moduli and Poisson's ratio (Voigt, Reuss, Hill), and the usual ratios;
- how much the relaxation of the atoms inside the cell softens each constant (clamped vs relaxed ions);
- sound velocities along [100], [110], [111] and the Debye temperature;
- the directional Young's modulus and linear compressibility;
- for a 2D sheet, the in-plane constants in N/m;
- with a fine-tuned model, the same with the raw foundation model, constant by constant.

A second mode, `--check-convergence`, repeats the fit at several strain amplitudes. Plots, gnuplot files and data files are written only when you ask for them.

Why a potential for elastic constants? The SIESTA route (`5.2`) needs one calculation per strained cell — a few dozen for a full tensor, each a relaxation of the atoms — and fights the real-space mesh ("egg-box") noise of SIESTA's stress. A MACE stress costs milliseconds and has no such noise: the whole tensor, with the foundation-model comparison and the clamped-ion scan, takes half a minute. The price is the usual one: the model is only as good as what it was trained on, and elastic constants are second derivatives of its energy surface — a demanding test (section 1.10).

The worked example (section 5) runs it on a GPU for bulk silicon, with the fine-tuned model of the `7.2` example (trained in the `7.1` example's silicon run), against experiment and against the phonons of the `7.3` example:

- the constants and moduli against ultrasonic measurements, and what the derived ratios say;
- sound velocities against the slopes of the `7.3` phonons;

- the internal relaxation, and why the force threshold of the relaxation matters;
- the strain amplitude, the cell size, and a structure that is not at a minimum;
- three foundation models, all too soft;
- hcp magnesium (five independent constants) and graphene (a 2D sheet) with the foundation model.

13.4.2.1 1. Theory

Symbols at a glance

Symbol	Meaning
σ_ij, ε_kl	stress (GPa) and strain (dimensionless) tensors
C_ijkl, C_mn	stiffness tensor; its 6×6 Voigt form (m, n = 1..6)
S_mn	compliance, S = C^{-1} (1/GPa; reported in 1/TPa)
ε_m	Voigt strain: ε_1..3 = ε_xx, ε_yy, ε_zz; ε_4..6 = 2ε_yz, 2ε_xz, 2ε_xy (engineering shear)
B, G, E, ν	bulk, shear and Young's moduli, Poisson's ratio
_V, _R, _H	Voigt, Reuss and Hill averages
A, A^U	Zener ratio (cubic), universal anisotropy index
u	internal coordinate: a displacement of the atoms inside the cell at fixed strain
ρ, v	mass density, sound velocity
θ_D	Debye temperature
n	atoms per unit volume

1.1 Hooke's law and the stiffness tensor. For small deformations stress is linear in strain [1]:

$$\sigma_{ij} = \Sigma_{kl}\, C_{ijkl}\, \varepsilon_{kl} \tag{1}$$

C_ijkl has 81 components, but σ and ε are symmetric, so only 36 are distinct; and because C is the second derivative of an energy, C_ijkl = C_klij, which leaves **21**. The Voigt notation stores them in a symmetric 6×6 matrix (xx → 1, yy → 2, zz → 3, yz → 4, xz → 5, xy → 6) [1]:

$$\sigma_m = \Sigma_n\, C_{mn}\, \varepsilon_n \qquad \varepsilon_{4,5,6} = 2\,\varepsilon_{yz},\ 2\,\varepsilon_{xz},\ 2\,\varepsilon_{xy} \quad \text{(engineering shear strains)} \tag{2}$$

$$E_{elastic} / V = \tfrac{1}{2}\, \Sigma_{mn}\, C_{mn}\, \varepsilon_m\, \varepsilon_n \tag{3}$$

The factor 2 in the shear strains is a convention, but it decides the numbers: with it, C44 is the ratio of shear stress to the *engineering* shear angle, which is what ultrasonic measurements and every table report. stb-mlelastic uses the same deformation matrices as the SIESTA workflow (elastic_inputs.get_strain_matrix), so a strain called xy means the same thing in both tools.

1.2 The stress-strain method. Apply a strain ε to the relaxed cell (h' = h (1 + ε)), relax the atoms at that fixed cell, compute the stress, and repeat at several amplitudes. For each of the six Voigt directions n, the stress components against the strain amplitude are straight lines whose slopes are the column n of C [4]:

$$C_{mn} = \partial\sigma_m / \partial\varepsilon_n \qquad \text{(a linear fit of } \sigma_m \text{ against } \varepsilon_n\text{, at --steps points in ±--max \%)} \tag{4}$$

- **Symmetric points** (-ε . . . +ε) cancel the second-order (quadratic) terms of the stress in the fitted slope; the first error left is of order ε^2 and comes from the third-order elastic constants (section 5.5).

- **The intercept** of each fit is the stress of the unstrained cell. It should be ≈ 0: the reference must be at a stress-free minimum of *this* model, which is why every model first relaxes the cell and the atoms (`--fmax`). A non-zero intercept means a residual stress that contaminates every constant.
- `C_mn` and `C_nm` come from two different directions; their agreement ($C = C^{T}$) is a check of the fits.

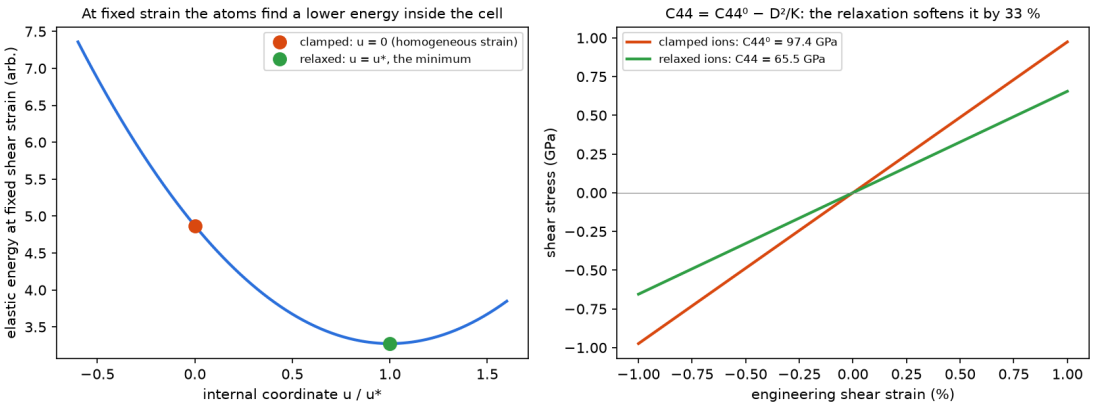
The alternative, the energy-strain method, fits $E(\varepsilon)$ (Eq. 3) instead. It needs larger strains, because the energy is quadratic, and it is what the SIESTA workflow offers to avoid SIESTA's stress noise. A MACE stress is the exact derivative of a smooth energy, so the stress method, with small strains, is the better choice here.

1.3 Internal relaxation: clamped and relaxed ions. When a crystal with more than one atom per primitive cell is strained, the atoms do not in general stay where the homogeneous strain puts them. In diamond-structure silicon, a shear ε_{yz} pushes the two atoms of the primitive cell along x in opposite directions, because the strain makes one of the four bonds shorter than the others. At fixed strain the energy has an extra internal coordinate u (Fig. 1) [11, 12]:

$$E / V = \tfrac{1}{2} C^{0} \varepsilon^{2} - D \varepsilon u + \tfrac{1}{2} K u^{2} \qquad \rightarrow \qquad u^{*} = (D / K) \varepsilon, \qquad C = C^{0} - D^{2} / K \tag{5}$$

- C^{0} is the **clamped-ion** constant ($u = 0$); C is the **relaxed-ion** constant, the one measured by any experiment slower than a lattice vibration.
- Where the symmetry pins every atom (silicon under xx, yy, zz strain; any crystal with all atoms at inversion centres), $D = 0$ and the two coincide.
- In silicon, Kleinman's internal-strain parameter describes u^{*}; the relaxation lowers C44 by about a third (section 5.4).

`stb-mlelastic` relaxes the atoms at every strained cell (`--no-relax` turns it off), and by default also runs the scan without relaxation (`--no-clamped` skips it) to report both.



Internal relaxation under shear

Fig. 1 — Left: energy against the internal coordinate at a fixed shear strain; clamped (`u = 0`) and relaxed (`u = u*`) positions. Right: the stress-strain lines of the clamped- and relaxed-ion constants, with the silicon model's numbers (`theory_figures.py`).

Why the force threshold matters. Under a 1 % shear, the silicon atoms move by only ~0.008 Å. A relaxation that stops at a residual force `f` leaves them ~`f/k` away from u^{*}, with `k` ≈ 13 eV/Å² the on-site stiffness: `f = 0.02 eV/Å` is 0.0015 Å, a fifth of the effect being measured. This is why `--fmax` defaults to 0.001 eV/Å (section 5.4).

1.4 Symmetry: which constants are independent. Neumann's principle: a physical property tensor has at least the symmetry of the crystal's point group [1]. For each rotation `R` of the point group, `C_ijkl = R_ia R_jb R_kc R_ld C_abcd`. That reduces the 21 constants:

Crystal system	Independent constants	Example
cubic	3: `C11 = C22 = C33`, `C12 = C13 = C23`, `C44 = C55 = C66`	Si, Al, NaCl
hexagonal	5: `C11 = C22`, `C12`, `C13 = C23`, `C33`, `C44 = C55`; `C66 = (C11 − C12)/2`	Mg, graphite, wurtzite
tetragonal	6 or 7	TiO_2 rutile
trigonal	6 or 7	α-quartz, Al_2O_3
orthorhombic	9	olivine
monoclinic	13	
triclinic	21	

(in the standard orientation; for hexagonal, `c` along `z`.)

`stb-mlelastic` finds the space and point group of the pre-relaxed cell with spglib [21], and **projects** the fitted tensor on the point group — averages `C` over every rotation, in the cell's own Cartesian frame, so it works for any orientation:

```
C_sym = (1 / |G|) Σ_{R ∈ G} R ⋆ C          (R ⋆ C: the rotation applied to all four indices)
(6)
```

The report then gives:

- the largest difference `|C − C_sym|` — a fit with the crystal's symmetry has none; a large one means noisy fits, an unrelaxed reference, or a symmetry found too high or too low;
- the independent constants, as groups of equal entries of `C_sym`. A linear relation that is not an equality — the hexagonal `C66 = (C11 − C12)/2` — is recognized and marked `= (C11 - C12)/2, not independent`, and not counted.

The numbers derived from `C` (moduli, velocities, stability) use the fitted tensor itself; the projection is a check. The symmetry tolerance of spglib is fixed at 10^{-3} Å.

1.5 Mechanical stability: Born's criteria. A crystal is mechanically stable when every small homogeneous strain raises its energy, i.e. when the quadratic form of Eq. 3 is positive definite: **all six eigenvalues of `C` are positive** — necessary and sufficient for any crystal system [5]. For the common systems this takes explicit forms [5], which the report lists too:

```
cubic:                C11 − C12 > 0,    C11 + 2 C12 > 0,    C44 > 0
(7)
hexagonal, tetragonal: C11 > |C12|,     2 C13^2 < C33 (C11 + C12),    C44 > 0   (tetragonal also C66 >
0)
2D sheet:             C11 > 0,          C66 > 0,            C11 C22 − C12^2 > 0
```

1.6 Polycrystal moduli: Voigt, Reuss and Hill. A polycrystal of randomly oriented grains is isotropic and has two constants, the bulk modulus `B` and the shear modulus `G`. They depend on how the grains share the load, and two limits bracket them [6, 7]:

- **Voigt**: every grain has the same strain (an upper bound);
- **Reuss**: every grain carries the same stress (a lower bound);
- **Hill**: their mean, the usual estimate [6].

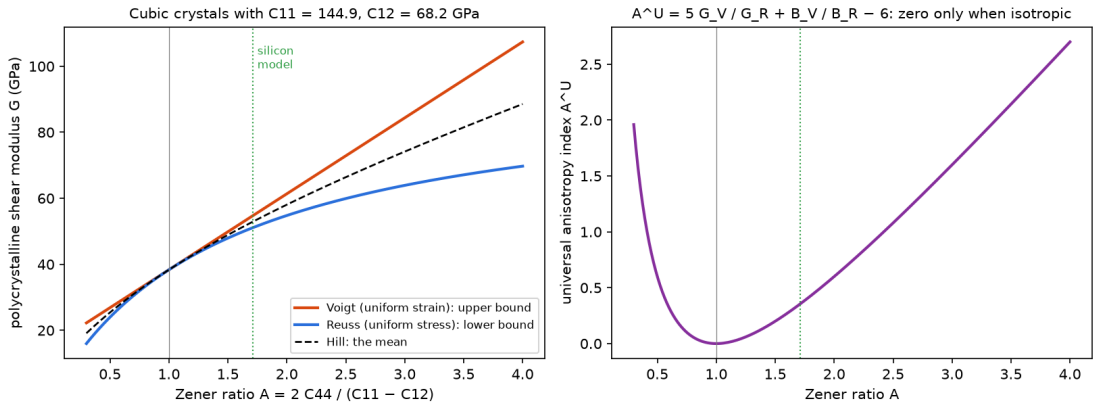
```
B_V = [(C11 + C22 + C33) + 2 (C12 + C13 + C23)] / 9
G_V = [(C11 + C22 + C33) − (C12 + C13 + C23) + 3 (C44 + C55 + C66)] / 15
1/B_R = (S11 + S22 + S33) + 2 (S12 + S13 + S23)
(8)
15/G_R = 4 (S11 + S22 + S33) − 4 (S12 + S13 + S23) + 3 (S44 + S55 + S66)
E = 9 B G / (3 B + G)            ν = (3 B − 2 G) / [2 (3 B + G)]
```

For a cubic crystal `B_V = B_R = (C11 + 2 C12)/3`; only the shear moduli differ, and the more anisotropic the crystal, the further apart they are (Fig. 2).

Ratios the report gives, with their usual reading:

Quantity	Formula	Reading
Zener ratio (cubic)	`A = 2 C44 / (C11 − C12)`	1 = isotropic
universal anisotropy	`A^U = 5 G_V/G_R + B_V/B_R − 6` [8]	0 = isotropic, for any crystal system
Pugh ratio	`B / G` [9]	above ~1.75 ductile, below brittle (an empirical rule for metals and covalent solids)
Cauchy pressure (cubic)	`C12 − C44`	negative: directional (covalent) bonding; positive: metallic. Zero for central pair forces
Vickers hardness	`H_V = 2 (k² G)^0.585 − 3, k = G/B` [10]	an empirical correlation, ±30 % at best
linear compressibility	`β_i = Σ_j S_ij`	fractional shortening of axis `i` per unit hydrostatic pressure

These ratios are rules of thumb built on the constants. They amplify the constants' errors (section 5.2).



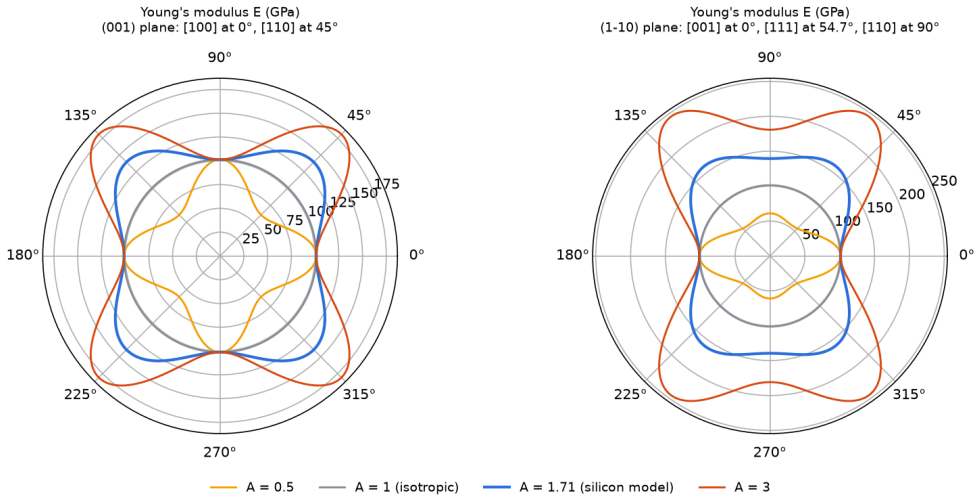
Voigt and Reuss bounds against anisotropy

Fig. 2 — Left: the Voigt and Reuss shear moduli of cubic crystals with the silicon model's `C11` *and* `C12`*, against the Zener ratio; they meet only for an isotropic crystal. Right: the universal anisotropy index (*`theory_figures.py`*).*

1.7 Directional properties. The Young's modulus along a unit direction `n` — the stiffness of a thin bar cut along `n` — and the linear compressibility along `n` follow from the compliance tensor [1]:

```
1 / E(n) = Σ_ijkl s_ijkl n_i n_j n_k n_l            β(n) = Σ_ijk s_ijkk n_i n_j
(9)
```

(s_ijkl is the 4-index compliance, with the factors ½ per engineering shear index). A cubic crystal is stiffest along <111> when A > 1 and along <100> when A < 1 (Fig. 3); its linear compressibility is the same in every direction. The report gives the extremes of E(n) over a sphere with their directions, and directional_moduli.png draws E and β in two Cartesian planes of the cell: xy, and the plane containing z and x + y (for a cubic cell, the (001) and (1-10) planes; for a hexagonal cell with c along z, the basal plane and a plane containing c).



Directional Young's modulus of cubic crystals

Fig. 3 — E(n) of cubic crystals with the silicon model's C11, C12 and four Zener ratios (theory_figures.py).

1.8 Sound velocities and the Debye temperature. A plane elastic wave along n obeys the **Christoffel equation** [14]:

$$\Sigma_k\ (\Sigma_{jl}\ C_{ijkl}\ n_j\ n_l - \rho\ v^2\ \delta_{ik})\ p_k = 0 \qquad (10)$$

Its three eigenvalues ρv^2 give one quasi-longitudinal and two quasi-transverse velocities. For a cubic crystal along [100], $\rho v_L^2 = C11$ and $\rho v_T^2 = C44$; along [110], $\rho v_{T1}^2 = (C11 - C12)/2$; and so on (7.3, section 1.7). These are the slopes of the acoustic phonon branches at $q \to 0$, an independent measurement of the same constants (section 5.3).

The report also gives the isotropic (Hill) velocities and the **elastic Debye temperature** [13]:

$$v_L = \sqrt{[(B + 4G/3)/\rho]}, \quad v_T = \sqrt{(G/\rho)}, \quad 3 / v_m^3 = 1/v_L^3 + 2/v_T^3, \quad \theta_D = (\hbar/k_B)\ (6\pi^2\ n)^{(1/3)}\ v_m \qquad (11)$$

It is the Debye model's cutoff fitted to the long-wavelength sound — the low-temperature heat capacity. For silicon the experimental constants give 649 K, the calorimetric value is 645 K. It differs from the "second-moment" θ_D of the phonon DOS that 7.3 reports, which weighs all frequencies.

1.9 Two-dimensional materials. For a sheet with vacuum along z, straining zz, yz or xz would only stretch empty space: those directions are skipped. The stress of the cell is an average over its height L_z, vacuum included, so $\sigma \times$ L_z is the force per unit length of the sheet, and the in-plane constants are reported **per area, in N/m**, independent of the vacuum:

> E_x = (C11 C22 − C12²) / C22, ν_xy = C12 / C22, G_xy = C66, γ = (C11 + C22 + 2 C12) / 4 (layer modulus) (12)

A hexagonal sheet (graphene, hBN, MoS_2) is isotropic in-plane: C11 = C22, C66 = (C11 − C12)/2, E(θ) constant. A 1D wire along c has a single constant, the axial stiffness C33 in nN (stress × cross-section).

1.10 Elastic constants from a machine-learned potential.

- **No noise.** The stress is the exact derivative of a smooth energy, so small strains (±1 %) and few points work. The limits are the relaxation threshold (section 1.3) and the strain amplitude (third-order terms).
- **Second derivatives are demanding.** The training fits energies, forces and stresses of distorted cells; the stiffness is the slope of the stress, constrained only by the strained configurations in the training set (6 of 57 in the example's model).
- **Foundation models are soft.** Universal potentials trained on near-equilibrium structures underestimate the curvature of the energy surface [19]. For silicon every MACE-MP-0 size puts C44 ~60 % below experiment (section 5.6).
- **The model card** (report section MODEL) is printed in full, and the strained cells are compared with its trained range of volume, pressure and interatomic distance (section TRAINED RANGE).

1.11 A calculation you can check by hand. From the reference run (section 5.2): C11 = 144.88, C12 = 68.15, C44 = 65.47 GPa, ρ = 2267.5 kg/m^3, 8 atoms in 164.5 Å^3.

- **Bulk modulus (Eq. 8):** B = (144.88 + 2 × 68.15)/3 = 93.73 GPa. Reported: 93.73 (Voigt = Reuss for a cubic crystal).
- **Voigt shear modulus:** G_V = (144.88 − 68.15 + 3 × 65.47)/5 = 54.63 GPa. Reported: 54.63.
- **Zener ratio:** A = 2 × 65.47/(144.88 − 68.15) = 1.707. Reported: 1.707.
- **Sound along [100]:** v_L = $\sqrt{(144.88 \times 10^9 / 2267.5)}$ = 7993 m/s, v_T = $\sqrt{(65.47 \times 10^9 / 2267.5)}$ = 5373 m/s. Reported: 7993 and 5374.
- **Linear compressibility:** for a cubic crystal β = 1/(3B) = 1000/(3 × 93.73) = 3.556 /TPa. Reported: 3.556 on every axis.
- **Young's modulus along [100]:** E = (C11 − C12)(C11 + 2C12)/(C11 + C12) = 76.73 × 281.18 / 213.03 = 101.28 GPa. Reported: 101.27.

1.12 What this tool simplifies — explicitly.

- **Static, 0 K constants.** No temperature: no thermal expansion of the reference cell, no vibrational contribution (adiabatic vs isothermal constants differ by ~1 % in solids).
- **Linear elasticity.** Small strains; the third-order constants appear only as the drift of --check-convergence.
- **The derived quantities use the fitted tensor**, not its symmetry projection; the projection is reported as a check.
- **spglib tolerance fixed** at 10^{-3} Å. A cell distorted beyond it is treated as lower symmetry.
- **Empirical ratios** (Pugh, Cauchy, hardness) are reported as such, without error bars.
- **One model, no error bars** on the constants; the fit's R^2 measures the stress noise, not the model's error.

13.4.2.2 2. Libraries and external dependencies used

Library	Used for
ASE [22]	`Atoms`, strained cells, the relaxations (through `stb.core.mace_relax`), neighbour lists
mace-torch [17, 18]	the MACE calculator, the foundation models (downloaded once, cached in `~/.cache/mace/`) and custom `.model` files
PyTorch	runs the model on `cpu` or `cuda` (`--device`)
spglib [21]	space group, point group and its rotations
pymatgen	structure conversion
NumPy	fits, tensors, eigenvalues, Christoffel
matplotlib	the PNG figures (always written), `--show`
gnuplot (optional)	renders the `.gplot` files written by `--save-gnuplot`
`stb.elastic_analysis`, `stb.elastic_inputs`	the stiffness fit, fit diagnostics and strain matrices shared with the SIESTA workflow (`5.2`)
`stb.core`	`.fdf` reading, the model card, the calculator, relaxations and device checks, library-noise capture

Install the `ml` extra: `pip install stb_suite[ml]`.

13.4.2.3 3. What `stb-mlelastic` does

3.1 One fit (the default).

1. **Read** the structure (`--file`, `.fdf`), find its dimensionality (vacuum-padded axes, from `--vacuum-gap`; `--dimensionality` overrides it: 2D expects the vacuum along `z`, 1D a wire along `c`), its symmetry.
2. **Load the model**: a foundation model (`--model`) or a custom model with its card (`--custom-model` + `--custom-model-card`; the card must match the model's SHA-256). Print the card; check the structure against it.
3. **Pre-relax** the cell and the atoms to the model's own minimum (`--fmax`, `--max-steps`; vacuum axes fixed), unless `--no-pre-relax`.
4. **Strain scan**: for each Voigt direction not blocked by a vacuum axis, `--steps` strains in ±`--max` %; at each, relax the atoms at fixed cell (unless `--no-relax`) and record the stress, the volume, the pressure, the shortest distance and the largest internal shift. Then the same without relaxation (clamped ions), unless `--no-clamped`.
5. **Fit** each direction (Eq. 4): `C`, R^2 and intercept per direction; symmetrize $(C + C^T)/2$.
6. **Analyse**: symmetry projection, Born criteria, moduli and ratios, clamped vs relaxed, Christoffel and Debye, directional properties (3D); in-plane moduli (2D).
7. With a custom model, repeat 3–6 with the **foundation model** (`--model`) unless `--skip-foundation-comparison`.

Each model is strained about **its own** relaxed cell: the comparison is between the two models' minima, not at a common lattice.

3.2 `--check-convergence`. Steps 1–5 at every amplitude of `--convergence-strains` (default: half, the given, and double `--max`), compared constant by constant (every independent constant, the smallest R^2, `B` and `G`), with the largest relative change and a warning above 2 %.

3.3 The report, section by section. Sections are numbered in order; those that do not apply (3D-only, a single model, the convergence mode) are left out.

Section	Content
`RUN METADATA`	date, versions, device (GPU name), input, mode, strains, relaxation settings, model, comparison

Section	Content
STRUCTURE	atoms, formula, cell, volume and density, shortest distance, space group, units of the constants
MODEL	the **model card** in full, or the foundation model's identity; load time
REFERENCE STATE	per model: pre-relax (forces, steps), lattice before → after and volume change, stress before → after; normalization (2D/1D)
STRESS-STRAIN FITS	per model and direction: the diagonal constant, R^2, intercept, largest stress, largest internal shift and per %, relaxation steps; skipped directions
TRAINED RANGE	volume per atom, pressure and shortest distance over all strained cells; with a card, each against its trained range
STIFFNESS TENSOR & CRYSTAL SYMMETRY	C, C = C^T check, space/point group, projection and its deviation, the independent constants, the compliance S
MECHANICAL STABILITY	per model: the Born criteria and the verdict
ELASTIC MODULI	Voigt/Reuss/Hill B, G, E, ν; Pugh, Cauchy, Zener, A^U, hardness, linear compressibilities (3D)
INTERNAL RELAXATION	clamped vs relaxed constants and their difference (3D, 2+ atoms)
SOUND VELOCITIES & DEBYE TEMPERATURE	density; Christoffel velocities along [100], [110], [111]; isotropic velocities; θ_D (3D)
DIRECTIONAL ANISOTROPY	E(n) extremes and their directions, β(n) extremes, E along the axes (3D)
IN-PLANE MODULI	E_x, E_y, ν, G_xy, layer modulus, E(θ) range (2D)
FINE-TUNED vs FOUNDATION	every independent constant, B, G, E, ν and the lattice of both models, with the difference
SUMMARY & FILES	status, headline numbers, files, the outputs not asked for, wall time
WARNINGS	every warning, numbered; the count of library messages captured

With --check-convergence: RUN METADATA, STRUCTURE, MODEL, STRAIN-AMPLITUDE CONVERGENCE CHECK, SUMMARY & FILES, WARNINGS.

3.4 Files.

Written	When
stress_strain.png (every direction, both models)	always
directional_moduli.png (polar E and β), anisotropy_surface.png (3D E surface, --skip-anisotropy-surface omits it)	3D
inplane_youngs.png	2D
strain_convergence.png	--check-convergence
stress_strain.dat, elastic_tensor.dat (C, its projection, S), directional.dat / strain_convergence.dat	--save-data
plot/*.dat, plot/*.gplot, and plot/*.pdf when gnuplot is installed	--save-gnuplot
stb_mlelastic_report.txt	--save-report
the figures on screen	--show (needs a display; without one it is a warning)

Everything goes into -o/--output-dir (default mlelastic_out).

3.5 Guard rails.

Situation	Behaviour
--custom-model without --custom-model-card, an unreadable card, or a SHA-256 mismatch	clean error
--steps below 2, --max not positive, --convergence-strains with one value	clean error
an isolated molecule (vacuum along every axis)	clean error: no elastic constants
vacuum along an axis the 2D/1D conventions do not expect	warning, treated as 3D (--dimensionality overrides)

Situation	Behaviour
a direction touching a vacuum axis	skipped and listed
the pre-relax hits `--max-steps`; a residual stress above 0.2 GPa at zero strain	warning
a model's own equilibrium volume more than 1 % from the input	warning (the constants are at the model's minimum)
a fit with R^2 below `--fit-quality-tolerance` or a large intercept	warning naming the directions
$C \neq C^T$ by more than 1 %; a tensor departing from the crystal's symmetry by more than 1 %	warning
no symmetry found (P1)	warning: no projection, 21 free constants
strained cells outside the card's trained volume or pressure	warning
Born criteria failed; an imaginary sound velocity; a singular tensor	warning; dependent sections skipped
`--device cuda` with no usable GPU; a missing input or model file	clean error
`--show` without a display; gnuplot not installed	warning; files written anyway

13.4.2.4 4. Running it both ways

Command line:

```
stb-mlelastic --file si8.fdf \
              --custom-model model/mlff_model_compiled.model --custom-model-card
              model/mlff_model_card.json \
              --device cuda --steps 7 --save-data --save-gnuplot --save-report
stb-mlelastic --file si8.fdf --custom-model ... --custom-model-card ... --check-convergence
--convergence-strains 0.25 0.5 1 2 3
stb-mlelastic --file graphene.fdf --model medium --max 0.5
```

Interactive menu: run `stb-suite` and type `7.4`. It asks, in this order:

1. the structure;
2. a custom model path, or Enter for a foundation model and then its size. With a custom model it asks for the card and offers `mlff_model_card.json` next to the model as the default;
3. the device (`cpu`/`cuda`; it checks the GPU) — for a custom model too;
4. the strain amplitude, the points per direction, the dimensionality, the output folder;
5. whether to run the convergence check instead (then the amplitudes); otherwise, with a custom model, whether to compare with the foundation model, and whether to skip the 3D surface;
6. one question per optional output (show, gnuplot, data, report).

The menu builds the same command line and calls `stb-mlelastic`; the guided script (Case 11) drives it with piped answers.

13.4.2.5 5. Worked example: silicon, magnesium, graphene

5.1 Set-up. The structures:

- `si8.fdf`, the 8-atom conventional diamond cell at SIESTA's relaxed lattice constant (5.48024 Å; experiment 5.4310 Å at 300 K); `si8_rattled.fdf`, the same with every atom moved by about 0.03 Å (section 5.8);
- `mg_hcp.fdf`, hcp magnesium at the experimental lattice, `c` along `z` (section 5.9);

- `graphene.fdf`, a graphene sheet with 20 Å along `c` (section 5.10).

The model, `../7.2-mlmd/model/` (shared with `7.2` and `7.3`): round 2 of the `7.1` silicon run, fine-tuned from MACE-MP-0 medium on 57 SIESTA single points (PBE, DZP, 8 and 64 atoms: rattled, strained, MD frames); test-set force error 28.7 meV/Å, stress error 0.18 GPa. It knows only silicon: Mg and graphene run with the foundation model.

Every run used `--device cuda` on one RTX 3060, float64; `make_reference.sh` lists them, section 6 explains how to regenerate them. The reference run (two models, 7 strains, clamped ions too: 126 relaxed cells) took 33 s.

Experimental references: silicon's constants by ultrasound at 298 K [15]; Mg [16]; graphene by nanoindentation [20].

5.2 The reference run: silicon's constants.

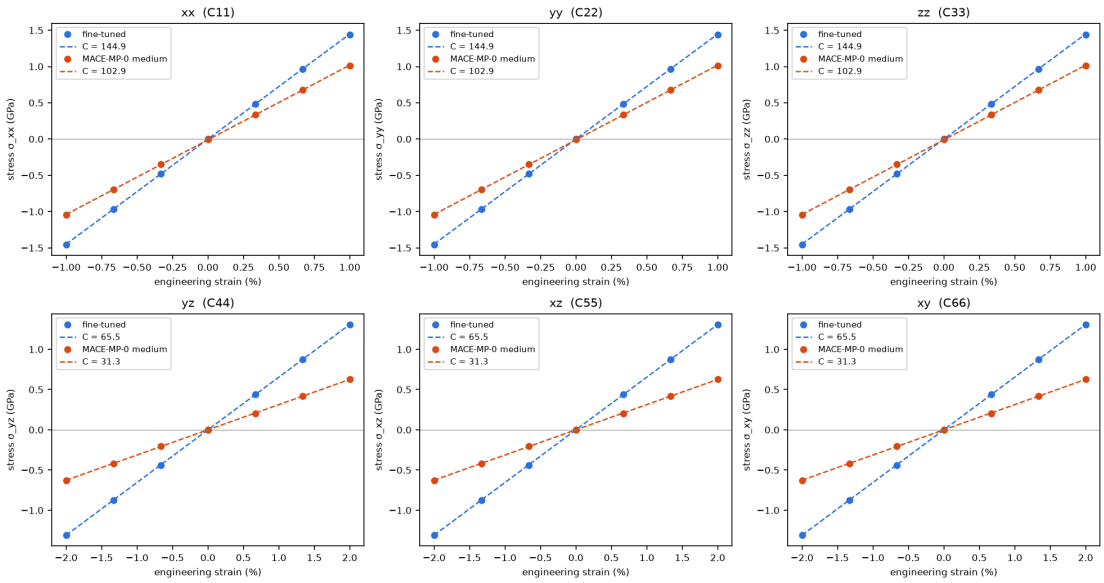
```
stb-mlelastic --file si8.fdf --custom-model ../7.2-mlmd/model/mlff_model_compiled.model \
              --custom-model-card ../7.2-mlmd/model/mlff_model_card.json --device cuda --steps 7 \
              --save-data --save-gnuplot --save-report
```

Report sections `REFERENCE STATE` and `STRESS-STRAIN FITS` (`reference_results/base/`), abridged:

```
--- fine-tuned
Pre-relax         : cell + atoms, max |F| 0.0000 -> 0.0000 eV/Ang in 6 step(s) (converged, --fmax)
Lattice a, b, c   : 5.48024 5.48024 5.48024 -> 5.47970 5.47970 5.47970 Ang (volume -0.029 %)
Stress            : pressure -0.034 -> -0.006 GPa; largest component after 0.006 GPa
--- MACE-MP-0 medium
Lattice a, b, c   : 5.48024 5.48024 5.48024 -> 5.45542 5.45542 5.45542 Ang (volume -1.353 %)

dir. | column | diagonal (GPa) | R^2      | intercept | max|σ| GPa | max shift Ang | Ang per % |
relax steps
xx   | C11    | 144.88         | 0.999963 | -0.000    | 1.46       | 0.0000        | 0.0000    | 0
yz   | C44    | 65.47          | 0.999997 | -0.000    | 1.31       | 0.0079        | 0.0079    | 110
```

- **The reference.** The fine-tuned model keeps SIESTA's lattice (−0.03 % in volume: it reproduces the level of theory it was trained on). The foundation model shrinks it by 1.35 % — its own silicon — and the report warns that its constants are at its own minimum.
- **The fits.** $R^2 \geq 0.99996$ and intercepts of 0.000 GPa: a stress-free reference and noise-free stresses. Under the normal strains the atoms do not move (0 relaxation steps, diamond's symmetry pins them); under the shears they move by 0.0079 Å per % strain.
- **Trained range.** Every strained cell is inside the model's trained volume (20.36–20.77 against 18.72–22.43 $Å^3$/atom) and pressure (±0.95 against −5.8…+10.5 GPa).



Stress-strain fits of silicon

Fig. 4 — The six stress-strain lines of both models (`--steps 7`). Each slope is a diagonal constant; the shear panels use the engineering strain.

Section `STIFFNESS TENSOR & CRYSTAL SYMMETRY`:

```
Crystal symmetry  : Fd-3m (227), point group m-3m (cubic, 48 rotations, spglib on the pre-relaxed
cell)
Projection        : C averaged over the point group; max |C - C_sym| = 0.000 GPa (0.00 % of the
largest constant)
Independent       : 3 non-zero independent constant(s) under m-3m:
  C11 = C22 = C33                          144.88 GPa
  C12 = C13 = C23                           68.15 GPa
  C44 = C55 = C66                           65.47 GPa
```

The six directions were fitted independently, yet the tensor has exact cubic symmetry — a measure of how clean the stresses are. The constants against experiment [15] and against the phonon slopes of `7.3` (same model, a different calculation):

	fine-tuned	`7.3` phonons	MACE-MP-0 medium	experiment	fine-tuned vs experiment
C11	144.88	148.0	102.90	165.6	−12.5 %
C12	68.15	70.2	60.61	63.9	+6.6 %
C44	65.47	65.1	31.31	79.5	−17.7 %
B (Hill)	93.73	93.5 (QHA B_0)	74.71	97.8	−4.2 %
G (Hill)	52.84	—	26.75	66.5	−20.5 %
E (Hill)	133.44	—	—	162.6	−17.9 %
Poisson ν	0.263	—	—	0.223	

GPa. The experimental moduli are the Hill averages of the measured constants.

- **Consistency.** Three calculations with the same model agree: the stress-strain constants, the phonon slopes of `7.3` (within 0.5–3 %, the phonon slopes coming from an interpolated 4×4×4 dispersion), and the curvature of the energy in the `7.3` QHA (B_0 = 93.5 against 93.7). Consistent

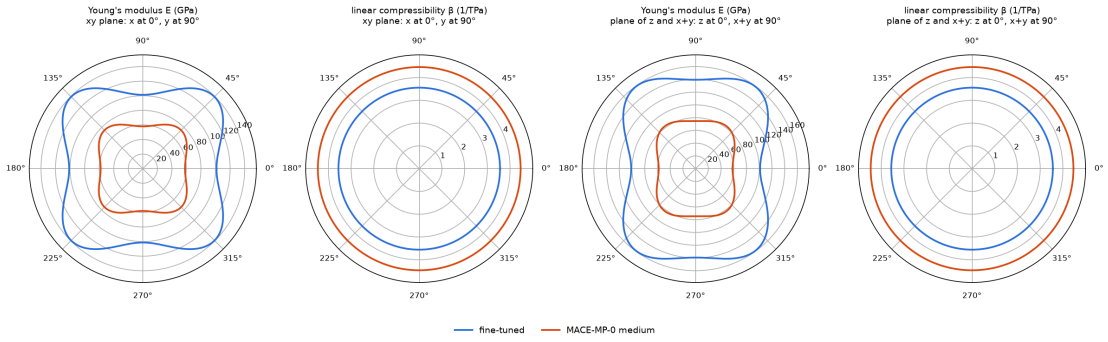
with itself, but not with SIESTA: at the same level of theory SIESTA gives `B = 87.5–89.0` (section 5.11).

- **Against experiment.** The bulk modulus is 4 % low, the shear constants further off: `C44` 18 % low, `G` 21 %. Section 5.11 splits this between PBE/DZP and the model: the shear errors are mostly the model's, and its `B` is closer to experiment than SIESTA's for the wrong reason.
- **The derived ratios amplify that.** Section `ELASTIC MODULI` reports a Pugh ratio of 1.77 ("ductile") and a Cauchy pressure of +2.7 GPa ("metallic"). Real silicon has 1.47 and −15.6 GPa: brittle and covalent, as it is. A 20 % error in `G` and an 18 % error in `C44` are enough to flip both rules of thumb. The empirical hardness, 7.4 GPa, is low for the same reason (silicon: about 10–12 GPa). Read these ratios as properties of the model — SIESTA itself gets both rules right (section 5.11).

5.3 Sound velocities. Section `SOUND VELOCITIES & DEBYE TEMPERATURE` against the acoustic slopes of the `7.3` phonons (4×4×4 supercell) and the velocities from the measured constants [15], m/s:

direction	Christoffel L / T1 / T2	`7.3` phonon slopes	experiment	largest difference vs `7.3`
[100]	7993 / 5374 / 5374	8081 / 5361 / 5361	8433 / 5843 / 5843	1.1 %
[110]	8709 / 5374 / 4113	8768 / 5365 / 4122	9134 / 5843 / 4673	0.7 %
[111]	8935 / 4572 / 4572	8994 / 4566 / 4566	9356 / 5093 / 5093	0.7 %

- Two different routes to the same physics: the curvature of the stress under homogeneous strain, and the slope of the phonon dispersion at `q` $\to$ `0`. They agree within 1.1 %.
- Along [110] the two transverse waves split: the one polarized along [001] travels at $\sqrt{(C44/\rho)}$, the one along [1-10] at $\sqrt{((C11 - C12)/2\rho)}$ — slower, since silicon's Zener ratio is 1.7.
- **Debye temperature** (Eq. 11): 583 K, against 649 K from the measured constants. The 10 % gap is that of the shear velocities, which dominate `v_m`.



Directional Young's modulus and compressibility

Fig. 5 — `E(n)` and β`(n)` in two planes of the cubic cell (`directional_moduli.png`). Silicon is stiffest along <111> (159 GPa) and softest along <100> (101 GPa); its compressibility is isotropic, as for every cubic crystal.

The same `E(n)` over the whole sphere is drawn in `anisotropy_surface.png` (`figures/si_anisotropy_surface.png`): the lobes point along the eight <111> directions.

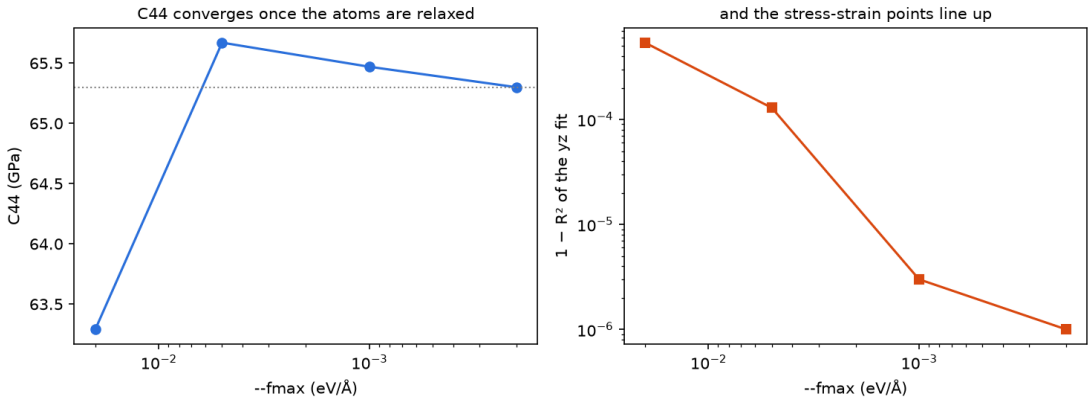
5.4 Internal relaxation and the force threshold. Section `INTERNAL RELAXATION`:

```
constant        | clamped (GPa) | relaxed (GPa) | relaxation
C11 = C22 = C33 | 144.88        | 144.88        | +0.0 %
C12 = C13 = C23 | 68.15          | 68.15         | +0.0 %
C44 = C55 = C66 | 97.36          | 65.47         | -32.8 %
```

- Under the normal strains nothing moves inside the cell, so clamped and relaxed constants are the same.
- Under shear the internal relaxation lowers `C44` by a third (Eq. 5, Fig. 1). DFT calculations of silicon find the same order: a clamped-ion $C44^0$ of about 110 GPa against a relaxed one of about 85 [12]. A calculation that forgets to relax the atoms overestimates the shear stiffness of every crystal without inversion at its atoms.

The precision of `C44` therefore depends on how well the atoms are relaxed. Four runs with different `--fmax` (`fmax_*`, 5 strains, no comparison):

`--fmax` (eV/Å)	C44 (GPa)	R^2 (yz)	relaxation steps (yz)	C11 (GPa)
0.02	63.29	0.999461	28	144.77
0.005	65.67	0.999871	40	144.77
0.001 (default)	65.47	0.999997	74	144.87
0.0002	65.30	0.999999	98	144.89



C44 against the force threshold

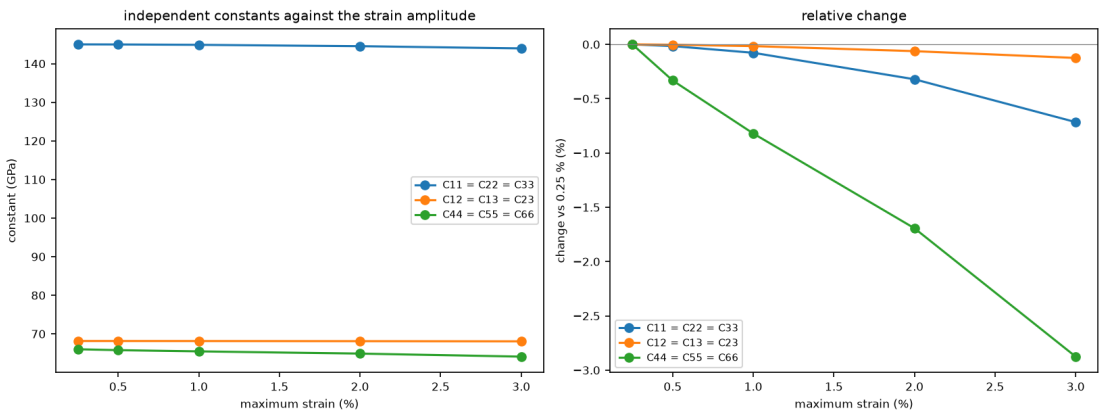
Fig. 6 — `C44` and the quality of the `yz` fit against `--fmax` (`results_figures.py`).

At 0.02 eV/Å (the previous default) the atoms stop ~0.0015 Å short of their relaxed positions — a fifth of the displacement being measured — and `C44` is 3 % low with scattered points. From 0.001 down, `C44` is stable to 0.3 % and the points line up ($1 - R^2 = 3 \times 10^{-6}$). `C11` does not care: nothing relaxes. Each tightening costs a few dozen relaxation steps, milliseconds with MACE.

5.5 The strain amplitude.

```
stb-mlelastic --file si8.fdf --custom-model ... --custom-model-card ... --steps 7
--check-convergence \
              --convergence-strains 0.25 0.5 1 2 3
```

```
max strain % | C11 = C22 ... | C12 = C13 ... | C44 = C55 ... | min R^2 | B Hill | G Hill
0.25         | 144.99        | 68.16         | 66.02         | 0.99994 | 93.77  | 53.13
0.5          | 144.97        | 68.16         | 65.80         | 0.99999 | 93.76  | 53.01
1            | 144.88        | 68.15         | 65.47         | 0.99996 | 93.73  | 52.84
2            | 144.52        | 68.12         | 64.90         | 0.99985 | 93.59  | 52.47
3            | 143.95        | 68.08         | 64.12         | 0.99968 | 93.37  | 51.95
Largest change    : C44 varies by 2.91 % over the scan
```



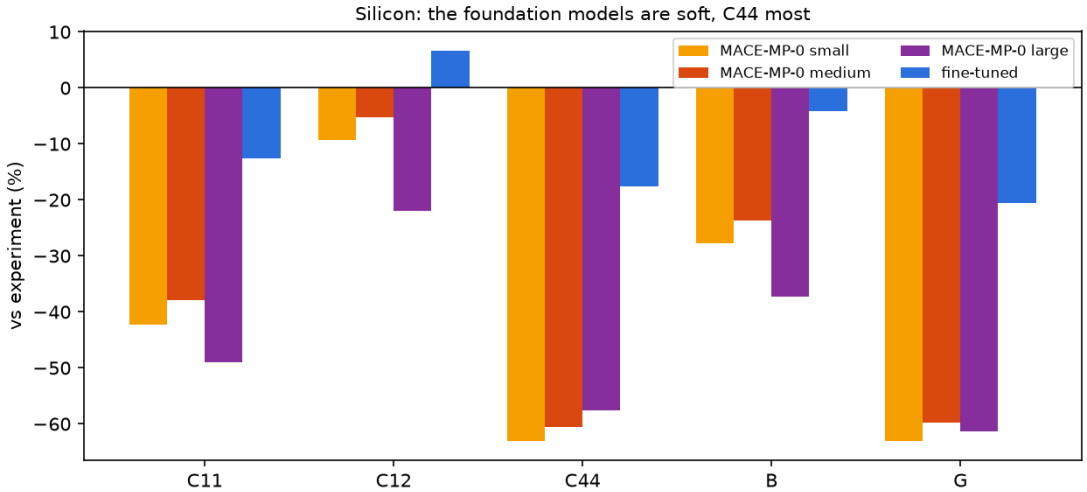
Constants against the strain amplitude

Fig. 7 — Every independent constant against the largest strain of the fit (`strain_convergence.png`*).*

- The constants drift smoothly with the amplitude: this is the third-order (anharmonic) response, not noise. `C44` drifts most — shear is the softest, most anharmonic deformation of the diamond lattice, as the negative Grüneisen parameters of its transverse acoustic phonons (`7.3`, section 5.10) also show.
- At the default 1 %, `C44` is 0.8 % below its small-strain limit, `C11` 0.1 %. Below 0.5 % nothing changes beyond the last digit; at 0.25 % the smallest R^2 drops a little: the stresses become small (0.4 GPa) next to the relaxation's residual.
- The report warns above 2 % change over the scan: here the 3 % amplitude is the one to avoid.

5.6 The foundation models. The same calculation with the three MACE-MP-0 sizes (`foundation_*`), GPa:

model	a (Å)	C11	C12	C44	B	G	ν
MACE-MP-0 small	5.4646	95.7	58.0	29.4	70.6	24.6	0.344
MACE-MP-0 medium	5.4554	102.9	60.6	31.3	74.7	26.8	0.340
MACE-MP-0 large	5.4872	84.4	49.9	33.7	61.4	25.7	0.316
fine-tuned	5.4797	144.9	68.2	65.5	93.7	52.8	0.263
experiment	5.4310	165.6	63.9	79.5	97.8	66.5	0.223



Foundation models against experiment

Fig. 8 — Each constant and modulus against experiment (`results_figures.py`*).*

- Every foundation model is soft: `C44` and `G` ~60 % low, `B` 24–37 %. The largest model is again not the best — softest in `C11` and `B`, as it was in the phonons of `7.3`.
- The Poisson ratio of 0.32–0.34 is that of a metal, not of a covalent semiconductor.
- These models are trained on the same functional (PBE) as our SIESTA labels: the softening is a property of universal potentials trained mostly on near-equilibrium structures [19], not a difference in the level of theory. Fine-tuning on 57 SIESTA configurations brings `C44` from 31 to 65 GPa.

5.7 The cell size. The 216-atom supercell (`stb-supercell -d 3 3 3`, `si216`) gives `C11 = 144.87`, `C12 = 68.16`, `C44 = 65.47 GPa` — the 8-atom values to 0.01 GPa. A homogeneous strain of a perfect crystal does not depend on the cell, which is a good check of the setup; the supercell only matters for a defect, an alloy or a disordered structure, whose environments need it. It took 82 s on the GPU against 34 s for 8 atoms.

5.8 A structure that is not at a minimum. `si8_rattled.fdf`, every atom moved by ~0.03 Å (largest force 0.85 eV/Å), with and without the pre-relax:

case	C11	C12	C44	symmetry found	independent	residual stress (GPa)	time
8 atoms, reference	144.88	68.15	65.47	Fd-3m	3	0.01	34 s
rattled, `--no-pre-relax`	144.77	68.04	65.33	P1	21 (no symmetry)	0.67	48 s
rattled, pre-relaxed (default)	144.87	68.15	65.47	Fd-3m	3	0.01	18 s

- **With the pre-relax** the atoms return to their sites, spglib finds Fd-3m again (its tolerance, 10^{-3} Å, is looser than phonopy's in `7.3`), and the constants are those of the crystal.
- **Without it**, the constants are still close — the atoms are relaxed at every strained cell anyway — but the reference carries 0.67 GPa of stress (the fits' intercepts), no symmetry is found, so there is nothing to project on and no check, and two warnings say so. The run is also slower: every strained cell relaxes the rattle away first.

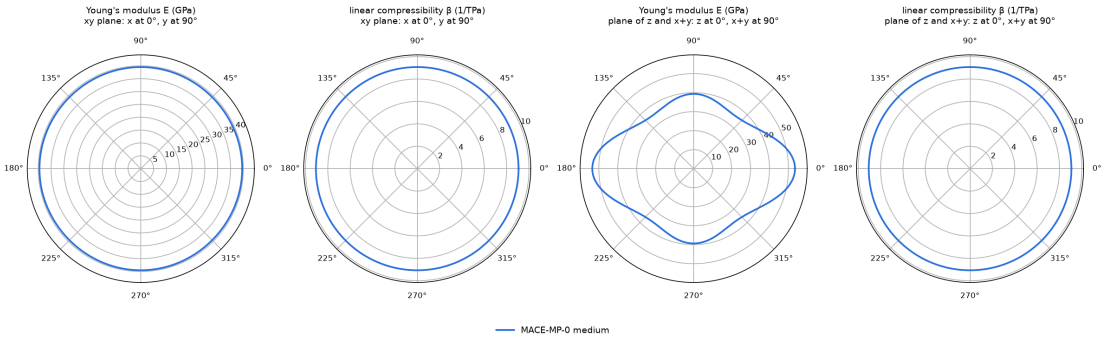
5.9 A hexagonal crystal: hcp magnesium. With MACE-MP-0 medium (the fine-tuned model knows only silicon), `mg_hcp`:

```
Crystal symmetry  : P6_3/mmc (194), point group 6/mmm (hexagonal, 24 rotations, spglib on the
pre-relaxed cell)
Projection        : C averaged over the point group; max |C - C_sym| = 0.052 GPa (0.08 % of the
largest constant)
  C11 = C22                              57.42 GPa
  C12                                    30.27 GPa
  C13 = C23                              22.60 GPa
  C33                                    65.11 GPa
  C44 = C55                              13.04 GPa
  C66                                    13.57 GPa  = (C11 - C12)/2, not independent
```

constant	MACE-MP-0 medium	experiment (≈, room temperature) [16]	difference
C11	57.5	59.4	−3 %

constant	MACE-MP-0 medium	experiment (≈, room temperature) [16]	difference
C12	30.2	25.6	+18 %
C13	22.6	21.4	+6 %
C33	65.1	61.6	+6 %
C44	13.0	16.4	−20 %
C66 = (C11 − C12)/2	13.6	16.9	

- The point group 6/mmm leaves five independent constants. `C66` is tied to the others by a linear relation, not an equality — 13.57 = (57.42 − 30.27)/2 — and the report marks it as not independent.
- The hexagonal Born criteria (Eq. 7) all pass.
- The foundation model does well on the normal constants of this metal (within 6 % except `C12`) and is again soft in shear (`C44` −20 %). It also shrinks the cell by 2 % in volume.
- In the basal plane (`xy`) the Young's modulus is the same in every direction, as hexagonal symmetry demands (Fig. 9, left); it varies only with the angle from `c`.



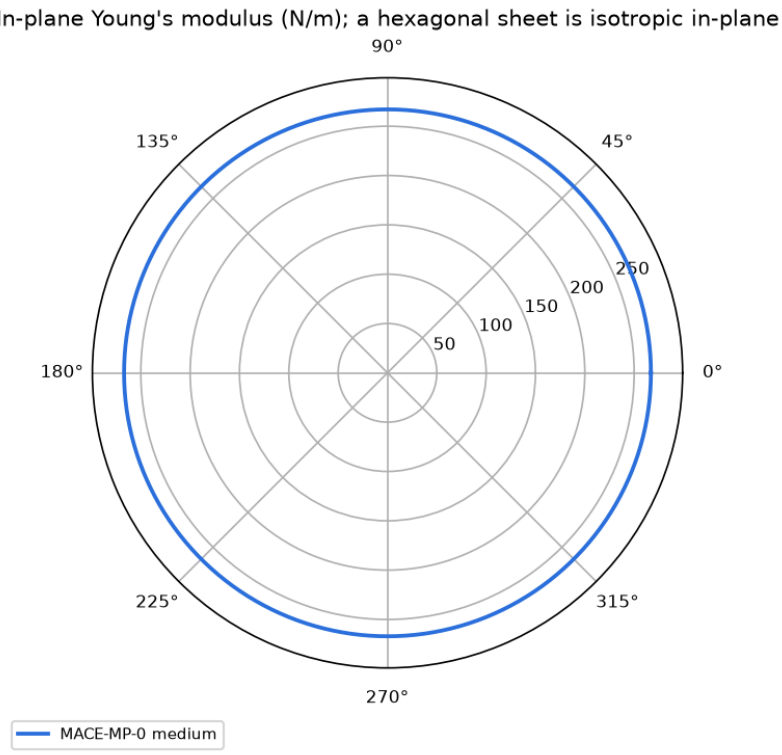
Directional Young's modulus of hcp Mg

Fig. 9 — `E(n)` and β`(n)` of hcp Mg: isotropic in the basal plane `xy`, anisotropic in a plane containing `c` (`z`).

5.10 A 2D sheet: graphene. With MACE-MP-0 medium, strains up to ±0.5 % (`graphene`):

```
Skipped          : zz, yz, xz -- touch a vacuum axis (no physical deformation)
Normalization    : cell height Lz = 20.0000 Ang (stress x Lz: per-area stiffness)
  C11 = C22                              303.44 N/m
  C12                                    105.16 N/m
  C66                                     99.14 N/m  = (C11 - C12)/2, not independent
Young's modulus E_x = (C11 C22 - C12^2)/C22 | 266.99 N/m
Poisson ratio ν_xy = C12/C22                | 0.346
```

- The directions that would strain the vacuum are skipped, and the constants are per area, independent of the 20 Å of vacuum chosen.
- In-plane isotropy holds: two independent constants, `E_x = E_y`, `C66 = (C11 − C12)/2` and `E(`θ`)` is a circle (Fig. 10).
- The values do not: experiment gives `E_2D = 340 ± 50 N/m` [20], and DFT calculations find about 345 N/m with a Poisson ratio near 0.17. The foundation model is 22 % soft, and its Poisson ratio is twice too large — `C12` far too high. A graphene model fine-tuned with `7.1` would be needed for numbers.



In-plane Young's modulus of graphene

Fig. 10 — `E(θ)` of graphene in the plane: a circle, as for every hexagonal sheet.

5.11 SIESTA at the model's own level of theory. Every comparison above with experiment mixes two errors: that of PBE with a DZP basis, which the model was trained to reproduce, and that of the model itself. To separate them, the same tensor is computed with SIESTA and the **training `calc.fdf`** (`reference_results/siesta_elastic/calc.fdf`: PBE, DZP, `PAO.EnergyShift 0.01 Ry`, `Mesh.Cutoff 300 Ry`, k 6×6×6, 300 K smearing, the PseudoDojo `Si.psml` — the settings and md5 the model card records), on `si8.fdf`, which is SIESTA's own minimum at these settings (residual pressure +0.016 GPa). The runs go through the SIESTA elastic workflow (`5.2`):

```
bash siesta_elastic.sh      # stb-elasticInputs -> SIESTA -> stb-elasticAnalysis; 37 runs of 8
atoms, ~6 min on 6 cores
python3 siesta_compare.py   # the tables below and figures/siesta_vs_model.png (numpy + ase +
matplotlib)
```

set	what	runs
`clamped`	`xx` and `xy` strains, ±1 %, 7 points, single points (`stb-elasticInputs` forces them)	14
`relaxed`	the same cells, atoms relaxed by CG at fixed cell to 0.001 eV/Å — the model's `--fmax`	14
`hydro`	isotropic strains, ±2 % in lattice, 9 points: `B` from `E(V)` and from the pressure	9

`stb-elasticInputs` writes single points only; for relaxed ions each folder's `config_extra.fdf` is replaced by `config_extra_relaxed.fdf` (`MD.TypeOfRun CG`, `MD.MaxForceTol 0.001 eV/Ang`, `MD.VariableCell false`) — the first-occurrence-wins `%include` at the top of `calc.fdf` makes it override everything else.

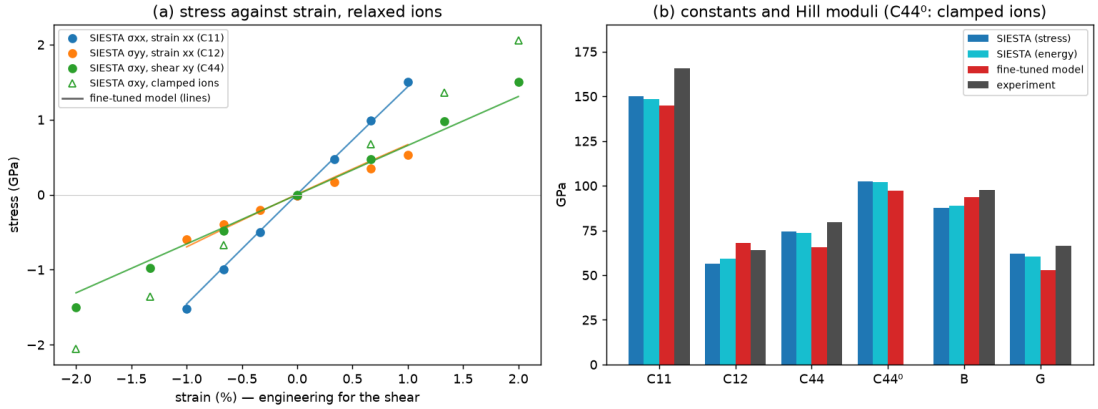
First, how precise is SIESTA here? The same runs give the constants two ways — from the stress, and from the curvature of the energy:

GPa	from the stress	from the energy
C11	150.03	148.53
C44 (relaxed)	74.40	73.74
$C44^0$ (clamped)	102.42	101.90
B	87.49 = (C11 + 2 C12)/3	88.98 (Birch–Murnaghan, ±2 %)
C12	56.22	59.21 = (3B − C11)/2

- The two routes agree to 1 % for the diagonal constants. C12 and B are the weak points: −V dP/dV between symmetric pairs of the isotropic scan gives 87.4, 90.3, 89.7 and 88.7 GPa at ±0.5, 1, 1.5 and 2 %. SIESTA's stress scatters by ~0.05 GPa at 300 Ry (the mesh and the SCF tolerance), a few percent of the stress at ±0.5 %. The fits show it too: R^2 0.9997–0.9998 against the model's 0.99996+.
- So SIESTA's constants carry 1–3 % of numerical noise at the training settings; the ranges below are the two routes.
- The E(V) of the perfect cell gives V_0 = 20.570 Å^3/atom and B_0 = 89.0 GPa, against the all-electron PBE reference of the Delta test, 20.453 Å^3/atom and 88.5 GPa [23]: DZP is close to the PBE limit here. (7.3 quoted B_0 = 94.4 from the 7.1 benchmark: that fit runs over the test set's strained cells, which are also rattled; the clean value is 89.0.)

SIESTA, the model and experiment (siesta_compare.py; ranges are stress / energy route):

GPa	SIESTA	fine-tuned	model vs SIESTA	experiment [15]	SIESTA vs experiment
C11	150.0 / 148.5	144.9	−3.4 / −2.5 %	165.6	−9.4 / −10.3 %
C12	56.2 / 59.2	68.2	+21 / +15 %	63.9	−12.0 / −7.3 %
C44	74.4 / 73.7	65.5	−12.0 / −11.2 %	79.5	−6.4 / −7.2 %
$C44^0$ (clamped)	102.4 / 101.9	97.4	−4.9 / −4.5 %	—	—
C' = (C11 − C12)/2	46.9 / 44.7	38.4	−18 / −14 %	50.9	−8 / −12 %
B (Hill)	87.5 / 89.0	93.7	+7.1 / +5.3 %	97.8	−10.5 / −9.0 %
G (Hill)	61.8 / 60.3	52.8	−14.6 / −12.4 %	66.5	−7.0 / −9.3 %
E (Hill)	150.1 / 147.6	133.4	−11.1 / −9.6 %	162.6	−7.6 / −9.2 %
Poisson ν	0.214 / 0.224	0.263		0.223	
Pugh B/G	1.41 / 1.48	1.77		1.47	
Cauchy C12 − C44	−18.2 / −14.5	+2.7		−15.6	
Debye temperature (K)	627 / 620	583		648	



SIESTA against the fine-tuned model

Fig. 11 — (a) SIESTA's stresses (points) against the model's stress-strain lines, same strains; the open triangles are SIESTA's clamped-ion shear. (b) The constants and Hill moduli: SIESTA by both routes, the model, experiment.

- **The error against experiment splits in two.** PBE/DZP puts silicon's constants 6–12 % below experiment — the usual softening of PBE at its 0.9 % larger lattice. On top of that the model adds its own error, and that error is not uniform. In `C11` and the clamped $C44^0$ it is only 3–5 %. In the shear constants it is larger: `C44` 11–12 % low, `C'` 14–18 % low. Of the 18 % by which the model's `C44` misses experiment, about 7 points are PBE/DZP and 11 are the model.
- **`C12` and `B` go the other way.** The model's `C12` is 15–21 % too high, and so is its `B`, by 5–7 %. It lands 4 % below experiment, closer than SIESTA's 9–11 %, but for the wrong reason: the model is not reproducing its own level of theory there. The `7.3` QHA gives the same 93.5, so this is the model's equation of state, not a fitting accident.
- **Most of the `C44` error is in the internal relaxation.** With the atoms clamped, the model is 5 % low; letting them relax lowers SIESTA's `C44` by 27 % (102.4 → 74.4) and the model's by 33 % (97.4 → 65.5). Under shear the model's atoms move 0.0079 Å per % strain, SIESTA's 0.0073: the model's internal relaxation is ~8 % too large. That response couples a homogeneous shear to the relative displacement of the two fcc sublattices. Only 6 of the 57 training configurations are strained cells, so the training set constrains it weakly.
- **The flipped rules of thumb are the model's.** SIESTA gives Pugh 1.41–1.48 and a Cauchy pressure of −18 to −15 GPa — brittle and covalent, as in experiment. The model's 1.77 and +2.7 GPa (section 5.2) come from its too-high `C12` and too-low shear constants, not from PBE.
- **What the model card does not tell you.** The card's stress error is 0.18 GPa on its test set. At ±1 % strain the stresses here are 0.5–1.5 GPa: an error of 0.1 GPa per % of strain is already 7 % of `C11` and 14 % of `C44` — and an error proportional to the strain hides easily in the RMS over a random test set. For elastic constants, check the model against SIESTA on the property itself, as here, and train with strained cells (`7.1`, `--volume-strains` and Stage 4 `--sampling-method strain`).

5.12 What to take from the example.

- A model fine-tuned on 57 SIESTA configurations gives silicon's elastic tensor with exact cubic symmetry, consistent with its own phonons and equation of state to 1 %; `B` is within 4 % of experiment, the shear constants ~20 % low.
- Against SIESTA at its own level of theory (section 5.11) the model is within 3–5 % in `C11` and the clamped `C44`, but 12–18 % low in the shear constants and 15–21 % high in `C12`. PBE/DZP itself is 6–12 % below experiment. Compare a model with its own level of theory before comparing it with experiment.
- Relax the atoms tightly: the internal relaxation is a third of `C44`, and a loose threshold makes it wrong.
- Use ±0.5–1 % strain; check with `--check-convergence`.
- The foundation models are 25–60 % soft; the largest is not the best.
- The symmetry analysis tells you what is independent and whether the fit respects it; a structure that is not at its minimum loses it.
- Rules of thumb built on the constants (Pugh, Cauchy, hardness) amplify their errors: a 20 % low `G` turned silicon "ductile" and "metallic".

13.4.2.6 6. Reproducing the reference data

`reference_results/` holds what `make_reference.sh` copies out of its work folder: each case's report and data files (`stress_strain.dat`, `elastic_tensor.dat`, `directional.dat` or `strain_convergence.dat`), and the gnuplot files of the `base` and `mg_hcp` cases. `figures/` holds the theory figures, the summary figures of

`results_figures.py`, and the tool's own figures of the reference runs, copied by `make_reference.sh`.

```
source ~/.anaconda3/etc/profile.d/conda.sh && conda activate stb   # or any environment with
stb_suite[ml]
bash make_reference.sh                  # every case into ./reference_run/ (GPU; DEVICE=cpu works),
then collect
python3 results_figures.py              # the tables of section 5 and figures/*.png (numpy +
matplotlib only)
python3 theory_figures.py               # the figures of section 1
```

All the cases take about 7 minutes on one RTX 3060; each is skipped when its report exists. On a CPU the 8-atom cases take about as long, the 216-atom one several minutes. The foundation models are downloaded on first use. A rerun gives the same constants to the printed digits.

The SIESTA runs of section 5.11 need a `siesta` executable (5.4.2 here, from conda-forge) and `mpirun`:

```
NP=6 bash siesta_elastic.sh            # 37 runs into ./siesta_run/ (~6 min on 6 cores), then collect
python3 siesta_compare.py               # the tables of section 5.11 and figures/siesta_vs_model.png
```

`reference_results/siesta_elastic/` keeps what they produce, without the SIESTA output files: `calc.fdf` and the two `config_extra` variants, `siesta_runs.dat` (strain measured from each cell, energy, stress tensor, residual force, CG steps and internal shift of every run), the two `stb-elasticAnalysis` reports and `siesta_compare.json`.

13.4.2.7 7. Known, deliberate limitations

- **0 K, static constants** (section 1.12): no temperature dependence, adiabatic and isothermal not distinguished.
- **Linear elasticity only.** Third-order constants appear only as the amplitude drift.
- **One model, no error bars.** The model card's stress error (0.18 GPa here) does not bound the constants; a SIESTA calculation of the constants themselves (`5.2`, section 5.11) does. It found the shear constants 12–18 % low.
- **The symmetry projection is a check**, not the tensor the moduli are computed from.
- **Empirical ratios** are rules of thumb, reported without uncertainty.

13.4.2.8 8. Step-by-step: elastic constants of your own material

1. **Choose the model.** The foundation model for a first look — expect soft shear constants. For numbers, a model trained with `7.1` that includes **strained cells** (`--volume-strains`, Stage 4 `--sampling-method strain`) — stiffness is what they teach.
2. **Start from a relaxed, symmetric structure.** The pre-relax finds the model's own minimum; `STRUCTURE` and `STIFFNESS TENSOR` say which symmetry it found. If it is lower than expected, symmetrize first (`stb-unitcell`).
3. **Run the reference fit** with `--save-report --save-data`, 5–7 strains in ±1 % (±0.5 % for soft or 2D materials).
4. **Read the report from the bottom**: `WARNINGS`, then `STRESS-STRAIN FITS` (R^2, intercepts), `TRAINED RANGE`, `STIFFNESS TENSOR` (projection deviation), `MECHANICAL STABILITY`, then the moduli.
5. **Check the amplitude** with `--check-convergence` and two or three amplitudes.
6. **Compare** with a fine-tuned model against the foundation one, with the phonon slopes (`7.3`), with the equation of state (`5.15`), and where it matters with SIESTA (`5.2`).

13.4.2.9 9. Files in this folder

File	What it is
`README.md`	this tutorial
`example_7.4.sh`	the guided walk-through (CPU, about 6 minutes; `DEVICE=cuda` for a GPU)
`si8.fdf`, `si8_rattled.fdf`	bulk silicon, 8-atom cell; the same with every atom moved by ~0.03 Å
`mg_hcp.fdf`	hcp magnesium, `c` along `z`
`graphene.fdf`	graphene sheet, 20 Å along `c`
`make_reference.sh`	regenerates `reference_results/` and the tool figures in `figures/` (GPU, about 7 minutes)
`results_figures.py`	reads `reference_results/`, prints the tables of section 5, draws the summary figures (numpy + matplotlib)
`siesta_elastic.sh`	the SIESTA runs of section 5.11 at the training level of theory (`5.2` workflow; needs `siesta`, ~6 min)
`siesta_compare.py`	SIESTA against the model and experiment: the tables of section 5.11, `figures/siesta_vs_model.png`
`theory_figures.py`	draws the figures of section 1
`figures/`	every figure of this README
`reference_results/`	reports and data of the reference runs; `siesta_elastic/` holds the SIESTA inputs and results
`../7.2-mlmd/model/`	the fine-tuned silicon model and its card, shared with `7.2` and `7.3`

13.4.2.10 10. Running the script

```
cd examples/7.4-mlelastic
bash example_7.4.sh               # CPU, ~6 min; press Enter between cases
DEVICE=cuda bash example_7.4.sh   # on a GPU
```

Each case writes into `output/<case>/`, which is wiped at every start. Cases 1, 7 and 8 download MACE-MP-0 medium on first use. Cases 12 and 13 need no MACE: they only read `reference_results/` (Case 13 the SIESTA runs of section 5.11).

13.4.2.11 11. Flag reference

`stb-mlelastic` — `--file FILE` (`.fdf`, required) · `--model {small,medium,large}` (default medium) · `--custom-model PATH` + `--custom-model-card JSON` (always both) · `--skip-foundation-comparison` · `--no-pre-relax` · `--no-relax` (clamped ions only) · `--no-clamped` (skip the clamped-ion scan) · `--fmax eV/Å` (both relaxations, default 0.001) · `--max-steps N` (default 200) · `--max %` (default 1.0) · `--steps N` (default 5, ≥ 2) · `--check-convergence` · `--convergence-strains PCT...` · `--dimensionality {auto,3d,2d,1d}` · `--vacuum-gap Å` (default 10) · `--fit-quality-tolerance` R^2 (default 0.995) · `--skip-anisotropy-surface` · `--device {cpu,cuda}` · `-o/--output-dir DIR` (default `mlelastic_out`) · `--show` · `--save-gnuplot` · `--save-data` · `--save-report` · `--no-intro` · `-v`.

13.4.2.12 12. What's next

- **`5.2` (Elastic workflow)** computes the same tensor with SIESTA — the reference to check a model against (section 5.11 does it for this model).
- **`7.1` (MLFF workflow)** trains the model; include strained cells for elastic constants.
- **`7.3` (`stb-mlphonons`)** gives the same constants from the acoustic phonon slopes (section 5.3 here) and the temperature dependence through the QHA.
- **`5.15` (equation of state)** fits SIESTA's `E(V)`: its B_0 is this tool's `B`, a cross-check.

- **`7.2` (`stb-mlmd`)** gives the finite-temperature behaviour the static constants leave out.

13.4.2.13 13. References

1. J. F. Nye, *Physical Properties of Crystals* (Oxford University Press, 1957; reprinted 1985).
2. M. Born, K. Huang, *Dynamical Theory of Crystal Lattices* (Oxford University Press, 1954).
3. D. C. Wallace, *Thermodynamics of Crystals* (Wiley, 1972).
4. Y. Le Page, P. Saxe, "Symmetry-general least-squares extraction of elastic data for strained materials from ab initio calculations of stress", *Phys. Rev. B* **65**, 104104 (2002).
5. F. Mouhat, F.-X. Coudert, "Necessary and sufficient elastic stability conditions in various crystal systems", *Phys. Rev. B* **90**, 224104 (2014).
6. R. Hill, "The elastic behaviour of a crystalline aggregate", *Proc. Phys. Soc. A* **65**, 349 (1952).
7. A. Reuss, "Berechnung der Fließgrenze von Mischkristallen auf Grund der Plastizitätsbedingung für Einkristalle", *Z. Angew. Math. Mech.* **9**, 49 (1929); W. Voigt, *Lehrbuch der Kristallphysik* (Teubner, 1928).
8. S. I. Ranganathan, M. Ostoja-Starzewski, "Universal elastic anisotropy index", *Phys. Rev. Lett.* **101**, 055504 (2008).
9. S. F. Pugh, "Relations between the elastic moduli and the plastic properties of polycrystalline pure metals", *Philos. Mag.* **45**, 823 (1954).
10. X.-Q. Chen, H. Niu, D. Li, Y. Li, "Modeling hardness of polycrystalline materials and bulk metallic glasses", *Intermetallics* **19**, 1275 (2011).
11. L. Kleinman, "Deformation potentials in silicon. I. Uniaxial strain", *Phys. Rev.* **128**, 2614 (1962).
12. O. H. Nielsen, R. M. Martin, "Quantum-mechanical theory of stress and force", *Phys. Rev. B* **32**, 3780 (1985).
13. O. L. Anderson, "A simplified method for calculating the Debye temperature from elastic constants", *J. Phys. Chem. Solids* **24**, 909 (1963).
14. M. J. P. Musgrave, *Crystal Acoustics* (Holden-Day, 1970).
15. J. J. Hall, "Electronic effects in the elastic constants of n-type silicon", *Phys. Rev.* **161**, 756 (1967).
16. G. Simmons, H. Wang, *Single Crystal Elastic Constants and Calculated Aggregate Properties: A Handbook*, 2nd ed. (MIT Press, 1971): room-temperature values of magnesium, rounded.
17. I. Batatia, D. P. Kovács, G. N. C. Simm, C. Ortner, G. Csányi, "MACE: Higher order equivariant message passing neural networks for fast and accurate force fields", *Adv. Neural Inf. Process. Syst.* **35**, 11423 (2022).
18. I. Batatia et al., "A foundation model for atomistic materials chemistry", arXiv:2401.00096 (2023).
19. B. Deng, Y. Choi, P. Zhong, J. Riebesell, S. Anand, Z. Li, K. Jun, K. A. Persson, G. Ceder, "Systematic softening in universal machine learning interatomic potentials", *npj Comput. Mater.* **11**, 9 (2025).
20. C. Lee, X. Wei, J. W. Kysar, J. Hone, "Measurement of the elastic properties and intrinsic strength of monolayer graphene", *Science* **321**, 385 (2008).
21. A. Togo, K. Shinohara, I. Tanaka, "Spglib: a software library for crystal symmetry search", *Sci. Technol. Adv. Mater.: Methods* **4**, 2384822 (2024).
22. A. H. Larsen et al., "The atomic simulation environment — a Python library for working with atoms", *J. Phys.: Condens. Matter* **29**, 273002 (2017).
23. K. Lejaeghere et al., "Reproducibility in density functional theory calculations of solids", *Science* **351**, aad3000 (2016); all-electron PBE reference (WIEN2k) of silicon from the Delta-codes DFT repository, `https://github.com/molmod/DeltaCodesDFT`, file `WIEN2k.txt`: `V0 = 20.4530 Å³/atom`, `B0 = 88.545 GPa`.

13.4.3 Command-line reference

13.4.3.1 `stb-mlelastic`

Elastic constants by the stress-strain method, driven entirely by a MACE potential – the MACE-MP-0 foundation model, or a custom model fine-tuned on your own SIESTA data via `stb-mlffAnalysis` (`--custom-model`, always with its model card). Strains the relaxed cell along every canonical Voigt direction not blocked by a vacuum axis, relaxes the atoms at each strained cell, fits the stiffness tensor and reports: each model's reference state, fit quality per direction, the strained cells against the card's trained range, the tensor projected on the crystal's point group, Born stability, Voigt/Reuss/Hill moduli, clamped- vs relaxed-ion constants, Christoffel sound velocities and the Debye temperature, directional Young's modulus and compressibility, and fine-tuned vs foundation; every warning in one final section. Or, with `--check-convergence`, the fit at several strain amplitudes. A fast preview, not a substitute for the SIESTA-based `stb-elasticInputs/stb-elasticAnalysis` workflow.

Synopsis.

```
usage: stb-mlelastic [-h] --file FILE [--model {small,medium,large}]
                     [--custom-model PATH] [--custom-model-card JSON]
                     [--skip-foundation-comparison] [--no-pre-relax] [--no-relax]
                     [--no-clamped] [--fmax FMAX] [--max-steps MAX_STEPS] [--max MAX]
                     [--steps STEPS] [--check-convergence]
                     [--convergence-strains PCT [PCT ...]]
                     [--dimensionality {auto,3d,2d,1d}] [--vacuum-gap VACUUM_GAP]
                     [--fit-quality-tolerance FIT_QUALITY_TOLERANCE]
                     [--skip-anisotropy-surface] [--device {cpu,cuda}] [-o OUTPUT_DIR]
                     [--show] [--save-gnuplot] [--save-data] [--save-report]
                     [--no-intro] [-v]
```

Options

`--file FILE`

Input structure (.fdf). required

`--model {small,medium,large}`

MACE-MP-0 model size (default: medium). Ignored if `--custom-model` is given. choices: small, medium, large

`--custom-model PATH`

Use a custom MACE model file instead of the MACE-MP-0 foundation potential – e.g. one fine-tuned via `stb-mlffAnalysis`. Overrides `--model`.

`--custom-model-card JSON`

Model card of the `--custom-model` file: the `mlff_model_card.json` that `stb-mlffAnalysis` wrote next to it (training data, level of theory, accuracy, energy scale). Required with `--custom-model` – the card is checked against the model file and used to warn when a structure is outside what the model was trained on.

`--skip-foundation-comparison`

With `--custom-model` the same calculation is repeated with the raw foundation model (`--model`) and compared constant by constant; this flag skips it. `--check-convergence` never compares. switch, off by default

`--no-pre-relax`

Skip the pre-relax of the reference structure (cell + atoms). On by default: the constants are second derivatives about a stress-free minimum; a residual stress shows up as the fits' intercepts. switch

`--no-relax`

Do not relax the atoms at each strained cell (clamped-ion constants only). On by default: the measured constants are relaxed-ion ones (diamond's C44 depends on it). switch

`--no-clamped`
: Skip the clamped-ion scan that is compared with the relaxed-ion constants (it doubles the number of stress evaluations; run only when the cell has 2+ atoms). switch

`--fmax FMAX`
: Force threshold of both relaxations, eV/Ang (default: 0.001). Tight on purpose: under shear the atoms of a diamond-type cell relax by only ~0.008 Ang per % strain, and a residual force f leaves them ~f/k off (k ~ 13 eV/Ang^2): 0.02 made silicon's C44 3 % low and noisy, 0.001 is converged to 0.3 %. With MACE the extra steps cost milliseconds. type: float

`--max-steps MAX_STEPS`
: Max optimizer steps of both relaxations (default: 200). type: int

`--max MAX`
: Max strain magnitude, % (default: 1.0). type: float

`--steps STEPS`
: Strain points per direction, spanning [-max, +max] (default: 5, which includes the unstrained cell). type: int

`--check-convergence`
: Instead of one fit, repeat it at several `--max` strain magnitudes (`--convergence-strains`) and compare every independent constant: does the linear regime hold? No foundation comparison in this mode. switch, off by default

`--convergence-strains PCT [PCT ...]`
: Max strain magnitudes (%) for `--check-convergence` (default: half, the given `--max`, and double it). type: float

`--dimensionality {auto,3d,2d,1d}`
: Physical dimensionality, controlling report units and which constants are physically meaningful (default: auto-detected from vacuum padding). Manual override assumes the same axis convention as `stb-elasticAnalysis`: in-plane xy for 2D, wire-along-c for 1D. choices: auto, 3d, 2d, 1d

`--vacuum-gap VACUUM_GAP`
: Gap (Ang) used to detect vacuum-padded axes (default: 10.0). type: float

`--fit-quality-tolerance FIT_QUALITY_TOLERANCE`
: Minimum R^2 of a direction's stress-strain fit before it is flagged (default: 0.995). type: float

`--skip-anisotropy-surface`
: Do not draw the 3D Young's-modulus surface (`anisotropy_surface.png`). The directional numbers and the polar plots are kept. switch, off by default

`--device {cpu,cuda}`
: Device to run the model on (default: cpu). choices: cpu, cuda

`-o, --output-dir OUTPUT_DIR`
: Output directory (default: mlelastic_out).

`--show`
: Open the figures on screen at the end (needs a display; without one it is a warning). The PNGs are written either way. switch, off by default

`--save-gnuplot`
: Write .dat + .gplot into <output-dir>/plot/ and run gnuplot there (one PDF per figure) when it is installed. Off by default. switch, off by default

`--save-data`
: Write the data behind the report: `stress_strain.dat` (every strained cell's stress), `elastic_tensor.dat` (C, its projection, S), `directional.dat`. Off by default. switch, off by default

`--save-report`
: Also persist the report to `stb_mlelastic_report.txt`. Off by default. switch, off by default

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Example usage:
  stb-mlelastic --file structure.fdf
  stb-mlelastic --file structure.fdf --custom-model mlff_model.model --custom-model-card mlff_model_card.json
 --device cuda --steps 7 --save-report
  stb-mlelastic --file structure.fdf --check-convergence --convergence-strains 0.25 0.5 1 2 3
```

13.5 ML NEB

At a glance

Menu code: 7.5 **Command:** `stb-mlneb`
Guided example: `examples/7.5-mlneb/`
Summary: Climbing-image NEB driven by a MACE potential – the foundation model or a fine-tuned one with its model card: the endpoints relaxed with the model, the band image by image, the barrier forward and backward, the saddle geometry, the band against the trained range, the foundation model and the committee on the same band. It closes the loop with SIESTA: `-siesta-check` writes single points of the ends and the saddle at the training level of theory and compares them with the model on the next run; `-to-mlff` adds the band to the training set of the next round of the MLFF workflow.

13.5.1 Overview

`stb-mlneb` computes the minimum-energy path and the barrier between two endpoint structures with a climbing-image NEB [73–75] in which every energy and force comes from a MACE potential [15]: the MACE-MP-0 foundation model [16] (`-model`) or a model fine-tuned with the MLFF workflow (7.1; `-custom-model` with `-custom-model-card`). It shares the geometric machinery of **`stb-neb`** (5.9) – composition check, wrapping into the cell, atom matching, linear interpolation refined by IDPP [76] – and the NEB itself is ASE's [8], relaxed with FIRE [45] in two stages (shaping, then climbing). A band of a 63-atom cell converges in under a minute on a GPU. The model's weak point is the saddle, the least likely structure in a training set, so the tool also measures it against SIESTA and can feed the band back to training (Fig. 13.4).

The climbing-image NEB in six equations

The minimum-energy path is where the force has no component perpendicular to the path, $\hat{\tau}$ its unit tangent [74]: $\mathbf{F}_\perp = -\nabla E + (\nabla E \cdot \hat{\tau})\hat{\tau} = 0$. Its highest point is a first-order saddle, and the barrier $E_a = E(\text{saddle}) - E(\text{initial})$ enters the rate of harmonic transition-state theory exponentially [110]:

$$\Gamma = \nu^* \, e^{-E_a/k_B T}, \qquad \nu^* = \frac{\prod_{j=1}^{3N} \nu_j^{\min}}{\prod_{j=1}^{3N-1} \nu_j^{\text{saddle}}};$$

an error δE_a multiplies Γ by $e^{\delta E_a/k_B T}$: 48 for 0.1 eV at 300 K. **Nudging** [74]. Each image keeps the true force perpendicular to the band and the spring force along it, so the band neither cuts corners nor slides into the minima:

$$\mathbf{F}_i = -\nabla E(\mathbf{R}_i) + \big(\nabla E(\mathbf{R}_i)\cdot\hat{\tau}_i\big)\hat{\tau}_i + k\big(|\mathbf{R}_{i+1}-\mathbf{R}_i| - |\mathbf{R}_i-\mathbf{R}_{i-1}|\big)\hat{\tau}_i.$$

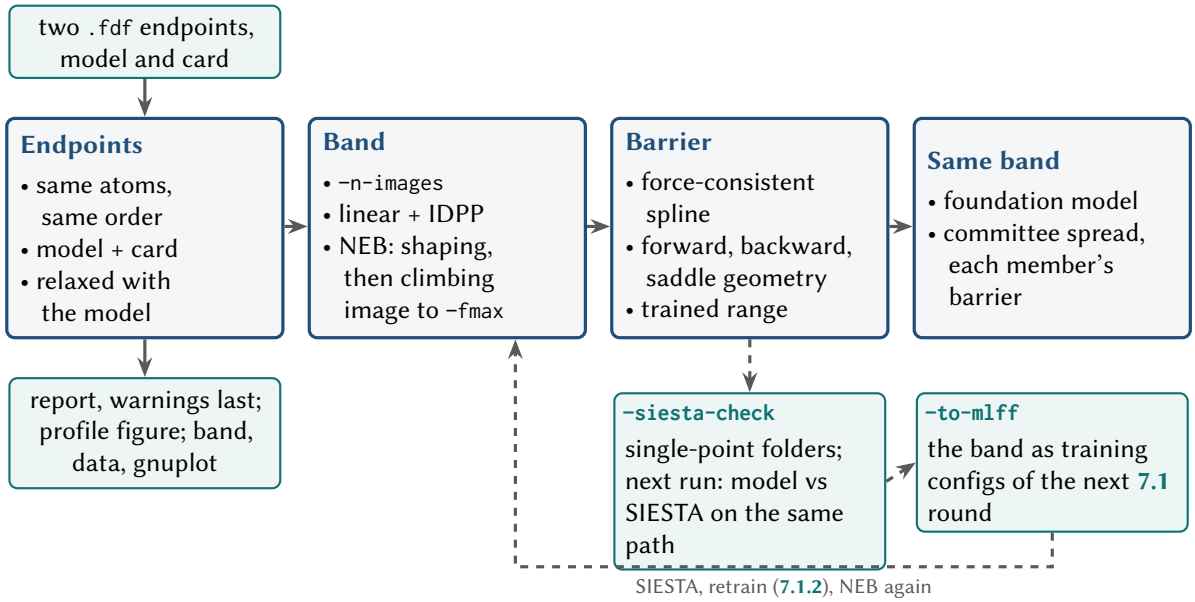


Figure 13.4: What one **stb-mlneb** run does, and the loop it closes. The SIESTA single points and the training (**stb-mlffAnalysis**, **stb-mlffValidate**) are run by the user between two calls.

Improved tangent [73]: $\boldsymbol{\tau}_i = \mathbf{R}_{i+1} - \mathbf{R}_i$ if $E_{i+1} > E_i > E_{i-1}$, $\mathbf{R}_i - \mathbf{R}_{i-1}$ if $E_{i+1} < E_i < E_{i-1}$, an energy-weighted mix at an extremum. **Climbing image** [75]: the highest image drops its springs and inverts its parallel force, $\mathbf{F}_{\max} = -\nabla E + 2(\nabla E \cdot \hat{\tau})\hat{\tau}$, and converges onto the saddle. **Starting path**: IDPP [76] relaxes each linearly interpolated image toward the interpolated interatomic distances, $S^{(i)} = \sum_{a<b} d_{ab}^{-4} \left[d_{ab}^{(i)} - d_{ab}\right]^2$, before any energy is computed. **The profile** between images is a cubic per segment matching the energies and the slopes $dE/ds = -\mathbf{F}_i \cdot \hat{\tau}_i$ (ASE `fit_images`); its maximum is the reported barrier, the highest image a lower bound. **SIESTA on the model's path.** Around SIESTA's own stationary point $\mathbf{R}^*$, the model's geometry $\mathbf{R}^* + \boldsymbol{\delta}$ gives $E_{\mathrm{S}}(\mathbf{R}^* + \boldsymbol{\delta}) = E_{\mathrm{S}}(\mathbf{R}^*) + \frac{1}{2}\boldsymbol{\delta}^{\mathsf{T}} H \boldsymbol{\delta} + O(\delta^3)$: the check approximates SIESTA's barrier to second order, and SIESTA's force $-H\boldsymbol{\delta}$ on each checked image measures $\boldsymbol{\delta}$.

Before you trust an ML barrier

- **A barrier is only as good as the training data near the saddle.** In the guided example a model with a 28.7 meV/Å test error on the perfect crystal gave 0.72 eV for a vacancy hop in silicon; SIESTA on the same path gave 0.25 eV (Section 13.5.2.6).
- **The trained-range check and the committee are not enough.** The band was inside every range of the card, and the committee's barriers (0.63–0.72 eV) agreed with each other: seeds fine-tuned from one foundation model are wrong together.
- **Check with SIESTA** (`-siesta-check`): single points on the ends and the saddle cost minutes and measure the model's error where it matters. Read SIESTA's forces there: an endpoint that is not SIESTA's minimum biases the barrier measured from it (the foundation model's 0.30 eV was close by chance).
- **Fix the model with the band** (`-to-mlff`, then 7.1.2): 14 configurations took the example from +186 % to +23 % in one round, and improved the model on the crystal too.
- **Use enough images**: a spline barrier more than 0.05 eV above the highest image means the band is too coarse near the saddle.
- **A barrier at the training settings is not the physical one**: converge the cell, the charge state

and the spin as in any DFT defect calculation, and confirm a number to publish with a SIESTA NEB (**stb-neb**).

13.5.2 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/7.5-mlneb/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

`stb-mlneb` finds the **minimum-energy path** between two stable structures and the **energy barrier** along it with the climbing-image nudged elastic band method (CI-NEB), with every energy and force from a MACE machine-learned potential: the MACE-MP-0 foundation model, or a model you fine-tuned on your own SIESTA calculations with the MLFF workflow (`7.1`), which always comes with its **model card**. You give it the two endpoints — a vacancy before and after a hop, an adsorbate on two sites, the two sides of a small rearrangement — and it reports:

- the atoms that move, the endpoints relaxed with the model, and the starting path;
- the band image by image, the barrier forward and backward and the reaction energy, the saddle-point geometry;
- the band against the model card's trained range;
- with a fine-tuned model: the same NEB with the foundation model, and the spread of the model's committee on the band.

It also closes the loop with SIESTA. `--siesta-check` writes SIESTA single points of the endpoints and the saddle at the model's own level of theory, and compares the model with SIESTA on the same geometries when they are done. `--to-mlff` adds the band's images to the training set of the next round of the MLFF workflow.

Why a potential for a barrier? A SIESTA NEB (`5.9`) costs one SIESTA calculation per image per optimizer step: seven images, a few dozen steps, a few hundred single points of a 63-atom supercell — a day of a small cluster. With a MACE model the same band converges in under a minute on a GPU. The price is the usual one: the model is only as good as its training data near the saddle. A transition state is the least likely structure in a training set built from perfect crystals, rattles and molecular dynamics.

The worked example (section 6) is a real loop on a GPU and 6 CPU cores: a vacancy hop in a 63-atom silicon supercell with the fine-tuned model of the `7.2`–`7.4` examples.

- The model, validated on the perfect crystal (test force error 28.7 meV/Å), gives a barrier of **0.72 eV**.
- Neither its trained range nor its committee sees anything wrong.
- **Seven SIESTA single points** on its own path give **0.25 eV**.
- Fourteen of those configurations, added to the training set, give a round-3 model with **0.31 eV**. Round 3 is also better on the perfect crystal.

That loop — NEB, check, retrain, NEB — is the main lesson of this example.

13.5.2.1 1. Theory

Symbols at a glance

Symbol	Meaning
`R`	the 3N atomic coordinates of a configuration; `E(R)` its energy, `F = −∇E` its forces
`R_0 . . . R_P`	the images of the band; `R_0` and `R_P` are the endpoints (fixed), `P + 1 = --n-images`

Symbol	Meaning
τ_i	unit tangent to the band at image i
F⊥, F∥	components of a force perpendicular and parallel to the band (F∥ = (F·τ) τ)
k	spring constant between neighbouring images (--k, eV/Å^2)
s	reaction coordinate: the path length along the band from R_0 (Å)
E_a, ΔE	barrier (E(saddle) − E(initial)) and reaction energy (E(final) − E(initial))
H	Hessian matrix ∂^2E/∂R∂R; its eigenvalues are k_B T-independent curvatures
ν*, Γ	attempt frequency and hop rate of transition-state theory
f_max	convergence threshold on the largest force component of the band (--fmax, eV/Å)

1.1 The potential energy surface: minima and saddle points. In the Born–Oppenheimer approximation the atoms move on a potential energy surface E(R). Two kinds of stationary points (∇E = 0) matter here:

- a **minimum**: every eigenvalue of the Hessian H is positive. A stable or metastable structure: the vacancy before and after the hop;
- a **first-order saddle point**: exactly one negative eigenvalue. Along its eigenvector the energy is a maximum; along every other direction a minimum. It is the **transition state** between two minima.

A **minimum-energy path** (MEP) connects two minima through a saddle point. At every point of it the force has no component perpendicular to the path [1]:

$$F\perp(R) = -\nabla E(R) + (\nabla E(R)\cdot\tau)\,\tau = 0 \qquad \text{on the MEP} \tag{1}$$

The MEP is the most probable route of a thermally activated jump at low temperature. Its highest point is the saddle, and the **barrier** is E_a = E(saddle) − E(initial). Forward and backward barriers differ by the reaction energy: E_a^back = E_a − ΔE. A path may also cross more than one saddle with intermediate minima in between (Fig. 1c). Then the transition is two steps, and each step is its own barrier.

1.2 Why the barrier: transition-state theory. Harmonic transition-state theory (Vineyard [2]) gives the rate of a jump over a saddle point:

$$\Gamma = \nu^* \exp(-E_a / k_B T), \qquad \nu^* = \Pi_{j=1}^{3N} \nu_j^{(min)} \,/\, \Pi_{j=1}^{3N-1} \nu_j^{(saddle)} \tag{2}$$

- The prefactor ν* is a ratio of the normal-mode frequencies at the minimum and at the saddle, the unstable mode left out. It is of the order of a phonon frequency, 10^{12}–10^{13} Hz. stb-mlneb does not compute it: it needs the Hessian at both points (a phonon calculation, 7.3, of each).
- **The barrier enters an exponential.** An error δE_a multiplies the rate by exp(δE_a / k_B T) (Fig. 4b): 0.1 eV is a factor 48 at 300 K and 3 at 1000 K. This is why a barrier needs an accuracy of a few hundredths of an eV, much tighter than the error a potential is usually judged by.

For a vacancy in the diamond lattice, each hop moves the vacancy by one bond length d = $\sqrt{3}$ a / 4 to one of Z = 4 neighbours. A random walk of such hops gives the vacancy diffusion coefficient [3]:

$$D_V = (1/6)\, Z\, d^2\, \Gamma = (2/3)\, d^2\, \nu^* \exp(-E_m / k_B T) \tag{3}$$

E_m, the migration barrier, is what the NEB of section 6 computes.

1.3 The elastic band: a chain of states. Searching for the saddle directly is hard: a minimizer runs away from it, and a saddle search must know which mode to climb. The **chain-of-states** methods instead discretize the whole path [1, 4]. `P − 1` intermediate images `R_1 . . . R_{P−1}` between the fixed endpoints are relaxed *together*, each one held to its neighbours by springs:

```
F_i^plain = −∇E(R_i) + k (R_{i+1} − 2 R_i + R_{i−1})          a plain elastic band (do not use)
(4)
```

This simple form fails in two opposite ways (Fig. 1a):

- **Corner cutting.** With stiff springs, the spring force perpendicular to a curved path pulls the band straight. It cuts across the corner, misses the saddle, and overestimates the barrier.
- **Sliding down.** With soft springs, the true force parallel to the band pushes images down into the minima. Too few are left near the saddle, and the barrier is underestimated.

1.4 Nudging. The **nudged** elastic band [4] removes both problems with two projections (Fig. 2). Each image keeps only the part of the true force perpendicular to the band, and only the part of the spring force along it:

```
F_i^NEB = F⊥_i  +  F∥,spring_i
        = [−∇E(R_i) + (∇E(R_i)·τ_i) τ_i]  +  k (|R_{i+1} − R_i| − |R_i − R_{i−1}|) τ_i
        (5)
```

- **The perpendicular true force** moves each image sideways, towards the MEP. It is zero on the MEP (Eq. 1), so a converged band lies on it, whatever `k`.
- **The parallel spring force** only keeps the images evenly spaced along the band. It never pulls the band off the MEP. The value of `k` changes the spacing, not the converged path. `--k` defaults to 0.1 eV/Å^2.

The tangent. The projections need τ`_i`, and its definition matters. The simple bisector of the two neighbouring segments gives kinks on a band where the force along the path is large compared with the perpendicular one. The **improved tangent** [5], the one used here, points toward the *higher-energy* neighbour:

```
τ_i = R_{i+1} − R_i   if E_{i+1} > E_i > E_{i−1};      τ_i = R_i − R_{i−1}   if E_{i+1} < E_i <
E_{i−1}            (6)
```

At an extremum of the band, the two segments are mixed with weights Δ`E_max = max(|E_{i+1} − E_i|, |E_{i−1} − E_i|)` and Δ`E_min`, the larger weight on the side of the higher neighbour.

1.5 The climbing image. A converged NEB samples the MEP at discrete images, and none of them sits exactly on the saddle. In the **climbing-image** variant [6], once the band is roughly in place, the highest image `i_max` drops its springs. Its force along the band is also inverted:

```
F_{i_max}^CI = −∇E(R_{i_max}) + 2 (∇E(R_{i_max})·τ) τ
(7)
```

- The image now **climbs** up along the band and **relaxes down** perpendicular to it. At convergence it sits exactly on the saddle point, so its energy is the barrier.
- The neighbouring images keep their springs. They space themselves evenly on each side of the climbing image, so the spacing differs on the two sides.
- Climbing from the start can send the image to a wrong maximum. `stb-mlneb` therefore climbs in a second stage.

1.6 The starting path: linear interpolation and IDPP. The band must start from somewhere. A **linear interpolation** of the Cartesian coordinates, `R_i = R_0 + (i/P)(R_P − R_0)`, can put two atoms on top of each other when one moves past another. The **image-dependent pair potential** (IDPP) [7] fixes that before any energy is computed. Each image is relaxed on an artificial surface that makes its interatomic distances `d_ab` close to the linearly interpolated distances `d_ab^(i)`:

```
S^(i)(R) = Σ_{a<b} w(d_ab) [d_ab^(i) − d_ab(R)]², d_ab^(i) = d_ab^(0) + (i/P)(d_ab^(P) −
d_ab^(0)), w(d) = 1/d⁴ (8)
```

The weights `w = 1/d⁴` favour the short distances. IDPP costs nothing (no model evaluation) and gives a path that keeps bond lengths reasonable. For the single-atom hop of section 6 it changes almost nothing; for concerted moves of many atoms it is the difference between a band that converges and one that does not. `--no-idpp` skips it (section 6.7).

Atom correspondence. Image `i` interpolates atom `j` of the initial structure to atom `j` of the final one. The two `.fdf` files must therefore list the same atoms in the same order. `stb-mlneb` checks the composition and re-matches the order by distance (`--autosort-tol`); the report lists the atoms that move and flags a jump in the starting path that looks like a wrong match.

1.7 The optimizer and convergence. All the images move together under the NEB forces, with the FIRE optimizer [8]: a velocity-Verlet integration with the velocity mixed toward the force and reset when the motion goes uphill. It is robust for NEB, where the forces are not the gradient of any single function, so a line search is ill-defined. Convergence is reached when the largest NEB force component on the band is below `f_max`. Note what is and is not converged:

- on an ordinary image, the **perpendicular** force (plus the spring). The raw force `|F|` of an interior image stays large, because its component along the path is never relaxed. The report prints the raw force, and a value above `f_max` there is normal;
- on the climbing image, the **full** force. It is a stationary point, so its raw `|F|` is below `f_max`.

`stb-mlneb` runs two stages:

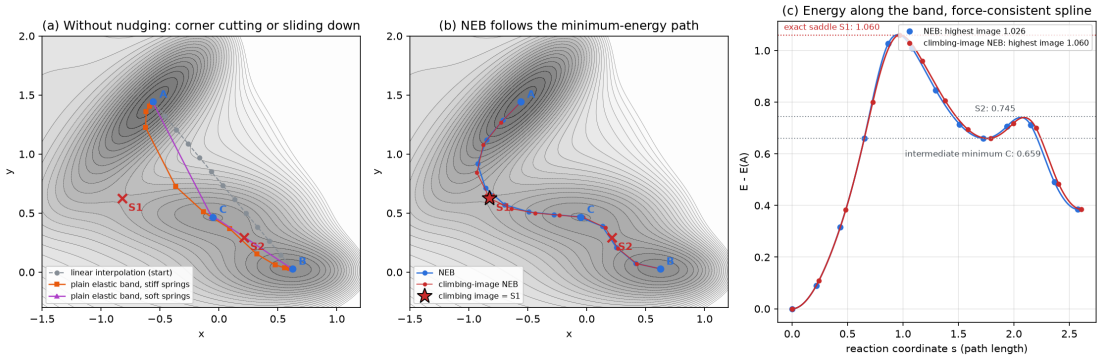
1. **Shaping**: a plain NEB to `max(2 f_max, 0.1)` eV/Å, in at most half of `--max-steps`.
2. **Climbing**: the climbing image to `f_max` (`--fmax`, default 0.05 eV/Å), with the rest of the steps.

The endpoints are relaxed with the model first (`--fmax`, positions only), so that the barrier is measured from the model's own minima (section 6.7 shows what happens without it).

1.8 Reading the barrier off the band: the force-consistent spline. The band gives the energy at `P + 1` points. Between them, `stb-mlneb` uses ASE's `fit_images` [9]: a cubic polynomial on each segment that matches the two image energies and the two **slopes**. The slope comes from the force along the band, which the NEB computes anyway:

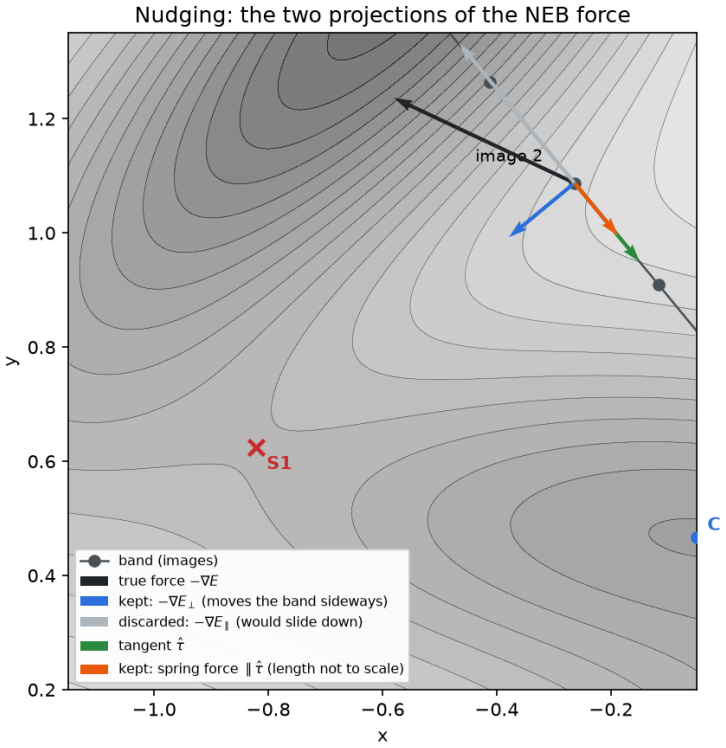
```
dE/ds |_i = −F_i · τ_i (Hermite cubic on each segment [s_i, s_{i+1}])
(9)
```

The spline's maximum is the **reported barrier**; the highest image is a lower bound. With a converged climbing image the two coincide, because the climbing image *is* the maximum. They differ when the band is too coarse, or when the climbing image did not converge (Fig. 3). A difference above 0.05 eV is a warning: use more images. The same spline, with the reaction energy, gives the backward barrier, and its minima between the images reveal an **intermediate minimum** — a two-step transition that needs two NEBs.



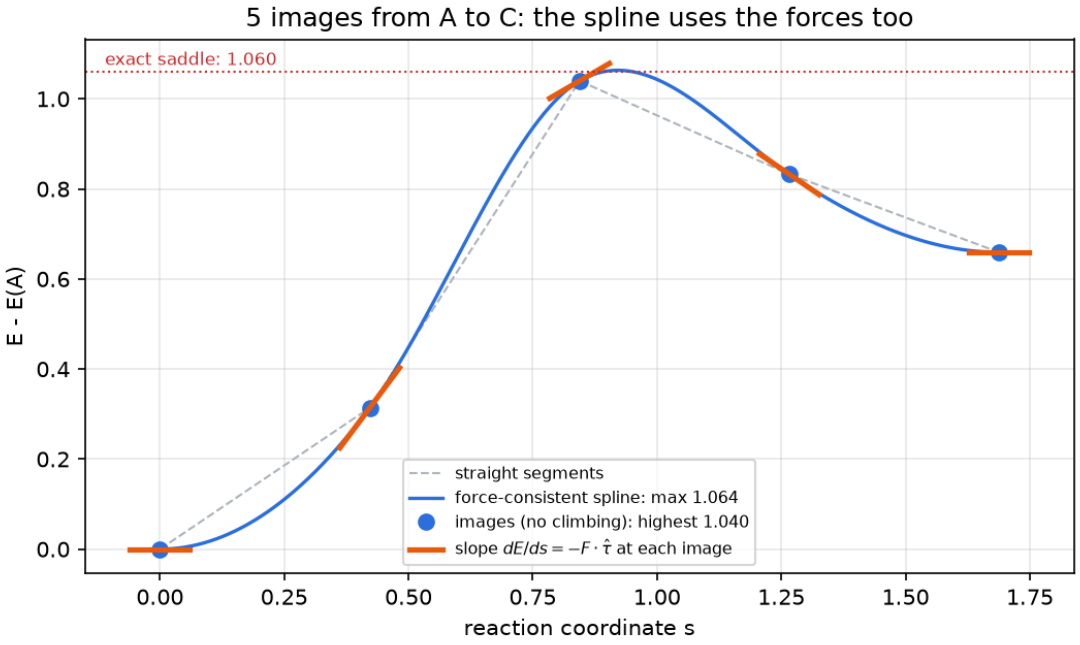
NEB on the Müller–Brown surface

Fig. 1 — The Müller–Brown surface [10] (energies ×0.01), three minima A, C, B and two saddles S1, S2. Every band here was computed by `theory_figures.py` *with Eqs. 4–7 and FIRE, 13 images from A to B. (a) A plain elastic band (Eq. 4): stiff springs cut the corner past S1, soft springs let the images slide into the minima. (b) NEB (Eq. 5) follows the MEP through C. The climbing image (Eq. 7) lands on S1 to 10^{-3} in the coordinates. (c) The energy along the band and the force-consistent spline (Eq. 9). The NEB's highest image is 0.03 below the exact saddle; the climbing image is exact. The intermediate minimum C makes this a two-step path.*



The two projections of the NEB force

Fig. 2 — The force on one image of a band that is not converged yet. The true force (black) is split along the tangent (green). Its perpendicular part (blue) is kept and moves the band toward the MEP. Its parallel part (grey) is discarded: it would slide the image downhill. The spring force acts only along the tangent (orange).



Highest image against the spline

Fig. 3 — Five images from A to C, without climbing. The highest image (1.040) misses the saddle (1.060). The force-consistent spline, which uses the slope $-\mathrm{F}\cdot\tau$ at each image (orange ticks), finds 1.064. Straight segments would only reproduce the highest image.

1.9 The test case: a vacancy hop in silicon. A vacancy in diamond-structure silicon leaves four atoms with one dangling bond each. It migrates when one of those four neighbours jumps into the empty site. The vacancy then sits where the neighbour was, one bond length away (`d = √3 a / 4 = 2.37 Å` at SIESTA's `a = 5.48 Å`).

- **At the endpoints**, the hopping atom has three bonds to its own neighbours (2.37–2.38 Å in the relaxed models) and faces the vacancy.
- **At the saddle**, it sits halfway, at the centre of two half-vacancies (the *split vacancy*, `D3d` symmetry). It is at the same distance, 2.76–2.78 Å, from six atoms, the three of each side; the report lists the four nearest.
- The path is symmetric: Δ`E = 0`, the saddle at 50 % of the path.

This is one of the best-studied defects in semiconductors, and its numbers are hard ones:

- **Experiment** (electron paramagnetic resonance after irradiation at cryogenic temperature [11, 12]) gives a migration energy of 0.45 eV for the neutral vacancy, 0.32 eV for V^{2+} and 0.18 eV for V^{2-}.
- **DFT** is notoriously slow to converge with the supercell. The vacancy's levels are deep and its distortions long ranged (Jahn–Teller), so 64-atom cells and dense k-grids give scattered results [13, 14]. Well-converged GGA calculations, extrapolated from 215–999-atom cells, give 0.36 eV for the neutral vacancy [15]; SIESTA with a 216-atom cell gave 0.40 eV [16].

The 63-atom cell of section 6 (a 2×2×2 supercell, non-spin-polarized, 3×3×3 k-points) is therefore **not** a converged vacancy calculation: its SIESTA barrier on the model's path, 0.25 eV, is a property of this cell and these settings. It is the right test for what this example asks — does the model reproduce SIESTA *at*

the training settings, in this cell? — because model and SIESTA see exactly the same cell. Section 6.8 comes back to this.

1.10 Barriers from a machine-learned potential. A MACE model is fitted to energies, forces and stresses of the training configurations. Its predictions are reliable where its training data are dense, and degrade gracefully, without any built-in warning, away from them. For a barrier this has three consequences:

1. **The saddle is out of distribution by nature.** Rattled crystals, strained cells and MD frames rarely contain a five-fold-coordinated atom between two half-vacancies. The model card can show how far the band is from the trained range in cell size, bond length, force and energy (report section `TRAINED RANGE`). It cannot show whether a *kind* of structure was in the training set. In section 6 the band is inside every range and the barrier is still 3× too high.
2. **A committee is a lower bound of the error.** Models fine-tuned with different seeds from the same foundation model share most of their errors. The spread of their forces measures how much the seeds disagree, not how wrong they all are. `stb-mlffValidate` calibrates the ratio error/spread on the test set (1.3–1.7 here), and `stb-mlneb` uses that calibration. Even so, the round-2 committee gave 0.63–0.72 eV against SIESTA's 0.25.
3. **The only real check is the reference method itself.** SIESTA single points on the model's own band cost a few minutes per image. They measure the model's error at exactly those geometries.

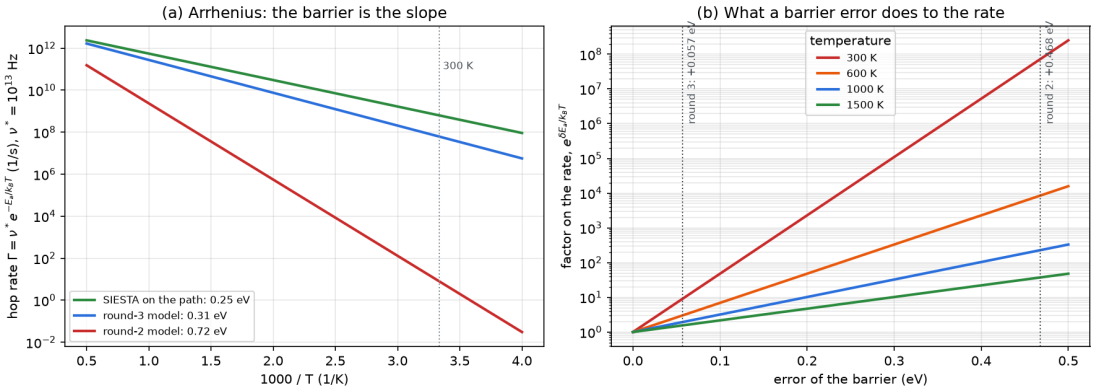
What a SIESTA single point on the model's path measures. Call `R*` the true SIESTA stationary point (minimum or saddle) and `R_m = R* + δ` the model's. Expanding SIESTA's energy around `R*`, where its gradient vanishes:

> `E_SIESTA(R_m) = E_SIESTA(R*) + ½ δ`^{T} `H δ + O(δ`^{3}`)`
> `(10)`

- The error is **second order** in the geometry error. SIESTA's barrier *on the model's path* approaches SIESTA's own barrier quadratically as the model improves.
- The SIESTA force on the model's geometry, `F = −H δ`, measures δ. The report prints SIESTA's `max|F|` for each checked image: a small one means the model's geometry is close to SIESTA's own stationary point.
- At a minimum the correction is positive: an endpoint that is not at SIESTA's minimum sits **too high**, which makes the barrier on that path look **too low**. At the saddle the correction can have either sign (one curvature is negative).

This is why the foundation model's path gives SIESTA only 0.185 eV in section 6.6: its endpoints are 0.05 eV above the fine-tuned ones in SIESTA's energy.

Closing the loop. The checked images are exactly the configurations the model lacks. Added to the training set (`--to-mlff`), with a few randomly displaced copies to sample their neighbourhood (`--to-mlff-rattle`), they teach it the region of the saddle in one round (section 6.5). This is active learning driven by the property of interest, and it complements the committee-driven sampling of `7.1.4`.



Rate against barrier

Fig. 4 — (a) Hop rate (Eq. 2, $\nu* = 10^{13}$ Hz) for the three barriers of section 6: the slope of each line is the barrier. (b) The factor an error in the barrier puts on the rate. The round-2 model's +0.47 eV is a factor 7 × 10^7 at 300 K; round 3's +0.06 eV is 9 at 300 K and 2 at 1000 K.

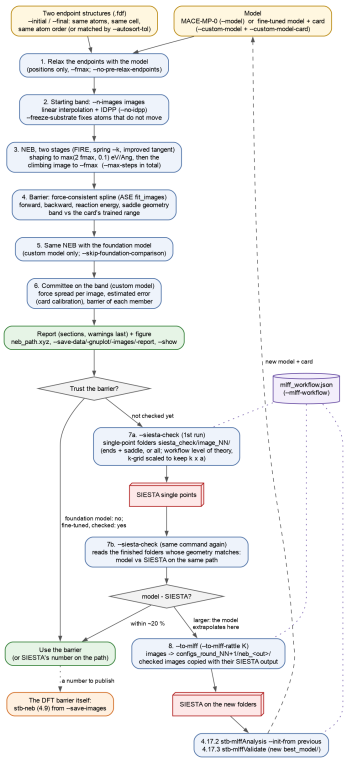
1.11 A calculation you can check by hand.

- **Hop length.** `d = √3/4 × 5.48024 = 2.3730 Å`. Report section `STRUCTURES`: one atom moves 2.3730 Å.
- **Rate error.** `k_B T` at 300 K is 0.025852 eV. Round 2's barrier error, 0.7193 − 0.2512 = 0.468 eV, gives `exp(0.468/0.025852) = 7.3 × 10^7` (Fig. 4b).
- **Vacancy diffusion (Eq. 3)** with SIESTA's 0.2512 eV and $\nu* = 10^{13}$ Hz: `D_V = (2/3)(2.373 × 10^-8 cm)^2 × 10^13 × exp(−0.2512/0.025852) = 2.3 × 10^-7 cm^2/s` at 300 K. With round 3's 0.3076 eV it is `2.6 × 10^-8`, with round 2's 0.7193 eV `3.1 × 10^-15` — eight orders of magnitude below SIESTA.
- **The k-grid of the check folders.** The training cell (`a = 5.48` Å) used 6×6×6, `k × a = 32.9` Å; the 2×2×2 supercell (`10.96` Å) keeps it with 3×3×3: `3 × 10.96 = 32.9` Å. Each folder's `config_extra.fdf` says `kgrid.MonkhorstPack [3 3 3]`.
- **Committee calibration.** Round 2's largest per-image force spread is 57.5 meV/Å (image 2). Times the card's calibration of 1.3, that is an estimated error of 75 meV/Å; report section `COMMITTEE UNCERTAINTY`: 75.

1.12 What this tool simplifies — explicitly.

- **0 K energy barriers**, not free-energy barriers. There is no prefactor, no entropy and no temperature: Eq. 2 needs the frequencies at the minimum and at the saddle.
- **One path, the one the endpoints define.** NEB finds the MEP nearest to the starting band. A different mechanism (a ring exchange, a reorientation) needs its own endpoints. Use more images, or an intermediate image, for a path you suspect has two steps.
- **Fixed cell.** The band lives in the initial cell. A lattice mismatch between the endpoints only warns.
- **Neutral, non-magnetic, same electron count.** The model knows no charge state or spin. A charged vacancy has another barrier (section 1.9).
- **The SIESTA check is single points on the model's geometries**, not a SIESTA NEB. It measures the model's error on that path and approximates SIESTA's barrier to second order (Eq. 10). For SIESTA's own barrier use `stb-neb` (`5.9`), starting from `--save-images`.
- **The committee spread is a lower bound** of the error (section 1.10).

13.5.2.2 2. The loop at a glance



stb-mlneb and the loop it closes

`stb-mlneb` itself is steps 1–8. Steps 7 and 8 talk to the MLFF workflow through its state file, `mlff_workflow.json` (`--mlff-workflow`). From it they take the level of theory: the reference `calc.fdf`, the pseudopotentials, the dispersion setting and the k-grid, scaled to keep `k × a`. `--to-mlff` also writes back to it: the new configs, their round and kind, and a history entry. The SIESTA runs and the training are yours to start, as everywhere in the suite.

13.5.2.3 3. Libraries and external dependencies used

Library	Used for
ASE [9]	`Atoms`, the NEB (`ase.mep.NEB`: improved tangent, climbing image), IDPP, FIRE, `fit_images` (the spline), neighbour distances, extended XYZ
mace-torch [17, 18]	the MACE calculator, the foundation models (downloaded once, cached in `~/.cache/mace/`) and custom `.model` files
PyTorch	runs the model on `cpu` or `cuda` (`--device`)
pymatgen	structure conversion
NumPy	geometry, the committee statistics
matplotlib	`neb_profile.png` (always written), `--show`
gnuplot (optional)	renders the `.gplot` file written by `--save-gnuplot`
`stb.neb`	composition check, wrapping into the cell, atom matching — shared with the SIESTA NEB (`5.9`)
`stb.core`	`.fdf` reading and writing, the model card, the calculator and the relaxations (`mace_relax`), device checks, pseudopotential banks, the workflow file (`mlff_workflow`), SIESTA log parsing, library-noise capture, report sections, gnuplot

Install the `ml` extra: `pip install stb_suite[ml]`. SIESTA is never called by the tool: `--siesta-check` and `--to-mlff` write input folders and read finished ones.

13.5.2.4 4. What `stb-mlneb` does

4.1 Step by step.

1. **Read** `--initial` and `--final` (`.fdf`). Check that they hold the same atoms (else a clean error). Match the atom order by distance (`--autosort-tol`, 0.5 Å). List the atoms that move by more than 0.1 Å. A lattice mismatch warns, and the band takes the initial cell.
2. **Load the model**: a foundation model (`--model small|medium|large`) or a custom model with its card (`--custom-model` + `--custom-model-card`; the card must match the model's SHA-256). Print the card in full.
3. **Relax both endpoints** with the model, positions only, to `--fmax` (`--no-pre-relax-endpoints` skips it). Report the force before and after, the largest move and the energy gained.
4. **Build the starting band**: `--n-images` images, endpoints included (default 7, at least 3), linear interpolation then IDPP (`--idpp-fmax`, `--idpp-steps`; `--no-idpp` skips it). With `--freeze-substrate`, the atoms that move less than `--freeze-threshold` (0.3 Å) between the endpoints are fixed. Report each image's step from the previous one and its shortest distance, and flag jumps.
5. **NEB** in two stages (section 1.7): shaping, then climbing. FIRE, spring `--k`, improved tangent, `--max-steps` in total. The residual force is recorded at every step.
6. **Barrier** (section 1.8): the spline through energies and forces, forward and backward, the reaction energy and the position of the saddle. Then the saddle geometry: the nearest neighbours of the atom that moves most, at the initial state, the saddle and the final state.
7. **Trained range** (custom model): volume per atom, shortest distance, largest force and energy per atom of the band against the card.
8. With a custom model, **repeat 3–6 with the foundation model** (`--model`) unless `--skip-foundation-comparison`. Each model relaxes its own endpoints, in the same cell.
9. **Committee** (custom model, unless `--no-committee`). Look for the card's committee next to the card's run folder, then in `--mlff-workflow`'s run, or take `--committee-models PATH...`. Every member evaluates the relaxed band. Report per image the energy and force spread (the largest per-atom deviation from the committee mean, as in `stb-mlffValidate`), the estimated error (spread × the card's calibrated error/spread ratio), the spread relative to the test set's, and each member's barrier.
10. **SIESTA check** (`--siesta-check`; needs `--mlff-workflow` or `--calc`). For each image of `--siesta-check-images` (`ends-saddle`, default, or `all`), look at `<out>/siesta_check/image_NN/`, then compare the model's profile with SIESTA's, image by image, and the barrier on that path:
    - **missing**: write it — the structure file the template `%includes`, `calc.fdf` with `config_extra.fdf` on top (k-grid, `NumberOfAtoms`, D3 if the workflow has it, single point), the pseudopotentials, and the stamp `stb_mlneb_image.xyz`;
    - **finished**, with the same geometry (0.02 Å): read SIESTA's energy and largest force;
    - **written for another geometry** (`stale`): leave it untouched and warn;
    - **not finished** (`pending`): say so.
11. **Export** (`--to-mlff`, needs `--mlff-workflow`). Write every image, and `--to-mlff-rattle K` copies of each displaced by a Gaussian of `--to-mlff-stdev` (0.05 Å; seed `--to-mlff-seed`), as training configs of the workflow's next round: `configs_round_NN/neb_<out>/image_II[_rattle_K]/`, kind `neb` / `neb-rattle`, stage `neb`. An image already finished by the check is copied with its SIESTA output, so it is not computed twice. A config already in the workflow is left alone, so the export can be repeated safely. The JSON is backed up and the call recorded in its history.
12. **Report, figure, files** (section 4.3).

The order matters for the SIESTA check: the check folders take the band of *this* run. Run the second pass with the same options and output folder. If the band changed (another model, other settings), the stamps no longer match: the folders are `stale`, and the report says so instead of comparing two different paths.

4.2 The report, section by section. Sections are numbered in order; those that do not apply (no card, no comparison, no committee, no check, no export) are left out.

Section	Content
`RUN METADATA`	date, versions, device (GPU name), endpoints, band and NEB settings, model, comparison
`STRUCTURES`	composition check, cell and volume per atom, shortest distance of each endpoint, the atoms that move and by how much
`MODEL`	the **model card** in full (identity and SHA-256, level of theory, training data and trained range, errors against SIESTA, calibration, stability, notes), or the foundation model's identity; load time
`ENDPOINTS`	per model: `max|F|` before and after, steps, largest move, energy gained, absolute energy; `E(final) − E(initial)`
`STARTING PATH`	per model: step between images and shortest distance after IDPP, with a check of each
`NEB BAND`	per model: converged or not, shaping and climbing steps, residual force, time; then each image: `s`, `E − E0`, raw `max|F|`, shortest distance, shift from the initial image, role (initial, climbing image, final)
`BARRIER`	per model: forward, backward and reaction energy from the spline and from the highest image; position of the saddle
`SADDLE POINT`	the four nearest neighbours of the largest mover at the initial state, the saddle and the final state; the saddle image's force and shortest distance
`TRAINED RANGE`	the band against the card: volume, shortest distance, largest force, energy per atom, each inside or outside
`FINE-TUNED vs FOUNDATION`	barrier (spline and image), backward barrier, reaction energy, saddle position, path length, shortest distance, NEB steps and time of both models
`COMMITTEE UNCERTAINTY`	where the members were found; per image `E − E0`, energy and force spread, worst atom, estimated error, spread vs the test set; each member's barrier; the calibration used
`SIESTA CHECK`	the template and k-grid; per image the model and SIESTA energies, their difference, SIESTA's `max|F|`, status; the barrier on the path, model vs SIESTA
`EXPORT TO THE MLFF`	workflow, round, folder, how many configs were written, copied or skipped;
`WORKFLOW`	the k-grid; the next commands
`SUMMARY & FILES`	status, headline numbers of every model, committee and check; the files; the outputs not asked for; wall time
`WARNINGS`	every warning, numbered; the count of library messages captured

The warnings include:

- an endpoint relaxation that hit `--max-steps`, or moved atoms far (the input was not a minimum of this model); `--no-pre-relax-endpoints` (the barrier is then not measured from the model's minima);
- an atom that moves very far between the endpoints (a wrong atom order), no atom that moves at all, consecutive images nearly identical, or one step of the starting path more than 3× the mean (atom correspondence);
- an NEB stopped at `--max-steps`;
- an intermediate minimum along the band (a two-step transition), or a band with no barrier;
- a spline barrier far from the highest image (too few images);
- the band outside the card's trained range: atoms closer than the shortest trained distance, energies below the lowest trained one, and the card's own domain checks;
- a committee spread far above the test set's, or members that disagree on the barrier;

- the SIESTA check: `stale` or `pending` folders, and a model–SIESTA difference that is "not reliable" (above `max(0.05 eV, 50 %)` of SIESTA's barrier) or "right in kind, not yet in value" (above `max(0.05 eV, 20 %)`);
- export folders that still need SIESTA; `--show` without a display.

4.3 Files.

Written	When
`neb_profile.png` (energy profile of every model with the spline; NEB convergence; with a check, SIESTA's points)	always; `--show` opens it
`neb_path.xyz` (the relaxed band, extended XYZ; `neb_path_foundation.xyz` for the comparison)	always
`neb_band.dat` (`neb_band_foundation.dat`): image, `s`, `E − E0`, `max|F|`, shortest distance, shift	`--save-data`
`plot/band_K.dat`, `plot/spline_K.dat`, `plot/convergence_K.dat`, `plot/neb_profile.gplot`, `plot/neb_profile.pdf`	`--save-gnuplot` (the PDF when gnuplot is installed)
`image_NN.fdf` (bare structures, e.g. for `stb-neb`)	`--save-images`
`stb_mlneb_report.txt`	`--save-report`
`siesta_check/image_NN/` (SIESTA inputs + stamp)	`--siesta-check`
`<workflow run>/configs_round_NN/neb_<out>/image_II[_rattle_K]/`, and the updated `mlff_workflow.json` (+ `.bak`)	`--to-mlff`

Everything else goes into `-o`/`--output-dir` (default `mlneb_out`).

4.4 Guard rails.

Situation	Behaviour
`--custom-model` without `--custom-model-card`, an unreadable card, a SHA-256 mismatch	clean error
`--n-images` below 3; `--to-mlff` without `--mlff-workflow`; a workflow file that does not exist	clean error
a missing endpoint or model file; endpoints with a different composition	clean error
`--siesta-check`/`--to-mlff` without `--calc` or `--mlff-workflow`; a `calc.fdf` that does not `%include` a structure file; no pseudopotential for an element	clean error, before the NEB
a lattice mismatch between the endpoints	warning; the band uses the initial cell
identical endpoints	warnings: no atom moves, images nearly identical, no barrier
an endpoint relaxation or the NEB hits its step limit	warning
a band outside the card's trained range; an element outside the model	warning; error
check folders written for another band, or not finished	`stale` / `pending`, left untouched, warning
`--device cuda` with no usable GPU	clean error
`--show` without a display; gnuplot not installed	warning; files written anyway

13.5.2.5 5. Running it both ways

Command line:

```
# a first look with the foundation model
stb-mlneb --initial vac_initial.fdf --final vac_final.fdf --model medium --device cuda --save-report

# a fine-tuned model, its committee and a SIESTA check of every image (first pass: writes the folders)
stb-mlneb --initial vac_initial.fdf --final vac_final.fdf \
```

```
          --custom-model best_model/mlff_model_compiled.model --custom-model-card
          best_model/mlff_model_card.json \
          --mlff-workflow mlff_runs/mlff_workflow.json --siesta-check --siesta-check-images all \
          --device cuda -o neb_r2 --save-data --save-gnuplot --save-report
for d in neb_r2/siesta_check/image_*/; do (cd $d && mpirun -np 6 siesta < calc.fdf > siesta.out); done

# second pass: the same command reads them; --to-mlff sends the band to the next training round
stb-mlneb ... the same options ... --to-mlff --to-mlff-rattle 1

# without a workflow: any calc.fdf that %includes the structure, a pseudopotential folder or bank, a
k-grid
stb-mlneb --initial i.fdf --final f.fdf --custom-model ... --custom-model-card ... \
          --siesta-check --calc calc.fdf --pseudo-dir dojo --kgrid 3 3 3
```

Interactive menu: run `stb-suite` and type `7.5`. It asks, in this order:

1. the initial and the final structure;
2. a custom model path, or Enter for a foundation model. With a custom model it asks for the card, offering `mlff_model_card.json` next to the model as the default;
3. the MACE-MP-0 size (with a custom model: the one of the comparison);
4. the device (`cpu`/`cuda`; it checks the GPU) and the number of images;
5. whether to freeze the substrate; with a custom model, whether to compare with the foundation model;
6. with a custom model, the `mlff_workflow.json` of its run (Enter to skip). With one: whether to write the SIESTA check (and of every image), whether to export to training (and how many rattled copies);
7. the output folder, then one question per optional output (show, gnuplot, data, images, report).

The menu builds the same command line and calls `stb-mlneb`; the guided script (Case 10) drives it with piped answers. The options without a menu question (`--k`, `--fmax`, `--max-steps`, `--calc`, `--kgrid`, `--committee-models`, ...) are for the command line.

13.5.2.6 6. Worked example: a vacancy hop in silicon, and one round to fix it

6.1 Set-up.

- **The cell.** `structure.fdf` is the 8-atom diamond cell at SIESTA's own lattice constant at the training settings, `a = 5.48024 Å` (experiment: 5.431 Å; PBE is 0.9 % too large). `make_endpoints.py` builds a 2×2×2 supercell (64 Si, `10.96 Å`), removes the atom at the origin (`vac_initial.fdf`, 63 atoms), and moves its neighbour at `(1/8, 1/8, 1/8)` into the hole (`vac_final.fdf`). One atom moves, by `d = 2.373 Å`; both files list the atoms in the same order.
- **The model of round 2** (`../7.2-mlmd/model/`, the model of `7.2`–`7.4`): the best model of the `7.1` silicon run, fine-tuned from MACE-MP-0 medium on 57 SIESTA single points of the **perfect crystal** — 39 rattled cells, 6 strained cells, 12 frames of an 800 K MD in a 64-atom cell. Test-set force error 28.7 meV/Å, phonons 1.10 THz RMS from SIESTA's. No vacancy, no transition state.
- **The level of theory**: `calc.fdf`, the training template (PBE, DZP, `PAO.EnergyShift 0.01 Ry`, `Mesh.Cutoff 300 Ry`, 300 K smearing, non-spin-polarized, PseudoDojo `Si.psml`). The check folders use it with `kgrid 3×3×3`, the training `k × a`.
- **Hardware**: every NEB on one RTX 3060 (`--device cuda`, float64), 45–60 s per band of the fine-tuned model, about 2 min with the foundation model. On a CPU (12 threads) the same band takes 9 min and comes out identical. Each SIESTA single point: about 3 min on 6 MPI processes (SIESTA 5.4.2).

The loop below was run once, by hand, in a copy of the `7.1` run folder (2 GB). `reference_results/loop_round2/` and `loop_round3/` keep its `stb-mlneb` reports, bands and the SIESTA single points (inputs, `siesta.out`, `siesta.FA`, stamp). `reference_results/mlff_round3/` keeps the training and validation reports of round 3. The guided script replays it without the 2 GB (section 11).

6.2 Round 2: the band.

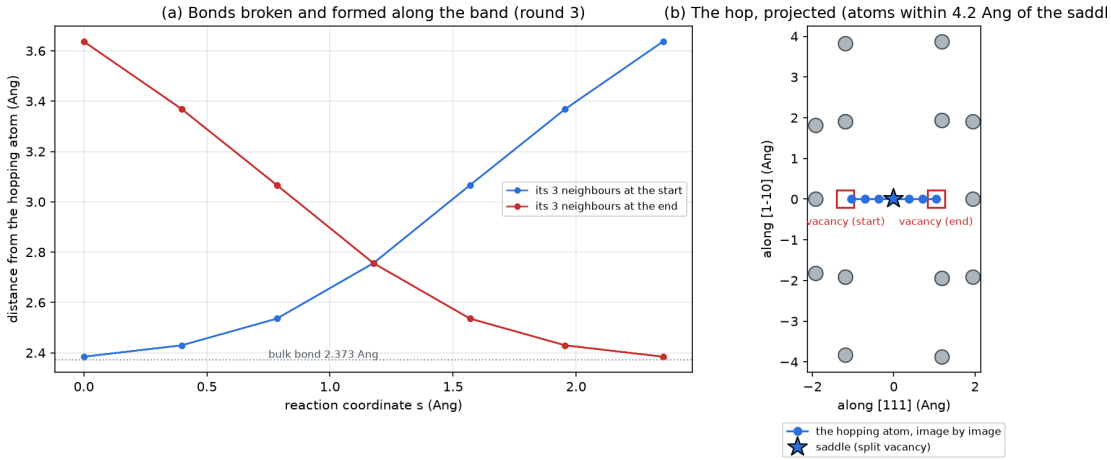
```
stb-mlneb --initial vac_initial.fdf --final vac_final.fdf \
          --custom-model best_model/mlff_model_compiled.model --custom-model-card
          best_model/mlff_model_card.json \
          --mlff-workflow mlff_runs/mlff_workflow.json --siesta-check --siesta-check-images all \
          --skip-foundation-comparison --device cuda -o neb_r2 --save-data --save-report
```

Endpoints. The hand-built vacancy is not a minimum: the model relaxes the three atoms around it by 0.13 Å (`max|F|` 0.31 → 0.023 eV/Å in 22 steps) and gains 0.110 eV. Both endpoints relax identically: they are the same defect, seen from two sites.

The band. IDPP gives seven images 0.355 Å apart, with nothing closer than 2.37 Å. The NEB converges in 16 shaping and 4 climbing steps (44 s):

image	s (Å)	`E − E0` (eV)	raw `max|F|` (eV/Å)	role
0	0.000	0.000	0.023	initial
1	0.411	+0.088	0.472	
2	0.789	+0.379	1.063	
3	1.175	**+0.719**	0.048	climbing image (saddle)
4–6		mirror of 2–0		final

- The barrier is **0.719 eV**, forward and backward (`ΔE = 0`). The spline and the highest image agree, since the climbing image is converged.
- The raw force on images 1, 2, 4, 5 is large (up to 1.06 eV/Å) — the component along the path, which the NEB never relaxes (section 1.7).
- **Saddle**: the hopping atom has its three starting bonds stretched from 2.372 to 2.776 Å, and three new ones at the same 2.776 Å: the split vacancy (Fig. 5). The path is symmetric, the saddle at 50 %.
- **Trained range**: volume, shortest distance (2.364 Å against the trained 1.852), forces (0.31 against 14.7 eV/Å) and energy per atom are all **inside** the card's range. Nothing in the report says the model is extrapolating.



The hop

Fig. 5 — (a) Distances from the hopping atom along the round-3 band: three bonds break while three form, and all six are 2.756 Å at the saddle. (b) The hopping atom image by image, projected on the plane of [111] (the hop) and [1-10], with the atoms within 4.2 Å of the saddle (`results_figures.py`*).*

6.3 The committee. The card records a committee of three (seeds 123–125, round 2). `stb-mlneb` found their models in the workflow's run folder and evaluated each on the band:

image	E spread (eV)	force spread (meV/Å)	estimated force error	vs the test set's spread
0	0.000	24.4	32	0.9×
1	0.010	44.6	58	1.6×
2	0.025	57.5	75	2.0×
3	0.040	17.2	22	0.6×

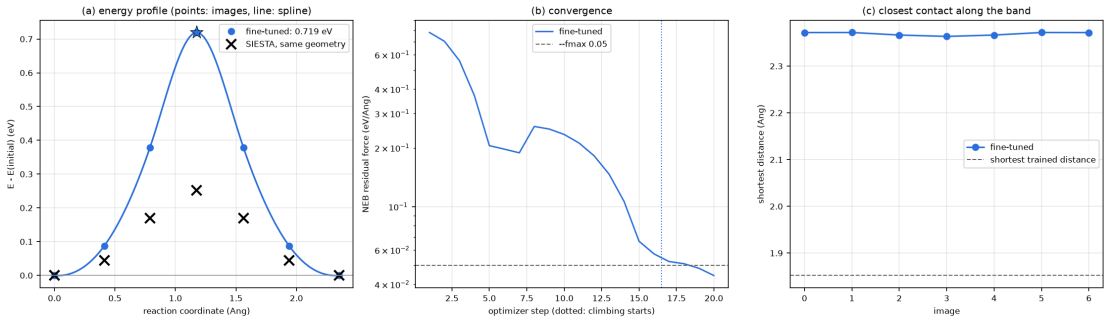
- **Member barriers: 0.634, 0.636, 0.719 eV.** Their spread, 0.04 eV, is a tenth of the error the SIESTA check is about to show.
- The force spread doubles on the flanks of the barrier, but only to 2× the test set's — no warning (the threshold is 3×).
- At the saddle itself the members agree *better* than at the endpoints (0.6×). The saddle is a high-symmetry point, where all three are wrong in the same way.

The committee says "a bit less certain on the flanks". It does not say "wrong by 0.47 eV" (section 1.10).

6.4 The SIESTA check. The first run wrote seven folders, `neb_r2/siesta_check/image_00` … `image_06`: the workflow's `calc.fdf` with a `config_extra.fdf` on top (`kgrid 3×3×3`, `NumberOfAtoms 63`, single point), `Si.psml`, the structure, and the stamp. SIESTA ran in each (7 × 2.8 min on 6 cores), and the same command, run again, read them:

image	model `E − E0` (eV)	SIESTA `E − E0` (eV)	model − SIESTA	SIESTA `max|F|` (eV/Å)
0	0.000	0.000	0.000	0.231
1	+0.088	+0.044	+0.044	0.236
2	+0.379	+0.170	+0.209	0.439
3	+0.719	**+0.251**	**+0.468**	0.167

- **On its own path, SIESTA's barrier is 0.251 eV. The model is +186 % off.** The error grows from the endpoints to the saddle: the farther from the crystal, the worse.
- **SIESTA's forces on the model's geometries** (0.17–0.44 eV/Å) show that even the model's relaxed vacancy is not SIESTA's minimum. Its endpoints sit 0.23 eV/Å away from equilibrium in SIESTA's eyes. By Eq. 10 they are a little too high in SIESTA's energy, so 0.251 eV, if anything, slightly underestimates SIESTA's barrier.
- The report's verdict, in `WARNINGS`: "the model is not reliable for this transition. Add the images to the training set (--to-mlff), retrain (stb-mlffAnalysis) and run the NEB again."



The tool's own figure of the round-2 run

Fig. 6 — `neb_profile.png` *of the round-2 run with the check read back (*`reference_results/round2/`*). (a) The band, the spline, the climbing image (star) and SIESTA's energies on the same geometries (crosses). (b) The NEB residual force: shaping, then climbing from the dotted line, until* `--fmax`*. (c) The shortest distance along the band against the card's shortest trained distance: far inside it.*

6.5 Retraining with the band.

```
stb-mlneb ... the same options ... --to-mlff --to-mlff-rattle 1
for d in mlff_runs/configs_round_03/neb_r2/*_rattle_1/; do (cd $d && mpirun -np 6 siesta < calc.fdf >
siesta.out); done
stb-mlffAnalysis --workflow mlff_runs/mlff_workflow.json --init-from previous --device cuda
stb-mlffValidate --workflow mlff_runs/mlff_workflow.json
```

- **The export**: 14 configs in `configs_round_03/neb_r2/` — the 7 checked images, copied with their SIESTA output, and one copy of each displaced by 0.05 Å (Gaussian, per Cartesian component). Only the 7 copies needed SIESTA: 21 min. In the workflow they are training configs of round 3, kinds `neb` and `neb-rattle`. (The real run's folder is called `neb_neb_r2`: that run predates a fix that keeps an `-o` name already starting with `neb`.)
- **Round 3**: `stb-mlffAnalysis` trained three members from round 2's model on 71 configs (64 train / 7 valid), about 10 min each on the GPU. `stb-mlffValidate` scored all three rounds on the same 16 held-out configs of the perfect crystal:

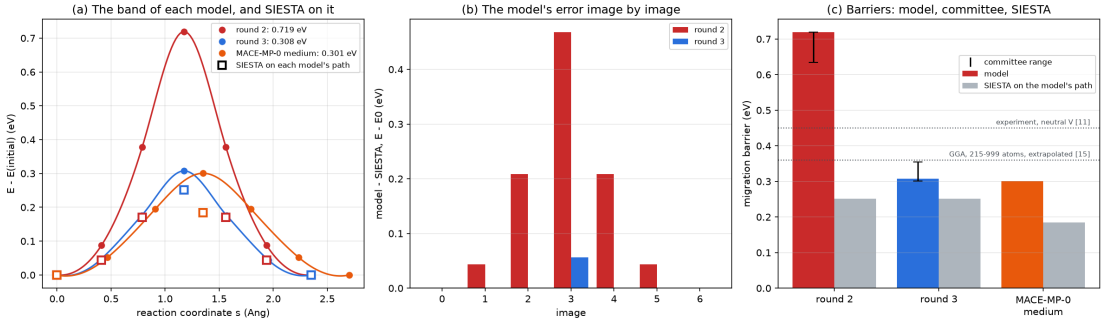
round	training configs	test F RMSE (meV/Å)	±	phonons vs SIESTA (THz)	verdict
1	45	23.2	4.3	1.14	
2	57	28.7	6.8	1.10	
3	71 (+14 NEB)	**22.7**	2.9	**1.09**	STOP, new best model

The band made the model better on the crystal too. Round 3 is the new `best_model/` (selected on the test set pooled with the 800 K MD frames), and its card lists `neb 7, neb-rattle 7` among the training kinds. It is shipped here as `model_round3/`.

6.6 Round 3, and the foundation model. The same NEB with round 3, now with the foundation-model comparison and the committee, and a SIESTA check of the ends and the saddle (3 single points):

	model barrier	committee barriers	SIESTA on the same path	SIESTA `max|F|`, endpoint / saddle
round 2	0.719 eV	0.634–0.719 eV	0.251 eV (+186 %)	0.23 / 0.17 eV/Å

	model barrier	committee barriers	SIESTA on the same path	SIESTA `max|F|`, endpoint / saddle
round 3 (+14 NEB configs)	**0.308 eV**	0.301–0.354 eV	0.251 eV (**+23 %**)	**0.04** / 0.09 eV/Å
MACE-MP-0 medium	0.301 eV	—	0.185 eV (+63 %)	0.16 / 0.11 eV/Å



The loop in numbers

Fig. 7 — (a) The relaxed band of each model (points and spline) and SIESTA's energies on it (open squares). (b) model – SIESTA image by image. (c) The barriers: each model, its committee's range, SIESTA on its path, and two literature values for a converged cell (section 6.8). `results_figures.py`.

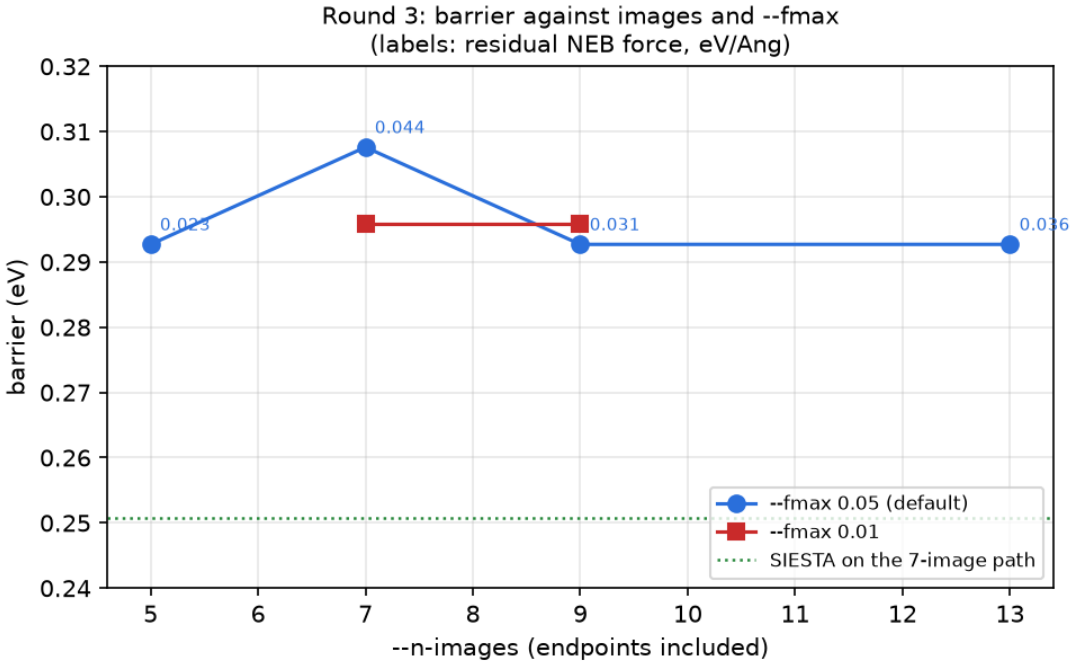
- **One round with 14 configs took the error from 0.47 eV to 0.06 eV.** The model's relaxed vacancy is now close to SIESTA's: the SIESTA force on its endpoints fell from 0.23 to 0.04 eV/Å.
- The verdict is now "right in kind, not yet in value": another round with the round-3 images would close the rest, or use SIESTA's 0.25 eV on this path.
- **The committee still does not bound the error.** Its members now span 0.30–0.35 eV and bracket the model, but SIESTA's 0.25 lies outside the range.
- **SIESTA gives 0.25 eV on both fine-tuned paths.** It does so although the two paths differ: their saddles are 2.776 and 2.756 Å from the six neighbours. This is Eq. 10 at work: near a stationary point the energy is insensitive to small geometry errors.
- **The foundation model's 0.30 eV is close by chance.** On its own path SIESTA gives 0.185 eV, because its endpoints are 0.047 eV *higher* in SIESTA's energy than round 2's (SIESTA force 0.16 eV/Å). A too-high minimum lowers the barrier measured from it (section 1.10). Its saddle is in fact 0.020 eV lower in SIESTA's energy than round 2's. The lowest SIESTA saddle minus the lowest SIESTA endpoint of all the checked geometries is 0.232 eV.

From all the single points, SIESTA's barrier in this cell at these settings is **0.23–0.25 eV**. Only a SIESTA NEB (`stb-neb`, `5.9`) would pin it down further.

6.7 Settings: images, IDPP, endpoints, foundation models. `make_reference.sh` repeats the NEB with other settings (all on the GPU, `reference_results/<case>/`):

case	barrier, spline = highest image (eV)	NEB steps	NEB time (s)
round 3, 7 images (the default)	0.3076	21	49
round 3, 5 images	0.2927	25	41
round 3, 9 images	0.2927	25	67
round 3, 13 images	0.2927	25	94
round 3, 7 images, `--no-idpp`	0.3076	21	48

case	barrier, spline = highest image (eV)	NEB steps	NEB time (s)
round 3, 7 images, `--fmax 0.01`	**0.2958**	34	73
round 3, 9 images, `--fmax 0.01`	**0.2958**	34	92
round 3, `--no-pre-relax-endpoints`	0.1792 (warnings)	29	58
MACE-MP-0 small	0.1505	28	44
MACE-MP-0 medium	0.3006	25	130
MACE-MP-0 large	0.0850 (warnings)	15	151



Barrier against the number of images and the force threshold

Fig. 8 — Round 3's barrier against `--n-images` *at the default* `--fmax 0.05` *(labels: the residual force where the NEB stopped) and at* `--fmax 0.01`*; SIESTA's barrier on the 7-image path for scale.*

- **The number of images does not matter here.** The path is a single symmetric hop with one saddle. The climbing image finds the saddle with 5 images as with 13, and the spline and the highest image coincide in every run. More images cost time (linearly) and give a smoother profile, not a different barrier. They matter for long or curved paths, where too few images cut corners or miss an intermediate minimum (Fig. 1).
- **`--fmax` does matter, at the 0.01 eV level.** At the default 0.05 eV/Å the barrier scatters between 0.293 and 0.308 eV with the image count. The difference is not the images but where FIRE happened to stop: the 7-image band crossed the threshold after 5 climbing steps (residual 0.044), the others after 10 (0.023–0.036). In a 63-atom cell a residual force of a few hundredths of an eV/Å on many atoms still holds ~0.01 eV of relaxation energy. At `--fmax 0.01` the endpoints relax 7 meV further, and 7 and 9 images agree: **0.2958 eV**. Use 0.01 when the second decimal matters. For the comparison with SIESTA (+23 %) the difference is immaterial.
- **IDPP changes nothing for a one-atom hop**: only one atom moves, and the linear path keeps every distance above 2.37 Å. The numbers are identical to the last digit. IDPP is for concerted moves, where a linear path pushes atoms through each other.
- **Unrelaxed endpoints give a wrong barrier.** With `--no-pre-relax-endpoints` the band starts from the hand-built vacancy, 0.11 eV above the model's minimum. Images 1 and 5 relax 0.096

eV *below* the endpoints, and the report warns of an intermediate minimum there. The reported "barrier" (0.18 eV) is measured from a point that is not a minimum; from images 1 and 5 it would be 0.275 eV, close to the right one.
- **The foundation models disagree by a factor of 3.5**: small 0.15, medium 0.30, large 0.085 eV. The large model even rebuilds the vacancy: its endpoints move by 0.40 Å during the relaxation (two warnings), and two of the dangling bonds pair up (the fourth neighbour at 3.23 Å instead of 3.66–3.78 Å). The saddle, the path (1.9 Å instead of 2.4–2.9 Å) and the barrier change with it. None of the three is SIESTA. As in `7.3` and `7.4`, a larger foundation model is not a better one for a given level of theory.

6.8 Is 0.25 eV silicon's vacancy migration barrier?. No — and the example does not claim it. It is the barrier of a neutral, non-spin-polarized vacancy in a 63-atom cell with a 3×3×3 k-grid. That is the level of theory the model was trained on, so it is the right reference for the model, and the wrong one for silicon:

- **Cell size.** The vacancy's relaxation is long ranged and its gap levels interact with their periodic images. Formation energies and barriers move by tenths of an eV between 64 and 216 atoms [13, 14, 19]. Converged GGA calculations, extrapolated from 215–999 atoms, give 0.36 eV for the neutral vacancy [15]; SIESTA with 216 atoms gave 0.40 eV [16].
- **Charge state.** Experiment separates the charge states: 0.45 eV for V^0, 0.32 for V^{2+}, 0.18 for V^{2-} [11, 12]. The model has no notion of charge.
- **Spin and symmetry.** The neutral vacancy's Jahn–Teller distortion (`D2d`) is subtle and cell-dependent; the non-spin-polarized calculation here keeps the high-symmetry relaxation.

For a converged barrier: train with vacancy configurations in a larger cell (Stage 4 of `7.1` with `--supercell 3 3 3` on a vacancy structure, plus `--to-mlff` from a 215-atom NEB), check with `--siesta-check` there, and confirm with a SIESTA NEB. The model makes the large cell affordable: a 215-atom band costs minutes on a GPU, while each SIESTA image costs hours.

6.9 What to take from the example.

- **A model is validated for the structures it was tested on.** A force error of 28.7 meV/Å on the perfect crystal said nothing about a vacancy at the saddle: the barrier was 3× too high.
- **The trained-range check and the committee are necessary, not sufficient.** The band was inside every range. The committee's spread (0.04 eV in the barrier) was a tenth of the error.
- **SIESTA single points on the model's own band are cheap and decisive**: 7 single points (20 min on 6 cores), against hundreds for a SIESTA NEB. Check at least the ends and the saddle; with `all`, you also see where the error grows.
- **The checked images are the best training data there is** for this property: 14 configs, one round, the error from 0.47 to 0.06 eV — and a model that also got better on the crystal.
- **Read SIESTA's forces in the check.** A model endpoint that is not SIESTA's minimum biases the barrier measured on its path (Eq. 10). The foundation model's "0.30 eV" was close by chance.
- **A barrier at the training settings is not the physical barrier.** Converge the cell, the charge and the spin as in any DFT defect calculation.

13.5.2.7 7. Reproducing the reference data

`reference_results/` holds:

- `loop_round2/`, `loop_round3/`: the real loop of sections 6.2–6.6 — reports (`loop_round2/stb_mlneb_report_pass1.txt` is the first pass, which wrote the folders), bands, figures, and `siesta_check/`

image_NN/ with SIESTA's inputs, siesta.out, siesta.FA and the stamp (the large SIESTA files are left out);
- foundation_siesta/: the single points of section 6.6 on the foundation model's endpoint and saddle (fd_*) and on round 2's (ft_*), with compare.txt;
- mlff_round3/: the stb-mlffAnalysis and stb-mlffValidate reports of round 3;
- one folder per case of make_reference.sh (section 6.7 and the round-2/round-3 bands): report, band data, figure, relaxed band, and the gnuplot files of round2 and round3.

```
source ~/.anaconda3/etc/profile.d/conda.sh && conda activate stb   # or any environment with
stb_suite[ml]
bash make_reference.sh                  # every case into ./reference_run/ (GPU; DEVICE=cpu works,
~20 min per band)
python3 results_figures.py              # the tables of section 6 and figures/loop.png,
vacancy_hop.png, images.png
python3 theory_figures.py               # the figures of section 1 (numpy + matplotlib, seconds)
dot -Tpng -Gdpi=150 figures/mlneb_flowchart.dot -o figures/mlneb_flowchart.png    # the flowchart
(graphviz)
```

make_reference.sh needs no SIESTA. For round2 and round3 it writes the check folders, copies SIESTA's output from loop_round*/siesta_check/ into those whose stamp matches its own band (within 0.02 Å; in practice identical), and runs the same command again to read them. All the cases take about 20 minutes on one RTX 3060; each is skipped when its report exists (FORCE=1 reruns). The foundation models are downloaded on first use. A rerun gives the same barriers to the printed digits.

The loop itself (sections 6.2–6.5) needs the 7.1 silicon run and SIESTA (5.4.2 here, from conda-forge). It is documented by its reports, not scripted: it is the commands of sections 6.2 and 6.5, plus SIESTA in each new folder.

13.5.2.8 8. Known, deliberate limitations

- **0 K barriers**, no prefactor: the rate needs the frequencies at the minimum and the saddle (section 1.2).
- **One path per run**, the one nearest to the starting band. Other mechanisms need other endpoints.
- **Fixed cell**, taken from --initial.
- **The SIESTA check is single points on the model's band.** It measures the model's error there and approximates SIESTA's barrier to second order (Eq. 10); a SIESTA NEB (5.9) gives SIESTA's own path.
- **The committee spread is a lower bound** of the error, calibrated on the test set, not on transition states.
- **No charge, no spin**: the model reproduces whatever its labels were.

13.5.2.9 9. Step-by-step: a barrier of your own

1. **Build the endpoints** with the same atoms in the same order (a script like make_endpoints.py, or stb-defect and an edit of one coordinate). Relax them first if you can; stb-mlneb relaxes them with the model anyway.
2. **A first look with the foundation model** (--model medium): does the band converge, is the path the one you expect, is there an intermediate minimum? The number itself is not trustworthy (section 6.6).
3. **With a fine-tuned model** (7.1), add --mlff-workflow of its run: the committee, the level of theory for the check. Read TRAINED RANGE and COMMITTEE UNCERTAINTY, but do not stop there.

4. **Check with SIESTA**: `--siesta-check` (ends and saddle; `all` to see where the error grows). Run SIESTA, run the same command again.
5. **If the model is off** by more than ~20 %: `--to-mlff --to-mlff-rattle 1`, SIESTA in the new folders, `stb-mlffAnalysis --init-from previous`, `stb-mlffValidate`, and the NEB again with the new `best_model/`. Repeat until the check agrees.
6. **Converge the physics**: the number of images (spline ≈ highest image), the cell size (the defect's images), and, for a number to publish, a SIESTA NEB (`stb-neb`, `5.9`) from `--save-images`.

13.5.2.10 10. Files in this folder

File	What it is
`README.md`	this tutorial
`example_7.5.sh`	the guided walk-through (about 30 min on a CPU, 5 min with `DEVICE=cuda`)
`structure.fdf`, `calc.fdf`	the 8-atom silicon cell at SIESTA's lattice constant, and the training `calc.fdf` of the `7.1` run (`%include structure.fdf`)
`make_endpoints.py`	builds `vac_initial.fdf` / `vac_final.fdf` from `structure.fdf`
`vac_initial.fdf`, `vac_final.fdf`	the two endpoints of the hop (63 Si)
`model_round3/`	the round-3 model (compiled, 11 MB) and its card; round 2 is `../7.2-mlmd/model/`
`make_reference.sh`	regenerates the cases of `reference_results/` (GPU, no SIESTA)
`results_figures.py`	reads `reference_results/`, prints the tables of section 6, draws `loop.png`, `vacancy_hop.png`, `images.png`
`theory_figures.py`	the figures of section 1: a NEB on the Müller–Brown surface, the force projections, the spline, the rate
`figures/`	every figure of this README; `mlneb_flowchart.dot` is the flowchart's source
`reference_results/`	the real loop, the SIESTA single points, the round-3 reports and every `make_reference.sh` case (section 7)

13.5.2.11 11. Running the script

```
cd examples/7.5-mlneb
bash example_7.5.sh              # CPU, ~30 min (three NEBs of 9 min); press Enter between cases
DEVICE=cuda bash example_7.5.sh  # on a GPU, ~5 min
```

Each case writes into `output/<case>/`, which is wiped at every start. The script never runs SIESTA. Case 4 copies the finished single points of the real loop into the folders Case 3 wrote, after comparing the stamps, and Case 7 puts those of round 3 in place before its run. `stb-mlneb` reads them as it would read your own. Case 2 builds a small workflow with `stb-mlff` (seconds), so that the export of Case 5 has somewhere to go. Cases 6 and 9 only read `reference_results/`. Case 10 (the menu) runs only with `DEVICE=cuda`.

13.5.2.12 12. Flag reference

`stb-mlneb` — `--initial FILE` and `--final FILE` (`.fdf`, required) · `--model {small,medium,large}` (default medium) · `--custom-model PATH` + `--custom-model-card JSON` (always both) · `--skip-foundation-comparison` · `--mlff-workflow JSON` · `--no-committee` · `--committee-models PATH...` · `--siesta-check` · `--siesta-check-images {ends-saddle,all}` · `--calc FILE` · `--pseudo-dir DIR|BANK` · `--kgrid KA KB KC` · `--to-mlff` · `--to-mlff-rattle N` (default 0) · `--to-mlff-stdev Å` (0.05) · `--to-mlff-seed N` (7) · `--n-images N` (default 7, ≥ 3) · `--autosort-tol Å` (0.5) · `--no-idpp` · `--idpp-fmax eV/Å` (0.1) · `--idpp-steps N` (100) · `--no-pre-relax-endpoints` · `--freeze-substrate` · `--freeze-threshold Å` (0.3) · `--k eV/Å²` (0.1) · `--fmax eV/`

Å (0.05) · `--max-steps N` (200, both stages) · `--device {cpu,cuda}` · `-o/--output-dir DIR` (default `mlneb_out`) · `--show` · `--save-gnuplot` · `--save-data` · `--save-images` · `--save-report` · `--no-intro` · `-v`.

13.5.2.13 13. What's next

- **`5.9` (NEB / reaction path)** runs the NEB with SIESTA, starting from this band (`--save-images`): SIESTA's own path and barrier.
- **`7.1` (MLFF workflow)** trains the model; `--to-mlff` feeds it the band. Stage 4 (`--supercell`) samples larger cells around a defect.
- **`7.3` (`stb-mlphonons`)** gives the frequencies the prefactor of Eq. 2 needs, at the minimum (and, on an image saved with `--save-images`, at the saddle).
- **`7.2` (`stb-mlmd`)** shows the hop happening at high temperature — a check of the barrier from the jump rate.

13.5.2.14 14. References

1. H. Jónsson, G. Mills, K. W. Jacobsen, "Nudged elastic band method for finding minimum energy paths of transitions", in *Classical and Quantum Dynamics in Condensed Phase Simulations*, ed. B. J. Berne, G. Ciccotti, D. F. Coker (World Scientific, 1998), p. 385.
2. G. H. Vineyard, "Frequency factors and isotope effects in solid state rate processes", *J. Phys. Chem. Solids* **3**, 121 (1957).
3. H. Mehrer, *Diffusion in Solids* (Springer, 2007).
4. G. Mills, H. Jónsson, "Quantum and thermal effects in H_2 dissociative adsorption: evaluation of free energy barriers in multidimensional quantum systems", *Phys. Rev. Lett.* **72**, 1124 (1994).
5. G. Henkelman, H. Jónsson, "Improved tangent estimate in the nudged elastic band method for finding minimum energy paths and saddle points", *J. Chem. Phys.* **113**, 9978 (2000).
6. G. Henkelman, B. P. Uberuaga, H. Jónsson, "A climbing image nudged elastic band method for finding saddle points and minimum energy paths", *J. Chem. Phys.* **113**, 9901 (2000).
7. S. Smidstrup, A. Pedersen, K. Stokbro, H. Jónsson, "Improved initial guess for minimum energy path calculations", *J. Chem. Phys.* **140**, 214106 (2014).
8. E. Bitzek, P. Koskinen, F. Gähler, M. Moseler, P. Gumbsch, "Structural relaxation made simple", *Phys. Rev. Lett.* **97**, 170201 (2006).
9. A. H. Larsen et al., "The atomic simulation environment — a Python library for working with atoms", *J. Phys.: Condens. Matter* **29**, 273002 (2017).
10. K. Müller, L. D. Brown, "Location of saddle points and minimum energy paths by a constrained simplex optimization procedure", *Theor. Chim. Acta* **53**, 75 (1979).
11. G. D. Watkins, "EPR studies of the lattice vacancy and low-temperature damage processes in silicon", in *Lattice Defects in Semiconductors 1974*, Inst. Phys. Conf. Ser. **23**, 1 (1975).
12. G. D. Watkins, "The vacancy in silicon: Identical diffusion properties at cryogenic and elevated temperatures", *J. Appl. Phys.* **103**, 106106 (2008), doi:10.1063/1.2937198.
13. M. J. Puska, S. Pöykkö, M. Pesola, R. M. Nieminen, "Convergence of supercell calculations for point defects in semiconductors: Vacancy in silicon", *Phys. Rev. B* **58**, 1318 (1998).
14. F. Corsetti, A. A. Mostofi, "System-size convergence of point defect properties: The case of the silicon vacancy", *Phys. Rev. B* **84**, 035209 (2011).
15. A. F. Wright, "Density-functional-theory calculations for silicon vacancy migration", *J. Appl. Phys.* **103** (2008), doi:10.1063/1.2906342.
16. F. El-Mellouhi, N. Mousseau, P. Ordejón, "Sampling the diffusion paths of a neutral vacancy in silicon with quantum mechanical calculations", *Phys. Rev. B* **70**, 205202 (2004).
17. I. Batatia, D. P. Kovács, G. N. C. Simm, C. Ortner, G. Csányi, "MACE: Higher order equivariant message passing neural networks for fast and accurate force fields", *Adv. Neural Inf. Process. Syst.*

35, 11423 (2022).
18. I. Batatia et al., "A foundation model for atomistic materials chemistry", arXiv:2401.00096 (2023).
19. M. I. J. Probert, M. C. Payne, "Improving the convergence of defect calculations in supercells: An ab initio study of the neutral silicon vacancy", *Phys. Rev. B* **67**, 075204 (2003).

13.5.3 Command-line reference

13.5.3.1 `stb-mlneb`

Climbing-image NEB (nudged elastic band) driven entirely by a MACE potential – the MACE-MP-0 foundation model, or a custom model fine-tuned on your own SIESTA data via `stb-mlffAnalysis` (`--custom-model`, always with its model card). Relaxes the two endpoint structures (`--initial`/`--final`, same composition and cell), interpolates the band (linear + IDPP), relaxes it with a two-stage climbing-image NEB and reports: the atoms that move, the endpoint relaxations, the starting path, the NEB convergence, the band image by image, the barrier forward and backward (spline and highest image), the saddle geometry, the band against the model card's trained range and, with a custom model, the same NEB with the foundation model and the uncertainty of its committee; every warning in one final section. Closing the loop with SIESTA: `--siesta-check` writes single-point folders for the endpoints and the saddle (then reads them on the next run and compares the model with SIESTA on the same geometries), and `--to-mlff` adds the images to the training set of the 7.1 workflow (`--mlff-workflow`) for the next round. For the full DFT barrier use `stb-neb` (5.9).

Synopsis.

```
usage: stb-mlneb [-h] --initial INITIAL --final FINAL [--model {small,medium,large}]
                 [--custom-model PATH] [--custom-model-card JSON]
                 [--skip-foundation-comparison] [--mlff-workflow JSON]
                 [--no-committee] [--committee-models PATH [PATH ...]]
                 [--siesta-check] [--siesta-check-images {ends-saddle,all}]
                 [--calc CALC] [--pseudo-dir PSEUDO_DIR] [--kgrid KA KB KC]
                 [--to-mlff] [--to-mlff-rattle N] [--to-mlff-stdev TO_MLFF_STDEV]
                 [--to-mlff-seed TO_MLFF_SEED] [--n-images N_IMAGES]
                 [--autosort-tol AUTOSORT_TOL] [--no-idpp] [--idpp-fmax IDPP_FMAX]
                 [--idpp-steps IDPP_STEPS] [--no-pre-relax-endpoints]
                 [--freeze-substrate] [--freeze-threshold FREEZE_THRESHOLD] [--k K]
                 [--fmax FMAX] [--max-steps MAX_STEPS] [--device {cpu,cuda}]
                 [-o OUTPUT_DIR] [--show] [--save-gnuplot] [--save-data]
                 [--save-images] [--save-report] [--no-intro] [-v]
```

Options

`--initial INITIAL`
: Initial (relaxed) endpoint structure (.fdf). required

`--final FINAL`
: Final (relaxed) endpoint structure (.fdf). required

`--model {small,medium,large}`
: MACE-MP-0 model size (default: medium). With `--custom-model`, the foundation model of the comparison.
choices: small, medium, large

`--custom-model PATH`
: Use a custom MACE model file instead of the MACE-MP-0 foundation potential – e.g. one fine-tuned via `stb-mlffAnalysis`. Overrides `--model`.

`--custom-model-card JSON`

Model card of the `--custom-model` file: the `mlff_model_card.json` that `stb-mlffAnalysis` wrote next to it (training data, level of theory, accuracy, energy scale). Required with `--custom-model` – the card is checked against the model file and used to warn when a structure is outside what the model was trained on.

`--skip-foundation-comparison`

With `--custom-model`, do not repeat the NEB with the foundation model (`--model`). The comparison is on by default: it shows what the fine-tuning changed in the barrier. switch, off by default

`--mlff-workflow JSON`

The `mlff_workflow.json` of the 7.1 run that trained `--custom-model`: locates its committee (for the uncertainty), and gives `--siesta-check`/`--to-mlff` the training level of theory (`calc.fdf`, pseudopotentials, dispersion, k x a).

`--no-committee`

Do not evaluate the committee of the custom model on the band. On by default when the card records a committee whose model files are found (next to the card's run folder, or in `--mlff-workflow`'s): the force spread per image and each member's barrier. switch

`--committee-models PATH [PATH ...]`

The committee's model files explicitly (each with its `mlff_model_card.json` next to it), instead of the card's own list.

`--siesta-check`

Write single-point SIESTA folders for images of the relaxed band into <output-dir>/siesta_check/image_NN/ (`--siesta-check-images`). Run SIESTA there and run `stb-mlneb` again with the same options: finished folders whose geometry matches the band are read and the model's profile is compared with SIESTA's on the same geometries. Needs `--calc` or `--mlff-workflow`. switch, off by default

`--siesta-check-images {ends-saddle,all}`

Which images `--siesta-check` writes: the two endpoints and the climbing image (default), or every image. choices: ends-saddle, all

`--calc CALC`

`calc.fdf` template for `--siesta-check`/`--to-mlff` (no geometry; it must %include the structure file, which each folder gets). Default with `--mlff-workflow`: its reference `calc.fdf`.

`--pseudo-dir PSEUDO_DIR`

Pseudopotentials for `--siesta-check` with `--calc`: a folder or a bank name (default: .).

`--kgrid KA KB KC`

k-grid of the check/export folders. Default: with `--mlff-workflow` the reference's grid scaled to this cell (same k x a as the training set); with `--calc` its own grid. type: int

`--to-mlff`

Add the band's images to the training set of the `--mlff-workflow` run, as configs of its next round (<run>/configs_round_NN/neb_<output-dir>/), with the workflow's level of theory. Images already labelled by `--siesta-check` are copied with their output. Then `stb-mlffAnalysis` trains the next round on them.switch, off by default

`--to-mlff-rattle N`

With `--to-mlff`, also N rattled copies of every image (default: 0): the neighbourhood of the path, not only the path itself. type: int

`--to-mlff-stdev TO_MLFF_STDEV`

Gaussian displacement of `--to-mlff-rattle` copies, Ang (default: 0.05). type: float

`--to-mlff-seed TO_MLFF_SEED`

Random seed of `--to-mlff-rattle` (default: 7). type: int

`--n-images N_IMAGES`

Total images including both endpoints (default: 7, at least 3). type: int

`--autosort-tol AUTOSORT_TOL`
Atom-correspondence tolerance between `--initial`/`--final`, Ang (default: 0.5, pymatgen's own documented value). type: float

`--no-idpp`
Skip the IDPP (Image Dependent Pair Potential) refinement of the interior images before the NEB. On by default: a geometric smoothing that usually gives a much better start than linear interpolation. switch

`--idpp-fmax IDPP_FMAX`
Force convergence for IDPP (default: 0.1). type: float

`--idpp-steps IDPP_STEPS`
Max steps for IDPP (default: 100). type: int

`--no-pre-relax-endpoints`
Skip the relaxation (positions only) of both endpoints with the model. On by default, so the barrier is measured from the model's own minima. switch

`--freeze-substrate`
Freeze the atoms that move less than `--freeze-threshold` between `--initial` and `--final` during the NEB: fewer degrees of freedom, no spectator atom wandering. Off by default. switch, off by default

`--freeze-threshold FREEZE_THRESHOLD`
Displacement threshold for `--freeze-substrate`, Ang (default: 0.3). type: float

`--k K`
NEB spring constant, eV/Ang^2 (default: 0.1). type: float

`--fmax FMAX`
Force convergence of the endpoint relaxations and of the NEB, eV/Ang (default: 0.05). The shaping stage stops at max(2 x fmax, 0.1); the climbing stage at fmax. type: float

`--max-steps MAX_STEPS`
Optimizer steps of each endpoint relaxation, and of the NEB in total: the shaping stage gets half, the climbing stage the rest (default: 200). type: int

`--device {cpu,cuda}`
Device to run the model on (default: cpu). choices: cpu, cuda

`-o, --output-dir OUTPUT_DIR`
Output directory (default: mlneb_out).

`--show`
Open the figure on screen at the end (needs a display; without one it is a warning). `neb_profile.png` is written either way. switch, off by default

`--save-gnuplot`
Write .dat + .gplot into <output-dir>/plot/ and run gnuplot there (`neb_profile.pdf`) when it is installed. Off by default. switch, off by default

`--save-data`
Write the band as a table: `neb_band.dat` (and `neb_band_foundation.dat` with the comparison): image, reaction coordinate, energy, max force, shortest distance, shift. Off by default. switch, off by default

`--save-images`
Also write each relaxed image as `image_NN.fdf` (bare structure, no calc or pseudopotentials) – e.g. to start a SIESTA NEB (`stb-neb`) from this band. Off by default. switch, off by default

`--save-report`
Also persist the report to `stb_mlneb_report.txt`. Off by default. switch, off by default

Like every STB command it also accepts `-h`/`-help`, `-v`/`-version` and `-no-intro` (Section 3.3).

From the built-in help.

```
Example usage:
  stb-mlneb --initial initial.fdf --final final.fdf
  stb-mlneb --initial initial.fdf --final final.fdf --custom-model mlff_model.model --custom-model-card
  mlff_model_card.json --device cuda --n-images 9 --show --save-gnuplot
  stb-mlneb --initial initial.fdf --final final.fdf --freeze-substrate --save-images --save-report
  stb-mlneb --initial i.fdf --final f.fdf --custom-model best_model/mlff_model_compiled.model
 --custom-model-card best_model/mlff_model_card.json --mlff-workflow mlff_runs/mlff_workflow.json --siesta-check
  # then run SIESTA, run it again, and add --to-mlff
```

13.6 ML Adsorption-Site Screening

At a glance

Menu code: 7.6 **Command:** `stb-mladsorb`
Guided example: `examples/7.6-mladsorb/`
Summary: Finds the candidate adsorption sites of a slab, relaxes the adsorbate on each with a MACE potential, reports where every run ended, and groups the runs into distinct minima with their energies, multiplicities and populations – with the committee, the foundation model and the model card's trained range alongside.

13.6.1 Overview

`stb-mladsorb` answers *where does this atom or molecule sit on this surface* with a MACE potential only: the foundation model (`-model`) or a model fine-tuned with the MLFF workflow (7.1; `-custom-model` with `-custom-model-card`). No SIESTA input is written; the result tells which sites deserve a SIESTA calculation (**`stb-adsorb`**, 5.8). One run:

1. **Clean slab.** Layers from the top, the model's forces on the slab as given and, with `-relax-depth` D, its relaxation with every atom deeper than D Å fixed. A slab cut from the bulk is not at the model's minimum (a Si(111) slab: 2.2 eV/Å before, −2.2 eV on relaxing): with `-relax-depth 0` (default) the substrate stays as given and the report says so.
2. **References.** The adsorbate alone in a box of `-vacuum-box` Å and, with `-bulk-reference`, each element's chemical potential μ from its bulk (positions and cell relaxed).
3. **Sites.** pymatgen's `AdsorbateSiteFinder` gives ontop, bridge and hollow sites, reduced by the slab's symmetry; every site of the cell is mapped to its symmetry-distinct one (its multiplicity) and described by the atom right below it. The finder counts every atom within 0.9 Å of the top as surface, so its names can mislead: on Si(111) its "ontop above L2" is the T4 site and its "bridge above L4" is H3 – the description says which is which.
4. **Relaxations.** One per adsorbate, site and starting distance (`-height`, `-start-heights`); for a molecule, `-n-orientations` random rotations are compared first by a single-point energy.
5. **Where each run ended.** The adsorbate's lowest atom is matched to the nearest site of the cell (0.6 Å): a bridge start that slides into a hollow is reported as a hollow. Runs that end at equivalent sites within 0.02 eV are one minimum.

6. **Minima.** For each minimum i, with E every model energy of the same model:

$$E_{\text{ads}} = E_{\text{slab+ads}} - E_{\text{slab}} - E_{\text{ads, isolated}}, \qquad E_{\text{form}} = E_{\text{slab+ads}} - E_{\text{slab}} - \sum_X n_X \mu_X, \qquad p_i = \frac{m_i\, e^{-\Delta E_i / k_B T}}{\sum_j m_j\, e^{-\Delta E_j / k_B T}}$$

with m_i the multiplicity, ΔE_i the energy above the most stable minimum and T = `-temperature` (configurational populations, no vibrational entropy). The report also lists how far each minimum is from the most stable one and prints the **`stb-mlneb`** command between their `.fdf` files: the surface-diffusion barrier is a **`stb-mlneb`** run (7.5), with its committee, SIESTA check and export to training.

Before you trust an ML adsorption energy

- **A model validated on the bulk is not validated on a surface.** In the guided example the round-3 silicon model of 7.5 (test force error 22.7 meV/Å) puts a Si adatom on Si(111) on T4, as SIESTA does, but its second minimum (B2) is not a minimum of SIESTA, its H3–T4 gap is 1.47 eV against SIESTA's 0.27 eV, its formation energy of T4 is -2.19 eV against -0.32 eV, and it relaxes the clean surface by -2.2 eV against -0.14 eV (Section 13.6.2.8).
- **The isolated adsorbate is outside a fine-tuned model**: there its isolated atom is 2.5 eV too low (cohesive energy 3.43 eV against SIESTA's 5.95 eV). Compare sites by ΔE, and use `-bulk-reference`.
- **Read the committee, not only the trained range**: the trained-range check flagged only the dimensionality; the committee's force spread was 37–78 times its test-set value, and its spread of E_{form} the size of the error.
- **A run that stays at a site proves it stationary, not a minimum**; check the best sites with SIESTA.

13.6.2 Guide

The guide below is the hands-on tutorial of the documentation site. It follows the repository folder `examples/7.6-mladsorb/`*; file names in it are relative to that folder, and its script* `example_*.sh` *reproduces every step.*

`stb-mladsorb` answers **where an atom or a molecule sits on a surface**, with every energy and force from a MACE machine-learned potential: the MACE-MP-0 foundation model, or a model fine-tuned on your own SIESTA calculations with the MLFF workflow (`7.1`), which always comes with its **model card**. You give it a slab and an adsorbate; it finds the candidate sites, relaxes the adsorbate on each, and reports:

- the clean slab, its layers and, with `--relax-depth`, its relaxation with the model;
- every candidate site, reduced by the slab's symmetry, with its multiplicity in the cell and the atom right below it;
- every relaxation, and **where it ended** — an adsorbate placed on a bridge that slides into a hollow is reported as a hollow;
- the **distinct minima**: their adsorption energy against the isolated adsorbate (`E_ads`) and, with `--bulk-reference`, against the element's bulk (`E_form`), the energy above the most stable one, the population at a temperature, the bonds, how far each is from the most stable one;
- with a fine-tuned model: the same screening with the foundation model, and the spread of the model's committee on every minimum;
- every structure against the model card's trained range, and all warnings in a last section.

It writes no SIESTA input. Its job is to say which sites deserve a SIESTA calculation (`stb-adsorb`, `5.8`), and to hand `stb-mlneb` (`7.5`) the minima between which a diffusion barrier is wanted.

Why a potential for a site screening? Six candidate sites of a 25-atom slab, each relaxed with the top layers free, cost six SIESTA relaxations: an afternoon on a workstation. With a MACE model on a GPU the same screening takes two minutes, and a dozen variations of it (how much of the slab relaxes, where the adsorbate starts, which model) cost less than one SIESTA run. The price is the one of every ML number: the model is only as good as its training data near the structures it is asked about — and a model trained on a bulk crystal has never seen a surface.

The worked example (sections 6 and 7) puts a silicon adatom on Si(111) with the round-3 silicon model of the `7.5` example, trained on bulk silicon only, and checks every minimum with SIESTA at the model's own level of theory.

- In two minutes on a GPU the model finds the right first site, **T4**, as the literature and SIESTA do.
- Almost everything else it reports is wrong:
  - its second minimum (B2, +1.17 eV) is not a minimum of SIESTA, which relaxes it into T4;
  - it puts H3 1.47 eV above T4, against SIESTA's **0.27 eV**;
  - it gives T4 a formation energy of −2.19 eV, against SIESTA's **−0.32 eV**;
  - it relaxes the clean surface by −2.2 eV, against SIESTA's −0.14 eV.
- Its isolated atom, the reference of `E_ads`, is 2.5 eV off.
- The trained-range check flags only the dimensionality. The **committee** is the alarm: a force spread 37–78 times its test-set value, and a spread of `E_form` (0.85 eV) of the size of the error.
- Five SIESTA relaxations at the model's level of theory (a few hours on 6 cores) settle it.

13.6.2.1 Contents

1. Theory
2. The screening at a glance
3. Libraries and external dependencies used
4. What `stb-mladsorb` does
5. Running it both ways
6. Worked example: a silicon adatom on Si(111)
7. The SIESTA check
8. What to take from the example
9. Reproducing the reference data
10. Known, deliberate limitations
11. Step-by-step: a surface of your own
12. Files in this folder
13. Running the script
14. Flag reference
15. What's next
16. References

13.6.2.2 1. Theory

1.1 What a site screening answers. An adsorbate on a crystal surface moves on a potential energy surface that has the periodicity of the surface. Its local minima are the **adsorption sites**: the lowest is

where the adsorbate sits at low temperature, the others are metastable sites it visits on the way, and the saddle points between neighbouring minima set how fast it diffuses (a barrier is a `stb-mlneb` calculation, `7.5`). A screening samples this surface: it places the adsorbate on a set of candidate positions, relaxes each, and collects the minima the relaxations end in.

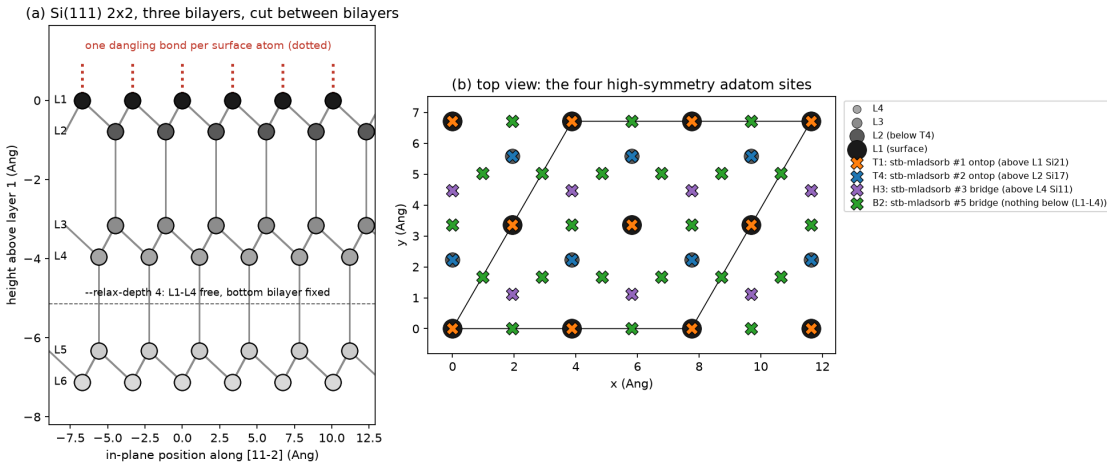
Two facts shape how the result must be read.

- **A relaxation finds the minimum of the basin it starts in, not the global one.** Starting points are the high-symmetry positions of the surface (on top of an atom, between two, in a hollow between three), because the minima of a crystalline surface usually sit there. Where a relaxation ends is a result; which site it started from is not.
- **A run that stays at a high-symmetry site proves the site is stationary, not that it is a minimum.** On a site whose symmetry has a three-fold axis or two mirror planes, the in-plane force on the adsorbate is zero by symmetry; an optimizer started exactly there cannot leave it even if it is a saddle or a maximum. Small asymmetries (the substrate relaxing, numerical noise, a lower-symmetry cell) usually let a run slide off a maximum — section 6.3 shows two that did — but not always. The test is a run started off the site (`--start-heights` changes only the height), the binding curve, or a SIESTA phonon calculation of the adsorbate.

1.2 The slab model. A surface is modelled by a **slab**: a few atomic layers, periodic in the plane, with vacuum above and below so that the periodic images along the normal do not interact. Three choices define it.

- **How it is cut.** The example's Si(111) slab is cut *between* two bilayers, through the long bonds along [111] that connect one bilayer to the next: every surface atom keeps three bonds and loses one (one *dangling bond*). A cut through the short bonds inside a bilayer would leave three dangling bonds per surface atom — a surface no calculation should start from (`make_slabs.py` drops those two single-atom layers; Figure 1).
- **How much of it may relax.** The deepest layers stand for the bulk below and are fixed; the top layers relax, alone first (the clean slab) and then with the adsorbate. `--relax-depth D` frees every atom within `D` Å of the top surface. A slab cut from the bulk is not at its minimum: with the round-3 model the example's slab relaxes by −2.2 eV (SIESTA: −0.14 eV, section 6.2). With `--relax-depth 0` (the default) the substrate stays exactly as given and only the adsorbate moves — fast, but the surface cannot respond to the adsorbate, and an unrelaxed surface makes every number an upper bound (section 6.6).
- **How large it is.** One adsorbate in an `n × n` surface cell is a periodic array of adsorbates at a coverage of $1/n^2$ monolayer (ML), each `n a` from its neighbours. The example's 2×2 cell is ¼ ML, adatoms 7.75 Å apart; the 3×3 cell (1/9 ML) and a thicker slab check what the choice changes (section 6.6).

The slab also has a bottom surface, here as dangling as the top one; it is fixed, far below (7.5 Å of silicon), and the same in every run, so it cancels in every energy difference.



Si(111) slab of the example

Figure 1. (a) The Si(111) 2×2 slab of the example, seen along [1-10]: three bilayers (L1–L2, L3–L4, L5–L6), cut between bilayers so every surface atom keeps one dangling bond (dotted). With `--relax-depth 4` *the top four layers relax, the bottom bilayer is fixed. (b) Top view with the four high-symmetry adatom sites as* `stb-mladsorb` *finds them: T1 on top of a first-layer atom, T4 above a second-layer atom, H3 above a fourth-layer atom (a hollow with nothing below it in the top three layers), B2 between two first-layer atoms. The legend gives the tool's own names (section 1.4).*

1.3 Adsorption energy, and its two references. With all three energies from the same model,

```
E_ads  = E(slab + X) − E(slab) − E(X, isolated)
E_form = E(slab + X) − E(slab) − Σ_i n_i μ_i ,      μ_i = E(bulk of element i) / N
```

`E_ads` is the energy released when the adsorbate X comes from vacuum (negative = bound). `E_form` measures the same structure from a reservoir: each atom of X taken from the bulk of its element, at chemical potential μ_i. For an adatom of the substrate's own element, `E_form` is the formation energy of the adatom (taken from a kink or a step, whose energy is the bulk's); for a Si adatom on the ideal Si(111) surface it is negative, because one adatom saturates three dangling bonds of a surface that is very far from its minimum.

The two are related by the **cohesive energy** of the element: for one atom, `E_ads − E_form` = μ − `E(atom) = −E_coh`. This identity is a check of the model: at the training level of theory SIESTA puts the cohesive energy of silicon at 5.95 eV against a spin-unpolarized atom (the setting of the training set) and 5.09 eV against a spin-polarized one (section 6.2; plane-wave PBE gives ~4.6 eV, the localized DZP basis overbinds the solid). A model whose `E_ads − E_form` is far from that has a wrong isolated atom.

Why the isolated adsorbate is the weak reference of a fine-tuned model. A model fine-tuned on condensed SIESTA configurations (crystals, rattled cells, MD frames) has never seen an atom or a molecule alone in vacuum. Its energy there is little more than its E0s — the per-element constants fitted to SIESTA's absolute energy scale — plus whatever the network predicts for "no neighbours", which nothing constrained. `E_ads` inherits that error in full; the energies of the sites *relative to each other* (ΔE) do not, and neither does `E_form`, whose reference (the bulk) is the centre of the training set. The report gives all three and says so; the example measures the error (section 6.2).

Neither number includes zero-point energy, vibrational entropy or the configurational entropy of a partial coverage: they are electronic energies at 0 K of one adsorbate per cell.

1.4 Finding candidate sites. The tool uses pymatgen's `AdsorbateSiteFinder` [11], built for high-throughput adsorption studies:

1. **Surface atoms.** Every atom within 0.9 Å (along the surface normal) of the topmost one.
2. **Triangulation.** The in-plane projections of the surface atoms, with their periodic images, are triangulated (Delaunay). An **ontop** site lies over each surface atom, a **bridge** at the midpoint of each edge, a **hollow** at the centroid of each triangle.
3. **Symmetry reduction.** The slab's space-group operations (`--symprec`) that keep the surface normal map sites onto each other; one representative of each class is kept. `stb-mladsorb` also counts how many sites of the cell each representative stands for — its **multiplicity**, used for the populations (section 1.6).
4. **Placement.** The adsorbate goes `--height` Å above the surface along the normal (for a molecule, pymatgen aligns its own axis with the normal; `--n-orientations` samples rotations, section 4.1).

The names can mislead — read what is below. Step 1 counts as "surface" every atom within 0.9 Å of the top. On Si(111) the second layer is only 0.79 Å below the first (0.58 Å once the slab relaxes), so both count, and their projections form a honeycomb: the finder's "ontop" sites lie over first- *and* second-layer atoms, and its "bridges" over midpoints of the honeycomb's edges and diagonals. The site names of the surface-science literature — T1, T4, H3, B2 (Figure 1b) — are about what lies *below* the site, and `stb-mladsorb` describes every site by the topmost atom within 0.5 Å below it:

Literature	Where	`stb-mladsorb` (this slab)	Description in the report
T1	on top of a first-layer atom	#1 ontop	above L1
T4	above a second-layer atom, between three first-layer atoms	#2 ontop	above L2
H3	above a fourth-layer atom, the "empty" hollow	#3 bridge	above L4
—	between a first- and a second-layer atom	#4 bridge	nothing below (L1–L4)
B2	between two first-layer atoms	#5 bridge	nothing below (L1–L4)
—	between T1 and H3	#6 bridge	nothing below (L1–L4)

The two unnamed bridges are no high-symmetry adatom site; on this surface both relaxations started there slid away (section 6.3). On an fcc(111) metal the same column separates the two hollows: above L2 is the hcp hollow, above L3 the fcc hollow.

1.5 The Si(111) adatom sites. On the ideal Si(111) surface every first-layer atom has one dangling bond. An adatom has four electrons and can bond to up to three of them; where it does so decides the site.

- **T1** (on top): one bond to one surface atom — two of the adatom's electrons stay unbonded, and the three dangling bonds around it are untouched.
- **T4** and **H3**: three bonds, one to each of three first-layer atoms. They differ in what lies below: at T4 a second-layer atom sits right under the adatom, ~2.5 Å away; at H3 there is nothing below in the top three layers.
- **B2**: two bonds, between two surface atoms.

Three bonds beat two beat one, so T4 and H3 should both be far below T1, and B2 in between. Between T4 and H3, the calculations of the 1980s settled the question for group-III and group-IV adatoms on Si(111): **T4** has the lower energy — for Al, Ga and In [3], and for Ge, Sn and Pb by 0.3–0.6 eV [5]. The extra interaction with the second-layer atom below (Figure 1) is what favours it [2, 4]. It is also the site of the silicon adatoms in nature: the 7×7 reconstruction of clean Si(111) — the dimer–adatom–stacking-fault (DAS) model [1] — places its twelve adatoms per cell on T4-like sites in local 2×2 arrangements, and

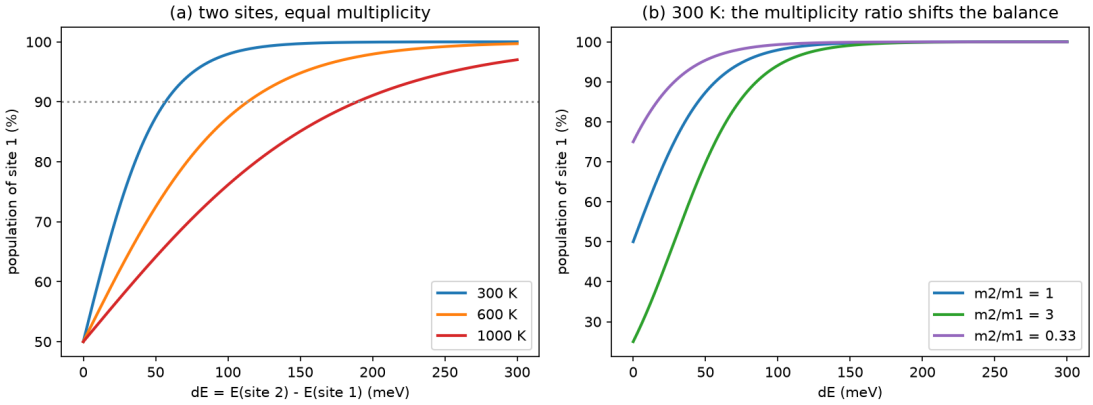
quenched Si(111) shows 2×2 and c(4×2) arrays of adatoms on T4 sites [6]. One adatom per 2×2 cell on T4 is the simplest of these structures [7]. The example is therefore a test with a known answer: T4 first, H3 second (a few tenths of an eV above), B2 and T1 well above.

1.6 From runs to minima, and their populations. Each relaxation is classified by **where it ended**: the adsorbate's lowest atom (its anchor, the one bonded to the surface) is matched to the nearest site of the whole cell, within 0.6 Å in-plane (else "off-site"). Runs of one adsorbate that end at symmetry-equivalent sites with energies within 0.02 eV are one **minimum**; runs that end at the same site with different energies (a molecule in two orientations) are two.

At temperature `T`, if the adsorbate can move between the minima (if the barriers are low enough for the time scale of interest), the fraction of time it spends in minimum `i` is the Boltzmann population

```
p_i = m_i exp(−ΔE_i / k_B T) / Σ_j m_j exp(−ΔE_j / k_B T)
```

with `ΔE_i` its energy above the most stable minimum and `m_i` its **multiplicity** — how many equivalent copies of the site the cell has. At 300 K, `k_B T` = 25.9 meV: a minimum 60 meV up holds ~9 % if the two have the same multiplicity (Figure 2), and a minimum three times as numerous needs about 28 meV more to be as rare. The populations are configurational only — no vibrational entropy, no zero-point energy — and say nothing about the time needed to reach equilibrium.



Populations

Figure 2. (a) Population of the more stable of two sites of equal multiplicity against their energy difference, at three temperatures. (b) At 300 K, the multiplicity ratio shifts the balance by `k_B T ln(m₂/m₁)`.

1.7 Binding curves at fixed height. `--height-sweep MIN MAX STEP` computes, for each site, the energy as a function of the adsorbate's height: at each height the anchor atom is **held** — vertically and laterally — while the rest of the adsorbate and the free substrate relax. The curve's minimum (a parabola through its three lowest points) is the equilibrium height on that site; its curvature is the stiffness of the bond along the normal; a curve that falls all the way to the lowest height means the adsorbate wants to go *into* the surface. Held laterally, the adsorbate stays on its site at every height, so the curve exists even for a site that is not a minimum in the plane — the curve of T1, for example, is defined even if T1 is a saddle.

1.8 Energies from a machine-learned potential. Every number above comes from a MACE model [8], an equivariant message-passing network that predicts a local energy per atom from its neighbourhood within a cutoff (6 Å here); forces are its exact gradient. Two kinds of model can be used.

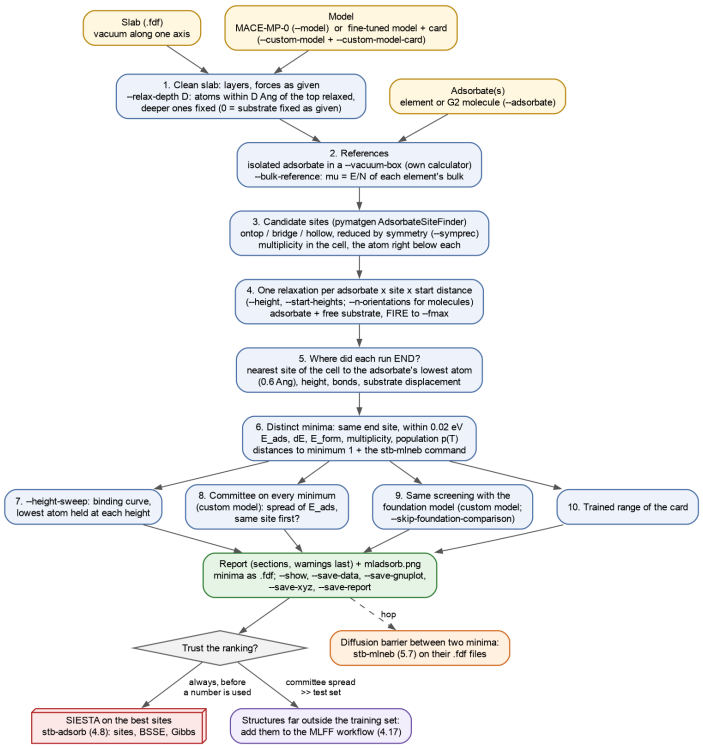
- The **MACE-MP-0 foundation models** [9] (`--model small|medium|large`), trained on ~1.5 million DFT relaxation frames of the Materials Project (PBE/PBE+U, VASP). They know every element, but not your level of theory (functional, pseudopotential, basis), and their energies are on their own absolute scale.
- A **fine-tuned model** (`--custom-model`), trained on your own SIESTA calculations with the MLFF workflow (`7.1`). It reproduces your level of theory where its training set reaches, and nothing guarantees it elsewhere. Its card records what that set covered (`Model card -- training data and trained domain`), and the report compares every structure with it (`TRAINED RANGE`).

A surface is the textbook case of "elsewhere" for a model trained on a bulk crystal: atoms with three neighbours instead of four, a dangling bond, a vacuum on one side. The trained-range check flags the dimensionality (`2D slab`), but it cannot say how wrong the model is there. The **committee** can bound it from below: the 7.1 workflow trains several members with different seeds, and where they disagree the training set did not constrain them. The report evaluates every member on every minimum (the spread of `E_ads`, of Δ`E` and of `E_form`, the force spread, and whether all members put the same site first). Members fine-tuned from the same foundation model share its habits and can be wrong together — the `7.5` example had a committee agree within 0.09 eV on a barrier that was 0.47 eV too high — so the spread is a lower bound of the error, and only SIESTA gives the error itself (section 7).

1.9 What this tool simplifies — explicitly.

- No reconstruction search: the slab is used as given (ideal here), the adsorbate is one atom or molecule per cell.
- No zero-point energy, no vibrational or configurational entropy in `E_ads`/`E_form`; populations are configurational.
- No spin, no charge, no dipole correction, no dispersion beyond what the model learned.
- The starting points are pymatgen's high-symmetry sites; nothing samples the surface between them, and a site where a run stays is stationary, not proven a minimum (section 1.1).
- A molecule's orientations are sampled at random by a single-point energy before the relaxation (`--n-orientations`), not exhaustively.

13.6.2.3 2. The screening at a glance



stb-mladsorb

`stb-mladsorb` itself is steps 1–10. It runs no SIESTA and writes no SIESTA input: the SIESTA side of the same question is the adsorption workflow `5.8` (`stb-adsorb` → `stb-adsorbBsse` → `stb-adsorbAnalysis` → `stb-adsorbGibbs`), whose own `--ml-rank` option uses the same site finder and the foundation model to pre-screen its candidates.

13.6.2.4 3. Libraries and external dependencies used

Library	Used for
pymatgen [12]	`AdsorbateSiteFinder` [11] (surface atoms, Delaunay triangulation, symmetry reduction, placing the adsorbate), `SpacegroupAnalyzer` (spglib: the slab's symmetry operations, for the multiplicities), structure conversion
ASE [10]	`Atoms`, FIRE [13] relaxations with `FixAtoms`, the G2 molecule database, neighbour distances, extended XYZ
mace-torch [8, 9]	the MACE calculator, the foundation models (downloaded once, cached in `~/.cache/mace/`) and custom `.model` files
PyTorch	runs the model on `cpu` or `cuda` (`--device`)
NumPy, SciPy	geometry, random orientations of a molecule (`scipy.spatial.transform.Rotation`)
matplotlib	`mladsorb.png` (always written), `--show`
gnuplot (optional)	renders the `.gplot` files written by `--save-gnuplot`
`stb.adsorb`	slab orientation, the adsorbate database, the isolated-adsorbate box — shared with the SIESTA workflow (`5.8`)
`stb.core`	`.fdf` reading and writing, the model card, the calculator and the relaxations (`mace_relax`), library-noise capture, report sections, gnuplot

Install the `ml` extra: `pip install stb_suite[ml]`.

13.6.2.5 4. What `stb-mladsorb` does

4.1 Step by step.

1. **Read the slab** (`.fdf`) and find its vacuum axis (`--vacuum-gap`, 10 Å); a slab whose vacuum is not along `c` is relabelled (warning). Split it into layers along the normal (0.35 Å tolerance).
2. **Load the model**: a foundation model (`--model`) or a custom model with its card (`--custom-model` + `--custom-model-card`; the card must match the model's SHA-256). Print the card in full.
3. **Clean slab.** Energy and forces as given; with `--relax-depth D`, relax the atoms within `D` Å of the top (FIRE, `--fmax`, `--max-steps`), the deeper ones fixed. The relaxed slab is the substrate of every run and the reference of every energy.
4. **References.** Each adsorbate alone in a cubic box of `--vacuum-box` Å, relaxed if it has more than one atom, with a calculator of its own (a compiled MACE model on CUDA fails on every later call once it has seen a structure with no neighbour in its cutoff). With `--bulk-reference FILE...`, each element's bulk is relaxed, positions and cell, and μ = `E/N`.
5. **Candidate sites** on the relaxed slab (section 1.4): `--site-type all|ontop|bridge|hollow`, reduced at `--symprec`. Equivalent positions are merged even when pymatgen labels them differently, and every site of the cell is generated as the orbit of its class under the slab's symmetry operations — the multiplicity is the orbit's size.
6. **Relaxations.** For each adsorbate, each site and each starting distance (`--height`, or `--start-heights H...`): place the adsorbate (for a molecule, the best of `--n-orientations` random rotations by single-point energy, seed `--seed`), relax it and the free substrate. Record energy, steps, residual force.
7. **Where each run ended** (section 1.6): the anchor atom's nearest site, its in-plane shift, its height above layer 1, its substrate neighbours (within 1.25× the sum of covalent radii) and bonds, the largest substrate displacement.
8. **Minima**, grouped and ranked; `E_ads`, Δ`E`, `E_form`, multiplicity, population at `--temperature`; the distance from the most stable minimum to every other one and the `stb-mlneb` command for the nearest.
9. **Binding curves** (`--height-sweep MIN MAX STEP`): the same sites with the anchor held at each height (section 1.7).
10. **Committee** (custom model, unless `--no-committee`): the card's members (next to its run folder, or `--mlff-workflow`'s, or `--committee-models`) evaluate the clean slab, the references and every minimum.
11. **Foundation comparison** (custom model, unless `--skip-foundation-comparison`): steps 3–8 again with `--model`, from the same sites.
12. **Trained range** (custom model): the structures against the card.
13. **Report, figure, files** (section 4.3).

4.2 The report, section by section. Sections are numbered in order; those that do not apply are left out.

Section	Content
`RUN METADATA`	date, versions, device, slab, adsorbates, sites and starting distances, substrate treatment, relaxation settings, extras, temperature
`MODEL`	the model card in full, or the foundation model's identity; load time
`CLEAN SLAB`	atoms, surface cell and area, thickness and vacuum; one row per layer (height as given and relaxed, spacing, atoms, free or fixed); energy and largest force as given; the relaxation (energy gained, forces before and after, steps, largest move)
`REFERENCES`	the isolated adsorbate (atoms, extent, box, energy, steps); with `--bulk-reference`, each element's volume per atom before and after and μ

Section	Content
CANDIDATE SITES	how many sites in the cell, how many distinct, how many symmetry operations; per site: label, type, what is below, multiplicity, in-plane fractional position
RELAXATIONS	one row per run: start and starting distance, where it ended, in-plane shift, height, coordination, nearest distance, E_ads, substrate displacement, steps, residual force; runs that left their site in cyan
DISTINCT MINIMA	rank, site, what is below, E_ads, ΔE, E_form, multiplicity, population, height, coordination, bonds, the runs that ended there; the gap between the two best; the sites no run ended at; the distances from minimum 1 and the stb-mlneb command
BINDING CURVES (FIXED HEIGHT)	per site: heights, energies, the minimum (parabola) and how it was found
COMMITTEE	per minimum: every member's E_ads, ΔE and E_form with their spread, the force spread and the estimated force error; whether every member puts the same site first
FOUNDATION MODEL COMPARISON	per run: where it ended and ΔE/E_form with each model; the most stable site and the gap to the second, both models
TRAINED RANGE	elements, dimensionality, shortest distance, largest force, energy per atom against the card
SUMMARY & FILES	status, the most stable site of each adsorbate, the committee's largest spread, the files, the outputs not asked for, the wall time of each stage
WARNINGS	every warning, numbered; the count of library messages captured

The warnings include: a clean slab whose relaxation hit its limit, or with every atom free; a fixed substrate under large forces; an adsorbate wider than half the box; sites the symmetry could not map; relaxations that hit --max-steps; an adsorbate ending closer than 1 Å to an atom; two minima closer than 50 meV; a binding curve whose minimum is at the edge of the sweep; a committee spread far above the test set's, members that disagree on the best site or on the gap; the fine-tuning changing the most stable site; structures outside the trained range; a bulk reference whose volume changes by more than 3 %; --show without a display.

4.3 Files.

Written	When
mladsorb.png: (a) top view — substrate, every site coloured by class, minima (stars) and moves (arrows); (b) the minima, with the committee's spread and the foundation model; (c) every run, start → end; (d) binding curves	always; --show opens it
rankNN_<type>_site<K>_<adsorbate>.fdf: each minimum (bare structure, positions in the cell)	always; --top-k N keeps the first N
mladsorb_configs.dat (every run), mladsorb_minima.dat (the minima), mladsorb_binding_curve.dat, mladsorb_configs_foundation.dat	--save-data
mladsorb_runs.xyz, mladsorb_minima.xyz, mladsorb_slab.xyz, mladsorb_binding_curve.xyz: extended XYZ with energy, forces, fixed atoms, per-atom adsorbate and layer, and the run's data in each frame	--save-xyz
plot/: substrate.dat, sites.dat, minima_xy.dat, cell.dat, minima.dat, binding_curve.dat, the .gplot scripts (top_view, site_energies, binding_curve) and their PDFs	--save-gnuplot (the PDFs when gnuplot is installed)
stb_mladsorb_report.txt	--save-report

Everything goes into -o/--output-dir (default mladsorb_out).

4.4 Guard rails.

Situation	Behaviour
`--custom-model` without `--custom-model-card`, an unreadable card, a SHA-256 mismatch	clean error
a missing slab, bulk-reference or workflow file; an unknown adsorbate; `--relax-depth` below 0	clean error
a "slab" with no vacuum axis	clean error
no site of the requested type	clean error
a slab whose vacuum is not along `c`	relabelled, warning
a relaxation that hits `--max-steps`	warning; the energy is an upper bound
`--relax-depth` deeper than the slab	warning: nothing is fixed
an element outside the model	error from the calculator
`--device cuda` with no usable GPU	clean error
`--show` without a display; gnuplot not installed	warning; files written anyway

13.6.2.6 5. Running it both ways

Command line:

```
# a first look with the foundation model, substrate fixed (fast)
stb-mladsorb --slab slab.fdf --adsorbate H --model medium --save-report

# a fine-tuned model, the top layers relaxed, a bulk reference, every output
stb-mladsorb --slab si111_2x2.fdf --adsorbate Si --relax-depth 4 --bulk-reference si_bulk.fdf \
             --custom-model best_model/mlff_model_compiled.model --custom-model-card
             best_model/mlff_model_card.json \
             --mlff-workflow mlff_runs/mlff_workflow.json --device cuda \
             --save-data --save-gnuplot --save-xyz --save-report --show

# a molecule: random orientations; several adsorbates; binding curves
stb-mladsorb --slab slab.fdf --adsorbate H2O --n-orientations 8 --seed 1
stb-mladsorb --slab slab.fdf --adsorbate O,H --site-type hollow --height-sweep 1.0 3.0 0.25

# then the barrier between the two best minima, with the command the report prints
stb-mlneb --initial mladsorb_out/rank01_....fdf --final mladsorb_out/rank03_....fdf --autosort-tol 0
...
```

Interactive menu: run `stb-suite` and type `7.6`. It asks, in this order:

1. the slab and the adsorbate(s);
2. a custom model path, or Enter for a foundation model. With a custom model it asks for the card (offering `mlff_model_card.json` next to the model) and whether to compare with the foundation model; without one, the MACE-MP-0 size;
3. the device (`cpu`/`cuda`; it checks the GPU);
4. the site type, the starting distance, the relaxation depth (Enter: substrate fixed), the bulk reference (Enter: none), how many minima to write;
5. the number of orientations for a molecule;
6. the output folder, then one question per optional output (show, data, gnuplot, XYZ, report).

The menu builds the same command line and calls `stb-mladsorb`; the guided script (Case 9) drives it with piped answers. The options without a menu question (`--start-heights`, `--height-sweep`, `--temperature`, `--symprec`, `--fmax`, `--max-steps`, `--committee-models`, ...) are for the command line.

13.6.2.7 6. Worked example: a silicon adatom on Si(111)

6.1 Set-up.

- **Slab:** `si111_2x2.fdf`, the ideal Si(111) surface in a 2×2 cell, three bilayers (24 atoms), cut between bilayers, at the lattice constant of the `7.1` training set (5.4802 Å; `make_slabs.py`). 7.5 Å of vacuum on each side.
- **Adsorbate:** one Si atom per cell, ¼ ML.
- **Model:** round 3 of the silicon MLFF run (`../7.5-mlneb/model_round3/`): fine-tuned from MACE-MP-0 medium on 71 SIESTA configurations of **bulk** silicon (rattled and strained 8-atom cells, MD frames of 64 atoms, and the 14 vacancy-hop configurations the `7.5` example added). Test-set force error 22.7 meV/Å, phonons within 1.1 THz of SIESTA. It has never seen a surface.
- **Settings of the main case:** `--relax-depth 4` (the top two bilayers relax, the bottom bilayer is fixed), `--bulk-reference si_bulk.fdf` (the 8-atom training cell), the committee of round 3 (`--mlff-workflow`), the foundation comparison (MACE-MP-0 medium), all outputs. 2–3 minutes on an RTX 3060.

```
stb-mladsorb --slab si111_2x2.fdf --adsorbate Si --relax-depth 4 --bulk-reference si_bulk.fdf \
             --custom-model ../7.5-mlneb/model_round3/mlff_model_compiled.model \
             --custom-model-card ../7.5-mlneb/model_round3/mlff_model_card.json \
             --mlff-workflow <4.17 run>/mlff_workflow.json --device cuda \
             --save-data --save-gnuplot --save-xyz --save-report -o main
```

6.2 The clean slab and the references.

```
layer | height as given (Ang) | height relaxed (Ang) | spacing (Ang) | atoms | in the relaxations
L1    | +0.000                | +0.000               | -             | 4     | free
L2    | -0.791                | -0.581               | 0.581         | 4     | free
L3    | -3.164                | -3.288               | 2.707         | 4     | free
L4    | -3.955                | -4.088               | 0.800         | 4     | free
L5    | -6.328                | -6.745               | 2.657         | 4     | fixed
L6    | -7.119                | -7.536               | 0.791         | 4     | fixed
As given          : E = -2743.310247 eV, largest force 2.2474 eV/Ang (atom 6, L5)
Relaxed           : --relax-depth 4 Ang -> 16 atom(s) free, 8 fixed; E = -2745.533265 eV
                    (relaxation -2.2230 eV), max|F| on the free atoms 2.2474 -> 0.0443 eV/Ang,
                    29 steps (converged), largest move 0.626 Ang
```

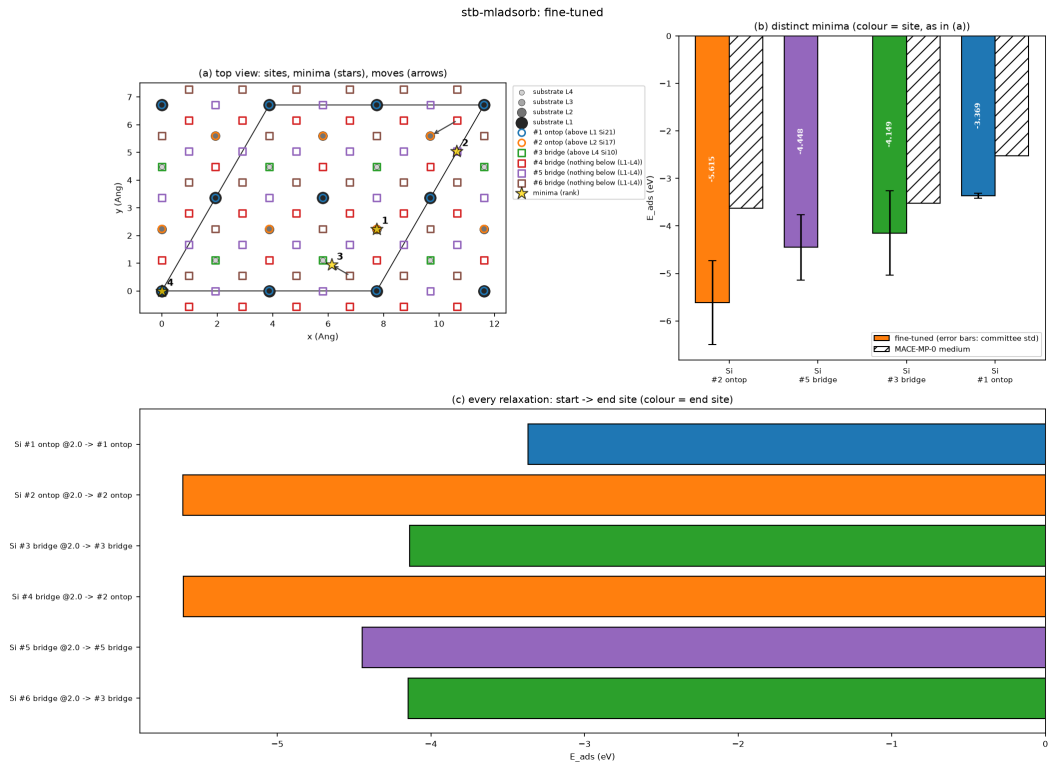
The model relaxes the surface strongly inwards: the first bilayer flattens (0.79 → 0.58 Å) and the slab gains 2.2 eV. Its largest force on the ideal slab, 2.25 eV/Å, is on the bottom bilayer (fixed); on the top layer it is 0.13 eV/Å. **SIESTA disagrees from the start**: on the same ideal slab its largest force is 0.69 eV/Å (also at the bottom), and its own relaxation of the top 4 Å gains only 0.14 eV — the model's relaxed slab is 0.94 eV *above* the ideal one in SIESTA (`reference_results/siesta/slab_ideal/`, section 7). The other models are closer to SIESTA here: round 2 sees 0.44 eV/Å and gains 0.06 eV, the foundation models see 0.08 eV/Å. The round-3 model has learned something about three-coordinated silicon that is wrong for a surface.

The references (`REFERENCES`): the model's isolated Si atom has E = −111.583 eV, its bulk μ = −115.008 eV/atom at 20.594 Å^3/atom (the file's 20.574). So the model's cohesive energy is `E(atom)` − μ = **3.43 eV**. SIESTA at the same level of theory gives μ = −115.018 eV/atom — the model's bulk is right to 9 meV — and an isolated, spin-unpolarized Si atom (the training's own setting) of −109.065 eV: a cohesive energy of **5.95 eV** (5.09 eV against a spin-polarized atom; `reference_results/siesta/atom*/`). The model's isolated atom is 2.5 eV too low: every `E_ads` below carries that error, `E_form` does not (section 1.3).

6.3 Where the runs ended. `CANDIDATE SITES` finds 48 sites in the cell, six distinct (24 symmetry operations keep the normal): T1, T4, H3, and three bridges with nothing below them in the top four layers, among them B2. One run per site:

```
run | start     | ended at  | moved (A) | height (A) | CN | d_nn (A) | E_ads (eV) | substrate (A) |
steps
1   | #1 ontop  | #1 ontop  | 0.00      | 2.574      | 1  | 2.376    | -3.3689    | 0.199         | 44
2   | #2 ontop  | #2 ontop  | 0.00      | 1.541      | 4  | 2.472    | -5.6155    | 1.075         | 51
3   | #3 bridge | #3 bridge | 0.00      | 1.444      | 3  | 2.666    | -4.1393    | 0.248         | 52
4   | #4 bridge | #2 ontop  | 1.05      | 1.536      | 4  | 2.471    | -5.6126    | 1.053         | 68
5   | #5 bridge | #5 bridge | 0.02      | 2.259      | 3  | 2.352    | -4.4483    | 1.123         | 61
6   | #6 bridge | #3 bridge | 0.76      | 1.578      | 5  | 2.615    | -4.1486    | 0.476         | 59
```

Two runs left their site: the bridge between a first- and a second-layer atom (#4) slid 1.05 Å into T4, and the bridge between T1 and H3 (#6) slid into H3 (0.36 Å short of its centre, 9 meV below run 3, which stayed at the centre). Four sites held their runs; section 1.1 says what that proves, and section 7 shows one of them was not a minimum of SIESTA. The substrate column shows how much the surface responds: more than 1 Å for T4 and B2.



The main case

Figure 3. `mladsorb.png` *of the main case: (a) top view, every site of the cell coloured by class, the four minima (stars, by rank) and the two runs that moved (arrows); (b) the minima with the committee's spread (error bars) and the foundation model (hatched); (c) every run, start → end, coloured by where it ended.*

6.4 The minima.

```
rank | site      | what is below | E_ads (eV) | dE (eV) | E_form (eV) | in cell | pop. 300 K (%) |
height (A) | CN
```

```
1    | #2 ontop  | above L2 Si17 | -5.6155    | +0.0000 | -2.1904     | 4       | 100.0          |
1.541      | 4
2    | #5 bridge | nothing below | -4.4483    | +1.1671 | -1.0233     | 12      | 0.0            |
2.259      | 3
3    | #3 bridge | above L4 Si10 | -4.1486    | +1.4669 | -0.7235     | 4       | 0.0            |
1.578      | 5
4    | #1 ontop  | above L1 Si21 | -3.3689    | +2.2466 | 0.0562      | 4       | 0.0            |
2.574      | 1
```

T4 first, as section 1.5 expects: three bonds of 2.47 Å to the first layer and the second-layer atom below at 2.51 Å (coordination 4). Then B2 (+1.17 eV), H3 (+1.47 eV) and T1 (+2.25 eV, one bond). At 300 K everything sits on T4. The order of the first two is not the literature's (H3 second), and the gaps are large; the committee and SIESTA will say why.

The report ends the section with the distances from T4 to the other minima and the `stb-mlneb` command for the nearest one as written in its file — H3, 2.08 Å away. That barrier is a `stb-mlneb` run (`7.5`); with this model, outside its domain on a surface, it is not worth computing before the model is fixed (section 8).

6.5 The committee, the trained range, and the foundation model. The three members of round 3, evaluated on the four minima (`COMMITTEE`):

```
rank | site      | members E_ads (eV)   | std   | members dE (eV)      | members E_form (eV)  | std
| F spread
1    | #2 ontop  | -5.111 -3.544 -5.615 | 0.882 | +0.000 +0.000 +0.000 | -1.685 -0.199 -2.190 | 0.845
| 1141
2    | #5 bridge | -3.592 -2.764 -4.448 | 0.688 | +1.519 +0.780 +1.167 | -0.166 0.581 -1.023  | 0.655
| 1951
3    | #3 bridge | -2.205 -2.332 -4.149 | 0.888 | +2.906 +1.212 +1.467 | 1.220 1.013 -0.723   | 0.872
| 2443
4    | #1 ontop  | -3.256 -3.260 -3.369 | 0.052 | +1.855 +0.284 +2.247 | 0.170 0.085 0.056    | 0.048
| 167
(F spread in meV/Ang)
```

All three put T4 first, but nothing else agrees: `E_form` of T4 spans −2.19 to −0.20 eV, and the force spread reaches 1.1–2.4 eV/Å on the three multiply-bonded sites — 37 to 78 times the 31 meV/Å the same members show on their test set. Only T1, one ordinary bond above an ideal surface atom, is in agreement (0.17 eV/Å). The report turns this into warnings: the model is far outside what its training set constrains. The trained-range table says only half of it — `dimensionality 3D vs 2D slab: OUTSIDE`, everything else "inside": distances, forces and energies per atom of a surface are within the bulk's ranges; the *environments* are not.

The foundation model (MACE-MP-0 medium, `FOUNDATION MODEL COMPARISON`) agrees on T4 first and disagrees on the rest: H3 only 0.10 eV above T4, T1 1.10 eV, and no B2 minimum (run 5 slides into T4). Its `E_form` of T4 is +0.43 eV — the adatom *costs* energy instead of releasing it. Its clean slab hardly relaxes (−0.01 eV, largest force 0.08 eV/Å on the ideal slab), where round 3 gains 2.2 eV and SIESTA 0.14 eV.

6.6 Settings: how much relaxes, where the run starts, slab size, other models. `make_reference.sh` runs the main case with one setting changed at a time; `results_figures.py` collects the lowest minimum at each named site:

```
E_form = E(slab+Si) - E(slab) - mu_Si (eV), lowest minimum at each site   |   energy above T4 (eV)
case                                        T4       H3       B2       T1   |      H3      B2      T1
round 3, top 4 A relaxed (main)      -2.190   -0.723   -1.023    0.056   |   1.467   1.167   2.247
```

```
round 3, substrate fixed            -1.255   -0.223       -   -0.562   |   1.031       -   0.692
round 3, top 1 A relaxed            -1.301   -2.162       -    0.012   |  -0.861       -   1.313
round 3, top 7 A relaxed            -2.233   -0.815  -1.149    0.055   |   1.418   1.084   2.288
round 3, 4 start heights            -2.190   -0.723  -1.024    0.054   |   1.467   1.166   2.244
round 3, 4 bilayers                 -1.472   -0.472  -0.196    0.109   |   1.000   1.276   1.581
round 3, 3x3 cell (1/9 ML)          -2.470   -1.189       -    0.012   |   1.281       -   2.482
round 2                             -0.240    1.187       -    1.335   |   1.427       -   1.575
MACE-MP-0 small                      0.766    0.747       -    2.075   |  -0.019       -   1.310
MACE-MP-0 medium                     0.430    0.531       -    1.526   |   0.101       -   1.096
MACE-MP-0 large                      0.602    0.606       -    2.007   |   0.004       -   1.405
SIESTA, relaxed (section 7)         -0.318   -0.044       -    1.918   |   0.275       -   2.236
'-': no run ended at that site (not a minimum of that model there; for SIESTA: its relaxation from
the model's
     geometry ended at another site).
```

- **How much of the slab relaxes** (Figure 5). With the substrate fixed as cut, T4 is bound 0.9 eV less than with the top two bilayers free, and T1 comes out *below* zero: on a surface that cannot relax, any bond helps. Freeing only the first bilayer (`--relax-depth 1`) is worse than either: the model then puts H3 0.86 eV *below* T4 — half-relaxing a slab traps it in a strained state. Between 4 and 7 Å the numbers move by less than 0.13 eV: four layers are enough.
- **Where the run starts** (`--start-heights 1.5 2.0 2.5 3.0`, 24 runs): the same four minima, within 2 meV; 8 runs slid to another site. One H3 run ended in a second, higher H3 structure (+0.63 eV) — two minima at one site, kept apart by the energy criterion.
- **Thickness and coverage**: a fourth bilayer moves `E_form` of T4 by 0.72 eV and H3–T4 by 0.47 eV; a 3×3 cell (1/9 ML) by 0.28 eV and loses the B2 minimum. With SIESTA these would be convergence tests; with this model they mostly measure how differently it extrapolates on two slabs.
- **Other models**: round 2 — the same training set without the 14 vacancy configurations of `7.5` — gives a T4 `E_form` of −0.24 eV and a clean slab that barely relaxes (−0.06 eV); the three foundation models put T4 and H3 within 0.1 eV of each other and every `E_form` above zero.

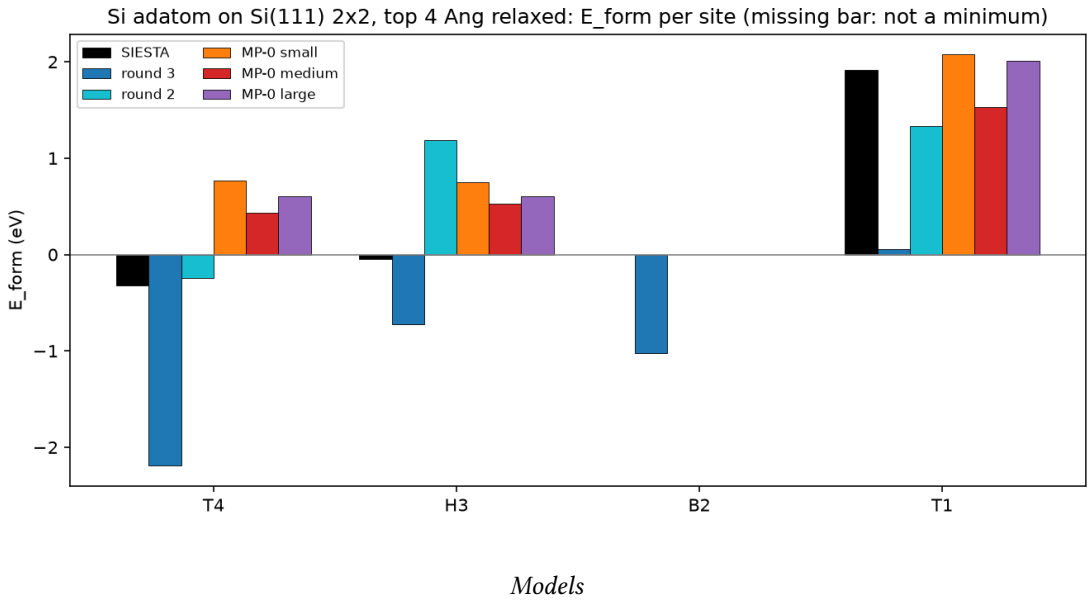
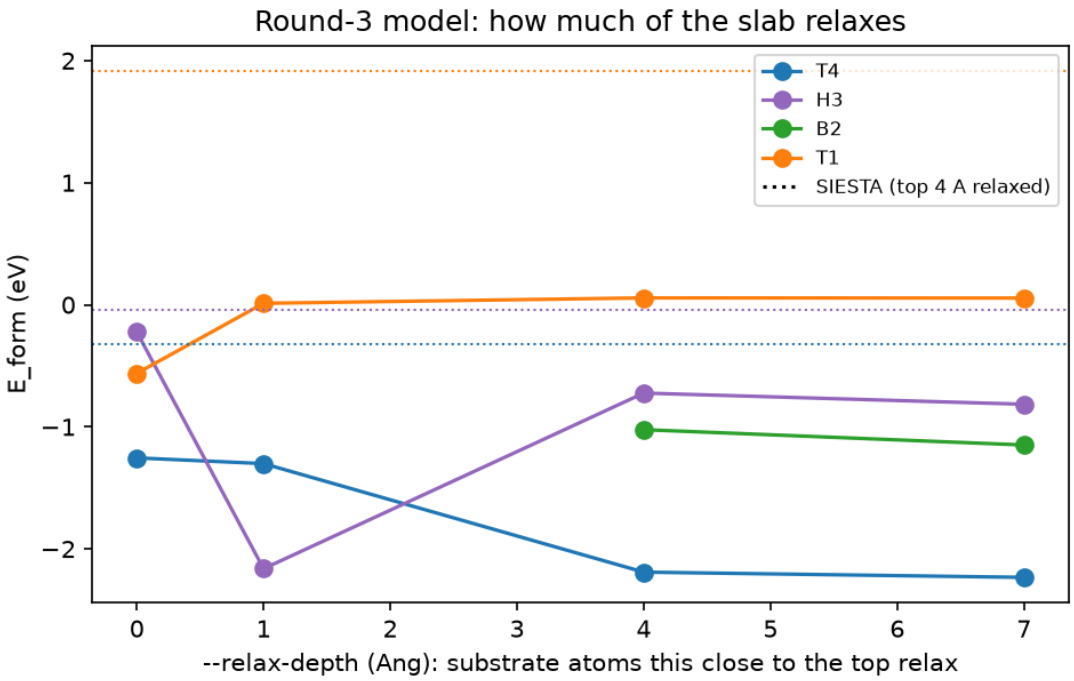


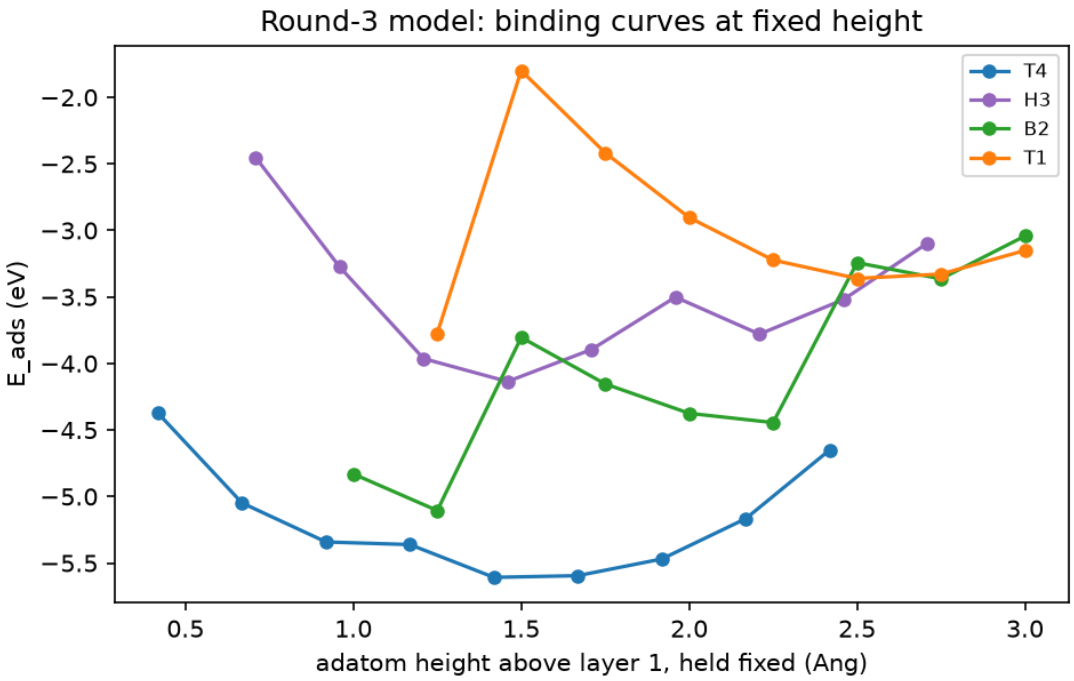
Figure 4. `E_form` *of each site with every model, top 4 Å relaxed, and with SIESTA (section 7). A missing bar: no run of that model ended at that site.*



Relaxation depth

Figure 5. `E_form` *of each site with the round-3 model against* `--relax-depth` *(0, 1, 4, 7 Å); dotted: SIESTA with the top 4 Å relaxed.*

Binding curves (`--height-sweep 1.0 3.0 0.25`, the adatom held at each height): for T4 and H3 the minimum of the curve falls where the free relaxation ended (1.53 vs 1.54 Å, −5.634 vs −5.616 eV; 1.44 vs 1.44 Å, −4.137 vs −4.139 eV) — a consistency check of both. The other curves show the model at its limit: at 1.0 Å above T1 the adatom is 0.82 Å from the surface atom below and the model returns −278 eV with forces of 257 eV/Å (left out of the curve and warned); the B2 and T1 curves jump by 1–2 eV between neighbouring heights as the free substrate falls into different arrangements.



Binding curves

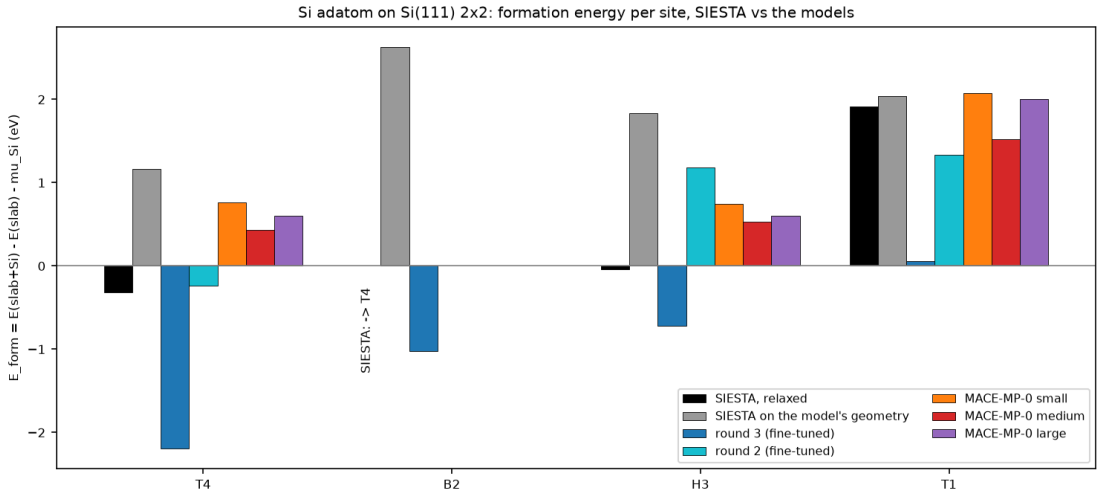
Figure 6. Binding curves of the round-3 model, the adatom held at each height (points closer than 0.8 × the bond length to an atom left out).

13.6.2.8 7. The SIESTA check

Each minimum of the main case, and its clean slab, relaxed with SIESTA at the training level of theory: PBE, DZP, `Mesh.Cutoff` 300 Ry, the training `calc.fdf`, k-grid 4×4×1 (`k × a` ≈ 31 Å in-plane, the training's 33 Å), CG to 0.05 eV/Å (the model's `--fmax`), fixed cell, the bottom bilayer fixed, each starting from the model's own geometry (`siesta/make_inputs.py`; 33–89 CG steps, 30–90 minutes each on 6 cores). Step 0 of each relaxation is SIESTA on the model's geometry. `siesta_compare.py`:

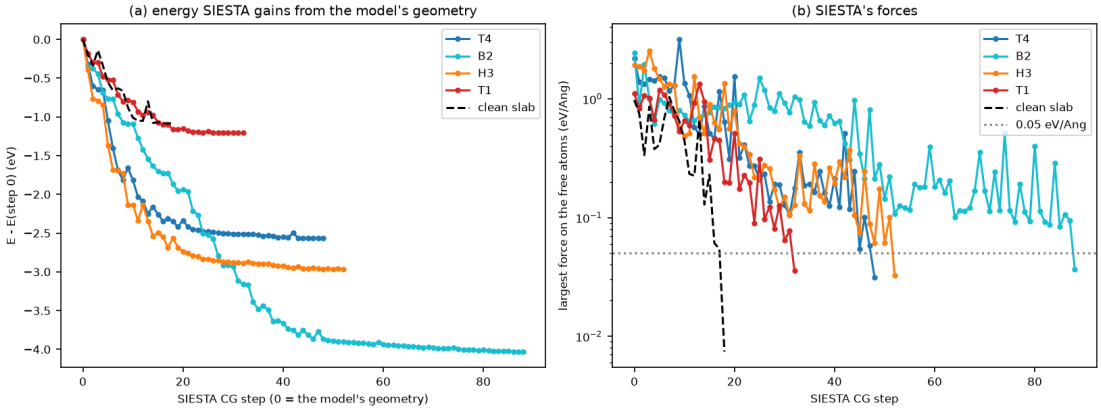
```
site  steps  F0 (eV/A)  Fend   E_form SIESTA  @ML geom   relax | round 3  round 2  MP-0 medium
T4       49      2.206  0.031         -0.318     1.162  -1.481 |  -2.190   -0.240        0.430
B2       89      2.427  0.037         -0.316     2.629  -2.945 |  -1.023       -            -
H3       53      1.922  0.033         -0.044     1.836  -1.880 |  -0.724    1.187        0.531
T1       33      1.106  0.036          1.918     2.041  -0.123 |   0.056    1.335        1.526

Where SIESTA's relaxation ended:
  T4: ended at T4 (0.00 Ang from it), 0.979 Ang above layer 1, bonds 2.516 2.516 2.516 2.547
  B2: ended at T4 (0.04 Ang from it), 0.959 Ang above layer 1, bonds 2.517 2.517 2.522 2.542
  H3: ended at H3 (0.06 Ang from it), 1.345 Ang above layer 1, bonds 2.471 2.474 2.474
  T1: ended at T1 (0.00 Ang from it), 2.059 Ang above layer 1, bonds 2.292
```



SIESTA vs the models

Figure 7. `E_form` *of each site: SIESTA relaxed, SIESTA on the model's geometry, and every model. B2 has no SIESTA bar: from the model's B2 geometry SIESTA relaxes into T4.*



SIESTA relaxations

Figure 8. The SIESTA relaxations from the model's geometries: (a) energy gained, (b) SIESTA's largest force. Step 0 is the model's minimum.

What it shows:

1. **The site order is the literature's, and the model got only the first place right.** SIESTA: T4, then H3 **0.27 eV** above (within the 0.3–0.6 eV calculated for Ge, Sn, Pb on the same site pair [5]), then T1 **2.24 eV** above. Round 3: T4, B2, H3 (1.47 eV), T1 (2.25 eV). The T1 gap agrees to 10 meV; the H3 gap is 5 times too large.
2. **B2 is not a minimum.** From the model's B2 geometry SIESTA's relaxation moves the adatom 1.1 Å into T4 (it ends 0.04 Å from T4, 2.5 meV above SIESTA's own T4). Round 3's second-best minimum is an artefact of the model; the foundation models and round 2 had no B2 minimum at all.
3. **The model's geometries are far from SIESTA's minima.** SIESTA's forces on them are 1.1–2.4 eV/Å (the model's own: below 0.05), and relaxing them gains 0.1 (T1) to 2.9 eV (B2). At T4 SIESTA puts the adatom 0.98 Å above the first layer, the model 1.54 Å: SIESTA's adatom sits deeper, its three bonds 2.52 Å and the second-layer atom below it at 2.55 Å.

4. **`E_form` is off by up to 2 eV.** SIESTA: T4 −0.32, H3 −0.04, T1 +1.92 eV. Round 3 overbinds every multiply-bonded site by 0.7–1.9 eV; round 2, without the vacancy configurations, is within 0.08 eV on T4 but has no H3 minimum; the foundation models put every site 0.6–0.8 eV too high. None is usable for a number.
5. **The round-3 model on SIESTA's own geometries** (single points): T4 −1.58 eV against SIESTA's −0.32, with forces of 1.8 eV/Å on the free atoms — it does not recognise SIESTA's minima as minima either.
6. **The committee was right to warn.** Its spread (std 0.85 eV in `E_form` of T4) is the size of the error (1.9 eV), and one member (−0.20 eV) was close to SIESTA — but the members still all chose T4, for the right reason or not.

13.6.2.9 8. What to take from the example

- **A screening reports where each relaxation ended.** Two of six runs left their starting site; on Si(111) the names pymatgen gives the sites (ontop, bridge) say little — the atom below says which site it is.
- **A run that stays at a site proves only that the site is stationary.** B2 held the model's run and is not a minimum of SIESTA.
- **Relax the top of the slab, and enough of it.** A slab cut from the bulk is far from any model's minimum; freeing the first bilayer only gave a wrong first site.
- **Measure energies from a reference the model knows.** The isolated atom of a fine-tuned model is off by 2.5 eV here; `E_form` against the bulk removes that, though not the error of the slab itself.
- **A model validated on the bulk is not validated on a surface.** Test error 22.7 meV/Å, phonons within 1 THz, and an `E_form` off by 2 eV on the first surface it sees. The trained-range check flags the dimensionality and nothing else; the committee spread (37–78× its test-set value) is the real alarm.
- **What to do with it**: use the screening to choose which sites to compute with SIESTA (`stb-adsorb`, `5.8`), and, if the model is to be used on surfaces, put the SIESTA-relaxed slabs and adatoms of section 7 into its training set (`7.1`) — the same loop that fixed the vacancy barrier in `7.5`.

13.6.2.10 9. Reproducing the reference data

`reference_results/` holds one folder per case of `make_reference.sh` (report, figure, data tables, minima as `.fdf` and extended XYZ, gnuplot files of the main case) and `siesta/` with the SIESTA relaxations of section 7 (inputs, `siesta.out`, the final geometry and `compare.txt`).

```
source ~/.anaconda3/etc/profile.d/conda.sh && conda activate stb   # or any environment with
stb_suite[ml]
python3 make_slabs.py                  # the three slabs (seconds; identical to the files shipped)
MLFF_RUN=<4.17 run folder> bash make_reference.sh   # every case into reference_run/ (GPU, ~30 min)
python3 results_figures.py             # the summary of section 6; relax_depth.png, models.png,
binding_curves.png
python3 siesta_compare.py              # section 7: compare.txt, figures/siesta_vs_ml.png,
siesta_relaxation.png
python3 theory_figures.py              # the figures of section 1 (seconds)
dot -Tpng -Gdpi=150 figures/mladsorb_flowchart.dot -o figures/mladsorb_flowchart.png    # the
flowchart (graphviz)
```

Each case is skipped when its report exists (`FORCE=1` reruns). The foundation models are downloaded on first use. The SIESTA relaxations are not scripted in `make_reference.sh` (about an hour each on 6

cores): `siesta/make_inputs.py` writes their folders from the main case's minima, and `siesta/run_siesta.sh` runs them.

13.6.2.11 10. Known, deliberate limitations

- **Starting points are high-symmetry sites**; nothing samples the surface between them, and a run that stays at a site proves it stationary, not a minimum (section 1.1).
- **One adsorbate per cell**: the coverage is set by the cell; adsorbate–adsorbate interactions are those of the periodic array (the 3×3 cell of section 6.6 measures them).
- **0 K electronic energies**: no zero-point energy, no vibrational entropy; populations are configurational.
- **No reconstruction**: the slab is used as given.
- **No barrier**: the diffusion barrier between two minima is a `stb-mlneb` run (`7.5`).
- **The committee spread is a lower bound** of the error.
- **No charge, no spin, no dipole correction**: the model reproduces whatever its labels were.

13.6.2.12 11. Step-by-step: a surface of your own

1. **Build the slab** with whole layers and the termination you mean (here: between bilayers), enough vacuum ($\geq$ 10 Å) and a bottom you will fix. `stb-slab` (`2.x`) or a script like `make_slabs.py`.
2. **A first look with the foundation model**, `--relax-depth` set to the top layers: which sites exist, where runs end, how far apart the minima are. Read the sites by what is below them.
3. **With a fine-tuned model** (`7.1`), add `--mlff-workflow` of its run (the committee) and `--bulk-reference` (a reference inside its domain). Read `TRAINED RANGE` and `COMMITTEE`: a model trained without surfaces is extrapolating, and its spread says how much at least.
4. **Converge the model's answer**: `--relax-depth`, `--start-heights`, the slab thickness and the cell (section 6.6).
5. **Check with SIESTA** the best two or three sites (`stb-adsorb`, `5.8`, or a relaxation from the minima's `.fdf`, as in section 7). If the model is far off, add those structures to the training set and train another round.
6. **The barrier** between neighbouring minima: `stb-mlneb` with the command the report prints.

13.6.2.13 12. Files in this folder

File	What it is
`README.md`	this tutorial
`example_7.6.sh`	the guided walk-through (about 15 min on a CPU, 3 min with `DEVICE=cuda`)
`make_slabs.py`	builds the three slabs below from the diamond structure
`si111_2x2.fdf`, `si111_2x2_4bl.fdf`, `si111_3x3.fdf`	Si(111) 2×2 with three and four bilayers, 3×3 with three (24, 32, 54 atoms)
`si_bulk.fdf`	the 8-atom silicon cell of the `7.1` training set, the bulk reference
`make_reference.sh`	regenerates the cases of `reference_results/` (GPU)
`results_figures.py`	reads `reference_results/`, prints the summary of section 6, draws `relax_depth.png`, `models.png`, `binding_curves.png`
`siesta_compare.py`	section 7: SIESTA against the models
`theory_figures.py`	the figures of section 1: the slab and its sites, the populations
`siesta/`	`make_inputs.py` and `run_siesta.sh`: the SIESTA relaxations of section 7
`figures/`	every figure of this README; `mladsorb_flowchart.dot` is the flowchart's source

`reference_results/`	every `make_reference.sh` case and the SIESTA relaxations (section 9)

The model is `../7.5-mlneb/model_round3/` (round 3 of the silicon run), round 2 is `../7.2-mlmd/model/`; neither is copied.

13.6.2.14 13. Running the script

```
cd examples/7.6-mladsorb
bash example_7.6.sh               # CPU, ~15 min; press Enter between cases
DEVICE=cuda bash example_7.6.sh   # on a GPU, ~3 min
```

Each case writes into `output/<case>/`, which is wiped at every start. The script never runs SIESTA: Case 7 reads the relaxations in `reference_results/siesta/`. Cases 1, 6 and 7 only read files; Cases 2, 4, 5 and 9 run the tool; Case 8 shows the guard rails.

13.6.2.15 14. Flag reference

`stb-mladsorb` — `--slab FILE` (`.fdf`, required) · `--adsorbate NAME[,NAME...]` (element or G2 molecule, required) · `--model {small,medium,large}` (default medium) · `--custom-model PATH` + `--custom-model-card JSON` (always both) · `--skip-foundation-comparison` · `--mlff-workflow JSON` · `--no-committee` · `--committee-models PATH...` · `--site-type {all,ontop,bridge,hollow}` (all) · `--height Å` (2.0) · `--start-heights Å...` · `--height-sweep MIN MAX STEP` · `--relax-depth Å` (0) · `--bulk-reference FILE...` · `--temperature K` (300) · `--n-orientations N` (1) · `--seed N` · `--symprec` (0.01) · `--vacuum-gap Å` (10) · `--vacuum-box Å` (20) · `--fmax eV/Å` (0.05) · `--max-steps N` (300) · `--top-k N` · `--device {cpu,cuda}` · `-o/--output-dir DIR` (default `mladsorb_out`) · `--show` · `--save-data` · `--save-gnuplot` · `--save-xyz` · `--save-report` · `--no-intro` · `-v`.

13.6.2.16 15. What's next

- **`5.8` (Adsorption)** computes the best sites with SIESTA, with the BSSE correction (`stb-adsorbBsse`) and the free energy (`stb-adsorbGibbs`) that this screening leaves out.
- **`7.5` (`stb-mlneb`)** gives the diffusion barrier between two minima, from their `.fdf` files, with its own committee, SIESTA check and export to training.
- **`7.1` (MLFF workflow)** trains the model; the minima and relaxations of a screening (`--save-xyz`, or the `.fdf` files) are the structures a surface-aware training set needs.
- **`7.2` (`stb-mlmd`)** shows the adatom moving between the sites at high temperature.

13.6.2.17 16. References

1. K. Takayanagi, Y. Tanishiro, M. Takahashi, S. Takahashi, "Structural analysis of Si(111)-7×7 by UHV-transmission electron diffraction and microscopy", *J. Vac. Sci. Technol. A* **3**, 1502 (1985), doi:10.1116/1.573160.
2. J. E. Northrup, "Si(111)$\sqrt{3}\times\sqrt{3}$-Al: An adatom-induced reconstruction", *Phys. Rev. Lett.* **53**, 683 (1984), doi:10.1103/PhysRevLett.53.683.
3. J. M. Nicholls, B. Reihl, J. E. Northrup, "Unoccupied surface states revealing the Si(111)$\sqrt{3}\times\sqrt{3}$-Al, -Ga, and -In adatom geometries", *Phys. Rev. B* **35**, 4137 (1987), doi:10.1103/PhysRevB.35.4137.
4. R. D. Meade, D. Vanderbilt, "Adatoms on Si(111) and Ge(111) surfaces", *Phys. Rev. B* **40**, 3905 (1989), doi:10.1103/PhysRevB.40.3905.

5. S. Özkaya et al., "Atomic and electronic structures of the group-IV elements on Si(111)-($\sqrt{3}\times\sqrt{3}$) surface", *J. Phys.: Conf. Ser.* **100**, 072025 (2008), doi:10.1088/1742-6596/100/7/072025.
6. R. S. Becker et al., "Dimer-adatom-stacking-fault (DAS) and non-DAS (111) semiconductor surfaces: A comparison of Ge(111)-c(2×8) to Si(111)-(2×2), -(5×5), -(7×7), and -(9×9) with scanning tunneling microscopy", *Phys. Rev. B* **39**, 1633 (1989), doi:10.1103/PhysRevB.39.1633.
7. J. E. Northrup et al., "Total energy of the adatom and pyramidal-cluster models for Si(111)", *Phys. Rev. B* **29**, 1966 (1984), doi:10.1103/PhysRevB.29.1966.
8. I. Batatia, D. P. Kovács, G. N. C. Simm, C. Ortner, G. Csányi, "MACE: Higher order equivariant message passing neural networks for fast and accurate force fields", *Adv. Neural Inf. Process. Syst.* **35**, 11423 (2022).
9. I. Batatia et al., "A foundation model for atomistic materials chemistry", arXiv:2401.00096 (2023).
10. A. H. Larsen et al., "The atomic simulation environment — a Python library for working with atoms", *J. Phys.: Condens. Matter* **29**, 273002 (2017).
11. J. H. Montoya, K. A. Persson, "A high-throughput framework for determining adsorption energies on solid surfaces", *npj Comput. Mater.* **3**, 14 (2017), doi:10.1038/s41524-017-0017-z.
12. S. P. Ong et al., "Python Materials Genomics (pymatgen): A robust, open-source python library for materials analysis", *Comput. Mater. Sci.* **68**, 314 (2013).
13. E. Bitzek, P. Koskinen, F. Gähler, M. Moseler, P. Gumbsch, "Structural relaxation made simple", *Phys. Rev. Lett.* **97**, 170201 (2006).
14. J. M. Soler et al., "The SIESTA method for ab initio order-N materials simulation", *J. Phys.: Condens. Matter* **14**, 2745 (2002).
15. J. P. Perdew, K. Burke, M. Ernzerhof, "Generalized gradient approximation made simple", *Phys. Rev. Lett.* **77**, 3865 (1996).

13.6.3 Command-line reference

13.6.3.1 `stb-mladsorb`

Adsorption-site screening driven entirely by a MACE potential – the MACE-MP-0 foundation model, or a model fine-tuned on your own SIESTA data with the 7.1 workflow (`--custom-model`, always with its model card). Finds the candidate ontop/bridge/hollow sites of the slab (symmetry-reduced, each described by the atom below it), relaxes the adsorbate on each (and the top layers of the slab with `--relax-depth`), reports where each run ENDED (a bridge start that slides into a hollow is a hollow), groups the runs into distinct minima with their multiplicity and Boltzmann population, and the adsorption energy relative to the isolated adsorbate and, with `--bulk-reference`, to the elements' bulk, and how far the other minima are from the most stable one (the input of a diffusion barrier with `stb-mlneb`, 7.5). Optional: binding curves at fixed height, the committee's spread on every minimum and the same screening with the foundation model. Every structure is checked against the model card's trained range; all warnings in the last section. No SIESTA input is written; for the DFT pipeline use `stb-adsorb` (5.8).

Synopsis.

```
usage: stb-mladsorb [-h] --slab SLAB --adsorbate ADSORBATE
                    [--model {small,medium,large}] [--custom-model PATH]
                    [--custom-model-card JSON] [--skip-foundation-comparison]
                    [--mlff-workflow JSON] [--no-committee]
                    [--committee-models PATH [PATH ...]]
                    [--site-type {all,ontop,bridge,hollow}] [--height HEIGHT]
                    [--start-heights H [H ...]] [--height-sweep MIN MAX STEP]
                    [--relax-depth ANG] [--bulk-reference FDF [FDF ...]]
                    [--temperature TEMPERATURE] [--n-orientations N_ORIENTATIONS]
                    [--seed SEED] [--symprec SYMPREC] [--vacuum-gap VACUUM_GAP]
```

```
[--vacuum-box VACUUM_BOX] [--fmax FMAX] [--max-steps MAX_STEPS]
[--top-k TOP_K] [--device {cpu,cuda}] [-o OUTPUT_DIR] [--show]
[--save-data] [--save-gnuplot] [--save-xyz] [--save-report]
[--no-intro] [-v]
```

Options

`--slab SLAB`

Slab/2D-material structure (.fdf), vacuum along one axis. required

`--adsorbate ADSORBATE`

Adsorbate(s): an ASE G2 molecule name (e.g. 'H2O') or an element symbol, comma-separated for several (e.g. 'H,O'). required

`--model {small,medium,large}`

MACE-MP-0 model size (default: medium). With `--custom-model`, the foundation model of the comparison. choices: small, medium, large

`--custom-model PATH`

A custom MACE model file instead of the MACE-MP-0 foundation potential – e.g. one fine-tuned via `stb-mlffAnalysis`. Overrides `--model`.

`--custom-model-card JSON`

Model card of the `--custom-model` file: the `mlff_model_card.json` that `stb-mlffAnalysis` wrote next to it (training data, level of theory, accuracy, energy scale). Required with `--custom-model` – the card is checked against the model file and used to warn when a structure is outside what the model was trained on.

`--skip-foundation-comparison`

With `--custom-model`, do not repeat the screening with the foundation model (`--model`). The comparison is on by default: it shows whether the fine-tuning changed the site ranking. switch, off by default

`--mlff-workflow JSON`

The `mlff_workflow.json` of the 7.1 run that trained `--custom-model`: locates its committee.

`--no-committee`

Do not evaluate the committee of the custom model on the minima. On by default when the card records a committee whose model files are found (next to the card's run folder, or in `--mlff-workflow`'s). switch

`--committee-models PATH [PATH ...]`

The committee's model files explicitly (each with its `mlff_model_card.json` next to it).

`--site-type {all,ontop,bridge,hollow}`

Which candidate site type(s) to screen (default: all). choices: all, ontop, bridge, hollow

`--height HEIGHT`

Starting distance of the adsorbate from the surface, Ang – pymatgen's AdsorbateSiteFinder definition (default: 2.0). type: float

`--start-heights H [H ...]`

Several starting distances (each one a run per site), e.g. 1.5 2.0 2.5: a relaxation can end in a different minimum depending on where it starts. Default: `--height` only. type: float

`--height-sweep MIN MAX STEP`

Binding curve per site: the adsorbate's lowest atom held at each height (the rest relaxes), E_ads(h) and its minimum. On top of the free relaxations. type: float

`--relax-depth ANG`

Substrate atoms within this depth (Ang) of the top surface relax – in the clean slab first, then with the adsorbate; deeper atoms stay fixed (default: 0 = the substrate is fixed as given and only the adsorbate moves). A slab cut from the bulk is not at the model's minimum: use the top layers (3-5 Ang). type: float

`--bulk-reference FDF [FDF ...]`
Bulk structure(s) of the adsorbate's element(s), one element per file: mu = E/N after relaxing positions and cell with the model, and E_form = E(slab+ads) - E(slab) - sum mu is reported next to E_ads.

`--temperature TEMPERATURE`
Temperature of the site populations, K (default: 300). type: float

`--n-orientations N_ORIENTATIONS`
For a multi-atom adsorbate, screen this many random orientations per site by a single-point energy and relax the best (default: 1 = the G2 orientation). No effect on a single atom. type: int

`--seed SEED`
Random seed of `--n-orientations` (default: unseeded). type: int

`--symprec SYMPREC`
Symmetry tolerance of the site reduction (default: 0.01). type: float

`--vacuum-gap VACUUM_GAP`
Gap (Ang) that identifies the slab's vacuum axis (default: 10.0). type: float

`--vacuum-box VACUUM_BOX`
Cubic box (Ang) of the isolated-adsorbate reference (default: 20.0). type: float

`--fmax FMAX`
Force convergence of every relaxation and of the NEB, eV/Ang (default: 0.05). type: float

`--max-steps MAX_STEPS`
Optimizer steps per relaxation (default: 300). type: int

`--top-k TOP_K`
Write only the N most stable minima as .fdf (default: all minima). type: int

`--device {cpu,cuda}`
Device of the model (default: cpu). choices: cpu, cuda

`-o, --output-dir OUTPUT_DIR`
Output directory (default: mladsorb_out).

`--show`
Open the figure on screen at the end (needs a display). `mladsorb.png` is written either way. switch, off by default

`--save-data`
Write the tables as .dat: every relaxation (`mladsorb_configs.dat`), the minima (`mladsorb_minima.dat`, with the committee spread) and the binding curves. Off by default. switch, off by default

`--save-gnuplot`
Write .dat + .gplot into <output-dir>/plot/ (top view, site energies, binding curve) and run gnuplot there when it is installed. Off by default. switch, off by default

`--save-xyz`
Write extended XYZ files: every relaxed run (`mladsorb_runs.xyz`), the minima (`mladsorb_minima.xyz`), the clean slab and the binding curves – energy, forces, fixed atoms, per-atom 'adsorbate' and 'layer', and the run's data in each frame's comment line. Off by default. switch, off by default

`--save-report`
Also write the report to `stb_mladsorb_report.txt`. switch, off by default

Like every STB command it also accepts `-h`/`--help`, `-v`/`--version` and `--no-intro` (Section 3.3).

From the built-in help.

```
Example usage:
  stb-mladsorb --slab slab.fdf --adsorbate H
```

```
stb-mladsorb --slab slab.fdf --adsorbate O,H --site-type hollow --relax-depth 3
stb-mladsorb --slab si111.fdf --adsorbate Si --relax-depth 4 --bulk-reference si_bulk.fdf --custom-model
best_model/mlff_model_compiled.model --custom-model-card best_model/mlff_model_card.json --device cuda --show
--save-data --save-gnuplot --save-xyz
stb-mladsorb --slab slab.fdf --adsorbate H2O --n-orientations 8 --seed 1
```

Part III

Developer reference

Chapter 14

Architecture

In this chapter

14.1 Repository layout . . . 854
14.2 Thin tools over a shared core . . . 854
14.3 The interactive menu . . . 855
14.4 Extensions . . . 855
14.5 Versioning . . . 856
14.6 Documentation . . . 856
14.7 Adding a tool . . . 856
14.8 Testing . . . 856

14.1 Repository layout

`stb-suite/src/stb/`	the package: one module per command, plus `core/`
`stb-suite/src/stb/core/`	the shared library
`stb-suite/src/stb/pseudopotentials/`	the bundled banks `dojo` and `virtual_vault`
`stb-suite/pyproject.toml`	metadata, dependencies, the `ml` extra and `[project.scripts]`
`stb-suite/meta.yaml`	conda recipe
`examples/<code>-<name>/`	one guided example per tool: README, script, fixtures
`test/`	smoke-test scripts, mirroring the menu categories
`docs/`	the documentation site generator, the `-help` snapshot and this manual

`stb-suite/build/` is a setuptools artefact, not source.

14.2 Thin tools over a shared core

Every command is a module `stb/<name>.py` with an argparse `main()`, registered in `[project.scripts]` (`stb-kgrid = "stb.kgrid:main"`). Logic shared by two or more tools lives in `stb.core`; a helper is moved there when a *second* tool needs it, not before (extract on second use). The main modules:

Module	Responsibility
`structure_io`	the only `.fdf` reader/writer (`read_fdf`, `write_fdf`, conversion to and from pymatgen), rewriting only the lattice or the coordinates of a full calculation file, supercells of a calculation file, `read_relaxed_or_input` (folder → `.XV` with species cross-check), `ChemicalSpeciesLabel` renumbering
`siesta_log`	parsers of `.out` logs (Fermi level, energies, stress in two forms, cell, MD trajectory, forces, dipole, spin moment, optical block, native charges, SCF/force diagnostics); every parser returns `None` instead of raising

Module	Responsibility
kspace	Monkhorst–Pack divisions, vacuum-axis detection, surface reference, dimensionality
fdf_directives, calc_directives	reading directives with SIESTA's rules (case, separators, first occurrence wins); forcing directives into a template text
pseudopotentials	bundled banks, resolution of -p, copying
symmetry	cell reduction, group labels, inequivalent sites, strain-mode equivalences and symmetry-allowed elastic bases
cli, menu, deps, citations	terminal output, reports and progress lines; run_tool; optional-dependency guards; BibTeX entries and merging
structure_checks, passivation, heterostructure	malformation checklist; dangling-bond capping; ZSL matching and bilayer construction
mace_relax, mace_phonons	MACE calculators, devices, relaxation and NEB; ML force constants
phonon_workflow, vibrational_symmetry, raman_symmetry, ir_symmetry, born_charges, spectrum, dielectric	phonopy plumbing with residual-force subtraction, mode classification, Born charges, spectra, .EPSIMG and Kramers–Kronig
siesta_bands, siesta_wfsx, orbitals, band_scatter, broadening, rho_io, grid_export	band files and edges; wavefunction loading modes; orbital names; scatter-on-bands plots; Gaussian broadening; density grids and their export
adsorption_sites, bsse, neb_manifest	slab/site geometry and forced blocks; ghost fragments; atom-correspondence manifests
eos_fit, gqca_solver, optical_setup, optical_properties, hirshfeld, xrd	equations of state, GQCA, optical symmetry and constants, Hirshfeld-I, powder diffraction
mlff_workflow, atomic_env, md_traj	the MLFF state file; environment fingerprints; trajectory cells and unwrapping
run_files	the kind of every file of a SIESTA run folder (from SIESTA's naming conventions), the files an .fdf reads (%include, < file), job-script detection; shared by **stb-clean** and **stb-archive**

14.3 The interactive menu

stb-suite builds its menu from seven dictionaries, in menu order INPUT_TOOLS, ANALYSIS_TOOLS, UTILITY_TOOLS, STRUCTURE_TOOLS, WORKFLOW_TOOLS, CATALYSIS_TOOLS (empty here, filled by an extension) and MLSIM_TOOLS, keyed by menu number to a title, a description and a function; a workflow entry, in any of them, has a stages dictionary instead. One list, CATEGORIES, gives their numbers and titles. The flat table of dotted codes is derived from them at import time. Codes have no gaps: when an item is removed or moved, the items after it are renumbered. Every menu function asks its questions, builds an argument list and calls run_tool(command, args), which runs the installed console command as a subprocess and handles errors and the final pause; it is the only dispatch path. Tools meant for job scripts (**stb-nebCycle**) are not in the menu.

14.4 Extensions

Tools that should not live in the public repository can be shipped as a separate package that plugs into the menu. Before building the code table, **stb-suite** loads every module registered under the stb.plugins entry-point group; each may define any of the seven dictionaries, whose entries are merged by number

(a taken number, or a plugin that fails to import, only produces a warning). Their entries run their own console commands through `run_tool`; `stb.core.menu` is the API they import. The documentation tooling can include extension repositories listed in `STB_DOCS_EXTRA_ROOTS`; the public site and this manual are built without them.

14.5 Versioning

One version for the whole suite, computed by `stb/_version.py`: `STB_VERSION` if set; on a release branch (`DEV = False`) the constant `RELEASE`; on dev `RELEASE.dev<N>`, with N = `git rev-list --count BASE_TAG..HEAD` in a checkout, else the installed metadata. Releases are tagged (`v2.0.0`) on `main`; fixes are made on `dev`. Tools import it (`from stb import __version__`); no tool carries its own number.

14.6 Documentation

The site (MkDocs) is generated at build time by `docs/gen_pages.py`: guides from `examples/*/README.md`, reference pages from the `--help` snapshot `docs/reference_help.json` (rewritten by `docs/dump_help.py`), menu facts read from `stb_suite.py` and `pyproject.toml` with `ast`. This manual (`docs/latex/stb_manual.tex`) reuses the same guides and reads each command's argument parser directly for the option lists.

14.7 Adding a tool

1. Write `stb/<name>.py` with `main()`, taking `VERSION` from `stb.__version__`, supporting `--no-intro` and `--save-report`, and importing shared logic from `stb.core`.
2. Register `stb-<name>` in `[project.scripts]` and reinstall.
3. Add a menu entry (or a stage of a workflow) with a `run_*` function that calls `run_tool`.
4. Follow the conventions: `config_extra.fdf` for forced directives, level-of-theory read back from earlier stages, wrapping into the cell, the trajectory formats, gnuplot for workflow analyses and matplotlib for ML tools, `--custom-model` with foundation comparison for MACE tools.
5. Add an example folder (README, script, fixtures) – it becomes the tool's guide on the site – and a smoke test under `test/`; refresh the help snapshot.

14.8 Testing

There is no unit-test suite. `test/` holds scripts per menu item that generate inputs, run the installed command and check its output (for example `test/3-utils/1-translator/test.sh`); `test/version/`, `test/menu/` and `test/docs/` check the version scheme, the menu and the documentation generator.

Appendix A

Menu code map

Every code accepted at the main prompt of **`stb-suite`**, with the command it runs. Codes of workflow stages have three fields. Click a tool to go to its section, or a command to its option list.

Code	Tool / stage	Command
1 · Inputs		
1.1	Input File Generator	`stb-inputfile`
1.2	Pseudopotential Resolver	`stb-pseudo`
1.3	K-Grid Generator	`stb-kgrid`
1.4	K-Path Generator	`stb-kpath`
1.5	DFT+U / Hubbard Block Generator	`stb-dftu`
1.6	Structure Fetcher	`stb-fetch`
1.7	ML Pre-Relaxation	`stb-mlrelax`
2 · Analysis		
2.1	Bands Analyzer	`stb-bands`
2.2	PDOS XML Parser	`stb-dos`
2.3	DOS Processor (Convolution)	`stb-convdos`
2.4	Structure Analyzer	`stb-structural`
2.5	Symmetry Analyzer	`stb-symmetry`
2.6	Bader Charge Analysis	`stb-bader`
2.7	Work Function Calculator	`stb-workfunction`
2.8	Density Plotter (RHO)	`stb-density`
2.9	Powder XRD Simulator	`stb-xrd`
2.10	Fatbands Analyzer	`stb-fatbands`
2.11	STM Simulator	`stb-stm`
2.12	Wavefunction Density	`stb-wfdensity`
2.13	STS Spectroscopy Simulator	`stb-sts`
2.14	COOP/COHP Bonding Analysis	`stb-coop`
2.15	IPR Analyzer	`stb-ipr`
2.16	Effective Mass / Velocity	`stb-effmass`
2.17	Spin Texture Analyzer	`stb-spintexture`
2.18	AIMD Trajectory Analysis	`stb-aimdAnalysis`
2.19	Native Charges	`stb-nativecharges`
3 · Utils		
3.1	File Translator	`stb-translate`
3.2	Clean File Tools	`stb-clean`
3.3	Grid to Cube Converter	`stb-cube`
3.4	Wantibexos Interface	`stb-siesta2wtb`
3.5	AIMD Trajectory Converter	`stb-ani2traj`
3.6	Status Checker	`stb-status`
3.7	Archive Packager	`stb-archive`
4 · Structures		
4.1	2D Monolayer Stacker	`stb-2Dstacking`
4.2	Supercell Builder	`stb-supercell`
4.3	Slab Builder	`stb-slab`
4.4	Nanotube/Nanoribbon Builder	`stb-nanotube`
4.5	Point Defect Generator	`stb-defect`

Code	Tool / stage	Command
4.6	SQS Generator	`stb-sqs`
4.7	Unit Cell Finder	`stb-unitcell`
4.8	Crystal Builder	`stb-crystalbuilder`
4.9	Surface Passivator	`stb-passivate`
4.10	Reference Molecule Builder	`stb-molecule`
4.11	Amorphous Structure Generator	`stb-amorphize`
4.12	Random Crystal Generator	`stb-crystalcast`
5 · Workflows		
5.1.1	Stress-Strain: Stage 1 - Prep	`stb-strain`
5.1.2	Stress-Strain: Stage 2 - Analysis	`stb-strainAnalysis`
5.2.1	Elastic Constants: Stage 1 - Prep	`stb-elasticInputs`
5.2.2	Elastic Constants: Stage 2 - Analysis	`stb-elasticAnalysis`
5.3.1	Cohesive Energy: Stage 1 - Prep	`stb-cohesive`
5.3.2	Cohesive Energy: Stage 2 - Analysis	`stb-cohesiveAnalysis`
5.4.1	Phonons: Stage 1 - Prep	`stb-phononsCreate`
5.4.2	Phonons: Stage 2 - Analysis	`stb-phononsPos`
5.5.1	Convergence Tests: Stage 1 - Prep	`stb-convergence`
5.5.2	Convergence Tests: Stage 2 - Analysis	`stb-convergenceAnalysis`
5.6.1	Structure Solution (XRD): Stage 1 - Prep	`stb-xrdsearch`
5.6.2	Structure Solution (XRD): Stage 2 - Analysis	`stb-xrdrank`
5.7.1	Hubbard U (Linear Response): Stage 1 - Reference Prep	`stb-hubbardu`
5.7.2	Hubbard U (Linear Response): Stage 2 - Perturbation Prep	`stb-hubbarduAlphas`
5.7.3	Hubbard U (Linear Response): Stage 3 - Analysis	`stb-hubbarduAnalysis`
5.8.1	Adsorption: Stage 1 - Prep	`stb-adsorb`
5.8.2	Adsorption: Stage 2 - BSSE Prep	`stb-adsorbBsse`
5.8.3	Adsorption: Stage 3 - Analysis	`stb-adsorbAnalysis`
5.8.4	Adsorption: Stage 4 - Gibbs Free Energy	`stb-adsorbGibbs`
5.9.1	NEB / Reaction Path: Stage 1 - Prep	`stb-neb`
5.9.2	NEB / Reaction Path: Stage 2 - Analysis	`stb-nebAnalysis`
5.10.1	2D Stacking Fault (Gamma-Surface): Stage 1 - Prep	`stb-stackingfault`
5.10.2	2D Stacking Fault (Gamma-Surface): Stage 2 - Analysis	`stb-stackingfaultAnalysis`
5.11.1	Raman Spectrum: Stage 1 - Phonon Displacements	`stb-raman`
5.11.2	Raman Spectrum: Stage 2 - Modes & Optical Displacements	`stb-ramanModes`
5.11.3	Raman Spectrum: Stage 3 - Analysis	`stb-ramanAnalysis`
5.12.1	IR Spectrum: Stage 1 - Phonon Displacements	`stb-ir`
5.12.2	IR Spectrum: Stage 2 - Modes & Dipole/Born-Charge Displacements	`stb-irModes`
5.12.3	IR Spectrum: Stage 3 - Analysis	`stb-irAnalysis`
5.13.1	Generalized Quasi-Chemical Approximation (GQCA): Stage 1 - Structure Generation	`stb-gqca`
5.13.2	Generalized Quasi-Chemical Approximation (GQCA): Stage 2 - Analysis	`stb-gqcaAnalysis`
5.14.1	Optical Properties: Stage 1 - Direction Folders	`stb-optical`
5.14.2	Optical Properties: Stage 2 - Analysis	`stb-opticalAnalysis`
5.15.1	Equation of State: Stage 1 - Prep	`stb-eosInputs`
5.15.2	Equation of State: Stage 2 - Analysis	`stb-eosAnalysis`
5.16.1	Charge Density Difference: Stage 1 - Prep	`stb-chargediffPrep`
5.16.2	Charge Density Difference: Stage 2 - Analysis	`stb-chargediffAnalysis`
5.17.1	Hirshfeld-I Charge Partitioning: Stage 1 - Neutral Prep	`stb-hirshfeldPrep`
5.17.2	Hirshfeld-I Charge Partitioning: Stage 2 - Ion Prep	`stb-hirshfeldIons`

Code	Tool / stage	Command
5.17.3	Hirshfeld-I Charge Partitioning: Stage 3 - Analysis	`stb-hirshfeldAnalysis`
6 · Catalysis		
Provided by an extension package (Chapter 12).		
7 · ML Simulations		
7.1.1	Custom ML Force Field (MACE fine-tuning): Stage 1 - Prep	`stb-mlff`
7.1.2	Custom ML Force Field (MACE fine-tuning): Stage 2 - Train	`stb-mlffAnalysis`
7.1.3	Custom ML Force Field (MACE fine-tuning): Stage 3 - Validate	`stb-mlffValidate`
7.1.4	Custom ML Force Field (MACE fine-tuning): Stage 4 - Active Learning	`stb-mlffActiveLearning`
7.2	ML Molecular Dynamics	`stb-mlmd`
7.3	ML Phonons	`stb-mlphonons`
7.4	ML Elastic Constants	`stb-mlelastic`
7.5	ML NEB	`stb-mlneb`
7.6	ML Adsorption-Site Screening	`stb-mladsorb`

Appendix B

All commands

Every console command installed by the package, alphabetically, with its menu code (– for commands outside the menu) and the first sentence of its description.

Command	Code	What it does
stb-2Dstacking	4.1	STB Monolayer Stacking Tool with Geometric Control & Symmetry Analysis.
stb-adsorb	5.8.1	Prepares SIESTA folders for an adsorption-energy study: a clean slab/2D material, an isolated adsorbate, and one folder per candidate adsorption site.
stb-adsorbAnalysis	5.8.3	Computes adsorption energies from an stb-adsorb sweep: E_ads = E_site - E_clean_slab - E_adsorbate.
stb-adsorbBsse	5.8.2	Stage 2 of the Adsorption workflow: generates the BSSE (counterpoise) ghost-fragment folders at each site's RELAXED geometry, once stb-adsorb's own sites/site_*/ have finished running in SIESTA.
stb-adsorbGibbs	5.8.4	Stage 4 of the Adsorption workflow: combines the vibrational Hessian/phonon folders stb-adsorbAnalysis --compute-gibbs (Stage 3) wrote into a Gibbs free energy of adsorption, DG = E_ads(BSSE if …
stb-aimdAnalysis	2.18	Physical post-processing of an AIMD/MD trajectory – a SIESTA AIMD run (<label>.ANI) or a generic ASE-readable trajectory (e.g.
stb-amorphize	4.11	Melt-quench amorphous structure generator using the MACE-MP-0 potential.
stb-ani2traj	3.5	Convert a SIESTA AIMD trajectory (<label>.ANI) into a multi-frame trajectory format that also carries the lattice – for opening in OVITO, VMD, etc.
stb-archive	3.7	Package a finished SIESTA calculation (inputs + essential outputs) into a single .tar.gz for sharing or reproducibility – the complement of stb-clean, which deletes that same class of outputs to …
stb-bader	2.6	Computes Bader (AIM) atomic charges from a SIESTA charge-density grid.
stb-bands	2.1	Process the band structure data.
stb-chargediffAnalysis	5.16.2	Stage 2 of the Charge Density Difference workflow: given just --dir (the same folder stb-chargediffPrep wrote into), auto-discovers the combined system and every fragment from …
stb-chargediffPrep	5.16.1	Stage 1 of the Charge Density Difference workflow: writes a 'combined/' SIESTA input folder (every atom real, no ghosting) plus one fragment folder per fragment of an N-fragment system (bilayer …
stb-clean	3.2	Clean a SIESTA calc folder to restart a fresh run: removes every file except the extensions in --keep (pseudopotentials, .fdf input, submission script by default).
stb-cohesive	5.3.1	Prepare folder structure for cohesive energy calculations.
stb-cohesiveAnalysis	5.3.2	Process cohesive energy results from SIESTA calculations.
stb-convdos	2.3	Apply Gaussian convolution to broaden a DOS file's columns.

Command	Code	What it does
stb-convergence	5.5.1	Stage 1 of 2: writes one ready-to-run SIESTA folder per swept value of one or more convergence-test parameters, one subfolder per parameter, fully relaxing the structure (positions + cell) at each.
stb-convergenceAnalysis	5.5.2	Stage 2 of 2: analyzes a stb-convergence sweep and reports where each swept parameter's energy AND relaxed structure converge.
stb-coop	2.14	Energy-resolved COOP/COHP bonding/antibonding curves for selected atom pairs, from a full-BZ SIESTA .WFSX + .HSX.
stb-crystalbuilder	4.8	Builds a structure from a space group and Wyckoff positions.
stb-crystalcast	4.12	Casts one or more random structures compatible with a given symmetry group.
stb-cube	3.3	Convert SIESTA Grid files (VT, VH, RHO, BADER) to Gaussian Cube format.
stb-defect	4.5	Introduces a point defect (vacancy, substitution, interstitial) into a SIESTA FDF structure.
stb-density	2.8	Converts a SIESTA charge-density grid (.RHO) to 2D maps, 3D point clouds, or planar-averaged 1D profiles.
stb-dftu	1.5	Generates a ready-to-use %block LDAU.proj snippet for one or more species.
stb-dos	2.2	Parse a PDOS.xml file and generate Gnuplot-ready .dat files.
stb-effmass	2.16	Effective mass tensor (Voigt) and band velocity for one chosen (k, band) eigenstate, from a SIESTA .WFSX + .HSX.
stb-elasticAnalysis	5.2.2	Siesta Elastic Properties Analyzer (STB Suite)
stb-elasticInputs	5.2.1	Stage 1 of 2: applies one or more canonical Voigt strains to a SIESTA structure and writes one ready-to-run folder (structure + calc.fdf + config_extra.fdf + pseudopotentials) per scanned strain …
stb-eosAnalysis	5.15.2	Aggregates a stb-eosInputs 'vol_*' sweep of real SIESTA calculations and fits an equation of state (Birch-Murnaghan by default) to get V0/E0/B0 (bulk modulus)/B0'.
stb-eosInputs	5.15.1	Generates a set of isotropically-scaled-volume SIESTA input folders for a real-DFT equation-of-state (E vs V) scan – run SIESTA in each one (fixed cell, positions may relax), then aggregate with …
stb-fatbands	2.10	Orbital-projected (fatbands) band structure from .bands + .WFSX.
stb-fetch	1.6	Fetches a structure from an online database and writes it as .fdf.
stb-gqca	5.13.1	Stage 1 of 2: generates the 3 ordered pair-cluster-representative structures (AA/AB/BB) for a substitutional alloy A(1-x)Bx.
stb-gqcaAnalysis	5.13.2	Stage 2 of 2: solves the GQCA pair-cluster mass-action equilibrium and reports alloy mixing thermodynamics.
stb-hirshfeldAnalysis	5.17.3	Stage 3 of the Hirshfeld-I workflow: reads back every neutral+ion isolated-atom folder, then iterates the Hirshfeld-I reference-density refinement against the combined/production system's real …
stb-hirshfeldIons	5.17.2	Stage 2 of the Hirshfeld-I workflow: reads back every neutral atom folder stb-hirshfeldPrep wrote (now run through SIESTA) and writes BOTH a cation and an anion reference folder for EVERY species – …
stb-hirshfeldPrep	5.17.1	Stage 1 of the Hirshfeld-I (iterative) charge-partitioning workflow: writes a combined/ folder (the full system, forced to a single-point SaveRho-true evaluation) plus one isolated-atom-in-vacuum-box …
stb-hubbardu	5.7.1	Stage 1 of 3: generates the 'reference' folder for a Hubbard U linear-response calculation (Cococcioni & de Gironcoli, Phys.

Command	Code	What it does
stb-hubbarduAlphas	5.7.2	Stage 2 of 3: generates the perturbed run folders for a Hubbard U linear-response calculation.
stb-hubbarduAnalysis	5.7.3	Stage 3 of 3: analyzes a stb-hubbardu linear-response sweep and computes the Hubbard U.
stb-inputfile	1.1	Script to prepare a Siesta input file (.fdf).
stb-ipr	2.15	Inverse Participation Ratio (IPR) scattered onto a band structure, from .bands + .WFSX.
stb-ir	5.12.1	Stage 1 of 3: generates the phonon-displacement folders for an IR spectrum workflow.
stb-irAnalysis	5.12.3	Stage 3 of 3: analyzes a stb-irModes displacement sweep and computes the IR spectrum.
stb-irModes	5.12.2	Stage 2 of 3: builds FORCE_SETS from Stage 1's phonon displacements, identifies the Gamma-point vibrational modes, and generates whichever follow-up SIESTA folder(s) the IR dipole-derivative …
stb-kgrid	1.3	Compute a Monkhorst-Pack k-point grid from a SIESTA structure file (.fdf) and a target k-point density.
stb-kpath	1.4	Finds the high-symmetry k-path for a SIESTA structure file (.fdf).
stb-mladsorb	7.6	Standalone adsorption-site screening driven entirely by a MACE potential – the MACE-MP-0 foundation model, or a custom model fine-tuned on your own SIESTA data via stb-mlffAnalysis (--custom-model).
stb-mlelastic	7.4	Standalone elastic-constants calculation driven entirely by a MACE potential – the MACE-MP-0 foundation model, or a custom model fine-tuned on your own SIESTA data via stb-mlffAnalysis …
stb-mlff	7.1.1	Stage 1 (Prep) of the Custom ML Force Field workflow: generates training configurations (rattled displacements and/or AIMD-sampled snapshots) from a reference SIESTA calculation, ready for the user …
stb-mlffActiveLearning	7.1.4	Stage 4 (Active Learning) of the Custom ML Force Field workflow: uses already fine-tuned model(s) (stb-mlffAnalysis) to screen a fresh batch of candidate configurations (independent rattled …
stb-mlffAnalysis	7.1.2	Stage 2 (Train) of the Custom ML Force Field workflow: aggregates the finished SIESTA calculations (energy + forces, best-effort stress), builds a MACE-format training set, and fine-tunes a MACE-MP …
stb-mlffValidate	7.1.3	Stage 3 (Validate) of the Custom ML Force Field workflow: scores every training round recorded in mlff_workflow.json (its committee mean and each member) against the held-out SIESTA test set from …
stb-mlmd	7.2	Molecular dynamics driven by a MACE potential – the MACE-MP-0 foundation model, or a custom model fine-tuned on your own SIESTA data via stb-mlffAnalysis (--custom-model).
stb-mlneb	7.5	Climbing-image NEB (nudged elastic band) driven entirely by a MACE potential – the MACE-MP-0 foundation model, or a custom model fine-tuned on your own SIESTA data via stb-mlffAnalysis …
stb-mlphonons	7.3	Standalone phonon calculation driven entirely by a MACE potential – the MACE-MP-0 foundation model, or a custom model fine-tuned on your own SIESTA data via stb-mlffAnalysis (--custom-model).
stb-mlrelax	1.7	Fast ML pre-relaxation with the MACE-MP-0 foundation potential.
stb-molecule	4.10	Builds a reference molecule from ASE's G2 database.

Command	Code	What it does
stb-nanotube	4.4	Rolls a 2D monolayer FDF into a nanotube or nanoribbon.
stb-nativecharges	2.19	Reads and tabulates SIESTA's own native Hirshfeld/Voronoi atomic-charge sections from a finished .out log (Charge.Hirshfeld end / Charge.Voronoi end).
stb-neb	5.9.1	Prepares SIESTA folders for a NEB (Nudged Elastic Band) reaction-path study: interpolates images between two already-relaxed endpoint structures, optionally refining the path with MACE-MP-0 before …
stb-nebAnalysis	5.9.2	Computes a NEB energy profile from an stb-neb band: reads each image_NN/'s SIESTA single-point energy and estimates the reaction barrier from the highest-energy image.
stb-nebCycle	–	One real-DFT NEB step: reads the latest cycle_NN/image_*/ (finished SIESTA single-points), takes exactly one climbing-image NEB step (ASE's own FIRE optimizer, restart-persisted between calls), and …
stb-optical	5.14.1	Stage 1 of 2: writes one SIESTA folder per requested Cartesian/biaxial polarization direction, ready for a linear-response Optical.Calculate run.
stb-opticalAnalysis	5.14.2	Stage 2 of 2: aggregates every direction folder written by stb-optical and derives the full set of linear optical properties.
stb-passivate	4.9	Caps dangling bonds on a cut surface with a passivating atom.
stb-phononsCreate	5.4.1	Automate SIESTA phonon displacement folders with Phonopy.
stb-phononsPos	5.4.2	Post-processing for SIESTA phonon calculations using Phonopy.
stb-pseudo	1.2	Resolves the pseudopotentials a structure needs and copies them into a run folder.
stb-raman	5.11.1	Stage 1 of 3: generates the phonon-displacement folders for a Raman spectrum workflow.
stb-ramanAnalysis	5.11.3	Stage 3 of 3: analyzes a stb-ramanModes Optical-calculation sweep and computes the Raman spectrum.
stb-ramanModes	5.11.2	Stage 2 of 3: builds FORCE_SETS from Stage 1's phonon displacements, identifies the Gamma-point vibrational modes, and generates the +/-delta Optical-calculation displacement folders needed for the …
stb-siesta2wtb	3.4	Convert SIESTA Hamiltonian to TB format for wantibexos Requires SIESTA calculation with SaveHS = .true.
stb-slab	4.3	Cuts a Miller-index slab from a bulk SIESTA FDF structure.
stb-spintexture	2.17	Spin texture (<Sx>,<Sy>,<Sz>) scattered onto a band structure, from a non-collinear/SOC SIESTA .WFSX.
stb-sqs	4.6	Generates a Special Quasirandom Structure (SQS) for a substitutional alloy.
stb-stackingfault	5.10.1	Prepares SIESTA folders for a 2D stacking-fault (generalized stacking fault energy / gamma-surface) study: rigidly slides one layer of a bilayer across a 2D grid of lateral offsets and writes one …
stb-stackingfaultAnalysis	5.10.2	Computes a stacking-fault (gamma-surface) energy landscape from an stb-stackingfault grid: reads each positions/shift_II_JJ/'s SIESTA energy and reports the equilibrium stacking, the highest-energy …
stb-status	3.6	Quick status summary of a SIESTA calc folder: run type (single-point/relaxation/AIMD), SCF convergence, max force, final energy, and which key output files (.XV, .bands, .DOS, .RHO, .ANI...) are …
stb-stm	2.11	Simulate an STM image (Tersoff-Hamann approximation) from a SIESTA energy-integrated LDOS grid (<label>.LDOS).

Command	Code	What it does
stb-strain	5.1.1	Stage 1 of 2: applies strain in Cartesian coordinates to a SIESTA structure and writes one ready-to-run folder (structure + calc.fdf + config_extra.fdf + pseudopotentials) per scanned strain value.
stb-strainAnalysis	5.1.2	Mechanical Properties from Stress-Strain Data (Stage 2 of stb-strain).
stb-structural	2.4	Compute ECN and structural properties from a SIESTA structure file.
stb-sts	2.13	Simulated STS dI/dV(E) spectroscopy curve at a fixed tip point, from a full-BZ SIESTA .WFSX.
stb-supercell	4.2	Builds a supercell from a SIESTA FDF structure file.
stb-symmetry	2.5	Analyze crystal symmetry (space group, Wyckoff positions, symmetry operations) from a SIESTA structure file.
stb-translate	3.1	File format converter using stb-translate.
stb-unitcell	4.7	Finds the primitive/conventional unit cell of a structure, or refines it.
stb-wfdensity	2.12	Real-space |psi|^2 of one chosen (k, band) eigenstate from a SIESTA .WFSX file -> Gaussian .cube.
stb-workfunction	2.7	Calculates the work function from a SIESTA electrostatic potential grid.
stb-xrd	2.9	Simulates a powder XRD pattern from a structure.
stb-xrdrank	5.6.2	Ranks candidate structures by similarity to an experimental XRD pattern (Stage 2 - Analysis).
stb-xrdsearch	5.6.1	Casts candidate structures across a set of space groups (Stage 1 - Prep).

Appendix C

Units and conversion factors

Quantity	Conversions used by the tools
length	1 Bohr = 0.529177 Å
energy	1 Ry = 13.6057 eV; 1 Ha = 27.2114 eV; 1 eV/atom = 96.485 kJ/mol = 23.061 kcal/mol
stress	1 eV/Å^3 = 160.218 GPa; 1 kBar = 0.1 GPa
2D stress	$\sigma_{2\mathrm{D}}$ [N/m] = 0.01 $\times$ σ [kBar] $\times$ c [Å] (cell height)
1D force	F [nN] = 10^{-3} $\times$ σ [kBar] $\times$ A [Å^2] (cell cross-section)
surface energy	1 meV/Å^2 = 16.02 mJ/m^2
frequency	1 THz = 33.356 cm^{-1} = 4.1357 meV
k-point density	d in Å^{-1} with $\lvert\mathbf{b}_i\rvert$ including 2π; $N_i = \lceil \lvert\mathbf{b}_i\rvert / d \rceil$
temperature	k_B = 8.617333$\times 10^{-5}$ eV/K; $k_B T$ = 25.85 meV at 300 K
diffusion	1 Å^2/fs = 0.1 cm^2/s
effective mass	$m^*[m_0]$ = 3.301296$\times 10^{-6}$ $\times$ sisl's value in (ps/Å)^2
velocity	1 Å/ps = 0.1 km/s
compressibility	input in bar^{-1}; $1/(B[\mathrm{GPa}] \times 10^4)$ for a material of bulk modulus B

Appendix D

Glossary

BSSE
basis-set superposition error: the artificial stabilization of a combined system when each fragment borrows the localized basis functions of the other; corrected by the counterpoise method with ghost atoms [71].

`config_extra.fdf`
the sidecar file in which the workflows force directives, included at the top of the copied template (Section 4.2).

Committee
several ML models trained with different seeds; their disagreement estimates the uncertainty of a prediction.

Eggbox effect
the small dependence of SIESTA's energies, forces and stresses on the position of atoms relative to the real-space grid; reduced by a larger `Mesh.Cutoff`.

Foundation model
a machine-learning potential trained on a broad database across the periodic table, here MACE-MP-0 [16].

Ghost atom
a species with negative atomic number in `ChemicalSpeciesLabel`: it carries basis functions but no nucleus or electrons.

Level of theory
the set of settings that define a calculation's physics (functional, basis, pseudopotentials, spin, dispersion, dipole correction, grids); quantities combined in one result must share it.

PAO
pseudo-atomic orbitals, the strictly localized numerical basis of SIESTA [6].

Single point
a calculation at fixed geometry (no ionic or cell relaxation).

SystemLabel
the SIESTA directive whose value names all output files.

Vacuum axis
a lattice direction along which the atoms leave a gap larger than `-vacuum-gap`; treated as non-periodic.

Bibliography

[1] Carlos M. O. Bastos. STB-SUITE: the siesta toolbox suite. https://github.com/stb-suite/stb, 2026. Source code, examples and documentation site https://stb-suite.github.io/stb/.

[2] José M Soler, Emilio Artacho, Julian D Gale, Alberto García, Javier Junquera, Pablo Ordejón, and Daniel Sánchez-Portal. The SIESTA method for ab initio order-N materials simulation. *Journal of Physics: Condensed Matter*, 14(11):2745–2779, 2002. doi: 10.1088/0953-8984/14/11/302.

[3] Alberto García, Nick Papior, Arsalan Akhtar, et al. Siesta: Recent developments and applications. *The Journal of Chemical Physics*, 152(20):204108, 2020. doi: 10.1063/5.0005077.

[4] P. Hohenberg and W. Kohn. Inhomogeneous Electron Gas. *Physical Review*, 136(3B):B864–B871, 1964. doi: 10.1103/physrev.136.b864.

[5] W. Kohn and L. J. Sham. Self-Consistent Equations Including Exchange and Correlation Effects. *Physical Review*, 140(4A):A1133–A1138, 1965. doi: 10.1103/physrev.140.a1133.

[6] Javier Junquera, Óscar Paz, Daniel Sánchez-Portal, and Emilio Artacho. Numerical atomic orbitals for linear-scaling calculations. *Physical Review B*, 64(23), 2001. doi: 10.1103/physrevb.64.235111.

[7] Shyue Ping Ong, William Davidson Richards, Anubhav Jain, et al. Python Materials Genomics (pymatgen): A robust, open-source python library for materials analysis. *Computational Materials Science*, 68:314–319, 2013. doi: 10.1016/j.commatsci.2012.10.028.

[8] Ask Hjorth Larsen, Jens Jørgen Mortensen, Jakob Blomqvist, et al. The atomic simulation environment—a Python library for working with atoms. *Journal of Physics: Condensed Matter*, 29 (27):273002, 2017. doi: 10.1088/1361-648X/aa680e.

[9] Atsushi Togo, Kohei Shinohara, and Isao Tanaka. Spglib: a software library for crystal symmetry search. *Science and Technology of Advanced Materials: Methods*, 4(1):2384822, 2024. doi: 10.1080/27660400.2024.2384822.

[10] Nick Papior and Pol Febrer Calabozo. sisl, 2026. Software, concept DOI (year of the latest release); cite the release used.

[11] Atsushi Togo and Isao Tanaka. First principles phonon calculations in materials science. *Scripta Materialia*, 108:1–5, 2015. doi: 10.1016/j.scriptamat.2015.07.021.

[12] Atsushi Togo, Laurent Chaput, Terumasa Tadano, and Isao Tanaka. Implementation strategies in phonopy and phono3py. *Journal of Physics: Condensed Matter*, 35(35):353001, 2023. doi: 10.1088/1361-648X/acd831.

[13] Mattias Ångqvist, William A. Muñoz, J. Magnus Rahm, Erik Fransson, Céline Durniak, Piotr Rozyczko, Thomas H. Rod, and Paul Erhart. ICET – A Python Library for Constructing and Sampling Alloy Cluster Expansions. *Advanced Theory and Simulations*, 2(7):1900015, 2019. doi: 10.1002/adts.201900015.

[14] Scott Fredericks, Kevin Parrish, Dean Sayre, and Qiang Zhu. PyXtal: A Python library for crystal structure generation and symmetry analysis. *Computer Physics Communications*, 261:107810, 2021. doi: 10.1016/j.cpc.2020.107810.

[15] Ilyes Batatia, Dávid Péter Kovács, Gregor N. C. Simm, Christoph Ortner, and Gábor Csányi. MACE: Higher order equivariant message passing neural networks for fast and accurate force fields, 2022. Presented at NeurIPS 2022.

[16] Ilyes Batatia, Philipp Benner, Yuan Chiang, et al. A foundation model for atomistic materials chemistry, 2023.

[17] Adam Kerrigan. pybader: a threaded implementation of grid-based bader charge analysis. https://github.com/adam-kerrigan/pybader, 2026. Python package; accessed 2026.

[18] Koichi Momma and Fujio Izumi. VESTA 3 for three-dimensional visualization of crystal, volumetric and morphology data. *Journal of Applied Crystallography*, 44(6):1272–1276, 2011. doi: 10.1107/s0021889811038970.

[19] Anton Kokalj. XCrySDen–a new program for displaying crystalline structures and electron densities. *Journal of Molecular Graphics and Modelling*, 17(3-4):176–179, 1999. doi: 10.1016/s1093-3263(99)00028-5.

[20] William Humphrey, Andrew Dalke, and Klaus Schulten. VMD: Visual molecular dynamics. *Journal of Molecular Graphics*, 14(1):33–38, 1996. doi: 10.1016/0263-7855(96)00018-5.

[21] Alexander Stukowski. Visualization and analysis of atomistic simulation data with OVITO–the Open Visualization Tool. *Modelling and Simulation in Materials Science and Engineering*, 18(1):015012, 2009. doi: 10.1088/0965-0393/18/1/015012.

[22] Jörg Neugebauer and Matthias Scheffler. Adsorbate-substrate and adsorbate-adsorbate interactions of Na and K adlayers on Al(111). *Physical Review B*, 46(24):16067–16080, 1992. doi: 10.1103/physrevb.46.16067.

[23] Lennart Bengtsson. Dipole correction for surface supercell calculations. *Physical Review B*, 59(19):12301–12304, 1999. doi: 10.1103/physrevb.59.12301.

[24] Stefan Grimme, Jens Antony, Stephan Ehrlich, and Helge Krieg. A consistent and accurate ab initio parametrization of density functional dispersion correction (DFT-D) for the 94 elements H-Pu. *The Journal of Chemical Physics*, 132(15):154104, 2010. doi: 10.1063/1.3382344.

[25] Stefan Grimme, Stephan Ehrlich, and Lars Goerigk. Effect of the damping function in dispersion corrected density functional theory. *Journal of Computational Chemistry*, 32(7):1456–1465, 2011. doi: 10.1002/jcc.21759.

[26] Alberto García, Matthieu J. Verstraete, Yann Pouillon, and Javier Junquera. The psml format and library for norm-conserving pseudopotential data curation and interoperability. *Computer Physics Communications*, 227:51–71, 2018. doi: 10.1016/j.cpc.2018.02.011.

[27] M.J. van Setten, M. Giantomassi, E. Bousquet, M.J. Verstraete, D.R. Hamann, X. Gonze, and G.-M. Rignanese. The PseudoDojo: Training and grading a 85 element optimized norm-conserving pseudopotential table. *Computer Physics Communications*, 226:39–54, 2018. doi: 10.1016/j.cpc.2018.01.012.

[28] N. Troullier and José Luriaas Martins. Efficient pseudopotentials for plane-wave calculations. *Physical Review B*, 43(3):1993–2006, 1991. doi: 10.1103/physrevb.43.1993.

[29] The SIESTA Group. SIESTA user's guide. https://siesta-project.org/siesta/Documentation/index.html, 2026. Accessed 2026; consulted for the fdf directives quoted in this manual.

[30] Antoine Loew, Dewen Sun, Hai-Chen Wang, Silvana Botti, and Miguel A. L. Marques. Universal machine learning interatomic potentials are ready for phonons. *npj Computational Materials*, 11 (1):178, 2025. doi: 10.1038/s41524-025-01650-1.

[31] Janosh Riebesell, Rhys E. A. Goodall, Philipp Benner, et al. A framework to evaluate machine learning crystal stability predictions. *Nature Machine Intelligence*, 7(6):836–847, 2025. doi: 10.1038/s42256-025-01055-1.

[32] John P. Perdew, Kieron Burke, and Matthias Ernzerhof. Generalized Gradient Approximation Made Simple. *Physical Review Letters*, 77(18):3865–3868, 1996. doi: 10.1103/physrevlett.77.3865.

[33] Shūichi Nosé. A molecular dynamics method for simulations in the canonical ensemble. *Molecular Physics*, 52(2):255–268, 1984. doi: 10.1080/00268978400101201.

[34] Hendrik J. Monkhorst and James D. Pack. Special points for Brillouin-zone integrations. *Physical Review B*, 13(12):5188–5192, 1976. doi: 10.1103/PhysRevB.13.5188.

[35] Wahyu Setyawan and Stefano Curtarolo. High-throughput electronic band structure calculations: Challenges and tools. *Computational Materials Science*, 49(2):299–312, 2010. doi: 10.1016/j.commatsci.2010.05.010.

[36] Yoyo Hinuma, Giovanni Pizzi, Yu Kumagai, Fumiyasu Oba, and Isao Tanaka. Band structure diagram paths based on crystallography. *Computational Materials Science*, 128:140–184, 2017. doi: 10.1016/j.commatsci.2016.10.015.

[37] S. L. Dudarev, G. A. Botton, S. Y. Savrasov, C. J. Humphreys, and A. P. Sutton. Electron-energy-loss spectra and the structural stability of nickel oxide: An LSDA+U study. *Physical Review B*, 57(3): 1505–1509, 1998. doi: 10.1103/physrevb.57.1505.

[38] Vladimir I. Anisimov, Jan Zaanen, and Ole K. Andersen. Band theory and Mott insulators: Hubbard U instead of Stoner I. *Physical Review B*, 44(3):943–954, 1991. doi: 10.1103/physrevb.44.943.

[39] A. I. Liechtenstein, V. I. Anisimov, and J. Zaanen. Density-functional theory and strong interactions: Orbital ordering in Mott-Hubbard insulators. *Physical Review B*, 52(8):R5467–R5470, 1995. doi: 10.1103/physrevb.52.r5467.

[40] Lei Wang, Thomas Maxisch, and Gerbrand Ceder. Oxidation energies of transition metal oxides within the GGA + U framework. *Physical Review B*, 73(19), 2006. doi: 10.1103/physrevb.73.195107.

[41] Matteo Cococcioni and Stefano de Gironcoli. Linear response approach to the calculation of the effective interaction parameters in the LDA + U method. *Physical Review B*, 71(3):035105, 2005. doi: 10.1103/PhysRevB.71.035105.

[42] Saulius Gražulis, Adriana Daškevič, Andrius Merkys, et al. Crystallography Open Database (COD): an open-access collection of crystal structures and platform for world-wide collaboration. *Nucleic Acids Research*, 40(D1):D420–D427, 2011. doi: 10.1093/nar/gkr900.

[43] Anubhav Jain, Shyue Ping Ong, Geoffroy Hautier, et al. Commentary: The Materials Project: A materials genome approach to accelerating materials innovation. *APL Materials*, 1(1):011002, 2013. doi: 10.1063/1.4812323.

[44] Casper W. Andersen, Rickard Armiento, Evgeny Blokhin, et al. OPTIMADE, an API for exchanging materials data. *Scientific Data*, 8(1):217, 2021. doi: 10.1038/s41597-021-00974-z.

[45] Erik Bitzek, Pekka Koskinen, Franz Gähler, Michael Moseler, and Peter Gumbsch. Structural Relaxation Made Simple. *Physical Review Letters*, 97(17), 2006. doi: 10.1103/physrevlett.97.170201.

[46] Richard F. W. Bader. *Atoms in Molecules: A Quantum Theory*. Clarendon Press, Oxford, 1990.

[47] Graeme Henkelman, Andri Arnaldsson, and Hannes Jónsson. A fast and robust algorithm for Bader decomposition of charge density. *Computational Materials Science*, 36(3):354–360, 2006. doi: 10.1016/j.commatsci.2005.04.010.

[48] W Tang, E Sanville, and G Henkelman. A grid-based Bader analysis algorithm without lattice bias. *Journal of Physics: Condensed Matter*, 21(8):084204, 2009. doi: 10.1088/0953-8984/21/8/084204.

[49] J. Tersoff and D. R. Hamann. Theory of the scanning tunneling microscope. *Physical Review B*, 31 (2):805–813, 1985. doi: 10.1103/physrevb.31.805.

[50] J. Bardeen. Tunnelling from a Many-Particle Point of View. *Physical Review Letters*, 6(2):57–59, 1961. doi: 10.1103/physrevlett.6.57.

[51] Timothy Hughbanks and Roald Hoffmann. Chains of trans-edge-sharing molybdenum octahedra: metal-metal bonding in extended systems. *Journal of the American Chemical Society*, 105(11): 3528–3537, 1983. doi: 10.1021/ja00349a027.

[52] Richard Dronskowski and Peter E. Bloechl. Crystal orbital Hamilton populations (COHP): energy-resolved visualization of chemical bonding in solids based on density-functional calculations. *The Journal of Physical Chemistry*, 97(33):8617–8624, 1993. doi: 10.1021/j100135a014.

[53] P. W. Anderson. Absence of Diffusion in Certain Random Lattices. *Physical Review*, 109(5): 1492–1505, 1958. doi: 10.1103/physrev.109.1492.

[54] R. S. Mulliken. Electronic Population Analysis on LCAO–MO Molecular Wave Functions. I. *The Journal of Chemical Physics*, 23(10):1833–1840, 1955. doi: 10.1063/1.1740588.

[55] F. L. Hirshfeld. Bonded-atom fragments for describing molecular charge densities. *Theoretica Chimica Acta*, 44(2):129–138, 1977. doi: 10.1007/bf00549096.

[56] Célia Fonseca Guerra, Jan-Willem Handgraaf, Evert Jan Baerends, and F. Matthias Bickelhaupt. Voronoi deformation density (VDD) charges: Assessment of the Mulliken, Bader, Hirshfeld, Weinhold, and VDD methods for charge analysis. *Journal of Computational Chemistry*, 25(2): 189–210, 2003. doi: 10.1002/jcc.10351.

[57] Patrick Bultinck, Christian Van Alsenoy, Paul W. Ayers, and Ramon Carbó-Dorca. Critical analysis and extension of the Hirshfeld atoms in molecules. *The Journal of Chemical Physics*, 126(14): 144111, 2007. doi: 10.1063/1.2715563.

[58] Alexandre C. Dias, Julian F.R.V. Silveira, and Fanyao Qu. WanTiBEXOS: A Wannier based Tight Binding code for electronic band structure, excitonic and optoelectronic properties of solids. *Computer Physics Communications*, 285:108636, 2023. doi: 10.1016/j.cpc.2022.108636.

[59] A. Zur and T. C. McGill. Lattice match: An application to heteroepitaxy. *Journal of Applied Physics*, 55(2):378–386, 1984. doi: 10.1063/1.333084.

[60] Alex Zunger, S.-H. Wei, L. G. Ferreira, and James E. Bernard. Special quasirandom structures. *Physical Review Letters*, 65(3):353–356, 1990. doi: 10.1103/PhysRevLett.65.353.

[61] Gus L. W. Hart and Rodney W. Forcade. Algorithm for generating derivative structures. *Physical Review B*, 77(22), 2008. doi: 10.1103/physrevb.77.224115.

[62] Larry A. Curtiss, Krishnan Raghavachari, Paul C. Redfern, and John A. Pople. Assessment of Gaussian-2 and density functional theories for the computation of enthalpies of formation. *The Journal of Chemical Physics*, 106(3):1063–1079, 1997. doi: 10.1063/1.473182.

[63] H. J. C. Berendsen, J. P. M. Postma, W. F. van Gunsteren, A. DiNola, and J. R. Haak. Molecular dynamics with coupling to an external bath. *The Journal of Chemical Physics*, 81(8):3684–3690, 1984. doi: 10.1063/1.448118.

[64] J. F. Nye. *Physical Properties of Crystals: Their Representation by Tensors and Matrices*. Oxford University Press, Oxford, 1985.

[65] Woldemar Voigt. *Lehrbuch der Kristallphysik*. B. G. Teubner, Leipzig, 1928.

[66] A. Reuss. Berechnung der Fließgrenze von Mischkristallen auf Grund der Plastizitätsbedingung für Einkristalle . *ZAMM - Journal of Applied Mathematics and Mechanics / Zeitschrift für Angewandte Mathematik und Mechanik*, 9(1):49–58, 1929. doi: 10.1002/zamm.19290090104.

[67] R Hill. The Elastic Behaviour of a Crystalline Aggregate. *Proceedings of the Physical Society. Section A*, 65(5):349–354, 1952. doi: 10.1088/0370-1298/65/5/307.

[68] Shivakumar I. Ranganathan and Martin Ostoja-Starzewski. Universal Elastic Anisotropy Index. *Physical Review Letters*, 101(5), 2008. doi: 10.1103/physrevlett.101.055504.

[69] Max Born and Kun Huang. *Dynamical Theory of Crystal Lattices*. Clarendon Press, Oxford, 1954.

[70] Félix Mouhat and François-Xavier Coudert. Necessary and sufficient elastic stability conditions in various crystal systems. *Physical Review B*, 90(22), 2014. doi: 10.1103/physrevb.90.224104.

[71] S.F. Boys and F. Bernardi. The calculation of small molecular interactions by the differences of separate total energies. Some procedures with reduced errors. *Molecular Physics*, 19(4):553–566, 1970. doi: 10.1080/00268977000101561.

[72] Martin T. Dove. *Introduction to Lattice Dynamics*. Cambridge University Press, Cambridge, 1993.

[73] Graeme Henkelman and Hannes Jónsson. Improved tangent estimate in the nudged elastic band method for finding minimum energy paths and saddle points. *The Journal of Chemical Physics*, 113(22):9978–9985, 2000. doi: 10.1063/1.1323224.

[74] HANNES JÓNSSON, GREG MILLS, and KARSTEN W. JACOBSEN. Nudged elastic band method for finding minimum energy paths of transitions. In *Classical and Quantum Dynamics in Condensed Phase Simulations*, pages 385–404, 1998. doi: 10.1142/9789812839664_0016.

[75] Graeme Henkelman, Blas P. Uberuaga, and Hannes Jónsson. A climbing image nudged elastic band method for finding saddle points and minimum energy paths. *The Journal of Chemical Physics*, 113(22):9901–9904, 2000. doi: 10.1063/1.1329672.

[76] Søren Smidstrup, Andreas Pedersen, Kurt Stokbro, and Hannes Jónsson. Improved initial guess for minimum energy path calculations. *The Journal of Chemical Physics*, 140(21):214106, 2014. doi: 10.1063/1.4878664.

[77] V. Vítek. Intrinsic stacking faults in body-centred cubic crystals. *Philosophical Magazine*, 18(154): 773–786, 1968. doi: 10.1080/14786436808227500.

[78] Dirk Porezag and Mark R. Pederson. Infrared intensities and Raman-scattering activities within density-functional theory. *Physical Review B*, 54(11):7830–7836, 1996. doi: 10.1103/physrevb.54.7830.

[79] Stefano Baroni, Stefano de Gironcoli, Andrea Dal Corso, and Paolo Giannozzi. Phonons and related crystal properties from density-functional perturbation theory. *Reviews of Modern Physics*, 73(2):515–562, 2001. doi: 10.1103/revmodphys.73.515.

[80] F. Albert Cotton. *Chemical Applications of Group Theory*. Wiley, New York, 3rd edition, 1990.

[81] Xavier Gonze and Changyol Lee. Dynamical matrices, Born effective charges, dielectric permittivity tensors, and interatomic force constants from density-functional perturbation theory. *Physical Review B*, 55(16):10355–10368, 1997. doi: 10.1103/physrevb.55.10355.

[82] R. D. King-Smith and David Vanderbilt. Theory of polarization of crystalline solids. *Physical Review B*, 47(3):1651–1654, 1993. doi: 10.1103/physrevb.47.1651.

[83] A. Sher, Mark van Schilfgaarde, An-Ban Chen, and William Chen. Quasichemical approximation in binary alloys. *Physical Review B*, 36(8):4279–4295, 1987. doi: 10.1103/physrevb.36.4279.

[84] An-Ban Chen and Arden Sher. *Semiconductor Alloys: Physics and Materials Engineering*. Plenum Press, New York, 1995.

[85] P.N. Ferreira, R. Lucrezi, I. Guilhon, M. Marques, L.K. Teles, C. Heil, and L.T.F. Eleno. Ab initio modeling of superconducting alloys. *Materials Today Physics*, 48:101547, 2024. doi: 10.1016/j.mtphys.2024.101547.

[86] Frederick Wooten. *Optical Properties of Solids*. Academic Press, New York, 1972.

[87] Mark Fox. *Optical Properties of Solids*. Oxford University Press, Oxford, 2nd edition, 2010.

[88] J. Tauc, R. Grigorovici, and A. Vancu. Optical Properties and Electronic Structure of Amorphous Germanium. *physica status solidi (b)*, 15(2):627–637, 1966. doi: 10.1002/pssb.19660150224.

[89] David R. Penn. Wave-Number-Dependent Dielectric Function of Semiconductors. *Physical Review*, 128(5):2093–2097, 1962. doi: 10.1103/physrev.128.2093.

[90] Danny E. P. Vanpoucke, Patrick Bultinck, and Isabel Van Driessche. Extending Hirshfeld-I to bulk and periodic materials. *Journal of Computational Chemistry*, 34(5):405–417, 2013. doi: 10.1002/jcc.23088.

[91] Roman F. Nalewajski and Robert G. Parr. Information theory, atoms in molecules, and molecular similarity. *Proceedings of the National Academy of Sciences*, 97(16):8879–8882, 2000. doi: 10.1073/pnas.97.16.8879.

[92] Farnaz Heidar-Zadeh, Paul W. Ayers, Toon Verstraelen, Ivan Vinogradov, Esteban Vöhringer-Martinez, and Patrick Bultinck. Information-theoretic approaches to atoms-in-molecules: Hirshfeld family of partitioning schemes. *The Journal of Physical Chemistry A*, 122(17):4219–4245, 2018. doi: 10.1021/acs.jpca.7b08966.

[93] Patrick Bultinck, Paul W. Ayers, Stijn Fias, Koen Tiels, and Christian Van Alsenoy. Uniqueness and basis set dependence of iterative Hirshfeld charges. *Chemical Physics Letters*, 444(1-3):205–208, 2007. doi: 10.1016/j.cplett.2007.07.014.

[94] Loup Verlet. Computer "experiments" on classical fluids. I. Thermodynamical properties of Lennard-Jones molecules. *Physical Review*, 159:98–103, 1967.

[95] William C. Swope, Hans C. Andersen, Peter H. Berens, and Kent R. Wilson. A computer simulation method for the calculation of equilibrium constants for the formation of physical clusters of molecules: Application to small water clusters. *The Journal of Chemical Physics*, 76:637–649, 1982.

[96] T. Schneider and E. Stoll. Molecular-dynamics study of a three-dimensional one-component model for distortive phase transitions. *Physical Review B*, 17:1302–1322, 1978.

[97] J. L. Lebowitz, J. K. Percus, and L. Verlet. Ensemble dependence of fluctuations with application to machine computations. *Physical Review*, 153:250–254, 1967.

[98] H. Flyvbjerg and H. G. Petersen. Error estimates on averages of correlated data. *The Journal of Chemical Physics*, 91:461–466, 1989.

[99] F. A. Lindemann. Über die Berechnung molekularer Eigenfrequenzen. *Physikalische Zeitschrift*, 11:609–612, 1910.

[100] K. Parlinski, Z. Q. Li, and Y. Kawazoe. First-principles determination of the soft mode in cubic ZrO_2. *Physical Review Letters*, 78:4063–4066, 1997.

[101] Atsushi Togo, Laurent Chaput, Isao Tanaka, and Gilles Hug. First-principles phonon calculations of thermal expansion in Ti_3SiC_2, Ti_3AlC_2, and Ti_3GeC_2. *Physical Review B*, 81:174301, 2010.

[102] P. Vinet, J. Ferrante, J. H. Rose, and J. R. Smith. Compressibility of solids. *Journal of Geophysical Research: Solid Earth*, 92(B9):9319–9325, 1987. doi: 10.1029/jb092ib09p09319.

[103] E. Grüneisen. Theorie des festen Zustandes einatomiger Elemente. *Annalen der Physik*, 344: 257–306, 1912.

[104] Bowen Deng, Yunyeong Choi, Peichen Zhong, Janosh Riebesell, Shashwat Anand, Zhuohan Li, KyuJung Jun, Kristin A. Persson, and Gerbrand Ceder. Systematic softening in universal machine learning interatomic potentials. *npj Computational Materials*, 11:9, 2025.

[105] Yvon Le Page and Paul Saxe. Symmetry-general least-squares extraction of elastic data for strained materials from ab initio calculations of stress. *Physical Review B*, 65:104104, 2002.

[106] Leonard Kleinman. Deformation potentials in silicon. I. Uniaxial strain. *Physical Review*, 128: 2614–2621, 1962.

[107] O. H. Nielsen and Richard M. Martin. Quantum-mechanical theory of stress and force. *Physical Review B*, 32:3780–3791, 1985.

[108] M. J. P. Musgrave. *Crystal Acoustics*. Holden-Day, San Francisco, 1970.

[109] Orson L. Anderson. A simplified method for calculating the debye temperature from elastic constants. *Journal of Physics and Chemistry of Solids*, 24(7):909–917, 1963. doi: 10.1016/0022-3697(63)90067-2.

[110] George H. Vineyard. Frequency factors and isotope effects in solid state rate processes. *Journal of Physics and Chemistry of Solids*, 3(1-2):121–127, 1957. doi: 10.1016/0022-3697(57)90059-8.

Index of commands, options and topics

adsorption
    MACE screening, 827

Born stability criteria, 773

climbing image, 800

elastic constants, 773

Grüneisen parameter, 741

Hirshfeld-I charges, 641

internal relaxation, 773

MD, *see* molecular dynamics
migration barrier, 800
molecular dynamics, 708

nudged elastic band
    MACE, 800

phonons, 741

QHA, *see* quasi-harmonic approximation
quasi-harmonic approximation, 741

`stb-2Dstacking`, 511
    `-batch_sym`, 287
    `-custom-model`, 288
    `-custom-model-card`, 288
    `-device`, 288
    `-gap`, 287
    `-gap_range`, 287
    `-interactive`, 287
    `-layer1`, 287
    `-layer2`, 287
    `-match_id`, 287
    `-max_area`, 287
    `-max_strain`, 287
    `-ml-relax`, 287
    `-ml-relax-cell`, 287
    `-model`, 288
    `-output`, 287
    `-save-report`, 287
    `-shift_x`, 287
    `-shift_y`, 287
    `-strain_mode`, 287
    `-symprec`, 287
    `-twist`, 287
    `-vacuum`, 287
    `-view`, 287
`stb-2Dstacking`, 281, **286**
`stb-adsorb`, 16, 17, 482, 827
    `-adsorbate`, 484
    `-all-sites`, 484
    `-both-sides`, 484
    `-calc`, 484
    `-force-spin`, 485
    `-height`, 484
    `-height-sweep`, 484
    `-list`, 484
    `-ml-custom-model`, 485
    `-ml-custom-model-card`, 485
    `-ml-device`, 485
    `-ml-fmax`, 485
    `-ml-model`, 485
    `-ml-prerelax`, 485
    `-ml-rank`, 485
    `-ml-steps`, 485
    `-n-orientations-azimuthal`, 486
    `-n-orientations-polar`, 486
    `-no-force-spin`, 485
    `-orientation-rmsd-tol`, 486
    `-orientation-top-k`, 486
    `-output-dir`, 486
    `-position`, 484
    `-positions-file`, 484
    `-pseudo-dir`, 484
    `-save-report`, 486
    `-site-index`, 484
    `-site-type`, 484
    `-structure`, 484
    `-symprec`, 485
    `-top-k`, 485
    `-vacuum-box`, 485
    `-vacuum-gap`, 485
    `-view`, 486
    `-view-plots`, 486
`stb-adsorb`, 464, **483**
`stb-adsorbAnalysis`, 483
    `-apply`, 488

--compute-gibbs, 488
--dir, 488
--displacement, 488
--file, 488
--force-tolerance, 488
--gibbs-supercell, 488
--output, 488
--save-report, 488
--view, 488
--view-plots, 488
--zpe-mode, 488
stb-adsorbAnalysis, 464, **487**
stb-adsorbBsse, 482
--dir, 487
--file, 487
--force-tolerance, 487
--save-report, 487
stb-adsorbBsse, 464, **487**
stb-adsorbGibbs, 483
--dir, 489
--file, 489
--force-tolerance, 489
--save-report, 489
--tmax, 489
--tmin, 489
--tstep, 489
--view-plots, 490
stb-adsorbGibbs, 464, **489**
stb-aimdAnalysis, 12, 60, 244, 351
--dt, 174
--fit-end, 174
--fit-start, 174
--geometry-file, 174
--label, 174
--list-atoms, 174
--n-bins, 174
--output-dir, 175
--pair, 174
--r-max, 174
--save-gnuplot, 175
--save-report, 175
--skip, 174
--stride, 174
--track-atom, 174
--track-pair, 174
--trajectory, 174
--view, 175
stb-aimdAnalysis, 168, **173**
stb-amorphize, 16
--compressibility, 352
--custom-model, 353
--custom-model-card, 353
--device, 353
--file, 352
--melt-steps, 352
--melt-temp, 352
--model, 352
--no-final-relax, 353
--output, 353
--quench-steps, 352
--quench-temp, 352
--save-data, 353
--save-report, 353
--save-traj, 353
--seed, 353
--stride, 353
--symprec, 353
--taup, 352
--taut, 352
--timestep, 352
--traj-format, 353
--view, 353
stb-amorphize, 346, **352**
stb-ani2traj, 8, 12, 244
--dry-run, 256
--fdf, 256
--label, 256
--log, 256
--out-format, 256
--output, 256
--save-report, 256
--skip, 256
--stride, 256
--unwrap, 256
stb-ani2traj, 244, **256**
stb-archive, 14, 203, 268, 855
--dry-run, 279
--exclude, 278
--force, 279
--include, 278
--label, 279
--mode, 278
--output, 279
--path, 278
--recursive, 279
--save-report, 279
stb-archive, 268, **278**
stb-bader, 12, 59
--export-volumes, 99
--label, 98
--no-cube, 99
--output-dir, 98

-ref, 99
-save-report, 99
-speed, 99
-threads, 99
-vacuum-tol, 99
stb-bader, 93, **98**
stb-bands, 12, 38, 59, 65, 71, 124
-eig-file, 66
-file, 66
-gap-tol, 66
-kp-file, 66
-label, 66
-manual-value, 66
-output-dir, 66
-save-report, 66
-shift, 66
stb-bands, 61, **65**
stb-chargediffAnalysis
-3d, 640
-axis, 640
-contour, 640
-dir, 640
-iso-min, 640
-no-cube, 640
-output-dir, 640
-png, 640
-pos, 640
-profile, 640
-save-data, 640
-save-gnuplot, 640
-save-report, 640
-show, 640
-spin, 640
-vmax, 640
-vmin, 640
stb-chargediffAnalysis, 617, **639**
stb-chargediffPrep
-calc, 638
-force-dipole, 638
-force-spin, 638
-force-vdw, 638
-fragment-manifest, 638
-fragment-sizes, 638
-fragment-species, 638
-mesh-cutoff, 638
-no-force-dipole, 638
-no-force-spin, 638
-no-force-vdw, 638
-output-dir, 638
-png, 639
-pseudo-dir, 638
-save-report, 639
-show, 639
-structure, 638
-view, 639
stb-chargediffPrep, 617, **637**
stb-clean, 14, 203, 268, 855
-dry-run, 216
-keep, 216
-no-archive, 216
-no-confirm, 216
-path, 215
-recursive, 216
-save-report, 216
stb-clean, 203, **215**
stb-cohesive
-bsse-convergence-check, 405
-bsse-convergence-increment, 405
-bsse-correction, 404
-bsse-cutoff, 404
-bsse-multi-site, 404
-dispersion, 405
-k-density, 403
-no-bsse-correction, 404
-no-bsse-multi-site, 404
-output-dir, 404
-pp-path, 403
-save-report, 405
-spin, 403
-structure, 403
-symprec, 404
-vacuum, 403
-vacuum-gap, 404
stb-cohesive, 394, **403**
stb-cohesiveAnalysis, 403
-dir, 405
-force-tolerance, 406
-out, 405
-save-report, 406
-view, 406
stb-cohesiveAnalysis, 394, **405**
stb-convdos, 59, 71, 144
-dir, 78
-file, 78
-fwhm, 78
-no-plot, 78
-out, 78
-output-dir, 78
-save-report, 78
-sigma, 78
-size, 78
stb-convdos, 73, **77**

stb-convergence
-calc, 432
-energyshift-range, 432
-kgrid-range, 432
-meshcutoff-range, 432
-output-dir, 432
-parameter, 432
-pseudo-dir, 432
-relax-steps, 432
-save-report, 432
-structure, 431
-vacuum-gap, 432
stb-convergence, 422, **431**
stb-convergenceAnalysis
-apply, 433
-dir, 433
-file, 433
-force-tolerance, 433
-output-dir, 433
-save-gnuplot, 433
-save-report, 433
-tolerance, 433
-vacuum-gap, 433
-view, 433
-volume-tolerance, 433
stb-convergenceAnalysis, 422, **432**
stb-coop, 12, 59, 60
-atol, 151
-bands-file, 151
-bond-order, 151
-dm-file, 151
-erange, 151
-fermi, 151
-fermi-file, 151
-fwhm, 151
-hsx-file, 151
-label, 150
-manual-value, 151
-npoints, 151
-output-dir, 152
-pair, 151
-pair-species, 151
-quantity, 151
-save-gnuplot, 152
-save-report, 152
-shift, 151
-sigma, 151
-view, 152
-wfsx, 151
stb-coop, 146, **150**
stb-crystalbuilder, 359
-a, 333
-alpha, 334
-angle-tolerance, 334
-b, 333
-beta, 334
-c, 333
-custom-model, 334
-custom-model-card, 334
-device, 334
-gamma, 334
-ml-relax, 334
-ml-relax-cell, 334
-model, 334
-output, 334
-reduce, 334
-save-report, 334
-site, 334
-spacegroup, 333
-symprec, 334
-view, 334
stb-crystalbuilder, 328, **333**
stb-crystalcast, 444
-angle-tolerance, 362
-area, 361
-count, 361
-custom-model, 361
-custom-model-card, 362
-d-tol, 360
-device, 362
-dim, 360
-eps, 360
-file, 360
-group, 360
-group-type, 360
-lattice, 361
-list-molecules, 361
-max-attempts, 361
-ml-rank, 361
-model, 361
-molecular, 360
-num-ions, 361
-output, 362
-save-report, 362
-seed, 361
-sites, 361
-species, 361
-subgroup, 360
-substitute, 360
-supergroup, 360
-symprec, 362
-target-group, 360

-thickness, 361
-vacuum, 361
-view, 362
-volume-factor, 361
stb-crystalcast, 354, **359**
stb-cube, 12, 216
-dry-run, 230
-label, 230
-output, 230
-potential-unit, 230
-save-report, 231
-spin, 230
-type, 230
stb-cube, 216, **230**
stb-defect, 16
-all-inequivalent-sites, 313
-custom-model, 314
-custom-model-card, 314
-device, 314
-file, 313
-filter-species, 313
-index, 313
-ml-rank, 313
-ml-relax, 314
-ml-relax-cell, 314
-model, 314
-nearest, 313
-nearest-format, 313
-new-species, 313
-output, 314
-position, 313
-position-format, 313
-save-report, 314
-species, 314
-symprec, 313
-type, 313
-view, 314
stb-defect, 308, **312**
stb-density, 12, 59, 138
-3d, 111
-axis, 111
-contour, 111
-cube, 111
-iso-min, 111
-label, 111
-output-dir, 111
-pos, 111
-profile, 111
-rho2, 111
-save-gnuplot, 112
-save-report, 111
-spin, 111
-view, 112
-vmax, 111
-vmin, 111
stb-density, 106, **110**
stb-dftu, 19, 461
-fdf, 44
-j, 44
-list-reference, 44
-output, 44
-save-report, 44
-shell, 44
-species, 44
-suggest, 44
-u, 44
-use-reference, 44
stb-dftu, 39, **43**
stb-dos, 12, 24, 59, 77
-dos-threshold-frac, 72
-estimate-from-dos, 72
-label, 72
-output-dir, 72
-projection, 72
-save-gnuplot, 72
-save-report, 72
-shift, 72
-type, 72
-view, 72
stb-dos, 67, **71**
stb-effmass, 12, 59, 60
-band, 162
-bands-file, 162
-fermi, 162
-fermi-file, 163
-gap-tol, 163
-hsx-file, 162
-k-index, 162
-k-point, 162
-label, 162
-output-dir, 163
-save-gnuplot, 163
-save-report, 163
-spin, 162
-view, 163
-wfsx, 162
stb-effmass, 157, **162**
stb-elasticAnalysis, 389, 773
-angle-tolerance, 393
-dimensionality, 392
-dir, 392
-eggbox-tolerance, 393

--file, 392
--fit-quality-tolerance, 393
--method, 392
--plot-dir, 393
--reference-structure, 393
--save-gnuplot, 393
--save-report, 393
--symmetry-method, 392
--symprec, 393
--vacuum-gap, 392
--view, 393
stb-elasticAnalysis, 379, **392**
stb-elasticInputs, 389
--angle-tolerance, 391
--calc, 390
--dirs, 391
--max, 391
--method, 390
--output-dir, 391
--pseudo-dir, 390
--save-report, 391
--steps, 391
--structure, 390
--symmetry-method, 391
--symprec, 391
--vacuum-gap, 391
stb-elasticInputs, 379, **390**
stb-eosAnalysis
--dir, 616
--eos, 616
--file, 616
--output, 616
--png, 617
--save-data, 617
--save-gnuplot, 617
--save-report, 617
--show, 617
--target-pressure, 616
stb-eosAnalysis, 595, **616**
stb-eosInputs
--calc, 615
--file, 615
--kgrid, 615
--kgrid-density, 615
--n-volumes, 615
--output-dir, 615
--pseudo-dir, 615
--relax-positions, 615
--relax-steps, 615
--save-report, 616
--strain-range, 615
--vacuum-gap, 615
stb-eosInputs, 595, **614**
stb-fatbands, 12, 59, 60, 156
--category, 125
--file, 125
--gap-tol, 125
--geometry-file, 125
--hsx-file, 125
--k-tol, 125
--label, 125
--manual-value, 125
--output-dir, 125
--projection, 125
--save-gnuplot, 126
--save-report, 126
--shift, 125
--view, 126
--wfsx, 125
stb-fatbands, 119, **124**
stb-fetch, 19
--angle-tolerance, 50
--api-key, 49
--cod-id, 49
--formula, 49
--limit, 50
--list-providers, 49
--material-id, 49
--most-stable, 49
--optimade-id, 49
--output, 50
--provider, 49
--save-report, 50
--source, 49
--symprec, 50
--unitcell, 50
--view, 50
stb-fetch, 45, **49**
stb-gqca, 567
--calc, 569
--file, 569
--kdensity, 569
--output-dir, 569
--pseudo-dir, 569
--species-b, 569
--sublattice, 569
--vacuum-gap, 569
stb-gqca, 549, **568**
stb-gqcaAnalysis, 567
--directory, 570
--export-relaxed, 571
--file, 570

-force-tolerance, 570
-output, 570
-plot-temp, 571
-png, 571
-save-report, 571
-show, 570
-temp, 570
-temp-max, 570
-temp-min, 570
-temp-points, 570
-view, 571
-x-max, 570
-x-min, 570
-x-points, 570
-x-target, 570
stb-gqcaAnalysis, 549, **569**
stb-hirshfeldAnalysis
-max-iter, 664
-output-dir, 664
-png, 664
-ref, 664
-save-data, 664
-save-report, 664
-show, 664
-tol, 664
stb-hirshfeldAnalysis, 641, **663**
stb-hirshfeldIons
-output-dir, 663
-pseudo-dir, 663
-ref, 663
-save-report, 663
stb-hirshfeldIons, 641, **663**
stb-hirshfeldPrep
-calc, 662
-mesh-cutoff, 662
-output-dir, 662
-pseudo-dir, 662
-save-report, 662
-structure, 662
-vacuum, 662
stb-hirshfeldPrep, 641, **662**
stb-hubbardu, 460
-atom-index, 462
-calc, 462
-j, 462
-output-dir, 462
-pseudo-dir, 462
-save-report, 462
-shell, 462
-species, 462
-structure, 462
stb-hubbardu, 449, **461**
stb-hubbarduAlphas, 461
-alphas, 463
-dir, 463
-frozen-iterations, 463
-save-report, 463
stb-hubbarduAlphas, 449, **462**
stb-hubbarduAnalysis, 461
-dir, 464
-file, 464
-output, 464
-r2-tolerance, 464
-save-gnuplot, 464
-save-report, 464
-view, 464
stb-hubbarduAnalysis, 449, **463**
stb-inputfile, 10, 19, 24, 444
-d3, 25
-pp-path, 25
-save-report, 25
-spin-polarized, 25
-type, 25
-view, 25
stb-inputfile, 20, **25**
stb-inputfile -t bands, 38
stb-inputfile -t relax, 299
stb-ipr, 12, 59, 60
-file, 156
-gap-tol, 157
-geometry-file, 156
-hsx-file, 156
-k-tol, 157
-label, 156
-manual-value, 157
-output-dir, 157
-q, 157
-save-gnuplot, 157
-save-report, 157
-shift, 157
-view, 157
-wfsx, 156
stb-ipr, 152, **156**
stb-ir, 542
-calc, 543
-distance, 543
-kgrid, 543
-kgrid-density, 544
-no-perfect-supercell, 544
-output-dir, 544
-pseudo-dir, 543
-save-report, 544

--structure, 543
--symprec, 544
--vacuum-gap, 544
stb-ir, 534, **543**
stb-irAnalysis, 12, 543
--animation-frames, 549
--degenerate-tol, 548
--directory, 548
--experimental, 548
--file, 548
--label, 548
--linewidth, 548
--output, 548
--peak-prominence, 548
--save-gnuplot, 549
--save-report, 549
--symprec, 549
--temperature, 548
--view, 549
--view-modes, 549
stb-irAnalysis, 534, **547**
stb-irModes, 542
--animation-frames, 547
--calc, 546
--directory, 546
--displacement, 546
--export-animations, 547
--fc-displ, 547
--freq-max, 546
--freq-min, 546
--label, 546
--modes, 546
--polarization-grid, 547
--pseudo-dir, 546
--rotational-mode-tol, 546
--skip-degenerate, 547
--symprec, 546
--use-symmetry, 546
--vacuum-gap, 546
stb-irModes, 534, **545**
stb-kgrid, i, 19, 24, 345, 431, 532
--density, 35
--file, 35
--save-report, 35
--vacuum-gap, 35
stb-kgrid, 31, **34**
stb-kpath, 19, 24, 38
--angle-tolerance, 39
--file, 39
--output, 39
--prec, 39
--save-report, 39
--symprec, 39
--vacuum-gap, 39
stb-kpath, 35, **39**
stb-mladsorb, 827
--adsorbate, 850
--bulk-reference, 850
--committee-models, 850
--custom-model, 850
--custom-model-card, 850
--device, 851
--fmax, 851
--height, 850
--height-sweep, 850
--max-steps, 851
--mlff-workflow, 850
--model, 850
--n-orientations, 851
--no-committee, 850
--output-dir, 851
--relax-depth, 850
--save-data, 851
--save-gnuplot, 851
--save-report, 851
--save-xyz, 851
--seed, 851
--show, 851
--site-type, 850
--skip-foundation-comparison, 850
--slab, 850
--start-heights, 850
--symprec, 851
--temperature, 851
--top-k, 851
--vacuum-box, 851
--vacuum-gap, 851
stb-mladsorb, 827, **849**
stb-mlelastic, 773, 774
--check-convergence, 799
--convergence-strains, 799
--custom-model, 798
--custom-model-card, 798
--device, 799
--dimensionality, 799
--file, 798
--fit-quality-tolerance, 799
--fmax, 799
--max, 799
--max-steps, 799
--model, 798
--no-clamped, 799

--no-pre-relax, 798
--no-relax, 798
--output-dir, 799
--save-data, 799
--save-gnuplot, 799
--save-report, 799
--show, 799
--skip-anisotropy-surface, 799
--skip-foundation-comparison, 798
--steps, 799
--vacuum-gap, 799
stb-mlelastic, 773, **798**
stb-mlff, 695
--calc, 697
--d3, 698
--dup-threshold, 699
--export-traj, 699
--file, 697
--from-aimd, 698
--ml-device, 698
--ml-model, 698
--ml-screen, 698
--n-configs, 698
--n-samples, 698
--n-test, 698
--no-plots, 699
--output-dir, 698
--pseudo-dir, 698
--save-report, 698
--seed, 698
--show, 699
--stdev, 698
--structure, 697
--view, 699
--volume-strains, 698
--workflow, 698
stb-mlff, 667, **697**
stb-mlffActiveLearning, 696
--auto-sampling, 706
--calc, 705
--d3, 707
--device, 707
--diversity, 706
--dry-run, 707
--file, 705
--frontier, 707
--max-energy, 707
--max-force, 706
--md-ensemble, 705
--md-pressure, 705
--md-steps, 706
--md-temperature, 706
--min-novelty, 706
--model, 705
--n-candidates, 706
--output-dir, 707
--pseudo-dir, 705
--round, 705
--sampling-method, 705
--save-data, 707
--save-report, 707
--scan-n, 706
--scan-values, 706
--score, 705
--seed, 707
--selection, 706
--show, 707
--stdev, 706
--strain-max, 705
--stride, 706
--structure, 705
--supercell, 705
--target-force, 706
--top-k, 706
--train-path, 706
--view-picks, 707
--workflow, 705
stb-mlffActiveLearning, 667, **704**
stb-mlffAnalysis, 17, 696, 801
--batch-size, 700
--committee, 700
--cv, 700
--device, 701
--e0s, 700
--epochs, 700
--export-lammps, 701
--foundation-model, 700
--init-from, 700
--lr, 700
--make-cards, 701
--name, 701
--no-stability, 701
--no-stress-training, 700
--path, 700
--save-data, 701
--save-report, 701
--seed, 701
--show, 701
--skip-foundation-comparison, 701
--skip-unlabelled, 700
--stability-steps, 701
--stability-temperature, 701

--stress-weight, 700
--valid-fraction, 701
--view-worst, 701
--work-dir, 701
--workflow, 700
stb-mlffAnalysis, 667, **699**
stb-mlffValidate, 696, 801
--compare, 703
--detail-round, 702
--device, 703
--foundation-model, 703
--md-check, 703
--md-check-frames, 703
--md-check-steps, 703
--md-check-supercell, 703
--md-check-temperature, 703
--md-check-to-train, 703
--model, 702
--phonon-mesh, 703
--phonon-ref, 702
--plateau, 703
--png, 703
--rounds, 702
--save-data, 703
--save-report, 703
--show, 703
--skip-foundation-comparison, 703
--target-force, 703
--test-path, 702
--view-worst, 703
--workflow, 702
stb-mlffValidate, 667, **702**
stb-mlmd, 8, 173, 708
--compressibility, 740
--custom-model, 739
--custom-model-card, 739
--device, 740
--ensemble, 739
--equilibration-steps, 739
--file, 739
--friction, 739
--model, 739
--n-steps, 739
--out-format, 740
--output, 740
--output-structure, 740
--png, 740
--pressure, 739
--save-data, 740
--save-gnuplot, 740
--save-report, 740
--seed, 740
--show, 740
--stride, 740
--supercell, 740
--taup, 740
--taut, 739
--temperature, 739
--timestep, 739
--vacuum-gap, 740
stb-mlmd, 708, **738**
stb-mlneb, 500, 800, 801, 828
--autosort-tol, 826
--calc, 825
--committee-models, 825
--custom-model, 824
--custom-model-card, 825
--device, 826
--final, 824
--fmax, 826
--freeze-substrate, 826
--freeze-threshold, 826
--idpp-fmax, 826
--idpp-steps, 826
--initial, 824
-k, 826
--kgrid, 825
--max-steps, 826
--mlff-workflow, 825
--model, 824
--n-images, 825
--no-committee, 825
--no-idpp, 826
--no-pre-relax-endpoints, 826
--output-dir, 826
--pseudo-dir, 825
--save-data, 826
--save-gnuplot, 826
--save-images, 826
--save-report, 826
--show, 826
--siesta-check, 825
--siesta-check-images, 825
--skip-foundation-comparison, 825
--to-mlff, 825
--to-mlff-rattle, 825
--to-mlff-seed, 825
--to-mlff-stdev, 825
stb-mlneb, 800, **824**
stb-mlphonons, 8, 741
--animate-amplitude, 772
--animate-dim, 772

--animate-format, 772
--animate-frames, 772
--animate-mode, 772
--animate-qpoint, 772
--band-points, 771
--check-convergence, 771
--convergence-dims, 772
--convergence-tol, 772
--custom-model, 771
--custom-model-card, 771
--device, 772
--dim, 771
--distance, 771
--eos, 771
--file, 770
--fmax, 771
--freeze-amplitude, 772
--freeze-unstable-mode, 772
--mesh, 771
--model, 770
--n-volumes, 771
--no-relax, 771
--output-dir, 772
--qha, 771
--save-data, 772
--save-gnuplot, 772
--save-report, 773
--show, 772
--skip-foundation-comparison, 771
--strain-range, 771
--tmax, 771
--tmin, 771
--tstep, 771
--vacuum-gap, 771
stb-mlphonons, 741, **770**
stb-mlrelax, 16, 19, 359, 742
--custom-model, 57
--custom-model-card, 57
--device, 58
--file, 57
--fmax, 57
--max-steps, 57
--model, 57
--optimizer, 57
--output, 58
--relax-cell, 57
--save-data, 58
--save-report, 58
--vacuum-gap, 57
--view, 58
stb-mlrelax, 50, **57**
stb-molecule
--custom-model, 345
--custom-model-card, 345
--device, 345
--list, 345
--ml-relax, 345
--model, 345
--name, 345
--output, 346
--save-report, 346
--vacuum, 345
--view, 346
stb-molecule, 341, **345**
stb-nanotube, 339
--bond-length, 307
--chirality, 306
--custom-model, 307
--custom-model-card, 307
--cutoff, 307
--device, 307
--file, 306
--lattice-tol, 306
--min-vacuum-size, 306
--ml-relax, 307
--ml-relax-cell, 307
--mode, 306
--model, 307
--output, 307
--passivant, 307
--passivate, 306
--repeats, 306
--save-report, 307
--search-bound, 306
--symprec, 306
--vacuum-gap, 306
--view, 307
stb-nanotube, 301, **306**
stb-nativecharges, 59, 175
--csv, 185
--details, 185
--label, 185
--path, 185
--save-report, 185
stb-nativecharges, 175, **185**
stb-neb, 16, 800, 802
--autosort-tol, 501
--calc, 501
--conda-env, 503
--cycle-fmax, 503
--final, 501
--force-dipole, 501

--force-spin, 501
--force-vdw, 501
--idpp, 502
--idpp-fmax, 502
--idpp-steps, 502
--initial, 501
--max-cycles, 502
--ml-device, 502
--ml-fmax, 502
--ml-freeze-substrate, 502
--ml-freeze-threshold, 502
--ml-k, 502
--ml-max-steps, 502
--ml-model, 502
--ml-prerelax-endpoints, 502
--mode, 502
--mpirun-np, 503
--n-images, 501
--no-force-dipole, 501
--no-force-spin, 501
--no-force-vdw, 501
--no-ml-freeze-substrate, 502
--output-dir, 502
--pseudo-dir, 501
--siesta-exe, 503
stb-neb, 490, **500**
stb-nebAnalysis, 500
--apply, 504
--dir, 503
--file, 503
--force-tolerance, 504
--output, 504
--save-gnuplot, 504
--save-path-xyz, 504
--save-report, 504
--view, 504
--view-path, 504
stb-nebAnalysis, 490, **503**
stb-nebCycle, 7, 500, 855
--climb-after, 505
--dir, 505
--file, 505
--fmax, 505
--k, 505
--save-report, 505
stb-nebCycle, **504**
stb-nebCycle --dir neb_run, 500
stb-optical, 590
--calc, 592
--directions, 592
--file, 592
--kgrid, 592
--kgrid-density, 592
--optical-broaden, 592
--optical-mesh, 592
--optical-nbands, 592
--output-dir, 592
--pseudo-dir, 592
--save-report, 592
--vacuum-gap, 592
stb-optical, 571, **591**
stb-opticalAnalysis, 12, 590
--dimensionality-correction, 594
--directory, 593
--energies, 594
--energy-max, 594
--energy-min, 594
--experimental, 594
--file, 593
--label, 594
--n-peaks, 595
--output, 594
--peak-prominence, 594
--plot-emax, 594
--plot-quantity, 594
--png, 594
--save-report, 594
--scissor, 595
--show, 594
--smoothing, 595
--thickness, 594
stb-opticalAnalysis, 571, **593**
stb-passivate, 298, 339
--bond-length, 340
--custom-model, 340
--custom-model-card, 340
--cutoff, 340
--device, 340
--file, 340
--ml-relax, 340
--ml-relax-cell, 340
--model, 340
--output, 340
--passivant, 340
--save-report, 340
--symprec, 340
--view, 340
stb-passivate, 335, **339**
stb-phononsCreate, 418, 526, 696
--calc, 419
--distance, 419
--kgrid, 419

-kgrid-density, 419
-ml-device, 419
-ml-fmax, 419
-ml-model, 419
-ml-prerelax, 419
-no-perfect-supercell, 420
-pseudo-dir, 419
-save-report, 420
-structure, 419
-symprec, 419
-vacuum-gap, 419
stb-phononsCreate, 406, **418**
stb-phononsPos, 418
-band-points, 421
-bands, 421
-directory, 420
-dos, 421
-freeze-amplitude, 421
-freeze-unstable-mode, 421
-label, 420
-mesh, 420
-pdos, 421
-plot-dir, 420
-save-gnuplot, 421
-save-report, 421
-symprec, 421
-thermal-displacements, 421
-tmax, 420
-tmin, 420
-tstep, 420
-vacuum-gap, 421
-view, 421
stb-phononsPos, 406, **420**
stb-pseudo, 11, 19, 25
-dry-run, 30
-fallback-dir, 30
-file, 30
-list-elements, 30
-output, 30
-pp-path, 30
-save-report, 30
-species, 30
stb-pseudo, 26, **29**
stb-raman, 526
-calc, 528
-distance, 528
-kgrid, 528
-kgrid-density, 528
-no-perfect-supercell, 529
-output-dir, 529
-pseudo-dir, 528
-save-report, 529
-structure, 528
-symprec, 528
-vacuum-gap, 528
stb-raman, 516, **528**
stb-ramanAnalysis, 12, 527
-directory, 533
-experimental, 533
-file, 533
-label, 533
-linewidth, 533
-output, 533
-peak-prominence, 533
-save-gnuplot, 533
-save-report, 533
-temperature, 533
-view, 533
stb-ramanAnalysis, 516, **532**
stb-ramanModes, 526
-animation-frames, 532
-calc, 530
-directory, 530
-displacement, 531
-export-animations, 532
-freq-max, 530
-freq-min, 530
-full-tensor, 531
-kgrid, 531
-kgrid-density, 531
-label, 530
-modes, 530
-optical-broaden, 532
-optical-mesh, 532
-optical-nbands, 532
-pseudo-dir, 530
-reduce-components, 531
-rotational-mode-tol, 531
-skip-degenerate, 531
-symprec, 531
-use-symmetry, 531
-vacuum-gap, 530
-view-animation, 532
stb-ramanModes, 516, **530**
stb-siesta2wtb, 12, 231
-dry-run, 244
-fermi-level, 243
-input, 243
-no-check, 243
-output, 243
-quiet, 244
-save-report, 244

stb-siesta2wtb, 231, **243**
stb-slab, 339
-all, 300
-bond-length, 300
-center-slab, 299
-custom-model, 300
-custom-model-card, 300
-cutoff, 300
-device, 300
-file, 299
-hkl, 299
-interactive, 299
-lll-reduce, 299
-min-slab-size, 299
-min-vacuum-size, 299
-ml-relax, 300
-ml-relax-cell, 300
-model, 300
-output, 300
-passivant, 300
-passivate, 300
-primitive, 299
-save-report, 300
-symmetrize, 299
-symprec, 299
-termination, 300
-view, 300
stb-slab, 294, **299**
stb-spintexture, 12, 59, 60
-bands-file, 167
-fermi, 167
-fermi-file, 167
-gap-tol, 167
-geometry-file, 167
-hsx-file, 167
-label, 167
-manual-value, 167
-output-dir, 167
-save-gnuplot, 167
-save-report, 167
-shift, 167
-view, 167
-wfsx, 167
stb-spintexture, 163, **166**
stb-sqs, 48
-cluster-cutoffs, 321
-composition, 320
-custom-model, 321
-custom-model-card, 321
-device, 321
-file, 320
-instances, 321
-mc-steps, 321
-method, 321
-ml-relax, 321
-ml-relax-cell, 321
-model, 321
-output, 321
-save-report, 321
-scaling, 321
-sublattice, 320
-symprec, 321
-temperature, 321
-view, 321
stb-sqs, 314, **320**
stb-stackingfault, 17, 511
-calc, 512
-d3, 513
-gap, 513
-grid-nx, 513
-grid-ny, 514
-layer1, 512
-layer2, 512
-match_id, 513
-max_area, 512
-max_strain, 512
-ml-custom-model, 514
-ml-custom-model-card, 514
-ml-device, 514
-ml-fmax, 514
-ml-model, 514
-ml-prerelax-layers, 514
-ml-preview, 514
-mode, 513
-no-d3, 513
-output-dir, 514
-pseudo-dir, 512
-relax-z-steps, 513
-save-report, 514
-scan, 513
-scan-points, 513
-strain_mode, 513
-twist, 513
-vacuum, 513
stb-stackingfault, 505, **512**
stb-stackingfaultAnalysis, 512
-apply, 515
-dir, 515
-file, 515
-force-tolerance, 515
-output, 515
-save-gnuplot, 515

--save-report, 515
--view, 515
--view-animation, 515
stb-stackingfaultAnalysis, 505, **515**
stb-status, 14, 257, 268
--csv, 268
--details, 268
--label, 268
--path, 268
--recursive, 268
--save-report, 268
stb-status, 257, **267**
stb-stm, 12, 59
--axis, 132
--file, 132
--geometry-file, 132
--iso, 132
--label, 132
--mode, 132
--output-dir, 132
--save-gnuplot, 132
--save-report, 132
--view, 132
--z, 132
--z-max, 132
stb-stm, 126, **131**
stb-strain, 375
--angle-tolerance, 377
--calc, 376
--output-dir, 377
--pseudo-dir, 377
--relax-mode, 377
--save-report, 377
--stdir, 377
--step, 377
--stmax, 377
--stmin, 377
--structure, 376
--symprec, 377
--vacuum-gap, 377
stb-strain, 365, **376**
stb-strainAnalysis, 375
--cross-section, 378
--dimensionality, 378
--dir, 378
--file, 378
--output-dir, 378
--save-gnuplot, 378
--save-report, 378
--thickness, 378
--vacuum-gap, 378
--view, 379
--yield, 378
stb-strainAnalysis, 365, **378**
stb-structural, 60, 351
--file, 85
--format, 85
--list, 85
--mode, 85
--no-rdf, 85
--output-dir, 85
--rdf-rmax, 85
--save-report, 85
stb-structural, 79, **85**
stb-sts, 12, 59, 60
--axis, 145
--bands-file, 145
--erange, 145
--fermi, 145
--fermi-file, 145
--fwhm, 146
--geometry-file, 145
--height, 145
--label, 145
--manual-value, 145
--npoints, 145
--output-dir, 146
--point, 145
--save-gnuplot, 146
--save-report, 146
--shift, 145
--sigma, 145
--view, 146
--wfsx, 145
--xy, 145
stb-sts, 140, **144**
stb-suite, i, 2, 6, 855, 857
stb-supercell, 312
--custom-model, 293
--custom-model-card, 293
--device, 293
--dim, 293
--file, 293
--ml-relax, 293
--ml-relax-cell, 293
--model, 293
--output, 293
--save-report, 293
--symprec, 293
--view, 293
stb-supercell, 288, **293**
stb-symmetry, 38, 60, 327

--angle-tolerance, 92
--compare-format, 92
--compare-to, 92
--file, 91
--format, 91
--no-operations, 92
--output-dir, 92
--save-report, 92
--scan-symprec, 92
--symprec, 91
--view, 92
--write-refined, 92
stb-symmetry, 86, **91**
stb-translate, 187, 280
--cif-symprec, 202
--coord-format, 202
--dry-run, 202
--in-file, 202
--in-format, 202
--lattice-vectors, 202
--out-dir, 202
--out-file, 202
--out-format, 202
--save-report, 202
--view, 202
--view-image, 202
stb-translate, 187, **201**
stb-unitcell, 48, 333
--angle-tolerance, 327
--custom-model, 327
--custom-model-card, 328
--device, 328
--file, 327
--ml-relax, 327
--ml-relax-cell, 327
--mode, 327
--model, 327
--output, 328
--save-report, 328
--symprec, 327
--view, 328
stb-unitcell, 322, **327**
stb-wfdensity, 12, 59, 60, 162
--axis, 139
--band, 139
--bands-file, 139
--fermi, 139
--fermi-file, 139
--gap-tol, 139
--geometry-file, 138
--k-index, 139
--k-point, 139
--label, 138
--output-dir, 139
--pos, 139
--profile, 139
--save-gnuplot, 139
--save-report, 139
--shape, 139
--spacing, 139
--spin, 139
--view, 139
--wfsx, 138
stb-wfdensity, 133, **138**
stb-workfunction, 12, 59
--asymmetric-tol, 105
--axis, 105
--fermi, 105
--file, 105
--grid, 105
--label, 105
--output-dir, 105
--save-gnuplot, 105
--save-report, 105
--view, 105
stb-workfunction, 99, **104**
stb-xrd, 60, 444
--compare-to, 118
--file, 117
--format, 117
--output-dir, 118
--raw-experimental, 118
--save-gnuplot, 118
--save-report, 118
--top, 118
--two-theta-range, 118
--view, 118
--wavelength, 118
stb-xrd, 112, **117**
stb-xrdrank, 444
--experimental, 448
--input-dir, 448
--output, 448
--raw-experimental, 448
--save-gnuplot, 448
--save-report, 448
--top, 448
--two-theta-range, 448
--view, 448
--wavelength, 448
stb-xrdrank, 434, **447**
stb-xrdsearch, 17, 444

--calc, 446
--count-per-group, 445
--d3, 446
--groups, 445
--keep-top-per-group, 446
--mace-custom-model, 446
--mace-custom-model-card, 446
--mace-device, 446
--mace-fmax, 446
--mace-max-steps, 446
--mace-model, 446
--mace-relax, 446
--max-attempts, 446
--ml-rank, 446
--molecular, 446
--num-ions, 445
--output-dir, 447
--pseudo-dir, 447
--save-report, 447
--seed, 446
--species, 445
--spin-polarized, 447
--view, 447
--volume-factor, 446
stb-xrdsearch, 434, **445**
stiffness tensor, *see* elastic constants
stockholder partition, *see* Hirshfeld-I charges

thermal expansion, 741

vacancy
  silicon, 800
Voigt–Reuss–Hill averages, 773